A HANDS-ON COURSE

Types, Proofs and Ownership

Type Theory, Logic and Category Theory through Rust and Lean 4

Luciano Di Cocco

Rust 1.99 (edition 2024) · Lean 4.34.1

First edition · October 2026

Types, Proofs and Ownership — Type Theory, Logic and Category Theory through Rust and Lean 4

Copyright © 2026 Luciano Di Cocco

First edition, October 2026. This book targets **Rust 1.99** (edition 2024) and **Lean 4.34.1** with **Mathlib** (release `v4.34.1`). The Rust code uses only the standard library.

Every program and every proof shown in this book is part of the accompanying `code/` directory: a Cargo workspace (`code/rust`) and a Lake project (`code/lean`). Code excerpts are included verbatim from those files when the book is built. Rust program outputs and compiler diagnostics were captured from real runs; Lean outputs and expected error messages are checked by Lean itself with `#guard_msgs` every time the project is built.

The text of this book and its diagrams are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). The code, including the excerpts shown in the book, is available under the MIT license or the Apache License, Version 2.0, at your option.

Rust is a trademark of the Rust Foundation. Lean is a project of the Lean FRO. This book is an independent work and is not affiliated with or endorsed by either project.

Contents

Preface xviii
Two languages xviii
Who this book is for xix
What you will build xix
How the book is organized xx
Conventions xxi
The code and how to run it xxii
A note on versions and accuracy xxii
Acknowledgements xxiii

Part I – Foundations

1 Two Languages, One Idea 2
1.1 What a type theory is made of 2
1.2 Rust's expressions, read as judgments 5
1.3 Three readings of a type 7
1.4 Why Rust and Lean 8
1.5 A first session with Lean 10
1.6 The question of this book 15
1.7 A map of the book 15
1.8 Common confusions 16
1.9 Summary 16
1.10 Exercises 17

2 Lean 4 for Rust Programmers 18
2.1 Reading and writing Lean files 18
2.2 Functions 19
2.3 Structures and inductive types 23
2.4 Options, errors and collections 26
2.5 Namespaces, dot notation and type classes 28
2.6 `do` notation, `IO` and loops 32
2.7 Totality: every function terminates 34
2.8 Values that never change 36
2.9 Putting it together: an order book in two languages 37
2.10 Proving things about programs 43

2.11 Mathlib 49
2.12 A Rust–Lean phrasebook 53
2.13 Common confusions 53
2.14 Summary 54
2.15 Exercises 55

3 Judgments, Rules and Derivations 56
3.1 Syntax, concrete and abstract 57
3.2 Judgments 59
3.3 Inference rules 61
3.4 Derivations 62
3.5 The rules in Lean 66
3.6 Rust's expressions as instances of rules 68
3.7 Evaluation and type safety 71
3.8 Checking, synthesis and inference 75
3.9 Structural and nominal types 75
3.10 Putting it together: the checker is correct 76
3.11 Common confusions 82
3.12 Summary 83
3.13 Exercises 83

4 Natural Deduction 85
4.1 Propositions and judgments 86
4.2 The rules of natural deduction 87
4.3 Proving it in Lean 89
4.4 Harmony 94
4.5 Normalization, subformulas and consistency 99
4.6 Sequent calculus and cut elimination 100
4.7 A proof checker in Rust 101
4.8 A deep embedding in Lean 108
4.9 Classical logic, in one rule 112
4.10 Putting it together: one formula, four checks 113
4.11 Common confusions 114
4.12 Summary 115
4.13 Exercises 115

5 Intuitionism and the Meaning of Proof 117
5.1 Brouwer and the meaning of "true" 118
5.2 The BHK interpretation 118
5.3 What fails, and what you gain 119
5.4 Classical logic is not wrong 124
5.5 Negation, truth and falsity 127

5.6 Decidable propositions in Lean 129
5.7 Kripke semantics 130
5.8 Heyting algebras 139
5.9 The categorical reading 145
5.10 Putting it together: the De Morgan law 146
5.11 Common confusions 146
5.12 Summary 147
5.13 Exercises 147

6 The Structural Rules 149
6.1 Rules about contexts 150
6.2 Where the structural rules hide 151
6.3 Primitive, derivable and admissible 153
6.4 Admissibility, proved in Lean 155
6.5 Counting uses 158
6.6 A first glance at Rust 165
6.7 The resource reading of assumptions 170
6.8 Putting it together: one formula, three views 171
6.9 Common confusions 172
6.10 Summary 173
6.11 Exercises 173

Part II – Programs as Proofs

7 The Lambda Calculus 176
7.1 Three constructs 176
7.2 Free and bound variables 177
7.3 Substitution, and the capture problem 178
7.4 β-reduction 181
7.5 Reduction strategies 182
7.6 Confluence: the answer does not depend on the route 185
7.7 Recursion without names 186
7.8 Church encodings 188
7.9 De Bruijn indices: names are the problem 191
7.10 The λ-calculus in Lean 195
7.11 Rust closures are typed λ-terms 203
7.12 Putting it together: the factorial of 3 204
7.13 Common confusions 205
7.14 Summary 205
7.15 Exercises 206

8 The Simply Typed Lambda Calculus and Closures 207
8.1 Types for the λ-calculus 208

8.2 The typing rules 208
8.3 Computation: β and η 210
8.4 What types buy: termination 210
8.5 Bidirectional type checking 212
8.6 The `stlc` crate 214
8.7 Functions in Rust 219
8.8 Intrinsically typed terms in Lean 229
8.9 Putting it together: one function, three ways 236
8.10 Common confusions 237
8.11 Summary 238
8.12 Exercises 238

9 Type Safety, Proved 240
9.1 The language 241
9.2 The typing rules 243
9.3 Running programs: small-step semantics 246
9.4 Determinism 251
9.5 Canonical forms 253
9.6 Progress 254
9.7 Substitution, carefully 255
9.8 Preservation 260
9.9 Type safety 261
9.10 Testing the theorems in Rust 263
9.11 Putting it together: one program, three views 270
9.12 What "going wrong" means for Rust 271
9.13 Common confusions 272
9.14 Summary 272
9.15 Exercises 273

10 The Curry–Howard Correspondence 274
10.1 Erase the terms 275
10.2 The dictionary 276
10.3 Quantifiers 281
10.4 Normalization is computation 282
10.5 Rust is inconsistent as a logic 283
10.6 Lean is consistent 284
10.7 Contraction: where Rust makes the logic honest 286
10.8 `Prop` and `Type` 288
10.9 Exists, Subtype and Sigma 290
10.10 `theorem` versus `def` 293
10.11 An inductive family: `Nat.le` 293
10.12 The term behind a tactic proof 294

10.13 Putting it together: one law, three proofs 295
10.14 Common confusions 295
10.15 Summary 296
10.16 Exercises 297

11 Algebraic Data Types and the Algebra of Types 298
11.1 Five ways to build a type 299
11.2 Counting values 303
11.3 Isomorphisms: when two types are the same 306
11.4 Proving isomorphisms in Lean 309
11.5 In Mathlib: `Equiv` and `Fintype.card` 315
11.6 Recursive types 317
11.7 Positivity: which recursive types are allowed 323
11.8 Layout: the algebra in memory 326
11.9 Derivatives of types and zippers 327
11.10 Putting it together: counting illegal states 333
11.11 Common confusions 336
11.12 Summary 336
11.13 Exercises 337

Part III – Abstraction

12 Parametric Polymorphism 339
12.1 One function, every type 340
12.2 System F 340
12.3 A type checker for System F in Lean 342
12.4 Rust generics are ∀ 347
12.5 Monomorphization 348
12.6 Rank-1 and rank-2 351
12.7 Polymorphism in Lean 357
12.8 Universes, gently 358
12.9 Church numerals in Lean 362
12.10 Putting it together: self-application in three languages 367
12.11 Common confusions 368
12.12 Summary 369
12.13 Exercises 369

13 Type Inference 371
13.1 Why infer types 372
13.2 Type variables and substitutions 372
13.3 Unification 374
13.4 Unification in Rust 378
13.5 Unification in Lean, proved sound 381

13.6 Hindley–Milner: polymorphism without annotations 387
13.7 Algorithm W 391
13.8 Algorithm W in Rust 392
13.9 The limits of Hindley–Milner 399
13.10 Rust's type inference 399
13.11 Lean's elaborator 404
13.12 Putting it together: `map` in three languages 408
13.13 Common confusions 409
13.14 Summary 409
13.15 Exercises 410

14 Traits, Type Classes and Dictionary Passing 411
14.1 Two kinds of polymorphism 412
14.2 Typing rules for type classes 414
14.3 Dictionary passing 416
14.4 Coherence 421
14.5 Associated types, supertraits and default methods 424
14.6 Laws 430
14.7 Type classes in Lean 437
14.8 Putting it together: statistics by batch 443
14.9 Common confusions 446
14.10 Summary 447
14.11 Exercises 448

15 Existential Types and Dynamic Dispatch 449
15.1 Hiding a type 450
15.2 Three places for a quantifier in Rust 451
15.3 Trait objects in memory 457
15.4 Dyn compatibility 460
15.5 `Any`: an existential you can open 462
15.6 Existentials in Lean 464
15.7 Encoding ∃ with ∀ 470
15.8 Putting it together: choosing where the quantifier goes 474
15.9 Common confusions 474
15.10 Summary 475
15.11 Exercises 476

Part IV – Categories

16 Categories for Programmers 478
16.1 Composition is what programmers do 479
16.2 The definition of a category 480
16.3 A gallery of categories 483

16.4 Commutative diagrams 492
16.5 Isomorphisms 493
16.6 Monomorphisms and epimorphisms 495
16.7 Initial and terminal objects 497
16.8 Products 499
16.9 Coproducts 503
16.10 Duality and the opposite category 504
16.11 Putting it together: one definition, many products 505
16.12 In Mathlib 506
16.13 Common confusions 510
16.14 Summary 510
16.15 Exercises 511

17 Functors and Higher-Kinded Types 512
17.1 Kinds: the types of types 513
17.2 Functors 514
17.3 Functors in Lean 517
17.4 Functors in Rust: the missing trait 520
17.5 Other variances 527
17.6 Functors between arbitrary categories 530
17.7 Variance is functoriality 534
17.8 Putting it together: map fusion, proved once 535
17.9 In Mathlib 536
17.10 Common confusions 537
17.11 Summary 538
17.12 Exercises 538

18 Natural Transformations and Free Theorems 540
18.1 From functors to maps between functors 541
18.2 Naturality 542
18.3 Checking naturality in Rust 543
18.4 Naturality in Lean 545
18.5 Composing natural transformations 547
18.6 Parametricity: theorems for free 551
18.7 Why Lean cannot prove the free theorem 554
18.8 Where Rust leaks parametricity 556
18.9 The Yoneda lemma, gently 559
18.10 In Mathlib 564
18.11 Putting it together: moving work past a transformation 565
18.12 Common confusions 567
18.13 Summary 568
18.14 Exercises 568

19 Cartesian Closed Categories **570**
19.1 Products and the terminal object, again 570
19.2 Exponentials 573
19.3 Currying is an adjunction 577
19.4 Heyting algebras are thin CCCs 578
19.5 Lambek: the λ-calculus is the language of CCCs 580
19.6 Putting it together: one term, three meanings 585
19.7 In Mathlib 587
19.8 Rust is not cartesian 588
19.9 Common confusions 590
19.10 Summary 591
19.11 Exercises 592

20 Monads and Effects **593**
20.1 When functions have effects 594
20.2 Monads: `pure`, `bind` and three laws 595
20.3 Monads you already use in Rust 598
20.4 async and await: a monad built into the compiler 603
20.5 Why Rust has no `Monad` trait 605
20.6 Monads in Lean 610
20.7 A monad class from scratch 613
20.8 The Kleisli category 619
20.9 Putting it together: an order workflow 622
20.10 In Mathlib: monads and adjunctions 626
20.11 Common confusions 627
20.12 Summary 628
20.13 Exercises 629

Part V – Ownership as Logic

21 Substructural Logics **631**
21.1 The structural rules, again 631
21.2 The family of logics 632
21.3 Conjunction splits in two 634
21.4 Disjunction, implication and the units 636
21.5 Splitting versus sharing 638
21.6 Linear logic in Rust 639
21.7 Proof search: the `linlogic` crate 643
21.8 A linear sequent calculus in Lean 648
21.9 Putting it together: the vending machine 656
21.10 Common confusions 657
21.11 Summary 658

21.12 Exercises 658

22 The Affine Lambda Calculus 660

22.1 From affine logic to a programming language 661
22.2 Syntax 661
22.3 Contexts and splitting 664
22.4 The typing rules 665
22.5 Worked derivations 668
22.6 Sharing: offers and branches 671
22.7 Why `⊗` has no projections 673
22.8 The usage theorem 674
22.9 More metatheory 677
22.10 Checking with leftovers 680
22.11 The `affine` crate 689
22.12 Putting it together: an upload 695
22.13 Common confusions 697
22.14 Summary 698
22.15 Exercises 698

23 Copy, Clone and the Exponential 700

23.1 Too affine to be useful 701
23.2 The exponential modality 701
23.3 `Copy` : contraction for free 703
23.4 `Clone` : duplication as a term 706
23.5 `Rc` and `Arc` : counted sharing 708
23.6 The formal statement 709
23.7 `Copy` in the `affine` crate 715
23.8 Putting it together: a measurement session 719
23.9 Common confusions 722
23.10 Summary 722
23.11 Exercises 723

24 Drop: Weakening with a Side Effect 724

24.1 Discarding is not free 725
24.2 The rules of drop order 728
24.3 Drop glue, and two things you cannot do 732
24.4 Affine, not linear 735
24.5 What linearity would buy 739
24.6 Lean: memory without destructors 740
24.7 A drop log in Lean 743
24.8 Putting it together: a transaction, from Rust to the log 751
24.9 Common confusions 752

24.10 Summary 752
24.11 Exercises 753

25 Typestate: Protocols in Types 755
25.1 Protocols as state machines 756
25.2 Typestate in Rust: a connection 757
25.3 States that carry data: a lab instrument 762
25.4 Builders with required fields 764
25.5 Session types, briefly 767
25.6 Where affinity falls short 771
25.7 Typestate in Lean: indexed families 773
25.8 Protocols as inductive relations 777
25.9 Putting it together: the lab instrument, end to end 782
25.10 Common confusions 783
25.11 Summary 784
25.12 Exercises 784

Part VI – Borrowing and Regions

26 References, Aliasing and Regions 787
26.1 What goes wrong with references 787
26.2 Why affinity is not enough 792
26.3 A heap language that goes wrong 794
26.4 Regions 800
26.5 Lifetimes are region variables 807
26.6 Permissions, not multiplicities 811
26.7 Interior mutability 814
26.8 Putting it together: one bug, four checkers 815
26.9 Common confusions 816
26.10 Summary 817
26.11 Exercises 817

27 Subtyping and Variance 819
27.1 Subtyping and subsumption 820
27.2 Rust's only subtyping: lifetimes 820
27.3 Variance 821
27.4 Why `&mut T` must be invariant 824
27.5 The variance of your own types 826
27.6 Variance as functoriality 828
27.7 Variance in Lean 832
27.8 A tiny language with subtyping 836
27.9 Coercions are not subtyping 840
27.10 Putting it together: the variance of a raw-pointer view 845

27.11 Common confusions 846
27.12 Summary 847
27.13 Exercises 847

28 A Core Calculus for Borrowing 849
28.1 What we need from Parts V and VI 850
28.2 The design of the calculus 850
28.3 Syntax 851
28.4 `Copy` and context splitting 854
28.5 The typing rules 856
28.6 Operational semantics 862
28.7 Metatheory 866
28.8 Worked derivation I: a `&mut` reborrow 868
28.9 Worked derivation II: the aliasing bug 873
28.10 The metatheorem: a mutable reference is used at most once 878
28.11 Putting it together: a checker for the calculus 881
28.12 Common confusions 882
28.13 Summary 883
28.14 Exercises 883

29 How rustc Really Checks Borrows 885
29.1 A type system or an analysis? 886
29.2 From source text to MIR 887
29.3 Dataflow analysis 896
29.4 Move analysis 898
29.5 Liveness 902
29.6 Dataflow, proved 904
29.7 Non-lexical lifetimes 909
29.8 Loans and conflicts 916
29.9 Two-phase borrows 920
29.10 Problem case #3 922
29.11 Polonius 924
29.12 Putting it together: `branch_borrow`, end to end 927
29.13 Common confusions 929
29.14 Summary 929
29.15 Exercises 930

Part VII – Beyond

30 Dependent Types 932
30.1 Types that mention terms 933
30.2 Indexed families 935
30.3 Lean's vectors and indexing with proofs 941

30.4 Equality as a type 943
30.5 The λ-cube 947
30.6 Rust's approximations 949
30.7 Matrices with dimensions in their types 961
30.8 Putting it together: shapes from data 964
30.9 Common confusions 966
30.10 Summary 966
30.11 Exercises 967

31 Proving Programs Correct 969
31.1 Specifications as types 969
31.2 Induction, revisited 970
31.3 Termination 973
31.4 A verified insertion sort 976
31.5 A verified binary search 982
31.6 Running verified code 985
31.7 Putting it together: model in Lean, implement in Rust 986
31.8 The landscape: verifying Rust itself 991
31.9 What a proof gives you, and what it does not 992
31.10 Common confusions 993
31.11 Summary 993
31.12 Exercises 994

32 Soundness, unsafe and Semantic Types 996
32.1 Syntactic soundness and its limits 997
32.2 Semantic types: types as predicates 998
32.3 Verifying a library that uses the primitive 1004
32.4 `unsafe` as a proof obligation 1006
32.5 Separation logic 1013
32.6 RustBelt 1019
32.7 Aliasing models: Stacked Borrows, Tree Borrows and Miri 1021
32.8 What types do not guarantee 1021
32.9 Where affinity reappears 1022
32.10 Putting it together: from `split_at_mut` to the frame rule 1024
32.11 Common confusions 1025
32.12 Summary 1025
32.13 Exercises 1025

33 The Verdict 1027
33.1 The question, made precise 1027
33.2 Ownership: affine, with three amendments 1028
33.3 Borrowing: a different theory 1030

33.4 How the compiler really does it 1033
33.5 Three senses, three answers 1033
33.6 The verdict in tables 1034
33.7 Where affinity reappears 1035
33.8 How to say it correctly 1036
33.9 Two languages, one idea, revisited 1036
33.10 Where to go next 1037
33.11 Summary 1037
33.12 Exercises 1038

Appendices

A Setting Up Rust and Lean 1040
A.1 The repository 1040
A.2 Rust 1040
A.3 Lean and Mathlib 1041
A.4 An editor for Lean 1042
A.5 Running and verifying the code 1043
A.6 Building the book 1044
A.7 Summary 1044

B Lean 4 for Rust Programmers: Cheat Sheet 1045
B.1 Syntax side by side 1046
B.2 Functions and definitions 1046
B.3 Structures and inductive types 1047
B.4 Option, Except and `do` 1048
B.5 Collections 1048
B.6 Type classes 1048
B.7 Imperative code 1049
B.8 Proofs 1049
B.9 Commands 1050
B.10 Things that surprise Rust programmers 1050

C Notation and Dictionary 1052
C.1 Conventions 1052
C.2 Judgments and contexts 1055
C.3 Logic 1057
C.4 Type formers 1058
C.5 The λ-calculus and evaluation 1059
C.6 Substructural logic 1061
C.7 Categories 1062
C.8 Regions, lifetimes and permissions 1064
C.9 Dependent types 1065

C.10 Lean notation 1065
C.11 Dictionary: logic, types, categories, Rust, Lean 1066
C.12 Dictionary: resources, ownership and borrowing 1068

D The Tactics Used in This Book 1071
D.1 Reading a goal 1071
D.2 Closing goals 1071
D.3 Introduction rules: working on the goal 1073
D.4 Elimination rules: working on hypotheses 1074
D.5 Rewriting and simplification 1075
D.6 Structuring a proof 1076
D.7 Recursion and termination 1077
D.8 Core Lean and Mathlib 1077
D.9 Searching and debugging 1077

E Glossary 1078
E.1 A 1078
E.2 B 1079
E.3 C 1079
E.4 D 1081
E.5 E 1083
E.6 F 1083
E.7 G 1084
E.8 H 1084
E.9 I 1085
E.10 J 1086
E.11 K 1086
E.12 L 1086
E.13 M 1087
E.14 N 1088
E.15 O 1089
E.16 P 1089
E.17 R 1090
E.18 S 1091
E.19 T 1093
E.20 U 1094
E.21 V 1095
E.22 W 1095
E.23 Y 1095
E.24 Z 1095

F Further Reading 1096

F.1 Lean and Mathlib . . . 1096
F.2 Type theory and programming languages . . . 1097
F.3 Logic and the Curry–Howard correspondence . . . 1097
F.4 Category theory . . . 1098
F.5 Polymorphism, inference and abstraction . . . 1098
F.6 Rust, ownership and regions . . . 1099
F.7 Verifying Rust programs . . . 1099

Preface

If you write Rust, you already spend a good part of your day in conversation with a type checker. It tells you that a value was moved, that a reference does not live long enough, that a trait is not implemented for a type, that two branches of an `if` disagree. Over time you learn to anticipate it, and then to use it: you design types so that the checker rejects the mistakes you are afraid of making. At that point a natural curiosity sets in. *Why* are these the rules? Where do they come from? Is there a theory behind them, and could you learn it?

There is, and you can. This book teaches it.

The theory is called **type theory**, and it has two close relatives that turn out to be the same subject seen from different angles: **logic**, where types are propositions and programs are proofs, and **category theory**, where types are objects and programs are arrows between them. Behind Rust's most distinctive feature, ownership, stands a fourth subject that grows out of the first three: **substructural logic**, the logic of resources that can be used at most once.

Most books on these subjects use Haskell, OCaml, Coq or pure mathematics. This one uses the language you already know, and a second language that makes the theory concrete.

Two languages

Rust is where the ideas live in production. Its type system is unusually rich: algebraic data types, generics that behave like System F, traits that are type classes, `impl Trait` and `dyn Trait` that are existential types, an affine ownership discipline, lifetimes that are regions, variance, const generics. Each of these is a chapter of type theory, and Rust lets you poke at it: write a program, see whether it compiles, read the error. Rust also deviates from the clean theory in instructive ways (every type is inhabited by `panic!()`, type inference stops at function boundaries, there are no higher-kinded types, the borrow checker is a dataflow analysis rather than a type system), and the book pays as much attention to the deviations as to the matches.

Lean 4 is where the ideas become precise. Lean is a functional programming language with *dependent types*, types that can mention values, and that makes it a proof assistant too: a proposition is a type, a proof is a program of that type, and Lean checks the proof the same way it checks any program. In this book Lean is the laboratory. Each toy language we study is defined in Lean, its typing rules are written as Lean definitions, and its properties are stated as theorems and proved. When the book says "the type checker is sound" or "a well-typed program never gets stuck" or "these two types are isomorphic", there is a Lean proof of it in the repository, and Lean has checked every step.

Lean comes with **Mathlib**, the large library of mathematics that its community maintains, which also supplies many of the *tactics* (automated proof steps) that make proofs short and readable. The book uses Mathlib from Chapter 2 on, and introduces it gently. Where building a definition yourself is the best way to understand it (a toy language, a typing judgment, the notion of a category), we build it from scratch first, and then look at the corresponding definition in Mathlib, so that you can carry on with the real library when you finish the book.

The two languages complement each other. Rust implements; Lean proves. A Rust test can check a property on ten thousand inputs; a Lean theorem establishes it for all of them. A Rust type checker is fast and gives good error messages; a Lean formalization of the same rules tells you exactly what those rules guarantee.

Who this book is for

The book is written for programmers who:

- use Rust actively: ownership and borrowing, structs and enums, traits and generics, closures, iterators, `Result`-based error handling and Cargo. You do not need to be an expert. When the book uses one of Rust's less familiar features (variance, higher-ranked trait bounds, generic associated types, `PhantomData`, `unsafe`), it explains it.
- know little or nothing about Lean, type theory, logic or category theory. Everything is built from first principles: the first inference rule appears in Chapter 1, the first Lean program in Chapter 1, the first category in Chapter 16.
- want to *understand*: to see the real definitions and the real proofs, not a tour of slogans. The book does not water the subject down. It goes slowly instead, and it explains every step.

You will need some patience with notation and a willingness to read a proof line by line. You will not need any mathematics beyond what a programmer already uses: induction on natural numbers and on trees, sets, functions and the idea of a proof. Everything else is introduced when it is needed.

What you will build

Every chapter is organized around code that compiles, runs, or is checked by Lean. Among other things you will build:

- a sequence of **toy languages**, each implemented in Rust and formalized in Lean: a small expression language with a type checker proved sound; the untyped λ-calculus with Church numerals; the simply typed λ-calculus, with a bidirectional checker in Rust, an intrinsically typed evaluator in Lean and a complete mechanized proof of type safety; a mini-ML with Hindley–Milner type inference;
- **proof tools**: a natural-deduction proof checker, a search for Kripke countermodels that shows which classical laws fail constructively, and a proof search for linear logic that shows why $A \vdash A \otimes A$ is not provable;

- a **library of category theory** in Lean: categories, functors, natural transformations, the Yoneda lemma, cartesian closed categories, the Kleisli category of a monad, each with its laws proved;
- the **formal core of ownership**: an affine λ-calculus with `Copy` types and its "use at most once" theorem, a model of drops, typestate protocols, a core calculus of borrows and lifetimes with derivations checked in Lean, and a miniature MIR with a non-lexical borrow checker that reproduces rustc's errors;
- **verified programs**: a sort and a binary search proved correct in Lean, with Rust implementations tested against the verified models, and a small separation logic.

How the book is organized

Part I — Foundations (Chapters 1–6) introduces the two languages, then judgments, inference rules and derivations, natural deduction, intuitionistic logic and the meaning of constructive proof, and the structural rules.

Part II — Programs as Proofs (Chapters 7–11) covers the λ-calculus, the simply typed λ-calculus and Rust's closures, a full proof of type safety, the Curry–Howard correspondence, and algebraic data types.

Part III — Abstraction (Chapters 12–15) covers parametric polymorphism, type inference, traits and type classes, and existential types.

Part IV — Categories (Chapters 16–20) introduces category theory for programmers: categories, functors and higher-kinded types, natural transformations and free theorems, cartesian closed categories, and monads.

Part V — Ownership as Logic (Chapters 21–25) covers substructural logics, the affine λ-calculus, `Copy` and `Clone` as the exponential modality, `Drop` as weakening with a side effect, and typestate.

Part VI — Borrowing and Regions (Chapters 26–29) covers references and regions, subtyping and variance, a core calculus for borrowing, and how rustc really checks borrows.

Part VII — Beyond (Chapters 30–33) covers dependent types, proving programs correct, soundness and semantic types, and ends with the verdict on the question asked in Chapter 1.

The **appendices** cover setup, a Lean cheat sheet for Rust programmers, notation and a dictionary between the fields, the tactics used in the book, a glossary and further reading.

Read Part I first, and Chapter 2 in particular: it is your introduction to Lean. Part II should come next. After that the parts can be read in several orders: Part IV needs only Parts I and II; Parts V and VI need Parts I and II and are best read together; Part VII draws on everything. Chapter 1 has a map.

Every chapter starts with a short list of what you will learn. It then explains the concepts, walks through complete examples, discusses common confusions, and ends with a summary and exercises with hints. The exercises mix Rust, Lean and pen-and-paper work; doing a few of them in each chapter is the fastest way to make the material your own.

Conventions

Code that you can run or check is always part of the book's repository. When a listing comes from a file, the text names the file the first time, for example `code/rust/examples/ch01_first_types.rs` or `code/lean/Book/Ch01/FirstSteps.lean`. Listings are copied from these files when the book is built, so the code you read is the code that compiles.

Commands you type in a terminal look like this:

```
cargo run --example ch01_first_types
```

The output of a Rust program is shown in a plain block. Every such block is a snapshot of a real run, and the repository's tools check that the program still produces exactly that output:

```
add(2, 3) = 5
```

Some Rust programs in the book are *supposed* to be rejected, because the rejection is the point. They live in `code/rust/compile_fail/`, and the diagnostics shown in the book were captured from the real compiler:

```
error[E0308]: `if` and `else` have incompatible types
```

Lean code is shown with its output written right above the command that produces it, in a special comment that Lean itself checks. In this block, Lean evaluates `2 + 3`, and the build fails unless the result is exactly the `5` written in the comment:

```
/-- info: 5 -/
#guard_msgs in
#eval 2 + 3
```

The same mechanism is used to show the errors Lean reports for programs and proofs it rejects. Chapter 1 explains it in detail.

Mathematics is written in the usual notation. Inference rules are written as fractions with the premises above the line, the conclusion below it and the name of the rule on the right:

$$\frac{\Gamma \vdash f : \sigma \to \tau \qquad \Gamma \vdash a : \sigma}{\Gamma \vdash f\ a : \tau}\ \text{T-App}$$

Appendix C lists every symbol used in the book.

Throughout the text you will find a few kinds of highlighted notes:

> **NOTE**
>
> Background information, history, or a detail that you can skip on a first reading.

> **TIP**
>
> A practical shortcut or a good habit.

> **WARNING**
>
> A common mistake or a subtle point that is easy to get wrong.

The code and how to run it

The repository has this layout:

```
book/              the chapters (Markdown sources, generated Markdown and PDFs)
code/
├── rust/          a Cargo workspace: examples/, crates/, compile_fail/
└── lean/          a Lake project: Book/Ch01/ ... Book/Ch33/
tools/             scripts that build the book and verify the code
```

Both toolchains are pinned: `code/rust/rust-toolchain.toml` selects Rust 1.99.0 and `code/lean/lean-toolchain` selects Lean 4.34.1. If you use rustup and elan (Appendix A explains how to install them), the right versions are downloaded automatically the first time you run `cargo` or `lake` in those directories.

```
cd code/rust
cargo run --example ch01_first_types          # single-file examples
cargo run -p stlc --example ch08_stlc         # examples that use a crate
cargo test -p arith                           # a crate's tests

cd ../lean
lake exe cache get                            # once: download prebuilt Mathlib
lake build                                    # check every Lean file
```

The Rust code uses only the standard library, so it builds in seconds. The Lean code depends on Mathlib, which is big: `lake exe cache get` downloads the prebuilt library once (under a gigabyte to download, about 8 GB on disk with its sources), after which checking the book's Lean files takes a few minutes. To verify the whole book at once (every example, every rejected program, every Lean file):

```
python3 tools/run_examples.py
```

The best way to read the Lean parts of the book is with the Lean files open in an editor (VS Code with the Lean 4 extension), so that you can put the cursor inside a proof and watch the goals change. Appendix A explains how to set this up.

A note on versions and accuracy

Rust is stable and backwards compatible; the programs in this book should keep compiling for a long time, although the exact wording of compiler diagnostics changes between releases. Lean 4 is younger and changes faster: names in its standard library are occasionally renamed, tactics grow more powerful, and the format of error messages evolves. Everything in this book was checked with the pinned versions. If you use a newer Lean and a proof breaks, the `lean-toolchain` file lets you go back to the version the book was written with.

On the theory side, the book states typing rules "for Rust" in many places. Rust has no official formal semantics, so such rules are *models* of what the compiler does, in the spirit of the research calculi that

formalize Rust's core. The text says so when it matters, and uses the symbol ≈ for "corresponds to, with caveats stated in the text".

Acknowledgements

This book stands on the work of the people who built the subject and the tools: the logicians and computer scientists whose ideas fill its chapters, the Rust project and its contributors, and the Lean community, whose documentation, examples and library made it possible to check every proof in these pages.

PART I

Foundations

Part I lays the foundations that everything else rests on. It starts with the two languages of the book: what a type theory is, why we study it through Rust and Lean together, and enough Lean to read and write the code of the coming chapters. Then it builds the basic apparatus of the subject from the ground up: judgments, inference rules and derivations; natural deduction, the proof system in which every later logic is written; intuitionism, which explains what a *constructive* proof is and why proofs can be programs; and the structural rules, three innocent-looking rules about assumptions whose removal will eventually give us ownership.

Every idea in this part is executed twice: implemented in Rust (a type checker, a proof checker, a search for countermodels) and formalized in Lean, where the first real theorems of the book are proved.

CHAPTER 1

Two Languages, One Idea

Every time the Rust compiler rejects one of your programs, it has done something remarkable. It has looked at a program it has never seen, considered every way that program could run, and concluded that at least one of them could go wrong. A value used after it was moved, a reference that outlives the data it points to, a `String` where an `i32` was expected: the compiler did not find these by running your code and watching it fail. It found them by *reasoning* about the code, following a small set of rules, before the program ran even once.

Those rules form a **type system**, and the mathematical study of type systems is called **type theory**. This book is about type theory and about its two closest relatives, logic and category theory, which turn out to be the same subject seen from different angles. It is also about the type system you use every day. Rust's is one of the richest in any mainstream language, and almost every part of it, from enums and generics to ownership and lifetimes, is a chapter of type theory in disguise.

We will study all of this with two languages side by side. Rust shows where the ideas live in a production language, and where that language deviates from the clean theory. Lean, a functional programming language that is also a proof assistant, lets us state the ideas precisely and have a machine check every proof. This first chapter introduces the basic vocabulary of type theory, takes a first look at both languages, and asks the question that the rest of the book will answer.

In this chapter you will learn:

- what a type theory is made of: syntax, judgments, inference rules and computation;
- how to read a typing rule and build a derivation, using Rust's own expressions as examples;
- the three readings of a type: as a classification of values, as a proposition and as an object;
- why this book pairs Rust with Lean, and what each language can do that the other cannot;
- how to evaluate, check and prove things in Lean, and how the book checks its own outputs;
- the question that runs through the whole book, and a map of how we will answer it.

1.1 What a type theory is made of

A type theory is a formal system, which means a precisely defined game played with symbols. Every type theory, from the smallest one in Chapter 3 to the one inside Lean, has four ingredients.

1. **Syntax.** The *terms* (also called expressions or programs) and the *types*. In Rust, `x + 1`, `if b { 1 } else { 2 }` and `|s: &str| s.len()` are terms; `i32`, `bool` and `Vec<String>` are types.

2. **Judgments.** Statements that the system can establish. The judgment you will see most often is the *typing judgment*, written $\Gamma \vdash e : \tau$ and read "in the context Γ, the term e has type τ".
3. **Inference rules.** Recipes that say how a judgment follows from other judgments.
4. **Computation rules.** Rules that say what terms *do*: how `2 + 3` becomes `5`, how a function call becomes its body.

Type safety, the property that makes a type system worth having, is a statement about how the third ingredient constrains the fourth. Let us look at each of them in turn.

Contexts and judgments

A term usually mentions variables, and its type depends on what we know about them. The term `x + y` has type `i32` if `x` and `y` do, and no type at all if `x` is a `String`. The **context** Γ (a capital Greek gamma) records what we assume about the free variables:

$$\Gamma = x_1 : \tau_1, x_2 : \tau_2, \ldots, x_n : \tau_n$$

The turnstile $\vdash$ separates the assumptions from the conclusion. So

$$x : \texttt{i32}, y : \texttt{i32} \vdash x + y : \texttt{i32}$$

is a judgment that says: assuming `x` and `y` are `i32`, the term `x + y` is an `i32`. It is important to understand what a judgment is *not*. It is not a piece of the program; it is a statement *about* the program, made in a language one level up (the *metalanguage*). It is not true or false the way `x > 0` is; it is *derivable* or not, according to the rules.

You have read judgments like this one many times without the notation. A function signature is one:

```rust
/// The signature is a typing judgment for the body: assuming `x: i32` and
/// `y: i32`, the expression `x + y` has type `i32`.
fn add(x: i32, y: i32) -> i32 {
    x + y
}
```

The parameter list is the context, the return type is the type, the body is the term. The signature promises: "under the assumptions `x: i32` and `y: i32`, the body has type `i32`". The compiler's job is to check that promise.

Inference rules

The rules of a type theory are written in a two-dimensional notation borrowed from logic. Here are five rules for a small fragment of Rust: integer literals, variables, addition, `if` and `let`.

$$\frac{}{\Gamma \vdash n : \texttt{i32}}\ \textsc{T-Lit} \qquad \frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{T-Var}$$

$$\frac{\Gamma \vdash e_1 : \texttt{i32} \qquad \Gamma \vdash e_2 : \texttt{i32}}{\Gamma \vdash e_1 + e_2 : \texttt{i32}}\ \textsc{T-Add}$$

$$\frac{\Gamma \vdash c : \texttt{bool} \qquad \Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \texttt{if}\ c\ \{e_1\}\ \texttt{else}\ \{e_2\} : \tau}\ \textsc{T-If}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1; e_2 : \tau_2}\ \textsc{T-Let}$$

Each rule has **premises** above the line and a **conclusion** below it. A rule says: if you have established all the premises, you may conclude the conclusion. The name on the right is just a label. A rule with no premises, like T-Lit, is an **axiom**: its conclusion holds outright. Read aloud:

- **T-Lit**: an integer literal has type `i32`, in any context. (Rust is subtler, with literal types inferred from context; this is a model of a fragment.)
- **T-Var**: a variable has the type the context assigns to it.
- **T-Add**: if both operands have type `i32`, so does their sum.
- **T-If**: if the condition is a `bool` and *both branches have the same type* τ, the whole `if` has type τ.
- **T-Let**: if e_1 has type τ_1, and e_2 has type τ_2 *when we additionally assume* $x : \tau_1$, then `let x = e1; e2` has type τ_2. The comma in $\Gamma, x : \tau_1$ means "extend the context". This is how a `let` brings a variable into scope.

The letters Γ, e_1, τ and so on are **metavariables**: placeholders for any context, term or type. A rule is a schema that stands for infinitely many concrete instances. T-Add with $e_1 := x$ and $e_2 := 1$ is one instance; with $e_1 := x + y$ and $e_2 := z$ it is another.

Derivations

A **derivation** is a tree of rule instances. Its root is the judgment we want, each node is justified by a rule whose premises are the node's children, and the leaves are axioms (or premises like $(x : \texttt{i32}) \in \Gamma$ that can be checked directly). Here is the derivation for the body of `add`, with $\Gamma = x : \texttt{i32}, y : \texttt{i32}$:

$$\dfrac{\dfrac{(x : \texttt{i32}) \in \Gamma}{\Gamma \vdash x : \texttt{i32}}\ \textsf{T-Var} \qquad \dfrac{(y : \texttt{i32}) \in \Gamma}{\Gamma \vdash y : \texttt{i32}}\ \textsf{T-Var}}{\Gamma \vdash x + y : \texttt{i32}}\ \textsf{T-Add}$$

Read it from the bottom: to show that `x + y` is an `i32`, use T-Add, which requires showing that `x` and `y` are `i32`; each of those follows from T-Var because the context says so. Every judgment the compiler accepts has a derivation like this one behind it, and **type checking is the search for a derivation**. When the search fails, you get an error message, which is the compiler's description of where the search got stuck.

Derivations will be everywhere in this book. In Chapter 3 we build a type checker in Rust that prints them, and in Lean we will represent them as data, construct them, and prove theorems about all of them at once.

Computation and type safety

The fourth ingredient says how programs run. We write $e \to e'$ for "e takes one step of computation and becomes e'":

$$2 + 3 \to 5 \qquad\qquad \texttt{if true}\ \{e_1\}\ \texttt{else}\ \{e_2\} \to e_1$$

A term that cannot step any further is either a **value** (a finished result such as `5` or `true`) or **stuck**: a nonsensical state such as `true + 5`, where no rule applies but the computation is not done. Getting stuck is the formal version of a crash, or worse.

The point of a type system is the **type safety** theorem, usually split in two:

- **Progress.** A well-typed closed term is either a value or can take a step. It is never stuck.
- **Preservation.** If a well-typed term takes a step, the result is well typed, with the same type.

Together they say that a well-typed program, run for any number of steps, never reaches a stuck state. Robin Milner put it in a famous slogan in 1978: *well-typed programs cannot "go wrong"*. What counts as "going wrong" depends on the semantics. For safe Rust it includes reading freed memory, data races and using uninitialized values, collectively called *undefined behavior*. It does not include panics, deadlocks, memory leaks or infinite loops: Rust's type system makes no promise about those, and we will see why in later chapters.

In Chapter 9 we will state progress and preservation for a small language and prove them completely in Lean. For now, keep the shape of the idea: rules that only look at the program text (typing) guarantee something about every possible run (computation).

1.2 Rust's expressions, read as judgments

Rust is an *expression-oriented* language: almost everything has a value and a type, including blocks, `if`, `match` and loops. That makes it a good place to practice reading rules. This chapter's Rust example, `code/rust/examples/ch01_first_types.rs`, walks through a few cases.

Blocks and the unit type

A block `{ s1; s2; e }` is an expression. Its statements are checked in order, each `let` extending the context the way rule T-Let does, and its type and value are those of the final expression `e`. If the block ends with a statement (a semicolon) instead of an expression, it has no final expression, and its value is `()`, the only value of the **unit type** `()`:

```
fn blocks() {
    // A block is an expression; its value is the value of its last expression.
    let area = {
        let width = 3;
        let height = 4;
        width * height
    };
    // When the block ends with a statement (note the semicolon), there is no
    // final expression: the block evaluates to the unit value `()`.
    let nothing = {
        let width = 3;
        println!("inside the block, width = {width}");
    };
    println!("area    = {area:<4} : {}", type_name_of_val(&area));
    println!("nothing = {nothing:<4?} : {}", type_name_of_val(&nothing));
}
```

```
inside the block, width = 3
area    = 12   : i32
nothing = ()   : ()
```

(The output comes from running the example; `type_name_of_val` is a standard library function that returns the name of a value's type.) The unit type has exactly one value, so a value of type `()` carries no information. It is what a function returns when it is called only for its effects. In type theory it is usually written $\mathbf{1}$, and in Chapter 11 you will see why it deserves the name of a number.

`if` and rule T-If

Since `if` is an expression, it has a type, and rule T-If says what that type is: both branches must have the same type τ, and then so does the `if`:

```
/// `if` is an expression: both branches must have the same type, and that
/// type is the type of the whole `if`.
fn sign(n: i64) -> &'static str {
    if n < 0 {
        "negative"
    } else if n == 0 {
        "zero"
    } else {
        "positive"
    }
}
```

Rule T-If is strict: it asks for *one* τ that fits both branches. This program violates it:

```
fn describe(ready: bool) {
    let status = if ready { 1 } else { "not yet" };
    println!("{status}");
}
```

The first branch has an integer type and the second has type `&str`, and no single type fits both, so no instance of T-If applies and no derivation exists. The compiler reports exactly that:

```
error[E0308]: `if` and `else` have incompatible types
 --> compile_fail/ch01_if_branches.rs:6:40
  |
6 |     let status = if ready { 1 } else { "not yet" };
  |                           -          ^^^^^^^^^^ expected integer, found `&str`
  |                           |
  |                           expected because of this
```

Notice what the compiler did *not* do. It did not look for some common supertype of the two branches, the way a language with subtyping might, nor did it decide the type at run time based on which branch is taken, the way a dynamically typed language would. Both choices are possible designs, and both have costs: the first needs a subtyping relation (Chapter 27), the second gives up checking before the program runs. Rust's rule is the simplest one, and it is the same rule we will write in every type theory in this book.

The never type

Some expressions never produce a value at all: `panic!(...)`, `loop {}` without a `break`, `return`, `break` and `continue`. Rust gives them the **never type**, written `!`. The never type has *no* values; an expression of type `!` can only diverge, exit or jump elsewhere.

That is exactly what makes it useful. Look at this function:

```
/// `parse_digit` returns a `u32` on every path. The `panic!` branch has type
/// `!` (never), which has no values, so it can stand in for a `u32`.
fn parse_digit(c: char) -> u32 {
    match c.to_digit(10) {
        Some(d) => d,
        None => panic!("not a digit: {c:?}"),
    }
}
```

The `match` has two arms, and by the same reasoning as T-If they must have the same type. The first arm is a `u32`. The second is a `panic!`, of type `!`. The compiler accepts it because `!` **coerces to any type**: since no value of type `!` can ever exist, pretending that this arm produces a `u32` can never cause a problem. In logic this principle is ancient: from a contradiction, anything follows (*ex falso quodlibet*). In type theory, the type with no values is written **0** or $\bot$, and it is the type-theoretic version of "false". You will meet it again in almost every chapter of Part I and Part II.

> **NOTE**
>
> The never type `!` is only partly stable: you can write `fn f() -> !`, but not yet use `!` as an ordinary type such as `Vec<!>`. The standard library provides `std::convert::Infallible`, an `enum` with no variants, as a stable type with no values, and you can define your own: `enum Void {}`. Chapter 5 uses one.

Rules are a model

The rules we wrote are a simplified *model* of what the Rust compiler does. The real compiler infers the types of integer literals, inserts coercions, autoreferences method receivers, and does a great many other things that our five rules ignore. Rust has no official formal semantics: there is a reference manual written in English and a compiler that is believed to implement it. Researchers have built formal *core calculi* that capture the essential parts of Rust, and we will build several small ones ourselves. When this book writes a typing rule "for Rust", it is always a model in this sense, and the text will point out where the model and the compiler differ.

1.3 Three readings of a type

So far we have treated types the way programmers do: as a classification of values. `bool` is the set of the two values `true` and `false`; `(i32, bool)` is the set of pairs; a function type is the set of functions. A type system is a lightweight, automatic way to rule out programs that mix up these sets.

That is the first reading. Two more readings will change how you see types.

The logician's reading: types are propositions. Look at this Rust function from the chapter's example:

```
/// For all types `A` and `B`: given an `A` and a `B`, produce a `B` and an `A`.
/// Read as logic: "if A and B, then B and A". Compare with `and_swap` in
/// `Book/Ch01/FirstSteps.lean`.
fn swap<A, B>((a, b): (A, B)) -> (B, A) {
    (b, a)
}
```

Ignore what it does and read its type as a sentence: "for all `A` and `B`, if you give me an `A` and a `B`, I can give you a `B` and an `A`". Now replace "give me a value of type" with "assume", and "a pair" with "and": "for all propositions A and B, if A and B, then B and A". That is a theorem of logic, and the function body is its proof: it shows how to turn evidence for "A and B" into evidence for "B and A". This is the **Curry–Howard correspondence**: types are propositions, programs are proofs, and a type is "true" exactly when some program has that type. Part II is devoted to it.

The category theorist's reading: types are objects. Category theory does not look inside types at all. It looks at how types are connected by functions: a function `f: A -> B` is an *arrow* from `A` to `B`, arrows can be composed, and there is an identity arrow on every type. From this point of view a pair type `(A, B)` is not "a set of pairs" but "the type with two projections such that every pair of functions into `A` and `B` factors uniquely through it". That sounds needlessly abstract, but it is the kind of description that works in *every* setting at once: for types and functions, for propositions and proofs, for sets and relations. Part IV develops it.

The three readings line up in a table that you will see, in more detail, many times in this book:

Type	As a set of values	As a proposition	As an object
`(A, B)`	pairs	A and B	product
`A -> B`	functions	A implies B	exponential
`enum Either<A, B>`	tagged union	A or B	coproduct
`()`	one value	true	terminal object
`!` / `enum Void {}`	no values	false	initial object

The remarkable fact, which took logicians and computer scientists most of the twentieth century to see clearly, is that these are not three subjects with a loose analogy between them. They are one subject.

1.4 Why Rust and Lean

What Rust brings

Rust's type system collects ideas from fifty years of programming-language research and puts them in the hands of working programmers:

- **algebraic data types** (structs, enums, `match`), from the ML family (Chapter 11);
- **parametric polymorphism**, generics that behave like System F, implemented by monomorphization (Chapter 12);
- **local type inference**, a deliberately restricted cousin of Hindley–Milner (Chapter 13);
- **traits**, which are type classes, the design that Haskell introduced (Chapter 14);
- **existential types**, as `impl Trait` and `dyn Trait` (Chapter 15);
- an **affine ownership discipline**: moves, `Copy`, `Clone` and `Drop` (Part V);
- **regions** and **permissions**, as lifetimes and the rules for `&` and `&mut` (Part VI);
- **subtyping and variance**, on lifetimes only (Chapter 27);
- a first step toward **dependent types**, as const generics (Chapter 30).

Because these are features of a real language, you can experiment with them: write a program, compile it, read the error. And because Rust is a real language with real constraints (performance, backwards compatibility, usability), it deviates from the textbook versions of these ideas in interesting ways. Every type in Rust has a value, `panic!()`, which makes Rust useless as a logic (Chapter 10). Type inference stops at function signatures (Chapter 13). There are no higher-kinded types, so there is no `Functor` trait (Chapter 17). The borrow checker is not a type system in the textbook sense at all, but a

dataflow analysis (Chapter 29). We will pay as much attention to these deviations as to the matches: they show what the theory costs in practice.

What Lean brings

Lean 4 is a functional programming language. You can write ordinary programs in it, with structures, inductive types (Lean's enums), pattern matching, type classes, `do` notation, mutable-looking loops and `IO`; it compiles to native code, and Lean itself is written in Lean. Chapter 2 introduces it from a Rust programmer's point of view.

What makes Lean different from Rust is that its types are much more expressive. In Lean a type can mention values. There is a type of "arrays of length 3", a type of "natural numbers less than `n`", and, most importantly, a type of "proofs that `2 * sumTo n = n * (n + 1)`". Since propositions are types and proofs are programs, writing a proof in Lean *is* writing a program of a certain type, and checking the proof *is* type checking. A small, carefully engineered part of Lean, its *kernel*, checks every proof, so a theorem that Lean accepts has been verified down to the axioms of its logic.

This makes Lean a *proof assistant*, and the book uses it as a laboratory. Each toy language we study is defined in Lean as a data type. Its typing rules become a Lean definition, so that "a derivation of $\Gamma \vdash e : \tau$" is a Lean value. Its theorems (the checker is sound, evaluation is deterministic, a well-typed term never gets stuck, a variable in an affine term is used at most once) are stated and proved. When the book claims something about a formal system, it usually backs the claim with a Lean proof that you can read, step through and modify.

Lean comes with **Mathlib**, a library of mathematics built and maintained by Lean's community. Think of it as Lean's equivalent of a large, carefully curated set of crates: definitions (numbers, sets, finite sets, functions, algebraic structures, categories), hundreds of thousands of proved theorems about them, and *tactics*, automated proof steps that make proofs short and readable. The book's Lean project depends on it, the way a Cargo project depends on a crate, and we will use it from this chapter on.

Rust implements, Lean proves

Here is the division of labor in miniature. Carl Friedrich Gauss is said to have noticed, as a schoolboy, that $0 + 1 + \ldots + n = n(n+1)/2$. In Rust, we can compute the sum both ways and compare:

```rust
/// The sum 0 + 1 + ... + n, computed by a loop.
fn sum_to(n: u64) -> u64 {
    (0..=n).sum()
}

/// Gauss's closed form for the same sum.
fn gauss(n: u64) -> u64 {
    n * (n + 1) / 2
}
```

```rust
// A test can check the formula for as many values as we have patience for,
// but only for finitely many. Chapter 1's Lean file proves it for all n.
for n in 0..=10_000 {
    assert_eq!(sum_to(n), gauss(n), "formula fails at n = {n}");
}
println!("sum_to(n) == gauss(n) for every n in 0..=10000");
```

```
sum_to(n) == gauss(n) for every n in 0..=10000
```

That is good evidence, and in a real project it would be a good test. But it is evidence about 10,001 numbers, and there are infinitely many. In Lean we can prove the formula for every `n`, as you will see in a moment. The pattern repeats throughout the book: the Rust programs are fast, practical, and checked on many inputs; the Lean developments are slower to write and checked for all inputs. Each makes the other more trustworthy. In Chapter 31 we will use this pattern deliberately, testing a Rust implementation against a verified Lean model.

1.5 A first session with Lean

The Lean code for this chapter is in `code/lean/Book/Ch01/FirstSteps.lean`. You can read it here, but you will get much more out of it with the file open in an editor that runs Lean as you type (Appendix A explains how to set up VS Code with the Lean 4 extension). The editor shows Lean's responses in a side panel called the **infoview**: the output of commands, the type of the expression under the cursor, and, inside a proof, the current goal.

Evaluating and checking

The file starts with the line `import Mathlib.Tactic.Ring`, which loads the part of Mathlib that this chapter uses (the `ring` tactic, and with it Mathlib's notation for numbers). Imports in Lean are like `use` declarations in Rust, except that they name modules of the whole project and its dependencies, and they must come first in the file.

Lean files contain declarations (definitions, theorems) and *commands* that ask Lean something. The two commands you will use most are `#eval`, which runs an expression and prints the result, and `#check`, which prints an expression's type without running it:

```
/-- info: 5 -/
#guard_msgs in
#eval 2 + 3

/-- info: 2 + 3 : ℕ -/
#guard_msgs in
#check 2 + 3

/-- info: fun n => n + 1 : ℕ → ℕ -/
#guard_msgs in
#check fun (n : ℕ) => n + 1
```

Each command is preceded by two extra lines, and this is a convention used throughout the book, so it deserves an explanation. The command `#guard_msgs in` tells Lean: "run the next command, capture everything it prints, and compare it with the doc comment above". The doc comment `/-- info: 5 -/` is the expected output. If the output differs in any way, Lean reports an error and the book's build fails. So the outputs you see in this book are not copied by hand: Lean checks them every time the project is built.

Read the three outputs. `#eval 2 + 3` prints `5`. `#check 2 + 3` prints `2 + 3 : ℕ`, which is Lean's way of writing a typing judgment with an empty context: the colon means "has type", exactly as in $\vdash 2 + 3 : \mathbb{N}$.

`ℕ` is the type of natural numbers $0, 1, 2, \ldots$, which Lean uses by default for numerals. (Its name in Lean's core library is `Nat`; `ℕ` is a notation that Mathlib adds, and Lean uses it when printing. You can type it in the editor as `\N`.) The third line checks an anonymous function, written `fun (n : ℕ) => n + 1` (Rust would write `|n: u64| n + 1`), and reports its type `ℕ → ℕ`.

A first definition

Definitions use `def`. Here is Rust's `add`, in Lean:

```
/-- The same judgment as Rust's `fn add(x: i32, y: i32) -> i32`. -/
def add (x y : ℤ) : ℤ := x + y
```

```
/-- info: Ch01.add (x y : ℤ) : ℤ -/
#guard_msgs in
#check add

/-- info: 5 -/
#guard_msgs in
#eval add 2 3
```

The parameters `(x y : ℤ)` are the context, `ℤ` after the colon is the result type, and the body follows `:=`. `ℤ` (Mathlib's notation for Lean's `Int`) is the type of all integers, without overflow: Lean's integers are arbitrary precision, and fixed-width types like `Int32` exist too. The definition lives in the namespace `Ch01` (every file of the book wraps its contents in a namespace named after its chapter), so its full name is `Ch01.add`.

Rule T-If holds in Lean too, and Lean rejects the analogue of our Rust program. The `#guard_msgs` mechanism also checks expected *errors*, which lets the book show you rejected programs that are still part of a file that compiles:

```
/--
error: Application type mismatch: The argument
  "not yet"
has type
  String
but is expected to have type
  ℕ
in the application
  if ready = true then 1 else "not yet"
-/
#guard_msgs in
def status (ready : Bool) : ℕ := if ready then 1 else "not yet"
```

The message is worded differently but says the same thing as Rust's: the `else` branch has type `String`, and it was expected to have type `ℕ`, the type of the whole `if`. (The `ready = true` in the last line is how Lean displays the condition `ready` of type `Bool` when it is used as a proposition; Chapter 5 explains the difference between `Bool` and `Prop`.)

A first theorem

Now the interesting part. Here is the sum $0 + 1 + \ldots + n$, defined by recursion:

```
/-- The sum `0 + 1 + ... + n`, defined by recursion on `n`. -/
def sumTo : ℕ → ℕ
  | 0 => 0
  | n + 1 => (n + 1) + sumTo n
```

```
/-- info: 5050 -/
#guard_msgs in
#eval sumTo 100
```

The definition is written by *pattern matching* on the argument, like a Rust `match`: the sum up to `0` is `0`, and the sum up to `n + 1` is `n + 1` plus the sum up to `n`. Every natural number is either `0` or `n + 1` for some `n`, so the two cases cover everything, and each recursive call is on a smaller number, so the recursion terminates. Lean checks both facts; it does not accept definitions that might not terminate unless you explicitly ask for it (Chapter 2 explains why).

Here is Gauss's formula as a Lean theorem, with its proof:

```
/-- Gauss's formula, for every natural number `n`. (Stated with `2 * ...` on
the left to avoid division, which on `ℕ` rounds down.) -/
theorem gauss (n : ℕ) : 2 * sumTo n = n * (n + 1) := by
  induction n with
  | zero =>
    -- Goal: 2 * sumTo 0 = 0 * (0 + 1). Both sides compute to 0.
    rfl
  | succ k ih =>
    -- ih   : 2 * sumTo k = k * (k + 1)
    -- Goal : 2 * sumTo (k + 1) = (k + 1) * (k + 1 + 1)
    rw [sumTo, mul_add, ih]
    -- Goal : 2 * (k + 1) + k * (k + 1) = (k + 1) * (k + 1 + 1)
    ring
```

A `theorem` looks like a `def`, and that is no accident. The part after the colon, `2 * sumTo n = n * (n + 1)`, is a *type*: the type of proofs of that equation. The part after `:=` is a *program* of that type, a proof. Here the program is written with `by`, which switches to **tactic mode**: instead of writing the proof term directly, we give Lean a sequence of instructions (tactics) that build it. The comments show the **goal**, what remains to be proved, at each point; in an editor you would see them in the infoview as you move the cursor.

Let us go through the proof line by line.

- `induction n with` starts a proof by induction on `n`, exactly as you would on paper. It creates two goals, one for each way a natural number can be built.
- In the `zero` case the goal is `2 * sumTo 0 = 0 * (0 + 1)`. Both sides compute to `0`: `sumTo 0` is `0` by the first equation of the definition. The tactic `rfl` (for *reflexivity*) proves an equation whose two sides compute to the same value.
- In the `succ k` case Lean gives us the **induction hypothesis** `ih`, the formula for `k`, and asks for the formula for `k + 1`.
- `rw [sumTo, mul_add, ih]` rewrites the goal three times, left to right. First it uses the second defining equation of `sumTo` to unfold `sumTo (k + 1)` into `k + 1 + sumTo k`. Then it uses `mul_add`, Mathlib's distributive law $a \cdot (b + c) = a \cdot b + a \cdot c$, to turn `2 * (k + 1 + sumTo k)` into `2 * (k + 1) + 2 * sumTo k`. Finally it uses `ih` to replace `2 * sumTo k` with `k * (k + 1)`. This is the step where the induction hypothesis is used.
- What remains, `2 * (k + 1) + k * (k + 1) = (k + 1) * (k + 1 + 1)`, no longer mentions `sumTo`: it is an identity between polynomials in `k`, true because both sides multiply out to $k^2 + 3k + 2$. The

`ring` tactic from Mathlib proves exactly such identities: it normalizes both sides as polynomials and compares them.

When the last goal is closed, Lean accepts the theorem. If any step were wrong (a rewrite that does not apply, a goal that `ring` cannot prove), Lean would report an error at that line, and the theorem would not exist.

There is more to proofs than tactics, and later chapters build up the toolbox slowly. For now, notice the shape: a theorem is a type, a proof is a program of that type, and the program is checked like any other.

Claims that are true and claims that are false

For a specific number, the equation is a computation, and Lean can check it by computing:

```
example : sumTo 3 = 6 := rfl

example : sumTo 10 = 55 := by decide
```

An `example` is a theorem without a name. The first proof is `rfl`: both sides compute to `6`. The second uses `decide`, which evaluates a decidable proposition to `true` or `false` and succeeds if it gets `true`. If the claim is false, Lean says so:

```
/--
error: Tactic `decide` proved that the proposition
  sumTo 3 = 7
is false
-/
#guard_msgs in
example : sumTo 3 = 7 := by decide
```

Types that carry proofs

Here is something Rust cannot express. In Rust, `xs[0]` on an empty slice panics at run time. In Lean, a function can demand, as an argument, a *proof* that the index is valid:

```
/-- The first element of an array. The second argument is a *proof* that the
array is not empty, so the indexing `xs[0]` cannot fail. -/
def first (xs : Array ℕ) (h : 0 < xs.size) : ℕ := xs[0]

/-- info: 5 -/
#guard_msgs in
#eval first #[5, 6] (by decide)
```

The parameter `h : 0 < xs.size` is a proof that the array is not empty. Its type mentions the value of another parameter, `xs`: this is a **dependent type**, and it is the feature that sets Lean apart from Rust. When we call `first #[5, 6]`, we must supply the proof; `by decide` computes it, because `0 < 2` is a decidable fact. If we try to index without a proof, Lean refuses, and helpfully lists the alternatives:

```
/--
error: failed to prove index is valid, possible solutions:
  - Use `have`-expressions to prove the index is valid
  - Use `a[i]!` notation instead, runtime check is performed, and 'Panic' error message is
produced if index is not valid
  - Use `a[i]?` notation instead, result is an `Option` type
  - Use `a[i]'h` notation instead, where `h` is a proof that index is valid
xs : Array ℕ
```

```
⊢ 0 < xs.size
-/
#guard_msgs in
def firstOrCrash (xs : Array ℕ) : ℕ := xs[0]
```

The options are illuminating: `xs[0]!` panics at run time when the index is invalid (that is what Rust's `xs[0]` does), `xs[0]?` returns an `Option` (that is Rust's `xs.get(0)`), and `xs[0]'h` takes a proof `h`. Lean offers all three, and the third has no counterpart in Rust. Chapter 30 is about dependent types.

Propositions as types, once more

The swap function from the Rust example has a Lean twin:

```
/-- A proposition and its proof. Compare with `swap` in `ch01_first_types.rs`. -/
theorem and_swap (p q : Prop) : p ∧ q → q ∧ p :=
  fun ⟨hp, hq⟩ => ⟨hq, hp⟩
```

`p ∧ q` is the proposition "p and q", and a proof of it is a pair of a proof of `p` and a proof of `q`, written with angle brackets `⟨hp, hq⟩`. The proof of `and_swap` is a function that takes such a pair apart and builds the swapped one: character for character the same program as Rust's `swap`. Chapter 10 explains why, and where the parallel between the two languages breaks down.

Which axioms did we use?

Lean can tell you what a theorem ultimately rests on. The command `#print axioms` lists the axioms that a proof uses, directly or through the library lemmas it relies on. To make the point, here is a second proof of Gauss's formula that leaves the arithmetic to a more powerful automatic tactic, `grind`:

```
/-- The same theorem, with the arithmetic left to the `grind` tactic. -/
theorem gauss' (n : ℕ) : 2 * sumTo n = n * (n + 1) := by
  induction n with
  | zero => rfl
  | succ k ih => simp only [sumTo]; grind
```

```
/-- info: 'Ch01.gauss' depends on axioms: [propext] -/
#guard_msgs in
#print axioms gauss

/-- info: 'Ch01.gauss'' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms gauss'

/-- info: 'Ch01.and_swap' does not depend on any axioms -/
#guard_msgs in
#print axioms and_swap
```

Our `and_swap` uses no axioms at all: it is a program built from the rules of the type theory alone. Our `gauss` uses one standard axiom of Lean's logic, `propext` (propositions that imply each other are equal), through the lemmas that `rw` and `ring` rely on. The `grind` version uses two more: `Quot.sound` (which underlies quotient types and, through them, function extensionality) and `Classical.choice`, from which the law of excluded middle follows: `grind` reasons classically. Neither proof is wrong. But they are different kinds of proof, and in Chapter 5 you will learn what the difference means and why a constructive proof (one that does not use `Classical.choice`) carries more information than a classical one.

1.6 The question of this book

There is a sentence you may have read in a blog post, a talk or a discussion thread: *Rust's ownership and lifetimes are based on affine types*. It sounds authoritative, it is repeated often, and it is rarely explained. This book takes it as its central question:

> Is it true that Rust's ownership and lifetime concepts are based on affine types?

To answer it we need a surprising amount of machinery. "Affine types" are the type-theoretic side of *affine logic*, a logic in which an assumption may be used at most once. To say what that means we need natural deduction and the **structural rules** (Chapters 4 and 6), the rules that silently allow ordinary logic to use an assumption twice or not at all. To connect logic to types we need the λ-calculus and the Curry–Howard correspondence (Part II). Then we can remove structural rules, build the affine λ-calculus and compare it with Rust's moves, `Copy`, `Clone` and `Drop` (Part V). Then we need references, regions, permissions and variance, a calculus with borrows, and a look at how the compiler actually checks them (Part VI). Only then can we give a verdict.

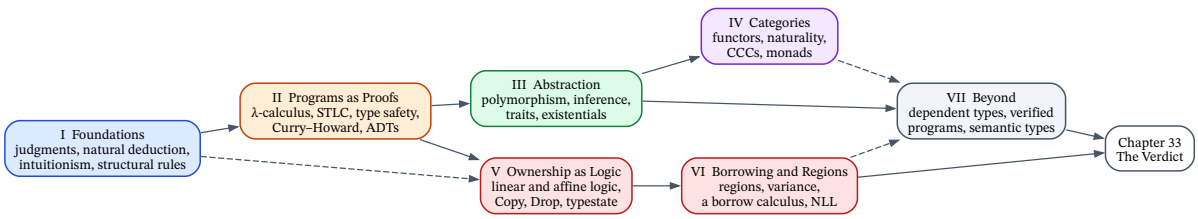
There is also a methodological trap in the question, and it is worth pointing out now. "Based on" can mean at least three different things:

1. **Historical**: the designers knew the research on linear and affine types and drew on it.
2. **Specificational**: the rules the compiler enforces are exactly the rules of an affine type system.
3. **Implementational**: the compiler works by building derivations in an affine type system.

These can have different answers, and keeping them apart is most of the work of answering honestly. Without spoiling the details, here is the shape of the answer we will reach in Chapter 33: the claim is **half true**. One half of Rust's discipline really is affine, in a precise sense that we will prove things about; the other half is a different theory altogether. Which half is which, and why the difference matters, is what Parts V and VI are about.

1.7 A map of the book

The book has seven parts. The diagram shows how they depend on each other.



The seven parts of the book and their dependencies. Solid arrows are strong dependencies; dashed arrows mean "uses some ideas from".

- **Part I — Foundations** (Chapters 1–6): the two languages, judgments and rules, natural deduction, intuitionism, the structural rules.

- **Part II — Programs as Proofs** (Chapters 7–11): the λ-calculus, simple types and closures, a mechanized proof of type safety, Curry–Howard, algebraic data types.
- **Part III — Abstraction** (Chapters 12–15): parametric polymorphism, type inference, traits and type classes, existential types.
- **Part IV — Categories** (Chapters 16–20): categories, functors, natural transformations, cartesian closed categories, monads.
- **Part V — Ownership as Logic** (Chapters 21–25): substructural logics, the affine λ-calculus, `Copy` and `Clone`, `Drop`, typestate.
- **Part VI — Borrowing and Regions** (Chapters 26–29): regions and permissions, variance, a borrow calculus, the real borrow checker.
- **Part VII — Beyond** (Chapters 30–33): dependent types, verified programs, semantic soundness and `unsafe`, the verdict.

Read Parts I and II in order; Chapter 2, which teaches Lean, is a prerequisite for everything that follows. After that you can follow your interest: Part IV only needs Parts I and II; Parts V and VI also need only Parts I and II and are best read together; Part VII draws on everything.

1.8 Common confusions

"**A judgment is a boolean expression.**" It is not. `x > 0` is a term of the language, evaluated when the program runs. $\Gamma \vdash e : \tau$ is a statement in the metalanguage, about the term e, established (or not) by building a derivation. Chapter 3 keeps the two levels carefully apart, and Lean makes the distinction concrete: a Rust `bool` corresponds to Lean's `Bool`, while a judgment corresponds to a Lean `Prop`.

"**Type safety means the program is correct.**" It means the program does not go wrong *in the specific sense the semantics defines*. A well-typed Rust program can still compute the wrong answer, panic, deadlock or run forever. A Lean theorem about a program is a much stronger statement, because its type can say what "correct" means; but it is only as good as the statement you wrote.

"**Lean is only for mathematicians.**" Lean is a general-purpose functional language, and a fast one; the proof-assistant side is what this book uses most, but every Lean file here also contains ordinary programs that you can run with `#eval`.

"**`#guard_msgs` is part of the program.**" It is a testing device: it changes nothing about the definitions in the file. If you delete the `#guard_msgs in` line and the doc comment above it, the command still runs and simply prints its output in the infoview.

1.9 Summary

- A type theory has four ingredients: syntax (terms and types), judgments such as $\Gamma \vdash e : \tau$, inference rules that derive judgments, and computation rules that run terms.
- A derivation is a tree of rule instances. Type checking is the search for a derivation; a type error is a report of where the search failed.

- Type safety (progress and preservation) says that well-typed programs never reach a stuck state. What "stuck" means depends on the semantics; for safe Rust it rules out undefined behavior but not panics, leaks or non-termination.
- Rust is expression-oriented, and its blocks, `if` and never type are instances of simple rules: rule T-If explains error E0308, and the never type is the empty type, which coerces to every type.
- Types can be read as sets of values, as propositions (Curry–Howard) and as objects of a category. These three readings are one subject.
- Rust shows where the theory lives in practice and where practice deviates from it; Lean states the theory precisely and checks proofs mechanically.
- In Lean, `#eval` runs, `#check` shows types, a `theorem` is a type with a program (a proof) of that type, and `#guard_msgs` checks the outputs shown in this book. `#print axioms` shows what a proof rests on.
- The book's question, whether Rust's ownership and lifetimes are "based on" affine types, has a precise answer that we will reach in Chapter 33.

1.10 Exercises

1. Write the full derivation tree for the body of `fn f(b: bool) -> i32 { if b { 1 } else { 2 } }`, using rules T-Var, T-Lit and T-If. You will need a rule for the boolean variable `b` in the context. *Hint:* the context is $\Gamma = b$: `bool`, and the derivation has three branches above the T-If line.
2. Which premise of T-If fails for `if c { 1 } else { "a" }`? Sketch two different changes to the type system that would accept the program, and for each, say what you would lose. *Hint:* one change introduces a type that contains both integers and strings; the other gives up knowing the type before the program runs.
3. What is the type of `loop {}`? Why does `let x: i32 = loop {};` compile, while `let x: i32 = loop { break; };` does not? Check your answers with the compiler. *Hint:* a `loop` that is exited with `break` has the type of the value given to `break`, and a bare `break` gives `()`.
4. In Lean, prove that `sumTo` is monotone: `theorem sumTo_le (n : ℕ) : sumTo n ≤ sumTo (n + 1)`. Add it to a scratch copy of `FirstSteps.lean` inside `namespace Ch01`. *Hint:* no induction is needed. Unfold one step with `simp only [sumTo]`, and finish with `omega`, a decision procedure for linear arithmetic.
5. Prove the three-way rotation `theorem rotate (p q r : Prop) : p ∧ q ∧ r → r ∧ p ∧ q` in Lean, and write the corresponding generic function `rotate` on triples in Rust. Then run `#print axioms rotate`. *Hint:* `p ∧ q ∧ r` means `p ∧ (q ∧ r)`, and the anonymous constructor notation flattens nested pairs: `fun ⟨hp, hq, hr⟩ => ⟨hr, hp, hq⟩`.
6. Prove `gauss` without Mathlib's `ring`, using only tactics from Lean's core library. Then compare the two proofs with `#print axioms`. *Hint:* after `rw [sumTo, Nat.mul_add, ih]`, multiply everything out with `simp only [Nat.add_mul, Nat.mul_add, Nat.mul_one, Nat.one_mul]`; the goal becomes linear if you treat `k * k` as an unknown, and `omega` (a decision procedure for linear arithmetic) can finish it.

CHAPTER 2

Lean 4 for Rust Programmers

Chapter 1 showed you a few lines of Lean: an `#eval`, a `#check`, a definition by pattern matching and a proof by induction. From now on, every chapter uses Lean to state its ideas precisely and to prove them. This chapter is where you learn the language properly.

The good news is that you already know most of it. Lean is a functional programming language with structures, enums, pattern matching, generics, traits and closures, and almost every Rust feature you use daily has a Lean counterpart with a different spelling. The chapter is organized around those counterparts. A few ideas, though, have no counterpart in Rust, and they are the reason Lean can be a proof assistant at all: values never change, every function must terminate, and a proof is just another program. We will meet each of them when it first matters.

The chapter then builds the same program, a small order book for a trading venue, in both languages, proves theorems about the Lean version that hold for every possible book, and ends with a first tour of Mathlib, the library of mathematics and tactics that the rest of the book relies on.

In this chapter you will learn:

- how Lean's `def`, `fun`, `structure`, `inductive` and `match` correspond to Rust's `fn`, closures, `struct`, `enum` and `match`;
- how `Option`, `Except`, `List` and `Array` compare to `Option`, `Result` and `Vec`, and what it means for a value to be *persistent*;
- how namespaces and dot notation replace `impl` blocks, and how type classes replace traits;
- how `do` notation gives you `?`, `let mut` and `for` loops in programs that are still pure;
- why Lean insists that every function terminates, and what `partial` and `termination_by` are for;
- how to prove properties of programs with `rfl`, `decide`, `simp`, `omega` and `induction`, and how to read goals and error messages;
- what Mathlib is, how the book's project depends on it, and how to find the lemma you need.

2.1 Reading and writing Lean files

A Lean file is a sequence of *declarations* (`def`, `theorem`, `structure`, `inductive`, `instance`...) and *commands* that ask Lean something (`#eval`, `#check`, `#print`). Imports come first, then usually a `namespace`. The Lean code of this chapter is in six files under `code/lean/Book/Ch02/`: `Basics.lean`,

`Classes.lean` and `Effects.lean` for the language, `Orders.lean` and `FirstProofs.lean` for the order book, and `MathlibTour.lean` for Mathlib.

As in Chapter 1, every output in this chapter is checked by Lean: the doc comment above a `#guard_msgs in` line is the exact text that the command below it must print. When you open the files in an editor you will see the same text in the **infoview**, the side panel that shows what Lean knows at the cursor: the output of commands, the type of the expression under the cursor and, inside a proof, the current goals. Lean checks the file continuously as you type, so there is no separate compile step; an error appears under the offending line within a second or two, like a `cargo check` that never stops running.

Lean uses Unicode freely. In the editor you type a backslash and a short name: `\to` gives `→`, `\a` gives `α`, `\N` gives `ℕ`, `\le` gives `≤`, `\ne` gives `≠`, `\<` and `\>` give `⟨` and `⟩`, and `\.` gives `·`. Most symbols also have an ASCII spelling (`->` for `→`, `<=` for `≤`), but the book uses the Unicode forms, because that is how Lean prints them.

Comments come in four kinds, and three of them have exact Rust counterparts: `-- line comment` is `//`, `/- block comment -/` is `/* */`, `/-- doc comment -/` before a declaration is `///`, and `/-! module doc -/` at the top of a file is `//!`.

2.2 Functions

Here is a first function, with its Rust version in the doc comment:

```
/-- Rust: `fn double(n: u64) -> u64 { 2 * n }` -/
def double (n : Nat) : Nat := 2 * n
```

```
/-- info: 42 -/
#guard_msgs in
#eval double 21
```

The shape is the same as in Rust, with different punctuation. Parameters are written `(name : Type)`, with spaces around the colon by convention. The result type follows a colon after the parameters, and the body follows `:=`. There are no braces, no semicolons and no `return`: the body is a single expression, and its value is the result. `Nat` is the type of natural numbers $0, 1, 2, \ldots$, which we will discuss in a moment.

Application and currying

A function with two parameters:

```
/-- Two parameters. Rust: `fn area(width: u64, height: u64) -> u64`. -/
def area (width height : Nat) : Nat := width * height

/-- info: 12 -/
#guard_msgs in
#eval area 3 4
```

Function application is written by juxtaposition: `area 3 4`, not `area(3, 4)`. Parentheses only group: `double (area 3 4)` applies `double` to the result of `area 3 4`, while `double area 3 4` would be an error. This is the most visible difference from Rust, and the one that takes longest to get used to.

The types tell you what is going on underneath:

```
/-- info: Ch02.area (width height : Nat) : Nat -/
#guard_msgs in
#check area

/-- info: area : Nat → Nat → Nat -/
#guard_msgs in
#check (area)

/-- info: area 3 : Nat → Nat -/
#guard_msgs in
#check area 3

/-- info: [30, 40, 50] -/
#guard_msgs in
#eval [3, 4, 5].map (area 10)
```

The first `#check` prints the *signature* of `area`, with parameter names. The second, with `area` in parentheses, prints its *type*: `Nat → Nat → Nat`. The arrow associates to the right, so this means `Nat → (Nat → Nat)`: `area` takes a width and returns a *function* that takes a height. Every multi-argument function in Lean works this way (it is *curried*, after the logician Haskell Curry), and `area 3 4` is really `(area 3) 4`. As a consequence you can stop early: `area 3` is a perfectly good value of type `Nat → Nat`, and the last line passes `area 10` to `List.map`. In Rust you would write the closure `|h| area(10, h)` by hand.

Anonymous functions

Closures are written `fun x => body`. For short ones there is a shorthand: a centered dot `·` inside parentheses stands for the argument, so `(· + 10)` means `fun x => x + 10`:

```
/-- info: [1, 4, 9] -/
#guard_msgs in
#eval [1, 2, 3].map (fun n => n * n)

/-- info: [11, 12, 13] -/
#guard_msgs in
#eval [1, 2, 3].map (· + 10)
```

Local definitions and conditionals

Inside a body, `let` names an intermediate value, and `if c then a else b` is an expression, exactly like Rust's `if`. String interpolation uses `s!"..."` with the expressions in braces:

```
/-- `let`, `if` and string interpolation. Everything is an expression, and
there is no `return`: the body's value is the result. -/
def describe (n : Nat) : String :=
  let parity := if n % 2 == 0 then "even" else "odd"
  s!"{n} is {parity}"

/-- info: "7 is odd" -/
#guard_msgs in
#eval describe 7
```

The `let` line has no semicolon and no `in`: the line break ends it, and the rest of the body is in its scope. The `else` branch is mandatory, for the reason you saw in Chapter 1: both branches must have the same type, and a missing branch would have none.

Numbers

Lean's numeric types differ from Rust's in ways that will matter in proofs:

```
/-- info: 0 -/
#guard_msgs in
#eval (3 : Nat) - 5          -- Nat subtraction stops at zero

/-- info: -2 -/
#guard_msgs in
#eval (3 : Int) - 5

/-- info: 0 -/
#guard_msgs in
#eval (255 : UInt8) + 1      -- fixed-width types wrap around

/-- info: 1267650600228229401496703205376 -/
#guard_msgs in
#eval 2 ^ 100                -- Nat has no upper bound
```

`Nat` is the type of *mathematical* natural numbers: there is no largest one, so `2 ^ 100` is computed exactly. (At run time Lean stores small numbers as machine words and switches to arbitrary-precision integers when they grow.) Since there are no negative naturals, subtraction is *truncated*: `3 - 5` is `0`. (A Rust `u64` subtraction that goes below zero panics in a debug build and wraps around in a release build.) Truncation keeps subtraction a total function, but it makes some familiar laws false: `a - b + b = a` holds only when `b ≤ a`. `Int` is the type of all integers, also unbounded. Fixed-width types (`UInt8`, `UInt64`, `Int32` ...) wrap around, and `Float` is the usual 64-bit floating-point type. The book uses `Nat` and `Int` almost everywhere, because they are the easiest types to prove theorems about.

Generic functions and implicit arguments

In Rust, a generic function lists its type parameters in angle brackets. In Lean, a type parameter is an ordinary parameter whose type is `Type`, the type of types:

```
/-- A generic function. `{α : Type}` is an *implicit* argument: Lean infers it
from the other arguments, as Rust infers `T` in
`fn first_or<T>(xs: Vec<T>, default: T) -> T`. -/
def firstOr {α : Type} (xs : List α) (default : α) : α :=
  match xs with
  | [] => default
  | x :: _ => x

/-- info: 3 -/
#guard_msgs in
#eval firstOr [3, 4] 0

/-- info: "none" -/
#guard_msgs in
#eval firstOr [] "none"
```

The braces around `{α : Type}` make the parameter **implicit**: callers do not pass it, and Lean works it out from the other arguments, the way Rust infers `T`. In `firstOr [3, 4] 0` it sees a list of naturals and sets `α := Nat`; in `firstOr [] "none"` the empty list says nothing, but the default `"none"` does. The

body uses `match` on the list, with the two cases `[]` (empty) and `x :: _` (a first element `x` followed by a tail we ignore).

When you want to pass an implicit argument yourself, prefix the function with `@`. All arguments then become explicit, which is Lean's version of the turbofish:

```
/-- info: @firstOr : {α : Type} → List α → α → α -/
#guard_msgs in
#check @firstOr

/-- info: 7 -/
#guard_msgs in
#eval @firstOr Nat [] 7      -- like Rust's turbofish: first_or::<u64>
```

The type `{α : Type} → List α → α → α` reads "for every type `α`, a function from lists of `α` and an `α` to an `α`"; Chapter 12 has much more to say about such types.

One more convenience, with a catch. If a signature mentions a name that is not defined anywhere, Lean adds it as an implicit argument automatically. This is called *auto-bound implicits*, and it is switched on in the book's project:

```
/-- `α` is not declared anywhere: Lean adds `{α}` as an implicit argument. -/
def identity (x : α) : α := x

/-- info: @identity : {α : Sort u_1} → α → α -/
#guard_msgs in
#check @identity
```

(`Sort u_1` is a slightly more general version of `Type`; Chapter 12 explains it.) Chapter 3 uses this to keep the typing rules of a language readable. The catch is that a *misspelled* or un-imported name in a signature is not reported as unknown: it silently becomes a type variable, and the error, if any, appears later and looks unrelated:

```
/--
error: failed to synthesize instance of type class
  HMul Nat Nta Nat

Hint: Type class instance resolution failures can be inspected with the `set_option
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
def triple (n : Nta) : Nat := 3 * n     -- a typo: `Nta` instead of `Nat`
```

Lean took `Nta` to be an arbitrary type and then could not multiply a `Nat` by a value of that type. When an error message mentions a type you never defined, look for a typo in the signature.

Definitions by pattern matching

A function can also be defined directly by cases on its argument, with one equation per case, as Chapter 1's `sumTo` was:

```
/-- A definition by pattern matching: one equation per case. -/
def fib : Nat → Nat
  | 0 => 0
  | 1 => 1
  | n + 2 => fib n + fib (n + 1)

/-- info: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34] -/
```

```
#guard_msgs in
#eval (List.range 10).map fib
```

The patterns `0`, `1` and `n + 2` cover every natural number, and Lean checks that they do. The recursive calls are on `n` and `n + 1`, both smaller than `n + 2`; Lean checks that too, as the section on totality explains.

2.3 Structures and inductive types

Structures

A `structure` is a Rust `struct`. Here is a point, and a Rust version from `code/rust/examples/ch02_rust_counterparts.rs`, a file that collects the Rust counterparts of this chapter's Lean definitions:

```
/-- Rust: `struct Point { x: i64, y: i64 }` with `#[derive(Debug, PartialEq)]`. -/
structure Point where
  x : Int
  y : Int
  deriving Repr, BEq

def origin : Point := { x := 0, y := 0 }

def p : Point := ⟨3, 4⟩
```

```
#[derive(Debug, Clone, Copy, PartialEq)]
struct Point {
    x: i64,
    y: i64,
}

impl Point {
    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}
```

A value is built either with field names, `{ x := 0, y := 0 }` (note `:=`, not `:`), or positionally with the *anonymous constructor* `⟨3, 4⟩`, which works for any structure when Lean knows the expected type. Every structure also has a named constructor, `Point.mk`, which is an ordinary function:

```
/-- info: 3 -/
#guard_msgs in
#eval p.x

/-- info: { x := 3, y := 10 } -/
#guard_msgs in
#eval { p with y := 10 }      -- Rust: Point { y: 10, ..p }

/-- info: { x := 3, y := 4 } -/
#guard_msgs in
#eval p                       -- p itself is unchanged

/-- info: Ch02.Point.mk (x y : Int) : Point -/
#guard_msgs in
#check Point.mk
```

`{ p with y := 10 }` is Rust's struct update syntax `Point { y: 10, ..p }`. It builds a *new* point, and `p` is unchanged, as the next line shows. In Lean there is no way to change `p` at all: values never change, a point we return to below.

Inductive types

An `inductive` type is a Rust `enum`: a fixed set of alternatives, called *constructors*, each of which may carry data:

```
/-- Rust: `enum Command { Ping, Get(String), Set(String, u64), Delete(String) }`. -/
inductive Command where
  | ping
  | get (key : String)
  | set (key : String) (value : Nat)
  | delete (key : String)
  deriving Repr, BEq

/-- `match` with one arm per constructor. -/
def Command.describe (c : Command) : String :=
  match c with
  | .ping => "ping"
  | .get key => s!"read {key}"
  | .set key value => s!"write {value} to {key}"
  | .delete key => s!"remove {key}"

/-- info: "write 3 to retries" -/
#guard_msgs in
#eval (Command.set "retries" 3).describe
```

```
#[derive(Debug, Clone, PartialEq)]
enum Command {
    Ping,
    Get(String),
    Set(String, u64),
    Delete(String),
}

impl Command {
    fn describe(&self) -> String {
        match self {
            Command::Ping => "ping".to_string(),
            Command::Get(key) => format!("read {key}"),
            Command::Set(key, value) => format!("write {value} to {key}"),
            Command::Delete(key) => format!("remove {key}"),
        }
    }
}
```

The function `Command.describe` matches on the command with one arm per constructor. In the patterns, `.get key` is short for `Command.get key`: when Lean knows the expected type, a leading dot looks the name up in that type's namespace, which saves writing `Command::` over and over.

As in Rust, a `match` must be exhaustive, and Lean tells you which cases you forgot:

```
/--
error: Missing cases:
ping
-/
#guard_msgs in
def Command.isWrite : Command → Bool
  | .set _ _ => true
  | .delete _ => true
  | .get _ => false
```

Rust reports the same mistake as error E0004 ("non-exhaustive patterns"). In both languages, exhaustiveness is what makes `match` trustworthy: a function defined by cases really does handle every value of the type.

Recursive types

Inductive types can be recursive. Here is a binary search tree, whose `node` constructor contains two smaller trees:

```
/-- A binary search tree. Rust needs `Box<Tree>` for the recursive fields;
Lean does not, because every value of an inductive type is already a pointer
to an immutable heap object. -/
inductive Tree where
  | leaf
  | node (left : Tree) (key : Nat) (right : Tree)
  deriving Repr

def Tree.insert (t : Tree) (k : Nat) : Tree :=
  match t with
  | .leaf => .node .leaf k .leaf
  | .node l x r =>
    if k < x then .node (l.insert k) x r
    else if x < k then .node l x (r.insert k)
    else t

def Tree.toList : Tree → List Nat
  | .leaf => []
  | .node l x r => l.toList ++ [x] ++ r.toList

/-- info: [1, 3, 4, 5, 9] -/
#guard_msgs in
#eval ([5, 3, 9, 1, 4, 3].foldl Tree.insert .leaf).toList
```

The Rust version needs `Box`, because a Rust `enum` is stored inline and a type that contains itself directly would have infinite size (Chapter 11 discusses this in detail):

```
/// The recursive fields need a `Box`: without it, `Tree` would contain itself
/// and have infinite size.
#[derive(Debug)]
enum Tree {
    Leaf,
    Node(Box<Tree>, u64, Box<Tree>),
}

impl Tree {
    /// Consumes the tree and returns a new one, like the Lean version.
    fn insert(self, k: u64) -> Tree {
        match self {
            Tree::Leaf => Tree::Node(Box::new(Tree::Leaf), k, Box::new(Tree::Leaf)),
            Tree::Node(l, x, r) => {
                if k < x {
                    Tree::Node(Box::new(l.insert(k)), x, r)
                } else if x < k {
                    Tree::Node(l, x, Box::new(r.insert(k)))
                } else {
                    Tree::Node(l, x, r)
                }
            }
        }
    }

    fn to_vec(&self) -> Vec<u64> {
        match self {
            Tree::Leaf => Vec::new(),
            Tree::Node(l, x, r) => {
```

```
                let mut v = l.to_vec();
                v.push(*x);
                v.extend(r.to_vec());
                v
            }
        }
    }
}
```

Lean needs nothing of the kind: a value of an inductive type is always a pointer to an immutable object on the heap, so a `Tree` field is one pointer wide. `Tree.insert` returns a new tree, but the subtrees it does not visit are not copied: the new tree points to them, and so does the old tree, which remains valid. (The Rust version avoids copies differently, by taking `self` by value and moving the subtrees into the result.) The `foldl` in the last line inserts the numbers one by one, starting from `.leaf`.

2.4 Options, errors and collections

`Option`

`Option α` is Rust's `Option<T>`, with the constructors spelled `none` and `some`:

```
/-- Rust: `fn safe_div(a: u64, b: u64) -> Option<u64>`. -/
def safeDiv (a b : Nat) : Option Nat :=
  if b == 0 then none else some (a / b)

/-- info: some 5 -/
#guard_msgs in
#eval safeDiv 10 2

/-- info: none -/
#guard_msgs in
#eval safeDiv 10 0

/-- info: 0 -/
#guard_msgs in
#eval (safeDiv 10 0).getD 0        -- Rust: .unwrap_or(0)

/-- info: some 6 -/
#guard_msgs in
#eval (safeDiv 10 2).map (· + 1)   -- Rust: .map(|n| n + 1)
```

The library functions have different names but the same meaning: `getD` is `unwrap_or`, `map` is `map`, `bind` is `and_then`, `isSome` is `is_some`. There is also `get!`, which panics on `none` like `unwrap`; the book avoids it.

`Except`

Rust's `Result<T, E>` is called `Except ε α` in Lean, with the constructors `.ok` and `.error`. Note that the error type comes *first*:

```
/-- The errors of a small parser. -/
inductive ParseError where
  | empty
  | notADigit (c : Char)
  deriving Repr

/-- `Except ε α` is Rust's `Result<α, ε>`: note that the error type comes
first. The constructors are `.ok` and `.error`. -/
def parseDigit (c : Char) : Except ParseError Nat :=
  if c.isDigit then .ok (c.toNat - '0'.toNat) else .error (.notADigit c)
```

```
/-- info: Except.ok 7 -/
#guard_msgs in
#eval parseDigit '7'

/-- info: Except.error (Ch02.ParseError.notADigit 'x') -/
#guard_msgs in
#eval parseDigit 'x'
```

(`Char.toNat` is the character's code point, so subtracting the code of `'0'` turns `'7'` into `7`.) Combining two fallible steps by hand is as tedious as in Rust without `?`:

```
/-- Combining two fallible steps with `match`, the long way. (Section "Effects"
shows the short way, with `do` and `←`, Lean's version of Rust's `?`.) -/
def parseTwoDigits (a b : Char) : Except ParseError Nat :=
  match parseDigit a with
  | .error e => .error e
  | .ok d₁ =>
    match parseDigit b with
    | .error e => .error e
    | .ok d₂ => .ok (10 * d₁ + d₂)

/-- info: Except.ok 42 -/
#guard_msgs in
#eval parseTwoDigits '4' '2'
```

The section on `do` notation shows Lean's equivalent of `?`.

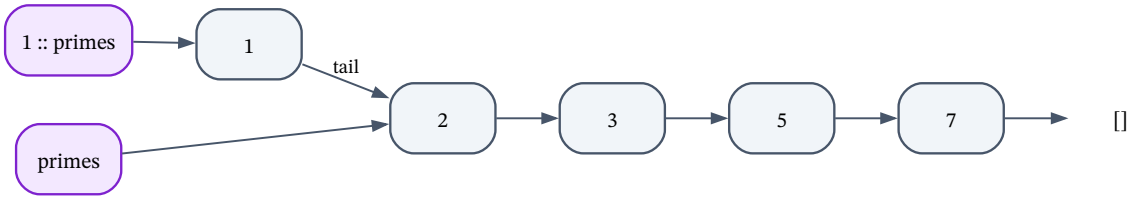
Lists

`List α` is a singly linked list: either empty, `[]`, or an element followed by a list, `x :: xs` (read "x cons xs"). Lists are the workhorse of functional programming, and Lean has a large library of functions on them:

```
def primes : List Nat := [2, 3, 5, 7]

/-- info: [1, 2, 3, 5, 7] -/
#guard_msgs in
#eval 1 :: primes              -- a new list whose tail *is* primes

/-- info: [2, 3, 5, 7] -/
#guard_msgs in
#eval primes                   -- primes itself is unchanged

/-- info: [6, 10, 14] -/
#guard_msgs in
#eval primes.map (· * 2) |>.filter (· > 5)

/-- info: 17 -/
#guard_msgs in
#eval primes.foldl (· + ·) 0

/-- info: (some 5, some 3, none) -/
#guard_msgs in
#eval (primes.find? (· > 4), primes[1]?, primes[10]?)
```

`xs.map f |>.filter g` is a pipeline: `|>.filter g` applies `filter g` to whatever is on its left, like a chained iterator adapter in Rust. Unlike Rust's iterators, Lean's list functions are not lazy: `map` builds a whole new list before `filter` runs. Indexing with `xs[i]?` returns an `Option`, like Rust's `get`.

The first two `#eval`s show the property that matters most. `1 :: primes` is a new list with one more element, and `primes` still exists, unchanged. Building the new list does not copy `primes`: the new first cell simply points to it.



Two lists sharing a tail. Consing 1 onto primes allocates one new cell that points to the existing list; both lists remain valid.

A data structure with this property is called **persistent**: operations produce new versions and the old versions remain available. Persistence is what you get automatically when values cannot change, and sharing is what makes it cheap.

Arrays

`Array α` is Lean's `Vec<T>`: a contiguous, growable buffer with constant-time indexing:

```
def readings : Array Nat := #[12, 15, 11]

/-- info: #[12, 15, 11, 14] -/
#guard_msgs in
#eval readings.push 14

/-- info: #[12, 99, 11] -/
#guard_msgs in
#eval readings.set! 1 99

/-- info: (3, 15) -/
#guard_msgs in
#eval (readings.size, readings[1]!)
```

Arrays are written `#[...]`. `push` adds an element at the end, `set!` replaces one (panicking if the index is out of bounds, hence the `!`), and `a[i]!` reads one with the same check. Chapter 1 showed the other two forms of indexing, `a[i]?` and `a[i]'h` with a proof `h`.

Arrays are persistent too: `readings.push 14` returns a new array, and `readings` is still `#[12, 15, 11]`. Whether that means every `push` copies the array is answered in the section on values that never change (it does not).

2.5 Namespaces, dot notation and type classes

Namespaces instead of `impl` blocks

Lean has no `impl` blocks. A "method" is an ordinary function that lives in the namespace of its type:

```
namespace Point

/-- Inside `namespace Point`, this defines `Point.add`. -/
def add (a b : Point) : Point := ⟨a.x + b.x, a.y + b.y⟩
```

```
/-- The Manhattan length. `Int.natAbs` is the absolute value, as a `Nat`. -/
def manhattan (a : Point) : Nat := a.x.natAbs + a.y.natAbs

end Point

/-- info: { x := 6, y := 8 } -/
#guard_msgs in
#eval p.add p                  -- dot notation: Point.add p p

/-- info: 7 -/
#guard_msgs in
#eval p.manhattan
```

Inside `namespace Point ... end Point`, the definition `add` gets the full name `Point.add`. (Writing `def Point.add` outside the namespace is equivalent, and that is how `Command.describe` was defined.) **Dot notation** then does the rest: when Lean sees `p.add p` and `p` has type `Point`, it looks for a function `Point.add` and passes `p` as its first argument of type `Point`. That is why `p.add p` means `Point.add p p`, why `primes.map f` means `List.map f primes`, and why `(safeDiv 10 0).getD 0` works.

It looks like method syntax, but it is only a naming convention, resolved by the type of the value before the dot. So anyone can add "methods" to any type, in any file, by defining a function in its namespace; Rust needs an extension trait for that. Namespaces also play the role of Rust's modules, and `open Point` brings the names of a namespace into scope, like `use`.

Type classes instead of traits

Rust's traits come from Haskell's **type classes**, and Lean has type classes in the original form. Chapter 14 studies them in depth; here is the practical correspondence. A class declares operations, possibly with defaults:

```
/-- Rust: `trait Summary { fn summary(&self) -> String; fn tag(&self) -> String
{ ... } }`. A class lists the operations an instance must provide; `tag` has
a default implementation, like a default method. -/
class Summary (α : Type) where
  summary : α → String
  tag : α → String := fun _ => "item"
```

An `instance` implements a class for a type, like an `impl Trait for Type` block. An instance can depend on another instance, like a blanket impl:

```
structure Sensor where
  name : String
  celsius : Int

/-- Rust: `impl Summary for Sensor { ... }`. -/
instance : Summary Sensor where
  summary s := s!"{s.name}: {s.celsius} °C"
  tag _ := "sensor"

/-- An instance that uses the default `tag`. -/
instance : Summary Nat where
  summary n := s!"the number {n}"

/-- Rust: `impl<T: Summary> Summary for Vec<T>`. The instance for `List α`
needs an instance for `α`, written as an instance argument `[Summary α]`. -/
instance {α : Type} [Summary α] : Summary (List α) where
  summary xs := ", ".intercalate (xs.map Summary.summary)
  tag _ := "list"
```

A generic function that needs the operations takes an **instance argument** in square brackets, which is Lean's trait bound:

```
/-- Rust: `fn report<T: Summary>(x: &T) -> String`. The square brackets ask
Lean to find the instance by itself, at every call site. -/
def report {α : Type} [Summary α] (x : α) : String :=
  s!"[{Summary.tag x}] {Summary.summary x}"

/-- info: "[sensor] boiler: 74 °C" -/
#guard_msgs in
#eval report (Sensor.mk "boiler" 74)

/-- info: "[item] the number 7" -/
#guard_msgs in
#eval report 7

/-- info: "[list] the number 1, the number 2" -/
#guard_msgs in
#eval report [1, 2]
```

Side by side with the Rust version, the correspondence is line for line:

```
trait Summary {
    fn summary(&self) -> String;
    fn tag(&self) -> String {
        "item".to_string()
    }
}

struct Sensor {
    name: String,
    celsius: i64,
}

impl Summary for Sensor {
    fn summary(&self) -> String {
        format!("{}: {} °C", self.name, self.celsius)
    }
    fn tag(&self) -> String {
        "sensor".to_string()
    }
}

impl<T: Summary> Summary for Vec<T> {
    fn summary(&self) -> String {
        let parts: Vec<String> = self.iter().map(|x| x.summary()).collect();
        parts.join(", ")
    }
    fn tag(&self) -> String {
        "list".to_string()
    }
}

fn report<T: Summary>(x: &T) -> String {
    format!("[{}] {}", x.tag(), x.summary())
}
```

The square brackets in `[Summary α]` tell Lean to *search* for the instance at each call site. For `report [1, 2]` it needs `Summary (List Nat)`, finds the list instance, which in turn needs `Summary Nat`, and finds that. When the search fails, you get Lean's equivalent of Rust's E0277 ("the trait bound is not satisfied"):

```
/--
error: failed to synthesize instance of type class
  Summary String
```

```
Hint: Type class instance resolution failures can be inspected with the `set_option
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
#eval report "hello"
```

One difference from Rust: the methods of a class are functions in the *class*'s namespace, so we call `Summary.summary x` (or `summary x` after `open Summary`). Dot notation `x.summary` would look for `Sensor.summary`, which does not exist.

The standard classes and `deriving`

Lean's core library defines classes for the operations that Rust's standard traits provide, and many of them can be derived:

Rust trait	Lean class	What it gives you
`Debug`	`Repr`	printing by `#eval`
`Display`	`ToString`	`toString x`, `s!"{x}"`
`PartialEq`	`BEq`	`x == y`
`Eq` (roughly)	`DecidableEq`	`x == y`, `if x = y`, and proofs
`Hash`	`Hashable`	`hash x`, keys of `HashMap`
`Default`	`Inhabited`	`default`
`Ord`	`Ord`	`compare x y`

`deriving` after a type plays the role of `#[derive]`:

```
/-- `deriving` asks Lean to write the instances, like `#[derive]`. -/
structure Endpoint where
  host : String
  port : Nat
  deriving Repr, BEq, Hashable, Inhabited

/-- `ToString` is Rust's `Display`: we write it by hand. -/
instance : ToString Endpoint where
  toString e := s!"{e.host}:{e.port}"

def api : Endpoint := { host := "api.example.com", port := 443 }
```

```
/-- info: { host := "api.example.com", port := 443 } -/
#guard_msgs in
#eval api                                   -- uses Repr, like {:?}

/-- info: "connecting to api.example.com:443" -/
#guard_msgs in
#eval s!"connecting to {api}"               -- uses ToString, like {}

/-- info: true -/
#guard_msgs in
#eval api == { host := "api.example.com", port := 443 }   -- uses BEq

/-- info: { host := "", port := 0 } -/
#guard_msgs in
#eval (default : Endpoint)                  -- uses Inhabited, like Default
```

`Std.HashMap` (from the `Std` library that ships with Lean) needs `BEq` and `Hashable` for its keys, as Rust's `HashMap` needs `Eq` and `Hash`:

```
open Std in
/-- `Std.HashMap` needs `BEq` and `Hashable` keys, as Rust's `HashMap` needs
`Eq` and `Hash`. -/
def failures : HashMap Endpoint Nat :=
  ({} : HashMap Endpoint Nat) |>.insert api 2 |>.insert { host := "db", port := 5432 } 1

/-- info: (some 2, none) -/
#guard_msgs in
#eval (failures.get? api, failures.get? default)
```

Two of these classes deserve a remark. `DecidableEq` is stronger than `BEq`: it decides the *proposition* `x = y`, with a guarantee that its answer is right, whereas a derived `BEq` is just a function returning a `Bool`, about which proofs know nothing. So when you intend to prove things about a type, derive `DecidableEq` (and `==` then comes from it), as the order book below does. `Inhabited` matters because some operations need a fallback value: `xs[i]!` must return *something* of the right type even while it reports a panic, and that something is `default`.

2.6 `do` notation, `IO` and loops

Effects are visible in types

In Rust any function can print, read a file or open a socket. In Lean a function of type `Nat → Nat` cannot: it can only compute its result. Effects live in special types, and the most important one is `IO α`, the type of *actions* that may perform input and output and then produce an `α`:

```
/-- An `IO` action: a description of effects, run by `#eval` or by `main`. -/
def greet (name : String) : IO Unit := do
  IO.println s!"hello, {name}"
  if name.length > 3 then
    IO.println "that is a long name"

/--
info: hello, Grace
that is a long name
-/
#guard_msgs in
#eval greet "Grace"
```

`do` starts a block of statements, one per line, executed in order. Calling `greet "Grace"` does not print anything by itself; it produces an action, and `#eval` runs it. A compiled Lean program is an action called `main` (usually of type `IO Unit`), which the runtime runs, just as Rust's runtime calls `fn main()`. The `if` without `else` is allowed here, because inside `do` a missing `else` means "do nothing".

`do` for errors: Lean's `?`

`do` is not tied to `IO`. It works for every type that describes a computation with some kind of effect (in technical terms, every *monad*, the subject of Chapter 20), including `Option` and `Except`. In a `do` block for `Except`, the arrow `←` runs a fallible step and either extracts its result or stops at the first error, which is exactly what Rust's `?` does:

```
/-- Parse a decimal number. In a `do` block for `Except`, `←` runs a fallible
step and stops at the first error, like Rust's `?`; `throw` is `return Err(..)`. -/
def parseNat (s : String) : Except ParseError Nat := do
  if s.isEmpty then throw .empty
```

```
  let mut n := 0
  for c in s.toList do
    let d ← parseDigit c
    n := 10 * n + d
  return n

/-- info: Except.ok 2024 -/
#guard_msgs in
#eval parseNat "2024"

/-- info: Except.error (Ch02.ParseError.notADigit 'O') -/
#guard_msgs in
#eval parseNat "2024"
```

```
#[derive(Debug, PartialEq)]
enum ParseError {
    Empty,
    NotADigit(char),
}

fn parse_digit(c: char) -> Result<u64, ParseError> {
    c.to_digit(10)
        .map(u64::from)
        .ok_or(ParseError::NotADigit(c))
}

fn parse_nat(s: &str) -> Result<u64, ParseError> {
    if s.is_empty() {
        return Err(ParseError::Empty);
    }
    let mut n = 0;
    for c in s.chars() {
        let d = parse_digit(c)?;
        n = 10 * n + d;
    }
    Ok(n)
}
```

Compare the two line by line. `throw .empty` is `return Err(ParseError::Empty)`; `let d ← parseDigit c` is `let d = parse_digit(c)?`; `return n` is `Ok(n)`. Lean also has `let mut`, reassignment with `:=`, and `for ... in ... do` loops, all inside `do`.

Mutable variables in pure code

Here is a loop that accumulates a sum in a mutable variable:

```
/-- A loop with a mutable accumulator, inside `Id.run do`: the function is
pure, and its type says so. -/
def sumSquares (xs : List Nat) : Nat := Id.run do
  let mut total := 0
  for x in xs do
    total := total + x * x
  return total

/-- info: 14 -/
#guard_msgs in
#eval sumSquares [1, 2, 3]

/-- The loop is a left fold in disguise, and Lean can prove it. -/
theorem sumSquares_eq_foldl (xs : List Nat) :
    sumSquares xs = xs.foldl (fun total x => total + x * x) 0 := by
  simp [sumSquares]
```

`Id.run do` uses `do` notation for a computation with *no* effects at all (`Id` is the "identity" monad), so `sumSquares` has the plain type `List Nat → Nat`, and it is as pure as `double`. How can a pure function

have a mutable variable? Because `let mut` and `for` are notation. Lean translates the block into ordinary functional code: each assignment introduces a new version of `total`, and the loop becomes a fold that threads the current version through the iterations. No memory cell is ever overwritten.

The theorem under the definition makes this concrete: the loop is equal to `List.foldl` with the function `fun total x => total + x * x`, and the `simp` tactic (explained later in this chapter) proves it by unfolding the translation. Inside `do` you can also `return` early, from the middle of a loop:

```
/-- Early `return` from inside a loop. -/
def firstNegative (xs : List Int) : Option Int := Id.run do
  for x in xs do
    if x < 0 then return some x
  return none

/-- info: some (-1) -/
#guard_msgs in
#eval firstNegative [3, -1, -5]
```

2.7 Totality: every function terminates

Rust lets you write any recursion you like, and a `loop {}` that never ends, which is the right choice for a language in which servers are supposed to run forever. Lean refuses such definitions, because of its proof-assistant side:

```
/--
error: fail to show termination for
  Ch02.bogus
with errors
failed to infer structural recursion:
Not considering parameter n of Ch02.bogus:
  it is unchanged in the recursive calls
no parameters suitable for structural recursion

well-founded recursion cannot be used, `Ch02.bogus` does not take any (non-fixed) arguments
-/
#guard_msgs in
def bogus (n : Nat) : Nat := bogus n + 1
```

If Lean accepted `bogus`, its defining equation `bogus 0 = bogus 0 + 1` would be a fact that proofs could use, and subtracting `bogus 0` from both sides gives `0 = 1`. From a false statement every statement follows, so a single non-terminating definition would make every theorem in the system worthless. Lean therefore accepts a recursive definition only when it can see that the recursion terminates. Functions that always terminate are called **total**, and totality is the price of being able to use programs inside proofs. Chapter 10 explains the same point from the logical side: in Rust, `loop {}` has every type, which is why Rust's types make a poor logic.

Lean has two ways to see termination. The easy one is **structural recursion**: every recursive call is on a piece of the argument, as in `fib` (calls on `n` and `n + 1` when the argument was `n + 2`) and `Tree.insert` (calls on the subtrees). It always terminates, because data is finite.

The other is **well-founded recursion**: Lean finds a quantity, called a *measure*, that decreases at every call and cannot decrease forever, and *proves* that it decreases. Often it manages on its own:

```
/-- Not structural (`n / 10` is not a sub-term of `n`), but Lean finds and
proves the decreasing measure on its own. -/
def digits (n : Nat) : List Nat :=
  if n < 10 then [n] else digits (n / 10) ++ [n % 10]

/-- Binary search for the first `n` in `[lo, hi)` where `ok` holds (for a
monotone `ok`). Lean guesses the measure `hi - lo` by itself. -/
def search (lo hi : Nat) (ok : Nat → Bool) : Nat :=
  if lo < hi then
    let mid := (lo + hi) / 2
    if ok mid then search lo mid ok else search (mid + 1) hi ok
  else lo

/-- info: ([2, 0, 2, 4], 8) -/
#guard_msgs in
#eval (digits 2024, search 0 100 (fun n => n * n ≥ 50))
```

For `digits`, the argument goes from `n` to `n / 10`, which is not a piece of `n` but is smaller when `n ≥ 10`, and Lean proves that automatically. For `search` it discovers that `hi - lo` shrinks at each call. When it fails, it says what it tried. Here is the famous Collatz function, which halves even numbers and maps an odd `n` to `3n + 1`:

```
/--
error: fail to show termination for
  Ch02.collatz
with errors
failed to infer structural recursion:
Cannot use parameter n:
  failed to eliminate recursive application
    collatz (n / 2)


failed to prove termination, possible solutions:
  - Use `have`-expressions to prove the remaining goals
  - Use `termination_by` to specify a different well-founded relation
  - Use `decreasing_by` to specify your own tactic for discharging this kind of goal
n : Nat
h✝¹ : ¬n ≤ 1
h✝ : ¬(n % 2 == 0) = true
⊢ 3 * n + 1 < n
-/
#guard_msgs in
def collatz (n : Nat) : Nat :=
  if n ≤ 1 then 0
  else if n % 2 == 0 then 1 + collatz (n / 2)
  else 1 + collatz (3 * n + 1)
```

The message has two parts. "Failed to infer structural recursion" says that `n / 2` is not a piece of `n`. Then Lean tried well-founded recursion with `n` itself as the measure, and got stuck on the goal printed at the end: in the odd branch (the hypotheses say `n` is not `≤ 1` and not even), it would have to show `3 * n + 1 < n`, which is false. In fact nobody knows whether this function terminates for every `n`: that is the Collatz conjecture, open since 1937.

For such cases there is `partial def`:

```
/-- Nobody knows whether this terminates for every `n`, so it must be
`partial`. Lean compiles and runs it, but will not unfold it in proofs. -/
partial def collatzSteps (n : Nat) : Nat :=
  if n ≤ 1 then 0
  else if n % 2 == 0 then 1 + collatzSteps (n / 2)
  else 1 + collatzSteps (3 * n + 1)
```

```
/-- info: 111 -/
#guard_msgs in
#eval collatzSteps 27
```

A `partial` function is compiled and runs normally, but for the logic it is an opaque constant: Lean knows its type and nothing about its values, so no proof can unfold it, and no contradiction can come from it. Lean also requires the function's type to be *inhabited*: it must be able to name some value of that type (here, for instance, the function that always returns `0`). That rules out a "partial proof" of a false proposition, whose type has no values at all. The book uses `partial` only when the point is that something may not terminate, as in the λ-calculus evaluator of Chapter 7.

Finally, when Lean cannot guess the measure, you can name it with `termination_by` and prove the decrease yourself with `decreasing_by`:

```
/-- Euclid's algorithm. Lean cannot guess why it terminates, so we say which
argument decreases (`termination_by`) and prove that it does (`decreasing_by`). -/
def gcd (a b : Nat) : Nat :=
  if b = 0 then a else gcd b (a % b)
termination_by b
decreasing_by
  -- Goal: a % b < b, knowing ¬b = 0.
  apply Nat.mod_lt   -- a % b < b holds whenever b > 0;
  omega              -- and b > 0 follows from ¬b = 0.

/-- info: 6 -/
#guard_msgs in
#eval gcd 12 18
```

Here the measure is `b`. The proof obligation is the goal `a % b < b`, under the hypothesis that `b` is not zero; `Nat.mod_lt` is the library fact "`a % b < b` whenever `b > 0`", and `omega`, a tactic you will meet below, proves `b > 0` from `¬b = 0`. Chapter 31 returns to well-founded recursion with harder examples.

2.8 Values that never change

In Lean no value is ever modified. You have seen the consequences: `{ p with ... }` builds a new structure, `xs.push x` returns a new array, `let mut` introduces new versions of a variable. This is what makes reasoning about Lean programs easy: a name always denotes the same value, so you can replace it by its definition anywhere, which is what proofs do all the time.

It would be terribly slow if every `push` copied the array, and it does not. Lean's runtime counts references to every heap object. When `Array.push` receives an array whose reference count is 1, nobody else can observe the array, so the runtime updates it *in place*, at the cost of Rust's `Vec::push`. Only when the array is shared (when some other variable still refers to the old version, as `readings` did above) does `push` copy it first. This technique is known as "functional but in place".

You know this situation from Rust: a `Vec` can be mutated through `&mut` because the ownership rules guarantee that nobody else can observe it meanwhile. Rust establishes that uniqueness *statically*, with the borrow checker; Lean checks it *dynamically*, with a reference count, and copies when uniqueness fails. Chapter 24 comes back to this with a demonstration.

2.9 Putting it together: an order book in two languages

Time for a complete program. An **order book** is the data structure at the heart of an exchange: traders send *limit orders* ("buy 10 at 100.50 or less", "sell 5 at 101.25 or more"), and the book keeps the orders that are waiting. The highest buy price is the *best bid*, the lowest sell price the *best ask*, and the gap between them is the *spread*. Our book is "post-only": an order that would trade immediately (a buy at or above the best ask, or a sell at or below the best bid) is rejected instead of matched, so the book is never *crossed*. Prices are in cents, so that arithmetic is exact.

The Rust version

The Rust program is `code/rust/examples/ch02_orders.rs`. Its types are what you would expect:

```rust
/// Which side of the book an order is on.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum Side {
    Buy,
    Sell,
}

/// A resting limit order. Prices are in cents, so that they are exact.
#[derive(Debug, Clone, PartialEq, Eq)]
struct Order {
    id: u64,
    trader: String,
    side: Side,
    price: u64,
    qty: u64,
}

/// Why an order or a cancellation was rejected.
#[derive(Debug, Clone, PartialEq, Eq)]
enum OrderError {
    ZeroQuantity,
    ZeroPrice,
    /// The order would trade against `best`, the best price on the other side.
    WouldCross {
        best: u64,
    },
    UnknownOrder(u64),
}

/// The resting orders, by id, and the id the next order will get.
#[derive(Debug)]
struct Book {
    orders: BTreeMap<u64, Order>,
    next_id: u64,
}
```

The queries use iterators over a `BTreeMap`, and `spread` uses `?` on `Option`:

```rust
impl Book {
    fn new() -> Book {
        Book {
            orders: BTreeMap::new(),
            next_id: 1,
        }
    }

    /// The orders on one side, in id order.
    fn side(&self, side: Side) -> impl Iterator<Item = &Order> {
        self.orders.values().filter(move |o| o.side == side)
    }
```

```
    /// The highest price a buyer is offering.
    fn best_bid(&self) -> Option<u64> {
        self.side(Side::Buy).map(|o| o.price).max()
    }

    /// The lowest price a seller is asking.
    fn best_ask(&self) -> Option<u64> {
        self.side(Side::Sell).map(|o| o.price).min()
    }

    /// The gap between the best ask and the best bid. The subtraction cannot
    /// underflow because the book is never crossed: Lean proves it.
    fn spread(&self) -> Option<u64> {
        let bid = self.best_bid()?;
        let ask = self.best_ask()?;
        Some(ask - bid)
    }

    /// The total quantity on one side.
    fn volume(&self, side: Side) -> u64 {
        self.side(side).map(|o| o.qty).sum()
    }

    /// The total quantity at each price on one side, by increasing price.
    fn depth(&self, side: Side) -> BTreeMap<u64, u64> {
        let mut levels = BTreeMap::new();
        for o in self.side(side) {
            *levels.entry(o.price).or_insert(0) += o.qty;
        }
        levels
    }
}
```

Placing an order validates it with early returns, then mutates the book through `&mut self`; cancelling removes the order from the map:

```
impl Book {
    /// The best price on the other side, if an order at `price` would trade
    /// against it.
    fn crossing(&self, side: Side, price: u64) -> Option<u64> {
        match side {
            Side::Buy => self.best_ask().filter(|&ask| ask <= price),
            Side::Sell => self.best_bid().filter(|&bid| price <= bid),
        }
    }

    /// Validate an order and add it to the book; return its id.
    fn place(
        &mut self,
        trader: &str,
        side: Side,
        price: u64,
        qty: u64,
    ) -> Result<u64, OrderError> {
        if qty == 0 {
            return Err(OrderError::ZeroQuantity);
        }
        if price == 0 {
            return Err(OrderError::ZeroPrice);
        }
        if let Some(best) = self.crossing(side, price) {
            return Err(OrderError::WouldCross { best });
        }
        let id = self.next_id;
        let trader = trader.to_string();
        let order = Order {
            id,
            trader,
```

```
            side,
            price,
            qty,
        };
        self.orders.insert(id, order);
        self.next_id += 1;
        Ok(id)
    }

    /// Remove an order from the book and return it.
    fn cancel(&mut self, id: u64) -> Result<Order, OrderError> {
        self.orders.remove(&id).ok_or(OrderError::UnknownOrder(id))
    }
}
```

The `main` function sends seven requests and runs a few queries (a small formatting function and two `Display` impls, in the same file, print prices as dollars and cents):

```
fn main() {
    let mut book = Book::new();
    let requests = [
        ("alice", Side::Buy, 10050, 10),
        ("bob", Side::Sell, 10125, 5),
        ("carol", Side::Buy, 10075, 7),
        ("dave", Side::Sell, 10125, 8),
        ("gina", Side::Sell, 10200, 4),
        ("erin", Side::Buy, 10150, 3),
        ("frank", Side::Sell, 9900, 0),
    ];
    for (trader, side, p, qty) in requests {
        match book.place(trader, side, p, qty) {
            Ok(id) => println!("placed   {}", book.orders[&id]),
            Err(e) => println!("rejected {trader}: {e}"),
        }
    }

    let (bid, ask) = (book.best_bid().unwrap(), book.best_ask().unwrap());
    let spread = book.spread().unwrap();
    println!(
        "best bid {}, best ask {}, spread {}",
        price(bid),
        price(ask),
        price(spread)
    );
    let (buys, sells) = (book.volume(Side::Buy), book.volume(Side::Sell));
    println!("volume: {buys} to buy, {sells} to sell");
    for (p, qty) in book.depth(Side::Sell) {
        println!("sell depth at {}: {qty}", price(p));
    }

    match book.cancel(3) {
        Ok(order) => println!("cancelled {order}"),
        Err(e) => println!("rejected cancel: {e}"),
    }
    match book.cancel(3) {
        Ok(order) => println!("cancelled {order}"),
        Err(e) => println!("rejected cancel: {e}"),
    }
    println!("best bid now {}", price(book.best_bid().unwrap()));
    assert_eq!(book.orders.len(), 4);
}
```

```
placed   #1 alice buys 10 @ 100.50
placed   #2 bob sells 5 @ 101.25
placed   #3 carol buys 7 @ 100.75
placed   #4 dave sells 8 @ 101.25
placed   #5 gina sells 4 @ 102.00
```

```
rejected erin: would cross the best price 101.25
rejected frank: quantity must be positive
best bid 100.75, best ask 101.25, spread 0.50
volume: 17 to buy, 17 to sell
sell depth at 101.25: 13
sell depth at 102.00: 4
cancelled #3 carol buys 7 @ 100.75
rejected cancel: no order #3
best bid now 100.50
```

Erin's buy at 101.50 is rejected because it would trade against the best ask of 101.25; Frank's sell has quantity zero.

The Lean version

The Lean program is `code/lean/Book/Ch02/Orders.lean`. The types translate one to one:

```
/-- Which side of the book an order is on. -/
inductive Side where
  | buy
  | sell
  deriving Repr, DecidableEq

/-- A resting limit order. Prices are in cents, so that they are exact. -/
structure Order where
  id : Nat
  trader : String
  side : Side
  price : Nat
  qty : Nat
  deriving Repr, BEq

/-- Why an order or a cancellation was rejected. -/
inductive OrderError where
  | zeroQuantity
  | zeroPrice
  | wouldCross (best : Nat)
  | unknownOrder (id : Nat)
  deriving Repr, BEq, DecidableEq

/-- The resting orders, oldest first, and the id the next order will get. -/
structure Book where
  orders : List Order
  nextId : Nat
  deriving Repr
```

The orders are kept in a `List`, oldest first, instead of a `BTreeMap`: lists are the simplest structure to prove things about, and our book is small. (Lean's `Std.TreeMap` is the counterpart of `BTreeMap`, and the `depth` query below uses it.) `Side` derives `DecidableEq` rather than `BEq`, for the reason given in the section on standard classes: the proofs below need to know that `==` on sides means equality.

The queries are functions in the `Book` namespace, so that dot notation works (`book.bestBid`):

```
def empty : Book := { orders := [], nextId := 1 }

/-- The orders on one side, oldest first. -/
def onSide (book : Book) (side : Side) : List Order :=
  book.orders.filter (fun o => o.side == side)

/-- The highest price a buyer is offering. -/
def bestBid (book : Book) : Option Nat :=
  (book.onSide .buy |>.map (·.price)).max?

/-- The lowest price a seller is asking. -/
```

```
def bestAsk (book : Book) : Option Nat :=
  (book.onSide .sell |>.map (·.price)).min?

/-- The gap between the best ask and the best bid. -/
def spread (book : Book) : Option Nat := do
  let bid ← book.bestBid
  let ask ← book.bestAsk
  return ask - bid

/-- The total quantity of a list of orders. -/
def totalQty : List Order → Nat
  | [] => 0
  | o :: os => o.qty + totalQty os

/-- The total quantity on one side. -/
def volume (book : Book) (side : Side) : Nat :=
  totalQty (book.onSide side)

/-- The total quantity at each price on one side, by increasing price.
`Std.TreeMap` is the counterpart of Rust's `BTreeMap`. -/
def depth (book : Book) (side : Side) : List (Nat × Nat) :=
  let addOrder (levels : Std.TreeMap Nat Nat) (o : Order) :=
    levels.alter o.price (fun q => some (q.getD 0 + o.qty))
  (book.onSide side |>.foldl addOrder ∅).toList

/-- The order with a given id, if it is in the book. -/
def find? (book : Book) (id : Nat) : Option Order :=
  book.orders.find? (fun o => o.id == id)
```

`bestBid` maps the orders to their prices and takes `List.max?`, which returns `none` for an empty list, like Rust's `Iterator::max`. `spread` uses `do` and `←` on `Option` where Rust used `?`. `totalQty` is a recursive function rather than `map(...).sum()`, because we will prove a theorem about it by induction. `depth` uses `alter` where Rust used the `entry` API.

The commands are where the two versions differ most:

```
/-- The best price on the other side, if an order at `price` would trade
against it. -/
def crossing (book : Book) (side : Side) (price : Nat) : Option Nat :=
  match side with
  | .buy => book.bestAsk.filter (fun ask => ask ≤ price)
  | .sell => book.bestBid.filter (fun bid => price ≤ bid)

/-- Validate an order and add it to the book. The result is the new order's
id together with the *new* book; the old book is unchanged. -/
def place (book : Book) (trader : String) (side : Side) (price qty : Nat) :
    Except OrderError (Nat × Book) :=
  if qty = 0 then .error .zeroQuantity
  else if price = 0 then .error .zeroPrice
  else match book.crossing side price with
    | some best => .error (.wouldCross best)
    | none =>
      let order := { id := book.nextId, trader, side, price, qty }
      .ok (book.nextId, { orders := book.orders ++ [order], nextId := book.nextId + 1 })

/-- Remove an order from the book; return it with the new book. -/
def cancel (book : Book) (id : Nat) : Except OrderError (Order × Book) :=
  match book.find? id with
  | none => .error (.unknownOrder id)
  | some order => .ok (order, { book with orders := book.orders.filter (·.id != id) })
```

`place` cannot mutate the book. It returns an `Except` whose success value is a pair: the new order's id and a *new book*. The old book still exists, and whoever called `place` decides which one to keep. The early returns of the Rust version become a chain of `if ... else if ... else match`. We could have

written `place` with `do`, `throw` and `return`, as `parseNat` was; the plain-expression style is used here because it makes the proofs later in the chapter shorter.

The driver is an `IO` action that keeps the current book in a `let mut` variable, replacing it after each successful command:

```
def requests : List (String × Side × Nat × Nat) :=
  [("alice", .buy, 10050, 10), ("bob", .sell, 10125, 5),
   ("carol", .buy, 10075, 7), ("dave", .sell, 10125, 8),
   ("gina", .sell, 10200, 4), ("erin", .buy, 10150, 3),
   ("frank", .sell, 9900, 0)]

/-- The book after all the requests, ignoring the rejected ones. -/
def demoBook : Book :=
  requests.foldl (init := Book.empty) fun book (trader, side, price, qty) =>
    match book.place trader side price qty with
    | .ok (_, book') => book'
    | .error _ => book

def cancelAndReport (book : Book) (id : Nat) : IO Book := do
  match book.cancel id with
  | .ok (order, book') =>
    IO.println s!"cancelled {order}"
    return book'
  | .error e =>
    IO.println s!"rejected cancel: {e}"
    return book

def demo : IO Unit := do
  let mut book := Book.empty
  for (trader, side, price, qty) in requests do
    match book.place trader side price qty with
    | .ok (id, book') =>
      book := book'
      if let some order := book.find? id then
        IO.println s!"placed   {order}"
    | .error e => IO.println s!"rejected {trader}: {e}"
  if let (some bid, some ask, some spread) := (book.bestBid, book.bestAsk, book.spread) then
    IO.println s!"best bid {showPrice bid}, best ask {showPrice ask}, spread {showPrice
spread}"
  IO.println s!"volume: {book.volume .buy} to buy, {book.volume .sell} to sell"
  for (price, qty) in book.depth .sell do
    IO.println s!"sell depth at {showPrice price}: {qty}"
  book ← cancelAndReport book 3
  book ← cancelAndReport book 3
  if let some bid := book.bestBid then
    IO.println s!"best bid now {showPrice bid}"
```

`for (trader, side, price, qty) in requests do` destructures each tuple, as Rust's `for (trader, side, p, qty) in requests` does. `if let some order := ... then` is Rust's `if let Some(order) = ...`. The function `cancelAndReport` returns the book to keep, and `book ← cancelAndReport book 3` runs it and rebinds `book`. Running the action prints exactly what the Rust program printed:

```
/--
info: placed   #1 alice buys 10 @ 100.50
placed   #2 bob sells 5 @ 101.25
placed   #3 carol buys 7 @ 100.75
placed   #4 dave sells 8 @ 101.25
placed   #5 gina sells 4 @ 102.00
rejected erin: would cross the best price 101.25
rejected frank: quantity must be positive
best bid 100.75, best ask 101.25, spread 0.50
volume: 17 to buy, 17 to sell
```

```
sell depth at 101.25: 13
sell depth at 102.00: 4
cancelled #3 carol buys 7 @ 100.75
rejected cancel: no order #3
best bid now 100.50
-/
#guard_msgs in
#eval demo
```

The file also defines `demoBook`, the book after all seven requests, computed by a `foldl` over the requests without any `IO`. We will prove things about it next.

2.10 Proving things about programs

The Rust program can be tested on a few books. The properties of the Lean program can be *proved*, for every book at once. The proofs are in `code/lean/Book/Ch02/FirstProofs.lean`.

Theorems and examples

You saw in Chapter 1 that a theorem looks like a definition whose type is a proposition, and whose body is a proof of that proposition. An `example` is a theorem without a name, useful for checks that nothing else will refer to. The simplest proofs are computations:

```
/-- Facts about the demo book, proved by computation. These are unit tests
that Lean runs every time the file is checked. -/
example : demoBook.bestBid = some 10075 := rfl

example : demoBook.spread = some 50 := by rfl

example : demoBook.volume .sell = 17 := by decide

example : demoBook.depth .sell = [(10125, 13), (10200, 4)] := by decide
```

These are unit tests, written as theorems: Lean evaluates both sides and checks that they agree, every time the file is checked. Two tactics do the work. `rfl` proves an equation whose two sides *compute* to the same value; here Lean runs `place` seven times inside the type checker. (The first example uses `rfl` as a term, without `by`; `by rfl` is the same proof written as a tactic.) `decide` asks Lean to evaluate a *decidable* proposition, one with an algorithm that answers yes or no, and succeeds if the answer is yes. Equality of numbers and lists is decidable, so either tactic works here.

Proofs for every book

The next theorem is not about one book. It says that an order with quantity zero is rejected, whatever the book, trader, side and price:

```
/-- An order with quantity zero is rejected, whatever the book, trader, side
and price. -/
theorem place_zero_qty (book : Book) (trader : String) (side : Side) (price : Nat) :
    book.place trader side price 0 = .error .zeroQuantity := by
  simp [Book.place]

/-- An order with price zero (and a positive quantity) is rejected. -/
theorem place_zero_price (book : Book) (trader : String) (side : Side) (qty : Nat)
    (h : qty ≠ 0) : book.place trader side 0 qty = .error .zeroPrice := by
  simp [Book.place, h]
```

The parameters before the colon are universally quantified: the theorem holds for *all* values of them. Rust can only check this property on the cases a test happens to try; Lean has checked it for all of them.

The proofs use `simp`, the simplifier, the tactic you will use most. `simp` rewrites the goal repeatedly with a large collection of equations from the library (lemmas tagged as simplification rules, such as `0 = 0 ↔ True` and "`if True then a else b` is `a`"), plus whatever you list in the brackets. Listing `Book.place` tells it to unfold the definition of `place`. In the first theorem, unfolding produces `if 0 = 0 then .error .zeroQuantity else ...`, the condition simplifies to `True`, the `if` to its first branch, and the goal becomes `.error .zeroQuantity = .error .zeroQuantity`, which `simp` closes. In the second, the hypothesis `h : qty ≠ 0` is listed too, so that `simp` can decide the first `if` the other way.

Induction on lists, and reading goals

The volume of a book is computed by `totalQty`, and a key fact about it is that the total of two lists appended is the sum of the totals. It is proved by induction on the first list, like Gauss's formula in Chapter 1. But let us first look at an *unfinished* version of the proof, to see what Lean shows you while you work:

```
/--
error: unsolved goals
case cons
ys : List Order
o : Order
os : List Order
ih : totalQty (os ++ ys) = totalQty os + totalQty ys
⊢ o.qty + (totalQty os + totalQty ys) = o.qty + totalQty os + totalQty ys
-/
#guard_msgs in
example (xs ys : List Order) :
    totalQty (xs ++ ys) = totalQty xs + totalQty ys := by
  induction xs with
  | nil => simp [totalQty]
  | cons o os ih => simp [totalQty, ih]
```

The error message is a snapshot of the infoview at the point where the proof ended, and learning to read it is the most useful skill in this chapter:

- `case cons` names the goal: we are in the `cons` case of the induction, where the list is `o :: os`.
- The lines above `⊢` are the **hypotheses**, everything we may use: the variables `ys`, `o` and `os`, with their types, and the induction hypothesis `ih`, which states the theorem for the shorter list `os`.
- The line after `⊢` (read "turnstile", typed `\vdash`) is the **goal**, what remains to be proved. It is the judgment notation of Chapter 1 again: the hypotheses play the role of the context Γ.

So `simp` unfolded `totalQty` on both sides and rewrote with `ih`, and what is left is an equation between sums that only differs in where the parentheses are. The finished proof adds one line:

```
/-- The total quantity of two lists of orders, one after the other. -/
theorem totalQty_append (xs ys : List Order) :
    totalQty (xs ++ ys) = totalQty xs + totalQty ys := by
  induction xs with
  | nil =>
    -- Goal: totalQty ([] ++ ys) = totalQty [] + totalQty ys
    simp [totalQty]
  | cons o os ih =>
    -- ih   : totalQty (os ++ ys) = totalQty os + totalQty ys
```

```
    -- Goal : totalQty (o :: os ++ ys) = totalQty (o :: os) + totalQty ys
    simp [totalQty, ih]
    -- Goal : o.qty + (totalQty os + totalQty ys) = o.qty + totalQty os + totalQty ys
    omega
```

`omega` is a decision procedure for linear arithmetic over `Nat` and `Int`: equations and inequalities built from variables, constants, `+`, `-`, and multiplication, division and remainder by constants. It treats anything else (here `o.qty`, `totalQty os` and `totalQty ys`) as an unknown number, and proves the goal if it follows from the hypotheses for *all* values of the unknowns. When the goal does not follow, `omega` describes a counterexample:

```
/--
error: omega could not prove the goal:
a possible counterexample may satisfy the constraints
  b ≥ 0
  a ≥ 0
  a - b ≥ 1
where
 a := ↑(totalQty ys)
 b := ↑(totalQty xs)
-/
#guard_msgs in
example (xs ys : List Order) : totalQty xs ≥ totalQty ys := by
  omega
```

The claim is false, and `omega` says why: the goal fails whenever `a`, the total of `ys`, exceeds `b`, the total of `xs`. (The arrows `↑` mean that `omega` moved the naturals into the integers to do its work.) Reading a failed `omega` this way often reveals a missing hypothesis or a false statement.

What a successful `place` did

To prove properties of `place` we need to know what a successful call returns. This lemma takes the definition apart, branch by branch:

```
/-- What a successful `place` did: the order did not cross the book, and it
was appended to the orders with the next id. -/
theorem place_ok {book book' : Book} {trader : String} {side : Side}
    {price qty id : Nat} (h : book.place trader side price qty = .ok (id, book')) :
    book.crossing side price = none ∧
      book'.orders = book.orders ++ [{ id := book.nextId, trader, side, price, qty }] := by
  unfold Book.place at h
  split at h            -- the `if qty = 0` in `place`
  · contradiction       -- that branch returned an error, not `.ok`
  · split at h          -- the `if price = 0`
    · contradiction
    · split at h        -- the `match` on `book.crossing side price`
      · contradiction
      · rename_i hcross -- hcross : book.crossing side price = none
        cases h         -- `.ok (book.nextId, {...}) = .ok (id, book')` fixes id and book'
        exact ⟨hcross, rfl⟩
```

The hypothesis `h` says that `place` returned `.ok (id, book')`. The proof walks through `place` in the same order as the program does:

- `unfold Book.place at h` replaces `book.place ...` in `h` by the body of `place`, a chain of `if`s and a `match`.
- `split at h` splits the proof into two cases, one per branch of the first `if`. In the branch `qty = 0`, `h` becomes `.error .zeroQuantity = .ok (id, book')`, an equation between different constructors,

which is impossible, and `contradiction` closes the case. Each bullet `·` (typed `\.`) focuses on one goal.

- The other two `split`s do the same for `price = 0` and for the `match` on `book.crossing side price`. Only the path on which no validation failed survives.
- On that path Lean knows `book.crossing side price = none`, under a name it made up, shown as `heq✝` in the infoview (you cannot refer to names with a dagger). `rename_i hcross` names it.
- Now `h` says `.ok (book.nextId, {...}) = .ok (id, book')`. Since constructors are injective, `cases h` learns that `id` is `book.nextId` and `book'` is the new book, and substitutes them everywhere.
- The goal is a conjunction, and `⟨hcross, rfl⟩` proves it from a proof of each half, with the angle brackets of Chapter 1's `and_swap`.

With this lemma, the volume theorems are one `simp` each:

```
/-- Placing an order adds its quantity to the volume of its side... -/
theorem volume_place {book book' : Book} {trader : String} {side : Side}
    {price qty id : Nat} (h : book.place trader side price qty = .ok (id, book')) :
    book'.volume side = book.volume side + qty := by
  have hord := (place_ok h).2
  simp [Book.volume, Book.onSide, hord, List.filter_append, totalQty_append, totalQty]

/-- ...and leaves the other side alone. -/
theorem volume_place_other {book book' : Book} {trader : String} {side other : Side}
    {price qty id : Nat} (h : book.place trader side price qty = .ok (id, book'))
    (hother : other ≠ side) : book'.volume other = book.volume other := by
  have hord := (place_ok h).2
  simp [Book.volume, Book.onSide, hord, List.filter_append, Ne.symm hother]
```

`have hord := (place_ok h).2` names the second half of the lemma's conclusion, the equation for `book'.orders`. The `simp` call then rewrites with it, unfolds `volume` and `onSide`, distributes the `filter` over `++` (`List.filter_append` is a library lemma), and uses `totalQty_append`. In the second theorem the hypothesis `other ≠ side` (flipped by `Ne.symm` into `side ≠ other`) lets `simp` see that the new order is filtered out. Cancellation gets the same treatment:

```
/-- After a successful cancellation, no order with that id is left. -/
theorem cancel_removes {book book' : Book} {id : Nat} {order : Order}
    (h : book.cancel id = .ok (order, book')) : book'.find? id = none := by
  unfold Book.cancel at h
  split at h
  · contradiction
  · cases h
    simp [Book.find?]
```

The proof has the same structure as the one of `place_ok`: unfold, `split` on the `match` in `cancel`, dismiss the error case with `contradiction`, and let `cases h` substitute the new book, whose orders are `book.orders.filter (·.id != id)`. What remains is the claim that searching that filtered list for `id` finds nothing, and `simp` knows enough about `find?` and `filter` to prove it.

`sorry`

While you develop a proof, you can leave a hole with `sorry`, and Lean will accept the theorem with a warning:

```
/-- warning: declaration uses `sorry` -/
#guard_msgs in
theorem unfinished (book : Book) : book.volume .buy ≤ totalQty book.orders := by
  sorry

/-- info: 'Ch02.unfinished' depends on axioms: [sorryAx] -/
#guard_msgs in
#print axioms unfinished
```

`sorry` is useful for working top-down: state the lemmas you need, `sorry` their proofs, and check that the main argument goes through. But a `sorry` is a lie that Lean remembers: `#print axioms` (Chapter 1) shows that the theorem depends on `sorryAx`, an axiom that proves *anything*, and so does every theorem that uses it. This is the one place where the book writes `sorry`, to show you the warning; no proof in the book depends on it, as `#print axioms` can confirm.

An invariant: the book is never crossed

The Rust `spread` subtracts the best bid from the best ask and claims, in a comment, that the subtraction cannot underflow because the book is never crossed. Comments like that are where bugs hide. In Lean we can prove it. First, the invariant, as a proposition about a book:

```
/-- A book is *uncrossed* when every bid is strictly below every ask. -/
def Book.Uncrossed (book : Book) : Prop :=
  ∀ b ∈ book.orders, ∀ s ∈ book.orders,
    b.side = .buy → s.side = .sell → b.price < s.price
```

`Book.Uncrossed` is a function from books to propositions; `∀ b ∈ book.orders, ...` reads "for every order `b` in the book's list". The theorem that `place` preserves the invariant is the most ambitious proof of this chapter, and uses a few tactics that Chapter 4 introduces properly, so feel free to come back to it later:

```
/-- `place` preserves the invariant: an uncrossed book stays uncrossed. -/
theorem place_uncrossed {book book' : Book} {trader : String} {side : Side}
    {price qty id : Nat} (hinv : book.Uncrossed)
    (h : book.place trader side price qty = .ok (id, book')) : book'.Uncrossed := by
  obtain ⟨hcross, hord⟩ := place_ok h
  intro b hb s hs hbuy hsell
  rw [hord, List.mem_append, List.mem_singleton] at hb hs
  rcases hb with hb | rfl <;> rcases hs with hs | rfl
  · -- an old bid and an old ask: the invariant of the old book
    exact hinv b hb s hs hbuy hsell
  · -- an old bid and the new ask: the new order is a sell that did not cross
    simp only at hsell
    subst hsell
    exact lt_of_not_crossing_sell hcross hb hbuy
  · -- the new bid and an old ask: the new order is a buy that did not cross
    simp only at hbuy
    subst hbuy
    exact lt_of_not_crossing_buy hcross hs hsell
  · -- the new order would be both a buy and a sell
    simp only at hbuy hsell
    rw [hbuy] at hsell
    contradiction
```

Step by step:

- `obtain ⟨hcross, hord⟩ := place_ok h` destructures the conjunction from the lemma into its two halves, like `let (a, b) = pair;` in Rust.

- The goal `book'.Uncrossed` is a "for all" statement. `intro b hb s hs hbuy hsell` says "let `b` be any order, assume `hb : b ∈ book'.orders`, let `s` be any order, assume `hs`, `hbuy` and `hsell`", and leaves the goal `b.price < s.price`.
- `rw [...] at hb hs` rewrites the two membership hypotheses: `book'.orders` is `book.orders ++ [new order]` by `hord`, membership in an append is a disjunction (`List.mem_append`), and membership in a one-element list is equality (`List.mem_singleton`). Now `hb` says "`b` is an old order, or `b` is the new one", and likewise `hs`.
- `rcases hb with hb | rfl` splits on that "or": either `b` is old (and `hb` says so), or `b` *is* the new order, and the pattern `rfl` substitutes it everywhere. The combinator `<;>` runs the second `rcases` on both resulting goals, so we get four cases.
- Old bid and old ask: that is exactly what the old invariant `hinv` covers.
- Old bid, new ask: `hsell` says the new order's side is `.sell`; `simp only` reduces `{...}.side` to `side`, and `subst` replaces `side` by `.sell` everywhere. Now `hcross` says that a *sell* at `price` does not cross, and the helper lemma `lt_of_not_crossing_sell` (in the same file) turns that into "every bid is below `price`", using the library fact that the maximum of a list is at least each of its elements.
- New bid, old ask: the mirror image, with the minimum of the sell prices.
- New bid and new ask: the new order would have side `.buy` and `.sell` at once; after rewriting, `hsell` claims `Side.buy = Side.sell`, two different constructors, and `contradiction` closes the goal.

With the invariant in hand, the claim in the Rust comment becomes a theorem:

```
/-- In an uncrossed book the best bid is below the best ask, so the
subtraction in `spread` (and in Rust's `spread`) never truncates. -/
theorem bestBid_lt_bestAsk {book : Book} (hinv : book.Uncrossed) {bid ask : Nat}
    (hbid : book.bestBid = some bid) (hask : book.bestAsk = some ask) : bid < ask := by
  have hb := List.max?_mem hbid
  have ha := List.min?_mem hask
  simp only [Book.onSide, List.mem_map, List.mem_filter] at hb ha
  obtain ⟨b, ⟨hb, hbuy⟩, rfl⟩ := hb
  obtain ⟨s, ⟨hs, hsell⟩, rfl⟩ := ha
  exact hinv b hb s hs (by simpa using hbuy) (by simpa using hsell)
```

`List.max?_mem` says that the maximum of a list is one of its elements, so `hb` says that `bid` is among the buy prices, and `ha` that `ask` is among the sell prices. `simp only` restates that as "there is an order in the book, on the buy side, whose price is `bid`", and `obtain ⟨b, ⟨hb, hbuy⟩, rfl⟩ := hb` takes it apart, naming the order `b` and replacing `bid` by `b.price` (the pattern `rfl`). After the same for the ask, the goal `b.price < s.price` is the invariant applied to `b` and `s`. The `simpa using hbuy` arguments convert `(b.side == .buy) = true`, the form in which `filter` states it, into `b.side = .buy`.

Finally, the empty book is uncrossed (`simp` sees that there are no orders to compare), and the demo book is too, which `decide` checks by evaluating the invariant on its five orders after `unfold` has exposed the definition:

```
theorem empty_uncrossed : Book.empty.Uncrossed := by
  simp [Book.Uncrossed, Book.empty]

example : demoBook.Uncrossed := by
  unfold Book.Uncrossed
  decide
```

By `place_uncrossed`, every book obtained from the empty one by successful calls to `place` is uncrossed too (cancelling an order also preserves the invariant, as Exercise 4 asks you to prove).

The pattern "the initial state satisfies the invariant, and every operation preserves it" is the same as in the type safety theorems of Chapter 9, where the invariant is "the program is well typed" and the operation is a step of evaluation.

2.11 Mathlib

What it is

Mathlib is the library of mathematics written by Lean's community: numbers of every kind, algebra, order theory, analysis, combinatorics, category theory and much more, with hundreds of thousands of theorems, all checked by Lean. It is also a library of **tactics**: many of the proof-automation tools Lean users rely on are defined in Mathlib rather than in Lean itself.

The division of labor is roughly this. Lean's core library (loaded automatically) and `Std` (which ships with Lean and is imported on demand, as `Classes.lean` imported `Std.Data.HashMap`) provide the programming language: `Nat`, `Int`, `String`, `List`, `Array`, `HashMap`, `IO`, basic lemmas about them, and the general-purpose tactics `rfl`, `decide`, `simp`, `omega`, `induction` and `grind`. Mathlib provides mathematics: the rationals and reals, algebraic structures such as groups and rings with their laws, order theory, sets and finite sets, categories, and the tactics that know about them. Everything in this chapter so far used only the core library and `Std`.

How the project depends on it

The book's Lean project is a **Lake** project. Lake is Lean's build tool and package manager, the counterpart of Cargo. Its configuration is `code/lean/lakefile.toml`:

```
# The book's Lean project. It depends on Mathlib, pinned to the release that
# matches the toolchain in lean-toolchain. Before the first build, download the
# prebuilt library (under 1 GB to download, about 8 GB on disk) with
#   lake exe cache get
# otherwise Lake would compile Mathlib from source, which takes hours.
name = "book"
version = "0.1.0"
defaultTargets = ["Book"]

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4"
rev = "v4.34.1"

[[lean_lib]]
name = "Book"
globs = ["Book.+"]
```

The `[[require]]` entry is the counterpart of a line under `[dependencies]` in `Cargo.toml`. The package is fetched from its Git repository and pinned to the tag `v4.34.1`, the Mathlib release that matches the Lean version in `lean-toolchain` (the two must be upgraded together). The exact commits of Mathlib and of the packages it depends on in turn (Batteries, Aesop and a few others) are recorded in `lake-manifest.json`, Lake's `Cargo.lock`, and the sources end up in `code/lean/.lake/packages/`.

Mathlib is big, about eight and a half thousand source files, and compiling it takes hours. Its maintainers therefore publish the compiled files (`.olean` files, Lean's counterpart of `.rlib`) for every release, and `lake exe cache get` downloads them. You run it once, after cloning.

Importing only what you use

An `import` names a module, which is a file: `import Mathlib.Tactic.Ring` loads `Mathlib/Tactic/Ring.lean` and everything it imports in turn. You could write `import Mathlib` and get the whole library, but every import is loaded each time a file is checked, so the book's files import only the modules they need. This chapter's Mathlib examples are in `code/lean/Book/Ch02/MathlibTour.lean`, which begins with:

```
import Mathlib.Tactic.Ring
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Positivity
```

To find the module that defines something, search the sources: for example, `grep -rn "theorem mul_le_mul" .lake/packages/mathlib/Mathlib` run in `code/lean`.

Notation

Mathlib adds the familiar mathematical names for the number types: `ℕ` for `Nat`, `ℤ` for `Int`, `ℚ` for the rationals and `ℝ` for the reals. They are only notation; `ℕ` and `Nat` are the same type, and once Mathlib is loaded Lean prints the short form:

```
/-- info: 3 : ℕ -/
#guard_msgs in
#check (3 : Nat)

/-- info: -3 : ℤ -/
#guard_msgs in
#check (-3 : Int)

/-- info: 1 / 3 : ℚ -/
#guard_msgs in
#check (1 / 3 : ℚ)

example : ℕ = Nat := rfl
```

This is why Chapter 1's outputs showed `ℕ`: its file imported Mathlib. The files earlier in this chapter do not, and print `Nat`.

Four tactics the book uses constantly

`ring` proves equations that hold in every commutative ring (or semiring, like `ℕ`), by normalizing both sides as polynomials. You saw it finish Gauss's formula in Chapter 1:

```
/-- `ring` proves equations that hold in every commutative ring (or semiring,
such as `ℕ`): it normalizes both sides as polynomials and compares them. -/
theorem square_of_sum (a b : ℤ) : (a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2 := by
  ring

/-- Two fills at the same price cost the same as one fill of the total. -/
theorem cost_split (price q₁ q₂ : ℕ) : price * q₁ + price * q₂ = price * (q₁ + q₂) := by
  ring
```

`linarith` proves a linear inequality (or equation) from linear hypotheses, over ordered fields such as `ℚ` and `ℝ` as well as over `ℕ` and `ℤ`. Here it shows that the mid price lies strictly between the bid and the ask:

```
/-- `linarith` proves a linear inequality from linear hypotheses. Here, over
the rationals: the mid price lies strictly between the bid and the ask. -/
theorem mid_between (bid ask : ℚ) (h : bid < ask) :
    bid < (bid + ask) / 2 ∧ (bid + ask) / 2 < ask := by
  constructor   -- split the ∧ into two goals
  · linarith
  · linarith
```

(`constructor` splits the goal `A ∧ B` into the goals `A` and `B`; Chapter 4 has the details.) `omega` cannot do this: it works only over `ℕ` and `ℤ`, and over the rationals it finds nothing it understands:

```
/--
error: omega could not prove the goal:
No usable constraints found. You may need to unfold definitions so `omega` can see linear
arithmetic facts about `Nat` and `Int`, which may also involve multiplication, division, and
modular remainder by constants.
-/
#guard_msgs in
example (bid ask : ℚ) (h : bid < ask) : bid < (bid + ask) / 2 := by
  omega
```

`norm_num` evaluates numerical expressions, including fractions, powers and comparisons, and proves the resulting equalities and inequalities:

```
/-- `norm_num` evaluates and normalizes numerical expressions: the spread of
the demo book, in dollars. -/
example : (10125 : ℚ) / 100 - 10075 / 100 = 1 / 2 := by
  norm_num

example : (2 : ℤ) ^ 10 - 24 = 1000 := by
  norm_num
```

`positivity` proves goals of the form `0 < e` and `0 ≤ e` by following the structure of `e`:

```
/-- `positivity` proves goals of the form `0 < e` or `0 ≤ e` by looking at the
shape of `e`: squares are nonnegative, sums of nonnegatives are, and so on. -/
example (x : ℤ) (n : ℕ) : 0 < x ^ 2 + n + 1 := by
  positivity
```

A good rule of thumb for arithmetic goals: `omega` for linear facts about `ℕ` and `ℤ`, `linarith` when the numbers are rationals or reals or the facts come from hypotheses, `ring` for algebraic identities, `norm_num` for concrete numbers, `positivity` for signs, and `simp` to clean up first.

Finding lemmas

Sooner or later you need a fact that is surely in the library, and you do not know its name. The tactic `exact?` searches the library (everything you have imported) for a lemma that closes the goal, and tells you what it found:

```
/--
info: Try this:
  [apply] exact Nat.mul_le_mul_left price h
-/
#guard_msgs in
```

```
example (price qty : ℕ) (h : qty ≤ 100) : price * qty ≤ price * 100 := by
  exact?
```

"Try this" is a suggestion: in the editor you click it, and the `exact?` is replaced by the proof. The finished theorem uses the lemma directly:

```
/-- The proof `exact?` found, pasted in. -/
theorem cost_le_cap (price qty : ℕ) (h : qty ≤ 100) : price * qty ≤ price * 100 :=
  Nat.mul_le_mul_left price h
```

Its sibling `apply?` is useful when no single lemma closes the goal: it lists lemmas whose conclusion *matches* the goal, together with the hypotheses each one would still require, so that you can pick the most promising and continue from there.

Searching works much better once you know that lemma names are systematic. A name describes the statement, starting from the conclusion, with symbols spelled out: `add` for `+`, `mul` for `*`, `le` for `≤`, `lt` for `<`, `comm` for commutativity, `assoc` for associativity. Hypotheses come after `_of_`, `_iff` marks an "if and only if", and a namespace prefix says which type the lemma is about:

```
/-- info: Nat.add_comm (n m : ℕ) : n + m = m + n -/
#guard_msgs in
#check Nat.add_comm

/-- info: add_comm.{u_1} {G : Type u_1} [AddCommMagma G] (a b : G) : a + b = b + a -/
#guard_msgs in
#check add_comm

/-- info: Nat.lt_of_lt_of_le {n m k : ℕ} : n < m → m ≤ k → n < k -/
#guard_msgs in
#check Nat.lt_of_lt_of_le

/-- info: Nat.succ_le_iff {m n : ℕ} : m.succ ≤ n ↔ m < n -/
#guard_msgs in
#check Nat.succ_le_iff
```

`Nat.add_comm` is commutativity of addition on naturals; `add_comm` is the same law for every type `G` with a commutative addition, which is what the instance argument `[AddCommMagma G]` says. `Nat.lt_of_lt_of_le` concludes `<` from a `<` and a `≤`. Many Mathlib lemmas are stated once for a whole family of types, which makes their signatures look intimidating at first:

```
/--
info: mul_le_mul.{u_1} {α : Type u_1} [Mul α] [Zero α] [Preorder α] {a b c d : α}
[PosMulMono α] [MulPosMono α] (h₁ : a ≤ b)
  (h₂ : c ≤ d) (c0 : 0 ≤ c) (b0 : 0 ≤ b) : a * c ≤ b * d
-/
#guard_msgs in
#check mul_le_mul
```

Read past the instance arguments to the parameters and the conclusion: if `a ≤ b` and `c ≤ d`, and `c` and `b` are nonnegative, then `a * c ≤ b * d`. The instance arguments just say that `α` must have a multiplication, a zero and an order that interact in the expected way, which they do for `ℕ`, `ℤ`, `ℚ` and `ℝ`.

2.12 A Rust–Lean phrasebook

The tables collect the correspondences of this chapter. Appendix B has a longer version, arranged for quick lookup.

Rust	Lean
`fn f(x: u64) -> u64 { x + 1 }`	`def f (x : Nat) : Nat := x + 1`
`f(a, b)`	`f a b`
a closure taking `x`	`fun x => x + 1` or `(· + 1)`
`fn id<T>(x: T) -> T`	`def id {α : Type} (x : α) : α`
`id::<u64>(5)`	`@id Nat 5`
`let y = e;`	`let y := e`
`struct P { x: i64 }`	`structure P where x : Int`
`P { x: 1 }`	`{ x := 1 }` or `⟨1⟩`
`P { x: 2, ..p }`	`{ p with x := 2 }`
`enum E { A, B(u64) }`	`inductive E` with constructors `a`, `b (n : Nat)`
`E::B(3)`	`E.b 3`, or `.b 3` when the type is known
`match e { ... }`	`match e with`, then one arm per case
`Box<Tree>` in a recursive enum	just `Tree`

Rust	Lean
`Option<T>`: `Some(x)`, `None`	`Option α`: `some x`, `none`
`Result<T, E>`: `Ok(x)`, `Err(e)`	`Except ε α`: `.ok x`, `.error e`
`e?`	`← e` inside `do`
`return Err(e)`	`throw e` inside `do`
`Vec<T>`, `vec![1, 2]`	`Array α`, `#[1, 2]`
(linked list)	`List α`, `[1, 2]`, `x :: xs`
`impl P { fn norm(&self) }`	`def P.norm (p : P)`, called as `p.norm`
`mod m`, `use m::x`	`namespace m`, `open m`
`trait T`, `impl T for P`	`class T`, `instance : T P`
`fn g<A: T>(a: A)`	`def g {α : Type} [T α] (a : α)`
`#[derive(Debug, PartialEq)]`	`deriving Repr, BEq`
`let mut x = 0; x += 1;`	`let mut x := 0` then `x := x + 1`, in `do`
`println!("{x}")`	`IO.println s!"{x}"`
`loop {}`, unbounded recursion	only with `partial def`
`assert_eq!(a, b)` in a test	`example : a = b := by decide`

2.13 Common confusions

"**`=` and `==` are the same thing.**" `a == b` is a `Bool` computed by a `BEq` instance, for use in programs. `a = b` is a *proposition*, a statement that may or may not have a proof, for use in theorems.

`if a = b then ...` works when equality is decidable (with a `DecidableEq` instance), and then the two agree. Chapter 5 explains the difference between `Bool` and `Prop` in depth.

"**`let mut` makes Lean imperative.**" `let mut`, reassignment and `for` exist only inside `do`, and Lean translates them into pure functional code, as `sumSquares_eq_foldl` showed. The function's type tells you whether it has effects: `List Nat → Nat` has none, `IO Unit` may have any.

"**A theorem about the Lean model proves the Rust program correct.**" It proves the Lean program correct. The two agree only as far as we translated faithfully; still, the Lean proof tells us which invariant the Rust code relies on, and Chapter 31 shows how to test a Rust implementation against a verified Lean model.

"**`simp` failed, so the statement is false.**" `simp` is a rewriting engine, not a decision procedure. When it gets stuck, look at the goal it leaves: often it needs a definition unfolded (`simp [f]`), a hypothesis (`simp [h]`), or another tactic (`omega`, `ring`) to finish.

2.14 Summary

- Lean's `def`, `fun`, `structure`, `inductive` and `match` are Rust's `fn`, closures, `struct`, `enum` and `match`, with application by juxtaposition and curried functions.
- Type parameters are ordinary parameters of type `Type`, usually implicit (`{α : Type}`); `@` makes them explicit.
- `Option` and `Except` play the roles of `Option` and `Result`; `List` is a persistent linked list, `Array` is the counterpart of `Vec`.
- Namespaces and dot notation replace `impl` blocks; classes and instances are traits and impls; instance arguments `[C α]` are trait bounds; `deriving` is `#[derive]`, and `DecidableEq` is the one to derive when you want proofs.
- `do` notation provides `←` (Rust's `?`), `let mut`, `for` and early `return`, translated into pure code; effects such as printing require `IO`.
- Every Lean function must terminate, because programs are used in proofs: structural recursion is accepted automatically, other recursion needs a decreasing measure, and `partial def` opts out at the cost of being opaque to proofs.
- Values never change; reference counting lets unshared values be updated in place.
- `theorem` and `example` state propositions; `rfl` and `decide` prove them by computation, `simp` by rewriting, `omega` by linear arithmetic, `induction` by induction. Goals show hypotheses above `⊢` and the goal below it.
- Mathlib is a Lake dependency pinned in `lakefile.toml` and `lake-manifest.json`, downloaded prebuilt with `lake exe cache get`; import only the modules you use. It provides `ℕ`, `ℤ`, `ℚ`, `ℝ`, the tactics `ring`, `linarith`, `norm_num` and `positivity`, and lemmas with systematic names that `exact?` and `apply?` help you find.

2.15 Exercises

1. Write `def Tree.size : Tree → Nat` and `def Tree.height : Tree → Nat` for the binary search tree of this chapter, and check them with `#eval` on the tree built from `[5, 3, 9, 1, 4, 3]`. Then write the Rust versions for the `Tree` in `ch02_rust_counterparts.rs`. *Hint:* both are structural recursions with two equations; for `height`, use `max`. Work in a copy of `Basics.lean`, where `Tree` is defined.
2. Give the `Command` type a function `Command.isWrite : Command → Bool` that handles every case (unlike the rejected one), and a `ToString` instance that uses `Command.describe`. Then prove `example : (Command.set "k" 1).isWrite = true := rfl`. *Hint:* the case missing from the rejected version was `.ping`. An instance can be written `instance : ToString Command := ⟨Command.describe⟩`, since a class with one field is a structure with one field.
3. Write `placeDo`, a version of `Book.place` that uses `do`, `throw` and `return`, as `parseNat` does. Check that it agrees with `place` on a few inputs with examples such as `example : placeDo demoBook "a" .buy 20000 1 = demoBook.place "a" .buy 20000 1 := rfl`. *Hint:* `if let some best := book.crossing side price then throw (.wouldCross best)` is a valid line in a `do` block. `rfl` needs no `BEq` instance: it only asks whether both sides compute to the same value.
4. Prove that cancelling an order preserves the invariant: `theorem cancel_uncrossed (hinv : book.Uncrossed) (h : book.cancel id = .ok (order, book')) : book'.Uncrossed`
   Hint: start like `cancel_removes` (unfold, `split at h`, `contradiction`, `cases h`), then `intro b hb s hs hbuy hsell`. The hypotheses `hb` and `hs` are memberships in a filtered list; `simp only [List.mem_filter] at hb hs` turns each into a conjunction whose first half (`hb.1`) is membership in the old list, so `hinv` applies.
5. Use `exact?` to find the name of the lemma "`a ≤ b` and `b ≤ c` imply `a ≤ c`" for natural numbers. Then guess the names of the lemmas "`a < b` and `b < c` imply `a < c`" and "`a ≤ b` and `b < c` imply `a < c`", and check your guesses with `#check`. *Hint:* transitivity lemmas end in `_trans`; lemmas that mix relations name the conclusion first and the hypotheses after `_of_`, in order, like `Nat.lt_of_lt_of_le`.
6. In Rust, change `Book::place` so that it accepts crossing orders, and find a sequence of requests for which `spread` panics in a debug build. Then, in a copy of the Lean files, make the same change to `Book.place` and see which proof in `FirstProofs.lean` breaks, and where. *Hint:* a buy above the best ask makes the best bid larger than the best ask. The first proof to break is `place_ok`, whose statement mentions `crossing`; once you remove that part, `place_uncrossed` breaks in the two cases that compare the new order with an old one.

CHAPTER 3

Judgments, Rules and Derivations

Chapter 1 introduced the vocabulary of type theory informally: a judgment such as $\Gamma \vdash e : \tau$, inference rules with premises above a line, and derivations, trees of rule instances that justify a judgment. We wrote five rules for a fragment of Rust and built one small derivation by hand. That was enough to read a rule, but not enough to *trust* one. What exactly is a rule? What does it mean for a judgment to be derivable? When a type checker says "this program has type `Nat`", how do we know it is telling the truth?

This chapter answers those questions for a complete, if tiny, language called **Arith**: natural numbers and booleans, addition, equality, `if` and `let`. We build it three times. On paper, as a grammar, a set of typing rules and a set of evaluation rules. In Rust, as the `arith` crate: a parser, a type checker that produces derivation trees as data and prints them, and an evaluator. And in Lean, where the typing rules become an inductive definition, derivations become proof terms, and the type checker becomes a function about which we prove the first real theorem of the book: **the checker is sound**, every type it reports is backed by a derivation. Along the way we look at Rust's own expressions through the same lens, and at what type safety does and does not promise.

In this chapter you will learn:

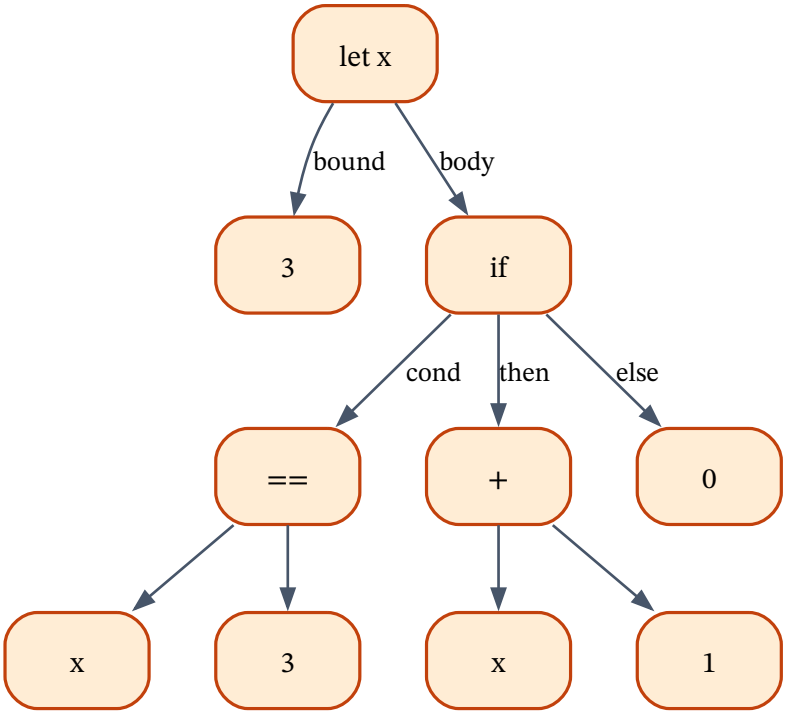
- the difference between concrete and abstract syntax, and how a BNF grammar defines syntax trees, which become enums in Rust and inductive types in Lean;
- what a judgment is, how inference rules define a relation inductively, and why a judgment holds exactly when it has a derivation;
- how to build derivations by hand, how a type checker searches for them, and how to represent them as data (Rust) and as proof terms (Lean);
- how Rust's blocks, `if`, unit type, never type and `Infallible` fit simple typing rules, and where the model needs care;
- what type safety means for Arith, what "going wrong" means, and why type systems reject some programs that would run fine;
- the difference between checking, synthesis and inference, and between structural and nominal types;
- how to prove, step by step in Lean, that the Arith checker is sound and complete, and that types are unique.

3.1 Syntax, concrete and abstract

Here is a program in Arith:

```
let x = 3 in if x == 3 then x + 1 else 0
```

That line of text is the program's **concrete syntax**: the characters a programmer types. A compiler does not work with characters for long. Its parser turns them into a tree, the program's **abstract syntax**, which records only the structure: which operation is applied to which sub-expressions.



The abstract syntax tree of let x = 3 in if x == 3 then x + 1 else 0. Each node is one construct of the language; its children are its sub-expressions.

Everything that the concrete syntax needs only to be readable and unambiguous is gone from the tree: spaces, parentheses, keywords, the precedence that says that `x + 1 == 2` means `(x + 1) == 2`. The tree for `(1 + 2) + 3` and the tree for `1 + 2 + 3` are the same tree. Type checkers, evaluators and proofs all work on abstract syntax, and so does the rest of this book.

Grammars

The set of Arith's trees is defined by a **grammar** in Backus–Naur form (BNF):

```
τ ::= Nat | Bool

e ::= n                          natural-number literal
    | true | false               boolean literals
    | x                          variable
    | e + e                      addition
    | e == e                     equality test
    | if e then e else e         conditional
    | let x = e in e             local definition
```

Read `::=` as "is one of" and `|` as "or". The first line says that a type τ is either `Nat` or `Bool`. The second says that an expression is a literal, a variable, or one of five compound forms built from smaller expressions. The letters n and x stand for any natural number and any variable name. This grammar describes *trees*: `e + e` means "a node labeled `+` with two children that are expressions", and the question of how to write such a tree as text, with which parentheses, is left to the concrete syntax.

A grammar is our first example of an **inductive definition**: the set of expressions is the *smallest* set that contains the literals and variables and is closed under the five ways of building bigger expressions. "Smallest" is what rules out infinite or circular trees: every expression is built in finitely many steps, which is what makes recursion and induction over expressions work. You will see the same pattern again in a moment, for judgments.

Arith in Rust

Each alternative of the grammar becomes a variant of an enum. The `arith` crate (in `code/rust/crates/arith`) defines them in `src/syntax.rs`:

```rust
/// The types of Arith: τ ::= Nat | Bool.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Ty {
    Nat,
    Bool,
}
```

```rust
/// The expressions of Arith, one variant per alternative of the grammar.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Expr {
    /// A natural-number literal `n`.
    Num(u64),
    /// A boolean literal, `true` or `false`.
    Bool(bool),
    /// A variable `x`.
    Var(String),
    /// `e1 + e2`
    Add(Box<Expr>, Box<Expr>),
    /// `e1 == e2`
    Eq(Box<Expr>, Box<Expr>),
    /// `if c then e1 else e2`
    If(Box<Expr>, Box<Expr>, Box<Expr>),
    /// `let x = e1 in e2`
    Let(String, Box<Expr>, Box<Expr>),
}
```

The sub-expressions are boxed because `Expr` is recursive. The crate also has a parser (`src/parse.rs`) that turns text into an `Expr`, with the usual recursive-descent structure: one function per precedence level, so that `+` binds tighter than `==` and both bind tighter than `if` and `let`. In the other direction, `Expr` implements `Display`, printing a tree with only the parentheses that are needed. A test in the crate prints thousands of random trees and parses them back, checking that the result is the same tree: concrete syntax is just a way of writing abstract syntax down.

Arith in Lean

In Lean, the grammar becomes two inductive types, in `code/lean/Book/Ch03/Arith.lean`:

```
/-- The types of Arith: `τ ::= Nat | Bool`. -/
inductive Ty where
  | nat
  | bool
  deriving Repr, DecidableEq
```

```
/-- The expressions of Arith, one constructor per alternative of the grammar.
`if` and `let` are Lean keywords, so the constructors are called `ite` and
`letE`. -/
inductive Expr where
  | num  (n : Nat)
  | bool (b : Bool)
  | var  (x : String)
  | add  (e₁ e₂ : Expr)
  | eq   (e₁ e₂ : Expr)
  | ite  (c e₁ e₂ : Expr)
  | letE (x : String) (e₁ e₂ : Expr)
  deriving Repr, DecidableEq
```

No `Box` is needed, as Chapter 2 explained. We do not write a parser in Lean; we write trees directly, using the constructors:

```
/-- `let x = 3 in if x == 3 then x + 1 else 0`, as a tree. -/
def sample : Expr :=
  .letE "x" (.num 3)
    (.ite (.eq (.var "x") (.num 3))
      (.add (.var "x") (.num 1))
      (.num 0))
```

The file also defines `Expr.render`, the same precedence-based printer as the Rust crate's, so that we can see a tree as text:

```
/-- info: "let x = 3 in if x == 3 then x + 1 else 0" -/
#guard_msgs in
#eval toString sample
```

3.2 Judgments

A **judgment** is a statement *about* syntax that we may or may not be able to establish. Arith has two kinds:

- the typing judgment $\Gamma \vdash e : \tau$, "in the context Γ, the expression e has type τ";
- the evaluation judgment $\rho \vdash e \Downarrow v$, "in the environment ρ, the expression e evaluates to the value v".

Other type theories have other judgment forms: "τ is a well-formed type", "e_1 and e_2 are equal", or, in logic, "A is true", which Chapter 4 studies in detail. The form of a judgment is a design decision, and a type theory is in large part a choice of judgment forms and of the rules that relate them.

A judgment is not part of the language it talks about. The Arith expression `x == 3` is a *term* of Arith: a program that computes a boolean when it runs. The judgment $x : \mathit{Nat} \vdash x == 3 : \mathit{Bool}$ is a statement in our *metalanguage*, the language in which we describe Arith, and it is about the term. It is not true or false at run time; it is derivable or not, once and for all. Keeping the two levels apart is the most important habit this chapter can give you. In Lean the separation is concrete: Arith expressions are

values of the Lean type `Expr`, and the typing judgment will be a Lean *proposition* about such values. Lean is our metalanguage, and Arith is *embedded* in it as data. (This style of formalization is called a *deep embedding*.)

Contexts

A context lists the types of the variables in scope:

```
Γ ::= ∅ | Γ, x : τ
```

A context is either empty or a smaller context extended with one more binding. We write $\Gamma(x) = \tau$ when the *most recent* binding of x in Γ has type τ. "Most recent" matters, because a `let` may bind a name that is already bound, and then the inner binding *shadows* the outer one, exactly as in Rust:

```
let x = true;
let x = 5;      // shadows the first x
x + 1           // this x is the number
```

The Rust crate represents a context as a vector of bindings, newest last, and searches it from the end:

```rust
/// A typing context Γ = x1 : τ1, ..., xn : τn. Bindings are kept in order,
/// most recent last, so that a new binding for `x` shadows older ones.
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct Ctx {
    bindings: Vec<(String, Ty)>,
}

impl Ctx {
    /// The empty context.
    pub fn new() -> Self {
        Ctx::default()
    }

    /// Γ, x : τ — a new context; `self` is left unchanged.
    pub fn extend(&self, x: &str, ty: Ty) -> Ctx {
        let mut bindings = self.bindings.clone();
        bindings.push((x.to_string(), ty));
        Ctx { bindings }
    }

    /// The type of the most recent binding of `x`, if there is one.
    pub fn lookup(&self, x: &str) -> Option<Ty> {
        self.bindings
            .iter()
            .rev()
            .find(|(name, _)| name == x)
            .map(|(_, ty)| *ty)
    }
}
```

The Lean version is a list of pairs, newest *first*, so that extending the context is just `::`:

```
/-- A typing context `Γ = x₁ : τ₁, ..., xₙ : τₙ`, newest binding first. -/
abbrev Ctx := List (String × Ty)
```

`abbrev` makes `Ctx` a transparent abbreviation: a `Ctx` *is* a list, and every list function applies to it. In particular `Γ.lookup x` is the core library's `List.lookup`, which returns the value paired with the *first* occurrence of the key, the most recent binding:

```
/-- info: some (Ch03.Ty.bool) -/
#guard_msgs in
#eval ([("x", .bool), ("x", .nat)] : Ctx).lookup "x"
```

3.3 Inference rules

The typing rules of Arith

Here are the typing rules, one for each form of expression:

$$\frac{}{\Gamma \vdash n : \mathit{Nat}}\ \text{T-Num} \qquad \frac{}{\Gamma \vdash b : \mathit{Bool}}\ \text{T-Bool} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \text{T-Var}$$

$$\frac{\Gamma \vdash e_1 : \mathit{Nat} \qquad \Gamma \vdash e_2 : \mathit{Nat}}{\Gamma \vdash e_1 + e_2 : \mathit{Nat}}\ \text{T-Add} \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 == e_2 : \mathit{Bool}}\ \text{T-Eq}$$

$$\frac{\Gamma \vdash e_1 : \mathit{Bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}\ \text{T-Let}$$

In T-Bool, b stands for either boolean literal. In words:

- **T-Num** and **T-Bool**: literals have their obvious types, in any context.
- **T-Var**: a variable has the type of its most recent binding.
- **T-Add**: a sum is a `Nat` if both operands are.
- **T-Eq**: an equality test is a `Bool` if both sides have the *same* type τ, whichever it is. So `1 == 2` and `true == false` are well typed, and `1 == true` is not.
- **T-If**: the condition must be a `Bool` and the branches must agree on a type τ, which is then the type of the whole expression (Rust's rule, from Chapter 1).
- **T-Let**: type e_1 first; then type e_2 in the context extended with $x : \tau_1$. The scope of x is e_2 and nothing else.

Anatomy of a rule

A rule is made of **premises** (above the line) and a **conclusion** (below it). A rule without premises, such as T-Num, is an **axiom**. The letters Γ, e_1, τ, n and x are **metavariables**: a rule is a *schema*, and every way of replacing its metavariables by actual contexts, expressions and types gives an *instance* of the rule. T-Add with $\Gamma := x : \mathit{Nat}$, $e_1 := x$ and $e_2 := 1$ is the instance

$$\frac{x : \mathit{Nat} \vdash x : \mathit{Nat} \qquad x : \mathit{Nat} \vdash 1 : \mathit{Nat}}{x : \mathit{Nat} \vdash x + 1 : \mathit{Nat}}\ \text{T-Add}$$

T-Var has a premise of a different kind, $\Gamma(x) = \tau$, which is not a typing judgment but a fact about the context that can be checked directly. Such premises are often called **side conditions**.

Notice something about T-Eq: the metavariable τ occurs in the premises but not in the conclusion. To use the rule backwards, from a goal $\Gamma \vdash e_1 == e_2 : \mathit{Bool}$, you must come up with a τ that works, and nothing in the goal tells you what it is. T-Let has the same feature with τ_1. We will see how a checker copes with this when we build one.

What the rules define

The rules do not *compute* anything. They define a **relation**: a set of triples (Γ, e, τ) for which the judgment $\Gamma \vdash e : \tau$ holds. Which set? Many sets are "closed under the rules", meaning that whenever the premises of a rule instance are in the set, so is the conclusion; the set of *all* triples is one of them, and it is useless. The rules define the **smallest** such set: a judgment holds only if the rules force it to. This is the same idea as in the grammar, and it has two consequences that we will use constantly.

1. **A judgment holds if and only if it has a derivation.** A derivation is the finite tree of rule instances that forces the judgment into the set. If there is no derivation, nothing forces it, and the smallest set leaves it out.
2. **Rule induction.** To prove that every derivable judgment has some property P, it is enough to show that each rule preserves P: if the premises have it, so does the conclusion. This is induction on the height of derivations, and it is how the theorems at the end of this chapter are proved.

A third consequence is specific to Arith. Each form of expression is the conclusion of exactly one rule: a sum can only be typed by T-Add, an `if` only by T-If. So if $\Gamma \vdash e_1 + e_2 : \tau$ is derivable, its derivation ends with T-Add, and therefore $\tau = \mathit{Nat}$ and both operands have type `Nat`. Reasoning backwards from a judgment to the premises that must have produced it is called **inversion**, and a system in which the syntax of the expression determines the rule is called **syntax-directed**.

3.4 Derivations

A derivation by hand

Let us derive the type of our sample program, $\texttt{let}\ x = 3\ \texttt{in}\ \texttt{if}\ x == 3\ \texttt{then}\ x + 1\ \texttt{else}\ 0$, in the empty context (which we do not write: $\vdash e : \tau$ means $\varnothing \vdash e : \tau$). The tree is too wide for one page, so we build it from named pieces. Let $\Gamma_1 = x : \mathit{Nat}$. The condition and the first branch:

$$\mathcal{D}_1 = \dfrac{\dfrac{\Gamma_1(x) = \mathit{Nat}}{\Gamma_1 \vdash x : \mathit{Nat}}\ \textsf{T-Var} \qquad \dfrac{}{\Gamma_1 \vdash 3 : \mathit{Nat}}\ \textsf{T-Num}}{\Gamma_1 \vdash x == 3 : \mathit{Bool}}\ \textsf{T-Eq}$$

$$\mathcal{D}_2 = \dfrac{\dfrac{\Gamma_1(x) = \mathit{Nat}}{\Gamma_1 \vdash x : \mathit{Nat}}\ \textsf{T-Var} \qquad \dfrac{}{\Gamma_1 \vdash 1 : \mathit{Nat}}\ \textsf{T-Num}}{\Gamma_1 \vdash x + 1 : \mathit{Nat}}\ \textsf{T-Add}$$

Then the `if`, writing e_{if} for the whole conditional:

$$\mathcal{D}_3 = \dfrac{\mathcal{D}_1 \qquad \mathcal{D}_2 \qquad \dfrac{}{\Gamma_1 \vdash 0 : \mathit{Nat}}\ \textsf{T-Num}}{\Gamma_1 \vdash e_{\text{if}} : \mathit{Nat}}\ \textsf{T-If}$$

and finally the `let`, whose second premise is typed in the extended context Γ_1:

$$\dfrac{\dfrac{}{\vdash 3 : \mathit{Nat}}\ \textsf{T-Num} \qquad \mathcal{D}_3}{\vdash \texttt{let}\ x = 3\ \texttt{in}\ e_{\text{if}} : \mathit{Nat}}\ \textsf{T-Let}$$

Check each step against the rules. In T-Eq the hidden τ is `Nat`; in T-Let the hidden τ_1 is `Nat`, and that choice is what puts $x : \mathit{Nat}$ into the context of $\mathcal{D}_3$. The whole tree has ten rule instances (counting the two T-Var nodes, whose side conditions are not rules) and height four.

Type checking is a search for derivations

How would a program find this tree? Read the rules from the bottom up. Given Γ and e, look at the form of e: since Arith is syntax-directed, exactly one rule can have produced it, and its premises say what to derive next. The hidden metavariables are not a problem if we *compute* types instead of guessing them: for T-Let, derive the type of e_1 first, and use the τ_1 we found to extend the context; for T-Eq, derive the types of both sides and compare them. A checker that computes the type from the expression is said to **synthesize** it, and for Arith synthesis always succeeds when a derivation exists.

The Rust checker, in `src/typing.rs`, returns the derivation itself. A `Derivation` is a tree whose nodes record the rule, the judgment it concludes and the sub-derivations of its premises:

```
/// The names of the typing rules.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Rule {
    Num,
    Bool,
    Var,
    Add,
    Eq,
    If,
    Let,
}

/// A judgment Γ ⊢ e : τ.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Judgment {
    pub ctx: Ctx,
    pub expr: Expr,
    pub ty: Ty,
}

/// A derivation: a tree of rule instances. The root's conclusion is the
/// judgment being derived; the children derive the rule's premises.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Derivation {
    pub rule: Rule,
    pub conclusion: Judgment,
    pub premises: Vec<Derivation>,
}
```

and `check` builds it by following the rules backwards, one arm per rule:

```
/// Search for a derivation of Γ ⊢ e : τ, for some τ. The search is guided by
/// the shape of `e`: for each kind of expression exactly one rule can apply,
/// so the checker never has to guess or backtrack.
pub fn check(ctx: &Ctx, e: &Expr) -> Result<Derivation, TypeError> {
    match e {
        Expr::Num(_) => Ok(leaf(Rule::Num, ctx, e, Ty::Nat)),
        Expr::Bool(_) => Ok(leaf(Rule::Bool, ctx, e, Ty::Bool)),
        Expr::Var(x) => match ctx.lookup(x) {
            Some(ty) => Ok(leaf(Rule::Var, ctx, e, ty)),
            None => Err(TypeError::Unbound { var: x.clone() }),
        },
        Expr::Add(a, b) => {
            let da = check(ctx, a)?;
            expect(Rule::Add, a, &da, Ty::Nat)?;
            let db = check(ctx, b)?;
            expect(Rule::Add, b, &db, Ty::Nat)?;
            Ok(node(Rule::Add, ctx, e, Ty::Nat, vec![da, db]))
        }
        Expr::Eq(a, b) => {
            let da = check(ctx, a)?;
            let db = check(ctx, b)?;
            agree(Rule::Eq, (a, &da), (b, &db))?;
```

```
            Ok(node(Rule::Eq, ctx, e, Ty::Bool, vec![da, db]))
        }
        Expr::If(c, a, b) => {
            let dc = check(ctx, c)?;
            expect(Rule::If, c, &dc, Ty::Bool)?;
            let da = check(ctx, a)?;
            let db = check(ctx, b)?;
            agree(Rule::If, (a, &da), (b, &db))?;
            let ty = da.ty();
            Ok(node(Rule::If, ctx, e, ty, vec![dc, da, db]))
        }
        Expr::Let(x, a, b) => {
            let da = check(ctx, a)?;
            // The body is checked in the extended context Γ, x : τ1.
            let db = check(&ctx.extend(x, da.ty()), b)?;
            let ty = db.ty();
            Ok(node(Rule::Let, ctx, e, ty, vec![da, db]))
        }
    }
}
```

Compare each arm with its rule. The `Add` arm derives both operands and insists (with the helper `expect`) that each has type `Nat`, then builds a T-Add node whose premises are the two sub-derivations. The `Eq` and `If` arms use `agree` to demand that two derived types coincide; this is how the hidden τ of T-Eq is found rather than guessed. The `Let` arm checks the body in `ctx.extend(x, ...)`, the context $\Gamma, x : \tau_1$.

Derivations as data

The example `crates/arith/examples/ch03_arith.rs` parses sample programs, checks them and prints the derivations with `Derivation::render`, which writes each judgment on its own line, the premises indented under their conclusion:

```
/// Type check a closed program, print its derivation, then run it.
fn show_derivation(src: &str) {
    let e = parse(src).expect("the samples are syntactically valid");
    let d = check(&Ctx::new(), &e).expect("this sample is well typed");
    assert!(d.verify(), "the checker only builds valid derivations");
    print!("{}", d.render());
    let v = eval(&Env::new(), &e).expect("well-typed programs do not go wrong");
    assert_eq!(v.ty(), d.ty());
    println!("({} rules, height {}; value {v})", d.size(), d.height());
    println!();
}
```

```
⊢ 1 + 2 : Nat   [T-Add]
  ⊢ 1 : Nat   [T-Num]
  ⊢ 2 : Nat   [T-Num]
(3 rules, height 2; value 3)

⊢ let x = 3 in if x == 3 then x + 1 else 0 : Nat   [T-Let]
  ⊢ 3 : Nat   [T-Num]
  x : Nat ⊢ if x == 3 then x + 1 else 0 : Nat   [T-If]
    x : Nat ⊢ x == 3 : Bool   [T-Eq]
      x : Nat ⊢ x : Nat   [T-Var]
      x : Nat ⊢ 3 : Nat   [T-Num]
    x : Nat ⊢ x + 1 : Nat   [T-Add]
      x : Nat ⊢ x : Nat   [T-Var]
      x : Nat ⊢ 1 : Nat   [T-Num]
    x : Nat ⊢ 0 : Nat   [T-Num]
(10 rules, height 4; value 4)
```

```
⊢ let x = true in let x = 5 in x + 1 : Nat   [T-Let]
  ⊢ true : Bool   [T-Bool]
  x : Bool ⊢ let x = 5 in x + 1 : Nat   [T-Let]
    x : Bool ⊢ 5 : Nat   [T-Num]
    x : Bool, x : Nat ⊢ x + 1 : Nat   [T-Add]
      x : Bool, x : Nat ⊢ x : Nat   [T-Var]
      x : Bool, x : Nat ⊢ 1 : Nat   [T-Num]
(7 rules, height 4; value 6)
```

The second tree is the derivation we built by hand, printed root first: the fraction notation draws trees with the root at the bottom, this printout with the root at the top, but the tree is the same. The third shows shadowing: in the context `x : Bool, x : Nat`, the variable `x` has type `Nat`, the most recent binding.

Having the derivation as data has a practical benefit: anyone can *check* it without trusting the checker that produced it. `Derivation::verify` walks the tree and confirms that every node is a correct instance of its rule:

```rust
/// Check that every node is a correct instance of its rule. This does not
/// search for anything: it only compares the judgments that are already
/// written in the tree, which is why checking a derivation is much easier
/// than finding one.
pub fn verify(&self) -> bool {
    let Judgment { ctx, expr, ty } = &self.conclusion;
    let p = &self.premises;
    let at = |i: usize, e: &Expr, t: Ty, c: &Ctx| {
        let j = &p[i].conclusion;
        j.expr == *e && j.ty == t && j.ctx == *c
    };
    let shape_ok = match (self.rule, expr) {
        (Rule::Num, Expr::Num(_)) => p.is_empty() && *ty == Ty::Nat,
        (Rule::Bool, Expr::Bool(_)) => p.is_empty() && *ty == Ty::Bool,
        (Rule::Var, Expr::Var(x)) => p.is_empty() && ctx.lookup(x) == Some(*ty),
        (Rule::Add, Expr::Add(a, b)) => {
            p.len() == 2
                && *ty == Ty::Nat
                && at(0, a, Ty::Nat, ctx)
                && at(1, b, Ty::Nat, ctx)
        }
        (Rule::Eq, Expr::Eq(a, b)) => {
            p.len() == 2
                && *ty == Ty::Bool
                && at(0, a, p[0].ty(), ctx)
                && at(1, b, p[0].ty(), ctx)
        }
        (Rule::If, Expr::If(c, a, b)) => {
            p.len() == 3
                && at(0, c, Ty::Bool, ctx)
                && at(1, a, *ty, ctx)
                && at(2, b, *ty, ctx)
        }
        (Rule::Let, Expr::Let(x, a, b)) => {
            p.len() == 2
                && at(0, a, p[0].ty(), ctx)
                && at(1, b, *ty, &ctx.extend(x, p[0].ty()))
        }
        _ => false,
    };
    shape_ok && p.iter().all(Derivation::verify)
}
```

`verify` searches for nothing. It only compares judgments that are already written in the tree, which makes it much simpler than `check`, and so much easier to get right. This division of labor, a clever

and possibly buggy program that *finds* a certificate and a simple program that *checks* it, is exactly how Lean itself is built: tactics and the elaborator search for proofs, and a small kernel checks the proof terms they produce. The crate's tests verify every derivation that `check` builds, and also confirm that tampered derivations (a wrong type, a wrong rule, a missing premise) are rejected.

When no derivation exists

When the search fails, the checker reports where it got stuck:

```
/// Why no derivation exists: the place where the search got stuck.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum TypeError {
    /// T-Var does not apply: the variable is not in the context.
    Unbound { var: String },
    /// A premise asks for one type and the sub-expression has another.
    Expected {
        rule: Rule,
        expr: Expr,
        expected: Ty,
        found: Ty,
    },
    /// A rule asks for two sub-expressions of the same type, and they differ.
    Disagree {
        rule: Rule,
        left: (Expr, Ty),
        right: (Expr, Ty),
    },
}
```

The example prints the error for a few ill-typed programs, and then runs them anyway with the evaluator, which we will meet shortly:

```
1 + true
  type error: T-Add: `true` should have type Nat, but has type Bool
  if run anyway: stuck: `1 + true` cannot use the value true
if 1 then 2 else 3
  type error: T-If: `1` should have type Bool, but has type Nat
  if run anyway: stuck: `if 1 then 2 else 3` cannot use the value 1
let x = 1 in y
  type error: T-Var: `y` is not in the context
  if run anyway: stuck: `y` has no value
if 1 == 1 then 1 else false
  type error: T-If: `1` : Nat and `false` : Bool should have the same type
  if run anyway: value 1
```

Each message names the rule that could not be applied and the premise that failed: for `1 + true`, rule T-Add needs `true : Nat`, and T-Bool can only give `true : Bool`. By inversion, T-Add is the *only* rule that could conclude a typing of `1 + true`, so no derivation exists at all, not just none that the checker found. The last program is more interesting: it is rejected, but it runs fine. We will come back to it.

3.5 The rules in Lean

The typing judgment as an inductive predicate

In Lean, the typing rules become one definition:

```
/-- The typing judgment `Γ ⊢ e : τ`. Each constructor is one inference rule:
its arguments are the premises, its type is the conclusion. -/
inductive HasType : Ctx → Expr → Ty → Prop where
```

```
  | num : HasType Γ (.num n) .nat
  | bool : HasType Γ (.bool b) .bool
  | var : Γ.lookup x = some τ →
      HasType Γ (.var x) τ
  | add : HasType Γ e₁ .nat → HasType Γ e₂ .nat →
      HasType Γ (.add e₁ e₂) .nat
  | eq : HasType Γ e₁ τ → HasType Γ e₂ τ →
      HasType Γ (.eq e₁ e₂) .bool
  | ite : HasType Γ c .bool → HasType Γ e₁ τ → HasType Γ e₂ τ →
      HasType Γ (.ite c e₁ e₂) τ
  | letE : HasType Γ e₁ τ₁ → HasType ((x, τ₁) :: Γ) e₂ τ₂ →
      HasType Γ (.letE x e₁ e₂) τ₂
```

`HasType` is an **inductive predicate**: an inductive type, like `Expr`, but one whose values are *proofs*. Its type, `Ctx → Expr → Ty → Prop`, says that for every context, expression and type, `HasType Γ e τ` is a proposition, the judgment $\Gamma \vdash e : \tau$. Each constructor is one rule. Its arguments are the premises and its result type is the conclusion: the constructor `add` says "from a proof of `HasType Γ e₁ .nat` and a proof of `HasType Γ e₂ .nat`, you get a proof of `HasType Γ (.add e₁ e₂) .nat`", which is rule T-Add. The side condition of T-Var becomes an ordinary premise, a proof that the lookup returns `τ`.

The metavariables of the rules are the names `Γ`, `e₁`, `τ` and so on in the constructors. We did not declare them: Lean's auto-bound implicits (Chapter 2) turn them into implicit arguments of each constructor, as `#check` shows:

```
/--
info: @HasType.letE : ∀ {Γ : Ctx} {e₁ : Expr} {τ₁ : Ty} {x : String} {e₂ : Expr} {τ₂ : Ty},
  HasType Γ e₁ τ₁ → HasType ((x, τ₁) :: Γ) e₂ τ₂ → HasType Γ (Expr.letE x e₁ e₂) τ₂
-/
#guard_msgs in
#check @HasType.letE
```

Read the `∀` as "for all values of the metavariables": this is the rule schema T-Let, stated as the type of a function from proofs of the premises to a proof of the conclusion.

Being an inductive definition, `HasType` is automatically the *smallest* relation closed under its constructors: a proof of `HasType Γ e τ` can only be built by applying constructors, finitely many times. Lean also generates the two principles that this smallness gives: `induction` on a proof of `HasType` (rule induction) and `cases` on it (inversion).

Derivations are proof terms

A derivation is now a Lean term built from the constructors. The derivation of $\vdash 1 + 2 : \mathit{Nat}$ is T-Add applied to two instances of T-Num:

```
/-- `⊢ 1 + 2 : Nat`: rule T-Add with two instances of T-Num above it. -/
example : HasType [] (.add (.num 1) (.num 2)) .nat :=
  HasType.add HasType.num HasType.num
```

and the derivation we built by hand for the sample program is this term, whose nesting is the shape of the tree:

```
/-- The derivation of `⊢ sample : Nat`, written out as a term. The shape of
the term is the shape of the derivation tree. -/
theorem sample_typed : HasType [] sample .nat :=
  HasType.letE HasType.num
    (HasType.ite
```

```
      (HasType.eq (HasType.var rfl) HasType.num)
      (HasType.add (HasType.var rfl) HasType.num)
      HasType.num)
```

Each `HasType.var rfl` is a T-Var node: its premise, that looking up `"x"` in `[("x", .nat)]` gives `some .nat`, holds by computation, so `rfl` proves it. The hidden metavariables of T-Let and T-Eq are found by Lean's unifier, much as our Rust checker found them, by matching the types of the sub-derivations.

Writing proof terms by hand is instructive once. The usual way is to build them with tactics, starting from the goal and working upwards, which is exactly the backwards reading of the rules:

```
/-- The same derivation, built backwards from the goal with tactics. -/
example : HasType [] sample .nat := by
  unfold sample
  apply HasType.letE
  · -- Goal: HasType [] (Expr.num 3) ?τ₁
    exact HasType.num
  · -- Goal: HasType [("x", Ty.nat)] (Expr.ite ...) Ty.nat
    apply HasType.ite
    · exact HasType.eq (HasType.var rfl) HasType.num
    · exact HasType.add (HasType.var rfl) HasType.num
    · exact HasType.num

/-- And with a loop: `constructor` applies whichever rule fits the goal. -/
example : HasType [] sample .nat := by
  repeat constructor
```

`unfold sample` replaces the name by its definition so that the goal shows the tree. `apply HasType.letE` says "the last rule is T-Let" and leaves one goal per premise. The first goal displays the unknown τ_1 as `?τ₁`, a *metavariable* in Lean's sense, a hole that later steps may fill: `exact HasType.num` fills it with `Ty.nat`, and from then on the second goal mentions `[("x", Ty.nat)]`. The second version lets the tactic `constructor` do the choosing. Given a goal of the form `HasType ...`, it tries the constructors and applies the first one whose conclusion fits. `repeat` applies it again to the first remaining goal, and again, until it fails; here it fails only when no goals are left. Along the way it also meets the `lookup` equations of T-Var, which `constructor` closes with `Eq.refl`. Because Arith is syntax-directed, the first constructor that fits is always the right one, so this naive search succeeds.

Here a Rust programmer may ask: where is the derivation *data*? In Rust we could print it and count its nodes. In Lean, `HasType` lives in `Prop`, and Lean treats any two proofs of the same proposition as equal (*proof irrelevance*), so a function that computes data cannot look inside a proof. Had we declared `HasType` with values in `Type` instead, derivations would be ordinary data that functions could inspect, like the Rust `Derivation`. Chapter 10 explains the difference between `Prop` and `Type`; for this chapter, what matters is that a judgment holds exactly when its proposition has a proof.

3.6 Rust's expressions as instances of rules

Chapter 1 used Rust's expressions to motivate the rules. With Arith in hand we can look at them more precisely, and see where Rust needs rules of a kind that Arith does not have. As

always, these are *models* of what the compiler does, not an official specification. The program is `code/rust/examples/ch03_expressions.rs`.

Blocks are nested `let`s

Everything in Rust is an expression, including blocks. A block with `let` statements and a final expression is Arith's `let` in different clothes:

```
/// A block with `let` statements is a nest of T-Let instances:
/// `{ let a = 2; let b = a + 3; a * b }` is `let a = 2 in let b = a + 3 in a * b`.
fn block_as_lets() -> u32 {
    let a = 2;
    let b = a + 3;
    a * b
}
```

The body is `let a = 2 in let b = a + 3 in a * b`, typed by two instances of T-Let. A statement that is not a `let`, such as `e1;`, adds a rule of its own, which we can call T-Seq: e_1 must have *some* type, which is then thrown away, and the block has the type of what follows.

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \{\, e_1 ; e_2 \,\} : \tau_2}\ \text{T-Seq}$$

Unit, and `if` without `else`

The unit type has one value, and one axiom:

$$\frac{}{\Gamma \vdash () : ()}\ \text{T-Unit}$$

A block that ends with a semicolon has no final expression, and is typed as if it ended with `()`. An `if` without `else` behaves as if it had `else { () }`, so rule T-If forces its `then` branch to have type `()` as well:

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : ()}{\Gamma \vdash \texttt{if}\ e_1\ \{e_2\} : ()}\ \text{T-IfUnit}$$

That is why such an `if` is only useful for its effects:

```
/// An `if` without `else` is an `if` whose missing branch is `()`. Rule T-If
/// then forces the `then` branch to have type `()` as well, which is why such
/// an `if` can only be used for its effects.
fn log_if_negative(n: i64, log: &mut Vec<String>) {
    if n < 0 {
        log.push(format!("{n} is negative"));
    }
}
```

and why using one for its value is an error, E0317, whose message spells out the rule:

```
fn bonus(points: u32) -> u32 {
    let extra = if points > 100 { 10 };
    points + extra
}
```

```
error[E0317]: `if` may be missing an `else` clause
 --> compile_fail/ch03_if_without_else.rs:7:17
  |
7 |     let extra = if points > 100 { 10 };
```

```
  |                 ^^^^^^^^^^^^^^^^^__^^
  |                 |                 |
  |                 |                 found here
  |                 expected integer, found `()`
  |
  = note: `if` expressions without `else` evaluate to `()`
  = help: consider adding an `else` block that evaluates to the expected type
```

The never type: a rule that is not syntax-directed

Some expressions never produce a value: `return`, `break`, `continue`, `panic!` and a `loop` without `break`. Rust gives them the never type `!`, and Chapter 1 explained that `!` coerces to every type. As a rule:

$$\frac{\Gamma \vdash e : !}{\Gamma \vdash e : \tau}\ \text{T-Never}$$

This rule is different from all of Arith's. Its conclusion is not tied to any form of expression: *any* expression of type `!` can be given *any* type. A system with such a rule is no longer syntax-directed, and types are no longer unique (an expression of type `!` has every type). The compiler copes by applying the coercion only where a specific type is expected. Here are expressions of type `!` in four positions:

```rust
/// Expressions of type `!` in four positions. Each of them stands in for a
/// value of whatever type the context expects.
fn first_even(xs: &[u32]) -> u32 {
    let mut i = 0;
    let found: u32 = loop {
        if i == xs.len() {
            return 0; // `return 0` has type `!`, coerced to `u32`
        }
        if xs[i] % 2 == 1 {
            i += 1;
            continue; // `continue` has type `!` too
        }
        break xs[i]; // the loop's type comes from `break`
    };
    let checked: u32 = if found < 1000 {
        found
    } else {
        panic!("too big")
    };
    checked
}
```

`return 0` and `continue` are typed `!` and coerced to whatever their context expects. The `loop` itself has the type of the values given to `break`, here `u32`. In the last `let`, the `else` branch is a `panic!`, coerced to `u32` so that T-If can apply.

`Infallible` and the empty `match`

The never type is only partly stable, as Chapter 1 noted, and the standard library offers `std::convert::Infallible`, an enum with no variants, as a stable type with no values. It is empty just like `!`, but it is an ordinary enum, and the coercion rule of `!` does not apply to it:

```rust
fn absurd(x: Infallible) -> u32 {
    x
}
```

```
error[E0308]: mismatched types
 --> compile_fail/ch03_infallible.rs:9:5
  |
8 | fn absurd(x: Infallible) -> u32 {
  |                             --- expected `u32` because of return type
9 |     x
  |     ^ expected `u32`, found `Infallible`
```

To use an empty enum as a value of another type you must write its *elimination* explicitly, as a `match` with no arms. A `match` needs one arm per variant, and there are none, so the empty `match` is exhaustive and, having no arms that could disagree, can be given any type:

$$\frac{\Gamma \vdash e : \texttt{Infallible}}{\Gamma \vdash \texttt{match}\ e\ \{\} : \tau}\ \text{T-Absurd}$$

```rust
/// `Infallible` is an enum with no variants: a type with no values. A function
/// that receives one can never be called, and its body proves that by
/// matching on the argument with no arms at all.
fn absurd<T>(x: Infallible) -> T {
    match x {}
}

/// A parser that cannot fail, written against an interface that allows
/// failure. With the error type `Infallible`, the pattern `Ok(n)` covers every
/// possible value, so the `let` needs no `else`.
fn parse_len(s: &str) -> Result<usize, Infallible> {
    Ok(s.len())
}

fn use_parse_len() -> usize {
    let Ok(n) = parse_len("hello");
    n
}
```

T-Absurd, unlike T-Never, is syntax-directed: the expression `match e {}` says which rule to use. The second half of the listing shows a more common use. A function with error type `Infallible` cannot fail, so the pattern `Ok(n)` covers every possible value of `Result<usize, Infallible>`, and the compiler accepts `let Ok(n) = ...;` without an `else`. In logic, the empty type is falsehood, and T-Absurd is the principle "from a contradiction, anything follows"; Chapter 5 builds negation on it.

3.7 Evaluation and type safety

The evaluation judgment

Types are only half of a language. The other half is what programs do. Arith's **values** are the results of evaluation:

```
v ::= n | true | false
```

and its evaluation judgment $\rho \vdash e \Downarrow v$ is defined by rules too, where an environment ρ maps variables to values. Here are the main ones (the rule for `==` compares two numbers or two booleans, and E-IfFalse is the mirror image of E-IfTrue):

$$\frac{}{\rho \vdash n \Downarrow n}\ \text{E-Num} \qquad \frac{\rho(x) = v}{\rho \vdash x \Downarrow v}\ \text{E-Var} \qquad \frac{\rho \vdash e_1 \Downarrow n_1 \qquad \rho \vdash e_2 \Downarrow n_2}{\rho \vdash e_1 + e_2 \Downarrow n_1 + n_2}\ \text{E-Add}$$

$$\frac{\rho \vdash e_1 \Downarrow \texttt{true} \qquad \rho \vdash e_2 \Downarrow v}{\rho \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Downarrow v}\ \text{E-IfTrue} \qquad \frac{\rho \vdash e_1 \Downarrow v_1 \qquad \rho, x \mapsto v_1 \vdash e_2 \Downarrow v_2}{\rho \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \Downarrow v_2}\ \text{E-Let}$$

Look closely at E-Add. The `+` in $e_1 + e_2$, in the expression, is Arith syntax: a node of the tree. The $+$ in $n_1 + n_2$, in the result, is ordinary addition of numbers, in the metalanguage. The rule says that the meaning of the syntax `+` is mathematical addition. It is the clearest example of the two levels of this chapter sitting side by side. Notice also that E-IfTrue does not mention e_3: the branch that is not taken is not evaluated.

This style of semantics, which relates an expression directly to its final value, is called **big-step**. Chapter 9 uses the other style, *small-step*, which describes one step of computation at a time. Each has its uses, and the difference matters for languages whose programs can run forever.

The evaluator

The Rust evaluator implements the rules as a recursive function, in `src/eval.rs`:

```
/// Evaluate `e` in the environment `env`.
pub fn eval(env: &Env, e: &Expr) -> Result<Value, EvalError> {
    match e {
        Expr::Num(n) => Ok(Value::Num(*n)),
        Expr::Bool(b) => Ok(Value::Bool(*b)),
        Expr::Var(x) => env.lookup(x).ok_or_else(|| EvalError::Unbound(x.clone())),
        Expr::Add(a, b) => {
            let n = as_num(e, eval(env, a)?)?;
            let m = as_num(e, eval(env, b)?)?;
            n.checked_add(m)
                .map(Value::Num)
                .ok_or_else(|| EvalError::Overflow { expr: e.clone() })
        }
        Expr::Eq(a, b) => match (eval(env, a)?, eval(env, b)?) {
            (Value::Num(n), Value::Num(m)) => Ok(Value::Bool(n == m)),
            (Value::Bool(p), Value::Bool(q)) => Ok(Value::Bool(p == q)),
            // Comparing a number with a boolean has no meaning in Arith.
            (_, v) => Err(EvalError::Stuck {
                expr: e.clone(),
                value: v,
            }),
        },
        Expr::If(c, a, b) => match eval(env, c)? {
            // Only the branch that is taken is evaluated.
            Value::Bool(true) => eval(env, a),
            Value::Bool(false) => eval(env, b),
            v => Err(EvalError::Stuck {
                expr: e.clone(),
                value: v,
            }),
        },
        Expr::Let(x, a, b) => {
            let v = eval(env, a)?;
            eval(&env.extend(x, v), b)
        }
    }
}
```

It never looks at types. When an operation meets a value of the wrong kind, such as adding a boolean, no rule applies: the evaluation is **stuck**. The evaluator reports that as an error:

```
/// The ways a run can fail.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum EvalError {
    /// A variable with no value: the run is stuck.
```

```
    Unbound(String),
    /// An operation met a value of the wrong kind: the run is stuck.
    Stuck { expr: Expr, value: Value },
    /// A sum does not fit in a `u64`. This is not a stuck state of the
    /// semantics but a limit of the machine, and types do not prevent it.
    Overflow { expr: Expr },
}
```

The Lean evaluator, in `Arith.lean`, is the same function, with `none` for a stuck evaluation:

```
/-- The evaluator: `some v` if `e` evaluates to `v`, `none` if the run gets
stuck. It never looks at types. -/
def eval (ρ : Env) : Expr → Option Value
  | .num n => some (.num n)
  | .bool b => some (.bool b)
  | .var x => ρ.lookup x
  | .add e₁ e₂ =>
    match eval ρ e₁, eval ρ e₂ with
    | some (.num n), some (.num m) => some (.num (n + m))
    | _, _ => none
  | .eq e₁ e₂ =>
    match eval ρ e₁, eval ρ e₂ with
    | some (.num n), some (.num m) => some (.bool (n == m))
    | some (.bool p), some (.bool q) => some (.bool (p == q))
    | _, _ => none
  | .ite c e₁ e₂ =>
    match eval ρ c with
    | some (.bool true) => eval ρ e₁
    | some (.bool false) => eval ρ e₂
    | _ => none
  | .letE x e₁ e₂ =>
    match eval ρ e₁ with
    | some v => eval ((x, v) :: ρ) e₂
    | none => none
```

```
/-- info: some (Ch03.Value.num 4) -/
#guard_msgs in
#eval eval [] sample

/-- info: none -/
#guard_msgs in
#eval eval [] (.add (.num 1) (.bool true))

/-- info: some (Ch03.Value.num 1) -/
#guard_msgs in
#eval eval [] (.ite (.bool true) (.num 1) (.bool false))
```

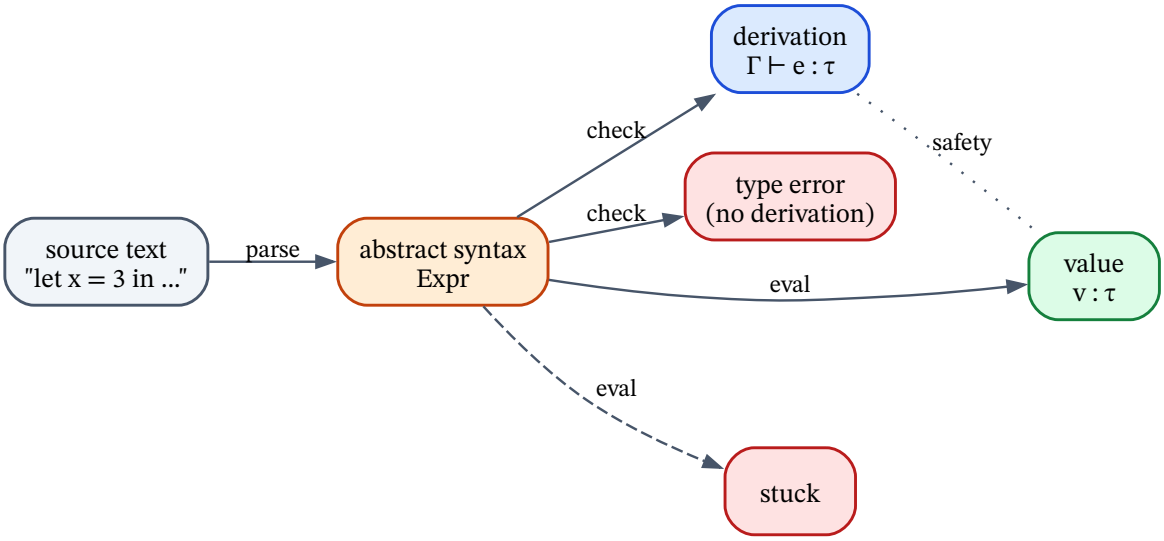
Type safety, informally

The output of the rejected programs above showed the connection between the two halves. `1 + true` is ill typed, and when we run it anyway it gets stuck. That is what the type system is for. For Arith, **type safety** says:

> If $\vdash e : \tau$, then $\vdash e \Downarrow v$ for some value v of type τ.

A well-typed closed program never gets stuck, and its result has the predicted type. This is Milner's slogan from Chapter 1, *well-typed programs cannot go wrong*, made precise for one language, and the precision lies in defining what "wrong" means: here, reaching a state where no evaluation rule applies. For a language whose programs may run forever, the statement is split into the two lemmas you met in Chapter 1, *progress* (a well-typed program is a value or can take a step) and *preservation* (a step

keeps the type), and Chapter 9 proves them completely for a richer language. Arith programs always terminate, so the big-step statement suffices.



The pieces of this chapter. The parser turns text into a tree; the checker either finds a derivation or reports a type error; the evaluator produces a value or gets stuck. Type safety (dotted) says that a derivation guarantees a value of the predicted type.

The crate *tests* the statement. On twenty thousand random expressions, it checks that every one the checker accepts verifies, evaluates without getting stuck, and produces a value of the predicted type:

```
#[test]
fn well_typed_programs_do_not_go_wrong() {
    let mut rng = Rng(2024);
    let mut well_typed = 0;
    for _ in 0..20_000 {
        let e = random_expr(&mut rng, 5);
        if let Ok(d) = check(&Ctx::new(), &e) {
            well_typed += 1;
            assert!(d.verify(), "invalid derivation for {e}");
            match eval(&Env::new(), &e) {
                Ok(v) => assert_eq!(v.ty(), d.ty(), "wrong type of value for {e}"),
                Err(err) => panic!("well-typed {e} went wrong: {err}"),
            }
        }
    }
    // Make sure the test is not vacuous.
    assert!(well_typed > 2_000, "only {well_typed} well-typed samples");
}
```

Two observations keep this honest. First, "wrong" means *stuck in the semantics*, and nothing else. Arith's numbers are unbounded, but the Rust implementation stores them in a `u64`, and a well-typed sum can overflow:

```
18446744073709551615 + 1
  type: Nat
  run: overflow in `18446744073709551615 + 1`
```

The type system does not, and cannot, prevent this: overflow is not a stuck state of the semantics but a limit of the machine that runs it. Rust is in the same position. Its type system rules out undefined

behavior, its notion of going wrong, and does not rule out panics, overflow (which panics in debug builds), deadlocks or infinite loops.

Second, type systems are **conservative**. `if 1 == 1 then 1 else false` is rejected, because T-If demands that both branches have the same type, yet it evaluates to `1` without trouble, since the `else` branch is never taken. The type checker does not run the program; it reasons about all runs at once, and here it cannot tell that the condition is always true. Every useful type system rejects some programs that would have been fine. For realistic languages this is unavoidable: whether a program in a Turing-complete language ever goes wrong is undecidable, so a checker that always terminates must err on one side, and type checkers err on the side of rejecting.

3.8 Checking, synthesis and inference

Our checker takes Γ and e and *produces* τ. Three related problems are worth telling apart, because later chapters solve each of them:

- **Type checking** proper: given Γ, e and τ, decide whether $\Gamma \vdash e : \tau$ holds.
- **Type synthesis**: given Γ and e, find a τ such that $\Gamma \vdash e : \tau$, or report that there is none. Our checker does this. For Arith, synthesis also solves checking: synthesize, then compare, which is correct because types are unique (proved below).
- **Type inference**: the program leaves out type information that the rules need, and the checker must reconstruct it. Arith never needs annotations, because the only binder, `let`, gets the type of its variable from e_1. A function `fun x => x + 1` is different: nothing in the term says what `x` is, and the checker must introduce an unknown type and solve equations about it. That is *unification*, and Chapter 13 builds the classic algorithm.

Real languages mix the three. Rust checks function bodies against their signatures, synthesizes the types of most expressions, and infers the types of local variables (as in `let v = Vec::new();`, whose element type is found from later uses), but only inside one function body: the signatures of functions are never inferred. Chapter 8 introduces *bidirectional* type checking, which alternates between checking and synthesis in a principled way, and Chapter 13 covers inference and Rust's place in it.

3.9 Structural and nominal types

When are two types the same? Arith's answer is trivial, `Nat` is `Nat`, but real languages need a policy. A type is **structural** if its identity is its structure: two tuple types `(f64, f64)` are the same type wherever they are written, and so are two array, reference or function-pointer types with the same components. A type alias does not even create a type, only a new name for an existing one. A type is **nominal** if its identity is its declaration: two structs with the same fields are different types, because they were declared separately.

```
/// Two structs with the same fields are different types (nominal typing).
struct Meters(f64);
struct Seconds(f64);
```

```
/// A type alias is just another name for the same type (structural, or
/// rather, no new type at all).
type Pair = (f64, f64);

fn speed(distance: Meters, time: Seconds) -> f64 {
    distance.0 / time.0
}

fn swap_pair(p: Pair) -> (f64, f64) {
    (p.1, p.0)
}
```

Nominal typing is what makes the *newtype* pattern useful. A distance and a duration are both `f64`, but `Meters` and `Seconds` are not interchangeable, and swapping them is a type error:

```
struct Meters(f64);
struct Seconds(f64);

fn speed(distance: Meters, time: Seconds) -> f64 {
    distance.0 / time.0
}

fn oops() -> f64 {
    speed(Seconds(8.0), Meters(100.0))
}
```

```
error[E0308]: arguments to this function are incorrect
  --> compile_fail/ch03_nominal.rs:13:5
   |
13 |     speed(Seconds(8.0), Meters(100.0))
   |     ^^^^^ ------------  -------------- expected `Seconds`, found `Meters`
   |           |
   |           expected `Meters`, found `Seconds`
   |
note: function defined here
  --> compile_fail/ch03_nominal.rs:8:4
   |
 8 | fn speed(distance: Meters, time: Seconds) -> f64 {
   |    ^^^^^
help: swap these arguments
   |
13 -     speed(Seconds(8.0), Meters(100.0))
13 +     speed(Meters(100.0), Seconds(8.0))
   |
```

The compiler even notices that swapping the arguments would fix the call. Lean behaves the same way: every `structure` and `inductive` declaration creates a new type, while `abbrev` (as for `Ctx`) only introduces a name. In type theory, structural types such as products are the primitive ones; when Chapter 11 writes $A \times B$, the honest Rust counterpart is the tuple `(A, B)`, and a `struct Pair(A, B)` is a product only up to an isomorphism.

3.10 Putting it together: the checker is correct

We now have two descriptions of Arith's type system: the rules, which say which judgments hold, and a checker, which computes types. Are they the same? A bug in the checker could make it accept a program that has no derivation, and then the type safety guarantee, which is about derivable judgments, would say nothing about that program. In Rust, the crate's tests give us evidence. In Lean we can settle the question.

The checker in Lean

The Lean checker has the same structure as the Rust one, returning `Option Ty` instead of a derivation:

```
/-- The type checker: synthesize the type of `e` in `Γ`, or fail with `none`.
For each kind of expression only one rule can apply, so the checker never
guesses. -/
def check (Γ : Ctx) : Expr → Option Ty
  | .num _ => some .nat
  | .bool _ => some .bool
  | .var x => Γ.lookup x
  | .add e₁ e₂ =>
    match check Γ e₁, check Γ e₂ with
    | some .nat, some .nat => some .nat
    | _, _ => none
  | .eq e₁ e₂ =>
    match check Γ e₁, check Γ e₂ with
    | some τ₁, some τ₂ => if τ₁ = τ₂ then some .bool else none
    | _, _ => none
  | .ite c e₁ e₂ =>
    match check Γ c, check Γ e₁, check Γ e₂ with
    | some .bool, some τ₁, some τ₂ => if τ₁ = τ₂ then some τ₁ else none
    | _, _, _ => none
  | .letE x e₁ e₂ =>
    match check Γ e₁ with
    | some τ₁ => check ((x, τ₁) :: Γ) e₂
    | none => none
```

```
/-- info: some (Ch03.Ty.nat) -/
#guard_msgs in
#eval check [] sample

/-- info: none -/
#guard_msgs in
#eval check [] (.add (.num 1) (.bool true))

/-- info: some (Ch03.Ty.bool) -/
#guard_msgs in
#eval check [("b", .bool)] (.ite (.var "b") (.var "b") (.eq (.num 1) (.num 2)))
```

The `match` on two (or three) results at once is the Lean counterpart of the Rust checker's sequence of `?` and `expect`: for a sum, only the case where both operands have type `Nat` succeeds, and any other combination falls through to `none`.

Soundness

Soundness says that the checker never claims a type without a derivation to back it. The theorem and its proof are in `code/lean/Book/Ch03/Soundness.lean`:

```
/-- **Soundness.** If the checker says `τ`, there is a derivation of
`Γ ⊢ e : τ`. -/
theorem check_sound {Γ : Ctx} {e : Expr} {τ : Ty}
    (h : check Γ e = some τ) : HasType Γ e τ := by
  induction e generalizing Γ τ with
  | num n =>
    -- h : check Γ (Expr.num n) = some τ, which computes to some Ty.nat = some τ
    simp only [check, Option.some.injEq] at h
    -- h : Ty.nat = τ
    subst h
    exact HasType.num
  | bool b =>
    simp only [check, Option.some.injEq] at h
    subst h
    exact HasType.bool
  | var x =>
```

```
    -- h : Γ.lookup x = some τ, exactly the premise of rule T-Var
    exact HasType.var h
  | add e₁ e₂ ih₁ ih₂ =>
    simp only [check] at h
    split at h
    · -- The checker found Nat for both operands.
      rename_i h₁ h₂
      -- h₁ : check Γ e₁ = some Ty.nat, h₂ : check Γ e₂ = some Ty.nat
      -- h  : some Ty.nat = some τ
      cases h
      exact HasType.add (ih₁ h₁) (ih₂ h₂)
    · -- The checker returned none, but h says it returned some τ.
      contradiction
  | eq e₁ e₂ ih₁ ih₂ =>
    simp only [check] at h
    split at h
    · rename_i τ₁ τ₂ h₁ h₂
      split at h
      · -- The two operands have the same type: τ₁ = τ₂.
        rename_i same
        cases h
        subst same
        exact HasType.eq (ih₁ h₁) (ih₂ h₂)
      · contradiction
    · contradiction
  | ite c e₁ e₂ ihc ih₁ ih₂ =>
    simp only [check] at h
    split at h
    · rename_i τ₁ τ₂ hc h₁ h₂
      split at h
      · rename_i same
        cases h
        subst same
        exact HasType.ite (ihc hc) (ih₁ h₁) (ih₂ h₂)
      · contradiction
    · contradiction
  | letE x e₁ e₂ ih₁ ih₂ =>
    simp only [check] at h
    split at h
    · -- The checker found τ₁ for e₁ and then checked e₂ in the extended context.
      rename_i τ₁ h₁
      exact HasType.letE (ih₁ h₁) (ih₂ h)
    · contradiction
```

The proof is by induction on the expression `e`, one case per constructor, and each case mirrors one arm of the checker. Let us go through it.

The induction. `induction e generalizing Γ τ with` sets up a structural induction on `e`. In each case for a compound expression, Lean provides an induction hypothesis for every sub-expression: in the `add` case, `ih₁` says that the theorem holds for `e₁`, and `ih₂` for `e₂`. The word `generalizing` matters. Without it, `ih₁` would speak about the *same* `Γ` and `τ` as the goal, but the sub-expressions have *other* types (the operands of a sum are `Nat` even when we know nothing yet about `τ`), and the body of a `let` lives in a *bigger* context. With `generalizing Γ τ`, each induction hypothesis reads "for every `Γ` and `τ`, if the checker says `τ` for this sub-expression in `Γ`, there is a derivation", which is strong enough to use at whatever context and type the case needs.

Literals. In the `num` case the hypothesis is `h : check Γ (Expr.num n) = some τ`. `simp only [check, Option.some.injEq] at h` unfolds `check`, which turns `h` into `some Ty.nat = some τ`, and then uses the injectivity of `some` (the lemma `Option.some.injEq`) to get `h : Ty.nat = τ`. `subst h`

replaces `τ` by `Ty.nat` everywhere, leaving the goal `HasType Γ (Expr.num n) Ty.nat`, which is rule T-Num: `exact HasType.num`. The `bool` case is identical.

Variables. `check Γ (.var x)` is *defined* as `Γ.lookup x`, so the hypothesis `h : check Γ (Expr.var x) = some τ` already is, up to unfolding, the premise of T-Var, and `exact HasType.var h` is accepted. (`exact` checks the term against the goal up to computation, so no explicit unfolding is needed.)

Sums. This is the case that shows the method. After unfolding, `h` says that a `match` on `check Γ e₁, check Γ e₂` returned `some τ`. `split at h` splits the proof into one case per arm of that `match`, adding to each case what Lean learned about the discriminants:

- In the first arm, both checks returned `some Ty.nat`. Lean records this in two hypotheses with names it invented (shown as `heq✝¹` and `heq✝` in the infoview), and `rename_i h₁ h₂` names them. Now `h : some Ty.nat = some τ`, and `cases h` solves this equation by substituting `Ty.nat` for `τ`. The goal is `HasType Γ (e₁.add e₂) Ty.nat`. Rule T-Add needs derivations for both operands, and the induction hypotheses provide them: `ih₁ h₁` turns "the checker says `Nat` for `e₁`" into "`e₁` has type `Nat`". So `exact HasType.add (ih₁ h₁) (ih₂ h₂)` closes the case: we have *built* the derivation from the derivations of the parts.
- In the other arm the `match` returned `none`, so `h : none = some τ`, which is impossible (different constructors), and `contradiction` closes the case.

Equality and conditionals. The same pattern, with one more `split` for the `if τ₁ = τ₂` inside the arm. In the branch where the two types are equal, `rename_i same` names the hypothesis `same : τ₁ = τ₂`, `cases h` fixes the type of the result, and `subst same` identifies `τ₁` and `τ₂`, so that the induction hypotheses give derivations at the *same* type, which is what T-Eq and T-If demand. In the branch where the types differ, the checker returned `none`, and `contradiction` closes the case.

`let`. After the split, `h₁ : check Γ e₁ = some τ₁` and `h : check ((x, τ₁) :: Γ) e₂ = some τ`. The first gives a derivation of `HasType Γ e₁ τ₁` by `ih₁ h₁`. The second is about the body *in the extended context*, and here `generalizing` pays off: `ih₂` holds for every context, including `(x, τ₁) :: Γ`, so `ih₂ h` is a derivation of the body. Rule T-Let combines them.

Every case built a derivation from the derivations of the sub-expressions, following the same rule the checker followed. That is what soundness means: the checker's recursion and the derivation's structure are the same tree.

Completeness

The converse says that the checker finds every type that can be derived:

```
/-- **Completeness.** If there is a derivation, the checker finds its type. -/
theorem check_complete {Γ : Ctx} {e : Expr} {τ : Ty}
    (h : HasType Γ e τ) : check Γ e = some τ := by
  induction h with
  | num => rfl
  | bool => rfl
  | var hx => exact hx
  | add _ _ ih₁ ih₂ => simp [check, ih₁, ih₂]
  | eq _ _ ih₁ ih₂ => simp [check, ih₁, ih₂]
```

```
  | ite _ _ _ ihc ih₁ ih₂ => simp [check, ihc, ih₁, ih₂]
  | letE _ _ ih₁ ih₂ => simp [check, ih₁, ih₂]
```

This time the induction is on the *derivation* `h`: rule induction, with one case per rule rather than per form of expression. In each case the goal is to show that the checker returns the type in the rule's conclusion, and the induction hypotheses say that it returns the right types for the premises. For T-Num and T-Bool the checker's answer is `some .nat` or `some .bool` by definition, so `rfl` works. For T-Var the premise *is* the claim. For the compound rules, `simp [check, ih₁, ih₂]` unfolds `check`, rewrites the recursive calls with the induction hypotheses (for example `check Γ e₁` becomes `some Ty.nat`), and then the `match` reduces to the right arm and the `if τ = τ` to its `then` branch.

Together the two theorems say that `check Γ e = some τ` and `HasType Γ e τ` are the same statement. That makes the typing judgment *decidable*, and Lean can be told so with an instance of the `Decidable` class, built from the checker:

```
/-- The checker is a decision procedure for the typing judgment. -/
theorem check_iff {Γ : Ctx} {e : Expr} {τ : Ty} :
    check Γ e = some τ ↔ HasType Γ e τ :=
  ⟨check_sound, check_complete⟩

instance (Γ : Ctx) (e : Expr) (τ : Ty) : Decidable (HasType Γ e τ) :=
  decidable_of_iff (check Γ e = some τ) check_iff

example : HasType [] sample .nat := by decide

example : ¬ HasType [] (.add (.num 1) (.bool true)) .nat := by decide
```

`decidable_of_iff` turns a decision procedure for one proposition into one for an equivalent proposition. With the instance in place, `decide` proves typing judgments, and their negations, by running the checker inside Lean, with the soundness and completeness theorems guaranteeing that the answer is right.

Uniqueness of types

An Arith expression has at most one type in a given context. We prove it twice. The first proof works directly on derivations:

```
/-- **Uniqueness of types.** An expression has at most one type in a given
context. Proved by induction on the first derivation and inversion of the
second. -/
theorem type_unique {Γ : Ctx} {e : Expr} {σ τ : Ty}
    (h₁ : HasType Γ e σ) (h₂ : HasType Γ e τ) : σ = τ := by
  induction h₁ generalizing τ with
  | num =>
    -- h₂ : HasType Γ (Expr.num n) τ. Only T-Num concludes that, and it forces
    -- τ = Ty.nat.
    cases h₂
    rfl
  | bool =>
    cases h₂
    rfl
  | var hσ =>
    -- hσ : Γ.lookup x = some σ; inverting h₂ gives hτ : Γ.lookup x = some τ.
    cases h₂ with
    | var hτ =>
      rw [hσ] at hτ
      -- hτ : some σ = some τ
      exact Option.some.inj hτ
```

```
  | add _ _ _ _ =>
    cases h₂
    rfl
  | eq _ _ _ _ =>
    cases h₂
    rfl
  | ite _ _ _ _ ih₁ _ =>
    -- The type of an `if` is the type of its first branch.
    cases h₂ with
    | ite _ h₂₁ _ => exact ih₁ h₂₁
  | letE _ _ ih₁ ih₂ =>
    cases h₂ with
    | letE h₂₁ h₂₂ =>
      -- Both derivations type e₁ first; by induction they agree on its type,
      have same := ih₁ h₂₁
      -- so they type e₂ in the same extended context, and agree again.
      subst same
      exact ih₂ h₂₂
```

Two derivations of the same expression are given, `h₁` with type `σ` and `h₂` with type `τ`. The proof is by induction on `h₁`, generalizing `τ` so that the induction hypotheses work for any second derivation. In each case, `cases h₂` performs **inversion** on the second derivation: it asks Lean which rules could have concluded `HasType Γ e τ` for this particular `e`. Since Arith is syntax-directed, only one rule fits, and Lean replaces `h₂` by the premises of that rule, solving any equations this imposes on `τ`.

- For a literal, the only rule is T-Num (or T-Bool), whose conclusion forces `τ = Ty.nat`. After `cases h₂`, the goal is `Ty.nat = Ty.nat`, and `rfl` closes it. The cases for `+` and `==` are the same, since their rules fix the result type.
- For a variable, both derivations end with T-Var, with premises `hσ : Γ.lookup x = some σ` and `hτ : Γ.lookup x = some τ`. Rewriting `hτ` with `hσ` gives `some σ = some τ`, and `Option.some.inj` (injectivity of `some`) gives `σ = τ`. Here, at the leaves of the tree, is where uniqueness really comes from: a context assigns each variable one type.
- For an `if`, the type of the whole is the type of the first branch in both derivations. Inverting `h₂` gives its derivation `h₂₁` of the first branch at type `τ`, and the induction hypothesis `ih₁`, for the first branch of `h₁`, turns it into `σ = τ`.
- For a `let`, the two derivations may *a priori* have typed `e₁` differently, and then typed the body in different contexts. The induction hypothesis for `e₁` rules that out: `have same := ih₁ h₂₁` proves that the two types of `e₁` are equal, `subst same` makes the two contexts identical, and the induction hypothesis for the body finishes.

The second proof is a corollary of the theorems about the checker:

```
/-- Uniqueness again, as a corollary: the checker is a function, and it
finds every type that can be derived. -/
theorem type_unique' {Γ : Ctx} {e : Expr} {σ τ : Ty}
    (h₁ : HasType Γ e σ) (h₂ : HasType Γ e τ) : σ = τ := by
  have hσ := check_complete h₁   -- check Γ e = some σ
  have hτ := check_complete h₂   -- check Γ e = some τ
  rw [hσ] at hτ                  -- some σ = some τ
  exact Option.some.inj hτ
```

By completeness, the checker returns `some σ` and `some τ` for the same input. It is a function, so the two results are equal, and `some` is injective. This proof is shorter because the work was done in the

completeness proof; the direct proof shows the technique, induction plus inversion, that later chapters use when there is no checker to lean on.

Uniqueness fails for Rust as a whole: the literal `1` can be an `i32`, a `u8` or any other integer type depending on context, and the never type has every type. That is part of why Rust needs inference rather than plain synthesis.

Proving that no derivation exists

Finally, a typing error is a claim that *no* derivation exists, and we can prove such claims:

```
/-- No derivation of `⊢ 1 + true : τ` exists, whatever `τ` is. Proved by
inversion: the only rule that could conclude it is T-Add, whose second premise
would be a derivation of `⊢ true : Nat`, and no rule concludes that. -/
theorem add_bool_untypable (τ : Ty) :
    ¬ HasType [] (.add (.num 1) (.bool true)) τ := by
  intro h
  cases h with
  | add _ h₂ =>
    -- h₂ : HasType [] (Expr.bool true) Ty.nat
    cases h₂
```

`¬ P` means "`P` implies false" (Chapter 4 explains why), so the proof starts with `intro h`, assuming a derivation `h`, and must reach a contradiction. `cases h` inverts it: only T-Add can conclude a typing of a sum, so `h` must have come from T-Add, whose second premise `h₂` would be a derivation of `HasType [] (.bool true) .nat`. Inverting `h₂` asks which rule concludes that a boolean literal has type `Nat`. None does: T-Bool only gives `Bool`. With no possible rule, `cases h₂` has no cases to produce, and the proof is complete. This is the formal version of the Rust checker's message for `1 + true`, and the `decide` example earlier proved a similar fact by running the checker; here we reasoned about the rules directly.

3.11 Common confusions

"**The grammar is about text.**" A BNF grammar in type theory describes trees. Parentheses, precedence and spacing belong to the concrete syntax and the parser. When a rule says $e_1 + e_2$, it means a `+` node with two children, however it was written.

"**The rules are the type checker.**" The rules *define* a relation; they do not say how to compute it, and a rule like T-Never shows that reading them as an algorithm is not always straightforward. A checker is a separate program, and whether it agrees with the rules is a theorem (soundness and completeness) that must be proved. For Arith it is; for Rust, the compiler is the de facto definition, and the research calculi that model it are checked against it.

"**Well typed means the program cannot fail.**" It means the program cannot go wrong in the specific sense the semantics defines. A well-typed Arith program can overflow the Rust evaluator, and a well-typed Rust program can panic or loop forever.

"**The never type is the unit type.**" `()` has exactly one value, so an expression of type `()` always produces it. `!` has no values, so an expression of type `!` never produces anything. The first is "true" and the second "false" in the logical reading of Chapter 1's table.

"**A derivation is just a proof tree on paper.**" It is a mathematical object that can be represented, built, checked and reasoned about: as a Rust struct, as a Lean proof term, and as the input to `induction` and `cases`. That is what makes the theorems of this chapter possible.

3.12 Summary

- Abstract syntax is a tree; a BNF grammar defines the set of trees inductively, and becomes an enum in Rust and an inductive type in Lean. Concrete syntax is how trees are written as text.
- A judgment is a statement about syntax made in the metalanguage. Arith has a typing judgment $\Gamma \vdash e : \tau$ and an evaluation judgment $\rho \vdash e \Downarrow v$.
- Inference rules have premises, a conclusion and metavariables; axioms have no premises; side conditions are premises of another kind. The rules define the smallest relation closed under them, so a judgment holds exactly when it has a derivation, and properties of derivable judgments are proved by rule induction.
- In a syntax-directed system each form of expression is concluded by one rule, which makes inversion and type checking straightforward. Rust's T-Never is a rule that is not syntax-directed.
- Type checking is a search for a derivation. The `arith` crate returns derivations as data, prints them, and verifies them independently; type errors report where the search failed.
- In Lean the typing judgment is an inductive predicate `HasType`; derivations are proof terms, built by hand or with `apply`, `exact`, `constructor` and `repeat`; `induction` and `cases` on derivations are rule induction and inversion.
- Type safety for Arith: well-typed closed programs evaluate to a value of the predicted type and never get stuck. "Wrong" means stuck in the semantics; type systems are conservative and reject some harmless programs.
- Synthesis computes a type, checking verifies a given one, inference reconstructs missing type information. Structs and enums are nominal; tuples, arrays, references and aliases are structural.
- The Lean checker is sound and complete with respect to the rules, which makes the judgment decidable, and types are unique; the proofs are inductions whose cases mirror the rules.

3.13 Exercises

1. Write out the complete derivation tree, in fraction notation, for $\vdash$ `let` $b = (1 == 2)$ `in if` b `then` 0 `else` $1 : \mathit{Nat}$. Then check your answer with the `arith` crate, by adding the program to `ch03_arith.rs`. *Hint:* name the sub-derivations as in this chapter; the context of the body is $b : \mathit{Bool}$, and T-Eq is used with $\tau = \mathit{Nat}$.
2. Add a comparison `e <= e` to Arith in Rust: a new `Expr` variant, a token and a parser case, a typing rule T-Le (both operands `Nat`, result `Bool`), an evaluation case, a `Rule::Le` with its `verify` case, and tests. The random safety test should keep passing once the generator produces the new form. *Hint:* parse `<=` at the same level as `==`. Write the rule as a fraction first.

3. Make the same extension in Lean: add `| le (e₁ e₂ : Expr)` to `Expr`, a constructor to `HasType`, a case to `check`, and repair the proofs of `check_sound`, `check_complete` and `type_unique`. *Hint:* Lean will tell you exactly which cases are missing. Each new case is a copy of the corresponding `add` case with the result type changed.
4. Prove type safety for Arith in Lean: if `HasType Γ e τ` and the environment `ρ` gives each variable of `Γ` a value of the right type, then `∃ v, eval ρ e = some v ∧ v.ty = τ`. *Hint:* state the assumption on `ρ` as `∀ x σ, Γ.lookup x = some σ → ∃ v, ρ.lookup x = some v ∧ v.ty = σ`, put it after the colon together with `∀ ρ`, and do induction on the derivation. For a value `v` with `v.ty = .nat`, `cases v` and `simp [Value.ty] at h` recover the number; in the `ite` case, do the same for the condition and then `cases` on the boolean. The `letE` case is the hardest: show that the extended environment matches the extended context by `by_cases` on whether the variable looked up is `x`, simplifying `List.lookup`.
5. Prove in Lean that `if true then 1 else false` has no type: `theorem ite_untypable (τ : Ty) : ¬ HasType [] (.ite (.bool true) (.num 1) (.bool false)) τ`, first by inversion as in `add_bool_untypable`, then with `cases τ <;> decide` (why is `decide` alone not enough?). Which premise of T-If fails? *Hint:* after `cases h` you get derivations of `1 : τ` and `false : τ`; inverting the first forces `τ = Ty.nat`, and then the second has no rule.
6. Explain, in terms of the rules T-Never and T-Absurd, why `fn f() -> u32 { return 7; }` and `fn g(x: Infallible) -> u32 { match x {} }` compile, while `fn h(x: Infallible) -> u32 { x }` does not. Then define your own `enum Void {}` and write a function `fn absurd<T>(v: Void) -> T`. *Hint:* `return 7;` is a statement whose expression has type `!`; the body block then has no final expression of type `u32`, but the compiler knows the block diverges.

CHAPTER 4

Natural Deduction

In Chapter 3 you met judgments, inference rules and derivations, and used them to describe which programs of a small language are well typed. This chapter applies the same machinery to a different subject: logic itself. The question is old and simple. When is a proof a proof? Mathematicians reason by assuming things, drawing consequences, splitting into cases and discharging assumptions when they are no longer needed, and for most of history nobody wrote down the rules of that game precisely. In the mid-1930s Gerhard Gentzen did, and he called his system *natural deduction*, because it follows the way people actually reason.

Natural deduction matters to us for three reasons. It is the logic that Lean checks whenever you write a proof. Its rules obey a precise design principle, *harmony*, which explains their shapes and shows how one badly designed rule can wreck a logic. And its proofs turn out to be programs: the rules are, almost symbol for symbol, the typing rules of a functional language, and the way proofs simplify is the way programs run. Part II is built on that fact.

In this chapter you will learn:

- the difference between a proposition and a judgment, and what a hypothetical judgment $\Gamma \vdash A$ is;
- the introduction and elimination rules for $\wedge$, $\to$, $\vee$, $\top$ and $\bot$, with negation defined as $A \to \bot$, in tree notation and in sequent notation;
- harmony: why detours can be removed (local soundness) and why nothing is lost (local completeness), and how Prior's `tonk` shows that harmony matters;
- normalization, the subformula property and consistency, and a first look at the sequent calculus and cut elimination;
- how to prove propositional formulas in Lean, with proof terms and with the tactics `intro`, `exact`, `apply`, `constructor`, `cases`, `rcases`, `obtain`, `left`, `right`, `exfalso` and `contradiction`;
- how to define natural deduction *inside* Lean and prove it sound with respect to truth tables;
- how to build a proof checker in Rust that gives good error messages, and what happens to it when you add `tonk`.

4.1 Propositions and judgments

Formulas

We start with the objects that logic is about. Fix a supply of *atomic propositions*: statements whose internal structure we do not analyze, such as "the connection is open" or "the order was paid". Formulas are built from atoms with a handful of connectives:

$$A, B ::= P \mid \top \mid \bot \mid A \wedge B \mid A \vee B \mid A \to B$$

where P ranges over atoms and A, B over formulas (in rules, letters such as A, B, C stand for any formula), $\top$ ("truth") is a proposition that is trivially true, $\bot$ ("falsity", read "bottom") is one that is never true, and $\wedge$, $\vee$, $\to$ are "and", "or" and "implies". This is a grammar of trees, like the grammar of `Arith` in Chapter 3: $A \wedge B \to C$ is a tree whose root is $\to$. The usual conventions keep the parentheses down: $\neg$ binds tightest, then $\wedge$, then $\vee$, then $\to$, and $\to$ groups to the right, so $A \to B \to C$ means $A \to (B \to C)$.

Two connectives are deliberately missing, because we define them:

$$\neg A := A \to \bot \qquad\qquad A \leftrightarrow B := (A \to B) \wedge (B \to A)$$

"Not A" *means* "A implies falsity": to refute A is to show that assuming A leads to absurdity. This definition is not a shortcut. As you will see, it gives negation exactly the rules one would want, and Chapter 5 explains why it is the right reading of "not".

"A is true" is a judgment

The most useful distinction in this subject, stressed by Per Martin-Löf, is the one between a **proposition** and a **judgment**. A proposition, such as $A \wedge B$, is a piece of syntax, an object we talk about. A judgment is an act of knowledge *about* such an object. The judgment we care about here is

$$A \ true$$

read "A is true", or more precisely "we have evidence for A". There are other judgment forms. You have already seen "e has type τ" in Chapter 3, and "A is a well-formed formula" is another. Inference rules relate judgments, never propositions: the horizontal line in a rule is not a connective, and it says "if you know the judgments above, you may conclude the one below".

Hypothetical judgments

We rarely know that something is true outright. Mostly we know that something is true *provided* other things are: "if the payment cleared and the item is in stock, the order can ship". This is a **hypothetical judgment**, written

$$A_1, \ldots, A_n \vdash B$$

and read "assuming $A_1, \ldots, A_n$, B is true". The list on the left of the turnstile $\vdash$ is the **context**, usually called Γ, and the whole thing $\Gamma \vdash B$ is called a **sequent**. With an empty context, $\vdash B$ says that B is true outright.

> **WARNING**
>
> $A \vdash B$ and $\vdash A \to B$ are different judgments. The first is a hypothetical judgment about B; the second is a categorical judgment about a formula that happens to contain the symbol $\to$. The rules below make them *interderivable* (each follows from the other), but they live at different levels: $\vdash$ belongs to the metalanguage, $\to$ to the object language. Keeping them apart is what will let Chapter 6 ask "which rules govern the left side of $\vdash$?", a question about the judgment, not about any connective.

Is Γ a set, a multiset or a list? Choosing "set" quietly decides that assumptions can be reordered, ignored and reused at will. We take Γ to be a **list**, and in this chapter we use it as freely as ordinary logic does; Chapter 6 turns each of those freedoms into an explicit rule and asks what happens when one is removed, which is where Rust's ownership enters the story.

4.2 The rules of natural deduction

Gentzen organized the rules by connective. Each connective has **introduction rules**, which say how to prove a formula with that connective at the root, and **elimination rules**, which say how to use one. We write the rules in sequent style, with the context in every judgment.

Hypotheses

The simplest rule says that you may use what you assumed:

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \mathsf{Hyp}$$

The premise $A \in \Gamma$ is a side condition that you check by looking at the list, not a judgment you derive.

Conjunction

To prove $A \wedge B$, prove A and prove B. To use $A \wedge B$, take either part:

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\ \wedge\mathsf{I} \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\ \wedge\mathsf{E}_1 \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\ \wedge\mathsf{E}_2$$

Notice that both premises of $\wedge$I have the *same* context Γ. Whatever assumptions are available are available to both halves of the proof. This is called context *sharing*, and it is exactly the point where linear and affine logic (Part V) will part ways with ordinary logic.

Implication

To prove $A \to B$, assume A and prove B. To use $A \to B$, supply an A:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\ \to\mathsf{I} \qquad \frac{\Gamma \vdash A \to B \qquad \Gamma \vdash A}{\Gamma \vdash B}\ \to\mathsf{E}$$

$\to$E is the ancient rule *modus ponens*. $\to$I is the interesting one: its premise has one more assumption than its conclusion. Reading upward, it says "to prove $A \to B$, add A to your assumptions and prove B". Reading downward, it says that once B is proved under the assumption A, the assumption can

be **discharged**: the conclusion $A \to B$ no longer depends on it. The two rules together explain the warning above: $\Gamma, A \vdash B$ holds exactly when $\Gamma \vdash A \to B$ does.

Two notations

Gentzen did not write contexts. In his **tree notation**, a derivation is a tree whose leaves are assumptions, and a rule that discharges an assumption marks the corresponding leaves with a label, here u:

$$\dfrac{\begin{array}{c}[A]^u \\ \vdots \\ B\end{array}}{A \to B}\ {\to}\mathsf{I}^u$$

The brackets say that the leaves A labeled u are closed by the $\to$I step with the same label; the dots stand for the derivation of B, which may use A any number of times. Sequent notation carries the same information by writing the open assumptions to the left of $\vdash$ at every step. Trees are compact on paper; sequents make the context explicit, which programs and the structural rules of Chapter 6 need. The Rust checker later in this chapter reads proofs in Gentzen's style and prints them in sequent style.

> **NOTE**
>
> Tree notation hides something. A single $\to$I step with label u closes *every* leaf labeled u, however many there are. Nothing in the notation counts the uses of an assumption, so using A twice is free. That silent freedom is the structural rule of *contraction*, and Chapter 6 brings it into the open.

Disjunction

To prove $A \vee B$, prove one of them. Using $A \vee B$ is reasoning by cases:

$$\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B}\ {\vee}\mathsf{I}_1 \qquad \dfrac{\Gamma \vdash B}{\Gamma \vdash A \vee B}\ {\vee}\mathsf{I}_2$$

$$\dfrac{\Gamma \vdash A \vee B \qquad \Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma \vdash C}\ {\vee}\mathsf{E}$$

$\vee$E says: if $A \vee B$ holds, and C follows from A, and C also follows from B, then C holds. Each case discharges its own assumption. The conclusion C is arbitrary: it is whatever you were trying to prove when you decided to split into cases. This rule looks very different from $\wedge$E, and we will see shortly that it has to.

Truth and falsity

$\top$ has one introduction rule with no premises, and no elimination rule. $\bot$ has no introduction rule, and one elimination rule:

$$\dfrac{}{\Gamma \vdash \top}\ {\top}\mathsf{I} \qquad \dfrac{\Gamma \vdash \bot}{\Gamma \vdash C}\ {\bot}\mathsf{E}$$

$\bot$E is *ex falso quodlibet*, "from falsity, anything", which you met in Chapter 1 as Rust's never type coercing to every type. It is the rule people find most suspicious, and harmony will explain why it is exactly right.

Negation, for free

Since $\neg A$ is $A \to \bot$, negation needs no rules of its own. The rules for $\to$ with $B := \bot$ read:

$$\frac{\Gamma, A \vdash \bot}{\Gamma \vdash \neg A}\,{\to}\mathsf{I} \qquad\qquad \frac{\Gamma \vdash \neg A \qquad \Gamma \vdash A}{\Gamma \vdash \bot}\,{\to}\mathsf{E}$$

To refute A, assume it and derive absurdity; a refutation of A together with a proof of A is absurd. Everything we prove about implication (harmony, normalization) therefore holds for negation automatically.

A first derivation

Let us derive $A \wedge B \vdash B \wedge A$. Write $\Gamma = A \wedge B$:

$$\dfrac{\dfrac{\dfrac{A \wedge B \in \Gamma}{\Gamma \vdash A \wedge B}\,\mathsf{Hyp}}{\Gamma \vdash B}\,{\wedge}\mathsf{E}_2 \qquad \dfrac{\dfrac{A \wedge B \in \Gamma}{\Gamma \vdash A \wedge B}\,\mathsf{Hyp}}{\Gamma \vdash A}\,{\wedge}\mathsf{E}_1}{\Gamma \vdash B \wedge A}\,{\wedge}\mathsf{I}$$

Read it from the bottom, the way you would search for it. The goal is a conjunction, so try $\wedge$I, which asks for B and A separately. Neither is an assumption, but both are parts of the assumption $A \wedge B$, so $\wedge$E2 and $\wedge$E1 extract them, and Hyp closes each branch. One more step, $\to$I, discharges the assumption and gives $\vdash A \wedge B \to B \wedge A$.

The single assumption $A \wedge B$ was used twice, once in each branch, which is legal because $\wedge$I shares its context. Remember this derivation: in Chapter 21 it becomes illegal, and in Rust, with a non-`Copy` pair taken apart in two places, it becomes error E0382, "use of moved value".

Here is a second example, with negation: $\vdash A \to \neg\neg A$, that is, $A \to ((A \to \bot) \to \bot)$. In tree notation:

$$\dfrac{\dfrac{\dfrac{[\neg A]^{v} \qquad [A]^{u}}{\bot}\,{\to}\mathsf{E}}{\neg\neg A}\,{\to}\mathsf{I}^{v}}{A \to \neg\neg A}\,{\to}\mathsf{I}^{u}$$

Applying the assumption $\neg A$ (label v) to the assumption A (label u) gives $\bot$; discharging v and then u gives the implication. Chapter 5 is largely about the fact that the converse, $\neg\neg A \to A$, has no derivation.

4.3 Proving it in Lean

Lean's logic is natural deduction, extended to a much richer language. Chapter 2 showed you `theorem`, `example` and the first tactics; this section shows how Lean represents propositions and proofs, and how each rule of natural deduction becomes either a function you can apply or a tactic. The code is in `code/lean/Book/Ch04/Tactics.lean`.

Propositions are types

In Lean a proposition is a type whose type is `Prop`:

```
/-- info: True : Prop -/
#guard_msgs in
#check True
```

```
/-- info: 2 + 2 = 4 : Prop -/
#guard_msgs in
#check 2 + 2 = 4

/-- info: 2 + 2 = 5 : Prop -/
#guard_msgs in
#check 2 + 2 = 5

/-- info: ∀ (p q : Prop), p ∧ q → q ∧ p : Prop -/
#guard_msgs in
#check ∀ (p q : Prop), p ∧ q → q ∧ p
```

`2 + 2 = 5` is a perfectly good proposition: false propositions are types too, just types with no elements. A **proof** of `p` is a term of type `p`, and checking a proof is type checking. This is the Curry–Howard correspondence of Chapter 10; for now, take it as Lean's way of implementing the judgment "A true": `h : p` means "`h` is evidence for `p`".

The rules are functions

Lean's core library provides the connectives `∧`, `∨`, `→`, `True`, `False` and `¬`, and their rules as named functions:

```
/-- info: @And.intro : ∀ {a b : Prop}, a → b → a ∧ b -/
#guard_msgs in
#check @And.intro

/-- info: @And.left : ∀ {a b : Prop}, a ∧ b → a -/
#guard_msgs in
#check @And.left

/-- info: @Or.inl : ∀ {a b : Prop}, a → a ∨ b -/
#guard_msgs in
#check @Or.inl

/-- info: @Or.elim : ∀ {a b c : Prop}, a ∨ b → (a → c) → (b → c) → c -/
#guard_msgs in
#check @Or.elim

/-- info: True.intro : True -/
#guard_msgs in
#check True.intro

/-- info: @False.elim : {C : Sort u_1} → False → C -/
#guard_msgs in
#check @False.elim
```

Read the types as rules. `And.intro` turns a proof of `a` and a proof of `b` into a proof of `a ∧ b`: that is ∧I, with `a` and `b` implicit (in braces), so that Lean infers them. `And.left` is ∧E1, `Or.inl` is ∨I1, `True.intro` is ⊤I, and `Or.elim` is ∨E: a disjunction, a way from `a` to `c`, a way from `b` to `c`, hence `c`. `False.elim` is ⊥E; its result can be any `Sort`, even a type of data, like Rust's never type coercing to anything. There is no function for →I or →E: implication *is* the function type, so →I is writing a `fun` and →E is applying it. Negation is defined exactly as in the text:

```
/--
info: @[implicit_reducible] def Not : Prop → Prop :=
fun a => a → False
-/
#guard_msgs in
#print Not
```

(The attribute `implicit_reducible` tells Lean to unfold `Not` freely when it compares types, so that a proof of `p → False` is accepted where `¬p` is expected.)

Proof terms

With these functions, a proof is an expression:

```
/-- `∧E₂` and `∧E₁` feed `∧I`: the derivation of `A ∧ B ⊢ B ∧ A`. -/
theorem and_swap (p q : Prop) (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)

/-- `→I` is a `fun`, `→E` is application. -/
theorem imp_trans (p q r : Prop) : (p → q) → (q → r) → (p → r) :=
  fun hpq hqr hp => hqr (hpq hp)

/-- `∨E`: one branch for each way the disjunction could have been proved. -/
theorem or_swap (p q : Prop) (h : p ∨ q) : q ∨ p :=
  Or.elim h (fun hp => Or.inr hp) (fun hq => Or.inl hq)

/-- `⊥E`: from a proof of `False`, anything. -/
theorem explode (p : Prop) (h : False) : p :=
  False.elim h
```

The first is the derivation of $A \wedge B \vdash B \wedge A$ as a term: the hypothesis `h` plays the role of Γ, and the nesting of `And.intro`, `And.right` and `And.left` is the shape of the tree. In `imp_trans`, `fun hpq hqr hp => ...` is three uses of $\to$I and `hqr (hpq hp)` two uses of $\to$E. Lean also offers shorthand: angle brackets for introductions, `.1` and `.2` for projections, and patterns in `fun`:

```
/-- The same proofs with Lean's shorthand: `⟨_, _⟩` builds a pair, `.1` and
`.2` project, and `fun ⟨hp, hq⟩ => ...` takes the pair apart. -/
theorem and_swap' (p q : Prop) : p ∧ q → q ∧ p :=
  fun ⟨hp, hq⟩ => ⟨hq, hp⟩

theorem and_swap'' (p q : Prop) (h : p ∧ q) : q ∧ p :=
  ⟨h.2, h.1⟩
```

Tactics: the rules read upward

For longer proofs it is easier to work the way you searched for the derivation above: from the goal upward. That is **tactic mode**. After `by`, Lean shows a *goal*, a sequent `Γ ⊢ A` listing the hypotheses you have and what you must prove, and each tactic applies a rule backwards, replacing the goal by the premises of that rule. Here is each rule as a tactic.

`intro` is $\to$I and `exact` closes a goal with a term of exactly its type (here rule Hyp):

```
/-- `intro` is rule →I, read bottom-up: to prove `p → q`, assume `p` (the
assumption gets the name `hp`) and prove `q`. `exact` closes the goal with a
term of exactly the goal's type, here the assumption itself (rule Hyp). -/
theorem identity (p : Prop) : p → p := by
  intro hp      -- goal: p, with hp : p in the context
  exact hp      -- rule Hyp
```

After `intro hp` the goal is the premise of $\to$I. The editor's infoview lists the hypotheses `p : Prop` and `hp : p` on separate lines above the target `⊢ p`: sequent notation, written vertically.

`apply` is $\to$E used backwards: if the goal is `r` and you have `f : q → r`, `apply f` leaves you to prove `q`:

```
/-- `apply f` is rule →E, read bottom-up: if `f : q → r` and the goal is `r`,
it is enough to prove `q`. -/
```

```
theorem imp_trans_tac (p q r : Prop) : (p → q) → (q → r) → (p → r) := by
  intro hpq hqr hp   -- three uses of →I at once
  apply hqr          -- goal was r; now it is q
  apply hpq          -- goal was q; now it is p
  exact hp
```

`constructor` applies the introduction rule of the goal's connective, here ∧I, and leaves one goal per premise; the bullets `·` focus on each goal in turn:

```
/-- `constructor` applies an introduction rule. For `∧` that is ∧I, and it
leaves one goal per premise. The bullets `·` focus on one goal at a time. -/
theorem and_intro_tac (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp    -- first premise of ∧I
  · exact hq    -- second premise of ∧I
```

`obtain ⟨hp, hq⟩ := h` eliminates a conjunction: it replaces `h : p ∧ q` with its two parts, which is ∧E1 and ∧E2 at once:

```
/-- `obtain ⟨hp, hq⟩ := h` takes a conjunction apart: both eliminations ∧E₁ and
∧E₂ at once. -/
theorem and_swap_tac (p q : Prop) : p ∧ q → q ∧ p := by
  intro h
  obtain ⟨hp, hq⟩ := h   -- now hp : p and hq : q
  exact ⟨hq, hp⟩         -- ∧I, written as an anonymous constructor
```

`left` and `right` are ∨I1 and ∨I2: they commit to one side of a disjunction:

```
/-- `left` and `right` are the two introduction rules ∨I₁ and ∨I₂: they choose
which side of the disjunction we are going to prove. -/
theorem or_intro_tac (p q : Prop) (hq : q) : p ∨ q := by
  right       -- ∨I₂: the goal becomes q
  exact hq
```

`cases h with` is ∨E. It produces one goal for each way `h` could have been built, each with the corresponding assumption, exactly like the two right-hand premises of the rule:

```
/-- `cases h with` is ∨E: one goal per constructor of `Or`, each with the
assumption that this constructor was used. -/
theorem or_swap_tac (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h with
  | inl hp =>     -- case 1: h was built by Or.inl, and hp : p
    right
    exact hp
  | inr hq =>     -- case 2: h was built by Or.inr, and hq : q
    left
    exact hq
```

The names `inl` and `inr` are the constructors of `Or` (the functions `Or.inl` and `Or.inr` you saw above). If you know Rust, `cases` will remind you of `match`: one arm per variant, each arm binding the variant's payload. That is not a coincidence, and Chapter 10 explains it.

`rcases` and `rintro` take nested structures apart in one step. The pattern `⟨hp, hq | hr⟩` says "split the conjunction, then split its second component, which is a disjunction, into two cases":

```
/-- `rcases` and `rintro` take apart nested conjunctions and disjunctions in
one step: `⟨_, _⟩` splits an `∧`, and `_ | _` splits an `∨` into two goals. -/
theorem and_or_distrib (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
  intro h
```

```
  rcases h with ⟨hp, hq | hr⟩
  · left
    exact ⟨hp, hq⟩
  · right
    exact ⟨hp, hr⟩

/-- `rintro` is `intro` followed by `rcases`. -/
theorem and_or_distrib' (p q r : Prop) : (p ∧ q) ∨ (p ∧ r) → p ∧ (q ∨ r) := by
  rintro (⟨hp, hq⟩ | ⟨hp, hr⟩)
  · exact ⟨hp, Or.inl hq⟩
  · exact ⟨hp, Or.inr hr⟩
```

`exfalso` is ⊥E read upward: whatever the goal is, replace it with `False`, since a proof of `False` proves anything. Here the proof of `False` comes from →E, applying `hnp : ¬p` (which *is* `p → False`) to `hp : p`:

```
/-- `exfalso` is ⊥E read bottom-up: whatever the goal, replace it by `False`.
Then we prove `False` by →E, applying `hnp : p → False` to `hp : p`. -/
theorem not_or_imp (p q : Prop) : ¬p ∨ q → (p → q) := by
  intro h hp
  rcases h with hnp | hq
  · exfalso       -- goal: False
    exact hnp hp  -- ¬p is p → False, so this is →E
  · exact hq
```

`contradiction` is a small automation: it closes any goal when the context is visibly inconsistent, for instance when it contains both `p` and `¬p`. Behind the scenes it uses →E and ⊥E, just as the previous proof did by hand:

```
/-- `contradiction` closes any goal when the context is visibly inconsistent:
it contains `False`, or both `p` and `¬p`. -/
theorem non_contradiction (p q : Prop) (hp : p) (hnp : ¬p) : q := by
  contradiction
```

Negation needs no tactics of its own. To prove `¬p`, `intro` a proof of `p` and derive `False`; to use `¬p`, apply it:

```
/-- Negation is implication into `False`, so its rules are those of `→`:
`intro` proves a negation, and applying it to a proof eliminates it. -/
theorem contrapositive (p q : Prop) : (p → q) → ¬q → ¬p := by
  intro hpq hnq hp   -- the last intro unfolds ¬p into p → False
  exact hnq (hpq hp)

theorem not_and_not (p : Prop) : ¬(p ∧ ¬p) := by
  intro ⟨hp, hnp⟩
  exact hnp hp
```

In `not_and_not`, the pattern after `intro` takes the conjunction apart as it is introduced. `↔` is a structure whose two fields are the two directions, so `constructor` splits it:

```
/-- Currying: a proof of `p ∧ q → r` and a proof of `p → q → r` can be turned
into each other. `↔` is a structure with two fields, built by `constructor`. -/
theorem curry_iff (p q r : Prop) : (p ∧ q → r) ↔ (p → q → r) := by
  constructor
  · intro h hp hq
    exact h ⟨hp, hq⟩
  · intro h ⟨hp, hq⟩
    exact h hp hq
```

Finally, when a step is wrong, Lean says which goal it could not close and why:

```
/--
error: Type mismatch
  hp
has type
  p
but is expected to have type
  q
-/
#guard_msgs in
theorem wrong (p q : Prop) : p → q := by
  intro hp
  exact hp
```

After `intro hp` the goal is `q`, and `hp` has type `p`, so `exact hp` fails: there is no rule that turns a proof of `p` into a proof of an unrelated `q`.

What a tactic proof really is

A tactic script is not the proof; it is a program that *builds* the proof term. The command `#print` shows the term that Lean stored:

```
/--
info: theorem Ch04.and_swap_tac : ∀ (p q : Prop), p ∧ q → q ∧ p :=
fun p q h => And.casesOn h fun hp hq => ⟨hq, hp⟩
-/
#guard_msgs in
#print and_swap_tac

/--
info: theorem Ch04.imp_trans_tac : ∀ (p q r : Prop), (p → q) → (q → r) → p → r :=
fun p q r hpq hqr hp => hqr (hpq hp)
-/
#guard_msgs in
#print imp_trans_tac
```

The second one is literally the term we wrote by hand for `imp_trans`. In the first, `obtain` produced `And.casesOn h fun hp hq => ...`: "take `h` apart into its two components and continue with them". Disjunction is more verbose:

```
/--
info: theorem Ch04.or_swap_tac : ∀ (p q : Prop), p ∨ q → q ∨ p :=
fun p q h => Or.casesOn (motive := fun t => h = t → q ∨ p) h (fun hp h => Or.inr hp) (fun hq
h => Or.inl hq) (Eq.refl h)
-/
#guard_msgs in
#print or_swap_tac
```

The `motive` describes the conclusion of the case analysis; the `h = t →` inside it and the final `Eq.refl h` are bookkeeping that `cases` adds so that each branch can remember which case it is in. What remains is `Or.casesOn h (fun hp => Or.inr hp) (fun hq => Or.inl hq)`: one function per case, which is the ∨E rule. Tactics are a convenient way to write natural-deduction trees, and the kernel checks the trees.

4.4 Harmony

Why these rules and not others? The answer is a design principle that natural deduction makes visible, and it is the conceptual heart of the chapter.

Introductions define, eliminations follow

Gentzen remarked that the introduction rules can be seen as *definitions* of the connectives: $A \wedge B$ is whatever you get from a proof of A and a proof of B, and nothing more. The eliminations must then be *justified* by the introductions: they may extract exactly what an introduction put in. Extracting more proves things that should not be provable; extracting less loses information. The two halves of the requirement are **local soundness** and **local completeness**; together they are **harmony**.

Local soundness: detours can be removed

A **detour** is an introduction rule immediately followed by an elimination of the same connective. Local soundness says that every detour can be removed, leaving a derivation of the same conclusion from the same assumptions. For conjunction, suppose the premises of $\wedge$I have derivations $\mathcal{D}_1$ (of $\Gamma \vdash A$) and $\mathcal{D}_2$ (of $\Gamma \vdash B$). The detour on the left becomes the derivation on the right:

$$\dfrac{\dfrac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\,\wedge\mathsf{I}}{\Gamma \vdash A}\,\wedge\mathsf{E}_1 \qquad \mapsto \qquad \Gamma \vdash A \;\text{ by }\; \mathcal{D}_1$$

We built $A \wedge B$ from a proof of A only to take A out again; we could have used $\mathcal{D}_1$ directly, and $\mathcal{D}_2$ is thrown away. $\wedge$E2 is symmetric. For implication, let $\mathcal{D}_1$ derive $\Gamma, A \vdash B$ and $\mathcal{D}_2$ derive $\Gamma \vdash A$:

$$\dfrac{\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\,\to\mathsf{I} \qquad \Gamma \vdash A}{\Gamma \vdash B}\,\to\mathsf{E} \qquad \mapsto \qquad \Gamma \vdash B \;\text{ by }\; \mathcal{D}_1[\mathcal{D}_2/A]$$

We proved B from the assumption A, packaged that as $A \to B$, and applied it to a proof of A. The direct proof, written $\mathcal{D}_1[\mathcal{D}_2/A]$, is $\mathcal{D}_1$ with every use of the assumption A (every Hyp leaf for it) replaced by a copy of $\mathcal{D}_2$. This is the most important reduction in the book. Proving a lemma and immediately applying it is a function definition followed by a call, and removing the detour is substituting the argument for the parameter. In the λ-calculus of Chapter 7 it is written

$$(\lambda x.\, e)\; v \to e[v/x]$$

and called **β-reduction**. The basic step of computation is local soundness for implication: computation was not added to logic from outside, it is the act of removing detours from proofs.

For disjunction, if the disjunction was built by $\vee$I1 from a derivation $\mathcal{D}$ of $\Gamma \vdash A$, we know which case of $\vee$E applies, and can take it. With $\mathcal{E}_1$ deriving the first case, $\Gamma, A \vdash C$:

$$\dfrac{\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B}\,\vee\mathsf{I}_1 \qquad \Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma \vdash C}\,\vee\mathsf{E} \qquad \mapsto \qquad \Gamma \vdash C \;\text{ by }\; \mathcal{E}_1[\mathcal{D}/A]$$

In a programming language this is a `match` on a value whose variant is known: `match Left(v) { Left(x) => e1, Right(y) => e2 }` evaluates to `e1` with `v` for `x`. Falsity has no introduction rule, so there is no detour to remove and local soundness holds vacuously; the same goes for truth, which has no elimination.

Local completeness: nothing is lost

Local completeness asks the opposite question: are the eliminations strong enough? Given *any* derivation of a formula, can we take it apart with the eliminations and rebuild it with the introductions? For conjunction, given any derivation $\mathcal{D}$ of $\Gamma \vdash A \wedge B$:

$$\Gamma \vdash A \wedge B \quad \mapsto \quad \dfrac{\dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\wedge \mathsf{E}_1 \qquad \dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\wedge \mathsf{E}_2}{\Gamma \vdash A \wedge B}\wedge \mathsf{I}$$

where both copies of $\Gamma \vdash A \wedge B$ on the right are proved by $\mathcal{D}$.

For implication, from $\mathcal{D}$ proving $A \to B$ we assume A, apply $\mathcal{D}$ with $\to$E and discharge with $\to$I. For disjunction, we split $\mathcal{D}$ with $\vee$E and rebuild each case with the matching $\vee$I. For truth, any derivation of $\top$ can be replaced by $\top$I. On programs these expansions are called **η**: a pair `p` is the same as `(p.0, p.1)`, a function `f` the same as `|x| f(x)`, and any value of type `()` is `()`.

Condition	Proof-theoretic content	Computational content
local soundness	detours can be removed	β-reduction
local completeness	eliminations recover everything	η-expansion

The two halves of harmony are exactly the two halves of the equational theory of the λ-calculus. You will meet β and η again in Chapters 7, 8 and 19.

Harmony in Lean

Lean takes Gentzen's remark literally. When you define a proposition with `inductive`, you give only its introduction rules, the constructors, and Lean *derives* the elimination rule. The file `code/lean/Book/Ch04/Harmony.lean` defines our own copies of the connectives:

```
/-- Conjunction: one introduction rule with two premises. -/
inductive MyAnd (a b : Prop) : Prop where
  | intro : a → b → MyAnd a b

/-- Disjunction: two introduction rules. -/
inductive MyOr (a b : Prop) : Prop where
  | inl : a → MyOr a b
  | inr : b → MyOr a b

/-- Truth: one introduction rule with no premises. -/
inductive MyTrue : Prop where
  | intro : MyTrue

/-- Falsity: no introduction rules at all. -/
inductive MyFalse : Prop
```

For each one Lean generates a *recursor*, its most general elimination rule:

```
/--
info: @MyOr.rec : ∀ {a b : Prop} {motive : MyOr a b → Prop},
  (∀ (a_1 : a), motive …) → (∀ (a_1 : b), motive …) → ∀ (t : MyOr a b), motive t
-/
#guard_msgs in
#check @MyOr.rec

/--
info: @MyTrue.rec : {motive : MyTrue → Sort u_1} → motive MyTrue.intro → (t : MyTrue) →
motive t
```

```
-/
#guard_msgs in
#check @MyTrue.rec

/-- info: MyFalse.rec : (motive : MyFalse → Sort u_1) → (t : MyFalse) → motive t -/
#guard_msgs in
#check @MyFalse.rec
```

The recursor of `MyOr` takes one function per constructor and proves `motive t` for any `t : MyOr a b`. The motive is the conclusion C of $\vee$E, here allowed to depend on the proof being eliminated (the `…` are proofs that Lean does not print). The recursor of `MyTrue` needs the motive in the single case `MyTrue.intro` and gives back nothing new: the useless elimination that $\top$ does without. The recursor of `MyFalse` has *no* cases and proves any motive: it is $\bot$E. With a constant motive, the familiar rules fall out:

```
/-- ∧E₁, derived from the recursor: take the pair apart, keep the left part. -/
theorem MyAnd.left {a b : Prop} (h : MyAnd a b) : a :=
  MyAnd.rec (motive := fun _ => a) (fun ha _ => ha) h

/-- ∨E, derived from the recursor with a constant motive `c`. -/
theorem MyOr.elim {a b c : Prop} (h : MyOr a b) (left : a → c) (right : b → c) : c :=
  MyOr.rec (motive := fun _ => c) left right h

/-- ⊥E, derived from the recursor: zero cases to handle. -/
theorem MyFalse.elim {c : Prop} (h : MyFalse) : c :=
  MyFalse.rec (motive := fun _ => c) h
```

Because elimination is derived from introduction, every inductive proposition in Lean is harmonious by construction. You cannot define `tonk` (next section) with `inductive`: you can only state its introduction rule, and Lean will hand you the matching, harmless elimination.

Local soundness and completeness are invisible on proofs in Lean, because two proofs of the same proposition are always considered equal (proof irrelevance, Chapter 10). They are visible on *data*, where Lean's evaluator performs exactly the reductions above:

```
/-! Local soundness, on data: an introduction followed by an elimination
computes away. All three hold by `rfl`: Lean evaluates both sides. -/
example (α β : Type) (a : α) (b : β) : (a, b).1 = a := rfl

example (α β : Type) (f : α → β) (a : α) : (fun x => f x) a = f a := rfl

example (α β γ : Type) (f : α → γ) (g : β → γ) (a : α) :
    Sum.elim f g (Sum.inl a) = f a := rfl
```

```
/-! Local completeness, on data: eliminating and re-introducing gives back
what we started with. For pairs, functions and `Unit` Lean knows this by
`rfl` (definitional η); for sums it needs a case analysis. -/
example (α β : Type) (p : α × β) : (p.1, p.2) = p := rfl

example (α β : Type) (f : α → β) : (fun x => f x) = f := rfl

example (u : Unit) : u = () := rfl

example (α β : Type) (s : α ⊕ β) : Sum.elim Sum.inl Sum.inr s = s := by
  cases s <;> rfl
```

All the β-rules hold by `rfl`, that is, by computation. So do the η-rules for pairs, functions and `Unit`, which Lean builds into its notion of definitional equality. The η-rule for sums does not: it needs `cases s`

and then `rfl` in each case (the combinator `<;>` runs the tactic after it on every goal produced by the one before it). The odd one out is again disjunction, whose elimination has a different shape, as we will see after the next section.

Tonk

In 1960 Arthur Prior challenged the idea that rules can define the meaning of a connective. If they can, he said, consider the connective `tonk`, with an introduction rule borrowed from $\vee$ and an elimination rule borrowed from $\wedge$:

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \mathit{tonk} B}\ \text{tonk-I} \qquad \frac{\Gamma \vdash A \mathit{tonk} B}{\Gamma \vdash B}\ \text{tonk-E}$$

Each rule looks innocent alone. Together they prove everything:

$$\dfrac{\dfrac{\dfrac{}{\vdash \top}\top\text{I}}{\vdash \top \mathit{tonk} \bot}\ \text{tonk-I}}{\vdash \bot}\ \text{tonk-E}$$

and from $\bot$, by $\bot$E, any formula at all. A logic in which everything is provable is useless. Harmony is the precise diagnosis: the detour tonk-I followed by tonk-E cannot be removed, because we have a derivation of A, need one of B, and nothing connects them. Prior's challenge was that "meaning given by rules" licenses nonsense; the answer is that only harmonious rules give meaning, and harmony is a checkable property.

In Lean we can state the catastrophe as a theorem: no operation on propositions satisfies both of tonk's rules, because any such operation would prove `False`. And no truth table can make both rules sound:

```
/-- No operation on propositions obeys both of Prior's rules for `tonk`:
if one did, `False` would be provable. -/
theorem no_tonk (tonk : Prop → Prop → Prop)
    (tonkI : ∀ {a b : Prop}, a → tonk a b)
    (tonkE : ∀ {a b : Prop}, tonk a b → b) : False :=
  tonkE (tonkI trivial)

/-- No truth table for `tonk` makes both rules sound: tonk-I needs
`a → tonk a b`, tonk-E needs `tonk a b → b`, and the row `a = true`,
`b = false` cannot satisfy both. -/
theorem no_tonk_table : ¬ ∃ f : Bool → Bool → Bool,
    (∀ a b, a = true → f a b = true) ∧ (∀ a b, f a b = true → b = true) := by
  intro ⟨f, hI, hE⟩
  have h : false = true := hE true false (hI true false rfl)
  contradiction
```

The proof of `no_tonk` is the derivation above, written as a term. In `no_tonk_table`, tonk-I demands that `f true b` be `true` and tonk-E demands that `f a false` be `false`, and the row `a = true, b = false` cannot satisfy both: `contradiction` closes the goal because the hypothesis `false = true` equates two different constructors of `Bool`.

Why $\vee$E and $\bot$E look strange

Most people find $\vee$E odd on first contact: $\wedge$E simply extracts a component, while $\vee$E has three premises and an arbitrary conclusion C. Harmony forces this shape. $A \vee B$ may have been introduced by either of two rules, and the elimination cannot know which. The strongest thing it can do is require the user

to handle *both* cases, and let them produce whatever they like. That is what makes local soundness work for both introductions at once. The pattern is general:

Introduction rules	Shape of the elimination	Examples
one	projections: take out what was put in	$\wedge$, $\to$
several	case analysis: handle every introduction	$\vee$
none	case analysis with zero cases: conclude anything	$\bot$
one, with no premises	nothing to take out: no elimination	$\top$

$\bot$E is therefore not a special license for absurdity. It is the case analysis of $\vee$E with zero cases, and Lean's `MyFalse.rec` above has exactly that shape. Rust has the same thing: an `enum` with no variants can be matched with no arms, and the match has any type you like.

```
enum Void {}

fn absurd<T>(v: Void) -> T {
    match v {} // zero variants, zero arms: this is rule ⊥E
}
```

(This fragment is compiled as part of `code/rust/examples/ch05_negation.rs`, which Chapter 5 discusses.) Is $\bot$E dangerous? Only in a context that is already contradictory: the consistency theorem below shows that $\bot$ is never provable from no assumptions. Explosion only says that an impossible situation constrains nothing, just as an unreachable `match` arm can claim any type because it never runs.

4.5 Normalization, subformulas and consistency

A derivation with no detours is called **normal**. Removing one detour can create others (substituting a derivation into a hole can place an introduction right above an elimination), so it is not obvious that repeatedly removing detours ever stops. It does:

- **Normalization theorem** (Dag Prawitz). Every derivation can be reduced to a normal derivation of the same sequent. Weak normalization says that some sequence of reductions terminates; strong normalization says that every sequence does, whatever detour you choose to remove first. Both hold for the system of this chapter.

Normal derivations have a remarkable property:

- **Subformula property.** Every formula that occurs in a normal derivation is a subformula of the conclusion or of an open assumption.

A normal proof never invents anything: it is built from pieces of its conclusion and assumptions. (The full theorem needs a few extra reductions, called *permutative conversions*, that move $\vee$E and $\bot$E out of the way; we will not need them.) Detours are where "foreign" formulas come from, as the Rust normalizer will show.

From these two theorems follows something that sounds like it should need a semantic argument:

- **Consistency.** There is no derivation of $\vdash \bot$.

Proof sketch. Suppose there were one; by normalization there is a normal one. Its last rule cannot be an introduction, because $\bot$ has none, nor Hyp, because the context is empty. So it is an elimination. Follow its *major* premise (the one containing the eliminated connective) upward, then that rule's major premise, and so on. In a normal derivation no major premise is the conclusion of an introduction, so the path runs through eliminations only, and along it the context never changes (eliminations extend the context only in the case premises of $\vee$E). The path ends at a Hyp, which needs a formula in the context, but the context is empty.

Consistency, normally a question about meaning, follows from a syntactic fact about how proofs simplify. Each of these results has a programming reading, which Part II develops:

Result	Statement	Reading for programs
weak normalization	some reduction sequence ends	some evaluation order terminates
strong normalization	every reduction sequence ends	every evaluation order terminates
subformula property	normal proofs use only subformulas	values are built from their types' parts
consistency	$\bot$ has no closed proof	the empty type has no closed value

Rust, with `loop {}`, has no normalization theorem, and indeed `loop {}` has every type including the empty one. Chapter 10 explains why that makes Rust useless as a logic and harmless as a programming language.

4.6 Sequent calculus and cut elimination

Gentzen gave a second presentation of the same logic, the **sequent calculus** (LJ, for the intuitionistic version). It keeps the introductions, now called *right rules*, and replaces each elimination by a *left rule* that introduces the connective on the left of $\vdash$, among the assumptions. For conjunction and implication:

$$\frac{\Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}\ \wedge\mathsf{L} \qquad\qquad \frac{\Gamma \vdash A \qquad \Gamma, B \vdash C}{\Gamma, A \to B \vdash C}\ \to\mathsf{L}$$

Read $\wedge$L as "an assumption $A \wedge B$ can be split into two assumptions", and $\to$L as "to use an assumption $A \to B$, prove A, then continue with B as an assumption". Every rule of the sequent calculus builds up formulas, never takes them apart, with one exception:

$$\frac{\Gamma \vdash A \qquad \Gamma, A \vdash C}{\Gamma \vdash C}\ \mathsf{Cut}$$

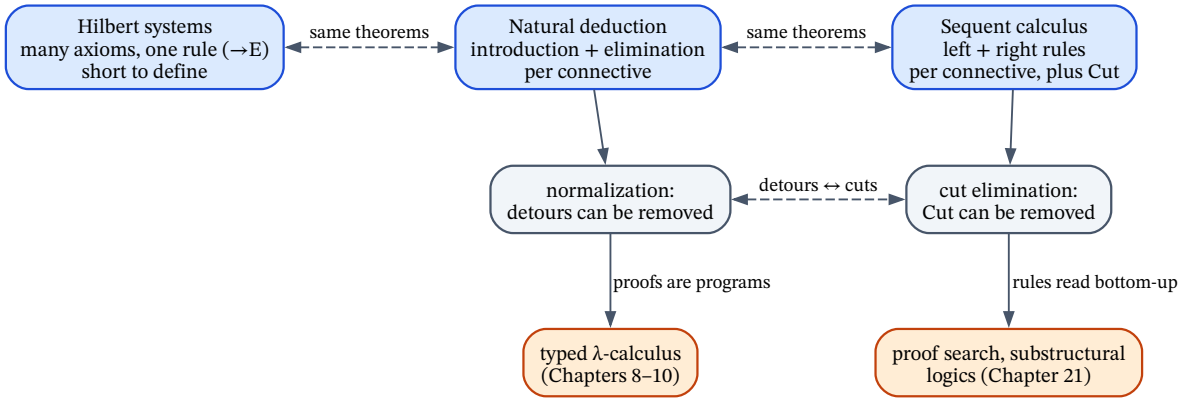
Cut is "prove a lemma A, then use it". Gentzen's main theorem, the *Hauptsatz*, is that cut is unnecessary:

- **Cut elimination.** Every derivation in the sequent calculus can be transformed into one that does not use Cut.

Cut elimination and normalization are two faces of one phenomenon: translating between the two systems maps cuts to detours. The price is size. Cut-free proofs can be enormously larger than proofs

with cuts, which is the proof-theoretic reason why mathematicians use lemmas even though, in principle, they need not.

Why have two systems? Because they are good at different things.



Three presentations of the same logic, the theorem that makes each one tick, and where each one leads in this book.

Natural deduction is where proofs look like programs. The sequent calculus is where proof *search* lives: in a cut-free proof every rule, read bottom-up, only takes apart the goal and the assumptions, so a program can search by trying the finitely many rules that apply. It is also where **substructural logic** was born: the phrase means "a sequent calculus with some structural rules removed", and Chapters 6 and 21 use exactly this vocabulary. *Hilbert systems*, with many axioms and only modus ponens, are short to define and awkward to use; we will not need them.

4.7 A proof checker in Rust

It is instructive to build a proof checker of our own, in the language you know, and watch it reject bad proofs. The crate `natded` (in `code/rust/crates/natded`) has a formula type with a parser and printer, proof trees, the checker with its error messages, truth tables, and detour removal.

Formulas and proofs as data

Formulas are an `enum`, one variant per case of the grammar, plus Prior's `tonk`, which the checker will reject unless we ask for it:

```
/// A formula of propositional logic.
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum Formula {
    /// An atomic proposition such as `A` or `Raining`.
    Atom(String),
    /// Truth, `⊤`.
    Top,
    /// Falsity, `⊥`.
    Bot,
    /// Conjunction, `A ∧ B`.
    And(Box<Formula>, Box<Formula>),
    /// Disjunction, `A ∨ B`.
    Or(Box<Formula>, Box<Formula>),
    /// Implication, `A → B`. Negation `¬A` is `A → ⊥`.
```

```
    Imp(Box<Formula>, Box<Formula>),
    /// Prior's `A tonk B`: not a real connective, see Chapter 4.
    Tonk(Box<Formula>, Box<Formula>),
}
```

A proof is a tree with one variant per rule. It is written in Gentzen's style: assumptions have names, and the rules that discharge an assumption (→I, and each case of ∨E) choose the name:

```
/// A proof tree. Each variant is one rule of natural deduction.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Proof {
    /// Hyp: use the assumption with this name.
    Hyp(String),
    /// ⊤I: truth, from no premises.
    TopI,
    /// ⊥E: from a proof of `⊥`, the given formula.
    BotE(Box<Proof>, Formula),
    /// ∧I: from proofs of `A` and of `B`, `A ∧ B`.
    AndI(Box<Proof>, Box<Proof>),
    /// ∧E1: from `A ∧ B`, `A`.
    AndE1(Box<Proof>),
    /// ∧E2: from `A ∧ B`, `B`.
    AndE2(Box<Proof>),
    /// ∨I1: from `A`, `A ∨ B`; the formula is `B`.
    OrI1(Box<Proof>, Formula),
    /// ∨I2: from `B`, `A ∨ B`; the formula is `A`.
    OrI2(Formula, Box<Proof>),
    /// ∨E: from `A ∨ B`, a proof of `C` assuming `A` (named `left.0`) and a
    /// proof of `C` assuming `B` (named `right.0`), conclude `C`.
    OrE {
        scrutinee: Box<Proof>,
        left: (String, Box<Proof>),
        right: (String, Box<Proof>),
    },
    /// →I: assume `A` under the given name, prove `B`, conclude `A → B`.
    /// The assumption is discharged: it cannot be used outside `body`.
    ImpI {
        name: String,
        assumption: Formula,
        body: Box<Proof>,
    },
    /// →E (modus ponens): from `A → B` and `A`, `B`.
    ImpE(Box<Proof>, Box<Proof>),
    /// tonk-I: from `A`, `A tonk B`; the formula is `B`.
    TonkI(Box<Proof>, Formula),
    /// tonk-E: from `A tonk B`, `B`.
    TonkE(Box<Proof>),
}
```

A few rules carry a formula. →I records the formula of the assumption it introduces (Gentzen's $[A]^u$), ∨I1 records the other disjunct, and ⊥E records its conclusion: these are formulas that cannot be computed from the premises. Everything else the checker works out. Short constructor functions (`hyp`, `and_i`, `imp_i`, ...) make proofs readable. Here is the derivation of $\vdash A \wedge B \to B \wedge A$, from the example program `code/rust/crates/natded/examples/ch04_proof_checker.rs`:

```
// ⊢ A ∧ B → B ∧ A. Assume A ∧ B under the name `u`, take it apart twice
// (∧E2 and ∧E1), put the parts back together in the other order (∧I),
// and discharge `u` (→I).
let and_comm = imp_i("u", f("A ∧ B"), and_i(and_e2(hyp("u")), and_e1(hyp("u"))));
let d1 = checker
    .prove(&[], &f("A ∧ B → B ∧ A"), &and_comm)
    .expect("a correct proof");
print!("{d1}");
```

The checker

`Checker::check` walks the tree, computing the formula each node proves and checking that each rule is applied to premises of the right shape. It returns a `Derivation`, the same tree with the full sequent at every node, or a `CheckError`. Here are the two rules for implication:

```
Proof::ImpI {
    name,
    assumption,
    body,
} => {
    // The body may use the new assumption; the conclusion may not.
    let inner = extended(context, name, assumption);
    let d = self.premise(&inner, body, path, 1)?;
    let c = Formula::imp(assumption.clone(), d.conclusion.clone());
    labels.push(name.clone());
    premises.push(d);
    c
}
Proof::ImpE(p, q) => {
    let d1 = self.premise(context, p, path, 1)?;
    let Formula::Imp(a, b) = &d1.conclusion else {
        let found = d1.conclusion;
        let kind = shape("first premise", "an implication", found);
        return Err(fail(path, kind));
    };
    let d2 = self.premise(context, q, path, 2)?;
    if d2.conclusion != **a {
        let kind = ErrorKind::ArgumentMismatch {
            implication: d1.conclusion.clone(),
            found: d2.conclusion,
        };
        return Err(fail(path, kind));
    }
    let c = (**b).clone();
    premises.extend([d1, d2]);
    c
}
```

For →I, the checker extends the context with the named assumption, checks the body there, and concludes $A \to B$ in the *original* context: discharge is the scoping of a variable. For →E, it insists that the first premise proves an implication and the second its antecedent. The other rules follow the same pattern. Printed, the derivation of $\vdash A \wedge B \to B \wedge A$ is the earlier tree in sequent notation, with each rule (and the assumption it uses or discharges) on the right:

```
⊢ A ∧ B → B ∧ A          →I u
└─ A ∧ B ⊢ B ∧ A         ∧I
   ├─ A ∧ B ⊢ B          ∧E2
   │  └─ A ∧ B ⊢ A ∧ B   Hyp u
   └─ A ∧ B ⊢ A          ∧E1
      └─ A ∧ B ⊢ A ∧ B   Hyp u
```

Each sequent is proved by the rule on its line from the sequents indented just below it. The context on the second line is $A \wedge B$ because →I added it; the root's context is empty because →I discharged it.

Error messages

A checker is only as useful as its error messages. Three wrong proofs:

```
// Three mistakes. First: projecting from something that is not a pair.
let wrong_shape = and_e1(hyp("u"));
let e1 = checker
```

```
    .check(&[("u", f("A → B"))], &wrong_shape)
    .unwrap_err();
println!("{e1}");

// Second: using an assumption after the →I that discharged it.
let out_of_scope = and_i(imp_i("u", f("A"), hyp("u")), hyp("u"));
let e2 = checker.check(&[], &out_of_scope).unwrap_err();
println!("{e2}");

// Third: modus ponens with the wrong argument.
let wrong_argument = imp_i("b", f("B"), imp_e(hyp("g"), hyp("b")));
let e3 = checker
    .check(&[("g", f("A → C"))], &wrong_argument)
    .unwrap_err();
println!("{e3}");
```

```
error in ∧E1 at root:
  the premise must prove a conjunction, but it proves `A → B`
error in Hyp at root.2:
  no assumption named `u` is available here (none are)
error in →E at root.1:
  the implication is `A → C`, so the argument must prove `A`, but it proves `B`
```

Each error names the failing rule and its position (`root.2.1` would mean "the first premise of the second premise of the root"). The first proof applies ∧E1 to an implication. The second uses `u` in the second premise of ∧I, but `u` was discharged by the ⟶I in the first premise: an assumption exists only inside the subproof that discharges it, as a variable exists only in its scope. The third applies $A \to C$ to a proof of B. These messages read like Rust's E0308 "mismatched types", and Chapter 10 explains why: a type checker is a proof checker.

Truth tables and soundness, tested

The crate also evaluates formulas in the classical two-valued way, the truth tables you may know from digital logic: an atom is true or false, and each connective computes a truth value from those of its parts.

```
/// The truth value of `f` in the row `v`. Atoms missing from `v` are false.
/// Returns `None` if `f` mentions `tonk`, which has no truth table.
pub fn eval(f: &Formula, v: &Valuation) -> Option<bool> {
    if f.mentions_tonk() {
        return None;
    }
    Some(eval_with(f, v, &[[false; 2]; 2]))
}

/// The truth value of `f`, reading `tonk` from the given table.
pub fn eval_with(f: &Formula, v: &Valuation, tonk: &TonkTable) -> bool {
    match f {
        Formula::Atom(name) => v.get(name).copied().unwrap_or(false),
        Formula::Top => true,
        Formula::Bot => false,
        Formula::And(a, b) => eval_with(a, v, tonk) && eval_with(b, v, tonk),
        Formula::Or(a, b) => eval_with(a, v, tonk) || eval_with(b, v, tonk),
        Formula::Imp(a, b) => !eval_with(a, v, tonk) || eval_with(b, v, tonk),
        Formula::Tonk(a, b) => {
            tonk[eval_with(a, v, tonk) as usize][eval_with(b, v, tonk) as usize]
        }
    }
}
```

(`eval_with` also takes a candidate truth table for `tonk`; we will need it at the end of this section.) Truth tables give a *semantics*, an account of what formulas mean that does not mention proofs. A proof system is **sound** for a semantics when everything it proves is true in that semantics. For truth tables this means: if $\Gamma \vdash A$ is derivable, then every row that makes all of Γ true makes A true. The example program checks this on the proofs it builds, and uses the contrapositive to show that a formula is *not* provable:

```
// Soundness, tested: every sequent the checker accepts is true in every
// row of its truth table.
for d in [&d1, &d2] {
    let hyps: Vec<Formula> = d.context.iter().map(|(_, a)| a.clone()).collect();
    assert!(counterexample(&hyps, &d.conclusion).is_none());
}
print!("{}", truth_table(&f("A ∧ B → B ∧ A")));

// A formula with a counterexample cannot have a proof: by soundness, the
// proof would make it true in the falsifying row.
let not_valid = f("A ∨ B → A");
let row = counterexample(&[], &not_valid).expect("not a tautology");
println!("{not_valid} is false when {}", show_valuation(&row));
```

```
A B │ A ∧ B → B ∧ A
F F │ T
F T │ T
T F │ T
T T │ T
A ∨ B → A is false when A = F, B = T
```

The second part is the useful one: since $A \vee B \to A$ is false in the row $A = F$, $B = T$, soundness guarantees that no proof exists. A Rust program can only check soundness proof by proof; in Lean we will prove it once and for all.

Normalizing proofs

The module `natded::normalize` implements the local-soundness reductions. Its core is a function that recognizes a detour at the root of a proof and removes it:

```
/// If `p` is itself a detour, removes it and names the reduction.
fn contract(p: &Proof) -> Option<(Proof, &'static str)> {
    match p {
        Proof::AndE1(q) => match &**q {
            // ∧I followed by ∧E1: keep the first component, drop the second.
            Proof::AndI(a, _) => Some(((**a).clone(), "∧I/∧E1")),
            _ => None,
        },
        Proof::AndE2(q) => match &**q {
            Proof::AndI(_, b) => Some(((**b).clone(), "∧I/∧E2")),
            _ => None,
        },
        Proof::ImpE(f, arg) => match &**f {
            // →I followed by →E: plug the argument into the body.
            Proof::ImpI { name, body, .. } => {
                Some((substitute(body, name, arg), "→I/→E"))
            }
            _ => None,
        },
        Proof::OrE {
            scrutinee,
            left,
            right,
        } => match &**scrutinee {
```

```
            // vI followed by vE: we know which case happened; take it.
            Proof::OrI1(a, _) => Some((substitute(&left.1, &left.0, a), "vI1/vE")),
            Proof::OrI2(_, b) => Some((substitute(&right.1, &right.0, b), "vI2/vE")),
            _ => None,
        },
        // tonk-I followed by tonk-E is a detour, but there is nothing to
        // reduce it to: the premise proves A and we need B.
        _ => None,
    }
}
```

These are the reductions of the harmony section, line by line. →I/→E calls `substitute`, which replaces every use of the assumption by the argument proof, renaming inner assumptions where needed so that the argument's own open assumptions are not captured (substitution in the λ-calculus needs the same care, Chapter 7). `normalize` applies `contract` anywhere in the tree until no detour is left. The example `ch04_harmony.rs` proves $A \wedge B \vdash B \wedge A$ through a lemma:

```
// A ∧ B ⊢ B ∧ A, proved by first proving a general lemma A ∧ B → B ∧ A
// (→I) and then applying it to the assumption `u` (→E): a detour.
let lemma = imp_i("x", f("A ∧ B"), and_i(and_e2(hyp("x")), and_e1(hyp("x"))));
let with_detour = imp_e(lemma, hyp("u"));
let hyps = [("u", f("A ∧ B"))];
let before = checker.check(&hyps, &with_detour).expect("correct");
print!("{before}");
println!("not subformulas: {}", show(&foreign_formulas(&before)));
```

```
A ∧ B ⊢ B ∧ A                    →E
├─ A ∧ B ⊢ A ∧ B → B ∧ A         →I x
│  └─ A ∧ B, A ∧ B ⊢ B ∧ A       ∧I
│     ├─ A ∧ B, A ∧ B ⊢ B        ∧E2
│     │  └─ A ∧ B, A ∧ B ⊢ A ∧ B  Hyp x
│     └─ A ∧ B, A ∧ B ⊢ A        ∧E1
│        └─ A ∧ B, A ∧ B ⊢ A ∧ B  Hyp x
└─ A ∧ B ⊢ A ∧ B                  Hyp u
not subformulas: {A ∧ B → B ∧ A}
```

The lemma $A \wedge B \to B \wedge A$ does not occur in the sequent, and `foreign_formulas` reports it: the subformula property fails. (Notice also that the context of the lemma's body contains $A \wedge B$ twice, once as `u` and once as `x`: contexts are lists.) One reduction later:

```
let result = normalize(&with_detour, 100);
println!("reductions: {:?}", result.steps);
let after = checker.check(&hyps, &result.proof).expect("still correct");
print!("{after}");
println!("not subformulas: {}", show(&foreign_formulas(&after)));
assert_eq!(before.sequent(), after.sequent());
```

```
reductions: ["→I/→E"]
A ∧ B ⊢ B ∧ A         ∧I
├─ A ∧ B ⊢ B          ∧E2
│  └─ A ∧ B ⊢ A ∧ B   Hyp u
└─ A ∧ B ⊢ A          ∧E1
   └─ A ∧ B ⊢ A ∧ B   Hyp u
not subformulas: {}
```

The normal proof is the direct one, it proves the same sequent, and it uses only subformulas. The checker re-checks it: reduction preserves correctness, which is the logical form of a property you will meet as *preservation* in Chapter 9.

Adding tonk

Now the experiment: `Checker::with_tonk()` also accepts Prior's two rules.

```
// ⊤ ⊢ ⊤ tonk ⊥ by tonk-I, then ⊥ by tonk-E.
let absurd = tonk_e(tonk_i(top_i(), Formula::Bot));
match checker.check(&[], &absurd) {
    Ok(_) => unreachable!("intuitionistic logic has no tonk"),
    Err(e) => println!("{e}"),
}
let tonk_checker = Checker::with_tonk();
let d = tonk_checker
    .check(&[], &absurd)
    .expect("accepted with tonk");
print!("{d}");

// Now every formula has a proof: wrap the proof of ⊥ in ⊥E.
let anything = bot_e(absurd.clone(), f("A ∧ ¬A"));
let d = tonk_checker
    .check(&[], &anything)
    .expect("accepted with tonk");
println!("accepted: {}", d.sequent());
```

```
error in tonk-E at root:
  this rule is not part of intuitionistic natural deduction
⊢ ⊥             tonk-E
└─ ⊢ ⊤ tonk ⊥   tonk-I
   └─ ⊢ ⊤       ⊤I
accepted: ⊢ A ∧ ¬A
```

The ordinary checker stops at the first rule it does not know, tonk-E at the root. The extended checker accepts a proof of $\vdash \bot$, and with one more $\bot$E a proof of $A \wedge \neg A$. Two diagnoses, one semantic and one proof-theoretic:

```
// Soundness is gone: ⊥ is false in every row of every truth table...
assert!(counterexample(&[], &Formula::Bot).is_some());
// ...and no truth table for tonk could rescue it.
let tables = all_tables();
let intro = tables.iter().filter(|t| tonk_intro_sound(t)).count();
let elim = tables.iter().filter(|t| tonk_elim_sound(t)).count();
let both = tables
    .iter()
    .filter(|t| tonk_intro_sound(t) && tonk_elim_sound(t))
    .count();
println!(
    "truth tables for tonk: {intro} fit tonk-I, {elim} fit tonk-E, {both} fit both"
);

// Local soundness fails: the detour tonk-I/tonk-E cannot be reduced.
let stuck = normalize(&absurd, 100);
println!("reductions: {:?}, stuck: {:?}", stuck.steps, stuck.stuck);
```

```
truth tables for tonk: 4 fit tonk-I, 4 fit tonk-E, 0 fit both
reductions: [], stuck: ["tonk-I/tonk-E"]
```

Of the 16 possible truth tables for a binary connective, four make tonk-I sound (those with $A \mathbin{tonk} B$ true whenever A is, such as disjunction) and four make tonk-E sound (those with it false whenever B is, such as conjunction), but none does both: this is the Rust twin of Lean's `no_tonk_table`. And the normalizer finds the detour but has no reduction for it. The checker itself has no bug; it faithfully implements a broken logic.

4.8 A deep embedding in Lean

The Rust checker tests soundness on examples. To prove it for every derivation, we need derivations to be mathematical objects. So far we have used Lean's own `Prop` and `∧`: a **shallow embedding**, in which the logic we study is Lean's logic. That is convenient for proving things *in* the logic, but not *about* it: there is no Lean statement "this proposition has no natural-deduction proof", because Lean's propositions are not data we can inspect. A **deep embedding** defines the syntax and the rules as ordinary inductive types, as Chapter 3 did for `Arith`; then formulas and derivations are values, and we can prove theorems about all of them. The code is in `code/lean/Book/Ch04/Deep.lean`.

Formulas and derivability

Formulas are an inductive type, with atoms numbered rather than named, and negation defined:

```
/-- Propositional formulas. Atoms are numbered: `var 0`, `var 1`, ... -/
inductive Formula where
  | var : Nat → Formula
  | top : Formula
  | bot : Formula
  | and : Formula → Formula → Formula
  | or  : Formula → Formula → Formula
  | imp : Formula → Formula → Formula
  deriving DecidableEq, Repr

/-- Negation is not a constructor: `¬A` abbreviates `A → ⊥`. -/
def Formula.neg (A : Formula) : Formula := .imp A .bot
```

```
/-- Three atoms to write examples with. -/
def P : Formula := .var 0
def Q : Formula := .var 1
def R : Formula := .var 2
```

The judgment $\Gamma \vdash A$ becomes an inductive predicate with one constructor per rule, the same technique as the typing judgment `HasType` of Chapter 3:

```
/-- `Proves Γ A` is the hypothetical judgment `Γ ⊢ A` of intuitionistic natural
deduction, with one constructor per rule. The context is a list, and the newest
assumption is at its head: the `A :: Γ` of the code is the `Γ, A` of the text. -/
inductive Proves : List Formula → Formula → Prop where
  | hyp   {Γ A}     : A ∈ Γ → Proves Γ A
  | topI  {Γ}       : Proves Γ .top
  | botE  {Γ C}     : Proves Γ .bot → Proves Γ C
  | andI  {Γ A B}   : Proves Γ A → Proves Γ B → Proves Γ (.and A B)
  | andE1 {Γ A B}   : Proves Γ (.and A B) → Proves Γ A
  | andE2 {Γ A B}   : Proves Γ (.and A B) → Proves Γ B
  | orI1  {Γ A B}   : Proves Γ A → Proves Γ (.or A B)
  | orI2  {Γ A B}   : Proves Γ B → Proves Γ (.or A B)
  | orE   {Γ A B C} : Proves Γ (.or A B) → Proves (A :: Γ) C → Proves (B :: Γ) C →
                      Proves Γ C
  | impI  {Γ A B}   : Proves (A :: Γ) B → Proves Γ (.imp A B)
  | impE  {Γ A B}   : Proves Γ (.imp A B) → Proves Γ A → Proves Γ B
```

Compare each constructor with its rule: the arguments are the premises, the result type is the conclusion, and the implicit arguments in braces are the metavariables. The context is a list, and an assumption added by $\to$I or $\vee$E goes at the front: `A :: Γ` in the code is Γ, A on paper. A value of type `Proves Γ A` *is* a derivation tree.

Derivations as terms and as tactic scripts

Here is $A \wedge B \vdash B \wedge A$ once more, now as a value of the deep embedding:

```
/-- The derivation of `P ∧ Q ⊢ Q ∧ P`, written as a term. Each `.hyp` needs a
proof that the formula is in the context; `List.Mem.head _` says "it is the
first element". -/
theorem and_comm_derivation : Proves [.and P Q] (.and Q P) :=
  .andI (.andE2 (.hyp (List.Mem.head _))) (.andE1 (.hyp (List.Mem.head _)))
```

The term has the shape of the tree: `.andI` at the root, `.andE2` and `.andE1` below it, `.hyp` at the leaves. Each `.hyp` needs a proof that the formula is in the context; `List.Mem.head _` says "it is the first element of the list". The same derivation can be built bottom-up with tactics, applying one constructor at a time:

```
/-- The same derivation under no assumptions, built bottom-up with `apply`.
Elimination rules have a formula in their premise that does not occur in their
conclusion (the `A` of ∧E₂ below), so we must supply it with `(A := P)`. -/
theorem and_comm_closed : Proves [] (.imp (.and P Q) (.and Q P)) := by
  apply Proves.impI           -- context is now [P ∧ Q]
  apply Proves.andI
  · apply Proves.andE2 (A := P)
    apply Proves.hyp
    simp                      -- P ∧ Q ∈ [P ∧ Q]
  · apply Proves.andE1 (B := Q)
    apply Proves.hyp
    simp
```

`apply Proves.impI` turns the goal `Proves [] (P ∧ Q → Q ∧ P)` into its premise `Proves [P ∧ Q] (Q ∧ P)`. After `apply Proves.andI`, there are two goals. For the first, `apply Proves.andE2 (A := P)` says "this Q comes from a conjunction $P \wedge Q$": we must name the left conjunct `P` ourselves, because ∧E2 has a formula in its premise that does not occur in its conclusion, and Lean cannot guess it. That is a general feature of elimination rules, and it is why searching for natural-deduction proofs is harder than searching in the sequent calculus. Finally `apply Proves.hyp` leaves the membership goal `P ∧ Q ∈ [P ∧ Q]`, which `simp` proves by simplifying it to `True`.

Truth tables in Lean

The truth-table semantics is a recursive function into `Bool`:

```
/-- The truth-table value of a formula, given a value for every atom. -/
def eval (v : Nat → Bool) : Formula → Bool
  | .var n   => v n
  | .top     => true
  | .bot     => false
  | .and A B => eval v A && eval v B
  | .or A B  => eval v A || eval v B
  | .imp A B => !eval v A || eval v B
```

A valuation `v : Nat → Bool` gives a truth value to every atom. To check a formula with n atoms we need the 2^n rows of its table:

```
/-- All valuations of the atoms `0, ..., n - 1` (the other atoms are `false`). -/
def valuations : Nat → List (Nat → Bool)
  | 0     => [fun _ => false]
  | n + 1 => (valuations n).flatMap fun v =>
      [v, fun k => if k = n then true else v k]

/-- A formula whose atoms are among `0, ..., n - 1` is a tautology when every
```

```
row of its truth table is `true`. -/
def isTautology (n : Nat) (A : Formula) : Bool :=
  (valuations n).all fun v => eval v A
```

```
/-- info: true -/
#guard_msgs in
#eval isTautology 2 (.imp (.imp P Q) (.imp (.or P R) (.or Q R)))

/-- info: false -/
#guard_msgs in
#eval isTautology 2 (.imp (.or P Q) P)

/-- info: true -/
#guard_msgs in
#eval isTautology 1 (.or P (.neg P))
```

(`flatMap` maps each valuation to a list and concatenates the lists, like Rust's `flat_map`; each existing row is kept and copied with atom `n` set to `true`.)

Soundness

Now the theorem. It says: if `Γ ⊢ A` is derivable, then every valuation that makes all of `Γ` true makes `A` true. The proof is by **induction on the derivation**, one case per rule, the same kind of proof as the soundness of the `Arith` checker in Chapter 3:

```
/-- **Soundness.** If `Γ ⊢ A` is derivable, then every valuation that makes all
of `Γ` true makes `A` true. By induction on the derivation: one case per rule. -/
theorem soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ v : Nat → Bool, (∀ B ∈ Γ, eval v B = true) → eval v A = true := by
  induction h with
  | hyp hmem =>
    -- The conclusion is one of the assumptions, which are all true.
    intro v hΓ
    exact hΓ _ hmem
  | topI =>
    intro v _
    rfl
  | botE _ ih =>
    -- The premise would make `false = true`: this case cannot happen.
    intro v hΓ
    have hbot := ih v hΓ
    simp [eval] at hbot
  | andI _ _ ih₁ ih₂ =>
    intro v hΓ
    simp [eval, ih₁ v hΓ, ih₂ v hΓ]
  | andE1 _ ih =>
    intro v hΓ
    have hand := ih v hΓ
    simp [eval] at hand
    exact hand.1
  | andE2 _ ih =>
    intro v hΓ
    have hand := ih v hΓ
    simp [eval] at hand
    exact hand.2
  | orI1 _ ih =>
    intro v hΓ
    simp [eval, ih v hΓ]
  | orI2 _ ih =>
    intro v hΓ
    simp [eval, ih v hΓ]
  | orE _ _ _ ih ihA ihB =>
    -- The disjunction is true, so one disjunct is; use the matching branch,
    -- whose context is Γ extended with that (true) disjunct.
    intro v hΓ
```

```
    have hor := ih v hΓ
    simp [eval] at hor
    rcases hor with hA | hB
    · apply ihA v
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hA
      · exact hΓ C hC
    · apply ihB v
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hB
      · exact hΓ C hC
  | @impI Γ A B _ ih =>
    -- If A is false the implication is true; if A is true, the premise
    -- (with A added to the context) gives B.
    intro v hΓ
    cases hA : eval v A with
    | false => simp [eval, hA]
    | true =>
      have hB : eval v B = true := by
        apply ih v
        intro C hC
        rcases List.mem_cons.mp hC with rfl | hC
        · exact hA
        · exact hΓ C hC
      simp [eval, hB]
  | impE _ _ ih₁ ih₂ =>
    intro v hΓ
    have himp := ih₁ v hΓ
    have hA := ih₂ v hΓ
    simp [eval, hA] at himp
    exact himp
```

`induction h with` produces one goal per constructor of `Proves`, and in each case the induction hypotheses (`ih`, `ih₁`, ...) state the theorem for the premises. Let us go through the cases.

- **hyp**: the conclusion is in the context, and every formula in the context is true by assumption `hΓ`.
- **topI**: `eval v .top` is `true` by definition, so `rfl` closes the goal.
- **botE**: the induction hypothesis says `eval v .bot = true`; `simp [eval] at hbot` rewrites it to `false = true`, recognizes it as impossible and closes the goal. The premise of ⊥E can never be true, so its conclusion is never at risk.
- **andI, orI1, orI2**: `simp` unfolds `eval` and uses the induction hypotheses to finish the Boolean computation.
- **andE1, andE2**: the hypothesis says `eval v A && eval v B = true`; `simp` turns it into a conjunction of two equations, and we take the one we need.
- **orE**: the main hypothesis says one of the disjuncts is true. In each case we apply the induction hypothesis for the corresponding branch, which needs every formula in the *extended* context to be true: the new head is the true disjunct, and the rest is `Γ`. The `rcases ... with rfl | hC` splits membership in `A :: Γ` into "it is `A`" and "it is in `Γ`".
- **impI**: the conclusion is `!eval v A || eval v B`. If `A` is false, this is true already. If `A` is true, the context `A :: Γ` is all true, so the induction hypothesis gives `eval v B = true`. The syntax `@impI Γ A B _ ih` names the implicit arguments so that we can talk about `A` and `B`.
- **impE**: both premises are true, and `simp` combines them.

There is no clever idea in this proof, and that is the point: soundness is checked rule by rule, and a rule like tonk would leave a case that no tactic can close.

What soundness buys

Soundness turns a semantic fact into a statement about proofs. The first consequence is consistency, this time with a complete proof:

```
/-- **Consistency.** `⊥` cannot be derived from no assumptions: under any
valuation, `eval v ⊥ = false`. -/
theorem consistency : ¬ Proves [] .bot := by
  intro h
  have := soundness h (fun _ => false) (by simp)
  simp [eval] at this
```

A derivation of `⊥` would make `eval v .bot` true for any valuation, but it is `false`. The same argument shows that specific formulas have no derivation, something no amount of failed searching could establish:

```
/-- An atom is not derivable: the valuation that makes every atom false is a
counterexample. -/
theorem atom_not_derivable : ¬ Proves [] P := by
  intro h
  have := soundness h (fun _ => false) (by simp)
  simp [eval, P] at this

/-- `(P ∨ Q) → P` is not derivable: take `P` false and `Q` true. -/
theorem or_imp_not_derivable : ¬ Proves [] (.imp (.or P Q) P) := by
  intro h
  have := soundness h (fun n => n = 1) (by simp)
  simp [eval, P, Q] at this
```

(`fun n => n = 1` is a valuation: the proposition `n = 1` is decidable, so Lean turns it into the `Bool` that is `true` exactly for atom 1, which is `Q`.)

Soundness has a limit, and it is a deep one. The converse, **completeness**, would say that every formula true in all rows is derivable. For the rules of this chapter it is false:

```
/-- Excluded middle is a tautology: true in every row of its truth table. So
soundness alone cannot show that it is *not* derivable. (Chapter 5 shows it
with a Kripke model.) -/
theorem lem_tautology (v : Nat → Bool) : eval v (.or P (.neg P)) = true := by
  cases h : v 0 <;> simp [eval, Formula.neg, P, h]
```

Excluded middle, $A \vee \neg A$, is a tautology, yet it has no derivation in our system. Truth tables cannot show this, because it is true in every row. Chapter 5 builds two finer semantics, Kripke models and Heyting algebras, and uses them to prove in Lean that `Proves [] (.or P P.neg)` is impossible.

4.9 Classical logic, in one rule

The system of this chapter is **intuitionistic** natural deduction. Classical logic, the logic of truth tables, is obtained by adding one more rule, for instance *reductio ad absurdum*:

$$\frac{\Gamma, \neg A \vdash \bot}{\Gamma \vdash A}\ \mathsf{RAA}$$

"To prove A, assume it is false and derive a contradiction." With RAA, $A \vee \neg A$ becomes derivable, and the system is complete for truth tables. RAA is neither an introduction nor an elimination: it belongs to no connective and falls outside the harmony story. That is not an argument against classical logic, but it is a first sign that the two systems differ in character. Chapter 5 is devoted to the difference, and to why Lean's core logic is intuitionistic while `Classical.em` sits in its library.

4.10 Putting it together: one formula, four checks

Let us follow a single formula through everything in this chapter:

$$(A \to B) \to A \vee C \to B \vee C$$

"If A implies B, then A or C implies B or C." In programming terms it is `map` for the left side of a sum.

On paper. Work bottom-up. Two $\to$I steps assume $A \to B$ (call it g) and $A \vee C$ (call it d), leaving the goal $B \vee C$. Split d with $\vee$E. In the case A (call it a), apply g to a with $\to$E and finish with $\vee$I1; in the case C, use $\vee$I2. With $\Gamma = A \to B, A \vee C$:

$$\dfrac{\Gamma \vdash A \vee C \qquad \dfrac{\dfrac{\Gamma, A \vdash A \to B \qquad \Gamma, A \vdash A}{\Gamma, A \vdash B}\,{\to}\mathsf{E}}{\Gamma, A \vdash B \vee C}\,{\vee}\mathsf{I}_1 \qquad \dfrac{\Gamma, C \vdash C}{\Gamma, C \vdash B \vee C}\,{\vee}\mathsf{I}_2}{\Gamma \vdash B \vee C}\,{\vee}\mathsf{E}$$

followed by two $\to$I steps, with the leaves closed by Hyp.

In Lean, shallowly. The tactic proof follows the paper proof step by step:

```
/-- The walkthrough formula of the chapter, `(A → B) → A ∨ C → B ∨ C`. -/
theorem or_map (p q r : Prop) : (p → q) → p ∨ r → q ∨ r := by
  intro hpq h              -- →I twice
  cases h with             -- ∨E on h : p ∨ r
  | inl hp =>
    left                   -- ∨I1
    exact hpq hp           -- →E
  | inr hr =>
    right                  -- ∨I2
    exact hr
```

In Lean, deeply. The same derivation as a value of `Proves`. The eliminated formulas must be given explicitly for $\vee$E (which disjunction we split) and $\to$E (which antecedent we prove):

```
/-- `⊢ (P → Q) → (P ∨ R) → (Q ∨ R)`: two →I, one ∨E, two ∨I, one →E. -/
theorem or_map_derivation : Proves [] (.imp (.imp P Q) (.imp (.or P R) (.or Q R))) := by
  apply Proves.impI                    -- assume P → Q
  apply Proves.impI                    -- assume P ∨ R
  apply Proves.orE (A := P) (B := R)   -- case analysis on P ∨ R
  · exact Proves.hyp (by simp)         -- the disjunction is an assumption
  · apply Proves.orI1                  -- case P: prove Q, then Q ∨ R
    apply Proves.impE (A := P)
    · exact Proves.hyp (by simp)       -- P → Q
    · exact Proves.hyp (by simp)       -- P
  · apply Proves.orI2                  -- case R: prove R, then Q ∨ R
    exact Proves.hyp (by simp)
```

In Rust. The proof tree for the checker:

```
// ⊢ (A → B) → A ∨ C → B ∨ C: a case analysis (∨E) on the disjunction `d`.
let or_map = imp_i(
```

```
    "g",
    f("A → B"),
    imp_i(
        "d",
        f("A ∨ C"),
        or_e(
            hyp("d"),
            "a", // case A: apply g, then choose the left side
            or_i1(imp_e(hyp("g"), hyp("a")), f("C")),
            "c", // case C: choose the right side
            or_i2(f("B"), hyp("c")),
        ),
    ),
);
let d2 = checker
    .prove(&[], &f("(A → B) → A ∨ C → B ∨ C"), &or_map)
    .expect("a correct proof");
print!("{d2}");
```

```
⊢ (A → B) → A ∨ C → B ∨ C                →I g
└─ A → B ⊢ A ∨ C → B ∨ C                 →I d
   └─ A → B, A ∨ C ⊢ B ∨ C                ∨E a c
      ├─ A → B, A ∨ C ⊢ A ∨ C             Hyp d
      ├─ A → B, A ∨ C, A ⊢ B ∨ C          ∨I1
      │  └─ A → B, A ∨ C, A ⊢ B           →E
      │     ├─ A → B, A ∨ C, A ⊢ A → B    Hyp g
      │     └─ A → B, A ∨ C, A ⊢ A        Hyp a
      └─ A → B, A ∨ C, C ⊢ B ∨ C          ∨I2
         └─ A → B, A ∨ C, C ⊢ C           Hyp c
```

The printed derivation is the paper tree, upside down and with every context written out: ∨E discharges `a` in its second premise and `c` in its third, and the contexts grow accordingly. Four representations, one proof; `#print or_map` would show a fifth, a program.

4.11 Common confusions

"**⊢ and → are the same thing.**" They are interderivable, by →I and →E, but not the same. $\Gamma \vdash A$ is a judgment about A, made in the metalanguage; $A \to B$ is a formula, an object the judgment talks about. The distinction is what lets Chapter 6 change the rules for contexts without touching any connective.

"**A tautology must be provable.**" For classical logic, yes: that is completeness. For the intuitionistic rules of this chapter, no: $A \vee \neg A$ is a tautology without a derivation. Soundness goes one way only, and Chapter 5 gives the semantics for which intuitionistic logic is complete.

"**`exfalso`, `contradiction` and ⊥E are classical reasoning.**" They are not. ⊥E is an ordinary intuitionistic rule: it uses a proof of ⊥ that you already have. Classical reasoning is RAA, which *produces* a proof of A from a refutation of $\neg A$. The difference between "from a contradiction, anything" and "if $\neg A$ is contradictory, then A" is the subject of Chapter 5.

"**Harmony is about truth.**" Harmony is a property of rules and proofs, checked without any semantics: can detours be removed, can formulas be rebuilt? That it also guarantees consistency (through normalization) is a theorem, not the definition.

"**Tactic proofs are a different kind of proof.**" A tactic script is a program that builds a proof term, and `#print` shows the term. The kernel never sees the tactics.

4.12 Summary

- A proposition is syntax; a judgment such as "A true" or $\Gamma \vdash A$ is an assertion about it. Inference rules relate judgments.
- Natural deduction gives each connective introduction rules (how to prove it) and elimination rules (how to use it). $\to$I and $\vee$E discharge assumptions; negation is $A \to \bot$ and inherits the rules of $\to$.
- Tree notation (with discharge labels) and sequent notation (with explicit contexts) carry the same information.
- Harmony: eliminations take out exactly what introductions put in. Local soundness is β-reduction, local completeness is η-expansion, and the shape of each elimination follows from its introductions. Prior's `tonk` fails local soundness and proves $\bot$.
- Normal derivations have the subformula property; normalization implies consistency. The sequent calculus with cut elimination is the same logic organized for proof search.
- In Lean, propositions are types in `Prop` and proofs are terms; tactics apply rules backwards, and `#print` shows the resulting term. Inductive propositions get harmonious eliminators for free.
- A deep embedding makes formulas and derivations data. We proved soundness for truth tables by induction on derivations, and derived consistency and non-derivability results from it. Completeness fails: excluded middle is a tautology without a derivation.
- The Rust crate `natded` checks proof trees, explains its rejections, tests soundness against truth tables and normalizes proofs; with `tonk` enabled it accepts a proof of $\bot$.

4.13 Exercises

1. Derive $\vdash A \to (B \to A)$ and $\vdash (A \to B \to C) \to (A \wedge B \to C)$ in tree notation with discharge labels. In the first, one assumption is discharged but never used: which? Then check both proofs with `natded`. *Hint:* the unused assumption is discharged by a $\to$I whose label appears on no leaf. Chapter 6 calls this weakening.
2. State local soundness and local completeness for $\top$. One of them is vacuous: which, and why? Write the expansion for $\top$ as a Lean equation on `Unit` and prove it. *Hint:* $\top$ has an introduction and no elimination, so no detour can form. The equation is in `Harmony.lean`.
3. In Lean, using only the tactics of this chapter, prove `p ∨ (q ∧ r) → (p ∨ q) ∧ (p ∨ r)`, `¬(p ∨ q) → ¬p ∧ ¬q` and `(p → q) → (p → r) → p → q ∧ r`. Then `#print` one of them and match the term against your tactic script. *Hint:* `rintro (hp | ⟨hq, hr⟩)` splits the first hypothesis; for the second, `intro h` and then `constructor`, and in each branch `intro` and `apply h`.
4. In the deep embedding, build a value of `Proves [] (.imp P P.neg.neg)` (the derivation of $A \to \neg\neg A$ from this chapter). Then prove `¬ Proves [] (.imp (.imp P Q) (.imp Q P))` using `soundness`. *Hint:* unfold `Formula.neg` mentally: you need two `impI` and one `impE`. For the second part,

find a row of the truth table that falsifies the formula and pass it to `soundness` as in `or_imp_not_derivable`.

5. Add the rule $\dfrac{\Gamma \vdash \top}{\Gamma \vdash C}$ ("$\top$E") to a copy of the `natded` checker and derive $\vdash \bot$. Which harmony condition does the rule violate? Compare it with `tonk`. *Hint:* $\top$I puts nothing in, so this elimination takes out more than was put in: the detour $\top$I/$\top$E has nothing to reduce to.
6. Consider a "conjunction" $A \circ B$ with the introduction rule of $\wedge$ and only the elimination $\wedge$E1. Show that it is locally sound but not locally complete, that adding it does not make $\bot$ derivable, and say what is lost. *Hint:* read $A \circ B$ as $A \wedge B$ to translate every derivation, so consistency is inherited; but B can never be recovered from $A \circ B$.

CHAPTER 5

Intuitionism and the Meaning of Proof

Chapter 4 ended with a puzzle. The formula $A \vee \neg A$, "every proposition is true or false", holds in every row of its truth table, yet the rules of natural deduction cannot derive it. And in Chapter 1, `#print axioms` showed that a proof by the `grind` tactic relied on something called `Classical.choice`, while a hand-written proof did not. Both observations point to the same fact: there are two respectable notions of proof, and the logic we have been using is the more demanding one.

That logic is called **intuitionistic** (or constructive). In it, to prove "A or B" you must say which, and to prove "there exists an x" you must produce one. This chapter explains where the idea comes from, what it costs (some classical laws fail), what it buys (proofs that carry information, which is why it is the logic of programming), and why classical logic is not wrong but sits inside it. We then build two semantics in which intuitionistic logic, unlike truth tables, is exactly right: Kripke models and Heyting algebras. With them we finally prove in Lean that excluded middle is not derivable in the system of Chapter 4.

In this chapter you will learn:

- the Brouwer–Heyting–Kolmogorov (BHK) interpretation, in which a proof of $A \vee B$ is a proof of one side *together with the information which*, and a proof of $A \to B$ is a method;
- which classical laws fail constructively (excluded middle, double-negation elimination, Peirce's law, one De Morgan law), and what you gain in exchange: the disjunction and existence properties;
- why classical logic is not wrong: the double-negation translation, and the constructive proof of $\neg\neg(A \vee \neg A)$;
- how negation as $A \to \bot$ looks in Rust, with empty types, `Infallible` and the never type;
- how Lean separates the two worlds: `Classical.em`, `Classical.byContradiction`, `Decidable`, and `#print axioms`;
- Kripke semantics, a two-world countermodel to excluded middle checked by `decide`, and an exhaustive countermodel search in Rust;
- Heyting algebras, built from scratch in Lean and then taken from Mathlib, and a first look at the categorical reading of logic.

5.1 Brouwer and the meaning of "true"

Intuitionism began in the early twentieth century with the Dutch mathematician L. E. J. Brouwer, as a philosophy of mathematics rather than a logic. Its central claim is that mathematical truth *is* provability: to assert A is to possess a construction, a mental proof, of A, not to report a fact about a realm that exists independently of what we can establish. Brouwer was suspicious of formal logic; the formal system was written down by his student Arend Heyting around 1930, and the logic it defines is exactly the logic of Chapter 4's rules.

The consequence for logic follows quickly. If asserting "A or B" means having a construction of A or having a construction of B, then asserting "$A \vee \neg A$ for every A" means claiming a method that, for every proposition, either proves it or refutes it. Nobody has such a method; for many open problems we have neither. So excluded middle cannot be asserted *in general*.

A classic example shows what a classical proof can leave out. *There exist irrational numbers a and b such that a^b is rational.* Proof: consider $\sqrt{2}^{\sqrt{2}}$. If it is rational, take $a = b = \sqrt{2}$. If not, take $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$, since then $a^b = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2$. The proof is classically impeccable, and at the end you still do not know which pair works. It used excluded middle on a question it never answered. An intuitionist does not say the proof is wrong; they say it proves something weaker than it claims: that the non-existence of such a pair is absurd.

> **NOTE**
>
> Intuitionism does not claim that excluded middle is *false*. As we will prove, $\neg\neg(A \vee \neg A)$ is intuitionistically provable: excluded middle can never be refuted. It is just not available as a general principle, because asserting a disjunction requires knowing which side holds.

5.2 The BHK interpretation

The **Brouwer–Heyting–Kolmogorov interpretation** says what counts as a proof of each kind of formula. Kolmogorov contributed a reading of propositions as *problems* and proofs as *solutions*, which is the same idea in a programmer's vocabulary. The table adds a column for Rust and one for Lean.

Formula	A proof is	Rust	Lean
$A \wedge B$	a proof of A and a proof of B	`(A, B)`	`⟨ha, hb⟩`
$A \vee B$	a tag saying which side, and a proof of it	`enum Either<A, B>`	`Or.inl ha`
$A \to B$	a method turning proofs of A into proofs of B	`fn(A) -> B`	`fun ha => ...`
$\top$	the trivial proof	`()`	`True.intro`
$\bot$	none exists	`enum Void {}`	(no constructor)
$\neg A$	a method turning proofs of A into proofs of $\bot$	`fn(A) -> Void`	`fun ha => ...`
$\forall x.\, A(x)$	a method giving, for each x, a proof of $A(x)$	generic `fn` (on types)	`fun x => ...`
$\exists x.\, A(x)$	a witness x and a proof of $A(x)$	`impl Trait` (on types)	`⟨x, h⟩`

Two observations. First, the BHK column *is* the type column: the explanation of the connectives that intuitionists gave in the 1930s is a description of data types, decades before Curry and Howard made the correspondence formal. That is why Chapter 10 can be short: the work is done here. (The Rust entries for the quantifiers are only approximations, since Rust cannot quantify over values; Chapter 10 discusses this.)

Second, the $\vee$ and $\exists$ rows are exactly where classical logic parts company. Both demand that a proof contain information: which side, which witness. A classical proof of $A \vee B$ may contain no such information, as the $\sqrt{2}$ example shows. Everything else in this chapter follows from taking those two rows seriously.

5.3 What fails, and what you gain

Excluded middle and its relatives

These classical laws have no intuitionistic proof:

Law	Formula
excluded middle	$A \vee \neg A$
double-negation elimination	$\neg\neg A \to A$
Peirce's law	$((A \to B) \to A) \to A$
one De Morgan law	$\neg(A \wedge B) \to \neg A \vee \neg B$
material implication	$(A \to B) \to \neg A \vee B$
converse contraposition	$(\neg B \to \neg A) \to (A \to B)$

Each has an easy classical proof. In Lean, classical reasoning is available through two library functions:

```
/-- info: Classical.em (p : Prop) : p ∨ ¬p -/
#guard_msgs in
#check Classical.em

/-- info: Classical.byContradiction {p : Prop} (h : ¬p → False) : p -/
#guard_msgs in
#check Classical.byContradiction

/-- info: axiom Classical.choice.{u} : {α : Sort u} → Nonempty α → α -/
#guard_msgs in
#print Classical.choice
```

`Classical.em` is excluded middle, and `Classical.byContradiction` is the rule RAA of Chapter 4: to prove `p`, show that `¬p` leads to `False`. Both are theorems, proved from the **axiom** `Classical.choice` (with help from two other axioms that we meet in a moment). Choice says that from the knowledge that a type is nonempty (`Nonempty α`, a proposition) you can obtain an element of it. That is precisely the step from "something exists" to "here it is" that BHK refuses. With these, the classical laws are short (the code is in `code/lean/Book/Ch05/Constructive.lean`):

```
/-- Double-negation elimination: proof by contradiction. -/
theorem dne (p : Prop) : ¬¬p → p := by
  intro hnnp
```

```
  apply Classical.byContradiction   -- goal: ¬p → False
  intro hnp
  exact hnnp hnp

/-- Peirce's law, by cases on excluded middle. -/
theorem peirce (p q : Prop) : ((p → q) → p) → p := by
  intro h
  cases Classical.em p with
  | inl hp => exact hp
  | inr hnp => exact h (fun hp => absurd hp hnp)

/-- The De Morgan law that fails constructively. `by_cases` splits on
`Classical.em`. -/
theorem not_and_or (p q : Prop) : ¬(p ∧ q) → ¬p ∨ ¬q := by
  intro h
  by_cases hp : p
  · right
    intro hq
    exact h ⟨hp, hq⟩
  · left
    exact hp
```

`dne` applies `byContradiction`, which turns the goal `p` into `¬p → False`. `peirce` splits on `Classical.em p`. `not_and_or` uses the tactic `by_cases`, which does the same split with a friendlier syntax. Now the tool that tells proofs apart, `#print axioms`, which you met in Chapter 1:

```
/-- info: 'Ch05.nn_em' does not depend on any axioms -/
#guard_msgs in
#print axioms nn_em

/-- info: 'Ch05.not_or_iff' does not depend on any axioms -/
#guard_msgs in
#print axioms not_or_iff

/-- info: 'Ch05.dne' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms dne

/-- info: 'Ch05.not_and_or' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms not_and_or
```

`nn_em` and `not_or_iff`, proved below, use no axioms at all: they are constructions in the BHK sense. `dne` and `not_and_or` depend on `Classical.choice` (and on `propext` and `Quot.sound`, two further axioms that Lean's library uses to derive excluded middle from choice; they are not classical by themselves). Whenever you want to know whether a Lean proof is constructive, this is the command to run.

All or nothing

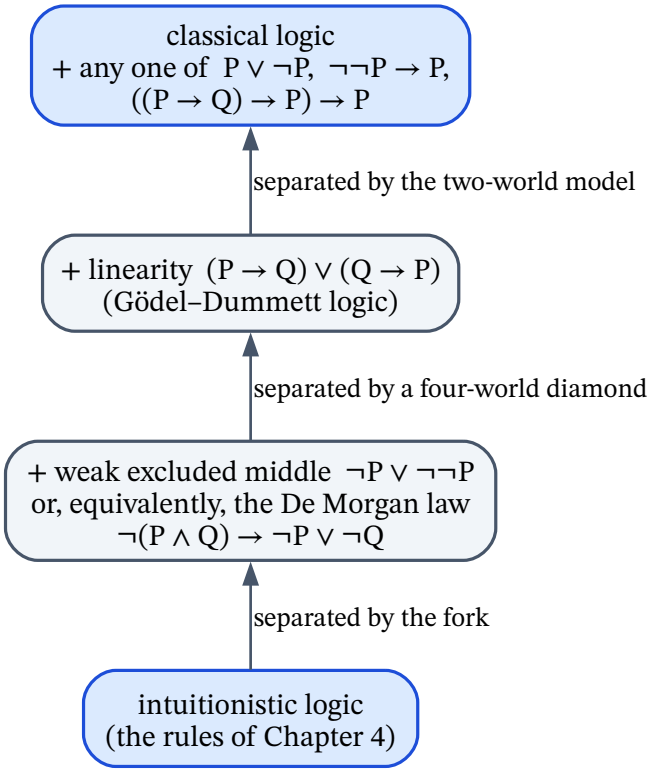
Excluded middle, double-negation elimination and Peirce's law are not three separate principles. Over intuitionistic logic, each one, assumed for all propositions, implies the others. In Lean the implications are constructive proofs that take the principle as a hypothesis:

```
/-- If excluded middle holds for every proposition, so does double-negation
elimination... -/
theorem dne_of_em (em : ∀ p : Prop, p ∨ ¬p) : ∀ p : Prop, ¬¬p → p := by
  intro p hnnp
  cases em p with
  | inl hp => exact hp
  | inr hnp => exact absurd hnp hnnp
```

```
/-- ...and conversely: apply double-negation elimination to `nn_em`. -/
theorem em_of_dne (dne : ∀ p : Prop, ¬¬p → p) : ∀ p : Prop, p ∨ ¬p :=
  fun p => dne (p ∨ ¬p) (nn_em p)

/-- Peirce's law implies excluded middle: take `q := False` and the
proposition `p ∨ ¬p`. -/
theorem em_of_peirce (peirce : ∀ p q : Prop, ((p → q) → p) → p) :
    ∀ p : Prop, p ∨ ¬p := by
  intro p
  apply peirce (p ∨ ¬p) False   -- goal: (p ∨ ¬p → False) → p ∨ ¬p
  intro k                       -- k : ¬(p ∨ ¬p)
  exact (nn_em p k).elim

/-- info: 'Ch05.em_of_peirce' does not depend on any axioms -/
#guard_msgs in
#print axioms em_of_peirce
```

The first proof splits on the excluded-middle hypothesis. The second applies double-negation elimination to `nn_em`, the constructive proof of $\neg\neg(p \vee \neg p)$ that we study below. The third instantiates Peirce's law with `q := False`: its premise `(p ∨ ¬p → False) → p ∨ ¬p` is again discharged by `nn_em`. And `#print axioms` confirms that these implications are themselves constructive. Adding any one of the three principles to the rules of Chapter 4 gives full classical logic; there is no "slightly classical" logic made of just one of them.

The De Morgan law is different, and the difference is easy to miss. It is equivalent not to excluded middle but to the weaker **weak excluded middle**, $\neg A \vee \neg\neg A$. Between intuitionistic and classical logic there are in fact infinitely many **intermediate logics**, and the two weaker principles above define two of them:



A few logics between intuitionistic and classical logic. Each step adds a principle; the label says which Kripke model separates the two logics.

The models in the labels are the subject of the Kripke section below, where a Rust program finds them.

What survives

Much of everyday logic survives. All of these are constructive:

```
/-- Double-negation introduction: a proof of `p` refutes every refutation. -/
theorem dni (p : Prop) : p → ¬¬p :=
  fun hp hnp => hnp hp

/-- Triple negation collapses to single negation. -/
theorem tne (p : Prop) : ¬¬¬p → ¬p :=
  fun hnnnp hp => hnnnp (dni p hp)

/-- Contraposition, in the direction that holds. -/
theorem contra (p q : Prop) : (p → q) → (¬q → ¬p) :=
  fun hpq hnq hp => hnq (hpq hp)

/-- Two De Morgan laws hold in both directions or in one. -/
theorem not_or_iff (p q : Prop) : ¬(p ∨ q) ↔ ¬p ∧ ¬q :=
  ⟨fun h => ⟨fun hp => h (Or.inl hp), fun hq => h (Or.inr hq)⟩,
   fun ⟨hnp, hnq⟩ h => h.elim hnp hnq⟩

theorem not_and_of_or_not (p q : Prop) : ¬p ∨ ¬q → ¬(p ∧ q) :=
  fun h ⟨hp, hq⟩ => h.elim (fun hnp => hnp hp) (fun hnq => hnq hq)
```

Read them as programs. `dni` takes a proof `hp` and a refutation `hnp` and applies the second to the first. `tne` turns a refutation of $\neg\neg p$ into a refutation of p by feeding it `dni p hp`: triple negation collapses to single negation, although double negation does not collapse to nothing. `contra` is composition. In `not_or_iff`, a refutation of $p \vee q$ yields refutations of each side by composing with `Or.inl` and

`Or.inr`, and conversely a pair of refutations handles both cases of a disjunction. `not_and_of_or_not` is the direction of De Morgan's law that needs no information: if one side is refuted, the conjunction is.

Constructive	Classical only
$A \to \neg\neg A$	$\neg\neg A \to A$
$\neg\neg\neg A \leftrightarrow \neg A$	$A \vee \neg A$
$\neg(A \wedge \neg A)$	$\neg(A \wedge B) \to \neg A \vee \neg B$
$\neg\neg(A \vee \neg A)$	$(\neg B \to \neg A) \to (A \to B)$
$\neg(A \vee B) \leftrightarrow \neg A \wedge \neg B$	$(A \to B) \to \neg A \vee B$
$(\neg A \vee \neg B) \to \neg(A \wedge B)$	$((A \to B) \to A) \to A$
$(A \to B) \to (\neg B \to \neg A)$	

A pattern is visible: the constructive laws produce negations, and the classical ones produce disjunctions or extract a positive fact from negative information.

What you gain: disjunction and existence

Here is the payoff, which an introduction that only lists failures hides.

- **Disjunction property.** If $\vdash A \vee B$ is derivable intuitionistically (with no assumptions), then $\vdash A$ or $\vdash B$ is derivable.
- **Existence property.** If $\vdash \exists x.\, A(x)$ is derivable in intuitionistic predicate logic, then $\vdash A(t)$ is derivable for some term t.

Both follow from normalization (Chapter 4). A normal derivation with no assumptions must end with an introduction rule, by the same argument that proved consistency: an elimination at the root would lead up a chain of eliminations to an assumption, and there are none. For $A \vee B$ that introduction is $\vee$I1 or $\vee$I2, and its premise is a proof of the chosen side. You will meet this argument again in Chapter 9 as the *canonical forms* lemma: a closed value of a type has the shape that the type's introduction rules allow (a value of type `Bool` is `true` or `false`).

Classical logic has neither property. $\vdash A \vee \neg A$ is classically derivable for an atom A, but neither $\vdash A$ nor $\vdash \neg A$ is (each is false in some row of the truth table). A classical proof of a disjunction may carry no information about which side holds, and a classical existence proof may contain no witness. Compare two Lean proofs:

```
/-- A constructive existence proof contains its witness. -/
theorem exists_square : ∃ n : Nat, n * n = 49 := ⟨7, rfl⟩

/-- The drinker: in a nonempty pub there is someone such that, if they drink,
everyone drinks. The proof asks whether everyone drinks, a question it cannot
answer by computation, and so it never says who the person is. -/
theorem drinker {α : Type} [Inhabited α] (drinks : α → Prop) :
    ∃ x, drinks x → ∀ y, drinks y := by
  by_cases h : ∀ y, drinks y
  · exact ⟨default, fun _ => h⟩       -- anyone will do
  · obtain ⟨y, hy⟩ := Classical.not_forall.mp h   -- someone does not drink
    exact ⟨y, fun hdy => absurd hdy hy⟩

/-- info: 'Ch05.exists_square' does not depend on any axioms -/
#guard_msgs in
#print axioms exists_square
```

```
/-- info: 'Ch05.drinker' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms drinker
```

`exists_square` contains its witness, 7, and uses no axioms. `drinker` is a famous puzzle, usually called the drinker paradox: in any nonempty pub there is someone such that, if they drink, everyone drinks. The proof asks whether everyone drinks. If so, anyone will do; if not, `Classical.not_forall` (a classical lemma: from "not everyone drinks", someone who does not) produces a non-drinker, who satisfies the claim vacuously. The proof is correct, and it never tells you who the person is, because the question it asked has no computable answer.

This is precisely why intuitionistic logic is the logic of programming, and why every proof assistant with computational content (Lean's core, Rocq, Agda) is built on it. A program that has computed a value of type `Result<T, E>` knows which variant it holds; a program must *have* the `T` to return it. If you want to run a proof, the proof needs the witness.

> **NOTE**
>
> In Lean, `∃` and `∨` live in `Prop`, where proofs are not meant to be run, and Lean deliberately forbids extracting the witness of an `∃` into ordinary data (Chapter 10 explains this *elimination restriction*). The informative versions are types of data: `Σ` and subtypes for existence, `Sum` and `Decidable` for disjunction. We meet `Decidable` below.

5.4 Classical logic is not wrong

Two facts keep the story balanced.

The double-negation translation

The first is a theorem of Glivenko:

- **Glivenko's theorem.** A propositional formula A is a classical tautology if and only if $\neg\neg A$ is intuitionistically derivable.

So classical propositional logic *embeds* into intuitionistic logic: put $\neg\neg$ in front and every classical theorem becomes an intuitionistic one. Gödel and Gentzen extended the idea to predicate logic with a translation that inserts double negations more carefully (in front of atoms, disjunctions and existentials). Nothing is lost in terms of what can be proved. What changes is what a proof *gives you*: a proof of $\neg\neg A$ is weaker information than a proof of A.

The most important instance is $\neg\neg(A \vee \neg A)$. Here is its proof in tactic mode:

```
/-- Excluded middle cannot be refuted. The refutation `k` is called twice. -/
theorem nn_em (p : Prop) : ¬¬(p ∨ ¬p) := by
  intro k              -- k : ¬(p ∨ ¬p); goal: False
  apply k              -- goal: p ∨ ¬p
  right                -- commit to the right disjunct; goal: ¬p
  intro hp             -- hp : p; goal: False
  apply k              -- use k a second time; goal: p ∨ ¬p
```

```
  left                -- now we can commit to the left disjunct
  exact hp
```

Follow the goals. We must prove `¬¬(p ∨ ¬p)`, which unfolds to `¬(p ∨ ¬p) → False`, so `intro k` gives us a refutation `k` of excluded middle and leaves the goal `False`. The only way to produce `False` is to use `k`, so `apply k`, and now we must prove `p ∨ ¬p`. We cannot know whether `p` holds, so we commit to the right side, `¬p`, which is a promise: "give me a `p` and I will produce `False`". `intro hp` accepts such a `p`. Now we have a proof of `p` in hand, so we go back to `k` a second time and give it the left side, `Or.inl hp`. The term is short:

```
/--
info: theorem Ch05.nn_em : ∀ (p : Prop), ¬¬(p ∨ ¬p) :=
fun p k => k (Or.inr fun hp => k (Or.inl hp))
-/
#guard_msgs in
#print nn_em
```

The proof answers "¬p", and if that answer is ever challenged with an actual proof of `p`, it changes its mind and answers "p" using the very proof that was used to challenge it. Notice that `k` is used twice. That will matter when we write the same program in Rust.

Double negation is a monad

The second fact explains Glivenko's theorem computationally. Double negation behaves like a *monad*, a structure you will meet properly in Chapter 20: there is a way to put a value in (`pure`) and a way to chain computations (`bind`):

```
/-- `pure` for double negation: the same term as `dni`. -/
theorem nn_pure {p : Prop} (hp : p) : ¬¬p :=
  fun k => k hp

/-- `bind` for double negation: to refute a refutation of `q`, run the
double-negated `p` with a continuation that feeds each `p` to `f`. -/
theorem nn_bind {p q : Prop} (nnp : ¬¬p) (f : p → ¬¬q) : ¬¬q :=
  fun k => nnp (fun hp => f hp k)

/-- Inside `¬¬`, excluded middle is available, so the classical De Morgan law
holds there, with no axioms. -/
theorem nn_not_and_or (p q : Prop) : ¬¬(¬(p ∧ q) → ¬p ∨ ¬q) := by
  apply nn_bind (nn_em p)   -- borrow `p ∨ ¬p` inside the double negation
  intro hem
  apply nn_pure             -- now prove the formula itself
  intro h
  cases hem with
  | inl hp => exact Or.inr (fun hq => h ⟨hp, hq⟩)
  | inr hnp => exact Or.inl hnp

/-- info: 'Ch05.nn_not_and_or' does not depend on any axioms -/
#guard_msgs in
#print axioms nn_not_and_or
```

`nn_pure` is `dni` again. `nn_bind` says: to refute a refutation `k` of `q`, run the double-negated `p` with a continuation that sends each `p` to `f`, and passes `k` along. The payoff is `nn_not_and_or`: inside the double negation we may borrow `p ∨ ¬p` from `nn_em` and then reason classically, and the result is a constructive proof of the double negation of the classical De Morgan law, with no axioms. The same trick gives the double negations of the other classical laws:

```
/-- The double negations of the other classical laws are constructive too. -/
theorem nn_dne (p : Prop) : ¬¬(¬¬p → p) :=
  nn_bind (nn_em p) fun hem =>
    nn_pure fun hnnp => hem.elim id (fun hnp => absurd hnp hnnp)

theorem nn_peirce (p q : Prop) : ¬¬(((p → q) → p) → p) :=
  nn_bind (nn_em p) fun hem =>
    nn_pure fun h => hem.elim id (fun hnp => h (fun hp => absurd hp hnp))
```

Programmers know $\neg\neg A$ under another name. Replace $\bot$ by an arbitrary answer type R: then $(A \to R) \to R$ is a value in **continuation-passing style**, a computation that, instead of returning an A, takes a consumer of As. Double-negation elimination would be a way to get the A out of a continuation-passing computation without supplying a consumer, which is impossible in a pure language. Timothy Griffin observed that classical logic corresponds to programs with first-class continuations: Peirce's law is the type of the control operator `call/cc`, and a classical proof by contradiction is a program that captures its continuation and jumps back to it. Classical logic is perfectly computational; it computes with control effects rather than with pure functions.

We can run `nn_em` in Rust, with `Void` replaced by an answer type so that the program can actually return something. The full program is `code/rust/examples/ch05_negation.rs`:

```
/// A sum type: a value of `A ∨ B` says which side it is on.
enum Either<L, R> {
    Left(L),
    Right(R),
}

/// "Refutations" relative to an answer type `R`: consumers of `A`s. With
/// `R = Void` this is `¬A`; with an ordinary `R` we can run it.
type Neg<A, R> = Rc<dyn Fn(A) -> R>;

/// `¬¬(A ∨ ¬A)`, the double negation of excluded middle. Given a consumer `k`
/// of `A ∨ ¬A`, first answer "¬A" with a refutation. If `k` ever applies that
/// refutation to some `a`, go back and call `k` again, this time with
/// `Left(a)`. The consumer is used twice, which is why it is an `Rc<dyn Fn>`.
fn nn_em<A: 'static, R: 'static>(k: Neg<Either<A, Neg<A, R>>, R>) -> R {
    let k_again = Rc::clone(&k);
    let refutation: Neg<A, R> = Rc::new(move |a| k_again(Either::Left(a)));
    k(Either::Right(refutation))
}
```

`Neg<A, R>` is a consumer of `A`s producing an `R`. The consumer `k` is called twice, so it must be shareable: an `Rc<dyn Fn>` rather than a `Box<dyn FnOnce>`. In the logic of Chapter 4, using an assumption twice is free; in Rust it costs an `Rc`, a first glimpse of Chapter 6. Here is a consumer that, when handed a refutation, tests it on the number 42:

```
// Run ¬¬(A ∨ ¬A) with A = u32 and answers of type String.
let log = Rc::new(RefCell::new(Vec::new()));
let seen = Rc::clone(&log);
let k: Neg<Either<u32, Neg<u32, String>>, String> =
    Rc::new(move |answer| match answer {
        Either::Left(a) => {
            seen.borrow_mut().push(format!("k received Left({a})"));
            format!("A holds, witness {a}")
        }
        Either::Right(refutation) => {
            seen.borrow_mut()
                .push("k received Right(refutation)".to_string());
            seen.borrow_mut()
```

```
                .push("k applies the refutation to 42".to_string());
            refutation(42)
        }
    });
let result = nn_em(k);
for line in log.borrow().iter() {
    println!("{line}");
}
println!("result: {result}");
```

```
k received Right(refutation)
k applies the refutation to 42
k received Left(42)
result: A holds, witness 42
```

The trace is the "change of mind" in action: `k` first receives `Right(refutation)`, applies the refutation to 42, and the refutation calls `k` again with `Left(42)`. The final answer comes from the second call.

5.5 Negation, truth and falsity

Three states of knowledge

Classically, every proposition is true or false. Intuitionistically there are three situations, and conflating them is the commonest error in reading intuitionistic statements:

Situation	Meaning	Rust analogy
$\vdash A$	we have a construction of A	a value of type `A` exists
$\vdash \neg A$	we have a refutation of A	a function `A -> Void` exists
neither	A is open: no judgment can be asserted	we know nothing

$\neg A$ is a positive claim that needs positive evidence: a method that turns any proof of A into absurdity, which amounts to showing that A has no proof. "A is not provable" is a different statement altogether, a fact *about* the logic (like "excluded middle has no derivation"), which cannot even be written as a formula inside it. Excluded middle would say that the third row is always empty.

Empty types in Rust

The Rust side of negation is the empty type. An `enum` with no variants has no values, and matching on it needs no arms:

```
/// Falsity: a type with no values at all.
#[derive(Debug)]
enum Void {}

/// Rule ⊥E: from a value of the empty type, a value of any type. The `match`
/// has no arms because there is no case to handle.
fn absurd<T>(v: Void) -> T {
    match v {}
}
```

A refutation is a function into `Void`. Here are three of the constructive laws as Rust functions, plus $\neg\bot$:

```
/// `¬A`: a refutation of `A`, a function that turns any `A` into a `Void`.
type Not<A> = Box<dyn FnOnce(A) -> Void>;
```

```
/// `A → ¬¬A`: given an `a`, refute every refutation of `A` by feeding it `a`.
fn dni<A: 'static>(a: A) -> Not<Not<A>> {
    Box::new(move |refutation: Not<A>| refutation(a))
}

/// `(A → B) → (¬B → ¬A)`: contraposition is composition.
fn contrapose<A: 'static, B: 'static>(
    f: impl FnOnce(A) -> B + 'static,
    not_b: Not<B>,
) -> Not<A> {
    Box::new(move |a| not_b(f(a)))
}

/// `¬¬¬A → ¬A`: triple negation collapses.
fn tne<A: 'static>(nnn: Not<Not<Not<A>>>) -> Not<A> {
    Box::new(move |a| nnn(dni(a)))
}

/// `¬⊥`: falsity is refutable, by the identity function.
fn not_false() -> Not<Void> {
    Box::new(|v| v)
}
```

These functions can be built but never run to completion, since that would produce a `Void`; the example only constructs them, and the compiler's acceptance is the point. (`Box<dyn FnOnce>` is used because each refutation is called at most once; `nn_em` above needed more.)

The standard library has its own empty type, `std::convert::Infallible`, used for operations that cannot fail. The compiler knows that empty types have no values and uses it: a pattern for the impossible case can be omitted.

```
/// A name can be parsed from any string, so parsing cannot fail: the error
/// type is `Infallible`, the standard library's empty enum.
struct Name(String);

impl FromStr for Name {
    type Err = Infallible;
    fn from_str(s: &str) -> Result<Name, Infallible> {
        Ok(Name(s.trim().to_lowercase()))
    }
}

fn parse_name(s: &str) -> Name {
    // The pattern `Ok(name)` is irrefutable: `Err(e)` would need an `e` of an
    // empty type, so the compiler knows that case cannot occur.
    let Ok(name) = s.parse::<Name>();
    name
}

/// An `Option` of an empty type can only be `None`, and the match needs no
/// `Some` arm.
fn only_none(x: Option<Void>) -> &'static str {
    match x {
        None => "None",
    }
}
```

`let Ok(name) = s.parse::<Name>();` is accepted as an irrefutable pattern because the `Err` variant would need a value of type `Infallible`. This is ⊥E used by the exhaustiveness checker: the `Err` case is handled "by absurdity", with zero arms. Finally, Rust's built-in empty type is the never type `!`:

```rust
/// `!` is Rust's built-in empty type. A function returning `!` never returns,
/// and a `!` can stand in for a value of any type (⊥E, built into the language).
fn give_up(reason: &str) -> ! {
    panic!("giving up: {reason}")
}

fn digit_value(c: char) -> u32 {
    match c.to_digit(10) {
        Some(d) => d,
        None => give_up("not a digit"), // has type `!`, coerced to `u32`
    }
}
```

```
size_of::<Void>() = 0
size_of::<Infallible>() = 0
parse_name("  Ada ").0 = "ada"
only_none(None) = None
digit_value('7') = 7
```

Empty types have size zero: no bits are needed to store a value that never exists. The never type coerces to `u32` in `digit_value`, which is ⊥E built into the language. The difference from `Void` is that `!` is also the type of divergence: `loop {}` and `panic!()` have type `!`, so in Rust the empty type *does* have "proofs", and Rust is inconsistent as a logic. Chapter 10 returns to this.

5.6 Decidable propositions in Lean

Lean draws the line between constructive and classical reasoning with a class. `Decidable p` is a *type of data* recording which of `p` and `¬p` holds, together with the proof:

```
/--
info: inductive Decidable : Prop → Type
number of parameters: 1
constructors:
Decidable.isFalse : {p : Prop} → ¬p → Decidable p
Decidable.isTrue : {p : Prop} → p → Decidable p
-/
#guard_msgs in
#print Decidable
```

Compare with `p ∨ ¬p`. Both have two cases, but `Or` is a `Prop`, whose proofs Lean erases when it compiles, while `Decidable p` lives in `Type`: a value of it is a bit of information a program can branch on. `Decidable p` is the BHK reading of excluded middle for one particular `p`, and an instance of the class is a decision procedure. That is why `if` on a proposition needs one:

```
/-- `if` on a proposition needs a `Decidable` instance: a procedure that
says which side holds. For `n % 2 = 0` on `Nat`, Lean has one. -/
def parity (n : Nat) : String :=
  if n % 2 = 0 then "even" else "odd"

/-- info: "odd" -/
#guard_msgs in
#eval parity 7

-- With an arbitrary `p`, there is no procedure, and Lean says so.
/--
error: failed to synthesize instance of type class
  Decidable p

Hint: Type class instance resolution failures can be inspected with the `set_option
```

```
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
def choose (p : Prop) : Nat := if p then 1 else 0

/-- Asking for the instance as an argument moves the obligation to the caller. -/
def choose' (p : Prop) [Decidable p] : Nat := if p then 1 else 0

/-- info: 1 -/
#guard_msgs in
#eval choose' (3 < 5)
```

Equality of natural numbers is decidable, so `parity` compiles and runs. For an arbitrary `p`, Lean cannot synthesize an instance: there is no procedure that decides every proposition. `choose'` asks its caller for the instance instead, and `choose' (3 < 5)` gets one from the library. The `decide` tactic that you have used since Chapter 1 works the same way: it evaluates the `Decidable` instance of the goal and succeeds if the answer is `isTrue`.

Classical logic offers a way out, at a price:

```
/-- `Classical.propDecidable` makes every proposition "decidable" by the
axiom of choice, but the result cannot be run: there is no code behind it. -/
noncomputable def chooseClassical (p : Prop) : Nat :=
  have := Classical.propDecidable p
  if p then 1 else 0
```

`Classical.propDecidable` produces a `Decidable p` for every `p`, using `Classical.choice`. The definition type-checks, but Lean requires the keyword `noncomputable`: there is no code to generate, because the axiom of choice has no algorithm behind it. In Lean the boundary between constructive and classical mathematics is not a matter of style; it is the boundary between what can be run and what cannot.

5.7 Kripke semantics

Truth tables are too coarse for intuitionistic logic: excluded middle is true in every row. Saul Kripke's semantics, from the 1960s, replaces the single row with a *process* of acquiring knowledge.

Worlds and forcing

A **Kripke model** consists of:

- a set W of **worlds**, thought of as states of knowledge;
- a preorder $\leq$ on W (reflexive and transitive), where $w \leq w'$ means "w' is a possible future of w, in which we know at least as much";
- for each world, the set of atoms known at that world, which must be **monotone**: once known, an atom stays known in every future.

The **forcing** relation $w \Vdash A$, "at w we know A", is defined by recursion on A:

- $w \Vdash P$ iff the atom P is known at w;
- $w \Vdash \top$ always, and $w \Vdash \bot$ never;
- $w \Vdash A \wedge B$ iff $w \Vdash A$ and $w \Vdash B$;
- $w \Vdash A \vee B$ iff $w \Vdash A$ or $w \Vdash B$;

- $w \Vdash A \to B$ iff for every $w' \geq w$, if $w' \Vdash A$ then $w' \Vdash B$.

The clauses for $\wedge$, $\vee$, $\top$ and $\bot$ are those of truth tables, evaluated at one world. Implication is the only clause that looks at other worlds: asserting $A \to B$ now is a commitment that holds *whatever we learn later*. For negation this gives

$$w \Vdash \neg A \quad \text{iff} \quad \text{no} \;\; w' \geq w \;\; \text{forces} \;\; A$$

$\neg A$ holds when A is refuted now *and will stay refuted forever*. That is much stronger than "A is not known yet", and the gap between the two is where intuitionistic logic lives. Similarly, $w \Vdash \neg\neg A$ iff every future of w can be extended to a world that forces A: A is not available yet, but it can never be ruled out.

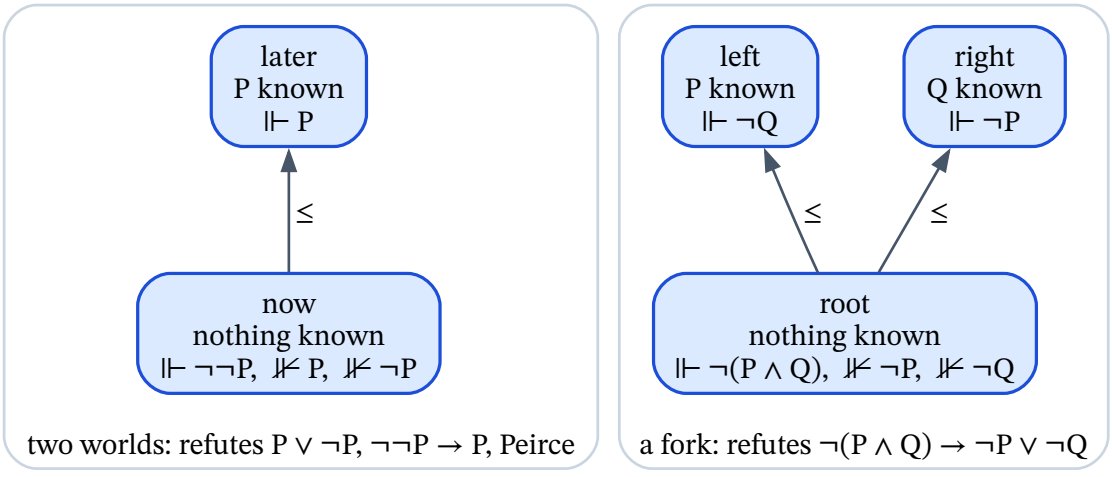
A sequent $\Gamma \vdash A$ is **valid** if in every Kripke model, every world that forces all formulas of Γ forces A. Two theorems connect the semantics with the rules of Chapter 4:

- **Soundness.** Every derivable sequent is valid.
- **Completeness** (Kripke). Every valid sequent is derivable.

We will prove soundness in Lean. Completeness is harder and we only state it; the Rust search below gives evidence for it on small examples.

A two-world countermodel

Take two worlds, *now* and *later*, with $now \leq later$. Nothing is known now; later, the atom P has been established.



Two Kripke countermodels. Left: a two-world model in which P becomes known later; it refutes excluded middle, double-negation elimination and Peirce's law at "now". Right: a fork in which P and Q become known on different branches; it refutes the De Morgan law.

Check excluded middle at *now*:

- $now \nVdash P$, because P is not known yet;
- $now \nVdash \neg P$, because a future world, *later*, forces P;

- so $now \nVdash P \vee \neg P$.

That is the whole counterexample, and the whole intuition: P is undecided now and becomes true later, so neither disjunct can be asserted now. The same model refutes double-negation elimination: every future of *now* can reach a world forcing P, so $now \Vdash \neg\neg P$, yet $now \nVdash P$. By soundness, none of these formulas is derivable.

Notice what happens with a single world: the quantifier "for every $w' \geq w$" ranges over w alone, and the clause for $\to$ becomes the truth table. A one-world Kripke model *is* a row of a truth table, and classical logic is intuitionistic logic in a world where nothing more can ever be learned.

Kripke models in Lean

The file `code/lean/Book/Ch05/Kripke.lean` imports the deep embedding of Chapter 4 (`Book.Ch04.Deep`, from the same pair of chapters) and gives it a Kripke semantics. A model is a structure over a type of worlds with a preorder, which we take from Mathlib's class `Preorder` (it provides `≤`, `le_refl` and `le_trans`):

```
/-- A Kripke model over a preordered set of worlds `W`. Read `w ≤ w'` as
"`w'` is a possible future of `w`, where we know at least as much", and
`val w n` as "at world `w`, atom `n` is known to hold". Knowledge is never
lost: that is `mono`. -/
structure KripkeModel (W : Type) [Preorder W] where
  val : W → Nat → Bool
  mono : ∀ {w w' : W} {n : Nat}, w ≤ w' → val w n = true → val w' n = true
```

```
/-- The forcing relation `w ⊩ A`. Only implication looks at other worlds: it
must hold in every future, not just now. -/
def Forces (M : KripkeModel W) : W → Formula → Prop
  | w, .var n   => M.val w n = true
  | _, .top     => True
  | _, .bot     => False
  | w, .and A B => Forces M w A ∧ Forces M w B
  | w, .or A B  => Forces M w A ∨ Forces M w B
  | w, .imp A B => ∀ w', w ≤ w' → Forces M w' A → Forces M w' B
```

`Forces` is a `Prop`-valued function defined by recursion on the formula, one equation per clause. The negation clause comes out as expected:

```
/-- Unfolding the definition for negation: `w ⊩ ¬A` means that no future of
`w` forces `A`. -/
theorem forces_neg (M : KripkeModel W) (w : W) (A : Formula) :
    Forces M w A.neg ↔ ∀ w', w ≤ w' → ¬ Forces M w' A := by
  simp [Formula.neg, Forces]
```

The key lemma is monotonicity: forcing is never lost.

```
/-- **Monotonicity.** What is forced stays forced in every future world. By
induction on the formula; the case of implication uses transitivity. -/
theorem Forces.mono (M : KripkeModel W) {w w' : W} (hle : w ≤ w') :
    ∀ A : Formula, Forces M w A → Forces M w' A
  | .var _, h => M.mono hle h
  | .top, _ => trivial
  | .bot, h => h
  | .and A B, ⟨hA, hB⟩ => ⟨Forces.mono M hle A hA, Forces.mono M hle B hB⟩
  | .or A _, .inl hA => .inl (Forces.mono M hle A hA)
  | .or _ B, .inr hB => .inr (Forces.mono M hle B hB)
  | .imp _ _, h => fun w'' hle' hA => h w'' (le_trans hle hle') hA
```

This proof is a recursive function, defined by pattern matching on the formula and the proof of forcing, and Lean checks that every recursive call is on a smaller formula. Atoms are monotone by the model's `mono` field. For a conjunction, both parts move forward; for a disjunction, whichever part holds moves forward. For an implication, `h` promises B in every future of w that forces A; a future w'' of w' is also a future of w, by `le_trans`. That is where transitivity is needed.

Soundness is proved by induction on derivations, like the truth-table version of Chapter 4:

```
/-- **Soundness.** A derivable sequent is forced at every world of every
Kripke model that forces all of its assumptions. -/
theorem kripke_soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ {W : Type} [Preorder W] (M : KripkeModel W) (w : W),
      (∀ B ∈ Γ, Forces M w B) → Forces M w A := by
  induction h with
  | hyp hmem => intro W _ M w hΓ; exact hΓ _ hmem
  | topI => intro W _ M w _; trivial
  | botE _ ih => intro W _ M w hΓ; exact (ih M w hΓ).elim
  | andI _ _ ih₁ ih₂ => intro W _ M w hΓ; exact ⟨ih₁ M w hΓ, ih₂ M w hΓ⟩
  | andE1 _ ih => intro W _ M w hΓ; exact (ih M w hΓ).1
  | andE2 _ ih => intro W _ M w hΓ; exact (ih M w hΓ).2
  | orI1 _ ih => intro W _ M w hΓ; exact .inl (ih M w hΓ)
  | orI2 _ ih => intro W _ M w hΓ; exact .inr (ih M w hΓ)
  | orE _ _ _ ih ihA ihB =>
    intro W _ M w hΓ
    rcases ih M w hΓ with hA | hB
    · apply ihA M w
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hA
      · exact hΓ C hC
    · apply ihB M w
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hB
      · exact hΓ C hC
  | impI _ ih =>
    -- To force A → B at w, take any future w' forcing A. The assumptions,
    -- forced at w, are still forced at w' (monotonicity), so the premise
    -- applies at w' with A added to the context.
    intro W _ M w hΓ w' hle hA
    apply ih M w'
    intro C hC
    rcases List.mem_cons.mp hC with rfl | hC
    · exact hA
    · exact Forces.mono M hle C (hΓ C hC)
  | impE _ _ ih₁ ih₂ =>
    -- Use the implication at w itself, which is its own future (w ≤ w).
    intro W _ M w hΓ
    exact ih₁ M w hΓ w (le_refl w) (ih₂ M w hΓ)
```

Most cases are as before. The two interesting ones are the rules for implication. For **impI** we must show that w forces $A \to B$: take any future w' with `hle : w ≤ w'` that forces A. The induction hypothesis applies at w' with context `A :: Γ`, so we must show that w' forces everything in it: A by assumption, and each formula of `Γ` because it was forced at w and forcing is monotone. Without monotonicity, the proof would break exactly here. For **impE** we use the implication at w itself, which is its own future by `le_refl`.

Deciding forcing with decide

For a model with finitely many worlds, forcing is decidable: each quantifier "for every w'" ranges over a finite list. We say so with an instance. First a small class for types whose elements can be listed:

```
/-- A type whose elements can be listed. (Mathlib's `Fintype` says the same,
but checking its instances by `decide` drags classical lemmas into the proof;
this small class keeps every result in this file constructive.) -/
class Enumerable (W : Type) where
  elems : List W
  complete : ∀ w, w ∈ elems

/-- Over an enumerable type, "for all `w`" is decidable: check every element
of the list. -/
instance decForallEnum {W : Type} [Enumerable W] (p : W → Prop) [DecidablePred p] :
    Decidable (∀ w, p w) :=
  decidable_of_iff (∀ w ∈ Enumerable.elems, p w)
    ⟨fun h w => h w (Enumerable.complete w), fun h w _ => h w⟩
```

`decidable_of_iff` builds a decision procedure for `∀ w, p w` from one for the equivalent `∀ w ∈ elems, p w`, which core Lean can decide by checking the list. Then forcing is decidable by recursion on the formula:

```
/-- In a finite model with a decidable order, forcing is decidable: every
quantifier ranges over finitely many worlds. -/
instance decForces [Enumerable W] [DecidableRel (α := W) (· ≤ ·)] (M : KripkeModel W) :
    ∀ (w : W) (A : Formula), Decidable (Forces M w A)
  | w, .var n => inferInstanceAs (Decidable (M.val w n = true))
  | _, .top => inferInstanceAs (Decidable True)
  | _, .bot => inferInstanceAs (Decidable False)
  | w, .and A B =>
    have := decForces M w A
    have := decForces M w B
    inferInstanceAs (Decidable (Forces M w A ∧ Forces M w B))
  | w, .or A B =>
    have := decForces M w A
    have := decForces M w B
    inferInstanceAs (Decidable (Forces M w A ∨ Forces M w B))
  | w, .imp A B =>
    have := fun w' => decForces M w' A
    have := fun w' => decForces M w' B
    inferInstanceAs (Decidable (∀ w', w ≤ w' → Forces M w' A → Forces M w' B))
```

Each case asks Lean to find the instance for the unfolded clause (`inferInstanceAs`), after making the instances for the subformulas available with `have`. Now the two-world model. The worlds are a two-element type with its order:

```
/-- Two stages of knowledge: `now`, and `later`, when we may know more. -/
inductive Stage where
  | now | later
  deriving DecidableEq

/-- `now ≤ later`, and each stage is below itself. -/
def Stage.le : Stage → Stage → Bool
  | .now, _ => true
  | .later, .later => true
  | .later, .now => false

instance : Preorder Stage where
  le a b := Stage.le a b = true
  le_refl a := by cases a <;> rfl
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide

instance : DecidableRel (α := Stage) (· ≤ ·) :=
  fun a b => inferInstanceAs (Decidable (Stage.le a b = true))

instance : Enumerable Stage where
  elems := [.now, .later]
  complete w := by cases w <;> decide
```

```
/-- Nothing is known `now`; `later`, the atom `P` (number 0) has been
established. -/
def twoWorlds : KripkeModel Stage where
  val
    | .later, 0 => true
    | _, _ => false
  mono {w w' n} hle hv := by
    change Stage.le w w' = true at hle
    cases w <;> cases w' <;> simp_all [Stage.le]
```

The valuation is a pattern match: atom 0 (`P`) is known `later`, nothing else anywhere. Monotonicity is proved by checking every pair of worlds. And now `decide` checks the countermodel, clause by clause, by evaluating the decision procedure:

```
/-- `P` is not forced now, but it will be. -/
example : ¬ Forces twoWorlds .now P ∧ Forces twoWorlds .later P := by decide

/-- `¬P` is not forced now either: a future world forces `P`. -/
example : ¬ Forces twoWorlds .now P.neg := by decide

/-- So excluded middle fails now. -/
theorem lem_fails : ¬ Forces twoWorlds .now (.or P P.neg) := by decide

/-- `¬¬P` holds now (`P` is never refuted), but `¬¬P → P` does not:
double-negation elimination fails. -/
theorem dne_fails : Forces twoWorlds .now P.neg.neg ∧
    ¬ Forces twoWorlds .now (.imp P.neg.neg P) := by decide

/-- Peirce's law `((P → Q) → P) → P` fails too (`Q` is never known). -/
theorem peirce_fails : ¬ Forces twoWorlds .now (.imp (.imp (.imp P Q) P) P) := by decide
```

With soundness, each failure becomes a statement about the proof system of Chapter 4:

```
/-- Excluded middle is not derivable in the natural deduction of Chapter 4:
if it were, soundness would make it forced at `now`. -/
theorem lem_not_derivable : ¬ Proves [] (.or P P.neg) := by
  intro h
  exact lem_fails (kripke_soundness h twoWorlds .now (by simp))

theorem dne_not_derivable : ¬ Proves [] (.imp P.neg.neg P) := by
  intro h
  exact dne_fails.2 (kripke_soundness h twoWorlds .now (by simp))

theorem peirce_not_derivable : ¬ Proves [] (.imp (.imp (.imp P Q) P) P) := by
  intro h
  exact peirce_fails (kripke_soundness h twoWorlds .now (by simp))
```

If excluded middle had a derivation `h`, then `kripke_soundness h` would make it forced at `now` in `twoWorlds` (the empty context is trivially forced, which `simp` checks), contradicting `lem_fails`. This is the theorem that Chapter 4 could not prove with truth tables. And yet:

```
/-- Yet all three are tautologies: truth tables cannot tell them apart from
the derivable formulas. -/
example : isTautology 1 (.or P P.neg) = true ∧
    isTautology 1 (.imp P.neg.neg P) = true ∧
    isTautology 2 (.imp (.imp (.imp P Q) P) P) = true := by decide
```

all three are tautologies. And the proof that excluded middle is not derivable is itself constructive (`propext` and `Quot.sound` come from `simp`; there is no `Classical.choice`):

```
-- The proof that excluded middle is not derivable is itself constructive.
/-- info: 'Ch05.lem_not_derivable' depends on axioms: [propext, Quot.sound] -/
```

```
#guard_msgs in
#print axioms lem_not_derivable
```

Finally, the claim that classical logic is the one-world case is a theorem:

```
/-- The only possible order on a one-element type. -/
local instance : Preorder Unit where
  le _ _ := True
  le_refl _ := trivial
  le_trans _ _ _ _ _ := trivial

/-- A model with a single world. -/
def oneWorld (v : Nat → Bool) : KripkeModel Unit where
  val _ n := v n
  mono _ h := h

/-- In a one-world model, forcing is truth-table evaluation: the quantifier
over futures has only the present to range over. -/
theorem forces_oneWorld (v : Nat → Bool) :
    ∀ A : Formula, Forces (oneWorld v) () A ↔ eval v A = true
  | .var _ => Iff.rfl
  | .top => by simp [Forces, eval]
  | .bot => by simp [Forces, eval]
  | .and A B => by
    simp [Forces, eval, ← forces_oneWorld v A, ← forces_oneWorld v B]
  | .or A B => by
    simp [Forces, eval, ← forces_oneWorld v A, ← forces_oneWorld v B]
  | .imp A B => by
    have hA := forces_oneWorld v A
    have hB := forces_oneWorld v B
    simp only [Forces, eval]
    cases h : eval v A <;> simp_all
```

(`local instance` gives `Unit` its trivial order in this file only.) The proof goes formula by formula; for implication, the "for every future" clause has only the present to check, and `simp_all` reduces both sides to the same Boolean condition.

A fork

Two worlds in a line are not enough to refute every classical law. The De Morgan law $\neg(P \wedge Q) \to \neg P \vee \neg Q$ needs a world with two incomparable futures, as on the right of the figure above:

```
/-- Three worlds: a root and two incomparable futures. -/
inductive Fork where
  | root | left | right
  deriving DecidableEq

/-- The order: the root is below everything, and each world is below itself. -/
def Fork.le : Fork → Fork → Bool
  | .root, _ => true
  | .left, .left => true
  | .right, .right => true
  | _, _ => false

instance : Preorder Fork where
  le a b := Fork.le a b = true
  le_refl a := by cases a <;> rfl
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide

instance : DecidableRel (α := Fork) (· ≤ ·) :=
  fun a b => inferInstanceAs (Decidable (Fork.le a b = true))

instance : Enumerable Fork where
  elems := [.root, .left, .right]
  complete w := by cases w <;> decide
```

```
/-- `P` becomes known on the left branch, `Q` on the right branch. -/
def fork : KripkeModel Fork where
  val
    | .left, 0 => true
    | .right, 1 => true
    | _, _ => false
  mono {w w' n} hle hv := by
    change Fork.le w w' = true at hle
    cases w <;> cases w' <;> simp_all [Fork.le]
```

```
/-- The De Morgan law `¬(P ∧ Q) → ¬P ∨ ¬Q` fails at the root. -/
theorem de_morgan_fails :
    ¬ Forces fork .root (.imp (Formula.and P Q).neg (.or P.neg Q.neg)) := by decide

/-- Weak excluded middle `¬P ∨ ¬¬P` fails at the root. -/
theorem wlem_fails : ¬ Forces fork .root (.or P.neg P.neg.neg) := by decide

theorem de_morgan_not_derivable : ¬ Proves [] (.imp (Formula.and P Q).neg (.or P.neg
Q.neg)) :=
  fun h => de_morgan_fails (kripke_soundness h fork .root (by simp))
```

At the root, $P \wedge Q$ is refuted, because no world knows both. But $\neg P$ fails at the root (the left branch learns P) and $\neg Q$ fails too (the right branch learns Q). The root commits to the refutation of the conjunction without being able to say which conjunct is to blame.

Searching for countermodels in Rust

How did we know which models to try? We searched. The program `code/rust/examples/ch05_kripke.rs` enumerates every Kripke model with at most three worlds and two atoms, and for each formula looks for a world where it fails. Forcing is the definition, transcribed:

```
/// A finite Kripke model. Worlds are `0..n`; `le[w][v]` says that `v` is a
/// possible future of `w`; `val[atom][w]` says that the atom is known at `w`.
struct Model {
    n: usize,
    le: Vec<Vec<bool>>,
    val: Vec<Vec<bool>>,
}

/// The forcing relation `w ⊩ A`. Only implication (and so negation) looks
/// beyond the present world: it must hold in every future.
fn forces(m: &Model, w: usize, a: &Formula) -> bool {
    match a {
        Atom(i) => m.val[*i][w],
        Bot => false,
        Formula::And(a, b) => forces(m, w, a) && forces(m, w, b),
        Formula::Or(a, b) => forces(m, w, a) || forces(m, w, b),
        Formula::Imp(a, b) => (0..m.n)
            .filter(|&v| m.le[w][v])
            .all(|v| !forces(m, v, a) || forces(m, v, b)),
    }
}
```

The enumeration builds every partial order on n worlds (every subset of ordered pairs that is reflexive, antisymmetric and transitive) and every monotone valuation (every set of worlds closed upward, for each atom). Restricting to partial orders loses nothing: worlds that are below each other force the same formulas and can be merged.

```
/// Every partial order on `n` worlds: reflexive, antisymmetric, transitive.
fn partial_orders(n: usize) -> Vec<Vec<Vec<bool>>> {
```

```
    let pairs: Vec<(usize, usize)> = (0..n)
        .flat_map(|i| (0..n).map(move |j| (i, j)))
        .filter(|(i, j)| i != j)
        .collect();
    let mut orders = Vec::new();
    for bits in 0..1u32 << pairs.len() {
        let mut le = vec![vec![false; n]; n];
        for (i, row) in le.iter_mut().enumerate() {
            row[i] = true;
        }
        for (k, &(i, j)) in pairs.iter().enumerate() {
            le[i][j] = bits >> k & 1 == 1;
        }
        let antisymmetric = pairs.iter().all(|&(i, j)| !(le[i][j] && le[j][i]));
        let transitive = (0..n).all(|i| {
            (0..n).all(|j| (0..n).all(|k| !(le[i][j] && le[j][k]) || le[i][k]))
        });
        if antisymmetric && transitive {
            orders.push(le);
        }
    }
    orders
}

/// Every monotone valuation of one atom: a set of worlds closed upward
/// (once known, an atom stays known).
fn up_sets(le: &[Vec<bool>]) -> Vec<Vec<bool>> {
    let n = le.len();
    (0..1u32 << n)
        .map(|bits| (0..n).map(|w| bits >> w & 1 == 1).collect::<Vec<bool>>())
        .filter(|set| (0..n).all(|w| (0..n).all(|v| !(set[w] && le[w][v]) || set[v])))
        .collect()
}

/// Every model with 1 to `max_worlds` worlds and two atoms.
fn all_models(max_worlds: usize) -> Vec<Model> {
    let mut models = Vec::new();
    for n in 1..=max_worlds {
        for le in partial_orders(n) {
            let ups = up_sets(&le);
            for vp in &ups {
                for vq in &ups {
                    models.push(Model {
                        n,
                        le: le.clone(),
                        val: vec![vp.clone(), vq.clone()],
                    });
                }
            }
        }
    }
    models
}
```

```
for (name, law) in &classical {
    // `models` is ordered by size, so the first hit is a smallest one.
    let found = models
        .iter()
        .find_map(|m| (0..m.n).find(|&w| !forces(m, w, law)).map(|w| (m, w)));
    match found {
        Some((m, w)) => println!("{name:<17} {law}\n  {}", describe(m, w)),
        None => println!("{name:<17} {law}\n  no countermodel found"),
    }
}
```

```
models with at most 3 worlds and 2 atoms: 564

excluded middle   P v ¬P
```

```
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
double negation   ¬¬P → P
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
Peirce            ((P → Q) → P) → P
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
De Morgan         ¬(P ∧ Q) → ¬P ∨ ¬Q
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0
weak excl. middle ¬P ∨ ¬¬P
  3 worlds, 0≤1 0≤2, P@{1} Q@{}, fails at 0
linearity         (P → Q) ∨ (Q → P)
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0

holds in all 564 models: ¬¬(P ∨ ¬P)
holds in all 564 models: P → ¬¬P
holds in all 564 models: ¬¬¬P → ¬P
holds in all 564 models: ¬(P ∨ Q) → ¬P ∧ ¬Q
holds in all 564 models: ¬P ∨ ¬Q → ¬(P ∧ Q)
holds in all 564 models: (P → Q) ∧ (P → ¬Q) → ¬P
```

Read a line such as `2 worlds, 0≤1, P@{1} Q@{}, fails at 0` as "worlds 0 and 1 with $0 \leq 1$, P known at world 1, Q nowhere; the formula fails at world 0": our two-world model. Because the models are enumerated by size, each one reported is a smallest countermodel. Excluded middle, double-negation elimination and Peirce's law all fall to two worlds. The De Morgan law, weak excluded middle and the linearity axiom $(P \to Q) \vee (Q \to P)$ survive every model with two worlds and need the fork. That is the semantic shadow of the diagram of intermediate logics: the two-world chain validates the De Morgan law but refutes excluded middle, so the De Morgan law cannot imply excluded middle. (The search only tries atoms for P and Q, but in fact every instance of the De Morgan law holds on any frame in which two futures of a world always have a common future, as in a chain.) The six intuitionistic laws at the bottom hold in all 564 models, as soundness predicts. A search is evidence, not proof; the proofs are the Lean files.

5.8 Heyting algebras

Kripke models describe intuitionistic logic with worlds; the algebraic semantics describes it with an ordered set of truth values. For classical logic the truth values form a **Boolean algebra**: `true` and `false`, or more generally the subsets of a set, with $\cap$, $\cup$ and complement. Intuitionistic logic needs a weaker structure.

Definition

A **Heyting algebra** is a partially ordered set $(H, \leq)$ with

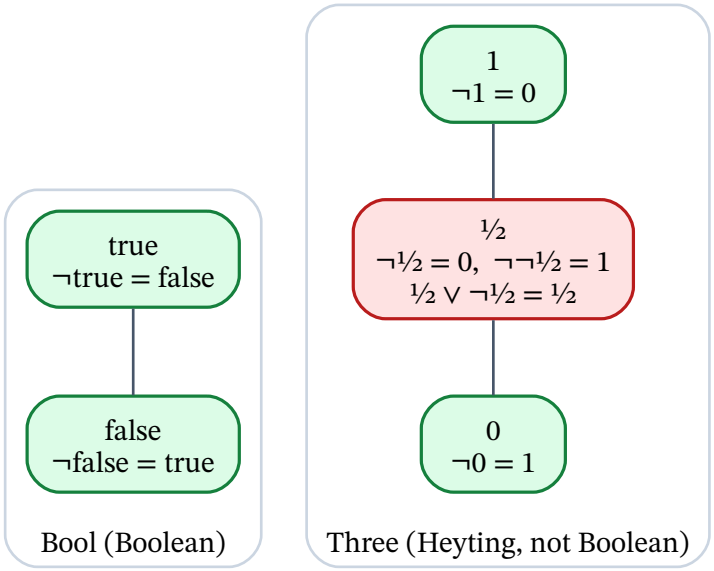
- a least element $\bot$ and a greatest element $\top$;
- binary meets $a \sqcap b$ (greatest lower bound, "and") and joins $a \sqcup b$ (least upper bound, "or");
- an implication $a \Rightarrow b$ characterized by **residuation**:

$$c \leq (a \Rightarrow b) \quad \text{iff} \quad c \sqcap a \leq b$$

Read $\leq$ as "entails". Residuation says that $a \Rightarrow b$ is the weakest element that, together with a, entails b: it is $\to$I and $\to$E rolled into one equivalence, exactly as $\Gamma, A \vdash B$ holds iff $\Gamma \vdash A \to B$. Negation is defined as in Chapter 4: $\neg a = a \Rightarrow \bot$, the largest element whose meet with a is $\bot$. In a Boolean

algebra, $a \Rightarrow b$ is $\neg a \sqcup b$ and $a \sqcup \neg a = \top$. In a Heyting algebra the second equation may fail: $\neg a$ is only a *pseudo-complement*.

The smallest example that is not Boolean is the three-element chain $0 < ½ < 1$: false, undecided, true. In a chain, meet is minimum, join is maximum, and $a \Rightarrow b$ is 1 if $a \le b$ and b otherwise. Then $\neg\, ½ = ½ \Rightarrow 0 = 0$, so $½ \sqcup \neg\, ½ = ½ \neq 1$: excluded middle fails at the undecided value. And $\neg\neg\, ½ = \neg 0 = 1 \neq ½$: double-negation elimination fails too.



The two-element Boolean algebra and the three-element Heyting chain, with their negations. The middle element is its own witness against excluded middle.

A Heyting algebra class from scratch

In `code/lean/Book/Ch05/Heyting.lean` we write the definition as a Lean class. We borrow only the *notation* from Lean and Mathlib: the classes `LE`, `Min`, `Max`, `HImp`, `Top` and `Bot` carry the symbols `≤`, `⊓`, `⊔`, `⇨`, `⊤` and `⊥` and no laws at all. Every law is a field of our class:

```
/-- A Heyting algebra: a bounded lattice with an implication `⇨` that is
related to `⊓` by residuation. (Inside `namespace Ch05` this name shadows
Mathlib's `HeytingAlgebra`, which we meet at the end of the file.) -/
class HeytingAlgebra (H : Type) extends LE H, Min H, Max H, HImp H, Top H, Bot H where
  -- `≤` is a partial order
  le_refl : ∀ a : H, a ≤ a
  le_trans : ∀ a b c : H, a ≤ b → b ≤ c → a ≤ c
  le_antisymm : ∀ a b : H, a ≤ b → b ≤ a → a = b
  -- `a ⊓ b` is the greatest lower bound ("and")
  inf_le_left : ∀ a b : H, a ⊓ b ≤ a
  inf_le_right : ∀ a b : H, a ⊓ b ≤ b
  le_inf : ∀ a b c : H, a ≤ b → a ≤ c → a ≤ b ⊓ c
  -- `a ⊔ b` is the least upper bound ("or")
  le_sup_left : ∀ a b : H, a ≤ a ⊔ b
  le_sup_right : ∀ a b : H, b ≤ a ⊔ b
  sup_le : ∀ a b c : H, a ≤ c → b ≤ c → a ⊔ b ≤ c
  -- `⊥` is the least element ("false"), `⊤` the greatest ("true")
  bot_le : ∀ a : H, ⊥ ≤ a
  le_top : ∀ a : H, a ≤ ⊤
```

```
  -- residuation: `b ⇨ c` is the largest `a` with `a ⊓ b ≤ c` ("implies")
  le_himp_iff : ∀ a b c : H, a ≤ b ⇨ c ↔ a ⊓ b ≤ c
```

`extends` makes our class include the notation classes, like a Rust trait with supertraits. Mathlib has a class with the same name; inside `namespace Ch05` ours shadows it, and we refer to Mathlib's as `_root_.HeytingAlgebra` at the end of the file. Negation is a definition:

```
/-- Negation is implication into the bottom element. -/
def neg {H : Type} [HeytingAlgebra H] (a : H) : H := a ⇨ ⊥
```

From the axioms alone we can prove the laws of intuitionistic negation:

```
/-- Meet is commutative (as an inequality; equality follows by antisymmetry). -/
theorem inf_comm_le (a b : H) : a ⊓ b ≤ b ⊓ a :=
  le_inf _ _ _ (inf_le_right a b) (inf_le_left a b)

/-- Modus ponens: `(a ⇨ b) ⊓ a ≤ b`. Residuation, read from right to left,
applied to `a ⇨ b ≤ a ⇨ b`. -/
theorem himp_inf_le (a b : H) : (a ⇨ b) ⊓ a ≤ b :=
  (le_himp_iff _ _ _).mp (le_refl _)

/-- `a ≤ ¬¬a`: unfolding, `a ≤ (a ⇨ ⊥) ⇨ ⊥`, which by residuation is
`a ⊓ (a ⇨ ⊥) ≤ ⊥`, modus ponens after swapping the meet. -/
theorem le_neg_neg (a : H) : a ≤ neg (neg a) :=
  (le_himp_iff _ _ _).mpr
    (le_trans _ _ _ (inf_comm_le _ _) (himp_inf_le a ⊥))

/-- Negation reverses the order. -/
theorem neg_anti {a b : H} (h : a ≤ b) : neg b ≤ neg a :=
  (le_himp_iff _ _ _).mpr
    (le_trans _ _ _
      (le_inf _ _ _ (inf_le_left _ _) (le_trans _ _ _ (inf_le_right _ _) h))
      (himp_inf_le b ⊥))

/-- Triple negation collapses: `¬¬¬a = ¬a`. -/
theorem neg_neg_neg (a : H) : neg (neg (neg a)) = neg a :=
  le_antisymm _ _ (neg_anti (le_neg_neg a)) (le_neg_neg (neg a))

/-- Non-contradiction: `a ⊓ ¬a ≤ ⊥`. -/
theorem inf_neg_le_bot (a : H) : a ⊓ neg a ≤ ⊥ :=
  le_trans _ _ _ (inf_comm_le _ _) (himp_inf_le a ⊥)
```

These are term-mode proofs that chain the axioms, and they follow the logic closely. `himp_inf_le` is modus ponens: residuation read from right to left, applied to `a ⇨ b ≤ a ⇨ b`. `le_neg_neg` is $a \le \neg\neg a$, the algebraic `dni`: by residuation it suffices that $a \sqcap \neg a \le \bot$, which is modus ponens after swapping the meet. `neg_anti` says that negation reverses the order, and `neg_neg_neg` combines the two into the collapse of triple negation. (Inside `namespace HeytingAlgebra`, names such as `le_trans` refer to the fields of our class.) Residuation also implies that every Heyting algebra is distributive, which soundness for ∨E needs:

```
/-- Every Heyting algebra is distributive. By residuation it suffices to
show `a ⊔ b ≤ c ⇨ d` for `d = (a ⊓ c) ⊔ (b ⊓ c)`, and a join is below
something when both parts are. -/
theorem sup_inf_le (a b c : H) : (a ⊔ b) ⊓ c ≤ (a ⊓ c) ⊔ (b ⊓ c) := by
  apply (le_himp_iff _ _ _).mp
  apply sup_le
  · exact (le_himp_iff _ _ _).mpr (le_sup_left _ _)
  · exact (le_himp_iff _ _ _).mpr (le_sup_right _ _)
```

Soundness in every Heyting algebra

A formula is interpreted by recursion, and a context by the meet of its formulas:

```
/-- The value of a formula in a Heyting algebra, given a value for each atom. -/
def interp {H : Type} [HeytingAlgebra H] (f : Nat → H) : Formula → H
  | .var n   => f n
  | .top     => ⊤
  | .bot     => ⊥
  | .and A B => interp f A ⊓ interp f B
  | .or A B  => interp f A ⊔ interp f B
  | .imp A B => interp f A ⇨ interp f B

/-- A context is the meet of its formulas (the empty context is `⊤`). -/
def interpCtx {H : Type} [HeytingAlgebra H] (f : Nat → H) : List Formula → H
  | []     => ⊤
  | A :: Γ => interp f A ⊓ interpCtx f Γ
```

```
/-- **Soundness.** If `Γ ⊢ A` then `⟦Γ⟧ ≤ ⟦A⟧` in every Heyting algebra, for
every interpretation of the atoms. -/
theorem soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ {H : Type} [HeytingAlgebra H] (f : Nat → H), interpCtx f Γ ≤ interp f A := by
  induction h with
  | hyp hmem =>
    intro H _ f
    induction hmem with
    | head => exact inf_le_left _ _
    | tail _ _ ih => exact le_trans _ _ _ (inf_le_right _ _) ih
  | topI => intro H _ f; exact le_top _
  | botE _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (bot_le _)
  | andI _ _ ih₁ ih₂ => intro H _ f; exact le_inf _ _ _ (ih₁ f) (ih₂ f)
  | andE1 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (inf_le_left _ _)
  | andE2 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (inf_le_right _ _)
  | orI1 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (le_sup_left _ _)
  | orI2 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (le_sup_right _ _)
  | orE _ _ _ ih ihA ihB =>
    -- ⟦Γ⟧ ≤ (a ⊔ b) ⊓ ⟦Γ⟧ ≤ (a ⊓ ⟦Γ⟧) ⊔ (b ⊓ ⟦Γ⟧) ≤ c
    intro H _ f
    apply le_trans _ _ _ (le_inf _ _ _ (ih f) (le_refl _))
    apply le_trans _ _ _ (sup_inf_le _ _ _)
    exact sup_le _ _ _ (ihA f) (ihB f)
  | impI _ ih =>
    -- residuation turns `a ⊓ ⟦Γ⟧ ≤ b` into `⟦Γ⟧ ≤ a ⇨ b`
    intro H _ f
    apply (le_himp_iff _ _ _).mpr
    exact le_trans _ _ _ (inf_comm_le _ _) (ih f)
  | impE _ _ ih₁ ih₂ =>
    -- ⟦Γ⟧ ≤ (a ⇨ b) ⊓ a ≤ b
    intro H _ f
    exact le_trans _ _ _ (le_inf _ _ _ (ih₁ f) (ih₂ f)) (himp_inf_le _ _)
```

The statement says that a derivation of $\Gamma \vdash A$ gives $[\![\Gamma]\!] \le [\![A]\!]$ in every Heyting algebra. Each case is one law: Hyp is "a meet is below its parts" (by induction on the position of the formula in the context), $\wedge$I is `le_inf`, the eliminations are `inf_le_left` and `inf_le_right`, $\vee$E is distributivity followed by `sup_le`, $\to$I is residuation, and $\to$E is modus ponens. The proof uses no axioms at all.

Two instances

The two-element algebra is `Bool`, and its laws are checked by `decide`, which tries every combination of values:

```
/-- The two-element Heyting algebra: the truth values. `a ⇨ b` is `!a || b`. -/
instance : HeytingAlgebra Bool where
  le a b := a = true → b = true
```

```
  min a b := a && b
  max a b := a || b
  himp a b := !a || b
  top := true
  bot := false
  -- Each law is checked on every combination of elements: up to 27 cases.
  le_refl a := by cases a <;> decide
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide
  le_antisymm a b := by cases a <;> cases b <;> decide
  inf_le_left a b := by cases a <;> cases b <;> decide
  inf_le_right a b := by cases a <;> cases b <;> decide
  le_inf a b c := by cases a <;> cases b <;> cases c <;> decide
  le_sup_left a b := by cases a <;> cases b <;> decide
  le_sup_right a b := by cases a <;> cases b <;> decide
  sup_le a b c := by cases a <;> cases b <;> cases c <;> decide
  bot_le a := by cases a <;> decide
  le_top a := by cases a <;> decide
  le_himp_iff a b c := by cases a <;> cases b <;> cases c <;> decide
```

The three-element chain needs a type of its own:

```
/-- The three-element chain `0 < ½ < 1`: "false", "undecided", "true". -/
inductive Three where
  | zero | half | one
  deriving DecidableEq, Repr

/-- The position of an element in the chain. -/
def Three.rank : Three → Nat
  | .zero => 0
  | .half => 1
  | .one  => 2

/-- In a chain, `a ⇨ b` is `1` when `a ≤ b`, and `b` otherwise. -/
instance : HeytingAlgebra Three where
  le a b := a.rank ≤ b.rank
  min a b := if a.rank ≤ b.rank then a else b
  max a b := if a.rank ≤ b.rank then b else a
  himp a b := if a.rank ≤ b.rank then .one else b
  top := .one
  bot := .zero
  -- Each law is checked on every combination of elements: up to 27 cases.
  le_refl a := by cases a <;> decide
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide
  le_antisymm a b := by cases a <;> cases b <;> decide
  inf_le_left a b := by cases a <;> cases b <;> decide
  inf_le_right a b := by cases a <;> cases b <;> decide
  le_inf a b c := by cases a <;> cases b <;> cases c <;> decide
  le_sup_left a b := by cases a <;> cases b <;> decide
  le_sup_right a b := by cases a <;> cases b <;> decide
  sup_le a b c := by cases a <;> cases b <;> cases c <;> decide
  bot_le a := by cases a <;> decide
  le_top a := by cases a <;> decide
  le_himp_iff a b c := by cases a <;> cases b <;> cases c <;> decide
```

Each law is checked by `cases` on every argument followed by `decide`: at most 27 combinations. Now the facts computed above:

```
/-- info: Ch05.Three.zero -/
#guard_msgs in
#eval neg Three.half

/-- info: Ch05.Three.one -/
#guard_msgs in
#eval neg (neg Three.half)

/-- Excluded middle fails at `½`: `½ ⊔ ¬½ = ½ ⊔ 0 = ½`, not `1`. -/
example : Three.half ⊔ neg Three.half ≠ ⊤ := by decide
```

```
/-- Double-negation elimination fails at `½`: `¬¬½ = 1`, which is not `≤ ½`. -/
example : ¬ (neg (neg Three.half) ≤ Three.half) := by decide

/-- In `Bool`, both hold for every element. -/
example : ∀ a : Bool, a ⊔ neg a = ⊤ ∧ neg (neg a) = a := by decide
```

And with soundness, a second, algebraic proof that excluded middle is not derivable, independent of Kripke models:

```
/-- A second proof that excluded middle is not derivable, this time
algebraic: interpret the atom `P` as `½`. -/
theorem lem_not_derivable' : ¬ Proves [] (.or P P.neg) := by
  intro h
  have := HeytingAlgebra.soundness h (fun _ => Three.half)
  revert this
  decide
```

Soundness gives `⊤ ≤ ½ ⊔ ¬½` for the interpretation that sends every atom to `½`; `revert this` turns that hypothesis into the goal `⊤ ≤ ... → False`, which `decide` refutes by computing.

The two semantics are closely related: the up-closed sets of worlds of a Kripke model (the "propositions" that respect monotonicity) form a Heyting algebra. The three-element chain comes from the two-world model: the up-sets are $\varnothing$, $\{later\}$ and $\{now, later\}$, which are 0, ½ and 1. Another classic source of Heyting algebras is topology: the open sets of a space, with $U \Rightarrow V$ the interior of $(X \setminus U) \cup V$. In the real line, $U = \mathbb{R} \setminus \{0\}$ has $\neg U = \operatorname{int}\{0\} = \varnothing$ and $\neg\neg U = \mathbb{R} \neq U$: the missing point is undecided.

Negation in a Heyting algebra

The elements with $\neg\neg a = a$ are called **regular**. Every negation is regular (that is `neg_neg_neg`), and one can show that the regular elements form a Boolean algebra. That is Glivenko's theorem in algebraic form: the map $a \mapsto \neg\neg a$ projects intuitionistic truth values onto classical ones, and classical logic is the logic of the regular elements, *inside* intuitionistic logic rather than beyond it. In the three-element chain the regular elements are 0 and 1; the undecided ½ is what the projection forgets.

In Mathlib

Mathlib's `HeytingAlgebra` states residuation the same way, and `Prop` is an instance:

```
/-- Mathlib's class states residuation in the same form as ours. -/
example {α : Type} [_root_.HeytingAlgebra α] (a b c : α) :
    a ≤ b ⇨ c ↔ a ⊓ b ≤ c :=
  le_himp_iff

/-- `Prop` is a Heyting algebra: `≤` is `→`, `⊓` is `∧`, `⊔` is `∨`, `⇨` is
`→`, and the complement `ᶜ` is `¬`. All of these hold by `rfl`. -/
example (p q : Prop) : (p ≤ q) = (p → q) := rfl
example (p q : Prop) : (p ⊓ q) = (p ∧ q) := rfl
example (p q : Prop) : (p ⊔ q) = (p ∨ q) := rfl
example (p q : Prop) : (p ⇨ q) = (p → q) := rfl
example (p : Prop) : pᶜ = ¬p := rfl

/-- The general laws we proved by hand are in Mathlib too. -/
example {α : Type} [_root_.HeytingAlgebra α] (a : α) : a ≤ aᶜᶜ := le_compl_compl
example {α : Type} [_root_.HeytingAlgebra α] (a : α) : aᶜᶜᶜ = aᶜ := compl_compl_compl a
```

Every one of these holds by `rfl`: in Mathlib's instance, `≤` on `Prop` *is* implication, meet *is* `∧`, Heyting implication *is* `→`, and the complement *is* `¬`. Propositions are the truth values of a Heyting algebra, with entailment as the order. Mathlib calls negation the *complement* `aᶜ`, and the laws we proved by hand are library lemmas (`le_compl_compl`, `compl_compl_compl`). Mathlib also makes `Prop` a `BooleanAlgebra`, which needs excluded middle, and `#print axioms` shows it:

```
/-- Excluded middle holds in every Boolean algebra... -/
theorem em_boolean {α : Type} [BooleanAlgebra α] (a : α) : a ⊔ aᶜ = ⊤ :=
  sup_compl_eq_top

/-- ...and Mathlib makes `Prop` a Boolean algebra, using classical logic. -/
theorem em_prop (p : Prop) : p ⊔ pᶜ = ⊤ := em_boolean p

/-- info: 'Ch05.em_prop' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms em_prop
```

5.9 The categorical reading

One more step of abstraction connects this chapter with Part IV. A preorder is a category in which there is at most one arrow between two objects: an arrow $a \to b$ exists exactly when $a \le b$ (Chapter 16 makes this precise). In that reading, meet is a *product*, join is a *coproduct*, $\top$ and $\bot$ are the terminal and initial objects, and residuation

$$c \sqcap a \le b \quad \text{iff} \quad c \le (a \Rightarrow b)$$

has the shape of currying, $\mathrm{Hom}(C \times A, B) \cong \mathrm{Hom}(C, B^A)$. A category with products and this currying property is called **cartesian closed**, and Chapter 19 shows that cartesian closed categories are exactly the models of the simply typed λ-calculus. A Heyting algebra is a cartesian closed category in which all proofs of the same entailment are identified; the general case keeps *proofs* as arrows instead of just "provable or not", which is the Curry–Howard correspondence one level up.

Logic	Categorical structure
intuitionistic	cartesian closed category (with coproducts)
intuitionistic, proofs ignored	Heyting algebra
classical, proofs ignored	Boolean algebra
linear (Chapter 21)	symmetric monoidal closed category
affine	symmetric monoidal closed, with a terminal unit

The last rows preview Part V. Residuation survives when the product $\sqcap$ is replaced by a tensor $\otimes$ that cannot duplicate or discard: the diagonal $a \to a \times a$ is contraction, the map $a \to \top$ is weakening, and dropping them is what turns intuitionistic logic into linear logic, and Rust's moves into an affine type system. Classical logic, on the other hand, does not have a good proof-relevant version of this table: a well-known argument (usually credited to André Joyal) shows that a cartesian closed category with an initial object, in which the canonical arrow $a \to \neg\neg a$ is an isomorphism for every object, collapses to a preorder. Classical proofs need control operators, not just functions.

5.10 Putting it together: the De Morgan law

Let us follow one formula through the chapter: $\neg(A \wedge B) \to \neg A \vee \neg B$.

Classically, it is easy. `not_and_or` splits on `p` with `by_cases`. If `p` holds, `q` must fail, since `p ∧ q` is refuted; if not, `¬p` is the left disjunct. `#print axioms` reports `Classical.choice`.

Constructively, half of it holds. The converse, $\neg A \vee \neg B \to \neg(A \wedge B)$, is `not_and_of_or_not`, with no axioms: given a refutation of one side, refute the conjunction by projecting out that side. The classical direction asks for more: from a refutation of the conjunction, a *decision* about which conjunct is to blame. BHK asks where that decision comes from, and there is no answer.

A countermodel shows it cannot be proved. The Rust search reports that no model with one or two worlds refutes the law, and that the fork does:

```
De Morgan         ¬(P ∧ Q) → ¬P ∨ ¬Q
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0
```

Lean confirms the countermodel by `decide` and turns it into a theorem about the proof system with `kripke_soundness`: `de_morgan_not_derivable`, in the fork section above. The root forces $\neg(P \wedge Q)$ but neither $\neg P$ nor $\neg Q$.

It is not as strong as excluded middle. The two-world model, which refutes excluded middle, validates every instance of this law (it is a chain, so any two futures have a common future). So adding the De Morgan law to intuitionistic logic does not give classical logic: it gives the intermediate logic of weak excluded middle.

Its double negation is constructive. `nn_not_and_or` proves $\neg\neg(\neg(A \wedge B) \to \neg A \vee \neg B)$ without axioms, by borrowing $A \vee \neg A$ inside the double-negation monad, as Glivenko's theorem promises. Classical reasoning is available constructively, as long as the conclusion is negative.

5.11 Common confusions

"**Intuitionists believe excluded middle is false.**" They do not; its negation is refutable, since $\neg\neg(A \vee \neg A)$ is a theorem. Excluded middle is not *assertible* in general, because asserting a disjunction requires knowing which side holds.

"**Proof by contradiction is classical.**" It depends on what is being proved. To prove $\neg A$, assuming A and deriving $\bot$ is just $\to$I, and perfectly constructive. To prove A by assuming $\neg A$ and deriving $\bot$ is RAA, which is classical. Mathematicians use the same phrase for both.

"**Lean is a classical proof assistant.**" Lean's core logic is intuitionistic; classical logic enters through one axiom, `Classical.choice`, which the library uses to prove `Classical.em`. Mathlib uses it freely, and so do some tactics (`grind`, `simp` with classical lemmas, `by_cases`). `#print axioms` tells you which of your results depend on it.

"**A Kripke world is a possible world where every proposition is true or false.**" A Kripke world is a state of knowledge. $w \nVdash A$ does not mean that A is false at w, only that it is not known there; falsity at w is $w \Vdash \neg A$, which constrains every future.

"**`Decidable p` is the same as `p ∨ ¬p`.**" Both have two cases, but `Decidable p` is data that a program can branch on, while `p ∨ ¬p` is a proposition whose proofs are erased. Every `Decidable p` gives `p ∨ ¬p`; the converse needs `Classical.choice` and gives up computability.

5.12 Summary

- In intuitionistic logic, truth is provability. The BHK interpretation says what a proof of each connective is, and it is a description of data types: pairs, tagged unions, functions, the unit type and the empty type.
- Excluded middle, double-negation elimination and Peirce's law fail constructively and are equivalent to one another; the De Morgan law $\neg(A \wedge B) \to \neg A \vee \neg B$ fails too, but is weaker.
- In exchange, intuitionistic proofs have the disjunction and existence properties: they carry the information which side and which witness. That is what makes them programs.
- Classical logic embeds into intuitionistic logic by double negation (Glivenko); $\neg\neg(A \vee \neg A)$ is constructive, and $\neg\neg$ behaves like a continuation monad. Classical proofs compute with control operators.
- Negation is $A \to \bot$: a refutation is a function into the empty type. Rust has empty enums, `Infallible` and `!`, and its exhaustiveness checker uses emptiness.
- In Lean, `Classical.em` and `byContradiction` rest on the axiom `Classical.choice`; `Decidable p` is the computable form of excluded middle, needed by `if` and by `decide`; `#print axioms` tells constructive proofs from classical ones.
- Kripke models (worlds of knowledge, monotone forcing, implication quantifying over the future) are sound and complete for intuitionistic logic. A two-world model refutes excluded middle; we proved soundness in Lean and checked the countermodel by `decide`, so excluded middle is not derivable.
- Heyting algebras are lattices with residuated implication. We built the class from scratch, proved soundness of natural deduction in every instance, and refuted excluded middle in the three-element chain. Mathlib's `Prop` is a Heyting algebra with `≤` as implication.
- Categorically, a Heyting algebra is a thin cartesian closed category; dropping the cartesian structure leads to the linear and affine logics of Part V.

5.13 Exercises

1. Find, by hand, a Kripke countermodel to material implication $(P \to Q) \to \neg P \vee Q$, and check each clause of the forcing definition. Then add the formula to the list in `ch05_kripke.rs` and compare with the program's answer. *Hint:* two worlds suffice. P must be undecided at the root, so that $\neg P$ fails there, and $P \to Q$ must hold at the root, so Q must become known wherever P does. (With Q never known, $P \to Q$ fails at the root and the implication holds vacuously.)
2. Prove in Lean, without `Classical`, that `¬¬(p → q) → ¬¬p → ¬¬q` (the double-negation monad applied to a function) and that `¬¬(¬p ∨ ¬¬p)`. Check both with `#print axioms`. *Hint:* for the

first, `intro h hp k` and then `apply h; intro hpq; apply hp; intro hp'; exact k (hpq hp')`. For the second, imitate `nn_em`: commit to the right disjunct first, and when it is challenged, switch to the left.

3. Prove that double-negation elimination implies Peirce's law, constructively: `(∀ p : Prop, ¬¬p → p) → ∀ p q : Prop, ((p → q) → p) → p`. *Hint:* apply the hypothesis to `p`, then `intro k` with `k : ¬p`; to use the assumption `(p → q) → p`, build a function `p → q` from `k` with `absurd`.
4. Add a three-world chain `now ≤ soon ≤ later` to `Kripke.lean`, with `P` known only at `later`. Use `decide` to check that excluded middle fails at `now` and at `soon`, and that `(P → Q) ∨ (Q → P)` holds at every world. *Hint:* copy `Stage` and its instances; the order is "rank of the first ≤ rank of the second", so `le_trans` follows by `decide` after `cases`.
5. In the three-element chain, compute Peirce's law $((a \Rightarrow b) \Rightarrow a) \Rightarrow a$ for $a = ½$, $b = 0$. Then prove `¬ Proves [] (.imp (.imp (.imp P Q) P) P)` in Lean using `HeytingAlgebra.soundness` and `Three`. *Hint:* $½ \Rightarrow 0 = 0$, $0 \Rightarrow ½ = 1$, $1 \Rightarrow ½ = ½$. Interpret atom 0 as `half` and atom 1 as `zero`: `fun n => if n = 0 then Three.half else Three.zero`, and finish as in `lem_not_derivable'`.
6. Extend the Rust search to four worlds and find a frame (an order, without a valuation) on which weak excluded middle $\neg P \vee \neg\neg P$ holds at every world for *every* valuation, but linearity $(P \to Q) \vee (Q \to P)$ fails for some valuation. *Hint:* weak excluded middle holds on every frame in which any two futures of a world have a common future. Try a diamond: a root, two incomparable worlds, and a top above both.

CHAPTER 6

The Structural Rules

Every rule of natural deduction in Chapter 4 was about a connective. Rule $\wedge$I says how to prove a conjunction, rule $\to$E says how to use an implication, and so on down the list. There is a second kind of rule, which mentions no connective at all. It talks only about the *context*, the list of assumptions to the left of the turnstile. It says that you may reorder the assumptions, that you may add an assumption you will never use, and that two copies of the same assumption may be merged into one. These are the **structural rules**: exchange, weakening and contraction. In ordinary logic they are so harmless that most textbooks never state them, and Chapter 4 did not state them either.

This chapter is short on new machinery, but it contains the idea that the second half of the book is built on. Whether a logic has these rules decides what an assumption *is*. With all three, an assumption is a fact: once you know it, you know it as many times as you like, and you are free to ignore it. Without contraction, an assumption is a resource, like a coin that can be spent once. Every time the Rust compiler reports error E0382, "use of moved value", it is refusing to apply contraction.

In this chapter you will learn:

- what exchange, weakening and contraction say, and how to read them in both directions;
- the difference between primitive, derivable and admissible rules, and between contexts as lists, multisets and sets;
- why the natural deduction of Chapter 4 has all three structural rules without stating them, with a Lean proof by induction on derivations;
- how to count how many times a derivation uses an assumption, and how each structural rule changes the count;
- where the three rules show up in Rust: unused values and `drop`, moves and E0382, `Clone` and `Copy`, evaluation order and drop order;
- the resource reading of assumptions, and the design decision that generates the substructural logics of Part V.

6.1 Rules about contexts

The context, once more

Chapter 4 wrote natural deduction in sequent notation. A sequent $\Gamma \vdash A$ says "from the assumptions in Γ, the formula A follows", and the context Γ is a list of formulas. Here are four of the rules again, to have them in front of us:

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \mathsf{Hyp} \qquad \frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\ \wedge\mathsf{I}$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\ \to\mathsf{I} \qquad \frac{\Gamma \vdash A \to B \qquad \Gamma \vdash A}{\Gamma \vdash B}\ \to\mathsf{E}$$

Rule Hyp says that an assumption can be used. Rule →I is the one that changes the context: to prove $A \to B$, add A to the assumptions and prove B. The comma in Γ, A means "the list Γ with A added at the end". The other rules pass the context along unchanged, from the conclusion to every premise.

The three structural rules

The structural rules are rules of the same shape, but no connective appears in them. The only thing that changes between premise and conclusion is the context:

$$\frac{\Gamma_1, A, B, \Gamma_2 \vdash C}{\Gamma_1, B, A, \Gamma_2 \vdash C}\ \mathsf{Exch}$$

$$\frac{\Gamma \vdash C}{\Gamma, A \vdash C}\ \mathsf{Weak} \qquad \frac{\Gamma, A, A \vdash C}{\Gamma, A \vdash C}\ \mathsf{Contr}$$

Exchange (Exch) says that two neighboring assumptions may swap places. Applied repeatedly, it puts the context in any order you like. **Weakening** (Weak, also called *thinning*) says that if C follows from Γ, it also follows from Γ with one more assumption. **Contraction** (Contr) says that if C follows from two copies of A, it also follows from one copy.

Gerhard Gentzen, who introduced both natural deduction and the sequent calculus in 1935, stated these rules explicitly as rules of his sequent calculus, and called them *structural* to set them apart from the *logical* rules that introduce and eliminate connectives.

Like every rule, these can be read in two directions, and both readings are useful.

Rule	Top-down (building a proof)	Bottom-up (searching for one)	An assumption…
Exch	swap two assumptions	swap two assumptions	can be used in any position
Weak	add an assumption	throw an assumption away	may go unused
Contr	merge two copies into one	duplicate an assumption	may be used more than once

> **WARNING**
>
> The rules are stated in the direction that is convenient for proofs by induction, and that direction is the *opposite* of the slogans. Weakening, read top-down, makes the context *bigger*; the slogan "you may throw an assumption away" is the bottom-up reading: to prove $\Gamma, A \vdash C$ it is enough to prove $\Gamma \vdash C$, so A was discarded. Contraction, read top-down, *merges* two copies; the slogan "you may duplicate an assumption" is again the bottom-up reading. When Part V says "affine means no contraction, so a value is used at most once", it is the bottom-up reading that is meant.

The same rules with terms

Chapter 1 introduced typing judgments $\Gamma \vdash e : \tau$, where the context gives types to variables. The structural rules have a version for them, and it is the version that will matter most for Rust:

$$\frac{\Gamma_1, x : \sigma, y : \tau, \Gamma_2 \vdash e : \rho}{\Gamma_1, y : \tau, x : \sigma, \Gamma_2 \vdash e : \rho}\ \textsf{Exch}$$

$$\frac{\Gamma \vdash e : \rho}{\Gamma, x : \sigma \vdash e : \rho}\ \textsf{Weak} \qquad \frac{\Gamma, x : \sigma, y : \sigma \vdash e : \rho}{\Gamma, z : \sigma \vdash e[z/x, z/y] : \rho}\ \textsf{Contr}$$

Read each rule as a statement about the program e:

- **Exchange** leaves the program unchanged. Reordering the variables in scope does not change what the program means.
- **Weakening** also leaves the program unchanged, while the context grows. So the new variable x does *not occur* in e: weakening is the permission to have a variable in scope and not use it.
- **Contraction** changes the program: $e[z/x, z/y]$ is e with both x and y replaced by z (Chapter 7 defines substitution carefully). Where e used two different variables, the new program uses the same variable z twice: contraction is the permission to use a variable more than once.

This gives the sentence that the rest of the book turns on:

$$\textbf{weakening} = \text{discarding} \qquad \textbf{contraction} = \text{duplicating}$$

You can already see where Rust stands. A function `fn first<A, B>(a: A, b: B) -> A { a }` never uses `b`; that is weakening, and Rust accepts it. A function `fn dup<T>(x: T) -> (T, T) { (x, x) }` uses `x` twice; that is contraction, and Rust rejects it unless `T` is `Copy`. We will look at both in detail below.

6.2 Where the structural rules hide

Look back at the rules of Chapter 4: there is no exchange, no weakening and no contraction among them. And yet natural deduction can prove $A, B \vdash A$, which seems to need weakening (the B is never used), and $A \vdash A \wedge A$, which seems to need contraction (the A is used twice). The structural rules are there; they are just hidden in two places.

The hypothesis rule. Rule Hyp asks only that $A \in \Gamma$. It does not care *where* in Γ the formula sits (that is exchange), it does not care what else is in Γ (that is weakening), and it does not care how many times A occurs in Γ (that is contraction, read the other way). The leaves of every derivation quietly absorb all three rules.

Context sharing. Rules with several premises, such as $\wedge$I, $\to$E and $\vee$E, hand the *same* context Γ to every premise. Each premise is free to use any assumption in Γ, so a single assumption can be used once in each branch. That is contraction, built into the shape of the rules.

Here is the clearest example, the commutativity of conjunction. (We write the leaves as Hyp steps with their membership premise left implicit.)

$$\dfrac{\dfrac{\dfrac{}{A \wedge B \vdash A \wedge B}\,\mathsf{Hyp}}{A \wedge B \vdash B}\,\wedge\mathsf{E}_2 \qquad \dfrac{\dfrac{}{A \wedge B \vdash A \wedge B}\,\mathsf{Hyp}}{A \wedge B \vdash A}\,\wedge\mathsf{E}_1}{A \wedge B \vdash B \wedge A}\,\wedge\mathsf{I}$$

There is one assumption, $A \wedge B$, and it is used twice, once by each projection. Rule $\wedge$I sent the context to both premises, and both premises used it. This is a surprise the first time you see it: swapping the components of a pair "needs" contraction in natural deduction. In Rust, `fn swap<A, B>((a, b): (A, B)) -> (B, A)` does not need `Clone`: the pattern `(a, b)` takes the pair apart *once*, producing two new variables that are used once each. Chapter 21 explains that Rust's pair behaves like a different conjunction, written $\otimes$, whose elimination rule is exactly this pattern match.

The second example shows weakening at a leaf. To prove $\vdash A \to (B \to A)$, apply $\to$I twice and then Hyp:

$$\dfrac{\dfrac{\dfrac{A \in A, B}{A, B \vdash A}\,\mathsf{Hyp}}{A \vdash B \to A}\,\to\mathsf{I}}{\vdash A \to (B \to A)}\,\to\mathsf{I}$$

The assumption B was added by $\to$I and never used: rule Hyp simply ignored it.

The same examples in Lean

The Lean code for this chapter is in `code/lean/Book/Ch06/Structural.lean` and `code/lean/Book/Ch06/Usage.lean`. The first file starts by redefining the formulas and the derivability relation of Chapter 4 (each chapter's Lean files are self-contained, so we do not import Chapter 4's):

```
/-- Propositional formulas, as in Chapter 4. -/
inductive Formula where
  | atom (name : String)
  | top
  | bot
  | conj (A B : Formula)
  | disj (A B : Formula)
  | imp (A B : Formula)
  deriving DecidableEq, Repr
```

```
/-- `Proves Γ A`: the sequent `Γ ⊢ A` is derivable in natural deduction.
Lean lists grow at the front, so the rules write `A :: Γ` for what the
mathematics writes `Γ, A`. -/
inductive Proves : List Formula → Formula → Prop where
  | hyp {Γ A} : A ∈ Γ → Proves Γ A
  | topI {Γ} : Proves Γ top
  | botE {Γ C} : Proves Γ bot → Proves Γ C
  | conjI {Γ A B} : Proves Γ A → Proves Γ B → Proves Γ (conj A B)
  | conjE₁ {Γ A B} : Proves Γ (conj A B) → Proves Γ A
  | conjE₂ {Γ A B} : Proves Γ (conj A B) → Proves Γ B
  | disjI₁ {Γ A B} : Proves Γ A → Proves Γ (disj A B)
  | disjI₂ {Γ A B} : Proves Γ B → Proves Γ (disj A B)
  | disjE {Γ A B C} :
```

```
      Proves Γ (disj A B) → Proves (A :: Γ) C → Proves (B :: Γ) C → Proves Γ C
  | impI {Γ A B} : Proves (A :: Γ) B → Proves Γ (imp A B)
  | impE {Γ A B} : Proves Γ (imp A B) → Proves Γ A → Proves Γ B
```

One detail needs attention. Lean lists are built from the front: `A :: Γ` is the list `Γ` with `A` added *before* its first element. So where the mathematics writes Γ, A, the Lean rules write `A :: Γ`, and the list `[A, B]` is the context with `A` first. The order is different, and as you will see in a moment, it makes no difference at all: that is precisely what exchange says.

The examples above, checked by Lean:

```
/-- `A ∧ B ⊢ B ∧ A`. Both premises of `conjI` receive the whole context, and
each of them uses the hypothesis `A ∧ B`: two uses of one assumption. -/
example : Proves [conj A B] (conj B A) :=
  .conjI (.conjE₂ (.hyp List.mem_cons_self)) (.conjE₁ (.hyp List.mem_cons_self))

/-- `A ⊢ A ∧ A`: one assumption, used twice. -/
example : Proves [A] (conj A A) :=
  .conjI (.hyp (by simp)) (.hyp (by simp))

/-- `A, B ⊢ A`: the assumption `B` is never used. -/
example : Proves [A, B] A :=
  .hyp (by simp)

/-- `⊢ A → B → A`: after two `impI` steps the context is `[B, A]`, and the
hypothesis rule picks `A` out of it, ignoring `B`. -/
example : Proves [] (imp A (imp B A)) :=
  .impI (.impI (.hyp (by simp)))
```

`List.mem_cons_self` is the library fact that the head of a list is a member of it ($a \in a :: l$). Where Lean can figure out which formula is meant, we let `by simp` prove the membership instead. Notice that each proof is a tree of constructors with the same shape as the derivation on paper: `.conjI` has two children, each `.conjE₂` or `.conjE₁` has one, and the leaves are `.hyp`.

6.3 Primitive, derivable and admissible

We keep saying that natural deduction "has" the structural rules without listing them. To make that precise we need to distinguish three ways in which a rule can belong to a system.

- A rule is **primitive** if it is one of the rules that define the system. The constructors of `Proves` are the primitive rules of our natural deduction.
- A rule is **derivable** if there is a fixed scheme that builds a derivation of its conclusion from derivations of its premises, using primitive rules only. A derivable rule is an abbreviation, a macro: wherever you use it, you could paste in the same few primitive steps instead.
- A rule is **admissible** if, whenever its premises are derivable, its conclusion is derivable too. The derivation of the conclusion may depend on the *shape* of the derivations of the premises; typically it is built by induction on them.

Every derivable rule is admissible: if the premises have derivations, plug them into the macro. The converse fails, and the structural rules are the standard examples. In natural deduction with lists as contexts, exchange, weakening and contraction are admissible but not derivable: there is no fixed combination of primitive rules that turns a derivation of $\Gamma \vdash C$ into one of $\Gamma, A \vdash C$ without looking

inside it, because no primitive rule adds an unused assumption to a context. What we can do is walk through the whole derivation and add A to every context in it.

A programmer's way to see the difference: a derivable rule is a function that wraps its argument in a few constructors and never inspects it; an admissible rule is a recursive function that takes its argument apart and rebuilds it, like a compiler pass over a syntax tree.

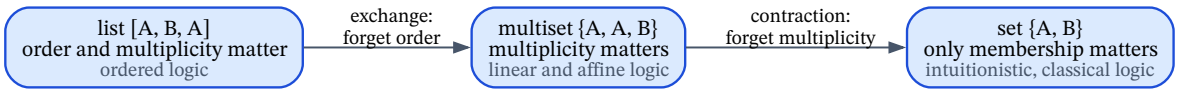
The difference matters when a system changes. If you add new primitive rules to a system, every derivable rule stays derivable: the macro still works. An admissible rule may stop being admissible, because its proof by induction now has a new case to handle, and that case may fail. Exercise 5 adds a single axiom to natural deduction and breaks weakening. Part V is full of systems designed so that weakening or contraction is *not* admissible, and the main danger there is a connective rule that brings a structural rule back without anyone noticing.

Lists, multisets and sets

There is a third status, which is not about rules at all but about data structures. If contexts are *sets* rather than lists, then Γ, A, B and Γ, B, A are the same set, and so are Γ, A, A and Γ, A. Exchange and contraction are then **built in**: they do not say anything, because the two contexts they relate are equal. The choice of data structure decides which structural rules come for free:

Contexts are	Exchange	Contraction	Weakening
lists	a statement	a statement	a statement
multisets	built in	a statement	a statement
sets	built in	built in	a statement

A **multiset** is a collection in which order does not matter but multiplicity does: $\{A, A, B\}$ and $\{A, B, A\}$ are the same multiset, while $\{A, A, B\}$ and $\{A, B\}$ are different. Weakening is a statement in all three cases, since adding an element does change a list, a multiset and a set.



Contexts as lists, multisets and sets. Exchange forgets the order of assumptions, contraction forgets their multiplicity; each choice of data structure suits a different family of logics.

Mathlib has all three data structures, and it is instructive to see Lean confirm the table:

```
-- As lists, order and multiplicity both matter:
example : [atom "A", atom "B"] ≠ [atom "B", atom "A"] := by decide
example : [atom "A", atom "A"] ≠ [atom "A"] := by decide

-- As multisets, order is forgotten (exchange is built in) but
-- multiplicity is not:
example (A B : Formula) (Γ : Multiset Formula) :
    A ::ₘ B ::ₘ Γ = B ::ₘ A ::ₘ Γ :=
  Multiset.cons_swap A B Γ
example : ({atom "A", atom "A"} : Multiset Formula) ≠ {atom "A"} := by decide
```

```
-- As sets, both are forgotten (exchange and contraction are built in):
example (A B : Formula) : ({A, B} : Finset Formula) = {B, A} :=
  Finset.pair_comm A B
example (A : Formula) (Γ : Finset Formula) : insert A (insert A Γ) = insert A Γ :=
  Finset.insert_idem A Γ
```

`Multiset Formula` is Mathlib's type of multisets, and `A ::ₘ Γ` adds one copy of `A` to the multiset `Γ`. `Multiset.cons_swap` is exactly exchange as an *equation*. `Finset Formula` is the type of finite sets, `insert A Γ` adds `A`, and `Finset.insert_idem` (idempotence: inserting twice is inserting once) is contraction as an equation. The two inequalities are checked by `decide`, which computes both sides and compares them.

Most presentations of ordinary logic use sets of assumptions and never mention the structural rules. That is a reasonable choice when the rules are all harmless. Substructural logic begins by refusing it: ordered logic keeps contexts as lists, linear and affine logic use multisets, and the question "is contraction admissible?" becomes the central design question.

6.4 Admissibility, proved in Lean

Our Lean contexts are lists, so all three structural rules are statements that need proofs. Let us prove them.

A first attempt, and why it fails

The natural first try is to prove weakening directly: if `Proves Γ C`, then `Proves (A :: Γ) C`, by induction on the derivation of `Γ ⊢ C`. Most cases are easy. In the case of rule Hyp, we know `C ∈ Γ`, so `C ∈ A :: Γ`. In the case of ∧I, the induction hypotheses give derivations of `A :: Γ ⊢ C₁` and `A :: Γ ⊢ C₂`, and ∧I combines them.

The case of →I breaks. There, `C` is `B → D` and the premise is `B :: Γ ⊢ D`. The induction hypothesis, applied to the premise, adds `A` at the front of *its* context and gives `A :: B :: Γ ⊢ D`. But to apply →I and obtain `A :: Γ ⊢ B → D`, we need `B :: A :: Γ ⊢ D`, with `B` at the front, where →I discharges it. The two contexts differ by an exchange. We are trying to prove weakening and we need exchange; if we tried to prove exchange first, we would run into the same problem one level down.

This is a very common situation in proofs by induction: the statement is true but too weak to be its own induction hypothesis. The cure is to prove something *stronger*, a statement general enough that the induction hypothesis is exactly what each case needs.

Monotonicity

The stronger statement is this: if every assumption of `Γ` is also an assumption of `Δ`, then everything provable from `Γ` is provable from `Δ`. The relation "every element of `Γ` is an element of `Δ`" is `Γ ⊆ Δ` (`List.Subset`), and the theorem is called *monotonicity*, because derivability only grows as the context grows.

```
/-- Derivability is monotone in the context: if every hypothesis of `Γ` is
also a hypothesis of `Δ`, everything provable from `Γ` is provable from `Δ`.
The proof transforms the derivation of `Γ ⊢ C`, rule by rule, into a
```

```
derivation of `Δ ⊢ C`. -/
theorem Proves.mono {Γ Δ : List Formula} {C : Formula}
    (h : Proves Γ C) (sub : Γ ⊆ Δ) : Proves Δ C := by
  induction h generalizing Δ with
  | hyp mem =>
    -- The only place where the context is inspected: `A ∈ Γ` and `Γ ⊆ Δ`.
    exact .hyp (sub mem)
  | topI => exact .topI
  | botE _ ih => exact .botE (ih sub)
  | conjI _ _ ih₁ ih₂ => exact .conjI (ih₁ sub) (ih₂ sub)
  | conjE₁ _ ih => exact .conjE₁ (ih sub)
  | conjE₂ _ ih => exact .conjE₂ (ih sub)
  | disjI₁ _ ih => exact .disjI₁ (ih sub)
  | disjI₂ _ ih => exact .disjI₂ (ih sub)
  | disjE _ _ _ ih ihA ihB =>
    -- The branches have one more hypothesis on both sides: `A :: Γ ⊆ A :: Δ`.
    exact .disjE (ih sub) (ihA (List.cons_subset_cons _ sub))
      (ihB (List.cons_subset_cons _ sub))
  | impI _ ih =>
    -- Same here: the premise lives in `A :: Γ`, so we extend `sub` by `A`.
    exact .impI (ih (List.cons_subset_cons _ sub))
  | impE _ _ ih₁ ih₂ => exact .impE (ih₁ sub) (ih₂ sub)
```

Let us go through it.

- `induction h generalizing Δ with` starts an induction on the derivation `h`, one case per rule, just like the soundness proof of Chapter 4. The words `generalizing Δ` matter. Without them, the induction hypothesis would be about the *fixed* `Δ` of the statement. With them, the induction hypothesis says "for every `Δ` with `Γ' ⊆ Δ`, ...", where `Γ'` is the context of the premise. We need that freedom, because in the →I and ∨E cases the premise's context is bigger than `Γ` and we must use the hypothesis with a bigger `Δ` too.
- In the `hyp` case, the derivation is a single leaf, `mem : C ∈ Γ`. The subset relation is, by definition, a function from membership in `Γ` to membership in `Δ`, so `sub mem : C ∈ Δ`, and `.hyp (sub mem)` is the new leaf. This is the *only* case where the context is looked at.
- In the cases that keep the context (`botE`, `conjI`, `conjE₁` and the rest), we apply the induction hypotheses with the same `sub` and rebuild the node with the same rule. This is where the transformation is a "compiler pass": it rebuilds the tree node by node.
- In the `impI` case, the premise lives in `A :: Γ`. We need a derivation in `A :: Δ`, and `List.cons_subset_cons` turns `sub : Γ ⊆ Δ` into `A :: Γ ⊆ A :: Δ`. The induction hypothesis, used with `A :: Δ`, gives what →I needs. The `disjE` case does the same for both branches.

The →I case that defeated the first attempt is now trivial: the induction hypothesis does not care where in `Δ` the new formula goes, so no exchange is needed.

The three rules as corollaries

With monotonicity, the structural rules are one-liners, because each of them relates two contexts with the same elements, or a context to a bigger one:

```
/-- Weakening: an extra hypothesis never hurts. -/
theorem weakening (h : Proves Γ C) : Proves (A :: Γ) C :=
  h.mono (List.subset_cons_self A Γ)
```

```
/-- Exchange: two neighboring hypotheses may swap places, anywhere in the
context. -/
theorem exchange (h : Proves (Γ₁ ++ A :: B :: Γ₂) C) :
    Proves (Γ₁ ++ B :: A :: Γ₂) C :=
  h.mono (by
    intro X hX
    -- hX : X ∈ Γ₁ ++ A :: B :: Γ₂,   goal : X ∈ Γ₁ ++ B :: A :: Γ₂
    simp only [List.mem_append, List.mem_cons] at hX ⊢
    tauto)
```

```
/-- Contraction: two copies of a hypothesis can be merged into one. -/
theorem contraction (h : Proves (A :: A :: Γ) C) : Proves (A :: Γ) C :=
  h.mono (by
    intro X hX
    simp only [List.mem_cons] at hX ⊢
    tauto)
```

For weakening, `List.subset_cons_self A Γ` is the library fact `Γ ⊆ A :: Γ`. For exchange and contraction, we prove the subset relation by hand. `intro X hX` takes an arbitrary formula `X` with `hX : X ∈ Γ₁ ++ A :: B :: Γ₂`, and the goal is `X ∈ Γ₁ ++ B :: A :: Γ₂`. The `simp only` step rewrites both memberships with two library lemmas, `List.mem_append` ($x \in l_1 + + l_2 \leftrightarrow x \in l_1 \vee x \in l_2$) and `List.mem_cons` ($x \in a :: l \leftrightarrow x = a \vee x \in l$). After it, the hypothesis says `X ∈ Γ₁ ∨ X = A ∨ X = B ∨ X ∈ Γ₂` and the goal says `X ∈ Γ₁ ∨ X = B ∨ X = A ∨ X ∈ Γ₂`. That is a propositional tautology, and `tauto`, a Mathlib tactic that proves goals by propositional reasoning (splitting `∧`, `∨` and `↔` in hypotheses and goals), finishes the proof.

Exchange is stated for an arbitrary position in the context (between `Γ₁` and `Γ₂`), which is what rule Exch says. Weakening and contraction are stated at the front, which is enough, since exchange can move any formula there.

In fact the same argument proves more: two contexts with the same elements prove exactly the same formulas.

```
/-- Contexts with the same elements prove the same formulas: for this system,
a context might as well be a *set*. -/
theorem proves_iff_of_same_elements {Γ Δ : List Formula}
    (h₁ : Γ ⊆ Δ) (h₂ : Δ ⊆ Γ) : Proves Γ C ↔ Proves Δ C :=
  ⟨fun h => h.mono h₁, fun h => h.mono h₂⟩
```

The proof builds the two directions of the `↔` with the anonymous constructor `⟨_, _⟩`, each direction a function that applies `mono`. For this system, a context might as well be a set. That is the precise sense in which natural deduction "has" all three structural rules.

Derivable versus admissible, in Lean

The distinction between derivable and admissible rules is visible in the Lean proofs. Here is a derivable rule:

```
/-- A *derivable* rule: from `Γ ⊢ A ∧ B` conclude `Γ ⊢ B ∧ A`. The proof is a
fixed combination of primitive rules (a macro); it never looks inside `h`. -/
theorem conj_comm_rule (h : Proves Γ (conj A B)) : Proves Γ (conj B A) :=
  .conjI (.conjE₂ h) (.conjE₁ h)
```

The proof of `conj_comm_rule` is a fixed term built from primitive rules, with `h` plugged in twice. It never inspects `h`. (It also uses `h` twice, which is contraction by sharing once more.)

And here is a rule whose proof needs an admissible rule:

```
/-- Double-negation introduction as a rule: from `Γ ⊢ A` conclude
`Γ ⊢ (A → ⊥) → ⊥`. The derivation of `A` is needed in the bigger context
`(A → ⊥) :: Γ`, and no fixed combination of primitive rules can move it
there: we call the admissible `weakening`, which rebuilds `h` by induction. -/
theorem dni_rule (h : Proves Γ A) : Proves Γ (imp (imp A bot) bot) :=
  .impI (.impE (.hyp List.mem_cons_self) (weakening h))
```

To prove $(A \to \bot) \to \bot$ from Γ, we assume $A \to \bot$ and apply it to A. But the derivation `h` proves A from Γ, and we need A in the bigger context `imp A bot :: Γ`. No primitive rule does that, so we call `weakening h`, which runs the whole induction of `Proves.mono` over `h`. The rule `dni_rule` is admissible; whether it is also derivable is a different question, and it is not answered by this proof.

6.5 Counting uses

"The assumption is used twice", "the assumption is never used": these are statements about derivations, and so far we have checked them by looking at pictures. Let us make Lean count.

There is an obstacle. `Proves Γ C` is a proposition: it lives in `Prop`, and Lean does not let you compute data, such as a number, from a proof of a proposition. Lean treats any two proofs of the same proposition as interchangeable, so a function that returned different numbers for different proofs would make no sense. Chapter 10 explains this *proof irrelevance* in detail. For us it means that to count we need derivations that are *data*: a type in `Type`, whose values are trees we can inspect, just like the abstract syntax trees of Chapter 3.

Positions

When we count uses, we must know *which* assumption a leaf uses. The leaf `hyp mem` of `Proves` only records that the formula is a member of the context, and if the context is `[A, A]`, that does not say which copy is meant. So the data version refers to assumptions by position:

```
/-- `Var Γ A`: a position in the context `Γ` at which the formula `A` sits.
`here` is the first position, `there x` is one step further than `x`. -/
inductive Var : List Formula → Formula → Type where
  | here {Γ A} : Var (A :: Γ) A
  | there {Γ A B} : Var Γ A → Var (B :: Γ) A

/-- The position as a number: `here` is `0`, `there here` is `1`, and so on. -/
def Var.idx {Γ A} : Var Γ A → ℕ
  | .here => 0
  | .there x => x.idx + 1
```

`Var Γ A` is an **inductive family**: a type that depends on a context and a formula. A value `x : Var Γ A` is evidence that `A` sits in `Γ`, and it also says where. The constructor `here` points at the first element of a context `A :: Γ` and has type `Var (A :: Γ) A`. The constructor `there` takes a position in `Γ` and makes it a position in `B :: Γ`, one step further. So in the context `[A, B, C]`, the value `there here` has type `Var [A, B, C] B` and points at `B`. `Var.idx` turns a position into a number. (Programmers who have met de Bruijn indices will recognize them here; Chapter 7 introduces them for the λ-calculus, and Chapter 8 uses the same idea for the variables of typed terms.)

Derivations as data

The data version of natural deduction has the same rules as `Proves`, with `Type` instead of `Prop` and a position in the hypothesis rule:

```
/-- Derivations as data: the same rules as `Proves`, but a derivation is a
tree that we can inspect, and a hypothesis leaf records *which* position of
the context it uses. -/
inductive Deriv : List Formula → Formula → Type where
  | hyp {Γ A} : Var Γ A → Deriv Γ A
  | topI {Γ} : Deriv Γ top
  | botE {Γ C} : Deriv Γ bot → Deriv Γ C
  | conjI {Γ A B} : Deriv Γ A → Deriv Γ B → Deriv Γ (conj A B)
  | conjE₁ {Γ A B} : Deriv Γ (conj A B) → Deriv Γ A
  | conjE₂ {Γ A B} : Deriv Γ (conj A B) → Deriv Γ B
  | disjI₁ {Γ A B} : Deriv Γ A → Deriv Γ (disj A B)
  | disjI₂ {Γ A B} : Deriv Γ B → Deriv Γ (disj A B)
  | disjE {Γ A B C} :
      Deriv Γ (disj A B) → Deriv (A :: Γ) C → Deriv (B :: Γ) C → Deriv Γ C
  | impI {Γ A B} : Deriv (A :: Γ) B → Deriv Γ (imp A B)
  | impE {Γ A B} : Deriv Γ (imp A B) → Deriv Γ A → Deriv Γ B
```

The two versions agree. Forgetting the tree turns a `Deriv` into a `Proves`:

```
/-- A position holds a member of the context. -/
theorem Var.mem {Γ A} : Var Γ A → A ∈ Γ
  | .here => List.mem_cons_self
  | .there x => List.mem_cons_of_mem _ x.mem

/-- Forgetting the tree: every `Deriv` gives a `Proves`. -/
theorem Deriv.toProves {Γ C} : Deriv Γ C → Proves Γ C
  | .hyp x => .hyp x.mem
  | .topI => .topI
  | .botE d => .botE d.toProves
  | .conjI d e => .conjI d.toProves e.toProves
  | .conjE₁ d => .conjE₁ d.toProves
  | .conjE₂ d => .conjE₂ d.toProves
  | .disjI₁ d => .disjI₁ d.toProves
  | .disjI₂ d => .disjI₂ d.toProves
  | .disjE d e f => .disjE d.toProves e.toProves f.toProves
  | .impI d => .impI d.toProves
  | .impE d e => .impE d.toProves e.toProves
```

These two theorems are proved by recursion, written with pattern matching like a function: each constructor of `Var` or `Deriv` is sent to the corresponding proof, with recursive calls on the subtrees. (A proof by recursion on a tree and a proof by induction on it are the same thing; the `induction` tactic writes the recursion for you.) The file also proves the converse, in the theorem `proves_iff_nonempty : Proves Γ C ↔ Nonempty (Deriv Γ C)`, by an induction just like the ones above. `Nonempty` is the proposition "this type has at least one value", so the theorem says that `Proves Γ C` holds exactly when some tree of type `Deriv Γ C` exists.

The structural rules as renamings

In the data version, the structural rules become *renamings*: functions that send each position of one context to a position of another context holding the same formula. Renaming a derivation replaces every leaf `hyp x` by `hyp (ρ x)`:

```
/-- A renaming sends each position of `Γ` to a position of `Δ` that holds the
same formula. -/
abbrev Renaming (Γ Δ : List Formula) : Type := ∀ A, Var Γ A → Var Δ A
```

```
/-- Under a binder both contexts grow by the same formula `B`, which is sent
to itself. -/
def Renaming.lift {Γ Δ B} (ρ : Renaming Γ Δ) : Renaming (B :: Γ) (B :: Δ)
  | _, .here => .here
  | _, .there x => .there (ρ _ x)

/-- Renaming a derivation: replace every hypothesis leaf `hyp x` by
`hyp (ρ x)`. The shape of the tree does not change. -/
def Deriv.rename {Γ Δ C} (ρ : Renaming Γ Δ) : Deriv Γ C → Deriv Δ C
  | .hyp x => .hyp (ρ _ x)
  | .topI => .topI
  | .botE d => .botE (d.rename ρ)
  | .conjI d e => .conjI (d.rename ρ) (e.rename ρ)
  | .conjE₁ d => .conjE₁ (d.rename ρ)
  | .conjE₂ d => .conjE₂ (d.rename ρ)
  | .disjI₁ d => .disjI₁ (d.rename ρ)
  | .disjI₂ d => .disjI₂ (d.rename ρ)
  | .disjE d e f => .disjE (d.rename ρ) (e.rename ρ.lift) (f.rename ρ.lift)
  | .impI d => .impI (d.rename ρ.lift)
  | .impE d e => .impE (d.rename ρ) (e.rename ρ)
```

`Renaming Γ Δ` is the type of functions that, for every formula `A`, map positions of `A` in `Γ` to positions of `A` in `Δ` (`abbrev` defines it as an abbreviation that Lean unfolds freely). The function `Renaming.lift` extends a renaming to contexts with one more formula in front; `Deriv.rename` uses it when it goes under →I or into the branches of ∨E, where the context grows. A renaming is the data version of `Γ ⊆ Δ`, and `Deriv.rename` is the data version of `Proves.mono`. The three structural rules are three particular renamings:

```
/-- Weakening: every old position moves one step to make room for `B`. -/
def Deriv.weaken (d : Deriv Γ C) : Deriv (B :: Γ) C :=
  d.rename (fun _ x => .there x)

/-- Exchange: positions `0` and `1` swap, the others stay. -/
def swapFirstTwo : Renaming (A :: B :: Γ) (B :: A :: Γ)
  | _, .here => .there .here
  | _, .there .here => .here
  | _, .there (.there x) => .there (.there x)

def Deriv.exchange (d : Deriv (A :: B :: Γ) C) : Deriv (B :: A :: Γ) C :=
  d.rename swapFirstTwo

/-- Contraction: positions `0` and `1` both go to `0`, the others move down. -/
def mergeFirstTwo : Renaming (A :: A :: Γ) (A :: Γ)
  | _, .here => .here
  | _, .there .here => .here
  | _, .there (.there x) => .there x

def Deriv.contract (d : Deriv (A :: A :: Γ) C) : Deriv (A :: Γ) C :=
  d.rename mergeFirstTwo
```

Weakening shifts every position by one, to make room for the new assumption at the front. Exchange swaps positions 0 and 1. Contraction sends *both* positions 0 and 1 to position 0, and shifts the others down. Each definition is by pattern matching on the position: `.here` is position 0, `.there .here` is position 1, and `.there (.there x)` is everything else.

Counting

To count the leaves that use some position, we select positions with a boolean-valued function and walk the tree:

```
/-- A selection of positions of `Γ`: `true` for the positions we count. -/
abbrev Sel (Γ : List Formula) : Type := ∀ A, Var Γ A → Bool

/-- Under a binder the new position `0` is not selected; the old ones keep
their selection, one step further. -/
def Sel.lift {Γ B} (s : Sel Γ) : Sel (B :: Γ)
  | _, .here => false
  | _, .there x => s _ x

/-- The number of hypothesis leaves of `d` that use a selected position. -/
def Deriv.uses {Γ C} : Deriv Γ C → Sel Γ → ℕ
  | .hyp x, s => if s _ x then 1 else 0
  | .topI, _ => 0
  | .botE d, s => d.uses s
  | .conjI d e, s => d.uses s + e.uses s
  | .conjE₁ d, s => d.uses s
  | .conjE₂ d, s => d.uses s
  | .disjI₁ d, s => d.uses s
  | .disjI₂ d, s => d.uses s
  | .disjE d e f, s => d.uses s + e.uses s.lift + f.uses s.lift
  | .impI d, s => d.uses s.lift
  | .impE d e, s => d.uses s + e.uses s

/-- How many times `d` uses the hypothesis at position `i`. -/
def Deriv.count {Γ C} (d : Deriv Γ C) (i : ℕ) : ℕ :=
  d.uses (fun _ x => x.idx == i)

/-- The usage of every hypothesis, in context order. -/
def Deriv.usage {Γ C} (d : Deriv Γ C) : List ℕ :=
  (List.range Γ.length).map d.count
```

`Sel Γ` assigns `true` or `false` to every position of `Γ`. `Deriv.uses` adds 1 for every leaf whose position is selected. When it goes under a binder (the body of →I, the branches of ∨E), the context has one more formula in front, so the selection is lifted: the new position 0 is not selected, and the old positions keep their selection, one step further. `count d i` selects exactly position `i`, and `usage d` lists the counts of all positions in order. Here are the derivations from the beginning of the chapter, as data:

```
def A : Formula := atom "A"
def B : Formula := atom "B"

/-- `A ∧ B ⊢ B ∧ A`, as in `Structural.lean`, now as data. -/
def swapD : Deriv [conj A B] (conj B A) :=
  .conjI (.conjE₂ (.hyp .here)) (.conjE₁ (.hyp .here))

/-- `A, B ⊢ A`. -/
def firstD : Deriv [A, B] A := .hyp .here

/-- `A → B, A ⊢ B` (modus ponens). -/
def mpD : Deriv [imp A B, A] B := .impE (.hyp .here) (.hyp (.there .here))

/-- `A ⊢ A ∧ A`. -/
def dupD : Deriv [A] (conj A A) := .conjI (.hyp .here) (.hyp .here)

/-- info: [2] -/
#guard_msgs in
#eval swapD.usage

/-- info: [1, 0] -/
#guard_msgs in
#eval firstD.usage

/-- info: [1, 1] -/
#guard_msgs in
#eval mpD.usage

/-- info: [2] -/
```

```
#guard_msgs in
#eval dupD.usage
```

The pictures were right. The swap uses its only assumption twice, `A, B ⊢ A` uses `A` once and `B` never, modus ponens uses each of its two assumptions once, and `A ⊢ A ∧ A` uses `A` twice.

These counts let us name the disciplines of Part V. Call a derivation *affine* if every assumption is used at most once, *relevant* if every assumption is used at least once, and *linear* if every assumption is used exactly once:

```
/-- Every hypothesis is used at most once. -/
def Deriv.affine {Γ C} (d : Deriv Γ C) : Bool := d.usage.all (· ≤ 1)
/-- Every hypothesis is used at least once. -/
def Deriv.relevant {Γ C} (d : Deriv Γ C) : Bool := d.usage.all (1 ≤ ·)
/-- Every hypothesis is used exactly once. -/
def Deriv.linear {Γ C} (d : Deriv Γ C) : Bool := d.usage.all (· == 1)

/-- The most specific of the three disciplines that a derivation follows. -/
def Deriv.discipline {Γ C} (d : Deriv Γ C) : String :=
  if d.linear then "linear"
  else if d.affine then "affine"
  else if d.relevant then "relevant"
  else "unrestricted"

/-- info: ["relevant", "affine", "linear", "relevant"] -/
#guard_msgs in
#eval [swapD.discipline, firstD.discipline, mpD.discipline, dupD.discipline]
```

`Deriv.discipline` reports the most specific of the three (a linear derivation is also affine and relevant). Only modus ponens is linear. The swap and the duplication are relevant but not affine; `A, B ⊢ A` is affine but not relevant.

How the structural rules change the counts

Now apply the structural rules to a derivation and watch the counts. `twoCopiesD` proves `A, A ⊢ A ∧ A` using each copy once:

```
/-- `A, A ⊢ A ∧ A`, each copy used once. -/
def twoCopiesD : Deriv [A, A] (conj A A) := .conjI (.hyp .here) (.hyp (.there .here))

/-- info: [1, 1] -/
#guard_msgs in
#eval twoCopiesD.usage

/-- info: [2] -/
#guard_msgs in
#eval twoCopiesD.contract.usage

/-- info: [0, 1, 1] -/
#guard_msgs in
#eval (twoCopiesD.weaken (B := B)).usage

/-- info: [1, 1] -/
#guard_msgs in
#eval mpD.exchange.usage
```

Contraction merges the two copies, and the merged assumption is used twice. Weakening adds a new assumption at the front, used zero times. Exchange swaps the counts (here both are 1). These are not coincidences of the examples; they are theorems. The cleanest statement is about contraction:

```
/-- Contraction adds up the uses of the two merged hypotheses. -/
theorem count_contract (d : Deriv (A :: A :: Γ) C) :
    d.contract.count 0 = d.count 0 + d.count 1 := by
  unfold Deriv.contract Deriv.count
  rw [Deriv.uses_rename, ← Deriv.uses_or]
  -- A position goes to 0 exactly when it is 0 or 1 (and it is never both).
  have hor : (fun X (x : Var (A :: A :: Γ) X) => ((mergeFirstTwo X x).idx == 0)) =
      fun _ x => (x.idx == 0 || x.idx == 1) := by
    funext _ x
    rcases x with _ | _ | x <;> simp [mergeFirstTwo, Var.idx]
  have hand : (fun X (x : Var (A :: A :: Γ) X) => (x.idx == 0 && x.idx == 1)) =
      fun _ _ => false := by
    funext _ x
    cases x.idx <;> simp
  rw [hor, hand, Deriv.uses_none, Nat.add_zero]
```

The uses of the merged assumption are the sum of the uses of the two copies. To prove it we need a lemma about renaming in general:

```
/-- The key lemma: counting the selected uses in a renamed derivation is the
same as counting, in the original derivation, the positions whose image is
selected. -/
theorem Deriv.uses_rename (d : Deriv Γ C) (ρ : Renaming Γ Δ) (s : Sel Δ) :
    (d.rename ρ).uses s = d.uses (fun X x => s X (ρ X x)) := by
  induction d generalizing Δ with
  | hyp x => rfl
  | topI => rfl
  | botE d ih => exact ih ρ s
  | conjI d e ihd ihe => simp only [rename, uses, ihd, ihe]
  | conjE₁ d ih => exact ih ρ s
  | conjE₂ d ih => exact ih ρ s
  | disjI₁ d ih => exact ih ρ s
  | disjI₂ d ih => exact ih ρ s
  | disjE d e f ihd ihe ihf =>
    simp only [rename, uses, ihd, ihe, ihf, Sel.lift_rename]
  | impI d ih =>
    simp only [rename, uses, ih, Sel.lift_rename]
  | impE d e ihd ihe => simp only [rename, uses, ihd, ihe]
```

It says: to count the selected leaves of a renamed derivation, count the leaves of the original derivation whose position is *sent* to a selected position. The proof is an induction on the derivation, again generalizing over the target context. In the `hyp` case both sides are the same expression, so `rfl` closes it. In the cases with one subtree and no new assumption, the induction hypothesis is exactly the goal (`exact ih ρ s`). In the cases with two subtrees, `simp only [rename, uses, ihd, ihe]` unfolds one step of `rename` and of `uses` and rewrites with the induction hypotheses. In the binder cases (`disjE` and `impI`) we also need the fact that lifting a selection and lifting a renaming fit together, which is the first of four small lemmas:

```
/-- Lifting commutes with renaming, with "nothing" and with `||` and `&&`.
Each proof looks at the two kinds of positions, `here` and `there x`. -/
theorem Sel.lift_rename (ρ : Renaming Γ Δ) (s : Sel Δ) :
    (fun X (x : Var (B :: Γ) X) => s.lift X (ρ.lift X x)) =
      Sel.lift (fun X x => s X (ρ X x)) := by
  funext _ x; cases x <;> rfl

theorem Sel.lift_none :
    Sel.lift (B := B) (fun _ _ => false : Sel Γ) = fun _ _ => false := by
  funext _ x; cases x <;> rfl

theorem Sel.lift_or (s t : Sel Γ) :
    Sel.lift (B := B) (fun X x => s X x || t X x) =
      fun X x => s.lift X x || t.lift X x := by
```

```
  funext _ x; cases x <;> rfl

theorem Sel.lift_and (s t : Sel Γ) :
    Sel.lift (B := B) (fun X x => s X x && t X x) =
      fun X x => s.lift X x && t.lift X x := by
  funext _ x; cases x <;> rfl
```

Each of them is an equation between two selections, which are functions, so the proof starts with `funext _ x`: two functions are equal if they agree on every argument, here every formula `_` and position `x`. Then `cases x` splits into the two kinds of position, `here` and `there x`, and the combinator `<;>` runs `rfl` on both resulting goals. In each case both sides compute to the same boolean.

Two more lemmas, `Deriv.uses_none` (selecting nothing counts nothing) and `Deriv.uses_or` (the count of a union plus the count of an intersection is the sum of the counts), are proved by the same kind of induction; their proofs use `omega`, the decision procedure for linear arithmetic, to add up the counts of the subtrees.

Now read the proof of `count_contract` again. `unfold` replaces `contract` and `count` by their definitions, and `rw [Deriv.uses_rename]` turns the left side into a count over the original derivation `d`, with the selection "positions that `mergeFirstTwo` sends to 0". The rewrite `← Deriv.uses_or` turns the right side into a count of a union plus a count of an intersection. The two `have` steps then prove that the selections are what we want. `hor` says that a position is sent to 0 exactly when it is 0 or 1; it is proved by `funext` (two functions are equal if they agree on every argument) and a case analysis with `rcases` on the position (`here`, `there here`, or `there (there x)`), each case closed by `simp` computing `Var.idx` and the boolean equalities. `hand` says that no position is both 0 and 1. The final `rw` replaces the selections, counts nothing for the empty intersection, and drops the `+ 0`.

The weakening and exchange theorems are proved the same way:

```
/-- Weakening adds a hypothesis that is used zero times... -/
theorem count_weaken (d : Deriv Γ C) : (d.weaken (B := B)).count 0 = 0 := by
  unfold Deriv.count Deriv.weaken
  rw [Deriv.uses_rename]
  -- The selection is now `fun X x => (x.idx + 1 == 0)`: always false.
  have hsel : (fun X (x : Var Γ X) => (Var.idx (Var.there (B := B) x) == 0)) =
      fun _ _ => false := by
    funext _ x; simp [Var.idx]
  rw [hsel, Deriv.uses_none]

/-- ...and leaves the uses of the old hypotheses unchanged. -/
theorem count_weaken_succ (d : Deriv Γ C) (i : ℕ) :
    (d.weaken (B := B)).count (i + 1) = d.count i := by
  unfold Deriv.count Deriv.weaken
  rw [Deriv.uses_rename]
  simp [Var.idx]
```

```
/-- Exchange swaps the uses of the first two hypotheses. -/
theorem count_exchange (d : Deriv (A :: B :: Γ) C) :
    d.exchange.count 0 = d.count 1 ∧ d.exchange.count 1 = d.count 0 := by
  unfold Deriv.exchange Deriv.count
  rw [Deriv.uses_rename, Deriv.uses_rename]
  constructor
  · congr 1
    funext _ x
    rcases x with _ | _ | x <;> simp [swapFirstTwo, Var.idx]
  · congr 1
```

```
    funext _ x
    rcases x with _ | _ | x <;> simp [swapFirstTwo, Var.idx]
```

Put together, these theorems say something worth stating in words. Exchange moves counts around without changing them. Weakening introduces an assumption with count 0, and leaves the other counts alone. Contraction is the only one of the three that can produce a count greater than 1 from counts that were at most

1. So a logic without contraction keeps every assumption at *most once*, and a logic without weakening keeps every assumption at *least once*. That is the whole idea of Part V in one paragraph, and Chapter 22 proves the "at most once" half as a theorem about a programming language.

6.6 A first glance at Rust

The structural rules are about contexts, and in a program the context is the set of variables in scope. So the question "which structural rules does Rust have?" becomes "what may a Rust program do with the variables in scope?". The answers are in this chapter's Rust example, `code/rust/examples/ch06_structural.rs`, and in two programs that the compiler rejects.

One caveat first, the same one Chapter 1 made about typing rules. Rust has no official formal semantics, and the part of the compiler that rejects a use of a moved value is not literally a sequent calculus: it is a dataflow analysis over the compiler's intermediate representation, which Chapter 29 describes. When we say "Rust refuses contraction", we are describing the compiler's behavior with the vocabulary of the rules. The description is accurate for the cases in this chapter, and Parts V and VI refine it.

The example uses two kinds of values. A `Connection` is a resource: it is not `Copy`, not `Clone`, and it prints a message when it is dropped. A `Coupon` can be cloned, but it is not `Copy`.

```
/// A connection to a server: a resource. Neither `Copy` nor `Clone`, and it
/// announces when it is closed (dropped).
struct Connection {
    host: String,
}

impl Connection {
    fn open(host: &str) -> Connection {
        println!("  open {host}");
        Connection {
            host: host.to_string(),
        }
    }
}

impl Drop for Connection {
    fn drop(&mut self) {
        println!("  close {}", self.host);
    }
}

/// A discount coupon. It can be cloned (printing a second coupon is a
/// deliberate act), but it is not `Copy`.
#[derive(Clone, Debug, PartialEq)]
struct Coupon {
```

```
    code: String,
}
```

Exchange

Rust keeps exchange. The order in which variables were declared, or in which parameters are listed, does not restrict how they can be used:

```
/// Exchange: from `a: A, b: B` we can build `(B, A)` as easily as `(A, B)`.
/// The order in which assumptions are listed is irrelevant to typing.
fn exchange<A, B>(a: A, b: B) -> (B, A) {
    (b, a)
}
```

But exchange is a statement about *typing*. Evaluation has an order, and Rust fixes it: the operands of a tuple or the arguments of a call are evaluated left to right, and local variables are dropped in the reverse order of their declaration.

```
println!("exchange:");
let (b, a) = exchange("order #17", 42_u32);
println!("  exchange(\"order #17\", 42) = ({b}, {a:?})");
// Typing does not care about order; evaluation does: left to right.
let pair = (note("left"), note("right"));
assert_eq!(pair, ("left", "right"));
// Drop order is fixed too: reverse order of declaration.
{
    let _db = Connection::open("db.internal");
    let _cache = Connection::open("cache.internal");
    println!("  end of scope");
}
```

```
exchange:
  exchange("order #17", 42) = (42, "order #17")
  evaluate left
  evaluate right
  open db.internal
  open cache.internal
  end of scope
  close cache.internal
  close db.internal
```

The connection to the cache was opened second and is closed first. If you swap the two `let` lines, the program still type checks, as exchange promises, but the output changes. A logic does not see effects; a programming language must specify them. Chapter 24 studies drop order in detail.

Weakening, with an effect

Rust keeps weakening. A function may ignore a parameter:

```
/// Weakening: the assumption `_unused` is never used. Rust accepts this, and
/// the value is dropped when the function returns.
fn weaken<A, B>(a: A, _unused: B) -> A {
    a
}
```

The leading underscore in `_unused` tells the compiler that the omission is deliberate. Without it you would get a warning from the `unused_variables` lint, and a warning is all you would get: a lint is

advice, not a typing rule. A language that rejected unused variables would be a *relevant* type system, and Rust is not one.

What Rust adds to weakening is an effect. A value that goes unused is still owned by someone, and when its owner goes out of scope, the value is dropped, which runs its destructor:

```
println!("weakening:");
let total = weaken(250_u32, Connection::open("audit.internal"));
println!("  weaken returned {total}");
let unused = Connection::open("metrics.internal");
drop(unused); // weakening made explicit: discard the assumption now
println!("  after drop");
```

```
weakening:
  open audit.internal
  close audit.internal
  weaken returned 250
  open metrics.internal
  close metrics.internal
  after drop
```

The audit connection is closed when `weaken` returns, before its result is printed: the unused parameter was dropped at the end of the function's body. The call `drop(unused)` is weakening made explicit. The standard library's `std::mem::drop` is a function that takes its argument by value and does nothing with it; its body is empty. Taking ownership and doing nothing is precisely discarding an assumption, and the destructor runs because the argument goes out of scope at the end of `drop`. Chapter 24 calls this *weakening with a side effect* and studies its consequences.

Contraction, refused

Rust does not have contraction for values that are not `Copy`. Here is a program that uses a coupon twice:

```
pub fn checkout(coupon: Coupon) -> u32 {
    let first = redeem(coupon);
    let second = redeem(coupon);
    first + second
}
```

```
error[E0382]: use of moved value: `coupon`
  --> compile_fail/ch06_use_after_move.rs:16:25
   |
14 | pub fn checkout(coupon: Coupon) -> u32 {
   |                 ------ move occurs because `coupon` has type `Coupon`, which does not
implement the `Copy` trait
15 |     let first = redeem(coupon);
   |                        ------ value moved here
16 |     let second = redeem(coupon);
   |                         ^^^^^^ value used here after move
   |
note: consider changing this parameter type in function `redeem` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch06_use_after_move.rs:9:23
   |
 9 | pub fn redeem(coupon: Coupon) -> u32 {
   |        ------         ^^^^^^ this parameter takes ownership of the value
   |        |
   |        in this function
note: if `Coupon` implemented `Clone`, you could clone the value
  --> compile_fail/ch06_use_after_move.rs:5:1
```

```
   |
 5 | pub struct Coupon {
   | ^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
15 |     let first = redeem(coupon);
   |                        ------ you could clone this value
```

Read the diagnostic with the rules in mind. "Move occurs because `coupon` has type `Coupon`, which does not implement the `Copy` trait" says that the assumption `coupon` is a resource, not a fact. "Value moved here" marks the first use, which consumed it. "Value used here after move" marks the second use, which would need contraction: the body uses one variable twice, which a typing derivation could only justify by merging two assumptions `coupon1` and `coupon2` into one. The compiler then offers two ways out. The first note suggests changing `redeem` to *borrow* the coupon, so that using it does not consume it; borrowing is the subject of Part VI. The second suggests implementing `Clone`, so that the duplication is done explicitly by a function call.

Contraction in a loop

A loop is a quieter way to ask for contraction. The body of a loop is checked once but runs many times, so a value moved in the body would be used once per iteration:

```
pub fn send_all(conn: Connection, orders: Vec<u32>) {
    for order in orders {
        send(conn, order);
    }
}
```

```
error[E0382]: use of moved value: `conn`
  --> compile_fail/ch06_move_in_loop.rs:16:14
   |
14 | pub fn send_all(conn: Connection, orders: Vec<u32>) {
   |                 ---- move occurs because `conn` has type `Connection`, which does not
implement the `Copy` trait
15 |     for order in orders {
   |     ------------------- inside of this loop
16 |         send(conn, order);
   |              ^^^^ value moved here, in previous iteration of loop
   |
note: consider changing this parameter type in function `send` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch06_move_in_loop.rs:9:19
   |
 9 | pub fn send(conn: Connection, order: u32) {
   |        ----       ^^^^^^^^^^ this parameter takes ownership of the value
   |        |
   |        in this function
note: if `Connection` implemented `Clone`, you could clone the value
  --> compile_fail/ch06_move_in_loop.rs:5:1
   |
 5 | pub struct Connection {
   | ^^^^^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
16 |         send(conn, order);
   |              ---- you could clone this value
```

"Value moved here, in previous iteration of loop" is the compiler's way of saying that the variable `conn` would be used as many times as there are orders. The fix in the example program is to *borrow* the connection in the loop body, reading `conn.host` through the variable without moving it out:

```
println!("loops:");
let conn = Connection::open("orders.internal");
for order in [101, 102, 103] {
    // Each iteration uses `conn`; borrowing makes that legal.
    println!("  send order {order} via {}", conn.host);
}
println!("  done sending");
```

```
loops:
  open orders.internal
  send order 101 via orders.internal
  send order 102 via orders.internal
  send order 103 via orders.internal
  done sending
  close orders.internal
```

The connection is used three times and closed once, at the end of `main`.

Contraction restored: `Clone` and `Copy`

There are two ways to get contraction back, and they are different in an instructive way:

```
/// Contraction, paid for: duplicating a value is an explicit call to `clone`.
fn contract_clone<A: Clone>(a: A) -> (A, A) {
    (a.clone(), a)
}

/// Contraction, for free: a `Copy` value is duplicated bit by bit, silently.
fn contract_copy<A: Copy>(a: A) -> (A, A) {
    (a, a)
}
```

`contract_clone` duplicates explicitly: the program calls `clone`, which may allocate memory and run arbitrary code. In the vocabulary of this chapter, `Clone` does not give you the structural rule; it gives you a *function* of type `A -> (A, A)`, and the program calls it. `contract_copy` duplicates implicitly: for `Copy` types, using a variable does not move it, the bits are copied, and using `a` twice needs nothing at all. For the types that implement `Copy` (integers, `bool`, `char`, shared references `&T`, and tuples and arrays of `Copy` types), Rust *has* contraction, silently.

```
println!("contraction:");
let coupon = Coupon {
    code: "SPRING-10".to_string(),
};
let (first, second) = contract_clone(coupon);
assert_eq!(first, second);
println!("  contract_clone gave {first} and {second}");
let (x, y) = contract_copy(7_u64);
println!("  contract_copy(7) = ({x}, {y})");
// A shared reference is `Copy`: duplicating it duplicates a permission to
// read, not the coupon (Part VI).
let (r1, r2) = contract_copy(&first);
println!(
    "  two references to the same {r1}: {}",
    std::ptr::eq(r1, r2)
);
```

```
contraction:
  contract_clone gave coupon SPRING-10 and coupon SPRING-10
  contract_copy(7) = (7, 7)
  two references to the same coupon SPRING-10: true
```

The last line deserves a second look. A shared reference `&Coupon` is `Copy`, so contraction applies to it: two references to the same coupon, as `std::ptr::eq` confirms. Duplicating a reference does not duplicate the coupon; it duplicates a *permission to read* it. That observation is the seed of Part VI, which argues that references are better explained by permissions and regions than by affinity.

Here is the summary of this first glance:

Rule	In Rust	Observed as
Exchange	kept	order of `let`s and parameters is irrelevant to typing
Weakening	kept, with an effect	unused values are allowed; `Drop` runs at scope end
Contraction	refused for non-`Copy` values	E0382, "use of moved value"
Contraction	kept for `Copy` types	`i32`, `bool`, `&T` used any number of times
Contraction	explicit via `Clone`	a call to `clone()`

Weakening without contraction has a name: it is the definition of an **affine** discipline. This table is the first, rough evidence for the claim that Chapter 1 called the question of the book, and Chapter 33 returns to it with all the evidence in hand.

6.7 The resource reading of assumptions

Why would anyone want a logic without contraction? Because of what assumptions stand for.

In mathematics, an assumption is a *fact*. If you know that n is even, you can use that fact once, twice or not at all, and it does not wear out. Contraction is the right rule for facts, and so is weakening: an irrelevant fact does no harm.

Now let an assumption stand for a *thing*: a coin, a ticket, an open file, a network connection, a heap buffer. Using it means spending it, handing it over, closing it. Contraction is now the wrong rule: from one coin it would conclude two. Jean-Yves Girard, who introduced *linear logic* in 1987, liked to explain it with purchases. Suppose a coin buys a coffee. If assumptions may be duplicated, then from one coin you get a coffee *and* still have the coin, and so you can buy as many coffees as you like. A logic that talks about resources must not have contraction.

Weakening is a subtler case. Throwing a thing away is sometimes fine (an unused coupon can expire) and sometimes not (an opened transaction must be committed or rolled back, a promise must be kept). A logic that keeps weakening but drops contraction describes resources that may be discarded but not copied: it is *affine*. A logic that drops both describes resources that must be used exactly once: it is *linear*.

In a programming language, a variable of a type that owns something (a `String` owns a heap buffer, a `File` owns a file descriptor) is exactly such an assumption. Two uses of it would mean two owners of one buffer, and in a language without a garbage collector, two owners means two frees: a double free, one of the classic memory-safety bugs. Refusing contraction is how Rust rules it out before the program runs.

The design decision

The move that generates all of Part V is easy to state: take a logic and delete some of its structural rules. What remains is a **substructural** logic. Which rules you delete decides what an assumption is: facts (keep all three), discardable resources (delete contraction), obligations that must be met (delete weakening), resources that must be used exactly once (delete both), or even resources that must be used in order (delete exchange as well).

But deleting a rule is not enough when the rule is hidden in other rules, and we have seen that natural deduction hides contraction in rule ∧I. If the context $\Gamma = x : A$ is given to *both* premises, both can use x, and contraction is back no matter what we deleted. A substructural logic must replace context *sharing* by context *splitting*: the conclusion's assumptions are divided between the premises, each assumption going to exactly one of them. The conjunction with a splitting rule is written ⊗ ("tensor"):

$$\frac{\Gamma \vdash A \qquad \Delta \vdash B}{\Gamma, \Delta \vdash A \otimes B}\ \otimes \mathsf{I}$$

To prove $A \otimes B$ from Γ, Δ, prove A from Γ and B from Δ. Similarly, weakening hides in rule Hyp, which accepts any context that contains A; a logic without weakening uses the stricter axiom $A \vdash A$, in which the context is exactly A.

Splitting the context is literally what Rust does when it checks a call `f(a, b)`: the owned variables in scope must be divided between the two arguments, and if both arguments need the same non-`Copy` variable, no division exists and the compiler reports E0382.

This is where the seed is planted. Part V grows it:

- Chapter 21 deletes the structural rules one by one and studies the resulting family of logics. It finds that, without contraction, conjunction splits into two different connectives ($A \otimes B$, which splits the context, and $A \,\&\, B$, which shares it), and it builds a proof search for linear and affine logic that shows exactly which sequents survive.
- Chapter 22 builds an affine λ-calculus, with the typing rules of a small programming language, and proves that every variable is used at most once.
- Chapter 23 explains `Copy` and `Clone` as a controlled way to bring contraction back, the *exponential* modality of linear logic.
- Chapter 24 studies weakening with a side effect: `Drop`, leaks and why Rust is affine rather than linear.
- Chapter 25 uses affinity to put protocols into types.

6.8 Putting it together: one formula, three views

Let us follow a single formula through the whole chapter: $A \to A \wedge A$, "from A, conclude A and A". Its proof needs one assumption twice, so it is the purest test of contraction.

The mathematics. In natural deduction, →I reduces the goal to $A \vdash A \wedge A$, and ∧I splits it into two premises that both receive the context A:

$$\dfrac{\dfrac{\dfrac{A \in A}{A \vdash A}\,\text{Hyp} \qquad \dfrac{A \in A}{A \vdash A}\,\text{Hyp}}{A \vdash A \wedge A}\,\wedge\text{I}}{\vdash A \to A \wedge A}\,\to\text{I}$$

With an explicit contraction step, the same proof reads: prove $A, A \vdash A \wedge A$ using each copy once, then contract. In a logic with $\otimes$I instead of $\wedge$I and without contraction, the context A would have to be split between the two premises, one of them would receive nothing, and the proof would fail. Chapter 21 proves, in Lean, that $A \vdash A \otimes A$ has *no* derivation at all in linear logic, by an invariant that counts atoms.

Lean. The derivation `dupD` has usage `[2]`. The derivation `twoCopiesD` of `A, A ⊢ A ∧ A` has usage `[1, 1]`: it is linear. Applying `Deriv.contract` to it produces a derivation with usage `[2]`, and the theorem `count_contract` guarantees that this is no accident: contraction adds the counts.

Rust. The formula becomes a generic function type, `fn dup<T>(x: T) -> (T, T)`. With the body `(x, x)` and no bound on `T`, the compiler rejects it with E0382, exactly as it rejected `checkout`. With the bound `T: Clone`, the body `(x.clone(), x)` is accepted: contraction paid for by a function call. With the bound `T: Copy`, the body `(x, x)` is accepted as it stands: contraction built in for a class of types. Three bodies, three different relationships with one structural rule.

Compare $A \to (B \to A)$, which needs weakening instead. Lean's `firstD` has usage `[1, 0]`, an affine derivation. Rust's `weaken` compiles without complaint and drops its second argument. One structural rule is kept, the other is not: that asymmetry is the shape of Rust's ownership discipline.

6.9 Common confusions

"**Weakening means deleting an assumption.**" Read top-down, weakening *adds* an assumption to the context of a sequent that was already provable. The intuition "discard an assumption" is the bottom-up reading, from the goal toward the premises. Both are correct readings of the same rule; mixing them up makes statements like "affine logic has weakening" sound backwards.

"**Natural deduction has structural rules among its rules.**" It does not, in the presentation of Chapter 4. The structural rules are admissible: they are theorems *about* the system, proved by induction, not rules *of* the system. Some presentations (especially of the sequent calculus) include them as primitive rules; the derivable sequents are the same either way.

"**Rust has no contraction.**" Rust has no *implicit* contraction for values that are not `Copy`. For `Copy` types, including every shared reference `&T`, contraction is silent and free, and `Clone` provides explicit duplication for many other types. Saying exactly where contraction is available is a large part of the answer to the book's question.

"**The unused-variable warning means Rust requires every variable to be used.**" It is a lint, and it can be silenced with a leading underscore or turned off. A type system that required every variable to be used would be relevant, and `#[must_use]`, which Chapter 24 discusses, is the closest Rust comes to it.

"**Exchange means that evaluation order does not matter.**" Exchange says that the order of assumptions does not matter for *derivability*, or for typing. Rust evaluates operands left to right and drops locals in reverse order of declaration, and programs can observe both.

"**An admissible rule is weaker than a derivable one.**" For the system as it is, they license exactly the same conclusions. The difference is robustness: a derivable rule survives the addition of new rules, while an admissible rule must be proved again, and may fail.

6.10 Summary

- The structural rules (exchange, weakening, contraction) mention no connective; they say what may be done with the context. Weakening is discarding, contraction is duplicating, exchange is freedom of order.
- Read top-down, weakening adds and contraction merges; the slogans "discard" and "duplicate" are the bottom-up readings.
- A rule can be primitive, derivable (a fixed macro of primitive rules) or admissible (its conclusion is derivable whenever its premises are, usually by induction). Derivable rules survive extensions of a system; admissible ones may not.
- Contexts can be lists, multisets or sets. Multisets build exchange in; sets also build in contraction.
- Natural deduction has all three structural rules, admissibly. They hide in the hypothesis rule and in the sharing of contexts between premises. Lean proves it through monotonicity: `Proves Γ C` and `Γ ⊆ Δ` give `Proves Δ C`.
- With derivations as data, the structural rules are renamings of positions. Weakening adds an assumption used zero times; contraction adds up the uses of two assumptions; only contraction can push a count above one.
- Rust keeps exchange (with a fixed evaluation and drop order), keeps weakening (with the effect of running destructors), refuses contraction for non-`Copy` values (E0382), restores it for `Copy` types, and offers it explicitly via `Clone`.
- An assumption can be a fact or a resource. Deleting structural rules, and replacing context sharing by context splitting, gives the substructural logics of Part V.

6.11 Exercises

1. Write out, on paper, the case for rule $\vee$E in the proof that weakening is admissible: assume derivations of $\Gamma \vdash A \vee B$, $\Gamma, A \vdash C$ and $\Gamma, B \vdash C$, and build one of $\Gamma, D \vdash C$. Where do you need exchange, if you insist on adding D at the end of the context? *Hint:* the induction hypothesis applied to the second premise gives $\Gamma, A, D \vdash C$, but $\vee$E needs a premise whose context ends with A.
2. In Lean, prove contraction at an arbitrary position: `theorem contraction_mid (h : Proves (Γ₁ ++ A :: A :: Γ₂) C) : Proves (Γ₁ ++ A :: Γ₂) C`. Then prove `Proves [A, B, A] C → Proves [B, A] C` using only `exchange`, `contraction` and `weakening`

(not `Proves.mono`). *Hint:* for the first, imitate the proof of `exchange`. For the second, bring the two copies of `A` next to each other first: `exchange (Γ₁ := [A])` swaps the second and third formulas, and `exchange (Γ₁ := [])` swaps the first two. The whole proof is one line.

3. Build a `Deriv` of $A \to B, A \to C, A \vdash B \wedge C$ in Lean and `#eval` its usage. Is it affine, relevant, linear? Then build a derivation of $(A \to A \to B) \to A \to B$ (an open version is enough: $A \to A \to B, A \vdash B$) and check that it uses the assumption A twice. *Hint:* the context is `[imp A B, imp A C, A]`, and `A` is at position `.there (.there .here)`. The second formula is the logician's combinator W, the standard test for contraction.
4. Write three Rust functions with the types of the formulas $A \wedge B \to A$, $(A \to B) \to (A \to C) \to A \to B \wedge C$ and $A \wedge B \to B \wedge A$, for generic `A`, `B`, `C` with no trait bounds (use `FnOnce` for the function arguments). Which ones compile? For the one that does not, say which structural rule it needs and add the smallest trait bound that makes it compile. *Hint:* in the second, the argument of type `A` is needed by both functions.
5. Add a new primitive rule to `Proves`: the axiom `Proves [] (atom "p")`, which is available only in the empty context. Show that weakening is no longer admissible: `Proves [] (atom "p")` holds but `Proves [atom "q"] (atom "p")` does not. Why does adding the same axiom in every context, `Proves Γ (atom "p")`, keep weakening admissible? *Hint:* in every rule, the contexts of the premises contain the context of the conclusion, so a derivation of $q \vdash p$ never reaches the empty context. Without the new axiom, a truth-table argument (Chapter 4's soundness theorem) shows that $q \vdash p$ is not derivable.
6. The theorem `count_weaken` is about weakening at the front of the context. State and prove in Lean that weakening does not change the *total* number of hypothesis leaves: `(d.weaken (B := B)).uses (fun _ _ => true) = d.uses (fun _ _ => true)`. *Hint:* rewrite with `Deriv.uses_rename`; the selection on the right-hand side is then the constant `true` function, and `rfl` may finish the proof.

PART II

Programs as Proofs

Part II is about the central discovery of the subject: proofs and programs are the same thing. It starts with the untyped λ-calculus, the smallest programming language there is, then adds simple types and meets Rust's closures along the way. It proves, completely and mechanically in Lean, that well-typed programs do not go wrong. With that in hand, the Curry–Howard correspondence stops being a slogan: propositions are types, proofs are programs, and simplifying a proof is running a program. The part ends with the algebra of types, where sums, products and functions obey the laws of arithmetic.

CHAPTER 7

The Lambda Calculus

Every closure you have written in Rust, `|x| x + 1`, `|order| order.total()`, `move || report`, descends from a notation that Alonzo Church introduced in the 1930s to study what it means for a function to be computable. His **λ-calculus** has exactly three constructs: variables, functions and applications. There are no numbers, no booleans, no loops and no data structures, and yet it can express every computation a Rust program can. It is the smallest programming language there is, and nearly every idea in Part II of this book (types, type safety, the Curry–Howard correspondence) is first stated for it.

This chapter studies the λ-calculus *without* types. That may seem like an odd place to start a book about type theory, but it is the right one: to see what types buy you, you need to see the language they constrain. You will meet programs that run forever, functions that take themselves as arguments, and numbers built out of nothing but functions. You will also meet the operation at the heart of every programming language implementation, **substitution**, and the subtle bug, *variable capture*, that makes it harder than it looks. In Chapter 8 we add types and watch most of the strangeness disappear.

In this chapter you will learn:

- the syntax of λ-terms, free and bound variables, and α-equivalence;
- capture-avoiding substitution, why the naive version is wrong, and how de Bruijn indices remove the problem altogether;
- β-reduction, normal forms, the reduction strategies normal order, call-by-name and call-by-value, and the Church–Rosser theorem;
- divergence (Ω) and recursion without names (the Y and Z combinators);
- Church encodings of booleans, pairs, numerals and arithmetic, computed by the book's `lambda` crate;
- how Lean represents λ-terms, defines reduction as an inductive relation, proves facts about it, and why it refuses a normalizer that has no fuel.

7.1 Three constructs

The terms of the untyped λ-calculus are given by a grammar with three cases:

$$t ::= x \mid \lambda x.\, t \mid t\ t$$

- A **variable** x.
- An **abstraction** $\lambda x.\, t$: the function with parameter x and body t. In Rust you would write `|x| t`.

- An **application** $t\ u$: the function t applied to the argument u. In Rust, `t(u)`.

That is all. The notation needs two conventions to keep parentheses under control. Application associates to the **left**, so $f\,a\,b$ means $(f\,a)\,b$: apply f to a, then apply the result to b. And the body of a λ extends **as far to the right as possible**, so $\lambda x.\, x\,y$ means $\lambda x.\, (x\,y)$, not $(\lambda x.\, x)\,y$.

Every function takes exactly one argument. A function of two arguments is a function that returns a function: $\lambda x.\, \lambda y.\, x$ takes x and returns a function that takes y and returns x. This is called **currying** (after Haskell Curry, although the idea is older), and we often abbreviate $\lambda x.\, \lambda y.\, t$ as $\lambda x\, y.\, t$. In Rust the same function is `|x| move |y| x`.

Some terms we will use over and over:

Term	Name	Rust closure
$\lambda x.\, x$	identity, I	`|x| x`
$\lambda x.\, \lambda y.\, x$	constant, K	`|x| move |_y| x`
$\lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x)$	composition	`|f| move |g| move |x| g(f(x))`
$\lambda x.\, x\,x$	self-application, δ	none (see the end of the chapter)
$(\lambda x.\, x\,x)\,(\lambda x.\, x\,x)$	Ω	none

The last two rows are the first sign that the untyped calculus can do things Rust cannot. We will come back to them.

The `lambda` crate

This chapter's Rust code is the crate `lambda` (in `code/rust/crates/lambda`). Its central type is the abstract syntax of λ-terms, an `enum` with one variant per grammar case:

```rust
/// A term of the untyped λ-calculus, with variables as names:
/// `t ::= x | λx. t | t t`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    /// A variable `x`.
    Var(String),
    /// An abstraction `λx. t`: the function that sends `x` to `t`.
    Lam(String, Box<Term>),
    /// An application `t u`: the function `t` applied to the argument `u`.
    App(Box<Term>, Box<Term>),
}
```

A `Lam` owns its body and an `App` owns both of its parts, so a term is a tree; the `Box`es are there because a recursive `enum` must have a known size (Chapter 11 explains why). The crate also has a small parser, so that examples can be written as text, `parse("λf. λx. f (f x)")` (with `\` accepted for `λ`), and a printer that follows the two conventions above and prints the minimum number of parentheses. We will not discuss the parser; it is a routine piece of code.

7.2 Free and bound variables

In $\lambda x.\, x\,y$ the two variables play different roles. The x refers to the parameter of the enclosing λ: it is **bound**. The y refers to nothing inside the term: it is **free**, and its meaning must come from outside,

the way a global variable or a variable from an enclosing scope gets its meaning in Rust. The set of free variables is defined by recursion on the term:

$$\mathrm{FV}(x) = \{x\} \qquad \mathrm{FV}(\lambda x.\, t) = \mathrm{FV}(t) \setminus \{x\} \qquad \mathrm{FV}(t\ u) = \mathrm{FV}(t) \cup \mathrm{FV}(u)$$

The middle equation is the only interesting one: a λ removes its own parameter from the free variables of its body. The **scope** of a binder λx is its body, and an occurrence of x is bound by the *nearest* enclosing λx. So in $\lambda x.\, \lambda x.\, x$ the occurrence of x belongs to the inner λ, exactly as an inner `let x` shadows an outer one in Rust. A term with no free variables is **closed**; closed terms are complete programs, open terms are fragments.

The same name can even be free and bound in one term: in $x\ (\lambda x.\, x)$ the first x is free and the second is bound. That is legal, but confusing, and you will see shortly that it is also where the bugs live. In the crate, `free_vars` is a direct transcription of the three equations:

```
/// The free variables of a term: the variables that are not bound by an
/// enclosing `λ`.
pub fn free_vars(&self) -> BTreeSet<String> {
    match self {
        Term::Var(x) => BTreeSet::from([x.clone()]),
        Term::Lam(x, body) => {
            let mut fv = body.free_vars();
            fv.remove(x);
            fv
        }
        Term::App(t, u) => {
            let mut fv = t.free_vars();
            fv.extend(u.free_vars());
            fv
        }
    }
}
```

The free variables of a function body are something you already know well under another name. When you write `|y| x + y` in Rust, the variable `x` is free in the closure body, and the compiler must arrange for the closure to have access to it: it **captures** `x`. The captures of a Rust closure are exactly the free variables of its body. Chapter 8 develops this connection.

7.3 Substitution, and the capture problem

How does a λ-term compute? By applying functions to arguments, and applying $\lambda x.\, t$ to s means: take the body t and replace every free occurrence of the parameter x by s. We write this operation $t[x := s]$, "t with s for x". (Chapters 4 and 6 wrote the same operation $t[s/x]$; both notations are common.) It is what a compiler does when it inlines a function call, and what you do by hand when you compute $(x \mapsto x^2 + 1)(3) = 3^2 + 1$.

The naive definition

The obvious definition replaces occurrences of x and stops at a λx, because inside it x means something else:

- $x[x := s] = s$, and $y[x := s] = y$ when $y \neq x$;
- $(t\ u)[x := s] = t[x := s]\ u[x := s]$;

- $(\lambda x.\, t)[x := s] = \lambda x.\, t$ (the parameter x shadows our x);
- $(\lambda y.\, t)[x := s] = \lambda y.\, t[x := s]$ when $y \neq x$.

A first example works fine. Substituting the identity for x in $x\, y$:

$$(x\ y)[x := \lambda z.\, z] = x[x := \lambda z.\, z]\ \ y[x := \lambda z.\, z] = (\lambda z.\, z)\ y$$

Now a second example. Take the constant function $\lambda y.\, x$: whatever you give it, it returns x. Substitute the variable y for x:

$$(\lambda y.\, x)[x := y] = \lambda y.\, (x[x := y]) = \lambda y.\, y$$

Something went badly wrong. We started with a function that ignores its argument and returns x, and we asked for the same function returning y instead (the y from outside, whatever it is). We ended with the identity function, which returns its argument. The free y that we substituted in has been **captured** by the λy that happened to be in the way: it now refers to the parameter, not to the outside variable it was supposed to mean.

This is not a contrived corner case. It changes the answers of programs. The term $K\, y\, z = (\lambda x.\, \lambda y.\, x)\, y\, z$ should evaluate to y, because K returns its first argument. Here it is, computed by a reducer that uses the naive substitution, next to the crate's real reducer (the naive substitution is written out in `crates/lambda/examples/ch07_capture.rs`, where it is clearly marked as wrong):

```
/// Textual replacement of `x` by `s`, stopping only at a `λx` that shadows
/// `x`. This is WRONG: it does not check whether a `λ` captures a free
/// variable of `s`.
fn naive_subst(t: &Term, x: &str, s: &Term) -> Term {
    match t {
        Term::Var(y) if y == x => s.clone(),
        Term::Var(_) => t.clone(),
        Term::Lam(y, _) if y == x => t.clone(),
        Term::Lam(y, body) => lam(y, naive_subst(body, x, s)),
        Term::App(f, a) => app(naive_subst(f, x, s), naive_subst(a, x, s)),
    }
}
```

```
(λy. x)[x := y]
  naive:            λy. y
  capture-avoiding: λy'. y

(λx. λy. x) y z
  with naive substitution:           z
  with capture-avoiding substitution: y
```

The naive reducer computes $(\lambda x.\, \lambda y.\, x)\, y\, z \to (\lambda y.\, y)\, z \to z$: the first step captured y, and the second step then returned the wrong argument.

You have met this bug before, in a different costume. In C, a macro that declares a local variable `x` and multiplies it by its argument goes wrong when the caller passes an expression that mentions the caller's own `x`: after textual expansion, that `x` refers to the macro's local variable instead. Rust's `macro_rules!` is **hygienic**: the local variables a macro introduces live in their own syntactic context and cannot capture the caller's, which is exactly capture avoidance. The chapter's closure example, `code/rust/examples/ch07_rust_lambdas.rs`, checks it:

```
/// Multiplies its argument by a local `x`. The `x` inside the macro and an `x`
/// at the call site are different variables: `macro_rules!` renames the
/// macro's local bindings, exactly like capture-avoiding substitution.
macro_rules! double_it {
    ($e:expr) => {{
        let x = 2;
        x * $e
    }};
}
```

```
double_it!(x) with x = 10 is 20, not 2 * 2 = 4
```

Capture-avoiding substitution

The fix is to notice that the *name* of a bound variable does not matter. The function $\lambda y.\, x$ is the same function as $\lambda w.\, x$: both ignore their argument and return x. So before substituting under a binder that would capture something, we rename the binder to a fresh name. The last case of the definition splits in two:

- $(\lambda y.\, t)[x := s] = \lambda y.\, t[x := s]$ when $y \neq x$ and $y \notin \mathrm{FV}(s)$: no danger, go ahead;
- $(\lambda y.\, t)[x := s] = \lambda y'.\, (t[y := y'])[x := s]$ when $y \neq x$ and $y \in \mathrm{FV}(s)$, where y' is a **fresh** name, one that occurs neither in t nor in s.

Let us redo the example step by step. We want $(\lambda y.\, x)[x := y]$.

1. The term is an abstraction $\lambda y.\, t$ with $t = x$, and the binder y is different from x, the variable we substitute for.
2. Is y free in the replacement $s = y$? Yes. If we went under the binder, the y in s would be captured. So we take the second case.
3. Choose a fresh name: y' occurs nowhere, so it will do.
4. Rename the binder: $t[y := y'] = x[y := y'] = x$, giving $\lambda y'.\, x$. This is the same function as before.
5. Now substitute safely: $\lambda y'.\, x[x := y] = \lambda y'.\, y$.

The result, $\lambda y'.\, y$, is a constant function returning the outside y, as it should be, and $K\, y\, z$ now evaluates to y. In the crate, the two cases look like this:

```
/// Capture-avoiding substitution `self[x := s]`: replace the free
/// occurrences of `x` in `self` by `s`, renaming bound variables of `self`
/// when they would capture a free variable of `s`.
pub fn subst(&self, x: &str, s: &Term) -> Term {
    match self {
        Term::Var(y) if y == x => s.clone(),
        Term::Var(_) => self.clone(),
        Term::App(t, u) => app(t.subst(x, s), u.subst(x, s)),
        // `λx. body` binds x: there is no free x inside, nothing to do.
        Term::Lam(y, _) if y == x => self.clone(),
        Term::Lam(y, body) => {
            if s.free_vars().contains(y) && body.free_vars().contains(x) {
                // Going under `λy` would capture the free `y` of `s`.
                // Rename the bound `y` to a name used nowhere nearby.
                let mut avoid = s.free_vars();
                avoid.extend(body.all_names());
                avoid.insert(x.to_string());
                let fresh_y = fresh(y, &avoid);
                let renamed = body.subst(y, &Term::Var(fresh_y.clone()));
                lam(&fresh_y, renamed.subst(x, s))
            } else {
```

```
                lam(y, body.subst(x, s))
            }
        }
    }
}
```

The condition also checks that x actually occurs free in the body: if it does not, nothing will be substituted, and there is nothing to rename. Fresh names are made by adding primes until the name is unused:

```
/// A variant of `base` (with primes appended) that is not in `avoid`.
pub fn fresh(base: &str, avoid: &BTreeSet<String>) -> String {
    let mut name = format!("{base}'");
    while avoid.contains(&name) {
        name.push('\'');
    }
    name
}
```

α-equivalence

Renaming a bound variable, consistently and without capturing anything, does not change what a term means. Two terms that differ only in this way are called **α-equivalent**, written $t \underset{\alpha}{\equiv} u$: $\lambda x.\, x \underset{\alpha}{\equiv} \lambda y.\, y$ and $\lambda x.\, \lambda y.\, x\, y \underset{\alpha}{\equiv} \lambda a.\, \lambda b.\, a\, b$, but $\lambda x.\, \lambda y.\, x \not\underset{\alpha}{\equiv} \lambda x.\, \lambda y.\, y$ (the first returns its first argument, the second its second).

From now on we treat α-equivalent terms as *the same term*, as your IDE does when it offers "rename variable" (a refactoring that is safe exactly when the new name captures nothing). Capture-avoiding substitution is only defined up to α anyway, since it depends on which fresh name you pick. Later in the chapter we will see a representation in which α-equivalent terms are *literally* equal.

7.4 β-reduction

With substitution in hand, computation is a single rule. A **redex** (reducible expression) is an application whose function part is an abstraction, $(\lambda x.\, t)\ s$, and it **β-reduces** to $t[x := s]$. Since a redex can appear anywhere inside a term, we also need rules that allow a step inside the parts of an application and inside the body of a λ. In the notation of Chapter 3, the one-step relation $t \to t'$ is defined by four rules:

$$\frac{}{(\lambda x.\, t)\ s \to t[x := s]}\ \textsf{Beta} \qquad \frac{t \to t'}{t\ s \to t'\ s}\ \textsf{AppL}$$

$$\frac{s \to s'}{t\ s \to t\ s'}\ \textsf{AppR} \qquad \frac{t \to t'}{\lambda x.\, t \to \lambda x.\, t'}\ \textsf{Abs}$$

Rule Beta does the work; the other three only say *where* it may be applied. We write $t \overset{*}{\to} t'$ for "zero or more steps". Here is a complete computation:

$$(\lambda x.\, \lambda y.\, x)\ a\ b \to (\lambda y.\, a)\ b \to a$$

The first step contracts the redex $(\lambda x.\, \lambda y.\, x)\ a$ (inside the left part of the outer application, by rule AppL), substituting a for x. The second contracts $(\lambda y.\, a)\ b$, substituting b for y; since y does not occur in a, the argument b simply disappears.

Look at what substitution does to its argument, because it will matter for the rest of the book. In $(\lambda x.\, x\ x)\ s \to s\ s$ the argument s is **duplicated**; in $(\lambda y.\, a)\ b \to a$ it is **discarded**. These are the term-level versions of the structural rules of Chapter 6, contraction and weakening. The λ-calculus uses them freely. Rust does not: duplicating a `String` needs an explicit `.clone()`, and in Part V we will build a λ-calculus that, like Rust, refuses to duplicate.

A term that contains no redex at all is in **normal form**: $\lambda x.\, x$, $x\ (\lambda y.\, y)$ and $\lambda f.\, \lambda x.\, f\ (f\ x)$ are normal forms; $\lambda x.\, (\lambda y.\, y)\ x$ is not, because there is a redex under the λ. A normal form is the "answer" of a computation. Not every term has one:

$$\Omega = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \to (x\ x)[x := \lambda x.\, x\ x] = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) = \Omega$$

Ω reduces to itself in one step, and that is the only step it can take, so every reduction sequence from Ω is infinite. It is the λ-calculus version of `loop {}`.

7.5 Reduction strategies

A term can contain several redexes, and the rules do not say which one to contract first. A **reduction strategy** makes that choice. Three strategies matter:

- **Normal order**: always contract the **leftmost, outermost** redex, the one whose λ comes first when you read the term from the left, and reduce under λ too. Arguments are substituted unevaluated.
- **Call-by-name**: like normal order, but never reduce under a λ and never inside an argument. A function body is not evaluated until the function is called, and an argument is not evaluated until it is used.
- **Call-by-value**: before contracting $(\lambda x.\, t)\ s$, first reduce the argument s to a **value** (an abstraction); never reduce under a λ.

You know call-by-value very well: it is what Rust does. When you call `f(g(x))`, Rust evaluates `g(x)` first, completely, and only then calls `f`. And Rust never evaluates the body of a closure until the closure is called. The crate implements the three strategies as one `step` function:

```
/// Which redex to contract when there are several.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Strategy {
    /// Leftmost-outermost redex first, also under `λ`. Finds the normal form
    /// whenever there is one.
    NormalOrder,
    /// Leftmost-outermost, but never under `λ` and never inside an argument:
    /// arguments are passed unevaluated.
    CallByName,
    /// Arguments are reduced to values before the call, never under `λ`.
    /// This is what Rust does.
    CallByValue,
}
```

```rust
/// One step of β-reduction with the given strategy, or `None` if the strategy
/// finds no redex to contract.
pub fn step(t: &Term, strategy: Strategy) -> Option<Term> {
    match t {
        Term::Var(_) => None,
        Term::Lam(x, body) => match strategy {
            Strategy::NormalOrder => step(body, strategy).map(|b| lam(x, b)),
            Strategy::CallByName | Strategy::CallByValue => None,
        },
        Term::App(f, a) => {
            if let Term::Lam(x, body) = &**f {
                // A redex (λx. body) a. Call-by-value contracts it only once
                // the argument is a value.
                if strategy != Strategy::CallByValue || is_value(a) {
                    return Some(body.subst(x, a));
                }
                return step(a, strategy).map(|a2| app((**f).clone(), a2));
            }
            if let Some(f2) = step(f, strategy) {
                return Some(app(f2, (**a).clone()));
            }
            match strategy {
                Strategy::CallByName => None,
                _ => step(a, strategy).map(|a2| app((**f).clone(), a2)),
            }
        }
    }
}
```

Read the `App` case carefully. If the function part is an abstraction, we have a redex at the top; normal order and call-by-name contract it at once, call-by-value only once the argument is a value (otherwise it steps inside the argument). If the function part is not an abstraction, all three strategies try to step inside it first; then call-by-name stops, and the other two continue with the argument. Around `step`, a function `reduce` runs a strategy for at most a given number of steps, the **fuel**, and reports whether it finished:

```rust
/// The result of running a reducer with a step limit.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Reduction {
    /// The last term reached.
    pub term: Term,
    /// The number of steps taken.
    pub steps: usize,
    /// `true` if no step was possible at the end; `false` if the fuel ran out.
    pub finished: bool,
}

/// Reduces `t` with `strategy` until no step applies, or until `fuel` steps
/// have been taken.
pub fn reduce(t: &Term, strategy: Strategy, fuel: usize) -> Reduction {
    let mut term = t.clone();
    let mut steps = 0;
    while steps < fuel {
        match step(&term, strategy) {
            Some(next) => {
                term = next;
                steps += 1;
            }
            None => {
                return Reduction {
                    term,
                    steps,
                    finished: true,
                };
            }
```

```
        }
    }
    let finished = step(&term, strategy).is_none();
    Reduction {
        term,
        steps,
        finished,
    }
}
```

The fuel is not an implementation detail. A reducer for the untyped λ-calculus *cannot* know in advance whether a term has a normal form (that is the halting problem in disguise), so a reducer that wants to be sure of stopping must give up after some number of steps. We will meet the same idea in Lean, where it is not optional.

Strategies disagree about termination

The example `crates/lambda/examples/ch07_strategies.rs` runs the three strategies on a few terms. The first is a function that throws its argument away, applied to Ω:

```
(λx. λy. y) ((λx. x x) (λx. x x))
  normal order   λy. y   (1 step)
  call-by-name   λy. y   (1 step)
  call-by-value  out of fuel after 1000 steps
```

Normal order and call-by-name contract the outer redex first, the argument is discarded, and the answer is $\lambda y.\, y$. Call-by-value insists on reducing the argument to a value first, and Ω never becomes a value. The same program terminates under one strategy and runs forever under another. You can see the same difference in Rust, where arguments are evaluated even if the function ignores them, and a closure is the standard way to delay evaluation (the call-by-name behavior, written by hand):

```
println!("Rust is call-by-value: both arguments are evaluated first");
let first = |a: &'static str, _b: &'static str| a;
let result = first(arg("a"), arg("b"));
println!("  result: {result}");
println!("A closure delays evaluation, as call-by-name would:");
let first_lazy = |a: &dyn Fn() -> &'static str, _b: &dyn Fn() -> &'static str| a();
let result = first_lazy(&|| arg("a"), &|| arg("b"));
println!("  result: {result}");
```

```
Rust is call-by-value: both arguments are evaluated first
  evaluating the argument "a"
  evaluating the argument "b"
  result: a
A closure delays evaluation, as call-by-name would:
  evaluating the argument "a"
  result: a
```

This is why `Option::unwrap_or_else` takes a closure while `unwrap_or` takes a value: `unwrap_or(expensive())` always runs `expensive()`, call-by-value, while `unwrap_or_else(|| expensive())` runs it only if needed.

The second term duplicates an argument that is itself a redex:

```
(λx. x x) ((λy. y) z)
  normal order   z z   (3 steps)
```

```
  call-by-name   z ((λy. y) z)   (2 steps)
  call-by-value  z z   (2 steps)
```

Normal order substitutes the unevaluated argument, duplicating it, and then has to reduce both copies: three steps. Call-by-value reduces the argument once, before substituting it: two steps. Call-by-name stops early, at $z\,((\lambda y.\, y)\, z)$: the head of the term is the variable z, which is not a function the strategy can call, and call-by-name does not look inside arguments. A term whose head cannot be reduced further like this is in **weak head normal form**; it is where lazy languages stop evaluating, and the remaining redex would be evaluated only if someone needed it. The third term shows the other difference, reduction under λ:

```
λx. (λy. y) x
  normal order   λx. x   (1 step)
  call-by-name   λx. (λy. y) x   (0 steps)
  call-by-value  λx. (λy. y) x   (0 steps)
```

For call-by-value and call-by-name, $\lambda x.\, (\lambda y.\, y)\, x$ is already finished: it is a function, and its body will be evaluated when it is called. That is how every programming language treats function bodies, Rust included. Normal order reduces inside and finds the normal form $\lambda x.\, x$.

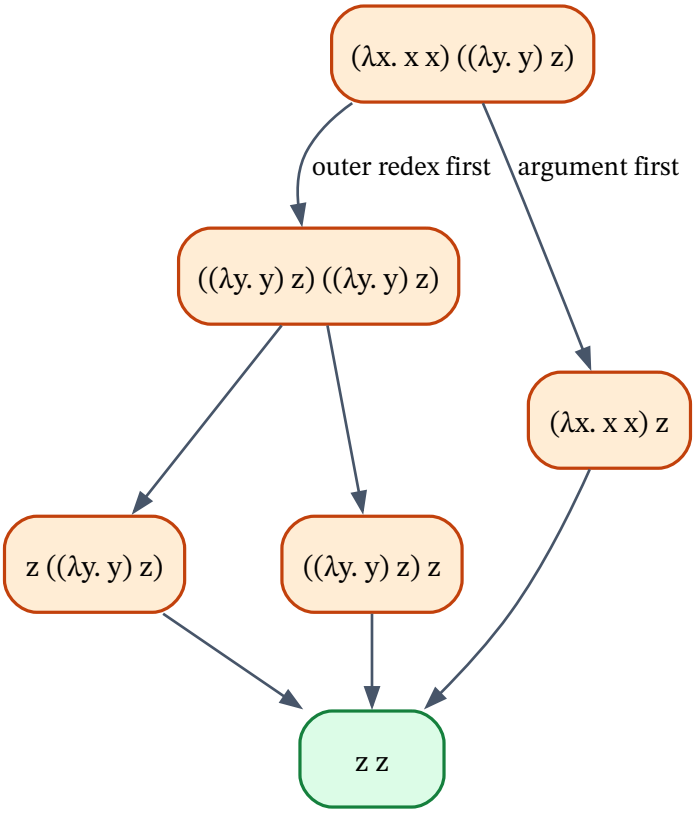
Normal order earns its name with a classical theorem (a consequence of the **standardization theorem**): if a term has a normal form at all, normal order reaches it. Normal order is the safe choice when you want *the* answer of a term. It is not the efficient choice: it can duplicate unevaluated work, as the second example showed. Lazy languages such as Haskell use a refinement, *call-by-need*, that substitutes unevaluated arguments but shares them, so that each one is evaluated at most once.

7.6 Confluence: the answer does not depend on the route

If different strategies can disagree about *whether* a term has an answer, can they disagree about *what* the answer is? No, and this is the most important theorem about the untyped λ-calculus.

> **Theorem (Church–Rosser, 1936).** If $t \xrightarrow{*} u_1$ and $t \xrightarrow{*} u_2$, then there is a term v with $u_1 \xrightarrow{*} v$ and $u_2 \xrightarrow{*} v$.

However two computations diverge, they can always be brought back together. This property is called **confluence**. The diagram shows the complete reduction graph of the second term above: every path ends at $z\,z$.



Every reduction path of (λx. x x) ((λy. y) z) ends in the same normal form z z: the λ-calculus is confluent.

The proof is not hard to understand but long to write down carefully (the standard method, due to Tait and Martin-Löf, goes through "parallel reduction", which contracts any set of redexes at once), and we will not do it. Its most useful consequence is immediate: **a term has at most one normal form**. If $t \xrightarrow{*} n_1$ and $t \xrightarrow{*} n_2$ with both normal, the theorem gives a common v; but a normal form cannot step, so $n_1 = v = n_2$ (up to α). Whatever strategy you use, if it finds an answer, it finds *the* answer. Confluence says nothing about termination, though; Ω is perfectly confluent.

> **NOTE**
>
> Rust has no such theorem, and could not: as soon as a language has side effects, the order of evaluation is observable. `f(print("a"), print("b"))` prints `ab` because Rust specifies left-to-right evaluation of arguments. The pure λ-calculus has no effects, so only termination can tell strategies apart.

7.7 Recursion without names

The λ-calculus has no `fn`, no names for functions, and therefore no obvious way to write a recursive function. A recursive definition such as $fact = \lambda n. \ldots fact\,(n-1) \ldots$ refers to itself, and the calculus has no way to say "itself". Yet recursion is possible, and Ω shows how: Ω manages to reproduce itself by applying a function to a copy of itself. Generalize the trick and you get Curry's **fixed-point combinator**:

$$Y = \lambda f.\,(\lambda x.\,f\,(x\,x))\;(\lambda x.\,f\,(x\,x))$$

Watch what it does to an arbitrary g. Write $W = (\lambda x.\,g\,(x\,x))\;(\lambda x.\,g\,(x\,x))$.

1. $Y\,g \to W$, substituting g for f.
2. $W \to g\,W$, substituting $\lambda x.\,g\,(x\,x)$ for x in $g\,(x\,x)$: the two copies of x become two copies of the same function, and their application is W again.

So $W \to g\,W \to g\,(g\,W) \to \cdots$: W is a **fixed point** of g, a term that g maps to itself (up to reduction). To define factorial, write its body with the recursive call as a parameter,

$$factStep = \lambda fact.\,\lambda n.\,if\;(iszero\;n)\;1\;(times\;n\;(fact\;(pred\;n)))$$

and take its fixed point: $Y\;factStep$ behaves as the factorial, because each time the body needs the recursive call, W unfolds once more. (The helpers if, $iszero$, $times$ and $pred$ are λ-terms too; we build them in the next section.)

Y only works with a lazy strategy. Under call-by-value, step 2 must first evaluate the argument W of g, which requires evaluating W again, forever: the self-application $x\,x$ is evaluated before anyone asks for it. The fix is to hide the self-application under a λ, which call-by-value does not enter. Replacing $x\,x$ by its **η-expansion** $\lambda v.\,x\,x\,v$ (a function that, when called with v, does what $x\,x$ would do) gives the call-by-value fixed-point combinator

$$Z = \lambda f.\,(\lambda x.\,f\,(\lambda v.\,x\,x\,v))\;(\lambda x.\,f\,(\lambda v.\,x\,x\,v))$$

The crate defines both, together with a factorial body for call-by-value whose branches are wrapped in $\lambda d.\,\ldots$ so that only the chosen branch runs (another use of λ to delay evaluation):

```rust
/// `Ω = (λx. x x) (λx. x x)`: reduces to itself, forever.
pub fn omega() -> Term {
    p("(λx. x x) (λx. x x)")
}

/// Curry's `Y = λf. (λx. f (x x)) (λx. f (x x))`. In two steps `Y g`
/// reduces to `g W`, where `W` is the term `Y g` reached after one step: so
/// `W` and `g W` are equal up to reduction, and `W` is a fixed point of `g`.
pub fn y() -> Term {
    p("λf. (λx. f (x x)) (λx. f (x x))")
}

/// `Z = λf. (λx. f (λv. x x v)) (λx. f (λv. x x v))`: the call-by-value fixed
/// point combinator. The extra `λv` delays the self-application.
pub fn z() -> Term {
    p("λf. (λx. f (λv. x x v)) (λx. f (λv. x x v))")
}
```

```
factorial of 3 under call-by-value:
  with Y: out of fuel after 200 steps; the term grew from 141 to 23536 nodes
  with Z: a value after 175 steps; normalizing it gives the numeral 6
```

With Y, call-by-value never gets anywhere: the term keeps unfolding, and in 200 steps it grew from 141 to more than 23,000 nodes. With Z, it computes a value in 175 steps. Because call-by-value stops at a λ, that value is not yet the numeral 6 but a function that still has work inside it; normal order finishes the job.

In Rust, recursion comes from named `fn` items, which may call themselves. A closure cannot refer to itself directly (it has no name, and, as Chapter 8 shows, its type would have to contain itself), which is why recursive helpers in Rust are written as nested `fn`s or receive themselves as an explicit argument.

7.8 Church encodings

The λ-calculus has no data, only functions. Church's insight was that data can be represented by *what you can do with it*. These representations are called **Church encodings**, and they are the clearest demonstration that three constructs are enough.

Booleans

What do you do with a boolean? You choose between two alternatives. So a boolean *is* a chooser:

$$true = \lambda t.\, \lambda f.\, t \qquad false = \lambda t.\, \lambda f.\, f \qquad if = \lambda b.\, \lambda x.\, \lambda y.\, b\ x\ y$$

$if\ true\ x\ y \xrightarrow{*} true\ x\ y \xrightarrow{*} x$. The if does nothing but pass its arguments to the boolean: a Church boolean is its own conditional. The logical operations follow by choosing well: $and = \lambda a.\, \lambda b.\, a\ b\ false$ ("if a then b, else false"), and similarly for or and not. In the crate:

```
/// `true = λt. λf. t`: of two alternatives, choose the first.
pub fn tru() -> Term {
    p("λt. λf. t")
}

/// `false = λt. λf. f`: of two alternatives, choose the second.
pub fn fls() -> Term {
    p("λt. λf. f")
}

/// `if = λb. λthen. λelse. b then else`: a boolean *is* its own `if`.
pub fn ite() -> Term {
    p("λb. λthen. λelse. b then else")
}

/// `and = λa. λb. a b false`
pub fn and() -> Term {
    with("λa. λb. a b false", &[("false", fls())])
}

/// `or = λa. λb. a true b`
pub fn or() -> Term {
    with("λa. λb. a true b", &[("true", tru())])
}

/// `not = λa. a false true`
pub fn not() -> Term {
    with("λa. a false true", &[("false", fls()), ("true", tru())])
}
```

The helper `with` parses a template and substitutes the given terms for its free variables, so that `and` can mention `false` by name:

```
/// Parses `src` and replaces its free variables by the given terms. The terms
/// are closed, so the substitution never has to rename anything: this is how
/// we write `and` in terms of `false` without a `let` in the calculus.
fn with(src: &str, defs: &[(&str, Term)]) -> Term {
    defs.iter().fold(p(src), |t, (x, def)| t.subst(x, def))
}
```

Pairs

What do you do with a pair? You take it apart. A pair is a function waiting for a selector, and the selectors are the booleans:

$$pair = \lambda a.\, \lambda b.\, \lambda s.\, s\ a\ b \qquad fst = \lambda p.\, p\ true \qquad snd = \lambda p.\, p\ false$$

$fst\ (pair\ a\ b) \xrightarrow{*} pair\ a\ b\ true \xrightarrow{*} true\ a\ b \xrightarrow{*} a$.

Numerals

What do you do with a natural number n? You do something n times. The Church numeral $\overline{n}$ takes a function f and a starting point x and applies f to x, n times:

$$\overline{0} = \lambda f.\, \lambda x.\, x \qquad \overline{1} = \lambda f.\, \lambda x.\, f\ x \qquad \overline{n} = \lambda f.\, \lambda x.\, \underbrace{f\ (f\ (\cdots (f}_{n}\ x)))$$

Arithmetic follows from this one idea:

- $succ = \lambda n.\, \lambda f.\, \lambda x.\, f\ (n\ f\ x)$: do it n times, then once more.
- $plus = \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\ f\ (n\ f\ x)$: do it n times, then m more times.
- $times = \lambda m.\, \lambda n.\, \lambda f.\, m\ (n\ f)$: "do f n times" is a function; do *that* m times.
- $power = \lambda m.\, \lambda n.\, n\ m$: apply m itself n times.
- $iszero = \lambda n.\, n\ (\lambda z.\, false)\ true$: start from $true$; any application of $\lambda z.\, false$ turns it into $false$.

The example `crates/lambda/examples/ch07_church.rs` shows $succ\ \overline{1}$ reduced in normal order, one step per line:

```
succ 1, step by step (normal order):
    (λn. λf. λx. f (n f x)) (λf. λx. f x)
  → λf. λx. f ((λf. λx. f x) f x)
  → λf. λx. f ((λx. f x) x)
  → λf. λx. f (f x)
```

Follow it. The first step substitutes $\overline{1}$ for n in the body of $succ$. Normal order then goes under the two λs, finds that the head f is a variable, and moves into the argument $(\lambda f.\, \lambda x.\, f\ x)\ f\ x$, whose leftmost redex is $(\lambda f.\, \lambda x.\, f\ x)\ f$: the numeral 1 receives the "outer" f (second line). Then $(\lambda x.\, f\ x)\ x$ contracts to $f\ x$, and the result is $\lambda f.\, \lambda x.\, f\ (f\ x) = \overline{2}$.

The predecessor is famously harder: there is no way to "undo" one application of f. The solution, a trick due to Kleene, counts up with pairs: start from $(0, 0)$ and apply $(a, b) \mapsto (b, b + 1)$ n times; the pair becomes $(n - 1, n)$, and its first component is the predecessor. Subtraction is then "apply $pred$ n times":

```rust
/// The predecessor, by Kleene's trick: count up pairs `(k, k + 1)` starting
/// from `(0, 0)`; after n steps the first component is n - 1.
/// `pred = λn. fst (n (λq. pair (snd q) (succ (snd q))) (pair 0 0))`
pub fn pred() -> Term {
    with(
        "λn. fst (n (λq. pair (snd q) (succ (snd q))) (pair zero zero))",
        &[
            ("pair", pair()),
            ("fst", fst()),
            ("snd", snd()),
            ("succ", succ()),
            ("zero", numeral(0)),
        ],
```

```
    )
}

/// `sub = λm. λn. n pred m`: truncated subtraction (`2 - 5 = 0`).
pub fn sub() -> Term {
    with("λm. λn. n pred m", &[("pred", pred())])
}
```

Here are the encodings at work. Each program is reduced to normal form in normal order, and the result is read back as a number or a boolean:

```
program             result   steps
plus 2 3                 5       6
times 2 3                6       7
power 2 3                8      16
pred 3                   2      36
sub 5 2                  3     106
iszero 0              true       3
iszero 2             false       4
and true false       false       4
fst (pair 1 2)           1       6
Y fact_step 3            6    1525
```

The last line is the factorial defined with Y: 1525 β-steps to compute $3! = 6$. The λ-calculus is not an efficient way to multiply, but it is a complete one: every function on natural numbers that a Turing machine can compute can be written as a λ-term on Church numerals. Kleene and Turing showed in the 1930s that the functions definable in the λ-calculus are exactly those that Turing machines compute, one of the pieces of evidence for the Church–Turing thesis.

One more line of the output deserves attention:

```
false = λt. λf. f
0     = λf. λx. x
α-equivalent: true
```

The boolean $false$ and the numeral $\overline{0}$ are the *same term* (up to α). Nothing in the untyped calculus can tell a boolean from a number; $iszero$ applied to $true$ happily produces an answer, and so does $plus$ applied to a pair. The calculus is not unsafe in the sense of crashing (every term is a function, and every function can be applied), but it gives no help at all against meaningless programs. That is the job of types.

Church numerals in Rust

Rust closures can play the same game, with one important difference: they are typed. A Church numeral at the fixed type `u64` is "a function that takes an `f: u64 -> u64` and returns `f` composed with itself n times":

```
/// A function on `u64`, shared so that it can be used many times.
type Fun = Rc<dyn Fn(u64) -> u64>;

/// A Church numeral *at the fixed type `u64`*: given `f`, return `f` composed
/// with itself n times.
type Church = Rc<dyn Fn(Fun) -> Fun>;

/// `0 = λf. λx. x`
fn zero() -> Church {
    Rc::new(|_f: Fun| -> Fun { Rc::new(|x| x) })
```

```
}

/// `succ = λn. λf. λx. f (n f x)`
fn succ(n: Church) -> Church {
    Rc::new(move |f: Fun| -> Fun {
        let n_f = n(f.clone());
        Rc::new(move |x| f(n_f(x)))
    })
}

/// `plus = λm. λn. λf. λx. m f (n f x)`
fn plus(m: Church, n: Church) -> Church {
    Rc::new(move |f: Fun| -> Fun {
        let m_f = m(f.clone());
        let n_f = n(f);
        Rc::new(move |x| m_f(n_f(x)))
    })
}

/// `times = λm. λn. λf. m (n f)`
fn times(m: Church, n: Church) -> Church {
    Rc::new(move |f: Fun| m(n(f)))
}

/// Read a numeral back: apply it to "add one" and start from 0.
fn to_u64(n: &Church) -> u64 {
    n(Rc::new(|x| x + 1))(0)
}
```

```
Church numerals at type u64:
  two = 2, three = 3
  plus(two, three)  = 5
  times(two, three) = 6
```

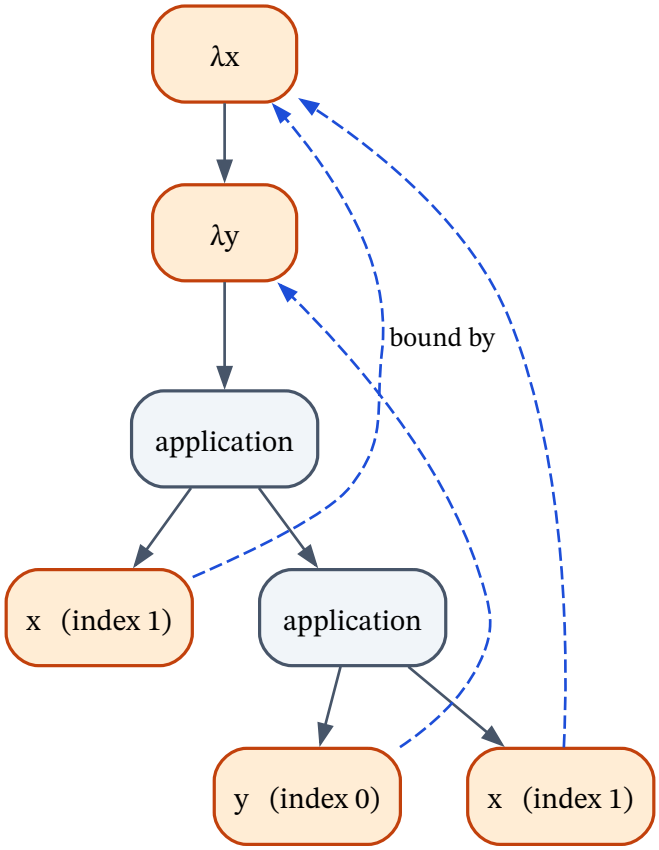
Three things in this code are worth noticing. First, the `Rc<dyn Fn>`: every closure in Rust has its own type (Chapter 8), so to give "all numerals" a single type we put them behind a pointer to a trait object. Second, the `f.clone()` in `succ` and `plus`: the λ-term $\lambda f.\, \lambda x.\, f\,(n\,f\,x)$ uses f twice, and Rust makes that duplication explicit, as it makes every duplication explicit. Third, what is missing: `power`. In the untyped calculus $power = \lambda m.\, \lambda n.\, n\,m$ applies the numeral n to the numeral m, which means using n at the type "functions on numerals" rather than "functions on `u64`". A numeral that works at *every* type at once needs parametric polymorphism of a kind Rust does not have for closures; Chapter 12 explains what is missing and how far traits can simulate it.

7.9 De Bruijn indices: names are the problem

Capture-avoiding substitution works, but it is fiddly: we must compute free variables, invent fresh names, and remember that the result is only defined up to α. Every implementation of a language with binders faces this, and the Dutch mathematician N. G. de Bruijn proposed a radical solution in the early 1970s, while working on Automath, one of the first proof checkers: get rid of the names.

A bound variable is replaced by a number, its **de Bruijn index**: how many λs you must cross, going outward from the occurrence, to reach its binder. The binders themselves carry no name at all:

$$\lambda x.\, x = \lambda.\, 0 \qquad \lambda x.\, \lambda y.\, x = \lambda.\, \lambda.\, 1 \qquad \lambda x.\, \lambda y.\, x\,(y\,x) = \lambda.\, \lambda.\, 1\,(0\,1)$$



The syntax tree of λx. λy. x (y x). Dashed arrows connect each variable to the λ that binds it; the de Bruijn index counts the λs crossed on the way up.

The same variable can have different indices at different places (in $\lambda x.\, x\,(\lambda y.\, x)$ the two occurrences of x are 0 and 1), and different variables can share an index. What the indices capture is the *binding structure* and nothing else. Free variables are numbered beyond the binders, according to an agreed list of free names (the **naming context**): in the context $[z]$, the term $\lambda x.\, x\, z$ is $\lambda.\, 0\, 1$.

The payoff is immediate: **α-equivalent terms have identical de Bruijn representations**, because renaming bound variables does not change any index. The crate's `alpha_eq` simply converts both terms and compares them with `==`:

```
α-equivalence is equality of de Bruijn terms:
  λx. λy. x  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
  λa. λb. a  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
  λx. λy. y  ↦ λ. λ. 0    α-equivalent to λx. λy. x: false
  λy. λx. y  ↦ λ. λ. 1    α-equivalent to λx. λy. x: true
```

The conversion walks the term with a stack of the enclosing binders' names; a variable's index is its distance from the top of the stack, which also handles shadowing correctly (the nearest binder wins):

```
/// Converts a named term to de Bruijn form. `free` lists the names of the free
/// variables: the free variable `free[i]` becomes index `i` (plus the number of
/// enclosing binders).
pub fn to_de_bruijn(t: &Term, free: &[&str]) -> Result<DbTerm, String> {
    // `scope` holds the names of the enclosing binders, innermost last.
    fn go(t: &Term, scope: &mut Vec<String>, free: &[&str]) -> Result<DbTerm, String> {
```

```
        match t {
            Term::Var(x) => {
                // The nearest binder named x wins: that is how shadowing works.
                if let Some(pos) = scope.iter().rev().position(|y| y == x) {
                    Ok(DbTerm::Var(pos))
                } else if let Some(i) = free.iter().position(|y| y == x) {
                    Ok(DbTerm::Var(scope.len() + i))
                } else {
                    Err(format!("free variable `{x}` is not in the naming context"))
                }
            }
            Term::Lam(x, body) => {
                scope.push(x.clone());
                let body = go(body, scope, free);
                scope.pop();
                Ok(db_lam(body?))
            }
            Term::App(f, a) => Ok(db_app(go(f, scope, free)?, go(a, scope, free)?)),
        }
    }
    go(t, &mut Vec::new(), free)
}
```

Shifting and substitution

Names are gone, but substitution still needs care, because indices are *relative*: the same variable has a different index depending on how many binders surround it. Two operations handle this.

Shifting. When a term is moved under an extra binder, its free variables must point one level further out. The function shift_c adds one to every index that is at least the **cutoff** c; indices below the cutoff are bound inside the term and stay put:

- $\mathit{shift}_c(k) = k$ if $k < c$, and $k + 1$ if $k \geq c$;
- $\mathit{shift}_c(\lambda.\, t) = \lambda.\, \mathit{shift}_{c+1}(t)$ (one more binder, so the cutoff grows);
- $\mathit{shift}_c(t\ u) = \mathit{shift}_c(t)\ \mathit{shift}_c(u)$.

Substitution. $t[j := s]$ replaces index j by s. For β-reduction we also want the indices above j to *decrease* by one, because the λ that bound j disappears:

- $k[j := s] = k$ if $k < j$, s if $k = j$, and $k - 1$ if $k > j$;
- $(\lambda.\, t)[j := s] = \lambda.\, t[j + 1 := \mathit{shift}_0(s)]$;
- $(t\ u)[j := s] = t[j := s]\ u[j := s]$.

Under a binder, the variable we are looking for is one level further away (so $j + 1$), and the replacement s must be shifted, so that its free variables skip the binder we just entered. That shift is precisely what capture-avoiding renaming did with names. β-reduction becomes

$$(\lambda.\, t)\ s \to t[0 := s]$$

and there are no names to clash, no fresh names to invent. The Rust code is a line-by-line transcription:

```
/// Adds one to every index that is at least `cutoff`. Indices below the
/// cutoff are bound inside the term and do not move; the others are free.
pub fn shift(&self, cutoff: usize) -> DbTerm {
    match self {
        DbTerm::Var(k) if *k < cutoff => DbTerm::Var(*k),
        DbTerm::Var(k) => DbTerm::Var(k + 1),
        DbTerm::Lam(body) => db_lam(body.shift(cutoff + 1)),
        DbTerm::App(t, u) => db_app(t.shift(cutoff), u.shift(cutoff)),
```

```
    }
}
```

```
/// `self[j := s]`: replaces index `j` by `s` and lowers the free indices
/// above `j` by one (the binder of `j` is about to disappear). Each time we
/// go under a `λ`, `j` grows by one and `s` is shifted, so that the free
/// variables of `s` keep pointing past the new binder.
pub fn subst(&self, j: usize, s: &DbTerm) -> DbTerm {
    match self {
        DbTerm::Var(k) if *k < j => DbTerm::Var(*k),
        DbTerm::Var(k) if *k == j => s.clone(),
        DbTerm::Var(k) => DbTerm::Var(k - 1),
        DbTerm::Lam(body) => db_lam(body.subst(j + 1, &s.shift(0))),
        DbTerm::App(t, u) => db_app(t.subst(j, s), u.subst(j, s)),
    }
}
```

> **NOTE**
>
> Pierce's *Types and Programming Languages* defines a general shift shift_c^d by any amount d, possibly negative, a substitution that does not lower indices, and β as "shift up the argument, substitute, shift the result down". Our version folds the lowering into the substitution, so only upward shifts are ever needed, which suits natural numbers (`usize` in Rust, `Nat` in Lean). The two definitions give the same results.

Now the capture example once more, nameless. In the naming context $[y]$, the term $(\lambda x.\, \lambda y.\, x)\, y$ is $(\lambda.\, \lambda.\, 1)\, 0$. The β-step substitutes 0 for index 0 in the body $\lambda.\, 1$:

1. The body is an abstraction, so we go under it: we now look for index 1, and the replacement 0 is shifted to $\mathit{shift}_0(0) = 1$.
2. Inside, we find index 1, which is the one we are looking for: replace it by the shifted replacement, 1.
3. The result is $\lambda.\, 1$: under one binder, index 1 points past it, to the free variable y. It is $\lambda y'.\, y$, exactly what capture-avoiding substitution produced.

If you forget the shift in step 1, the replacement stays 0, and the result is $\lambda.\, 0$, the identity: the capture bug, back again in a new notation.

```
(λx. λy. x) y  is  (λ. λ. 1) 0  in the naming context [y]
  with the shift:    λ. 1
  without the shift: λ. 0
```

De Bruijn indices turn the capture problem into arithmetic that is easy to get right once and for all, which is why most proof assistants, Lean included, use them internally. Their cost is readability, so real implementations keep names for printing. The crate checks that both representations compute the same thing: the de Bruijn reducer takes the same 1525 steps on the factorial and reaches the same normal form.

```
Y fact_step 3 with de Bruijn indices: 1525 steps, same normal form: true
```

7.10 The λ-calculus in Lean

Now we formalize the calculus in Lean, in `code/lean/Book/Ch07/Untyped.lean` and `code/lean/Book/Ch07/Normalize.lean` (the second imports the first). We use de Bruijn indices from the start; you have just seen why.

Terms, shifting and substitution

A term is an inductive type with three constructors, as in Rust. Lean needs no `Box`: an inductive type may refer to itself directly, and Lean's runtime represents the values as pointers anyway.

```
/-- Terms of the untyped λ-calculus with de Bruijn indices: `var k` refers to
the binder `k` levels up. -/
inductive Term where
  | var : Nat → Term
  | lam : Term → Term
  | app : Term → Term → Term
  deriving DecidableEq
```

The file also defines a printer that writes terms the way the Rust crate does, and a few named terms. (`deriving DecidableEq` asks Lean to generate a decision procedure for equality of terms, which we will use later.)

```
/-- `λx. x` -/
def I : Term := lam (var 0)
/-- `λx. λy. x` -/
def K : Term := lam (lam (var 1))
/-- `λx. x x` -/
def delta : Term := lam (app (var 0) (var 0))
/-- `Ω = (λx. x x) (λx. x x)` -/
def omega : Term := app delta delta

/-- info: λ. λ. 1 -/
#guard_msgs in
#eval K

/-- info: (λ. 0 0) (λ. 0 0) -/
#guard_msgs in
#eval omega
```

`shift` and `subst` are the definitions from the previous section, written by pattern matching. Each recursive call is on a direct subterm, so Lean accepts them as **structural recursion** without further ado (Chapter 2 explains how Lean checks this):

```
/-- `shift c t` adds one to every index of `t` that is at least `c`. The
indices below the cutoff `c` are bound inside `t`; the others are free. -/
def shift (c : Nat) : Term → Term
  | var k => if k < c then var k else var (k + 1)
  | lam t => lam (shift (c + 1) t)
  | app t u => app (shift c t) (shift c u)
```

```
/-- `subst j s t` replaces the index `j` in `t` by `s`, and lowers the free
indices above `j` by one, because the binder of `j` disappears. Under a
binder, `j` grows by one and `s` is shifted. -/
def subst (j : Nat) (s : Term) : Term → Term
  | var k => if k < j then var k else if k = j then s else var (k - 1)
  | lam t => lam (subst (j + 1) (shift 0 s) t)
  | app t u => app (subst j s t) (subst j s u)
```

Two facts about them, proved by `rfl` because both sides compute to the same term: the nameless capture example, and the step that turns Ω into itself.

```
/-- `(λx. λy. x) z`, with `z` free (index 0): the body `λ. 1` becomes `λ. 1`
again, the constant function returning `z`. -/
example : subst 0 (var 0) (lam (var 1)) = lam (var 1) := rfl

/-- Contracting `Ω`'s redex rebuilds `Ω`. -/
example : subst 0 delta (app (var 0) (var 0)) = omega := rfl
```

Reduction as an inductive relation

The four rules Beta, AppL, AppR and Abs become the four constructors of an inductive predicate, exactly as typing rules did in Chapter 3. A value of type `Step t u` is a derivation that t reduces to u in one step:

```
/-- One step of β-reduction, anywhere in a term. -/
inductive Step : Term → Term → Prop where
  /-- `(λ. t) s` steps to `t[0 := s]`. -/
  | beta (t s : Term) : Step (app (lam t) s) (subst 0 s t)
  /-- Reduce inside the function part of an application. -/
  | appL {t t' : Term} (s : Term) : Step t t' → Step (app t s) (app t' s)
  /-- Reduce inside the argument of an application. -/
  | appR (t : Term) {s s' : Term} : Step s s' → Step (app t s) (app t s')
  /-- Reduce inside the body of a λ. -/
  | body {t t' : Term} : Step t t' → Step (lam t) (lam t')
```

For "many steps" we use a definition from Mathlib. `Relation.ReflTransGen r` is the **reflexive-transitive closure** of a relation `r`: `ReflTransGen r a b` holds when you can get from `a` to `b` in zero or more `r`-steps. It is defined as an inductive predicate with two constructors, `refl` (zero steps) and `tail` (some steps, then one more), and Mathlib proves the expected lemmas about it, among them `single` (one step is many steps), `head` (one step followed by many) and `trans`. We import it with `import Mathlib.Logic.Relation`.

```
/-- Many steps: the reflexive-transitive closure of `Step`, from Mathlib. -/
abbrev Steps : Term → Term → Prop := Relation.ReflTransGen Step

/-- A term is in normal form when it cannot take any step. -/
def Normal (t : Term) : Prop := ∀ t', ¬ Step t t'
```

A first proof: $K\,a\,b \xrightarrow{*} a$, with a and b the free variables 0 and 1.

```
/-- `K a b` reduces to `a` in two steps (here `a` and `b` are the free
variables 0 and 1). -/
example : Steps (app (app K (var 0)) (var 1)) (var 0) := by
  -- First step: contract `K (var 0)` inside the left part.
  apply Relation.ReflTransGen.head (Step.appL _ (Step.beta _ _))
  -- Second step: contract the remaining redex `(λ. 1) (var 1)`.
  apply Relation.ReflTransGen.single
  exact Step.beta _ _
```

The goal is a `Steps` statement, and we build the sequence from the front. `Relation.ReflTransGen.head` takes a first step and leaves the rest as a new goal; we give it the derivation `Step.appL _ (Step.beta _ _)`, which says "contract the redex $K\,(\texttt{var}\,0)$ inside the left part". The underscores are filled in by unification: Lean works out the terms from the goal. Notice that we never wrote down the intermediate term: Lean computed `subst 0 (var 0) (lam (var 1))` itself, and it is

`lam (var 1)`, the constant function returning variable 0 (which is index 1 under the binder). What remains is `Steps (app (lam (var 1)) (var 1)) (var 0)`, and `single` turns it into one more `Step`, a Beta step whose substitution drops the argument and lowers index 1 to 0.

Ω, formally

Ω steps to itself:

```
/-- `Ω` steps to itself. -/
theorem omega_step : Step omega omega :=
  Step.beta (app (var 0) (var 0)) delta
```

The proof is a Beta step whose stated result is `subst 0 delta (app (var 0) (var 0))`; Lean accepts it as a proof of `Step omega omega` because that substitution *computes* to `omega` (the two are **definitionally equal**). More interesting is that Ω can *only* step to itself, so it never reaches a normal form. These are statements about *all* possible reductions, the kind of fact a test cannot establish:

```
/-- Moreover, the only step `Ω` can take leads back to `Ω`. -/
theorem omega_step_inv {t : Term} (h : Step omega t) : t = omega := by
  cases h with
  | beta => rfl
  | appL _ h =>
    -- A step inside `delta = λ. 0 0`: impossible, `0 0` has no redex.
    cases h with
    | body h =>
      cases h with
      | appL _ h => cases h
      | appR _ h => cases h
  | appR _ h =>
    cases h with
    | body h =>
      cases h with
      | appL _ h => cases h
      | appR _ h => cases h

/-- So every reduction sequence from `Ω` stays at `Ω`. -/
theorem omega_steps_inv {t : Term} (h : Steps omega t) : t = omega := by
  induction h with
  | refl => rfl
  | tail _ hstep ih =>
    subst ih
    exact omega_step_inv hstep

/-- `Ω` has no normal form. -/
theorem omega_no_normal_form : ¬ ∃ t, Steps omega t ∧ Normal t := by
  rintro ⟨t, hsteps, hnormal⟩
  rw [omega_steps_inv hsteps] at hnormal
  exact hnormal omega omega_step
```

The first theorem is proved by **case analysis on the derivation** `h`: `cases h` asks which rule could have produced `Step omega t`. Rule `beta` fits, with `t` forced to be the substitution, which computes to `omega` (`rfl`). Rule `appL` fits only if `delta` itself steps; a second `cases` finds that only `body` applies, then that `app (var 0) (var 0)` would have to step, and finally that no rule lets a variable step: the case is impossible and disappears. Rule `appR` is the same, for the argument. `omega_steps_inv` extends this to many steps by induction on the `ReflTransGen` derivation, and `omega_no_normal_form` combines the two: a reachable normal term would be `omega` (the `rw`), and `omega` is not normal because it steps.

A normalizer, and a proof that it is right

Next, a function that performs one step of normal-order reduction, the Lean twin of the crate's `step_normal`:

```
/-- One step of normal-order reduction: contract the leftmost-outermost redex,
or return `none` if there is no redex. -/
def step : Term → Option Term
  | var _ => none
  | lam t => (step t).map lam
  | app (lam t) s => some (subst 0 s t)
  | app t s =>
    match step t with
    | some t' => some (app t' s)
    | none => (step s).map (app t)
```

The patterns overlap: `app (lam t) s` is listed before the general `app t s`, and Lean, like Rust's `match`, tries them in order. We now prove that this function agrees with the relation `Step`: every step it takes is a `Step` (soundness), and it finds a step whenever one exists (completeness). Soundness first:

```
/-- Every step that `step` takes is a `Step`. -/
theorem step_sound {t u : Term} (h : step t = some u) : Step t u := by
  fun_induction step t generalizing u with
  | case1 =>
    -- `step (var k) = none`, so `h : none = some u` is impossible.
    contradiction
  | case2 t ih =>
    -- `h : Option.map lam (step t) = some u`: the body stepped to some `t'`.
    obtain ⟨t', ht', rfl⟩ := Option.map_eq_some_iff.mp h
    exact Step.body (ih ht')
  | case3 t s =>
    -- A redex: `h : some (subst 0 s t) = some u`.
    cases h
    exact Step.beta t s
  | case4 t s _ t' ht ih =>
    -- The function part stepped: `ht : step t = some t'`.
    cases h
    exact Step.appL s (ih ht)
  | case5 t s _ _ _ ih =>
    -- The function part is normal and the argument stepped.
    obtain ⟨s', hs', rfl⟩ := Option.map_eq_some_iff.mp h
    exact Step.appR t (ih hs')
```

The tactic `fun_induction step t` is **functional induction**: an induction that follows the branches of the *function* `step` rather than the constructors of the *type* `Term`. When we defined `step`, Lean generated an induction principle with one case per branch (`case1` to `case5`, in order) and an induction hypothesis per recursive call, so each case knows which branch `step` took. In `case1` the hypothesis `none = some u` is absurd (`contradiction`). In `case2`, the core lemma `Option.map_eq_some_iff` turns `Option.map lam (step t) = some u` into `step t = some t'` with `u = lam t'`; `obtain` unpacks it (the `rfl` pattern replaces `u` by `lam t'`), and rule `body` lifts the induction hypothesis under the λ. In `case3`, `cases h` on `some (subst 0 s t) = some u` identifies `u` with the substitution, and `beta` applies. `case4` and `case5` use `appL` and `appR` the same way.

Completeness is proved by induction on the derivation of `Step t u`, showing in each case that `step t` is `some _`. The only subtlety is the overlapping pattern: to unfold `step` on an application, Lean must know that its function part is not a λ, and the lemma `step_app` packages that fact (`step.eq_4` is the equation Lean generated for the fourth branch of `step`):

```
/-- How `step` unfolds on an application whose function part is not a `λ`. -/
theorem step_app {t s : Term} (ht : ∀ b, t ≠ lam b) :
    step (app t s) =
      match step t with
      | some t' => some (app t' s)
      | none => (step s).map (app t) := by
  rw [step.eq_4]
  exact fun b h => ht b h
```

```
/-- If any `Step` is possible, `step` finds one: `step` returns `none` only on
normal forms. -/
theorem step_complete {t u : Term} (h : Step t u) : (step t).isSome := by
  induction h with
  | beta t s =>
    -- A redex at the root: the third equation of `step` applies.
    simp [step]
  | @appL t t' s _ ih =>
    -- `t` can step, so `step t` is `some _` (the induction hypothesis).
    cases t with
    | var k => simp [step] at ih
    | lam b => simp [step]
    | app t₁ t₂ =>
      obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
      rw [step_app (by simp), hu]
      rfl
  | @appR t s s' _ ih =>
    -- `s` can step. Either `t` is a `λ` (then the root is a redex), or
    -- `step` tries `t` first and, if `t` is normal, steps `s`.
    cases t with
    | lam b => simp [step]
    | _ =>
      rw [step_app (by simp)]
      split
      · rfl
      · obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
        simp [hu]
  | body _ ih =>
    obtain ⟨u, hu⟩ := Option.isSome_iff_exists.mp ih
    simp [step, hu]

/-- So `step t = none` means that `t` is in normal form. -/
theorem normal_of_step_none {t : Term} (h : step t = none) : Normal t := by
  intro t' hstep
  have := step_complete hstep
  rw [h] at this
  contradiction
```

In the `appL` and `appR` cases, `cases t` separates a λ in function position (the root is then a redex, and `simp [step]` sees that `step` returns `some`) from the other shapes, where `step_app` applies; its side condition, "this is not a λ", is discharged by `simp`. The corollary `normal_of_step_none` is what we need: if `step` returns `none`, the term is normal.

With these, a **fuel-based normalizer** is easy to write and to prove correct. It takes a number of steps as an extra argument and decreases it at each step, so Lean sees structural recursion on the fuel:

```
/-- Normal-order reduction for at most `fuel` steps: `some u` if a normal form
`u` was reached, `none` if the fuel ran out. -/
def normalize : Nat → Term → Option Term
  | 0, _ => none
  | fuel + 1, t =>
    match step t with
    | none => some t
    | some t' => normalize fuel t'
```

```
/-- What `normalize` returns is a normal form of its input. -/
theorem normalize_sound {fuel : Nat} {t u : Term} (h : normalize fuel t = some u) :
    Steps t u ∧ Normal u := by
  induction fuel generalizing t with
  | zero =>
    -- No fuel: `normalize 0 t = none`, so `h` is impossible.
    simp [normalize] at h
  | succ fuel ih =>
    cases hs : step t with
    | none =>
      -- `t` is already normal and `normalize` returned it.
      simp [normalize, hs] at h
      subst h
      exact ⟨Relation.ReflTransGen.refl, normal_of_step_none hs⟩
    | some t' =>
      -- One step to `t'`, then the induction hypothesis for the rest.
      simp [normalize, hs] at h
      obtain ⟨hsteps, hnormal⟩ := ih h
      exact ⟨Relation.ReflTransGen.head (step_sound hs) hsteps, hnormal⟩
```

The proof is an induction on the fuel. With zero fuel the hypothesis is false (`simp [normalize] at h` reduces it to `False`). With `fuel + 1`, `cases hs : step t` splits on what `step t` returned and remembers it as `hs`. If `none`, `normalize` returned `t`, which reaches itself in zero steps and is normal by `normal_of_step_none`. If `some t'`, the induction hypothesis covers the rest of the run from `t'`, and `head` prepends the first step, justified by `step_sound`.

Computing proofs

Now Church numerals, in de Bruijn form (the comments give the named version):

```
/-- `f (f (... (f x)))` with `n` copies of `f`, where `f` is the variable `k`
and `x` is the variable `0`. -/
def iterApp (k : Nat) : Nat → Term
  | 0 => var 0
  | n + 1 => app (var k) (iterApp k n)

/-- The Church numeral `n = λf. λx. f (... (f x))`: `f` is index 1, `x` is 0. -/
def church (n : Nat) : Term := lam (lam (iterApp 1 n))

/-- `succ = λn. λf. λx. f (n f x)` -/
def succ : Term := lam (lam (lam (app (var 1) (app (app (var 2) (var 1)) (var 0)))))

/-- `plus = λm. λn. λf. λx. m f (n f x)` -/
def plus : Term :=
  lam (lam (lam (lam (app (app (var 3) (var 1)) (app (app (var 2) (var 1)) (var 0))))))

/-- `times = λm. λn. λf. m (n f)` -/
def times : Term := lam (lam (lam (app (var 2) (app (var 1) (var 0)))))
```

```
/-- info: λ. λ. 1 (1 (1 0)) -/
#guard_msgs in
#eval church 3

/-- info: some (λ. λ. 1 (1 (1 (1 (1 0))))) -/
#guard_msgs in
#eval normalize 100 (app (app plus (church 2)) (church 3))

/-- info: none -/
#guard_msgs in
#eval normalize 100 omega
```

`#eval` runs `normalize`: $2 + 3$ normalizes to the numeral 5, and Ω exhausts its fuel. The soundness theorem turns such computations into proofs:

```
/-- `2 + 3` reduces to `5`: `normalize` computes the answer, and
`normalize_sound` turns the computation into a proof. -/
theorem two_plus_three : Steps (app (app plus (church 2)) (church 3)) (church 5) :=
  (normalize_sound (fuel := 100) (by decide)).1

theorem two_times_three : Steps (app (app times (church 2)) (church 3)) (church 6) :=
  (normalize_sound (fuel := 100) (by decide)).1
```

`normalize_sound` needs a proof that `normalize 100 (...) = some (church 5)`. Since `Term` has decidable equality, that equation is a decidable proposition, and `decide` asks Lean's kernel (the small trusted core that checks every proof) to evaluate it. This pattern, "write a procedure, prove it sound, run it", is called **proof by reflection**, and many large automated proofs work this way.

For specific numbers, computation is enough. For *all* numbers, we need a real proof. Here is one: $succ\ \overline{n} \xrightarrow{*} \overline{n+1}$ for every n.

```
/-- Church numerals are closed, so shifting does not change them. -/
theorem shift_iterApp (c n : Nat) : shift (c + 2) (iterApp 1 n) = iterApp 1 n := by
  induction n with
  | zero => simp [iterApp, shift]
  | succ n ih => simp [iterApp, shift, ih]

theorem shift_church (c n : Nat) : shift c (church n) = church n := by
  simp [church, shift, shift_iterApp]

/-- Substituting for `f` (index 1) renames it to index 2... -/
theorem subst_iterApp_f (n : Nat) :
    subst 1 (var 2) (iterApp 1 n) = iterApp 2 n := by
  induction n with
  | zero => simp [iterApp, subst]
  | succ n ih => simp [iterApp, subst, ih]

/-- ...and substituting `x` (index 0) for itself lowers index 2 back to 1. -/
theorem subst_iterApp_x (n : Nat) :
    subst 0 (var 0) (iterApp 2 n) = iterApp 1 n := by
  induction n with
  | zero => simp [iterApp, subst]
  | succ n ih => simp [iterApp, subst, ih]
```

```
/-- Reduction sequences can be carried out under a `λ`... -/
theorem steps_body {t t' : Term} (h : Steps t t') : Steps (lam t) (lam t') := by
  induction h with
  | refl => exact Relation.ReflTransGen.refl
  | tail _ hstep ih => exact ih.tail (Step.body hstep)

/-- ...and inside the argument of an application. -/
theorem steps_appR (t : Term) {s s' : Term} (h : Steps s s') :
    Steps (app t s) (app t s') := by
  induction h with
  | refl => exact Relation.ReflTransGen.refl
  | tail _ hstep ih => exact ih.tail (Step.appR t hstep)
```

```
/-- For every `n`, `succ (church n)` reduces to `church (n + 1)`. -/
theorem succ_correct (n : Nat) : Steps (app succ (church n)) (church (n + 1)) := by
  -- Step 1, the outer redex: `succ (church n)` becomes `λ. λ. 1 (church n 1 0)`.
  have h1 : Step (app succ (church n))
      (lam (lam (app (var 1) (app (app (church n) (var 1)) (var 0))))) := by
    have := Step.beta (lam (lam (app (var 1) (app (app (var 2) (var 1)) (var 0)))))
      (church n)
    simp [subst, shift_church] at this
    exact this
  -- Step 2, inside: `church n 1` becomes `λ. iterApp 2 n`.
  have h2 : Step (app (app (church n) (var 1)) (var 0))
```

```
      (app (lam (iterApp 2 n)) (var 0)) := by
    have := Step.appL (var 0) (Step.beta (lam (iterApp 1 n)) (var 1))
    simp [subst, shift, subst_iterApp_f] at this
    exact this
  -- Step 3: `(λ. iterApp 2 n) 0` becomes `iterApp 1 n`.
  have h3 : Step (app (lam (iterApp 2 n)) (var 0)) (iterApp 1 n) := by
    have := Step.beta (iterApp 2 n) (var 0)
    rwa [subst_iterApp_x] at this
  -- Steps 2 and 3 happen under two binders, in the argument of `1`.
  have h23 : Steps (app (app (church n) (var 1)) (var 0)) (iterApp 1 n) :=
    Relation.ReflTransGen.head h2 (Relation.ReflTransGen.single h3)
  -- `church (n + 1)` is, by definition, `λ. λ. 1 (iterApp 1 n)`.
  exact Relation.ReflTransGen.head h1 (steps_body (steps_body (steps_appR _ h23)))
```

The proof follows the reduction you saw in the Rust trace, three steps, but for a symbolic n:

- **Step 1** contracts the outer redex. We take the Beta step as `this` and let `simp` compute its substitution; since `church n` is closed, shifting it changes nothing (`shift_church`, by induction on `n`). `exact this` matches the result against `h1`.
- **Step 2** contracts `church n 1`. The substitution turns the numeral's own `f` (index 1) into index 2, the outer f seen from under the numeral's remaining binder (`subst_iterApp_f`, again by induction).
- **Step 3** substitutes x for x, lowering index 2 back to 1 (`subst_iterApp_x`).

Steps 2 and 3 happen under two λs and in the argument of f; `steps_body` and `steps_appR` carry a reduction sequence into such a context, each by a short induction. The final term is `lam (lam (app (var 1) (iterApp 1 n)))`, which is `church (n + 1)` by definition, so Lean accepts it as it is.

Why Lean insists on fuel

Why not write a normalizer without fuel, one that simply keeps stepping until it reaches a normal form? Here is the attempt, and Lean's answer:

```
/--
error: fail to show termination for
  Ch07.normalizeAll
with errors
failed to infer structural recursion:
Cannot use parameter t:
  failed to eliminate recursive application
    normalizeAll t'


failed to prove termination, possible solutions:
  - Use `have`-expressions to prove the remaining goals
  - Use `termination_by` to specify a different well-founded relation
  - Use `decreasing_by` to specify your own tactic for discharging this kind of goal
t t' : Term
⊢ sizeOf t' < sizeOf t
-/
#guard_msgs in
def normalizeAll (t : Term) : Term :=
  match step t with
  | none => t
  | some t' => normalizeAll t'
```

Lean requires every function to terminate, and it must see why. It first tried **structural recursion**, but `t'` is the result of a reduction step, not a subterm of `t`. It then tried **well-founded recursion**

on the size of the term (`sizeOf`) and left the goal it could not prove: `sizeOf t' < sizeOf t`. That goal is false, since a β-step can make a term bigger (you saw Y grow from 141 to 23,536 nodes). In fact *no* measure can work, because the function does not terminate on Ω. Lean is not being fussy; it is right. And it must be strict: if it accepted `def loop (n : Nat) : Nat := loop n + 1`, the equation `loop 0 = loop 0 + 1` would prove `0 = 1` (Chapter 10 makes this precise). What Lean offers instead is `partial`:

```
/-- The same function, marked `partial`: Lean accepts it without a termination
proof, compiles it, and lets us run it. -/
partial def normalizeAll' (t : Term) : Term :=
  match step t with
  | none => t
  | some t' => normalizeAll' t'

/-- info: λ. λ. 1 (1 (1 (1 (1 0)))) -/
#guard_msgs in
#eval normalizeAll' (app (app times (church 2)) (church 3))
```

A `partial def` is compiled and runs, so `#eval` works (and would loop forever on Ω). The price is that the logic knows nothing about it. Lean only requires its type to be inhabited (here it is: `fun t => t` has type `Term → Term`), and then treats the function as an *opaque* constant: you cannot unfold it in a proof, even on an input where it obviously returns at once.

```
/--
error: Tactic `rfl` failed: The left-hand side
  normalizeAll' (var 0)
is not definitionally equal to the right-hand side
  var 0

⊢ normalizeAll' (var 0) = var 0
-/
#guard_msgs in
example : normalizeAll' (var 0) = var 0 := by
  rfl
```

So the choice in Lean is: fuel, which makes the function total and lets you prove theorems about it (`normalize_sound`), or `partial`, which makes it runnable and nothing more. Rust makes the second choice for every function: `loop` and unrestricted recursion are always available, and termination is never checked. Chapter 8 shows that adding types to the λ-calculus brings termination back, for free.

7.11 Rust closures are typed λ-terms

Rust's closures are λ-abstractions, and calling a closure is β-reduction in call-by-value order. Here are the combinators from the beginning of the chapter, written in Rust:

```
/// `λx. x`
fn identity<A>(x: A) -> A {
    x
}

/// `λx. λy. x`: given x, return a closure that ignores its argument.
fn konst<A: Clone, B>(x: A) -> impl Fn(B) -> A {
    move |_y| x.clone()
}

/// `λf. λg. λx. g (f x)`: first f, then g.
```

```
fn compose<A, B, C>(f: impl Fn(A) -> B, g: impl Fn(B) -> C) -> impl Fn(A) -> C {
    move |x| g(f(x))
}
```

```
identity(7)              = 7
konst("kept")(42)        = kept
compose(inc, double)(5)  = 12
```

The difference is in the signatures: every parameter and every result has a type. Most of the time the types are generic, so the combinators work for any types `A`, `B` and `C`, but each use of a combinator picks specific ones. That restriction is what rules out the strange terms of this chapter. Try to write the self-application $\delta = \lambda x.\, x\, x$ as a closure:

```
pub fn delta() {
    let self_apply = |x| x(x);
}
```

```
error[E0282]: type annotations needed
 --> compile_fail/ch07_self_application.rs:7:23
  |
7 |     let self_apply = |x| x(x);
  |                       ^  - type must be known at this point
  |
help: consider giving this closure parameter an explicit type
  |
7 |     let self_apply = |x: /* Type */| x(x);
  |                        ++++++++++++
```

The compiler must know the type of `x` to call it, and nothing tells it. But annotations cannot help either. Suppose `x` had some type T. Since `x` is called, T must be a function type, say $A \to B$. Since `x` is its own argument, A must be T. So $T = T \to B$: a type that contains itself as a proper part, which no finite type does. Chapter 8 turns this argument into a theorem (proved in Lean), and Chapter 13 shows the same failure in type inference, where it is detected by a check called the *occurs check*. Without δ there is no Ω, no Y and no Z, and, as Chapter 8 shows, no non-termination at all.

7.12 Putting it together: the factorial of 3

Let us retrace how the crate computes 3!, the last line of the arithmetic table, because it uses nearly every idea of this chapter.

1. **Encoding.** The numerals, $iszero$, $times$ and $pred$ are Church encodings; $pred$ uses Church pairs, and the conditional is a Church boolean. The `with` helper assembles them by substituting closed terms for names, so no renaming is ever needed there.
2. **Recursion.** `fact_step` abstracts the body of factorial over the recursive call, and Y ties the knot: $Y\ factStep$ unfolds once for every recursive call.
3. **Strategy.** Normal order finds the answer in 1525 steps. Call-by-value loops with Y; with Z and delayed branches it computes a value, which normal order then tidies up into the numeral.
4. **Substitution and confluence.** Every step is a capture-avoiding substitution, and by the Church–Rosser theorem any strategy that reaches a normal form reaches this one, $\overline{6}$.

5. **Two representations.** The de Bruijn reducer, which never renames anything, makes the same choices and reaches the same normal form.
6. **Reading back.** `decode_numeral` recognizes $\lambda f.\, \lambda x.\, f\,(\cdots(f\,x))$ and counts the fs. It must be told to expect a number, because $false$ and $\overline{0}$ are the same term.

In Lean, the same computation would be a theorem: `normalize` with enough fuel, plus `normalize_sound`, proves that the factorial term reduces to `church 6`. The Rust program computes; the Lean development guarantees that the computation means what we think.

7.13 Common confusions

"**Substitution is search and replace.**" It is not, as the capture example shows: replacing text inside a binder can change which binder a variable refers to. Substitution is defined on syntax trees, respects scopes, and renames bound variables when necessary.

"**α-equivalent terms are different programs.**" They are the same program written with different parameter names. Every reasonable operation on terms gives α-equivalent results on α-equivalent inputs, which is why we identify them, and why de Bruijn indices, which make them equal, are so convenient.

"**A de Bruijn index is a position in the term.**" It counts *binders*, not symbols: it is the number of λs between the occurrence and its binder. The same variable can have different indices in different places.

"**Normal order is the efficient strategy.**" It is the *complete* one: it finds a normal form whenever there is one. It can be slower than call-by-value, because it may duplicate unevaluated arguments. Real languages choose call-by-value (Rust, OCaml) or call-by-need (Haskell).

"**Church–Rosser means every computation terminates.**" It means computations never disagree about the result. Ω is confluent and has no result.

"**The untyped λ-calculus can go wrong.**" Not in the sense of crashing: every term is a function, so every application can proceed. What it lacks is any protection against *meaningless* programs, such as adding a boolean to a pair; it computes an answer, just not a sensible one. Once we add genuine constants like numbers in Chapter 8, "applying the number 3" really does get stuck, and types become indispensable.

7.14 Summary

- The untyped λ-calculus has three constructs: variables, abstractions $\lambda x.\, t$ and applications $t\ u$. Functions of several arguments are curried.
- A λ binds its parameter in its body; the other variables are free. The free variables of a Rust closure's body are its captures.
- Substitution $t[x := s]$ must avoid capture: when a binder would capture a free variable of s, rename the binder first. Naive substitution changes the meaning of programs; `macro_rules!` hygiene is the same idea.

- β-reduction, $(\lambda x.\, t)\, s \to t[x := s]$, is the only computation rule. It duplicates and discards arguments freely.
- Strategies choose the redex: normal order (complete), call-by-name (lazy), call-by-value (Rust's). They can disagree on termination, never on the result: the Church–Rosser theorem makes normal forms unique.
- Ω diverges; Y and Z produce recursion from self-application; Church encodings represent booleans, pairs and numbers as functions, enough for all computable functions.
- De Bruijn indices replace names by binder distances: α-equivalence becomes equality, and capture avoidance becomes shifting.
- In Lean, terms are an inductive type, reduction is an inductive relation, a fuel-based normalizer is proved sound, and computation can produce proofs. Lean rejects a normalizer without fuel because it cannot terminate on every input; `partial` makes it runnable but opaque to proofs.
- Rust closures are typed λ-terms, and types rule out self-application: `|x| x(x)` has no type.

7.15 Exercises

1. Compute the free variables of $\lambda x.\, y\, (\lambda y.\, x\, y\, z)$, then compute $(\lambda x.\, y\, (\lambda y.\, x\, y\, z))[y := x\, z]$ by hand, step by step, renaming where necessary. Check your answer with the crate (`parse` and `Term::subst`, comparing with `alpha_eq`). *Hint:* the outer λx would capture the x of $x\, z$, so it must be renamed; the inner λy binds y itself, so the substitution stops there.
2. Reduce $(\lambda x.\, \lambda y.\, y\, x)\, ((\lambda z.\, z)\, a)\, (\lambda w.\, w)$ in normal order and in call-by-value, writing every step. Count the steps. Which strategy is faster here, and why? Confirm with `trace`. *Hint:* look at how many times x is used in the body.
3. Define Church encodings of `xor` and of $\mathit{double} = \lambda n.\, \mathit{plus}\ n\ n$ with the crate's `with` helper (copy it into the example, since it is private to the crate), and add rows for them to the table in `ch07_church.rs`. *Hint:* $\mathit{xor} = \lambda a.\, \lambda b.\, a\ (\mathit{not}\ b)\ b$.
4. Convert $\lambda f.\, (\lambda x.\, f\, (x\, x))\, (\lambda x.\, f\, (x\, x))$ (Y) to de Bruijn indices, then perform one step of β-reduction on $Y\, g$ (with g free, index 0 in the naming context $[g]$) using the definitions of shift and substitution, by hand. *Hint:* Y is $\lambda.\, (\lambda.\, 1\, (0\, 0))\, (\lambda.\, 1\, (0\, 0))$. The substitution enters one binder (the λx) on its way to each occurrence of f, so it looks for index 1 there and inserts g, shifted to index 1. The result looks exactly like the body of Y, but its index 1 now means g.
5. In Lean, prove `Steps (app I (app I I)) I` (with `I` from `Untyped.lean`) in two ways: by building the sequence with `Relation.ReflTransGen.head` and `Step` constructors, and by computation with `normalize_sound` and `decide`. Then state and prove that `I` is normal. *Hint:* for normality, `intro t h` and then `cases h` twice: a variable cannot step.
6. Prove in Lean that `normalize` is monotone in its fuel: if `normalize n t = some u` then `normalize (n + 1) t = some u`. *Hint:* induction on `n` generalizing `t`, and in the successor case `cases hs : step t` followed by `simp [normalize, hs]` on both the hypothesis and the goal.

CHAPTER 8

The Simply Typed Lambda Calculus and Closures

The untyped λ-calculus of Chapter 7 can compute anything, and it cannot tell anything apart. The boolean $false$ and the number $\overline{0}$ are the same term; $plus$ applied to a pair produces an answer; $\lambda x.\, x\, x$ applies a function to itself, and Ω runs forever. In 1940 Alonzo Church added **types** to his calculus, and most of this strangeness disappeared at once. The result, the **simply typed λ-calculus** (STLC), is the smallest interesting type theory, the core of every typed functional language, and the system that Chapter 9 proves safe and Chapter 10 reads as logic.

Rust's functions are typed λ-terms too, but a Rust programmer meets function types in several costumes that the clean theory does not have: `fn` items, function pointers, closures with anonymous types, `impl Fn`, `dyn Fn`, and three traits, `Fn`, `FnMut` and `FnOnce`. The second half of this chapter explains them one by one, and shows that the three traits answer a question the λ-calculus never asks: how many times may a function be called? That question leads straight to Part V.

In this chapter you will learn:

- the types and typing rules of the STLC with unit, booleans, naturals, pairs and sums, as introduction and elimination rules;
- the computation rules β and η, why every well-typed term terminates, and why $\lambda x.\, x\, x$ has no type;
- bidirectional type checking, where λ checks but does not synthesize, and how the book's `stlc` crate implements it with an evaluator based on closures;
- what Rust's `fn` items, function pointers, closures, `impl Fn` and `dyn Fn` are, and what each costs;
- capture modes, `move`, and the `Fn` / `FnMut` / `FnOnce` hierarchy read as "how many times may this resource be used";
- why currying, free in the λ-calculus, has a price in Rust;
- how Lean represents intrinsically typed terms whose evaluator is total and safe by construction, and how that differs from the extrinsic style of Chapter 9.

8.1 Types for the λ-calculus

The types are built from three base types and three type formers:

$$\tau, \sigma ::= \texttt{Unit} \mid \texttt{Bool} \mid \texttt{Nat} \mid \sigma \to \tau \mid \sigma \times \tau \mid \sigma + \tau$$

$\sigma \to \tau$ is the type of functions from σ to τ, $\sigma \times \tau$ the type of pairs, and $\sigma + \tau$ the type of **tagged unions**: a value of $\sigma + \tau$ is either a σ tagged "left" or a τ tagged "right", like a Rust `enum Either<S, T> { Left(S), Right(T) }`. The arrow associates to the right, $\texttt{Nat} \to \texttt{Nat} \to \texttt{Nat}$ meaning $\texttt{Nat} \to (\texttt{Nat} \to \texttt{Nat})$, the type of curried two-argument functions.

The terms are those of Chapter 7, with an annotation on the parameter of each λ, plus constants and operations for the new types:

$$t ::= x \mid \lambda x : \tau . t \mid t\ t \mid () \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ t\ \texttt{then}\ t\ \texttt{else}\ t$$

$$\mid 0 \mid \texttt{succ}\ t \mid \texttt{iter}\ t\ t\ t \mid (t, t) \mid \texttt{fst}\ t \mid \texttt{snd}\ t$$

$$\mid \texttt{inl}\ t \mid \texttt{inr}\ t \mid \texttt{case}\ t\ \texttt{of}\ \texttt{inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow t$$

Most of these are familiar. `iter n z s` applies the function `s` to `z`, `n` times: it is the way to compute with a natural number, the typed version of a Church numeral (Chapter 7), and the same thing as `(0..n).fold(z, |acc, _| s(acc))` in Rust. `case` is a two-armed `match` on a sum, binding `x` in the first arm and `y` in the second.

Annotating every parameter, as in $\lambda x : \texttt{Nat}.\, \texttt{succ}\ x$, is called the **Church style** of typing: types are part of the terms. In the **Curry style** terms stay unannotated and a type is something you assign to them afterwards, possibly in several ways ($\lambda x. x$ has every type $\tau \to \tau$). We will use both: Church style for the rules, Curry style in the Lean theorem about self-application, and a mixture, bidirectional checking, in the Rust checker.

8.2 The typing rules

The judgment is $\Gamma \vdash t : \tau$, as in Chapter 3, with a context $\Gamma = x_1 : \tau_1, \ldots, x_n : \tau_n$ that later entries may shadow. The heart of the system is three rules, for variables, abstraction and application:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsf{T-Var} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda x : \sigma . t : \sigma \to \tau}\ \textsf{T-Lam} \qquad \frac{\Gamma \vdash t : \sigma \to \tau \qquad \Gamma \vdash u : \sigma}{\Gamma \vdash t\ u : \tau}\ \textsf{T-App}$$

Rule T-Lam says: to type a function, assume its parameter has the annotated type and type the body. Rule T-App says: a function of type $\sigma \to \tau$ applied to a σ gives a τ. You have relied on both every time Rust checked a closure: `|x: u32| x + 1` has type "`u32` to `u32`" because, with `x: u32`, the body is a `u32`.

The other types follow one pattern. Each type former has **introduction** rules, which build values of the type, and **elimination** rules, which use them. For booleans, naturals, pairs and sums:

$$\frac{}{\Gamma \vdash \texttt{true} : \texttt{Bool}}\ \textsf{T-True} \qquad \frac{\Gamma \vdash c : \texttt{Bool} \qquad \Gamma \vdash t : \tau \qquad \Gamma \vdash u : \tau}{\Gamma \vdash \texttt{if}\ c\ \texttt{then}\ t\ \texttt{else}\ u : \tau}\ \textsf{T-If}$$

$$\frac{}{\Gamma \vdash 0 : \texttt{Nat}}\ \textsf{T-Zero} \qquad \frac{\Gamma \vdash n : \texttt{Nat}}{\Gamma \vdash \texttt{succ}\ n : \texttt{Nat}}\ \textsf{T-Succ} \qquad \frac{\Gamma \vdash n : \texttt{Nat} \qquad \Gamma \vdash z : \tau \qquad \Gamma \vdash s : \tau \to \tau}{\Gamma \vdash \texttt{iter}\ n\ z\ s : \tau}\ \textsf{T-Iter}$$

$$\frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau}{\Gamma \vdash (t, u) : \sigma \times \tau}\ \textsf{T-Pair} \qquad \frac{\Gamma \vdash p : \sigma \times \tau}{\Gamma \vdash \texttt{fst}\ p : \sigma}\ \textsf{T-Fst} \qquad \frac{\Gamma \vdash p : \sigma \times \tau}{\Gamma \vdash \texttt{snd}\ p : \tau}\ \textsf{T-Snd}$$

$$\frac{\Gamma \vdash t : \sigma}{\Gamma \vdash \texttt{inl}\ t : \sigma + \tau}\ \textsf{T-Inl} \qquad \frac{\Gamma \vdash t : \tau}{\Gamma \vdash \texttt{inr}\ t : \sigma + \tau}\ \textsf{T-Inr}$$

$$\frac{\Gamma \vdash e : \sigma + \tau \qquad \Gamma, x : \sigma \vdash t : \rho \qquad \Gamma, y : \tau \vdash u : \rho}{\Gamma \vdash \texttt{case}\ e\ \texttt{of inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow u : \rho}\ \textsf{T-Case}$$

plus T-False (like T-True) and T-Unit, $\Gamma \vdash () : \texttt{Unit}$. Rule T-If is Chapter 1's rule, unchanged. Rule T-Case is Rust's `match`: both arms must produce the same type ρ, and each arm may use the payload of its variant. The table sums up the pattern, with the Rust feature that plays each role:

Type	Introduction	Elimination	In Rust
$\sigma \to \tau$	$\lambda x : \sigma.\, t$	application	closures, `fn`
$\sigma \times \tau$	(t, u)	`fst`, `snd`	tuples, `.0`, `.1`
$\sigma + \tau$	`inl`, `inr`	`case`	`enum`, `match`
`Bool`	`true`, `false`	`if`	`bool`, `if`
`Nat`	`0`, `succ`	`iter`	`u64`, loops, `fold`
`Unit`	`()`	(none needed)	`()`

If this table reminds you of Chapter 4, it should. Erase the terms from the rules and keep only the types: T-Lam becomes the introduction rule for implication, T-App is *modus ponens*, T-Pair and T-Fst are the rules for $\wedge$, T-Inl and T-Case the rules for $\vee$. That is not a coincidence, and Chapter 10 is devoted to it.

A derivation

Here is the derivation for `twice`, the function that applies its argument twice:

$$\vdash \lambda f : \texttt{Nat} \to \texttt{Nat}.\, \lambda x : \texttt{Nat}.\, f\,(f\,x) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$$

Write $\Gamma = f : \texttt{Nat} \to \texttt{Nat}, x : \texttt{Nat}$. The inner application $f\,x$ is typed by T-App from two variables:

$$\mathcal{D} = \frac{\dfrac{}{\Gamma \vdash f : \texttt{Nat} \to \texttt{Nat}}\ \textsf{T-Var} \qquad \dfrac{}{\Gamma \vdash x : \texttt{Nat}}\ \textsf{T-Var}}{\Gamma \vdash f\,x : \texttt{Nat}}\ \textsf{T-App}$$

The outer application uses $\mathcal{D}$ as its second premise, and two uses of T-Lam move x and then f out of the context and into the type:

$$\frac{\dfrac{\dfrac{\Gamma \vdash f : \texttt{Nat} \to \texttt{Nat} \qquad \mathcal{D}}{\Gamma \vdash f\,(f\,x) : \texttt{Nat}}\ \textsf{T-App}}{f : \texttt{Nat} \to \texttt{Nat} \vdash \lambda x : \texttt{Nat}.\, f\,(f\,x) : \texttt{Nat} \to \texttt{Nat}}\ \textsf{T-Lam}}{\vdash \lambda f : \texttt{Nat} \to \texttt{Nat}.\, \lambda x : \texttt{Nat}.\, f\,(f\,x) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}}\ \textsf{T-Lam}$$

Notice that f is used twice. The context is shared by both premises of T-App, so a variable may be used as often as we like; that is contraction (Chapter 6) at work, and it is exactly what Part V will restrict.

8.3 Computation: β and η

The computation rules are the β-rule of Chapter 7 plus one rule for each elimination applied to an introduction:

- $(\lambda x : \tau.\, t)\ v \to t[x := v]$;
- $\texttt{fst}\ (v_1, v_2) \to v_1$ and $\texttt{snd}\ (v_1, v_2) \to v_2$;
- $\texttt{case}\ (\texttt{inl}\ v)\ \texttt{of inl}\ x \Rightarrow t \mid \texttt{inr}\ y \Rightarrow u \to t[x := v]$, and symmetrically for `inr`;
- $\texttt{if true then}\ t\ \texttt{else}\ u \to t$, and symmetrically for `false`;
- $\texttt{iter}\ 0\ z\ s \to z$ and $\texttt{iter}\ (\texttt{succ}\ n)\ z\ s \to s\ (\texttt{iter}\ n\ z\ s)$.

Each rule removes a *detour*: something was built only to be taken apart at once. Chapter 4 called the logical version of this **local soundness**, and you can now see why the slogan "proof simplification is computation" is literally true.

The **η-rules** go the other way. They say that every value of a type is determined by what you can do with it:

$$\lambda x : \sigma.\, f\ x \equiv f \quad (x \notin \mathrm{FV}(f)) \qquad\qquad (\texttt{fst}\ p, \texttt{snd}\ p) \equiv p$$

A function that only passes its argument on to f *is* f; a pair rebuilt from its two projections *is* the pair. Chapter 4 called this **local completeness**. η is usually stated as an equation rather than a reduction step, and it is the principle behind a refactoring you have probably done: Clippy's `redundant_closure` lint suggests replacing `.map(|x| f(x))` with `.map(f)`. Rust shows why η needs care in a real language, though. In `.map(|x| make_f()(x))`, `make_f()` runs once per element, while in `.map(make_f())` it runs once, before the call: under call-by-value with side effects, η-reducing can change when things happen.

8.4 What types buy: termination

The most striking consequence of the typing rules is a theorem.

> **Theorem (strong normalization).** If $\vdash t : \tau$ in the STLC, then every reduction sequence starting from t is finite.

Every well-typed program terminates, whatever the evaluation order. The theorem was proved by William Tait in the 1960s with a method now called **logical relations**, which Chapter 32 explains; for System T, the STLC with `iter`, the proof is a little longer but the result is the same. We will not prove it here, but we can see the most important thing it rules out: self-application.

Why $\lambda x.\, x\ x$ has no type

Suppose $\Gamma \vdash \lambda x : \sigma.\, x\ x : \tau$ for some annotation σ. Read the rules backward:

1. Only T-Lam concludes a judgment about a λ, so $\tau = \sigma \to \rho$ for some ρ, and $\Gamma, x : \sigma \vdash x\ x : \rho$.
2. Only T-App concludes a judgment about an application, so the first x has a type $\sigma' \to \rho$ and the second has type σ'.
3. Only T-Var types a variable, and it gives x the type σ in both places. So $\sigma = \sigma' \to \rho$ and $\sigma = \sigma'$.

Together: $\sigma = \sigma \to \rho$. But the type $\sigma \to \rho$ contains σ as a proper part, so it is strictly bigger than σ, and no type is equal to a strictly bigger type. There is no such σ. Without $\delta = \lambda x.\, x\, x$ there is no Ω, no Y and no Z.

This argument is short enough to check in Lean, and the file `code/lean/Book/Ch08/Extrinsic.lean` does. It defines raw, unannotated λ-terms with de Bruijn indices and a Curry-style typing relation `HasType`, one constructor per rule (we look at it more closely at the end of the chapter). First, the size argument:

```
/-- No type is a function type whose domain is the type itself: `σ ⇒ τ` is
always strictly bigger than `σ`. -/
theorem arrow_ne_self (σ τ : Ty) : (σ ⇒ τ) ≠ σ := by
  intro h
  -- Compare sizes: `sizeOf (σ ⇒ τ) = 1 + sizeOf σ + sizeOf τ`.
  have := congrArg sizeOf h
  simp at this
  omega
```

Every inductive type in Lean comes with a function `sizeOf` that counts constructors, and Lean proves equations such as `sizeOf (σ ⇒ τ) = 1 + sizeOf σ + sizeOf τ` automatically. `congrArg sizeOf h` applies `sizeOf` to both sides of the assumed equation `h : (σ ⇒ τ) = σ`; `simp` rewrites with the size equations, and `omega` (a decision procedure for linear arithmetic over the naturals) sees that `1 + sizeOf σ + sizeOf τ = sizeOf σ` is impossible. Then the theorem, which follows the three steps above exactly:

```
/-- `λx. x x` has no type, in any context. -/
theorem self_application_untypable (Γ : List Ty) (τ : Ty) :
    ¬ HasType Γ (.lam (.app (.var 0) (.var 0))) τ := by
  intro h
  -- Only rule `lam` concludes a judgment about a `λ`: τ = σ ⇒ ρ for some σ, ρ,
  -- and the body `x x` has type ρ when `x : σ`.
  cases h with
  | lam hbody =>
    -- Only rule `app` concludes a judgment about an application: the first
    -- `x` has some type σ' ⇒ ρ and the second has type σ'.
    cases hbody with
    | app hf ha =>
      -- Both are the variable 0, whose type is σ: so σ = σ' ⇒ ρ and σ = σ'.
      cases hf with
      | var e₁ =>
        cases ha with
        | var e₂ =>
          simp at e₁ e₂
          -- e₁ : σ = σ' ⇒ ρ and e₂ : σ = σ', hence σ' ⇒ ρ = σ'.
          subst e₂
          exact arrow_ne_self _ _ e₁.symm
```

Each `cases` is "only rule X concludes a judgment of this shape": Lean tries every constructor of `HasType`, discards those whose conclusion cannot match, and gives us the premises of the one that can. After the last two, `e₁` and `e₂` say what looking up index 0 in the context `σ :: Γ` returned; `simp` computes the lookup, leaving `σ = σ' ⇒ ρ` and `σ = σ'`, and `arrow_ne_self` finishes. Notice that the theorem holds for every context and every type: no annotation, no context, nothing can rescue self-application.

Rust agrees. Chapter 7 showed `|x| x(x)` failing because the type of `x` was unknown; giving `x` a type cannot help, by the argument above. Generics do not help either. Here a generic function asks for a closure `F` that can be called with a reference to its own type, and the closure we pass would need a type that contains itself:

```
fn self_apply<F: Fn(&F)>(f: F) {
    f(&f)
}

pub fn demo() {
    self_apply(|_me| println!("called"));
}
```

```
error[E0644]: closure/coroutine type that references itself
  --> compile_fail/ch08_closure_takes_itself.rs:11:16
   |
11 |     self_apply(|_me| println!("called"));
   |                ^^^^^ recursive type with infinite-size name
   |
   = note: closures cannot capture themselves or take themselves as argument;
           this error may be the result of a recent compiler bug-fix,
           see issue #46062 <https://github.com/rust-lang/rust/issues/46062>
           for more information
note: required by a bound in `self_apply`
  --> compile_fail/ch08_closure_takes_itself.rs:6:18
   |
 6 | fn self_apply<F: Fn(&F)>(f: F) {
   |                  ^^^^^^ required by this bound in `self_apply`
```

"Recursive type with infinite-size name" is the compiler's way of saying $\sigma = \sigma \to \rho$. (The note about a bug fix refers to the history of this check and can be ignored.)

The price of termination

Strong normalization is a strong guarantee, and it has a price: a language in which every program terminates cannot express every computable function. (A total language cannot even contain an interpreter for itself, by the same diagonal argument that makes the halting problem undecidable.) To get general recursion back, a language adds a fixed-point operator `fix` with the typing rule "if $f : \tau \to \tau$ then `fix` $f : \tau$", and loses termination. That extended calculus, usually called PCF, is the core of languages such as ML and Haskell.

Rust makes the same choice: recursive `fn`s and `loop` are unrestricted, so Rust is Turing complete, and its type system promises nothing about termination. A Rust function of type `fn(u64) -> u64` may run forever, and `loop {}` has every type (Chapter 1). Lean makes the opposite choice, and Chapter 7 showed what it costs: functions must be shown to terminate, or be marked `partial` and become opaque to proofs. Chapter 10 explains why a proof assistant has no alternative.

8.5 Bidirectional type checking

Writing a type on every λ is tedious, and Rust rarely asks you to. In `orders.iter().map(|o| o.total)` you do not annotate `o`; the compiler knows it from `map`'s signature. The type of the closure's parameter flows *into* the closure from its surroundings. **Bidirectional type checking** turns this observation into a disciplined algorithm by splitting the typing judgment in two:

- **synthesis**, $\Gamma \vdash t \Rightarrow \tau$: from the term alone, the checker computes its type τ (the type is an *output*);
- **checking**, $\Gamma \vdash t \Leftarrow \tau$: the checker is *given* τ and verifies that t has it (the type is an *input*).

Each rule is written for one mode. Variables, applications, projections and annotated terms synthesize; an unannotated λ is checked:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x \Rightarrow \tau}\ \text{S-Var} \qquad \frac{\Gamma \vdash t \Rightarrow \sigma \to \tau \qquad \Gamma \vdash u \Leftarrow \sigma}{\Gamma \vdash t\ u \Rightarrow \tau}\ \text{S-App} \qquad \frac{\Gamma \vdash t \Leftarrow \tau}{\Gamma \vdash (t : \tau) \Rightarrow \tau}\ \text{S-Ann}$$

$$\frac{\Gamma, x : \sigma \vdash t \Leftarrow \tau}{\Gamma \vdash \lambda x.\, t \Leftarrow \sigma \to \tau}\ \text{C-Lam} \qquad \frac{\Gamma \vdash t \Leftarrow \sigma}{\Gamma \vdash \texttt{inl}\ t \Leftarrow \sigma + \tau}\ \text{C-Inl} \qquad \frac{\Gamma \vdash t \Rightarrow \sigma \qquad \sigma = \tau}{\Gamma \vdash t \Leftarrow \tau}\ \text{C-Switch}$$

Read S-App as an algorithm: synthesize the type of the function; it must be an arrow $\sigma \to \tau$; now *check* the argument against σ, which we just learned; the result is τ. Read C-Lam the same way: we are given $\sigma \to \tau$, so we know the type of x without an annotation, and we check the body against τ. C-Switch connects the modes: to check a term that can synthesize, synthesize and compare. S-Ann goes the other way: an annotation turns a term that can only be checked into one that synthesizes.

The rules follow a simple recipe: **introduction forms check, elimination forms synthesize**. An elimination form has a "principal" subterm whose type determines everything (the function in an application, the pair in a projection), so it can synthesize. An introduction form often cannot: from $\lambda x.\, x$ alone there is no way to know the type of x, and from `inl` 3 there is no way to know the right half of the sum. That is why **a λ checks but does not synthesize**: the information it lacks, the type of its parameter, can only come from outside, from an expected type or an annotation. (A λ whose parameter *is* annotated can synthesize, and our checker allows that too.) In practice annotations are needed in only two places: at the top of a definition, and where a λ is applied on the spot, as in $(\lambda x.\, t)\ u$, which is rare in real code.

Bidirectional checking in Rust

Rust is not literally a bidirectional checker (Chapter 13 shows that it also solves equations between types), but its handling of closures has exactly this flavor. When a closure appears where a function type is expected, the expected type supplies its parameter types: checking mode. When it does not, the compiler must find the parameter type some other way, and if it needs it too early, it gives up:

```
pub fn total_length() -> usize {
    let len = |s| s.len();
    len("checked") + len("synthesized")
}
```

```
error[E0282]: type annotations needed
 --> compile_fail/ch08_closure_needs_annotation.rs:7:16
  |
7 |     let len = |s| s.len();
  |                ^  - type must be known at this point
  |
help: consider giving this closure parameter an explicit type
  |
7 |     let len = |s: /* Type */| s.len();
  |                 ++++++++++++
```

To call the method `len` the compiler must know the type of `s`, and at that point nothing has told it; the later call `len("checked")` comes too late. Two fixes correspond to the two rules above: annotate the parameter (`|s: &str| s.len()`, which makes the closure synthesize), or put the closure where a type is expected, as in `let len: fn(&str) -> usize = |s| s.len();` or `words.iter().map(|s| s.len())`

(checking mode, rule C-Lam). The same happens with sums: `let r = Ok(3);` alone is rejected with "type annotations needed for `Result<i32, _>`", because, like `inl` 3, `Ok(3)` says nothing about the error type.

8.6 The `stlc` crate

This chapter's crate, `code/rust/crates/stlc`, implements the language with a bidirectional checker and an evaluator. The syntax is two `enum`s:

```
/// Simple types: `τ ::= Unit | Bool | Nat | τ → τ | τ × τ | τ + τ`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    Unit,
    Bool,
    Nat,
    /// Functions `σ → τ`.
    Fun(Box<Ty>, Box<Ty>),
    /// Pairs `σ × τ`.
    Prod(Box<Ty>, Box<Ty>),
    /// Tagged unions `σ + τ`: a value is `inl` of a `σ` or `inr` of a `τ`.
    Sum(Box<Ty>, Box<Ty>),
}
```

```
/// Terms. `Lam` may carry an annotation on its parameter; `Ann` is a type
/// annotation `(e : τ)` on any term.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(String),
    /// `λx. e` or `λx: τ. e`
    Lam(String, Option<Ty>, Box<Term>),
    /// `e1 e2`
    App(Box<Term>, Box<Term>),
    /// `(e : τ)`
    Ann(Box<Term>, Ty),
    /// `let x = e1 in e2`
    Let(String, Box<Term>, Box<Term>),
    /// `()`
    Unit,
    /// `true`, `false`
    Bool(bool),
    /// `if c then e1 else e2`
    If(Box<Term>, Box<Term>, Box<Term>),
    /// `0`, `1`, `2`, ...: numerals stand for `succ (succ (... 0))`
    Nat(u64),
    /// `succ e`
    Succ(Box<Term>),
    /// `iter n z s`: apply `s` to `z`, `n` times
    Iter(Box<Term>, Box<Term>, Box<Term>),
    /// `(e1, e2)`
    Pair(Box<Term>, Box<Term>),
    /// `fst e`
    Fst(Box<Term>),
    /// `snd e`
    Snd(Box<Term>),
    /// `inl e`
    Inl(Box<Term>),
    /// `inr e`
    Inr(Box<Term>),
    /// `case e of inl x => e1 | inr y => e2`
    Case(Box<Term>, String, Box<Term>, String, Box<Term>),
}
```

`Lam` carries an `Option<Ty>`: `None` for $\lambda x.\, t$, `Some` for $\lambda x : \tau.\, t$. A parser and a printer handle the concrete syntax used in this chapter, with `λ`, `→` and `×` (or `\`, `->` and `*` in ASCII).

The checker

The checker is a struct holding the context, with two mutually recursive methods, one per judgment. Here are the synthesis rules for the λ-fragment:

```
Var(x) => self.lookup(x),
// An annotation switches to checking mode.
Ann(e, ty) => {
    self.check(e, ty)?;
    Ok(ty.clone())
}
// Elimination of →: synthesize the function's type, then check
// the argument against its domain.
App(f, a) => match self.infer(f)? {
    Ty::Fun(dom, cod) => {
        self.check(a, &dom)?;
        Ok(*cod)
    }
    found => Err(TypeError::NotAFunction {
        term: (**f).clone(),
        found,
    }),
},
// A λ with an annotated parameter can synthesize...
Lam(x, Some(dom), body) => {
    let cod = self.under(x, dom.clone(), |c| c.infer(body))?;
    Ok(fun(dom.clone(), cod))
}
// ...but without one it cannot: what is the type of x?
Lam(_, None, _) => Err(TypeError::CannotSynthesize {
    term: e.clone(),
    form: "an unannotated λ",
}),
```

Each arm is one rule: `Var` is S-Var, `Ann` is S-Ann (check, then return the annotation), `App` is S-App, an annotated `Lam` synthesizes by extending the context and synthesizing the body, and an unannotated `Lam` is an error. The rules for the other forms follow the recipe: literals synthesize, `if` synthesizes the type of its first branch and checks the second against it, and `inl` / `inr` refuse to synthesize. The checking rules:

```
/// The checking rules, `Γ ⊢ e ⇐ τ`: does `e` have the given type `τ`? The
/// type is an input.
fn check_inner(&mut self, e: &Term, ty: &Ty) -> Result<(), TypeError> {
    use Term::*;
    let wrong_shape = |form| TypeError::WrongShape {
        term: e.clone(),
        expected: ty.clone(),
        form,
    };
    match (e, ty) {
        // Introduction of →: the expected type tells us the type of x.
        (Lam(x, ann, body), Ty::Fun(dom, cod)) => {
            if let Some(ann) = ann
                && ann != &**dom
            {
                return Err(TypeError::Mismatch {
                    term: e.clone(),
                    expected: ty.clone(),
                    found: fun(ann.clone(), (**cod).clone()),
                });
            }
            self.under(x, (**dom).clone(), |c| c.check(body, cod))
        }
        (Lam(..), _) => Err(wrong_shape("a function")),
        (Pair(a, b), Ty::Prod(ta, tb)) => {
            self.check(a, ta)?;
```

```
            self.check(b, tb)
        }
        (Pair(..), _) => Err(wrong_shape("a pair")),
        (Inl(a), Ty::Sum(left, _)) => self.check(a, left),
        (Inr(b), Ty::Sum(_, right)) => self.check(b, right),
        (Inl(_) | Inr(_), _) => Err(wrong_shape("an injection into a sum")),
        (If(c, a, b), _) => {
            self.check(c, &Ty::Bool)?;
            self.check(a, ty)?;
            self.check(b, ty)
        }
        (Let(x, e1, e2), _) => {
            let ty1 = self.infer(e1)?;
            self.under(x, ty1, |c| c.check(e2, ty))
        }
        (Case(scrut, x, l, y, r), _) => {
            let (left, right) = self.infer_sum(scrut)?;
            self.under(x, left, |c| c.check(l, ty))?;
            self.under(y, right, |c| c.check(r, ty))
        }
        (Iter(n, z, s), _) => {
            self.check(n, &Ty::Nat)?;
            self.check(z, ty)?;
            self.check(s, &fun(ty.clone(), ty.clone()))
        }
        // Everything else: synthesize a type and compare (rule Switch).
        _ => {
            let found = self.infer(e)?;
            if &found == ty {
                Ok(())
            } else {
                Err(TypeError::Mismatch {
                    term: e.clone(),
                    expected: ty.clone(),
                    found,
                })
            }
        }
    }
}
```

The first arm is C-Lam: the expected type $\sigma \to \tau$ provides the parameter's type (and if there is an annotation too, it must agree). Pairs, injections, `if`, `let`, `case` and `iter` push the expected type into their parts. The last arm is C-Switch. The method `under` extends the context for the duration of a closure and pops the binding afterwards, so that inner bindings shadow outer ones and disappear when their scope ends.

The crate can record every judgment the checker visits. The example `crates/stlc/examples/ch08_stlc.rs` checks an unannotated `twice` against $(\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$:

```
Checking λf. λx. f (f x) against (Nat → Nat) → Nat → Nat:
  ⊢ λf. λx. f (f x) ⇐ (Nat → Nat) → Nat → Nat
    f: Nat → Nat ⊢ λx. f (f x) ⇐ Nat → Nat
      f: Nat → Nat, x: Nat ⊢ f (f x) ⇐ Nat
        f: Nat → Nat, x: Nat ⊢ f (f x) ⇒ Nat
          f: Nat → Nat, x: Nat ⊢ f ⇒ Nat → Nat
          f: Nat → Nat, x: Nat ⊢ f x ⇐ Nat
            f: Nat → Nat, x: Nat ⊢ f x ⇒ Nat
              f: Nat → Nat, x: Nat ⊢ f ⇒ Nat → Nat
              f: Nat → Nat, x: Nat ⊢ x ⇐ Nat
                f: Nat → Nat, x: Nat ⊢ x ⇒ Nat
```

Read it from the top. Two C-Lam steps move f and x into the context, with types taken from the expected type. The body `f (f x)` is checked against `Nat`, but an application cannot be checked directly:

C-Switch synthesizes its type instead (the `⇒` line one level down, with the result `Nat`, compared with the expected `Nat`). S-App synthesizes the type of `f`, `Nat → Nat`, and checks the argument `f x` against `Nat`, which repeats the pattern one level deeper, ending with the variable `x`, checked by switching to synthesis. Information flows down through `⇐` and back up through `⇒`, and no annotation was needed.

When the search for a derivation fails, the error points at the subterm where it failed:

```
Rejected by the checker:
  3 4
      `3` has type Nat; it is not a function
  λx. x
      cannot synthesize a type for `λx. x` (an unannotated λ can only be checked)
  if true then 1 else false
      `false` has type Bool, but Nat was expected
  (λx. x : Nat)
      `λx. x` is a function, but Nat was expected
  λx: Nat → Nat. x x
      `x` has type Nat → Nat, but Nat was expected
```

The last one is self-application with an annotation: whatever type we give `x`, the second `x` is checked against the domain of the first, and the types do not match.

The evaluator: environments and closures

To *run* programs, the crate does not substitute, as Chapter 7 did; it uses an **environment**, a list of variable bindings, the way real interpreters and compilers work. Evaluating a λ cannot evaluate its body (the parameter has no value yet), so it produces a **closure**: the parameter, the body, and the values of the body's other free variables, captured from the environment.

```rust
/// The results of evaluation.
#[derive(Clone, Debug, PartialEq)]
pub enum Value {
    Unit,
    Bool(bool),
    Nat(u64),
    Pair(Rc<Value>, Rc<Value>),
    Inl(Rc<Value>),
    Inr(Rc<Value>),
    Closure(Rc<Closure>),
}

/// A function value: a parameter, a body, and the captured variables.
#[derive(Debug, PartialEq)]
pub struct Closure {
    pub param: String,
    pub body: Term,
    /// The values of the body's free variables (other than the parameter),
    /// copied from the environment when the closure was created.
    pub captured: Env,
}
```

```rust
/// Builds the closure for `λparam. body` in `env`, capturing only the free
/// variables of the λ, as a Rust closure does.
fn make_closure(env: &Env, param: &str, body: &Term) -> Result<Value, Stuck> {
    let lam = Term::Lam(param.to_string(), None, Box::new(body.clone()));
    let mut captured = Vec::new();
    for x in lam.free_vars() {
        captured.push((x.clone(), lookup(env, &x)?));
    }
    Ok(Value::Closure(Rc::new(Closure {
        param: param.to_string(),
        body: body.clone(),
```

```
        captured,
    }))
}
```

```
/// Calls a function value: evaluate the body in the captured environment
/// extended with the argument.
pub fn apply(f: &Value, arg: Value) -> Result<Value, Stuck> {
    match f {
        Value::Closure(c) => eval(&extend(&c.captured, &c.param, arg), &c.body),
        other => Err(Stuck(format!("cannot call {other}: it is not a function"))),
    }
}
```

`make_closure` computes the free variables of the λ (Chapter 7) and copies their values, and only theirs, out of the environment. Calling a closure evaluates its body in the captured environment extended with the argument. This is **lexical scope**: a function sees the variables of the place where it was *defined*, not of the place where it is called. The evaluator itself is call-by-value, like Rust:

```
/// Evaluates a term in an environment, call-by-value.
pub fn eval(env: &Env, e: &Term) -> Result<Value, Stuck> {
    use Term::*;
    match e {
        Var(x) => lookup(env, x),
        Lam(x, _, body) => make_closure(env, x, body),
        App(f, a) => {
            let f = eval(env, f)?;
            let a = eval(env, a)?; // call-by-value: the argument first
            apply(&f, a)
        }
        Ann(e, _) => eval(env, e),
        Let(x, e1, e2) => {
            let v = eval(env, e1)?;
            eval(&extend(env, x, v), e2)
        }
        Unit => Ok(Value::Unit),
        Bool(b) => Ok(Value::Bool(*b)),
        If(c, a, b) => match eval(env, c)? {
            Value::Bool(true) => eval(env, a),
            Value::Bool(false) => eval(env, b),
            other => Err(Stuck(format!("`if` on {other}, which is not a boolean"))),
        },
        Nat(n) => Ok(Value::Nat(*n)),
        Succ(n) => match eval(env, n)? {
            Value::Nat(k) => Ok(Value::Nat(k + 1)),
            other => Err(Stuck(format!("`succ` of {other}, which is not a number"))),
        },
        Iter(n, z, s) => match eval(env, n)? {
            Value::Nat(k) => {
                let s = eval(env, s)?;
                let mut acc = eval(env, z)?;
                for _ in 0..k {
                    acc = apply(&s, acc)?;
                }
                Ok(acc)
            }
            other => Err(Stuck(format!("`iter` on {other}, which is not a number"))),
        },
        Pair(a, b) => Ok(Value::Pair(Rc::new(eval(env, a)?), Rc::new(eval(env, b)?))),
        Fst(p) | Snd(p) => match eval(env, p)? {
            Value::Pair(a, b) => {
                let v = if matches!(e, Fst(_)) { a } else { b };
                Ok((*v).clone())
            }
            other => Err(Stuck(format!("projection of {other}, which is not a pair"))),
        },
        Inl(a) => Ok(Value::Inl(Rc::new(eval(env, a)?))),
```

```
        Inr(b) => Ok(Value::Inr(Rc::new(eval(env, b)?))),
        Case(scrut, x, l, y, r) => match eval(env, scrut)? {
            Value::Inl(v) => eval(&extend(env, x, (*v).clone()), l),
            Value::Inr(v) => eval(&extend(env, y, (*v).clone()), r),
            other => Err(Stuck(format!("`case` on {other}, which is not a sum"))),
        },
    }
}
```

Here is the crate's small library, with the types the checker synthesizes, and some programs that use it (`with_library` wraps each program in `let`s that define the library):

```
The library, with synthesized types:
  twice  : (Nat → Nat) → Nat → Nat
  add    : Nat → Nat → Nat
  mul    : Nat → Nat → Nat
  iszero : Nat → Bool
  pred   : Nat → Nat
  swap   : Nat × Bool → Bool × Nat
```

```
Evaluation:
  twice (add 3) 4  :  Nat  ⇓  10
  mul 6 7  :  Nat  ⇓  42
  pred 10  :  Nat  ⇓  9
  iszero 0  :  Bool  ⇓  true
  swap (7, true)  :  Bool × Nat  ⇓  (true, 7)
  case (inr true : Nat + Bool) of inl n => n | inr b => 1  :  Nat  ⇓  1
  add 3  :  Nat → Nat  ⇓  <closure λn. iter m n (λk: Nat. succ k) | m = 3>
```

`pred` is Kleene's pair trick from Chapter 7, now typed. The last line is the interesting one: `add 3` is a function, and its value is a closure whose body is `λn. iter m n (λk: Nat. succ k)` and whose captured environment is `m = 3`. Only `m` is captured, because it is the only free variable of the body; the library functions in scope are not.

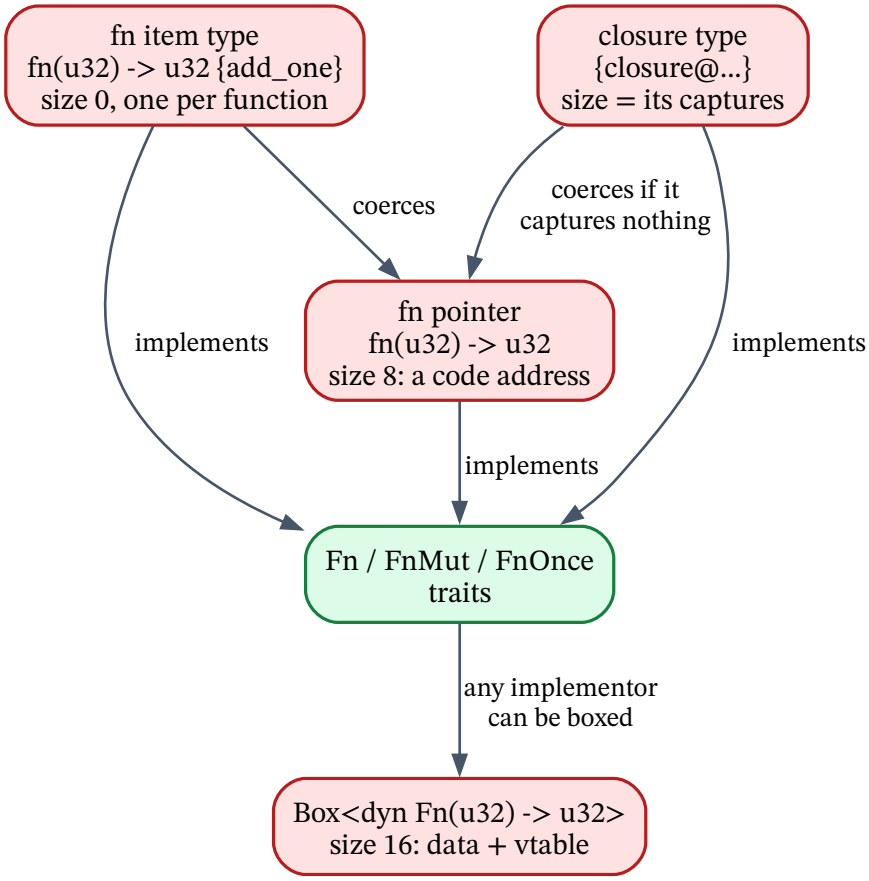
Look at the `Err(Stuck(...))` arms of `eval`. They are reached when the evaluator finds something it cannot handle, such as an `if` on a number. If we skip the checker, ill-typed programs reach them:

```
Run without checking:
  3 4  ⇓  stuck: cannot call 3: it is not a function
  if 0 then 1 else 2  ⇓  stuck: `if` on 0, which is not a boolean
```

For every program the checker accepts, these arms are dead code. That claim is the **progress** theorem, and Chapter 9 proves it, for a language like this one, in Lean.

8.7 Functions in Rust

In the STLC there is one kind of function type, $\sigma \to \tau$. In Rust there are several, because Rust cares about two things the calculus ignores: what a function value occupies in memory, and what it is allowed to do with the data it captured. The example `code/rust/examples/ch08_closures.rs` explores them all.



Rust's function types and how they relate: fn items and non-capturing closures coerce to fn pointers, and all of them implement the Fn traits, through which any of them can be boxed as a trait object.

`fn` items and function pointers

A function declared with `fn` is called a **`fn` item**, and each one has its own type, distinct from every other function's, even one with the same signature:

```
fn add_one(x: u32) -> u32 {
    x + 1
}

fn double(x: u32) -> u32 {
    x * 2
}
```

```
let f = add_one;
println!(
    "add_one: type {}, size {}",
    type_name_of_val(&f),
    size_of_val(&f)
);
let table: [fn(u32) -> u32; 2] = [add_one, double];
let p = table[1];
println!(
    "table[1]: type {}, size {}",
    type_name_of_val(&p),
    size_of_val(&p)
);
println!("table[1](21) = {}", p(21));
```

```
add_one: type ch08_closures::add_one, size 0
table[1]: type fn(u32) -> u32, size 8
table[1](21) = 42
```

The type of `add_one` is a **zero-sized type**: the compiler knows statically which function it is, so a value of that type needs no bytes at all, and a call through it compiles to a direct call. (The compiler writes this type as `fn(u32) -> u32 {add_one}` in error messages.) To put two different functions in one array, they must have one type, and Rust **coerces** both to the **function pointer** type `fn(u32) -> u32`: an 8-byte code address, called indirectly. A function pointer is the closest thing in Rust to the $\sigma \to \tau$ of the STLC: one type for all functions with that signature.

Closures and their anonymous types

A closure is more than code: it carries the values of the variables it captures. The compiler implements each closure expression as an anonymous struct with one field per capture, plus an implementation of the call operator. Roughly (this sketch is not compiled; the real traits cannot be implemented by hand in stable Rust):

```
// let percent: u16 = 3;
// let by_value = move |x: u16| x * percent;
// becomes, approximately:
struct ByValueClosure {
    percent: u16, // the captured variable
}
impl Fn(u16) -> u16 for ByValueClosure {
    fn call(&self, x: u16) -> u16 {
        x * self.percent
    }
}
```

This is the same representation as the `stlc` crate's `Closure`: code plus captured environment. The size of a closure is the size of its captures:

```
let percent: u16 = 3;
let label = String::from("invoice 2041");
let nothing = |x: u16| x + 1;
let by_ref = |x: u16| x * percent; // captures &percent: a pointer
let by_value = move |x: u16| x * percent; // captures a copy of percent
let owns_label = move || label.len(); // captures the String itself
println!("closure type: {}", type_name_of_val(&nothing));
println!(
    "sizes: nothing {}, by_ref {}, by_value {}, owns_label {}",
    size_of_val(&nothing),
    size_of_val(&by_ref),
    size_of_val(&by_value),
    size_of_val(&owns_label)
);
assert_eq!(
    (nothing(1), by_ref(1), by_value(1), owns_label()),
    (2, 3, 3, 12)
);
```

```
closure type: ch08_closures::main::{{closure}}
sizes: nothing 0, by_ref 8, by_value 2, owns_label 24
```

A closure that captures nothing is zero-sized, like a `fn` item. One that captures `percent` by reference holds a pointer (8 bytes); one that captures it by value holds a copy of the `u16` (2 bytes); one that owns a `String` holds the `String` itself (24 bytes: pointer, capacity and length).

Since each closure expression defines its own struct, **every closure has its own type**, and no two closures, even textually identical ones, have the same type. The type has no name you can write, which is why the compiler prints it as `{closure@...}` with a source location. This runs into a rule you know from Chapter 1, T-If:

```
pub fn scaler(double: bool, k: u32) -> impl Fn(u32) -> u32 {
    let times_k = move |x| x * k;
    let plus_k = move |x| x + k;
    if double { times_k } else { plus_k }
}
```

```
error[E0308]: `if` and `else` have incompatible types
 --> compile_fail/ch08_closure_types_differ.rs:9:34
  |
7 |     let times_k = move |x| x * k;
  |                   -------- the expected closure
8 |     let plus_k = move |x| x + k;
  |                  -------- the found closure
9 |     if double { times_k } else { plus_k }
  |                 -------          ^^^^^^ expected closure, found a different closure
  |                 |
  |                 expected because of this
  |
  = note: expected closure `{closure@compile_fail/ch08_closure_types_differ.rs:7:19: 7:27}`
             found closure `{closure@compile_fail/ch08_closure_types_differ.rs:8:18: 8:26}`
  = note: no two closures, even if identical, have the same type
  = help: consider boxing your closure and/or using it as a trait object
help: you could change the return type to be a boxed trait object
  |
6 - pub fn scaler(double: bool, k: u32) -> impl Fn(u32) -> u32 {
6 + pub fn scaler(double: bool, k: u32) -> Box<dyn Fn(u32) -> u32> {
  |
help: if you change the return type to expect trait objects, box the returned expressions
  |
9 |     if double { Box::new(times_k) } else { Box::new(plus_k) }
  |                 +++++++++       +          +++++++++      +
```

The two branches have two different types, so no instance of T-If applies. The compiler's suggestion is the standard fix: erase the difference behind a pointer, `Box<dyn Fn(u32) -> u32>`. (If neither closure captured anything, the program would compile: both would be coerced to the function pointer type `fn(u32) -> u32`.) A capturing closure can never become a function pointer, because a code address has nowhere to keep the captured data:

```
pub fn shift_all(xs: &mut [u32], offset: u32) {
    let shift: fn(u32) -> u32 = |x| x + offset;
    for x in xs {
        *x = shift(*x);
    }
}
```

```
error[E0308]: mismatched types
 --> compile_fail/ch08_capture_to_fn_pointer.rs:7:33
  |
7 |     let shift: fn(u32) -> u32 = |x| x + offset;
  |                ---------------   ^^^^^^^^^^^^^^ expected fn pointer, found closure
  |                |
```

```
   |                expected due to this
   |
  = note: expected fn pointer `fn(u32) -> u32`
                found closure `{closure@compile_fail/ch08_capture_to_fn_pointer.rs:7:33:
7:36}`
note: closures can only be coerced to `fn` types if they do not capture any variables
 --> compile_fail/ch08_capture_to_fn_pointer.rs:7:41
  |
7 |     let shift: fn(u32) -> u32 = |x| x + offset;
  |                                         ^^^^^^ `offset` captured here
```

`impl Fn` and `dyn Fn`

Since closure types cannot be named, functions that accept or return closures name them through the traits they implement. `impl Fn(u64) -> u64` in argument position means "some type implementing this trait, chosen by the caller": it is a generic parameter in disguise, and each call is compiled separately for the closure's actual type (monomorphization, Chapter 12), so the call is direct and can be inlined. In return position it means "some type chosen by the function, which the caller may not know": an *existential* type, the subject of Chapter 15. `dyn Fn(u64) -> u64` is a **trait object**: a pointer to the closure's data together with a pointer to a table of its methods (the *vtable*). It is the only way to treat closures of different types uniformly, for example in a `Vec`:

```
// Different closures have different types; to store them together we
// erase the types behind a pointer: `Box<dyn Fn>`.
let discount = 10;
let add_fee = fee_curried(3);
let pipeline: Vec<Box<dyn Fn(u64) -> u64>> = vec![
    Box::new(move |c| c - c * discount / 100), // 10% off
    Box::new(move |c| c + add_fee(c)),         // plus a 3% fee
    Box::new(|c| c / 100 * 100),               // whole units only
];
let result = pipeline.iter().fold(25_000, |amount, step| step(amount));
println!("pipeline(25000) = {result}");
println!(
    "size of Box<dyn Fn(u64) -> u64> = {}",
    size_of_val(&pipeline[0])
);
```

```
pipeline(25000) = 23100
size of Box<dyn Fn(u64) -> u64> = 16
```

The box costs a heap allocation (unless the closure is zero-sized), and each call goes through the vtable, an indirect call the compiler usually cannot inline. The three representations, generic `impl Fn`, function pointer and `dyn Fn`, are three engineering answers to one mathematical type $\sigma \to \tau$.

Capture modes

How does a closure capture a variable? The compiler picks the least powerful mode that the body needs:

- **by shared reference** (`&x`) if the body only reads `x`;
- **by mutable reference** (`&mut x`) if the body modifies `x`;
- **by value** (moving `x` into the closure) if the body moves `x` out.

The keyword `move` overrides the choice: every captured variable is captured by value (moved, or copied if its type is `Copy`). Since the 2021 edition, a closure that uses only a field `x.a` captures just that field, not all of `x`.

```
// By shared reference: the closure only reads `prices`.
let prices = vec![120, 80, 45];
let total = || prices.iter().sum::<u32>();
println!(
    "total = {}, and prices is still usable: {:?}",
    total(),
    prices
);

// By mutable reference: the closure updates `calls`.
let mut calls = 0;
let mut count = || {
    calls += 1;
    calls
};
count();
count();
println!("calls = {calls}");

// By value: the closure moves `report` out of itself when called.
let report = String::from("Q3 report");
let send = move || report;
println!("sent: {}", call_once(send));
```

```
total = 245, and prices is still usable: [120, 80, 45]
calls = 2
sent: Q3 report
```

`total` borrows `prices`, so `prices` stays usable. `count` borrows `calls` mutably for as long as `count` is in use, so `calls` can be printed only after the last call. `send` owns `report`. Borrowing captures are governed by the borrow checker like any other reference, and the most common surprise is a closure that outlives what it borrows:

```
pub fn make_adder(n: u32) -> impl Fn(u32) -> u32 {
    |x| x + n
}
```

```
error[E0373]: closure may outlive the current function, but it borrows `n`, which is owned
by the current function
 --> compile_fail/ch08_closure_escapes.rs:7:5
  |
7 |     |x| x + n
  |     ^^^     - `n` is borrowed here
  |     |
  |     may outlive borrowed value `n`
  |
note: closure is returned here
 --> compile_fail/ch08_closure_escapes.rs:7:5
  |
7 |     |x| x + n
  |     ^^^^^^^^^
help: to force the closure to take ownership of `n` (and any other referenced variables),
use the `move` keyword
  |
7 |     move |x| x + n
  |     ++++
```

The closure reads `n`, so it captures `&n`, a reference to a local variable that dies when `make_adder` returns. `move` fixes it by storing a copy of `n` in the closure. Lifetimes of references are the subject of Part VI; here it is enough to see that the λ-calculus has no such issue, because its variables are not places in memory.

`Fn`, `FnMut` and `FnOnce`

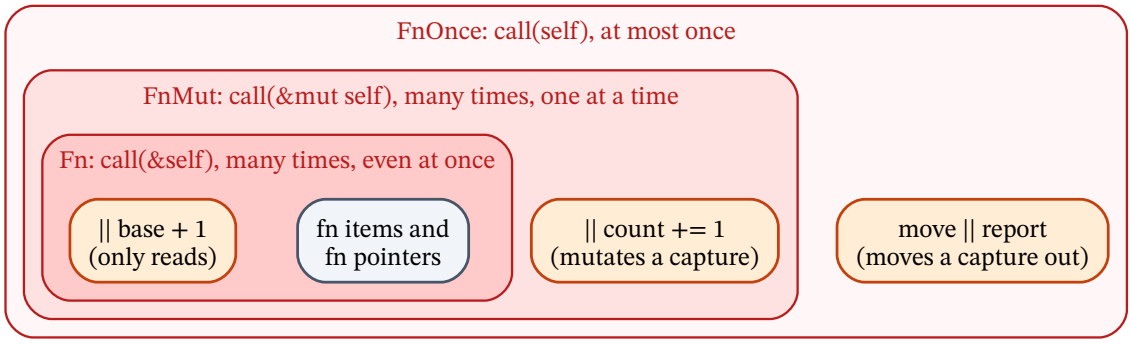
Calling a closure is calling a method on its struct, and the question is how the method receives `self`. The three possibilities are three traits, here in simplified form (the real ones are generic over the argument tuple and cannot be implemented in stable Rust):

```
trait FnOnce<Args> {
    type Output;
    fn call_once(self, args: Args) -> Self::Output; // consumes the closure
}
trait FnMut<Args>: FnOnce<Args> {
    fn call_mut(&mut self, args: Args) -> Self::Output; // exclusive access
}
trait Fn<Args>: FnMut<Args> {
    fn call(&self, args: Args) -> Self::Output; // shared access
}
```

What a closure's body does with its captures decides which traits it implements:

- if it moves a captured value out, it can only be called by value, once: it implements `FnOnce` only;
- if it mutates a capture, it needs exclusive access: `FnMut` (and therefore `FnOnce`);
- if it only reads, shared access suffices: `Fn` (and therefore `FnMut` and `FnOnce`).

The supertrait chain `Fn: FnMut: FnOnce` says that every `Fn` closure can be used where an `FnMut` or `FnOnce` is expected: a function that may be called many times through a shared reference may certainly be called through an exclusive one, or once. `fn` items and function pointers implement all three.



The three closure traits as nested sets: every Fn closure is also FnMut, and every FnMut closure is also FnOnce.

Functions that *accept* closures choose the weakest trait they need, which gives the caller the most freedom:

```
/// Calls `f` twice. `Fn` means: callable through a shared reference.
fn call_twice(f: impl Fn() -> u32) -> u32 {
    f() + f()
```

```
}

/// Calls `f` twice. `FnMut` means: callable through a mutable reference.
fn call_twice_mut(mut f: impl FnMut() -> u32) -> u32 {
    f() + f()
}

/// Calls `f` once. `FnOnce` means: callable by value, consuming it.
fn call_once(f: impl FnOnce() -> String) -> String {
    f()
}
```

```
let base = 20;
let reads = || base + 1; // Fn: only reads its capture
println!("call_twice(reads)     = {}", call_twice(reads));
println!("call_twice_mut(reads) = {}", call_twice_mut(reads));
let mut ticks = 0;
let bumps = || {
    ticks += 10;
    ticks
}; // FnMut: mutates its capture
println!("call_twice_mut(bumps) = {}", call_twice_mut(bumps));
// `move` decides HOW variables are captured, not WHICH trait you get:
let greeting = String::from("hello");
let greet = move || greeting.len() as u32; // moved in, but only read
println!("call_twice(greet)     = {}", call_twice(&greet));
println!("greet is still callable: {}", greet());
```

```
call_twice(reads)     = 42
call_twice_mut(reads) = 42
call_twice_mut(bumps) = 30
call_twice(greet)     = 10
greet is still callable: 5
```

`reads` is `Fn`, so it can be passed to both `call_twice` and `call_twice_mut`. `bumps` mutates `ticks`, so it is only `FnMut`. `greet` was declared `move`, but its body only reads `greeting`, so it is `Fn`: **`move` decides how variables are captured, not which traits the closure implements**. Passing `&greet` works because a shared reference to an `Fn` closure is itself an `Fn` closure, and afterwards `greet` is still ours. The compiler rejects the two combinations that would be unsound. A mutating closure where `Fn` is required:

```
fn call_twice<F: Fn()>(f: F) {
    f();
    f();
}

pub fn count_calls() {
    let mut count = 0;
    call_twice(|| count += 1);
}
```

```
error[E0594]: cannot assign to `count`, as it is a captured variable in a `Fn` closure
  --> compile_fail/ch08_fn_mutates.rs:13:19
   |
 6 | fn call_twice<F: Fn()>(f: F) {
   |                        - change this to accept `FnMut` instead of `Fn`
...
12 |     let mut count = 0;
   |         --------- `count` declared here, outside the closure
13 |     call_twice(|| count += 1);
   |     ---------- -- ^^^^^^^^^^ cannot assign
   |     |          |
```

```
   |     |          in this closure
   |     expects `Fn` instead of `FnMut`
```

and an `FnOnce` closure called twice:

```
pub fn send_twice() {
    let report = String::from("Q3 report");
    let send = move || report; // returns `report`, moving it out
    let first = send();
    let second = send();
}
```

```
error[E0382]: use of moved value: `send`
  --> compile_fail/ch08_fnonce_twice.rs:10:18
   |
 9 |     let first = send();
   |                 ------ `send` moved due to this call
10 |     let second = send();
   |                  ^^^^ value used here after move
   |
note: closure cannot be invoked more than once because it moves the variable `report` out of
its environment
  --> compile_fail/ch08_fnonce_twice.rs:8:24
   |
 8 |     let send = move || report; // returns `report`, moving it out
   |                        ^^^^^^
note: this value implements `FnOnce`, which causes it to be moved when called
  --> compile_fail/ch08_fnonce_twice.rs:9:17
   |
 9 |     let first = send();
   |                 ^^^^
```

The second error is E0382, "use of moved value", the error of Chapter 6. The first call moved `send` (calling an `FnOnce` consumes it), so there is nothing left to call. Read the three traits as answers to the question **how many times may this function be used?** `FnOnce` : at most once, because a call uses up resources. `FnMut` : any number of times, one at a time. `Fn` : any number of times, even simultaneously from several places. In the STLC the question does not arise: every function is a value that can be used any number of times, because the calculus has contraction. Rust's `FnOnce` is a function that can be used *at most once*, an **affine** function, and Chapter 21 will write its type as $A \multimap B$, the linear implication of linear logic. This is the first place where the book's question, whether Rust's ownership is "based on affine types", shows up in the type of a function.

The cost of currying

In the λ-calculus, a two-argument function and its curried form are interchangeable at no cost: $(A \times B) \to C$ and $A \to (B \to C)$ are isomorphic (Chapter 11 proves it), and partial application is just application. In Rust, currying a particular function by hand is easy and cheap:

```
/// A fee of `percent` percent on `amount` cents.
fn fee(percent: u64, amount: u64) -> u64 {
    amount * percent / 100
}

/// The same function, curried by hand: the result captures `percent`.
fn fee_curried(percent: u64) -> impl Fn(u64) -> u64 {
    move |amount| fee(percent, amount)
}
```

`fee_curried(3)` is a closure that captures `percent`: 8 bytes, no allocation. But a *generic* `curry`, turning any two-argument function into a curried one, runs into three costs at once. The natural signature does not even compile:

```
pub fn curry<A: Clone, B, C>(
    f: impl Fn(A, B) -> C + Clone,
) -> impl Fn(A) -> impl Fn(B) -> C {
    move |a: A| {
        let f = f.clone();
        move |b: B| f(a.clone(), b)
    }
}
```

```
error[E0562]: `impl Trait` is not allowed in the return type of `Fn` trait bounds
 --> compile_fail/ch08_curry_nested_impl.rs:8:20
  |
8 | ) -> impl Fn(A) -> impl Fn(B) -> C {
  |                    ^^^^^^^^^^^^^^^
  |
  = note: `impl Trait` is only allowed in arguments and return types of functions and
methods
  = note: see issue #99697 <https://github.com/rust-lang/rust/issues/99697> for more
information
```

Rust allows `impl Trait` in the return type of a function, but not inside the return type of an `Fn` bound. The version that compiles boxes the inner closure:

```
/// Currying, for any two-argument function. The inner closure is returned as
/// a `Box<dyn Fn>` (Rust does not allow `impl Fn(A) -> impl Fn(B) -> C`), and
/// it must clone `a` on every call, because it may be called many times.
fn curry<A, B, C>(
    f: impl Fn(A, B) -> C + Clone + 'static,
) -> impl Fn(A) -> Box<dyn Fn(B) -> C>
where
    A: Clone + 'static,
    B: 'static,
    C: 'static,
{
    move |a: A| {
        let f = f.clone();
        Box::new(move |b: B| f(a.clone(), b))
    }
}
```

1. **An allocation.** The inner closure is a `Box<dyn Fn>`: one heap allocation per partial application (it holds at least `a`), and an indirect call per use.
2. **Clones.** The inner closure is `Fn`, so it may be called many times, and each call needs its own `A` to pass to `f`, which takes `A` by value. It must clone `a` on every call. The outer closure must also clone `f` into each inner closure, hence `Clone` on `f`.
3. **`'static` bounds.** A boxed trait object must say how long the data it points to lives; here we require it to own everything (`'static`, Chapter 26).

The example counts the clones with a type that increments a counter in its `Clone` implementation:

```
println!("fee(3, 2000)            = {}", fee(3, 2000));
println!("fee_curried(3)(2000)    = {}", fee_curried(3)(2000));
println!("curry(fee)(3)(2000)     = {}", curry(fee)(3)(2000));
println!("size of fee_curried(3)  = {}", size_of_val(&fee_curried(3)));
println!(
    "size of curry(fee)(3)   = {} (plus a heap allocation)",
```

```
    size_of_val(&curry(fee)(3))
);

let tag = |t: Tracked, line: &str| format!("[{}] {line}", t.0);
let tagger = curry(tag)(Tracked("audit".to_string()));
for line in ["opened", "approved", "closed"] {
    println!("{}", tagger(line));
}
println!(
    "clones of the captured Tracked: {}",
    CLONES.load(Ordering::Relaxed)
);
```

```
fee(3, 2000)            = 60
fee_curried(3)(2000)    = 60
curry(fee)(3)(2000)     = 60
size of fee_curried(3)  = 8
size of curry(fee)(3)   = 16 (plus a heap allocation)
[audit] opened
[audit] approved
[audit] closed
clones of the captured Tracked: 3
```

Three calls, three clones of the captured value. None of this is a flaw in Rust: it is what currying *means* once values have owners and functions may be called many times. The λ-calculus duplicates freely (contraction) and never frees anything; Rust makes duplication explicit and memory management visible, and the cost of currying is the bill.

8.8 Intrinsically typed terms in Lean

So far, a term and its type have been separate things: there is a set of terms, some of which are well typed, and a typing relation that says which. This is called **extrinsic** typing ("types from outside"), and it is the style of Chapter 3, of the `stlc` crate, and of Chapter 9. Lean, with its dependent types, allows another style. In **intrinsic** typing, the type of a term is part of the term's *Lean type*: `Term Γ τ` is the type of terms of type `τ` in context `Γ`, and a term that is not well typed cannot be constructed at all. The file is `code/lean/Book/Ch08/Intrinsic.lean`.

Types, and what they mean

The object language's types are an ordinary inductive type, with a scoped notation `⇒` for the arrow (scoped so that it does not leak into other files), and each type has a *meaning*: the Lean type of its values.

```
/-- The simple types: unit, booleans, naturals, functions, pairs and sums. -/
inductive Ty where
  | unit
  | bool
  | nat
  | arrow (σ τ : Ty)
  | prod (σ τ : Ty)
  | sum (σ τ : Ty)
  deriving DecidableEq, Repr

/-- `σ ⇒ τ` is the function type (scoped: it does not leak into other files). -/
scoped infixr:25 " ⇒ " => Ty.arrow
```

```
/-- What a type means: the Lean type of its values. -/
@[reducible] def Ty.denote : Ty → Type
  | unit => Unit
  | bool => Bool
  | nat => Nat
  | arrow σ τ => σ.denote → τ.denote
  | prod σ τ => σ.denote × τ.denote
  | sum σ τ => σ.denote ⊕ τ.denote
```

`Ty.denote` is a function from object-language types to Lean types: `nat` means `Nat`, an arrow means a Lean function type, `prod` means a Lean pair, and `sum` means Lean's sum type `⊕`. (`@[reducible]` lets Lean unfold it freely while checking types, so that a value of `nat.denote` can be used as a `Nat`.)

Variables are proofs of membership

A variable must carry its type. We use the indexed family that Chapter 6 introduced for contexts: `Var Γ τ` is a position in the list `Γ` at which the type `τ` occurs. `here` is the most recent binding, de Bruijn index 0, and `there` skips one; but unlike Chapter 7's bare numbers, a `Var` *proves* that the position holds the right type.

```
/-- `Var Γ τ` is a variable of type `τ` in the context `Γ`: a position in the
list `Γ` where `τ` occurs. `here` is the most recent binding (de Bruijn index
0); `there x` skips one binding. -/
inductive Var : List Ty → Ty → Type where
  | here {Γ : List Ty} {τ : Ty} : Var (τ :: Γ) τ
  | there {Γ : List Ty} {σ τ : Ty} : Var Γ τ → Var (σ :: Γ) τ
```

Terms are typing derivations

Now the terms. Each constructor is a typing rule: the indices in its result type are the conclusion, its arguments are the premises.

```
/-- `Term Γ τ` is a term of type `τ` in the context `Γ`. Each constructor is
a typing rule; an ill-typed term cannot even be written down. -/
inductive Term : List Ty → Ty → Type where
  | var {Γ τ} : Var Γ τ → Term Γ τ
  | lam {Γ σ τ} : Term (σ :: Γ) τ → Term Γ (σ ⇒ τ)
  | app {Γ σ τ} : Term Γ (σ ⇒ τ) → Term Γ σ → Term Γ τ
  | unit {Γ} : Term Γ .unit
  | tt {Γ} : Term Γ .bool
  | ff {Γ} : Term Γ .bool
  | ite {Γ τ} : Term Γ .bool → Term Γ τ → Term Γ τ → Term Γ τ
  | zero {Γ} : Term Γ .nat
  | succ {Γ} : Term Γ .nat → Term Γ .nat
  | iter {Γ τ} : Term Γ .nat → Term Γ τ → Term Γ (τ ⇒ τ) → Term Γ τ
  | pair {Γ σ τ} : Term Γ σ → Term Γ τ → Term Γ (.prod σ τ)
  | fst {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ σ
  | snd {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ τ
  | inl {Γ σ τ} : Term Γ σ → Term Γ (.sum σ τ)
  | inr {Γ σ τ} : Term Γ τ → Term Γ (.sum σ τ)
  | case {Γ σ τ ρ} :
      Term Γ (.sum σ τ) → Term (σ :: Γ) ρ → Term (τ :: Γ) ρ → Term Γ ρ
```

Compare `lam` with rule T-Lam: from a body of type `τ` in the context `σ :: Γ` (the context extended with the parameter), build a term of type `σ ⇒ τ` in `Γ`. `app` is T-App, `case` is T-Case, and so on. There is no annotation on `lam`: the parameter type `σ` is part of the term's type index. Because the constructors *are* the rules, a value of type `Term Γ τ` is at once a term and its typing derivation. Some example terms (`lit n` builds the numeral `n` from `zero` and `succ`):

```
/-- The numeral `n` as a term: `succ (succ (... zero))`. -/
def lit {Γ : List Ty} : Nat → Term Γ .nat
  | 0 => .zero
  | n + 1 => .succ (lit n)

/-- `twice = λf. λx. f (f x)`. In the body, `f` is `there here` (one binder
up) and `x` is `here`. -/
def twice : Term [] ((nat ⇒ nat) ⇒ nat ⇒ nat) :=
  .lam (.lam (.app (.var (.there .here)) (.app (.var (.there .here)) (.var .here))))

/-- `add = λm. λn. iter m n (λk. succ k)` -/
def add : Term [] (nat ⇒ nat ⇒ nat) :=
  .lam (.lam (.iter (.var (.there .here)) (.var .here) (.lam (.succ (.var .here)))))

/-- `swap = λp. (snd p, fst p)` -/
def swap {σ τ : Ty} : Term [] (.prod σ τ ⇒ .prod τ σ) :=
  .lam (.pair (.snd (.var .here)) (.fst (.var .here)))

/-- `toNat = λs. case s of inl n => n | inr b => if b then 1 else 0` -/
def toNat : Term [] (.sum nat bool ⇒ nat) :=
  .lam (.case (.var .here) (.var .here) (.ite (.var .here) (lit 1) (lit 0)))
```

An ill-typed term is not a term with a type error; it is a Lean type error. Applying the number zero to zero:

```
/--
error: Application type mismatch: The argument
  Term.zero
has type
  Term ?m.5 nat
but is expected to have type
  Term [] (nat ⇒ nat)
in the application
  Term.zero.app
-/
#guard_msgs in
example : Term [] .nat := .app .zero .zero
```

Lean's own type checker reports that `.zero` has type `Term _ nat`, where a function type `Term [] (nat ⇒ nat)` was needed. Self-application fails the same way: the variable `here` has type `σ` in the context `[σ]`, and the application needs it to have type `σ ⇒ τ`:

```
/--
error: Application type mismatch: The argument
  Var.here
has type
  Var (?m.11 :: ?m.10) ?m.11
but is expected to have type
  Var [σ] (σ ⇒ τ)
in the application
  Term.var Var.here
-/
#guard_msgs in
example (σ τ : Ty) : Term [] (σ ⇒ τ) := .lam (.app (.var .here) (.var .here))
```

An evaluator that cannot go wrong

An environment for `Γ` supplies a value of the right Lean type for each variable, and the evaluator maps a term of type `τ` to a Lean value of type `τ.denote`:

```
/-- An environment for `Γ`: a value of the right type for every variable. -/
abbrev Env (Γ : List Ty) : Type := ∀ {τ : Ty}, Var Γ τ → τ.denote
```

```
/-- The empty environment. There are no variables to look up. -/
def Env.nil : Env [] := fun x => nomatch x

/-- Extend an environment with a value for the new variable `here`. -/
def Env.cons {Γ : List Ty} {σ : Ty} (v : σ.denote) (ρ : Env Γ) : Env (σ :: Γ)
  | _, .here => v
  | _, .there x => ρ x
```

```
/-- `iterate f n a` applies `f` to `a`, `n` times. -/
def iterate {α : Type} (f : α → α) : Nat → α → α
  | 0, a => a
  | n + 1, a => f (iterate f n a)
```

```
/-- The evaluator: every term of type `τ` denotes a Lean value of type
`τ.denote`. There is no error case and no fuel: Lean checks that every case
is covered and that the recursion terminates. -/
def Term.eval {Γ : List Ty} {τ : Ty} : Term Γ τ → Env Γ → τ.denote
  | .var x, ρ => ρ x
  | .lam b, ρ => fun v => b.eval (ρ.cons v)
  | .app f a, ρ => f.eval ρ (a.eval ρ)
  | .unit, _ => ()
  | .tt, _ => true
  | .ff, _ => false
  | .ite c a b, ρ => if c.eval ρ then a.eval ρ else b.eval ρ
  | .zero, _ => 0
  | .succ n, ρ => n.eval ρ + 1
  | .iter n z s, ρ => iterate (s.eval ρ) (n.eval ρ) (z.eval ρ)
  | .pair a b, ρ => (a.eval ρ, b.eval ρ)
  | .fst p, ρ => (p.eval ρ).1
  | .snd p, ρ => (p.eval ρ).2
  | .inl a, ρ => .inl (a.eval ρ)
  | .inr b, ρ => .inr (b.eval ρ)
  | .case s l r, ρ =>
    match s.eval ρ with
    | .inl v => l.eval (ρ.cons v)
    | .inr v => r.eval (ρ.cons v)
```

Compare this with the Rust evaluator. There is no `Result`, no `Stuck`, no fuel, and no case for "applied something that is not a function". In the `app` case, `f.eval ρ` has type `(σ ⇒ τ).denote`, which *is* the Lean function type `σ.denote → τ.denote`, so applying it to `a.eval ρ` is just Lean function application. In the `ite` case, `c.eval ρ` is a `Bool`. The types guarantee that every case has what it needs, and Lean has checked that the match covers every constructor and that the recursion is structural. The evaluator is **total and safe by construction**: progress and preservation, which Chapter 9 must prove, are built into the types here. A `lam` evaluates to a genuine Lean function (`fun v => ...`), which we can apply to Lean functions directly:

```
/-- info: 10 -/
#guard_msgs in
#eval (Term.app (.app twice (.app add (lit 3))) (lit 4)).eval Env.nil

/-- The meaning of `twice` is a Lean function, which we can apply to Lean
functions. -/
example : twice.eval Env.nil (· * 3) 5 = 45 := rfl

/-- info: (true, 7) -/
#guard_msgs in
#eval (Term.app (swap (σ := nat) (τ := bool)) (.pair (lit 7) .tt)).eval Env.nil

/-- info: 1 -/
#guard_msgs in
#eval (Term.app toNat (.inr .tt)).eval Env.nil
```

This kind of evaluator, which maps the syntax of a language to mathematical objects (here, Lean values), is called a **denotational semantics**; Chapter 19 develops it into the interpretation of the STLC in a cartesian closed category.

β and η, proved

With the evaluator we can state β and η as theorems about meanings. β holds by definition, for functions and for the other types:

```
/-- β holds in the semantics, by computation: applying a λ means evaluating its
body with the argument's value bound to the parameter. -/
theorem eval_beta {Γ : List Ty} {σ τ : Ty} (b : Term (σ :: Γ) τ) (a : Term Γ σ)
    (ρ : Env Γ) : (Term.app (.lam b) a).eval ρ = b.eval (ρ.cons (a.eval ρ)) :=
  rfl

/-- So do the β-rules of the other types. -/
example {Γ : List Ty} {σ τ : Ty} (a : Term Γ σ) (b : Term Γ τ) (ρ : Env Γ) :
    (Term.fst (.pair a b)).eval ρ = a.eval ρ := rfl

example {Γ : List Ty} {τ : Ty} (z : Term Γ τ) (s : Term Γ (τ ⇒ τ)) (ρ : Env Γ) :
    (Term.iter .zero z s).eval ρ = z.eval ρ := rfl
```

`rfl` works because the left-hand side computes to the right-hand side by the equations of `eval`. η is more interesting, because it mentions the term `f.weaken`: the function `f`, moved into a context with one more variable (the parameter `x`) that it does not use. Weakening is an instance of **renaming**, which applies a type-preserving map on variables to every variable of a term:

```
/-- A renaming sends each variable of `Γ` to a variable of `Δ` with the same
type. -/
abbrev Ren (Γ Δ : List Ty) : Type := ∀ {τ : Ty}, Var Γ τ → Var Δ τ

/-- Going under a binder: the new variable stays `here`, the others move as
before, one position further out. -/
def Ren.ext {Γ Δ : List Ty} {σ : Ty} (r : Ren Γ Δ) : Ren (σ :: Γ) (σ :: Δ)
  | _, .here => .here
  | _, .there x => .there (r x)

/-- Apply a renaming to every variable of a term. -/
def Term.rename {Γ Δ : List Ty} (r : Ren Γ Δ) {τ : Ty} : Term Γ τ → Term Δ τ
  | .var x => .var (r x)
  | .lam b => .lam (b.rename r.ext)
  | .app f a => .app (f.rename r) (a.rename r)
  | .unit => .unit
  | .tt => .tt
  | .ff => .ff
  | .ite c a b => .ite (c.rename r) (a.rename r) (b.rename r)
  | .zero => .zero
  | .succ n => .succ (n.rename r)
  | .iter n z s => .iter (n.rename r) (z.rename r) (s.rename r)
  | .pair a b => .pair (a.rename r) (b.rename r)
  | .fst p => .fst (p.rename r)
  | .snd p => .snd (p.rename r)
  | .inl a => .inl (a.rename r)
  | .inr b => .inr (b.rename r)
  | .case e l rr => .case (e.rename r) (l.rename r.ext) (rr.rename r.ext)

/-- Weakening: a term of `Γ` is also a term of `σ :: Γ`. -/
def Term.weaken {Γ : List Ty} {σ τ : Ty} (t : Term Γ τ) : Term (σ :: Γ) τ :=
  t.rename Var.there
```

The key lemma says that evaluating a renamed term is evaluating the original term in an environment that "looks through" the renaming:

```
/-- Evaluating a renamed term in `ρ` is evaluating the original term in an
environment `ρ'` that agrees with `ρ` through the renaming. -/
theorem eval_rename {Γ Δ : List Ty} {τ : Ty} (t : Term Γ τ) (r : Ren Γ Δ)
    (ρ : Env Δ) (ρ' : Env Γ) (h : ∀ {σ : Ty} (x : Var Γ σ), ρ' x = ρ (r x)) :
    (t.rename r).eval ρ = t.eval ρ' := by
  -- Under a binder, the extended environments still agree.
  have ext {σ : Ty} (v : σ.denote) {Γ Δ : List Ty} (r : Ren Γ Δ) (ρ : Env Δ)
      (ρ' : Env Γ) (h : ∀ {σ : Ty} (x : Var Γ σ), ρ' x = ρ (r x)) :
      ∀ {σ' : Ty} (x : Var (σ :: Γ) σ'), (ρ'.cons v) x = (ρ.cons v) (r.ext x) := by
    intro σ' x
    cases x with
    | here => rfl
    | there x => exact h x
  induction t generalizing Δ with
  | var x => exact (h x).symm
  | lam b ih =>
    funext v
    exact ih r.ext (ρ.cons v) (ρ'.cons v) (ext v r ρ ρ' h)
  | case e l rr ihe ihl ihr =>
    simp only [Term.rename, Term.eval, ihe r ρ ρ' h]
    split
    · exact ihl r.ext _ _ (ext _ r ρ ρ' h)
    · exact ihr r.ext _ _ (ext _ r ρ ρ' h)
  -- The remaining cases: rename the subterms, then use the induction
  -- hypotheses (`ih r ρ ρ' h` rewrites a renamed subterm's value).
  | app f a ihf iha => simp only [Term.rename, Term.eval, ihf r ρ ρ' h, iha r ρ ρ' h]
  | ite c a b ihc iha ihb =>
    simp only [Term.rename, Term.eval, ihc r ρ ρ' h, iha r ρ ρ' h, ihb r ρ ρ' h]
  | succ n ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | iter n z s ihn ihz ihs =>
    simp only [Term.rename, Term.eval, ihn r ρ ρ' h, ihz r ρ ρ' h, ihs r ρ ρ' h]
  | pair a b iha ihb => simp only [Term.rename, Term.eval, iha r ρ ρ' h, ihb r ρ ρ' h]
  | fst p ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | snd p ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | inl a ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | inr b ih => simp only [Term.rename, Term.eval, ih r ρ ρ' h]
  | unit | tt | ff | zero => rfl
```

The proof is by induction on the term. For a variable, the hypothesis `h` is exactly what is needed (`symm` flips the equation). For the binders, `lam` and `case`, the environments are extended by one value, and the local lemma `ext` shows, by cases on the variable, that the extended environments still agree through `r.ext`. For a `lam`, both sides are Lean functions, so `funext v` reduces the goal to their values at an arbitrary argument `v`. In the `case` case, `split` considers the two possible results of evaluating the scrutinee. The other cases unfold `rename` and `eval` (`simp only`) and rewrite each subterm with its induction hypothesis. η is now a short calculation:

```
/-- η holds in the semantics: `λx. f x` means the same as `f` (when `x` is
not free in `f`, which the weakening guarantees). -/
theorem eval_eta {Γ : List Ty} {σ τ : Ty} (f : Term Γ (σ ⇒ τ)) (ρ : Env Γ) :
    (Term.lam (.app f.weaken (.var .here))).eval ρ = f.eval ρ := by
  funext v
  -- After unfolding `eval`: (f.weaken).eval (ρ.cons v) v = f.eval ρ v.
  show (f.rename Var.there).eval (ρ.cons v) v = f.eval ρ v
  rw [eval_rename f Var.there (ρ.cons v) ρ (fun _ => rfl)]
```

After `funext v`, the goal is the value of both sides at `v`. `show` restates the left side as `eval` unfolds it: the renamed `f`, evaluated in the environment extended with `v`, applied to `v`. `eval_rename` with the renaming `Var.there` turns it into `f` evaluated in `ρ`, since looking up `there x` in `ρ.cons v` is looking up `x` in `ρ` (the `fun _ => rfl` proves that, for every variable, by computation).

Extrinsic versus intrinsic

The file `code/lean/Book/Ch08/Extrinsic.lean` builds the extrinsic version for comparison: raw terms, with the same constructors but no type indices, and a separate typing relation whose constructors are the same typing rules.

```
/-- The typing judgment `Γ ⊢ t : τ` as an inductive relation: one constructor
per typing rule. The λ rule does not say where `σ` comes from: any `σ` will
do, if the body can be typed with it (Curry style). -/
inductive HasType : List Ty → Raw → Ty → Prop where
  | var {Γ : List Ty} {k : Nat} {τ : Ty} :
      Γ[k]? = some τ → HasType Γ (.var k) τ
  | lam {Γ : List Ty} {t : Raw} {σ τ : Ty} :
      HasType (σ :: Γ) t τ → HasType Γ (.lam t) (σ ⇒ τ)
  | app {Γ : List Ty} {f a : Raw} {σ τ : Ty} :
      HasType Γ f (σ ⇒ τ) → HasType Γ a σ → HasType Γ (.app f a) τ
  | unit {Γ : List Ty} : HasType Γ .unit .unit
  | tt {Γ : List Ty} : HasType Γ .tt .bool
  | ff {Γ : List Ty} : HasType Γ .ff .bool
  | ite {Γ : List Ty} {c a b : Raw} {τ : Ty} :
      HasType Γ c .bool → HasType Γ a τ → HasType Γ b τ → HasType Γ (.ite c a b) τ
  | zero {Γ : List Ty} : HasType Γ .zero .nat
  | succ {Γ : List Ty} {n : Raw} : HasType Γ n .nat → HasType Γ (.succ n) .nat
  | iter {Γ : List Ty} {n z s : Raw} {τ : Ty} :
      HasType Γ n .nat → HasType Γ z τ → HasType Γ s (τ ⇒ τ) →
        HasType Γ (.iter n z s) τ
  | pair {Γ : List Ty} {a b : Raw} {σ τ : Ty} :
      HasType Γ a σ → HasType Γ b τ → HasType Γ (.pair a b) (.prod σ τ)
  | fst {Γ : List Ty} {p : Raw} {σ τ : Ty} :
      HasType Γ p (.prod σ τ) → HasType Γ (.fst p) σ
  | snd {Γ : List Ty} {p : Raw} {σ τ : Ty} :
      HasType Γ p (.prod σ τ) → HasType Γ (.snd p) τ
  | inl {Γ : List Ty} {a : Raw} {σ τ : Ty} :
      HasType Γ a σ → HasType Γ (.inl a) (.sum σ τ)
  | inr {Γ : List Ty} {b : Raw} {σ τ : Ty} :
      HasType Γ b τ → HasType Γ (.inr b) (.sum σ τ)
  | case {Γ : List Ty} {e l r : Raw} {σ τ ρ : Ty} :
      HasType Γ e (.sum σ τ) → HasType (σ :: Γ) l ρ → HasType (τ :: Γ) r ρ →
        HasType Γ (.case e l r) ρ
```

The two styles are connected by **erasure**: forgetting the types of an intrinsic term gives a raw term, and the theorem `erase_typed` says that the raw term is well typed, at the type the intrinsic term carried. The proof is an induction on the term in which every case is "apply the matching rule to the induction hypotheses", which makes the slogan precise: an intrinsic term *is* a typing derivation.

```
/-- A typed variable's index points at its type in the context. -/
theorem Var.index_lookup {Γ : List Ty} {τ : Ty} (x : Var Γ τ) : Γ[x.index]? = some τ := by
  induction x with
  | here => rfl
  | there x ih => simpa [Var.index] using ih

/-- Every intrinsically typed term, once its types are erased, is accepted by
the extrinsic typing relation, at the same type: the constructors of `Term`
*are* the typing rules. -/
theorem Term.erase_typed {Γ : List Ty} {τ : Ty} (t : Term Γ τ) :
    HasType Γ t.erase τ := by
  induction t with
  | var x => exact .var x.index_lookup
  | lam _ ih => exact .lam ih
  | app _ _ ihf iha => exact .app ihf iha
  | unit => exact .unit
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iha ihb => exact .ite ihc iha ihb
  | zero => exact .zero
  | succ _ ih => exact .succ ih
```

```
  | iter _ _ _ ihn ihz ihs => exact .iter ihn ihz ihs
  | pair _ _ iha ihb => exact .pair iha ihb
  | fst _ ih => exact .fst ih
  | snd _ ih => exact .snd ih
  | inl _ ih => exact .inl ih
  | inr _ ih => exact .inr ih
  | case _ _ _ ihe ihl ihr => exact .case ihe ihl ihr
```

The only case that needs a lemma is the variable: `Var.index_lookup` says that the de Bruijn index of a typed variable finds its type in the context. For `here` the index is 0 and the lookup returns the head of the list (`rfl`); for `there x` both the index and the context grow by one, and `simpa` (which simplifies the goal and the hypothesis and then closes one with the other) reduces the goal to the induction hypothesis.

```
/-- Erasing `twice` gives back the untyped `λ. λ. 1 (1 0)` of Chapter 7. -/
example : twice.erase = .lam (.lam (.app (.var 1) (.app (.var 1) (.var 0)))) := rfl

/-- So `HasType` accepts it, with the type that `twice` carries. -/
example : HasType [] twice.erase ((nat ⇒ nat) ⇒ nat ⇒ nat) := twice.erase_typed
```

Which style is better depends on what you want:

	Extrinsic (Chapters 3, 9)	**Intrinsic (this chapter)**
Ill-typed terms	exist; the checker rejects them	cannot be written
Typing	a separate relation `HasType`	the indices of `Term Γ τ`
Safety	a theorem to prove	true by construction
Evaluator	may get stuck (`Option`, `Stuck`)	total, no error cases
Good for	type checkers, studying errors	interpreters, semantics

Chapter 9 chooses the extrinsic style on purpose: to *prove* that well-typed programs do not go wrong, it needs to talk about programs that do, and to show that the typing rules keep evaluation away from them. The intrinsic style makes that theorem invisible, which is wonderful for building an interpreter and useless for studying why it works.

8.9 Putting it together: one function, three ways

Let us follow `twice (add 3) 4`, which should be 10, through all three implementations of this chapter.

In the `stlc` crate, the program is text. The parser builds a `Term`; the bidirectional checker synthesizes `Nat` for it, using C-Lam to type the unannotated `twice` against its annotation and S-App for each application; the evaluator builds closures and calls them:

```
twice (add 3) 4  :  Nat  ⇓  10
```

On the way, `add 3` became the closure `<closure λn. ... | m = 3>`, a body plus one captured value.

In Rust, `twice` is a generic function and `add` a closure returning a closure:

```
/// `twice = λf. λx. f (f x)`, for functions on `u64`.
fn twice(f: impl Fn(u64) -> u64) -> impl Fn(u64) -> u64 {
```

```
    move |x| f(f(x))
}
```

```
let add = |m: u64| move |n: u64| m + n;
let add3 = add(3);
println!("size of add(3) = {}", size_of_val(&add3));
println!("twice(add(3))(4) = {}", twice(add3)(4));
```

```
size of add(3) = 8
twice(add(3))(4) = 10
```

`add(3)` is a value of an anonymous closure type holding `m = 3`: 8 bytes, the same data as the crate's closure, laid out by the compiler instead of stored in a vector. `twice` is monomorphized for that type, so the two calls to `f` are direct.

In Lean, `twice` and `add` are intrinsically typed terms, and evaluation produces Lean values: the first `#eval` of the `run` block above printed 10. The types of the terms were checked by Lean's own type checker when we wrote them; the evaluator could not have failed.

Three implementations, one mathematical object: the term $\lambda f.\, \lambda x.\, f\,(f\,x)$ of type $(\texttt{Nat} \to \texttt{Nat}) \to \texttt{Nat} \to \texttt{Nat}$, typed by the derivation we wrote by hand at the start of the chapter.

8.10 Common confusions

"**A λ without an annotation has no type.**" It has many: $\lambda x.\, x$ has type $\tau \to \tau$ for every τ. What it lacks is a type that can be *synthesized* from the term alone. In checking mode, against an expected type, it is fine.

"**`move` makes a closure `FnOnce`.**" `move` only decides *how* variables are captured (by value). Which traits the closure implements depends on what its body does with the captures. A `move` closure that only reads is `Fn`.

"**`Fn` is the most general closure trait.**" It is the most *demanding* for the closure and the most *permissive* for the caller. A function that accepts `impl FnOnce` accepts every closure; one that accepts `impl Fn` accepts only those that can be called through a shared reference.

"**Closures are function pointers.**" A function pointer is a code address. A closure is a struct of captured values with a call method, and only a non-capturing closure can be coerced to a function pointer.

"**Two closures with the same code have the same type.**" Every closure expression has its own type. To mix them, use generics with one closure at a time, or a trait object.

"**Strong normalization means Rust programs terminate.**" It is a theorem about the STLC. Rust adds unrestricted recursion and `loop`, and its types promise nothing about termination.

"**Intrinsic typing makes the safety proof unnecessary.**" It moves the proof into the definition: the type of `Term.eval` *is* the safety statement, and Lean checked it when it accepted the definition. What you lose is the ability to talk about ill-typed terms at all.

8.11 Summary

- The STLC assigns types `Unit`, `Bool`, `Nat`, $\sigma \to \tau$, $\sigma \times \tau$ and $\sigma + \tau$ to λ-terms. Each type former has introduction and elimination rules; erased of terms, they are the rules of natural deduction.
- β-rules remove detours (an introduction followed by an elimination); η-rules say a value is determined by its uses. Clippy's `redundant_closure` is an η-reduction.
- Every well-typed STLC term terminates (strong normalization). $\lambda x.\, x\, x$ has no type, because $\sigma = \sigma \to \rho$ has no solution, a fact proved in Lean. Rust gives up termination to be Turing complete.
- Bidirectional checking splits typing into synthesis ($\Rightarrow$) and checking ($\Leftarrow$): introduction forms check, elimination forms synthesize, and a λ needs an expected type or an annotation. Rust's closures get their parameter types the same way.
- The `stlc` crate implements a bidirectional checker and an environment-based evaluator whose closures capture the free variables of their body, as Rust's do.
- Rust has several function types: zero-sized `fn` items, `fn` pointers, closures with unique anonymous types sized by their captures, generic `impl Fn`, and trait objects `dyn Fn`.
- Closures capture by shared reference, mutable reference or value; `move` forces by value. `Fn`, `FnMut` and `FnOnce` say how a closure may be called, and `FnOnce` is an affine function: usable at most once.
- Currying is free in the λ-calculus and costs allocations, clones and bounds in Rust.
- In Lean, intrinsically typed terms `Term Γ τ` are typing derivations; their denotational evaluator is total and safe by construction, and β and η hold as theorems. The extrinsic style, used in Chapter 9, keeps ill-typed terms around so that safety can be stated and proved.

8.12 Exercises

1. Write typing derivations for $\vdash \lambda p : \sigma \times \tau.\, (\mathtt{snd}\ p, \mathtt{fst}\ p) : \sigma \times \tau \to \tau \times \sigma$ and for $\vdash \lambda x : \sigma.\, \lambda y : \tau.\, x : \sigma \to \tau \to \sigma$. For each, say whether some variable is used twice, or not at all. *Hint:* in the second derivation y goes into the context and is never looked up; that is weakening (Chapter 6).
2. Extend the `stlc` crate with a primitive `isZero e`: give its rule in both modes, add it to the parser, checker and evaluator, and define `iszero` without `iter` in the example. Which mode should it be in? *Hint:* it is an elimination form for `Nat` with a known result type, so it synthesizes `Bool` after checking `e` against `Nat`.
3. Give a bidirectional rule for `case` in checking mode and one in synthesis mode, and explain why the synthesis rule must *check* the second branch against the type synthesized from the first. Find both in `check.rs`. *Hint:* compare with rule T-If in Chapter 1, and with how Rust reports the type of a `match` whose arms disagree.
4. Write a Rust function `compose3` that composes three closures, first with `impl Fn` arguments and result, then with `Box<dyn Fn>`. Use `size_of_val` to compare the sizes of the results, and explain the difference. *Hint:* the `impl Fn` result is a closure holding three closures; the boxed version is a fat pointer, whatever it holds.
5. Which of `Fn`, `FnMut`, `FnOnce` does each closure implement? `|v: &mut Vec<u32>| v.push(1)`, `move || drop(buffer)`, `|| counter.get()` (with `counter: Cell<u32>`), `move |x| x + offset`. Check

your answers by passing each to functions that require each trait. *Hint:* the parameter `v` is not a capture; `Cell::get` takes `&self`.

6. In Lean, add a constructor `isZero : Term Γ .nat → Term Γ .bool` to `Intrinsic.lean` (in a copy), extend `eval`, `rename` and the proof of `eval_rename`, and prove `(Term.app (.lam (.isZero (.var .here))) (lit 0)).eval Env.nil = true` by `rfl`. *Hint:* `eval` maps it to `n.eval ρ == 0`; the `eval_rename` case is like `succ`.

CHAPTER 9

Type Safety, Proved

In Chapter 1 we quoted Robin Milner's slogan, *well-typed programs cannot go wrong*, and split it into two halves: **progress** (a well-typed program is never stuck) and **preservation** (a step of computation does not change the type). Chapter 3 stated both for the small language `Arith`, and Chapter 8 built the simply typed λ-calculus. All along, the theorem has been a promise. In this chapter we keep it.

We take the simply typed λ-calculus with booleans and natural numbers, define how its programs run, one small step at a time, and prove in Lean that no well-typed program ever reaches a stuck state, no matter how long it runs; we also prove that evaluation is deterministic. The proof is the classic one, due in this form to Andrew Wright and Matthias Felleisen (1994) and presented in almost every textbook on programming languages since (Pierce's *Types and Programming Languages* is the standard reference). It is built from half a dozen lemmas, and we will go through every one of them, line by line. If you have never read a proof about a programming language, this chapter shows you how one is put together and where the real difficulty hides.

Then we look at the same theorems from the other side: a Rust crate generates thousands of random well-typed programs and runs them, checking progress and preservation at every step. Putting tests and proofs side by side is the best way to understand what each one gives you. Finally we ask what "going wrong" means for Rust itself.

In this chapter you will learn:

- how to define a small-step operational semantics as an inductive relation, and what values and stuck terms are;
- the canonical forms lemmas, and how they make the progress theorem work;
- why preservation needs weakening and substitution lemmas, and how de Bruijn indices make those lemmas precise enough for a machine to check;
- how progress and preservation combine into type safety for runs of any length, and why evaluation is deterministic;
- how to test the same theorems with a property-based test harness in Rust, and what such tests can and cannot tell you;
- what "going wrong" means for Rust, and what its type system does not promise.

9.1 The language

The language of this chapter is the simply typed λ-calculus of Chapter 8, cut down to what the proofs need: functions, booleans and natural numbers.

$$\tau ::= \mathit{bool} \mid \mathit{nat} \mid \tau_1 \to \tau_2$$

$$t ::= \#i \mid \lambda : \tau . t \mid t_1\, t_2 \mid \mathit{true} \mid \mathit{false} \mid \mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \mid n \mid t_1 + t_2 \mid \mathit{iszero}\ t$$

Variables are **de Bruijn indices**, as in Chapter 7: instead of a name, a variable is the number of binders between it and the λ that binds it. We write them $\#0$, $\#1$, $\#2$, with a `#` so that they cannot be confused with the numeric literals $0, 1, 2$ of the language. The function $\lambda(x : \mathit{nat}).\, x + 1$ becomes $\lambda : \mathit{nat}.\, \#0 + 1$, and the function that applies its first argument twice to its second, $\lambda(f : \mathit{nat} \to \mathit{nat}).\, \lambda(x : \mathit{nat}).\, f\,(f\,x)$, becomes $\lambda : (\mathit{nat} \to \mathit{nat}).\, \lambda : \mathit{nat}.\, \#1\,(\#1\,\#0)$: inside the inner λ, x is one binder away ($\#0$) and f is two ($\#1$).

Why de Bruijn indices, when names are so much easier to read? Because the proof of preservation hinges on a lemma about substitution, and with names, substitution must avoid capturing variables, terms must be identified up to renaming of bound variables (α-equivalence), and every lemma acquires side conditions about fresh names. On paper these are usually waved away. In a proof assistant nothing can be waved away, and de Bruijn indices make all of it disappear: there are no names to clash, and two α-equivalent terms are literally equal. The price is that indices must be adjusted when a term moves under a binder, and we will pay that price, carefully, when we get to substitution.

The λ carries a type annotation, $\lambda : \tau . t$: the type of the parameter is written down, not inferred. Chapter 8's bidirectional checker could sometimes do without it; here every λ has one, so that type checking is a simple recursive function and every term has at most one type.

In Lean

The language lives in `code/lean/Book/Ch09/Syntax.lean`. Types and terms are two inductive types:

```
/-- The types of the language: booleans, natural numbers and functions. -/
inductive Ty where
  | bool : Ty
  | nat : Ty
  | arrow : Ty → Ty → Ty
  deriving DecidableEq
```

```
/-- Terms in de Bruijn notation: `var i` refers to the binder `i` levels up.
`lam τ t` is `λ (x : τ). t`, with the bound variable written `var 0` in `t`.
`tt` and `ff` are the boolean literals (`true` and `false` are Lean's own). -/
inductive Term where
  | var : Nat → Term
  | lam : Ty → Term → Term
  | app : Term → Term → Term
  | tt : Term
  | ff : Term
  | ite : Term → Term → Term → Term
  | lit : Nat → Term
  | add : Term → Term → Term
  | isZero : Term → Term
  deriving DecidableEq
```

The constructor `ite` is "if-then-else" (the name Lean itself uses for `if`, which is a keyword), and the boolean literals are `tt` and `ff`, because `true` and `false` already name Lean's own booleans. The file also defines a printer, so that `#eval` shows terms in the notation of this chapter:

```
/-- `(λ (x : nat). x + 1) 41` -/
def incr41 : Term := .app (.lam .nat (.add (.var 0) (.lit 1))) (.lit 41)

/-- `λ (f : nat → nat). λ (x : nat). f (f x)`: apply a function twice. The inner
`#1` is `f`, one binder further out than `x`, which is `#0`. -/
def twice : Term :=
  .lam (.arrow .nat .nat) (.lam .nat (.app (.var 1) (.app (.var 1) (.var 0))))

/-- info: (λ:nat. #0 + 1) 41 -/
#guard_msgs in
#eval incr41

/-- info: λ:(nat → nat). λ:nat. #1 (#1 #0) -/
#guard_msgs in
#eval twice
```

There is one important difference from Chapter 8. There, terms were **intrinsically** typed: the type of a term was an index of its Lean type (`Term Γ τ`), so an ill-typed term could not even be written down, and the evaluator was safe by construction. Here terms are **extrinsically** typed: `Term` is plain syntax, ill-typed terms such as `true + 1` are perfectly good values of type `Term`, and typing is a separate relation that some terms satisfy and others do not. The intrinsic approach gives safety for free; the extrinsic approach makes safety a *theorem*, something we can state, prove and break. To talk about programs that go wrong, we need to be able to write them.

In Rust

The Rust crate `safety` (`code/rust/crates/safety`) mirrors the Lean files module by module. Its types and terms are the same two definitions, as Rust enums:

```
/// The types: booleans, natural numbers and functions.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    Bool,
    Nat,
    Arrow(Box<Ty>, Box<Ty>),
}
```

```
/// Terms in de Bruijn notation. `Var(i)` refers to the binder `i` levels up;
/// `Lam(τ, t)` is `λ (x : τ). t`, with the bound variable written `Var(0)` in `t`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(usize),
    Lam(Ty, Box<Term>),
    App(Box<Term>, Box<Term>),
    True,
    False,
    If(Box<Term>, Box<Term>, Box<Term>),
    Lit(u64),
    Add(Box<Term>, Box<Term>),
    IsZero(Box<Term>),
}
```

The Rust printer produces exactly the same notation as the Lean one, so the outputs of the two languages can be compared character by character.

9.2 The typing rules

A context Γ is a list of types: entry i is the type of the variable $\#i$. The variable bound by the innermost λ is $\#0$, so when the type checker enters a $\lambda : \sigma$, it puts σ at the *front* of the list, and every variable that was already in scope moves one position back, exactly as its index does. We write $\sigma :: \Gamma$ for that extended context and $\Gamma(i)$ for entry i. The rules are those of Chapter 8, restricted to our types:

$$\frac{\Gamma(i) = \tau}{\Gamma \vdash \#i : \tau}\ \text{T-Var} \qquad \frac{\sigma :: \Gamma \vdash t : \tau}{\Gamma \vdash \lambda : \sigma.\, t : \sigma \to \tau}\ \text{T-Lam}$$

$$\frac{\Gamma \vdash t_1 : \sigma \to \tau \qquad \Gamma \vdash t_2 : \sigma}{\Gamma \vdash t_1\ t_2 : \tau}\ \text{T-App}$$

$$\frac{}{\Gamma \vdash \mathit{true} : \mathit{bool}}\ \text{T-True} \qquad \frac{}{\Gamma \vdash \mathit{false} : \mathit{bool}}\ \text{T-False} \qquad \frac{}{\Gamma \vdash n : \mathit{nat}}\ \text{T-Lit}$$

$$\frac{\Gamma \vdash t_1 : \mathit{bool} \qquad \Gamma \vdash t_2 : \tau \qquad \Gamma \vdash t_3 : \tau}{\Gamma \vdash \mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash t_1 : \mathit{nat} \qquad \Gamma \vdash t_2 : \mathit{nat}}{\Gamma \vdash t_1 + t_2 : \mathit{nat}}\ \text{T-Add} \qquad \frac{\Gamma \vdash t : \mathit{nat}}{\Gamma \vdash \mathit{iszero}\ t : \mathit{bool}}\ \text{T-IsZero}$$

Rule T-If is the rule from Chapter 1 that both branches share one type; it will come back later in the chapter, when we break it on purpose.

In Lean, contexts are lists, `lookup` is $\Gamma(i)$, and the judgment is an inductive predicate with one constructor per rule, the technique of Chapter 3 (file `code/lean/Book/Ch09/Typing.lean`):

```
/-- A context lists the types of the free variables: entry `i` is the type of
`var i`. The most recently bound variable comes first. -/
abbrev Ctx := List Ty

/-- The type of variable `i` in `Γ`, if `Γ` has an entry `i`. -/
def lookup : Ctx → Nat → Option Ty
  | [], _ => none
  | σ :: _, 0 => some σ
  | _ :: Γ, i + 1 => lookup Γ i
```

```
/-- `HasType Γ t τ`: in context `Γ`, the term `t` has type `τ`. -/
inductive HasType : Ctx → Term → Ty → Prop
  | var {Γ : Ctx} {i : Nat} {τ : Ty} :
      lookup Γ i = some τ → HasType Γ (.var i) τ
  | lam {Γ : Ctx} {σ τ : Ty} {t : Term} :
      HasType (σ :: Γ) t τ → HasType Γ (.lam σ t) (.arrow σ τ)
  | app {Γ : Ctx} {σ τ : Ty} {t₁ t₂ : Term} :
      HasType Γ t₁ (.arrow σ τ) → HasType Γ t₂ σ → HasType Γ (.app t₁ t₂) τ
  | tt {Γ : Ctx} : HasType Γ .tt .bool
  | ff {Γ : Ctx} : HasType Γ .ff .bool
  | ite {Γ : Ctx} {τ : Ty} {c t e : Term} :
      HasType Γ c .bool → HasType Γ t τ → HasType Γ e τ →
      HasType Γ (.ite c t e) τ
  | lit {Γ : Ctx} {n : Nat} : HasType Γ (.lit n) .nat
  | add {Γ : Ctx} {t₁ t₂ : Term} :
      HasType Γ t₁ .nat → HasType Γ t₂ .nat → HasType Γ (.add t₁ t₂) .nat
  | isZero {Γ : Ctx} {t : Term} :
      HasType Γ t .nat → HasType Γ (.isZero t) .bool
```

Compare each constructor with its rule: the hypotheses before the last arrow are the premises, the type after it is the conclusion, and the variables in curly braces are the metavariables of the rule, which Lean will fill in by itself whenever it can. (We wrote our own `lookup` rather than using Lean's

list indexing `Γ[i]?`. The two compute the same thing, but `lookup` is three lines that you can read, and it keeps the whole development free of axioms, a point we will come back to at the end.)

A derivation is a value of this inductive type. Here is the typing derivation for `twice`, built by hand:

```
/-- A derivation, built by hand: `twice` has type `(nat → nat) → nat → nat`.
Read the proof term from the outside in: two `lam` rules, then the outer
application, whose function is `#1` and whose argument is another application. -/
example : HasType [] twice (.arrow (.arrow .nat .nat) (.arrow .nat .nat)) :=
  .lam (.lam
    (.app (.var rfl)                          -- #1 : nat → nat
      (.app (.var rfl) (.var rfl))))          -- #1 #0 : nat
```

Read it as the derivation tree written sideways. The outermost `.lam` is T-Lam for $\lambda : (\mathit{nat} \to \mathit{nat})$, the next one for $\lambda : \mathit{nat}$, and below them the body $\#1\ (\#1\ \#0)$ is an application (T-App) whose function is the variable $\#1$ and whose argument is another application. Each `.var rfl` is T-Var: its premise is an equation such as `lookup [nat, nat → nat] 1 = some (nat → nat)`, and `rfl` proves it because both sides compute to the same value. Lean worked out every type in the tree from the conclusion we asked for.

A checker, proved sound

Building derivations by hand gets tedious quickly. As in Chapter 3, a type checker does it for us. The checker `typeOf` has one case per rule:

```
/-- The type checker: `some τ` if the term has type `τ`, `none` if it has no
type. -/
def typeOf (Γ : Ctx) : Term → Option Ty
  | .var i => lookup Γ i
  | .lam σ t =>
    match typeOf (σ :: Γ) t with
    | some τ => some (.arrow σ τ)
    | none => none
  | .app t₁ t₂ =>
    match typeOf Γ t₁, typeOf Γ t₂ with
    | some (.arrow σ τ), some σ' => if σ = σ' then some τ else none
    | _, _ => none
  | .tt => some .bool
  | .ff => some .bool
  | .ite c t e =>
    match typeOf Γ c, typeOf Γ t, typeOf Γ e with
    | some .bool, some τ, some τ' => if τ = τ' then some τ else none
    | _, _, _ => none
  | .lit _ => some .nat
  | .add t₁ t₂ =>
    match typeOf Γ t₁, typeOf Γ t₂ with
    | some .nat, some .nat => some .nat
    | _, _ => none
  | .isZero t =>
    match typeOf Γ t with
    | some .nat => some .bool
    | _ => none
```

```
/-- info: some ((nat → nat) → nat → nat) -/
#guard_msgs in
#eval typeOf [] twice

/-- info: none -/
#guard_msgs in
#eval typeOf [] (.add .tt (.lit 1))
```

and it is proved sound with respect to the rules: whenever it says `some τ`, a derivation of type `τ` exists. The proof goes by induction on the term, with the same technique as Chapter 3; here are the first cases:

```
/-- The checker is sound: when it answers `some τ`, a derivation exists. -/
theorem typeOf_sound {Γ : Ctx} {t : Term} {τ : Ty} :
    typeOf Γ t = some τ → HasType Γ t τ := by
  induction t generalizing Γ τ with
  | var i => intro h; exact .var h
  | lam σ t ih =>
    intro h
    simp only [typeOf] at h
    split at h
    · next τ' h' => cases h; exact .lam (ih h')
    · cases h
  | app t₁ t₂ ih₁ ih₂ =>
    intro h
    simp only [typeOf] at h
    split at h
    · next σ τ' σ' h₁ h₂ =>
      split at h
      · next hσ => cases h; subst hσ; exact .app (ih₁ h₁) (ih₂ h₂)
      · cases h
    · cases h
```

In each case with subterms, `simp only [typeOf] at h` unfolds one step of the checker in the hypothesis `h : typeOf Γ t = some τ`, and `split at h` splits the proof into one branch per arm of the checker's `match`; `next ... =>` names the hypotheses the branch provides. Where the checker returns `none`, `h` has become `none = some τ`, which `cases h` dismisses (no constructor of `=` can prove it). In the successful branch, `cases h` learns what `τ` is, and the derivation is assembled from the induction hypotheses. The remaining cases, in the file, are the same. With a sound checker, the typing derivation of a concrete program is just a computation:

```
/-- With a sound checker, a derivation for a concrete program is a computation:
`rfl` checks that the checker says `some nat`. -/
theorem incr41_typed : HasType [] incr41 .nat := typeOf_sound rfl
```

The Rust checker is the same function, returning a `Result` with an error message instead of an `Option`, and with a switch, `rules`, that we will use later to break rule T-If:

```
/// The type of `t` in context `ctx`, under the given rules.
pub fn type_of_with(rules: Rules, ctx: &[Ty], t: &Term) -> Result<Ty, TypeError> {
    let expect = |place, u: &Term, expected: Ty| -> Result<(), TypeError> {
        let found = type_of_with(rules, ctx, u)?;
        if found == expected {
            Ok(())
        } else {
            Err(TypeError::Mismatch {
                place,
                expected,
                found,
            })
        }
    };
    match t {
        // T-Var: look the variable up in the context.
        Term::Var(i) => ctx.get(*i).cloned().ok_or(TypeError::UnboundVariable(*i)),
        // T-Lam: check the body in the context extended with the parameter.
        Term::Lam(param, body) => {
            let mut inner = vec![param.clone()];
            inner.extend_from_slice(ctx);
```

```
            let result = type_of_with(rules, &inner, body)?;
            Ok(Ty::arrow(param.clone(), result))
        }
        // T-App: a function of type σ → τ applied to an argument of type σ.
        Term::App(f, a) => match type_of_with(rules, ctx, f)? {
            Ty::Arrow(param, result) => {
                expect("the argument", a, *param)?;
                Ok(*result)
            }
            other => Err(TypeError::NotAFunction(other)),
        },
        // T-True, T-False, T-Lit.
        Term::True | Term::False => Ok(Ty::Bool),
        Term::Lit(_) => Ok(Ty::Nat),
        // T-If: a boolean condition and two branches of the same type.
        Term::If(c, then_branch, else_branch) => {
            expect("the condition of `if`", c, Ty::Bool)?;
            let ty_then = type_of_with(rules, ctx, then_branch)?;
            let ty_else = type_of_with(rules, ctx, else_branch)?;
            match rules {
                Rules::Sound if ty_then != ty_else => {
                    Err(TypeError::BranchesDiffer(ty_then, ty_else))
                }
                // The broken rule never compares the two branches.
                _ => Ok(ty_then),
            }
        }
        // T-Add and T-IsZero.
        Term::Add(a, b) => {
            expect("the left operand of `+`", a, Ty::Nat)?;
            expect("the right operand of `+`", b, Ty::Nat)?;
            Ok(Ty::Nat)
        }
        Term::IsZero(n) => {
            expect("the argument of `iszero`", n, Ty::Nat)?;
            Ok(Ty::Bool)
        }
    }
}
```

9.3 Running programs: small-step semantics

A type system is a claim about how programs behave, so before we can prove anything we must say precisely how programs run. We do it with a **small-step operational semantics**: a relation $t \to t'$ that says "the term t takes one step of computation and becomes t'". A whole run is a chain of such steps.

Values

Some terms are finished: they are the possible results of a computation. These are the **values**:

$$v ::= \lambda : \tau . t \mid \mathit{true} \mid \mathit{false} \mid n$$

A λ-abstraction is a value even if its body could compute further: we never evaluate under a binder, just as Rust does not run the body of a closure until the closure is called. In Lean, values are an inductive predicate on terms (file `code/lean/Book/Ch09/Semantics.lean`):

```
/-- Values: the terms that are finished computing. -/
inductive Value : Term → Prop
  | lam {τ : Ty} {t : Term} : Value (.lam τ t)
  | tt : Value .tt
  | ff : Value .ff
  | lit {n : Nat} : Value (.lit n)
```

The evaluation rules

The rules come in two kinds. **Computation rules** do the actual work: they apply when a term is a redex, an elimination form applied to a value of the right shape.

$$\frac{v \text{ value}}{(\lambda : \tau.\, t)\, v \to t[v]}\ \text{E-Beta}$$

$$\frac{}{\mathit{if}\ \mathit{true}\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to t_2}\ \text{E-IfTrue} \qquad \frac{}{\mathit{if}\ \mathit{false}\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to t_3}\ \text{E-IfFalse}$$

$$\frac{}{m + n \to m + n}\ \text{E-AddLit} \qquad \frac{}{\mathit{iszero}\ 0 \to \mathit{true}}\ \text{E-IsZeroZero} \qquad \frac{}{\mathit{iszero}\ (n+1) \to \mathit{false}}\ \text{E-IsZeroSucc}$$

In E-AddLit, m and n are literals and $m + n$ on the right is the literal for their sum: the step turns the *expression* $2 + 3$ into the *number* 5. In E-Beta, $t[v]$ is the body t with its bound variable $\#0$ replaced by v; we will define it precisely in the section on substitution.

Congruence rules say where to look for the next redex when the term is not one yet:

$$\frac{t_1 \to t_1'}{t_1\ t_2 \to t_1'\ t_2}\ \text{E-AppL} \qquad \frac{v_1 \text{ value} \qquad t_2 \to t_2'}{v_1\ t_2 \to v_1\ t_2'}\ \text{E-AppR}$$

$$\frac{t_1 \to t_1'}{\mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to \mathit{if}\ t_1'\ \mathit{then}\ t_2\ \mathit{else}\ t_3}\ \text{E-If}$$

$$\frac{t_1 \to t_1'}{t_1 + t_2 \to t_1' + t_2}\ \text{E-AddL} \qquad \frac{v_1 \text{ value} \qquad t_2 \to t_2'}{v_1 + t_2 \to v_1 + t_2'}\ \text{E-AddR} \qquad \frac{t \to t'}{\mathit{iszero}\ t \to \mathit{iszero}\ t'}\ \text{E-IsZero}$$

Together these rules fix an **evaluation strategy**, call-by-value with left-to-right order, the strategy of Chapter 7 and of Rust. The premise "v_1 value" in E-AppR and E-AddR is what imposes the order: the right operand may only step once the left one is finished. The premise "v value" in E-Beta says that arguments are evaluated before the call. Rust behaves the same way: in `f(a(), b())`, the function expression is evaluated first, then `a()`, then `b()`, and only then is `f` called. Notice also what the rules do *not* say: no rule evaluates the branches of an `if` before the condition is known, and no rule looks inside a λ.

The rules transcribe into one inductive predicate, one constructor per rule:

```
/-- One step of call-by-value evaluation. The *congruence* rules (`appL`, `appR`,
`ite`, `addL`, `addR`, `isZero`) say where to look for work; the *computation*
rules (`beta`, `iteTrue`, `iteFalse`, `addLit`, `isZeroZero`, `isZeroSucc`) do
the work. -/
inductive Step : Term → Term → Prop
  | appL {t₁ t₁' t₂ : Term} :
      Step t₁ t₁' → Step (.app t₁ t₂) (.app t₁' t₂)
  | appR {v₁ t₂ t₂' : Term} :
      Value v₁ → Step t₂ t₂' → Step (.app v₁ t₂) (.app v₁ t₂')
  | beta {τ : Ty} {body v : Term} :
      Value v → Step (.app (.lam τ body) v) (instantiate body v)
  | ite {c c' t e : Term} :
      Step c c' → Step (.ite c t e) (.ite c' t e)
  | iteTrue {t e : Term} : Step (.ite .tt t e) t
  | iteFalse {t e : Term} : Step (.ite .ff t e) e
  | addL {t₁ t₁' t₂ : Term} :
      Step t₁ t₁' → Step (.add t₁ t₂) (.add t₁' t₂)
  | addR {v₁ t₂ t₂' : Term} :
      Value v₁ → Step t₂ t₂' → Step (.add v₁ t₂) (.add v₁ t₂')
  | addLit {m n : Nat} : Step (.add (.lit m) (.lit n)) (.lit (m + n))
  | isZero {t t' : Term} : Step t t' → Step (.isZero t) (.isZero t')
```

```
  | isZeroZero : Step (.isZero (.lit 0)) .tt
  | isZeroSucc {n : Nat} : Step (.isZero (.lit (n + 1))) .ff
```

Runs

A run is a sequence of steps. Its relation, written $t \xrightarrow{*} t'$, is the **reflexive-transitive closure** of $\to$: zero steps, or one step followed by a run.

```
/-- Zero or more steps: the reflexive-transitive closure of `Step`. -/
inductive Steps : Term → Term → Prop
  | refl {t : Term} : Steps t t
  | step {t t' t'' : Term} : Step t t' → Steps t' t'' → Steps t t''
```

```
/-- Two runs, one after the other, form a run. -/
theorem Steps.trans {t₁ t₂ t₃ : Term} (h₁₂ : Steps t₁ t₂) (h₂₃ : Steps t₂ t₃) :
    Steps t₁ t₃ := by
  induction h₁₂ with
  | refl => exact h₂₃
  | step hstep _ ih => exact .step hstep (ih h₂₃)
```

`Steps.trans` glues two runs together, by induction on the first run. If it is empty, the second run is already the answer. If it starts with a step `hstep`, the induction hypothesis `ih` glues the rest of it to the second run, and `.step hstep` puts the first step back in front.

Here is a complete run, written as a proof:

```
/-- `(λ:nat. #0 + 1) 41` takes two steps to reach `42`. Each step is a rule
instance; `.step` chains them and `.refl` ends the run. -/
example : Steps incr41 (.lit 42) :=
  .step (.beta .lit)            -- (λ:nat. #0 + 1) 41  →  41 + 1
    (.step .addLit              -- 41 + 1              →  42
      .refl)
```

The first step is E-Beta, whose premise is that the argument `41` is a value (`.lit` is the constructor of `Value` for literals). It produces `instantiate (#0 + 1) 41`, which Lean computes to `41 + 1`. The second step is E-AddLit, which produces the literal `lit (41 + 1)`. Lean computes `41 + 1` to `42`, so the run can end there with `.refl`: the statement asked for `.lit 42`, and that is where we are.

> **NOTE**
>
> Mathlib has the reflexive-transitive closure of any relation, `Relation.ReflTransGen`. It adds steps at the *end* of a run instead of at the front, but it is the same relation, as `steps_iff_reflTransGen` at the end of `Semantics.lean` proves. We defined our own because two constructors are easier to read than a general library, and because building it is the lesson.

Stuck terms

What happens to `true + 1`? It is not a value, and no rule applies: E-AddLit needs two literals, and the congruence rules need one of the operands to step, which `true` and `1` cannot. The term is **stuck**:

```
/-- A term is stuck when it is not a value and no rule applies to it. -/
def Stuck (t : Term) : Prop := ¬ Value t ∧ ∀ t', ¬ Step t t'
```

A stuck term is the formal model of a program that has gone wrong: it has not finished, but there is nothing it can do next. This is where an interpreter would crash, or where compiled code without checks would add a boolean to a number and continue with garbage. Proving that a particular term is stuck is a good first exercise in reasoning about an inductive relation:

```
/-- `true + 1` is stuck: it is not a value, and no rule has a conclusion of the
form `true + 1 → _`. -/
theorem stuck_example : Stuck (.add .tt (.lit 1)) := by
  constructor
  · -- Not a value: no constructor of `Value` builds an `add`.
    intro h
    cases h
  · -- No step: try every rule whose conclusion starts with `add`.
    intro t' h
    cases h with
    | addL h₁ => cases h₁    -- `true` would have to step: no rule for that
    | addR _ h₂ => cases h₂  -- `1` would have to step: no rule for that
```

`constructor` splits the conjunction into its two halves. For "not a value", we assume a proof `h` that `true + 1` is a value and run `cases h`: Lean examines every constructor of `Value`, finds that none of them produces an `add`, and closes the goal because there is no case left to consider. For "no step", we assume a step `h` and run `cases h`. This time two constructors could have produced a step from an `add`: `addL`, which needs a step from `true`, and `addR`, which needs a step from `1`. (`addLit` is ruled out because `true` is not a literal.) In each case, a second `cases` on the premise finds no rule that lets a literal step, and the goal is closed. This pattern, `cases` until no constructor fits, is how every "this cannot happen" argument in this chapter works.

The evaluator in Rust

The Rust crate has the same semantics in two forms, and the difference between them will matter for testing. The first, `successors`, is a transcription of the *relation*: it tries every rule independently, checks that rule's premises and returns every step that some rule allows.

```
/// Every step that the rules allow from `t`: one `if` per rule, each checking
/// exactly that rule's premises.
pub fn successors(t: &Term) -> Vec<Step> {
    let mut out = Vec::new();
    match t {
        Term::App(f, a) => {
            // E-AppL: f → f'
            for s in successors(f) {
                out.push(s.under(Rule::AppL, |f2| Term::App(Box::new(f2), boxed(a))));
            }
            // E-AppR: f is a value and a → a'
            if is_value(f) {
                for s in successors(a) {
                    out.push(
                        s.under(Rule::AppR, |a2| Term::App(boxed(f), Box::new(a2))),
                    );
                }
            }
            // E-Beta: f is a λ and a is a value
            if let (Term::Lam(_, body), true) = (&**f, is_value(a)) {
                out.push(Step::axiom(Rule::Beta, instantiate(body, a)));
            }
        }
        Term::If(c, then_branch, else_branch) => {
            // E-If: c → c'
            for s in successors(c) {
```

```
                out.push(s.under(Rule::If, |c2| {
                    Term::If(Box::new(c2), boxed(then_branch), boxed(else_branch))
                }));
            }
            // E-IfTrue and E-IfFalse
            match **c {
                Term::True => {
                    out.push(Step::axiom(Rule::IfTrue, (**then_branch).clone()))
                }
                Term::False => {
                    out.push(Step::axiom(Rule::IfFalse, (**else_branch).clone()))
                }
                _ => {}
            }
        }
        Term::Add(a, b) => {
            // E-AddL: a → a'
            for s in successors(a) {
                out.push(s.under(Rule::AddL, |a2| Term::Add(Box::new(a2), boxed(b))));
            }
            // E-AddR: a is a value and b → b'
            if is_value(a) {
                for s in successors(b) {
                    out.push(
                        s.under(Rule::AddR, |b2| Term::Add(boxed(a), Box::new(b2))),
                    );
                }
            }
            // E-AddLit: two literals
            if let (Term::Lit(m), Term::Lit(n)) = (&**a, &**b) {
                out.push(Step::axiom(Rule::AddLit, Term::Lit(m + n)));
            }
        }
        Term::IsZero(n) => {
            // E-IsZero: n → n'
            for s in successors(n) {
                out.push(s.under(Rule::IsZero, |n2| Term::IsZero(Box::new(n2))));
            }
            // E-IsZeroZero and E-IsZeroSucc
            match **n {
                Term::Lit(0) => out.push(Step::axiom(Rule::IsZeroZero, Term::True)),
                Term::Lit(_) => out.push(Step::axiom(Rule::IsZeroSucc, Term::False)),
                _ => {}
            }
        }
        // No rule has a variable or a value on its left-hand side.
        Term::Var(_) | Term::Lam(..) | Term::True | Term::False | Term::Lit(_) => {}
    }
    out
}
```

The second, `step`, is an *evaluator*: a function that returns the one step that call-by-value takes. It decides which rule to use by looking for the leftmost subterm that is not yet a value, which is the strategy written as code.

```
/// The evaluator: the step that call-by-value takes from `t`, or `None` if `t`
/// is a value or stuck. Each case first finds the leftmost subterm that is not
/// yet a value, which is what makes it a function.
pub fn step(t: &Term) -> Option<Step> {
    match t {
        Term::App(f, a) if !is_value(f) => {
            step(f).map(|s| s.under(Rule::AppL, |f2| Term::App(Box::new(f2), boxed(a))))
        }
        Term::App(f, a) if !is_value(a) => {
            step(a).map(|s| s.under(Rule::AppR, |a2| Term::App(boxed(f), Box::new(a2))))
        }
        Term::App(f, a) => match &**f {
            Term::Lam(_, body) => Some(Step::axiom(Rule::Beta, instantiate(body, a))),
```

```
            _ => None, // a value that is not a function, applied: stuck
        },
        Term::If(c, t1, t2) if !is_value(c) => step(c).map(|s| {
            s.under(Rule::If, |c2| Term::If(Box::new(c2), boxed(t1), boxed(t2)))
        }),
        Term::If(c, t1, t2) => match **c {
            Term::True => Some(Step::axiom(Rule::IfTrue, (**t1).clone())),
            Term::False => Some(Step::axiom(Rule::IfFalse, (**t2).clone())),
            _ => None, // a condition that is not a boolean: stuck
        },
        Term::Add(a, b) if !is_value(a) => {
            step(a).map(|s| s.under(Rule::AddL, |a2| Term::Add(Box::new(a2), boxed(b))))
        }
        Term::Add(a, b) if !is_value(b) => {
            step(b).map(|s| s.under(Rule::AddR, |b2| Term::Add(boxed(a), Box::new(b2))))
        }
        Term::Add(a, b) => match (&**a, &**b) {
            (Term::Lit(m), Term::Lit(n)) => {
                Some(Step::axiom(Rule::AddLit, Term::Lit(m + n)))
            }
            _ => None, // adding something that is not a number: stuck
        },
        Term::IsZero(n) if !is_value(n) => {
            step(n).map(|s| s.under(Rule::IsZero, |n2| Term::IsZero(Box::new(n2))))
        }
        Term::IsZero(n) => match **n {
            Term::Lit(0) => Some(Step::axiom(Rule::IsZeroZero, Term::True)),
            Term::Lit(_) => Some(Step::axiom(Rule::IsZeroSucc, Term::False)),
            _ => None,
        },
        // Variables (free ones: a closed term has none at the top) and values.
        Term::Var(_) | Term::Lam(..) | Term::True | Term::False | Term::Lit(_) => None,
    }
}
```

A `Step` records the derivation of the step along with its result: the congruence rules from the outside in, ending with the computation rule that did the work. A step is itself a derivation, a chain of rule instances, exactly like the Lean proof term `.appR .lam (.beta .lit)`, which says "E-AppR, whose premise is E-Beta".

```
/// One step `t → result`, with its derivation: the congruence rules from the
/// outside in, ending with the computation rule that did the work.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Step {
    pub derivation: Vec<Rule>,
    pub result: Term,
}
```

The relation is the specification and the function is the implementation. The theorem that connects them is determinism.

9.4 Determinism

Theorem (determinism). If $t \to t_1$ and $t \to t_2$, then $t_1 = t_2$.

The key lemma is that values are finished: no rule has a value on its left-hand side.

```
/-- Values do not step: no rule has a value on its left-hand side. -/
theorem Value.no_step {v t : Term} (hv : Value v) : ¬ Step v t := by
  intro h
  cases hv <;> cases h
```

`cases hv` considers each kind of value, `cases h` each rule that could make it step, and the combinator `<;>` runs the second tactic on every goal produced by the first. There are four kinds of values and no rule applies to any of them, so nothing is left.

The proof of determinism is by induction on the first step, h_1, followed by a case analysis of the second step, h_2. Here are the three cases for application:

```
/-- Evaluation is deterministic: a term has at most one next step. -/
theorem Step.deterministic {t t₁ t₂ : Term} (h₁ : Step t t₁) (h₂ : Step t t₂) :
    t₁ = t₂ := by
  induction h₁ generalizing t₂ with
  | appL h ih =>
    cases h₂ with
    | appL h' => rw [ih h']
    | appR hv _ => exact absurd h (hv.no_step)
    | beta _ => exact absurd h Value.lam.no_step
  | appR hv h ih =>
    cases h₂ with
    | appL h' => exact absurd h' hv.no_step
    | appR _ h' => rw [ih h']
    | beta hv' => exact absurd h hv'.no_step
  | beta hv =>
    cases h₂ with
    | appL h' => exact absurd h' Value.lam.no_step
    | appR _ h' => exact absurd h' hv.no_step
    | beta _ => rfl
```

`generalizing t₂` makes the induction hypothesis talk about any second step, not just the one we started with. Then each pair of rules falls into one of three patterns.

- **The same congruence rule twice** (`appL` with `appL`): both steps rewrote the same subterm, so by the induction hypothesis `ih h'` the two results of that subterm are equal, and `rw` rewrites one into the other, which closes the goal.
- **Two different rules** (`appL` with `appR`, `appL` with `beta`, and so on): one rule says that a subterm steps, the other that the same subterm is a value. `Value.no_step` says that both cannot hold, and `absurd h (...)` derives anything at all, the goal included, from a statement together with its negation. In the `appL` / `beta` cases the value is the λ itself (`Value.lam`).
- **The same computation rule twice** (`beta` with `beta`): both results are literally the same term, and `rfl` proves the equation.

The remaining nine cases, for `if`, `+` and `iszero`, follow the same three patterns. Whenever a pair of rules is not covered by the `with` clauses, it is because the two conclusions cannot have the same left-hand side, and `cases` has already discarded it.

Determinism has a satisfying consequence: a term has at most one result.

```
/-- A consequence of determinism: a term evaluates to at most one value. -/
theorem Steps.value_unique {t v₁ v₂ : Term} (h₁ : Steps t v₁) (h₂ : Steps t v₂)
    (hv₁ : Value v₁) (hv₂ : Value v₂) : v₁ = v₂ := by
  induction h₁ with
  | refl =>
    -- t = v₁ is a value, so the run to v₂ must be empty.
    cases h₂ with
    | refl => rfl
    | step h _ => exact absurd h hv₁.no_step
  | step h _ ih =>
    cases h₂ with
```

```
    | refl => exact absurd h hv₂.no_step
    | step h' rest =>
      -- Both runs take a first step; by determinism it is the same step.
      rw [Step.deterministic h h'] at *
      exact ih rest hv₁
```

The proof walks down the two runs together. If the first run is empty, it starts at the value v_1, from which the second run cannot step, so $v_1 = v_2$. If both runs take a first step, determinism says it is the same step; `rw [...] at *` replaces one intermediate term by the other in every hypothesis, and the induction hypothesis handles the rest of the runs.

Notice that determinism has nothing to do with types: it holds for every term, well typed or not. It is a property of the evaluation rules alone, and the premises "v_1 value" in the congruence rules are what make it true.

9.5 Canonical forms

We now turn to the theorems that involve types, and we start with a simple observation that the progress proof will need three times. If a *value* has type *bool*, which values could it be? Look at the typing rules: a λ has an arrow type, a literal has type *nat*, so the only candidates are *true* and *false*. That is a **canonical forms** lemma: it says what the values of each type look like.

```
/-- A value of type `bool` is `tt` or `ff`. -/
theorem canonical_bool {v : Term} (hv : Value v) (ht : HasType [] v .bool) :
    v = .tt ∨ v = .ff := by
  cases hv with
  | lam => cases ht        -- a λ has an arrow type, not bool
  | tt => left; rfl
  | ff => right; rfl
  | lit => cases ht        -- a literal has type nat, not bool

/-- A value of type `nat` is a literal. -/
theorem canonical_nat {v : Term} (hv : Value v) (ht : HasType [] v .nat) :
    ∃ n, v = .lit n := by
  cases hv with
  | lit => exact ⟨_, rfl⟩
  | _ => cases ht          -- every other value has a different type

/-- A value of type `σ → τ` is a λ whose parameter has type `σ`. -/
theorem canonical_arrow {v : Term} {σ τ : Ty} (hv : Value v)
    (ht : HasType [] v (.arrow σ τ)) : ∃ body, v = .lam σ body := by
  cases hv with
  | lam => cases ht; exact ⟨_, rfl⟩
  | _ => cases ht
```

Each proof starts with `cases hv`, one goal for each kind of value. In the impossible goals, `cases ht` finds that no rule could give, say, a λ the type `bool`: the conclusion of T-Lam is an arrow type, a different constructor, and the case is discarded. (`| _ =>` handles all the remaining kinds of value at once.) In `canonical_arrow`, `cases ht` does something more interesting: it unifies the T-Lam conclusion $\sigma' \to \tau'$, where σ' is the annotation of the λ, with the expected type $\sigma \to \tau$, and so learns that the annotation is σ.

9.6 Progress

Theorem (progress). If $\vdash t : \tau$ (in the empty context), then t is a value or there is a t' with $t \to t'$.

The context must be empty. A term with a free variable, such as $\#0 + 1$, is well typed in the context $[\mathit{nat}]$, but it is not a value and cannot step: it is waiting for a value to be substituted for $\#0$. Progress is a statement about *closed* programs, the ones you can actually run.

The proof is by induction on the typing derivation. In each case we know the last rule used, and the induction hypotheses tell us that each premise's term is a value or steps. Here it is in full:

```
/-- Progress: a closed, well-typed term is a value or can take a step. -/
theorem progress {t : Term} {τ : Ty} (ht : HasType [] t τ) :
    Value t ∨ ∃ t', Step t t' := by
  -- Induction on the derivation. The context is `[]` in every case, so Lean
  -- needs to remember that: `generalize` names it `Γ` with `hΓ : [] = Γ`.
  generalize hΓ : ([] : Ctx) = Γ at ht
  induction ht with
  | var h =>
    -- A variable in the empty context. `h : lookup [] i = some τ` computes to
    -- `none = some τ`, which no constructor of `=` can prove.
    subst hΓ
    cases h
  | lam _ _ => left; exact .lam
  | @app _ σ τ t₁ t₂ ht₁ _ ih₁ ih₂ =>
    right
    rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩
    · rcases ih₂ hΓ with hv₂ | ⟨t₂', h₂⟩
      · -- Both are values: by canonical forms, t₁ is a λ, and β applies.
        subst hΓ
        obtain ⟨body, rfl⟩ := canonical_arrow hv₁ ht₁
        exact ⟨_, .beta hv₂⟩
      · exact ⟨_, .appR hv₁ h₂⟩     -- t₁ is a value, t₂ steps
    · exact ⟨_, .appL h₁⟩           -- t₁ steps
  | tt => left; exact .tt
  | ff => left; exact .ff
  | @ite _ τ c t e hc _ _ ihc _ _ =>
    right
    rcases ihc hΓ with hvc | ⟨c', hc'⟩
    · -- The condition is a value of type bool: `tt` or `ff`.
      subst hΓ
      rcases canonical_bool hvc hc with rfl | rfl
      · exact ⟨_, .iteTrue⟩
      · exact ⟨_, .iteFalse⟩
    · exact ⟨_, .ite hc'⟩
  | lit => left; exact .lit
  | @add _ t₁ t₂ ht₁ ht₂ ih₁ ih₂ =>
    right
    rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩
    · rcases ih₂ hΓ with hv₂ | ⟨t₂', h₂⟩
      · -- Two values of type nat: two literals, and `addLit` applies.
        subst hΓ
        obtain ⟨m, rfl⟩ := canonical_nat hv₁ ht₁
        obtain ⟨n, rfl⟩ := canonical_nat hv₂ ht₂
        exact ⟨_, .addLit⟩
      · exact ⟨_, .addR hv₁ h₂⟩
    · exact ⟨_, .addL h₁⟩
  | @isZero _ t ht' ih =>
    right
    rcases ih hΓ with hv | ⟨t', h⟩
    · subst hΓ
      obtain ⟨n, rfl⟩ := canonical_nat hv ht'
      cases n with
      | zero => exact ⟨_, .isZeroZero⟩
      | succ n => exact ⟨_, .isZeroSucc⟩
    · exact ⟨_, .isZero h⟩
```

Let us go through it.

- `generalize hΓ : ([] : Ctx) = Γ at ht` replaces the empty context in `ht` by a variable `Γ` and records the equation `hΓ : [] = Γ`. Induction needs this: the rule T-Lam changes the context of its premise, so an induction over derivations must work for every context, and the equation carries the fact that ours is empty into every case. Whenever we need to know that the context is empty, `subst hΓ` replaces `Γ` by `[]` everywhere.
- **Variables.** A variable cannot be typed in the empty context: the premise `h : lookup [] i = some τ` computes to `none = some τ`, and `cases h` finds no way to prove that. The case is impossible, which is exactly why progress needs the context to be empty.
- **Values.** For `lam`, `tt`, `ff` and `lit` we choose the `left` side of the disjunction and give the `Value` constructor.
- **Application.** This is the interesting case. The pattern `@app _ σ τ t₁ t₂ ht₁ _ ih₁ ih₂` names the implicit arguments of the constructor (the `@` makes them visible), the typing derivation `ht₁` of the function, and the two induction hypotheses. We choose `right`: an application is never a value, so we must find a step. `rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩` splits on the induction hypothesis for the function: either it is a value (`hv₁`) or it steps to some `t₁'` (`h₁`). If it steps, E-AppL gives the step: `⟨_, .appL h₁⟩` is the pair of a witness (`_`, which Lean infers) and a proof that `t` steps to it. If it is a value, we split on the argument the same way. If the argument steps, E-AppR applies. If both are values, we finally need the canonical forms lemma: the function is a value of type $\sigma \to \tau$, so it is a λ. The pattern `⟨body, rfl⟩` in `obtain` does more than name the body: the `rfl` replaces `t₁` by `.lam σ body` everywhere, so that the goal becomes "`(λ:σ. body) t₂` steps", which E-Beta proves.
- **Conditionals.** The condition is a value or steps. If it steps, E-If. If it is a value, canonical forms says it is `tt` or `ff`, and `rcases ... with rfl | rfl` substitutes each possibility in turn; E-IfTrue and E-IfFalse finish.
- **Addition** is the application case again: left operand steps (E-AddL), or is a value and the right one steps (E-AddR), or both are values, hence literals by canonical forms, and E-AddLit applies.
- `iszero`: the argument steps (E-IsZero), or it is a literal n; `cases n` distinguishes 0 (E-IsZeroZero) from a successor (E-IsZeroSucc).

The shape of this proof is the same for every language: one canonical forms lemma per type with an elimination form, and evaluation rules that cover every combination of values the typing rules allow. Had we forgotten E-IsZeroSucc, the `succ` case would fail, and Lean would show us the stuck term. Progress is a **completeness check of the evaluator against the type system**.

9.7 Substitution, carefully

Preservation will need to know that E-Beta does not change the type, and E-Beta performs a substitution. So before preservation we need the **substitution lemma**: substituting a term of the right type for a variable preserves typing. On paper it is one line,

$$\text{if } \sigma :: \Gamma \vdash t : \tau \text{ and } \Gamma \vdash s : \sigma \text{ then } \Gamma \vdash t[s] : \tau,$$

but to prove it we first have to say exactly what $t[s]$ is, and that is where de Bruijn indices ask for care.

Renaming and shifting

Substitution must move terms under binders, and a term that moves under a binder must have its free variables renumbered: under one more λ, the variable that was #3 outside is #4 inside. The cleanest way to organize this is to treat a **renaming** as a function ρ from indices to indices, and to define how it acts on a term:

```
/-- Push a renaming under one binder: index 0 is the variable that the binder
introduces, and it stays 0; every other index `i + 1` refers to the variable `i`
of the outer scope, which is renamed by `ρ` and then pushed up by one again. -/
def liftRen (ρ : Nat → Nat) : Nat → Nat
  | 0 => 0
  | i + 1 => ρ i + 1

/-- Apply a renaming (a function from indices to indices) to every free variable
of a term. Under a λ the renaming is lifted. -/
def rename (ρ : Nat → Nat) : Term → Term
  | .var i => .var (ρ i)
  | .lam τ t => .lam τ (rename (liftRen ρ) t)
  | .app t₁ t₂ => .app (rename ρ t₁) (rename ρ t₂)
  | .tt => .tt
  | .ff => .ff
  | .ite c t e => .ite (rename ρ c) (rename ρ t) (rename ρ e)
  | .lit n => .lit n
  | .add t₁ t₂ => .add (rename ρ t₁) (rename ρ t₂)
  | .isZero t => .isZero (rename ρ t)

/-- Shifting: every free variable moves up by one. This is what a term needs
when it is carried under one more binder. -/
def shift (t : Term) : Term := rename (· + 1) t
```

The only subtle case is the λ. Inside the body, index 0 is the variable that this λ binds, which the renaming must leave alone; an index $i+1$ inside refers to the variable that was i outside, which the renaming sends to $\rho(i)$, and which inside the body must again be one higher, $\rho(i)+1$. That is `liftRen ρ`. With renaming in hand, **shifting** (moving a term under one more binder) is the renaming $i \mapsto i+1$:

```
/-- In `λ:nat. #0 + #1`, the `#0` is bound and the `#1` is free (it refers to
the variable just outside the λ, written `#0` out there). Shifting moves only
the free one. -/
example : shift (.lam .nat (.add (.var 0) (.var 1)))
    = .lam .nat (.add (.var 0) (.var 2)) := rfl
```

Substitution

A **substitution** is a function σ from indices to terms, and it acts on a term by replacing every free variable $\#i$ with $\sigma(i)$. Under a λ it must be lifted, in the same way as a renaming, with one extra twist: the term that replaces an outer variable now lives under one more binder, so it must be shifted.

```
/-- Push a substitution under one binder: index 0 is the new bound variable and
maps to itself; the term `σ i` that replaces outer variable `i` is shifted,
because it now lives under one more binder. -/
def liftSub (σ : Nat → Term) : Nat → Term
  | 0 => .var 0
  | i + 1 => shift (σ i)

/-- Simultaneous substitution: replace every free variable `var i` by `σ i`. -/
def subst (σ : Nat → Term) : Term → Term
```

```
  | .var i => σ i
  | .lam τ t => .lam τ (subst (liftSub σ) t)
  | .app t₁ t₂ => .app (subst σ t₁) (subst σ t₂)
  | .tt => .tt
  | .ff => .ff
  | .ite c t e => .ite (subst σ c) (subst σ t) (subst σ e)
  | .lit n => .lit n
  | .add t₁ t₂ => .add (subst σ t₁) (subst σ t₂)
  | .isZero t => .isZero (subst σ t)

/-- The substitution used by β-reduction: variable 0 becomes `s`, and every
other variable moves down by one, because the λ that bound variable 0 is gone. -/
def single (s : Term) : Nat → Term
  | 0 => s
  | i + 1 => .var i

/-- `instantiate body s` is the body of a λ with its bound variable replaced by
`s`; on paper we write it $t[s]$. -/
def instantiate (body s : Term) : Term := subst (single s) body
```

`single s` is the substitution performed by E-Beta. When the λ disappears, its bound variable $\#0$ becomes the argument s, and every other free variable of the body moves *down* by one, because there is one binder fewer between it and its own λ. The examples show all three effects:

```
/-- The body of `λ:nat. #0 + 1`, instantiated with `41`. -/
example : instantiate (.add (.var 0) (.lit 1)) (.lit 41)
    = .add (.lit 41) (.lit 1) := rfl

/-- Substituting under a binder. The body is `λ:nat. #1 #0` (inside it, `#1` is
the outer bound variable) and the argument is a free variable `#5`. The result
mentions `#6`: under the inner λ, the argument had to be shifted. -/
example : instantiate (.lam .nat (.app (.var 1) (.var 0))) (.var 5)
    = .lam .nat (.app (.var 6) (.var 0)) := rfl

/-- A free variable of the body that is *not* the one being replaced moves down
by one, because the λ that bound `#0` has disappeared. -/
example : instantiate (.add (.var 0) (.var 3)) (.lit 7)
    = .add (.lit 7) (.var 2) := rfl
```

In the second example the body is $\lambda : \mathit{nat}.\ \#1\ \#0$, whose $\#1$ is the outer bound variable, and we substitute the free variable $\#5$ for it. The result mentions $\#6$, not $\#5$: inside the inner λ, the variable that is $\#5$ outside is called $\#6$, and `liftSub` shifted it accordingly. Forget that shift and the substituted term would silently refer to the wrong variable, which is the de Bruijn version of variable capture. This is the kind of bug that a proof assistant will not let you make.

The Rust crate has the same functions, with renamings and substitutions as closures (file `code/rust/crates/safety/src/subst.rs`):

```
/// Replaces every free variable `Var(i)` of `t` by `sigma(i)`.
pub fn subst(t: &Term, sigma: &dyn Fn(usize) -> Term) -> Term {
    let go = |u: &Term| Box::new(subst(u, sigma));
    match t {
        Term::Var(i) => sigma(*i),
        Term::Lam(ty, body) => {
            // Under the binder, 0 maps to itself, and the replacement for an
            // outer variable is shifted: it now lives under one more λ.
            let lifted = |i: usize| {
                if i == 0 {
                    Term::Var(0)
                } else {
                    shift(&sigma(i - 1))
                }
```

```
            };
            Term::Lam(ty.clone(), Box::new(subst(body, &lifted)))
        }
        Term::App(f, a) => Term::App(go(f), go(a)),
        Term::True => Term::True,
        Term::False => Term::False,
        Term::If(c, t, e) => Term::If(go(c), go(t), go(e)),
        Term::Lit(n) => Term::Lit(*n),
        Term::Add(a, b) => Term::Add(go(a), go(b)),
        Term::IsZero(t) => Term::IsZero(go(t)),
    }
}

/// The body of a λ with its bound variable replaced by `s`: variable 0
/// becomes `s`, and every other variable moves down by one.
pub fn instantiate(body: &Term, s: &Term) -> Term {
    subst(body, &|i| if i == 0 { s.clone() } else { Term::Var(i - 1) })
}
```

The renaming lemma, and weakening

We prove the typing lemmas in the same order as the definitions: renaming first, then substitution, then the special case that β needs. A renaming preserves types provided it respects the contexts, that is, it sends each variable to a variable of the same type:

```
/-- Renaming preserves types, provided the renaming respects the contexts:
whenever `Γ` gives variable `i` type `τ`, `Δ` gives `ρ i` the same type. -/
theorem rename_typing {Γ Δ : Ctx} {ρ : Nat → Nat} {t : Term} {τ : Ty}
    (hρ : ∀ i τ, lookup Γ i = some τ → lookup Δ (ρ i) = some τ)
    (ht : HasType Γ t τ) : HasType Δ (rename ρ t) τ := by
  induction ht generalizing Δ ρ with
  | var h => exact .var (hρ _ _ h)
  | lam _ ih =>
    -- Under the binder, the lifted renaming respects the extended contexts.
    apply HasType.lam
    apply ih
    intro i τ' h
    cases i with
    | zero => exact h                    -- the bound variable: 0 ↦ 0
    | succ i => exact hρ i τ' h          -- an outer variable: i + 1 ↦ ρ i + 1
  | app _ _ ih₁ ih₂ => exact .app (ih₁ hρ) (ih₂ hρ)
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iht ihe => exact .ite (ihc hρ) (iht hρ) (ihe hρ)
  | lit => exact .lit
  | add _ _ ih₁ ih₂ => exact .add (ih₁ hρ) (ih₂ hρ)
  | isZero _ ih => exact .isZero (ih hρ)
```

The proof is by induction on the typing derivation, `generalizing Δ ρ` so that the induction hypotheses work for every target context and every renaming (we will need them for different ones). Most cases just rebuild the same rule from the induction hypotheses, like `.app (ih₁ hρ) (ih₂ hρ)`. Two cases do real work.

- **Variables.** The term is $\#i$ with `h : lookup Γ i = some τ`, the renamed term is $\#\rho(i)$, and the hypothesis `hρ` says precisely that `lookup Δ (ρ i) = some τ`. Rule T-Var does the rest.
- **λ.** After `apply HasType.lam`, the goal is to type the renamed body, `rename (liftRen ρ) t`, in the context `σ :: Δ`. The induction hypothesis `ih` can do that, provided we show that the lifted renaming respects the extended contexts `σ :: Γ` and `σ :: Δ`. That is the goal after `apply ih` and `intro i τ' h`, and we prove it by cases on the index. For `0`, the lookup in both contexts is the new entry `σ`, and `liftRen ρ 0` is `0`, so the hypothesis `h` already *is* the goal:

both compute to `some σ = some τ'`, and `exact h` succeeds because Lean compares types up to computation. For `i + 1`, the hypothesis computes to `lookup Γ i = some τ'` and the goal to `lookup Δ (ρ i) = some τ'`, which is `hρ` applied to `i`.

The most important instance is **weakening**: a term that is typed in Γ is still typed, after shifting, in a context with one more variable in front. It is the renaming lemma with $\rho(i) = i + 1$:

```
/-- Weakening: a term typed in `Γ` is still typed, after shifting, in a
context with one more variable in front. -/
theorem weakening {Γ : Ctx} {t : Term} {τ : Ty} (σ : Ty) (ht : HasType Γ t τ) :
    HasType (σ :: Γ) (shift t) τ :=
  rename_typing (fun _ _ h => h) ht
```

The side condition says that `lookup Γ i = some τ` implies `lookup (σ :: Γ) (i + 1) = some τ`, and the second statement computes to the first, so the identity function `fun _ _ h => h` proves it. This is the structural rule of weakening from Chapter 6, in de Bruijn clothing: adding an unused assumption does not invalidate a derivation, provided you renumber.

The substitution lemma

A substitution preserves types provided it replaces every variable by a term of that variable's type:

```
/-- Substitution preserves types, provided every variable is replaced by a term
of its own type: whenever `Γ` gives `i` type `τ`, the term `σ i` has type `τ`
in `Δ`. -/
theorem subst_typing {Γ Δ : Ctx} {σ : Nat → Term} {t : Term} {τ : Ty}
    (hσ : ∀ i τ, lookup Γ i = some τ → HasType Δ (σ i) τ)
    (ht : HasType Γ t τ) : HasType Δ (subst σ t) τ := by
  induction ht generalizing Δ σ with
  | var h => exact hσ _ _ h
  | @lam Γ α β t _ ih =>
    apply HasType.lam
    apply ih
    intro i τ' h
    cases i with
    | zero =>
      -- The bound variable is replaced by itself.
      exact .var h
    | succ i =>
      -- An outer variable is replaced by `shift (σ i)`, typed by weakening.
      exact weakening α (hσ i τ' h)
  | app _ _ ih₁ ih₂ => exact .app (ih₁ hσ) (ih₂ hσ)
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iht ihe => exact .ite (ihc hσ) (iht hσ) (ihe hσ)
  | lit => exact .lit
  | add _ _ ih₁ ih₂ => exact .add (ih₁ hσ) (ih₂ hσ)
  | isZero _ ih => exact .isZero (ih hσ)
```

The structure is identical to the renaming lemma. In the variable case, the variable $\#i$ is replaced by $\sigma(i)$, and the hypothesis `hσ` is exactly the statement that $\sigma(i)$ has the right type. In the λ case, the pattern `@lam Γ α β t _ ih` names the parameter type `α`, and we must show that the lifted substitution respects `α :: Γ` and `α :: Δ`. Index `0` is replaced by the variable $\#0$, which has type `α` by T-Var. Index `i + 1` is replaced by `shift (σ i)`, and here the substitution lemma uses the renaming lemma: `σ i` has type `τ'` in `Δ` by `hσ`, so its shift has type `τ'` in `α :: Δ` by weakening. This is the precise point where the two lemmas meet, and the reason renaming had to come first.

Finally, the form that β-reduction needs:

```
/-- The substitution lemma in the form β-reduction needs: if the body of a λ
has type `τ` when its parameter has type `σ`, and `s` has type `σ`, then the
instantiated body has type `τ`. -/
theorem instantiate_typing {Γ : Ctx} {σ τ : Ty} {body s : Term}
    (hbody : HasType (σ :: Γ) body τ) (hs : HasType Γ s σ) :
    HasType Γ (instantiate body s) τ := by
  apply subst_typing _ hbody
  intro i τ' h
  cases i with
  | zero =>
    -- Variable 0 has type σ in σ :: Γ: `h` computes to `some σ = some τ'`,
    -- and `cases` concludes that τ' is σ. Variable 0 is replaced by s : σ.
    cases h
    exact hs
  | succ i =>
    -- Variable i + 1 becomes var i, with the same type in Γ.
    exact .var h
```

`apply subst_typing _ hbody` reduces the goal to the side condition for the substitution `single s`, with `σ :: Γ` as the source context and `Γ` as the target. Index `0` has type `σ` and is replaced by `s`, which has type `σ` by `hs`; `cases h` turns `h : some σ = some τ'` into the knowledge that `τ'` is `σ`. Index `i + 1` is replaced by $\#i$, whose type in `Γ` is the type of $\#(i+1)$ in `σ :: Γ`, which is what `h` says.

That is the hard part of type safety, and it is all about variables: booleans and numbers appear only in routine cases. Add a construct to the language and these proofs get one more routine case each; change how binders work and this is the part you must redo.

9.8 Preservation

Theorem (preservation). If $\Gamma \vdash t : \tau$ and $t \to t'$, then $\Gamma \vdash t' : \tau$.

```
/-- Preservation: a step does not change the type of a term. -/
theorem preservation {Γ : Ctx} {t t' : Term} {τ : Ty}
    (ht : HasType Γ t τ) (hstep : Step t t') : HasType Γ t' τ := by
  -- Induction on the step; in each case, take the typing derivation apart.
  induction hstep generalizing τ with
  | appL _ ih =>
    cases ht with
    | app ht₁ ht₂ => exact .app (ih ht₁) ht₂
  | appR _ _ ih =>
    cases ht with
    | app ht₁ ht₂ => exact .app ht₁ (ih ht₂)
  | beta _ _ =>
    -- ht : HasType Γ (app (lam σ body) v) τ. Take it apart twice:
    cases ht with
    | app hlam hv =>
      cases hlam with
      | lam hbody =>
        -- hbody : HasType (σ :: Γ) body τ,  hv : HasType Γ v σ
        exact instantiate_typing hbody hv
  | ite _ ih =>
    cases ht with
    | ite hc ht₁ ht₂ => exact .ite (ih hc) ht₁ ht₂
  | iteTrue =>
    cases ht with
    | ite _ ht₁ _ => exact ht₁
  | iteFalse =>
    cases ht with
    | ite _ _ ht₂ => exact ht₂
  | addL _ ih =>
    cases ht with
    | add ht₁ ht₂ => exact .add (ih ht₁) ht₂
  | addR _ _ ih =>
```

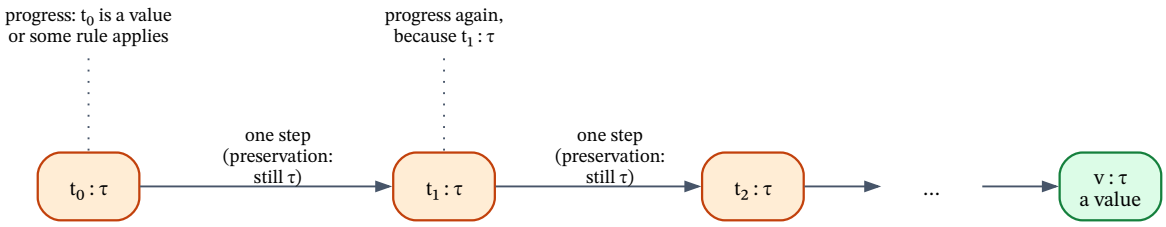
```
    cases ht with
    | add ht₁ ht₂ => exact .add ht₁ (ih ht₂)
  | addLit =>
    cases ht
    exact .lit
  | isZero _ ih =>
    cases ht with
    | isZero ht' => exact .isZero (ih ht')
  | isZeroZero =>
    cases ht
    exact .tt
  | isZeroSucc =>
    cases ht
    exact .ff
```

Textbooks usually prove preservation by induction on the typing derivation. We do it by induction on the *step*, `generalizing τ`, and then take the typing derivation apart with `cases`: the step tells us which rule fired, and in each case there is exactly one typing rule that could have typed the left-hand side. This "taking apart" is called **inversion**: from the fact that `app t₁ t₂` has type `τ`, `cases ht` concludes that there is a σ with $t_1 : \sigma \to \tau$ and $t_2 : \sigma$, because T-App is the only rule whose conclusion is an application.

- **Congruence rules** (`appL`, `appR`, `ite`, `addL`, `addR`, `isZero`): inversion gives the derivations of the subterms, the induction hypothesis `ih` says that the stepped subterm keeps its type, and the same typing rule reassembles the whole.
- **E-Beta**: here is where all the work of the previous section pays off. The term is $(\lambda : \sigma'.\, body)\, v$. Inversion of T-App gives `hlam`, a derivation for the λ with some type $\sigma \to \tau$, and `hv`, for the argument with type σ. Inversion of T-Lam on `hlam` gives `hbody : HasType (σ :: Γ) body τ` (and, silently, the fact that the annotation σ' is σ). The substitution lemma `instantiate_typing hbody hv` is exactly the conclusion.
- **E-IfTrue and E-IfFalse**: the result is one of the branches, and inversion of T-If gives its derivation, with the type `τ` of the whole `if`. This is where rule T-If's insistence on a single type for both branches is used.
- **E-AddLit, E-IsZeroZero, E-IsZeroSucc**: the result is a literal or a boolean, typed by an axiom. `cases ht` is still needed, to learn that `τ` is `nat` or `bool`.

9.9 Type safety

Progress talks about one state and preservation about one step. Safety talks about whole runs, and it follows from the two by an induction on the length of the run.



A run of a well-typed program: preservation keeps the type at every step, and progress, applied to each state in turn, says that the run never stops at a term that is not a value.

```
/-- Preservation for runs of any length. -/
theorem preservation_steps {t t' : Term} {τ : Ty}
    (ht : HasType [] t τ) (hsteps : Steps t t') : HasType [] t' τ := by
  induction hsteps with
  | refl => exact ht
  | step hstep _ ih => exact ih (preservation ht hstep)

/-- Type safety: a closed well-typed term never reaches a stuck state. -/
theorem safety {t t' : Term} {τ : Ty}
    (ht : HasType [] t τ) (hsteps : Steps t t') : ¬ Stuck t' := by
  intro ⟨hnotval, hnostep⟩
  -- t' is still well typed, so progress applies to it.
  rcases progress (preservation_steps ht hsteps) with hv | ⟨t'', h⟩
  · exact hnotval hv
  · exact hnostep t'' h
```

`preservation_steps` extends preservation to runs, by induction on the run: an empty run changes nothing, and a run that starts with a step keeps the type first by `preservation` and then by the induction hypothesis. `safety` then assumes that the end of the run is stuck, which `intro ⟨hnotval, hnostep⟩` splits into "not a value" and "no step". The term at the end of the run is still well typed, by `preservation_steps`, so progress applies to it: it is a value, contradicting `hnotval`, or it steps, contradicting `hnostep`.

That is the theorem Milner's slogan promised: a closed, well-typed term never reaches a stuck state, however many steps it takes. Specialized to one program, it is a one-line corollary:

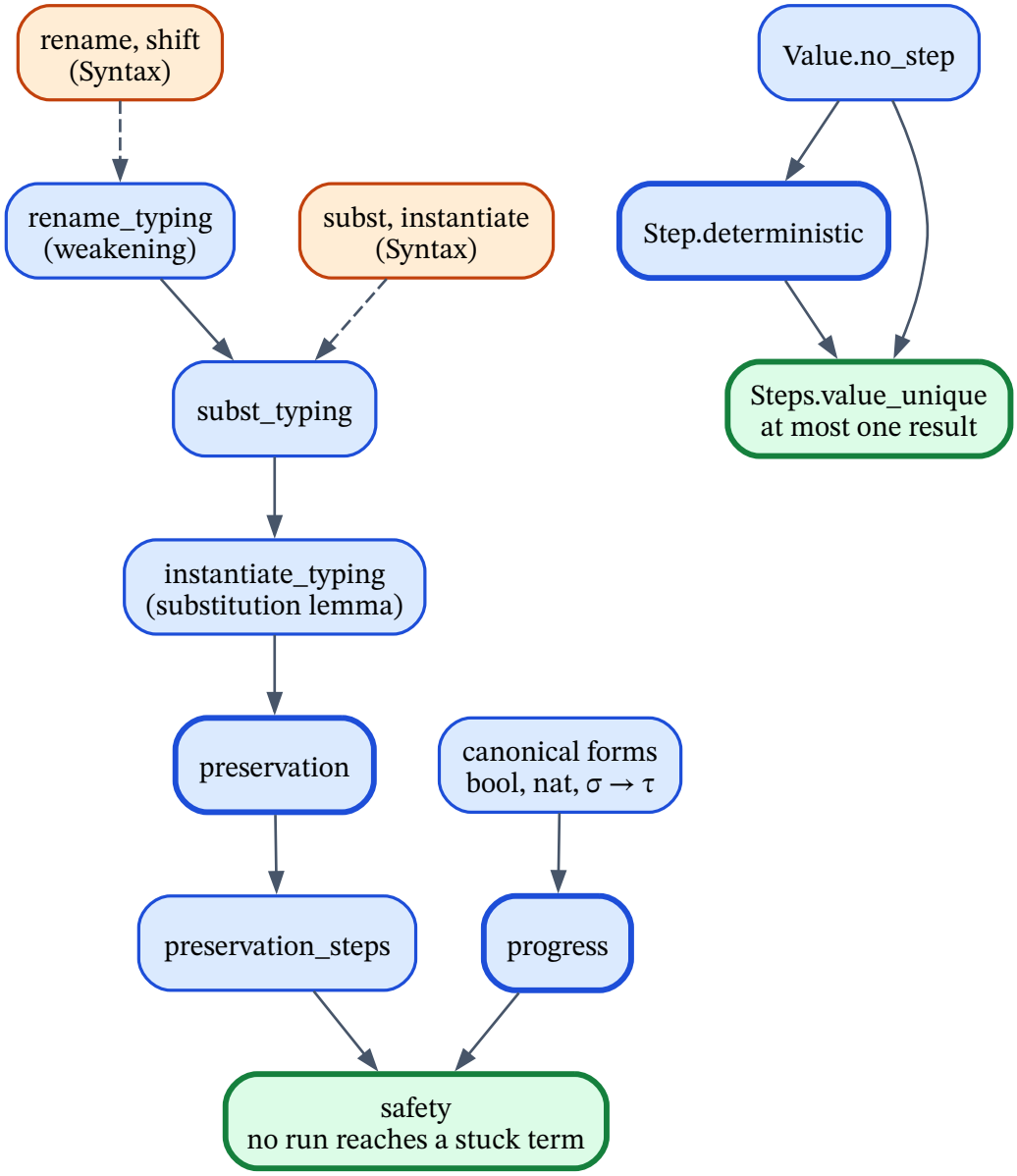
```
/-- For example: the program `(λ:nat. #0 + 1) 41` never gets stuck, whatever
it reduces to. The typing derivation comes from the checker. -/
theorem incr41_safe {t' : Term} (h : Steps incr41 t') : ¬ Stuck t' :=
  safety incr41_typed h
```

And one more check, of a different kind:

```
/-- info: 'Ch09.safety' does not depend on any axioms -/
#guard_msgs in
#print axioms safety
```

Chapter 1 used `#print axioms` to show that some proofs depend on axioms such as `propext` or `Classical.choice`. The proof of type safety depends on none: it is built from the rules of Lean's type theory alone, every case analysis is constructive, and the theorem holds in the strongest sense Lean has. (This is the reason for our own `lookup`: the definition of Lean's list indexing `Γ[i]?` happens to bring in the axiom `propext`. That would not make the proof wrong, but it is satisfying to know that it is not needed.)

The whole development fits in one picture. Arrows go from a lemma to the results that use it.



The structure of the proof: the substitution lemmas feed preservation, the canonical forms feed progress, and the two together give safety; determinism is a separate branch that does not involve types.

9.10 Testing the theorems in Rust

The Lean proof covers every term at once. The Rust crate `safety` takes the other road: it implements the same language and checks the same theorems on many concrete terms. The example `ch09_fuzz_safety`, in `code/rust/crates/safety/examples/ch09_fuzz_safety.rs`, does the whole tour; you run it with

```
cargo run -p safety --example ch09_fuzz_safety
```

Random numbers without a crate

The standard library has no random number generator, and the book's Rust code uses no other crates. A property-based test does not need good randomness, only *reproducible* variety, and a few lines of SplitMix64 provide it:

```
/// SplitMix64: a 64-bit state advanced by a constant, then scrambled.
/// Not for cryptography; good enough to drive a test generator.
#[derive(Clone, Debug)]
pub struct Rng {
    state: u64,
}

impl Rng {
    pub fn new(seed: u64) -> Rng {
        Rng { state: seed }
    }

    pub fn next_u64(&mut self) -> u64 {
        self.state = self.state.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.state;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }

    /// A number in `0..n` (with a negligible bias for small `n`).
    pub fn below(&mut self, n: usize) -> usize {
        assert!(n > 0, "below(0)");
        (self.next_u64() % n as u64) as usize
    }

    /// An element of a non-empty slice.
    pub fn pick<'a, T>(&mut self, items: &'a [T]) -> &'a T {
        &items[self.below(items.len())]
    }
}
```

The same seed always produces the same sequence, so every failure can be replayed exactly. (Crates such as `proptest` and `quickcheck` package this whole approach, with better generators and automatic shrinking; we build a small version ourselves so that every piece is visible.)

Generating well-typed terms

A random term is almost never well typed: generate `if`, `+`, `true` and λ at random and you get `iszero (λ:bool. #0)` most of the time. Testing progress and preservation on such terms tests nothing, because the theorems only speak about well-typed terms. The generator must produce well-typed terms, and in large numbers.

The trick is to run the typing rules **backwards**. To generate a term of type τ, pick any rule whose conclusion can have type τ, and generate its premises recursively. To produce a *nat*, T-Lit, T-Add, T-If and T-App all qualify; for T-App, invent an argument type σ, then generate a function of type $\sigma \to \tau$ and an argument of type σ. To produce a $\sigma \to \tau$, use T-Lam: put σ in the context and generate a body of type τ, where T-Var may now pick the new variable.

```
/// A random term of type `ty` in context `ctx`, with roughly `size`
/// constructors, following the typing rules `rules` backwards.
pub fn gen_term(rng: &mut Rng, rules: Rules, ctx: &[Ty], ty: &Ty, size: usize) -> Term {
    // Collect the rules whose conclusion can have type `ty`.
    let mut choices = Vec::new();
```

```
    for (i, var_ty) in ctx.iter().enumerate() {
        if var_ty == ty {
            choices.push(Choice::Var(i)); // T-Var
        }
    }
    match ty {
        Ty::Bool => choices.extend([Choice::True, Choice::False]),
        Ty::Nat => choices.push(Choice::Lit),
        Ty::Arrow(..) => choices.push(Choice::Lam),
    }
    if size > 1 {
        // Rules with premises, when there is room for subterms. Applications
        // are listed twice: they create the β-redexes that make runs long.
        choices.extend([Choice::App, Choice::App, Choice::If]);
        match ty {
            Ty::Nat => choices.push(Choice::Add),
            Ty::Bool => choices.push(Choice::IsZero),
            Ty::Arrow(..) => {}
        }
    }
    let budget = size.saturating_sub(1);
    match *rng.pick(&choices) {
        Choice::Var(i) => Term::Var(i),
        Choice::True => Term::True,
        Choice::False => Term::False,
        Choice::Lit => Term::Lit(rng.below(10) as u64),
        Choice::Lam => {
            // T-Lam: ty is σ → τ; generate a body of type τ with σ in scope.
            let Ty::Arrow(param, result) = ty else {
                unreachable!()
            };
            let mut inner = vec![(**param).clone()];
            inner.extend_from_slice(ctx);
            lam(
                (**param).clone(),
                gen_term(rng, rules, &inner, result, budget),
            )
        }
        Choice::App => {
            // T-App: invent an argument type σ, then a function σ → ty and an
            // argument σ.
            let arg_ty = gen_type(rng, 1);
            let (left, right) = split(rng, budget);
            let f = gen_term(
                rng,
                rules,
                ctx,
                &Ty::arrow(arg_ty.clone(), ty.clone()),
                left,
            );
            app(f, gen_term(rng, rules, ctx, &arg_ty, right))
        }
        Choice::If => {
            // T-If: a boolean condition and two branches of type ty. The broken
            // rule lets the else branch have any type at all.
            let c = gen_term(rng, rules, ctx, &Ty::Bool, budget / 3);
            let t = gen_term(rng, rules, ctx, ty, budget / 3);
            let else_ty = match rules {
                Rules::Sound => ty.clone(),
                Rules::BrokenIf => gen_type(rng, 1),
            };
            ite(c, t, gen_term(rng, rules, ctx, &else_ty, budget / 3))
        }
        Choice::Add => {
            let (left, right) = split(rng, budget);
            let a = gen_term(rng, rules, ctx, &Ty::Nat, left);
            add(a, gen_term(rng, rules, ctx, &Ty::Nat, right))
        }
        Choice::IsZero => is_zero(gen_term(rng, rules, ctx, &Ty::Nat, budget)),
```

```
    }
}
```

The size budget shrinks at every rule with premises, which guarantees that generation stops. The terms are well typed by construction, but "by construction" is a claim, not a proof, so the checker verifies it for every term.

Checking a run

For each generated term, the harness runs it to a value and checks the three properties at every state along the way:

```
/// Runs `t`, which should have type `ty`, to a value, checking at every state:
/// the state has type `ty` (preservation; at the start, the generator's claim),
/// it is a value or the rules allow a step (progress), and the rules allow at
/// most one step, the one the evaluator takes (determinism).
/// Returns the number of steps taken.
pub fn check_run(
    rules: Rules,
    t: &Term,
    ty: &Ty,
    fuel: usize,
) -> Result<usize, Violation> {
    let found = type_of_with(rules, &[], t);
    if found.as_ref() != Ok(ty) {
        return Err(Violation::Generator {
            term: t.clone(),
            ty: ty.clone(),
            found,
        });
    }
    let mut current = t.clone();
    for steps_taken in 0..fuel {
        if is_value(&current) {
            return Ok(steps_taken);
        }
        let all = successors(&current);
        if all.is_empty() {
            return Err(Violation::Progress {
                state: current,
                ty: ty.clone(),
            });
        }
        if all.len() > 1 || step(&current).as_ref() != Some(&all[0]) {
            return Err(Violation::Determinism {
                state: current,
                steps: all,
            });
        }
        let next = all[0].result.clone();
        let found = type_of_with(rules, &[], &next);
        if found.as_ref() != Ok(ty) {
            let (before, after, ty) = (current, next, ty.clone());
            return Err(Violation::Preservation {
                before,
                after,
                ty,
                found,
            });
        }
        current = next;
    }
    Err(Violation::OutOfFuel { state: current })
}
```

Each check is a direct transcription of a theorem. **Preservation**: the type checker agrees that every new state has the original type. **Progress**: if the state is not a value, the relation allows at least one step. **Determinism**: it allows at most one, and it is the one the evaluator `step` chose. This last check is where having two versions of the semantics pays off: the relation `successors` is the specification, written rule by rule, and the function `step` is the implementation, written for efficiency. The test compares them on every state it visits.

Here is a run of ten thousand random programs:

```
== Property test: random well-typed terms ==
terms        : 10000 (seed 2024)
types        : 3389 bool, 3263 nat, 3348 function
largest term : 68 constructors
steps        : 32305 in total, longest run 19
progress     : held in all 42305 states visited
preservation : held for all 32305 steps
determinism  : one rule at most in every state, as the evaluator chose
```

Every theorem held in every state. The runs are short (a few steps on average): random well-typed programs tend to be shallow, which is a limitation of the generator worth keeping in mind.

What the type system rules out

The same crate can generate terms with no regard for types. Closed random terms of up to twelve constructors give a feeling for what typing is for:

```
== Random closed terms, ignoring types ==
well typed   : 3380
ill typed    : 6620
   stuck              : 5299
   reached a value    : 1321
   ran out of fuel    : 0
```

Of the terms that the checker rejects, about four in five really do get stuck. The rest run to a value without trouble: `if true then 1 else false` is rejected because its branches have different types, yet it evaluates to `1`. A type system is **conservative**: it rejects every program that could go wrong, and some that would not have. Four hand-picked examples:

```
== Ill-typed terms ==
true + 1
   rejected: the left operand of `+` should be nat, found bool
   it is stuck at true + 1 after 0 steps
(λ:nat. #0 + 1) true
   rejected: the argument should be nat, found bool
   it is stuck at true + 1 after 1 step
if true then 1 else false
   rejected: the branches of `if` differ: nat and bool
   it reaches the value 1 in 1 step
(λ:bool. #0 #0) (λ:bool. #0 #0)
   rejected: applying a non-function of type bool
   it is still running after 100 steps
```

The second one takes a perfectly good β-step before it gets stuck at `true + 1`: in an ill-typed program the error may appear far from its cause, which is exactly what preservation prevents for well-typed ones. The last is the Ω of Chapter 7, which runs forever. It is ill typed too; as Chapter 8 said, every

well-typed term of the simply typed λ-calculus terminates. Our safety theorem does not need that fact, as we will see at the end of the chapter.

Breaking a rule on purpose

A test suite that never fails says little about its own power, so let us break the type system and see whether the tests notice. The crate's `Rules::BrokenIf` changes one rule: T-If takes the type of its `then` branch and ignores the `else` branch entirely. The generator follows the broken rule too, so it produces terms like `if c then 1 else true`, which the broken checker calls `nat`. Here is what the harness says:

```
== The same test, with the broken rule for `if` ==
term #3 (29 constructors) violates preservation
shrunk to    : if false then 0 else false : nat
   preservation fails: if false then 0 else false : nat  →  false : bool
and the counterexample from Safety.lean:
   (if false then 1 else true) + 2  : nat
→  true + 2  : ill typed: the left operand of `+` should be nat, found bool
      by E-AddL(E-IfFalse)
   (stuck)
```

Term number 3 (counting from 0) already violates preservation. The raw counterexample has 29 constructors, which is not very informative, so the harness **shrinks** it: it repeatedly tries smaller candidates (a subterm in place of the whole, a constant in place of a subterm) and keeps any candidate that still has a type under the broken rules and still violates a theorem.

```rust
/// Shrinks a failing term: repeatedly replaces it by a smaller candidate that
/// still has a type (under `rules`) and still violates a theorem.
pub fn shrink(rules: Rules, term: &Term, fuel: usize) -> (Term, Ty, Violation) {
    let ty = type_of_with(rules, &[], term).expect("a failing term has a type");
    let violation = check_run(rules, term, &ty, fuel).expect_err("the term fails");
    let mut best = (term.clone(), ty, violation);
    'improve: loop {
        for candidate in shrink_candidates(&best.0) {
            // Only candidates that are still "well typed" count...
            let Ok(ty) = type_of_with(rules, &[], &candidate) else {
                continue;
            };
            // ...and that still break a theorem.
            if let Err(violation) = check_run(rules, &candidate, &ty, fuel) {
                best = (candidate, ty, violation);
                continue 'improve;
            }
        }
        return best;
    }
}
```

The result, `if false then 0 else false : nat`, is as small as a counterexample can be: one step later it is `false`, a `bool`. Preservation fails, and with it safety: the counterexample that the Lean file uses, `(if false then 1 else true) + 2`, steps to `true + 2`, which is stuck.

The Lean side can say the same thing as a theorem. `BadHasType` is `HasType` with the broken rule:

```
/-- A *broken* type system: identical to `HasType`, except that the rule for
`if` only looks at the `then` branch. -/
inductive BadHasType : Ctx → Term → Ty → Prop
  | var {Γ : Ctx} {i : Nat} {τ : Ty} :
      lookup Γ i = some τ → BadHasType Γ (.var i) τ
  | lam {Γ : Ctx} {σ τ : Ty} {t : Term} :
      BadHasType (σ :: Γ) t τ → BadHasType Γ (.lam σ t) (.arrow σ τ)
```

```
  | app {Γ : Ctx} {σ τ : Ty} {t₁ t₂ : Term} :
      BadHasType Γ t₁ (.arrow σ τ) → BadHasType Γ t₂ σ →
      BadHasType Γ (.app t₁ t₂) τ
  | tt {Γ : Ctx} : BadHasType Γ .tt .bool
  | ff {Γ : Ctx} : BadHasType Γ .ff .bool
  | ite {Γ : Ctx} {τ τ' : Ty} {c t e : Term} :          -- τ' is ignored!
      BadHasType Γ c .bool → BadHasType Γ t τ → BadHasType Γ e τ' →
      BadHasType Γ (.ite c t e) τ
  | lit {Γ : Ctx} {n : Nat} : BadHasType Γ (.lit n) .nat
  | add {Γ : Ctx} {t₁ t₂ : Term} :
      BadHasType Γ t₁ .nat → BadHasType Γ t₂ .nat →
      BadHasType Γ (.add t₁ t₂) .nat
  | isZero {Γ : Ctx} {t : Term} :
      BadHasType Γ t .nat → BadHasType Γ (.isZero t) .bool
```

and the broken system is provably unsafe:

```
/-- The broken system is not safe: `(if false then 1 else true) + 2` is
"well typed" in it, and after one step it is stuck. -/
theorem bad_typing_unsafe :
    ∃ t t' τ, BadHasType [] t τ ∧ Steps t t' ∧ Stuck t' := by
  refine ⟨.add (.ite .ff (.lit 1) .tt) (.lit 2),   -- the program
          .add .tt (.lit 2),                        -- where it ends up
          .nat, ?_, ?_, ?_⟩
  · -- "Well typed": the `then` branch is a nat, the `else` branch is ignored.
    exact .add (.ite .ff .lit .tt) .lit
  · -- One step: the `if` reduces to its `else` branch.
    exact .step (.addL .iteFalse) .refl
  · -- `true + 2` is stuck, for the same reason as `true + 1`.
    constructor
    · intro h; cases h
    · intro t'' h
      cases h with
      | addL h₁ => cases h₁
      | addR _ h₂ => cases h₂
```

`refine` fills in the three witnesses and leaves three goals, marked `?_`, each proved as before. And if you copy the proof of `preservation` and replace `HasType` by `BadHasType`, it breaks in exactly one place: in the `iteFalse` case, inversion gives the `else` branch a type `τ'` unrelated to `τ`, and `exact ht₂` is rejected. The proof does not just say "unsafe"; it points at the rule.

Tests and proofs, compared

We now have two kinds of evidence for the same theorems, and it is worth being precise about what each gives.

	Property tests (Rust)	Proof (Lean)
Covers	the terms generated	every term
Checks	the Rust code that runs	a model of the language
On failure	a concrete, shrunk counterexample	a goal that cannot be closed
Cost	minutes to write, seconds to run	hours to write, then free
Explains	nothing: it found a bug	why: which lemma, which case

The first two rows are the important ones. The proof covers *all* terms; the tests cover those the generator happens to produce, and our generator has blind spots (short runs, small types), so a bug that only shows up in deep terms could escape ten thousand tests. On the other hand, the proof is

about the Lean definitions, while the program that runs is the Rust crate, and nothing in the Lean file prevents the Rust `step` function from having a bug of its own. That is why the harness checks the Rust evaluator against a rule-by-rule transcription of the relation. Proofs are about models, tests are about code, and a careful project uses both; Chapter 31 tests a Rust implementation against a verified Lean model.

9.11 Putting it together: one program, three views

Take the program $twice\ (\lambda : \mathit{nat}.\ \#0 + 2)\ 3$, which applies "add two" twice to three. In Rust, the evaluator shows the run, with the type of every intermediate term and the derivation of every step:

```
let add_two = lam(Ty::Nat, add(Term::Var(0), Term::Lit(2)));
let program = app(app(twice(), add_two), Term::Lit(3));
println!("== One run, step by step ==");
show_run(Rules::Sound, &program);
```

```
== One run, step by step ==
   (λ:(nat → nat). λ:nat. #1 (#1 #0)) (λ:nat. #0 + 2) 3  : nat
→  (λ:nat. (λ:nat. #0 + 2) ((λ:nat. #0 + 2) #0)) 3  : nat
      by E-AppL(E-Beta)
→  (λ:nat. #0 + 2) ((λ:nat. #0 + 2) 3)  : nat
      by E-Beta
→  (λ:nat. #0 + 2) (3 + 2)  : nat
      by E-AppR(E-Beta)
→  (λ:nat. #0 + 2) 5  : nat
      by E-AppR(E-AddLit)
→  5 + 2  : nat
      by E-Beta
→  7  : nat
      by E-AddLit
   (a value)
```

Read the derivations. In the first step, `E-AppL(E-Beta)`, the function part `twice (λ:nat. #0 + 2)` is itself a β-redex, so E-AppL descends into it and E-Beta fires there, replacing both occurrences of $\#1$ by the argument. The second step is a plain β at the top. The third, `E-AppR(E-Beta)`, evaluates the argument of the outer call first, as call-by-value demands, and so on down to `7`. Every line has type `nat`: preservation, visibly.

In Lean, the same program, its type, its run and its result:

```
/-- `λ (x : nat). x + 2` -/
def addTwo : Term := .lam .nat (.add (.var 0) (.lit 2))

/-- `twice (λ (x : nat). x + 2) 3` -/
def program : Term := .app (.app twice addTwo) (.lit 3)

/-- info: (λ:(nat → nat). λ:nat. #1 (#1 #0)) (λ:nat. #0 + 2) 3 -/
#guard_msgs in
#eval program

theorem program_typed : HasType [] program .nat := typeOf_sound rfl
```

```
/-- The run of `program`, one rule instance per step. Lean computes each
intermediate term itself (by unfolding `instantiate`), and checks that the
rules apply. -/
theorem program_runs : Steps program (.lit 7) :=
  .step (.appL (.beta .lam)) <|   -- twice addTwo 3  →  (λ:nat. addTwo (addTwo #0)) 3
```

```
  .step (.beta .lit) <|           --                → addTwo (addTwo 3)
  .step (.appR .lam (.beta .lit)) <| --             → addTwo (3 + 2)
  .step (.appR .lam .addLit) <|   --                → addTwo 5
  .step (.beta .lit) <|           --                → 5 + 2
  .step .addLit <|                --                → 7
  .refl

/-- Safety and determinism together: `program` never gets stuck, and `7` is the
only value it can produce. -/
theorem program_result {v : Term} (h : Steps program v) (hv : Value v) :
    v = .lit 7 :=
  Steps.value_unique h program_runs hv .lit
```

The proof of `program_runs` is the Rust trace, written as a proof term. The derivation `E-AppL(E-Beta)` is `.appL (.beta .lam)`: the argument of `.beta` is the proof that the argument of the call, `addTwo`, is a value. The operator `<|` applies the function on its left to everything on its right, and saves us six levels of nested parentheses: `f <| g <| x` means `f (g x)`. Lean computes each intermediate term itself, by unfolding `instantiate`, and checks that each rule applies; if the trace were wrong in any detail, the proof would fail. Finally, `program_result` combines the run with determinism: since `program` reaches `7` and a term has at most one value, `7` is the only value it can ever produce.

9.12 What "going wrong" means for Rust

Our language goes wrong in one way: it gets stuck. Real languages are more complicated, and it is worth asking what the analogue of a stuck state is for Rust, and what the analogue of the safety theorem promises.

Rust has no official formal semantics, so there is no theorem to quote. But the language reference does say what counts as going wrong: **undefined behavior**. Reading freed or uninitialized memory, a data race, producing an invalid value (a `bool` that is neither `true` nor `false`, a null or dangling reference) are all undefined behavior. These are Rust's stuck states: situations to which the language assigns no meaning at all. Safe Rust's promise, the analogue of our safety theorem, is that code without `unsafe` blocks cannot cause undefined behavior, provided the `unsafe` code inside the libraries it uses is correct. (For a formal core of Rust and several standard library types, that promise has been proved by the RustBelt project, with a technique quite different from progress and preservation; Chapter 32 is about it.)

Equally important is what does *not* count as going wrong.

- **Panics.** Indexing out of bounds, `unwrap` on `None`, integer overflow in a debug build, an explicit `panic!`: these are all defined behavior (the thread unwinds, running destructors, or the process aborts). In the vocabulary of this chapter, a panic is not a stuck state but a transition to an explicit *error* state, like the `error` term that textbooks add to a language with exceptions; progress then says "a value, a step, or an error".
- **Non-termination.** `loop {}` never gets stuck: it takes steps forever, and a diverging program is perfectly safe. Our safety theorem would survive the addition of general recursion to the language (with one more case in each proof); strong normalization is a separate, stronger property, which Rust does not have.

- **Leaks.** Forgetting to free memory (with `mem::forget`, or a cycle of `Rc` pointers) is safe. Chapter 24 explains why Rust's designers had to accept this.
- **Deadlocks** and **logic errors**. Two threads waiting for each other's locks are stuck in an informal sense, but every thread is in a well-defined state. And a program that computes the wrong answer is, as far as the type system is concerned, a correct program for a different specification.

So "well-typed programs cannot go wrong" is true for safe Rust, with "wrong" meaning undefined behavior. It is a strong and valuable guarantee, but a narrow one, and knowing its edges is part of using the language well. To say more about a program you need a stronger type (Chapter 30) or a proof about the program itself (Chapter 31).

9.13 Common confusions

"**Progress means the program terminates.**" Progress says that a well-typed program that has not finished can take *one more step*. A program can take one more step forever. Termination is a separate property: our language happens to have it, Rust does not, and safety does not depend on it.

"**Preservation means the term does not change.**" The term changes at every step; its *type* does not. `5 + 2` and `7` are different terms of the same type.

"**Safety means the result is correct.**" Safety means the result is a value of the promised type. `λ:nat. #0 + 1` and `λ:nat. #0 + 2` have the same type, and the type system cannot tell them apart.

"**The tests passed ten thousand times, so the theorem holds.**" The tests passed for ten thousand terms drawn from a particular distribution. The theorem holds for all terms because of the proof. The two statements are different, and the second one does not follow from the first. (The converse also fails: the proof does not show that the Rust code is correct.)

"**De Bruijn indices are an implementation trick.**" They are a design decision about the *mathematics*: they make α-equivalent terms equal, which removes a whole family of side conditions about fresh names from the substitution lemma.

9.14 Summary

- A small-step operational semantics is an inductive relation $t \to t'$ with computation rules (E-Beta, E-IfTrue, ...) and congruence rules (E-AppL, E-AppR, ...). The premises "value" in the congruence rules fix the evaluation order and make evaluation deterministic.
- A stuck term is neither a value nor able to step. Type safety says that a closed, well-typed term never reaches one.
- **Progress** is proved by induction on the typing derivation, with one canonical forms lemma per type ("a value of type bool is true or false").
- **Preservation** is proved by induction on the step with inversion of the typing derivation; its only hard case is E-Beta, which needs the **substitution lemma**, which in turn needs the **renaming lemma** (weakening is its special case).

- With de Bruijn indices, renaming and substitution are functions on indices, lifted under binders, and the lemmas about them are short inductions. All of it is checked by Lean, with no axioms.
- Determinism is independent of types; with it, a term has at most one value.
- Property-based tests check the same theorems on concrete terms, generated by running the typing rules backwards. They find bugs quickly and give concrete, shrinkable counterexamples; proofs cover all terms but are about a model.
- For Rust, "going wrong" means undefined behavior. Safe Rust rules it out; it does not rule out panics, non-termination, leaks or deadlocks.

9.15 Exercises

1. Write, on paper, the full derivation tree of the step $(\lambda : \mathit{nat}.\, \#0 + 2)\,(3 + 2) \to (\lambda : \mathit{nat}.\, \#0 + 2)\,5$, and the typing derivations of the terms on both sides. *Hint:* the step is E-AppR, whose premises are "the λ is a value" and the step $3 + 2 \to 5$, which is E-AddLit. Compare with the Lean term `.appR .lam .addLit` in `program_runs`.
2. Suppose we dropped the premise "v_1 value" from E-AppR. Find a term with two different next steps, so that determinism fails. Does safety still hold? *Hint:* take an application in which both the function and the argument can step, such as $((\lambda : \mathit{nat}.\, \lambda : \mathit{nat}.\, \#0)\,1)\,(2 + 3)$. For safety, look at which lemmas the proof of preservation uses: none of them mentions evaluation order.
3. Prove in Lean that a run can be continued inside a context: `theorem Steps.addL {t t' u : Term} (h : Steps t t') : Steps (.add t u) (.add t' u)`. Add it to a scratch copy of `Semantics.lean`. *Hint:* induction on `h`. The `refl` case is `.refl`; in the `step` case, combine `.addL hstep` with the induction hypothesis.
4. Prove that types are unique: `theorem HasType.unique {Γ t τ₁ τ₂} (h₁ : HasType Γ t τ₁) (h₂ : HasType Γ t τ₂) : τ₁ = τ₂`. *Hint:* induction on `h₁` generalizing `τ₂`, then `cases h₂`. In the `var` case, rewrite one `lookup` equation with the other and use `Option.some.inj`. In the `app` case, the induction hypothesis for the function gives an equation between two arrow types, and `cases` on it (or `Ty.arrow.inj`) gives the equation between their results.
5. Break a different rule in the Rust crate: make T-IsZero return `nat` instead of `bool` (add a variant to `Rules`). Which theorem does the harness report as violated, and what does the shrunk counterexample look like? Then state and prove the corresponding `bad_..._unsafe` theorem in Lean. *Hint:* `iszero 0` would then have type `nat` but step to `true`. Expect a preservation failure on a term of two or three constructors; wrapping it in `+ 1` gives a stuck term.
6. Extend the language with a boolean negation `not t`, with rules $\mathit{not}\ \mathit{true} \to \mathit{false}$, $\mathit{not}\ \mathit{false} \to \mathit{true}$ and a congruence rule, in both Lean and Rust. Update every definition and proof until everything compiles and the harness passes again. *Hint:* Lean will tell you exactly which cases are missing: each `match` and each induction needs one more case, and progress needs `canonical_bool` once more. Count how many of the new cases are routine.

CHAPTER 10

The Curry–Howard Correspondence

In Chapter 1 we read the type of a small Rust function as a sentence of logic: `fn swap<A, B>((a, b): (A, B)) -> (B, A)` says "for all A and B, if A and B, then B and A", and its body is a proof. At the time this was a promise of things to come. Since then you have seen natural deduction (Chapter 4), the constructive reading of proofs (Chapter 5), the structural rules (Chapter 6), the λ-calculus and its simply typed version (Chapters 7 and 8), and a complete proof that well-typed programs do not get stuck (Chapter 9). We now have everything we need to make the promise precise.

The **Curry–Howard correspondence** says that the typing rules of the simply typed λ-calculus and the rules of intuitionistic natural deduction are the same rules. Erase the terms from a typing derivation and you get a proof; decorate a proof with terms and you get a program. Propositions are types, proofs are programs, and simplifying a proof is running a program. This chapter works out the dictionary connective by connective, in logic, in Rust and in Lean, and then looks hard at where the dictionary breaks down. Rust turns out to be an *inconsistent* logic, in which every proposition has a proof, and also a logic that refuses to use a hypothesis twice. Lean is consistent, and it separates propositions from data in a way that has surprising consequences: a proof that a number exists does not let you compute the number.

In this chapter you will learn:

- the Curry–Howard dictionary: implication, conjunction, disjunction, truth, falsity, negation and the quantifiers, as types in Rust and Lean;
- why normalizing a proof is evaluating a program, and why that makes a terminating language a consistent logic;
- why Rust is inconsistent as a logic (`loop {}`, `panic!`, `todo!`), how Lean stays consistent (totality, `partial`, `sorry` and `#print axioms`), and where Rust makes the logic *more* honest than Lean: contraction needs `Clone`;
- the difference between `Prop` and `Type`, proof irrelevance, and the impredicativity of `Prop`;
- `Exists`, `Subtype`, `PSigma` and `Sigma`, and why the witness of an existential cannot be extracted into data;
- `theorem` versus `def`, `Nat.le` as an inductive family, and how to see the term behind a tactic proof.

10.1 Erase the terms

Put two rules next to each other: implication introduction from natural deduction (Chapter 4), written with a context, and rule T-Lam of the simply typed λ-calculus (Chapter 8).

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B} \to \mathrm{I} \qquad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.\, t : A \to B}\ \textsf{T-Lam}$$

The right-hand rule is the left-hand rule with terms attached. Erase the variable name x and the terms t and $\lambda x.\, t$, and what remains is $\to$ I exactly: to prove $A \to B$, assume A and prove B. Now application and modus ponens:

$$\frac{\Gamma \vdash A \to B \qquad \Gamma \vdash A}{\Gamma \vdash B} \to \mathrm{E} \qquad \frac{\Gamma \vdash f : A \to B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\, a : B}\ \textsf{T-App}$$

Again the same rule, with and without terms. And pairs:

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge \mathrm{I} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B}\ \textsf{T-Pair}$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge \mathrm{E}_1 \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \mathit{fst}\, p : A}\ \textsf{T-Fst}$$

The pattern continues through every rule of the simply typed λ-calculus with pairs, sums, unit and the empty type: the introduction rules of a connective are the constructors of a type, and its elimination rules are the ways of using a value of that type. The precise statement has two halves.

1. **Erasure.** If $\Gamma \vdash t : \tau$ is derivable in the simply typed λ-calculus, then $|\Gamma| \vdash \tau$ is derivable in intuitionistic natural deduction, where $|\Gamma|$ forgets the variable names and the type formers are read as connectives ($\times$ as $\wedge$, $+$ as $\vee$, and so on).
2. **Decoration.** Conversely, every natural-deduction derivation of $|\Gamma| \vdash \tau$ can be decorated with terms to give a typing derivation, and different derivations give different terms.

So a proof *is* a term, recorded in the shape of its derivation, and a term *is* a proof. Haskell Curry noticed the correspondence for combinatory logic and implication in the 1930s; William Howard extended it to natural deduction and the λ-calculus in a manuscript written in 1969 (and published in 1980), which gave the correspondence its second name. Since then it has grown far beyond propositional logic: Lean itself, with its dependent types, is a direct descendant.

Each reading illuminates the other. A logically unsound rule, such as Prior's `tonk` from Chapter 4, corresponds to a type former whose elimination does not match its introduction, and for programs that use it preservation fails. A proof that uses a hypothesis twice corresponds to a program that uses a variable twice. That last fact will matter a great deal in Rust.

10.2 The dictionary

Logic	Type theory	Rust	Lean
$A \wedge B$	$A \times B$	`(A, B)`	`p ∧ q`, `α × β`
$A \vee B$	$A + B$	`enum Either<A, B>`	`p ∨ q`, `α ⊕ β`
$A \to B$	$A \to B$	`impl FnOnce(A) -> B`	`p → q`, `α → β`
$\top$	$\mathbf{1}$	`()`	`True`, `Unit`
$\bot$	$\mathbf{0}$	`enum Void {}`	`False`, `Empty`
$\neg A$	$A \to \mathbf{0}$	`impl FnOnce(A) -> Void`	`¬p`
$\forall x.\, P(x)$	Π-type	generics, over types only	`∀ x, P x`
$\exists x.\, P(x)$	Σ-type	`impl Trait`, over types only	`∃ x, P x`

The Lean column has two entries in most rows, because Lean keeps two parallel versions of every connective: one for propositions (`∧`, `∨`, `True`, `False`) and one for data (`×`, `⊕`, `Unit`, `Empty`). The reason for the split is the subject of the second half of this chapter; for now, treat them as twins.

The Rust code of this chapter is in `code/rust/examples/ch10_proofs_as_programs.rs` and the Lean code in `code/lean/Book/Ch10/Dictionary.lean` and `code/lean/Book/Ch10/PropVsType.lean`. We go through the dictionary one row at a time.

The connectives in Rust

Rust already has pairs, functions and the unit type. For disjunction and falsity we define our own types, as in Chapter 5:

```
/// Falsity, ⊥: a type with no values at all.
enum Void {}

/// Disjunction, A ∨ B: a value of one of two types, tagged with which.
#[derive(Debug, PartialEq)]
enum Either<A, B> {
    Left(A),
    Right(B),
}
// Conjunction A ∧ B is the pair (A, B); truth ⊤ is the unit type ();
// implication A → B is a function; negation ¬A is a function A → Void.
```

Conjunction is the pair. Its introduction rule builds a pair, its elimination rules project out a component, and commutativity is the swap of Chapter 1:

```
/// ∧-introduction: from proofs of A and B, a proof of A ∧ B.
fn and_intro<A, B>(a: A, b: B) -> (A, B) {
    (a, b)
}

/// ∧-elimination: from A ∧ B, a proof of A (and, symmetrically, of B).
fn and_elim_left<A, B>(pair: (A, B)) -> A {
    pair.0
}

/// A ∧ B → B ∧ A
fn and_comm<A, B>((a, b): (A, B)) -> (B, A) {
    (b, a)
}
```

Disjunction is the tagged union. Introduction picks a side and attaches the tag; elimination is a `match`, which needs one branch for each side, exactly as $\vee$ E needs a proof of C from A and a proof of C from B. Chapter 4 called $\vee$ E strange; in Rust it is the most familiar thing in the world.

```
/// ∨-introduction on the left: from A, a proof of A ∨ B.
fn or_intro_left<A, B>(a: A) -> Either<A, B> {
    Either::Left(a)
}

/// ∨-elimination: if A → C and B → C, then A ∨ B → C. It is a `match`.
fn or_elim<A, B, C>(
    e: Either<A, B>,
    if_left: impl FnOnce(A) -> C,
    if_right: impl FnOnce(B) -> C,
) -> C {
    match e {
        Either::Left(a) => if_left(a),
        Either::Right(b) => if_right(b),
    }
}

/// A ∨ B → B ∨ A
fn or_comm<A, B>(e: Either<A, B>) -> Either<B, A> {
    or_elim(e, Either::Right, Either::Left)
}
```

Notice that `or_comm` passes the tuple-variant constructors `Either::Right` and `Either::Left` as functions: an enum constructor *is* an introduction rule, and Rust lets you use it as an ordinary function.

Implication is the function type. Modus ponens is a call, the hypothetical syllogism is composition, and the curried and uncurried forms of a two-premise implication are interchangeable:

```
/// →-elimination (modus ponens): from A → B and A, a proof of B.
fn modus_ponens<A, B>(f: impl FnOnce(A) -> B, a: A) -> B {
    f(a)
}

/// (A → B) → (B → C) → (A → C): composition.
fn compose<A, B, C>(
    f: impl FnOnce(A) -> B,
    g: impl FnOnce(B) -> C,
) -> impl FnOnce(A) -> C {
    move |a| g(f(a))
}

/// (A → B → C) → (A ∧ B → C): uncurrying.
fn uncurry<A, B, C, G>(f: impl FnOnce(A) -> G) -> impl FnOnce((A, B)) -> C
where
    G: FnOnce(B) -> C,
{
    move |(a, b)| f(a)(b)
}

/// A ∧ (B ∨ C) → (A ∧ B) ∨ (A ∧ C): distributivity.
fn distribute<A, B, C>((a, bc): (A, Either<B, C>)) -> Either<(A, B), (A, C)> {
    match bc {
        Either::Left(b) => Either::Left((a, b)),
        Either::Right(c) => Either::Right((a, c)),
    }
}
```

We use `FnOnce` for implications, the most permissive of Rust's closure traits: a proof of $A \to B$ may be used once, to turn one proof of A into one proof of B. That choice will come back when we discuss contraction. (Writing currying in the other direction, from `(A, B) -> C` to a function returning

a function, runs into a limitation of Rust: `impl Trait` is not allowed in the return type of an `Fn` bound, error E0562, so the nested closure needs a `Box`. Chapter 8 discussed the cost of currying in Rust.)

Truth, falsity and negation. Truth is the unit type: a proof of $\top$ carries no information, and `()` is the one value that carries none. Falsity is a type with no values. Negation $\neg A$ is $A \to \bot$, a function that would turn a proof of A into a proof of falsity, which is to say, a refutation of A:

```
/// ⊥ → A: ex falso quodlibet. A `match` with no arms covers every case of a
/// type with no values, and so it can have any type.
fn ex_falso<A>(v: Void) -> A {
    match v {}
}

/// A → ¬¬A: given A, any refutation of A can be refuted.
fn double_negation_intro<A, N>(a: A) -> impl FnOnce(N) -> Void
where
    N: FnOnce(A) -> Void,
{
    move |not_a| not_a(a)
}

/// (A → B) → (¬B → ¬A): contraposition.
fn contrapositive<A, B, F, NB>(f: F, not_b: NB) -> impl FnOnce(A) -> Void
where
    F: FnOnce(A) -> B,
    NB: FnOnce(B) -> Void,
{
    move |a| not_b(f(a))
}

/// ¬(A ∧ ¬A): no contradiction.
fn non_contradiction<A, N>((a, not_a): (A, N)) -> Void
where
    N: FnOnce(A) -> Void,
{
    not_a(a)
}

/// ¬A ∧ ¬B → ¬(A ∨ B): each refutation is used at most once.
fn de_morgan_and<A, B, NA, NB>(
    not_a: NA,
    not_b: NB,
) -> impl FnOnce(Either<A, B>) -> Void
where
    NA: FnOnce(A) -> Void,
    NB: FnOnce(B) -> Void,
{
    move |e| match e {
        Either::Left(a) => not_a(a),
        Either::Right(b) => not_b(b),
    }
}
```

`ex_falso` is the rule $\bot$E: from falsity, anything. Its body is a `match` with *no arms*. A `match` must cover every value of the scrutinee's type, `Void` has no values, so zero arms cover them all, and the match can be given any type at all. This is the same reasoning that lets the never type `!` coerce to every type (Chapter 1): code that has a value of an empty type is code that can never run.

Here is what the positive part of the dictionary does when it runs:

```
let show = |label: &str, value: String| println!("{label:<28} = {value}");
show(
    "and_comm((1, \"one\"))",
    format!("{:?}", and_comm((1, "one"))),
```

```
);
show(
    "and_elim_left(('a', 2.5))",
    format!("{:?}", and_elim_left(and_intro('a', 2.5))),
);
let e: Either<u8, &str> = or_intro_left(7);
show("or_comm(Left(7))", format!("{:?}", or_comm(e)));
let len_is_even = compose(|s: &str| s.len(), |n: usize| n % 2 == 0);
show(
    "compose(len, even)(\"four\")",
    format!("{}", len_is_even("four")),
);
show(
    "modus_ponens(double, 21)",
    format!("{}", modus_ponens(|x: i32| 2 * x, 21)),
);
let add = uncurry(|a: i32| move |b: i32| a + b);
show("uncurry(add)((2, 3))", format!("{}", add((2, 3))));
let pair = (1, Either::<bool, char>::Right('c'));
show(
    "distribute((1, Right('c')))",
    format!("{:?}", distribute(pair)),
);
show(
    "dup(String::from(\"x\"))",
    format!("{:?}", dup(String::from("x"))),
);
show("identity(\"same\")", format!("{:?}", identity("same")));
show("some_displayable()", format!("{}", some_displayable()));
```

```
and_comm((1, "one"))          = ("one", 1)
and_elim_left(('a', 2.5))     = 'a'
or_comm(Left(7))              = Right(7)
compose(len, even)("four")    = true
modus_ponens(double, 21)      = 42
uncurry(add)((2, 3))          = 5
distribute((1, Right('c')))   = Right((1, 'c'))
dup(String::from("x"))        = ("x", "x")
identity("same")              = "same"
some_displayable()            = 42
```

The negation proofs are not in the output, and cannot be. To call a refutation of A you need a proof of A, and then the refutation would return a value of type `Void`, which does not exist. Those functions are checked by the compiler and never run. That is the essence of propositions as types: *type checking is proof checking*, and running the program is beside the point. (It is not beside the point for the positive connectives, as the next section shows.)

The connectives in Lean

In Lean the same proofs are written as terms, and Lean's syntax makes them shorter. `⟨hp, hq⟩` is the anonymous constructor (here, of `And`), `fun ⟨hp, hq⟩ => ...` takes a pair apart in its argument, and `.inl` / `.inr` are the two constructors of `Or`. The theorem and its twin on data are the same program:

```
/-- ∧-introduction: a pair of proofs. -/
theorem and_intro (hp : p) (hq : q) : p ∧ q := ⟨hp, hq⟩

/-- ∧-elimination: a projection. -/
theorem and_left : p ∧ q → p := fun h => h.1

/-- p ∧ q → q ∧ p -/
theorem and_swap : p ∧ q → q ∧ p := fun ⟨hp, hq⟩ => ⟨hq, hp⟩
```

```
/-- The same program on data: a pair of values, swapped. -/
def swap : α × β → β × α := fun ⟨a, b⟩ => ⟨b, a⟩
```

```
/-- info: Ch10.and_swap (p q : Prop) : p ∧ q → q ∧ p -/
#guard_msgs in
#check and_swap

/-- info: Ch10.swap (α β : Type) : α × β → β × α -/
#guard_msgs in
#check swap
```

The file starts with `variable (p q r : Prop)` and `variable (α β γ : Type)`, which add these parameters to every declaration that mentions them; that is why `#check` shows `and_swap` taking `p` and `q`. Compare the two signatures: one says `Prop` and `∧`, the other `Type` and `×`, and the bodies are identical. The same holds for disjunction, where the elimination rule is a `match`:

```
/-- ∨-elimination is a `match` with one arm per side. -/
theorem or_swap : p ∨ q → q ∨ p := fun h =>
  match h with
  | .inl hp => .inr hp
  | .inr hq => .inl hq

/-- The same program on data, with `Sum` (Rust's `Either`, or `Result`). -/
def sumSwap : α ⊕ β → β ⊕ α := fun s =>
  match s with
  | .inl a => .inr a
  | .inr b => .inl b
```

and for implication, where every proof is a λ-term:

```
/-- →-elimination, modus ponens: application. -/
theorem modus_ponens : (p → q) → p → q := fun f hp => f hp

/-- Hypothetical syllogism: composition. -/
theorem syllogism : (p → q) → (q → r) → (p → r) := fun f g hp => g (f hp)

/-- Currying, in both directions: an `Iff` is a pair of implications. -/
theorem curry_iff : (p ∧ q → r) ↔ (p → q → r) :=
  ⟨fun f hp hq => f ⟨hp, hq⟩, fun g ⟨hp, hq⟩ => g hp hq⟩

/-- Distributivity of ∧ over ∨. -/
theorem distrib : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := fun ⟨hp, hqr⟩ =>
  match hqr with
  | .inl hq => .inl ⟨hp, hq⟩
  | .inr hr => .inr ⟨hp, hr⟩
```

`curry_iff` proves an equivalence, and an equivalence `↔` is a pair of implications, built with the anonymous constructor like any other pair. The two directions are the functions `curry` and `uncurry`.

For negation, Lean *defines* `¬p` as `p → False`, and the definitional equation is checked by `rfl`:

```
/-- ⊤ is the proposition with one trivial proof, like the unit type. -/
theorem truth : True := trivial

/-- Ex falso quodlibet: `False` has no constructors, so a `match` on a proof
of `False` needs no arms. -/
theorem ex_falso : False → p := fun h => nomatch h

/-- The same for data: `Empty` is Lean's `enum Void {}`. -/
def emptyElim : Empty → α := fun e => nomatch e

/-- `¬p` is *defined* as `p → False`. -/
```

```
example : (¬p) = (p → False) := rfl

theorem double_negation_intro : p → ¬¬p := fun hp hnp => hnp hp

theorem contrapositive : (p → q) → ¬q → ¬p := fun f hnq hp => hnq (f hp)

theorem non_contradiction : ¬(p ∧ ¬p) := fun ⟨hp, hnp⟩ => hnp hp
```

`nomatch h` is Lean's `match h {}`: a match with no cases on a value of an empty type. `False` is an inductive proposition with no constructors, `Empty` an inductive type with no constructors, and `nomatch` works for both.

10.3 Quantifiers

The propositional connectives are where Rust and Lean agree. With the quantifiers they part ways, because a quantifier ranges over something, and the two languages disagree about what types may mention.

Universal quantification is a dependent function

In Lean, a proof of $\forall n : \mathbb{N}, n + 0 = n$ is a function that takes a number n and returns a proof about *that* n:

```
/-- ∀ over values: a *dependent* function. Given `n`, it returns a proof about
that very `n`; the result type depends on the argument. -/
theorem add_zero_all : ∀ n : ℕ, n + 0 = n :=
  fun n => show n + 0 = n from rfl

/-- info: add_zero_all 5 : 5 + 0 = 5 -/
#guard_msgs in
#check add_zero_all 5

/-- ∀ over types: a generic function, Rust's `fn identity<A>(a: A) -> A`. -/
def identity (α : Type) : α → α := fun a => a

/-- ∀ over propositions: the same program, read as a theorem. -/
theorem self_implication : ∀ (p : Prop), p → p := fun _ hp => hp
```

The type of the result, `n + 0 = n`, mentions the argument `n`. A function whose result type depends on the value of its argument is a **dependent function**, and its type is called a Π-type. Applying it to `5` gives a proof of `5 + 0 = 5`, as `#check` shows. When the bound variable ranges over types instead of values, as in `identity`, the dependent function is a generic function; when it ranges over propositions, as in `self_implication`, it is a proof that works for every proposition. In Lean these are three uses of one construct.

Rust has the second one and not the first. A generic function `fn identity<A>(a: A) -> A` is a proof of $\forall A.\, A \to A$, quantifying over types (Chapter 12 studies this as System F):

```
/// ∀A. A → A: a generic function is a proof for every type at once.
fn identity<A>(a: A) -> A {
    a
}

/// ∃T. T: Display. The caller learns that *some* displayable type comes back,
/// not which one: the witness (here `u32`) is hidden.
fn some_displayable() -> impl std::fmt::Display {
```

```
    42_u32
}
```

But a Rust type cannot mention a run-time value. There is no Rust type that says "a `u32` equal to `n + 0`" for an argument `n`, and therefore no Rust function whose type is the theorem $\forall n.\, n + 0 = n$. Const generics (`[T; N]`) allow a few compile-time values in types, and Chapter 30 shows how far that goes, but they cannot express properties of the values a program computes. In the dictionary, Rust's quantifiers range over types only.

Existential quantification is a dependent pair

A proof of $\exists n, n > 100$ is a pair: a witness, and a proof that the witness has the property. The type of the second component depends on the value of the first, which makes this a **dependent pair**, a Σ-type:

```
/-- ∃ is a dependent pair: a witness, and a proof about that witness. -/
theorem exists_big : ∃ n : ℕ, n > 100 := ⟨101, by decide⟩

/-- Using an existential: take the pair apart and use both components. -/
theorem exists_bigger (h : ∃ n : ℕ, n > 100) : ∃ m : ℕ, m > 101 := by
  obtain ⟨n, hn⟩ := h      -- n : ℕ, hn : n > 100
  exact ⟨n + 1, by omega⟩  -- the new witness is n + 1
```

To use an existential, you take the pair apart (`obtain ⟨n, hn⟩ := h`, as in Chapter 4) and use the witness and its property. Here we build a new witness, `n + 1`, and `omega` proves that it is greater than 101.

Rust's closest relative is `impl Trait` in return position. The signature of `some_displayable` above says "there exists a type `T` such that `T: Display`, and here is a value of it". The caller cannot see the witness type (it happens to be `u32`), only the promised property. Chapter 15 studies this in depth. As with $\forall$, Rust's $\exists$ ranges over types, never over values.

10.4 Normalization is computation

Chapter 4 introduced the idea of a **detour** in a proof: an introduction rule immediately followed by the elimination rule of the same connective. Proving $A \wedge B$ by $\wedge\,\mathrm{I}$ and then using $\wedge\,\mathrm{E}_1$ to get A back is a detour; the proof of A we started from would have done. Chapter 4 showed that every detour can be removed (local soundness, or harmony), and that repeatedly removing detours *normalizes* a proof.

Under Curry–Howard, a detour is a **redex**. $\wedge\,\mathrm{I}$ followed by $\wedge\,\mathrm{E}_1$ is $\mathit{fst}\,(a, b)$, which reduces to a. $\to\mathrm{I}$ followed by $\to\mathrm{E}$ is $(\lambda x.\, t)\, a$, which β-reduces to $t[a]$. Removing a detour is taking a step of computation, and normalizing a proof is running a program to its normal form:

```
/-- A program with a detour: build a pair, then immediately take it apart. As
a proof, this is ∧-introduction followed at once by ∧-elimination. -/
def detour (a : α) : α := (fun pair : α × α => pair.1) (a, a)

-- `#reduce` normalizes: the β-redex and the projection disappear.
/-- info: fun α a => a -/
#guard_msgs in
#reduce detour
```

```
/-- The same detour as a proof. -/
theorem detour_proof (hp : p) : p := (fun h : p ∧ p => h.1) ⟨hp, hp⟩
```

`#reduce` normalizes a term, and it turns `detour` into the identity function: the β-redex and the projection are gone. In Rust, `(|pair: (A, A)| pair.0)((a, a))` is the same detour, and the optimizer typically removes it when it inlines the closure call. (We do not `#reduce` the proof `detour_proof`: Lean does not unfold theorems, for reasons that the section on `Prop` will make clear.)

This connection has a deep consequence. Suppose a language has an empty type $\mathbf{0}$, and suppose every closed term has a normal form. Could there be a closed term $t : \mathbf{0}$? Normalize it: by preservation (Chapter 9), the normal form still has type $\mathbf{0}$; by progress, a closed normal form is a value; and by canonical forms, there are no values of type $\mathbf{0}$, since the empty type has no introduction rule. So there is no such t. Read as logic: there is no proof of $\bot$. **A terminating, type-safe language is a consistent logic.** The argument uses exactly the three ingredients of Chapter 9, plus termination, and termination is the one that real programming languages give up.

10.5 Rust is inconsistent as a logic

Here are proofs, in Rust, of every proposition there is:

```
/// "Proofs" of every proposition. Each one type-checks.
#[allow(dead_code)]
fn by_looping<A>() -> A {
    loop {}
}

fn by_panicking<A>() -> A {
    panic!("this proof has nothing inside")
}

#[allow(dead_code)]
fn by_todo<A>() -> A {
    todo!()
}

#[allow(dead_code, unconditional_recursion)]
fn by_recursion<A>() -> A {
    by_recursion() // the compiler warns (a lint), but accepts
}

/// A "proof" of falsity. It compiles.
#[allow(dead_code)]
fn proof_of_false() -> Void {
    by_looping()
}
```

Each function claims, for every type `A`, to produce a value of type `A`, and each one type-checks. `loop {}` and `panic!` have the never type `!`, which coerces to `A` (Chapter 1); `todo!()` is a `panic!` with a standard message; `by_recursion` calls itself forever. The compiler does notice the last one and emits the `unconditional_recursion` warning, but a warning is a lint, not a rejection, and the `#[allow(...)]` attribute silences it. (The `dead_code` attributes only silence warnings about functions that the example never calls.) And `proof_of_false` returns a `Void`, a value of a type that has no values: a proof of $\bot$, accepted by the compiler.

None of these functions ever returns. When you run one, the "proof" turns out to be empty:

```rust
// Running a "proof" by panicking: the program compiled, but the proof was
// never there. (The panic message is silenced; we only report it.)
panic::set_hook(Box::new(|_| {}));
let outcome = panic::catch_unwind(|| {
    let n: u32 = by_panicking();
    n
});
match outcome {
    Ok(n) => println!("by_panicking::<u32>() returned {n}"),
    Err(_) => {
        println!("by_panicking::<u32>() panicked: no value was ever produced")
    }
}
```

```
by_panicking::<u32>() panicked: no value was ever produced
```

So as a logic, Rust is **inconsistent**: every proposition, including $\bot$, has a proof. This is not a design mistake. A general-purpose language needs unbounded loops and recursion (a server runs forever by design), and it needs a way to give up (`panic!`). Both break the termination half of the argument in the previous section, and with it consistency.

What survives is a weaker, still useful principle: *a total function is an honest proof*. If you know that a function always returns (no `panic!`, no infinite loop, no call to something that might do either), then its type is a true theorem, and its body is a proof. When we read Rust signatures as theorems in this book, that is the understanding: the reading is valid for the total part of the language. A well-known paper on Haskell, which has the same issue, put this in its title: "Fast and Loose Reasoning is Morally Correct" (2006).

10.6 Lean is consistent

Lean accepts none of the four tricks. A recursive definition must be seen to terminate. Here is `by_recursion` in Lean:

```
-- The Rust "proof" `fn by_recursion<A>() -> A { by_recursion() }`, in Lean:
-- rejected, because Lean cannot see why it would terminate.
/--
error: fail to show termination for
  Ch10.byRecursion
with errors
failed to infer structural recursion:
Not considering parameter p of Ch10.byRecursion:
  it is unchanged in the recursive calls
no parameters suitable for structural recursion

well-founded recursion cannot be used, `Ch10.byRecursion` does not take any (non-fixed)
arguments
-/
#guard_msgs in
def byRecursion (p : Prop) : p := byRecursion p
```

Lean tries two ways of justifying the recursion, as Chapter 2 explained: structural recursion (some argument gets smaller at each call), which fails because `p` is passed unchanged, and well-founded recursion (some measure decreases), which fails because there are no arguments that could decrease. So the definition is rejected.

Lean does have an escape hatch for programs that may not terminate: `partial`. But `partial` comes with a condition that makes it harmless for logic:

```
/-- `partial` turns off the termination check, but only for types that have a
value anyway: then the loop cannot be the only "proof". -/
partial def spin (n : ℕ) : ℕ := spin (n + 1)

-- A `partial` definition is opaque: Lean knows its type and nothing else.
/-- info: opaque Ch10.spin : ℕ → ℕ -/
#guard_msgs in
#print spin

-- For `False` there is no value to fall back on, and `partial` is refused.
/--
error: failed to compile 'partial' definition `Ch10.loopProof`, could not prove that the
type
  ∀ (n : ℕ), False
is nonempty.

This process uses multiple strategies:
- It looks for a parameter that matches the return type.
- It tries synthesizing 'Inhabited' and 'Nonempty' instances for the return type, while
making every parameter into a local 'Inhabited' instance.
- It tries unfolding the return type.

If the return type is defined using the 'structure' or 'inductive' command, you can try
adding a 'deriving Nonempty' clause to it.
-/
#guard_msgs in
partial def loopProof (n : ℕ) : False := loopProof n
```

A `partial` definition is compiled and can run forever, but in the logic it is an `opaque` constant: Lean knows its type and nothing about its body, so no theorem can be proved by unfolding it. And Lean accepts a `partial` definition only if it can show that its type is nonempty, so that an arbitrary value of that type could stand in for the loop. `spin` returns a `ℕ`, which has values; `loopProof` would return a proof of `False`, which has none, and the error message lists the strategies Lean tried to find one. So `partial` can never create a proof of something that had no proof before.

What about `sorry`? It is Lean's `todo!()`: a placeholder that has every type. Lean accepts it, but warns, and remembers:

```
-- `sorry` proves anything, like Rust's `todo!()`. Lean warns, and remembers.
/-- warning: declaration uses `sorry` -/
#guard_msgs in
theorem cheat : False := sorry

/-- info: 'Ch10.cheat' depends on axioms: [sorryAx] -/
#guard_msgs in
#print axioms cheat

/-- info: 'Ch10.de_morgan' does not depend on any axioms -/
#guard_msgs in
#print axioms de_morgan
```

`#print axioms` shows that `cheat` depends on `sorryAx`, the axiom behind `sorry`, and that axiom propagates to everything that uses `cheat`. A theorem whose `#print axioms` list does not contain `sorryAx` does not depend on any `sorry`, however deep in the library. Our `de_morgan` depends on no axioms at all. (Lean's standard axioms, `propext`, `Quot.sound` and `Classical.choice`, which you met in Chapter 1, are believed to be consistent with its type theory; `sorryAx` is not, which is exactly why it is tracked.) This is how a proof assistant can be both a programming language and a trustworthy logic:

the language may contain non-terminating programs and placeholders, but the kernel keeps track of which theorems they could have touched.

10.7 Contraction: where Rust makes the logic honest

So far Rust has been the less faithful of the two languages. In one respect it is the more faithful one, or at least the more revealing.

In Lean, a hypothesis can be used as many times as you like:

```
/-- ¬(p ∨ q) → ¬p ∧ ¬q. The hypothesis `h` is used twice, and nothing has to
be said about it: Lean's logic has contraction built in. -/
theorem de_morgan : ¬(p ∨ q) → ¬p ∧ ¬q :=
  fun h => ⟨fun hp => h (.inl hp), fun hq => h (.inr hq)⟩

/-- p → p ∧ p, and its twin on data, α → α × α. Both use their argument twice. -/
theorem dup : p → p ∧ p := fun hp => ⟨hp, hp⟩

def dupData : α → α × α := fun a => (a, a)
```

In `de_morgan`, the hypothesis `h` (a refutation of $p \vee q$) is used twice, once to refute p and once to refute q. In `dup`, `hp` is used twice to build a pair. Neither use needs any comment: as Chapter 6 explained, ordinary logic has the structural rule of **contraction**, which lets an assumption be used any number of times, and Lean's type theory has it built in.

Now write `dup` in Rust, for an arbitrary type `A`:

```
fn dup<A>(a: A) -> (A, A) {
    (a, a)
}
```

```
error[E0382]: use of moved value: `a`
 --> compile_fail/ch10_contraction.rs:7:9
  |
6 | fn dup<A>(a: A) -> (A, A) {
  |           - move occurs because `a` has type `A`, which does not implement the `Copy`
trait
7 |     (a, a)
  |      -  ^ value used here after move
  |      |
  |      value moved here
  |
help: if `A` implemented `Clone`, you could clone the value
 --> compile_fail/ch10_contraction.rs:6:8
  |
6 | fn dup<A>(a: A) -> (A, A) {
  |        ^ consider constraining this type parameter with `Clone`
7 |     (a, a)
  |      - you could clone this value
help: consider restricting type parameter `A` with trait `Copy`
  |
6 | fn dup<A: Copy>(a: A) -> (A, A) {
  |         ++++++
```

The value `a` is moved into the first component of the pair, and then it is gone. For an arbitrary type, Rust has no contraction. The compiler's two suggestions are the two ways of buying it back: require `A: Clone`, and duplicate the value explicitly with `.clone()`, or require `A: Copy`, and let the compiler duplicate it implicitly. The working `dup` in the example asks for `Clone`.

De Morgan's law shows the same thing in a less artificial setting. The proof needs its hypothesis twice, so the Rust version fails in the same way:

```
fn de_morgan_or<A, B, N>(n: N) -> (impl FnOnce(A) -> Void, impl FnOnce(B) -> Void)
where
    N: FnOnce(Either<A, B>) -> Void,
{
    (move |a| n(Either::Left(a)), move |b| n(Either::Right(b)))
}
```

```
error[E0382]: use of moved value: `n`
  --> compile_fail/ch10_de_morgan.rs:17:35
   |
13 | fn de_morgan_or<A, B, N>(n: N) -> (impl FnOnce(A) -> Void, impl FnOnce(B) -> Void)
   |                          - move occurs because `n` has type `N`, which does not
implement the `Copy` trait
...
17 |     (move |a| n(Either::Left(a)), move |b| n(Either::Right(b)))
   |      -------- -                   ^^^^^^^^ - use occurs due to use in closure
   |      |        |                   |
   |      |        |                   value used here after move
   |      |        variable moved due to use in closure
   |      value moved into closure here
   |
help: if `N` implemented `Clone`, you could clone the value
  --> compile_fail/ch10_de_morgan.rs:13:23
   |
13 | fn de_morgan_or<A, B, N>(n: N) -> (impl FnOnce(A) -> Void, impl FnOnce(B) -> Void)
   |                       ^ consider constraining this type parameter with `Clone`
...
17 |     (move |a| n(Either::Left(a)), move |b| n(Either::Right(b)))
   |               - you could clone this value
```

The first closure captures `n` by move (that is what `move` means), and the second closure has nothing left to capture. The fix in the example asks for `N: Clone` and makes the duplication visible:

```
/// ¬(A ∨ B) → ¬A ∧ ¬B. The hypothesis is needed twice, once in each
/// component: that is contraction, and Rust makes us ask for it with `Clone`.
fn de_morgan_or<A, B, N>(n: N) -> (impl FnOnce(A) -> Void, impl FnOnce(B) -> Void)
where
    N: FnOnce(Either<A, B>) -> Void + Clone,
{
    let n2 = n.clone(); // contraction, spelled out
    (move |a| n(Either::Left(a)), move |b| n2(Either::Right(b)))
}

/// A → A ∧ A, the purest form of contraction: it needs `Clone` too.
fn dup<A: Clone>(a: A) -> (A, A) {
    (a.clone(), a)
}
```

The line `let n2 = n.clone();` is contraction, written out. Where Lean's logic duplicates hypotheses silently, Rust makes you say so, and only lets you when the type supports it. Many types do: integers, shared references `&T` and function pointers are `Copy`, a closure is `Copy` when everything it captures is, and most library types are `Clone`. But a `String`, a file handle or a `Vec` cannot be duplicated for free, and an arbitrary generic `A` cannot be duplicated at all.

Weakening, the other structural rule, is still available in Rust: you may ignore a value, and it is dropped. Exchange is available too: the order in which you use variables does not matter to the type checker. A logic with weakening and exchange but without contraction is called **affine**. So the proofs that Rust's generic functions can express, for arbitrary types, are the proofs of an affine logic, and

`Clone` and `Copy` are the controlled way back to ordinary logic. This is the first piece of evidence for the question of Chapter 1. Part V builds the affine λ-calculus and makes the connection precise; Chapter 23 shows that `Copy` and `Clone` behave like the exponential modality $!A$ of linear logic.

10.8 `Prop` and `Type`

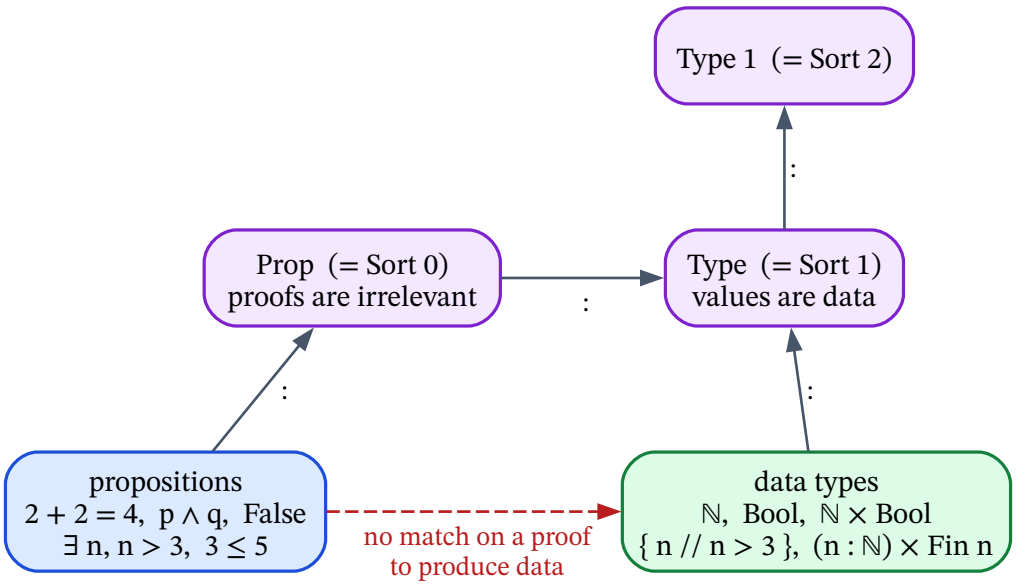
We now turn to the Lean side of the dictionary, and to the question we postponed: why does Lean have two versions of every connective?

Two universes

Every Lean type lives in a **universe**, a type of types. Propositions live in `Prop`, data types in `Type`:

```
/-- info: 2 + 2 = 4 : Prop -/
#guard_msgs in
#check 2 + 2 = 4

/-- info: ℕ : Type -/
#guard_msgs in
#check ℕ

/-- info: Prop : Type -/
#guard_msgs in
#check Prop

/-- info: Type : Type 1 -/
#guard_msgs in
#check Type
```

`Prop` and `Type` are themselves types, and they live in bigger universes: `Prop : Type` and `Type : Type 1`, and so on upward (`Type 1 : Type 2`, ...). In Lean's own notation, `Prop` is `Sort 0` and `Type` is `Sort 1`. Chapter 12 studies universes and why the hierarchy never stops; here we only need the bottom two levels.



Lean's universes: propositions are types whose elements are proofs and live in Prop; data types live in Type. Prop is itself a type, in Type, and Type is in Type 1. A proof cannot be taken apart to produce data.

The difference between the two is not what you can *write*: `p ∧ q` and `α × β` are built in the same way. It is what the elements *mean*. An element of a type in `Type` is a piece of data, which a program can inspect and compute with. An element of a proposition is a proof, and Lean treats all proofs of the same proposition as equal.

Proof irrelevance

Chapter 5 contrasted `Bool`, a data type with two values that a program can branch on, with `Prop`, the type of statements. Here is the deepest difference between them:

```
/-- A `Bool` is data: two different values that a program can inspect. -/
example : true ≠ false := by decide

/-- A proposition is a type whose values are proofs, and any two proofs of
the same proposition are equal, by definition: `rfl` proves it. -/
theorem proof_irrelevance (p : Prop) (h₁ h₂ : p) : h₁ = h₂ := rfl
```

`true ≠ false` holds for booleans. But for any proposition `p`, any two proofs `h₁ h₂ : p` are equal, and `rfl` proves it: they are equal *by definition*, without any argument. This is **definitional proof irrelevance**, and it is a rule of Lean's type theory. Two proofs built in completely different ways are the same proof:

```
/-- Two proofs of `3 ≤ 5`, built differently (one by a decision procedure, one
by arithmetic), are the same proof as far as Lean is concerned. -/
example : (by decide : 3 ≤ 5) = (by omega : 3 ≤ 5) := rfl
```

Proof irrelevance says that the *existence* of a proof is all that matters, not its identity. It has several practical consequences. Lean's compiler erases proofs: they cost nothing at run time, much as a zero-sized type costs nothing in Rust. The kernel never needs to look inside a proof to compare it with another, which is why `#reduce` leaves theorems alone, and why `theorem` and `def` are different commands (more on that below). And, as we will see shortly, it forbids certain programs.

Impredicativity

One more difference between the two universes concerns quantification:

```
-- Quantifying over all propositions still gives a proposition...
/-- info: ∀ (p : Prop), p → p : Prop -/
#guard_msgs in
#check ∀ p : Prop, p → p

-- ...but quantifying over all types gives something bigger than a type.
/-- info: (α : Type) → α → α : Type 1 -/
#guard_msgs in
#check (α : Type) → α → α
```

A proposition that quantifies over *all propositions*, including itself, is still a proposition: `∀ p : Prop, p → p` is in `Prop`. This is called **impredicativity**. A type that quantifies over all types is not a type in `Type`; it lands one universe up, in `Type 1`. `Type` is **predicative**. The asymmetry is deliberate. Universes of data that quantify over themselves are dangerous (the extreme case, `Type : Type`, is inconsistent, a result known as Girard's paradox), while an impredicative `Prop` is safe in Lean's type theory, thanks in part to proof irrelevance and to the elimination restriction of the next section. Impredicativity is what lets you state theorems such as "for every proposition p, ..." without climbing

the universe hierarchy, and it is the corner of Lean where System F's polymorphism (Chapter 12) survives unchanged.

10.9 Exists, Subtype and Sigma

There are four ways in Lean to pair a value with something about it, and the differences between them are the clearest illustration of `Prop` versus `Type`:

```
/-- info: ∃ n, n > 3 : Prop -/
#guard_msgs in
#check ∃ n : ℕ, n > 3

/-- info: { n // n > 3 } : Type -/
#guard_msgs in
#check {n : ℕ // n > 3}

/-- info: (n : ℕ) ×' n > 3 : Type -/
#guard_msgs in
#check (n : ℕ) ×' n > 3

/-- info: (n : ℕ) × Fin n : Type -/
#guard_msgs in
#check (n : ℕ) × Fin n
```

Type	Lives in	First component	Second component
`∃ n, P n`	`Prop`	a value (hidden)	a proof
`{n // P n}` (`Subtype`)	`Type`	a value	a proof
`(n : ℕ) ×' P n` (`PSigma`)	`Type`	a value	a proof (or data)
`(n : ℕ) × Fin n` (`Sigma`)	`Type`	a value	data

`Exists` is a proposition: a proof that some suitable `n` exists. `Subtype` (written `{n // P n}`) is a data type whose elements are numbers that carry a proof of `P`: the refinement types of other languages, and the type of "values with an invariant". `PSigma` (`×'`) is the general dependent pair that may hold a proof or data in either component, and `Sigma` (`×`) is the dependent pair of data, such as a length together with an index below it.

The elements of a `Subtype` are data, and their values can be computed with:

```
/-- A number greater than 3, with the proof attached. -/
def four : {n : ℕ // n > 3} := ⟨4, by decide⟩

/-- The witness of a `Subtype` is data, and a program can return it. -/
def Subtype.witness (x : {n : ℕ // n > 3}) : ℕ := x.val

/-- info: 4 -/
#guard_msgs in
#eval Subtype.witness four

/-- Two elements of a subtype are equal as soon as their values are: the
proofs are irrelevant. -/
example (x y : {n : ℕ // n > 3}) (h : x.val = y.val) : x = y := Subtype.ext h

/-- A dependent pair whose second component is data: a length, and an index
below it. -/
def lastIndex : (n : ℕ) × Fin (n + 1) := ⟨3, 3⟩
```

Two elements of a subtype are equal as soon as their values are, by `Subtype.ext`: the proofs never get in the way, because they are all equal.

Rust has no way to attach a proof to a value, but it has a common idiom that plays the same role: a **newtype with a private field**, whose only constructor establishes the invariant.

```rust
/// The closest Rust gets to Lean's `{ v : Vec<u32> // sorted v }`: a value
/// together with an invariant. The field is private, so the only way to get a
/// `Sorted` is through `Sorted::new`, which checks. The "proof" lives in this
/// module's code, not in the type.
mod sorted {
    pub struct Sorted(Vec<u32>);

    impl Sorted {
        pub fn new(mut v: Vec<u32>) -> Sorted {
            v.sort();
            Sorted(v)
        }

        pub fn as_slice(&self) -> &[u32] {
            &self.0
        }

        /// Relies on the invariant: the smallest element comes first.
        pub fn min(&self) -> Option<u32> {
            self.0.first().copied()
        }
    }
}
```

```
Sorted::new(vec![5, 3, 9, 1]) holds [1, 3, 5, 9]
its minimum, read off the front: Some(1)
```

Code outside the module can only obtain a `Sorted` from `Sorted::new`, which sorts, so every `Sorted` holds a sorted vector, and `min` can rely on that. The difference from Lean is where the proof lives. In Lean, the proof is part of the value and the kernel checks it. In Rust, the "proof" is the code of the module, and a reviewer checks it. Chapter 30 returns to this idiom, under the name "parse, don't validate".

The elimination restriction

Now try to write the most natural function in the world: given a proof that a number greater than 3 exists, return the number.

```
/--
error: Invalid projection: Cannot project a value of non-propositional type
  ℕ
from the expression
  h
which has propositional type
  ∃ n, n > 3
-/
#guard_msgs in
def witness (h : ∃ n : ℕ, n > 3) : ℕ := h.1
```

Lean refuses. The projection `h.1` would turn a proof (an element of a type in `Prop`) into data (a number, in `Type`), and Lean does not allow that. If you try with a `match` instead, the error is longer but says the same thing at its core: "recursor `Exists.casesOn` can only eliminate into `Prop`". This is the **elimination restriction**: a proof of a proposition can be taken apart only to build another proof.

Proof irrelevance explains why the restriction is necessary:

```
/-- Why the restriction is necessary: these two proofs have different
witnesses... -/
theorem proofWith4 : ∃ n : ℕ, n > 3 := ⟨4, by decide⟩
theorem proofWith5 : ∃ n : ℕ, n > 3 := ⟨5, by decide⟩

/-- ...and yet they are equal, by proof irrelevance. A function that returned
the witness would map equal arguments to 4 and to 5. -/
example : proofWith4 = proofWith5 := rfl
```

`proofWith4` and `proofWith5` have different witnesses, and yet they are equal, because they are proofs of the same proposition. If `witness` existed, it would map these two equal arguments to 4 and to 5. A function that gives different results on equal arguments is not a function, and the logic would be inconsistent. So Lean must choose between proof irrelevance and extracting witnesses, and it chooses irrelevance.

Inside a proof, on the other hand, you may take an existential apart freely, because the result is again a proof and the witness cannot leak into data:

```
/-- Inside a proof, taking an `Exists` apart is fine: the result is a proof
too, so the witness cannot escape into data. -/
theorem exists_gt_two (h : ∃ n : ℕ, n > 3) : ∃ n : ℕ, n > 2 := by
  obtain ⟨n, hn⟩ := h
  exact ⟨n, by omega⟩
```

That is what `obtain` and `rcases` have been doing all along. There are also exceptions in the other direction. A proposition with at most one constructor, whose fields are all proofs themselves (or appear among its indices, as in `Eq`), carries no information beyond the fact that it holds, and it may be eliminated into data. `False` (no constructors) and `And` (one constructor with two proof fields) qualify; `Exists` (whose constructor has a data field, the witness) and `Or` (two constructors) do not.

```
/-- The exceptions: a proposition with at most one constructor whose fields
are all proofs (or appear in the type) can be eliminated into data, because
there is nothing to leak. `False` has no constructors... -/
def fromFalse (h : False) : ℕ := h.elim

/-- ...and `And` has one, with two proof fields. -/
def fromAnd (p q : Prop) (h : p ∧ q) : ℕ :=
  match h with
  | ⟨_, _⟩ => 42
```

And if you really need the witness, there are two escape hatches, with very different characters:

```
/-- Classical choice picks *some* witness, but no program can compute it. -/
noncomputable def someWitness (h : ∃ n : ℕ, n > 3) : ℕ := Classical.choose h

theorem someWitness_spec (h : ∃ n : ℕ, n > 3) : someWitness h > 3 :=
  Classical.choose_spec h

/-- For a decidable property of natural numbers, the witness can be *found*:
`Nat.find` searches 0, 1, 2, ... and the proof of existence guarantees that the
search stops. -/
def firstAbove3 : ℕ := Nat.find proofWith5

/-- info: 4 -/
#guard_msgs in
#eval firstAbove3
```

`Classical.choose` picks *some* witness, and `Classical.choose_spec` guarantees that it has the property. But it is an axiom, not a program: the definition must be marked `noncomputable`, and `#eval` on it fails. `Nat.find` (from Mathlib) is a program: for a property of natural numbers that a program can decide, it searches $0, 1, 2, \ldots$ and returns the first number that has it. The proof of existence is used only to guarantee that the search terminates. Notice that `firstAbove3` was given `proofWith5`, a proof built from the witness 5, and returns 4: `Nat.find` cannot see the witness inside the proof (nothing can), and it does not need to.

This is the constructive content of Chapter 5, seen from a new angle. A constructive proof of $\exists n.\, P(n)$ does contain a witness, and in Lean you can get at it inside proofs. But a *program* that needs the witness should ask for a `Subtype` or a `Sigma` instead, which carry the witness as data. Choosing between `∃ n, P n` and `{n // P n}` is choosing between a fact and a piece of data.

10.10 `theorem` versus `def`

Lean has two commands for named declarations, and the difference follows from everything above:

```
/--
error: type of theorem `Ch10.three` is not a proposition
  ℕ
-/
#guard_msgs in
theorem three : ℕ := 3

/--
warning: Definition `twoPlusTwo` is a proposition; use `theorem` instead of `def`

Note: This linter can be disabled with `set_option linter.defProp false`
-/
#guard_msgs in
def twoPlusTwo : 2 + 2 = 4 := rfl
```

A `theorem` must have a proposition as its type: its body is a proof, and by proof irrelevance Lean never needs to look at it again. Lean treats theorem bodies as opaque for computation, which keeps the kernel fast (it never unfolds a long proof). A `def` defines data, whose value matters and is unfolded when needed. Lean accepts a `def` whose type is a proposition, but a linter (a style check, like Clippy's) points out that `theorem` is the right command. The rule of thumb: `theorem` for propositions, `def` for everything else, `example` for anonymous checks.

10.11 An inductive family: `Nat.le`

Propositions in Lean are usually not built from the connectives alone. Most interesting ones are **inductive families**, inductive types indexed by values, just like the `HasType` and `Step` relations of Chapter 9. Lean's own `≤` on natural numbers is one:

```
/--
info: protected inductive Nat.le : ℕ → ℕ → Prop
number of parameters: 1
constructors:
Nat.le.refl : ∀ {n : ℕ}, n.le n
Nat.le.step : ∀ {n m : ℕ}, n.le m → n.le m.succ
```

```
-/
#guard_msgs in
#print Nat.le

/-- `3 ≤ 5`, built from the constructors: reflexivity gives `3 ≤ 3`, and two
steps give `3 ≤ 4` and `3 ≤ 5`. -/
theorem three_le_five : 3 ≤ 5 := Nat.le.step (Nat.le.step Nat.le.refl)

/-- No constructor can build `5 ≤ 3`: `cases` tries both and finds that each
would need a smaller instance of the same impossible fact, down to `5 ≤ 0`. -/
theorem not_five_le_three : ¬ 5 ≤ 3 := by
  intro h
  cases h with
  | step h => cases h with
    | step h => cases h with
      | step h => cases h
```

`#print` shows the definition: `Nat.le n` is a family of propositions indexed by a second number, with two constructors. `refl` proves `n ≤ n`, and `step` turns a proof of `n ≤ m` into a proof of `n ≤ m + 1` (`m.succ` is `m + 1`; the `number of parameters: 1` line says that `n` is a fixed parameter while the second number varies). So a proof of `3 ≤ 5` is literally a chain of two `step`s on top of a `refl`: a derivation, and a natural number in unary, at the same time. In the other direction, `cases` shows that `5 ≤ 3` has no proof: the only constructor that can conclude `5 ≤ 3` is `step`, which needs `5 ≤ 2`, which needs `5 ≤ 1`, which needs `5 ≤ 0`, and no constructor concludes that. Each `cases h with | step h => ...` peels one constructor off.

This is the Curry–Howard view of a judgment: a relation defined by rules is an inductive family, a derivation is an element of it, and induction on derivations is recursion over that element. Every inductive relation in this book (`HasType`, `Step`, `Steps`, the natural-deduction `Proves` of Chapter 4) works this way.

10.12 The term behind a tactic proof

Tactic proofs look different from term proofs, but by Curry–Howard there is only one kind of proof: a term. Tactics are a way of *building* the term, and `#print` shows the result (Chapter 4 did this for a first example):

```
theorem and_swap_tactics (p q : Prop) : p ∧ q → q ∧ p := by
  intro h
  obtain ⟨hp, hq⟩ := h
  exact ⟨hq, hp⟩

/--
info: theorem Ch10.and_swap_tactics : ∀ (p q : Prop), p ∧ q → q ∧ p :=
fun p q h => And.casesOn h fun hp hq => ⟨hq, hp⟩
-/
#guard_msgs in
#print and_swap_tactics
```

The three tactics became a three-part term. `intro h` became `fun p q h =>` (the binders for `p` and `q` come from the statement), `obtain ⟨hp, hq⟩ := h` became `And.casesOn h fun hp hq => ...`, which is ∧ E in its most general form (a function that receives both components), and `exact ⟨hq, hp⟩` became the anonymous constructor, ∧ I. Up to the spelling of the elimination, it is the term we wrote by hand for `and_swap`.

10.13 Putting it together: one law, three proofs

Let us follow one theorem through all three languages: De Morgan's law $\neg(A \vee B) \to \neg A \wedge \neg B$, which is valid in intuitionistic logic (unlike its dual $\neg(A \wedge B) \to \neg A \vee \neg B$, which needs excluded middle, as Chapter 5 explained).

In logic. Assume $h : \neg(A \vee B)$ and call that context Γ. To refute A, assume $a : A$; then $\mathit{inl}\, a$ proves $A \vee B$, and h applied to it proves $\bot$. Written as a derivation with its terms attached, this is one half of the proof:

$$\dfrac{\dfrac{\Gamma, a : A \vdash h : \neg(A \vee B) \qquad \dfrac{\Gamma, a : A \vdash a : A}{\Gamma, a : A \vdash \mathit{inl}\, a : A \vee B}\ \text{T-Inl}}{\Gamma, a : A \vdash h\,(\mathit{inl}\, a) : \bot}\ \text{T-App}}{\Gamma \vdash \lambda a.\, h\,(\mathit{inl}\, a) : \neg A}\ \text{T-Lam}$$

The other half, $\Gamma \vdash \lambda b.\, h\,(\mathit{inr}\, b) : \neg B$, is the same with inr. The pair rule combines them:

$$\dfrac{\Gamma \vdash \lambda a.\, h\,(\mathit{inl}\, a) : \neg A \qquad \Gamma \vdash \lambda b.\, h\,(\mathit{inr}\, b) : \neg B}{\Gamma \vdash (\lambda a.\, h\,(\mathit{inl}\, a), \lambda b.\, h\,(\mathit{inr}\, b)) : \neg A \wedge \neg B}\ \text{T-Pair}$$

and a final T-Lam discharges h. Erase the terms, and you have the natural-deduction proof. Look at the T-Pair step: both premises have the full context Γ, so the hypothesis h is available to both halves. That is contraction, hidden in the shape of the rule: the context is *shared* between the premises, not *split*.

In Lean. The term at the root of the derivation is, symbol for symbol, the proof in `Dictionary.lean`: `fun h => (fun hp => h (.inl hp), fun hq => h (.inr hq))`. Lean's rule for pairs shares the context, as in the derivation, and Lean accepts it without comment. `#print axioms de_morgan` reports no axioms: it is a proof from the rules alone.

In Rust. The same term, transcribed, is the program in `ch10_de_morgan.rs`, and Rust rejects it with E0382: the refutation `n` is moved into the first closure and cannot be used by the second. Rust's rule for building a pair (or a tuple of closures) *splits* the owned values in scope between the components. To recover the proof, we must pay for the contraction with `N: Clone` and an explicit `n.clone()`. Once we do, Rust accepts it, and the proof is valid for every refutation type that can be cloned.

Three languages, one theorem, three verdicts: valid in logic, accepted silently by Lean, accepted by Rust only after the duplication is made explicit. And if we had cheated, writing `fn de_morgan_or<A, B, N>(n: N) -> ... { todo!() }`, Rust would have accepted that too, while Lean would have marked the theorem with `sorryAx`. The dictionary is real, and so are its exceptions.

10.14 Common confusions

"**Every program is a proof of something interesting.**" Every well-typed program proves its type, but most types are dull theorems. `fn(i32) -> i32` is "if there is an integer, there is an integer", which `|x| x` already proves. Curry–Howard is illuminating for generic types (where the program cannot cheat by inventing values) and for dependent types (where the type can say something precise), not for `i32 -> i32`.

"**If my Rust code compiles, the theorem is proved.**" Only if the code is total. `panic!`, `todo!`, `loop {}` and unbounded recursion prove anything.

"**`Prop` is Lean's `Bool`.**" A `Bool` is a value a program can branch on; a `Prop` is a type of proofs, which a program cannot inspect at all. Chapter 5 explains when a `Prop` can be turned into a `Bool` (when it is `Decidable`).

"**Lean's `∃` is like `impl Trait`.**" Both hide a witness, but `impl Trait` hides a *type*, which the compiler knows and the caller does not, while Lean's `∃` hides a *value*, which no program can extract. The analogue of `impl Trait` in Lean is closer to a structure with a type field (Chapter 15).

"**Proof irrelevance means proofs do not matter.**" It means that *which* proof you have does not matter. *Whether* you have one is the whole point.

"**Rust lacks contraction.**" For values of arbitrary type, yes. For `Copy` types contraction is free and implicit, and for `Clone` types it is available on request. Rust's logic is affine with a controlled way back, which is what Part V is about.

10.15 Summary

- Curry–Howard: the typing rules of the simply typed λ-calculus are the rules of intuitionistic natural deduction with terms attached. Types are propositions, terms are proofs, introduction rules are constructors, elimination rules are the ways of using a value.
- The dictionary: $\wedge$ is the pair, $\vee$ the tagged union, $\to$ the function, $\top$ the unit type, $\bot$ the empty type, $\neg A$ a function into the empty type, $\forall$ a dependent function and $\exists$ a dependent pair. Rust's quantifiers range over types only.
- Removing a detour from a proof is a step of computation. Termination together with progress, preservation and canonical forms makes a language a consistent logic.
- Rust is inconsistent as a logic: `loop {}`, `panic!`, `todo!` and unbounded recursion inhabit every type. Total Rust functions are honest proofs.
- Lean stays consistent: definitions must terminate, `partial` requires a nonempty type and is opaque, and `sorry` is tracked by `#print axioms`.
- Rust has no contraction for arbitrary types: $A \to A \wedge A$ needs `Clone` or `Copy`. Its generic functions form an affine logic.
- Lean separates `Prop` (proofs, irrelevant, impredicative, erased) from `Type` (data). `Exists` lives in `Prop`; `Subtype`, `PSigma` and `Sigma` in `Type`. A proof of `∃` cannot be eliminated into data, because proof irrelevance would make such a function inconsistent; use a `Subtype`, `Nat.find`, or (noncomputably) `Classical.choose`.
- `theorem` is for propositions, `def` for data. Inductive families such as `Nat.le` are relations defined by rules, and tactic proofs build ordinary terms, which `#print` shows.

10.16 Exercises

1. Prove $\neg\neg\neg A \to \neg A$ in Lean (as a term) and in Rust (as a generic function with `FnOnce` bounds). Is the converse, $\neg A \to \neg\neg\neg A$, also provable? *Hint:* given `h : ¬¬¬A` and `a : A`, the function `fun na => na a` is a proof of `¬¬A`; apply `h` to it. The converse is an instance of `double_negation_intro`.
2. Write the curried direction of `curry_iff` in Rust: from `impl FnOnce((A, B)) -> C` to a function that takes an `A` and returns a function that takes a `B`. First try `impl FnOnce(A) -> impl FnOnce(B) -> C` and read the error; then return `Box<dyn FnOnce(B) -> C>`. Which bounds on the type parameters does the box need, and why? *Hint:* the error is E0562. A `Box<dyn ...>` without a lifetime means `+ 'static`, and the boxed closure captures both `f` and `a`.
3. Prove in Lean that `(∃ x, P x) → ¬ ∀ x, ¬ P x`, for `P : α → Prop`. Then explain why the converse cannot be proved constructively, using the BHK reading of Chapter 5. *Hint:* take the existential apart with `obtain`, and apply the universal statement to the witness. For the converse, a constructive proof would have to produce a witness from a proof that refutes the absence of one.
4. Prove `theorem toExists : {n : ℕ // n > 3} → ∃ n : ℕ, n > 3` in Lean (why is it a `theorem`, although it looks like a function?). Then explain, in one sentence each, why the function in the other direction cannot be written, and why `Nat.find` nevertheless gives you a number from a proof of `∃ n : ℕ, n > 3`. *Hint:* `fun x => ⟨x.val, x.property⟩`. A function type whose result is a proposition is itself a proposition (impredicativity again). Compare what `Nat.find` uses from the proof (only that it exists) with what `witness` would use.
5. On paper, take the proof of $A \to A$ that goes through a detour, $(\lambda p.\, \mathit{fst}\, p)\,(a, a)$ under a λa, write its natural-deduction derivation, and normalize it step by step, removing one detour at a time. Check your answer against `#reduce detour`. *Hint:* there are two detours: $\to$ I followed by $\to$ E (the β-redex) and then $\wedge$ I followed by $\wedge\,\mathrm{E}_1$.
6. In Rust, try to write excluded middle, `fn lem<A>() -> Either<A, Box<dyn FnOnce(A) -> Void>>`, without cheating. Then write it with `todo!()` and explain what this does and does not show. *Hint:* to return `Left` you need an `A` out of nothing; to return `Right` you need to refute every `A`, including types that have values. With `todo!()` the function compiles, which shows only that Rust is inconsistent.

CHAPTER 11

Algebraic Data Types and the Algebra of Types

Look at almost any Rust program and you will find the same few building blocks. A `struct` or a tuple glues several values together. An `enum` offers a choice between alternatives. The unit type `()` is the boring answer of a function called for its effects, and `Infallible` is the error type of an operation that cannot fail. Functions turn values into values. With these five ingredients, nested inside each other and made recursive with a `Box`, you can describe configuration files, network packets, abstract syntax trees, state machines and almost everything else a program needs.

These building blocks are called **algebraic data types**, and the word *algebraic* is not decoration. If you count the values of each type, products multiply, sums add, the unit type is the number 1, the empty type is 0, and function types are powers. The familiar laws of school algebra (distributivity, $a^{b+c} = a^b \cdot a^c$, $a \cdot 1 = a$) turn into statements about types, and each of those statements is witnessed by a pair of programs that convert back and forth without losing information. The algebra explains why `Option<bool>` fits in one byte, why a recursive `enum` needs a `Box`, why Lean refuses some recursive types that Rust happily accepts, and how to tell, by counting, that a data structure admits states your program should never see.

This chapter studies the type formers one by one, then the algebra, then recursion. Rust shows the algebra at work, down to the byte layout; Lean proves every law we state, first with a hand-written structure and then with Mathlib's library of isomorphisms and cardinalities.

In this chapter you will learn:

- the five basic type formers (products, sums, unit, empty, functions), their introduction and elimination rules, and why exhaustive `match` *is* the elimination rule for sums;
- how counting values turns types into an algebra, and how each law of that algebra becomes an isomorphism of types;
- how to prove isomorphisms in Lean with an `Iso` structure of our own, then with Mathlib's `Equiv` and `Fintype.card`;
- recursive types as solutions of equations ($\text{List}\, A \cong 1 + A \times \text{List}\, A$), why Rust needs a `Box` for them, and the fold as their elimination rule;
- why Lean rejects a type like `inductive Bad | mk : (Bad → Nat) → Bad`, and what Rust gains and loses by accepting its equivalent;

- how Rust lays out algebraic data types in memory: zero-sized types, empty types and the niche optimization;
- how to use counting to find illegal states in a design, and (as a bonus) how the *derivative* of a type gives you a zipper.

11.1 Five ways to build a type

You have met all of the type formers of this chapter before. Chapter 8 gave the simply typed λ-calculus products and sums with their rules, and Chapter 10 read them as conjunction and disjunction. This chapter looks at them as *data*: what their values are, how many there are, and how they are stored. Here is the cast:

Type former	Theory	Rust	Lean	Values
product	$A \times B$	`(A, B)`, `struct`	`α × β`, `structure`	one `A` and one `B`
sum	$A + B$	`enum`, `Result<A, B>`	`α ⊕ β`, `inductive`	an `A` or a `B`, tagged
unit	$\mathbf{1}$	`()`, `struct Marker;`	`Unit`	exactly one
empty	$\mathbf{0}$	`enum Void {}`, `Infallible`	`Empty`	none
function	$A \to B$, B^A	`fn(A) -> B`, closures	`α → β`	a `B` for every `A`

Each type former comes with rules that say how to build its values (*introduction*) and how to use them (*elimination*), as in Chapter 4.

Products

A value of $A \times B$ is built from a value of each component, and used by projecting a component out:

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B}\ \textsf{T-Pair} \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \mathtt{fst}\ p : A}\ \textsf{T-Fst} \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \mathtt{snd}\ p : B}\ \textsf{T-Snd}$$

with the computation rules $\mathtt{fst}\,(a, b) \to a$ and $\mathtt{snd}\,(a, b) \to b$. In Rust the anonymous product is the tuple, with projections `.0` and `.1` and destructuring patterns `let (a, b) = p;`. A `struct` is a product whose components have names. The names matter to the compiler: `struct Meters(f64)` and `struct Seconds(f64)` are different types with the same values, because Rust's structs are *nominal* (Chapter 3). The algebra ignores names; below, "same values, different type" becomes the difference between isomorphism and equality.

Sums

A value of $A + B$ is a value of A or a value of B, *tagged* with which one it is. Two introduction rules put a value in, and one elimination rule takes it apart by cases:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \mathtt{inl}\ a : A + B}\ \textsf{T-Inl} \qquad \frac{\Gamma \vdash b : B}{\Gamma \vdash \mathtt{inr}\ b : A + B}\ \textsf{T-Inr}$$

$$\frac{\Gamma \vdash s : A + B \qquad \Gamma, x : A \vdash e_1 : C \qquad \Gamma, y : B \vdash e_2 : C}{\Gamma \vdash \mathtt{case}\ s\ \mathtt{of}\ \mathtt{inl}\ x \Rightarrow e_1 \mid \mathtt{inr}\ y \Rightarrow e_2 : C}\ \textsf{T-Case}$$

Rule T-Case says: to use a value of a sum, you must say what to do in *each* case, and both branches must produce the same type C. You have seen this shape twice already. It is the rule $\vee$E of Chapter 4 (to use "A or B", prove the goal from A and from B), and its special case `bool` $= \mathbf{1} + \mathbf{1}$ is rule T-If of Chapter 1.

The tag is essential. The sum `u8` $+$ `u8` has 512 values, not 256: `inl 7` and `inr 7` are different. That is what distinguishes a sum (a *tagged union*) from a C `union`, which has no tag.

In Rust, every `enum` is a sum, and every variant with fields is a product, so an `enum` is a *sum of products*. The standard library is full of sums: `Option<T>` is $\mathbf{1} + T$ (`None` carries nothing, `Some` carries a `T`), and `Result<T, E>` is $T + E$. The plain two-way sum is not in the standard library, so this chapter's Rust example, `code/rust/examples/ch11_type_algebra.rs`, defines it:

```rust
/// The sum type A + B: a value is an `A` or a `B`, tagged with which one.
#[derive(Clone, Copy, Debug, PartialEq)]
enum Either<A, B> {
    Left(A),
    Right(B),
}

/// The empty type 0: no variants, hence no values.
#[derive(Clone, Copy, Debug, PartialEq)]
enum Void {}

/// The elimination rule for 0: a `match` with no arms. It type-checks at any
/// result type `T` because there is no case to handle.
fn absurd<T>(v: Void) -> T {
    match v {}
}
```

The unit type and the empty type

The unit type $\mathbf{1}$ has one value and an introduction rule with no premises; there is nothing to eliminate, because a value of type $\mathbf{1}$ carries no information:

$$\frac{}{\Gamma \vdash () : \mathbf{1}}\ \text{T-Unit} \qquad\qquad \frac{\Gamma \vdash e : \mathbf{0}}{\Gamma \vdash \texttt{absurd}\ e : C}\ \text{T-Absurd}$$

The empty type $\mathbf{0}$ is the opposite: no introduction rule at all, and an elimination rule that produces a value of *any* type C. Rule T-Absurd is the case rule for a sum with zero summands: to handle every case of a value of $\mathbf{0}$, you handle no cases, and the conclusion can be anything. In Rust it is a `match` with no arms, which is exactly the body of `absurd` above. As Chapter 10 explained, $\mathbf{0}$ is falsity under Curry–Howard, and T-Absurd is *ex falso quodlibet*.

An empty type is useful precisely because it rules things out. The standard library's `Infallible` is an empty `enum`, used as the error type of conversions that cannot fail. A function returning `Result<u64, Infallible>` promises that it never returns `Err`, and the compiler can check code that relies on the promise. The chapter's second example, `code/rust/examples/ch11_recursive_types.rs`, shows two ways to unwrap such a result:

```rust
/// A parser that cannot fail: its error type has no values.
fn parse_trusted(digits: &str) -> Result<u64, Infallible> {
    Ok(digits.bytes().fold(0, |n, d| 10 * n + u64::from(d - b'0')))
}
```

```rust
/// The long way: the `Err` arm is there, and its body is a `match` with no
/// arms, because there is no `Infallible` value to take apart.
fn unwrap_long<T>(r: Result<T, Infallible>) -> T {
    match r {
        Ok(t) => t,
        Err(e) => match e {},
    }
}

/// The short way: the `Err` case would need a value of type `Infallible`, so
/// the pattern `Ok(t)` alone is exhaustive and `let` accepts it.
fn unwrap_short<T>(r: Result<T, Infallible>) -> T {
    let Ok(t) = r;
    t
}
```

In `unwrap_long`, the `Err` arm binds a value `e` of type `Infallible` and handles it with `match e {}`: no arms, because no value of `Infallible` exists. In `unwrap_short` the compiler goes one step further. Since Rust 1.82, a pattern that only an empty type could match may be left out, so `let Ok(t) = r;` is accepted as exhaustive: the missing case `Err(_)` would need a value of `Infallible`.

Functions

The function type $A \to B$ completes the list. Its rules are the λ-abstraction and application rules of Chapter 8, and its values are the functions themselves. Algebraically, function types will turn out to be *exponentials*, so we will often write B^A for $A \to B$.

Pattern matching is the elimination rule

The rule T-Case demands a branch for each summand. Rust's `match` generalizes it to sums with any number of variants, nested patterns and guards, and keeps the demand: a `match` must be **exhaustive**. A `match` that forgets a variant is like a proof by cases that forgets a case, and the compiler rejects it (`code/rust/compile_fail/ch11_non_exhaustive.rs`):

```rust
enum Payment {
    Card { last4: u16 },
    Invoice { days: u32 },
    Voucher(u32),
}

fn describe(p: &Payment) -> String {
    match p {
        Payment::Card { last4 } => format!("card ending in {last4}"),
        Payment::Invoice { days } => format!("invoice, due in {days} days"),
    }
}
```

```
error[E0004]: non-exhaustive patterns: `&Payment::Voucher(_)` not covered
  --> compile_fail/ch11_non_exhaustive.rs:13:11
   |
13 |     match p {
   |           ^ pattern `&Payment::Voucher(_)` not covered
   |
note: `Payment` defined here
  --> compile_fail/ch11_non_exhaustive.rs:6:6
   |
 6 | enum Payment {
   |      ^^^^^^^
...
 9 |     Voucher(u32),
```

```
   |     ------- not covered
   = note: the matched value is of type `&Payment`
help: ensure that all possible cases are being handled by adding a match arm with a wildcard
pattern or an explicit pattern as shown
   |
15 ~         Payment::Invoice { days } => format!("invoice, due in {days} days"),
16 ~         &Payment::Voucher(_) => todo!(),
   |
```

The error names the missing case, `&Payment::Voucher(_)`, and suggests adding an arm for it. Exhaustiveness checking is what makes `enum` safer than the "tag field plus a few optional fields" pattern common in other languages: when you add a variant to an `enum`, every `match` that does not handle it stops compiling, and the compiler gives you the list of places to update.

> **TIP**
>
> A wildcard arm `_ => ...` satisfies the checker, but it also switches the check off for every variant added later. For enums you own, prefer listing the variants explicitly. Library authors can do the opposite with `#[non_exhaustive]`, which forces code in *other* crates to include a wildcard, so that adding a variant is not a breaking change: the sum is declared to be open.

The type formers in Lean

Lean has all five type formers too, and it lets us build the first four from scratch with `inductive` and `structure` (Chapter 2). The file `code/lean/Book/Ch11/Formers.lean` does exactly that:

```
/-- A + B: a value is built by `inl` from an `α` or by `inr` from a `β`. -/
inductive MySum (α β : Type) where
  | inl : α → MySum α β
  | inr : β → MySum α β

/-- A × B: one constructor with two fields. -/
structure MyProd (α β : Type) where
  fst : α
  snd : β

/-- 1: one constructor with no fields. -/
inductive MyUnit where
  | star : MyUnit

/-- 0: no constructors at all. -/
inductive MyEmpty : Type
```

The definitions read like the table at the start of the section. A sum has two constructors, each with one field. A product is a structure with two fields. The unit type has one constructor with no fields, and the empty type has no constructors at all, so there is no way to build one of its values. The elimination rules are ordinary functions defined by pattern matching:

```
/-- Rule +E: to use a sum, say what to do in each case. -/
def MySum.elim {α β γ : Type} (f : α → γ) (g : β → γ) : MySum α β → γ
  | .inl a => f a
  | .inr b => g b

/-- Rule 0E: from a value of the empty type, anything. There is no case to
write, and `nomatch` says so. -/
def MyEmpty.elim {γ : Type} (e : MyEmpty) : γ := nomatch e

/-- Rule ×E: the projections come with the structure; pattern matching works
```

```
too. -/
def MyProd.swap {α β : Type} : MyProd α β → MyProd β α
  | ⟨a, b⟩ => ⟨b, a⟩
```

`MyEmpty.elim` plays the role of Rust's `absurd`. Its body `nomatch e` is a pattern match with no cases, the Lean spelling of Rust's `match e {}`, and it type-checks at any result type `γ`. The built-in versions are called `Sum` (written `α ⊕ β`, with constructors `Sum.inl` and `Sum.inr`), `Prod` (written `α × β`), `Unit` (whose value is written `()`) and `Empty`:

```
/-- info: Sum.inl 3 : ℕ ⊕ String -/
#guard_msgs in
#check (Sum.inl 3 : ℕ ⊕ String)

/-- info: (3, "three") : ℕ × String -/
#guard_msgs in
#check ((3, "three") : ℕ × String)

/-- info: "3" -/
#guard_msgs in
#eval (Sum.elim toString id (Sum.inl 3 : ℕ ⊕ String) : String)
```

`Sum.elim f g` is the case rule packaged as a function: it applies `f` to a left value and `g` to a right value. Lean checks exhaustiveness as Rust does, with an error that lists the missing patterns:

```
inductive Payment where
  | card (last4 : ℕ)
  | invoice (days : ℕ)
  | voucher (code : ℕ)

/--
error: Missing cases:
(voucher _)
-/
#guard_msgs in
def Payment.describe : Payment → String
  | .card last4 => s!"card ending in {last4}"
  | .invoice days => s!"invoice, due in {days} days"
```

11.2 Counting values

Now the algebra. Write $|A|$ for the number of values of a type A, and consider only types with finitely many values for the moment. Then:

$$|\mathbf{0}| = 0 \qquad |\mathbf{1}| = 1 \qquad |A + B| = |A| + |B| \qquad |A \times B| = |A| \cdot |B| \qquad |A \to B| = |B|^{|A|}$$

The first two are definitions. A value of $A + B$ is either one of the $|A|$ left values or one of the $|B|$ right values, and the tags keep the two groups apart, so the counts add. A value of $A \times B$ chooses an A *and*, independently, a B, so the counts multiply. A function from A to B chooses, for each of the $|A|$ arguments, one of $|B|$ results, independently: $|B|$ choices $|A|$ times, which is $|B|^{|A|}$. That is why function types are called **exponentials**, and why we write B^A for $A \to B$.

Some consequences are worth checking by hand. `bool` is $\mathbf{1} + \mathbf{1}$, so it has $1 + 1 = 2$ values. `Option<bool>` is $\mathbf{1} + 2$, three values. There are $2^2 = 4$ functions from `bool` to `bool`: the identity, negation, and the two constant functions. There is exactly one function from $\mathbf{0}$ to any type, the empty function that never has to return anything ($B^0 = 1$), and no function at all from an inhabited type into

$\mathbf{0}$ ($0^A = 0$ when $|A| > 0$). The corner case $|\mathbf{0} \to \mathbf{0}| = 0^0 = 1$ settles a question that sometimes divides mathematicians: for types, 0^0 is 1, and the single function is `absurd`.

In Rust we can make the counting executable. The example defines a trait for types with finitely many values, with an associated constant computed from the structure of the type and a method that lists the values:

```
/// Types with finitely many values, listed explicitly. `CARD` is computed
/// from the structure of the type, `all()` enumerates the values; the
/// counting laws say the two must agree.
trait Finite: Sized {
    const CARD: usize;
    fn all() -> Vec<Self>;
}

impl Finite for Void {
    const CARD: usize = 0;
    fn all() -> Vec<Self> {
        vec![]
    }
}

impl Finite for () {
    const CARD: usize = 1;
    fn all() -> Vec<Self> {
        vec![()]
    }
}

impl Finite for bool {
    const CARD: usize = 2;
    fn all() -> Vec<Self> {
        vec![false, true]
    }
}

/// |A × B| = |A| · |B|
impl<A: Finite + Clone, B: Finite + Clone> Finite for (A, B) {
    const CARD: usize = A::CARD * B::CARD;
    fn all() -> Vec<Self> {
        let mut out = Vec::new();
        for a in A::all() {
            for b in B::all() {
                out.push((a.clone(), b));
            }
        }
        out
    }
}

/// |A + B| = |A| + |B|
impl<A: Finite, B: Finite> Finite for Either<A, B> {
    const CARD: usize = A::CARD + B::CARD;
    fn all() -> Vec<Self> {
        let lefts = A::all().into_iter().map(Either::Left);
        let rights = B::all().into_iter().map(Either::Right);
        lefts.chain(rights).collect()
    }
}

/// |Option A| = 1 + |A|
impl<A: Finite> Finite for Option<A> {
    const CARD: usize = 1 + A::CARD;
    fn all() -> Vec<Self> {
        std::iter::once(None)
            .chain(A::all().into_iter().map(Some))
            .collect()
    }
```

```
}

/// |Result<T, E>| = |T| + |E|
impl<T: Finite, E: Finite> Finite for Result<T, E> {
    const CARD: usize = T::CARD + E::CARD;
    fn all() -> Vec<Self> {
        let oks = T::all().into_iter().map(Ok);
        let errs = E::all().into_iter().map(Err);
        oks.chain(errs).collect()
    }
}
```

Each `impl` is one line of the counting table. `CARD` is computed at compile time from the components: the `impl` for pairs says `A::CARD * B::CARD`, the one for `Either` says `A::CARD + B::CARD`. The method `all` enumerates the values, and the example asserts that the length of the enumeration agrees with the formula. For function types we only compute the count:

```
/// |A → B| = |B| ^ |A|: one independent choice in B for each value of A.
/// (We only count functions here; enumerating them is an exercise.)
const fn pow(base: usize, exp: usize) -> usize {
    let mut result = 1;
    let mut i = 0;
    while i < exp {
        result *= base;
        i += 1;
    }
    result
}

fn card_fn<A: Finite, B: Finite>() -> usize {
    pow(B::CARD, A::CARD)
}
```

```
println!();
println!(
    "|(bool, Option<bool>)|      = {}",
    <(bool, Option<bool>)>::CARD
);
println!("|Either<bool, ()>|          = {}", <Either<bool, ()>>::CARD);
println!(
    "|bool -> Option<bool>|      = {}",
    card_fn::<bool, Option<bool>>()
);
println!(
    "|Option<bool> -> bool|      = {}",
    card_fn::<Option<bool>, bool>()
);
println!("|Void -> bool|              = {}", card_fn::<Void, bool>());
println!("|bool -> Void|              = {}", card_fn::<bool, Void>());
println!("|Void -> Void|              = {}", card_fn::<Void, Void>());
// The formula and the enumeration agree.
assert_eq!(
    <(bool, Option<bool>)>::all().len(),
    <(bool, Option<bool>)>::CARD
);
assert_eq!(<Either<bool, Option<()>>>::all().len(), 4);
```

```
|(bool, Option<bool>)|      = 6
|Either<bool, ()>|          = 3
|bool -> Option<bool>|      = 9
|Option<bool> -> bool|      = 8
|Void -> bool|              = 1
|bool -> Void|              = 0
|Void -> Void|              = 1
```

The last three lines are the corner cases: one empty function, no function into `Void`, and $0^0 = 1$.

11.3 Isomorphisms: when two types are the same

Counting suggests that `Option<bool>` and `Either<(), bool>` are "the same type": both have three values. They are not equal, as far as Rust is concerned, but they are interchangeable in a precise sense. An **isomorphism** between types A and B, written $A \cong B$, is a pair of functions

$$f : A \to B \qquad g : B \to A \qquad \text{with} \qquad g(f(a)) = a \ \text{ for all } \ a, \qquad f(g(b)) = b \ \text{ for all } \ b.$$

The two equations are the *round trips*: converting to B and back loses nothing, and neither does converting to A and back. If $A \cong B$, any program that works with A can be rewritten to work with B by converting at the boundaries, and nothing observable changes. For a programmer, an isomorphism is a lossless, reversible change of representation, the kind of refactoring that is safe by construction.

For finite types, an isomorphism exists exactly when the two types have the same number of values. But the interesting isomorphisms are the ones that come from the *laws* of the algebra, because they work uniformly for all types, finite or not. Each law of arithmetic becomes one:

Law of numbers	Isomorphism of types	In words
$a \cdot 1 = a$	$A \times \mathbf{1} \cong A$	pairing with `()` adds nothing
$a + 0 = a$	$A + \mathbf{0} \cong A$	an impossible alternative adds nothing
$a \cdot 0 = 0$	$A \times \mathbf{0} \cong \mathbf{0}$	no pair can be built
$a \cdot b = b \cdot a$	$A \times B \cong B \times A$	swap
$a(b + c) = ab + ac$	$A \times (B + C) \cong A \times B + A \times C$	distributivity
$c^{ab} = (c^b)^a$	$(A \times B \to C) \cong (A \to B \to C)$	currying
$a^{b+c} = a^b \cdot a^c$	$(B + C \to A) \cong (B \to A) \times (C \to A)$	one function per case
$a^1 = a$, $1^a = 1$, $a^0 = 1$	$(\mathbf{1} \to A) \cong A$, ...	small exponents
$2 = 1 + 1$	`bool` $\cong \mathbf{1} + \mathbf{1}$	a tag and nothing else

The law $a^{b+c} = a^b \cdot a^c$ is one you use without noticing: a function out of an `enum` with two variants is the same thing as a pair of functions, one for each variant. That is what a `match` with two arms *is*.

Isomorphisms in Rust

In Rust, an isomorphism is a pair of functions. The example stores them as function pointers and checks both round trips on every value of a finite instance:

```
/// An isomorphism: two functions that undo each other.
struct Iso<A, B> {
    name: &'static str,
    forward: fn(A) -> B,
    backward: fn(B) -> A,
}

impl<A, B> Iso<A, B>
where
    A: Finite + Clone + PartialEq + Debug,
    B: Finite + Clone + PartialEq + Debug,
```

```
{
    /// Check both round trips on *every* value of both types. For these
    /// particular finite instances this is a complete proof; for all types at
    /// once we need Lean.
    fn check(&self) {
        for a in A::all() {
            assert_eq!((self.backward)((self.forward)(a.clone())), a);
        }
        for b in B::all() {
            assert_eq!((self.forward)((self.backward)(b.clone())), b);
        }
        assert_eq!(A::CARD, B::CARD);
        println!("{:<26} ok: {} values each side", self.name, A::CARD);
    }
}
```

Here are some of the laws as generic functions. Each pair works for all types `A`, `B`, `C`; the check instantiates them at small finite types:

```
/// A × 1 ≅ A
fn drop_unit<A>((a, ()): (A, ())) -> A {
    a
}
fn add_unit<A>(a: A) -> (A, ()) {
    (a, ())
}

/// A + 0 ≅ A: the `Right` case cannot happen, and `absurd` says so.
fn drop_void<A>(e: Either<A, Void>) -> A {
    match e {
        Either::Left(a) => a,
        Either::Right(v) => absurd(v),
    }
}
fn add_void<A>(a: A) -> Either<A, Void> {
    Either::Left(a)
}

/// A × (B + C) ≅ A × B + A × C
fn distribute<A, B, C>((a, bc): (A, Either<B, C>)) -> Either<(A, B), (A, C)> {
    match bc {
        Either::Left(b) => Either::Left((a, b)),
        Either::Right(c) => Either::Right((a, c)),
    }
}
fn factor<A, B, C>(e: Either<(A, B), (A, C)>) -> (A, Either<B, C>) {
    match e {
        Either::Left((a, b)) => (a, Either::Left(b)),
        Either::Right((a, c)) => (a, Either::Right(c)),
    }
}

/// Option A ≅ 1 + A
fn option_to_sum<A>(o: Option<A>) -> Either<(), A> {
    match o {
        None => Either::Left(()),
        Some(a) => Either::Right(a),
    }
}
fn sum_to_option<A>(e: Either<(), A>) -> Option<A> {
    match e {
        Either::Left(()) => None,
        Either::Right(a) => Some(a),
    }
}

/// bool ≅ 1 + 1
fn bool_to_sum(b: bool) -> Either<(), ()> {
```

```
    if b {
        Either::Left(())
    } else {
        Either::Right(())
    }
}
fn sum_to_bool(e: Either<(), ()>) -> bool {
    matches!(e, Either::Left(()))
}
```

```
Iso {
    name: "A × 1 ≅ A",
    forward: drop_unit::<Option<bool>>,
    backward: add_unit,
}
.check();
Iso {
    name: "A + 0 ≅ A",
    forward: drop_void::<(bool, bool)>,
    backward: add_void,
}
.check();
Iso {
    name: "A × (B + C) ≅ A×B + A×C",
    forward: distribute::<bool, Option<bool>, ()>,
    backward: factor,
}
.check();
Iso {
    name: "Option A ≅ 1 + A",
    forward: option_to_sum::<bool>,
    backward: sum_to_option,
}
.check();
Iso {
    name: "bool ≅ 1 + 1",
    forward: bool_to_sum,
    backward: sum_to_bool,
}
.check();
// The standard library's `transpose` methods: 1 + (A + E) ≅ (1 + A) + E.
Iso {
    name: "1 + (A + E) ≅ (1 + A) + E",
    forward: Option::<Result<bool, ()>>::transpose,
    backward: Result::<Option<bool>, ()>::transpose,
}
.check();
```

```
A × 1 ≅ A                 ok: 3 values each side
A + 0 ≅ A                 ok: 4 values each side
A × (B + C) ≅ A×B + A×C   ok: 8 values each side
Option A ≅ 1 + A          ok: 3 values each side
bool ≅ 1 + 1              ok: 2 values each side
1 + (A + E) ≅ (1 + A) + E  ok: 4 values each side
```

The last check uses two methods from the standard library. `Option::transpose` turns an `Option<Result<T, E>>` into a `Result<Option<T>, E>`, and `Result::transpose` goes back. Both types are $1 + T + E$, grouped differently: whoever added the methods to the standard library was doing type algebra.

Because the check enumerates every value, it is a complete proof for *these instances*, but not for every `A`. For that we need Lean.

Function types cannot be enumerated or compared with `==`, so the currying isomorphism is checked pointwise, on arguments we choose. Its Rust version also shows a deviation from the clean theory:

```rust
/// (A × B → C) ≅ (A → (B → C)). The inner function must own a copy of `a`
/// and of `f`, which is why Rust asks for `Clone` and a box.
fn curry<A, B, C, F>(f: F) -> impl Fn(A) -> Box<dyn Fn(B) -> C>
where
    A: Clone + 'static,
    B: 'static,
    C: 'static,
    F: Fn((A, B)) -> C + Clone + 'static,
{
    move |a: A| {
        let f = f.clone();
        Box::new(move |b: B| f((a.clone(), b)))
    }
}

fn uncurry<A, B, C, G, H>(g: G) -> impl Fn((A, B)) -> C
where
    G: Fn(A) -> H,
    H: Fn(B) -> C,
{
    move |(a, b)| g(a)(b)
}
```

```rust
// Function types cannot be compared for equality, so we check the round
// trip pointwise: on every argument we try, the functions agree.
let f = |(a, b): (u32, u32)| 10 * a + b;
let back_again = uncurry(curry(f));
for a in 0..10 {
    for b in 0..10 {
        assert_eq!(back_again((a, b)), f((a, b)));
        assert_eq!(curry(f)(a)(b), f((a, b)));
    }
}
println!("{:<26} ok: agree on 100 arguments", "C^(A×B) ≅ (C^B)^A");
```

```
C^(A×B) ≅ (C^B)^A          ok: agree on 100 arguments
```

On paper, `curry f` is just `fun a => fun b => f (a, b)`. In Rust, the inner closure outlives the call that created it, so it must *own* a copy of `a` and of `f`. Hence the `Clone` and `'static` bounds, and the `Box<dyn Fn>` (Rust does not allow `impl Fn` in the return type of a closure type). Chapter 8 discussed the cost of currying; here it is the price of an isomorphism, and a first sign of the theme of Part V: Rust values are resources, and a program that uses a variable twice needs permission to duplicate it.

11.4 Proving isomorphisms in Lean

Rust can test the laws on examples. Lean can prove them for all types at once. The file `code/lean/Book/Ch11/Iso.lean` starts by defining what an isomorphism is:

```
/-- An isomorphism between `α` and `β`: a function in each direction, and proofs
that each one undoes the other. -/
structure Iso (α β : Type) where
  /-- The forward function. -/
  toFun : α → β
  /-- The backward function. -/
  invFun : β → α
  /-- Going forward and then back is the identity on `α`. -/
  left_inv : ∀ a, invFun (toFun a) = a
```

```
  /-- Going back and then forward is the identity on `β`. -/
  right_inv : ∀ b, toFun (invFun b) = b

/-- `α ≅ β` is the type of isomorphisms between `α` and `β`. -/
scoped infix:25 " ≅ " => Iso
```

Compare this with the Rust `Iso`. Both have two functions. The Lean structure has two more fields, `left_inv` and `right_inv`, whose types are *propositions*: "for every `a`, going forward and back gives `a`", and the same in the other direction. A value of type `α ≅ β` therefore contains the two functions *and proofs* of both round trips. You cannot build one without proving the laws, in the same way that you cannot build a Rust struct without supplying every field. (The notation `≅` is declared `scoped`, so it is only active inside the chapter's namespace. Mathlib uses the same symbol for isomorphisms in a category, which is what our `Iso` will become in Chapter 16; here all our types live in `Type`, and Chapter 12 explains what lies above it.)

The first three constructions show that "isomorphic" behaves like equality: it is reflexive, symmetric and transitive.

```
/-- Every type is isomorphic to itself. -/
def refl (α : Type) : α ≅ α where
  toFun a := a
  invFun a := a
  left_inv _ := rfl
  right_inv _ := rfl

/-- An isomorphism can be run backwards. -/
def symm (i : α ≅ β) : β ≅ α where
  toFun := i.invFun
  invFun := i.toFun
  left_inv := i.right_inv
  right_inv := i.left_inv

/-- Isomorphisms compose. -/
def trans (i : α ≅ β) (j : β ≅ γ) : α ≅ γ where
  toFun a := j.toFun (i.toFun a)
  invFun c := i.invFun (j.invFun c)
  left_inv a := by
    -- Goal: i.invFun (j.invFun (j.toFun (i.toFun a))) = a
    rw [j.left_inv, i.left_inv]
  right_inv c := by
    -- Goal: j.toFun (i.toFun (i.invFun (j.invFun c))) = c
    rw [i.right_inv, j.right_inv]
```

The `where` syntax fills in the fields one by one. In `refl`, both functions are the identity and both round trips hold by `rfl`, since `(fun a => a) ((fun a => a) x)` computes to `x`. In `symm`, the fields are swapped, and so are the proofs: `i.right_inv` is exactly the statement needed for the new `left_inv`. In `trans`, the round trip `i.invFun (j.invFun (j.toFun (i.toFun a)))` is simplified from the inside out: `rw [j.left_inv]` rewrites `j.invFun (j.toFun _)` to its argument, leaving `i.invFun (i.toFun a)`, and `rw [i.left_inv]` finishes.

Units, commutativity and associativity

The law $A \times \mathbf{1} \cong A$:

```
/-- `α × 1 ≅ α`: pairing with the unit value adds no information. -/
def prodUnit (α : Type) : α × Unit ≅ α where
  toFun p := p.1
```

```
  invFun a := (a, ())
  left_inv := by
    intro ⟨a, u⟩  -- take the pair apart
    cases u       -- the only value of `Unit` is `()`
    rfl           -- goal: (a, ()) = (a, ()), true by computation
  right_inv _ := rfl
```

The interesting proof is `left_inv`, which must show `(p.1, ()) = p` for every pair `p`. The tactic `intro ⟨a, u⟩` introduces the pair and takes it apart in one step, naming its components `a` and `u`. Then `cases u` does case analysis on `u : Unit`, which has a single constructor, so it replaces `u` by `()`. The goal becomes `(a, ()) = (a, ())`, which `rfl` closes. The other round trip, `(a, ()).1 = a`, holds by computation.

Commutativity and associativity of products need no tactics at all:

```
/-- `α × β ≅ β × α`. -/
def prodComm (α β : Type) : α × β ≅ β × α where
  toFun p := (p.2, p.1)
  invFun p := (p.2, p.1)
  left_inv _ := rfl
  right_inv _ := rfl

/-- `(α × β) × γ ≅ α × (β × γ)`. -/
def prodAssoc (α β γ : Type) : (α × β) × γ ≅ α × (β × γ) where
  toFun p := (p.1.1, (p.1.2, p.2))
  invFun p := ((p.1, p.2.1), p.2.2)
  left_inv _ := rfl
  right_inv _ := rfl
```

For `prodComm`, `left_inv` must show that swapping twice gives back `p`, which computes to `(p.1, p.2) = p`. That `rfl` proves it is not obvious, since `p` is a variable, not a pair written out. Lean accepts it because of **eta for structures**: a value of a structure type is equal, by definition, to the structure built from its own projections.

Sums and the empty type

```
/-- `α + β ≅ β + α`. -/
def sumComm (α β : Type) : α ⊕ β ≅ β ⊕ α where
  toFun
    | .inl a => .inr a
    | .inr b => .inl b
  invFun
    | .inl b => .inr b
    | .inr a => .inl a
  left_inv s := by cases s <;> rfl
  right_inv s := by cases s <;> rfl

/-- `α + 0 ≅ α`: a summand with no values contributes nothing. -/
def sumEmpty (α : Type) : α ⊕ Empty ≅ α where
  toFun
    | .inl a => a
    | .inr e => nomatch e
  invFun a := .inl a
  left_inv s := by
    cases s with
    | inl a => rfl
    | inr e => nomatch e  -- no value of `Empty`: nothing to prove
  right_inv _ := rfl

/-- `α × 0 ≅ 0`: a pair needs a component of type `Empty`, which does not exist. -/
def prodEmpty (α : Type) : α × Empty ≅ Empty where
  toFun p := nomatch p.2
```

```
  invFun e := nomatch e
  left_inv p := nomatch p.2
  right_inv e := nomatch e
```

Two new pieces of syntax appear. First, `toFun` and `invFun` are defined by pattern matching directly after the field name, one line per constructor, as a `match` would. Second, `cases s <;> rfl` means: split `s` into its two cases, and run `rfl` on *every* resulting goal (the combinator `<;>` applies the tactic on its right to all goals produced by the tactic on its left).

The empty type appears in three places. In `sumEmpty`, the case `.inr e` cannot happen, and `nomatch e` disposes of it, both in the function and in the proof. In `prodEmpty`, *every* case is impossible: a pair of type `α × Empty` would contain an `Empty`, so all four fields are defined by `nomatch` on that component. The isomorphism $A \times \mathbf{0} \cong \mathbf{0}$ consists of two functions that can never be called and two proofs about values that do not exist.

Distributivity

```
/-- `α × (β + γ) ≅ α × β + α × γ`: distributivity. -/
def distrib (α β γ : Type) : α × (β ⊕ γ) ≅ (α × β) ⊕ (α × γ) where
  toFun
    | (a, .inl b) => .inl (a, b)
    | (a, .inr c) => .inr (a, c)
  invFun
    | .inl (a, b) => (a, .inl b)
    | .inr (a, c) => (a, .inr c)
  left_inv := by
    intro ⟨a, s⟩
    cases s <;> rfl
  right_inv := by
    intro s
    cases s <;> rfl
```

This is the Rust pair `distribute` / `factor`, plus proofs. To prove `left_inv`, we take the pair apart with `intro ⟨a, s⟩` and then split `s` into its two cases; in each, both sides compute to the same pair and `rfl` closes the goal.

Exponentials and function extensionality

The laws for function types need one more ingredient:

```
/-- `γ ^ (α × β) ≅ (γ ^ β) ^ α`: currying. -/
def curry (α β γ : Type) : (α × β → γ) ≅ (α → β → γ) where
  toFun f a b := f (a, b)
  invFun g p := g p.1 p.2
  left_inv f := by
    funext p  -- two functions are equal if they agree on every argument
    rfl       -- goal: f (p.1, p.2) = f p, true because p = (p.1, p.2)
  right_inv _ := rfl

/-- `α ^ (β + γ) ≅ α ^ β × α ^ γ`: a function out of a sum is a pair of
functions, one per summand. -/
def sumArrow (α β γ : Type) : (β ⊕ γ → α) ≅ (β → α) × (γ → α) where
  toFun f := (fun b => f (.inl b), fun c => f (.inr c))
  invFun p
    | .inl b => p.1 b
    | .inr c => p.2 c
  left_inv f := by
    funext s
    cases s <;> rfl
```

```
  right_inv _ := rfl

/-- `α ^ 1 ≅ α`: a function out of `Unit` is just one value. -/
def unitArrow (α : Type) : (Unit → α) ≅ α where
  toFun f := f ()
  invFun a := fun _ => a
  left_inv f := by
    funext u
    cases u
    rfl
  right_inv _ := rfl

/-- `1 ^ α ≅ 1`: there is exactly one function into `Unit`. -/
def arrowUnit (α : Type) : (α → Unit) ≅ Unit where
  toFun _ := ()
  invFun _ := fun _ => ()
  left_inv _ := rfl
  right_inv _ := rfl

/-- `α ^ 0 ≅ 1`: there is exactly one function out of `Empty`. -/
def emptyArrow (α : Type) : (Empty → α) ≅ Unit where
  toFun _ := ()
  invFun _ := fun e => nomatch e
  left_inv f := by
    funext e
    nomatch e
  right_inv _ := rfl
```

In `curry`, the goal of `left_inv` is an equation between *functions*: `(fun p => f (p.1, p.2)) = f`. The tactic `funext p` applies the principle of **function extensionality**: two functions are equal if they give equal results on every argument. It turns the goal into `f (p.1, p.2) = f p` for an arbitrary `p`, which `rfl` proves using eta for pairs again. (Function extensionality is a theorem in Lean, derived from the quotient types built into its logic; some other type theories must assume it as an axiom.) It is exactly what the Rust version could not check: Rust compared the two functions on 100 arguments, Lean compares them on all of them.

`sumArrow` is the law $A^{B+C} \cong A^B \times A^C$: the forward direction splits a function on a sum into its two restrictions, and the backward direction is a function defined by cases. `unitArrow`, `arrowUnit` and `emptyArrow` are the small exponents; in the last one, `funext e` followed by `nomatch e` proves that any two functions out of `Empty` are equal, because there is no argument on which they could differ.

The remaining laws of the table have the same shape:

```
/-- `Option α ≅ 1 + α`. -/
def optionSum (α : Type) : Option α ≅ Unit ⊕ α where
  toFun
    | none => .inl ()
    | some a => .inr a
  invFun
    | .inl () => none
    | .inr a => some a
  left_inv o := by cases o <;> rfl
  right_inv s := by
    cases s with
    | inl u => cases u; rfl
    | inr a => rfl

/-- `Bool ≅ 1 + 1`: a boolean is a choice between two empty messages. -/
def boolSum : Bool ≅ Unit ⊕ Unit where
  toFun b := if b then .inl () else .inr ()
  invFun
    | .inl () => true
```

```
    | .inr () => false
  left_inv b := by cases b <;> rfl
  right_inv s := by
    cases s with
    | inl u => cases u; rfl
    | inr u => cases u; rfl
```

Chaining isomorphisms like equations

On paper you would derive $A^2 \cong A \times A$ by a chain of equalities: $A^2 = A^{1+1} = A^1 \times A^1 = A \times A$. Each step replaces a part of the expression by something isomorphic. To do this with isomorphisms we need *congruence*: if the parts are isomorphic, so are the wholes.

```
/-- Isomorphic components give isomorphic products. -/
def prodCongr (i : α ≅ α') (j : β ≅ β') : α × β ≅ α' × β' where
  toFun p := (i.toFun p.1, j.toFun p.2)
  invFun p := (i.invFun p.1, j.invFun p.2)
  left_inv p := by simp only [i.left_inv, j.left_inv]
  right_inv p := by simp only [i.right_inv, j.right_inv]

/-- Isomorphic domains and codomains give isomorphic function types. -/
def arrowCongr (i : α ≅ α') (j : β ≅ β') : (α → β) ≅ (α' → β') where
  toFun f := fun a' => j.toFun (f (i.invFun a'))
  invFun g := fun a => j.invFun (g (i.toFun a))
  left_inv f := by
    funext a
    simp only [i.left_inv, j.left_inv]
  right_inv g := by
    funext a'
    simp only [i.right_inv, j.right_inv]
```

`arrowCongr` is worth reading slowly. To turn `f : α → β` into a function `α' → β'`, convert the argument back to `α` with `i.invFun`, apply `f`, and convert the result forward with `j.toFun`. The round-trip proofs use `simp only [...]`, which rewrites with the listed equations wherever they apply, as many times as needed: here it cancels each `invFun (toFun x)` pair. With these two lemmas, the chain of equalities becomes a chain of isomorphisms, glued by `trans`:

```
/-- `(Bool → α) ≅ α × α`, derived step by step like the equation
`α ^ 2 = α ^ (1 + 1) = α ^ 1 × α ^ 1 = α × α`. -/
def boolArrow (α : Type) : (Bool → α) ≅ α × α :=
  (arrowCongr boolSum (refl α))                        -- α ^ 2       ≅ α ^ (1 + 1)
    |>.trans (sumArrow α Unit Unit)                    --             ≅ α ^ 1 × α ^ 1
    |>.trans (prodCongr (unitArrow α) (unitArrow α))   --             ≅ α × α
```

The syntax `x |>.trans y` means `x.trans y`, written so that each step of the chain sits on its own line, next to the equation it implements. The result is not just a proof that the two types are isomorphic: it is a pair of functions that we can run:

```
/-- info: (10, 20) -/
#guard_msgs in
#eval (boolArrow ℕ).toFun (fun b => if b then 10 else 20)

/-- info: [7, 8] -/
#guard_msgs in
#eval [true, false].map ((boolArrow ℕ).invFun (7, 8))
```

The function `fun b => if b then 10 else 20` becomes the pair `(10, 20)`, and the pair `(7, 8)` becomes the function that sends `true` to `7` and `false` to `8`. We never wrote these conversions directly: they were assembled from the laws.

11.5 In Mathlib: `Equiv` and `Fintype.card`

Mathlib has the same notion under the name `Equiv`, written `α ≃ β`. Its fields have the same names and the same meaning as ours, so the translation in both directions is field by field:

```
/-- Our `Iso` is Mathlib's `Equiv`, field for field. -/
def toEquiv (i : α ≅ β) : α ≃ β where
  toFun := i.toFun
  invFun := i.invFun
  left_inv := i.left_inv
  right_inv := i.right_inv

/-- And back. -/
def ofEquiv (e : α ≃ β) : α ≅ β where
  toFun := e
  invFun := e.symm
  left_inv := e.left_inv
  right_inv := e.right_inv
```

(In `ofEquiv`, `toFun := e` uses a *coercion*: an `Equiv` can be used wherever a function is expected, and Lean inserts the conversion. Mathlib's documentation asks users to write `e x` and `e.symm y` rather than reading the fields directly.) Mathlib's library contains every isomorphism of this chapter and hundreds more, with systematic names:

```
/-- info: Equiv.prodSumDistrib ℕ Bool String : ℕ × (Bool ⊕ String) ≃ ℕ × Bool ⊕ ℕ × String
-/
#guard_msgs in
#check Equiv.prodSumDistrib ℕ Bool String

/-- info: Equiv.curry ℕ Bool String : (ℕ × Bool → String) ≃ (ℕ → Bool → String) -/
#guard_msgs in
#check Equiv.curry ℕ Bool String

/-- info: Sum.inr (3, "hi") -/
#guard_msgs in
#eval Equiv.prodSumDistrib ℕ Bool String (3, .inr "hi")

/-- An `Equiv` is applied like a function; `e.symm` is its inverse. -/
example : (Equiv.prodSumDistrib ℕ Bool String).symm (.inl (3, true)) = (3, .inl true) :=
  rfl

/-- Library isomorphisms combine with `Equiv.prodCongr`, `Equiv.trans` and so on. -/
example (α : Type) : α × (Bool ⊕ Empty) ≃ α × Bool :=
  Equiv.prodCongr (Equiv.refl α) (Equiv.sumEmpty Bool Empty)
```

Other useful entries are `Equiv.prodComm`, `Equiv.prodAssoc`, `Equiv.prodPUnit`, `Equiv.sumComm`, `Equiv.sumEmpty`, `Equiv.sumArrowEquivProdArrow` and `Equiv.optionEquivSumPUnit`. (`PUnit` is a version of `Unit` that can live in any universe; Chapter 12 explains universes.)

Counting with `Fintype.card`

For counting, Mathlib has the class `Fintype`: an instance of `Fintype α` is a finite list of all the values of `α`, without repetitions, in the spirit of our Rust trait `Finite`. `Fintype.card α` is the length of that

list. Instances exist for `Bool`, `Unit`, `Empty`, products, sums, `Option`, and function types whose domain has decidable equality. The file `code/lean/Book/Ch11/Cardinality.lean` puts them to work:

```
/-- info: 2 -/
#guard_msgs in
#eval Fintype.card Bool

/-- info: 6 -/
#guard_msgs in
#eval Fintype.card (Bool × Option Bool)

/-- info: 9 -/
#guard_msgs in
#eval Fintype.card (Bool → Option Bool)
```

These are computed from the enumeration that each `Fintype` instance provides. The counting laws are theorems of the library:

```
example (α β : Type) [Fintype α] [Fintype β] :
    Fintype.card (α × β) = Fintype.card α * Fintype.card β :=
  Fintype.card_prod α β

example (α β : Type) [Fintype α] [Fintype β] :
    Fintype.card (α ⊕ β) = Fintype.card α + Fintype.card β :=
  Fintype.card_sum

example (α β : Type) [Fintype α] [Fintype β] [DecidableEq α] :
    Fintype.card (α → β) = Fintype.card β ^ Fintype.card α :=
  Fintype.card_fun

example (α : Type) [Fintype α] : Fintype.card (Option α) = Fintype.card α + 1 :=
  Fintype.card_option
```

and with them `simp` can compute the cardinality of any type built from these formers, as an equation proved for good rather than a number printed once:

```
example : Fintype.card (Bool × Option Bool) = 6 := by simp
example : Fintype.card (Bool ⊕ Unit) = 3 := by simp
example : Fintype.card (Bool → Option Bool) = 9 := by simp
example : Fintype.card (Option Bool → Bool) = 8 := by simp
example : Fintype.card (Empty → Bool) = 1 := by simp  -- the empty function
example : Fintype.card (Bool → Empty) = 0 := by simp  -- no function at all
example : Fintype.card (Empty → Empty) = 1 := by simp -- 0 ^ 0 = 1
```

Each `simp` call rewrites `Fintype.card` of a compound type with the laws above, replaces `Fintype.card Bool` by `2`, `Fintype.card Empty` by `0` and so on, and finishes the arithmetic. The last three lines are our corner cases again, now proved.

The connection between the two halves of the chapter is a theorem: isomorphic types have the same cardinality. Mathlib proves it as `Fintype.card_congr`, and we can feed it our own isomorphisms through `toEquiv`:

```
/-- Isomorphic types have the same number of values. -/
theorem card_eq_of_iso {α β : Type} [Fintype α] [Fintype β] (i : α ≃ β) :
    Fintype.card α = Fintype.card β :=
  Fintype.card_congr i.toEquiv

/-- Distributivity of types implies distributivity of numbers. -/
theorem mul_add_from_types (α β γ : Type) [Fintype α] [Fintype β] [Fintype γ] :
    Fintype.card α * (Fintype.card β + Fintype.card γ) =
      Fintype.card α * Fintype.card β + Fintype.card α * Fintype.card γ := by
```

```
  have h := card_eq_of_iso (distrib α β γ)
  -- h : card (α × (β ⊕ γ)) = card ((α × β) ⊕ (α × γ))
  simpa [Fintype.card_prod, Fintype.card_sum] using h
```

The second theorem runs the algebra backwards. The isomorphism `distrib` holds for all types, so the cardinalities of both sides are equal, and simplifying both sides with the counting laws gives the distributive law of *numbers*. The tactic `simpa [...] using h` simplifies both the hypothesis `h` and the goal with the given lemmas, and closes the goal with the simplified `h`. Distributivity of multiplication over addition, for all natural numbers that are sizes of finite types (which is all of them), proved by an isomorphism of types.

For finite types the converse holds too: equal cardinality gives an isomorphism.

```
/-- For finite types the converse holds too: the same number of values means
isomorphic. Mathlib's statement: -/
example (α β : Type) [Fintype α] [Fintype β] :
    Fintype.card α = Fintype.card β ↔ Nonempty (α ≃ β) :=
  Fintype.card_eq

/-- So `Bool × Bool` and `Option (Option Bool)` are isomorphic (both have 4
values), even though nobody wrote the isomorphism down. -/
example : Nonempty (Bool × Bool ≃ Option (Option Bool)) :=
  Fintype.card_eq.mp (by simp)
```

`Nonempty (α ≃ β)` is the proposition "there exists an isomorphism", which asserts existence without handing you one. Any such isomorphism comes from numbering the values of both types, and is arbitrary: it has nothing to do with the structure of `Bool × Bool` or `Option (Option Bool)`. The isomorphisms that come from the laws are better: they are defined by the same program for every type, a property that Chapter 18 calls *naturality*.

11.6 Recursive types

So far every type has been built from smaller types. A list is built from *itself*: a list of `A` is either empty, or an `A` followed by a list of `A`. Written as an equation between types:

$$\text{List}\ A \cong \mathbf{1} + A \times \text{List}\ A$$

A recursive type is a solution of such an equation, $X \cong F(X)$ for some type expression F; the solution that recursive data types denote is the *least* one, written $\mu X.\, F(X)$ ("mu"), which contains exactly the values that can be built by applying the constructors finitely many times. So $\text{List}\ A = \mu X.\, \mathbf{1} + A \times X$. If you unfold the equation repeatedly and pretend that the algebra of numbers applies, you get

$$\text{List}\ A = \mathbf{1} + A + A^2 + A^3 + \ldots$$

which says something true: a list is empty, or a single element, or a pair of elements, or a triple, and so on. The number of lists of length n is $|A|^n$.

Lists in Rust, and why the `Box`

Rust writes the equation almost literally, with one extra word:

```
/// List<T> = 1 + T × List<T>. The `Box` stores the tail behind a pointer, so
/// that a `List<T>` has a size known at compile time.
```

```
#[derive(Debug, PartialEq)]
enum List<T> {
    Nil,
    Cons(T, Box<List<T>>),
}
```

Without the `Box`, the compiler refuses (`code/rust/compile_fail/ch11_recursive_without_box.rs`):

```
enum List<T> {
    Nil,
    Cons(T, List<T>),
}
```

```
error[E0072]: recursive type `List` has infinite size
 --> compile_fail/ch11_recursive_without_box.rs:5:1
  |
5 | enum List<T> {
  | ^^^^^^^^^^^^
6 |     Nil,
7 |     Cons(T, List<T>),
  |             ------- recursive without indirection
  |
help: insert some indirection (e.g., a `Box`, `Rc`, or `&`) to break the cycle
  |
7 |     Cons(T, Box<List<T>>),
  |             ++++       +
```

The problem is not with the type theory, which is happy with $\mu X.\, \mathbf{1} + A \times X$, but with the memory layout. Rust stores the fields of a variant *inline*, so the size of a `List<T>` would have to be at least the size of a `T` plus the size of a `List<T>`, which has no finite solution. Rust requires every type stored inline (every field, local variable and by-value argument) to be `Sized`, with a size known at compile time, and `List<T>` would not be. A `Box<List<T>>` is a pointer of fixed size, and the cycle is broken. Languages with a garbage collector store recursive fields behind pointers automatically; in Rust the indirection is visible because you decide where values live. (The example below prints `size_of::<List<u64>>() = 16`: eight bytes for the `u64`, eight for the pointer, and *nothing* for the tag that distinguishes `Nil` from `Cons`. The section on layout explains where the tag went.)

The equation $\mathrm{List}\ A \cong \mathbf{1} + A \times \mathrm{List}\ A$ is an isomorphism, and we can write both directions. Unrolling exposes one layer, rolling puts it back:

```
/// One layer of the list, with the tail left as it is: 1 + T × List<T>.
type Layer<T> = Option<(T, List<T>)>;

impl<T> List<T> {
    /// Unfold one layer.
    fn unroll(self) -> Layer<T> {
        match self {
            List::Nil => None,
            List::Cons(head, tail) => Some((head, *tail)),
        }
    }

    /// Fold one layer back.
    fn roll(layer: Layer<T>) -> List<T> {
        match layer {
            None => List::Nil,
            Some((head, tail)) => List::Cons(head, Box::new(tail)),
        }
    }
}
```

In Lean, the same isomorphism comes with its proofs. The file `code/lean/Book/Ch11/Recursive.lean` defines lists from scratch:

```
/-- Lists, from scratch: a list is empty, or a head followed by a list. -/
inductive MyList (α : Type) where
  | nil : MyList α
  | cons : α → MyList α → MyList α
deriving Repr
```

```
/-- Unfold one layer: a list *is* `1 + α × MyList α`. -/
def unroll : MyList α → Unit ⊕ (α × MyList α)
  | nil => .inl ()
  | cons a as => .inr (a, as)

/-- Fold one layer back. -/
def roll : Unit ⊕ (α × MyList α) → MyList α
  | .inl () => nil
  | .inr (a, as) => cons a as

/-- `MyList α` is a solution of the equation `X ≅ 1 + α × X`. -/
def fixedPoint (α : Type) : MyList α ≅ Unit ⊕ (α × MyList α) where
  toFun := unroll
  invFun := roll
  left_inv xs := by cases xs <;> rfl
  right_inv s := by
    rcases s with ⟨⟩ | ⟨a, as⟩  -- `()` on the left, a pair on the right
    · rfl
    · rfl
```

`left_inv` is proved by cases on the list. For `right_inv`, the pattern `rcases s with ⟨⟩ | ⟨a, as⟩` splits the sum into its two cases and, at the same time, takes apart the `()` on the left (the empty pattern `⟨⟩` matches the only value of `Unit`) and the pair on the right. Both cases then hold by `rfl`.

The fold is the elimination rule

Products have projections and sums have `case`. What is the elimination rule of a recursive type? To use a list, you must say what to do with the empty list, and what to do with a head and a tail; but the tail is again a list, and the natural thing to do with it is to *use it the same way*, recursively. That is the **fold**: given a result `nil` for the empty list and a function `cons` that combines a head with the result already computed for the tail, it computes a result for every list.

```
impl<T> List<T> {
    /// The elimination rule for lists: a result for `Nil`, and a way to
    /// combine a head with the result already computed for the tail.
    fn fold<R>(self, nil: R, cons: &impl Fn(T, R) -> R) -> R {
        match self {
            List::Nil => nil,
            List::Cons(head, tail) => {
                let rest = tail.fold(nil, cons);
                cons(head, rest)
            }
        }
    }

    fn from_vec(items: Vec<T>) -> List<T> {
        let mut list = List::Nil;
        for item in items.into_iter().rev() {
            list = List::Cons(item, Box::new(list));
        }
        list
    }
}
```

```rust
fn length<T>(list: List<T>) -> usize {
    list.fold(0, &|_, n| n + 1)
}

fn sum(list: List<u64>) -> u64 {
    list.fold(0, &|x, total| x + total)
}

fn map<T, U>(list: List<T>, f: impl Fn(T) -> U) -> List<U> {
    list.fold(List::Nil, &|x, mapped| List::Cons(f(x), Box::new(mapped)))
}
```

```rust
let list = List::from_vec(vec![3, 4, 5]);
println!("list     = {list:?}");
let layer = list.unroll();
println!("unrolled = {layer:?}");
let list = List::roll(layer);
assert_eq!(list, List::from_vec(vec![3, 4, 5]));

println!("length   = {}", length(List::from_vec(vec!['a', 'b', 'c'])));
println!("sum      = {}", sum(list));
println!(
    "map      = {:?}",
    map(List::from_vec(vec![1, 2]), |x| x * 10)
);
println!("size_of::<List<u64>>() = {}", size_of::<List<u64>>());
```

```
list     = Cons(3, Cons(4, Cons(5, Nil)))
unrolled = Some((3, Cons(4, Cons(5, Nil))))
length   = 3
sum      = 12
map      = Cons(10, Cons(20, Nil))
size_of::<List<u64>>() = 16
```

`length`, `sum` and `map` contain no recursion of their own: the recursion is inside `fold`, written once. The Lean version is the same program:

```
/-- The fold: give a result for the empty list, and a way to combine a head
with the result for the tail. -/
def fold (n : β) (c : α → β → β) : MyList α → β
  | nil => n
  | cons a as => c a (fold n c as)

def length (xs : MyList α) : ℕ := xs.fold 0 (fun _ k => k + 1)
def sum (xs : MyList ℕ) : ℕ := xs.fold 0 (fun a s => a + s)
def map (f : α → β) (xs : MyList α) : MyList β :=
  xs.fold nil (fun a bs => cons (f a) bs)
```

```
/-- info: 3 -/
#guard_msgs in
#eval (cons 4 (cons 5 (cons 6 nil))).length

/-- info: 15 -/
#guard_msgs in
#eval (cons 4 (cons 5 (cons 6 nil))).sum

/-- info: Ch11.MyList.cons 40 (Ch11.MyList.cons 50 (Ch11.MyList.nil)) -/
#guard_msgs in
#eval (cons 4 (cons 5 nil)).map (· * 10)
```

Why call the fold *the* elimination rule rather than one useful function among many? Because of the following theorem, which says that the two equations defining `fold` determine it completely:

```
/-- Any function `h` that satisfies the two equations of `fold` *is* `fold`.
This is what "the fold is the elimination principle of lists" means. -/
theorem fold_unique (n : β) (c : α → β → β) (h : MyList α → β)
    (h_nil : h nil = n) (h_cons : ∀ a as, h (cons a as) = c a (h as)) :
    ∀ xs, h xs = xs.fold n c := by
  intro xs
  induction xs with
  | nil =>
    -- Goal: h nil = fold n c nil, and the right side computes to n.
    rw [h_nil]
    rfl
  | cons a as ih =>
    -- ih   : h as = fold n c as
    -- Goal : h (cons a as) = fold n c (cons a as)
    rw [h_cons, ih]
    rfl

/-- Folding with the constructors themselves rebuilds the list. -/
theorem fold_nil_cons (xs : MyList α) : xs.fold nil cons = xs := by
  induction xs with
  | nil => rfl
  | cons a as ih =>
    show cons a (as.fold nil cons) = cons a as
    rw [ih]
```

The statement of `fold_unique` reads: if a function `h` sends `nil` to `n`, and sends `cons a as` to `c a (h as)` for all `a` and `as`, then `h` *is* the fold, on every list. The proof is an induction on the list (Chapter 2 and Chapter 1 showed `induction`). In the `nil` case we rewrite `h nil` to `n` with the hypothesis `h_nil`, and `fold n c nil` computes to `n`. In the `cons` case the hypothesis `h_cons` turns `h (cons a as)` into `c a (h as)`, the induction hypothesis `ih` turns `h as` into `fold n c as`, and the right-hand side computes to the same thing. So any function on lists that is defined by "a value for `nil`, and a way to extend the result for the tail" is a fold. (Category theorists say that lists form an *initial algebra*, and the fold is the unique homomorphism out of it.)

`fold_nil_cons` is a pleasant special case: folding with the constructors themselves rebuilds the list. In the `cons` case, `show` restates the goal in the form it takes after one step of computation (Lean checks that the restated goal is the same up to computation), and `rw [ih]` finishes. Two consequences:

```
/-- Two consequences of uniqueness, proved by induction: `map` preserves the
length, and mapping the identity does nothing. -/
theorem length_map (f : α → β) (xs : MyList α) : (xs.map f).length = xs.length := by
  induction xs with
  | nil => rfl
  | cons a as ih =>
    show (as.map f).length + 1 = as.length + 1
    rw [ih]

theorem map_id (xs : MyList α) : xs.map (fun a => a) = xs :=
  fold_nil_cons xs
```

The fold does not capture every recursive function on lists directly: a function that needs the tail itself, not just the result computed from it, does not fit its shape. Lean's most general elimination principle for a type is its **recursor**, which Lean generates automatically for every `inductive` declaration and which also hands you the tail:

```
/-- The recursor that Lean generated for `MyList`, used to prove a property `P`
of every list: prove it for `nil`, and prove it for `cons a as` assuming it for
`as`. This is the induction principle that the `induction` tactic applies. -/
```

```
example {α : Type} (P : MyList α → Prop)
    (h_nil : P .nil) (h_cons : ∀ a as, P as → P (.cons a as)) : ∀ xs, P xs :=
  fun xs => MyList.rec h_nil h_cons xs

/-- The same recursor, used to compute a value of a fixed type `β`, is the fold. -/
example {α β : Type} (n : β) (c : α → β → β) (xs : MyList α) :
    MyList.rec (motive := fun _ => β) n (fun a _ r => c a r) xs = xs.fold n c := by
  induction xs with
  | nil => rfl
  | cons a as ih => exact congrArg (c a) ih
```

The first `example` uses `MyList.rec` to prove a property `P` of every list: the argument `h_cons` receives the head `a`, the tail `as` *and* a proof of `P as`. This is the induction principle, and the `induction` tactic applies it for you. The second `example` shows that the same recursor, used to compute a value of a fixed type `β` and ignoring the tail, is our `fold`. The recursor is the elimination rule; induction and the fold are two ways of using it. (Its argument `motive`, given explicitly in the second example, says what is computed or proved for each list. It may be a type or a proposition, which is why the recursor's full type, shown by `#check @MyList.rec`, mentions the `Sort u` of Chapter 12.)

Empty and infinite fixed points

Not every equation has the solution you expect. Consider $X \cong X$, or in Rust, a type whose only variant contains the type itself:

```
/// μX. X: the only variant needs a value of the type itself. The type is
/// accepted, but no value of it can ever be built.
#[allow(dead_code)]
enum Loop {
    Next(Box<Loop>),
}

/// Like `Infallible`, a `Loop` can be turned into anything; unlike it, the
/// proof is a recursion that would never end (and is never started). The
/// compiler notices that this function cannot return, and warns: we allow it.
#[allow(dead_code, unconditional_recursion)]
fn absurd_loop<T>(l: Loop) -> T {
    match l {
        Loop::Next(inner) => absurd_loop(*inner),
    }
}
```

The `Box` gives it a size, so Rust accepts the type. But you can never build a value of it: to build a `Loop` you need a `Loop` first. Its least fixed point is empty, $\mu X.\, X \cong \mathbf{0}$, and indeed the function `absurd_loop` turns a `Loop` into anything, like `absurd` for `Void`. The compiler even notices that the function can never return and warns about unconditional recursion, which we silence because here it is the point. In Lean the emptiness is a theorem, proved by induction:

```
/-- `μX. X`: a type whose only constructor needs a value of the type itself. -/
inductive Loop where
  | next : Loop → Loop

/-- No value of `Loop` can ever be built: by induction, every `Loop` gives a
proof of `False`. -/
theorem Loop.elim (l : Loop) : False := by
  induction l with
  | next _ ih => exact ih

/-- So `Loop` is isomorphic to the empty type. -/
def loopEmpty : Loop ≃ Empty where
```

```
  toFun l := (Loop.elim l).elim
  invFun e := nomatch e
  left_inv l := (Loop.elim l).elim
  right_inv e := nomatch e
```

In the induction, there is a single case, `next l`, and the induction hypothesis `ih` already proves `False` for the inner `l`. There is no base case, so the induction never needs to start from anything.

The opposite surprise is a type that solves an equation no number solves:

```
/-- `ℕ ≃ 1 + ℕ`: the natural numbers solve `X ≃ 1 + X`. In arithmetic,
`n = 1 + n` has no solution; for infinite types it does, so the algebra of
types cannot be "cancelled" like the algebra of numbers. -/
def natFixedPoint : ℕ ≃ Unit ⊕ ℕ where
  toFun
    | 0 => .inl ()
    | n + 1 => .inr n
  invFun
    | .inl () => 0
    | .inr n => n + 1
  left_inv n := by cases n <;> rfl
  right_inv s := by
    rcases s with (()) | n
    · rfl
    · rfl
```

$\mathbb{N} \cong \mathbf{1} + \mathbb{N}$: a natural number is zero or the successor of a natural number, which is the definition of `Nat`. For numbers, $n = 1 + n$ has no solution, and you can subtract n from both sides to get $0 = 1$. For types you cannot: the algebra of types has no subtraction and no cancellation in general. The counting laws apply to finite types; for infinite types they become the laws of cardinal arithmetic, where $\infty = 1 + \infty$ is perfectly fine.

11.7 Positivity: which recursive types are allowed

Rust accepts any recursive type that has a finite size. Lean is pickier. Try to define a type that contains functions *out of* itself, in `code/lean/Book/Ch11/Positivity.lean`:

```
/--
error: (kernel) arg #1 of 'Ch11.Bad.mk' has a non positive occurrence of the datatypes being
declared
-/
#guard_msgs in
inductive Bad where
  | mk : (Bad → ℕ) → Bad
```

The message comes from Lean's *kernel*, the small trusted core that checks every definition (Chapter 1). An inductive type must be **strictly positive**: the type being defined may appear in the types of its constructors' arguments only as the argument itself, or as the *result* of a function type whose domain does not mention it, or inside another inductive type that is itself positive in that position. It must never appear to the *left* of an arrow. In `Bad.mk`, the argument has type `Bad → ℕ`, and `Bad` is on the left.

Positive occurrences are fine, even when they are infinite or nested:

```
/-- Infinitely branching trees: `Tree` occurs to the right of the arrow, which is
fine. A node holds one subtree for every natural number. -/
inductive Tree where
```

```
  | leaf : Tree
  | node : (ℕ → Tree) → Tree

/-- Rose trees: a node holds a list of subtrees. `Rose` occurs inside `List`,
which Lean checks is itself positive in its parameter. -/
inductive Rose where
  | node : String → List Rose → Rose
```

A `Tree.node` holds a *function* from `ℕ` to subtrees, so a node has infinitely many children, and that is allowed because `Tree` is on the right of the arrow. A `Rose.node` holds a `List Rose`; Lean looks inside the definition of `List`, sees that its parameter appears only positively, and accepts. It also looks inside other structures, so a negative occurrence cannot be hidden:

```
/-- A negative occurrence hidden behind a type constructor is found too. -/
structure Pred (α : Type) where
  test : α → Bool

/--
error: (kernel) arg #1 of '_nested.Ch11.Pred_1.mk' has a non positive occurrence of the
datatypes being declared
-/
#guard_msgs in
inductive Sneaky where
  | mk : Pred Sneaky → Sneaky
```

Why the rule exists

The rule is there because a negative type would make Lean's logic inconsistent. Suppose a type `B` had a constructor `mk : (B → False) → B`. Every inductive type also comes with a way to take its values apart, so there would be a function `unmk : B → (B → False)`. With these two functions alone we can prove `False`:

```
/-- Suppose some type `B` had a constructor `mk : (B → False) → B` and, as every
inductive type does, a way `unmk` to take a value apart again. Then we could
prove `False`. -/
theorem no_negative_false (B : Type) (mk : (B → False) → B) (unmk : B → B → False) :
    False :=
  let f : B → False := fun b => unmk b b  -- "b applied to itself"
  f (mk f)                                -- f applied to (a code for) itself
```

Read the proof as a program. `f` takes a value `b` and applies `b`'s own function to `b` itself. Then `f (mk f)` packages `f` as a value of `B` and feeds it to `f`. The result has type `False`, so it is a proof of `False`, and the theorem is proved with no assumptions beyond the existence of `mk` and `unmk`. Under Curry–Howard (Chapter 10), a type with $B \cong (B \to \mathbf{0})$ is a proposition equivalent to its own negation: the liar paradox, "this sentence is false".

With `ℕ` in place of `False`, the same trick gives a function that would have to satisfy $n = n + 1$. This is Cantor's diagonal argument:

```
/-- The same diagonal argument for `Bad`: if `mk` and `unmk` existed, with the
computation rule `unmk (mk f) = f`, then some number would equal its successor. -/
theorem no_bad (B : Type) (mk : (B → ℕ) → B) (unmk : B → B → ℕ)
    (unmk_mk : ∀ f, unmk (mk f) = f) : False := by
  -- the diagonal function: apply a value to itself, and add one
  let f : B → ℕ := fun b => unmk b b + 1
  have h : f (mk f) = f (mk f) + 1 :=
    calc f (mk f) = unmk (mk f) (mk f) + 1 := rfl
```

```
      _ = f (mk f) + 1 := by rw [unmk_mk]
  -- n = n + 1 is impossible for natural numbers
  omega
```

The proof uses `calc`, which proves an equation as a chain of steps, each with its own justification: `f (mk f)` is by definition `unmk (mk f) (mk f) + 1` (proved by `rfl`), and rewriting `unmk (mk f)` to `f` with the computation rule gives `f (mk f) + 1`. The hypothesis `h` then says that some natural number equals its successor, and `omega`, the decision procedure for linear arithmetic, derives the contradiction.

These theorems do not mention `Bad` (which Lean refused to create): they say that *no* type, of any kind, can have such a pair of functions (with, in the second case, the computation rule that every inductive type provides). Positivity is the syntactic check that guarantees an `inductive` declaration never asks for one.

> **NOTE**
>
> Lean has an escape hatch, `unsafe inductive`, which skips the positivity check. Unsafe definitions can be used in programs, but not in anything that the logic relies on: no theorem may depend on them. This book never uses it.

Rust accepts negative types

Rust has no positivity check, and the equivalent of `Bad` compiles. The function inside may even take the type itself as an argument:

```
/// A type that occurs to the *left* of an arrow in its own definition: a
/// `SelfApply` contains a function that takes a `SelfApply`. Lean rejects
/// the analogous `inductive`; Rust accepts it.
struct SelfApply(Box<dyn Fn(&SelfApply, u64) -> u64>);

/// Factorial, without any function that calls itself by name: the closure
/// receives *itself* as an argument and uses it for the recursive call.
fn factorial(n: u64) -> u64 {
    let fact = SelfApply(Box::new(
        |me: &SelfApply, k: u64| {
            if k == 0 { 1 } else { k * (me.0)(me, k - 1) }
        },
    ));
    (fact.0)(&fact, n)
}
```

```
factorial(10) = 3628800
```

There is no recursive *function* in this program: `factorial` does not call itself, and the closure has no name to call. The recursion comes entirely from the type: the closure receives a `SelfApply` that contains the closure itself, and calls it. This is the self-application trick of the Y combinator from Chapter 7, made typable by a negative recursive type. Replace the body by `(me.0)(me, k)` and you get a recursion that never ends (until the stack overflows), written without any recursive function.

For Rust this is harmless. As Chapter 10 showed, Rust is not consistent as a logic anyway: `loop {}` and `panic!()` inhabit every type. For Lean, a non-terminating term of type `False` would be a proof of `False`, and every theorem in Mathlib would become worthless. Positivity, together with the termination checker of Chapter 2, keeps every definition that proofs can compute with terminating.

11.8 Layout: the algebra in memory

The number of values of a type tells you how much information a value carries: a type with n values needs about $\log_2 n$ bits. Rust's compiler uses the algebra to choose layouts, and `std::mem::size_of` lets us watch it. The last part of `ch11_type_algebra.rs` prints a table (on a 64-bit machine):

```
fn layout_table() {
    fn row<T>(name: &str, values: &str) {
        println!("{name:<26} {values:>9} {:>6}", size_of::<T>());
    }
    println!("{:<26} {:>9} {:>6}", "type", "values", "bytes");
    row::<()>("()", "1");
    row::<Void>("Void", "0");
    row::<bool>("bool", "2");
    row::<Option<bool>>("Option<bool>", "3");
    row::<Option<Option<bool>>>("Option<Option<bool>>", "4");
    row::<u32>("u32", "2^32");
    row::<Option<u32>>("Option<u32>", "1+2^32");
    row::<NonZeroU32>("NonZeroU32", "2^32-1");
    row::<Option<NonZeroU32>>("Option<NonZeroU32>", "2^32");
    row::<char>("char", "1112064");
    row::<Option<char>>("Option<char>", "1112065");
    row::<&u8>("&u8", "");
    row::<Option<&u8>>("Option<&u8>", "");
    row::<Box<u64>>("Box<u64>", "");
    row::<Option<Box<u64>>>("Option<Box<u64>>", "");
    row::<Vec<u8>>("Vec<u8>", "");
    row::<Option<Vec<u8>>>("Option<Vec<u8>>", "");
    row::<Result<u32, Infallible>>("Result<u32, Infallible>", "2^32");
    row::<(u8, u32)>("(u8, u32)", "2^40");
    row::<Option<(u8, u32)>>("Option<(u8, u32)>", "1+2^40");
}
```

```
type                          values  bytes
()                                 1      0
Void                               0      0
bool                               2      1
Option<bool>                       3      1
Option<Option<bool>>               4      1
u32                             2^32      4
Option<u32>                    1+2^32      8
NonZeroU32                     2^32-1      4
Option<NonZeroU32>              2^32      4
char                           1112064      4
Option<char>                   1112065      4
&u8                                       8
Option<&u8>                               8
Box<u64>                                  8
Option<Box<u64>>                          8
Vec<u8>                                  24
Option<Vec<u8>>                          24
Result<u32, Infallible>         2^32      4
(u8, u32)                       2^40      8
Option<(u8, u32)>              1+2^40     12
```

Read it from the top.

- **Zero-sized types.** `()` has one value, so it carries no information and takes zero bytes. Unit structs (`struct Marker;`) are zero-sized for the same reason, which is what makes the marker types of Chapter 25 free.
- **Empty types.** `Void` takes zero bytes too: there will never be a value to store. `Result<u32, Infallible>` has the size of `u32`: the law $A + \mathbf{0} \cong A$ holds in memory as well, and no tag is needed.

- **Niches.** A `bool` occupies a byte but uses only 2 of its 256 bit patterns. `Option<bool>` stores `None` in an unused pattern, a **niche**, and still fits in one byte; `Option<Option<bool>>` uses another one. A `char` has 1,112,064 values in 32 bits, which leaves many niches. `NonZeroU32` gives up the value `0`, and `Option<NonZeroU32>` puts `None` there.
- **No niche.** A `u32` uses all 2^{32} patterns, and $1 + 2^{32}$ values do not fit in 32 bits, so `Option<u32>` needs a separate tag (8 bytes, with alignment).
- **Pointers.** A reference or a `Box` is never null, so `Option<&u8>` and `Option<Box<u64>>` use the null pointer for `None`. The same happens inside `Vec` (whose buffer pointer is never null) and inside our `List<u64>`, where `Nil` is the null pointer in the `Box` field. That is where its tag went.
- **Padding is not a niche.** `(u8, u32)` is 8 bytes: 5 of data and 3 of padding to keep the `u32` aligned. The padding bytes can hold anything (copying a tuple does not preserve them), so the compiler cannot store a tag there, and `Option<(u8, u32)>` grows to 12 bytes.

The example also states the niche optimization as assertions:

```
// The niche optimization, as assertions: when the payload has a bit
// pattern that no valid value uses, `Option` stores `None` there.
assert_eq!(size_of::<Option<&u8>>(), size_of::<&u8>());
assert_eq!(size_of::<Option<Box<u64>>>(), size_of::<Box<u64>>());
assert_eq!(size_of::<Option<NonZeroU32>>(), size_of::<u32>());
assert!(size_of::<Option<u32>>() > size_of::<u32>());
```

> **IMPORTANT**
>
> Only some of these layouts are promises. The documentation of `std::option` guarantees that `Option<T>` has the same size as `T` when `T` is a reference, a `Box`, a function pointer, a `NonZero` integer or `NonNull`, which is what makes `Option<&T>` usable as a nullable pointer across a C interface. The others (`Option<bool>`, `Option<char>`, `Option<Vec<u8>>`) are what the current compiler does, and code should not depend on them.

11.9 Derivatives of types and zippers

This bonus section shows a beautiful piece of type algebra that produces a useful data structure. Take a data structure and punch a hole in it where one element used to be. What is the type of "the structure with a hole"?

For a pair $X \times X$, the hole is either in the first component, and then the second component is still there, or in the second, and then the first is still there. So the type of pairs with a hole is $X + X = 2X$. Now notice that $\frac{d}{dX} X^2 = 2X$. This is not a coincidence. Write ∂F for the type of one-hole contexts of $F(X)$ (positions where an X was removed). Then:

Type $F(X)$	One-hole contexts ∂F	Why
C (no X in it)	$\mathbf{0}$	there is no X to remove
X	$\mathbf{1}$	the hole is the whole thing
$F + G$	$\partial F + \partial G$	the hole is in the left or the right alternative
$F \times G$	$\partial F \times G + F \times \partial G$	the hole is in F (and G is intact), or in G

These are the rules of differentiation from calculus, including the product rule. Conor McBride noticed this correspondence in the early 2000s, and it gives a recipe for a data structure that Gérard Huet had described in 1997: the **zipper**, a structure with a *focus* that can move around and be edited in constant time per step.

For lists, $L = \mathbf{1} + X \times L$. Differentiating both sides with the rules gives $\partial L = L + X \times \partial L$, an equation of the same shape as the list equation, whose solution is a list of X followed by an L: $\partial L \cong L \times L$. A list with a hole is the list of elements before the hole and the list of elements after it. Put an element in the hole and you get a list with a focused position: $L \times X \times L$.

A list zipper in Lean

The file `code/lean/Book/Ch11/Zipper.lean` turns this into a data structure:

```
/-- A list with one position in focus. `before` holds the elements to the left
of the focus, nearest first (so it is stored reversed). -/
structure Zipper (α : Type) where
  before : List α
  focus : α
  after : List α
deriving Repr

namespace Zipper

variable {α : Type}

/-- Forget the focus: the list the zipper represents. -/
def toList (z : Zipper α) : List α := z.before.reverse ++ z.focus :: z.after

/-- Focus on the first element of a nonempty list. -/
def ofList : List α → Option (Zipper α)
  | [] => none
  | a :: as => some ⟨[], a, as⟩

/-- Move the focus one step right, if there is an element there. -/
def right (z : Zipper α) : Option (Zipper α) :=
  match z.after with
  | [] => none
  | b :: bs => some ⟨z.focus :: z.before, b, bs⟩

/-- Move the focus one step left, if there is an element there. -/
def left (z : Zipper α) : Option (Zipper α) :=
  match z.before with
  | [] => none
  | b :: bs => some ⟨bs, b, z.focus :: z.after⟩

/-- Change the focused element: constant time, wherever the focus is. -/
def modify (f : α → α) (z : Zipper α) : Zipper α := { z with focus := f z.focus }
```

`before` holds the elements to the left of the focus with the nearest one first, so moving the focus is a constant-time operation: `right` pushes the old focus onto `before` and takes the new focus from

`after`. Moves can fail at the ends, so they return an `Option`, and a `do` block chains them, stopping at the first `none`:

```
/-- info: some [1, 2, 300, 4] -/
#guard_msgs in
#eval do
  let z ← ofList [1, 2, 3, 4]
  let z ← z.right
  let z ← z.right
  pure (z.modify (· * 100)).toList
```

A zipper is only useful if moving does not corrupt the data, and that is something we can prove:

```
/-- Moving right does not change the list. -/
theorem toList_right {z z' : Zipper α} (h : z.right = some z') :
    z'.toList = z.toList := by
  rcases z with ⟨bef, x, aft⟩
  cases aft with
  | nil =>
    -- `right` returned `none`, so `h : none = some z'` is impossible.
    simp [right] at h
  | cons b bs =>
    -- `right` returned the zipper ⟨x :: bef, b, bs⟩, so that is z'.
    simp only [right, Option.some.injEq] at h
    subst h
    -- Goal: (x :: bef).reverse ++ b :: bs = bef.reverse ++ x :: b :: bs
    simp [toList]

/-- A step right followed by a step left comes back to the same zipper. -/
theorem left_of_right {z z' : Zipper α} (h : z.right = some z') :
    z'.left = some z := by
  rcases z with ⟨bef, x, aft⟩
  cases aft with
  | nil => simp [right] at h
  | cons b bs =>
    simp only [right, Option.some.injEq] at h
    subst h
    rfl

/-- Focusing on a list does not change it either. -/
theorem toList_ofList {xs : List α} {z : Zipper α} (h : ofList xs = some z) :
    z.toList = xs := by
  cases xs with
  | nil => simp [ofList] at h
  | cons a as =>
    simp only [ofList, Option.some.injEq] at h
    subst h
    rfl
```

Take `toList_right` step by step. `rcases z with ⟨bef, x, aft⟩` takes the zipper apart, and we split on `aft`, since `right` looks at it. If `aft` is empty, the hypothesis `h` says `none = some z'`, and `simp [right] at h` reduces it to `False`, which closes the goal. If `aft = b :: bs`, then `simp only [right, Option.some.injEq] at h` unfolds `right` and uses the fact that `some` is injective, turning `h` into `⟨x :: bef, b, bs⟩ = z'`, and `subst h` replaces `z'` by that zipper. What remains, `(x :: bef).reverse ++ b :: bs = bef.reverse ++ x :: b :: bs`, is proved by `simp` with the library's lemmas about `reverse` and `++`. The other two proofs follow the same pattern.

A tree zipper in Rust

The same recipe works for trees, and gives a cursor for an editor or a refactoring tool: focus on a subexpression of a syntax tree, rewrite it, move on. The example `code/rust/examples/ch11_zipper.rs` uses arithmetic expressions:

```
/// Expressions: E = Num + Var + E × E + E × E.
#[derive(Clone, Debug, PartialEq)]
enum Expr {
    Num(i64),
    Var(char),
    Add(Box<Expr>, Box<Expr>),
    Mul(Box<Expr>, Box<Expr>),
}
```

As a type equation, $E = N + V + E \times E + E \times E$, where N and V are the constants `i64` and `char`. One layer of context, a hole for a subexpression directly inside a node, is the derivative with respect to E: $\partial = \mathbf{0} + \mathbf{0} + (E + E) + (E + E)$. Four variants, each holding the sibling that stays behind:

```
/// One step on the way from the root down to the cursor: which operand of
/// which constructor we went into, and the operand we left behind. These four
/// variants are the four terms of the derivative ∂E = E + E + E + E: the
/// constants `Num` and `Var` contribute nothing, and each `E × E` contributes
/// "hole on the left, keep the right" plus "keep the left, hole on the right".
#[derive(Clone, Debug)]
enum Frame {
    AddLeft(Expr),
    AddRight(Expr),
    MulLeft(Expr),
    MulRight(Expr),
}

/// Fill the hole of a frame with an expression.
fn plug(frame: Frame, e: Expr) -> Expr {
    match frame {
        Frame::AddLeft(right) => add(e, right),
        Frame::AddRight(left) => add(left, e),
        Frame::MulLeft(right) => mul(e, right),
        Frame::MulRight(left) => mul(left, e),
    }
}
```

A hole deep inside the tree is described by the path from the root: a list of frames. The zipper is that path together with the focused subexpression:

```
/// The cursor: the subexpression in focus, and the path of frames from the
/// root down to it (innermost frame last).
#[derive(Clone, Debug)]
struct Zipper {
    focus: Expr,
    path: Vec<Frame>,
}

impl Zipper {
    fn new(e: Expr) -> Zipper {
        Zipper {
            focus: e,
            path: Vec::new(),
        }
    }

    /// Move into the left operand. Fails on a leaf.
    fn left(mut self) -> Option<Zipper> {
        let (child, frame) = match self.focus {
```

```
            Expr::Add(a, b) => (*a, Frame::AddLeft(*b)),
            Expr::Mul(a, b) => (*a, Frame::MulLeft(*b)),
            Expr::Num(_) | Expr::Var(_) => return None,
        };
        self.path.push(frame);
        Some(Zipper {
            focus: child,
            path: self.path,
        })
    }

    /// Move into the right operand. Fails on a leaf.
    fn right(mut self) -> Option<Zipper> {
        let (child, frame) = match self.focus {
            Expr::Add(a, b) => (*b, Frame::AddRight(*a)),
            Expr::Mul(a, b) => (*b, Frame::MulRight(*a)),
            Expr::Num(_) | Expr::Var(_) => return None,
        };
        self.path.push(frame);
        Some(Zipper {
            focus: child,
            path: self.path,
        })
    }

    /// Move to the parent: plug the focus into the innermost frame.
    fn up(mut self) -> Option<Zipper> {
        let frame = self.path.pop()?;
        Some(Zipper {
            focus: plug(frame, self.focus),
            path: self.path,
        })
    }

    /// Replace the subexpression under the cursor.
    fn replace(self, e: Expr) -> Zipper {
        Zipper {
            focus: e,
            path: self.path,
        }
    }

    /// Close the zipper: plug the focus into every frame, innermost first.
    fn rebuild(self) -> Expr {
        self.path
            .into_iter()
            .rev()
            .fold(self.focus, |e, frame| plug(frame, e))
    }

    /// The whole expression with a hole `[ ]` where the cursor is.
    fn context(&self) -> String {
        let mut text = String::from("[ ]");
        for frame in self.path.iter().rev() {
            text = match frame {
                Frame::AddLeft(r) => format!("({text} + {r})"),
                Frame::AddRight(l) => format!("({l} + {text})"),
                Frame::MulLeft(r) => format!("({text} * {r})"),
                Frame::MulRight(l) => format!("({l} * {text})"),
            };
        }
        text
    }
}
```

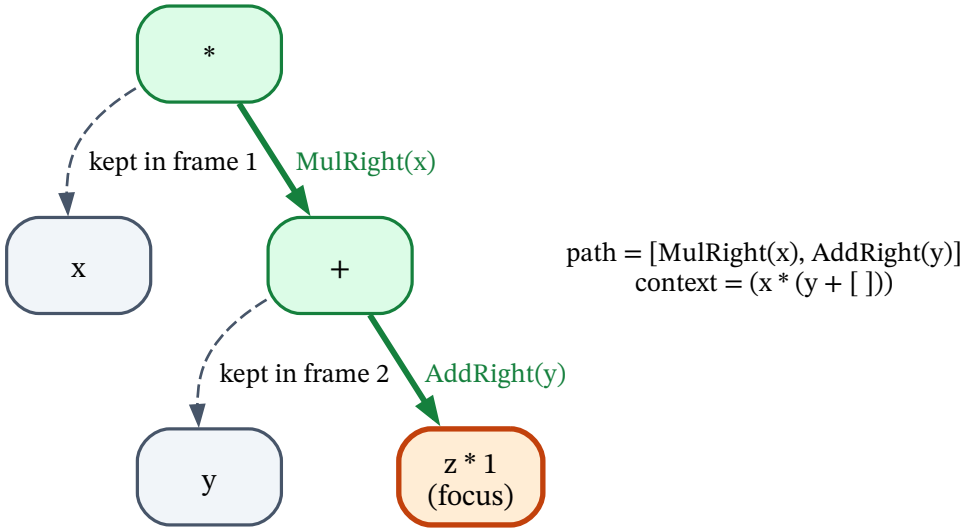
Moving down pushes a frame and moving up pops one and plugs the focus into it; each step costs constant time, however big the tree. `rebuild` plugs the focus into all frames, innermost first, which is a fold over the path. A small rewriting step uses the zipper:

```
/// Simplify the node under the cursor using x + 0 = x and x * 1 = x.
fn simplify_here(z: Zipper) -> Zipper {
    let simpler = match &z.focus {
        Expr::Add(a, b) if **b == num(0) => Some((**a).clone()),
        Expr::Mul(a, b) if **b == num(1) => Some((**a).clone()),
        _ => None,
    };
    match simpler {
        Some(e) => z.replace(e),
        None => z,
    }
}
```

```
let e = mul(add(var('x'), num(0)), add(var('y'), mul(var('z'), num(1))));
println!("start: {e}");

let z = Zipper::new(e);
show("root", &z);
let z = z.left().expect("a product has a left operand");
show("left", &z);
let z = simplify_here(z);
show("simplify", &z);
let z = z.up().expect("not at the root");
let z = z.right().and_then(Zipper::right).expect("two steps right");
show("right, right", &z);
let z = simplify_here(z);
show("simplify", &z);

let result = z.rebuild();
println!("result: {result}");
assert_eq!(result, mul(var('x'), add(var('y'), var('z'))));
```

```
start: ((x + 0) * (y + (z * 1)))
root           focus = ((x + 0) * (y + (z * 1))) context = [ ]
left           focus = (x + 0)    context = ([ ] * (y + (z * 1)))
simplify       focus = x          context = ([ ] * (y + (z * 1)))
right, right   focus = (z * 1)    context = (x * (y + [ ]))
simplify       focus = z          context = (x * (y + [ ]))
result: (x * (y + z))
```

The `context` column prints the frames as an expression with a hole `[ ]`: it is the value of the derivative, displayed. After moving right twice, the zipper looks like this:



*The zipper focused on the subexpression z * 1. The bold path from the root is the list of frames; each frame keeps the sibling that was not entered (x, then y).*

11.10 Putting it together: counting illegal states

Here is the algebra used for design. A network client tracks the state of a link: connected or not, and, when connected, the role the server granted. A first design uses two independent fields; a second uses an enum (`code/rust/examples/ch11_illegal_states.rs`):

```
#[derive(Clone, Copy, Debug, PartialEq)]
enum Role {
    Reader,
    Writer,
    Admin,
}

/// Design 1: two independent fields, 2 × (1 + 3) = 8 states. The code that
/// uses it must trust an invariant: a link has a role exactly when it is
/// connected.
#[derive(Clone, Copy, Debug, PartialEq)]
struct LinkFlags {
    connected: bool,
    role: Option<Role>,
}

/// Design 2: one enum, 1 + 3 = 4 states, and no invariant to trust.
#[derive(Clone, Copy, Debug, PartialEq)]
enum Link {
    Down,
    Up(Role),
}
```

Count. `LinkFlags` is `bool` $\times$ $(\mathbf{1} + \mathrm{Role})$, so it has $2 \cdot (1 + 3) = 8$ values. `Link` is $\mathbf{1} + \mathrm{Role}$, with $1 + 3 = 4$ values. The meaningful states are "down" and "up with a role", four of them. So the flags design has four values that mean nothing: connected without a role, and disconnected with each of the three roles. Every function that receives a `LinkFlags` must either check for them or trust that no one ever builds one. The program lists them, and checks that the legal flags and the enum are isomorphic:

```
impl LinkFlags {
    /// Every value of the type: |bool| × |Option<Role>| of them.
    fn all() -> Vec<LinkFlags> {
        let mut roles = vec![None];
        roles.extend(ROLES.map(Some));
        let mut out = Vec::new();
        for connected in [false, true] {
            for &role in &roles {
                out.push(LinkFlags { connected, role });
            }
        }
        out
    }

    fn is_legal(&self) -> bool {
        self.connected == self.role.is_some()
    }

    /// Partial: illegal combinations have no counterpart.
    fn to_link(self) -> Option<Link> {
        match (self.connected, self.role) {
            (false, None) => Some(Link::Down),
            (true, Some(role)) => Some(Link::Up(role)),
            (false, Some(_)) | (true, None) => None,
        }
    }
}

impl Link {
    /// Every value of the type: 1 + |Role| of them.
    fn all() -> Vec<Link> {
        let mut out = vec![Link::Down];
        out.extend(ROLES.map(Link::Up));
        out
    }

    /// Total: every `Link` is a legal `LinkFlags`.
    fn to_flags(self) -> LinkFlags {
        match self {
            Link::Down => LinkFlags {
                connected: false,
                role: None,
            },
            Link::Up(role) => LinkFlags {
                connected: true,
                role: Some(role),
            },
        }
    }
}
```

```
connected: false  role: None          legal    -> Down
connected: false  role: Some(Reader)  ILLEGAL
connected: false  role: Some(Writer)  ILLEGAL
connected: false  role: Some(Admin)   ILLEGAL
connected: true   role: None          ILLEGAL
connected: true   role: Some(Reader)  legal    -> Up(Reader)
connected: true   role: Some(Writer)  legal    -> Up(Writer)
connected: true   role: Some(Admin)   legal    -> Up(Admin)
LinkFlags: 8 states, 4 legal, 4 illegal
Link:      4 states
legal LinkFlags ≅ Link: both round trips checked
```

The algebra gives the general rule. With r roles, the flags have $2(1 + r) = 2 + 2r$ states, the enum has $1 + r$, and the difference, $1 + r$ illegal states, grows with every role you add. The enum makes illegal states *unrepresentable*: there is no value of `Link` that means nothing, so no code has to handle one.

In Lean we can prove this for every finite type of roles at once (`code/lean/Book/Ch11/States.lean`). First, the legal states as a type of their own:

```
/-- The invariant of the flags design: a role exactly when connected. -/
def Legal : Bool × Option R → Prop
  | (connected, role) => connected = role.isSome

instance : DecidablePred (Legal (R := R)) :=
  fun s => inferInstanceAs (Decidable (s.1 = s.2.isSome))

/-- The legal flags, as a type: a *subtype*, the values of `Bool × Option R`
that come with a proof of `Legal`. -/
abbrev LegalFlags (R : Type) : Type := {s : Bool × Option R // Legal s}
```

`{s : Bool × Option R // Legal s}` is a **subtype**: its values are pairs `⟨s, h⟩` of a state `s` and a proof `h` that `s` is legal. Rust has no counterpart, but a "parse, don't validate" newtype with a private field comes close (Chapter 30). The `DecidablePred` instance tells Lean how to *decide* legality by computing, which Mathlib needs in order to count the subtype. The isomorphism with $\mathbf{1} + R$:

```
/-- The legal flags are isomorphic to `Option R`, i.e. to `1 + R`. -/
def legalIso (R : Type) : LegalFlags R ≃ Option R where
  toFun s := s.val.2
  invFun role := ⟨(role.isSome, role), rfl⟩
  left_inv := by
    intro ⟨⟨connected, role⟩, h⟩
    -- h : connected = role.isSome, so `connected` is determined by `role`
    simp only [Legal] at h
    subst h
    rfl
  right_inv _ := rfl
```

The interesting direction is `left_inv`. A legal state is `⟨(connected, role), h⟩` where `h : connected = role.isSome`. The tactic `subst h` replaces `connected` everywhere by `role.isSome`, which says precisely that the flag carried no information of its own, and then both sides are the same term. Finally, the counts:

```
/-- The flags design has `2 * (|R| + 1)` states... -/
theorem card_flags [Fintype R] :
    Fintype.card (Bool × Option R) = 2 * (Fintype.card R + 1) := by
  simp

/-- ...of which `|R| + 1` are legal... -/
theorem card_legal [Fintype R] :
    Fintype.card (LegalFlags R) = Fintype.card R + 1 := by
  rw [card_eq_of_iso (legalIso R), Fintype.card_option]

/-- ...so exactly `|R| + 1` of them are illegal. -/
theorem card_illegal [Fintype R] :
    Fintype.card (Bool × Option R) - Fintype.card (LegalFlags R) =
      Fintype.card R + 1 := by
  rw [card_flags, card_legal]
  omega
```

`card_legal` rewrites the cardinality of the subtype with our `card_eq_of_iso` and the isomorphism just built, then uses `Fintype.card_option`; `card_illegal` lets `omega` do the subtraction. The lesson is worth keeping: when a type has more values than your domain has states, the difference is a list of bugs waiting for a constructor call, and the algebra tells you how long the list is.

11.11 Common confusions

"**Isomorphic types are the same type.**" They have the same information, not the same name. Rust will not pass an `Option<bool>` where an `Either<(), bool>` is expected, and that is usually what you want. Lean is equally strict: `Iso` and `Equiv` are data that you apply explicitly, not equations that the type checker uses silently.

"**`()` and `!` (or `Infallible`) are both 'nothing'.**" They are opposites. `()` has exactly one value, so returning it says "done, nothing to report". An empty type has no values, so returning it says "this never returns". A function `fn() -> ()` can be called; a value of type `Infallible` can never exist, and code that receives one is unreachable.

"**An empty type is a zero-sized type.**" Both take zero bytes, but `()` has a value and `Void` does not. `Vec<()>` can hold a million elements; `Vec<Void>` can only ever be empty. The algebra distinguishes $\mathbf{1}$ from $\mathbf{0}$ even where `size_of` does not.

"**`Box` changes the type theory of a recursive type.**" It changes the representation. The type $\mu X.\, \mathbf{1} + A \times X$ is the same whether the recursive field is stored inline (impossible), behind a `Box`, an `Rc` or a reference; the choice of pointer is about ownership and memory, which later parts of the book study.

"**The counting laws prove the isomorphisms.**" For finite types, equal counts give *some* isomorphism, but an arbitrary one. The laws of this chapter give isomorphisms defined by a single program for all types, including infinite ones, where counting says nothing useful.

11.12 Summary

- The basic type formers are products ($A \times B$), sums ($A + B$), the unit type ($\mathbf{1}$), the empty type ($\mathbf{0}$) and functions (B^A). Rust's structs and tuples are products, its enums are sums of products, `()` is $\mathbf{1}$, and `Infallible` or `enum Void {}` is $\mathbf{0}$.
- Each former has introduction and elimination rules. The elimination rule for sums demands a branch for every case: exhaustive `match` in Rust, the missing-cases check in Lean. The empty `match` is the elimination rule for $\mathbf{0}$.
- Counting values gives $|A + B| = |A| + |B|$, $|A \times B| = |A|\,|B|$ and $|A \to B| = |B|^{|A|}$. Each law of arithmetic becomes an isomorphism of types, a pair of mutually inverse functions.
- In Lean, an isomorphism is a structure with two functions and two round-trip proofs. We proved the laws, chained them with congruence and transitivity, and connected our `Iso` to Mathlib's `Equiv`; `Fintype.card` counts values, and isomorphic finite types have equal cardinality.
- Recursive types solve type equations: $\mathrm{List}\ A \cong \mathbf{1} + A \times \mathrm{List}\ A$. Rust needs a `Box` to give them a finite size. Their elimination rule is the fold (in Lean, the generated recursor), which is uniquely determined by its equations.
- Lean requires inductive types to be strictly positive; a negative type would allow a diagonal argument that proves `False`. Rust allows negative types, and with them recursion without recursive functions.

- Rust's layouts follow the algebra: zero-sized types, empty summands that cost nothing, and niches that store an `Option`'s `None` in an unused bit pattern.
- Counting the values of a design reveals its illegal states; choosing a type with exactly as many values as the domain has states makes them unrepresentable.
- The derivative of a type is its type of one-hole contexts, and a zipper is a derivative plus a focus.

11.13 Exercises

1. Compute the number of values of `Result<Option<bool>, bool>`, of `fn(Option<bool>) -> Option<bool>` and of `(bool, ()) -> Either<(), Void>`. Check the first two in Lean with `Fintype.card` and `simp`. *Hint:* for the third, simplify first: $(2 \times 1) \to (1 + 0)$ is 1^2.
2. Write, in Rust, the isomorphism $(A \times B)^C \cong A^C \times B^C$ (a function returning a pair is a pair of functions) as two generic functions, and check the round trips pointwise. Which bounds do you need, and why? *Hint:* splitting one function into two needs the function twice, so it must be `Clone` (or shared through an `Rc`); joining two functions into one needs no bound.
3. In Lean, prove `prodSumDistrib`'s cousin `(α ⊕ β) × γ ≃ (α × γ) ⊕ (β × γ)` using our `Iso`, without looking at `distrib`. Then build it a second way, by chaining `prodComm`, `distrib` and a congruence lemma for sums that you write yourself (`sumCongr`). *Hint:* `sumCongr` has the same shape as `prodCongr`, with the functions defined by cases.
4. Define `length` and `map` for the Rust `List<T>` without `fold`, by direct recursion, and then prove in Lean that `MyList.length` satisfies `(xs.map f).length = xs.length` using `fold_unique` instead of induction. *Hint:* take `h := fun xs => (xs.map f).length`, `n := 0` and `c := fun _ k => k + 1`; both equations hold by `rfl`.
5. The type `enum E { A(Box<E>), B(Box<E>) }` compiles. How many values does it have? What about `enum F { Leaf, Node(Box<F>, Box<F>) }`? Write the type equation of each and say which fixed point is empty. Prove the emptiness of the first in Lean. *Hint:* follow `Loop.elim`; the induction has two cases, each with its own induction hypothesis.
6. Write a list zipper in Rust with the same operations as the Lean `Zipper` (`from_vec`, `left`, `right`, `modify`, `to_vec`), and test that `right` followed by `left` gives back the same zipper. Then derive the type of one-hole contexts of a binary tree with values in the nodes, $T = \mathbf{1} + T \times X \times T$, with respect to X. *Hint:* a hole for a value sits in one node, and that node sits somewhere in the tree; the answer involves a list of "which way did we go, and what did we pass".

PART III

Abstraction

Part III is about abstraction: writing one piece of code that works at many types. It covers parametric polymorphism and System F, the type inference that lets Rust and Lean fill in most types for you, traits and type classes and the dictionaries the compiler passes behind your back, and existential types, the theory behind `impl Trait` and `dyn Trait`. Each of these is a feature you use in Rust every day; each turns out to be a precise logical idea.

CHAPTER 12

Parametric Polymorphism

In the simply typed λ-calculus of Chapter 8, every function has one type. The identity function on booleans, $\lambda(x : \mathit{Bool}).\, x$, and the identity function on naturals, $\lambda(x : \mathit{Nat}).\, x$, are two different terms, and if you need the identity on pairs of lists of booleans you must write a third. The bodies are identical; only the annotations differ. Every programmer recognizes the situation, and every modern language has an answer to it. Rust's answer is generics: `fn id<T>(x: T) -> T` is written once and works for every type.

The type theory behind this answer is **System F**, the polymorphic λ-calculus, discovered independently by the logician Jean-Yves Girard (1972) and the computer scientist John Reynolds (1974). It adds two things to the simply typed calculus: functions that take a *type* as an argument, and types of the form $\forall \alpha.\, \tau$ that describe them. The idea is simple, and it is surprisingly powerful: with nothing but functions and ∀, System F can express booleans, pairs, sums and natural numbers, and its quantifier can range over types that contain the quantifier itself.

That last feature, *impredicativity*, is where the three languages of this book part ways. System F has it everywhere. Rust has a deliberately restricted version of System F, with quantifiers only on the outside of item signatures, and recovers a little more through traits and lifetimes. Lean has dependent types, which contain System F's ∀ as a special case, but it organizes its types in a hierarchy of *universes* that keeps impredicativity for propositions only. This chapter explains all three, and in particular takes the time to explain Lean's universes from scratch, since they are one of the most confusing parts of Lean for newcomers.

In this chapter you will learn:

- the syntax, typing rules and computation rules of System F, and how to build a derivation with type abstraction and type application;
- how Church encodings express booleans, pairs, sums and numbers in System F, and what impredicativity means;
- how Rust generics correspond to ∀, how the turbofish is type application, and how monomorphization compiles them (with its costs and benefits, compared with erasure and dictionary passing);
- why Rust's polymorphism is rank-1, and how traits with generic methods and higher-ranked trait bounds (`for<'a>`) give you rank-2 types;

- how Lean writes polymorphism with implicit arguments and `@`, and how its universes (`Type u`, `Sort u`, `Prop`) work, and why they exist;
- why System F's impredicativity survives in Lean only in `Prop`, and how universe polymorphism works around the restriction;
- a type checker for System F, written and proved sound and complete in Lean.

12.1 One function, every type

Here is the polymorphic identity function in Rust, from the chapter's example `code/rust/examples/ch12_generics.rs`:

```
/// Λα. λ(x : α). x — the polymorphic identity. Its type is ∀α. α → α.
fn id<T>(x: T) -> T {
    x
}
```

Its type, written in the notation of this chapter, is $\forall\alpha.\,\alpha \to \alpha$: "for every type α, a function from α to α". The type parameter `T` plays the role of α. When you call `id(5)`, the compiler works out that `T` must be `i32` and uses the function at type `i32 -> i32`. When you call `id("five")`, it uses it at `&str -> &str`. One definition, every type.

It helps to make the hidden step explicit. Using a polymorphic function happens in two stages: first you choose the type (`T := i32`), which gives you an ordinary monomorphic function, and then you apply that function to a value. System F makes both stages visible in the syntax, and gives each its own typing rule.

12.2 System F

Syntax

The types of System F are type variables, function types, universally quantified types, and whatever base types we want to add (we will use *Bool*):

$$\tau ::= \alpha \mid \mathit{Bool} \mid \tau \to \tau \mid \forall\alpha.\,\tau$$

The terms are those of the simply typed λ-calculus, plus two new forms:

$$e ::= x \mid \mathit{true} \mid \mathit{false} \mid \lambda(x : \tau).\,e \mid e\ e \mid \Lambda\alpha.\,e \mid e\,[\tau]$$

Type abstraction $\Lambda\alpha.\,e$ (a capital lambda) is a function whose parameter is a type. **Type application** $e\,[\tau]$ passes a type to such a function. The polymorphic identity is $\mathit{id} = \Lambda\alpha.\,\lambda(x : \alpha).\,x$, and $\mathit{id}\,[\mathit{Bool}]\ \mathit{true}$ first instantiates it at *Bool* and then applies it to *true*.

$\forall\alpha.\,\tau$ binds the type variable α in τ, as λ binds a term variable. Everything Chapter 7 said about bound variables carries over: $\forall\alpha.\,\alpha \to \alpha$ and $\forall\beta.\,\beta \to \beta$ are the same type (α-equivalence), and substituting a type for a type variable, written $\tau[\sigma/\alpha]$, must avoid capturing free variables.

Typing rules

Contexts now record two kinds of assumptions: term variables with their types, as before, and type variables that are in scope: $\Gamma ::= \varnothing \mid \Gamma, x : \tau \mid \Gamma, \alpha$. The rules for variables, abstraction and application are those of Chapter 8:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsf{T-Var} \qquad \frac{\Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash \lambda(x : \sigma).\, e : \sigma \to \tau}\ \textsf{T-Abs} \qquad \frac{\Gamma \vdash f : \sigma \to \tau \qquad \Gamma \vdash a : \sigma}{\Gamma \vdash f\ a : \tau}\ \textsf{T-App}$$

and two new rules govern the new forms:

$$\frac{\Gamma, \alpha \vdash e : \tau}{\Gamma \vdash \Lambda\alpha.\, e : \forall\alpha.\, \tau}\ \textsf{T-TAbs} \qquad \frac{\Gamma \vdash e : \forall\alpha.\, \tau}{\Gamma \vdash e\,[\sigma] : \tau[\sigma/\alpha]}\ \textsf{T-TApp}$$

Rule T-TAbs says: if e has type τ for an *arbitrary* type α, one about which nothing is assumed, then $\Lambda\alpha.\, e$ has type $\forall\alpha.\, \tau$. The variable α must be new: it may not already appear in Γ. Without that side condition we could type $\lambda(x : \alpha).\, \Lambda\alpha.\, x$ as $\alpha \to \forall\alpha.\, \alpha$, a function that turns a value of one particular type into a value of every type. Rule T-TApp is the elimination rule: a value of type $\forall\alpha.\, \tau$ can be used at any type σ, and its type becomes τ with σ in place of α. In logic, T-TAbs and T-TApp are the introduction and elimination rules for the universal quantifier, $\forall$I and $\forall$E, applied to propositions instead of individuals: this is *second-order propositional* logic, and System F is its Curry–Howard twin, as Chapter 10 anticipated.

The computation rules are two β-rules, one for each kind of abstraction:

$$(\lambda(x : \sigma).\, e)\ a \to e[a/x] \qquad\qquad (\Lambda\alpha.\, e)\,[\sigma] \to e[\sigma/\alpha]$$

Notice that the second rule does nothing to the *shape* of the computation: it only changes type annotations. If you erase all types from a System F term, type abstractions and applications disappear and what remains is an untyped λ-term that computes the same result. Types in System F are a compile-time affair, and this observation is the starting point of every implementation strategy we will see.

A derivation

Here is the derivation that *id* has type $\forall\alpha.\, \alpha \to \alpha$. The context Γ_1 is $\alpha, x : \alpha$:

$$\dfrac{\dfrac{\dfrac{(x : \alpha) \in \Gamma_1}{\alpha, x : \alpha \vdash x : \alpha}\ \textsf{T-Var}}{\alpha \vdash \lambda(x : \alpha).\, x : \alpha \to \alpha}\ \textsf{T-Abs}}{\vdash \Lambda\alpha.\, \lambda(x : \alpha).\, x : \forall\alpha.\, \alpha \to \alpha}\ \textsf{T-TAbs}$$

and here is its use at *Bool*, where the left premise is the derivation above followed by T-TApp, which computes $(\alpha \to \alpha)[\mathit{Bool}/\alpha] = \mathit{Bool} \to \mathit{Bool}$ (T-True is the axiom that types the constant *true*):

$$\dfrac{\dfrac{\vdash \mathit{id} : \forall\alpha.\, \alpha \to \alpha}{\vdash \mathit{id}\,[\mathit{Bool}] : \mathit{Bool} \to \mathit{Bool}}\ \textsf{T-TApp} \qquad \dfrac{}{\vdash \mathit{true} : \mathit{Bool}}\ \textsf{T-True}}{\vdash \mathit{id}\,[\mathit{Bool}]\ \mathit{true} : \mathit{Bool}}\ \textsf{T-App}$$

Church encodings

Chapter 7 encoded booleans, pairs and numbers as untyped λ-terms. System F can give those encodings *types*, and the types say exactly what each encoding is.

A boolean is a choice between two alternatives. The Church boolean takes the two alternatives, of any type α, and returns one of them:

$$\mathit{CBool} = \forall\alpha.\,\alpha \to \alpha \to \alpha \qquad \mathit{true} = \Lambda\alpha.\,\lambda(t:\alpha).\,\lambda(f:\alpha).\,t \qquad \mathit{false} = \Lambda\alpha.\,\lambda(t:\alpha).\,\lambda(f:\alpha).\,f$$

A natural number n is "apply a function n times":

$$\mathit{CNat} = \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha \qquad \mathit{two} = \Lambda\alpha.\,\lambda(f:\alpha\to\alpha).\,\lambda(x:\alpha).\,f\,(f\,x)$$

and the types of Chapter 11 have encodings too:

$$A \times B = \forall\gamma.\,(A \to B \to \gamma) \to \gamma \qquad A + B = \forall\gamma.\,(A \to \gamma) \to (B \to \gamma) \to \gamma \qquad \mathbf{0} = \forall\alpha.\,\alpha$$

Look at these types with the eyes of Chapter 11. The encoding of a sum is its case rule: give me what to do with an A and what to do with a B, for any result type γ, and I will do it. The encoding of a pair is its elimination rule too: give me a consumer of an A and a B, and I will feed it. The encoding of a natural number is the fold of Chapter 11, for the type $\mathbb{N} = \mu X.\,\mathbf{1} + X$: a value for zero ($x:\alpha$) and a step ($f:\alpha\to\alpha$). A Church-encoded value *is* its own elimination rule. And $\mathbf{0} = \forall\alpha.\,\alpha$ is a type that would produce a value of every type; under Curry–Howard it says "everything is true", and System F has no closed term of this type.

Impredicativity

The quantifier $\forall\alpha$ ranges over *all* types, and that includes the types of the form $\forall\beta.\ldots$, including the very type being defined. A CNat can be instantiated at CNat. This is what makes the following definition of exponentiation possible:

$$\mathit{pow} = \lambda(m:\mathit{CNat}).\,\lambda(n:\mathit{CNat}).\,n\,[\mathit{CNat}]\,(\mathit{mul}\,m)\,\mathit{one}$$

The numeral n iterates "multiply by m" over numerals, starting from one. A quantifier whose range includes the type it is part of is called **impredicative**. Its most striking use is self-application:

$$\lambda(x:\forall\alpha.\,\alpha\to\alpha).\,x\,[\forall\alpha.\,\alpha\to\alpha]\,x : (\forall\alpha.\,\alpha\to\alpha) \to (\forall\alpha.\,\alpha\to\alpha)$$

The argument x is a polymorphic identity; we instantiate it at its own type and apply it to itself. Chapter 8 showed that $\lambda x.\,x\,x$ has no type in the simply typed calculus; in System F a version of it does, because the two occurrences of x are used at two different types.

System F is remarkably strong. Girard proved that every well-typed term terminates (strong normalization), so the logic is consistent, and yet the functions on numerals it can define go far beyond addition and exponentiation. The price is paid by type inference: J. B. Wells showed in the 1990s that deciding whether an unannotated term has a System F type is undecidable. That is why practical languages restrict System F, as Chapter 13 explains for ML and Haskell, and as we will now see for Rust.

12.3 A type checker for System F in Lean

Before turning to Rust, let us make the rules concrete. With all annotations present, *checking* a System F term is easy, and the file `code/lean/Book/Ch12/SystemF.lean` implements a checker and proves it correct. Type variables are represented by de Bruijn indices, as term variables were in Chapter 7:

`var 0` refers to the nearest enclosing ∀, `var 1` to the next one out, and so on. This makes α-equivalent types literally equal, so the checker can compare types with `=`.

```
/-- Types: `Bool`, type variables, functions and `∀`. A type variable is a
number: `var 0` refers to the nearest enclosing `∀`, `var 1` to the one
outside it, and so on (de Bruijn indices, as in Chapter 7). -/
inductive Ty where
  | bool : Ty
  | var : ℕ → Ty
  | arr : Ty → Ty → Ty
  | all : Ty → Ty
deriving DecidableEq, Repr
```

Two operations on types are needed for rule T-TApp. Substituting under a binder requires *shifting* the indices of the substituted type, exactly as in Chapter 7:

```
/-- `τ.lift c` adds one to every type variable of `τ` that points outside the
innermost `c` binders. Needed when a type is moved under one more `∀`. -/
def lift : Ty → ℕ → Ty
  | bool, _ => bool
  | var k, c => if k < c then var k else var (k + 1)
  | arr a b, c => arr (a.lift c) (b.lift c)
  | all t, c => all (t.lift (c + 1))

/-- `τ.subst k σ` replaces variable `k` of `τ` by `σ`; the variables above `k`
move down by one, because the `∀` that bound `k` is gone. -/
def subst : Ty → ℕ → Ty → Ty
  | bool, _, _ => bool
  | var n, k, σ => if n < k then var n else if n = k then σ else var (n - 1)
  | arr a b, k, σ => arr (a.subst k σ) (b.subst k σ)
  | all t, k, σ => all (t.subst (k + 1) (σ.lift 0))
```

`τ.subst 0 σ` is $\tau[\sigma/\alpha]$ for the variable bound by an outer ∀ that has just been removed: variable `0` becomes `σ`, and the other variables move down by one, since one binder is gone. When `subst` goes under a `∀`, the index to replace grows by one and `σ` is lifted, so that its own free variables keep pointing to the same binders. The terms mirror the grammar:

```
/-- Terms. Term variables are de Bruijn indices into the context too. -/
inductive Term where
  | var : ℕ → Term              -- x
  | tt : Term                   -- true
  | ff : Term                   -- false
  | lam : Ty → Term → Term      -- λ (x : τ). e
  | app : Term → Term → Term    -- e₁ e₂
  | tlam : Term → Term          -- Λα. e
  | tapp : Term → Ty → Term     -- e [τ]
deriving Repr
```

The typing rules become an inductive relation, one constructor per rule, exactly as in Chapter 3:

```
/-- The typing rules of System F. The context lists the types of the term
variables in scope, nearest first. Going under a `Λ` adds a type variable, so
every type already in the context is lifted past it. -/
inductive HasType : List Ty → Term → Ty → Prop
  | var {Γ i τ} : Γ[i]? = some τ → HasType Γ (.var i) τ
  | tt {Γ} : HasType Γ .tt .bool
  | ff {Γ} : HasType Γ .ff .bool
  | lam {Γ σ e τ} : HasType (σ :: Γ) e τ → HasType Γ (.lam σ e) (.arr σ τ)
  | app {Γ f a σ τ} :
      HasType Γ f (.arr σ τ) → HasType Γ a σ → HasType Γ (.app f a) τ
  | tlam {Γ e τ} : HasType (Γ.map (·.lift 0)) e τ → HasType Γ (.tlam e) (.all τ)
  | tapp {Γ e τ σ} : HasType Γ e (.all τ) → HasType Γ (.tapp e σ) (τ.subst 0 σ)
```

Read the constructors against the rules above. `lam` is T-Abs, with the new variable's type pushed on the context. `tlam` is T-TAbs: the context does not record type variables explicitly, because with de Bruijn indices "a new type variable in scope" means "every existing index now points one binder further", so all the types in the context are lifted. The side condition of T-TAbs is automatic in this representation. `tapp` is T-TApp with `subst`. The checker is the same rules read from the conclusion upwards:

```
/-- The type checker: the rules read from the conclusion upwards. -/
def typeOf : List Ty → Term → Option Ty
  | Γ, .var i => Γ[i]?
  | _, .tt => some .bool
  | _, .ff => some .bool
  | Γ, .lam σ e => (typeOf (σ :: Γ) e).map (.arr σ)
  | Γ, .app f a =>
    match typeOf Γ f, typeOf Γ a with
    | some (.arr σ τ), some σ' => if σ = σ' then some τ else none
    | _, _ => none
  | Γ, .tlam e => (typeOf (Γ.map (·.lift 0)) e).map .all
  | Γ, .tapp e σ =>
    match typeOf Γ e with
    | some (.all τ) => some (τ.subst 0 σ)
    | _ => none

/-- Check a closed term and print its type. -/
def check (e : Term) : String :=
  match typeOf [] e with
  | some τ => τ.toStr 0
  | none => "type error"
```

To show types with names instead of numbers, a small printer names the binders α, β, γ by depth. Under `d` binders, the variable with index `k` refers to the binder at depth `d - 1 - k`, so that is the name it gets:

```
/-- A name for the type variable bound at depth `d`. -/
def name (d : ℕ) : String :=
  (["α", "β", "γ", "δ"][d]?).getD s!"α{d}"

/-- Print a type under `d` binders, with names instead of numbers. -/
def toStr (d : ℕ) : Ty → String
  | bool => "Bool"
  | var k => if k < d then name (d - 1 - k) else s!"#{k}"
  | arr a b =>
    let left := match a with
      | arr .. | all .. => s!"({a.toStr d})"
      | _ => a.toStr d
    s!"{left} → {b.toStr d}"
  | all t => s!"∀{name d}. {t.toStr (d + 1)}"
```

Here are some terms and their types:

```
/-- `Λα. λ (x : α). x` -/
def polyId : Term := .tlam (.lam (.var 0) (.var 0))

/-- The type of Church numerals, `∀α. (α → α) → α → α`. -/
def CNat : Ty := .all (.arr (.arr (.var 0) (.var 0)) (.arr (.var 0) (.var 0)))

/-- `Λα. λ (f : α → α). λ (x : α). f (f x)` -/
def two : Term :=
  .tlam (.lam (.arr (.var 0) (.var 0))
    (.lam (.var 0) (.app (.var 1) (.app (.var 1) (.var 0)))))

/-- `λ (n : CNat). Λα. λ (f : α → α). λ (x : α). f (n [α] f x)` -/
def succ : Term :=
  .lam CNat (.tlam (.lam (.arr (.var 0) (.var 0)) (.lam (.var 0)
```

```
    (.app (.var 1) (.app (.app (.tapp (.var 2) (.var 0)) (.var 1)) (.var 0))))))

/-- Self-application, typable thanks to impredicativity:
`λ (x : ∀α. α → α). x [∀α. α → α] x` -/
def selfApp : Term :=
  .lam (.all (.arr (.var 0) (.var 0)))
    (.app (.tapp (.var 0) (.all (.arr (.var 0) (.var 0)))) (.var 0))

/-- A rank-2 function: `λ (f : ∀α. α → α). f [Bool] true` -/
def useTwice : Term :=
  .lam (.all (.arr (.var 0) (.var 0))) (.app (.tapp (.var 0) .bool) .tt)
```

```
/-- info: "∀α. α → α" -/
#guard_msgs in
#eval check polyId

/-- info: "Bool" -/
#guard_msgs in
#eval check (.app (.tapp polyId .bool) .tt)

/-- info: "∀α. (α → α) → α → α" -/
#guard_msgs in
#eval check two

/-- info: "(∀α. (α → α) → α → α) → ∀α. (α → α) → α → α" -/
#guard_msgs in
#eval check succ

/-- info: "∀α. (α → α) → α → α" -/
#guard_msgs in
#eval check (.app succ two)

/-- info: "(∀α. α → α) → ∀α. α → α" -/
#guard_msgs in
#eval check selfApp

/-- info: "(∀α. α → α) → Bool" -/
#guard_msgs in
#eval check useTwice

/-- info: "type error" -/
#guard_msgs in
#eval check (.lam .bool (.app (.var 0) (.var 0)))  -- λ (x : Bool). x x
```

The checker confirms the polymorphic identity, its use at `Bool`, the Church numeral two, the successor function on Church numerals and its application to two, the impredicative self-application, and a rank-2 function (more on rank below). It rejects $\lambda(x : \mathit{Bool}).\, x\, x$, since a boolean is not a function.

A checker is only as good as its agreement with the rules, and in Lean that agreement is a theorem:

```
/-- Soundness: if the checker returns a type, there is a derivation of it. -/
theorem typeOf_sound {Γ : List Ty} {e : Term} {τ : Ty} (h : typeOf Γ e = some τ) :
    HasType Γ e τ := by
  induction e generalizing Γ τ with
  | var i => exact .var h
  | tt =>
    -- h : some bool = some τ
    cases h
    exact .tt
  | ff =>
    cases h
    exact .ff
  | lam σ e ih =>
    -- h : (typeOf (σ :: Γ) e).map (arr σ) = some τ
    simp only [typeOf, Option.map_eq_some_iff] at h
    obtain ⟨τ', h', rfl⟩ := h
```

```
    exact .lam (ih h')
  | app f a ihf iha =>
    simp only [typeOf] at h
    -- Look at what the checker found for `f` and `a`.
    split at h
    · rename_i σ τ' σ' hf ha
      -- h : (if σ = σ' then some τ' else none) = some τ
      split at h
      · rename_i hσ
        cases h
        subst hσ
        exact .app (ihf hf) (iha ha)
      · cases h
    · cases h
  | tlam e ih =>
    simp only [typeOf, Option.map_eq_some_iff] at h
    obtain ⟨τ', h', rfl⟩ := h
    exact .tlam (ih h')
  | tapp e σ ih =>
    simp only [typeOf] at h
    split at h
    · rename_i τ' he
      cases h
      exact .tapp (ih he)
    · cases h
```

The proof of `typeOf_sound` is by induction on the term, with `Γ` and `τ` generalized because the recursive calls of the checker use different contexts and types. Each case looks at what the checker did. For a variable, the checker's answer `Γ[i]? = some τ` is literally the premise of the `var` rule. For `tt` and `ff`, `cases h` turns `h : some bool = some τ` into the fact `τ = bool`. For a λ-abstraction, `simp only [typeOf, Option.map_eq_some_iff] at h` turns "mapping `arr σ` over the result gave `some τ`" into "the body had some type `τ'`, and `τ = arr σ τ'`"; `obtain` takes this apart (the pattern `rfl` substitutes the equation away), and the induction hypothesis turns the body's result into a derivation. For an application, the checker made two choices (the `match` on the results for `f` and `a`, then the `if`), and `split at h` follows each of them: in the branch where everything succeeded, the hypotheses `hf` and `ha` record what the checker found, `hσ` records that the argument type matched, and the derivation is built from the two induction hypotheses. In the failing branches, `h` says `none = some τ`, and `cases h` closes the goal because those two constructors can never be equal. Type abstraction and application follow the same pattern.

Completeness, the converse, is an induction on the *derivation* instead, and `simp` does the work in each case, using the induction hypotheses to compute the checker's result. Together the two theorems say that the checker decides the typing relation, and they give uniqueness of types for free:

```
/-- Completeness: every derivable type is the one the checker reports. -/
theorem typeOf_complete {Γ : List Ty} {e : Term} {τ : Ty} (h : HasType Γ e τ) :
    typeOf Γ e = some τ := by
  induction h with
  | var hi => exact hi
  | tt => rfl
  | ff => rfl
  | lam _ ih => simp [typeOf, ih]
  | app _ _ ihf iha => simp [typeOf, ihf, iha]
  | tlam _ ih => simp [typeOf, ih]
  | tapp _ ih => simp [typeOf, ih]

/-- A consequence: a term has at most one type. -/
theorem type_unique {Γ : List Ty} {e : Term} {τ₁ τ₂ : Ty}
```

```
    (h₁ : HasType Γ e τ₁) (h₂ : HasType Γ e τ₂) : τ₁ = τ₂ := by
  have := (typeOf_complete h₁).symm.trans (typeOf_complete h₂)
  exact Option.some.inj this
```

(The checker does not verify that the types written in annotations are well-scoped, for example that `var 5` does not appear where only one type variable is bound. Adding that check is Exercise 6.)

12.4 Rust generics are ∀

A Rust generic function is a type abstraction. `fn id<T>(x: T) -> T` is $\Lambda T.\, \lambda(x : T).\, x$, and its type is $\forall T.\, T \to T$. Calling it with an explicit type is type application, and Rust's syntax for it is the **turbofish** `::<>`:

```
// Type application, explicit (the turbofish) and inferred.
let a = id::<u8>(7); // id [u8] 7
let b = id(String::from("seven")); // id [String] "seven", inferred
let n = "42".parse::<i64>().expect("a number"); // parse [i64]
let squares = (1..=4).map(|x| x * x).collect::<Vec<u32>>();
println!("{a} {b} {n} {squares:?}");

let shout = compose(|s: String| s + "!", |n: u32| n.to_string());
println!("compose: {}", shout(42));
```

```
7 seven 42 [1, 4, 9, 16]
compose: 42!
```

Most of the time you do not write the turbofish, because the compiler infers the type argument from the value arguments or from the expected result (Chapter 13 is about how). You need it when nothing else determines the type: `"42".parse()` could produce any type that implements `FromStr`, and `collect()` could build any collection, so `parse::<i64>()` and `collect::<Vec<u32>>()` supply the missing type application. Functions with several type parameters take several type arguments, in order:

```
/// ∀α β γ. (β → γ) → (α → β) → α → γ
fn compose<A, B, C>(g: impl Fn(B) -> C, f: impl Fn(A) -> B) -> impl Fn(A) -> C {
    move |x| g(f(x))
}
```

Here `compose` has type $\forall A\, B\, C.\, (B \to C) \to (A \to B) \to A \to C$.

Bounded quantification

Rust generics usually carry *bounds*:

```
/// ∀T. (T: PartialOrd + Copy) ⇒ &[T] → T. The first line reports which
/// compiled copy of the function is running.
fn largest<T: PartialOrd + Copy>(items: &[T]) -> T {
    println!("  running {}", type_name_of_val(&largest::<T>));
    let mut best = items[0];
    for &item in items {
        if item > best {
            best = item;
        }
    }
    best
}
```

Read the signature as $\forall T.\,(T : \mathit{PartialOrd} \wedge T : \mathit{Copy}) \Rightarrow \&\,[T] \to T$: for every type T that supports comparison and copying. A bound restricts the range of the quantifier and, in exchange, gives the body something to work with: without `PartialOrd`, the body could not compare two `T`s; without `Copy`, it could not copy one out of the slice. A function with *no* bounds can do almost nothing with its `T`: it can move values around, store them and return them, but not inspect them. That observation, made precise, is *parametricity*, and it is the subject of Chapter 18. Bounds and the trait system behind them are the subject of Chapter 14; in the rest of this chapter, think of a bound as a constraint attached to $\forall$.

Where Rust differs from System F

Rust's generics are System F with four important differences.

1. **Quantifiers only at the outside.** A type parameter is introduced by an *item*: a function, a struct, an enum, a trait or an `impl`. There is no type `∀T. T -> T` that you can write in a field, a parameter or a variable. In particular, a closure cannot be generic, and a function cannot take a polymorphic function as an argument. This is the *rank-1* restriction, and the next sections are about it.
2. **Bounds.** Quantifiers come with trait bounds, which System F does not have.
3. **An implicit bound.** Every type parameter has the bound `Sized` unless you write `T: ?Sized`: generic code must know the size of its values, which is a first hint of how Rust compiles generics.
4. **Lifetimes.** Rust also quantifies over lifetimes, and for lifetimes it does allow quantifiers in nested positions, as we will see.

12.5 Monomorphization

How does a compiler turn `largest` into machine code? The comparison `item > best` is an integer comparison for `i32`, a floating-point comparison for `f64`, and a call to a user-defined function for a struct. The copying of `best` moves 4 bytes for an `i32` and 8 for an `f64`. There is no single sequence of machine instructions that works for every `T`.

Rust's answer is **monomorphization**: for every type at which a generic function is used, the compiler generates a separate copy of the function, with the type parameter replaced by that type. The polymorphic function exists in the source code and in the type checker; the binary contains only monomorphic copies. We can watch the copies run, because inside each copy the expression `type_name_of_val(&largest::<T>)` names the copy:

```
let big_int = largest(&[3, 9, 4]);
let big_char = largest(&['q', 'z', 'a']);
let big_float = largest(&[2.5, 1.5]);
println!("results: {big_int} {big_char} {big_float}");
assert_eq!(big_int, largest_i32(&[3, 9, 4]));
assert_eq!(big_char, largest_char(&['q', 'z', 'a']));
```

```
  running ch12_generics::largest<i32>
  running ch12_generics::largest<char>
  running ch12_generics::largest<f64>
results: 9 z 2.5
```

Three calls, three different functions. Each copy is what you would get if you wrote the function by hand for one type:

```
/// What monomorphization produces for `largest::<i32>`, written by hand:
/// `T` replaced by `i32` everywhere, and `>` compiled to an integer compare.
fn largest_i32(items: &[i32]) -> i32 {
    let mut best = items[0];
    for &item in items {
        if item > best {
            best = item;
        }
    }
    best
}

/// And for `largest::<char>`: the same text, different machine code.
fn largest_char(items: &[char]) -> char {
    let mut best = items[0];
    for &item in items {
        if item > best {
            best = item;
        }
    }
    best
}
```

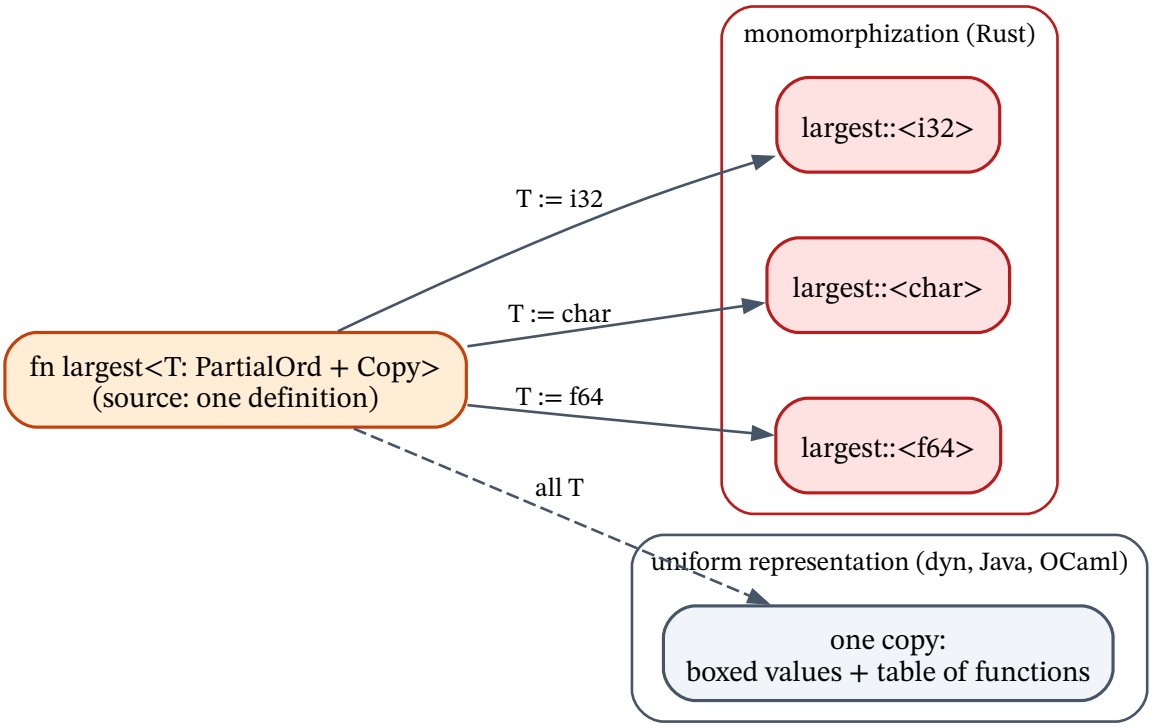
If you list the symbols of the compiled example (with `nm`, in a debug build), you find three local symbols for `largest`, with the type argument encoded in the mangled name by a single letter (`c` for `char`, `d` for `f64`, `l` for `i32` in the "v0" mangling scheme that the pinned toolchain uses), next to the two hand-written copies.

Because each copy is specialized, it knows everything about its type at compile time, including its size:

```
/// Each copy knows the size of its `T` as a compile-time constant. A
/// language that erases types (one copy for every `T`) could not offer this.
fn layout<T>() -> String {
    format!("{:<22} {:>2} bytes", type_name::<T>(), size_of::<T>())
}
```

```
u8                      1 bytes
(u8, u32)               8 bytes
alloc::string::String  24 bytes
&str                   16 bytes
```

In the theory, $(\Lambda\alpha.\, e)\,[\sigma] \to e[\sigma/\alpha]$ is a computation step. Monomorphization performs all those steps at compile time.



One generic definition compiled two ways: monomorphization makes one copy per type argument; a uniform representation uses one copy for all types.

The alternatives

Monomorphization is not the only way to compile polymorphism, and not the most common one. The alternatives keep a *single* copy of the code and make it work for all types by making all values look alike:

Strategy	Used by	Costs	Benefits
monomorphization	Rust generics, C++ templates	one copy per type: binary size, compile time	static calls, inlining, exact layouts
uniform representation	Java (erasure), OCaml, Haskell	values boxed behind pointers, indirection	one copy, fast compilation
dictionary passing	Haskell type classes, Rust `dyn`	indirect calls through a table of functions	one copy, mixed types in one collection

Rust offers the second and third strategies too, through `dyn Trait`, at the programmer's choice:

```
/// The other strategy: one compiled function for every element type, working
/// on a uniform representation (a pointer to the data plus a pointer to a
/// table of functions). Chapter 15 studies `dyn` in detail.
fn show_all(items: &[&dyn Display]) -> String {
    let parts: Vec<String> = items.iter().map(|item| item.to_string()).collect();
    parts.join(", ")
}

/// The monomorphized counterpart: one copy per `T`, and all items must have
```

```
/// the same type.
fn show_all_generic<T: Display>(items: &[T]) -> String {
    let parts: Vec<String> = items.iter().map(|item| item.to_string()).collect();
    parts.join(", ")
}
```

```
dyn:     1, two, 3.5
generic: 1, 2, 3
size_of::<&i32>()          = 8
size_of::<&dyn Display>()  = 16
```

`show_all` is compiled once. Its elements are *fat pointers*: a pointer to the value plus a pointer to a table of the functions that `Display` requires for that value's type (a vtable), 16 bytes instead of 8. Every call to `to_string` goes through the table. In exchange, one slice can hold an integer, a string and a float. `show_all_generic` is compiled once per element type, makes direct calls, and requires all elements to have the same type. Chapter 14 explains dictionaries and Chapter 15 explains `dyn` as an existential type.

> **NOTE**
>
> Monomorphization has a limit that System F does not. A generic function that calls itself at a *bigger* type, for example `fn nest<T: Debug>(x: T, n: u32) { if n > 0 { nest((x,), n - 1) } }`, needs copies for `T`, `(T,)`, `((T,),)` and so on without end. The type checker accepts it (it is a perfectly good System F program), but code generation fails with "reached the recursion limit while instantiating". Because the failure happens after type checking, this program is not part of the book's compile-fail suite, which only runs the type checker. Languages with a uniform representation compile such *polymorphic recursion* without trouble.

12.6 Rank-1 and rank-2

The **rank** of a type measures how deeply $\forall$ can be nested to the left of arrows. A type with no $\forall$ has rank 0. A type of the form $\forall \alpha_1 \ldots \alpha_n . \tau$ with τ of rank 0 has rank 1: all quantifiers are at the outside, as in every Rust function signature. A function whose *argument* is polymorphic, such as

$$(\forall \alpha. \mathrm{Vec}\,\alpha \to \mathrm{Vec}\,\alpha) \to \mathrm{Vec}\,u32 \to \mathrm{Vec}\,str \to \ldots$$

has rank 2: the function receives a polymorphic function, and can use it at any type it likes. System F allows any rank; Rust's types are rank-1.

The difference is about *who chooses the type*. In a rank-1 type, the caller of the function chooses every type argument. Try to write a function that applies the same operation to two vectors of different element types (`code/rust/compile_fail/ch12_rank2_closure.rs`):

```
fn lengths_after<A, B, F>(f: F, a: Vec<A>, b: Vec<B>) -> (usize, usize)
where
    F: Fn(Vec<A>) -> Vec<A>,
{
    (f(a).len(), f(b).len())
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch12_rank2_closure.rs:10:20
   |
 6 | fn lengths_after<A, B, F>(f: F, a: Vec<A>, b: Vec<B>) -> (usize, usize)
   |                  -  - found type parameter
   |                  |
   |                  expected type parameter
...
10 |     (f(a).len(), f(b).len())
   |                  - ^ expected `Vec<A>`, found `Vec<B>`
   |                  |
   |                  arguments to this function are incorrect
   |
   = note: expected struct `Vec<A>`
              found struct `Vec<B>`
   = note: a type parameter was expected, but a different one was found; you might be
missing a type parameter or trait bound
   = note: for more information, visit https://doc.rust-lang.org/book/ch10-02-traits.html#
traits-as-parameters
note: callable defined here
  --> compile_fail/ch12_rank2_closure.rs:8:8
   |
 8 |     F: Fn(Vec<A>) -> Vec<A>,
   |        ^^^^^^^^^^^^^^^^^^^
```

The signature says: for every `A`, `B` and `F` such that `F` is a function from `Vec<A>` to `Vec<A>`. The caller picks all three, so inside the body `f` is a function on `Vec<A>`, for the one `A` the caller chose, and applying it to a `Vec<B>` is a type error. What we wanted was the rank-2 type: "give me a function that works for *every* element type". In System F, Λ and $[\,]$ make that expressible, and our Lean checker accepted the rank-2 term `useTwice`, $\lambda(f : \forall\alpha.\,\alpha \to \alpha).\, f\,[\mathit{Bool}]\ \mathit{true}$.

Rank-2 through a trait with a generic method

Rust has no polymorphic function values, but it has traits whose *methods* are generic, and that is enough. The chapter's second example, `code/rust/examples/ch12_rank2.rs`, starts with list operations:

```rust
/// A value of a type implementing `ListOp` is a polymorphic function
/// ∀T. Vec<T> → Vec<T>: the *caller of* `apply` chooses `T`, every time.
trait ListOp {
    fn apply<T>(&self, items: Vec<T>) -> Vec<T>;
}

struct Reverse;
struct TakeFirst(usize);
struct RotateLeft(usize);

impl ListOp for Reverse {
    fn apply<T>(&self, mut items: Vec<T>) -> Vec<T> {
        items.reverse();
        items
    }
}

impl ListOp for TakeFirst {
    fn apply<T>(&self, mut items: Vec<T>) -> Vec<T> {
        items.truncate(self.0);
        items
    }
}

impl ListOp for RotateLeft {
    fn apply<T>(&self, mut items: Vec<T>) -> Vec<T> {
        if !items.is_empty() {
            let k = self.0 % items.len();
            items.rotate_left(k);
```

```
        }
        items
    }
}

/// The rank-2 function: (∀T. Vec<T> → Vec<T>) → Vec<u32> → Vec<&str> → ...
/// It uses its argument at two different element types.
fn on_both<'s>(
    op: &impl ListOp,
    numbers: Vec<u32>,
    words: Vec<&'s str>,
) -> (Vec<u32>, Vec<&'s str>) {
    (op.apply(numbers), op.apply(words))
}
```

A value of a type implementing `ListOp` is a polymorphic function: its method `apply` has its own type parameter `T`, and that parameter is chosen at each call of `apply`, not when the `ListOp` value is passed around. So `on_both` takes one operation and uses it at two element types, which is exactly the rank-2 type we wanted, with the trait standing in for $\forall T.\, \mathrm{Vec}\, T \to \mathrm{Vec}\, T$:

```
let numbers = vec![1, 2, 3, 4];
let words = vec!["alpha", "beta", "gamma"];
println!(
    "reverse:  {:?}",
    on_both(&Reverse, numbers.clone(), words.clone())
);
println!(
    "take 2:   {:?}",
    on_both(&TakeFirst(2), numbers.clone(), words.clone())
);
println!("rotate 1: {:?}", on_both(&RotateLeft(1), numbers, words));
```

```
reverse:  ([4, 3, 2, 1], ["gamma", "beta", "alpha"])
take 2:   ([1, 2], ["alpha", "beta"])
rotate 1: ([2, 3, 4, 1], ["beta", "gamma", "alpha"])
```

This is not a curiosity: it is how a large part of the Rust ecosystem is built. A serialization library must traverse a data structure whose fields have different types and hand each field to a *format* (JSON, binary, a debug printer), and the format must accept fields of any type. The format is therefore a value of a trait with generic methods:

```
/// A visitor must handle a field of *any* type that can be printed.
trait FieldVisitor {
    fn field<T: Debug>(&mut self, name: &str, value: &T);
}

struct ServerConfig {
    host: String,
    port: u16,
    verbose: bool,
    retries: Vec<u32>,
}

impl ServerConfig {
    /// Calls the visitor at four different types: `String`, `u16`, `bool`
    /// and `Vec<u32>`. Serialization libraries are built on this pattern.
    fn visit(&self, v: &mut impl FieldVisitor) {
        v.field("host", &self.host);
        v.field("port", &self.port);
        v.field("verbose", &self.verbose);
        v.field("retries", &self.retries);
    }
```

```
}

/// Renders `name = value` lines.
struct Printer(Vec<String>);

impl FieldVisitor for Printer {
    fn field<T: Debug>(&mut self, name: &str, value: &T) {
        self.0.push(format!("{name} = {value:?}"));
    }
}

/// Measures how much memory the field values occupy inline.
struct Sizer(usize);

impl FieldVisitor for Sizer {
    fn field<T: Debug>(&mut self, _name: &str, value: &T) {
        self.0 += size_of_val(value);
    }
}
```

```
let config = ServerConfig {
    host: "db.internal".to_string(),
    port: 5432,
    verbose: false,
    retries: vec![100, 200, 400],
};
let mut printer = Printer(Vec::new());
config.visit(&mut printer);
println!("{}", printer.0.join("; "));
let mut sizer = Sizer(0);
config.visit(&mut sizer);
println!("inline bytes of the fields: {}", sizer.0);
```

```
host = "db.internal"; port = 5432; verbose = false; retries = [100, 200, 400]
inline bytes of the fields: 51
```

`ServerConfig::visit` calls the visitor at four different types. Two very different visitors, one that prints and one that measures sizes, are passed to the same `visit`. (The serde library is built on this pattern: for example, its `SerializeStruct` trait has a generic method `serialize_field` that accepts a field of any serializable type.)

Church numerals are the classic rank-2 example, and they work the same way:

```
/// A Church numeral: for every type A, a way to apply a function n times.
/// This is the System F type ∀A. (A → A) → A → A.
trait Church {
    fn iterate<A>(&self, f: &dyn Fn(A) -> A, x: A) -> A;
}

struct Zero;
struct Succ<N>(N);
struct Add<M, N>(M, N);
struct Mul<M, N>(M, N);

impl Church for Zero {
    fn iterate<A>(&self, _f: &dyn Fn(A) -> A, x: A) -> A {
        x
    }
}

impl<N: Church> Church for Succ<N> {
    fn iterate<A>(&self, f: &dyn Fn(A) -> A, x: A) -> A {
        f(self.0.iterate(f, x))
    }
```

```
}

/// m + n: apply f n times, then m more times.
impl<M: Church, N: Church> Church for Add<M, N> {
    fn iterate<A>(&self, f: &dyn Fn(A) -> A, x: A) -> A {
        self.0.iterate(f, self.1.iterate(f, x))
    }
}

/// m * n: apply "apply f n times" m times. The inner numeral is used at the
/// same type A, the outer one with a function built from the inner one.
impl<M: Church, N: Church> Church for Mul<M, N> {
    fn iterate<A>(&self, f: &dyn Fn(A) -> A, x: A) -> A {
        self.0.iterate(&|y| self.1.iterate(f, y), x)
    }
}

fn to_u32(n: &impl Church) -> u32 {
    n.iterate(&|k| k + 1, 0)
}
```

```
let two = || Succ(Succ(Zero));
let three = || Succ(Succ(Succ(Zero)));
println!("2 + 3 = {}", to_u32(&Add(two(), three())));
println!("2 * 3 = {}", to_u32(&Mul(two(), three())));
// The same numeral at other types: strings and powers of two.
println!(
    "{}",
    three().iterate(&|s: String| s + "ab", String::from(">"))
);
println!("2^3 = {}", three().iterate(&|k: u64| 2 * k, 1));
```

```
2 + 3 = 5
2 * 3 = 6
>ababab
2^3 = 8
```

The trait `Church` is the type $\forall A.\,(A \to A) \to A \to A$. The numerals are used at `u32` (by `to_u32`, to count), at `String` and at `u64` (to compute 2^3), and inside `Mul` the outer numeral is used with a function built from the inner numeral. Two limitations show where the encoding differs from System F. Each numeral here is a value of a *different Rust type* (`Succ<Succ<Zero>>` is not `Succ<Zero>`), so a function cannot return "some numeral" computed at run time without further tricks. And `dyn Church` is not allowed: a trait with a generic method has infinitely many instantiations of that method, which no finite vtable can hold. Chapter 15 explains this rule (dyn compatibility, error E0038).

Higher-ranked trait bounds over lifetimes

For lifetimes, Rust does have nested quantifiers. The problem is the same as with types: if the caller chooses a lifetime, the function cannot use it for data that it creates itself (`code/rust/compile_fail/ch12_lifetime_rank1.rs`):

```
fn normalized_len<'a, F>(text: &str, pick: F) -> usize
where
    F: Fn(&'a str) -> &'a str,
{
    let lowered = text.to_lowercase();
    pick(&lowered).len()
}
```

```
error[E0597]: `lowered` does not live long enough
  --> compile_fail/ch12_lifetime_rank1.rs:11:10
   |
 6 | fn normalized_len<'a, F>(text: &str, pick: F) -> usize
   |                   -- lifetime `'a` defined here
...
10 |     let lowered = text.to_lowercase();
   |         ------- binding `lowered` declared here
11 |     pick(&lowered).len()
   |     -----^^^^^^^^-
   |     |    |
   |     |    borrowed value does not live long enough
   |     argument requires that `lowered` is borrowed for `'a`
12 | }
   | - `lowered` dropped here while still borrowed
   |
note: requirement that the value outlives `'a` introduced here
  --> /rustc/b940084d7eb6a299eb4bfeb8e34901bc051e7ac4/library/core/src/ops/function.rs:76:34
   |
76 | pub const trait Fn<Args: Tuple>: [const] FnMut<Args> {
   |                                  ^^^^^^^^^^^^^^^^^^^
```

The lifetime `'a` is a parameter of `normalized_len`, so the caller chooses it, and any lifetime the caller can name outlives the call. The callback accepts only `&'a str`, so `&lowered` would have to live for `'a`, longer than the function body in which `lowered` lives. The error says exactly that: `lowered` would need to be borrowed for `'a`, but it is dropped at the end of the function.

What the function needs is a callback that works for *every* lifetime, including the short one of a local variable. That is a **higher-ranked trait bound** (HRTB), written with `for<'a>`:

```rust
/// The callback is applied to a string that is created inside this function,
/// so no lifetime the caller could name would fit it. The bound says: for
/// every lifetime 'a, `pick` maps a `&'a str` to a `&'a str`.
fn normalized_len<F>(text: &str, pick: F) -> usize
where
    F: for<'a> Fn(&'a str) -> &'a str,
{
    let lowered = text.to_lowercase(); // a local String
    pick(&lowered).len()
}

/// The same bound, written with the usual sugar: an elided lifetime in an
/// `Fn` bound means `for<'a>`.
fn normalized_len_sugar(text: &str, pick: impl Fn(&str) -> &str) -> usize {
    let lowered = text.to_lowercase();
    pick(&lowered).len()
}

fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}
```

```rust
println!(
    "trim:       {}",
    normalized_len("  Hello World ", str::trim)
);
println!("first word: {}", normalized_len("Hello World", first_word));
println!(
    "with sugar: {}",
    normalized_len_sugar("Hello World", first_word)
);
// A function pointer type can be higher-ranked too.
let picker: for<'a> fn(&'a str) -> &'a str = str::trim_start;
println!("fn pointer: {}", normalized_len("   abc", picker));
```

```
trim:       11
first word: 5
with sugar: 5
fn pointer: 3
```

The bound `F: for<'a> Fn(&'a str) -> &'a str` reads "for every lifetime `'a`, `F` maps a `&'a str` to a `&'a str`", with the quantifier *inside* the bound: a rank-2 type over lifetimes. Now the *function* chooses `'a` at each call of `pick`, and it can choose the lifetime of `lowered`. You rarely see `for<'a>` written out, because the familiar `impl Fn(&str) -> &str` means exactly the same thing: when a lifetime is elided in an `Fn` bound, Rust quantifies over it in the bound, as `normalized_len_sugar` shows. Function pointer types can be higher-ranked too, as the last line shows: `for<'a> fn(&'a str) -> &'a str` is the type of `str::trim_start`. So Rust has first-class $\forall$ over lifetimes and only rank-1 $\forall$ over types; the lifetime part of Rust's type system is studied in Part VI.

12.7 Polymorphism in Lean

Lean does not need a separate Λ: a type is a value like any other, so a function can simply take a type as an argument. The file `code/lean/Book/Ch12/Universes.lean` starts with a small polymorphic function:

```
/-- Apply a function twice. The type `α` is an *implicit* argument: callers do
not pass it, Lean works it out from the other arguments. -/
def twice {α : Type} (f : α → α) (a : α) : α := f (f a)
```

The parameter `{α : Type}` is a type parameter, in curly braces to make it *implicit*: Lean fills it in by unification, as Rust infers type arguments. The `@` prefix turns all implicit parameters into explicit ones, which gives System F's type application; a named argument `(α := String)` supplies one implicit argument by name, much like a turbofish:

```
/-- info: 16 -/
#guard_msgs in
#eval twice (· + 3) 10

-- `@` makes every argument explicit: here we pass the type ourselves.
/-- info: 16 -/
#guard_msgs in
#eval @twice ℕ (· + 3) 10

/-- info: "hi!!" -/
#guard_msgs in
#eval twice (α := String) (· ++ "!") "hi"

/-- info: @twice : {α : Type} → (α → α) → α → α -/
#guard_msgs in
#check @twice
```

The type of `@twice` is `{α : Type} → (α → α) → α → α`. This is a *dependent* function type: the type of the later arguments mentions the first argument `α`. System F's $\forall \alpha.\, \tau$ is the special case of a dependent function type whose argument is a type; Lean also writes it `∀ α, τ`. Chapter 30 studies dependent functions in general.

Rank-2 functions need no tricks in Lean. A polymorphic function is an ordinary value, and a function can take one as an argument (`code/lean/Book/Ch12/Church.lean`):

```
/-- A rank-2 function: its *argument* is polymorphic, and it uses that argument
at two different types. In Lean this is an ordinary dependent function. -/
def onBoth (f : {α : Type} → List α → List α) (xs : List ℕ) (ys : List String) :
    List ℕ × List String :=
  (f xs, f ys)

/-- info: ([3, 2, 1], ["b", "a"]) -/
#guard_msgs in
#eval onBoth List.reverse [1, 2, 3] ["a", "b"]

/-- info: ([1], ["a"]) -/
#guard_msgs in
#eval onBoth (List.take 1) [1, 2, 3] ["a", "b"]
```

`onBoth` uses its argument at `ℕ` and at `String`. Compare it with the trait `ListOp` and the method `on_both` in Rust: same idea, without the indirection.

12.8 Universes, gently

So far we have written `α : Type` without asking what `Type` is. It is time to ask, because the answer explains several error messages you will meet sooner or later, and it is the place where Lean departs from System F. We go in small steps.

Every type has a type

In Lean every expression has a type, and types are expressions. So types have types too:

```
/-- info: ℕ : Type -/
#guard_msgs in
#check ℕ

/-- info: ℕ → List ℕ : Type -/
#guard_msgs in
#check ℕ → List ℕ

/-- info: Type : Type 1 -/
#guard_msgs in
#check Type

/-- info: Type 1 : Type 2 -/
#guard_msgs in
#check Type 1
```

`ℕ`, `ℕ → List ℕ`, `Bool` and `String` are types of "ordinary data", and their type is `Type`. `Type` is itself an expression, and its type is `Type 1`. The type of `Type 1` is `Type 2`, and so on forever. These are the **universes**, and they form a hierarchy: `Type` (also written `Type 0`) is the universe of small types, `Type 1` contains `Type` itself and everything built from it, and so on.

Why not stop at one level, with `Type : Type`? Because that makes the logic inconsistent. Girard showed that a type theory with a type of all types allows a version of the Burali-Forti paradox (about the set of all ordinals), now called Girard's paradox, from which a term of the empty type can be constructed. Lean therefore refuses to put `Type` in `Type`:

```
/--
error: Type mismatch
  Type
has type
  Type 1
```

```
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1`
-/
#guard_msgs in
example : Type := Type
```

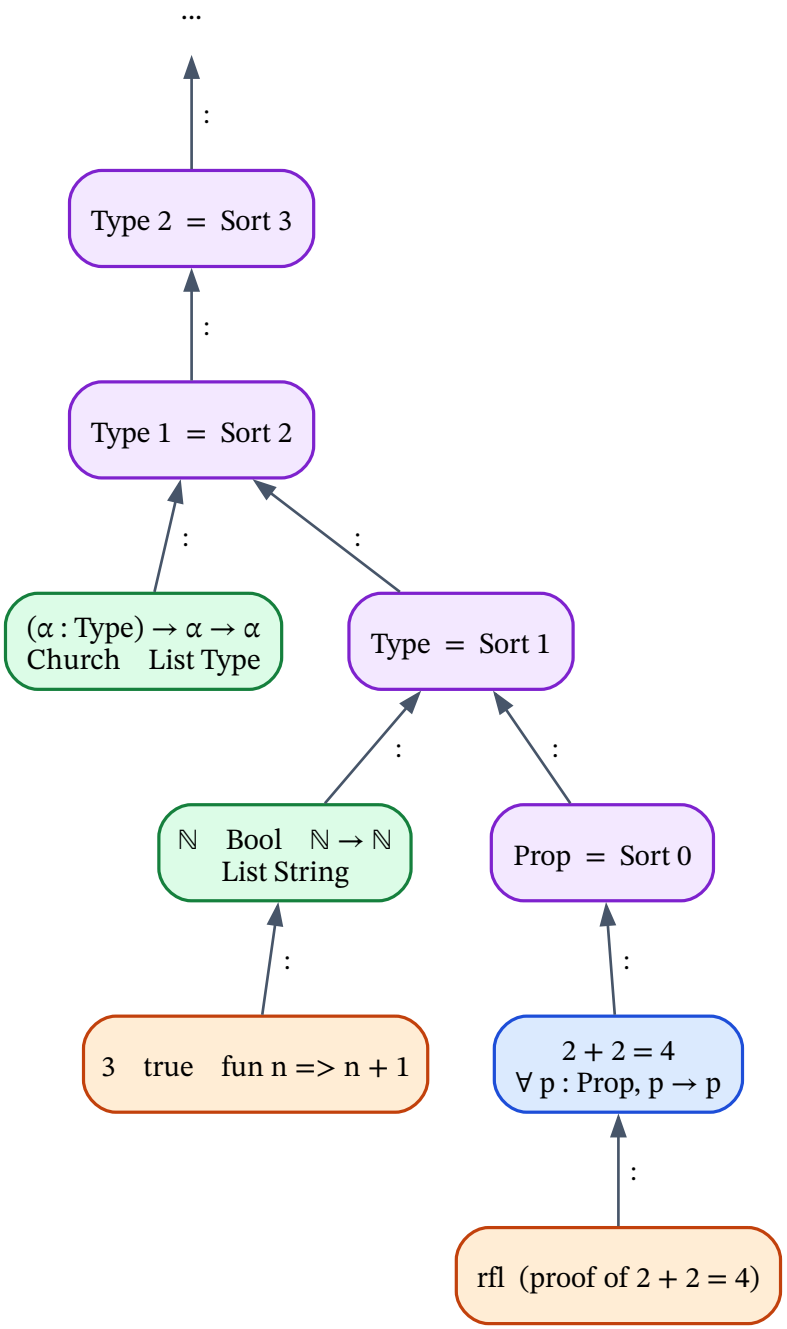
The message says: `Type` has type `Type 1`, not `Type`. (The extra line "of sort `Type 2`" is the type of the type, one level higher.)

Polymorphic types live one level up

Now consider the type of the polymorphic identity. It quantifies over all types in `Type`, and a type that talks about *all of* *`Type`* cannot itself be one of the types in `Type`: that would be circular in the same way. So it lives one level up:

```
/-- info: (α : Type) → α → α : Type 1 -/
#guard_msgs in
#check (α : Type) → α → α

/--
error: Type mismatch
  (α : Type) → α → α
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1`
-/
#guard_msgs in
def PolyId : Type := (α : Type) → α → α

/-- The type of polymorphic identity functions is fine one level up. -/
def PolyId' : Type 1 := (α : Type) → α → α

/-- Its only (sensible) inhabitant. -/
def polyId : PolyId' := fun _ a => a
```

This is **predicativity**: a type that quantifies over a universe lives in a higher universe than the one it quantifies over. The general rule for function types is that `(x : A) → B` lives in the larger of the universes of `A` and `B`. In `(α : Type) → α → α`, the domain `Type` lives in `Type 1`, so the whole type lives in `Type 1`.



Lean's universes. Each arrow means "has type": Prop has type Type, Type has type Type 1, and polymorphic types such as (a : Type) → a → a live in Type 1, one level above the types they quantify over.

Prop is different

Propositions have their own universe, `Prop`, which sits below `Type`:

```
/-- info: Prop : Type -/
#guard_msgs in
#check Prop

/-- info: 2 + 2 = 4 : Prop -/
#guard_msgs in
```

```
#check 2 + 2 = 4

-- A quantifier over *all* propositions is still a proposition:
-- `Prop` is impredicative.
/-- info: ∀ (p : Prop), p → p : Prop -/
#guard_msgs in
#check ∀ p : Prop, p → p

theorem polyIdProof : ∀ p : Prop, p → p := fun _ hp => hp
```

The last check is the surprise. `∀ p : Prop, p → p` quantifies over *all* propositions, and yet it is itself a proposition, in `Prop`, not in a higher universe. `Prop` is **impredicative**, exactly like System F's types: a proposition may quantify over all propositions, including itself. Chapter 10 explained why this is safe for `Prop` in particular: proofs are irrelevant (any two proofs of the same proposition are equal), and information cannot flow from proofs into ordinary data except in restricted ways, which blocks the paradoxes. So Lean has impredicativity where it is safe, for propositions, and predicativity for data.

Sort: one name for all levels

Lean's internal name for universes is `Sort`. `Prop` is `Sort 0`, and `Type u` is `Sort (u + 1)`:

```
/-- info: Prop : Type -/
#guard_msgs in
#check Sort 0

/-- info: Type : Type 1 -/
#guard_msgs in
#check Sort 1

/-- info: Type 1 : Type 2 -/
#guard_msgs in
#check Sort 2
```

With this naming, the rule for function types can be stated once. If `A` lives in `Sort u` and `B` lives in `Sort v`, then

$$\frac{\Gamma \vdash A : \mathit{Sort}\ u \qquad \Gamma, x : A \vdash B : \mathit{Sort}\ v}{\Gamma \vdash (x : A) \to B : \mathit{Sort}\ (\mathrm{imax}\ u\ v)}\ \text{T-Pi}$$

where $\mathrm{imax}\ u\ v$ is 0 if $v = 0$, and the maximum of u and v otherwise. The first case is impredicativity: whatever you quantify over, if the body is a proposition, the result is a proposition. The second case is predicativity. Check the two examples: in `(α : Type) → α → α`, the domain `Type` has type `Sort 2` and the body `α → α` has type `Sort 1`, so the result is in `Sort 2`, that is `Type 1`. In `∀ p : Prop, p → p`, the domain `Prop` has type `Sort 1` and the body `p → p` has type `Sort 0`, so the result is in `Sort 0`, that is `Prop`.

Universe polymorphism

If polymorphic types live one level up, do we need a separate identity function for each level? No: definitions can be polymorphic in the level itself.

```
universe u

/-- One identity function for every universe: `u` is a *universe variable*. -/
def myId {α : Sort u} (a : α) : α := a
```

```
/-- info: myId 3 : ℕ -/
#guard_msgs in
#check myId 3            -- α := ℕ,       ℕ : Type   = Sort 1

/-- info: myId ℕ : Type -/
#guard_msgs in
#check myId ℕ            -- α := Type,    Type : Type 1 = Sort 2

/-- info: myId polyIdProof : ∀ (p : Prop), p → p -/
#guard_msgs in
#check myId polyIdProof  -- α is a proposition, α : Prop = Sort 0

/-- info: List.{u} (α : Type u) : Type u -/
#guard_msgs in
#check List
```

`universe u` declares a *universe variable*, and `{α : Sort u}` lets `myId` work at every level: on numbers (`α := ℕ`, in `Sort 1`), on types (`α := Type`, in `Sort 2`) and on proofs (`α` a proposition, in `Sort 0`). Library definitions are written this way: `List.{u}` takes a type in any universe `Type u`. When you see `Type u_1` or `.{u_1, u_2}` in a Mathlib signature, these are universe variables that Lean introduced automatically (Mathlib often writes `Type*` in its sources to ask for a fresh one). In everyday code you can mostly ignore them; they matter when a definition must work for types that are themselves polymorphic.

Finally, `ULift` moves a type up the hierarchy when some API demands a type in a higher universe:

```
/-- `ULift` copies a type into a higher universe, when an API demands one. -/
def liftedThree : ULift.{1} ℕ := ULift.up 3

/-- info: ULift ℕ : Type 1 -/
#guard_msgs in
#check ULift.{1} ℕ

/-- info: 3 -/
#guard_msgs in
#eval liftedThree.down
```

12.9 Church numerals in Lean

With universes understood, we can transcribe System F's Church numerals into Lean (`code/lean/Book/Ch12/Church.lean`):

```
/-- Church numerals: the number `n` is "apply `f` to `x`, `n` times", for every
type `α`. The `∀α` makes it a type of `Type 1`. (`abbrev` is a definition that
Lean unfolds freely, like a `type` alias in Rust.) -/
abbrev Church : Type 1 := (α : Type) → (α → α) → α → α

namespace Church

def zero : Church := fun _ _ x => x
def succ (n : Church) : Church := fun α f x => f (n α f x)
def add (m n : Church) : Church := fun α f x => m α f (n α f x)
def mul (m n : Church) : Church := fun α f => m α (n α f)

/-- Read a numeral back: instantiate `α := ℕ`, `f := (· + 1)`, `x := 0`. -/
def toNat (n : Church) : ℕ := n ℕ (· + 1) 0

def ofNat : ℕ → Church
  | 0 => zero
  | k + 1 => succ (ofNat k)
```

The type `Church` quantifies over `Type`, so it lives in `Type 1`, as the previous section predicts. `abbrev` declares a definition that Lean unfolds whenever it needs to, like a `type` alias in Rust, so that Lean sees a `Church` as the function type it stands for. Addition runs `n` first and `m` after it; multiplication iterates "apply `f` `n` times", `m` times. `toNat` instantiates a numeral at `ℕ` with the successor function and zero, which counts the applications:

```
/-- info: 7 -/
#guard_msgs in
#eval toNat (add (ofNat 3) (ofNat 4))

/-- info: 12 -/
#guard_msgs in
#eval toNat (mul (ofNat 3) (ofNat 4))

-- The same numeral used at another type: three times "append a bang".
/-- info: "go!!!" -/
#guard_msgs in
#eval ofNat 3 String (· ++ "!") "go"
```

Now we can prove that Church arithmetic is arithmetic. The key lemma says what a numeral does: `ofNat k` applies `f` exactly `k` times. Mathlib writes the `k`-th iterate of `f` as `f^[k]`:

```
/-- The numeral `ofNat k` applies `f` exactly `k` times: `f^[k]` is Mathlib's
notation for the `k`-th iterate of `f`. -/
theorem ofNat_apply (k : ℕ) (α : Type) (f : α → α) (x : α) :
    ofNat k α f x = f^[k] x := by
  induction k with
  | zero => rfl
  | succ k ih =>
    -- ofNat (k + 1) α f x = f (ofNat k α f x) = f (f^[k] x) = f^[k + 1] x
    show f (ofNat k α f x) = f^[k + 1] x
    rw [ih, Function.iterate_succ_apply']

/-- The same fact about the function itself (by function extensionality). -/
theorem ofNat_eq_iterate (k : ℕ) (α : Type) (f : α → α) : ofNat k α f = f^[k] :=
  funext (ofNat_apply k α f)

/-- Counting up from 0, `k` times, gives `k`. -/
theorem succ_iterate (k : ℕ) : (· + 1)^[k] 0 = k := by
  induction k with
  | zero => rfl
  | succ k ih => rw [Function.iterate_succ_apply', ih]

theorem toNat_ofNat (k : ℕ) : toNat (ofNat k) = k := by
  unfold toNat
  rw [ofNat_apply, succ_iterate]
```

`ofNat_apply` is proved by induction on `k`. In the step case, `show` unfolds one layer of `ofNat (k + 1)` (Lean checks that the new goal is the old one up to computation), the induction hypothesis replaces `ofNat k α f x` by `f^[k] x`, and Mathlib's lemma `Function.iterate_succ_apply'` states that `f^[k + 1] x = f (f^[k] x)`. `toNat_ofNat` uses it at `ℕ`, plus a lemma `succ_iterate` (adding one, `k` times, starting from zero, gives `k`). Then:

```
/-- Church addition is addition. The two numerals are functions, so we prove
them equal with `funext`: they agree on every type, function and start value. -/
theorem add_ofNat (a b : ℕ) : add (ofNat a) (ofNat b) = ofNat (a + b) := by
  funext α f x
  -- Goal: ofNat a α f (ofNat b α f x) = ofNat (a + b) α f x
  simp only [add, ofNat_apply, Function.iterate_add_apply]

/-- Church multiplication is multiplication: `f^[b]` iterated `a` times is
```

```
`f^[b * a]`. -/
theorem mul_ofNat (a b : ℕ) : mul (ofNat a) (ofNat b) = ofNat (a * b) := by
  funext α f x
  simp only [mul, ofNat_apply, ofNat_eq_iterate]
  -- Goal: f^[b]^[a] x = f^[a * b] x
  rw [← Function.iterate_mul, Nat.mul_comm]
```

Church numerals are functions, so to prove two of them equal we use function extensionality (Chapter 11), here with three arguments at once: `funext α f x`. For addition, after rewriting both numerals as iterates, the goal is `f^[a] (f^[b] x) = f^[a + b] x`, which is Mathlib's `Function.iterate_add_apply` read backwards. For multiplication the goal is `(f^[b])^[a] x = f^[a * b] x`: iterating `f^[b]` `a` times. `Function.iterate_mul` says `f^[m * n] = (f^[m])^[n]`, so rewriting with it from right to left turns the left side into `f^[b * a] x`, and commutativity of multiplication finishes.

The predicativity wall

System F defined exponentiation by instantiating a numeral at the type of numerals. In Lean that is exactly what predicativity forbids:

```
/--
error: Application type mismatch: The argument
  Church
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1` in the application
  n Church
-/
#guard_msgs in
def Church.pow (m n : Church) : Church := n Church (Church.mul m) (Church.ofNat 1)
```

A `Church` accepts types from `Type`, and `Church` itself lives in `Type 1`. The way around is universe polymorphism: define numerals at every level, and let a numeral of level 1 iterate over numerals of level 0:

```
universe u

/-- Universe-polymorphic numerals: one type of numerals for each level `u`. -/
abbrev ChurchU : Type (u + 1) := (α : Type u) → (α → α) → α → α

def ChurchU.ofNat : ℕ → ChurchU.{u}
  | 0 => fun _ _ x => x
  | k + 1 => fun α f x => f (ChurchU.ofNat k α f x)

def ChurchU.mul (m n : ChurchU.{u}) : ChurchU.{u} := fun α f => m α (n α f)

/-- Exponentiation `m ^ n`: the exponent `n` is a numeral of the level above,
so it can iterate over the type of numerals of level 0. -/
def ChurchU.pow (m : ChurchU.{0}) (n : ChurchU.{1}) : ChurchU.{0} :=
  n ChurchU.{0} (ChurchU.mul m) (ChurchU.ofNat 1)

/-- info: 81 -/
#guard_msgs in
#eval ChurchU.pow (ChurchU.ofNat 3) (ChurchU.ofNat 4) ℕ (· + 1) 0
```

`ChurchU.{1}` quantifies over `Type 1`, which contains `ChurchU.{0}`, so `n` can be instantiated at the type of level-0 numerals. Each level can talk about the levels below it, never about itself, and that is enough for exponentiation: $3^4 = 81$.

Booleans, pairs and impredicative propositions

The other Church encodings carry over in the same way:

```
/-- Booleans: a boolean chooses between two alternatives. -/
abbrev CBool : Type 1 := (α : Type) → α → α → α
def ctrue : CBool := fun _ t _ => t
def cfalse : CBool := fun _ _ f => f
def cnot (b : CBool) : CBool := fun α t f => b α f t
def CBool.toBool (b : CBool) : Bool := b Bool true false

/-- Pairs: a pair is "give me a two-argument consumer, and I will feed it". -/
abbrev CPair (α β : Type) : Type 1 := (γ : Type) → (α → β → γ) → γ
def CPair.mk {α β : Type} (a : α) (b : β) : CPair α β := fun _ k => k a b
def CPair.fst {α β : Type} (p : CPair α β) : α := p α (fun a _ => a)
def CPair.snd {α β : Type} (p : CPair α β) : β := p β (fun _ b => b)

/-- info: false -/
#guard_msgs in
#eval (cnot ctrue).toBool

/-- info: (3, "three") -/
#guard_msgs in
#eval let p := CPair.mk 3 "three"; (p.fst, p.snd)
```

In `Prop`, the encodings need no universe bump, because `Prop` is impredicative. They define the logical connectives from ∀ and → alone, and Lean can prove that they agree with the built-in ones:

```
/-- "p and q", encoded impredicatively: for every proposition r, if p and q
together give r, then r. This quantifies over all of `Prop`, and is a `Prop`. -/
def CAnd (p q : Prop) : Prop := ∀ r : Prop, (p → q → r) → r

/-- "p or q": for every r, if each of p and q gives r, then r. -/
def COr (p q : Prop) : Prop := ∀ r : Prop, (p → r) → (q → r) → r

/-- "false": every proposition holds. -/
def CFalse : Prop := ∀ r : Prop, r

theorem cand_iff (p q : Prop) : CAnd p q ↔ p ∧ q := by
  constructor
  · intro h
    -- instantiate r := p ∧ q and give the pairing function
    exact h (p ∧ q) (fun hp hq => ⟨hp, hq⟩)
  · intro ⟨hp, hq⟩ r k
    exact k hp hq

theorem cor_iff (p q : Prop) : COr p q ↔ p ∨ q := by
  constructor
  · intro h
    exact h (p ∨ q) Or.inl Or.inr
  · intro h r kp kq
    cases h with
    | inl hp => exact kp hp
    | inr hq => exact kq hq

theorem cfalse_iff : CFalse ↔ False := by
  constructor
  · intro h
    exact h False
  · intro h
    exact h.elim
```

`CAnd p q` says: for every proposition `r`, if `p` and `q` together imply `r`, then `r`. To get `p ∧ q` from it, instantiate `r` with `p ∧ q` itself and supply the pairing function, which is an impredicative instantiation: `CAnd p q` quantifies over all propositions, including `p ∧ q`. The converse direction takes the pair apart and feeds the consumer `k`. `COr` is the same with the case rule, and `CFalse`, "every

proposition holds", implies `False` by instantiating `r` with `False`. These are the logical readings of the System F encodings of products, sums and the empty type, and they show that in an impredicative system the connectives are definable from $\forall$ and $\to$.

Not every Church numeral is a numeral

We proved `toNat (ofNat k) = k`. What about the other direction, `ofNat (toNat n) = n` for every `n : Church`? In System F, every closed term of type $CNat$ is equal to a numeral: it has a normal form, and the only normal forms of that type are numerals. Parametricity (Chapter 18) explains the same fact semantically: a function that must work uniformly for every type can only apply `f` some fixed number of times. Lean does not guarantee that. With classical logic, a "numeral" can ask whether its type argument is `Bool` and behave differently:

```
/-- The natural numbers and the booleans are different types. -/
theorem nat_ne_bool : ℕ ≠ Bool := by
  intro h
  -- If they were equal, ℕ would be finite, like Bool.
  have : Finite ℕ := h ▸ inferInstance
  exact not_finite ℕ

open Classical in
/-- With classical logic we can ask "is α the type Bool?" and behave
differently: once on `Bool`, twice everywhere else. -/
noncomputable def weird : Church :=
  fun α f x => if α = Bool then f x else f (f x)

/-- `weird` is not `ofNat k` for any `k`: on `Bool` with `not` it acts like an
odd numeral, on `ℕ` with `(· + 1)` like the numeral 2. -/
theorem weird_ne_ofNat (k : ℕ) : weird ≠ Church.ofNat k := by
  intro h
  have hb := congrFun (congrFun (congrFun h Bool) not) true
  have hn := congrFun (congrFun (congrFun h ℕ) (· + 1)) 0
  -- hb : weird Bool not true = ofNat k Bool not true, and so on
  simp [weird, Church.ofNat_apply, nat_ne_bool] at hb hn
  -- hb : not^[k] true = false        hn : 2 = (fun x => x + 1)^[k] 0
  rw [Church.succ_iterate] at hn
  subst hn
  -- hb : not^[2] true = false, but not (not true) = true
  exact absurd hb (by decide)
```

`weird` applies `f` once on `Bool` and twice on every other type. The theorem shows that no standard numeral behaves like this. Applying the assumed equality `h : weird = ofNat k` to `Bool`, `not` and `true`, and to `ℕ`, `(· + 1)` and `0` (with `congrFun`, which applies both sides of an equality of functions to the same argument) gives two equations. `simp` evaluates `weird` on both, using `nat_ne_bool` to decide the `if` on `ℕ`. The equation on `ℕ` says that `k` is 2, and then the equation on `Bool` says that `not (not true) = false`, which `decide` refutes. (`nat_ne_bool` itself is proved by noticing that `ℕ` is infinite and `Bool` is finite.)

So Lean's type `Church` contains more than the numerals. Parametricity is a fact *about* System F, provable from outside it, and it fails for Lean with classical logic, which can inspect types through `if α = Bool`. Rust breaks it too, in its own ways, as Chapter 18 will show.

12.10 Putting it together: self-application in three languages

The impredicative self-application $\lambda(x : \forall\alpha.\,\alpha \to \alpha).\, x\,[\forall\alpha.\,\alpha \to \alpha]\,x$ is a good test of what each language allows. Let us take it through all three.

System F. Write $I = \forall\alpha.\,\alpha \to \alpha$ and $\Gamma = x : I$. The derivation has two branches, one per occurrence of x:

$$\dfrac{\dfrac{\dfrac{(x : I) \in \Gamma}{\Gamma \vdash x : \forall\alpha.\,\alpha \to \alpha}\,\text{T-Var}}{\Gamma \vdash x\,[I] : I \to I}\,\text{T-TApp} \qquad \dfrac{(x : I) \in \Gamma}{\Gamma \vdash x : I}\,\text{T-Var}}{\Gamma \vdash x\,[I]\,x : I}\,\text{T-App}$$

followed by T-Abs, which gives $\vdash \lambda(x : I).\, x\,[I]\,x : I \to I$. The step that matters is T-TApp with $\sigma = I$: substituting I for α in $\alpha \to \alpha$ gives $I \to I$, so the first occurrence of x becomes a function that accepts the second. Our Lean checker reached the same conclusion: `check selfApp` printed `(∀α. α → α) → ∀α. α → α`.

Rust. There is no type I to write, but a trait with a generic method plays its role, and the method can be instantiated at the type of the value itself:

```
/// ∀α. α → α as a trait: implementors behave like a polymorphic identity.
trait PolyId {
    fn call<T>(&self, x: T) -> T;
}

struct Identity;

impl PolyId for Identity {
    fn call<T>(&self, x: T) -> T {
        x
    }
}

/// System F's λ(x : ∀α. α → α). x [∀α. α → α] x, in Rust: the method is
/// instantiated at the type of `x` itself (here `&F`), and applied to `x`.
fn self_apply<F: PolyId>(x: &F) -> &F {
    x.call(x)
}
```

```
let same = self_apply(&Identity);
println!("self-application: {}", same.call("still the identity"));
```

```
self-application: still the identity
```

In `x.call(x)`, the method's `T` is chosen to be `&F`, the type of `x`. Rust's generic methods are quantified over all types, including the implementing type, so in this narrow sense the trait encoding is impredicative.

Lean. In `Type`, predicativity rejects the program; in `Prop`, it is accepted; and with universe levels, a polymorphic identity of level 1 can take one of level 0:

```
/--
error: Application type mismatch: The argument
  (α : Type) → α → α
has type
  Type 1
```

```
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1` in the application
  x ((α : Type) → α → α)
-/
#guard_msgs in
def selfApp (x : (α : Type) → α → α) : (α : Type) → α → α :=
  x ((α : Type) → α → α) x

/-- In `Prop`, the same self-application is accepted: `Prop` is impredicative. -/
theorem selfAppProp (x : ∀ p : Prop, p → p) : ∀ p : Prop, p → p :=
  x (∀ p : Prop, p → p) x

/-- In `Type`, a polymorphic identity of level 1 can take one of level 0. -/
def PolyIdU : Type (u + 1) := (α : Type u) → α → α

def applyUp (x : PolyIdU.{1}) (y : PolyIdU.{0}) : PolyIdU.{0} :=
  x PolyIdU.{0} y
```

The three outcomes summarize the chapter. System F's ∀ ranges over everything, which makes it expressive and makes type inference undecidable. Rust keeps ∀ over types at the outside of item signatures and compiles it by monomorphization; traits with generic methods and `for<'a>` bounds give back the higher ranks that real programs need. Lean's dependent functions contain System F's ∀, and its universes keep impredicativity for propositions, where it is safe, while data climbs the hierarchy one level at a time.

12.11 Common confusions

"**A generic function is a function that takes a type at run time.**" In System F it is, in principle, but types never influence the computation, and every implementation strategy exploits that. Rust resolves all type arguments at compile time and generates one copy per instantiation; no type is passed at run time.

"**`impl Fn(&str) -> &str` has a lifetime parameter chosen by the caller.**" The elided lifetime in an `Fn` bound is quantified *inside* the bound: it means `for<'a> Fn(&'a str) -> &'a str`. That is why such a callback can be applied to borrows of local variables. A lifetime declared on the function (`fn f<'a>`) is chosen by the caller.

"**`Type` is the type of all types.**" It is the type of small types. `Type` itself, `(α : Type) → α → α` and `List Type` live in `Type 1`, and the hierarchy goes on. `Prop` is a separate universe at the bottom.

"**Universes are about size.**" They are about *what a type quantifies over*. A polymorphic function type is in `Type 1` not because its values are large but because its definition mentions all of `Type`.

"**Impredicative means inconsistent.**" System F and Lean's `Prop` are impredicative and consistent. What is inconsistent is `Type : Type`, and, in Chapter 11, negative inductive types; impredicativity has to be combined with care, but it is not a paradox by itself.

"**Monomorphization is the same as C++ templates, so Rust generics are templates.**" Both generate one copy per instantiation, but the body of a C++ template is fully checked only when it is instantiated, while a Rust generic function is type-checked once, against its bounds, before any copy is made. Rust's generics are System F with bounds, compiled like templates.

12.12 Summary

- System F adds type abstraction $\Lambda\alpha.\, e$, type application $e\,[\tau]$ and the type $\forall\alpha.\, \tau$ to the simply typed λ-calculus, with rules T-TAbs and T-TApp, the introduction and elimination rules for $\forall$.
- Church encodings express booleans, pairs, sums and numerals with $\forall$ and $\to$; each encoding is the type's elimination rule. Impredicativity lets a quantifier range over types that contain it, which allows definitions like exponentiation and self-application.
- A type checker for System F with de Bruijn indices fits in a page of Lean, and is proved sound and complete with respect to the typing rules.
- Rust generics are rank-1 System F with trait bounds. The turbofish is type application. Monomorphization compiles one copy per instantiation, trading binary size and compile time for speed and exact layouts; `dyn Trait` offers a single copy with dictionary passing instead.
- Rust recovers rank-2 polymorphism with traits whose methods are generic (list operations, visitors, Church numerals) and has first-class quantification over lifetimes through higher-ranked trait bounds `for<'a>`, which is what `Fn(&str) -> &str` means.
- In Lean, type parameters are ordinary (often implicit) arguments; `@` and named arguments make type application explicit, and rank-n functions are ordinary dependent functions.
- Every Lean type has a type: `Type : Type 1 : Type 2 : ...`, with `Prop` below. Types that quantify over a universe live one level up (predicativity), except in `Prop`, which is impredicative. `Sort u` names all levels, and universe polymorphism lets one definition serve every level.

12.13 Exercises

1. Write the full derivation of $\vdash \Lambda\alpha.\, \lambda(f : \alpha \to \alpha).\, \lambda(x : \alpha).\, f\,(f\,x) : \mathit{CNat}$, then type-check the same term with the Lean checker (it is `two` in `SystemF.lean`). *Hint:* the context at the top of the tree is $\alpha, f : \alpha \to \alpha, x : \alpha$, and the body needs T-App twice.
2. Define Church-encoded `and`, `or` and `not` on `CBool` in Lean, and prove that `(cand a b).toBool = (a.toBool && b.toBool)` when `a` and `b` are `ctrue` or `cfalse` (four cases). *Hint:* `cand a b := fun α t f => a α (b α t f) f`; each case holds by `rfl`.
3. In Rust, add a `Pow<M, N>` numeral to the `Church` trait of `ch12_rank2.rs`, computing m^n. Then explain why it type-checks even though Lean needed two universe levels for the same definition. *Hint:* m^n applied to `f` is "compose `f` with itself m times", iterated n times: instantiate `N`'s `iterate` at `Box<dyn Fn(A) -> A + 'a>`, in a helper function that names the lifetime `'a` of the borrowed `f` and `m`. Rust's generic methods range over all Rust types, which form a single "universe".
4. Write a function `apply_to_all` that takes a `&impl ListOp` and a `Vec<Vec<u32>>` and applies the operation both to the outer vector and to each inner vector. Which instantiations of `apply` does it use? How many copies of `RotateLeft::apply` does monomorphization generate for your program? *Hint:* one at `T = Vec<u32>`, one at `T = u32`; use the `type_name_of_val` trick of `ch12_generics.rs` inside `apply` to see them.
5. In Lean, what are the types of `List Type`, `Type → Type`, `(α : Type 1) → α` and `∀ (α : Type) (p : α → Prop), ∃ x, p x`? Predict each with rule T-Pi, then check with `#check`.

Hint: `Type → Type` is a function type whose domain and codomain are both in `Type 1`; the last one is a proposition, whatever it quantifies over.

6. Extend the System F checker with well-formedness: a function `Ty.wf : ℕ → Ty → Bool` that checks that every type variable index is smaller than the number of type variables in scope, a counter of type variables in `typeOf`, and a check of every annotation. Adapt `HasType` and the soundness proof. *Hint:* add a parameter `n : ℕ` (type variables in scope) to both `typeOf` and `HasType`; `tlam` increases it, and `lam` and `tapp` require `wf n` of their annotation.

CHAPTER 13

Type Inference

Look at this Rust function body:

```
let mut log = Vec::new();
log.push("started");
```

You never wrote the type of `log`, yet the compiler knows it is a `Vec<&str>`. It found out by noticing that `Vec::new()` produces a vector of *something*, that `push` adds a `&str` to it, and that the something must therefore be `&str`. This is **type inference**: reconstructing the types a programmer left out, by treating them as unknowns and solving the equations the program imposes on them.

In the 1970s Robin Milner designed type inference into ML, a language in which a whole program can be written without a single type annotation and still be statically typed. The algorithm, now called Hindley–Milner inference, is simple, fast in practice, and comes with a theorem saying that it always finds the *best* type a program can have. Rust's inference is a deliberately restricted cousin of it; Lean's elaborator is a far more ambitious relative. This chapter builds the algorithm, implements it in Rust for a small ML-like language, proves its core in Lean, and then uses it as a lens on what the Rust and Lean compilers do when you leave a type out.

In this chapter you will learn:

- what it means to infer a type, and what makes one type the *principal* type of a program;
- type variables and substitutions, and how substitutions compose;
- unification: how to solve equations between types, by hand and by machine, and why the occurs check rejects $\lambda x.\, x\, x$;
- the Hindley–Milner type system: let-polymorphism, generalization, instantiation, algorithm W, and the limits of all of it;
- how Rust's own inference works: local to function bodies, bounded by signatures, with literal defaulting, closures that are never generalized, and error E0282 when an unknown stays unknown;
- how Lean's elaborator uses metavariables and unification, how to read its messages, and how to prove that a unification algorithm is sound.

13.1 Why infer types

In Chapter 8 the simply typed λ-calculus came with annotations on its binders: you wrote $\lambda x : \texttt{Int}.\, x + 1$, and the checker read the type of `x` off the annotation. Bidirectional checking let you drop some of them, but a λ that is not checked against a known type still needs its annotation, because there is nothing else to tell the checker what `x` is.

Now take a function with no annotations at all:

$$twice = \lambda f.\, \lambda x.\, f\,(f\,x)$$

What is its type? It has many. If `f` is a function on integers, then $twice : (\texttt{Int} \to \texttt{Int}) \to \texttt{Int} \to \texttt{Int}$. If `f` is a function on booleans, then $twice : (\texttt{Bool} \to \texttt{Bool}) \to \texttt{Bool} \to \texttt{Bool}$. But some types are wrong: $(\texttt{Int} \to \texttt{Bool}) \to \texttt{Int} \to \texttt{Bool}$ fails, because the outer `f` would receive the `Bool` that the inner `f` returns. Every correct type has the same shape: $(a \to a) \to a \to a$ for some type a. That shape, with a left as a variable, is the **principal type** of `twice`: it is a correct type, and every other correct type is obtained from it by replacing the variable with something.

So type inference is not about guessing one type, but about computing the most general description of all the types a program can have. Chapter 3 distinguished *checking* a term against a type, *synthesizing* a type from an annotated term, and *inferring* one for a term without annotations. The first two follow the rules. The third needs a new idea: types with holes in them, and a way to fill the holes.

13.2 Type variables and substitutions

The holes are **type variables**. We write them α, β, γ in mathematics and `?0`, `?1`, `?2` in code, following the convention that rustc and Lean use for unknowns. The types of this chapter are:

$$\tau ::= \alpha \mid \texttt{Int} \mid \texttt{Bool} \mid \tau \to \tau \mid \tau \times \tau \mid \texttt{List}\ \tau$$

A type with variables stands for all the types you get by filling in the variables. $\alpha \to \alpha$ stands for $\texttt{Int} \to \texttt{Int}$, $\texttt{Bool} \to \texttt{Bool}$, $(\texttt{Int} \to \texttt{Bool}) \to (\texttt{Int} \to \texttt{Bool})$, and so on, but not for $\texttt{Int} \to \texttt{Bool}$.

A **substitution** says how to fill in variables: it maps finitely many type variables to types. We write $S = [\alpha \mapsto \texttt{Int}, \beta \mapsto \alpha \to \alpha]$ for the substitution that replaces α by `Int` and β by $\alpha \to \alpha$, *simultaneously*, and $S\,\tau$ for the result of applying it to τ:

$$S\,(\beta \to \alpha) = (\alpha \to \alpha) \to \texttt{Int}$$

The α inside $S(\beta)$ is not replaced again: the substitution looks at the original type once. Variables that S does not mention are left alone.

If S_1 and S_2 are substitutions, the **composition** $S_2 \circ S_1$ applies S_1 first and then S_2:

$$(S_2 \circ S_1)\,\tau = S_2\,(S_1\,\tau)$$

For example, with $S_1 = [\alpha \mapsto \beta \to \beta]$ and $S_2 = [\beta \mapsto \texttt{Int}]$, the composition maps α to $\texttt{Int} \to \texttt{Int}$ and β to `Int`. Composition is how inference accumulates knowledge: each step discovers a little substitution, and the answer is all of them composed.

Finally, τ' is an **instance** of τ when $\tau' = S\,\tau$ for some substitution S, and τ is then **more general** than τ'. The principal type of a program is a type of which every other type of the program is an instance.

Substitutions in Lean

The Lean development of this chapter starts in `code/lean/Book/Ch13/Unify.lean`. Types are an inductive type, with variables numbered by natural numbers, and a scoped notation `⇒` for the arrow (we leave pairs out of the Lean version; they would add a case to every proof and teach nothing new):

```
/-- The types of the chapter's examples: variables, two base types, functions
and lists. -/
inductive Ty where
  | var : Nat → Ty
  | int : Ty
  | bool : Ty
  | arrow : Ty → Ty → Ty
  | list : Ty → Ty
  deriving DecidableEq

/-- `a ⇒ b` is the function type `Ty.arrow a b`. -/
scoped infixr:25 " ⇒ " => Ty.arrow
```

A substitution is represented as a *function* from variable numbers to types. That looks wasteful (a function assigns a type to infinitely many variables), but a substitution that leaves a variable alone simply maps `n` to `.var n`, and functions make the algebra easy to state:

```
/-- A substitution assigns a type to every variable. (The variables it does
not change are mapped to themselves.) -/
def Subst := Nat → Ty

/-- Applying a substitution: replace every variable by its image. -/
def Ty.subst (σ : Subst) : Ty → Ty
  | .var n => σ n
  | .int => .int
  | .bool => .bool
  | .arrow a b => .arrow (a.subst σ) (b.subst σ)
  | .list a => .list (a.subst σ)

/-- The identity substitution changes nothing. -/
def Subst.id : Subst := Ty.var

/-- `τ.comp σ` is "first `σ`, then `τ`": the image of `n` is `σ n` with `τ`
applied to it. -/
def Subst.comp (τ σ : Subst) : Subst := fun n => (σ n).subst τ

/-- The substitution `[x := t]`, which changes only the variable `x`. -/
def Subst.single (x : Nat) (t : Ty) : Subst :=
  fun n => if n = x then t else .var n
```

`Ty.subst σ` walks a type and replaces each variable by its image; `comp` composes in the same order as $\circ$; `single x t` is the one-variable substitution $[x \mapsto t]$. Two theorems pin down how these behave:

```
/-- Applying the identity substitution changes nothing. -/
theorem subst_id (t : Ty) : t.subst Subst.id = t := by
  induction t with
  | var n => rfl
  | int => rfl
  | bool => rfl
  | arrow a b iha ihb => simp [Ty.subst, iha, ihb]
  | list a ih => simp [Ty.subst, ih]

/-- Applying a composition is applying one substitution after the other. -/
theorem subst_comp (τ σ : Subst) (t : Ty) :
```

```
    t.subst (τ.comp σ) = (t.subst σ).subst τ := by
  induction t with
  | var n => rfl
  | int => rfl
  | bool => rfl
  | arrow a b iha ihb => simp [Ty.subst, iha, ihb]
  | list a ih => simp [Ty.subst, ih]
```

Both are proved by induction on `t`. In the `var` case both sides compute to the same thing, so `rfl` closes the goal (for `subst_id`, `(Ty.var n).subst Subst.id` unfolds to `Ty.var n`). In the `arrow` case `simp` unfolds `Ty.subst` one level and rewrites with the induction hypotheses `iha` and `ihb`, after which both sides are identical; `list` is the same with one hypothesis. The second theorem justifies the definition of `comp`: applying the composition is applying one substitution after the other.

With these two facts, the usual algebraic laws follow in one line each:

```
/-- Substitutions under composition form a monoid: `id` is a unit on both
sides and composition is associative. -/
theorem Subst.id_comp (σ : Subst) : Subst.id.comp σ = σ := by
  funext n
  exact subst_id (σ n)

theorem Subst.comp_id (σ : Subst) : σ.comp Subst.id = σ := by
  funext n
  rfl

theorem Subst.comp_assoc (ρ τ σ : Subst) :
    (ρ.comp τ).comp σ = ρ.comp (τ.comp σ) := by
  funext n
  exact subst_comp ρ τ (σ n)
```

`funext` reduces an equation between functions to an equation between their values at an arbitrary `n` (`comp_id` even holds by definition). Substitutions under composition form a **monoid**, a structure you will meet again in Chapter 14 and, as a category with one object, in Chapter 16.

13.3 Unification

Inference turns a program into equations between types: a function of type $\alpha \to \texttt{Int}$ applied to a `Bool` needs α to be `Bool`; an `if` with branches of types β and `Int` needs them equal. The question is always: given τ_1 and τ_2, find a substitution S with $S\,\tau_1 = S\,\tau_2$. Such an S is a **unifier** of the two types, and the process of finding one is **unification**. We write the equation to be solved as $\tau_1 \doteq \tau_2$, to distinguish it from an equation that already holds.

A pair of types usually has many unifiers. $\alpha \to \beta \doteq \gamma \to \gamma$ is solved by $[\alpha \mapsto \gamma, \beta \mapsto \gamma]$, and also by $[\alpha \mapsto \texttt{Int}, \beta \mapsto \texttt{Int}, \gamma \mapsto \texttt{Int}]$. The first is better: the second is the first followed by $[\gamma \mapsto \texttt{Int}]$, so it commits to a choice the equation never asked for. A **most general unifier** (mgu) S is one through which every other unifier factors: for every unifier S' there is an R with $S' = R \circ S$. Inference always wants the mgu, because any extra commitment would make the inferred type less general than it could be.

Five problems by hand

The fastest way to understand unification is to do it. Here are five problems; the code later in the chapter solves the same five.

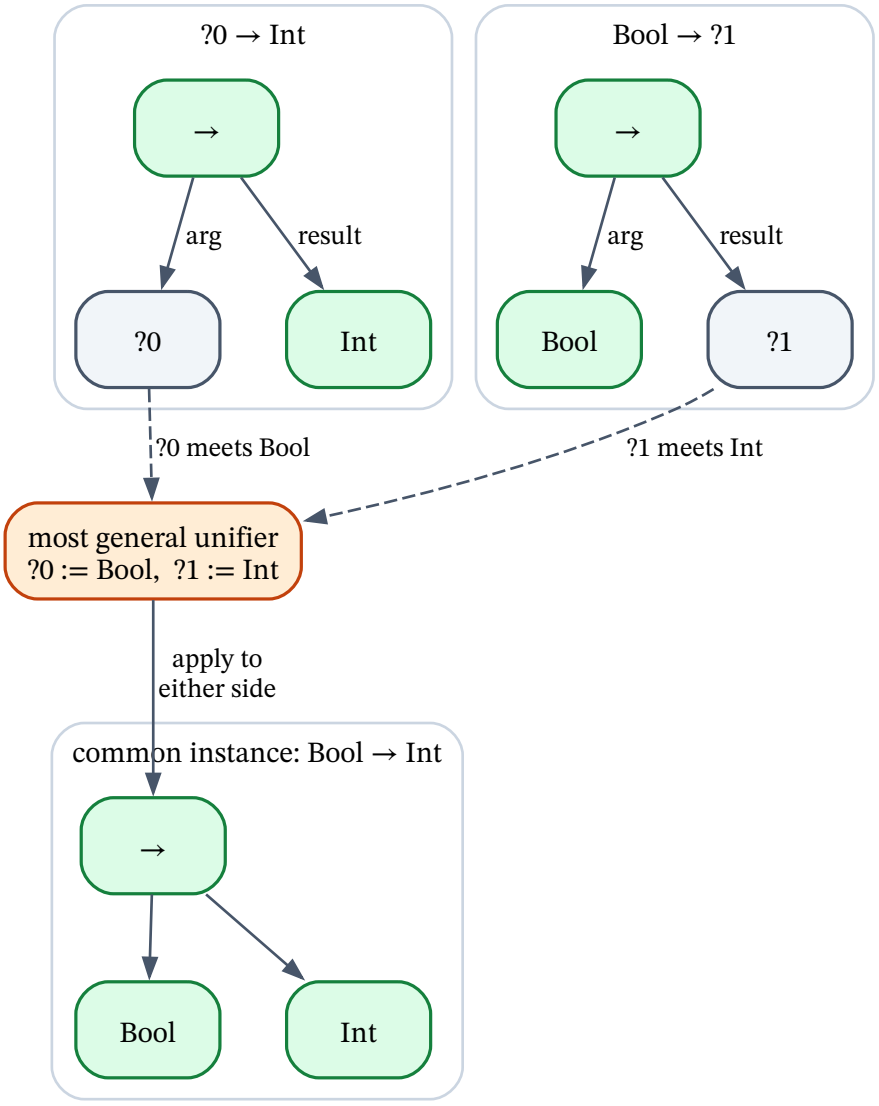
(1) $\alpha \to \texttt{Int} \doteq \texttt{Bool} \to \beta$. Both sides are arrows, and two arrows are equal exactly when their arguments are equal and their results are equal. So the problem *decomposes* into two smaller ones: $\alpha \doteq \texttt{Bool}$ and $\texttt{Int} \doteq \beta$. Each says that a variable must be a certain type, so we *solve* them: $\alpha \mapsto \texttt{Bool}$ and $\beta \mapsto \texttt{Int}$. Both sides become $\texttt{Bool} \to \texttt{Int}$.

(2) $\alpha \to \alpha \doteq \texttt{Int} \to \beta$. Decomposing gives $\alpha \doteq \texttt{Int}$ and $\alpha \doteq \beta$. Solving the first, $\alpha \mapsto \texttt{Int}$, changes the second: everything we still have to solve must be read *through* what we already know, and $\alpha \doteq \beta$ becomes $\texttt{Int} \doteq \beta$. So $\beta \mapsto \texttt{Int}$ too. This is the step that is easy to forget by hand and that every implementation must get right: after solving a variable, substitute its solution into the remaining equations.

(3) $\texttt{List}\ \alpha \to \beta \doteq \beta \to \texttt{List Int}$. Decompose: $\texttt{List}\ \alpha \doteq \beta$ and $\beta \doteq \texttt{List Int}$. The first solves $\beta \mapsto \texttt{List}\ \alpha$ (an equation can have the variable on either side). The second becomes $\texttt{List}\ \alpha \doteq \texttt{List Int}$, which decomposes to $\alpha \doteq \texttt{Int}$. The unifier is $[\alpha \mapsto \texttt{Int}, \beta \mapsto \texttt{List Int}]$: the solution for β was found in two stages, first a shape and then its contents.

(4) $\texttt{Int} \to \alpha \doteq \texttt{Bool} \to \alpha$. Decomposing gives $\texttt{Int} \doteq \texttt{Bool}$, and two different type constructors can never be made equal by any substitution, because substitutions only replace variables. This is a **clash**, and there is no unifier.

(5) $\alpha \doteq \alpha \to \beta$. The left side is a variable, so it looks like problem (1): solve $\alpha \mapsto \alpha \to \beta$. But applying that "solution" to the left side gives $\alpha \to \beta$, and to the right side gives $(\alpha \to \beta) \to \beta$. They are not equal, and substituting again only makes both sides grow. The equation asks for a type that is equal to a function type *whose argument is itself*. No finite type is like that: its size would have to equal its own size plus at least two. Unification must refuse to bind a variable to a type that contains it. This test is called the **occurs check**.



Unifying ?0 → Int with Bool → ?1: each variable meets the subtree in the same position on the other side, and the resulting substitution turns both trees into the same tree.

The rules

The five examples used five kinds of step. Collected and named, they are the unification algorithm, in the formulation of Martelli and Montanari as rules that transform a set E of equations (an earlier formulation is Robinson's, from 1965, who invented unification for automated theorem proving):

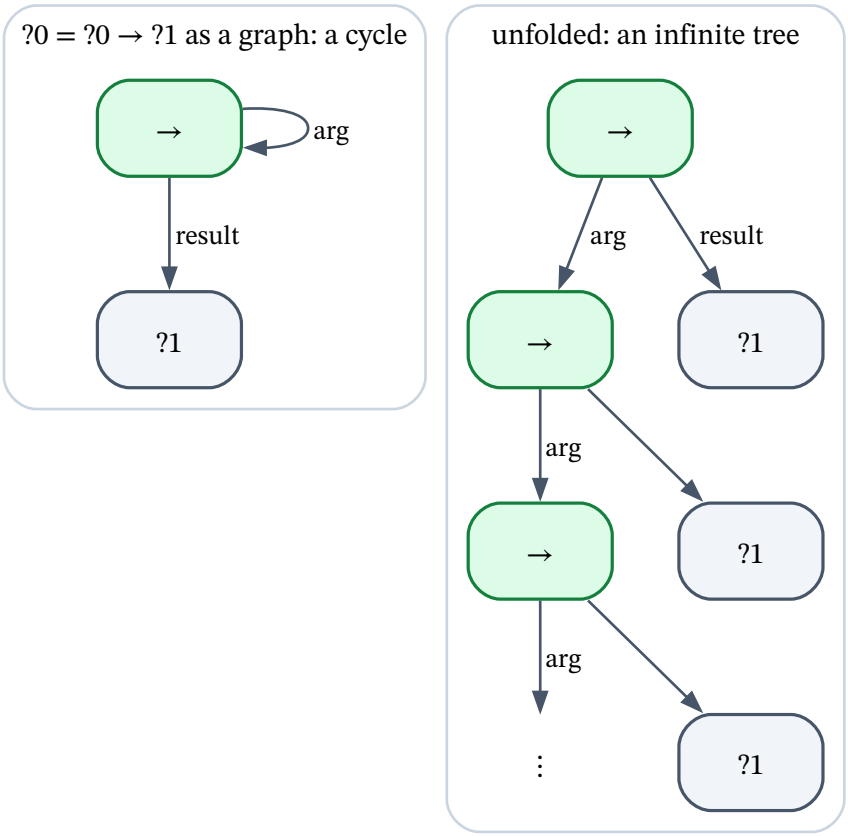
Rule	When the first equation is	Then
Delete	$\tau \doteq \tau$	drop it
Decompose	$C(\tau_1, \ldots, \tau_n) \doteq C(\tau'_1, \ldots, \tau'_n)$	replace it by $\tau_i \doteq \tau'_i$
Clash	$C(\ldots) \doteq D(\ldots)$ with $C \neq D$	fail
Occurs	$\alpha \doteq \tau$, α in τ, $\tau \neq \alpha$	fail
Eliminate	$\alpha \doteq \tau$, α not in τ	record $\alpha \mapsto \tau$, substitute in E

Here C and D are type constructors (`Int`, `Bool`, $\to$, $\times$, `List`), each applied to its arguments, and an equation $\tau \doteq \alpha$ with the variable on the right is first turned around (some presentations call that step Orient). The algorithm applies rules until it fails or no equation is left; the recorded bindings, composed in order, form the unifier.

The algorithm **terminates** (every Eliminate removes a variable for good, and between two Eliminates the other rules make the equations smaller) and is **correct**: if the equations have a unifier, it returns a most general one, and if it fails, there is none. Robinson proved this for his version, and the proof carries over. In Lean we will prove that what the algorithm returns is a unifier, and that the occurs check never rejects a solvable problem.

Why $\lambda x.\, x\, x$ has no type

You met $\lambda x.\, x\, x$ in Chapter 7, where it was the building block of the looping term Ω, and in Chapter 8, where the simply typed λ-calculus could not give it a type. Unification explains why no annotation could ever help. Give `x` an unknown type α. In `x x`, the first `x` is applied, so its type must be a function type $\alpha \doteq \alpha \to \beta$, whose argument type is the type of the second `x`, which is again α. That is problem (5), and the occurs check rejects it. The term would need an infinite type:



The equation ?0 = ?0 → ?1 has no finite solution. Read as a graph, a solution would be a cycle; unfolded, an infinite tree.

Some type systems allow such types (*equi-recursive* types; OCaml accepts them behind a compiler flag), but then Ω becomes typable and typed programs may loop. The occurs check keeps the types of ML finite.

13.4 Unification in Rust

The crate `hm` (in `code/rust/crates/hm`) implements everything in this chapter for a small ML-like language. Its types, in `src/types.rs`, use a uniform representation that matches the table of rules: a type is either a variable or a constructor applied to a list of argument types.

```
/// A type variable. While inference runs it is an unknown, printed `?0`,
/// `?1`, ...; in a final answer it is printed with a letter, `a`, `b`, ...
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct TyVar(pub u32);

/// The type constructors of mini-ML. Each has a fixed number of arguments.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum TyCon {
    Int,
    Bool,
    /// `a -> b`, the type of functions from `a` to `b`.
    Fun,
    /// `a * b`, the type of pairs.
    Pair,
    /// `List a`, the type of lists of `a`.
    List,
}

/// A monotype: a type variable, or a constructor applied to argument types.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Type {
    Var(TyVar),
    Con(TyCon, Vec<Type>),
}

/// A type scheme `forall vars. ty`. The variables in `vars` are *bound*: each
/// use of a variable with this scheme may choose them afresh.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Scheme {
    pub vars: Vec<TyVar>,
    pub ty: Type,
}
```

So `Int` is `Con(TyCon::Int, vec![])` and $\alpha \to \beta$ is `Con(TyCon::Fun, vec![Var(a), Var(b)])`. A `Scheme` is a type with some variables marked as bound; it becomes important when we reach polymorphism.

The substitution, in `src/unify.rs`, is stored the way rustc stores its own inference variables: as a table indexed by variable number, where entry `n` is `Some(t)` once $?n \mapsto t$ is known.

```
/// The substitution built so far, as a table of bindings.
#[derive(Default)]
pub struct Subst {
    bindings: Vec<Option<Type>>,
    /// When `Some`, every unification step is recorded here.
    trace: Option<Vec<String>>,
}

/// Why two types cannot be unified.
#[derive(Clone, Debug, PartialEq)]
pub enum UnifyError {
    /// Two different constructors met: `Int` against `Bool`, say.
    Mismatch(Type, Type),
```

```
    /// The variable would have to contain itself: `?0 = ?0 -> ?1`.
    Occurs(TyVar, Type),
}

impl Subst {
    pub fn new() -> Subst {
        Subst::default()
    }

    /// A type variable that is not bound and has never been used.
    pub fn fresh(&mut self) -> TyVar {
        self.bindings.push(None);
        TyVar(self.bindings.len() as u32 - 1)
    }

    /// The binding of `v`, if `v` has been solved.
    pub fn lookup(&self, v: TyVar) -> Option<&Type> {
        self.bindings[v.0 as usize].as_ref()
    }

    /// Applies the substitution to a type: every bound variable is replaced
    /// by its binding, recursively, until only unbound variables remain.
    pub fn apply(&self, ty: &Type) -> Type {
        match ty {
            Type::Var(v) => match self.lookup(*v) {
                Some(bound) => self.apply(bound),
                None => ty.clone(),
            },
            Type::Con(con, args) => {
                Type::Con(*con, args.iter().map(|a| self.apply(a)).collect())
            }
        }
    }
}
```

The table is in *triangular* form: a binding may mention a variable solved later (in problem (3), β is bound to `List` α before α is known), so `apply` follows bindings recursively. Composition is implicit: adding a binding *is* composing a one-variable substitution after everything found so far, and the table never has to be rewritten.

Unification itself transcribes the rules:

```
impl Subst {
    /// Extends the substitution so that `left` and `right` become equal, or
    /// explains why that is impossible.
    pub fn unify(&mut self, left: &Type, right: &Type) -> Result<(), UnifyError> {
        self.unify_at(left, right, 0)
    }

    fn unify_at(
        &mut self,
        left: &Type,
        right: &Type,
        depth: usize,
    ) -> Result<(), UnifyError> {
        // Look through variables that are already solved: we always compare
        // the current knowledge about both sides.
        let left = self.shallow(left);
        let right = self.shallow(right);
        self.log(depth, format!("unify {left}  ~  {right}"));
        match (&left, &right) {
            // Rule Delete: a variable is equal to itself.
            (Type::Var(a), Type::Var(b)) if a == b => Ok(()),
            // Rules Orient and Eliminate: solve the variable, unless the
            // occurs check fails.
            (Type::Var(v), other) | (other, Type::Var(v)) => {
                let other = self.apply(other);
```

```
                if other.mentions(*v) {
                    self.log(
                        depth + 1,
                        format!("occurs check: ?{} occurs in {other}", v.0),
                    );
                    return Err(UnifyError::Occurs(*v, other));
                }
                self.log(depth + 1, format!("?{} := {other}", v.0));
                self.bindings[v.0 as usize] = Some(other);
                Ok(())
            }
            // Rule Decompose: same constructor, unify the arguments pairwise.
            (Type::Con(c1, args1), Type::Con(c2, args2)) if c1 == c2 => {
                for (a1, a2) in args1.iter().zip(args2) {
                    self.unify_at(a1, a2, depth + 1)?;
                }
                Ok(())
            }
            // Rule Clash: different constructors can never be made equal.
            _ => {
                self.log(depth + 1, format!("clash: {left} is not {right}"));
                Err(UnifyError::Mismatch(self.apply(&left), self.apply(&right)))
            }
        }
    }
```

Walk through it with the table of rules beside you.

- `shallow` looks through solved variables at the root. This is problem (2)'s "read the remaining equations through what we know", done lazily: each equation is resolved when we reach it.
- Two equal variables: rule Delete.
- A variable on either side: rule Eliminate (and Orient, since the or-pattern `(Var(v), other) | (other, Var(v))` handles both orientations). Before binding, the code applies the substitution to `other` and checks whether `v` occurs in it: the occurs check. Applying first matters, because `v` may hide behind another variable that is already bound to a type mentioning `v`.
- Two constructors that are the same: rule Decompose, unify the arguments pairwise. For `Int` against `Int` there are no arguments, so this also covers Delete for constants.
- Anything else is a Clash.

The `log` calls record each step when tracing is on. The example `crates/hm/examples/ch13_unify.rs` runs the five problems of the previous section with tracing:

```
/// Pairs of types to unify. `?n` is a type variable.
const PROBLEMS: [(&str, &str); 5] = [
    ("?0 -> Int", "Bool -> ?1"),
    ("?0 -> ?0", "Int -> ?1"),
    ("List ?0 -> ?1", "?1 -> List Int"),
    ("Int -> ?0", "Bool -> ?0"),
    ("?0", "?0 -> ?1"),
];
```

```
(1) ?0 -> Int  ~  Bool -> ?1
  unify ?0 -> Int  ~  Bool -> ?1
    unify ?0  ~  Bool
      ?0 := Bool
    unify Int  ~  ?1
      ?1 := Int
  unifier: ?0 = Bool, ?1 = Int
```

```
  both sides become Bool -> Int
(2) ?0 -> ?0  ~  Int -> ?1
  unify ?0 -> ?0  ~  Int -> ?1
    unify ?0  ~  Int
      ?0 := Int
    unify Int  ~  ?1
      ?1 := Int
  unifier: ?0 = Int, ?1 = Int
  both sides become Int -> Int
(3) List ?0 -> ?1  ~  ?1 -> List Int
  unify List ?0 -> ?1  ~  ?1 -> List Int
    unify List ?0  ~  ?1
      ?1 := List ?0
    unify List ?0  ~  List Int
      unify ?0  ~  Int
        ?0 := Int
  unifier: ?0 = Int, ?1 = List Int
  both sides become List Int -> List Int
(4) Int -> ?0  ~  Bool -> ?0
  unify Int -> ?0  ~  Bool -> ?0
    unify Int  ~  Bool
      clash: Int is not Bool
  fails: Int and Bool clash
(5) ?0  ~  ?0 -> ?1
  unify ?0  ~  ?0 -> ?1
    occurs check: ?0 occurs in ?0 -> ?1
  fails: ?0 occurs in ?0 -> ?1
```

The trace follows the hand computations. In problem (2) the second equation is printed as `unify Int ~ ?1`, because `shallow` read it through the table; in problem (3) the second equation becomes `List ?0 ~ List Int`; (4) and (5) fail with a clash and with the occurs check.

13.5 Unification in Lean, proved sound

The Lean version of the algorithm works on a list of equations, the way the table of rules does, and returns either a unifier or the reason it failed:

```
/-- A substitution in triangular form: a list of bindings, applied from left
to right. Later bindings never mention earlier variables. -/
abbrev Bindings := List (Nat × Ty)

def Bindings.toSubst : Bindings → Subst
  | [] => Subst.id
  | (x, t) :: rest => (Bindings.toSubst rest).comp (Subst.single x t)

/-- What unification can answer. -/
inductive Outcome where
  | unifier : Bindings → Outcome
  | clash : Ty → Ty → Outcome
  | occurs : Nat → Ty → Outcome
  | outOfFuel : Outcome

/-- Applies `[x := t]` to both sides of every remaining equation. -/
def substEqs (x : Nat) (t : Ty) (eqs : List (Ty × Ty)) : List (Ty × Ty) :=
  eqs.map fun (a, b) => (a.subst (.single x t), b.subst (.single x t))

/-- The variable case: solve the equation `?x = t`, then unify the remaining
equations with `k`, and add the binding `x := t` in front of their unifier. -/
def solveVar (k : List (Ty × Ty) → Outcome) (x : Nat) (t : Ty)
    (rest : List (Ty × Ty)) : Outcome :=
  if t = .var x then k rest                          -- Delete
  else if t.occurs x then .occurs x t                -- Occurs check
  else match k (substEqs x t rest) with              -- Eliminate
    | .unifier σ => .unifier ((x, t) :: σ)
```

```
    | failure => failure

/-- Unification of a list of equations, with `fuel` bounding the number of
steps (each equation processed costs one unit). -/
def unify : (fuel : Nat) → List (Ty × Ty) → Outcome
  | 0, _ => .outOfFuel
  | _ + 1, [] => .unifier []
  | fuel + 1, (a, b) :: rest =>
    match a, b with
    | .var x, t | t, .var x => solveVar (unify fuel) x t rest
    | .int, .int | .bool, .bool => unify fuel rest      -- Delete
    | .arrow a₁ b₁, .arrow a₂ b₂ =>                     -- Decompose
      unify fuel ((a₁, a₂) :: (b₁, b₂) :: rest)
    | .list a₁, .list a₂ => unify fuel ((a₁, a₂) :: rest)
    | a, b => .clash a b                                -- Clash
```

A unifier is a list of bindings in triangular form: `(x, t) :: rest` means "first $[x \mapsto t]$, then `rest`", and `Bindings.toSubst` composes them in that order. Lists are easy to print, functions easy to reason about, and `toSubst` connects the two. The or-pattern `| .var x, t | t, .var x =>` sends equations with a variable on either side to `solveVar`, which implements Delete, the occurs check and Eliminate; it receives the recursive call as an argument `k`, so that we can reason about the variable case once.

Each equation processed costs one unit of `fuel`, the device you saw in Chapter 7. Unification always terminates, but not for a reason Lean sees by itself: Decompose makes the list longer and Eliminate can make types bigger. The real argument uses a lexicographic measure (number of variables, then total size) and would double the length of the file; fuel lets us concentrate on what the algorithm computes.

Here it is on the five problems:

```
/-- Unifies two types, with plenty of fuel for small examples. -/
def unify₁ (a b : Ty) : Outcome := unify 100 [(a, b)]

section Examples
open Ty

/-- info: unifier: ?0 := Bool, ?1 := Int -/
#guard_msgs in
#eval unify₁ (var 0 ⇒ int) (bool ⇒ var 1)

/-- info: unifier: ?0 := Int, ?1 := Int -/
#guard_msgs in
#eval unify₁ (var 0 ⇒ var 0) (int ⇒ var 1)

/-- info: unifier: ?1 := List Int, ?0 := Int -/
#guard_msgs in
#eval unify₁ (list (var 0) ⇒ var 1) (var 1 ⇒ list int)

/-- info: clash: Int is not Bool -/
#guard_msgs in
#eval unify₁ (int ⇒ var 0) (bool ⇒ var 0)

/-- info: occurs check: ?0 occurs in ?0 → ?1 -/
#guard_msgs in
#eval unify₁ (var 0) (var 0 ⇒ var 1)

end Examples
```

The answers agree with the Rust trace (problem (3) lists its bindings in the order they were made, each with its final value). Fuel is a real limit: problem (1) needs four steps, so three are not enough:

```
/-- info: out of fuel -/
#guard_msgs in
#eval unify 3 [(.var 0 ⇒ .int, .bool ⇒ .var 1)]

/-- info: unifier: ?0 := Bool, ?1 := Int -/
#guard_msgs in
#eval unify 4 [(.var 0 ⇒ .int, .bool ⇒ .var 1)]
```

The occurs check is necessary

Is the occurs check a limitation of the algorithm, or a fact about types? A fact, and Lean can prove it: a solution would have to be bigger than itself. First, sizes:

```
/-- Whether the variable `x` occurs in a type. -/
def Ty.occurs (x : Nat) : Ty → Bool
  | .var n => n == x
  | .int | .bool => false
  | .arrow a b => a.occurs x || b.occurs x
  | .list a => a.occurs x

/-- The size of a type: the number of constructors in it. -/
def Ty.size : Ty → Nat
  | .arrow a b => a.size + b.size + 1
  | .list a => a.size + 1
  | _ => 1
```

The key lemma says that if `x` occurs in `t`, then whatever a substitution puts in place of `x` ends up somewhere inside the substituted `t`, so it cannot be bigger:

```
/-- If `x` occurs in `t`, then whatever `σ` puts in place of `x` ends up inside
`t.subst σ`, so it is no bigger. -/
theorem size_le_of_occurs (σ : Subst) (x : Nat) (t : Ty) (h : t.occurs x = true) :
    (σ x).size ≤ (t.subst σ).size := by
  induction t with
  | var n =>
    -- `n == x` is true, so `n = x` and both sides are `(σ x).size`.
    simp [Ty.occurs] at h
    simp [Ty.subst, h]
  | int => simp [Ty.occurs] at h
  | bool => simp [Ty.occurs] at h
  | arrow a b iha ihb =>
    -- `x` occurs in `a` or in `b`; either way `σ x` is inside the image.
    simp only [Ty.occurs, Bool.or_eq_true] at h
    simp only [Ty.subst, Ty.size]
    rcases h with ha | hb
    · have := iha ha; omega
    · have := ihb hb; omega
  | list a ih =>
    simp only [Ty.occurs] at h
    simp only [Ty.subst, Ty.size]
    have := ih h; omega

/-- The occurs check is not a limitation of the algorithm: if `x` occurs in `t`
and `t` is not `x` itself, then *no* substitution makes them equal, because
the right-hand side is always strictly bigger. -/
theorem no_unifier_of_occurs (x : Nat) (t : Ty) (h : t.occurs x = true)
    (hne : t ≠ .var x) (σ : Subst) : (Ty.var x).subst σ ≠ t.subst σ := by
  intro heq
  have hsize := congrArg Ty.size heq
  simp only [Ty.subst] at hsize
  cases t with
  | var n =>
    simp [Ty.occurs] at h
    exact hne (by rw [h])
  | int => simp [Ty.occurs] at h
  | bool => simp [Ty.occurs] at h
```

```
  | arrow a b =>
    simp only [Ty.occurs, Bool.or_eq_true] at h
    simp only [Ty.subst, Ty.size] at hsize
    rcases h with ha | hb
    · have := size_le_of_occurs σ x a ha; omega
    · have := size_le_of_occurs σ x b hb; omega
  | list a =>
    simp only [Ty.occurs] at h
    simp only [Ty.subst, Ty.size] at hsize
    have := size_le_of_occurs σ x a h; omega
```

In `size_le_of_occurs`, the `var n` case turns `h` into `n = x`, and both sides are the size of `σ x`. In the `int` and `bool` cases `x` cannot occur, and `simp` turns `h` into `False`. In the `arrow` case `h` says that `x` occurs in `a` *or* in `b` (`Bool.or_eq_true` turns `||` into `∨`), `rcases` splits the two possibilities, and in each the induction hypothesis plus "an arrow is bigger than its parts" is linear arithmetic, which `omega` decides.

`no_unifier_of_occurs` argues by contradiction: taking sizes of both sides of `heq` (`congrArg Ty.size heq`) gives an equation between numbers. `t` cannot be a variable (the only one containing `x` is `x`, excluded by `hne`), nor `Int` or `Bool`. So it is an arrow or a list, whose substituted size is *strictly* bigger than `σ x`, and `omega` finds the contradiction.

Instantiated at the equation of $\lambda x.\, x\, x$, this is a two-line theorem:

```
/-- Typing `fun x -> x x` needs a solution of `?0 = ?0 → ?1`. There is none. -/
theorem self_application_untypable (σ : Subst) :
    (Ty.var 0).subst σ ≠ (.var 0 ⇒ .var 1).subst σ :=
  no_unifier_of_occurs 0 (.var 0 ⇒ .var 1) rfl (by simp) σ
```

(`rfl` proves that `?0` occurs in `?0 ⇒ ?1` by computation, and `by simp` that `?0 ⇒ ?1` is not the variable `?0`.)

The returned unifier unifies

Now the main theorem of the file: whatever `unify` returns as a unifier really is one. First, what "unifies" means for a list of equations:

```
/-- `σ` unifies a list of equations when it makes the two sides of each one
equal. -/
def Unifies (σ : Subst) (eqs : List (Ty × Ty)) : Prop :=
  ∀ e ∈ eqs, e.1.subst σ = e.2.subst σ

theorem unifies_nil (σ : Subst) : Unifies σ [] := by
  intro e he
  cases he

theorem unifies_cons {σ : Subst} {a b : Ty} {eqs : List (Ty × Ty)} :
    Unifies σ ((a, b) :: eqs) ↔ a.subst σ = b.subst σ ∧ Unifies σ eqs := by
  simp [Unifies]
```

`unifies_cons` says that a substitution unifies a list whose first equation is $a \doteq b$ exactly when it equates a and b and unifies the rest; `simp` proves it from the definition of membership in a list. Two more lemmas correspond to two sentences of the informal argument:

```
/-- A substitution that does not mention `x` leaves `t` alone. -/
theorem subst_single_of_not_occurs (x : Nat) (s t : Ty) (h : t.occurs x = false) :
    t.subst (.single x s) = t := by
```

```
  induction t with
  | var n =>
    simp [Ty.occurs] at h
    simp [Ty.subst, Subst.single, h]
  | int => rfl
  | bool => rfl
  | arrow a b iha ihb =>
    simp only [Ty.occurs, Bool.or_eq_false_iff] at h
    simp [Ty.subst, iha h.1, ihb h.2]
  | list a ih =>
    simp only [Ty.occurs] at h
    simp [Ty.subst, ih h]

/-- Applying the bindings `(x, s) :: σ` is applying `[x := s]`, then `σ`. -/
theorem toSubst_cons (x : Nat) (s : Ty) (σ : Bindings) (t : Ty) :
    t.subst (Bindings.toSubst ((x, s) :: σ)) =
      (t.subst (.single x s)).subst σ.toSubst := by
  rw [Bindings.toSubst, subst_comp]
```

The first says that $[x \mapsto s]$ does not change a type in which x does not occur: this is where the occurs check pays off (in the `arrow` case, `Bool.or_eq_false_iff` splits "`a.occurs x || b.occurs x` is false" into two halves, `h.1` and `h.2`). The second says that the bindings `(x, s) :: σ` apply $[x \mapsto s]$ and then `σ`; it is `subst_comp`, which is why we proved that first.

The heart of the proof is the variable case. Its hypothesis `hk` says that the recursive call `k` is sound, and the conclusion is that `solveVar` is:

```
/-- The variable case is sound, provided the recursive call `k` is. -/
theorem solveVar_sound {k : List (Ty × Ty) → Outcome}
    (hk : ∀ eqs σ, k eqs = .unifier σ → Unifies σ.toSubst eqs)
    {x : Nat} {t : Ty} {rest : List (Ty × Ty)} {σ : Bindings}
    (h : solveVar k x t rest = .unifier σ) :
    Unifies σ.toSubst ((.var x, t) :: rest) := by
  unfold solveVar at h
  split at h
  · -- Delete: `t` is the variable `x` itself.
    rename_i ht
    subst ht
    exact unifies_cons.mpr ⟨rfl, hk rest σ h⟩
  · split at h
    · -- The occurs check failed: `h` says `.occurs x t = .unifier σ`.
      cases h
    · -- Eliminate: `x` does not occur in `t`, and `k` solved the rest.
      rename_i hne hocc
      split at h
      · rename_i σ' hσ'
        -- `h` says that `σ` is `(x, t) :: σ'`.
        cases h
        have hrest := hk _ _ hσ'
        have hfix : t.subst (.single x t) = t :=
          subst_single_of_not_occurs x t t (by simpa using hocc)
        refine unifies_cons.mpr ⟨?_, ?_⟩
        · -- The equation `?x = t` itself: both sides become `t.subst σ'`.
          rw [toSubst_cons, toSubst_cons, hfix]
          simp [Ty.subst, Subst.single]
        · -- The other equations: this is what `k` solved.
          intro e he
          rw [toSubst_cons, toSubst_cons]
          exact hrest _ (List.mem_map_of_mem he)
      · -- `k` failed, so `h` contradicts the hypothesis of this branch.
        rename_i hfail
        exact absurd h (hfail σ)
```

`unfold solveVar at h` exposes the `if`s inside `h`, and `split at h` makes one case per branch of the first `if` or `match` in `h`, adding the branch condition as an anonymous hypothesis (`rename_i` names it).

- If `t` is `x` itself, `subst ht` replaces `t` by `.var x`; the first equation holds by `rfl` and the rest by `k`.
- If the occurs check fails, `h` claims that `.occurs x t` equals `.unifier σ`; different constructors are never equal, and `cases h` closes the goal.
- In the Eliminate case, `k` returned some `σ'` and `cases h` replaces `σ` by `(x, t) :: σ'`. After `toSubst_cons`, the equation $x \doteq t$ has `t` on both sides before `σ'` is applied: on the left by definition of `single`, on the right by `hfix`, an instance of the first lemma. Each remaining equation is exactly a substituted equation that `k` solved (`List.mem_map_of_mem` finds it in the substituted list).
- If `k` failed, `h` contradicts the branch hypothesis `hfail`, and `absurd` concludes.

The main theorem is an induction on the fuel:

```
/-- Swapping the two sides of the first equation does not change what unifies. -/
theorem unifies_swap {σ : Subst} {a b : Ty} {eqs : List (Ty × Ty)}
    (h : Unifies σ ((a, b) :: eqs)) : Unifies σ ((b, a) :: eqs) := by
  rw [unifies_cons] at h ⊢
  exact ⟨h.1.symm, h.2⟩

/-- **Soundness of unification**: when `unify` returns a unifier, it really
unifies every equation it was given. -/
theorem unify_sound : ∀ (fuel : Nat) (eqs : List (Ty × Ty)) (σ : Bindings),
    unify fuel eqs = .unifier σ → Unifies σ.toSubst eqs := by
  intro fuel
  induction fuel with
  | zero =>
    intro eqs σ h
    simp [unify] at h
  | succ fuel ih =>
    intro eqs σ h
    match eqs with
    | [] => exact unifies_nil _
    | (a, b) :: rest =>
      -- One case for each pair of constructors: 25 cases. Unfolding one
      -- step of `unify` leaves `h` in one of four shapes.
      cases a <;> cases b <;> simp only [unify] at h
      -- (1) A clash: `h : .clash _ _ = .unifier σ` is impossible.
      all_goals try (cases h; done)
      -- (2) An equation `?x = t` or `t = ?x`: the variable case.
      all_goals try exact solveVar_sound ih h
      all_goals try exact unifies_swap (solveVar_sound ih h)
      -- (3) `Int = Int` and `Bool = Bool`: deleted; `ih` covers the rest.
      all_goals try exact unifies_cons.mpr ⟨rfl, ih _ _ h⟩
      -- (4) Decompose: two arrows, or two lists.
      case arrow.arrow a₁ b₁ a₂ b₂ =>
        have hs := ih _ _ h
        rw [unifies_cons, unifies_cons] at hs
        obtain ⟨ha, hb, hrest⟩ := hs
        exact unifies_cons.mpr ⟨by simp [Ty.subst, ha, hb], hrest⟩
      case list.list a₁ a₂ =>
        have hs := ih _ _ h
        rw [unifies_cons] at hs
        obtain ⟨ha, hrest⟩ := hs
        exact unifies_cons.mpr ⟨by simp [Ty.subst, ha], hrest⟩
```

With no fuel, `h` is impossible. With fuel, the empty list is unified by anything, and a non-empty list is split by the constructors of its first equation: `cases a <;> cases b` produces all $5 \times 5 = 25$ combinations (`<;>` runs the second tactic on every goal the first produces), and `simp only [unify] at h` unfolds one step of `unify` in each. Then `h` has one of four shapes, and `all_goals try t` applies a tactic wherever it succeeds: a clash (impossible); a variable on either side (`solveVar_sound`, with `unifies_swap` for the right side); `Int ≐ Int` or `Bool ≐ Bool` (`rfl` and `ih`); and two arrows or two

lists, selected by name with `case`, where the induction hypothesis unifies the parts and `simp` rebuilds the whole.

This is *soundness*: the algorithm never lies. *Completeness* (if a unifier exists, enough fuel finds a most general one) is the harder half, needing the termination measure we avoided. With `no_unifier_of_occurs`, we can already trust both a returned unifier and an occurs-check failure.

13.6 Hindley–Milner: polymorphism without annotations

A function is useful because it is used many times, and different uses may need different types. Chapter 12's System F gives such a function a type like $\forall\alpha.\,\alpha \to \alpha$, but type inference for System F is undecidable (Joe Wells proved it in the 1990s). Hindley and Milner found a restriction that makes polymorphism inferable, and it is surprisingly mild.

Monotypes and type schemes

The types τ of the previous sections, possibly containing variables, are called **monotypes**. A **type scheme** puts quantifiers in front of a monotype, and only there:

$$\sigma ::= \tau \mid \forall\alpha.\,\sigma$$

So $\forall\alpha.\,\alpha \to \alpha$ is a scheme, but $(\forall\alpha.\,\alpha \to \alpha) \to \texttt{Int}$ is not anything at all: a quantifier may not appear inside an arrow. In the vocabulary of Chapter 12, Hindley–Milner is **rank-1** polymorphism, the same restriction that Rust imposes on the types of values. A context Γ now binds variables to schemes; a monotype counts as a scheme with no quantifiers.

Two operations connect monotypes and schemes. A monotype τ is an **instance** of a scheme, written $\tau \sqsubseteq \forall\alpha_1 \ldots \alpha_n.\,\tau'$, when $\tau = [\alpha_1 \mapsto \tau_1, \ldots, \alpha_n \mapsto \tau_n]\,\tau'$ for some monotypes τ_i: you get an instance by choosing a type for each quantified variable. In the other direction, **generalization** quantifies every variable of a type that the context does not constrain:

$$\mathrm{Gen}(\Gamma, \tau) = \forall\alpha_1 \ldots \alpha_n.\,\tau \quad \text{where} \quad \{\alpha_1, \ldots, \alpha_n\} = \mathrm{ftv}(\tau) \setminus \mathrm{ftv}(\Gamma)$$

Here ftv means "free type variables": in Γ, those that occur in its schemes without being quantified.

The rules

The typing rules of Hindley–Milner (often abbreviated HM) for the core of mini-ML are those of the simply typed λ-calculus, with two changes. Variables are instantiated, and `let` generalizes:

$$\frac{(x : \sigma) \in \Gamma \qquad \tau \sqsubseteq \sigma}{\Gamma \vdash x : \tau}\ \textsf{T-Var} \qquad\qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.\, e : \tau_1 \to \tau_2}\ \textsf{T-Abs}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\, e_2 : \tau_2}\ \textsf{T-App}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \mathrm{Gen}(\Gamma, \tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}\ \textsf{T-Let}$$

The rules for literals (T-Int, T-Bool), `if` (T-If, as in Chapter 1) and pairs are the expected ones.

Read T-Var and T-Let together. A `let`-bound variable receives a scheme, the most general one its definition allows, and each *use* of the variable picks its own instance. A λ-bound variable, in T-Abs, receives a plain monotype τ_1, whose only instance is itself: every use of it has the same type. That asymmetry is the whole of let-polymorphism.

Same computation, different typing

Here is the asymmetry on a concrete program. With `let`:

$$\texttt{let}\ id = \lambda x.\, x\ \texttt{in}\ (id\ 1, id\ \texttt{true})$$

The definition $\lambda x.\, x$ has type $\alpha \to \alpha$ in the empty context, so T-Let binds $id : \forall \alpha.\, \alpha \to \alpha$, and each use instantiates the scheme differently. With $\Gamma' = id : \forall \alpha.\, \alpha \to \alpha$, the derivation for the first component is

$$\dfrac{\dfrac{(id : \forall \alpha.\, \alpha \to \alpha) \in \Gamma' \qquad \texttt{Int} \to \texttt{Int} \sqsubseteq \forall \alpha.\, \alpha \to \alpha}{\Gamma' \vdash id : \texttt{Int} \to \texttt{Int}}\ \textsf{T-Var} \qquad \dfrac{}{\Gamma' \vdash 1 : \texttt{Int}}\ \textsf{T-Int}}{\Gamma' \vdash id\ 1 : \texttt{Int}}\ \textsf{T-App}$$

and the second is the same with `Bool` and `true`, so the program has type $\texttt{Int} \times \texttt{Bool}$. Now write the same computation with a λ:

$$(\lambda id.\, (id\ 1, id\ \texttt{true}))\ (\lambda x.\, x)$$

One β-step turns it into the same pair as the `let` version. But by T-Abs, id has a single monotype τ_1; the first use requires $\tau_1 = \texttt{Int} \to \tau$, the second $\tau_1 = \texttt{Bool} \to \tau'$, and `Int` is not `Bool`. There is no derivation: `let x = e1 in e2` and `(fun x -> e2) e1` are equal as computations and different as typing problems.

Why not give λ-bound variables schemes too? Because a polymorphic argument is rank-2 polymorphism, and inference would become undecidable again. A `let` is special because the definition is right there: the checker can compute its most general type. A λ-bound variable stands for an argument that comes from elsewhere, and nothing says how polymorphic it will be.

Why Gen looks at the context

The side condition in Gen, "variables not free in Γ", is easy to read past, and essential. Consider

$$\lambda x.\, \texttt{let}\ y = x\ \texttt{in}\ (y\ 1, y\ \texttt{true})$$

Inside the λ, $x : \alpha$, so the definition of y has type α. If we generalized it to $y : \forall \alpha.\, \alpha$, both uses would type-check. But y *is* x, one value supplied by the caller: pass `fun n -> n + 1` and the program evaluates `true + 1`, a stuck term. Because α is free in the context, Gen leaves it alone and the program is rejected. Free in Γ means "someone else still gets to decide what this type is".

The asymmetry, proved in Lean

The file `code/lean/Book/Ch13/Polymorphism.lean` writes the rules as an inductive relation over the types of `Unify.lean`, which it imports. Our Lean types have no pairs, so `id` serves two types in an `if`: `if id true then id 1 else 2`. An instance may change the bound variables of a scheme and nothing else:

```
/-- A fragment of mini-ML: enough for the chapter's example. -/
inductive Expr where
```

```
  | var : String → Expr
  | lit : Nat → Expr
  | tt : Expr
  | lam : String → Expr → Expr
  | app : Expr → Expr → Expr
  | let_ : String → Expr → Expr → Expr
  | ite : Expr → Expr → Expr → Expr

/-- A type scheme `∀ vars. ty`. -/
structure Scheme where
  vars : List Nat
  ty : Ty

/-- A context binds names to schemes; the most recent binding comes first. -/
abbrev Ctx := List (String × Scheme)

def Ctx.lookup : Ctx → String → Option Scheme
  | [], _ => none
  | (y, σ) :: Γ, x => if x = y then some σ else Ctx.lookup Γ x

/-- The type variable `v` is free in the context: some scheme mentions it
without binding it. -/
def Ctx.Free (Γ : Ctx) (v : Nat) : Prop :=
  ∃ x σ, (x, σ) ∈ Γ ∧ σ.ty.occurs v = true ∧ v ∉ σ.vars

/-- `τ` is an instance of `∀ vars. ty`: substitute for the bound variables
(and only for them). -/
def Scheme.Inst (σ : Scheme) (τ : Ty) : Prop :=
  ∃ s : Subst, (∀ n, n ∉ σ.vars → s n = .var n) ∧ τ = σ.ty.subst s
```

The side condition of Gen is a premise of `let_`: the quantified variables `vs` must not be free in the context (a derivation may quantify fewer than Gen would; nothing is lost).

```
/-- The Hindley–Milner typing rules. -/
inductive HasType : Ctx → Expr → Ty → Prop
  /-- Var: instantiate the scheme of `x`. -/
  | var {Γ x σ τ} :
      Γ.lookup x = some σ → σ.Inst τ → HasType Γ (.var x) τ
  | lit {Γ n} : HasType Γ (.lit n) .int
  | tt {Γ} : HasType Γ .tt .bool
  /-- Abs: the parameter gets a monotype (a scheme with no bound variables). -/
  | lam {Γ x e τ₁ τ₂} :
      HasType ((x, ⟨[], τ₁⟩) :: Γ) e τ₂ → HasType Γ (.lam x e) (τ₁ ⇒ τ₂)
  | app {Γ f a τ₁ τ₂} :
      HasType Γ f (τ₁ ⇒ τ₂) → HasType Γ a τ₁ → HasType Γ (.app f a) τ₂
  /-- Let: generalize over variables that are not free in the context. -/
  | let_ {Γ x e₁ e₂ vs τ₁ τ₂} :
      HasType Γ e₁ τ₁ → (∀ v ∈ vs, ¬ Γ.Free v) →
      HasType ((x, ⟨vs, τ₁⟩) :: Γ) e₂ τ₂ → HasType Γ (.let_ x e₁ e₂) τ₂
  | ite {Γ c a b τ} :
      HasType Γ c .bool → HasType Γ a τ → HasType Γ b τ →
      HasType Γ (.ite c a b) τ
```

```
/-- `let id = fun x -> x in if id true then id 1 else 2` -/
def withLet : Expr :=
  .let_ "id" (.lam "x" (.var "x"))
    (.ite (.app (.var "id") .tt) (.app (.var "id") (.lit 1)) (.lit 2))

/-- `(fun id -> if id true then id 1 else 2)`, the same body under `fun`. -/
def withFun : Expr :=
  .lam "id" (.ite (.app (.var "id") .tt) (.app (.var "id") (.lit 1)) (.lit 2))
```

The `let` version has a derivation, which we build with tactics:

```
/-- The scheme `∀ ?0. ?0 → ?0` has `τ → τ` as an instance, for every `τ`. -/
theorem id_inst (τ : Ty) : (Scheme.mk [0] (.var 0 ⇒ .var 0)).Inst (τ ⇒ τ) := by
```

```
  refine ⟨Subst.single 0 τ, ?_, rfl⟩
  -- Every variable other than `?0` is left alone by `[?0 := τ]`.
  intro n hn
  simp only [List.mem_singleton] at hn
  simp [Subst.single, hn]

/-- With `let`, the program has type `Int`: `id` is used at `Bool → Bool`
and at `Int → Int`. -/
theorem withLet_typed : HasType [] withLet .int := by
  apply HasType.let_ (vs := [0]) (τ₁ := .var 0 ⇒ .var 0)
  · -- `fun x => x : ?0 → ?0`
    apply HasType.lam
    apply HasType.var rfl
    exact ⟨Subst.id, fun _ _ => rfl, (subst_id _).symm⟩
  · -- The empty context has no free variables, so `?0` may be generalized.
    intro v _ ⟨x, σ, hmem, _⟩
    cases hmem
  · -- The body, with `id : ∀ ?0. ?0 → ?0`.
    apply HasType.ite
    · exact HasType.app (HasType.var rfl (id_inst .bool)) HasType.tt
    · exact HasType.app (HasType.var rfl (id_inst .int)) HasType.lit
    · exact HasType.lit
```

`id_inst` instantiates $\forall ?0.\, ?0 \to ?0$ with $[?0 \mapsto \tau]$: `refine` supplies the substitution and the equation (`rfl`) and leaves the goal that other variables are untouched. In `withLet_typed`, `apply HasType.let_` fixes the scheme and leaves one goal per premise: the definition (T-Abs, then T-Var with the identity substitution), the side condition (nothing is a member of the empty context, which `cases hmem` observes), and the body, with `id` instantiated at `Bool` and at `Int`.

The `fun` version has no derivation, and this time the proof shows that *no* derivation of *any* type can exist:

```
/-- A scheme with no bound variables has exactly one instance: itself. -/
theorem mono_inst {τ₁ τ : Ty} (h : (Scheme.mk [] τ₁).Inst τ) : τ = τ₁ := by
  obtain ⟨s, hs, rfl⟩ := h
  -- `s` changes no variable, so it is the identity substitution.
  have : s = Subst.id := funext fun n => hs n (List.not_mem_nil)
  rw [this, subst_id]

/-- With `fun`, the program has no type at all. -/
theorem withFun_untypable (τ : Ty) : ¬ HasType [] withFun τ := by
  intro h
  -- Invert the derivation, rule by rule, from the bottom up.
  cases h with
  | lam hbody =>
    -- `id : τ₁`, a monotype, and the body is an `if`.
    cases hbody with
    | ite hcond hthen _ =>
      -- `id true : Bool` forces `id : Bool → Bool`...
      cases hcond with
      | app hid htrue =>
        cases htrue
        cases hid with
        | var hlook hinst =>
          simp [Ctx.lookup] at hlook
          subst hlook
          have h1 := mono_inst hinst
          -- ...and `id 1` forces `id : Int → τ`.
          cases hthen with
          | app hid' hone =>
            cases hone
            cases hid' with
            | var hlook' hinst' =>
              simp [Ctx.lookup] at hlook'
              subst hlook'
```

```
            have h2 := mono_inst hinst'
            -- `Bool → Bool = Int → τ` is impossible: `Bool ≠ Int`.
            rw [← h1] at h2
            cases h2
```

The proof is by **inversion**, as in Chapter 9: the last rule of a derivation must be the one whose conclusion matches the term, and `cases` on the derivation finds it and exposes its premises. The term is a λ, so the rule is `lam`, which binds `id` to a monotype `⟨[], τ₁⟩`; the body is an `if`, so next comes `ite`. In the condition `id true`, `cases htrue` finds that `true : Bool`, so `id` is used at `Bool ⇒ Bool`, an instance of its scheme (`simp [Ctx.lookup]` looks it up). `mono_inst` says that a scheme with no bound variables has only itself as an instance (the substitution changes no variable, so `funext` shows it is `Subst.id`), so `h1 : Bool ⇒ Bool = τ₁`. The `then` branch gives `h2 : Int ⇒ τ = τ₁`. Rewriting with `h1` turns `h2` into `Int ⇒ τ = Bool ⇒ Bool`, and `cases h2` observes that `Int` and `Bool` are different constructors. No derivation; no type.

13.7 Algorithm W

The rules do not say how to find the types: T-Abs needs a τ_1 for the parameter and T-Var an instance, and nothing says which. **Algorithm W**, from Milner's 1978 paper, invents a fresh type variable whenever a rule needs an unknown type, and lets unification decide later. It returns a substitution and a type, $W(\Gamma, e) = (S, \tau)$:

- **Variable.** If $(x : \forall \alpha_1 \ldots \alpha_n . \tau) \in \Gamma$, return the identity substitution and $[\alpha_i \mapsto \beta_i]\,\tau$, with fresh variables β_i. Instantiation with fresh variables is the most general instance.
- **Abstraction** $\lambda x.\, e$. Pick a fresh β for the parameter. Let $(S, \tau) = W((\Gamma, x : \beta), e)$. Return $(S, S\beta \to \tau)$: the body's inference may have learned something about β.
- **Application** $e_1\ e_2$. Let $(S_1, \tau_1) = W(\Gamma, e_1)$ and $(S_2, \tau_2) = W(S_1\Gamma, e_2)$. Pick a fresh β for the result and unify: $S_3 = \mathrm{mgu}(S_2\tau_1, \tau_2 \to \beta)$. Return $(S_3 \circ S_2 \circ S_1, S_3\beta)$.
- **Let** `let` $x = e_1$ `in` e_2. Let $(S_1, \tau_1) = W(\Gamma, e_1)$, then $(S_2, \tau_2) = W((S_1\Gamma, x : \mathrm{Gen}(S_1\Gamma, \tau_1)), e_2)$. Return $(S_2 \circ S_1, \tau_2)$.

The composed substitutions are the knowledge accumulated so far, applied to the context before each recursive call. If any mgu fails, so does W. The theorem that makes W worth having is due to Damas and Milner (1982), building on Hindley's result for combinatory logic (1969):

- **Soundness.** If $W(\Gamma, e) = (S, \tau)$, then $S\Gamma \vdash e : \tau$.
- **Completeness.** If e has any type in any instance of Γ, then W succeeds, and that type is an instance of the generalized type that W returns.

For a closed program, W computes its **principal type**, or proves that it has none.

W by hand: `twice`

Let us run W on $\lambda f.\, \lambda x.\, f\,(f\,x)$, keeping a single growing substitution as the code will.

1. λf: $f : \alpha_0$. λx: $x : \alpha_1$.

2. The inner application $f\,x$: f has type α_0, x has type α_1, pick α_2 for the result. Unify $\alpha_0 \doteq \alpha_1 \to \alpha_2$: solved by $\alpha_0 \mapsto \alpha_1 \to \alpha_2$. So $f\,x : \alpha_2$.
3. The outer application $f\,(f\,x)$: f now has type $\alpha_1 \to \alpha_2$ (read through the substitution), the argument has type α_2, pick α_3. Unify $\alpha_1 \to \alpha_2 \doteq \alpha_2 \to \alpha_3$: decompose, $\alpha_1 \mapsto \alpha_2$, then $\alpha_2 \mapsto \alpha_3$.
4. The whole term has type $\alpha_0 \to \alpha_1 \to \alpha_3$, which the substitution turns into $(\alpha_3 \to \alpha_3) \to \alpha_3 \to \alpha_3$. Nothing is in the context, so everything generalizes: $\forall a.\,(a \to a) \to a \to a$.

The end of the output of `ch13_unify.rs` traces the crate's own run on `twice`, and on the `let` example, where you can see generalization and two instantiations with fresh variables:

```
algorithm W on: fun f x -> f (f x)
  fun f -> ..       f : ?0
  fun x -> ..       x : ?1
  (f x)             unify ?0 ~ ?1 -> ?2  =>  ?0 := ?1 -> ?2
  f (f x)           unify ?1 -> ?2 ~ ?2 -> ?3  =>  ?1 := ?2, ?2 := ?3
  (program)         result forall a. (a -> a) -> a -> a
principal type: forall a. (a -> a) -> a -> a

algorithm W on: let id = fun x -> x in (id 1, id true)
  fun x -> ..       x : ?0
  let id = ..       generalize id : forall a. a -> a
  id                instantiate forall a. a -> a as ?1 -> ?1
  id 1              unify ?1 -> ?1 ~ Int -> ?2  =>  ?1 := Int, ?2 := Int
  id                instantiate forall a. a -> a as ?3 -> ?3
  id true           unify ?3 -> ?3 ~ Bool -> ?4  =>  ?3 := Bool, ?4 := Bool
  (program)         result Int * Bool
principal type: Int * Bool
```

13.8 Algorithm W in Rust

The `hm` crate's inference engine is in `src/infer.rs`. Instead of returning and composing substitutions, it keeps one `Subst` table that only grows: "compose a new substitution" becomes "add bindings to the table". Milner's paper describes this variant too, as algorithm J; it computes the same types with less copying, and production compilers, rustc included, work this way.

The language and its prelude

The language is a small ML, with a parser (`src/parser.rs`) that reduces a friendlier surface syntax to these forms:

```rust
/// A region of the source text, as byte offsets `start..end`.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Span {
    pub start: usize,
    pub end: usize,
}

/// An expression together with the part of the source it came from, so that
/// error messages can point at it.
#[derive(Clone, Debug, PartialEq)]
pub struct Expr {
    pub kind: ExprKind,
    pub span: Span,
}

#[derive(Clone, Debug, PartialEq)]
pub enum ExprKind {
    Int(i64),
```

```
    Bool(bool),
    Var(String),
    Lam(String, Box<Expr>),
    App(Box<Expr>, Box<Expr>),
    Let(String, Box<Expr>, Box<Expr>),
    LetRec(String, Box<Expr>, Box<Expr>),
    If(Box<Expr>, Box<Expr>, Box<Expr>),
    Pair(Box<Expr>, Box<Expr>),
}
```

Every expression carries a `Span` of source text, for error messages. Operators and lists are not built into the checker: `a + b` is parsed as `(+) a b` and `[1, 2]` as `cons 1 (cons 2 nil)`, and the names get their types from a **prelude**, the initial environment:

```
impl<'src> Infer<'src> {
    /// A fresh inference run, with the prelude in the environment.
    pub fn new(source: &'src str) -> Infer<'src> {
        let mut infer = Infer {
            source,
            subst: Subst::new(),
            env: Vec::new(),
            trace: None,
        };
        let (int, bool) = (Type::int, Type::bool);
        let arith = || Type::fun(int(), Type::fun(int(), int()));
        let compare = || Type::fun(int(), Type::fun(int(), bool()));
        for op in ["+", "-", "*"] {
            infer.builtin(op, 0, |_| arith());
        }
        for op in ["==", "<"] {
            infer.builtin(op, 0, |_| compare());
        }
        infer.builtin("not", 0, |_| Type::fun(bool(), bool()));
        infer.builtin("fst", 2, |v| {
            Type::fun(Type::pair(v[0].clone(), v[1].clone()), v[0].clone())
        });
        infer.builtin("snd", 2, |v| {
            Type::fun(Type::pair(v[0].clone(), v[1].clone()), v[1].clone())
        });
        infer.builtin("nil", 1, |v| Type::list(v[0].clone()));
        infer.builtin("cons", 1, |v| {
            let list = Type::list(v[0].clone());
            Type::fun(v[0].clone(), Type::fun(list.clone(), list))
        });
        infer.builtin("head", 1, |v| {
            Type::fun(Type::list(v[0].clone()), v[0].clone())
        });
        infer.builtin("tail", 1, |v| {
            Type::fun(Type::list(v[0].clone()), Type::list(v[0].clone()))
        });
        infer.builtin("null", 1, |v| Type::fun(Type::list(v[0].clone()), bool()));
        infer
    }

    /// Adds a prelude function whose scheme quantifies over `n` variables.
    fn builtin(&mut self, name: &str, n: usize, ty: impl Fn(&[Type]) -> Type) {
        let vars: Vec<TyVar> = (0..n).map(|_| self.subst.fresh()).collect();
        let args: Vec<Type> = vars.iter().map(|v| Type::Var(*v)).collect();
        let scheme = Scheme {
            vars,
            ty: ty(&args),
        };
        self.env.push(Binding {
            name: name.to_string(),
            scheme,
            origin: Origin::Prelude,
        });
```

```
    }
}
```

`nil` gets $\forall a.\, \texttt{List}\ a$ and `cons` $\forall a.\, a \to \texttt{List}\ a \to \texttt{List}\ a$, so lists are polymorphic by rule T-Var. The operators are monomorphic: `==` compares integers only, because "for every a *that supports equality*" is not an HM type; type classes (Chapter 14) add exactly that.

Generalization and instantiation

```
impl Infer<'_> {
    /// The type variables free in the environment: those of every binding's
    /// type (after applying the substitution), minus the binding's own
    /// quantified variables.
    fn env_vars(&self) -> BTreeSet<TyVar> {
        let mut vars = BTreeSet::new();
        for binding in &self.env {
            for v in self.subst.apply(&binding.scheme.ty).vars() {
                if !binding.scheme.vars.contains(&v) {
                    vars.insert(v);
                }
            }
        }
        vars
    }

    /// Gen(Γ, τ) = forall (ftv(τ) \ ftv(Γ)). τ
    /// Also reports whether some variable was kept monomorphic because the
    /// environment mentions it.
    fn generalize(&self, ty: &Type) -> (Scheme, bool) {
        let ty = self.subst.apply(ty);
        let in_env = self.env_vars();
        let (vars, blocked): (Vec<TyVar>, Vec<TyVar>) =
            ty.vars().into_iter().partition(|v| !in_env.contains(v));
        (Scheme { vars, ty }, !blocked.is_empty())
    }

    /// Inst(forall a1 ... an. τ) = τ[a1 := ?1, ..., an := ?n], with fresh ?i.
    fn instantiate(&mut self, scheme: &Scheme) -> Type {
        let fresh: Vec<(TyVar, Type)> = scheme
            .vars
            .iter()
            .map(|v| (*v, Type::Var(self.subst.fresh())))
            .collect();
        rename(&scheme.ty, &fresh)
    }
}

fn rename(ty: &Type, map: &[(TyVar, Type)]) -> Type {
    match ty {
        Type::Var(v) => match map.iter().find(|(w, _)| w == v) {
            Some((_, replacement)) => replacement.clone(),
            None => ty.clone(),
        },
        Type::Con(con, args) => {
            Type::Con(*con, args.iter().map(|a| rename(a, map)).collect())
        }
    }
}
```

`generalize` is Gen transcribed, and also reports whether a variable was *blocked* by the environment, for the error messages. Scanning the whole environment at every `let` is fine for teaching; production checkers such as OCaml's avoid it by recording the `let` depth (*level*) at which each variable was created.

The cases of W

The main function has one arm per rule. Variables are instantiated:

```
// Rule Var: look the name up and instantiate its scheme.
ExprKind::Var(name) => {
    let Some(binding) = self.env.iter().rev().find(|b| &b.name == name)
    else {
        return Err(Diagnostic::error(format!(
            "unbound variable `{name}`"
        ))
        .primary(expr.span, "not found in this scope"));
    };
    let scheme = binding.scheme.clone();
    let ty = self.instantiate(&scheme);
    if !scheme.vars.is_empty() {
        self.log(
            expr,
            format!("instantiate {} as {ty}", scheme_raw(&scheme)),
        );
    }
    Ok(ty)
}
```

Abstractions bind the parameter to a fresh variable, as a scheme with no quantifiers (a monotype):

```
// Rule Abs: the parameter gets a fresh, monomorphic type.
ExprKind::Lam(param, body) => {
    let alpha = Type::Var(self.subst.fresh());
    self.log(expr, format!("{param} : {alpha}"));
    self.bind(param, Scheme::mono(alpha.clone()), Origin::Lambda);
    let body_ty = self.infer(body);
    self.env.pop();
    Ok(Type::fun(alpha, body_ty?))
}
```

The environment is a stack (`bind` pushes, `pop` ends the scope). Applications follow W exactly:

```
// Rule App: the function's type must unify with arg -> result.
ExprKind::App(func, arg) => {
    let func_ty = self.infer(func)?;
    let arg_ty = self.infer(arg)?;
    let result = Type::Var(self.subst.fresh());
    let wanted = Type::fun(arg_ty.clone(), result.clone());
    let before = (self.subst.apply(&func_ty), self.subst.apply(&arg_ty));
    let wanted_now = self.subst.apply(&wanted);
    let outcome = self.subst.unify(&func_ty, &wanted);
    self.log_unify(expr, &before.0, &wanted_now);
    match outcome {
        Ok(()) => Ok(result),
        Err(err) => {
            Err(self.application_error(expr, func, arg, before, err))
        }
    }
}
```

`before` saves the two types as currently known, so that an error message is built from them rather than from a half-updated table. `let` infers, generalizes, and checks the body:

```
// Rule Let: infer, generalize, then check the body.
ExprKind::Let(name, bound, body) => {
    let bound_ty = self.infer(bound)?;
    let (scheme, blocked) = self.generalize(&bound_ty);
    self.log(expr, format!("generalize {name} : {}", scheme_raw(&scheme)));
    self.bind(name, scheme, Origin::Let { blocked });
    let body_ty = self.infer(body);
```

```
    self.env.pop();
    body_ty
}
```

`let rec` goes beyond the rules above: inside its own definition the function is a fresh *monotype*, generalized only afterwards, for the body:

```
// Recursion: inside its definition the function is monomorphic.
ExprKind::LetRec(name, bound, body) => {
    let alpha = Type::Var(self.subst.fresh());
    self.bind(name, Scheme::mono(alpha.clone()), Origin::Recursive);
    let bound_ty = self.infer(bound);
    self.env.pop();
    let bound_ty = bound_ty?;
    self.unify_or(expr, &alpha, &bound_ty, || {
        Diagnostic::error("recursive function used at an inconsistent type")
            .primary(bound.span, "in this definition")
    })?;
    let (scheme, blocked) = self.generalize(&bound_ty);
    self.log(expr, format!("generalize {name} : {}", scheme_raw(&scheme)));
    self.bind(name, scheme, Origin::Let { blocked });
    let body_ty = self.infer(body);
    self.env.pop();
    body_ty
}
```

Running it

The example `crates/hm/examples/ch13_infer.rs` infers the principal types of a list of programs:

```
/// Sample programs, from the identity function to `map`.
const PROGRAMS: [&str; 9] = [
    "fun x -> x",
    "fun x y -> x",
    "fun f x -> f (f x)",
    "fun f g x -> f (g x)",
    "fun p -> (snd p, fst p)",
    "fun x -> x + 1",
    "let rec length xs =
       if null xs then 0 else 1 + length (tail xs)
     in length",
    "let rec map f xs =
       if null xs then [] else f (head xs) :: map f (tail xs)
     in map",
    "let rec map f xs =
       if null xs then [] else f (head xs) :: map f (tail xs)
     in map (fun n -> n < 3) [1, 2, 3]",
];
```

```
== principal types ==
fun x -> x
  : forall a. a -> a
fun x y -> x
  : forall a b. a -> b -> a
fun f x -> f (f x)
  : forall a. (a -> a) -> a -> a
fun f g x -> f (g x)
  : forall a b c. (a -> b) -> (c -> a) -> c -> b
fun p -> (snd p, fst p)
  : forall a b. a * b -> b * a
fun x -> x + 1
  : Int -> Int
let rec length xs =
  if null xs then 0 else 1 + length (tail xs)
in length
```

```
  : forall a. List a -> Int
let rec map f xs =
  if null xs then [] else f (head xs) :: map f (tail xs)
in map
  : forall a b. (a -> b) -> List a -> List b
let rec map f xs =
  if null xs then [] else f (head xs) :: map f (tail xs)
in map (fun n -> n < 3) [1, 2, 3]
  : List Bool
```

Each answer is a principal type. In composition, the arrows record which types must match (the result of `g` feeds `f`); `fun x -> x + 1` has no variables, because `+` forced `Int`; `length` works on lists of anything; and applying `map` instantiates its scheme at `Int` and `Bool`. The second half of the output runs this chapter's `let`-versus-`fun` programs:

```
/// The same computation written with `let` and with `fun`, and a `let` whose
/// variable cannot be generalized.
const POLYMORPHISM: [&str; 3] = [
    "let id = fun x -> x in (id 1, id true)",
    "(fun id -> (id 1, id true)) (fun x -> x)",
    "fun x -> let y = x in (y 1, y true)",
];
```

```
== let versus fun ==
let id = fun x -> x in (id 1, id true)
  : Int * Bool
(fun id -> (id 1, id true)) (fun x -> x)
  error: mismatched types
fun x -> let y = x in (y 1, y true)
  error: mismatched types
```

Error messages

When W fails, the user wrote no types, so the message cannot say "you declared `Int` but wrote `Bool`"; it must explain which uses conflicted. The crate's messages imitate rustc's: a headline, the culprit underlined, and notes. The example `crates/hm/examples/ch13_errors.rs` renders five:

```
/// Each program is ill-typed; the name says why.
const PROGRAMS: [(&str, &str); 5] = [
    ("self-application", "fun x -> x x"),
    ("fun-bound", "(fun id -> (id 1, id true)) (fun x -> x)"),
    ("not-generalized", "fun x -> let y = x in (y 1, y true)"),
    ("branches", "fun b -> if b then 1 else false"),
    (
        "polymorphic-recursion",
        "let rec f x = if true then 0 else f 1 + f true in f",
    ),
];
```

```
error: infinite type: `a` would have to equal `a -> b`
 --> self-application:1:10
  |
1 | fun x -> x x
  |          ^^^ in this application
  |
  = note: the function `x` has type `a`
  = note: its argument `x` has type `a`
  = note: no finite type satisfies `a = a -> b` (the occurs check)

error: mismatched types
 --> fun-bound:1:22
```

```
  |
1 | (fun id -> (id 1, id true)) (fun x -> x)
  |                  -- ^^^^ expected `Int`, found `Bool`
  |                  |
  |                  this function has type `Int -> a`
  |
  = note: `id` is a parameter of `fun`, so all its uses share one type
  = help: bind `id` with `let` to make it polymorphic

error: mismatched types
 --> not-generalized:1:31
  |
1 | fun x -> let y = x in (y 1, y true)
  |                           - ^^^^ expected `Int`, found `Bool`
  |                           |
  |                           this function has type `Int -> a`
  |
  = note: `y` was not generalized: its type variables occur in the environment

error: `if` and `else` have incompatible types
 --> branches:1:27
  |
1 | fun b -> if b then 1 else false
  |                   -      ^^^^^ expected `Int`, found `Bool`
  |                   |
  |                   expected because of this

error: mismatched types
 --> polymorphic-recursion:1:43
  |
1 | let rec f x = if true then 0 else f 1 + f true in f
  |                                         - ^^^^ expected `Int`, found `Bool`
  |                                         |
  |                                         this function has type `Int -> Int`
  |
  = note: `f` is used in its own definition, where it is not yet polymorphic
```

The first is the occurs check on $\lambda x.\, x\, x$, in this chapter's terms. The second is the λ-bound `id`, with the reason and the fix. The third is the side condition of Gen at work. The fourth is rule T-If, phrased as rustc phrases E0308 (compare Chapter 1). The fifth is the monomorphic recursion of `let rec`, discussed below. The notes come from the origin recorded with each binding and from the `blocked` flag of `generalize`:

```
/// When the function is a variable, says how it was bound: this is what
/// decides whether it may be used at several types.
fn explain_binding(&self, diag: Diagnostic, func: &Expr) -> Diagnostic {
    let ExprKind::Var(name) = &func.kind else {
        return diag;
    };
    let Some(binding) = self.env.iter().rev().find(|b| &b.name == name) else {
        return diag;
    };
    match binding.origin {
        Origin::Lambda => diag
            .note(format!(
                "`{name}` is a parameter of `fun`, so all its uses share one type"
            ))
            .help(format!("bind `{name}` with `let` to make it polymorphic")),
        Origin::Let { blocked: true } => diag.note(format!(
            "`{name}` was not generalized: its type variables occur in the \
             environment"
        )),
        Origin::Recursive => diag.note(format!(
            "`{name}` is used in its own definition, where it is not yet \
             polymorphic"
        )),
```

```
            Origin::Prelude | Origin::Let { blocked: false } => diag,
        }
    }
```

One more detail makes the messages readable: a single `Namer` per message shows unknown types as `a`, `b`, ... in order of appearance, instead of the raw `?17` of the table.

13.9 The limits of Hindley-Milner

HM sits at a sweet spot; just outside it lie the following.

Polymorphic arguments. `(fun id -> (id 1, id true))` is rejected although System F gives it the rank-2 type $(\forall \alpha.\, \alpha \to \alpha) \to \texttt{Int} \times \texttt{Bool}$. Inferring such types is undecidable; languages that support them (Haskell with extensions, Lean) require an annotation on the polymorphic parameter.

Polymorphic recursion. A `let rec` function is monomorphic in its own body, so the fifth error program is rejected. Given the annotation $f : \forall a.\, a \to \texttt{Int}$ it could be checked, but inferring such *polymorphic recursion* is known to be undecidable.

Overloading. HM cannot say "for all types a that support `+`". Type classes (Chapter 14) extend HM with exactly this constrained quantifier, and Rust's traits are the same idea.

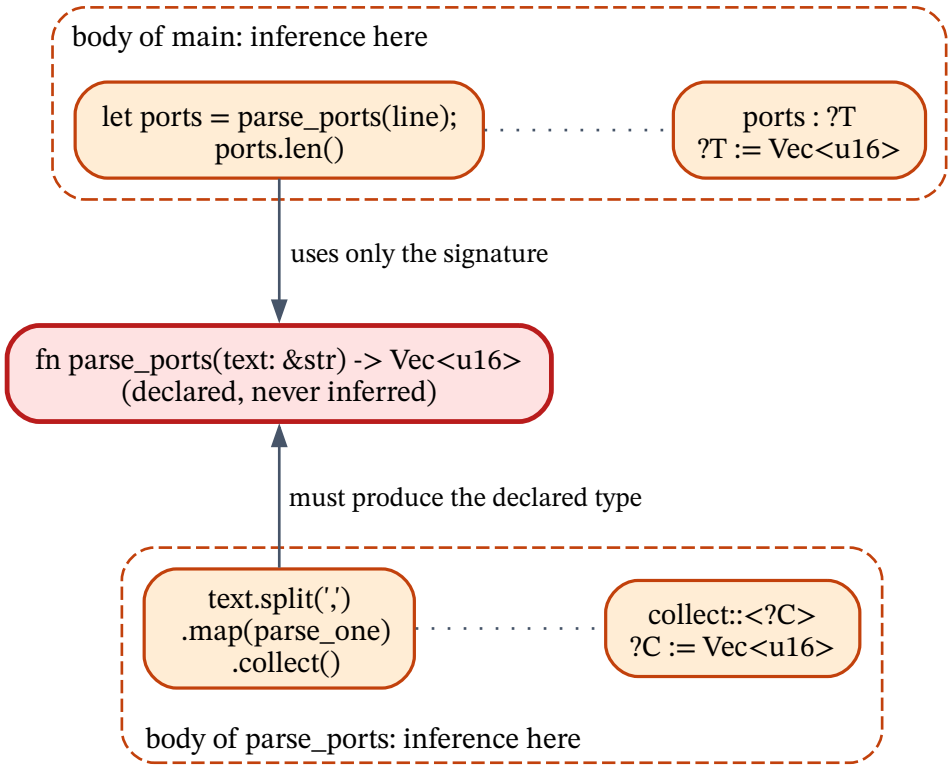
Effects. With mutable references, let-polymorphism is unsound: a reference of type $\forall \alpha.\, \texttt{Ref}\ (\texttt{List}\ \alpha)$ could be written at `Int` and read at `Bool`. ML languages add the *value restriction*: only syntactic values (such as a `fun`) are generalized.

Subtyping. With subtyping (Chapter 27), equations become inequations, and principal types become large sets of constraints.

Cost. A chain of `let`s that each duplicate the previous type makes types double in size at every step, so the worst case is exponential, and it was shown around 1990 that this is unavoidable. Real programs never do this, and inference is close to linear in practice.

13.10 Rust's type inference

Rust's compiler infers types by the method of this chapter: unknown types become inference variables, solved by unification in a table much like the crate's `Subst`. The differences from HM are deliberate, and each shows up as an error message you have probably met.



Rust infers types inside each function body separately. Bodies communicate only through declared signatures, which are never inferred.

Signatures are boundaries

ML infers the types of whole programs. Rust infers types only *inside* function bodies: every `fn` item states its full signature, and a placeholder there is an error even when the compiler could work out the answer (`code/rust/compile_fail/ch13_signature_placeholder.rs`):

```rust
fn double(x: i32) -> _ {
    x * 2
}
```

```
error[E0121]: the placeholder `_` is not allowed within types on item signatures for return
types
 --> compile_fail/ch13_signature_placeholder.rs:6:22
  |
6 | fn double(x: i32) -> _ {
  |                      ^ not allowed in type signatures
  |
help: replace with the correct return type
  |
6 - fn double(x: i32) -> _ {
6 + fn double(x: i32) -> i32 {
  |
```

The compiler suggests `i32` and refuses anyway, by design. A signature is a contract that callers rely on, and should not change because someone edited a body; errors stay local to the function that contains them; and Rust's types carry trait bounds and lifetimes, which would make whole-program inference

slow and its errors hard to explain. (The controlled exception is `impl Trait` in return position, Chapter 15.)

Generic items are schemes; closures are not generalized

A generic function *is* a type scheme: `fn identity<T>(x: T) -> T` declares $\forall T.\, T \to T$, and every call instantiates it with fresh inference variables, like rule T-Var (`code/rust/examples/ch13_rust_inference.rs` ; the turbofish of Chapter 12 instantiates explicitly):

```
/// A generic item: its type is a scheme, `forall T. T -> T`, instantiated
/// afresh at every call.
fn identity<T>(x: T) -> T {
    x
}
```

```
let n = identity(5);
let s = identity("five");
let v = identity::<Vec<u8>>(vec![]); // turbofish: explicit instantiation
```

A closure bound with `let` looks like a `let` -bound function, but it gets a single parameter type, fixed by its first use, like the λ-bound `id` . The compiler's message says so:

```
fn use_twice() {
    let identity = |x| x;
    let n = identity(5);
    let s = identity("five");
}
```

```
error[E0308]: mismatched types
 --> compile_fail/ch13_closure_not_generic.rs:9:22
  |
9 |     let s = identity("five");
  |             -------- ^^^^^^ expected integer, found `&str`
  |             |
  |             arguments to this function are incorrect
  |
note: expected because the closure was earlier called with an argument of type `{integer}`
 --> compile_fail/ch13_closure_not_generic.rs:8:22
  |
8 |     let n = identity(5);
  |             -------- ^ expected because this argument is of type `{integer}`
  |             |
  |             in this closure call
note: closure parameter defined here
 --> compile_fail/ch13_closure_not_generic.rs:7:21
  |
7 |     let identity = |x| x;
  |                     ^
```

This is not an oversight. A closure is a value of one anonymous type with one `Fn` signature, and Rust has no polymorphic values: there is no syntax for a closure generic over types. Rust's equivalent of `let` generalization is the generic item, with its signature written out.

Unknowns must be solved

In HM, a variable that nothing constrains is generalized: in mini-ML, `let entries = [] in null entries` has type `Bool` . Rust never generalizes inside a body, so by the end

of type checking every inference variable must be solved (every local needs a known type, possibly mentioning the function's own type parameters). When one is not, you get E0282:

```
fn count_entries() -> usize {
    let entries = Vec::new();
    entries.len()
}
```

```
error[E0282]: type annotations needed for `Vec<_>`
 --> compile_fail/ch13_annotations_needed.rs:7:9
  |
7 |     let entries = Vec::new();
  |         ^^^^^^^   ---------- type must be known at this point
  |
help: consider giving `entries` an explicit type, where the type for type parameter `T` is
specified
  |
7 |     let entries: Vec<T> = Vec::new();
  |                ++++++++
```

The program could run (an empty vector has length 0 whatever its element type), but nothing ever says what `?T` in `Vec<?T>` is.

Literals and information that flows backward

Integer literals get a special inference variable, printed `{integer}`, that only an integer type can solve. If nothing decides, it falls back to `i32` (`{float}` to `f64`). But anything later in the body may decide first:

```
let a = 42; // nothing constrains it: {integer} falls back to i32
let b = 2.5; // {float} falls back to f64
let c = 42; // an {integer} variable for now...
let small: u8 = c; // ...that this later use unifies with u8
let mut total = 0; // likewise, 0 is {integer}...
total += 3_u64; // ...until it meets a u64
```

```
let mut log = Vec::new(); // Vec<?T>: the element type is unknown here
log.push("started"); // ...and solved here: ?T = &str
log.push("stopped");

let add_one = |x| x + 1; // the parameter type is unknown here
let port = add_one(8079_u16); // ...and fixed by the first call: u16
```

```
let a = 42               : i32 = 42
let b = 2.5              : f64 = 2.5
let c = 42 (later u8)    : u8 = 42
small                    : u8 = 42
total (+= 3_u64)         : u64 = 3
log                      : Vec<&str> = ["started", "stopped"]
add_one(8079_u16)        : u16 = 8080
identity(5)              : i32 = 5
identity("five")         : &str = "five"
identity::<Vec<u8>>(..)  : Vec<u8> = []
```

`c` is a `u8` because of the *next* line, `log` learns its element type from `push`, and `add_one` its parameter type from its first call: unification does not care in which order it learns things.

Where order matters

Pure unification is order-independent, but rustc is not purely unification. To resolve a method call `x.m()` it must know, *at that point*, the type of `x`, because `i32::pow` and `u64::pow` are different functions and method lookup starts from the receiver's type:

```
fn area() -> u64 {
    let side = 12;
    let squared = side.pow(2);
    squared
}
```

```
error[E0689]: can't call method `pow` on ambiguous numeric type `{integer}`
 --> compile_fail/ch13_ambiguous_numeric.rs:8:24
  |
8 |     let squared = side.pow(2);
  |                        ^^^
  |
help: you must specify a type for this binding, like `i32`
  |
7 |     let side: i32 = 12;
  |             +++++
```

In HM, the return type `u64` would eventually flow back to `side`; rustc checks the body in order and gets stuck at `pow` first. (The suggestion is a heuristic: with `i32` the return would fail; `12_u64` is the right fix.) The same order dependence is why the closure `|x| x(x)` of Chapter 7 fails with E0282, "type annotations needed", before any occurs check could happen.

`collect` and `_` as a hole

`Iterator::collect` is polymorphic in its *result*: the requested type decides whether it builds a `Vec`, a `String` or a `Result<Vec<_>, _>`. If nothing asks, the compiler reports E0283, the cousin of E0282 for an unknown that must implement a trait:

```
fn parse_ports(text: &str) {
    let ports = text
        .split(',')
        .map(|p| p.trim().parse::<u16>().unwrap())
        .collect();
    println!("{ports:?}");
}
```

```
error[E0283]: type annotations needed
    --> compile_fail/ch13_collect_needs_type.rs:6:9
     |
   6 |     let ports = text
     |         ^^^^^
...
   9 |         .collect();
     |          ------- type must be known at this point
     |
     = note: the type must implement `FromIterator<u16>`
note: required by a bound in `collect`
    --> /rustc/b940084d7eb6a299eb4bfeb8e34901bc051e7ac4/library/core/src/iter/traits/
iterator.rs:2073:19
     |
2073 |     fn collect<B: FromIterator<Self::Item>>(self) -> B
     |                   ^^^^^^^^^^^^^^^^^^^^^^^^^^ required by this bound in
`Iterator::collect`
help: consider giving `ports` an explicit type
     |
```

```
   6 |     let ports: Vec<_> = text
     |              ++++++++
```

The fix is to say which collection, and only that:

```
let words = text.split(' ').collect::<Vec<_>>(); // Vec<&str>
let distinct: BTreeSet<_> = text.split(' ').collect(); // BTreeSet<&str>
let joined: String = text.split(' ').collect(); // String
let lengths: Vec<_> = words.iter().map(|w| w.len()).collect(); // Vec<usize>
let ports: Result<Vec<u16>, _> = "80,443".split(',').map(str::parse).collect();
let bad: Result<Vec<u16>, ParseIntError> =
    "80,x".split(',').map(str::parse).collect();
```

```
words                    : Vec<&str> = ["to", "be", "or", "not", "to", "be"]
distinct                 : BTreeSet<&str> = {"be", "not", "or", "to"}
joined                   : String = "tobeornottobe"
lengths                  : Vec<usize> = [2, 2, 2, 3, 2, 2]
ports                    : Result<Vec<u16>, ParseIntError> = Ok([80, 443])
bad                      : Result<Vec<u16>, ParseIntError>
                         = Err(ParseIntError { kind: InvalidDigit })
```

The `_` in `collect::<Vec<_>>()` is a hole: "a `Vec` of something; you work out what". It becomes a fresh inference variable, solved by unification with the item type. In the last two lines, one annotation decides both what `collect` builds and which type `str::parse` parses into. Choosing an implementation from a type is trait resolution, not HM: the subject of Chapter 14.

13.11 Lean's elaborator

Lean's kernel checks fully explicit terms, with every implicit argument, instance and coercion written out. The **elaborator** fills them in the way W does: every unknown becomes a **metavariable**, written `?m.5`, solved by unification. The examples are in `code/lean/Book/Ch13/Elaboration.lean`.

Metavariables at work

```
/-- info: fun x => x + 1 : Nat → Nat -/
#guard_msgs in
#check fun x => x + 1

/-- info: fun x => x + 1 : Int → Int -/
#guard_msgs in
#check fun (x : Int) => x + 1
```

In the first command `x`'s type is a metavariable, and `+` and `1` leave pending problems. When nothing decides, a **default instance** makes numerals natural numbers: the analogue of `{integer}` falling back to `i32`. The option `pp.explicit` shows what the elaborator built:

```
set_option pp.explicit true in
/-- info: @id Bool true : Bool -/
#guard_msgs in
#check id true
```

The implicit type argument of `id` (Chapter 12) was a metavariable, solved as `Bool` by unification. A metavariable still unsolved at the end is an error, as in Rust's E0282:

```
/--
error: Failed to infer type of binder `x`
---
error: Failed to infer type of definition `idFun`
-/
#guard_msgs in
def idFun := fun x => x
```

```
/--
error: don't know how to synthesize implicit argument `α`
  @List.nil ?m.3
context:
⊢ Type u_1
---
error: Failed to infer type of definition `emptyList`
-/
#guard_msgs in
def emptyList := []

def emptyNats : List Nat := []
```

The second is `Vec::new()` again: nothing determines the element type of `[]` (`⊢ Type u_1` says that the missing argument is a type). Lean also does not generalize local definitions: a `let`-bound function behaves like Rust's closure and like HM's λ-bound variable:

```
/--
error: Application type mismatch: The argument
  true
has type
  Bool
but is expected to have type
  Nat
in the application
  f true
-/
#guard_msgs in
example : Nat × Bool :=
  let f := fun x => x
  (f 1, f true)

/-- With an explicit polymorphic type, the local function is fine. -/
example : Nat × Bool :=
  let f : {α : Type} → α → α := fun x => x
  (f 1, f true)
```

With a polymorphic annotation the `let` works: Lean can *check* a polymorphic local function, but will not *infer* one. Top-level definitions are polymorphic by declaration, like Rust items, except that Lean binds unknown type names automatically as implicit arguments:

```
/-- A polymorphic definition: `α` is bound automatically, as an implicit
argument. -/
def myId (x : α) : α := x

/-- info: @myId : {α : Sort u_1} → α → α -/
#guard_msgs in
#check @myId

example : Nat × Bool := (myId 1, myId true)
```

Order, holes and ascriptions

Lean, like rustc, elaborates mostly from left to right, and field notation needs the receiver's type *now*, like Rust's methods:

```
/--
error: Invalid field notation: Type of
  x
is not known; cannot resolve field `succ`

Hint: Consider replacing the field projection with a call to one of the following:
  • `Fin.succ`
  • `Nat.succ`
  • `Std.PRange.succ`
-/
#guard_msgs in
def succAll := fun x => x.succ

/-- info: fun x => x.succ : Nat → Nat -/
#guard_msgs in
#check (fun x => x.succ : Nat → Nat)
```

The fix is a **type ascription** `(e : τ)`, which elaborates `e` *against* `τ`, so `x : Nat` is known before `x.succ` is looked at. Ascriptions also decide how literals and operators are elaborated:

```
/-- info: -1 -/
#guard_msgs in
#eval (1 - 2 : Int)

/-- info: 0 -/
#guard_msgs in
#eval ((1 - 2 : Nat) : Int)

/-- info: ↑(1 - 2) : Int -/
#guard_msgs in
#check ((1 - 2 : Nat) : Int)
```

The second subtracts in `Nat`, where the result is truncated to 0, and then inserts a coercion `↑` to `Int`. Finally, a `_` is a hole for the elaborator to fill by unification, and a `?_` a hole for *you* to fill, as a new goal:

```
/--
error: don't know how to synthesize placeholder for argument `a`
context:
⊢ Nat
-/
#guard_msgs in
def sevens : List Nat := List.replicate 3 _

/-- `?_` is a hole too, but a *named goal*: tactics fill it in later. -/
def sevens' : List Nat := by
  refine List.replicate 3 ?_
  exact 7

/-- info: [7, 7, 7] -/
#guard_msgs in
#eval sevens'
```

Unification up to computation

Lean's types contain terms, so its unifier must sometimes *compute*: `?n + 0` and `5` unify, because addition recurses on its second argument and `?n + 0` reduces to `?n`. But `0 + ?n` is stuck on the unknown `?n`, and the unifier gives up:

```
/-- Unification computes: `?n + 0` reduces to `?n`, so `?n := 5`. -/
example : ∃ n : Nat, n + 0 = 5 := ⟨_, rfl⟩

/--
error: Application type mismatch: The argument
  rfl
has type
  ?m.15 = ?m.15
but is expected to have type
  0 + ?m.13 = 5
in the application
  Exists.intro ?m.13 rfl
-/
#guard_msgs in
example : ∃ n : Nat, 0 + n = 5 := ⟨_, rfl⟩
```

The `show` tactic replaces the goal by a type that unifies with it up to computation (*definitional equality*, Chapter 30), so it accepts `n = n` for `n + 0 = n` but not for `0 + n = n`:

```
example (n : Nat) : n + 0 = n := by
  show n = n      -- accepted: `n + 0` computes to `n`
  rfl

/--
error: 'show' tactic failed, pattern
  n = n
is not definitionally equal to target
  0 + n = n
-/
#guard_msgs in
example (n : Nat) : 0 + n = n := by
  show n = n
  rfl
```

Metavariables can also stand for *functions*, making unification **higher-order**, which is undecidable in general (results due to Huet and, for the second-order case, Goldfarb). Problems can have many incomparable solutions: `?f 3 ≟ 6` is solved by `fun x => x + 3`, `fun x => 2 * x`, `fun _ => 6` and more. Lean solves the fragment that Dale Miller identified, *patterns* (a metavariable applied to distinct bound variables), where the solution is unique, and otherwise prefers to fail rather than guess:

```
/-- A higher-order *pattern*: `?f x = x + 1` with `x` a bound variable has
exactly one solution, `?f := fun x => x + 1`. -/
example : ∃ f : Nat → Nat, ∀ x, f x = x + 1 := ⟨_, fun _ => rfl⟩

/--
error: Application type mismatch: The argument
  rfl
has type
  ?m.11 = ?m.11
but is expected to have type
  ?m.9 3 = 6
in the application
  Exists.intro ?m.9 rfl
-/
#guard_msgs in
example : ∃ f : Nat → Nat, f 3 = 6 := ⟨_, rfl⟩
```

The first example meets `?f x ≟ x + 1` with `x` bound, a pattern, solved by abstracting `x`; the second meets `?f 3 ≟ 6` and refuses.

13.12 Putting it together: `map` in three languages

Let us follow one function through the chapter. In mini-ML, nothing is annotated:

```
let rec map f xs =
  if null xs then [] else f (head xs) :: map f (tail xs)
in map
```

W proceeds as follows.

1. `let rec` binds `map : α` (a monotype, for now), and the definition binds `f : β` and `xs : γ`.
2. `null xs` : `null` is instantiated to `List` $\delta \to$ `Bool`, and unification solves $\gamma \mapsto$ `List` δ. The condition is a `Bool`, as T-If requires.
3. `[]` is `nil`, instantiated to `List` ϵ: the `then` branch.
4. `head xs` has type δ, so `f (head xs)` solves $\beta \mapsto \delta \to \zeta$ and has type ζ.
5. `map f (tail xs)` : `map` is still the monotype α, so applying it twice solves $\alpha \mapsto (\delta \to \zeta) \to$ `List` $\delta \to \eta$. Then `::` (`cons`, instantiated) needs $\eta =$ `List` ζ, and the `else` branch has type `List` ζ.
6. T-If unifies the branches: $\epsilon \mapsto \zeta$.
7. The definition has type $(\delta \to \zeta) \to$ `List` $\delta \to$ `List` ζ, which also unifies with α. Nothing is in the context, so it generalizes to $\forall a\, b.\, (a \to b) \to$ `List` $a \to$ `List` b, which is what the crate printed.

In Rust the signature is written and inference works inside the body; in Lean the signature is written too, with type variables bound automatically:

```
fn map<A, B>(f: impl Fn(A) -> B, xs: Vec<A>) -> Vec<B> {
    xs.into_iter().map(f).collect() // collect::<Vec<B>>, from the signature
}
```

```
def map (f : α → β) : List α → List β
  | [] => []
  | x :: xs => f x :: map f xs
```

(Both fragments are shown for comparison and are not compiled as part of the book.) The three languages put the line between "written" and "inferred" in different places:

	mini-ML (HM)	Rust	Lean
function signatures	inferred	written	written, type variables auto-bound
local `let` of a function	generalized	closure: one type	not generalized
unknown left at the end	generalized	error E0282	error
numeric literals	`Int` only	`{integer}`, defaults to `i32`	default instance `Nat`
unification	first-order	first-order, plus traits	up to computation, higher-order patterns

13.13 Common confusions

"**Inferred types are dynamic types.**" No. An HM program is as statically typed as a fully annotated one; inference only spares you from writing the types. Every expression still has a type before the program runs.

"**The principal type is the most specific type.**" It is the most *general* one: every other type of the program is an instance of it.

"**The occurs check is a limitation of the algorithm.**" It is a fact about types: `no_unifier_of_occurs` proves that no substitution solves $\alpha \doteq \tau$ when α occurs inside τ. Removing the check does not make more programs typable; it makes the inference engine loop or build cyclic types.

"`let x = e1 in e2` **is just** `(fun x -> e2) e1`." As computations, yes; as typing problems, no. That is the point of let-polymorphism, and it is proved in `withLet_typed` and `withFun_untypable`.

"**Rust has Hindley–Milner type inference.**" Rust has unification-based inference, with a table of variables as in algorithm J. But it is local to function bodies, never generalizes, defaults literals, needs some types at specific points (method calls), and interleaves trait resolution with unification. "HM-flavored local inference" is more accurate.

"`_` **means any type.**" In both Rust and Lean, `_` means *one specific* type that the compiler must work out. If it cannot, that is an error, not a wildcard.

13.14 Summary

- Type inference turns missing types into type variables and the program into equations, solved by unification. The answer is a principal type, of which every other valid type is an instance.
- Substitutions compose and form a monoid; a most general unifier commits to nothing the equations do not demand.
- Unification decomposes equal constructors, fails on a clash, and solves a variable unless it occurs in its would-be solution. The occurs check is why $\lambda x.\, x\, x$ has no type, and Lean proves no substitution could help.
- Hindley–Milner allows quantifiers only at the outside of a type, gives `let`-bound variables schemes and λ-bound variables monotypes, and never generalizes variables free in the context.
- Algorithm W (fresh variables plus unification) computes principal types; the `hm` crate implements it as algorithm J, with rustc-style error messages.
- HM cannot infer rank-2 types, polymorphic recursion or overloading, and needs the value restriction in the presence of effects.
- Rust infers inside function bodies only: signatures are boundaries (E0121), closures are not generalized (E0308), unknowns must be solved (E0282, E0283), `{integer}` defaults to `i32`, method calls need their receiver's type (E0689), and `_` is a hole.
- Lean's elaborator solves metavariables by unification up to computation and for higher-order patterns, defaults numerals to `Nat`, and does not generalize `let`. In Lean we proved unification sound, the occurs check necessary, and let-polymorphism strictly stronger than λ-binding.

13.15 Exercises

1. By hand, find a most general unifier of $(\alpha \to \beta) \to \texttt{List}\ \alpha$ and $(\texttt{Int} \to \gamma) \to \gamma$, or show there is none. Then do the same for $\alpha \to \beta$ and $\beta \to \alpha$. Check your answers with `unify₁` in Lean or with the `hm` crate's `Subst::unify`. *Hint:* in the first problem, solve γ twice and look at what the second equation for γ says about β and α.
2. Run algorithm W by hand on `fun f g x -> f x (g x)` and give its principal type. Then add it to `PROGRAMS` in `ch13_infer.rs` and compare. *Hint:* you need fresh variables for `f`, `g`, `x` and for the result of each of the three applications. The answer has three type variables.
3. In the `hm` crate, remove the occurs check from `unify` (the `if` that returns `UnifyError::Occurs`) and run `fun x -> x x`. What happens, and why? Restore the check afterwards. *Hint:* look at what `Subst::apply` does with the binding `?0 := ?0 -> ?1`.
4. In Lean, prove that a one-variable substitution is idempotent when the variable does not occur in its solution: `(Subst.single x t).comp (Subst.single x t) = Subst.single x t` whenever `t.occurs x = false`. *Hint:* `funext n`, unfold `Subst.comp` and `Subst.single`, split on whether `n = x` with `by_cases`, and use `subst_single_of_not_occurs` in the case `n = x`.
5. In a copy of `Polymorphism.lean`, delete the side condition `(∀ v ∈ vs, ¬ Γ.Free v)` from the `let_` rule, and prove that the program `fun x -> let y = x in if y true then y 1 else 2` now has a type. Explain why that is a disaster, using the program from the section on Gen. *Hint:* give `x` the monotype `?0` and `y` the scheme `∀ ?0. ?0`; then `y` has every type, including `Bool → Bool` and `Int → Int`.
6. Fix the two Rust programs `ch13_ambiguous_numeric.rs` and `ch13_closure_not_generic.rs` in as many different ways as you can (an annotation, a literal suffix, a generic `fn`, a fully qualified call such as `u64::pow(side, 2)`). For each fix, say which inference variable it solves, and when. *Hint:* for the closure, ask whether the fix gives one closure two types, or gives you two closures, or replaces the closure by a scheme.

CHAPTER 14

Traits, Type Classes and Dictionary Passing

A generic function in Rust promises to work for every type, and Chapter 12 showed what that promise costs: a function `fn f<T>(x: T) -> T` cannot look inside its argument, so it can do almost nothing with it. Most useful generic code needs to do *something* with its values: compare them to sort a list, add them up to compute a total, format them to write a log line. Rust's answer is the trait bound. `fn largest<T: PartialOrd>(items: &[T])` still works for many types, but no longer for all of them: only for those that come with a comparison.

That small `: PartialOrd` is the subject of this chapter. In type theory a trait is a **type class**, an idea that Haskell introduced at the end of the 1980s to make *ad-hoc polymorphism*, functions that behave differently at different types, as principled as the parametric kind. We will see how a type checker reasons about type classes (instance resolution turns out to be proof search, with a familiar Curry–Howard twist), how a compiler implements them (by passing *dictionaries* of functions, which Rust then optimizes away), why Rust insists that every instance be unique, and what a trait cannot say at all: that its methods obey laws. Lean, whose classes can carry proofs, will say it for us.

In this chapter you will learn:

- the difference between parametric and ad-hoc polymorphism, and how type classes make the second one principled;
- the typing rules for constrained types, and why instance resolution is proof search whose proofs are dictionaries;
- how a trait bound is elaborated into an extra argument, with the same Rust program written both ways;
- what coherence is, why Rust enforces it with the orphan rule and the ban on overlapping impls, and how Lean does without it;
- associated types versus type parameters, supertraits and default methods, in Rust and in Lean (`outParam`, `extends`);
- why laws matter, how Rust can only test them on samples, and how a Lean class makes every instance prove them;
- how Lean's classes work: structures, instance arguments, `inferInstance`, local instances and priorities, `Decidable`, `Inhabited`, and Mathlib's `Monoid`.

14.1 Two kinds of polymorphism

The word *polymorphism* means "many shapes": one piece of code that works at many types. Christopher Strachey, in lecture notes from 1967, distinguished two kinds, and his terms are still the standard ones.

Parametric polymorphism is the kind you studied in Chapter 12. A parametric function works *uniformly* at every type: the same code runs whatever the type is, and it cannot tell which type it got. `fn swap<A, B>((a, b): (A, B)) -> (B, A)` and `Vec::len` are parametric. In System F their types begin with $\forall$.

Ad-hoc polymorphism is one name with *different* code at different types. The `+` in `2 + 3` and the `+` in `2.5 + 0.5` are different machine instructions, and `+` on `String` allocates memory. `==`, `to_string` and `clone` are the same kind of thing: an operation that exists at many types, but that each type implements in its own way.

Ad-hoc polymorphism is older than its theory. Many languages resolve overloading at each call site: the compiler sees `x + y` with `x: f64` and picks the floating-point addition. That works for a single call but does not compose: you cannot write a function `sum_all` that works "for every type with a `+`", because inside `sum_all` the type is not known, and there is nothing to resolve the `+` against.

In 1989, Philip Wadler and Stephen Blott proposed a principled solution, in a paper with the memorable title "How to make ad-hoc polymorphism less ad hoc", and Haskell adopted it. The idea has three parts:

1. A **class** names a set of operations with their types: "a type α is a `Monoid` if it has an `empty` of type α and a `combine` of type $\alpha \to \alpha \to \alpha$".
2. An **instance** declares that a particular type belongs to the class, and gives the operations.
3. A polymorphic type may **quantify over the types in a class**: $\forall \alpha.\ \mathit{Monoid}\ \alpha \Rightarrow \mathrm{Vec}\ \alpha \to \alpha$, "for every type α that is a monoid, a function from vectors of α to α".

A Rust trait is a class, an `impl` is an instance, and a bound `M: Monoid` is the constraint $\mathit{Monoid}\ M \Rightarrow$. Chapter 12 described bounds as *bounded quantification*; this chapter looks at what a bound means and how it is implemented.

A monoid trait

Our running example is the **monoid**, the simplest algebraic structure that earns its keep in programs. A monoid is a type with a neutral element and an associative operation: numbers with `+` and `0`, numbers with `*` and `1`, strings with concatenation and `""`, vectors with `extend` and `vec![]`, and, as we will see at the end of the chapter, summary statistics with "merge". The full program is in `code/rust/examples/ch14_dictionary_passing.rs`.

```
/// A monoid: a type with a neutral element and an associative operation.
///
/// Laws (stated here in prose, because Rust has no way to state them in code):
/// - associativity: `a.combine(b).combine(c) == a.combine(b.combine(c))`
/// - identity: `M::empty().combine(a) == a == a.combine(M::empty())`
trait Monoid {
    fn empty() -> Self;
```

```
    fn combine(self, other: Self) -> Self;
}
```

Two details deserve attention. `empty` has no `self` parameter: it is not called *on* a value but *at* a type, as `M::empty()`. Interfaces in object-oriented languages cannot express this, since a method there is always dispatched on the object it is called on; type classes can, because the instance is chosen by a type, wherever that type comes from. And the doc comment states laws that the trait itself does not mention. We will come back to them.

Integers form a monoid in more than one way, and a type can implement a trait only once (the section on coherence explains why). So each monoid structure on `i64` gets its own *newtype*, a one-field struct whose only job is to select an instance:

```
/// `i64` has (at least) two monoid structures, addition and multiplication.
/// A type can implement a trait only once, so each structure gets a newtype.
#[derive(Clone, Copy, Debug, PartialEq)]
struct Sum(i64);

#[derive(Clone, Copy, Debug, PartialEq)]
struct Product(i64);

#[derive(Clone, Copy, Debug, PartialEq)]
struct Max(i64);
```

```
impl Monoid for Sum {
    fn empty() -> Self {
        Sum(0)
    }
    fn combine(self, other: Self) -> Self {
        Sum(self.0 + other.0)
    }
}

impl Monoid for Product {
    fn empty() -> Self {
        Product(1)
    }
    fn combine(self, other: Self) -> Self {
        Product(self.0 * other.0)
    }
}

impl Monoid for Max {
    fn empty() -> Self {
        Max(i64::MIN)
    }
    fn combine(self, other: Self) -> Self {
        Max(self.0.max(other.0))
    }
}

impl Monoid for String {
    fn empty() -> Self {
        String::new()
    }
    fn combine(mut self, other: Self) -> Self {
        self.push_str(&other);
        self
    }
}

impl<T> Monoid for Vec<T> {
    fn empty() -> Self {
```

```
        Vec::new()
    }
    fn combine(mut self, other: Self) -> Self {
        self.extend(other);
        self
    }
}
```

The instance for pairs is different from the others: it has a *context*. A pair is a monoid if both components are, and its operations are built from theirs:

```
/// An instance with a context: a pair is a monoid *if* both components are.
impl<A: Monoid, B: Monoid> Monoid for (A, B) {
    fn empty() -> Self {
        (A::empty(), B::empty())
    }
    fn combine(self, other: Self) -> Self {
        (self.0.combine(other.0), self.1.combine(other.1))
    }
}
```

Read its first line as a rule: "for all `A` and `B`, if `A: Monoid` and `B: Monoid`, then `(A, B): Monoid`". Inside the body, `A::empty()` and `self.0.combine(other.0)` use the instances for `A` and `B`, whatever they turn out to be.

Now the generic code:

```
/// Combine all the items, left to right. Works for every monoid `M`.
fn mconcat<M: Monoid>(items: Vec<M>) -> M {
    let mut acc = M::empty();
    for item in items {
        acc = acc.combine(item);
    }
    acc
}

/// Combine each batch, then combine the batch results. A generic function
/// that calls another generic function at the same type.
fn mconcat_batches<M: Monoid>(batches: Vec<Vec<M>>) -> M {
    let partials: Vec<M> = batches.into_iter().map(mconcat).collect();
    mconcat(partials)
}
```

Inside `mconcat`, `M` is as abstract as any type parameter in Chapter 12, with one difference: the bound gives the body two operations on `M`, `M::empty()` and `combine`. Nothing else is known about `M`. The function cannot print an `M` or compare two of them.

14.2 Typing rules for type classes

Let us write down rules for this, as we have for every other feature so far. As always, the rules are a *model* of what rustc does: rustc's trait solver handles much more (lifetimes, associated types, auto traits, negative reasoning), but the core is the following system, which follows Mark Jones's theory of *qualified types*.

Constraints and entailment

A **constraint** π is a statement about types, such as *Monoid* `Sum` or *Monoid* α. A type may be **qualified** by a constraint, $\pi \Rightarrow \tau$, and a polymorphic type may quantify over a qualified one. The type of `mconcat` is

$$mconcat : \forall \alpha.\ \mathit{Monoid}\ \alpha \Rightarrow \mathrm{Vec}\ \alpha \to \alpha$$

and the class declaration gives each method a qualified type too:

$$empty : \forall \alpha.\ \mathit{Monoid}\ \alpha \Rightarrow \alpha \qquad\qquad combine : \forall \alpha.\ \mathit{Monoid}\ \alpha \Rightarrow \alpha \to \alpha \to \alpha$$

Whether a constraint holds is decided by a second judgment, **entailment**, written $P \Vdash \pi$: "from the constraints P that we assume, the constraint π follows". Each instance declaration becomes a rule of entailment. An instance without a context is an axiom; an instance with a context is a rule with premises:

$$\frac{}{P \Vdash \mathit{Monoid}\ \texttt{Sum}}\ \textsf{E-Sum} \qquad \frac{}{P \Vdash \mathit{Monoid}\ \texttt{Max}}\ \textsf{E-Max}$$

$$\frac{P \Vdash \mathit{Monoid}\ \alpha \qquad P \Vdash \mathit{Monoid}\ \beta}{P \Vdash \mathit{Monoid}\ (\alpha \times \beta)}\ \textsf{E-Pair} \qquad \frac{\pi \in P}{P \Vdash \pi}\ \textsf{E-Assume}$$

The last rule says that an assumed constraint holds. It is what a generic function's body relies on: inside `mconcat`, P contains *Monoid* M, and that is why the body may call `M::empty()`.

Typing with constraints

The typing judgment now carries the assumed constraints as well: $P \mid \Gamma \vdash e : \tau$. Two rules connect it with entailment. The first *uses* a qualified type, by choosing the type and discharging the constraint:

$$\frac{P \mid \Gamma \vdash e : \forall \alpha.\ \pi \Rightarrow \tau \qquad P \Vdash \pi[\alpha := \sigma]}{P \mid \Gamma \vdash e : \tau[\alpha := \sigma]}\ \textsf{T-Inst}$$

The second *introduces* one, when a generic function is defined:

$$\frac{P, \pi \mid \Gamma \vdash e : \tau \qquad \alpha \ \text{ not free in } \ P \ \text{ or } \ \Gamma}{P \mid \Gamma \vdash e : \forall \alpha.\ \pi \Rightarrow \tau}\ \textsf{T-Gen}$$

Rule T-Gen is the type-class version of the type abstraction rule of Chapter 12: to show that the body of `mconcat` has type $\mathrm{Vec}\ \alpha \to \alpha$ for every monoid α, check it with *Monoid* α added to the assumptions. Rule T-Inst is what happens at a call. When `main` calls `mconcat(stats)` with `stats: Vec<(Sum, Max)>`, the checker instantiates $\alpha := \texttt{Sum} \times \texttt{Max}$ and must establish the premise $\Vdash \mathit{Monoid}\ (\texttt{Sum} \times \texttt{Max})$, by a derivation:

$$\frac{\dfrac{}{\Vdash \mathit{Monoid}\ \texttt{Sum}}\ \textsf{E-Sum} \qquad \dfrac{}{\Vdash \mathit{Monoid}\ \texttt{Max}}\ \textsf{E-Max}}{\Vdash \mathit{Monoid}\ (\texttt{Sum} \times \texttt{Max})}\ \textsf{E-Pair}$$

Instance resolution is proof search

Look at the entailment rules again. Each has a conclusion and some premises, all of them simple statements about types. Logicians call rules of this shape **Horn clauses**, and they are exactly what a Prolog program is made of. Finding a derivation for a goal such as *Monoid* $(\texttt{Sum} \times \texttt{Max})$ works the way Prolog works: find a rule whose conclusion matches the goal (here E-Pair, with $\alpha := \texttt{Sum}$ and $\beta := \texttt{Max}$), then solve its premises as new goals, recursively. This process is called **instance resolution**. rustc's

trait solver, the constraint solver of Haskell's main compiler GHC, and Lean's instance synthesizer are all, at heart, proof-search engines of this kind.

When the search fails, there is no derivation, and the program is rejected. Here is a call at a type with no instance (`code/rust/compile_fail/ch14_no_instance.rs`):

```
fn all_ready(flags: Vec<bool>) -> bool {
    mconcat(flags)
}
```

```
error[E0277]: the trait bound `bool: Monoid` is not satisfied
  --> compile_fail/ch14_no_instance.rs:16:13
   |
16 |     mconcat(flags)
   |     ------- ^^^^^ the trait `Monoid` is not implemented for `bool`
   |     |
   |     required by a bound introduced by this call
   |
help: this trait has no implementations, consider adding one
  --> compile_fail/ch14_no_instance.rs:5:1
   |
 5 | trait Monoid {
   | ^^^^^^^^^^^^
note: required by a bound in `mconcat`
  --> compile_fail/ch14_no_instance.rs:10:15
   |
10 | fn mconcat<M: Monoid>(items: Vec<M>) -> M {
   |               ^^^^^^ required by this bound in `mconcat`
```

Every part of the message has a place in the rules. "The trait bound `bool: Monoid` is not satisfied" says that $\nvdash$ *Monoid* `bool` has no derivation. "Required by a bound introduced by this call" points at the use of rule T-Inst whose second premise failed. And "this trait has no implementations" tells you that there are no rules at all whose conclusion has the form *Monoid* ..., so the search had nothing to try.

14.3 Dictionary passing

The rules tell us *whether* a program is well typed. A compiler also needs to know what the program *does*. Inside `mconcat` the type `M` is unknown, so what code should `M::empty()` run? Wadler and Blott's answer, which is still how type classes are explained (and, in Haskell and Lean, implemented), is a translation called **dictionary passing**. It turns a program with classes into an ordinary program without them:

With classes	After the translation
a class (`trait Monoid`)	a record type, one field per method
an instance (`impl Monoid for Sum`)	a record value
an instance with a context	a function from records to a record
a constraint in a signature (`M: Monoid`)	an extra parameter
a method call (`M::empty()`)	a field access
a call of a constrained function	a call that passes the record found by resolution

The record is called a **dictionary** because it is a lookup table: given the name of a method, it returns that method's implementation for one particular type. Let us carry out the translation by hand, in the second half of `ch14_dictionary_passing.rs`. No trait appears in that half of the file.

The class becomes a struct

```
/// The class `Monoid`, as a record type: one field per method.
struct MonoidDict<M> {
    empty: Box<dyn Fn() -> M>,
    combine: Box<dyn Fn(M, M) -> M>,
}
```

One field per method. The fields hold closures behind `Box<dyn Fn>` (Chapter 8), for a reason we will see in a moment.

Instances become values

```
/// The instance `impl Monoid for Sum`, as a value.
fn sum_dict() -> MonoidDict<Sum> {
    MonoidDict {
        empty: Box::new(|| Sum(0)),
        combine: Box::new(|a, b| Sum(a.0 + b.0)),
    }
}

fn product_dict() -> MonoidDict<Product> {
    MonoidDict {
        empty: Box::new(|| Product(1)),
        combine: Box::new(|a, b| Product(a.0 * b.0)),
    }
}

fn max_dict() -> MonoidDict<Max> {
    MonoidDict {
        empty: Box::new(|| Max(i64::MIN)),
        combine: Box::new(|a, b| Max(a.0.max(b.0))),
    }
}

fn string_dict() -> MonoidDict<String> {
    MonoidDict {
        empty: Box::new(String::new),
        combine: Box::new(|mut a, b| {
            a.push_str(&b);
            a
        }),
    }
}

/// `impl<T> Monoid for Vec<T>` is generic, so its dictionary is built by a
/// generic function: one dictionary per element type.
fn vec_dict<T: 'static>() -> MonoidDict<Vec<T>> {
    MonoidDict {
        empty: Box::new(Vec::new),
        combine: Box::new(|mut a, b| {
            a.extend(b);
            a
        }),
    }
}
```

Each instance is now a function that builds its dictionary, and the closures are the bodies of the methods in the `impl` blocks, nearly character for character. The generic instance `impl<T> Monoid for Vec<T>` became a generic function `vec_dict<T>`, which builds one dictionary for each element type. The bound

`T: 'static` says that `T` contains no short-lived references, which a `Box<dyn Fn>` requires by default (Chapter 15 explains the `+ 'static` hidden in `Box<dyn Trait>`).

An instance with a context becomes a function on dictionaries

```
/// The instance with a context, `impl<A: Monoid, B: Monoid> Monoid for (A, B)`,
/// becomes a function from dictionaries to a dictionary.
fn pair_dict<A: 'static, B: 'static>(
    da: MonoidDict<A>,
    db: MonoidDict<B>,
) -> MonoidDict<(A, B)> {
    // Move each field of the two dictionaries into the closure that needs it.
    let (empty_a, combine_a) = (da.empty, da.combine);
    let (empty_b, combine_b) = (db.empty, db.combine);
    MonoidDict {
        empty: Box::new(move || (empty_a(), empty_b())),
        combine: Box::new(move |(a1, b1), (a2, b2)| {
            (combine_a(a1, a2), combine_b(b1, b2))
        }),
    }
}
```

This is the most instructive part of the translation. The instance for pairs had a context, `A: Monoid, B: Monoid`. After the translation the context is two parameters, the dictionaries for `A` and `B`, and the result is the dictionary for `(A, B)`. Its closures *capture* the functions of the component dictionaries, which is why the fields of `MonoidDict` must be able to hold capturing closures and not only plain `fn` pointers. In logical terms, rule E-Pair is an implication ("if α and β are monoids, so is $\alpha \times \beta$"), and its translation is a function from evidence to evidence: Curry–Howard again.

Constraints become parameters

```
/// `mconcat`, translated: the bound `M: Monoid` became the parameter `dict`.
fn mconcat_dict<M>(dict: &MonoidDict<M>, items: Vec<M>) -> M {
    let mut acc = (dict.empty)();
    for item in items {
        acc = (dict.combine)(acc, item);
    }
    acc
}

/// `mconcat_batches`, translated: the dictionary it received is passed on to
/// the functions it calls.
fn mconcat_batches_dict<M>(dict: &MonoidDict<M>, batches: Vec<Vec<M>>) -> M {
    let partials: Vec<M> = batches
        .into_iter()
        .map(|batch| mconcat_dict(dict, batch))
        .collect();
    mconcat_dict(dict, partials)
}
```

Compare `mconcat_dict` with `mconcat` line by line: `M::empty()` became `(dict.empty)()`, `acc.combine(item)` became `(dict.combine)(acc, item)`, and the bound `M: Monoid` became the parameter `dict`. (The parentheses in `(dict.empty)()` tell Rust to call the function stored in the field, rather than look for a method named `empty`.) The second function shows what happens when one constrained function calls another: `mconcat_batches_dict` has a dictionary for `M`, its own parameter, and passes it on. In the rules, that is E-Assume: the constraint is in P because it was assumed, and the evidence for it is the variable `dict`.

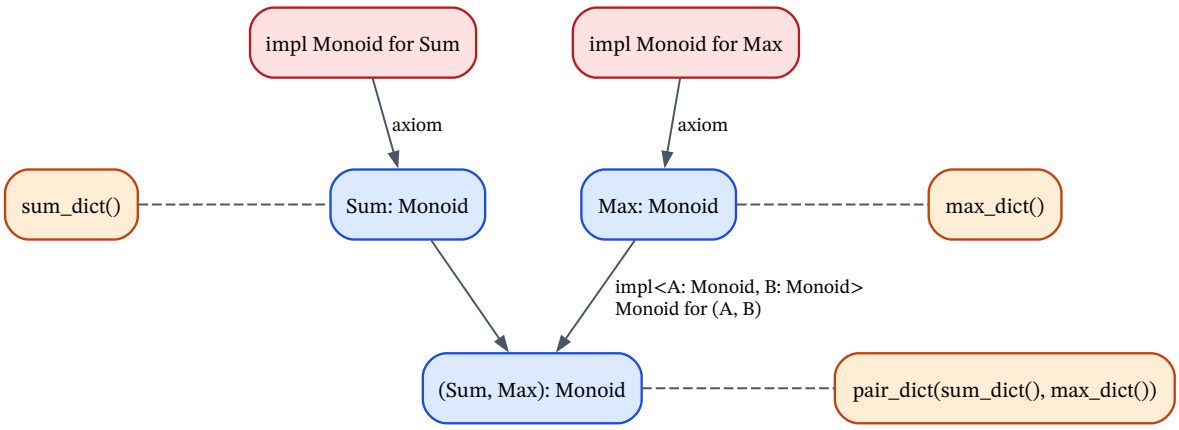
The two programs agree

The `main` function runs every computation both ways, with the trait and with explicit dictionaries, and asserts that the results are equal:

```
let readings = [3, 1, 4, 1, 5];

let sums: Vec<Sum> = readings.iter().map(|&n| Sum(n)).collect();
report(
    "Sum",
    mconcat(sums.clone()),
    mconcat_dict(&sum_dict(), sums),
);

let products: Vec<Product> = readings.iter().map(|&n| Product(n)).collect();
report(
    "Product",
    mconcat(products.clone()),
    mconcat_dict(&product_dict(), products),
);

let words = vec!["dictionary".to_string(), "-".into(), "passing".into()];
report(
    "String",
    mconcat(words.clone()),
    mconcat_dict(&string_dict(), words),
);

let lists = vec![vec![1, 2], vec![], vec![3]];
report(
    "Vec<i32>",
    mconcat(lists.clone()),
    mconcat_dict(&vec_dict(), lists),
);

// The pair instance: total and maximum in one pass.
let stats: Vec<(Sum, Max)> = readings.iter().map(|&n| (Sum(n), Max(n))).collect();
report(
    "(Sum, Max)",
    mconcat(stats.clone()),
    mconcat_dict(&pair_dict(sum_dict(), max_dict()), stats),
);

// A generic function that calls another generic function.
let batches = vec![vec![Sum(3), Sum(1)], vec![Sum(4)], vec![Sum(1), Sum(5)]];
report(
    "batches of Sum",
    mconcat_batches(batches.clone()),
    mconcat_batches_dict(&sum_dict(), batches),
);
```

```
Sum               trait: Sum(14)               dict: Sum(14)
Product           trait: Product(60)           dict: Product(60)
String            trait: "dictionary-passing"  dict: "dictionary-passing"
Vec<i32>          trait: [1, 2, 3]             dict: [1, 2, 3]
(Sum, Max)        trait: (Sum(14), Max(5))     dict: (Sum(14), Max(5))
batches of Sum    trait: Sum(14)               dict: Sum(14)
empty (Sum, Prod) trait: (Sum(0), Product(1))  dict: (Sum(0), Product(1))
```

The last line comes from a call on an empty vector, where both versions return the neutral element. Look at the `(Sum, Max)` call: where the trait version says `mconcat(stats.clone())`, the dictionary version says `mconcat_dict(&pair_dict(sum_dict(), max_dict()), stats)`. The expression `pair_dict(sum_dict(), max_dict())` is the derivation of *Monoid* (`Sum` × `Max`) that we wrote above, with each rule replaced by the function that implements it: E-Pair by `pair_dict`, E-Sum by `sum_dict`, E-Max by `max_dict`.



Instance resolution for (Sum, Max): Monoid. The goals form a derivation built from the instances; reading each rule as a function turns the derivation into the dictionary.

This is the central idea of the chapter, so let us state it plainly. **Constraints are propositions, instances are axioms and implications, resolution is proof search, and the dictionary is the proof.** The elaborated program passes proofs around at run time, and these proofs are useful: they contain the code.

We can make the evidence part of the rules. Write $P \Vdash d : \pi$ for "d is evidence for π", where P now gives a name to each assumed constraint ($dict : \mathit{Monoid}\ \alpha$). Rule E-Pair becomes:

$$\frac{P \Vdash d_1 : \mathit{Monoid}\ \alpha \qquad P \Vdash d_2 : \mathit{Monoid}\ \beta}{P \Vdash \texttt{pair_dict}(d_1, d_2) : \mathit{Monoid}\ (\alpha \times \beta)}\ \text{E-Pair}$$

Rule T-Inst then translates a use of a constrained function into an application of it to the evidence, and rule T-Gen translates a generic definition into a function that takes the evidence as a parameter. That translation, from a derivation in the system with classes to a program in a language without them, is called **elaboration**, and producing it is the job of a type checker for type classes.

What rustc actually does

If you compiled `mconcat` exactly as `mconcat_dict`, every `combine` would be an indirect call through a pointer, and every dictionary would be a heap allocation. Rust does not do that. It **monomorphizes** generic functions (Chapter 12): for each type at which `mconcat` is used, it compiles a separate copy, `mconcat::<Sum>`, `mconcat::<String>`, and so on. In each copy the dictionary is a constant known at compile time, so `(dict.combine)(acc, item)` becomes a direct call to `<Sum as Monoid>::combine`, which can then be inlined down to a single machine addition. You can think of monomorphization as dictionary passing followed by specialization: the dictionary argument is evaluated at compile time, and disappears.

Haskell and Lean take the other route: their compiled code passes dictionaries at run time, as `mconcat_dict` does, and their optimizers specialize calls when it pays off. Chapter 12 described the trade-off.

There is one place where Rust keeps a dictionary at run time: the **trait object**. A `Box<dyn Trait>` is a pointer to a value together with a pointer to the dictionary for that value's type, which Rust calls

a *vtable*. The difference from our `MonoidDict` is where the dictionary travels. In dictionary passing it is an argument of the *function*; in a trait object it is attached to the *value*, so that values of different types, each with its own dictionary, can sit in the same `Vec`. That is the subject of Chapter 15, which also explains why `dyn Monoid` is impossible (hint: `empty` has no value to carry a vtable).

14.4 Coherence

Dictionary passing defines the meaning of a program through the derivations of its constraints. What if a constraint has two derivations, with different dictionaries? Then the meaning of the program depends on which derivation the compiler happened to find. A type-class system is **coherent** when this cannot happen: all the derivations of a judgment elaborate to programs with the same behavior. The simplest way to guarantee it, and the one Rust chooses, is to make sure that two instance rules never apply to the same constraint. Rust enforces this with two restrictions.

No overlapping impls

The first restriction is that two impls of a trait may not apply to the same type. The instance for `(A, B)` and a separate instance for `(Sum, Max)` would overlap, since the first also covers `(Sum, Max)`. The most common way to run into this is a **blanket impl**, an impl for every type that satisfies some bound. The file `code/rust/examples/ch14_trait_features.rs` has one:

```
/// A blanket impl: *every* type that implements `Display` gets `Log`.
/// As an instance declaration: for all T, Display T implies Log T.
trait Log {
    fn log_line(&self) -> String;
}

impl<T: Display> Log for T {
    fn log_line(&self) -> String {
        format!("[log] {self}")
    }
}
```

As a rule, it reads $\mathit{Display}\ \alpha \Rightarrow \mathit{Log}\ \alpha$ for *every* α, so it gives `Log` to every type that can be displayed, including types defined later and in other crates:

```
// `Log` was never implemented for these types by name: the blanket impl
// covers every `Display` type, from the standard library's or ours.
println!("{}", 42.log_line());
println!("{}", "door opened".log_line());
println!("{}", Celsius(-18.0).log_line());
```

```
[log] 42
[log] door opened
[log] -18.0 °C
```

The standard library uses the same pattern: it has an `impl<T: Display + ?Sized> ToString for T`, which is why every type you can format with `{}` has a `to_string` method. But a blanket impl claims a lot of territory. Adding a special case for `i32` (`code/rust/compile_fail/ch14_overlap.rs`) is rejected:

```
trait Log {
    fn log_line(&self) -> String;
}
```

```
impl<T: Display> Log for T {
    fn log_line(&self) -> String {
        format!("[log] {self}")
    }
}

impl Log for i32 {
    fn log_line(&self) -> String {
        format!("[log] the number {self}")
    }
}
```

```
error[E0119]: conflicting implementations of trait `Log` for type `i32`
  --> compile_fail/ch14_overlap.rs:18:1
   |
12 | impl<T: Display> Log for T {
   | -------------------------- first implementation here
...
18 | impl Log for i32 {
   | ^^^^^^^^^^^^^^^^ conflicting implementation for `i32`
```

Both rules conclude *Log* `i32` (the first because `i32: Display`), with different code, so a call `42.log_line()` would have two meanings.

Some languages allow overlapping instances and resolve them by choosing the most specific one. Rust has such a feature, called **specialization**, but it is unstable: it has been available only on nightly compilers for years, because in its general form it interacts unsoundly with lifetimes, which the compiler erases before it generates code. It is worth knowing that it exists, and that the standard library uses a restricted form internally; it is not part of the language you can use, and if it matters to you, check its current status.

The orphan rule

Forbidding overlap within one crate is not enough. Suppose two independent crates each implement `Display` for `Vec<u8>`. Each compiles on its own, since each sees only one impl; but a program that depends on both would have two instances, and no single crate would be at fault. Rust prevents this situation from arising with the **orphan rule**: an impl must be written in the crate that defines the trait or in a crate that defines the type. (More precisely, a local type must appear in the right position among the impl's types; the Rust Reference has the full rule.) An impl in neither is an *orphan*:

```
impl fmt::Display for Vec<u8> {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{} bytes", self.len())
    }
}
```

```
error[E0117]: only traits defined in the current crate can be implemented for types defined
outside of the crate
 --> compile_fail/ch14_orphan.rs:8:1
  |
8 | impl fmt::Display for Vec<u8> {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^_______
  |                         |
  |                         `Vec` is not defined in the current crate
  |
  = note: impl doesn't have any local type before any uncovered type parameters
```

```
  = note: for more information see https://doc.rust-lang.org/reference/items/implementations
.html#orphan-rules
  = note: define and implement a trait or new type instead
```

The orphan rule makes coherence a *local* property: for any trait and type, only the crates that own one of them can provide an impl, and those crates can see each other, so the compiler can check for overlap where the impl is written. The price is the one the last note of the diagnostic suggests. To give a foreign type a foreign trait, you wrap the type in a newtype of your own. That is the same move as `Sum` and `Product`: in a coherent system, a newtype is how you say "this type, but with a different instance".

Lean chooses differently

Lean does not enforce coherence. You may declare as many instances of a class for a type as you like; resolution tries them in order of priority and, among instances of equal priority, the most recent first. The file `code/lean/Book/Ch14/Dictionaries.lean` builds on `Monoid.lean`, which (as we will see in the section on laws) defines a `Monoid` class and an instance for `Nat` with addition:

```
section Multiplication

/-- A second `Monoid Nat` instance, visible only until the end of this
section. Lean accepts it; Rust would reject it (E0119). -/
local instance natMul : Monoid Nat where
  empty := 1
  combine a b := a * b

-- The most recently declared instance wins: the same expression as above
-- now means a product.
/-- info: 24 -/
#guard_msgs in
#eval mconcat [2, 3, 4]

end Multiplication

-- After the section, the local instance is gone.
/-- info: 9 -/
#guard_msgs in
#eval mconcat [2, 3, 4]

section Priorities

/-- A low-priority instance is tried only after the default-priority ones. -/
local instance (priority := low) natMax : Monoid Nat where
  empty := 0
  combine a b := max a b

/-- info: 9 -/
#guard_msgs in
#eval mconcat [2, 3, 4]

end Priorities

-- `letI` adds an instance for a single expression.
/-- info: 4 -/
#guard_msgs in
#eval
  letI : Monoid Nat := ⟨0, max⟩
  mconcat [2, 3, 4]
```

The same expression, `mconcat [2, 3, 4]`, evaluates to `9` with the addition instance and to `24` inside the section that declares the multiplication instance `natMul`; the keyword `local` makes that instance

disappear at the end of the section. An instance with lower priority (`natMax`) does not displace the default one, and `letI` installs an instance for a single expression. Rust would have rejected `natMul` with E0119.

Lean leaves coherence to convention: Mathlib keeps one canonical instance per structure and type, and when two chains of instances lead to the same class (a "diamond"), it makes them equal by definition, so that it does not matter which one resolution finds. In your own Lean code, that is your job. Rust, which must make guarantees about programs written by many people who never coordinate, made coherence a rule of the language.

14.5 Associated types, supertraits and default methods

Three more features complete the toolbox. Each has a reading in terms of dictionaries, and a counterpart in Lean.

Type parameters and associated types

A trait can have type parameters of its own. Then it describes a *relation* between types, and a type can implement it several times, once for each value of the parameter:

```
/// A trait with a type parameter: a *relation* between two types. One type
/// may implement it many times, once for each target.
trait ConvertTo<Target> {
    fn convert(&self) -> Target;
}

impl ConvertTo<Fahrenheit> for Celsius {
    fn convert(&self) -> Fahrenheit {
        Fahrenheit(self.0 * 9.0 / 5.0 + 32.0)
    }
}

impl ConvertTo<Kelvin> for Celsius {
    fn convert(&self) -> Kelvin {
        Kelvin(self.0 + 273.15)
    }
}
```

Because `Celsius` implements `ConvertTo` twice, a call `c.convert()` is ambiguous unless something says which target is meant. The program says it with type annotations:

```
let c = Celsius(21.5);
// With a type parameter, the caller must say which instance it means:
// the *target* type selects the impl.
let f: Fahrenheit = c.convert();
let k: Kelvin = c.convert();
println!("{c} = {f:?} = {k:?}");
```

```
21.5 °C = Fahrenheit(70.7) = Kelvin(294.65)
```

Without an annotation (`code/rust/compile_fail/ch14_ambiguous_target.rs`), resolution finds two candidates and cannot choose:

```
fn show(c: &Celsius) {
    let t = c.convert();
}
```

```
error[E0283]: type annotations needed
  --> compile_fail/ch14_ambiguous_target.rs:27:9
   |
27 |     let t = c.convert();
   |         ^     ------- type must be known at this point
   |
note: multiple `impl`s satisfying `Celsius: ConvertTo<_>` found
  --> compile_fail/ch14_ambiguous_target.rs:13:1
   |
13 | impl ConvertTo<Fahrenheit> for Celsius {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
...
19 | impl ConvertTo<Kelvin> for Celsius {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: consider giving `t` an explicit type
   |
27 |     let t: /* Type */ = c.convert();
   |          ++++++++++++
```

Notice how different this is from E0277. There, the search found no derivation; here it found two possible ones, `Celsius: ConvertTo<Fahrenheit>` and `Celsius: ConvertTo<Kelvin>`, and nothing in the program decides between them.

An **associated type** is a type that each instance must define, and that the implementing type therefore *determines*. Our sensors have one:

```
/// Every instrument has a name.
trait Device {
    fn name(&self) -> String;
}

/// A sensor is a device that produces readings. `Device` is a supertrait:
/// a type can implement `Sensor` only if it also implements `Device`.
trait Sensor: Device {
    /// An associated type: a *function* from sensor types to reading types.
    type Reading: Debug;

    fn read(&mut self) -> Self::Reading;

    /// A default method, written once in terms of the other methods (including
    /// the supertrait's `name`). Instances may use it or override it.
    fn report(&mut self) -> String {
        let reading = self.read();
        format!("{}: {reading:?}", self.name())
    }
}
```

```
/// Replays recorded temperatures.
struct Thermometer {
    label: String,
    recorded: Vec<Celsius>,
    next: usize,
}

impl Device for Thermometer {
    fn name(&self) -> String {
        self.label.clone()
    }
}

impl Sensor for Thermometer {
    type Reading = Celsius;
    fn read(&mut self) -> Celsius {
        let reading = self.recorded[self.next % self.recorded.len()];
        self.next += 1;
```

```
        reading
    }
    // `report` is not written here: the default is used.
}

/// A door switch that reads `true` when the door is open.
struct DoorSwitch {
    open: bool,
}

impl Device for DoorSwitch {
    fn name(&self) -> String {
        "lab door".to_string()
    }
}

impl Sensor for DoorSwitch {
    type Reading = bool;
    fn read(&mut self) -> bool {
        self.open
    }
    // This instance overrides the default method.
    fn report(&mut self) -> String {
        let state = if self.read() { "open" } else { "closed" };
        format!("{} is {state}", self.name())
    }
}
```

A `Thermometer` has readings of type `Celsius`, a `DoorSwitch` has readings of type `bool`, and no type can have two reading types, because a type can implement `Sensor` only once. Mathematically, `Reading` is a **function from types to types**: $\mathit{Reading}(\texttt{Thermometer}) = \texttt{Celsius}$. Generic code can apply it, and can put bounds on its result:

```
/// Read `n` values. The return type mentions `S::Reading`, the value of the
/// type-level function `Reading` at `S`.
fn sample<S: Sensor>(sensor: &mut S, n: usize) -> Vec<S::Reading> {
    (0..n).map(|_| sensor.read()).collect()
}

/// A bound on the *result* of the type-level function: the readings must be
/// comparable (`PartialOrd`) and copyable.
fn peak<S>(sensor: &mut S, n: usize) -> Option<S::Reading>
where
    S: Sensor,
    S::Reading: PartialOrd + Copy,
{
    let readings = sample(sensor, n);
    let mut best = *readings.first()?;
    for &r in &readings {
        if r > best {
            best = r;
        }
    }
    Some(best)
}
```

The return type of `sample` is `Vec<S::Reading>`, the value of the type-level function at `S`. The function `peak` additionally requires the readings to be comparable and copyable. A trait with a type parameter could not do this: `S::Reading` makes sense only because there is a single impl to read it from.

Haskell reached the same idea from the other side. Its classes may have several parameters, and Mark Jones proposed **functional dependencies** to declare that some parameters determine others: a class `Sensor s r | s -> r` says that `s` determines `r`, which is exactly what an associated type says. Rust's

`Add` trait shows both kinds of parameter at once: in `trait Add<Rhs = Self> { type Output; ... }`, the right operand `Rhs` is an input (an `i32` can be added to an `i32` and to an `&i32`, two impls) and the result `Output` is an output (once both operands are known, the result type is fixed).

Lean expresses the same distinction with **`outParam`**, a marker on a class parameter that says "resolution computes this one". Lean's own addition is declared this way, with the result type `γ` as the output:

```
/--
info: @HAdd.hAdd : {α : Type u_1} → {β : Type u_2} → {γ : outParam (Type u_3)} → [self :
HAdd α β γ] → α → β → γ
-/
#guard_msgs in
#check @HAdd.hAdd
```

The file `code/lean/Book/Ch14/Features.lean` defines a `Sensor` class both ways:

```
/-- Sensors and their readings. `reading` is an *output* parameter: once the
sensor type is known, resolution determines the reading type, as an
associated type does in Rust. -/
class Sensor (σ : Type) (reading : outParam Type) where
  read : σ → reading

/-- The same class with an ordinary (input) parameter. -/
class Sensor' (σ : Type) (reading : Type) where
  read : σ → reading

structure Thermometer where
  tenths : Int

instance : Sensor Thermometer Int := ⟨fun t => t.tenths⟩
instance : Sensor' Thermometer Int := ⟨fun t => t.tenths⟩

def readTwice {σ r : Type} [Sensor σ r] (s : σ) : r × r :=
  (Sensor.read s, Sensor.read s)

def readTwice' {σ r : Type} [Sensor' σ r] (s : σ) : r × r :=
  (Sensor'.read s, Sensor'.read s)

/-- info: (215, 215) -/
#guard_msgs in
#eval readTwice (Thermometer.mk 215)
```

With `outParam`, `readTwice (Thermometer.mk 215)` works: resolution starts from the known sensor type and finds the instance, and with it the reading type. Without `outParam`, Lean refuses to start the search while one of the class's arguments is still unknown:

```
/--
error: typeclass instance problem is stuck
  Sensor' Thermometer ?m.3

Note: Lean will not try to resolve this typeclass instance problem because the second type
argument to `Sensor'` is a metavariable. This argument must be fully determined before Lean
will try to resolve the typeclass.

Hint: Adding type annotations and supplying implicit arguments to functions can give Lean
more information for typeclass resolution. For example, if you have a variable `x` that you
intend to be a `Nat`, but Lean reports it as having an unresolved type like `?m`, replacing
`x` with `(x : Nat)` can get typeclass resolution un-stuck.
-/
#guard_msgs in
#eval readTwice' (Thermometer.mk 215)

/-- info: (215, 215) -/
```

```
#guard_msgs in
#eval (readTwice' (Thermometer.mk 215) : Int × Int)
```

This is Lean's version of E0283, and the fix is the same: an annotation that fixes the unknown type.

Supertraits

The declaration `trait Sensor: Device` makes `Device` a **supertrait** of `Sensor`: a type can implement `Sensor` only if it also implements `Device`, and code that knows `S: Sensor` may call `name`. In the rules, a supertrait adds an entailment rule that goes from the subclass to the superclass:

$$\frac{P \Vdash \mathit{Sensor}\ \alpha}{P \Vdash \mathit{Device}\ \alpha}\ \text{E-Super}$$

In the dictionary translation, the `Sensor` dictionary *contains* the `Device` dictionary, so that the evidence for rule E-Super is a field access. The default method `report` relies on this: it calls `self.name()`.

Default methods

A **default method** is written once, in the trait, in terms of the other methods. `Thermometer` uses the default `report`; `DoorSwitch` overrides it:

```
let mut fridge = thermometer();
let mut door = DoorSwitch { open: true };
println!("sample:  {:?}", sample(&mut fridge, 4));
println!("peak:    {:?}", peak(&mut fridge, 3));
println!("default: {:?}", report_twice(&mut fridge));
println!("custom:  {:?}", report_twice(&mut door));
```

```
sample:  [Celsius(4.5), Celsius(5.0), Celsius(3.5), Celsius(4.5)]
peak:    Some(Celsius(5.0))
default: ["fridge: Celsius(5.0)", "fridge: Celsius(3.5)"]
custom:  ["lab door is open", "lab door is open"]
```

(The thermometer replays its three recorded temperatures in a cycle, so successive reads differ.) The standard library's `Iterator` is the extreme case: it has one required method, `next`, and dozens of default ones (`map`, `filter`, `sum`, `collect` and many more) written in terms of `next`.

Here is the translation of all three features into dictionaries, from the same file:

```
/// The dictionary of `Device`.
struct DeviceDict<T> {
    name: fn(&T) -> String,
}

/// The dictionary of `Sensor`. The supertrait's dictionary is a field, the
/// associated type is a type parameter fixed by the instance, and the default
/// method is a field like any other.
struct SensorDict<T, R> {
    device: DeviceDict<T>,
    read: fn(&mut T) -> R,
    report: fn(&SensorDict<T, R>, &mut T) -> String,
}

/// The default method, translated: a function that receives the dictionary
/// it belongs to, so that it can call the other methods.
fn default_report<T, R: Debug>(dict: &SensorDict<T, R>, sensor: &mut T) -> String {
    let reading = (dict.read)(sensor);
    format!("{}: {reading:?}", (dict.device.name)(sensor))
}
```

```
/// `impl Sensor for Thermometer` uses the default `report`...
fn thermometer_dict() -> SensorDict<Thermometer, Celsius> {
    SensorDict {
        device: DeviceDict {
            name: <Thermometer as Device>::name,
        },
        read: <Thermometer as Sensor>::read,
        report: default_report,
    }
}

/// ...while `impl Sensor for DoorSwitch` overrides it. The override does not
/// need the dictionary, so this field wraps it in a closure that ignores it.
fn door_dict() -> SensorDict<DoorSwitch, bool> {
    SensorDict {
        device: DeviceDict {
            name: <DoorSwitch as Device>::name,
        },
        read: <DoorSwitch as Sensor>::read,
        report: |_dict, door| <DoorSwitch as Sensor>::report(door),
    }
}

/// A generic function with a `Sensor` bound, translated.
fn report_twice_dict<T, R>(dict: &SensorDict<T, R>, sensor: &mut T) -> [String; 2] {
    [(dict.report)(dict, sensor), (dict.report)(dict, sensor)]
}
```

The supertrait's dictionary is the field `device`. The associated type became a type parameter `R` of the dictionary, fixed by each instance (`SensorDict<Thermometer, Celsius>`): the instance determines it, as a functional dependency says. And a default method is a function that receives *the dictionary it belongs to*, so that it can call the instance's other methods: `default_report` reads `dict.read` and `dict.device.name`. An instance that keeps the default stores `default_report` in its `report` field; an instance that overrides it stores its own function. No closure here captures anything, so plain `fn` pointers suffice. The two versions agree:

```
let (mut fridge_a, mut fridge_b) = (thermometer(), thermometer());
let with_trait = report_twice(&mut fridge_a);
let with_dict = report_twice_dict(&thermometer_dict(), &mut fridge_b);
assert_eq!(with_trait, with_dict);
let with_trait = report_twice(&mut DoorSwitch { open: false });
let with_dict = report_twice_dict(&door_dict(), &mut DoorSwitch { open: false });
assert_eq!(with_trait, with_dict);
println!("dictionary versions agree: {with_dict:?}");
```

```
dictionary versions agree: ["lab door is closed", "lab door is closed"]
```

In Lean, a default method is a default value of a class field:

```
/-- A class with a default method: `describe` has a default implementation,
written in terms of `name`. -/
class Describe (α : Type) where
  name : α → String
  describe : α → String := fun x => s!"<{name x}>"

/-- This instance uses the default `describe`. -/
instance : Describe Nat where
  name n := s!"nat {n}"

/-- This instance overrides it. -/
instance : Describe Bool where
```

```
  name b := toString b
  describe b := if b then "yes" else "no"

/-- info: "<nat 5>" -/
#guard_msgs in
#eval Describe.describe (5 : Nat)

/-- info: "yes" -/
#guard_msgs in
#eval Describe.describe true
```

and a supertrait is written with `extends`, which we use for laws in a moment.

14.6 Laws

The `Monoid` trait's documentation says that `combine` is associative and that `empty` is neutral. Nothing in the trait enforces it. The compiler checks that an instance supplies two functions of the right types and nothing more, so it happily accepts these two instances (`code/rust/examples/ch14_laws.rs`):

```
/// Subtraction with 0 as the "neutral" element. The compiler accepts this
/// instance: the trait only asks for two functions of the right types.
#[derive(Clone, Copy, Debug, PartialEq)]
struct Diff(i64);

impl Monoid for Diff {
    fn empty() -> Self {
        Diff(0)
    }
    fn combine(self, other: Self) -> Self {
        Diff(self.0 - other.0)
    }
}

/// Floating-point addition: associative for real numbers, but not for `f64`,
/// because every addition rounds its result.
#[derive(Clone, Copy, Debug, PartialEq)]
struct FloatSum(f64);

impl Monoid for FloatSum {
    fn empty() -> Self {
        FloatSum(0.0)
    }
    fn combine(self, other: Self) -> Self {
        FloatSum(self.0 + other.0)
    }
}
```

Subtraction is not associative, and 0 is neutral for it only on the right ($a - 0 = a$, but $0 - a \neq a$). Floating-point addition is more insidious: every addition rounds its result, and the rounding errors depend on the order of the additions.

Why laws matter

Generic code relies on laws, usually without saying so. Here are two ways to combine a list:

```
/// Left to right: ((e ⋄ x1) ⋄ x2) ⋄ ... ⋄ xn.
fn mconcat<M: Monoid>(items: Vec<M>) -> M {
    let mut acc = M::empty();
    for item in items {
        acc = acc.combine(item);
    }
    acc
```

```
}

/// Divide and conquer: split in halves, combine each half, combine the two
/// results. This is how a parallel reduction splits its work. It computes the
/// same value as `mconcat` exactly when the monoid laws hold.
fn mconcat_tree<M: Monoid>(mut items: Vec<M>) -> M {
    match items.len() {
        0 => M::empty(),
        1 => items.pop().unwrap(),
        n => {
            let right = items.split_off(n / 2);
            mconcat_tree(items).combine(mconcat_tree(right))
        }
    }
}
```

The tree-shaped fold splits the list in two halves, combines each half (in parallel, in a real program) and combines the two results. That is how parallel reductions and map-reduce systems divide their work, and it computes the same value as the left-to-right fold *exactly when the monoid laws hold*: associativity lets you move the parentheses, and the neutral element lets you start each half from scratch. With an unlawful instance, the two folds disagree:

```
left fold vs tree fold:
  Sum       left: Sum(21)                       tree: Sum(21)
  Diff      left: Diff(-21)                     tree: Diff(-3)
  FloatSum  left: FloatSum(0.6000000000000001)  tree: FloatSum(0.6)
```

`Diff` computes $0 - 1 - 2 - \ldots - 6 = -21$ one way and -3 the other. `FloatSum` gives `0.6000000000000001` one way and `0.6` the other, a small difference of the kind that makes a numerical result change with the number of threads that computed it.

The standard library has the same problem, and handles it the same way, with documentation. `Eq` promises that `==` is an equivalence relation (reflexive, symmetric, transitive), `Ord` that `cmp` is a total order consistent with `PartialOrd`, `Hash` that equal values have equal hashes. None of it is checked. If you implement `Ord` inconsistently, `sort` and `BTreeMap` will misbehave; the documentation of `BTreeMap` calls this a logic error whose behavior is unspecified, but not undefined behavior. Safe code cannot break memory safety this way, and `unsafe` code must not trust these laws, as Chapter 32 discusses. The split between `PartialEq` and `Eq`, and between `PartialOrd` and `Ord`, is itself a law written into the trait hierarchy: `Eq` adds no new operations to `PartialEq`, only the promise that `==` is reflexive. `f64` cannot make that promise (`NaN != NaN`), so it implements `PartialEq` but not `Eq`.

Testing laws in Rust

What Rust *can* do is test the laws, like any other property. This function checks both identity laws on every sample and associativity on every triple of samples:

```
/// Check the three monoid laws on every sample (identity) and on every
/// triple of samples (associativity). Returns the number of checks performed,
/// or a description of the first counterexample.
fn check_laws<M>(samples: &[M]) -> Result<usize, String>
where
    M: Monoid + Clone + PartialEq + Debug,
{
    let mut checks = 0;
    for a in samples {
```

```
        let left = M::empty().combine(a.clone());
        if left != *a {
            return Err(format!("left identity fails: empty ⋄ {a:?} = {left:?}"));
        }
        let right = a.clone().combine(M::empty());
        if right != *a {
            return Err(format!("right identity fails: {a:?} ⋄ empty = {right:?}"));
        }
        checks += 2;
    }
    for a in samples {
        for b in samples {
            for c in samples {
                let ab_c = a.clone().combine(b.clone()).combine(c.clone());
                let a_bc = a.clone().combine(b.clone().combine(c.clone()));
                if ab_c != a_bc {
                    return Err(format!(
                        "associativity fails for {a:?}, {b:?}, {c:?}\n\
                         (a ⋄ b) ⋄ c = {ab_c:?}, a ⋄ (b ⋄ c) = {a_bc:?}"
                    ));
                }
                checks += 1;
            }
        }
    }
    Ok(checks)
}
```

```
println!("law checks on samples:");
let ints = [-7, -1, 0, 1, 2, 3, 10];
print_check("Sum", &ints.map(Sum));
print_check("String", &["", "a", "bc", "def"].map(String::from));
print_check("Stats", &ints.map(Stats::of));
print_check("Diff", &ints.map(Diff));
print_check("FloatSum", &[0.1, 0.2, 0.3, 1.0e16].map(FloatSum));
```

```
law checks on samples:
  Sum       ok (357 checks)
  String    ok (72 checks)
  Stats     ok (357 checks)
  Diff      FAILED: left identity fails: empty ⋄ Diff(-7) = Diff(7)
  FloatSum  FAILED: associativity fails for FloatSum(0.1), FloatSum(0.2), FloatSum(0.3)
            (a ⋄ b) ⋄ c = FloatSum(0.6000000000000001), a ⋄ (b ⋄ c) = FloatSum(0.6)
```

For seven integer samples the check performs $2 \cdot 7 + 7^3 = 357$ comparisons. It catches both broken instances: `Diff` fails left identity on the first sample, since $0 - (-7) = 7$, and `FloatSum` fails associativity on the first triple that exposes a rounding error. (`Stats` is the monoid of this chapter's walkthrough.) This is the Rust way, and a good one, especially with a property-testing library generating the samples. But it is evidence about 357 cases, not a proof about all of them.

Laws as proof fields in Lean

In Lean a class is a structure (the next section looks at this closely), and a structure may have fields whose types are propositions. The values of such fields are proofs. That is all it takes to put laws into a class. The file `code/lean/Book/Ch14/Monoid.lean` starts with the operations alone, the counterpart of the Rust trait:

```
/-- A monoid, operations only: the counterpart of the Rust trait `Monoid`. -/
class Monoid (α : Type) where
  /-- The neutral element. -/
```

```
  empty : α
  /-- The operation. -/
  combine : α → α → α

/-- `a ⋄ b` is `Monoid.combine a b`. -/
scoped infixl:70 " ⋄ " => Monoid.combine
```

The `scoped infixl` line defines the notation `a ⋄ b` for `combine a b`, available wherever the namespace `Ch14` is open; `infixl:70` makes it left-associative, with the same precedence as `*`. This `Monoid` is our own class, `Ch14.Monoid`. Mathlib has a different class with the same name, which this file does not import, and which we will meet at the end of the chapter. Next come instances, including one with a context, and the generic function, which in Lean is most natural as a fold from the right:

```
instance instMonoidNat : Monoid Nat where
  empty := 0
  combine a b := a + b

instance instMonoidString : Monoid String where
  empty := ""
  combine a b := a ++ b

instance instMonoidList {α : Type} : Monoid (List α) where
  empty := []
  combine xs ys := xs ++ ys

/-- An instance with a context: a pair is a monoid if both components are. -/
instance instMonoidProd {α β : Type} [Monoid α] [Monoid β] : Monoid (α × β) where
  empty := (empty, empty)
  combine p q := (p.1 ⋄ q.1, p.2 ⋄ q.2)
```

```
/-- Combine all the elements of a list: `x₁ ⋄ (x₂ ⋄ (... ⋄ empty))`. -/
def mconcat {α : Type} [Monoid α] : List α → α
  | [] => empty
  | x :: xs => x ⋄ mconcat xs
```

```
/-- info: 14 -/
#guard_msgs in
#eval mconcat [3, 1, 4, 1, 5]

/-- info: "dictionary-passing" -/
#guard_msgs in
#eval mconcat ["dictionary", "-", "passing"]

/-- info: (14, "abc") -/
#guard_msgs in
#eval mconcat [(3, "a"), (1, "b"), (10, "c")]
```

The binder `[Monoid α]` in `mconcat` is the constraint: square brackets mark an *instance-implicit* argument, which Lean fills in by resolution. Now the class with laws:

```
/-- A lawful monoid: the operations, plus proofs of the three laws. Every
instance must supply the proofs; Lean checks them like any other field. -/
class LawfulMonoid (α : Type) extends Monoid α where
  combine_assoc : ∀ a b c : α, (a ⋄ b) ⋄ c = a ⋄ (b ⋄ c)
  empty_combine : ∀ a : α, empty ⋄ a = a
  combine_empty : ∀ a : α, a ⋄ empty = a
```

`extends Monoid α` makes `Monoid` a superclass, exactly like `trait LawfulMonoid: Monoid` in Rust, and the three new fields are the laws. `#print` shows what Lean made of the declaration:

```
/--
info: class Ch14.LawfulMonoid (α : Type) : Type
number of parameters: 1
parents:
  Ch14.LawfulMonoid.toMonoid : Monoid α
fields:
  Ch14.Monoid.empty : α
  Ch14.Monoid.combine : α → α → α
  Ch14.LawfulMonoid.combine_assoc : ∀ (a b c : α), a ⋄ b ⋄ c = a ⋄ (b ⋄ c)
  Ch14.LawfulMonoid.empty_combine : ∀ (a : α), empty ⋄ a = a
  Ch14.LawfulMonoid.combine_empty : ∀ (a : α), a ⋄ empty = a
constructor:
  Ch14.LawfulMonoid.mk {α : Type} [toMonoid : Monoid α] (combine_assoc : ∀ (a b c : α), a ⋄
b ⋄ c = a ⋄ (b ⋄ c))
    (empty_combine : ∀ (a : α), empty ⋄ a = a) (combine_empty : ∀ (a : α), a ⋄ empty = a) :
LawfulMonoid α
field notation resolution order:
  Ch14.LawfulMonoid, Ch14.Monoid
-/
#guard_msgs in
#print LawfulMonoid
```

Here is the dictionary translation of a supertrait, made by Lean itself: the parent is a field `toMonoid`, and the constructor takes it as an instance argument `[toMonoid : Monoid α]`. (The printed laws write `a ⋄ b ⋄ c` for `(a ⋄ b) ⋄ c`, since `⋄` associates to the left.)

An instance of `LawfulMonoid` must fill in every field, proofs included. For natural numbers, strings and lists, the laws are theorems of Lean's library:

```
/-- Addition on `Nat`: the three laws are library theorems. -/
instance instLawfulNat : LawfulMonoid Nat where
  toMonoid := instMonoidNat
  combine_assoc := Nat.add_assoc
  empty_combine := Nat.zero_add
  combine_empty := Nat.add_zero

instance instLawfulString : LawfulMonoid String where
  toMonoid := instMonoidString
  combine_assoc _ _ _ := String.append_assoc
  empty_combine _ := String.empty_append
  combine_empty _ := String.append_empty

instance instLawfulList {α : Type} : LawfulMonoid (List α) where
  toMonoid := instMonoidList
  combine_assoc := List.append_assoc
  empty_combine := List.nil_append
  combine_empty := List.append_nil
```

`Nat.add_assoc` is a proof that `n + m + k = n + (m + k)` for all `n`, `m` and `k`, and that is what the field `combine_assoc` requires once `⋄` is unfolded to `+`. Lean checks this: the type of the field, with `instMonoidNat` as the monoid, *computes* to the type of `Nat.add_assoc`. The string lemmas take their arguments implicitly, hence the pattern `_ _ _ :=`, which accepts the three arguments of the law and ignores them.

The instance for pairs must prove the laws itself, from the laws of the components:

```
/-- Pairs of lawful monoids are lawful: each law holds componentwise. -/
instance instLawfulProd {α β : Type} [LawfulMonoid α] [LawfulMonoid β] :
    LawfulMonoid (α × β) where
  toMonoid := instMonoidProd
  combine_assoc p q r := by
    -- Unfold the pair operation: the goal becomes an equation between pairs.
```

```
    show ((p.1 ⋄ q.1) ⋄ r.1, (p.2 ⋄ q.2) ⋄ r.2) = (p.1 ⋄ (q.1 ⋄ r.1), p.2 ⋄ (q.2 ⋄ r.2))
    -- Rewrite each component with the associativity law of its own monoid.
    rw [LawfulMonoid.combine_assoc p.1, LawfulMonoid.combine_assoc p.2]
  empty_combine p := by
    show (empty ⋄ p.1, empty ⋄ p.2) = p
    rw [LawfulMonoid.empty_combine, LawfulMonoid.empty_combine]
  combine_empty p := by
    show (p.1 ⋄ empty, p.2 ⋄ empty) = p
    rw [LawfulMonoid.combine_empty, LawfulMonoid.combine_empty]
```

Let us go through `combine_assoc`. The goal is `(p ⋄ q) ⋄ r = p ⋄ (q ⋄ r)` for pairs `p`, `q`, `r`. The `show` tactic replaces it with a goal that is equal by definition, in which the pair operation is unfolded: an equation between two pairs, component by component. (`show` succeeds only if the new goal is definitionally equal to the old one, so it doubles as a check that we unfolded correctly.) Then `rw [LawfulMonoid.combine_assoc p.1, ...]` rewrites the first component with the associativity law of `α`, turning `(p.1 ⋄ q.1) ⋄ r.1` into `p.1 ⋄ (q.1 ⋄ r.1)`, and does the same for the second component with the law of `β`. The two sides are now identical, and `rw` closes the goal by itself, as it always tries `rfl` after rewriting. The two identity laws follow the same pattern; after their rewrites the goal is `(p.1, p.2) = p`, which holds by definition for pairs.

What we gain is that the laws can now be *used*. Here is the theorem that justifies the tree-shaped fold, proved once for every lawful monoid:

```
/-- `mconcat` can be computed in two batches and the results combined.
This is the law that makes splitting work across threads correct. -/
theorem mconcat_append (xs ys : List α) :
    mconcat (xs ++ ys) = mconcat xs ⋄ mconcat ys := by
  induction xs with
  | nil =>
    -- Goal: mconcat ([] ++ ys) = mconcat [] ⋄ mconcat ys,
    -- that is, mconcat ys = empty ⋄ mconcat ys.
    show mconcat ys = empty ⋄ mconcat ys
    rw [LawfulMonoid.empty_combine]
  | cons x xs ih =>
    -- ih   : mconcat (xs ++ ys) = mconcat xs ⋄ mconcat ys
    -- Goal : x ⋄ mconcat (xs ++ ys) = (x ⋄ mconcat xs) ⋄ mconcat ys
    show x ⋄ mconcat (xs ++ ys) = (x ⋄ mconcat xs) ⋄ mconcat ys
    rw [ih, LawfulMonoid.combine_assoc]
```

The proof is by induction on `xs`. When `xs` is empty, `[] ++ ys` is `ys` and `mconcat []` is `empty`, so the goal says `mconcat ys = empty ⋄ mconcat ys`; the `show` makes this explicit, and rewriting with the left identity law turns the right side into the left side. When `xs` is `x :: xs`, both `mconcat` and `++` unfold one step, and the goal becomes `x ⋄ mconcat (xs ++ ys) = (x ⋄ mconcat xs) ⋄ mconcat ys`. The induction hypothesis `ih` rewrites `mconcat (xs ++ ys)` on the left, giving `x ⋄ (mconcat xs ⋄ mconcat ys)`, and associativity rewrites the right side into the same expression. The proof uses two of the three laws, which is why the theorem is stated for a `LawfulMonoid`: for a bare `Monoid` it would be false.

Two consequences follow. Combining the results of batches gives the same value as combining everything at once, the Lean version of `mconcat_batches`:

```
/-- Combining the batch results gives the same value as combining everything
at once: the Lean version of `mconcat_batches` in the Rust example. -/
theorem mconcat_batches (batches : List (List α)) :
    mconcat (batches.map mconcat) = mconcat batches.flatten := by
  induction batches with
```

```
  | nil => rfl
  | cons b bs ih =>
    -- Goal: mconcat b ⋄ mconcat (bs.map mconcat) = mconcat (b ++ bs.flatten)
    show mconcat b ⋄ mconcat (bs.map mconcat) = mconcat (b ++ bs.flatten)
    rw [ih, mconcat_append]
```

And the left fold, which our Rust `mconcat` computes, agrees with the right fold, which our Lean `mconcat` computes:

```
/-- A left fold started at `a` equals `a` combined with the right fold. -/
theorem foldl_combine (xs : List α) (a : α) :
    xs.foldl (· ⋄ ·) a = a ⋄ mconcat xs := by
  induction xs generalizing a with
  | nil =>
    -- Goal: a = a ⋄ empty
    show a = a ⋄ empty
    rw [LawfulMonoid.combine_empty]
  | cons x xs ih =>
    -- Goal: xs.foldl (· ⋄ ·) (a ⋄ x) = a ⋄ (x ⋄ mconcat xs)
    show xs.foldl (· ⋄ ·) (a ⋄ x) = a ⋄ (x ⋄ mconcat xs)
    rw [ih (a ⋄ x), LawfulMonoid.combine_assoc]

/-- For a lawful monoid, folding from the left and from the right agree. -/
theorem foldl_eq_mconcat (xs : List α) :
    xs.foldl (· ⋄ ·) empty = mconcat xs := by
  rw [foldl_combine, LawfulMonoid.empty_combine]
```

The lemma needs `induction xs generalizing a`. A left fold changes its accumulator at every step, so the induction hypothesis must hold for every starting value `a`, not only for the one in the goal (Chapter 31 uses this pattern often). The step case rewrites with the hypothesis at the new accumulator `a ⋄ x` and then with associativity. The theorem sets `a := empty` and finishes with left identity.

Finally, subtraction. Lean's `Monoid` class accepts it, as Rust's trait did (the file shows the instance as an `example`), since only the types of the operations are checked. A `LawfulMonoid` instance is another matter: the proofs must go through, and they do not:

```
/--
error: omega could not prove the goal:
a possible counterexample may satisfy the constraints
  d ≥ 1
where
 d := c
---
error: omega could not prove the goal:
a possible counterexample may satisfy the constraints
  b ≥ 1
where
 b := a
-/
#guard_msgs in
example : LawfulMonoid Int where
  empty := 0
  combine a b := a - b
  combine_assoc a b c := by
    show a - b - c = a - (b - c)
    omega
  empty_combine a := by
    show 0 - a = a
    omega
  combine_empty a := by
    show a - 0 = a
    omega
```

The `omega` tactic, a decision procedure for linear arithmetic over integers and natural numbers, fails twice. For associativity, `a - b - c = a - (b - c)` is false whenever `c ≠ 0`; for left identity, `0 - a = a` is false whenever `a ≠ 0`. In both cases `omega` reports the condition under which it found a counterexample, naming the relevant variable (`c` and `a`) with an internal alias. The third law, `a - 0 = a`, is true, and its proof goes through. We can do better than a failed attempt, and *prove* that no lawful instance can use subtraction:

```
/-- Subtraction with `0`: the types are right, so `Monoid` accepts it. -/
example : Monoid Int where
  empty := 0
  combine a b := a - b

/-- But no lawful monoid on `Int` can have subtraction as its operation.
A class is a structure, so we can quantify over all its instances. -/
theorem sub_not_lawful (m : LawfulMonoid Int)
    (h : ∀ a b : Int, m.combine a b = a - b) : False := by
  -- Instantiate associativity at a = 0, b = 0, c = 1.
  have assoc := m.combine_assoc 0 0 1
  -- It says (0 - 0) - 1 = 0 - (0 - 1), that is, -1 = 1.
  rw [h, h, h, h] at assoc
  omega
```

Since a class is a structure, an instance is an ordinary value, and we can quantify over all of them. The theorem takes an arbitrary `m : LawfulMonoid Int` whose operation is subtraction (hypothesis `h`) and derives `False`. The proof instantiates `m`'s associativity law at `0`, `0` and `1`, rewrites every `◇` into `-` with `h`, and is left with the hypothesis `0 - 0 - 1 = 0 - (0 - 1)`, that is, `-1 = 1`, which `omega` refutes.

So the laws cost nothing at run time (proofs live in `Prop`, and Lean erases them when it compiles), but they move a burden from the users of a class to its implementers: an instance that cannot prove its laws cannot exist. Lean's core library uses lawful classes too, in a slightly different style, as separate classes that sit next to the operations: `LawfulBEq` says that a `BEq` instance agrees with equality, `LawfulFunctor` and `LawfulMonad` appear in Chapters 17 and 20, and `Std.Associative op` states a single law.

14.7 Type classes in Lean

Lean's classes are the same idea as Rust's traits, but Lean is more open about how they work: the elaborated program is ordinary Lean that you can inspect. The code is in `code/lean/Book/Ch14/Dictionaries.lean` and `Features.lean`.

A class is a structure, an instance argument is a parameter

```
/--
info: class Ch14.Monoid (α : Type) : Type
number of parameters: 1
fields:
  Ch14.Monoid.empty : α
  Ch14.Monoid.combine : α → α → α
constructor:
  Ch14.Monoid.mk {α : Type} (empty : α) (combine : α → α → α) : Monoid α
-/
#guard_msgs in
#print Monoid

/-- info: @combine : {α : Type} → [self : Monoid α] → α → α → α -/
#guard_msgs in
```

```
#check @Monoid.combine

/-- info: @mconcat : {α : Type} → [Monoid α] → List α → α -/
#guard_msgs in
#check @mconcat
```

The `#print` output is that of a structure: two fields and a constructor `Monoid.mk`. The method `combine` is the structure's projection, with one special feature: its `self` argument, the dictionary, is in square brackets, which tells Lean to find it by resolution. And `mconcat` takes the dictionary as its second argument, `[Monoid α]`. The dictionary-passing translation is not something we have to imagine: it is the type of the elaborated function.

We can also write the translation out by hand, with the dictionary as an explicit parameter, and prove that nothing changed:

```
/-- `mconcat` after the dictionary-passing translation: the dictionary `d` is
an explicit argument, and every method call reads a field of it. -/
def mconcatExplicit {α : Type} (d : Monoid α) : List α → α
  | [] => d.empty
  | x :: xs => d.combine x (mconcatExplicit d xs)

/-- The translation is faithful: given the instance that resolution would
find, the explicit version computes exactly what `mconcat` computes. -/
theorem mconcatExplicit_eq {α : Type} [inst : Monoid α] (xs : List α) :
    mconcatExplicit inst xs = mconcat xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih =>
    -- Goal: inst.combine x (mconcatExplicit inst xs) = x ⋄ mconcat xs
    show inst.combine x (mconcatExplicit inst xs) = x ⋄ mconcat xs
    -- Rewrite with the induction hypothesis; both sides are then identical.
    rw [ih]
```

`mconcatExplicit` takes a value `d : Monoid α`, using the class as an ordinary type, and reads its fields `d.empty` and `d.combine`, exactly like `mconcat_dict` in Rust. The theorem says that when `d` is the instance that resolution finds, the two functions agree on every list. The proof is by induction: in the base case both sides compute to `empty`, and in the step case, after `show` displays the unfolded goal, rewriting with the induction hypothesis leaves two sides that are the same by definition (`inst.combine x y` is what `x ⋄ y` means). Notice the binder `[inst : Monoid α]`: an instance argument can be given a name, so that the statement can refer to it.

Because the dictionary is just an argument, we can also pass it by hand, either to the explicit version or to `mconcat` itself, using `@` to make all its implicit arguments explicit (Chapter 12):

```
/-- info: 9 -/
#guard_msgs in
#eval mconcat [2, 3, 4]

/-- info: 24 -/
#guard_msgs in
#eval mconcatExplicit ⟨1, (· * ·)⟩ [2, 3, 4]

/-- info: 24 -/
#guard_msgs in
#eval @mconcat Nat ⟨1, (· * ·)⟩ [2, 3, 4]
```

With the dictionary `⟨1, (· * ·)⟩`, a `Monoid Nat` built with the anonymous constructor, the same list combines to $2 \cdot 3 \cdot 4 = 24$.

Instance resolution

The command `#synth` runs resolution and prints the dictionary it finds. By default it hides implicit arguments, which leaves only the name of the outermost instance; with the option `pp.explicit` it prints the whole term:

```
/-- info: instMonoidProd -/
#guard_msgs in
#synth Monoid (Nat × List String)

/-- info: @instMonoidProd Nat (List String) instMonoidNat (@instMonoidList String) -/
#guard_msgs in
set_option pp.explicit true in
#synth Monoid (Nat × List String)

-- `inferInstance` asks resolution for a dictionary, as an ordinary term.
/-- info: (5, ["a", "b"]) -/
#guard_msgs in
#eval mconcatExplicit (inferInstance : Monoid (Nat × List String)) [(2, ["a"]), (3, ["b"])]
```

Read the second output as the derivation it is: `instMonoidProd`, applied to the types `Nat` and `List String` and to the evidence for its two premises, `instMonoidNat` and `instMonoidList` (itself applied to `String`). It has the same shape as the Rust expression `pair_dict(sum_dict(), vec_dict())`. `inferInstance` asks for the same thing inside a term: it is a function that returns its instance argument, so elaborating it triggers resolution, and the result is the dictionary as a value. (If you are curious about the search itself, `set_option trace.Meta.synthInstance true` makes Lean print every goal it tries.)

When the search fails, Lean says so, just as rustc did with E0277:

```
/--
error: failed to synthesize
  Monoid Bool

Hint: Additional diagnostic information may be available using the `set_option diagnostics
true` command.
-/
#guard_msgs in
#synth Monoid Bool

/--
error: failed to synthesize instance of type class
  Monoid Bool

Hint: Type class instance resolution failures can be inspected with the `set_option
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
def allReady (flags : List Bool) : Bool := mconcat flags
```

Local instances and priorities appeared in the section on coherence: they are where Lean's design departs most from Rust's.

`Decidable` and `Inhabited`

Two classes from Lean's core library have appeared in earlier chapters, and they look different now. **`Decidable`** is the class behind `if` on propositions (Chapter 5) and behind the `decide` tactic:

```
/--
info: inductive Decidable : Prop → Type
number of parameters: 1
constructors:
Decidable.isFalse : {p : Prop} → ¬p → Decidable p
Decidable.isTrue : {p : Prop} → p → Decidable p
-/
#guard_msgs in
#print Decidable

/-- A batch of readings is *paired* when it has an even number of entries
(every measurement is taken twice). This is a proposition, not a `Bool`. -/
def Paired (xs : List Int) : Prop := xs.length % 2 = 0

/-- The instance is a decision procedure: it decides `Paired xs` for every
`xs`, producing a proof either way. Here we reuse the instance for equality
of natural numbers. -/
instance (xs : List Int) : Decidable (Paired xs) :=
  inferInstanceAs (Decidable (xs.length % 2 = 0))

/-- info: "paired" -/
#guard_msgs in
#eval if Paired [215, 218, 221, 219] then "paired" else "unpaired"

example : Paired [215, 218] := by decide
example : ¬ Paired [215] := by decide
```

`Decidable` is declared with `class inductive`: a class whose instances are values of an inductive type with two constructors. An instance of `Decidable p` is *either* a proof of `¬p` or a proof of `p`. Since an instance may depend on the arguments of the class, an instance such as `Decidable (Paired xs)`, given for every `xs`, is a **decision procedure**: an algorithm that returns one of the two proofs for each `xs`. That is why `if Paired xs then ... else ...` can run (it evaluates the instance and looks at the constructor), and why `by decide` can prove `Paired [215, 218]` (it evaluates the instance and checks that the result is `isTrue`). Our instance reuses the one for equality of natural numbers, through `inferInstanceAs`, which runs resolution at a type that is equal by definition to the one we need. A proposition without an instance cannot be used in an `if`:

```
/-- A proposition about all natural numbers has no `Decidable` instance. -/
def AllEven (f : Nat → Nat) : Prop := ∀ n, f n % 2 = 0

/--
error: failed to synthesize instance of type class
  Decidable (AllEven fun n => 2 * n)

Hint: Type class instance resolution failures can be inspected with the `set_option
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
#eval if AllEven (fun n => 2 * n) then "yes" else "no"
```

No algorithm can check `f n % 2 = 0` for infinitely many `n` by looking at them, and Lean does not pretend otherwise. Rust has no counterpart to this class: its `if` takes a `bool`, a value already computed.

`Inhabited` says that a type has a default value. It is Rust's `Default`, and `deriving Inhabited` is `#[derive(Default)]`:

```
/--
info: class Inhabited.{u} (α : Sort u) : Sort (max 1 u)
```

```
number of parameters: 1
fields:
  Inhabited.default : α
constructor:
  Inhabited.mk.{u} {α : Sort u} (default : α) : Inhabited α
-/
#guard_msgs in
#print Inhabited

structure Reading where
  sensor : String
  value : Int
  deriving Repr, Inhabited

/-- info: { sensor := "", value := 0 } -/
#guard_msgs in
#eval (default : Reading)

/-- info: (0, "", []) -/
#guard_msgs in
#eval (default : Nat × String × List Bool)
```

Chapter 10 met it as the reason why `partial def` requires its result type to be inhabited: Lean needs some value of the type to keep a possibly non-terminating definition logically harmless.

In Mathlib: `Monoid`

Mathlib defines `Monoid` too, with the same mathematics and a richer design. The file `code/lean/Book/Ch14/MathlibMonoid.lean` does not import our files, so `Monoid` there is Mathlib's:

```
/--
info: class Monoid.{u_2} (M : Type u_2) : Type u_2
number of parameters: 1
parents:
  Monoid.toOne : One M
  Monoid.toMul : Mul M
  Monoid.toSemigroup : Semigroup M
  Monoid.toMulOneClass : MulOneClass M
  Monoid.toNPow : NPow M
fields:
  One.one : M
  Mul.mul : M → M → M
  Semigroup.mul_assoc : ∀ (a b c : M), a * b * c = a * (b * c)
  MulOneClass.one_mul : ∀ (a : M), 1 * a = a
  MulOneClass.mul_one : ∀ (a : M), a * 1 = a
  NPow.npow : ℕ → M → M :=
    npowRecAuto
  Monoid.npow_zero : ∀ (x : M), x ^ 0 = 1 := by
    intros; rfl
  Monoid.npow_succ : ∀ (n : ℕ) (x : M), x ^ (n + 1) = x ^ n * x := by
    intros; rfl
constructor:
  Monoid.mk.{u_2} {M : Type u_2} [toOne : One M] [toMul : Mul M] (mul_assoc : ∀ (a b c : M),
a * b * c = a * (b * c))
    (one_mul : ∀ (a : M), 1 * a = a) (mul_one : ∀ (a : M), a * 1 = a) [toNPow : NPow M]
    (npow_zero : ∀ (x : M), x ^ 0 = 1 := by intros; rfl)
    (npow_succ : ∀ (n : ℕ) (x : M), x ^ (n + 1) = x ^ n * x := by intros; rfl) : Monoid M
field notation resolution order:
  Monoid, One, Mul, Semigroup, MulOneClass, NPow, MulOne
-/
#guard_msgs in
#print Monoid
```

Four things are worth noticing. Mathlib writes monoids *multiplicatively*, with `*` and `1`; the additive version, `AddMonoid` with `+` and `0`, is generated from the multiplicative one by an attribute called

`to_additive`. The class is assembled from smaller ones with `extends`: `Semigroup` contributes associativity, `MulOneClass` the two identity laws, `NPow` the power operation, so that a theorem that needs only associativity can be stated for every semigroup. The field `npow` (the power `x ^ n`) has a **default implementation**, `npowRecAuto`, which multiplies `n` copies; an instance may override it with something faster. And the fields `npow_zero` and `npow_succ`, the laws of `npow`, have **default proofs**, `by intros; rfl`, which work whenever `npow` is the default. Default methods and default proofs are one and the same mechanism.

Here is an instance for a type of our own, giving only `one`, `mul` and the three laws:

```
/-- Elapsed times, combined by adding them up. -/
structure Stopwatch where
  ms : ℕ
  deriving Repr

/-- The instance gives `one`, `mul` and the three laws. `npow` and its two
laws are not given: the defaults are used. -/
instance : Monoid Stopwatch where
  one := ⟨0⟩
  mul a b := ⟨a.ms + b.ms⟩
  mul_assoc a b c := by
    show Stopwatch.mk (a.ms + b.ms + c.ms) = Stopwatch.mk (a.ms + (b.ms + c.ms))
    rw [Nat.add_assoc]
  one_mul a := by
    show Stopwatch.mk (0 + a.ms) = a
    rw [Nat.zero_add]
  mul_one _ := rfl
```

In return, the whole library applies. `List.prod` (defined in Lean's core library for any type with `*` and `1`) is `mconcat` under another name; powers come from the default `npow`; and every theorem proved for monoids in general holds for `Stopwatch`, including `List.prod_append`, the counterpart of our `mconcat_append`, whose assumptions are core lawful classes that Mathlib provides for every monoid:

```
/-- info: { ms := 6 } -/
#guard_msgs in
#eval [Stopwatch.mk 1, ⟨2⟩, ⟨3⟩].prod

/-- info: { ms := 15 } -/
#guard_msgs in
#eval (⟨5⟩ : Stopwatch) ^ 3

/-- Library theorems about every monoid now apply to `Stopwatch`. -/
example (a b c : Stopwatch) : a * b * c = a * (b * c) := mul_assoc a b c

example (x : Stopwatch) (m n : ℕ) : x ^ (m + n) = x ^ m * x ^ n := pow_add x m n

example (xs ys : List Stopwatch) : (xs ++ ys).prod = xs.prod * ys.prod :=
  List.prod_append
```

Finally, Mathlib's answer to "two monoids on ℕ" is the same as Rust's: types that act as newtypes. `ℕ` has a multiplicative instance (with `*`) and an additive one (with `+`), and `Multiplicative ℕ` is a copy of `ℕ` whose `*` is the addition of `ℕ`:

```
/-- info: Nat.instMonoid -/
#guard_msgs in
#synth Monoid ℕ

/-- info: Nat.instAddMonoid -/
#guard_msgs in
```

```
#synth AddMonoid ℕ

/-- info: 24 -/
#guard_msgs in
#eval ([2, 3, 4] : List ℕ).prod

/-- info: 9 -/
#guard_msgs in
#eval ([2, 3, 4] : List ℕ).sum

-- `Multiplicative ℕ` is a copy of `ℕ` whose `*` is the addition of `ℕ`:
-- a newtype, as `Sum` was in the Rust example.
/-- info: 9 -/
#guard_msgs in
#eval ([.ofAdd 2, .ofAdd 3, .ofAdd 4] : List (Multiplicative ℕ)).prod
```

14.8 Putting it together: statistics by batch

Our walkthrough is a small data pipeline. Three instruments in a lab each produce a batch of temperature readings, in tenths of a degree. We want the count, total, minimum and maximum of all the readings; to save work, each instrument summarizes its own batch, and the summaries are merged. The question is whether merging summaries gives the same answer as summarizing everything in one pass. Monoids are exactly the tool for this question.

In Rust, the summary is a struct whose four fields are combined by four lawful monoids (sum, sum, minimum, maximum):

```
/// Summary statistics of a batch of measurements. Every field is combined by
/// a lawful monoid (sum, sum, min, max), so `Stats` is lawful too.
#[derive(Clone, Copy, Debug, PartialEq)]
struct Stats {
    count: u64,
    total: i64,
    min: i64,
    max: i64,
}

impl Stats {
    /// The statistics of a single measurement.
    fn of(x: i64) -> Stats {
        Stats {
            count: 1,
            total: x,
            min: x,
            max: x,
        }
    }
}

impl Monoid for Stats {
    fn empty() -> Self {
        Stats {
            count: 0,
            total: 0,
            min: i64::MAX,
            max: i64::MIN,
        }
    }
    fn combine(self, other: Self) -> Self {
        Stats {
            count: self.count + other.count,
            total: self.total + other.total,
            min: self.min.min(other.min),
```

```
            max: self.max.max(other.max),
        }
    }
}
```

The neutral element uses `i64::MAX` for the minimum and `i64::MIN` for the maximum, so that combining with it changes nothing. The pipeline computes the statistics both ways:

```
// Three instruments, three batches of measurements (in tenths of a degree).
let batches: Vec<Vec<i64>> = vec![
    vec![215, 218, 221, 219],
    vec![202, 199],
    vec![230, 228, 233],
];

// Each instrument summarizes its own batch...
let per_batch: Vec<Stats> = batches
    .iter()
    .map(|batch| mconcat(batch.iter().map(|&x| Stats::of(x)).collect()))
    .collect();
// ...and the summaries are merged.
let merged = mconcat(per_batch.clone());

// The same statistics, computed in one pass over all the measurements.
let all: Vec<Stats> = batches.concat().into_iter().map(Stats::of).collect();
let one_pass = mconcat(all);

println!("walkthrough:");
for (i, s) in per_batch.iter().enumerate() {
    println!("  batch {i}: {s:?}");
}
println!("  merged : {merged:?}");
println!("  one pass agrees: {}", merged == one_pass);
assert_eq!(merged, one_pass);
```

```
walkthrough:
  batch 0: Stats { count: 4, total: 873, min: 215, max: 221 }
  batch 1: Stats { count: 2, total: 401, min: 199, max: 202 }
  batch 2: Stats { count: 3, total: 691, min: 228, max: 233 }
  merged : Stats { count: 9, total: 1965, min: 199, max: 233 }
  one pass agrees: true
```

The two results agree, and the law checks reported `Stats ok (357 checks)`. That is good evidence that merging is correct for *these* batches and for the 343 triples of samples. It also has a gap that shows up only when you look for it: `total` is an `i64`, and addition of `i64` is associative only as long as it does not overflow (in a debug build an overflow panics; in a release build it wraps around).

In Lean (`code/lean/Book/Ch14/Stats.lean`), we build the same summary out of parts. Integers with addition form a lawful monoid, with the proofs taken from the library:

```
instance instMonoidInt : Monoid Int where
  empty := 0
  combine a b := a + b

instance instLawfulInt : LawfulMonoid Int where
  toMonoid := instMonoidInt
  combine_assoc := Int.add_assoc
  empty_combine := Int.zero_add
  combine_empty := Int.add_zero
```

Lean's `Int` has no largest value to serve as the neutral element of the minimum, so the minimum and the maximum are optional values, with `none` for "no readings yet":

```
/-- Combine two optional values with `f`; `none` means "no value yet" and is
the neutral element. -/
def merge (f : Int → Int → Int) : Option Int → Option Int → Option Int
  | none, y => y
  | x, none => x
  | some x, some y => some (f x y)

/-- The smallest value seen so far. -/
structure Low where
  val : Option Int

/-- The largest value seen so far. -/
structure High where
  val : Option Int

instance : Monoid Low where
  empty := ⟨none⟩
  combine a b := ⟨merge min a.val b.val⟩

instance : Monoid High where
  empty := ⟨none⟩
  combine a b := ⟨merge max a.val b.val⟩
```

The laws need a proof by cases, since each value may be `none` or `some _`:

```
-- The parent field `toMonoid` is omitted: resolution finds the instance above.
instance : LawfulMonoid Low where
  combine_assoc := by
    -- Eight cases: each of the three values is `none` or `some _`.
    rintro ⟨_ | x⟩ ⟨_ | y⟩ ⟨_ | z⟩ <;> simp [Monoid.combine, merge] <;> omega
  empty_combine := by rintro ⟨_ | x⟩ <;> rfl
  combine_empty := by rintro ⟨_ | x⟩ <;> rfl

instance : LawfulMonoid High where
  combine_assoc := by
    rintro ⟨_ | x⟩ ⟨_ | y⟩ ⟨_ | z⟩ <;> simp [Monoid.combine, merge] <;> omega
  empty_combine := by rintro ⟨_ | x⟩ <;> rfl
  combine_empty := by rintro ⟨_ | x⟩ <;> rfl
```

`rintro ⟨_ | x⟩ ⟨_ | y⟩ ⟨_ | z⟩` introduces three values of type `Low`, takes each one apart (a `Low` is a structure with one field, an `Option Int`) and splits on whether that option is `none` or `some`: eight goals in all. The combinator `<;>` runs the tactic after it on *every* goal produced by the tactic before it. `simp [Monoid.combine, merge]` unfolds the operation and simplifies; in the cases with a `none` that finishes the goal, and in the case with three values it leaves an equation about `min`, which `omega` proves (it understands `min` and `max` on integers). The identity laws hold by computation in both cases, so `rfl` proves them. The `toMonoid` field is not given: since the constructor takes it as an instance argument, resolution fills it in.

The summary is then a product of four lawful monoids, and resolution builds its lawful instance from the pair instance `instLawfulProd`, which we proved once and for all:

```
/-- Count, total, minimum and maximum: a product of four monoids. -/
abbrev Stats := Nat × Int × Low × High

/-- The statistics of a single measurement. -/
def Stats.of (x : Int) : Stats := (1, x, ⟨some x⟩, ⟨some x⟩)

/-- info: instLawfulProd -/
#guard_msgs in
#synth LawfulMonoid Stats
```

On the walkthrough's data, Lean computes the same numbers as Rust (a small `Repr` instance, defined in the file, prints the minimum and maximum compactly):

```
/-- The measurements of the Rust walkthrough, in tenths of a degree. -/
def batches : List (List Int) :=
  [[215, 218, 221, 219], [202, 199], [230, 228, 233]]

/--
info: [(4, 873, min 215, max 221), (2, 401, min 199, max 202), (3, 691, min 228, max 233)]
-/
#guard_msgs in
#eval batches.map (fun b => mconcat (b.map Stats.of))

/-- info: (9, 1965, min 199, max 233) -/
#guard_msgs in
#eval mconcat (batches.map (fun b => mconcat (b.map Stats.of)))
```

And now the statement that the Rust program could only sample: for *every* list of batches, merging the per-batch summaries equals the summary of all the readings.

```
/-- Merging per-batch statistics gives the statistics of all the data,
for every list of batches. -/
theorem stats_by_batch (bs : List (List Int)) :
    mconcat (bs.map (fun b => mconcat (b.map Stats.of)))
      = mconcat (bs.flatten.map Stats.of) := by
  -- Mapping over a flattened list = flattening the mapped batches.
  rw [List.map_flatten]
  -- mconcat of a flattened list = mconcat of the batch results.
  rw [← mconcat_batches]
  -- Two consecutive maps are one map of the composite function.
  rw [List.map_map]
  -- Both sides now apply the same function to every batch.
  rfl
```

The proof is three rewrites with general facts, and no induction. `List.map_flatten` moves the map inside the flattening: summarizing each reading of the flattened list is the same as flattening the lists of summarized readings. `← mconcat_batches` (the arrow means "use the equation from right to left") replaces the `mconcat` of a flattened list with the `mconcat` of the per-batch results. `List.map_map` fuses two consecutive maps into one. The two sides then differ only in how the function applied to each batch is written, and `rfl` confirms that they are the same function. All the real work was done by `mconcat_batches`, which holds for every lawful monoid, and by the instances, which proved the laws. This is the pattern that type classes with laws make possible: prove a theorem once, for the class, and every instance inherits it.

Note also what the Lean version does *not* model: overflow. Lean's `Int` is unbounded, so the theorem is about mathematical integers. If the Rust code must handle totals beyond the range of `i64`, it needs a different type or checked arithmetic, and the proof tells you precisely which assumption is at stake.

14.9 Common confusions

"**Traits are interfaces.**" They are related, but a trait can do several things an interface in Java or C# cannot. An impl can be added to an existing type after the fact, by the crate that owns the trait. A method may have no receiver at all, like `M::empty()`, because the instance is selected by a *type*, which can come from a parameter, a return type or an annotation, rather than by the object a method is

called on. And dispatch is static by default: there is no object carrying its methods around unless you ask for one with `dyn`.

"**A trait bound costs an indirect call.**" Not in Rust. Generic code is monomorphized, the dictionary is resolved at compile time, and calls are direct and can be inlined. Only `dyn Trait` keeps the dictionary until run time.

"**An associated type is a type parameter with a different syntax.**" An associated type is an *output*: the implementing type determines it, so there is exactly one. A type parameter is an *input*: a type may implement the trait once for each parameter. That is why `Iterator::Item` is associated (an iterator type yields one kind of item) and `From<T>` has a parameter (a type can be built from many other types).

"**`T: Monoid` guarantees that `combine` is associative.**" It guarantees that `combine` exists and has the right type. In Rust the laws are documentation and tests; only a class with proof fields, as in Lean, guarantees them.

14.10 Summary

- Parametric polymorphism runs the same code at every type; ad-hoc polymorphism runs different code at different types. Type classes (Wadler and Blott, 1989) make ad-hoc polymorphism principled: a class names operations, instances provide them, and types quantify over the members of a class, $\forall \alpha.\ \mathit{Monoid}\ \alpha \Rightarrow \tau$. Rust's traits are type classes.
- Instances are the rules of an entailment judgment $P \Vdash \pi$: axioms, or rules with premises when the instance has a context. Instance resolution is proof search with these Horn clauses. E0277 means that no derivation exists; E0283 that the program does not determine which candidate to use.
- Elaboration by dictionary passing turns a class into a record of functions, an instance into a record (or a function on records), a constraint into a parameter and a method call into a field access. The dictionary is the proof of the constraint. Rust monomorphizes the dictionaries away; `dyn Trait` keeps one at run time, attached to the value.
- Coherence (one meaning per program) requires that every constraint have essentially one derivation. Rust enforces it by forbidding overlapping impls (E0119) and orphan impls (E0117); newtypes select alternative instances. Lean allows several instances, chosen by priority and recency.
- Associated types are type-level functions determined by the implementing type (functional dependencies; Lean's `outParam`). Supertraits put the parent's dictionary inside the child's. Default methods are functions that receive their own dictionary.
- Laws are part of what a class means but not part of what Rust checks. Rust can test them on samples. A Lean class can carry them as proof fields, so that every instance must prove them and every theorem about the class applies to every instance, at no run-time cost.
- In Lean a class is a structure, and an instance argument `[C α]` is an ordinary parameter filled in by resolution; `#synth`, `inferInstance`, `@` and local instances expose the machinery. `Decidable` instances are decision procedures; `Inhabited` is Rust's `Default`. Mathlib's `Monoid` is built from smaller classes, with default methods and default proofs.

14.11 Exercises

1. Translate `fn show_all<T: Show>(xs: &[T]) -> String`, which joins the shown elements with `"; "`, into dictionary-passing style, given `trait Show { fn show(&self) -> String; }`. Then translate the instance `impl<T: Show> Show for Vec<T>` and use it to show a `Vec<Vec<i32>>`. *Hint:* the dictionary is `struct ShowDict<T> { show: Box<dyn Fn(&T) -> String> }`; the instance for vectors is a function `fn vec_show<T: 'static>(d: ShowDict<T>) -> ShowDict<Vec<T>>`, and for `Vec<Vec<i32>>` you apply it twice.
2. Write the entailment derivation, with evidence, for *Monoid* $(\mathrm{Vec}\,\texttt{Sum} \times (\texttt{String} \times \texttt{Max}))$, using the instances of `ch14_dictionary_passing.rs`. Which Rust expression is the evidence? *Hint:* the derivation has three leaves and two uses of E-Pair; the evidence starts with `pair_dict(vec_dict(), ...)`.
3. In Rust, give `Option<M>` a monoid instance for every monoid `M`, in which `None` is neutral and `Some(a) ⋄ Some(b) = Some(a ⋄ b)`. Check the laws with `check_laws` on samples that include `None`. Is the instance lawful when `M` is `Diff`? *Hint:* match on the pair `(self, other)`. For the last question, think about what `None` is neutral for, and look for a failing triple without `None`.
4. In Lean, give `Option α` the monoid of exercise 3, and prove `instance [LawfulMonoid α] : LawfulMonoid (Option α)`. *Hint:* follow the instance for `Low` in `Stats.lean`. After `rintro (_ | a) (_ | b) (_ | c)`, the call `simp [Monoid.combine, LawfulMonoid.combine_assoc]` handles all eight cases of associativity; the identity laws are `rfl` in both cases.
5. In Lean, define `pow (a : α) : Nat → α` by `pow a 0 = empty` and `pow a (n + 1) = a ⋄ pow a n`, and prove `pow a (m + n) = pow a m ⋄ pow a n` for every lawful monoid. This is the law that makes exponentiation by repeated squaring correct. *Hint:* induction on `m`. In the step case, first rewrite `m + 1 + n` as `(m + n) + 1` (prove that equation with `omega` in a `have`), then `show` the unfolded goal and use the induction hypothesis and `combine_assoc`.
6. Pen and paper. Suppose Rust allowed both `impl<T: Display> Log for T` and `impl Log for i32`. Write the two derivations of *Log* `i32` and the two dictionaries they elaborate to. What does `42.log_line()` print in each case? Which property of the type-class system fails, and which rule of Rust protects it? *Hint:* one derivation uses the blanket impl and the instance `Display for i32`; the other is an axiom. The property's name starts with "co".

CHAPTER 15

Existential Types and Dynamic Dispatch

In the last three chapters the *caller* always chose the type. When you call `mconcat::<Sum>`, or let the compiler infer the type for you, you pick one instance of a polymorphic function and use it. The function promises to work for every type you might pick; that is what $\forall$ means.

Often the roles are reversed. A function that builds an iterator chain wants to return it without naming its type, which may be impossible to write down anyway, since closures have anonymous types. A configuration file decides at run time which of several drivers a program uses, and the program wants to keep them all in one `Vec`. A library wants to hide how its data is represented, so that it can change the representation later. In each case the *callee* (or the implementer) chooses the type, and the caller is told only what it can do with it. This is the other quantifier, $\exists$: "there is some type, and here is what you may do with its values".

Existential types are the theory behind Rust's `impl Trait` in return position and `dyn Trait`, behind abstract data types and module systems, and behind the vtables that make dynamic dispatch work. They also explain an error message that puzzles many Rust programmers, the one about traits that are not "dyn compatible".

In this chapter you will learn:

- the existential type $\exists\alpha.\,\tau$, its pack and unpack rules, and why a side condition on the unpack rule is what makes a type abstract;
- why abstract data types have existential type (Mitchell and Plotkin), and how this relates to modules and information hiding;
- the three places a quantifier can go in Rust (generics, `impl Trait` in return position, `dyn Trait`), who chooses the type in each, and when;
- what a `dyn Trait` value is in memory, measured with `size_of`: a fat pointer to the value and its vtable;
- why some traits cannot be used with `dyn` (E0038), explained from the typing rules;
- `dyn Any` and downcasting: an existential that can be opened, and what that costs;
- how to encode $\exists$ with $\forall$, in Rust and in Lean, and why a Lean structure with a type field lives in `Type 1`.

15.1 Hiding a type

Who chooses

A universally quantified type $\forall\alpha.\,\tau$ is a promise made by the *implementer* of a function to its *caller*: "whatever type you choose for α, I will work". The caller chooses, the implementer must cope with any choice. An existentially quantified type $\exists\alpha.\,\tau$ reverses the roles: "I have chosen some type α, and I give you a τ that mentions it; you must cope with whatever I chose". The implementer chooses, and the client must work without knowing the choice.

Chapter 10 met the logical side of this: a proof of $\exists x.\,P(x)$ consists of a witness a and a proof of $P(a)$. In Lean it has two incarnations, the proposition `∃ x, P x` and dependent pairs such as `{ x // P x }`, which are data. An existential *type* is the same idea one level up: the witness is a type.

Pack and unpack

Let us extend the types of System F (Chapter 12) with existentials:

$$\tau ::= \ldots \mid \exists\alpha.\,\tau$$

The introduction form **packs** a type together with a value; the elimination form **unpacks** a package and gives the client a name for the hidden type and a name for the value:

$$\frac{\Gamma \vdash e : \tau[\alpha := \sigma]}{\Gamma \vdash \mathit{pack}\ \langle \sigma, e\rangle\ \mathit{as}\ \exists\alpha.\,\tau : \exists\alpha.\,\tau}\ \textsf{T-Pack}$$

$$\frac{\Gamma \vdash e_1 : \exists\alpha.\,\tau \qquad \Gamma, \alpha, x : \tau \vdash e_2 : \rho \qquad \alpha \notin \mathrm{ftv}(\rho)}{\Gamma \vdash \mathit{unpack}\ \langle \alpha, x\rangle = e_1\ \mathit{in}\ e_2 : \rho}\ \textsf{T-Unpack}$$

Rule T-Pack says: if e has the type τ with the *concrete* type σ in place of α, then you may package σ and e together and forget that the type was σ. The resulting type, $\exists\alpha.\,\tau$, no longer mentions σ; many different packages, with different hidden types, have this same type.

Rule T-Unpack is the interesting one. To use a package e_1, you write a client e_2 that must type-check with a *fresh* type variable α in the context and a variable x of type τ. Since α is a variable, the client cannot assume anything about it: it is not `u32`, not `String`, not anything in particular. The client may only use x through whatever operations τ provides. The third premise, $\alpha \notin \mathrm{ftv}(\rho)$ (the type variable α is not among the *free type variables* of the result type ρ), forbids the hidden type from escaping: the client's result must make sense without α, so it cannot be, say, the value x itself. That side condition is what makes the type *abstract*.

The computation rule opens the package and substitutes both of its parts:

$$\mathit{unpack}\ \langle \alpha, x\rangle = (\mathit{pack}\ \langle \sigma, v\rangle\ \mathit{as}\ \exists\alpha.\,\tau)\ \mathit{in}\ e_2 \to e_2[\alpha := \sigma][x := v]$$

The client was checked for an arbitrary α, so it is correct for σ in particular. Notice the asymmetry: *running* the program reveals σ, but *type-checking* the client never does.

If you have read Chapter 4, these rules will look familiar. Erase the terms, and T-Pack and T-Unpack are the introduction and elimination rules for $\exists$ in natural deduction:

$$\frac{\Gamma \vdash P(a)}{\Gamma \vdash \exists x.\,P(x)}\ \exists\,\mathsf{I} \qquad\qquad \frac{\Gamma \vdash \exists x.\,P(x) \qquad \Gamma, P(x) \vdash Q \qquad x \ \text{not free in}\ \ \Gamma\ \ \text{or}\ \ Q}{\Gamma \vdash Q}\ \exists\,\mathsf{E}$$

The elimination rule has the same shape as the one for $\vee$: to use $\exists x.\, P(x)$, show that Q follows from $P(x)$ for an arbitrary x, about which you know nothing else, and Q must not mention x. Logicians call the side condition the *eigenvariable condition*. It is the same restriction as $\alpha \notin \mathrm{ftv}(\rho)$, read in logic instead of programs.

Abstract types have existential type

Here is the existential type of a counter: some state type S, with an initial state, a way to step and a way to read the count.

$$\mathit{Counter} = \exists S.\, \{\, \mathit{init} : S,\ \mathit{step} : S \to S,\ \mathit{get} : S \to \mathbb{N} \,\}$$

One implementation packs $S := \mathbb{N}$ with 0, $n \mapsto n + 1$ and the identity. Another packs $S := \mathrm{List}\ \mathbf{1}$ (a list of units, one per step) with the empty list, "add one element" and "length". Both packages have type $\mathit{Counter}$. A client unpacks one, steps a few times and reads the count, and it cannot tell which representation it got: by rule T-Unpack, it was checked without knowing S.

John Mitchell and Gordon Plotkin made this observation in 1988, in a paper whose title states it: "Abstract types have existential type". An **abstract data type** (a type whose representation is hidden behind a set of operations) *is* an existential package, and the guarantees of data abstraction (clients cannot depend on the representation, so it can be changed without breaking them) follow from the typing rules. We will prove such a guarantee for our two counters in Lean later in the chapter.

You use this idea every day in a weaker form. A Rust module that exports a struct with private fields, and functions that operate on it, is an abstract type: code outside the module cannot see the fields. The ML family of languages goes further, with *signatures* that declare a type without revealing its definition, so that a module can be sealed to hide its representation. What existential types add is that the package is a *value*: you can choose an implementation at run time, store packages in a list, and pass them to functions.

15.2 Three places for a quantifier in Rust

All the Rust code of this section is in `code/rust/examples/ch15_existentials.rs`. The interface is a trait, and there are three implementations with three different state types:

```
/// The interface: a counter can take a step and report its value.
trait Counter {
    fn step(&mut self);
    fn get(&self) -> u32;
}

/// Counts by one.
struct Simple(u32);

impl Counter for Simple {
    fn step(&mut self) {
        self.0 += 1;
    }
    fn get(&self) -> u32 {
        self.0
    }
}
```

```
/// Counts by one and keeps a log: a different representation with the same
/// observable behavior.
struct Logged {
    count: u32,
    log: Vec<String>,
}

impl Counter for Logged {
    fn step(&mut self) {
        self.count += 1;
        self.log.push(format!("step {}", self.count));
    }
    fn get(&self) -> u32 {
        self.count
    }
}

/// Counts by a fixed stride: different behavior, too.
struct Stride {
    value: u32,
    by: u32,
}

impl Counter for Stride {
    fn step(&mut self) {
        self.value += self.by;
    }
    fn get(&self) -> u32 {
        self.value
    }
}
```

Rust lets you write code against `Counter` in three ways, and the difference between them is exactly *where the quantifier goes* and *who chooses the type*.

The caller chooses: generics

```
/// ∀C. Counter C ⇒ C → u32 → u32. The *caller* chooses `C`.
fn run_generic<C: Counter>(mut counter: C, steps: u32) -> u32 {
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}

/// The same quantifier, written with `impl Trait` in argument position.
fn run_impl(mut counter: impl Counter, steps: u32) -> u32 {
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}
```

`run_generic` has type $\forall C.\ \mathit{Counter}\ C \Rightarrow C \to \mathtt{u32} \to \mathtt{u32}$ (Chapter 14). The caller chooses `C`, and the compiler produces one copy of the function for each choice. The second function shows that **`impl Trait` in argument position is universal**: `fn run_impl(counter: impl Counter, ...)` is another way of writing `run_generic`, with an anonymous type parameter. This is a common source of confusion, since the same syntax in return position means the opposite.

```
println!(
    "generic:    {} {}",
    run_generic(Simple(0), 3),
```

```
    run_impl(Stride { value: 0, by: 10 }, 3)
);
```

```
generic:    3 30
```

The callee chooses, statically: `impl Trait` in return position

```
/// ∃C. Counter C ∧ C. The *callee* chooses the type (here `Logged`); the
/// caller only learns that it implements `Counter`.
fn make_counter() -> impl Counter {
    Logged {
        count: 0,
        log: Vec::new(),
    }
}
```

The signature says: "this function returns a value of *some* type that implements `Counter`". The function's body chooses the type (here `Logged`), and the caller gets a package of type $\exists C.\ \mathit{Counter}\ C \wedge C$: a value of an unknown type, usable only through the trait.

```
let counter = make_counter();
println!("impl Trait: {}", run_generic(counter, 3));
```

```
impl Trait: 3
```

The caller may not rely on the hidden type, even though the compiler knows it perfectly well (`code/rust/compile_fail/ch15_opaque_type.rs`):

```
fn make_counter() -> impl Counter {
    Simple(0)
}

fn peek() -> u32 {
    let counter: Simple = make_counter();
    counter.0
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch15_opaque_type.rs:27:27
   |
22 | fn make_counter() -> impl Counter {
   |                      ------------ the found opaque type
...
27 |     let counter: Simple = make_counter();
   |                  ------   ^^^^^^^^^^^^^^ expected `Simple`, found opaque type
   |                  |
   |                  expected due to this
   |
   = note:   expected struct `Simple`
           found opaque type `impl Counter`
```

Rust calls the type of the result an **opaque type**: `impl Counter` is a type distinct from every other type, which the caller can use only through `Counter`. This is rule T-Unpack at work. The client `peek` would need $\alpha = \texttt{Simple}$, and T-Unpack checks the client for an arbitrary α.

Return-position `impl Trait` is a restricted existential, in two ways. First, the witness is chosen once, *per function* (and per instantiation of the function's type parameters), not per call: all the return statements

must produce the same type, so a function cannot decide at run time between two representations (`code/rust/compile_fail/ch15_impl_two_types.rs`):

```
fn make_counter(fast: bool) -> impl Counter {
    if fast {
        Stride { value: 0, by: 10 }
    } else {
        Simple(0)
    }
}
```

```
error[E0308]: `if` and `else` have incompatible types
  --> compile_fail/ch15_impl_two_types.rs:39:9
   |
36 | /     if fast {
37 | |         Stride { value: 0, by: 10 }
   | |         -------------------------- expected because of this
38 | |     } else {
39 | |         Simple(0)
   | |         ^^^^^^^^^ expected `Stride`, found `Simple`
40 | |     }
   | |_____- `if` and `else` have incompatible types
   |
help: you could change the return type to be a boxed trait object
   |
35 - fn make_counter(fast: bool) -> impl Counter {
35 + fn make_counter(fast: bool) -> Box<dyn Counter> {
   |
help: if you change the return type to expect trait objects, box the returned expressions
   |
37 ~         Box::new(Stride { value: 0, by: 10 })
38 |     } else {
39 ~         Box::new(Simple(0))
   |
```

The compiler's suggestion, a boxed trait object, is the third placement, which we look at next. Second, the hiding is for the *programmer*, not the compiler. The compiler knows the hidden type, monomorphizes the caller for it and calls `Logged::step` directly, with no run-time cost. (There is one documented leak: auto traits such as `Send` and `Sync` "see through" an opaque type, so a caller can rely on `make_counter()` being `Send` if `Logged` is, even though the signature does not say so.)

This is the form of existential you use most, often without noticing: every function that returns an iterator chain or a closure as `impl Iterator<...>` or `impl Fn(...)` returns a package whose hidden type is too complicated, or too anonymous, to write down.

The value carries its choice: `dyn Trait`

```
/// The witness type is chosen at run time, so it can differ from call to
/// call. The value is packed together with its vtable behind a pointer.
fn make_counter_dyn(kind: &str) -> Box<dyn Counter> {
    match kind {
        "simple" => Box::new(Simple(0)),
        "logged" => Box::new(Logged {
            count: 0,
            log: Vec::new(),
        }),
        _ => Box::new(Stride { value: 0, by: 10 }),
    }
}

/// Unpacking: the client works with any hidden type, through the interface.
fn run_dyn(counter: &mut dyn Counter, steps: u32) -> u32 {
```

```
    for _ in 0..steps {
        counter.step();
    }
    counter.get()
}
```

`Box<dyn Counter>` is an existential package in the full sense. The function chooses the hidden type at run time, differently on each call; each value carries, next to the data, the dictionary (vtable) for its own type; and a client such as `run_dyn` works for every package, through the interface. Since each value carries its own witness, values with different hidden types can live in one collection:

```
// A heterogeneous collection: three different hidden types in one Vec.
let mut counters: Vec<Box<dyn Counter>> = ["simple", "logged", "stride"]
    .into_iter()
    .map(make_counter_dyn)
    .collect();
let results: Vec<u32> = counters
    .iter_mut()
    .map(|c| run_dyn(c.as_mut(), 3))
    .collect();
println!("dyn Trait:  {results:?}");
```

```
dyn Trait:  [3, 3, 30]
```

In the notation of this chapter, `Vec<Box<dyn Counter>>` is $\mathrm{Vec}\,(\exists C.\ \mathit{Counter}\ C \wedge C)$: the quantifier is *inside* the vector, once per element. Compare `Vec<impl Counter>` as a return type, $\exists C.\ \mathit{Counter}\ C \wedge \mathrm{Vec}\ C$, with the quantifier outside and one hidden type shared by all the elements.

A module with an abstract type

The trait `Counter` describes *objects*: each value comes with its own state and operations. Mitchell and Plotkin's abstract data type is a slightly different thing: one hidden type, and a package of operations that create and manipulate *any number* of values of that type. Rust can express this too, with an associated type as the hidden type:

```
/// An abstract data type in the style of Mitchell and Plotkin:
/// ∃S. { init: S, step: S → S, get: S → u32 }. The hidden type is the
/// associated type `State`; a value of a type implementing `CounterModule`
/// is the package of operations.
trait CounterModule {
    type State;
    fn init(&self) -> Self::State;
    fn step(&self, state: Self::State) -> Self::State;
    fn get(&self, state: &Self::State) -> u32;
    /// A binary operation: fine here, because every state of this module has
    /// the same hidden type.
    fn merge(&self, a: Self::State, b: Self::State) -> Self::State;
}

struct UnaryCounters;

impl CounterModule for UnaryCounters {
    /// One `()` per step: a deliberately different representation.
    type State = Vec<()>;
    fn init(&self) -> Vec<()> {
        Vec::new()
    }
    fn step(&self, mut state: Vec<()>) -> Vec<()> {
        state.push(());
        state
```

```
    }
    fn get(&self, state: &Vec<()>) -> u32 {
        state.len() as u32
    }
    fn merge(&self, mut a: Vec<()>, b: Vec<()>) -> Vec<()> {
        a.extend(b);
        a
    }
}

/// The package is returned as `impl CounterModule`: callers cannot learn
/// that the states are vectors.
fn counter_module() -> impl CounterModule {
    UnaryCounters
}

/// A client of the module. It manipulates states of type `M::State` without
/// knowing what they are.
fn two_runs<M: CounterModule>(module: &M, first: u32, second: u32) -> u32 {
    let mut a = module.init();
    for _ in 0..first {
        a = module.step(a);
    }
    let mut b = module.init();
    for _ in 0..second {
        b = module.step(b);
    }
    module.get(&module.merge(a, b))
}
```

`counter_module()` returns a package, $\exists S.\ \{init : S, \ldots\}$, in the shape of the formula at the start of the chapter. The client `two_runs` creates *two* states, manipulates them, and merges them, all without knowing that a state is a `Vec<()>`:

```
let module = counter_module();
println!("module:     {}", two_runs(&module, 2, 3));
```

```
module:     5
```

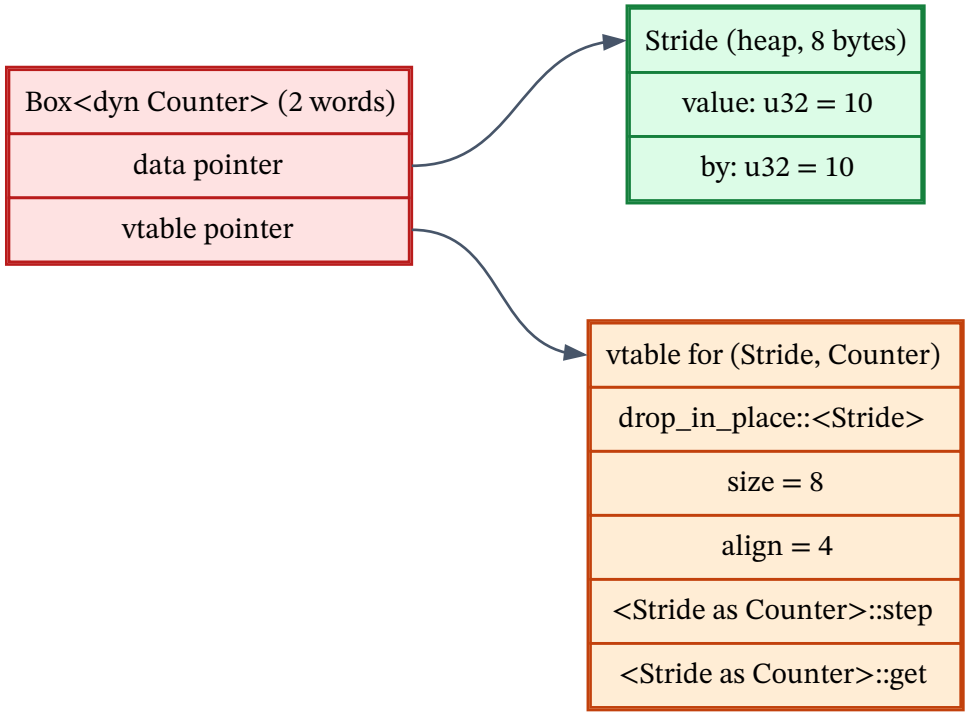
Notice `merge`, a **binary operation** on the hidden type. It is fine here, because both states come from the same package and therefore have the same hidden type. With objects, as we will see in the section on dyn compatibility, binary operations are a problem: two `Box<dyn Counter>` values may hide two different types.

Summary of the three placements

Rust	Quantifier	Who chooses	When the type is known	Dispatch
`fn f<C: Counter>(c: C)`	$\forall$	caller	compile time	static
`fn f(c: impl Counter)`	$\forall$	caller	compile time	static
`fn f() -> impl Counter`	$\exists$, one per function	callee	compile time, hidden	static
`Box<dyn Counter>`	$\exists$, one per value	whoever builds it	run time	vtable

15.3 Trait objects in memory

A value of type `dyn Counter` is a package $\langle \sigma, v \rangle$ whose type σ has been erased: nothing at run time says "this is a `Stride`". What survives of σ is the information the client needs to use v through the interface, and that is a dictionary in the sense of Chapter 14: the vtable for `Stride` as a `Counter`. A pointer to a trait object is therefore two pointers, one to the value and one to the vtable. Rust calls such a pointer a **fat pointer**.



A boxed trait object of type dyn Counter in memory: a data pointer to the value of the hidden type, and a vtable pointer to the dictionary for (Stride, Counter), which also records the hidden type's size and alignment.

The program `code/rust/examples/ch15_fat_pointers.rs` measures pointers with `size_of`, in machine words, so that the output is the same on 32-bit and 64-bit machines:

```
/// The size of `T` in machine words (8 bytes on a 64-bit machine, 4 on a
/// 32-bit one), so that the output does not depend on the platform.
fn words<T>() -> usize {
    size_of::<T>() / size_of::<usize>()
}

/// Print one row of the table.
fn row<T>(name: &str) {
    println!("{name:<26}{}", words::<T>());
}
```

```
println!("{:<26}words", "pointer type");
row::<&u32>("&u32");
row::<&[u32; 4]>("&[u32; 4]");
row::<Box<Simple>>("Box<Simple>");
row::<fn(u32) -> u32>("fn(u32) -> u32");
```

```
row::<&[u32]>("&[u32]");
row::<&str>("&str");
row::<&dyn Counter>("&dyn Counter");
row::<&mut dyn Counter>("&mut dyn Counter");
row::<Box<dyn Counter>>("Box<dyn Counter>");
row::<Rc<dyn Counter>>("Rc<dyn Counter>");
row::<Option<Box<dyn Counter>>>("Option<Box<dyn Counter>>");
row::<&dyn Fn(u32) -> u32>("&dyn Fn(u32) -> u32");
```

```
pointer type              words
&u32                      1
&[u32; 4]                 1
Box<Simple>               1
fn(u32) -> u32            1
&[u32]                    2
&str                      2
&dyn Counter              2
&mut dyn Counter          2
Box<dyn Counter>          2
Rc<dyn Counter>           2
Option<Box<dyn Counter>>  2
&dyn Fn(u32) -> u32       2
```

The first four are thin pointers: a reference to a sized type, a `Box` of one, and a function pointer are one word each. Every pointer to a `dyn` type is two words, whatever the pointer (`&`, `&mut`, `Box`, `Rc`): the second word is the vtable pointer. `Option<Box<dyn Counter>>` is still two words, because a `Box` is never null and `None` can use the null value (the niche optimization of Chapter 11). And `&dyn Fn(u32) -> u32` is two words too: a closure behind a `dyn` pointer is a trait object like any other, which is how Chapter 14's `MonoidDict` stored its functions.

Slices are fat as well: `&[u32]` and `&str` carry a length as their second word. A slice type `[u32]` is, in the same spirit, an existential over a *number*, "some n, and an array `[u32; n]`", and the hidden n travels in the pointer:

```
// A slice reference is a fat pointer too: the hidden quantity is a length.
let readings = [215u32, 218, 221, 219];
let first_two: &[u32] = &readings[..2];
let all: &[u32] = &readings;
println!(
    "slices of length {} and {}: both {} words",
    first_two.len(),
    all.len(),
    words::<&[u32]>()
);
```

```
slices of length 2 and 4: both 2 words
```

An existential over a value rather than a type is a dependent pair, a Σ-type; Chapter 30 returns to it.

The vtable

In the current implementation of Rust, a vtable starts with three entries that every trait object needs (the function that drops the hidden value, and its size and alignment) followed by one entry per method; the exact layout is not guaranteed. The size and alignment are needed because the *type* is gone: to free a `Box<dyn Counter>`, the program must tell the allocator how big the value is, and it can only find out at run time. `size_of_val` and `align_of_val` do exactly that, reading the vtable:

```
// Three packages with three different hidden types.
let mut counters: Vec<Box<dyn Counter>> = vec![
    Box::new(Small(0)),
    Box::new(Simple(0)),
    Box::new(Stride { value: 0, by: 10 }),
];
// The size and alignment of each hidden type are not known statically;
// `size_of_val` and `align_of_val` read them from the vtable.
for counter in counters.iter_mut() {
    counter.step();
    let hidden: &dyn Counter = counter.as_ref();
    println!(
        "value {:>2}: hidden type of size {} and alignment {}",
        hidden.get(),
        size_of_val(hidden),
        align_of_val(hidden),
    );
}
```

```
value  1: hidden type of size 2 and alignment 2
value  1: hidden type of size 4 and alignment 4
value 10: hidden type of size 8 and alignment 4
```

The three packages hide types of sizes 2, 4 and 8 bytes, and the program finds this out from three different vtables. A consequence is that `dyn Counter` has no size known at compile time: it is a **dynamically sized type**, which can exist only behind a pointer. The compiler rejects a parameter of type `dyn Counter` (`code/rust/compile_fail/ch15_unsized.rs`):

```
fn value_of(counter: dyn Counter) -> u32 {
    counter.get()
}
```

```
error[E0277]: the size for values of type `(dyn Counter + 'static)` cannot be known at
compilation time
  --> compile_fail/ch15_unsized.rs:10:22
   |
10 | fn value_of(counter: dyn Counter) -> u32 {
   |                      ^^^^^^^^^^^ doesn't have a size known at compile-time
   |
   = help: the trait `Sized` is not implemented for `(dyn Counter + 'static)`
help: you can use `impl Trait` as the argument type
   |
10 - fn value_of(counter: dyn Counter) -> u32 {
10 + fn value_of(counter: impl Counter) -> u32 {
   |
help: function arguments must have a statically known size, borrowed types always have a
known size
   |
10 | fn value_of(counter: &dyn Counter) -> u32 {
   |                      +
```

Its two suggestions are the two quantifiers. `impl Counter` makes the function universal, so that each caller's type is known at compile time; `&dyn Counter` keeps the existential but puts it behind a pointer that carries the vtable.

Packing and unpacking in Rust

In Rust, rule T-Pack is a **coercion**: where a `Box<Stride>` is used as a `Box<dyn Counter>`, the compiler converts it by adding the vtable pointer for `(Stride, Counter)`, a constant it generates once for each pair of type and trait. That conversion is called an *unsizing coercion*, and it happens implicitly at the

points where the expected type is known, as in the arms of `make_counter_dyn`. Rule T-Unpack is a method call: `counter.step()` on a `&mut dyn Counter` loads the `step` entry from the vtable and calls it with the data pointer. Inside `Stride::step`, `Self` is `Stride` again; the client never learns it.

The hidden type may contain references, and the existential must say how long they live. The full form of the type is `dyn Counter + 'a`: some type that implements `Counter` *and outlives* `'a`. When you omit the bound, Rust fills in a default: for `Box<dyn Counter>` it is `'static` (the box may live forever, so the hidden type must not borrow anything temporary), and for `&'a dyn Counter` it is `'a`. That is the `'static` that Chapter 14's dictionaries had to require of their type parameters. Chapter 26 is about lifetimes.

Dynamic dispatch has costs that `impl Trait` avoids: an indirect call that the compiler usually cannot inline, a heap allocation for each `Box`, and fat pointers. What it buys is the run-time choice of type, heterogeneous collections, and a single copy of the client's code instead of one per type.

15.4 Dyn compatibility

Not every trait can be used as `dyn Trait`. Here is one that cannot (`code/rust/compile_fail/ch15_dyn_incompatible.rs`):

```
trait Mergeable {
    fn total(&self) -> u32;
    fn merge(&self, other: &Self) -> Self;
    fn accept<V: Visitor>(&self, visitor: &mut V);
}

fn store(items: &mut Vec<Box<dyn Mergeable>>) {}
```

```
error[E0038]: the trait `Mergeable` is not dyn compatible
  --> compile_fail/ch15_dyn_incompatible.rs:17:30
   |
17 | fn store(items: &mut Vec<Box<dyn Mergeable>>) {}
   |                              ^^^^^^^^^^^^^ `Mergeable` is not dyn compatible
   |
note: for a trait to be dyn compatible it needs to allow building a vtable
      for more information, visit <https://doc.rust-lang.org/reference/items/traits.html#
dyn-compatibility>
  --> compile_fail/ch15_dyn_incompatible.rs:13:28
   |
11 | trait Mergeable {
   |       --------- this trait is not dyn compatible...
12 |     fn total(&self) -> u32;
13 |     fn merge(&self, other: &Self) -> Self;
   |                            ^^^^^     ^^^^ ...because method `merge` references the
`Self` type in its return type
   |                            |
   |                            ...because method `merge` references the `Self` type in this
parameter
14 |     fn accept<V: Visitor>(&self, visitor: &mut V);
   |        ^^^^^^ ...because method `accept` has generic type parameters
   = help: consider moving `merge` to another trait
   = help: consider moving `accept` to another trait
```

The rules for which traits are **dyn compatible** (older documentation says "object safe") are listed in the Rust Reference and in `rustc --explain E0038`. They can look arbitrary, but each one follows from the typing rules of existentials and from the representation we have just seen. Think of a call through

`dyn Mergeable` as an unpack: the receiver is a package $\langle \alpha, x \rangle$ with α unknown, and the call must type-check for that unknown α using only one vtable entry.

A method that returns `Self`. `merge` returns a `Self`, that is, a value of type α. After unpacking, the result type of the call would mention α, violating $\alpha \notin \mathrm{ftv}(\rho)$; and at the machine level the caller would not know how much space to reserve for the result. (A method returning `&Self` has the same problem in a milder form.)

A method that takes another `Self`. `merge` also takes `other: &Self`, a second value of the hidden type. But a client holding two `&dyn Mergeable` values holds two packages, which may hide two *different* types, a `Batch` and a `Single`; there is no way to call `merge` on them. This is the classic **binary method** problem of object-oriented type systems, and it is why the module-style existential (one package, many values of the same hidden type) can have binary operations and the object-style one cannot.

A generic method. `accept<V: Visitor>` is polymorphic: its type is $\forall V.\ \mathit{Visitor}\ V \Rightarrow \ldots$. Rust implements polymorphism by monomorphization, so the vtable would need one entry for each type `V` with which `accept` will ever be called, and the vtable is fixed when the package is created, long before anyone decides which `V` to use. A polymorphic function inside an existential package is a *rank-2* type (Chapter 12), and Rust has no single machine-code entry that implements one.

A method without a receiver. A function like `fn new() -> Self` has no `self`, so a call has no package to unpack and no vtable to read it from. (Chapter 14's `Monoid` fails here, with `empty`, and again with `combine`, which takes a second `Self`. That is no accident: a monoid is about combining values of the *same* type, which is the module-style kind of existential.)

The cure, in each case, is either to remove the method from the vtable or to change it so that one vtable entry serves every use. A bound `where Self: Sized` removes a method from `dyn` use (a `dyn` type is never `Sized`), while keeping it available on concrete types. And a generic method can be replaced by one that takes a trait object, so that the method receives the visitor's dictionary at run time instead of being monomorphized for it: dictionary passing, from Chapter 14, replacing monomorphization. The repaired trait is in `code/rust/examples/ch15_dyn_compat.rs`:

```
trait Mergeable {
    fn total(&self) -> u32;

    /// Mentions `Self`: excluded from the vtable, so it can be called on
    /// concrete types only.
    fn merge(&self, other: &Self) -> Self
    where
        Self: Sized;

    /// Takes a trait object instead of a type parameter: one vtable entry
    /// that works for every visitor.
    fn accept(&self, visitor: &mut dyn Visitor);
}
```

```
// `merge` works on concrete types, where both arguments have the same type.
let merged = Batch(vec![3, 1]).merge(&Batch(vec![4]));
println!("merged batch total: {}", merged.total());

// Through `dyn Mergeable`, `total` and `accept` are available...
let items: Vec<Box<dyn Mergeable>> = vec![
```

```
    Box::new(merged),
    Box::new(Single(9)),
    Box::new(Batch(vec![2, 6])),
];
let totals: Vec<u32> = items.iter().map(|item| item.total()).collect();
println!("totals: {totals:?}");

// ...and `accept` works with any visitor, chosen at run time too.
let mut all = Collect(Vec::new());
let mut largest = Largest(0);
for item in &items {
    item.accept(&mut all);
    item.accept(&mut largest);
}
println!("visited: {:?}, largest: {}", all.0, largest.0);
```

```
merged batch total: 8
totals: [8, 9, 8]
visited: [3, 1, 4, 9, 2, 6], largest: 9
```

`merge` is still there, for code that knows the concrete type: merging two batches gives a batch with total 8. Through `dyn Mergeable`, `total` and `accept` work on a heterogeneous collection, and `accept` works with every visitor, each passed as a `&mut dyn Visitor`.

15.5 `Any`: an existential you can open

The standard library has a trait object that breaks the rules on purpose: `dyn Any`. Every type that holds no short-lived references (every `T: 'static`) implements `Any`, and the trait's one method returns a `TypeId`, a unique identifier of the hidden type. With it, the methods `downcast_ref`, `downcast_mut` and `Box::downcast` ask a package "is your hidden type `T`?" and, if the answer is yes, return the value at type `T`. The examples are in `code/rust/examples/ch15_any.rs`.

```
/// A `&dyn Any` hides its type, but `downcast_ref` asks "is it a `T`?" and,
/// if so, returns the value at type `T`.
fn describe(value: &dyn Any) -> String {
    if let Some(n) = value.downcast_ref::<i32>() {
        format!("an i32: {n}")
    } else if let Some(s) = value.downcast_ref::<String>() {
        format!("a String of length {}", s.len())
    } else if let Some(b) = value.downcast_ref::<bool>() {
        format!("a bool: {b}")
    } else {
        "something else".to_string()
    }
}
```

```
let packages: Vec<Box<dyn Any>> = vec![
    Box::new(42i32),
    Box::new(String::from("fridge")),
    Box::new(1.5f64),
];
for p in &packages {
    println!("{}", describe(p.as_ref()));
}
```

```
an i32: 42
a String of length 6
something else
```

In terms of rule T-Unpack, `downcast_ref` is a client that does what the side condition was meant to prevent: it learns the hidden type and lets it escape. The price is that abstraction no longer holds. A client with `Any` can tell our two counter representations apart, which no client using only the interface could do.

The same leak reaches generic functions. A function of type $\forall\alpha.\,\alpha \to \alpha$ cannot do anything with its argument but return it, and Chapter 18 will make this precise: the only total function of that type that cannot inspect types is the identity. Rust gives generic functions a way to inspect types:

```
/// The signature looks like ∀T. T → T, whose only total, parametric
/// inhabitant is the identity. The bound `T: Any` is the warning sign: the
/// body may look at the type, and it does.
fn not_identity<T: Any>(mut value: T) -> T {
    if let Some(n) = (&mut value as &mut dyn Any).downcast_mut::<i32>() {
        *n += 1;
    }
    value
}
```

```
println!(
    "not_identity: {} {:?} {}",
    not_identity(41i32),
    not_identity("unchanged"),
    not_identity(41i64)
);
```

```
not_identity: 42 "unchanged" 41
```

`not_identity` returns its argument unchanged for a `&str` and an `i64`, and adds one to an `i32`. Rust is honest about this, in a way: the bound `T: Any` appears in the signature and warns the reader that the function may look at the type. Not every leak is announced, though. `std::mem::size_of::<T>()` and `std::any::type_name::<T>()` need no bound at all, and this chapter uses both: the `words` helper of `ch15_fat_pointers.rs` is a generic function whose result depends on the type, and the `TypeName` client in the section on encodings prints a type that was supposed to be hidden. Chapter 18 calls these features **parametricity leaks** and studies what they cost.

Downcasting is still useful. A map with one value per type (used, for example, to attach typed extensions to a request) is a classic application:

```
/// A map with at most one value per *type*, as used for request extensions
/// and plugin registries. The key is the hidden type's `TypeId`.
#[derive(Default)]
struct Extensions {
    values: BTreeMap<TypeId, Box<dyn Any>>,
}

impl Extensions {
    fn insert<T: Any>(&mut self, value: T) {
        self.values.insert(TypeId::of::<T>(), Box::new(value));
    }

    fn get<T: Any>(&self) -> Option<&T> {
        self.values.get(&TypeId::of::<T>())?.downcast_ref::<T>()
    }
}

struct RequestId(u64);
```

```
struct Operator(String);
```

```
let mut ext = Extensions::default();
ext.insert(RequestId(7));
ext.insert(Operator("ada".to_string()));
let id: Option<&RequestId> = ext.get::<RequestId>();
let operator: Option<&Operator> = ext.get::<Operator>();
println!("request id: {:?}", id.map(|r| r.0));
println!("operator:   {:?}", operator.map(|o| o.0.as_str()));
println!("a u8:       {:?}", ext.get::<u8>());
```

```
request id: Some(7)
operator:   Some("ada")
a u8:       None
```

And `Box<dyn Error>`, the standard way to return "some error", is an existential whose clients may downcast to the error types they know how to handle:

```
/// `Box<dyn Error>` hides the type of the error; a caller that cares can
/// downcast to the concrete error types it knows how to handle.
fn parse_reading(text: &str) -> Result<i64, Box<dyn Error>> {
    let value: i64 = text.trim().parse()?;
    if value < -2730 {
        return Err("below absolute zero".into());
    }
    Ok(value)
}

fn classify(result: Result<i64, Box<dyn Error>>) -> String {
    match result {
        Ok(v) => format!("ok: {v}"),
        Err(e) => match e.downcast_ref::<ParseIntError>() {
            Some(parse) => format!("not a number ({parse})"),
            None => format!("invalid reading ({e})"),
        },
    }
}
```

```
for text in ["215", "21.5", "-3000"] {
    println!("{text:>6}: {}", classify(parse_reading(text)));
}
```

```
  215: ok: 215
 21.5: not a number (invalid digit found in string)
-3000: invalid reading (below absolute zero)
```

The `'static` requirement of `Any` has a reason: `TypeId` identifies types after lifetimes have been erased, so it could not tell `&'a str` from `&'static str`, and a downcast between them would be unsound.

15.6 Existentials in Lean

Lean has no `exists` type former of the System F kind, because it does not need one: a dependent pair whose first component is a *type* is an existential package. The code is in `code/lean/Book/Ch15/Existential.lean`.

Packages as dependent pairs

```
/-- info: (α : Type) × (α → String) : Type 1 -/
#guard_msgs in
#check ((α : Type) × (α → String))

/-- info: (α : Type) × α × (α → String) : Type 1 -/
#guard_msgs in
#check ((α : Type) × α × (α → String))
```

The type `(α : Type) × (α → String)` is the Lean spelling of $\exists \alpha.\, \alpha \to$ `String`: a pair of a type and a function on it. It is a useless package, though: its client receives a function from α and no α to apply it to. Add a value, and you get something useful, Lean's counterpart of `Box<dyn Display>`:

```
/-- A value of some hidden type, packed with a way to display it: the
counterpart of Rust's `Box<dyn Display>`. -/
abbrev Showable := (α : Type) × α × (α → String)

/-- Packing: choose the hidden type, give a value and an implementation. -/
def readings : List Showable :=
  [⟨Nat, 42, fun n => s!"{n} ticks"⟩,
   ⟨String, "fridge", fun s => s!"sensor {s}"⟩,
   ⟨Bool, true, fun b => if b then "door open" else "door closed"⟩]

/-- Unpacking: the client sees a type `α`, a value of type `α` and a function
on `α`, and can only combine them. -/
def Showable.show (p : Showable) : String :=
  match p with
  | ⟨_, x, f⟩ => f x

/-- info: ["42 ticks", "sensor fridge", "door open"] -/
#guard_msgs in
#eval readings.map Showable.show
```

Each element of `readings` packs a different type with a value and a function on that type. `Showable.show` unpacks with a pattern match: it receives some type `α` (the underscore), a value `x : α` and a function `f : α → String`, and can only apply `f` to `x`.

Look at the type in the `#check` outputs: **`Type 1`**, not `Type`. This is the universe bump that Chapter 12 explained in general. A package contains a type, an element of the universe `Type`, and `Type` itself lives in `Type 1`; so a structure with a field of type `Type` must live in `Type 1` too. Lean's universes are *predicative*: a type that quantifies over, or contains, all the types of universe u lives in a higher universe, because allowing `Type : Type` would make the logic inconsistent (a result known as Girard's paradox).

A counter abstract data type

Here is the counter of the start of the chapter, as a Lean structure with a type field:

```
/-- The operations of a counter whose states have type `σ`. -/
structure CounterOps (σ : Type) where
  init : σ
  step : σ → σ
  get : σ → Nat

/-- An abstract data type: ∃σ. CounterOps σ. A value of this structure is
a package of a type and an implementation. -/
structure Counter where
  State : Type
  ops : CounterOps State
```

```
/-- info: Ch15.Counter : Type 1 -/
#guard_msgs in
#check Counter

/-- info: CounterOps Nat : Type -/
#guard_msgs in
#check CounterOps Nat
```

`CounterOps Nat` is an ordinary type, in `Type`; `Counter`, which contains the type `State` as a field, is in `Type 1`. The three implementations are packages with two different state types:

```
/-- Counting with a natural number. -/
def natOps : CounterOps Nat := { init := 0, step := (· + 1), get := id }

/-- Counting with a list of `()`, one per step: a different representation. -/
def unaryOps : CounterOps (List Unit) :=
  { init := [], step := (() :: ·), get := List.length }

/-- Counting by ten: a different behavior. -/
def tenOps : CounterOps Nat := { init := 0, step := (· + 10), get := id }

/-- Three packages: each hides its state type behind the `Counter` interface. -/
def byOne : Counter := ⟨Nat, natOps⟩
def unary : Counter := ⟨List Unit, unaryOps⟩
def byTen : Counter := ⟨Nat, tenOps⟩
```

Pack and unpack are ordinary functions, and the computation rule holds by definition:

```
/-- Rule T-Pack: a type and an implementation for that type make a package. -/
def Counter.pack (σ : Type) (ops : CounterOps σ) : Counter := ⟨σ, ops⟩

/-- Rule T-Unpack: the client `k` must work for every state type `σ`, and its
result type `ρ` is fixed before `σ` is in scope, so it cannot mention `σ`. -/
def Counter.unpack {ρ : Type} (c : Counter) (k : (σ : Type) → CounterOps σ → ρ) : ρ :=
  k c.State c.ops

/-- The computation rule: unpacking a package hands its contents to the client. -/
theorem Counter.unpack_pack {ρ : Type} (σ : Type) (ops : CounterOps σ)
    (k : (σ : Type) → CounterOps σ → ρ) :
    (Counter.pack σ ops).unpack k = k σ ops := rfl
```

The side condition of rule T-Unpack is enforced by scoping. The result type `ρ` is a parameter of `Counter.unpack`, fixed before the client `k` receives `σ`, so `ρ` cannot depend on `σ`. The client `k` must be a function of *every* type `σ`, which is rule T-Unpack's "fresh α". A client written against this interface works for every package, including packages with different hidden types in the same list:

```
/-- Apply `f` to `s`, `n` times. -/
def iterate {σ : Type} (f : σ → σ) : Nat → σ → σ
  | 0, s => s
  | n + 1, s => iterate f n (f s)

/-- A client: take `n` steps from the initial state and read the value. It is
written once and works for every package. -/
def Counter.run (c : Counter) (n : Nat) : Nat :=
  c.unpack (fun _ ops => ops.get (iterate ops.step n ops.init))

/-- A heterogeneous list: three packages with two different hidden types. -/
def counters : List Counter := [byOne, unary, byTen]

/-- info: [3, 3, 30] -/
#guard_msgs in
#eval counters.map (·.run 3)
```

Abstract and transparent

Inside a function that receives an arbitrary package `c`, the hidden type is `c.State`, and Lean knows nothing about it. A client that tries to treat the initial state as a number is rejected, as `peek` was in Rust:

```
/--
error: Type mismatch
  c.ops.init
has type
  c.State
but is expected to have type
  Nat
-/
#guard_msgs in
def peek (c : Counter) : Nat := c.ops.init
```

The message has the same shape as Rust's E0308 with its opaque type: `c.State` is not `Nat`. But Lean's packages are not opaque in Rust's sense. For a *specific* package, the type field is an ordinary definition, and Lean can see through it:

```
/-- For a *specific* package, Lean can see through the type field. -/
example : byOne.State = Nat := rfl

def peekOne : Nat := byOne.ops.init
```

`byOne.State` computes to `Nat`, so `byOne.ops.init` *is* a natural number. In the terminology of module systems, a dependent pair is a *transparent* package (the projections `.1` and `.2`, here `.State` and `.ops`, are always available), while System F's ∃ and Rust's `impl Trait` are *opaque*. Lean code gets abstraction from the way clients are written (a function of an arbitrary package cannot know which package it has), not from the type of the package.

The universe bump has consequences

Living in `Type 1` is not a mere technicality. Many functions and type constructors expect a type in `Type`, and a `Counter` does not qualify:

```
/--
error: Type mismatch
  Counter
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1`
-/
#guard_msgs in
def CounterType : Type := Counter

/--
error: Application type mismatch: The argument
  Counter
has type
  Type 1
of sort `Type 2` but is expected to have type
  Type
of sort `Type 1` in the application
  IO Counter
-/
#guard_msgs in
def chooseCounter : IO Counter := pure byOne
```

The second error is the one that matters in practice: Lean's `IO` takes a type in `Type`, so an `IO` action cannot return a package with a type field. Rust has no such restriction, because in Rust types are not values: a `Box<dyn Counter>` contains a vtable, not a type, and there is no universe of types for it to live in. In Lean the hidden type is a real component of the package. The compiler erases it at run time, but the logic has to account for it.

`Exists` versus `Sigma`, again

Chapter 10 contrasted the two ways of saying "there exists" in Lean. They meet again here:

```
/-- info: ∃ σ, Nonempty (CounterOps σ) : Prop -/
#guard_msgs in
#check ∃ σ : Type, Nonempty (CounterOps σ)

/-- The proposition is provable: `Nat`, with `natOps`, is a witness. -/
theorem some_counter_exists : ∃ σ : Type, Nonempty (CounterOps σ) :=
  ⟨Nat, ⟨natOps⟩⟩

/--
error: Invalid projection: Cannot project a value of non-propositional type
  Type
from the expression
  h
which has propositional type
  ∃ σ, Nonempty (CounterOps σ)
-/
#guard_msgs in
def witness (h : ∃ σ : Type, Nonempty (CounterOps σ)) : Type := h.1
```

`∃ σ : Type, Nonempty (CounterOps σ)` is a proposition, in `Prop`, and stays there even though it quantifies over all types: `Prop` is *impredicative* (Chapter 12), so no universe bump happens. But a proof of an `Exists` cannot be taken apart to produce data: the witness cannot be extracted, because of the elimination restriction of Chapter 10. The `Sigma`-style structure `Counter` pays the universe bump and in exchange gives full access to its contents. Rust's `dyn Counter` sits in between: like `Exists`, it does not let you extract the witness type (short of `Any`), and like `Sigma`, it is data that you can compute with.

Representation independence, proved

The promise of abstract types is that a client cannot tell two implementations apart if they behave the same through the interface. For our counters, counting with a `Nat` and counting with a `List Unit` should be indistinguishable. The file `code/lean/Book/Ch15/Independence.lean` proves this for every client written in a small command language:

```
/-- A client program: a list of commands. -/
inductive Cmd where
  | step
  | read

/-- Run a client program against an implementation, collecting the values it
reads. -/
def exec {σ : Type} (ops : CounterOps σ) : List Cmd → σ → List Nat
  | [], _ => []
  | .step :: cs, s => exec ops cs (ops.step s)
  | .read :: cs, s => ops.get s :: exec ops cs s
```

A client program is a list of commands; `exec` runs it against an implementation, starting from a given state, and collects the values that the `read` commands observe. The proof technique is a **simulation**:

a relation between the states of the two implementations that holds at the start, is preserved by `step`, and relates only states that `get` reports equally.

```
/-- A simulation between two implementations: a relation between their
states that holds initially, is preserved by `step`, and relates only states
that `get` cannot tell apart. -/
structure Simulation {σ τ : Type} (A : CounterOps σ) (B : CounterOps τ) where
  R : σ → τ → Prop
  init : R A.init B.init
  step : ∀ s t, R s t → R (A.step s) (B.step t)
  get : ∀ s t, R s t → A.get s = B.get t
```

```
/-- Related states produce the same outputs, for every program. -/
theorem exec_sim {σ τ : Type} {A : CounterOps σ} {B : CounterOps τ}
    (sim : Simulation A B) :
    ∀ (cs : List Cmd) (s : σ) (t : τ), sim.R s t → exec A cs s = exec B cs t := by
  intro cs
  induction cs with
  | nil =>
    -- No commands: both runs output the empty list.
    intro s t _
    rfl
  | cons c cs ih =>
    intro s t hR
    cases c with
    | step =>
      -- Both sides take a step; the new states are related again.
      exact ih (A.step s) (B.step t) (sim.step s t hR)
    | read =>
      -- Both sides output a value (equal, by `sim.get`) and continue
      -- from the same, related states.
      show A.get s :: exec A cs s = B.get t :: exec B cs t
      rw [sim.get s t hR, ih s t hR]

/-- Hence the two implementations are indistinguishable from their initial
states. -/
theorem indistinguishable {σ τ : Type} {A : CounterOps σ} {B : CounterOps τ}
    (sim : Simulation A B) (cs : List Cmd) :
    exec A cs A.init = exec B cs B.init :=
  exec_sim sim cs A.init B.init sim.init
```

The proof of `exec_sim` is by induction on the program. Notice that it introduces only `cs` before the induction, so that the induction hypothesis `ih` speaks about *all* pairs of related states, which the step case needs (the same pattern as `generalizing` in Chapter 14). For the empty program, both runs output `[]`. For a program that starts with `step`, both implementations take a step, the new states are related by `sim.step`, and the induction hypothesis applies to them. For a program that starts with `read`, both runs output a value and continue from the same states; `show` exposes the two lists, `sim.get` proves that the outputs are equal, and the induction hypothesis proves that the rest are equal. The corollary `indistinguishable` applies this to the initial states, which `sim.init` relates.

For the two counters, the relation is "the list's length is the number":

```
/-- The simulation between counting with `Nat` and counting with a list of
units: the list's length is the number. -/
def natUnary : Simulation natOps unaryOps where
  R n xs := xs.length = n
  init := rfl
  step n xs h := by
    -- h    : xs.length = n
    -- Goal : R (n + 1) (() :: xs), that is, (() :: xs).length = n + 1
    show xs.length + 1 = n + 1
    rw [h]
```

```
  get n xs h := by
    -- Goal : natOps.get n = unaryOps.get xs, that is, n = xs.length
    show n = xs.length
    rw [h]

/-- No client program can tell the two implementations apart. -/
theorem nat_unary (cs : List Cmd) :
    exec natOps cs natOps.init = exec unaryOps cs unaryOps.init :=
  indistinguishable natUnary cs
```

Each obligation is a one-line rewrite with the hypothesis `h`, after `show` has unfolded the operations (`(() :: xs).length` is `xs.length + 1` by definition). The theorem `nat_unary` then says that no program in the command language can tell the two representations apart. A counter that counts by ten, on the other hand, is told apart by a two-command program, and `decide` checks it:

```
/-- Counting by ten, on the other hand, is told apart by a two-command
program. -/
example : exec natOps [.step, .read] natOps.init ≠
    exec tenOps [.step, .read] tenOps.init := by
  decide
```

The command language makes "every client" precise enough to prove something. The full statement, for every client of type $\forall S.\, \mathit{CounterOps}\ S \to \rho$, is an instance of **parametricity**, which says that polymorphic functions preserve relations between types; Chapter 18 explains why it cannot be proved inside Lean, and Rust's `Any` shows that it fails when the language lets clients inspect types.

15.7 Encoding $\exists$ with $\forall$

There is a remarkable fact about existential types: you do not need them as a primitive, because they can be encoded with universal ones.

$$\exists \alpha.\, \tau \cong \forall \beta.\, (\forall \alpha.\, \tau \to \beta) \to \beta$$

Read the right side as a description of what you can *do* with a package: for any result type β, if you give me a client that works for every α (a function $\forall \alpha.\, \tau \to \beta$), I will run it and return its result. Packing is "remember σ and v, and when a client arrives, run it on them"; unpacking is "hand the package your client":

$$\mathit{pack}\ \langle \sigma, v \rangle = \Lambda \beta.\, \lambda k.\, k\ [\sigma]\ v \qquad \mathit{unpack}\ \langle \alpha, x \rangle = p\ \mathit{in}\ e = p\ [\rho]\ (\Lambda \alpha.\, \lambda x.\, e)$$

The side condition of T-Unpack is built in: the client is a polymorphic function, so it cannot know α, and its result type ρ is chosen before α is bound. In logic, the encoding is the fact that $\exists x.\, P(x)$ holds exactly when, for every conclusion Q, "for all x, $P(x)$ implies Q" implies Q. In programming, it is *continuation-passing style*: instead of returning a package, call the continuation with its contents.

In Rust

Rust cannot write the type $\forall \alpha.\, \tau \to \beta$ as a closure type, because closures are not generic (Chapter 13 showed E0308 when one closure is used at two types). Chapter 12's workaround applies: a rank-2 type is a trait with a generic method. The client is a *visitor*:

```
/// A client that works for *every* counter type: ∀C. Counter C ⇒ C → Output.
/// Closures cannot be generic, so the client is a trait with a generic method.
trait CounterClient {
    type Output;
    fn visit<C: Counter>(self, counter: C) -> Self::Output;
}

/// The existential, encoded with a universal: instead of returning a counter
/// of a hidden type, take a client and hand it the counter.
/// ∃C. Counter C ∧ C  ≅  ∀R. (∀C. Counter C ⇒ C → R) → R
fn with_counter<K: CounterClient>(kind: &str, client: K) -> K::Output {
    match kind {
        "simple" => client.visit(Simple(0)),
        "logged" => client.visit(Logged {
            count: 0,
            log: Vec::new(),
        }),
        _ => client.visit(Stride { value: 0, by: 10 }),
    }
}

/// One client: take some steps and report the value.
struct StepAndGet(u32);

impl CounterClient for StepAndGet {
    type Output = u32;
    fn visit<C: Counter>(self, counter: C) -> u32 {
        run_generic(counter, self.0)
    }
}

/// Another client, with a different result type: the name of the hidden type,
/// which this client may print but not use.
struct TypeName;

impl CounterClient for TypeName {
    type Output = &'static str;
    fn visit<C: Counter>(self, _counter: C) -> &'static str {
        std::any::type_name::<C>()
    }
}
```

`with_counter` is the package, in encoded form. It does not return a counter; it receives a client and calls `client.visit` with the counter it chooses. Each arm of the `match` calls `visit` at a *different* type, which is fine, because `visit` is generic, and the compiler monomorphizes it three times. So this encoding achieves what `impl Counter` could not (choosing the hidden type at run time) without a `Box` or a vtable:

```
for kind in ["simple", "logged", "stride"] {
    let value = with_counter(kind, StepAndGet(3));
    let name = with_counter(kind, TypeName);
    println!("cps:        {kind:<7} -> {value:>2}  (hidden type {name})");
}
```

```
cps:        simple  ->  3  (hidden type ch15_existentials::Simple)
cps:        logged  ->  3  (hidden type ch15_existentials::Logged)
cps:        stride  -> 30  (hidden type ch15_existentials::Stride)
```

The two clients have different result types (`u32` and `&'static str`): that is the $\forall\beta$ of the encoding, here the associated type `Output`. The second client, `TypeName`, shows the parametricity leak mentioned above: `type_name` prints the hidden type, which the client was not supposed to know. The price of the encoding is inversion of control: every client must be written as a type that implements a trait,

rather than as a closure. The `serde` library's `Visitor` traits are a well-known use of this pattern. Note also that the encoding could not be turned into a trait object: a trait with a generic method is not dyn compatible.

In Lean

In Lean, the encoding is a definition, and its properties are theorems (`code/lean/Book/Ch15/Encoding.lean`):

```
/-- ∃α. τ α, encoded: for every result type `ρ`, given a client that works
for every `α`, produce a `ρ`. The result types range over `Type u`. -/
def Hidden (τ : Type → Type) : Type (u + 1) :=
  (ρ : Type u) → ((α : Type) → τ α → ρ) → ρ

/-- Pack: remember `α` and `x`; when a client arrives, run it on them. -/
def Hidden.pack {τ : Type → Type} (α : Type) (x : τ α) : Hidden.{u} τ :=
  fun _ k => k α x

/-- Unpack: hand the client to the package. -/
def Hidden.unpack {τ : Type → Type} {ρ : Type u} (p : Hidden.{u} τ)
    (k : (α : Type) → τ α → ρ) : ρ :=
  p ρ k

/-- The computation rule holds by definition. -/
theorem Hidden.unpack_pack {τ : Type → Type} {ρ : Type u} (α : Type) (x : τ α)
    (k : (α : Type) → τ α → ρ) :
    (Hidden.pack α x).unpack k = k α x := rfl
```

`Hidden τ` is $\forall \rho.\, (\forall \alpha.\, \tau\ \alpha \to \rho) \to \rho$, with the result types ρ ranging over the universe `Type u`. `Hidden.pack` and `Hidden.unpack` are the two definitions above, and the computation rule of existentials, unpacking a package hands its contents to the client, holds by `rfl`: it is literally β-reduction. Here is Rust's `with_counter`, with the counter operations of `Existential.lean`:

```
/-- The Rust function `with_counter`, in Lean: the package is chosen at run
time, and each branch packs a different state type. -/
def withCounter (fast : Bool) : Hidden.{0} CounterOps :=
  if fast then Hidden.pack Nat tenOps
  else Hidden.pack (List Unit) unaryOps

/-- A client: three steps, then read. It must work for every state type. -/
def stepThree (σ : Type) (ops : CounterOps σ) : Nat :=
  ops.get (ops.step (ops.step (ops.step ops.init)))

/-- info: (30, 3) -/
#guard_msgs in
#eval ((withCounter true).unpack stepThree, (withCounter false).unpack stepThree)
```

Universes appear again. The encoding quantifies over all result types in `Type u`, so it lives one universe higher:

```
/-- info: Hidden CounterOps : Type 1 -/
#guard_msgs in
#check Hidden.{0} CounterOps

/-- info: Hidden CounterOps : Type 2 -/
#guard_msgs in
#check Hidden.{1} CounterOps
```

With ρ ranging over `Type` (universe level 0), `Hidden CounterOps` is in `Type 1`, the same universe as the dependent pair. Can we convert between the two? One direction is easy, but the other needs the

client to return a *dependent pair*, a type in `Type 1`, so the result types must range over `Type 1`, and the encoding moves up to `Type 2`:

```
/-- From a dependent pair to the encoding. The result types must range over
`Type 1`, because the pair type itself lives in `Type 1`. -/
def toHidden {τ : Type → Type} : ((α : Type) × τ α) → Hidden.{1} τ
  | ⟨α, x⟩ => Hidden.pack α x

/-- Back: use the pair type itself as the result type `ρ`. -/
def fromHidden {τ : Type → Type} (p : Hidden.{1} τ) : (α : Type) × τ α :=
  p.unpack (fun α x => ⟨α, x⟩)

/-- One round trip is the identity, by computation. -/
theorem fromHidden_toHidden {τ : Type → Type} (s : (α : Type) × τ α) :
    fromHidden (toHidden s) = s := by
  -- Split the pair into its type and its value; then both sides compute
  -- to the same pair.
  cases s
  rfl
```

Going from a pair to the encoding and back gives the pair you started with, and the proof is a computation: after `cases` splits the pair into its type and its value, both sides reduce to the same pair. The other round trip, from an encoded package `p` to a pair and back, should also give `p` back, but proving it requires knowing that `p` treats its client's type parameter uniformly. That is a free theorem, true of every `p` you can write without inspecting types, and, as Chapter 18 explains, not provable inside Lean. In System F, whose polymorphism is impredicative, the $\forall\beta$ ranges over all types, including the encoding itself, so everything stays in one universe (and, with parametricity, the two sides are isomorphic); in a predicative hierarchy like Lean's, each step up costs a universe.

In `Prop`, which is impredicative, none of these problems arise, and the encoding is a theorem in both directions:

```
/-- In `Prop`, the encoding is an equivalence, and both directions are
provable: a proof of `∃ x, P x` is exactly something that can serve every
client `∀ x, P x → Q`. -/
theorem exists_iff_forall {α : Type} (P : α → Prop) :
    (∃ x, P x) ↔ ∀ Q : Prop, (∀ x, P x → Q) → Q := by
  constructor
  · -- Unpack the witness and hand it to the client `k`.
    intro ⟨x, hx⟩ Q k
    exact k x hx
  · -- Choose the client that packs its argument again.
    intro h
    exact h (∃ x, P x) (fun x hx => ⟨x, hx⟩)

/-- info: 'Ch15.exists_iff_forall' does not depend on any axioms -/
#guard_msgs in
#print axioms exists_iff_forall
```

From left to right: take the witness `x` and its proof `hx` apart, and hand them to the client `k`. From right to left: the hypothesis `h` works for every conclusion `Q`, so choose `Q := ∃ x, P x` and the client that packs its arguments again. The proof uses no axioms at all. Since proofs of propositions carry no information beyond their existence, the round trip is automatic.

15.8 Putting it together: choosing where the quantifier goes

Let us use the chapter's ideas on a concrete design question. A lab control program reads a configuration string that names a counter implementation (`"simple"`, `"logged"` or `"stride"`), and must run the chosen counter. Which Rust feature should express "the counter chosen by the configuration"?

A generic parameter does not fit. `fn run<C: Counter>(c: C)` lets the *caller* choose `C`, at compile time. Here the choice is made by a string, at run time, inside the function that reads the configuration.

`impl Trait` in return position does not fit either. A function `fn from_config(name: &str) -> impl Counter` must return one type, decided at compile time, and the compiler rejects two arms that return different types (E0308, as we saw). The existential is there, but its witness is fixed per function.

`dyn Trait` fits. `make_counter_dyn` returns a `Box<dyn Counter>`: each call may choose a different type, each value carries its vtable, and any client of `&mut dyn Counter` can use it. The costs are a heap allocation, an indirect call per method, and the restriction that `Counter` must be dyn compatible. That is the usual answer, and the compiler's own suggestion.

The encoding fits too. `with_counter` takes a `CounterClient` and calls it with the chosen counter: three monomorphized calls, no allocation, no vtable, and no dyn-compatibility requirement. The price is that the code that uses the counter must be written as a client type with a generic method, and that the counter cannot outlive the call.

And if the counters must be combined, as in `merge`, neither objects nor the encoding are enough: binary operations need several values with *the same* hidden type, which is the module-style existential, a trait with an associated type such as `CounterModule`.

Whichever you choose, the Lean side tells you what is guaranteed. A client that uses only the interface cannot distinguish `Simple` from `Logged` (in Lean, `natOps` from `unaryOps`), because there is a simulation between them, and `nat_unary` proves that no program in the command language observes a difference. A client that uses `Any`, `type_name` or `size_of` can, and the `TypeName` client did: abstraction holds only as long as clients play by the rules of T-Unpack.

15.9 Common confusions

"**`impl Trait` always means an existential.**" Only in return position. In argument position, `fn f(x: impl Trait)` is a generic function: the caller chooses the type. The same syntax denotes $\forall$ in one position and $\exists$ in the other.

"**`impl Trait` hides the type from the compiler.**" It hides it from the *caller's code*. The compiler knows it, generates direct calls, and even lets auto traits such as `Send` leak through. Only `dyn Trait` erases the type at run time.

"**`dyn Trait` is a type like any other.**" It is a dynamically sized type: it has no size known at compile time, so you can only use it behind a pointer (`&dyn Trait`, `Box<dyn Trait>`, `Rc<dyn Trait>`), and that pointer is two words long.

"**A trait that is not dyn compatible is badly designed.**" It may simply describe a module-style abstraction rather than objects. `Clone`, `Default`, `PartialEq` with its binary `eq`, and Chapter 14's `Monoid` are all excellent traits that are not dyn compatible, because their methods return `Self`, lack a receiver, or take a second `Self`.

"**A Lean structure with a type field hides that type.**" It does so only from code that receives an arbitrary package. For a specific package, the field computes to the actual type. Lean's dependent pairs are transparent; Rust's opaque types are not.

15.10 Summary

- The existential type $\exists\alpha.\,\tau$ packages a hidden type with a value. Rule T-Pack forgets the type; rule T-Unpack lets a client use the package for an arbitrary α, with the side condition $\alpha \notin \mathrm{ftv}(\rho)$ (the eigenvariable condition of ∃E) keeping the type from escaping. That side condition is what makes types abstract.
- Abstract data types have existential type (Mitchell and Plotkin, 1988), and representation independence is the guarantee that clients cannot distinguish implementations related by a simulation; we proved it in Lean for two counters and every program of a command language.
- Rust offers three placements. Generics and `impl Trait` in argument position are ∀ (the caller chooses). `impl Trait` in return position is ∃ with one statically known, opaque witness per function. `dyn Trait` is ∃ with a witness per value, chosen at run time. A trait with an associated type gives module-style existentials.
- A pointer to a `dyn Trait` is fat: a data pointer and a vtable pointer (two words, as `size_of` shows). The vtable is the dictionary of Chapter 14 plus the drop function, size and alignment of the erased type. Slices are fat pointers too, hiding a length.
- A trait is dyn compatible only if every method can be called through a single vtable entry on one unpacked value: no `Self` in the result or in other arguments, no type parameters, a receiver. `where Self: Sized` and trait-object parameters are the standard repairs (E0038).
- `dyn Any` and downcasting reveal the hidden type at run time. They are useful (type maps, error inspection) and they break abstraction and parametricity (Chapter 18), as do `size_of` and `type_name`.
- $\exists\alpha.\,\tau \cong \forall\beta.\,(\forall\alpha.\,\tau \to \beta) \to \beta$: in Rust, a client trait with a generic method gives run-time choice of type without a vtable; in Lean, the encoding computes, one round trip is `rfl`, the other is a free theorem, and each step up in the encoding costs a universe. In `Prop` the encoding is a theorem in both directions.
- In Lean, packages are dependent pairs such as `(α : Type) × α × (α → String)` or structures with a type field. They live in `Type 1`, which has practical consequences (`IO Counter` is ill-typed). `Exists` stays in `Prop` but cannot release its witness.

15.11 Exercises

1. For each signature, say whether the type is universally or existentially quantified, who chooses it, and when it is known: `fn a(x: impl Iterator<Item = u8>)`, `fn b() -> impl Iterator<Item = u8>`, `fn c() -> Box<dyn Iterator<Item = u8>>`, `fn d<I: Iterator<Item = u8>>(x: I)`. *Hint:* two are universal and identical in meaning; the two existentials differ in when the hidden type is fixed.
2. Explain, using rule T-Unpack, why a method `fn duplicate(&self) -> Self` makes a trait not dyn compatible, and why adding `where Self: Sized` to that method makes the trait dyn compatible again. Can `duplicate` then be called on a `Box<dyn Trait>`? *Hint:* look at the result type of the unpacked call; `dyn Trait` is never `Sized`, so the method is not in the vtable.
3. In Rust, write the CPS encoding of `Box<dyn Display>`: a trait `ShowClient { type Output; fn visit<T: Display>(self, value: T) -> Self::Output; }` and a function `with_reading(kind: &str, client: K) -> K::Output` that chooses between an `i32`, a `String` and an `f64`. Write a client that formats the value inside brackets. *Hint:* follow `with_counter` and `StepAndGet`; the client's `visit` can use `format!("[{value}]")` because of the `Display` bound.
4. In Lean, define `twoOps : CounterOps Nat` whose state counts by two and whose `get` divides by two, and prove that it is indistinguishable from `natOps` by constructing a `Simulation natOps twoOps`. *Hint:* take `R n m := m = 2 * n`. Each obligation becomes a goal about `Nat` after `show`; `omega` proves all of them, including the one with division.
5. In Lean, use `nat_unary` to prove that `byOne.run n = unary.run n` for every `n`. *Hint:* first prove a lemma relating `exec` on the program `List.replicate n .step ++ [.read]` to `iterate`, by induction on `n` generalizing the starting state; then both sides of the goal are the single element of an output list.
6. Pen and paper. Why does `∃ σ : Type, Nonempty (CounterOps σ)` live in `Prop` while `(σ : Type) × CounterOps σ` lives in `Type 1`? What would go wrong if `Sigma` over `Type` were allowed to live in `Type`? *Hint:* recall from Chapter 12 which of Lean's universes is impredicative, and the remark on Girard's paradox in this chapter.

PART IV

Categories

Part IV steps back and looks at the structure that all the previous chapters share. Category theory is the mathematics of composition: objects, arrows between them, and laws about how arrows combine. Types and functions form a category, and so do propositions and proofs. Functors explain `map`, natural transformations explain why generic functions are so well behaved, cartesian closed categories are what the simply typed λ-calculus *is*, and monads explain `?`, `and_then` and `do` notation. The definitions are written in Lean, with the laws as proof obligations, and every instance is proved lawful.

CHAPTER 16

Categories for Programmers

Look at the last program you wrote and count how often you composed two things. An iterator chain such as `lines.iter().map(parse).filter(valid).collect()` is four steps glued end to end. A web server passes a request through a stack of middleware. A compiler is a pipeline: lexer, parser, type checker, code generator. Even the humblest refactoring, extracting three lines into a helper function, relies on a fact so obvious that nobody states it: gluing `f` to `g` and then to `h` gives the same program however you group the steps. Composition is what programmers do all day.

Category theory is the mathematics of composition. It studies things that can be glued end to end, and deliberately forgets what they are made of. The effect is the opposite of vagueness: by looking only at how arrows compose, category theory finds definitions that work in many settings at once. One definition of "product" gives you pairs of values, the minimum of two numbers and the conjunction of two propositions. One definition of "terminal object" gives you `()`, `True` and the largest element of an order. Part IV uses this language to explain `map` (Chapter 17), generic functions (Chapter 18), closures and the λ-calculus (Chapter 19), and `?` with `and_then` (Chapter 20). This chapter lays the foundation.

In this chapter you will learn:

- what a category is: objects, morphisms, identities, composition and the three laws, and how the book's `Category` class states them in Lean;
- many small examples: types and functions, preorders, monoids, discrete categories and the free category of paths in a graph, in Lean and in Rust;
- how to read commutative diagrams, and why a commuting square is a property test with a proof;
- isomorphisms, monomorphisms and epimorphisms, and what they are in the category of types;
- initial and terminal objects, products and coproducts, defined by universal properties and proved in Lean for `Empty`, `Unit`, pairs and sums, together with their uniqueness up to unique isomorphism;
- duality: the opposite category, and how it turns every theorem into two;
- how Mathlib's category theory library writes all of this (`⟶`, `≫`, `𝟙`, `Type u` as a category, products through `Limits`).

16.1 Composition is what programmers do

Here is composition in Rust, written once for all types. The full program is in `code/rust/examples/ch16_categories.rs`.

```
/// `compose(g, f)` is "first `f`, then `g`": the mathematician's `g ∘ f`.
fn compose<A, B, C>(g: impl Fn(B) -> C, f: impl Fn(A) -> B) -> impl Fn(A) -> C {
    move |a| g(f(a))
}

/// The identity morphism on every type `A`.
fn identity<A>(a: A) -> A {
    a
}
```

`compose(g, f)` turns a function from `A` to `B` and a function from `B` to `C` into a function from `A` to `C`. The argument order follows the mathematical notation $g \circ f$, read "g after f": the function on the right runs first (Mathlib uses the opposite convention, as we will see at the end of the chapter). The types must *meet*: the output type of `f` must be the input type of `g`. `identity` does nothing, for every type. Two facts about these functions are so familiar that they are easy to overlook:

- composing with `identity`, on either side, changes nothing;
- when you compose three functions, it does not matter which two you glue first.

The example checks both facts on a few inputs:

```
let len = |s: &str| s.len();
let double = |n: usize| 2 * n;
let is_even = |n: usize| n % 2 == 0;
let samples = ["", "a", "hello", "category"];
for s in samples {
    // The unit laws: composing with the identity changes nothing.
    assert_eq!(compose(identity, len)(s), len(s));
    assert_eq!(compose(len, identity)(s), len(s));
    // Associativity: the bracketing does not matter.
    let left = compose(compose(is_even, double), len);
    let right = compose(is_even, compose(double, len));
    assert_eq!(left(s), right(s));
}
println!("functions: laws hold on {} samples", samples.len());
println!(
    "functions: (double ∘ len)(\"hello\") = {}",
    compose(double, len)("hello")
);
```

```
functions: laws hold on 4 samples
functions: (double ∘ len)("hello") = 10
```

The second fact is the one your refactorings rely on. If a pipeline does `len`, then `double`, then `is_even`, you may extract the first two steps into a helper (`compose(is_even, compose(double, len))`) or the last two (`compose(compose(is_even, double), len)`), and the program means the same thing. That is **associativity**. The first fact says that `identity` is a neutral element for composition, like `0` for addition: the **unit laws**.

The checks never looked at what `len` or `double` compute, only at their source and target types, at how they compose, and at the identity. Category theory keeps exactly this and throws away the rest.

16.2 The definition of a category

A **category** $\mathcal{C}$ consists of:

- a collection of **objects** $a, b, c, \ldots$;
- for every two objects a and b, a collection $\mathrm{Hom}(a, b)$ of **morphisms** (also called **arrows**) from a to b; we write $f : a \to b$ for $f \in \mathrm{Hom}(a, b)$, and call a the *source* and b the *target* of f;
- for every object a, an **identity** morphism $\mathrm{id}_a : a \to a$;
- for every $f : a \to b$ and $g : b \to c$, a **composite** $g \circ f : a \to c$.

These must satisfy three laws, for all $f : a \to b$, $g : b \to c$ and $h : c \to d$:

$$\mathrm{id}_b \circ f = f \qquad f \circ \mathrm{id}_a = f \qquad h \circ (g \circ f) = (h \circ g) \circ f$$

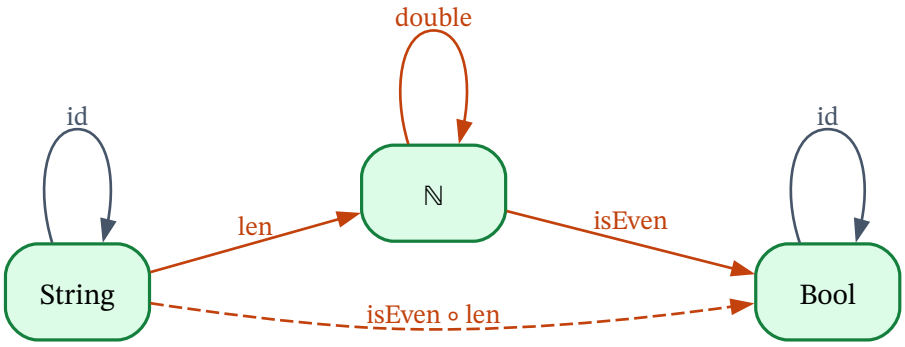
That is the whole definition. Three remarks will save you confusion later.

First, composition is only defined when the arrows meet: $g \circ f$ exists only if the target of f is the source of g. This is exactly the type-checking rule of `compose`.

Second, **objects need not be sets, and morphisms need not be functions.** In the examples below you will meet categories whose morphisms are proofs of inequalities, elements of a monoid, and paths in a graph. Morphisms are whatever the category says they are, as long as they compose and obey the laws.

Third, category theory never looks *inside* an object. Everything it says about an object, it says through the arrows that go in and out of it. This will turn out to be a strength: it is why the same definitions work in so many places.

A small picture helps. The diagram shows three objects of the category of types and some of the arrows between them: the identities (drawn as loops), `len`, `isEven`, their composite, and `double`, an arrow from `ℕ` to itself that is not the identity.



A few objects and morphisms of the category of types. Loops are morphisms from an object to itself; the dashed arrow is the composite of the two solid arrows it bypasses.

The book's `Category` class

Here is the definition in Lean. It lives in `code/lean/Book/Ch16/Category.lean`, the module that all the chapters of Part IV share:

```
/-- A category: objects, morphisms between them, an identity morphism on every
object and an associative composition for which the identities are units. -/
class Category (Obj : Type u) where
  /-- The morphisms from `a` to `b`. -/
  Hom : Obj → Obj → Type
  /-- The identity morphism of `a`. -/
  id : (a : Obj) → Hom a a
  /-- Composition in the order of function composition: `comp g f` is
  "first `f`, then `g`". -/
  comp : {a b c : Obj} → Hom b c → Hom a b → Hom a c
  /-- The identity is a left unit for composition. -/
  id_comp : ∀ {a b : Obj} (f : Hom a b), comp (id b) f = f
  /-- The identity is a right unit for composition. -/
  comp_id : ∀ {a b : Obj} (f : Hom a b), comp f (id a) = f
  /-- Composition is associative. -/
  assoc : ∀ {a b c d : Obj} (h : Hom c d) (g : Hom b c) (f : Hom a b),
    comp h (comp g f) = comp (comp h g) f
```

Let us read it field by field.

- `class Category (Obj : Type u)` declares a type class (Chapter 14) indexed by the type of objects. An *instance* `Category Obj` says how the elements of `Obj` form a category. The universe variable `u` (Chapter 12) allows `Obj` to be big: the type of all types, `Type`, itself lives in `Type 1`, and we want `Type` to be a category.
- `Hom : Obj → Obj → Type` is the heart of the definition: for every two objects, a *type* of morphisms, which depends on the *values* `a` and `b` (a dependent type, in the sense of Chapter 1). This is why "composition only when the arrows meet" is checked by Lean's type checker for free.
- `id` and `comp` are the data: an identity for every object, and a composition for every two arrows that meet. The objects of `comp` are implicit arguments (in braces), because Lean can read them off the arrows. As the comment says, `comp g f` means "first `f`, then `g`", like $g \circ f$.
- `id_comp`, `comp_id` and `assoc` are the three laws, as **fields**: an instance is rejected until it supplies proofs of them. This is the difference between a Lean class and a Rust trait that Chapter 14 pointed out: a trait can list operations but not demand laws.

Why does `Hom` land in `Type`, rather than in an arbitrary universe as in Mathlib? For simplicity: every category in this book has morphisms that fit in `Type`, and a fixed universe keeps universe annotations out of every statement. The price is that a category whose arrows between two objects are too many to form a `Type` cannot be an instance; we will not need one. The choice also has a pleasant consequence that Chapter 17 uses: since `Hom a b` is a type, it is an object of the category of types.

Two notations make statements readable:

```
/-- `a ⟶ b` is the type of morphisms from `a` to `b`. -/
scoped infixr:10 " ⟶ " => Category.Hom
/-- `g ⊚ f` is the composite "`f`, then `g`". -/
scoped infixr:80 " ⊚ " => Category.comp
```

`a ⟶ b` (a long arrow, typed `\hom` in the editor) is the type of morphisms from `a` to `b`, and `g ⊚ f` (typed `\oo`) is their composite. Both are `scoped`: they are active inside `namespace Ch16` and in files that `open Ch16`, and nowhere else, so that they do not leak into the rest of the book. `infixr` makes them right-associative, and the numbers are precedences: `⟶` binds more loosely than almost everything, like `→`.

The category of types

The first instance is the one we started with:

```
/-- The category whose objects are Lean types and whose morphisms are functions. -/
instance typeCategory : Category Type where
  Hom α β := α → β
  id _ := fun x => x
  comp g f := fun x => g (f x)
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

The objects are the Lean types in `Type`, the morphisms from `α` to `β` are the functions `α → β`, the identity is `fun x => x`, and composition is function composition. All three laws are proved by `rfl`. Why does that work? Take `id_comp`: the left side, `comp (id b) f`, unfolds to `fun x => (fun y => y) (f x)`, which computes (by β-reduction) to `fun x => f x`. Lean's definitional equality includes η for functions, so `fun x => f x` *is* `f`. Associativity is similar: both sides compute to `fun x => h (g (f x))`. The laws of the category of types hold "by computation", which is why programmers never notice them.

With the instance in place, functions can be written and composed as morphisms. The file `code/lean/Book/Ch16/Examples.lean` starts with:

```
/-- In `typeCategory`, a morphism `String → ℕ` is just a function. -/
def len : String → ℕ := String.length

def double : ℕ → ℕ := fun n => 2 * n

def isEven : ℕ → Bool := fun n => n % 2 == 0

/-- info: 10 -/
#guard_msgs in
#eval (double ∘ len) "hello"

/-- info: true -/
#guard_msgs in
#eval (isEven ∘ double ∘ len) "abc"
```

The declared type `String → ℕ` unfolds to `String → ℕ`, so an ordinary function can be its value. The composite `isEven ∘ double ∘ len` parses as `isEven ∘ (double ∘ len)` and runs from right to left: the length of `"abc"` is 3, doubled is 6, which is even. Because the laws hold by computation, specific instances of them are `rfl` too:

```
/-- The laws hold by `rfl`: both sides are the same function, up to unfolding. -/
example : (isEven ∘ double) ∘ len = isEven ∘ (double ∘ len) := rfl
example : double ∘ Category.id ℕ = double := rfl
```

Rust types and functions, with caveats

Do Rust's types and functions (or closures) form a category in the same way, with `identity` and `compose`? Approximately, and the approximation is instructive, because a Rust function is more than a mapping from inputs to outputs:

- **Side effects.** A function can print or mutate shared state. Composition stays associative, but "two morphisms are equal" now has to mean "the same outputs *and* the same effects", and we will see that this breaks one of the nicest facts about `()`.

- **Panics and non-termination.** `|_| panic!()` and `|_| loop {}` have type `A -> B` for every `B`, but never return a `B`.
- **Equality of functions is not checkable.** Lean proves `f = g` with `funext` (functions that agree on every input are equal); a Rust test compares outputs on samples.
- **Ownership.** A function `fn(A) -> B` consumes its argument. Composition is unaffected, but products will show where it matters.

The usual way out is to reason about the *pure, total* fragment: functions that always return and have no observable effects. Within it, Rust types and functions form a category as honestly as Lean's do, and we will point out each place where stepping outside changes the answer.

16.3 A gallery of categories

The category of types is only one example, and if it were the only one, category theory would be pointless. This section collects small categories that look nothing like it. Each comes in Lean (as an instance of the book's class, with its laws proved) and most come in Rust too.

Categories as data, in Rust

In Rust we cannot write `Hom(a, b)` as a *type* that depends on the *values* `a` and `b`: that needs dependent types, the subject of Chapter 30. What we can do is represent a category whose objects and morphisms are runtime values, and check at run time that two morphisms meet before composing them:

```
/// A category whose objects and morphisms are runtime values. Rust cannot make
/// the *type* of a morphism depend on the *values* of its source and target
/// (that needs dependent types, Chapter 30), so `compose` checks at run time
/// that the morphisms meet, and returns `None` if they do not.
trait Category {
    type Obj: Copy + PartialEq + Debug;
    type Mor: Clone + PartialEq + Debug;

    fn source(&self, f: &Self::Mor) -> Self::Obj;
    fn target(&self, f: &Self::Mor) -> Self::Obj;
    fn id(&self, a: Self::Obj) -> Self::Mor;
    /// `compose(g, f)`: first `f`, then `g`.
    fn compose(&self, g: &Self::Mor, f: &Self::Mor) -> Option<Self::Mor>;
}
```

With the trait, a single generic function can test the laws of every instance on sample morphisms. It checks the two unit laws for every sample, and associativity for every triple of samples that can be composed:

```
/// Checks the unit laws for every sample, and associativity for every
/// composable triple of samples. Returns the number of equations checked.
fn check_laws<C: Category>(cat: &C, samples: &[C::Mor]) -> usize {
    let mut checked = 0;
    for f in samples {
        let (a, b) = (cat.source(f), cat.target(f));
        assert_eq!(cat.compose(&cat.id(b), f).as_ref(), Some(f), "id ∘ f = f");
        assert_eq!(cat.compose(f, &cat.id(a)).as_ref(), Some(f), "f ∘ id = f");
        checked += 2;
        for g in samples.iter().filter(|g| cat.source(g) == b) {
            for h in samples.iter().filter(|h| cat.source(h) == cat.target(g)) {
                let g_f = cat.compose(g, f).expect("g and f meet");
                let h_g = cat.compose(h, g).expect("h and g meet");
```

```
                let left = cat.compose(h, &g_f);
                let right = cat.compose(&h_g, f);
                assert_eq!(left, right, "h ∘ (g ∘ f) = (h ∘ g) ∘ f");
                checked += 1;
            }
        }
    }
    checked
}
```

The contrast with Lean is worth stating once. In Lean, composing two arrows that do not meet is a *type error*, caught before the program runs. In Rust it is a `None` at run time. In Lean the laws are proved for all morphisms; in Rust they are tested on the samples we thought of.

Preorders

A **preorder** is a relation $\leq$ that is reflexive ($a \leq a$) and transitive ($a \leq b$ and $b \leq c$ give $a \leq c$). Every preorder is a category:

- the objects are the elements;
- there is exactly one morphism $a \to b$ when $a \leq b$, and none otherwise;
- the identity on a is reflexivity, $a \leq a$;
- composition is transitivity.

The laws hold automatically, because between two objects there is at most one morphism, so any two parallel morphisms are equal. A category with this property is called **thin**. Examples are everywhere: numbers with $\leq$; positive integers ordered by divisibility; propositions ordered by implication ($p \leq q$ when p implies q); and, in Chapter 27, Rust's lifetimes ordered by "outlives", where `'a: 'b` says that `'a` lasts at least as long as `'b`.

In Lean, a morphism $a \to b$ is a *proof* of `a ≤ b`:

```
/-- Every preorder is a category with at most one morphism between two objects:
a morphism `a → b` is a proof of `a ≤ b`, wrapped by `PLift` so that it lives
in `Type`. -/
instance preorderCategory (α : Type) [Preorder α] : Category α where
  Hom a b := PLift (a ≤ b)
  id a := ⟨le_refl a⟩
  comp g f := ⟨le_trans f.down g.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

`Preorder α` is Mathlib's class of preorders: a relation `≤` with the lemmas `le_refl` and `le_trans`. A proof of `a ≤ b` lives in `Prop`, but `Hom` must return a `Type`, so we wrap it with `PLift`, a structure with a single field `down` that turns anything (including a proof) into a value of a type in `Type`: `⟨h⟩` builds one, and `f.down` takes it apart. The laws are `rfl` because of **proof irrelevance** (Chapter 10): any two proofs of the same proposition are definitionally equal, so `⟨le_trans f.down (id b).down⟩` and `f` are the same morphism as far as Lean is concerned.

```
/-- The morphism `3 → 5` in the category `ℕ`: a proof that `3 ≤ 5`. -/
def threeToFive : (3 : ℕ) → 5 := ⟨by decide⟩

/-- Any two parallel morphisms of a preorder are equal: proofs are irrelevant. -/
theorem preorder_thin {α : Type} [Preorder α] {a b : α} (f g : a → b) : f = g := by
```

```
  cases f   -- `f` is `⟨h₁⟩` for some proof `h₁ : a ≤ b`
  cases g   -- `g` is `⟨h₂⟩` for some proof `h₂ : a ≤ b`
  rfl       -- `h₁` and `h₂` are proofs of the same proposition, hence equal

/-- In `Prop`, ordered by implication, a morphism `p → q` is a proof of `p → q`. -/
def andElim (p q : Prop) : (p ∧ q) → p := ⟨fun h => h.1⟩
```

`threeToFive` is the morphism $3 \to 5$ in the category `ℕ`; its "content" is a proof that `3 ≤ 5`, found by `decide`. `preorder_thin` states thinness. Its proof needs one step more than `rfl`: `cases f` and `cases g` take the two morphisms apart into `⟨h₁⟩` and `⟨h₂⟩` (Lean's `rfl` does not see through two unknown structures at once), and then `rfl` succeeds because `h₁` and `h₂` are proofs of the same proposition. Finally, `andElim` is a morphism in `Prop`, where Mathlib defines `p ≤ q` as `p → q`: its content is the proof `fun h => h.1` of $p \wedge q \to p$.

The Rust version uses divisibility. A morphism carries its source and target, and the only way to build one is through `divides`, which checks the relation:

```
/// The preorder of positive integers ordered by divisibility: there is one
/// morphism `a → b` exactly when `a` divides `b`.
struct Divisibility;

#[derive(Clone, PartialEq, Debug)]
struct Divides {
    from: u32,
    to: u32,
}

/// The only way to build a morphism: check that `a` divides `b`.
fn divides(a: u32, b: u32) -> Option<Divides> {
    (a > 0 && b % a == 0).then_some(Divides { from: a, to: b })
}

impl Category for Divisibility {
    type Obj = u32;
    type Mor = Divides;
    fn source(&self, f: &Divides) -> u32 {
        f.from
    }
    fn target(&self, f: &Divides) -> u32 {
        f.to
    }
    fn id(&self, a: u32) -> Divides {
        Divides { from: a, to: a }
    }
    fn compose(&self, g: &Divides, f: &Divides) -> Option<Divides> {
        // Transitivity: a | b and b | c give a | c.
        (f.to == g.from).then(|| Divides {
            from: f.from,
            to: g.to,
        })
    }
}
```

```
let div = Divisibility;
let (a, b) = (divides(3, 12).unwrap(), divides(12, 36).unwrap());
println!(
    "divisibility: (12 → 36) ∘ (3 → 12) = {:?}",
    div.compose(&b, &a)
);
let samples: Vec<Divides> = (1..=12)
    .flat_map(|a| (1..=12).filter_map(move |b| divides(a, b)))
    .collect();
println!(
```

```
    "divisibility: {} equations checked",
    check_laws(&div, &samples)
);
```

```
divisibility: (12 → 36) ∘ (3 → 12) = Some(Divides { from: 3, to: 36 })
divisibility: 203 equations checked
```

The samples are all the divisibility facts among the numbers 1 to 12, and the checker verified 203 equations: two unit laws for each sample morphism, and associativity for every composable triple.

Monoids are categories with one object

A **monoid** is a set M with an associative operation $\cdot$ and a unit element e: $e \cdot x = x = x \cdot e$ and $x \cdot (y \cdot z) = (x \cdot y) \cdot z$. Examples: numbers with $+$ and 0 or with $\times$ and 1, strings with concatenation and `""`, and functions from a type to itself with composition and the identity.

Compare the monoid laws with the category laws: they are the same equations. That is not a coincidence. A monoid *is* a category with exactly one object $\star$: the morphisms $\star \to \star$ are the elements of M, composition is the operation, and the identity is the unit. Since there is only one object, every two morphisms meet, and the "type-checking" part of composition disappears.

```
/-- The single object of the category built from a monoid `M`. -/
structure OneObject (M : Type) where
  /-- The unique value: the type has exactly one element, `()`. -/
  mk ::

/-- A monoid is a category with one object: the morphisms are the elements of
the monoid, composition is multiplication, the identity is `1`. -/
instance monoidCategory (M : Type) [Monoid M] : Category (OneObject M) where
  Hom _ _ := M
  id _ := 1
  comp g f := g * f
  id_comp f := one_mul f
  comp_id f := mul_one f
  assoc h g f := (mul_assoc h g f).symm
```

`OneObject M` is a structure with no fields, so it has exactly one value, written `⟨⟩`: the single object. `Monoid M` is Mathlib's class of monoids, which provides `*`, `1` and the three laws `one_mul`, `mul_one` and `mul_assoc` (Chapter 14 built its own version of this class). Each category law is discharged by the corresponding monoid law; associativity is stated the other way around in Mathlib, hence the `.symm`, which turns a proof of $x = y$ into a proof of $y = x$.

```
/-- An element of the monoid, seen as a morphism of its one-object category. -/
def OneObject.arrow {M : Type} [Monoid M] (m : M) : (⟨⟩ : OneObject M) → ⟨⟩ := m

/-- In the one-object category of `(ℕ, *, 1)`, composing is multiplying. -/
def six : (⟨⟩ : OneObject ℕ) → ⟨⟩ := OneObject.arrow 2 ⊚ OneObject.arrow 3

/-- info: 6 -/
#guard_msgs in
#eval (six : ℕ)
```

`OneObject.arrow` only tells Lean the type of the (single) object. Composing the arrows `2` and `3` in the one-object category of $(\mathbb{N}, \times, 1)$ multiplies them.

The connection runs the other way too. In *any* category, the morphisms from an object to itself (its **endomorphisms**) form a monoid under composition, and the monoid laws are instances of the category laws:

```
/-- Conversely, in any category the morphisms from an object to itself form a
monoid under composition: the monoid laws are instances of the category laws. -/
theorem endo_monoid {Obj : Type u} [Category Obj] (a : Obj) (f g h : a ⟶ a) :
    Category.id a ≫ f = f ∧ f ≫ Category.id a = f ∧ h ≫ (g ≫ f) = (h ≫ g) ≫ f :=
  ⟨Category.id_comp f, Category.comp_id f, Category.assoc h g f⟩
```

The proof is the three laws, glued with the anonymous constructor `⟨_, _, _⟩` that builds a conjunction. So the functions `ℕ → ℕ`, such as `double` in the picture above, form a monoid.

In Rust, strings under concatenation give a one-object category whose single object is `()`:

```
/// The monoid of strings under concatenation, as a category with one object
/// `()`: every string is a morphism `() → ()`.
struct Strings;

impl Category for Strings {
    type Obj = ();
    type Mor = String;
    fn source(&self, _: &String) {}
    fn target(&self, _: &String) {}
    fn id(&self, _: ()) -> String {
        String::new()
    }
    fn compose(&self, g: &String, f: &String) -> Option<String> {
        Some(format!("{f}{g}")) // first f's text, then g's
    }
}
```

```
let words: Vec<String> = ["", "con", "cat", "egory"].map(String::from).to_vec();
let concat = Strings.compose(&words[2], &words[1]).unwrap();
println!("strings: \"cat\" ∘ \"con\" = {concat:?}");
println!(
    "strings: {} equations checked",
    check_laws(&Strings, &words)
);
```

```
strings: "cat" ∘ "con" = "concat"
strings: 72 equations checked
```

`compose(g, f)` puts `f`'s text first, to match "first `f`, then `g`"; the other choice would give a perfectly good category too, as duality will explain.

Discrete categories

At the other extreme from a monoid is a category with many objects and no interesting arrows: the only morphisms are the identities. Any set gives such a **discrete category**. In Lean a morphism $a \to b$ is a proof that $a = b$:

```
/-- A copy of `α` whose only morphisms are identities. -/
structure Discrete (α : Type) where
  /-- The underlying element. -/
  val : α

/-- A morphism `a ⟶ b` is a proof that `a = b`. -/
instance discreteCategory (α : Type) : Category (Discrete α) where
  Hom a b := PLift (a = b)
```

```
  id _ := ⟨rfl⟩
  comp g f := ⟨f.down.trans g.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

/-- Between two different objects there is no morphism at all. -/
theorem discrete_no_arrow (f : (⟨1⟩ : Discrete ℕ) → ⟨2⟩) : False := by
  have h : (⟨1⟩ : Discrete ℕ) = ⟨2⟩ := f.down
  simp at h
```

`Discrete α` is a copy of `α` (a structure with one field), so that the instance does not compete with other instances on `α`, such as the preorder on `ℕ`. Composition is transitivity of equality. The theorem `discrete_no_arrow` shows that there is no morphism between two different objects: from `f` we extract a proof `h` of `⟨1⟩ = ⟨2⟩`, and `simp at h` simplifies it. `simp` knows that two values built with the same constructor are equal only if their fields are, turns `h` into `1 = 2`, evaluates that to `False`, and a hypothesis `False` closes any goal.

Discrete categories look trivial, and they are; they serve as building blocks. A pair of objects of a category is the same thing as a way of sending the two objects of a two-element discrete category into it, a remark that Mathlib's `Limits` library turns into its definition of products.

Free categories: paths in a graph

Take a directed graph: vertices, and edges between them. Its **paths** form a category. The objects are the vertices; a morphism from a to b is a path that starts at a and ends at b (a sequence of edges, each starting where the previous one ended); composition is concatenation; the identity on a is the empty path at a. This is the **free category** on the graph: "free" because it satisfies no equations beyond the laws, so two paths are equal only if they consist of the same edges.

```
/-- A path in a graph whose edges from `a` to `b` form the type `E a b`:
either empty, or an edge followed by a path. -/
inductive Path {V : Type} (E : V → V → Type) : V → V → Type where
  | nil (a : V) : Path E a a
  | cons {a b c : V} (e : E a b) (p : Path E b c) : Path E a c

namespace Path

variable {V : Type} {E : V → V → Type}

/-- Concatenation: first `p`, then `q`. -/
def append : {a b c : V} → Path E a b → Path E b c → Path E a c
  | _, _, _, nil _, q => q
  | _, _, _, cons e p, q => cons e (append p q)

/-- The number of edges in a path. -/
def length : {a b : V} → Path E a b → ℕ
  | _, _, nil _ => 0
  | _, _, cons _ p => length p + 1
```

`Path E` is an *inductive family*: one type `Path E a b` for every pair of vertices, defined all at once, like the typing relations of Chapter 3 but in `Type` rather than `Prop`. A path is either `nil a`, the empty path at `a`, or `cons e p`, an edge from `a` to `b` followed by a path from `b` to `c`; the type of `cons` makes it impossible to build a path whose edges do not meet. `append` concatenates by recursion on the first path (the leading `_` in its patterns stand for the implicit vertices).

The laws are theorems about `append`:

```
/-- The empty path is a left unit: true by the first equation of `append`. -/
theorem nil_append {a b : V} (p : Path E a b) : append (nil a) p = p := rfl

/-- The empty path is a right unit: needs induction on the path. -/
theorem append_nil {a b : V} (p : Path E a b) : append p (nil b) = p := by
  induction p with
  | nil a => rfl
  | cons e p ih =>
    -- Goal: append (cons e p) (nil _) = cons e p
    simp only [append]  -- unfold one step: cons e (append p (nil _)) = cons e p
    rw [ih]             -- the induction hypothesis closes it

/-- Concatenation is associative, again by induction on the first path. -/
theorem append_assoc {a b c d : V} (p : Path E a b) (q : Path E b c) (r : Path E c d) :
    append (append p q) r = append p (append q r) := by
  induction p with
  | nil a => rfl
  | cons e p ih => simp only [append, ih]
```

`nil_append` is `rfl`: it is literally the first equation of `append`. The other unit law is not: `append p (nil b)` is stuck until we know whether `p` is `nil` or `cons`, so we prove it by induction on `p`. In the `nil` case both sides are `nil a`. In the `cons e p` case, `simp only [append]` unfolds one step of the definition, leaving `cons e (append p (nil _)) = cons e p`, and the induction hypothesis `ih : append p (nil _) = p` rewrites the left side into the right. Associativity is the same pattern; there `simp only [append, ih]` unfolds and rewrites with the hypothesis in one go.

Now the instance:

```
/-- The objects of the free category on a graph `E` are its vertices. -/
structure Free {V : Type} (E : V → V → Type) where
  /-- The vertex. -/
  vertex : V

/-- The free category: morphisms are paths, composition is concatenation. -/
instance freeCategory {V : Type} (E : V → V → Type) : Category (Free E) where
  Hom a b := Path E a.vertex b.vertex
  id a := Path.nil a.vertex
  comp g f := Path.append f g
  id_comp f := Path.append_nil f
  comp_id f := Path.nil_append f
  assoc h g f := Path.append_assoc f g h
```

The wrapper `Free E` attaches the instance to the graph, since the vertex type `V` alone does not say which edges we mean. Note the order in `comp g f := Path.append f g`: the path walks `f` first. Because of that order, the category's left unit law (`id ⊚ f = f`) is `append_nil` and its right unit law is `nil_append`.

Here is a graph that every network programmer knows: the states of a connection and the operations that move between them.

```
/-- The states of a network connection. -/
inductive Conn where
  | closed | connected | authenticated

/-- The protocol: which operation leads from which state to which. -/
inductive Step : Conn → Conn → Type where
  | connect : Step .closed .connected
  | login : Step .connected .authenticated
  | logout : Step .authenticated .connected
```

```
  | close : Step .connected .closed

/-- The name of an operation, for printing. -/
def Step.name : {a b : Conn} → Step a b → String
  | _, _, .connect => "connect"
  | _, _, .login => "login"
  | _, _, .logout => "logout"
  | _, _, .close => "close"

/-- The operations along a path, in order. -/
def Path.names : {a b : Conn} → Path Step a b → List String
  | _, _, .nil _ => []
  | _, _, .cons e p => e.name :: p.names

/-- Morphisms of the free category on the protocol graph are valid traces. -/
abbrev Trace (a b : Conn) : Type := (⟨a⟩ : Free Step) ⟶ ⟨b⟩

def signIn : Trace .closed .authenticated :=
  .cons .connect (.cons .login (.nil _))

def signOut : Trace .authenticated .closed :=
  .cons .logout (.cons .close (.nil _))

/-- info: ["connect", "login", "logout", "close"] -/
#guard_msgs in
#eval (signOut ◎ signIn).names
```

`Step` is an inductive family again: `Step a b` is the type of operations that go from state `a` to state `b`, so `Step.login` exists only from `.connected` to `.authenticated`. A morphism of the free category, a `Trace`, is a sequence of operations that the protocol allows. Composing `signIn` with `signOut` gives the whole session. Composing `signIn` with itself is not "an error at run time": it does not type-check, because `signIn` ends at `.authenticated` and starts at `.closed`. Chapter 25 builds whole APIs on this idea, under the name typestate.

The Rust version represents the same graph as a transition function, and traces as vectors of operations validated when they are built:

```
/// The states of a connection, and the operations of its protocol.
#[derive(Clone, Copy, PartialEq, Debug)]
enum State {
    Closed,
    Connected,
    Authenticated,
}

#[derive(Clone, Copy, PartialEq, Debug)]
enum Op {
    Connect,
    Login,
    Logout,
    Close,
}

/// The edges of the protocol graph: where each operation may start and end.
fn step(state: State, op: Op) -> Option<State> {
    use State::*;
    match (state, op) {
        (Closed, Op::Connect) => Some(Connected),
        (Connected, Op::Login) => Some(Authenticated),
        (Authenticated, Op::Logout) => Some(Connected),
        (Connected, Op::Close) => Some(Closed),
        _ => None,
    }
}
```

```
/// A morphism of the free category: a path in the graph, that is, a sequence
/// of operations that the protocol allows, starting from `from`.
#[derive(Clone, PartialEq, Debug)]
struct Trace {
    from: State,
    to: State,
    ops: Vec<Op>,
}

impl Trace {
    fn new(from: State, ops: &[Op]) -> Option<Trace> {
        let mut to = from;
        for &op in ops {
            to = step(to, op)?;
        }
        Some(Trace {
            from,
            to,
            ops: ops.to_vec(),
        })
    }
}
```

```
/// The free category on the protocol graph.
struct Protocol;

impl Category for Protocol {
    type Obj = State;
    type Mor = Trace;
    fn source(&self, f: &Trace) -> State {
        f.from
    }
    fn target(&self, f: &Trace) -> State {
        f.to
    }
    fn id(&self, a: State) -> Trace {
        Trace {
            from: a,
            to: a,
            ops: Vec::new(),
        }
    }
    fn compose(&self, g: &Trace, f: &Trace) -> Option<Trace> {
        if f.to != g.from {
            return None; // the paths do not meet
        }
        let ops = f.ops.iter().chain(&g.ops).copied().collect();
        Some(Trace {
            from: f.from,
            to: g.to,
            ops,
        })
    }
}
```

```
use {Op::*, State::*};
let sign_in = Trace::new(Closed, &[Connect, Login]).unwrap();
let sign_out = Trace::new(Authenticated, &[Logout, Close]).unwrap();
let session = Protocol.compose(&sign_out, &sign_in).unwrap();
println!("protocol: sign_out ∘ sign_in = {:?}", session.ops);
println!(
    "protocol: sign_in ∘ sign_in = {:?}",
    Protocol.compose(&sign_in, &sign_in)
);
let traces: Vec<Trace> = [
    (Closed, vec![Connect]),
    (Connected, vec![Login, Logout]),
    (Connected, vec![Close, Connect]),
```

```
    (Authenticated, vec![Logout, Close]),
    (Closed, vec![Connect, Login]),
]
.iter()
.filter_map(|(from, ops)| Trace::new(*from, ops))
.collect();
println!(
    "protocol: {} equations checked",
    check_laws(&Protocol, &traces)
);
```

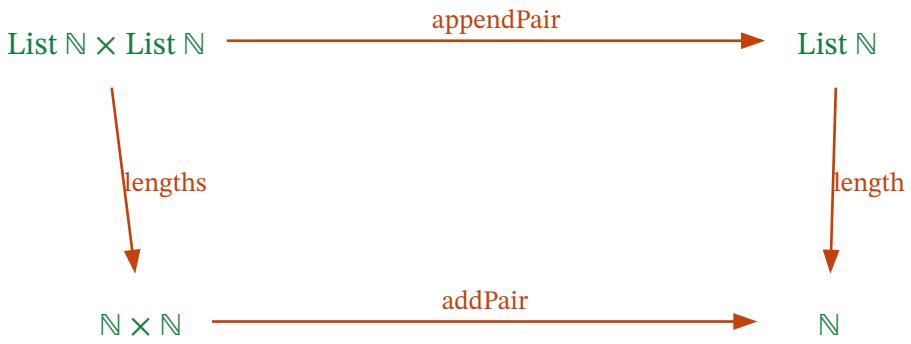
```
protocol: sign_out ∘ sign_in = [Connect, Login, Logout, Close]
protocol: sign_in ∘ sign_in = None
protocol: 27 equations checked
```

The composite session is the four operations in order; composing `sign_in` with itself gives `None`, the run-time counterpart of Lean's type error.

16.4 Commutative diagrams

Equations between composites quickly become hard to read, so category theorists draw them. A **diagram** is a picture of some objects and some arrows between them. A diagram **commutes** if any two paths of arrows with the same start and the same end compose to the same morphism.

Here is a commutative square from everyday programming. Start with a pair of lists. You can append them and take the length of the result, or take the two lengths and add them:



A commutative square: appending two lists and then measuring the result gives the same number as measuring both lists and adding.

The square commutes: both paths from the top-left corner to the bottom-right corner are the same function. In Lean, that is one equation between composites:

```
/-- Concatenate the two lists of a pair. -/
def appendPair : List ℕ × List ℕ → List ℕ := fun p => p.1 ++ p.2
/-- Measure both lists of a pair. -/
def lengths : List ℕ × List ℕ → ℕ × ℕ := fun p => (p.1.length, p.2.length)
/-- Add the two numbers of a pair. -/
def addPair : ℕ × ℕ → ℕ := fun p => p.1 + p.2

/-- The square commutes: "append, then measure" equals "measure both, then add". -/
```

```
theorem length_square :
    (List.length : List ℕ → ℕ) ∘ appendPair = addPair ∘ lengths := by
  funext p                  -- two functions are equal if they agree on every `p`
  exact List.length_append  -- (p.1 ++ p.2).length = p.1.length + p.2.length
```

The proof uses `funext p`, which reduces an equation between functions to an equation between their values at an arbitrary argument `p`, and then Lean's library lemma `List.length_append`, which says exactly `(p.1 ++ p.2).length = p.1.length + p.2.length`.

If you have written property-based tests, you have written commutative diagrams without the pictures: "for all pairs of lists, `len(a ++ b) == len(a) + len(b)`" is this square, tested on random samples, while the Lean version is proved for all inputs. Chapter 18 is full of such squares, under the name naturality squares.

16.5 Isomorphisms

When are two objects "the same"? Category theory must answer with arrows. A morphism $f : a \to b$ is an **isomorphism** if it has an **inverse** $g : b \to a$:

$$g \circ f = \mathrm{id}_a \qquad\qquad f \circ g = \mathrm{id}_b$$

Then a and b are **isomorphic**, written $a \cong b$, and nothing category theory can say distinguishes them. You have seen this idea in Chapter 11, where `Iso` structures recorded that types such as `A × ()` and `A` carry the same information: isomorphisms in the category of types. The shared module generalizes the definition to every category:

```
/-- An isomorphism between `a` and `b`: morphisms in both directions whose
composites are the identities. -/
structure Iso {Obj : Type u} [Category Obj] (a b : Obj) where
  hom : a → b
  inv : b → a
  inv_hom : inv ∘ hom = Category.id a
  hom_inv : hom ∘ inv = Category.id b
```

An `Iso a b` packages the two arrows and the two equations; the module also defines the notation `a ≅ b` for it. Isomorphism behaves like an equality: every object is isomorphic to itself, and isomorphisms can be reversed and composed. In `Iso.refl` both equations say `id a ∘ id a = id a`, an instance of the unit law; `Iso.symm` swaps the two arrows and the two proofs.

```
/-- Every object is isomorphic to itself. -/
def Iso.refl {Obj : Type u} [Category Obj] (a : Obj) : a ≅ a where
  hom := Category.id a
  inv := Category.id a
  inv_hom := Category.id_comp _
  hom_inv := Category.id_comp _

/-- Isomorphisms can be reversed. -/
def Iso.symm {Obj : Type u} [Category Obj] {a b : Obj} (i : a ≅ b) : b ≅ a where
  hom := i.inv
  inv := i.hom
  inv_hom := i.hom_inv
  hom_inv := i.inv_hom
```

```
/-- Isomorphisms compose. -/
def Iso.trans {Obj : Type u} [Category Obj] {a b c : Obj} (i : a ≅ b) (j : b ≅ c) :
    a ≅ c where
  hom := j.hom ≫ i.hom
  inv := i.inv ≫ j.inv
  inv_hom := by
    rw [Category.assoc, ← Category.assoc i.inv, j.inv_hom, Category.comp_id, i.inv_hom]
  hom_inv := by
    rw [Category.assoc, ← Category.assoc j.hom, i.hom_inv, Category.comp_id, j.hom_inv]
```

The proof of `inv_hom` in `Iso.trans` is a good exercise in reading `rw`. The goal is `(i.inv ≫ j.inv) ≫ (j.hom ≫ i.hom) = id a`: undo `j`, undo `i`, after doing `i` and then `j`. The rewrites move the brackets until the inner pair meets:

1. `Category.assoc` rewrites the left side, of the shape `h ≫ (g ≫ f)`, into `((i.inv ≫ j.inv) ≫ j.hom) ≫ i.hom`.
2. `← Category.assoc i.inv` uses the same law from right to left, on the subterm that starts with `i.inv`: `(i.inv ≫ j.inv) ≫ j.hom` becomes `i.inv ≫ (j.inv ≫ j.hom)`.
3. `j.inv_hom` replaces `j.inv ≫ j.hom` with `id b`.
4. `Category.comp_id` removes it: `i.inv ≫ id b` becomes `i.inv`.
5. `i.inv_hom` turns `i.inv ≫ i.hom` into `id a`, and the goal `id a = id a` is closed by `rfl`, which `rw` tries automatically.

The proof of `hom_inv` is the mirror image. Neither mentions what the objects or the morphisms are: they hold in every category at once.

Two isomorphisms in the category of types:

```
/-- `Bool` and `Option Unit` are isomorphic in the category of types. -/
def boolIso : Bool ≅ Option Unit where
  hom := fun b => if b then some () else none
  inv := fun o => o.isSome
  inv_hom := by
    funext b      -- compare the two functions on every `b : Bool`
    cases b <;> rfl
  hom_inv := by
    funext o
    cases o <;> rfl

/-- Swapping the components of a pair is an isomorphism `α × β ≅ β × α`. -/
def swapIso (α β : Type) : α × β ≅ β × α where
  hom := fun p => (p.2, p.1)
  inv := fun p => (p.2, p.1)
  inv_hom := rfl
  hom_inv := rfl
```

For `boolIso`, the equations compare two functions, so `funext b` reduces them to a statement about an arbitrary `b : Bool`, `cases b` splits into the cases `false` and `true`, and `<;> rfl` runs `rfl` on both. For `swapIso` the equations hold by computation (swapping twice gives back the pair, by η for pairs).

A morphism can have at most one inverse. The proof is a classic of equational reasoning, written here with `calc`, which chains equations step by step, each with its own justification:

```
/-- A morphism has at most one inverse: if `g` is a left inverse and `g'` a
right inverse of `f`, then `g = g'`. -/
theorem inverse_unique {Obj : Type u} [Category Obj] {a b : Obj}
    (f : a → b) (g g' : b → a)
```

```
    (hg : g ⊚ f = Category.id a) (hg' : f ⊚ g' = Category.id b) : g = g' :=
  calc g = g ⊚ Category.id b := (Category.comp_id g).symm
    _ = g ⊚ (f ⊚ g') := by rw [hg']
    _ = (g ⊚ f) ⊚ g' := Category.assoc g f g'
    _ = Category.id a ⊚ g' := by rw [hg]
    _ = g' := Category.id_comp g'
```

Read the chain from top to bottom: insert an identity, replace it by $f \circ g'$, regroup, replace $g \circ f$ by an identity, remove it. Each `_ = ...` line continues from the right-hand side of the previous one. So "the inverse" of an isomorphism is well defined, which is why `Iso` can store it as a field.

In a preorder, $a \cong b$ means $a \leq b$ and $b \leq a$. In an order where that forces $a = b$ (a partial order, such as $\mathbb{N}$), the only isomorphisms are identities.

16.6 Monomorphisms and epimorphisms

Injective and surjective functions have categorical versions stated with arrows only. A morphism $f : a \to b$ is a **monomorphism** (mono) if it can be cancelled on the left, and an **epimorphism** (epi) if it can be cancelled on the right:

$$f \circ g = f \circ h \Rightarrow g = h \qquad\qquad g \circ f = h \circ f \Rightarrow g = h$$

```
/-- `f` is a monomorphism if it can be cancelled on the left. -/
def Mono {Obj : Type u} [Category Obj] {a b : Obj} (f : a ⟶ b) : Prop :=
  ∀ {x : Obj} (g h : x ⟶ a), f ⊚ g = f ⊚ h → g = h

/-- `f` is an epimorphism if it can be cancelled on the right. -/
def Epi {Obj : Type u} [Category Obj] {a b : Obj} (f : a ⟶ b) : Prop :=
  ∀ {y : Obj} (g h : b ⟶ y), g ⊚ f = h ⊚ f → g = h
```

In the category of types these are exactly the injective and the surjective functions. Here is the first half:

```
/-- In the category of types, the monomorphisms are the injective functions. -/
theorem mono_iff_injective {α β : Type} (f : α → β) :
    Mono f ↔ Function.Injective f := by
  constructor
  · -- A mono is injective: test it with two functions out of `Unit`.
    intro hf x y hxy
    have h := hf (fun _ : Unit => x) (fun _ : Unit => y) (funext fun _ => hxy)
    exact congrFun h ()
  · -- An injective function can be cancelled: compare `g` and `h` pointwise.
    intro hf x g h hgh
    funext t
    exact hf (congrFun hgh t)
```

`constructor` splits the `↔` into two implications.

- *Mono implies injective.* We have `hf : Mono f` and `x y` with `f x = f y`, and must show `x = y`. The trick is to test cancellation with two functions out of `Unit`: the constant functions `fun _ => x` and `fun _ => y`. Composing them with `f` gives `fun _ => f x` and `fun _ => f y`, which are equal by `funext` and the hypothesis. Mono cancels `f`, so the two constant functions are equal, and `congrFun h ()` applies both sides to `()` to get `x = y`. (Functions out of `Unit` are the categorical stand-in for elements, a trick you will see again.)
- *Injective implies mono.* Given `f ∘ g = f ∘ h`, compare `g` and `h` at any `t`: `congrFun hgh t` gives `f (g t) = f (h t)`, and injectivity removes `f`.

And the second half:

```
/-- In the category of types, the epimorphisms are the surjective functions. -/
theorem epi_iff_surjective {α β : Type} (f : α → β) :
    Epi f ↔ Function.Surjective f := by
  constructor
  · -- An epi is surjective: test it with two predicates on `β`.
    intro hf b
    let g : β → Prop := fun y => ∃ a, f a = y
    let h : β → Prop := fun _ => True
    have hgf : g ∘ f = h ∘ f := by
      funext a
      exact eq_true ⟨a, rfl⟩  -- `∃ a', f a' = f a` holds, so it equals `True`
    have hgh : g = h := hf g h hgf
    exact of_eq_true (congrFun hgh b)
  · -- A surjective function can be cancelled: every `b` is some `f a`.
    intro hf y g h hgh
    funext b
    obtain ⟨a, rfl⟩ := hf b
    exact congrFun hgh a
```

- *Epi implies surjective.* Given `b`, we want some `a` with `f a = b`. We test cancellation with two functions from `β` into `Prop` (which is itself a type, so it is an object of our category): `g y` is the proposition "`y` is hit by `f`", and `h y` is `True`. Composed with `f` they agree, since every `f a` is hit by `f`: `eq_true` turns the proof `⟨a, rfl⟩` of `∃ a', f a' = f a` into the equation `(∃ a', f a' = f a) = True`. Epi cancels `f`, so `g = h`; at `b` this says `(∃ a, f a = b) = True`, and `of_eq_true` turns that back into a proof.
- *Surjective implies epi.* To compare `g` and `h` at `b`, write `b` as `f a` (`obtain ⟨a, rfl⟩ := hf b` both extracts `a` and replaces `b` by `f a` everywhere), and use the hypothesis at `a`.

Do not conclude that "mono and epi" means "isomorphism", as "injective and surjective" means "bijective" for functions. In a preorder, any two parallel morphisms are equal, so *every* morphism is both mono and epi; but $0 \leq 1$ in $\mathbb{N}$ is not an isomorphism, because there is no morphism back:

```
/-- In a preorder every morphism is both mono and epi (parallel morphisms are
equal), but `0 → 1` in `ℕ` is not an isomorphism: that would need `1 ≤ 0`. -/
def zeroToOne : (0 : ℕ) → 1 := ⟨by decide⟩

theorem zeroToOne_mono : Mono zeroToOne :=
  fun g h _ => preorder_thin g h

theorem zeroToOne_epi : Epi zeroToOne :=
  fun g h _ => preorder_thin g h

theorem zero_one_not_iso (i : (0 : ℕ) ≅ 1) : False := by
  have h : 1 ≤ 0 := i.inv.down
  omega
```

The last proof extracts from the inverse a proof of `1 ≤ 0`, and `omega` (the decision procedure for linear arithmetic of Chapter 2) finds the contradiction. Categorical definitions agree with the familiar ones in the category of types and can behave very differently elsewhere.

16.7 Initial and terminal objects

Some objects are characterized by the arrows around them alone. An object t is **terminal** if from every object a there is *exactly one* morphism $a \to t$. Dually, an object i is **initial** if to every object a there is exactly one morphism $i \to a$.

```
/-- `t` is terminal: from every object there is exactly one morphism to `t`. -/
structure IsTerminal {Obj : Type u} [Category Obj] (t : Obj) where
  /-- The morphism from `a` to `t`. -/
  arrow : (a : Obj) → a ⟶ t
  /-- It is the only one. -/
  unique : ∀ {a : Obj} (f : a ⟶ t), f = arrow a

/-- `i` is initial: to every object there is exactly one morphism from `i`. -/
structure IsInitial {Obj : Type u} [Category Obj] (i : Obj) where
  /-- The morphism from `i` to `a`. -/
  arrow : (a : Obj) → i ⟶ a
  /-- It is the only one. -/
  unique : ∀ {a : Obj} (f : i ⟶ a), f = arrow a
```

The definitions live in `code/lean/Book/Ch16/Universal.lean`. Each is a structure with the morphism (`arrow`) and a proof that it is the only one (`unique`). In the category of types, `Unit` is terminal and `Empty` is initial:

```
/-- `Unit` is terminal in the category of types. -/
def unitTerminal : IsTerminal Unit where
  arrow _ := fun _ => ()
  unique _ := by
    funext x           -- compare `f` and `fun _ => ()` on every input `x`
    rfl                -- both give `()`: `Unit` has only one value

/-- `Empty` is initial in the category of types. -/
def emptyInitial : IsInitial Empty where
  arrow _ := fun e => nomatch e
  unique f := by
    funext e           -- compare the two functions on every `e : Empty`...
    nomatch e          -- ...of which there are none
```

For `Unit` there is only one function from any type: the one that returns `()`. Any other function `f` agrees with it everywhere, because `Unit` has only one value, so after `funext x` the goal `f x = ()` is closed by `rfl` (Lean has η for `Unit`: every value of `Unit` is definitionally `()`). For `Empty` there is exactly one function *to* any type, the empty function: `nomatch e` is a match with no cases, accepted because `Empty` has no constructors. Uniqueness is vacuous: two functions out of `Empty` agree on all of the (zero) inputs.

Rust has both, and Chapter 1's table already named them:

```
/// The unique function from any type into the terminal object `()`.
fn to_unit<A>(_: A) {}

/// A type with no values, and the unique function out of it: the initial object.
enum Void {}

fn absurd<A>(v: Void) -> A {
    match v {}
}
```

`match v {}` is Rust's `nomatch`: a `match` with no arms on a type with no variants, which has every type. The logician reads these as `True` (provable from anything) and `False` (from which anything follows): in the category `Prop`, `True` is terminal and `False` is initial.

Not every category has these objects. In `ℕ` with `≤`, `0` is initial, since `0 ≤ n` for every `n`; but there is no terminal object, since no number is above every number:

```
/-- In `ℕ` ordered by `≤`, `0` is initial... -/
def zeroInitial : IsInitial (0 : ℕ) where
  arrow n := ⟨Nat.zero_le n⟩
  unique _ := preorder_thin _ _

/-- ...but there is no terminal object: no number is above every number. -/
theorem nat_no_terminal (n : ℕ) (h : IsTerminal n) : False := by
  have hle : n + 1 ≤ n := (h.arrow (n + 1)).down
  omega
```

Uniqueness for `0` is `preorder_thin` again. For the negative result, a terminal `n` would have an arrow from `n + 1`, that is, a proof of `n + 1 ≤ n`, which `omega` refutes.

Now the side effects come back, as promised. In pure code, a function into `()` carries no information, which is what "exactly one morphism into `()`" says. In Rust, two functions into `()` can differ:

```
// Two functions `i32 -> ()` that return the same thing on every input...
let calls = Cell::new(0);
let quiet = |_: i32| ();
let noisy = |_: i32| calls.set(calls.get() + 1);
for n in 0..3 {
    assert_eq!(quiet(n), noisy(n));
}
// ...but they are different programs: `()` is terminal only for pure code.
println!("effects: same outputs, but noisy ran {} times", calls.get());
```

```
effects: same outputs, but noisy ran 3 times
```

`quiet` and `noisy` return the same value on every input, yet they are different programs. Once effects count, `()` is no longer terminal. This is the first sign of a theme of Chapter 20: effects need a category of their own.

Unique up to unique isomorphism

How many terminal objects can a category have? Possibly many (any one-element type is terminal in the category of types), but they are all isomorphic, and in exactly one way:

```
/-- Two morphisms into a terminal object are always equal. -/
theorem IsTerminal.hom_ext {Obj : Type u} [Category Obj] {t a : Obj}
    (ht : IsTerminal t) (f g : a ⟶ t) : f = g := by
  rw [ht.unique f, ht.unique g]

/-- Terminal objects are unique up to isomorphism... -/
def IsTerminal.iso {Obj : Type u} [Category Obj] {t₁ t₂ : Obj}
    (h₁ : IsTerminal t₁) (h₂ : IsTerminal t₂) : t₁ ≅ t₂ where
  hom := h₂.arrow t₁
  inv := h₁.arrow t₂
  inv_hom := h₁.hom_ext _ _   -- both sides are morphisms `t₁ ⟶ t₁`
  hom_inv := h₂.hom_ext _ _   -- both sides are morphisms `t₂ ⟶ t₂`

/-- ...and the isomorphism is unique: there is only one morphism `t₁ ⟶ t₂`. -/
theorem IsTerminal.iso_unique {Obj : Type u} [Category Obj] {t₁ t₂ : Obj}
```

```
    (h₁ : IsTerminal t₁) (h₂ : IsTerminal t₂) (i : t₁ ≅ t₂) :
    i.hom = (h₁.iso h₂).hom :=
  h₂.hom_ext _ _
```

The whole argument rests on `hom_ext`: two morphisms into a terminal object are equal, because both are equal to the `arrow`. To build the isomorphism between terminal objects t_1 and t_2, take the unique arrows in both directions. Their composite $t_1 \to t_2 \to t_1$ is a morphism into the terminal object t_1, and so is id_{t_1}; by `hom_ext` they are equal. The same argument works at t_2. And the isomorphism is unique because there is only one morphism $t_1 \to t_2$ at all.

This pattern, "unique up to unique isomorphism", is the precise sense in which we may speak of *the* terminal object.

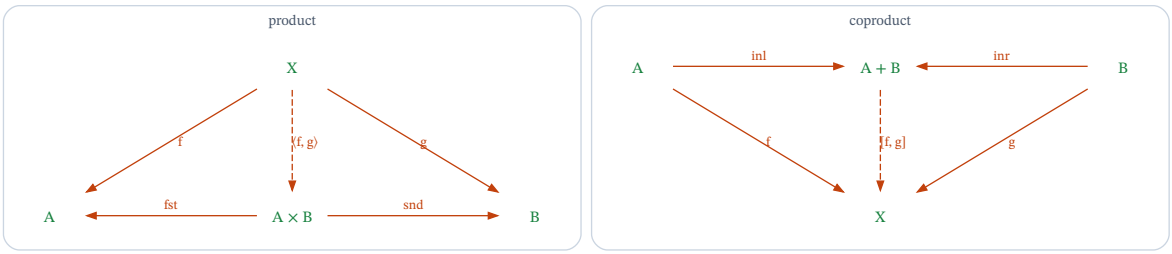
16.8 Products

Now the main event. What is a pair? In Rust, `(A, B)` is a type whose values hold an `A` and a `B`. Category theory cannot look at values, so it describes the pair type by what you can *do* with it: you can project out of it, and you can build it from anything that gives you both components.

A **product** of objects a and b is an object p with two morphisms $\mathrm{fst} : p \to a$ and $\mathrm{snd} : p \to b$ such that for every object x and every pair of morphisms $f : x \to a$, $g : x \to b$, there is **exactly one** morphism $\langle f, g \rangle : x \to p$ with

$$\mathrm{fst} \circ \langle f, g \rangle = f \qquad\qquad \mathrm{snd} \circ \langle f, g \rangle = g$$

In pictures, the left half of the next figure: whatever x, f and g are, the dashed arrow exists, makes both triangles commute, and is the only arrow that does. The right half is the coproduct, which we will meet next.



The universal properties of the product (left) and the coproduct (right). The dashed arrow exists, is unique, and makes the triangles commute.

A definition of this shape is called a **universal property**. Think of it as an interface specification: it does not say how a product is built, only what any product must offer (two projections and a way to pair) and the laws these operations satisfy. Any construction meeting the specification is a product, and by the uniqueness theorem below, any two are interchangeable, with a canonical translation between them. In Lean the specification is a structure:

```
/-- `p`, with projections `fst` and `snd`, is a product of `a` and `b`: every
pair of morphisms `f : x → a`, `g : x → b` factors through `p` in exactly one
```

```
way. -/
structure IsProduct {Obj : Type u} [Category Obj] {a b p : Obj}
    (fst : p ⟶ a) (snd : p ⟶ b) where
  /-- The morphism into the product built from `f` and `g`. -/
  lift : {x : Obj} → (x ⟶ a) → (x ⟶ b) → (x ⟶ p)
  /-- Projecting the lift on the first component gives back `f`. -/
  fst_lift : ∀ {x : Obj} (f : x ⟶ a) (g : x ⟶ b), fst ≫ lift f g = f
  /-- Projecting the lift on the second component gives back `g`. -/
  snd_lift : ∀ {x : Obj} (f : x ⟶ a) (g : x ⟶ b), snd ≫ lift f g = g
  /-- Any morphism with these two properties is the lift. -/
  uniq : ∀ {x : Obj} (f : x ⟶ a) (g : x ⟶ b) (h : x ⟶ p),
    fst ≫ h = f → snd ≫ h = g → h = lift f g
```

The four fields mirror the definition: `lift f g` is $\langle f, g \rangle$, the next two fields are the two triangles, and `uniq` says that any `h` making both triangles commute is the lift. Note that `IsProduct` is *data* (a structure in `Type`, because `lift` is a function), not just a proposition: a proof that something is a product comes with the recipe for pairing.

Pairs are products

Now the theorem: Lean's pair type, with its projections, is a product in the category of types.

```
/-- The pair type `α × β`, with `Prod.fst` and `Prod.snd`, is a product in the
category of types. -/
def pairIsProduct (α β : Type) :
    IsProduct (Obj := Type) (Prod.fst : α × β → α) Prod.snd where
  lift f g := fun x => (f x, g x)
  fst_lift _ _ := rfl
  snd_lift _ _ := rfl
  uniq f g h hf hg := by
    -- hf : Prod.fst ≫ h = f      hg : Prod.snd ≫ h = g
    funext x
    -- Goal: h x = (f x, g x). Two pairs are equal if their components are.
    apply Prod.ext
    · exact congrFun hf x      -- (h x).1 = f x
    · exact congrFun hg x      -- (h x).2 = g x
```

The named argument `(Obj := Type)` tells Lean in which category we work. Without it Lean would have to guess the category from `Prod.fst`, whose type is a plain function type. Then the fields:

- `lift f g` is the function that runs both `f` and `g` on the same input and pairs the results.
- `fst_lift` and `snd_lift` hold by computation: `(f x, g x).1` is `f x`.
- `uniq` is the real content. We have `h : x ⟶ α × β` with `hf : Prod.fst ≫ h = f` and `hg : Prod.snd ≫ h = g`, and must show `h = lift f g`. `funext x` reduces this to `h x = (f x, g x)`. `apply Prod.ext` says that two pairs are equal when their first components are equal and their second components are equal, leaving two goals. The first is `(h x).1 = f x`, which is `hf` evaluated at `x`: `congrFun hf x`. The second is the same with `hg`.

Uniqueness deserves a moment's thought: it says that *a function into a pair type is determined by its two components*. There is no hidden third piece of information that a pair could carry. The lift computes:

```
/-- The lift of two functions runs both on the same input. -/
def squareAndLabel : ℕ ⟶ ℕ × String :=
  (pairIsProduct ℕ String).lift (fun n => n * n) (fun n => s!"#{n}")

/-- info: (49, "#7") -/
#guard_msgs in
#eval squareAndLabel 7
```

Products in Rust, and the price of a copy

In Rust the pairing function looks like this:

```
/// The pairing of the universal property of products: from `f: X -> A` and
/// `g: X -> B`, the unique `X -> (A, B)` whose projections are `f` and `g`.
/// Rust needs `X: Clone` to feed the same input to both functions.
fn pair<X: Clone, A, B>(
    f: impl Fn(X) -> A,
    g: impl Fn(X) -> B,
) -> impl Fn(X) -> (A, B) {
    move |x| (f(x.clone()), g(x))
}
```

Look at the bound `X: Clone`. The lift must give *the same input* to both `f` and `g`, and a function `Fn(X) -> A` consumes its argument, so `x` must be duplicated. In Lean using a variable twice is free; in Rust it is an operation that not every type supports. (Taking `Fn(&X)` would share a reference instead, a different trick with its own theory, the subject of Part VI.) The example checks the two triangles and the uniqueness condition on samples:

```
let square = |n: u32| n * n;
let label = |n: u32| format!("#{n}");
let both = pair(square, label);
for n in [0, 1, 7, 12] {
    // The two triangles of the product diagram commute.
    assert_eq!(both(n).0, square(n));
    assert_eq!(both(n).1, label(n));
    // Any other `h` with the same projections agrees with `pair`.
    let h = |n: u32| (n.pow(2), format!("#{}", n));
    assert_eq!(h(n), both(n));
}
println!("product: pair(square, label)(7) = {:?}", both(7));

let describe = either(
    |n: u32| format!("number {n}"),
    |s: &str| format!("word {s}"),
);
println!("coproduct: {}", describe(Either::Left(42)));
println!("coproduct: {}", describe(Either::Right("hello")));

let units: Vec<()> = ["a", "b"].into_iter().map(to_unit).collect();
println!(
    "terminal: to_unit maps {} strings to {:?}",
    units.len(),
    units
);
let nothing: Vec<Void> = Vec::new();
let numbers: Vec<u32> = nothing.into_iter().map(absurd).collect();
println!("initial: absurd maps no values to {numbers:?}");
```

```
product: pair(square, label)(7) = (49, "#7")
coproduct: number 42
coproduct: word hello
terminal: to_unit maps 2 strings to [(), ()]
initial: absurd maps no values to []
```

The candidate `h` was written differently (`n.pow(2)` and a different `format!`), and it agrees with `pair` on every sample, as uniqueness predicts.

The need for `Clone` is not a detail. The morphism $a \to a \times a$ that pairs the identity with itself, $\langle \mathrm{id}, \mathrm{id} \rangle$, is called the **diagonal**, and it is the categorical name for *using a value twice*. Logic calls the same thing

contraction. Rust's ownership system is, among other things, a refusal to provide the diagonal for free. Chapter 19 makes this precise, and Part V builds the whole affine story on it.

Products are unique

Like terminal objects, products are unique up to a unique isomorphism, and the proof has the same shape: everything follows from the uniqueness clause.

```
/-- Products are unique up to isomorphism: if `p` and `q` are both products of
`a` and `b`, the lifts of each other's projections are inverse to each other. -/
def IsProduct.iso {Obj : Type u} [Category Obj] {a b p q : Obj}
    {fst : p ⟶ a} {snd : p ⟶ b} {fst' : q ⟶ a} {snd' : q ⟶ b}
    (P : IsProduct fst snd) (Q : IsProduct fst' snd') : p ≅ q where
  hom := Q.lift fst snd
  inv := P.lift fst' snd'
  inv_hom := by
    -- Both `P.lift fst' snd' ≫ Q.lift fst snd` and `id p` are lifts of
    -- `fst` and `snd` through `p`, so both are equal to `P.lift fst snd`.
    have h₁ : P.lift fst' snd' ≫ Q.lift fst snd = P.lift fst snd :=
      P.uniq _ _ _
        (by rw [Category.assoc, P.fst_lift, Q.fst_lift])
        (by rw [Category.assoc, P.snd_lift, Q.snd_lift])
    have h₂ : Category.id p = P.lift fst snd :=
      P.uniq _ _ _ (Category.comp_id fst) (Category.comp_id snd)
    rw [h₁, h₂]
  hom_inv := by
    -- The same argument with the roles of `p` and `q` exchanged.
    have h₁ : Q.lift fst snd ≫ P.lift fst' snd' = Q.lift fst' snd' :=
      Q.uniq _ _ _
        (by rw [Category.assoc, Q.fst_lift, P.fst_lift])
        (by rw [Category.assoc, Q.snd_lift, P.snd_lift])
    have h₂ : Category.id q = Q.lift fst' snd' :=
      Q.uniq _ _ _ (Category.comp_id fst') (Category.comp_id snd')
    rw [h₁, h₂]

/-- The isomorphism commutes with the projections. -/
theorem IsProduct.iso_fst {Obj : Type u} [Category Obj] {a b p q : Obj}
    {fst : p ⟶ a} {snd : p ⟶ b} {fst' : q ⟶ a} {snd' : q ⟶ b}
    (P : IsProduct fst snd) (Q : IsProduct fst' snd') :
    fst' ≫ (P.iso Q).hom = fst :=
  Q.fst_lift fst snd
```

Suppose p (with `fst`, `snd`) and q (with `fst'`, `snd'`) are both products of a and b. Each product can lift the other's projections, which gives arrows $q \to p$ and $p \to q$. For `inv_hom` we must show that the round trip $p \to q \to p$ is the identity. The trick is to show that *both* the round trip and `id p` are lifts of `fst` and `snd` through p, so that `P.uniq` makes both equal to `P.lift fst snd`:

- `h₁`: the round trip followed by `fst` is `fst`. The `rw` chain regroups with `Category.assoc`, then uses `P.fst_lift` (the projection of the lift into p is `fst'`) and `Q.fst_lift` (the projection of the lift into q is `fst`). The same for `snd`. So by `P.uniq`, the round trip equals `P.lift fst snd`.
- `h₂`: `id p` followed by `fst` is `fst`, by `Category.comp_id`, and the same for `snd`. So `id p` also equals `P.lift fst snd`.
- `rw [h₁, h₂]` rewrites both sides of the goal into the same thing.

`iso_fst` adds that the isomorphism respects the projections; its proof is one of the triangles of `Q`, `Q.fst_lift`. So the pair type is not just *a* product of `α` and `β`; it is *the* product, in the only sense in which category theory can say "the".

16.9 Coproducts

Turn every arrow in the definition of a product around, and you get a **coproduct**: an object s with **injections** $\text{inl} : a \to s$ and $\text{inr} : b \to s$, such that for every $f : a \to x$ and $g : b \to x$ there is exactly one $[f, g] : s \to x$ with

$$[f, g] \circ \text{inl} = f \qquad [f, g] \circ \text{inr} = g$$

```
/-- `s`, with injections `inl` and `inr`, is a coproduct of `a` and `b`: every
pair of morphisms `f : a → x`, `g : b → x` factors through `s` in exactly one
way. The same definition as `IsProduct`, with every arrow reversed. -/
structure IsCoproduct {Obj : Type u} [Category Obj] {a b s : Obj}
    (inl : a → s) (inr : b → s) where
  /-- The morphism out of the coproduct built from `f` and `g`. -/
  desc : {x : Obj} → (a → x) → (b → x) → (s → x)
  /-- On the left injection, the desc behaves like `f`. -/
  desc_inl : ∀ {x : Obj} (f : a → x) (g : b → x), desc f g ∘ inl = f
  /-- On the right injection, the desc behaves like `g`. -/
  desc_inr : ∀ {x : Obj} (f : a → x) (g : b → x), desc f g ∘ inr = g
  /-- Any morphism with these two properties is the desc. -/
  uniq : ∀ {x : Obj} (f : a → x) (g : b → x) (h : s → x),
    h ∘ inl = f → h ∘ inr = g → h = desc f g
```

In the category of types, the coproduct is the sum type: Rust's `enum` with two variants, Lean's `α ⊕ β` with constructors `Sum.inl` and `Sum.inr`. The morphism out of the sum (`desc`, for "descend") handles each case with the corresponding function: it is a `match`.

```
/-- The sum type `α ⊕ β`, with `Sum.inl` and `Sum.inr`, is a coproduct in the
category of types. -/
def sumIsCoproduct (α β : Type) :
    IsCoproduct (Obj := Type) (Sum.inl : α → α ⊕ β) Sum.inr where
  desc f g := fun s => match s with
    | .inl a => f a
    | .inr b => g b
  desc_inl _ _ := rfl
  desc_inr _ _ := rfl
  uniq f g h hf hg := by
    -- hf : h ∘ Sum.inl = f      hg : h ∘ Sum.inr = g
    funext s
    cases s with                          -- a value of `α ⊕ β` is `inl a` or `inr b`
    | inl a => exact congrFun hf a        -- h (inl a) = f a
    | inr b => exact congrFun hg b        -- h (inr b) = g b
```

Uniqueness is proved by cases: after `funext s`, a value of `α ⊕ β` is either `inl a` or `inr b`, and in each case the hypothesis about the corresponding triangle, evaluated at that value, is exactly the goal. The proof says that *a function out of a sum is determined by what it does on each variant*, which is why exhaustive `match` is a complete way to consume an `enum`.

In Rust:

```
/// A sum type with two cases: the coproduct of `A` and `B`.
#[derive(Debug)]
enum Either<A, B> {
    Left(A),
    Right(B),
}

/// The copairing: from `f: A -> X` and `g: B -> X`, the unique
/// `Either<A, B> -> X` that behaves like `f` on `Left` and like `g` on `Right`.
/// No `Clone` is needed: each input is used once.
fn either<A, B, X>(
    f: impl Fn(A) -> X,
```

```
    g: impl Fn(B) -> X,
) -> impl Fn(Either<A, B>) -> X {
    move |e| match e {
        Either::Left(a) => f(a),
        Either::Right(b) => g(b),
    }
}
```

No `Clone` this time: the input goes to exactly one of the two functions. Products and coproducts are perfectly dual in the category of types, but not in Rust: pairing needs a copy, case analysis does not. This asymmetry, the shadow of the diagonal, reappears in Part V as the difference between the two conjunctions of linear logic.

16.10 Duality and the opposite category

You have now seen several pairs of definitions that are mirror images: terminal and initial, product and coproduct, mono and epi. Category theory makes the mirror into a construction. The **opposite** of a category $\mathcal{C}$, written $\mathcal{C}^{\mathrm{op}}$, has the same objects and every arrow reversed: a morphism $a \to b$ in $\mathcal{C}^{\mathrm{op}}$ is a morphism $b \to a$ in $\mathcal{C}$, and composition is done in the other order.

```
/-- The opposite of a category has the same objects and every arrow reversed.
`Op C` is a copy of `C`, so that the two category structures do not clash. -/
structure Op (Obj : Type u) where
  /-- The object of the original category. -/
  unop : Obj

instance opCategory {Obj : Type u} [Category Obj] : Category (Op Obj) where
  Hom a b := b.unop → a.unop
  id a := Category.id a.unop
  comp g f := f ⊚ g
  id_comp f := Category.comp_id f
  comp_id f := Category.id_comp f
  assoc h g f := (Category.assoc f g h).symm
```

`Op Obj` is a copy of `Obj` (one field, `unop`), so that the opposite category is a different instance on a different type and the two structures never clash. `Hom a b` is `b.unop → a.unop`, reversed. The identity is the old identity; `comp g f` is `f ⊚ g`. The laws come from the original ones: the left unit law of the opposite is the right unit law of the original (and vice versa), and associativity is the original associativity read backwards (`.symm`).

The **duality principle** follows: a terminal object of $\mathcal{C}^{\mathrm{op}}$ is an initial object of $\mathcal{C}$, a product in $\mathcal{C}^{\mathrm{op}}$ is a coproduct in $\mathcal{C}$, a mono in $\mathcal{C}^{\mathrm{op}}$ is an epi in $\mathcal{C}$. So every theorem about terminal objects, applied to the opposite category, *is* a theorem about initial objects. In Lean we can watch this happen:

```
/-- An initial object of `C` is a terminal object of the opposite category. -/
def IsInitial.op {Obj : Type u} [Category Obj] {i : Obj} (h : IsInitial i) :
    IsTerminal (⟨i⟩ : Op Obj) where
  arrow a := h.arrow a.unop       -- a morphism `a → ⟨i⟩` in `Op` is `i → a.unop`
  unique f := h.unique f

/-- An isomorphism in the opposite category is an isomorphism in `C`, reversed. -/
def Iso.unop {Obj : Type u} [Category Obj] {a b : Obj}
    (i : (⟨a⟩ : Op Obj) ≅ ⟨b⟩) : b ≅ a where
  hom := i.hom
  inv := i.inv
  inv_hom := i.hom_inv
```

```
  hom_inv := i.inv_hom

/-- Initial objects are unique up to isomorphism: the dual of `IsTerminal.iso`,
obtained by applying it in the opposite category. No new proof is needed. -/
def IsInitial.iso {Obj : Type u} [Category Obj] {i₁ i₂ : Obj}
    (h₁ : IsInitial i₁) (h₂ : IsInitial i₂) : i₁ ≅ i₂ :=
  (IsTerminal.iso h₂.op h₁.op).unop
```

`IsInitial.op` repackages an initial object of `C` as a terminal object of `Op C`; the fields are the same terms, because the types agree once `Op` is unfolded. `Iso.unop` turns an isomorphism in the opposite category back into one in `C`, swapping `inv_hom` and `hom_inv`. And `IsInitial.iso`, the uniqueness of initial objects, has **no proof of its own**: it is `IsTerminal.iso`, applied in the opposite category. As an example of its use:

```
/-- For example, any type that is initial is isomorphic to `Empty`. -/
def initialIsoEmpty {α : Type} (h : IsInitial α) : α ≅ Empty :=
  h.iso emptyInitial
```

Duality halves the work of category theory. (It also explains the remark about the string monoid: reversing the order of concatenation gives the opposite category.)

16.11 Putting it together: one definition, many products

A universal property is a single definition with many implementations. Let us run this chapter's definitions on two more categories.

In `ℕ` ordered by `≤`, what is a product of `a` and `b`? It is an object p with morphisms $p \to a$ and $p \to b$ (so $p \le a$ and $p \le b$), such that any x with $x \le a$ and $x \le b$ has a morphism to p (so $x \le p$). That is the *greatest lower bound*: `min a b`. Dually, the coproduct is `max a b`.

```
/-- In `ℕ` ordered by `≤`, the candidate projections out of `min a b`... -/
def minFst (a b : ℕ) : min a b → a := ⟨Nat.min_le_left a b⟩
def minSnd (a b : ℕ) : min a b → b := ⟨Nat.min_le_right a b⟩

/-- ...make `min a b` a product of `a` and `b`. -/
def minIsProduct (a b : ℕ) : IsProduct (minFst a b) (minSnd a b) where
  lift f g := ⟨Nat.le_min.mpr ⟨f.down, g.down⟩⟩   -- x ≤ a and x ≤ b give x ≤ min a b
  fst_lift _ _ := preorder_thin _ _
  snd_lift _ _ := preorder_thin _ _
  uniq _ _ _ _ _ := preorder_thin _ _

/-- Dually, `max a b` is a coproduct. -/
def maxInl (a b : ℕ) : a → max a b := ⟨Nat.le_max_left a b⟩
def maxInr (a b : ℕ) : b → max a b := ⟨Nat.le_max_right a b⟩

def maxIsCoproduct (a b : ℕ) : IsCoproduct (maxInl a b) (maxInr a b) where
  desc f g := ⟨Nat.max_le.mpr ⟨f.down, g.down⟩⟩   -- a ≤ x and b ≤ x give max a b ≤ x
  desc_inl _ _ := preorder_thin _ _
  desc_inr _ _ := preorder_thin _ _
  uniq _ _ _ _ _ := preorder_thin _ _
```

All the equations (`fst_lift`, `uniq` and so on) are discharged by `preorder_thin`: in a thin category, parallel arrows are always equal, so the triangles commute and uniqueness holds automatically. The entire content of the universal property is in `lift`, which needs the lemma `Nat.le_min` ("$x \le \min(a, b)$ if and only if $x \le a$ and $x \le b$"); `.mpr` takes the right-to-left direction of the `↔`.

In `Prop` ordered by implication, the product is conjunction and the coproduct is disjunction:

```
/-- In `Prop` ordered by implication, `p ∧ q` is a product of `p` and `q`... -/
def andFst (p q : Prop) : (p ∧ q) → p := ⟨fun h => h.1⟩
def andSnd (p q : Prop) : (p ∧ q) → q := ⟨fun h => h.2⟩

def andIsProduct (p q : Prop) : IsProduct (andFst p q) (andSnd p q) where
  lift f g := ⟨fun hx => ⟨f.down hx, g.down hx⟩⟩
  fst_lift _ _ := preorder_thin _ _
  snd_lift _ _ := preorder_thin _ _
  uniq _ _ _ _ _ := preorder_thin _ _

/-- ...and `p ∨ q` is a coproduct. -/
def orInl (p q : Prop) : p → (p ∨ q) := ⟨Or.inl⟩
def orInr (p q : Prop) : q → (p ∨ q) := ⟨Or.inr⟩

def orIsCoproduct (p q : Prop) : IsCoproduct (orInl p q) (orInr p q) where
  desc f g := ⟨fun h => h.elim f.down g.down⟩
  desc_inl _ _ := preorder_thin _ _
  desc_inr _ _ := preorder_thin _ _
  uniq _ _ _ _ _ := preorder_thin _ _
```

The lift of `andIsProduct` is the familiar introduction rule for $\wedge$ from Chapter 4: from proofs of $x \to p$ and $x \to q$, a proof of $x \to p \wedge q$. The projections are the elimination rules, and the coproduct's `desc` is the elimination rule for $\vee$ (proof by cases). Natural deduction's rules for conjunction and disjunction *are* the universal properties of products and coproducts in the thin category of propositions.

Here is the comparison, with Rust's approximate answers in the last row:

Category	Product	Coproduct	Terminal	Initial
Lean types	`α × β`	`α ⊕ β`	`Unit`	`Empty`
`ℕ` with `≤`	`min a b`	`max a b`	none	`0`
`Prop` with `→`	`p ∧ q`	`p ∨ q`	`True`	`False`
Rust types (pure code)	`(A, B)`, with `Clone`	`enum`	`()`	`Void`, `!`

This is Chapter 1's table of three readings, now with a precise meaning for the third column: each entry is defined by the *same* universal property, in a different category. The rows also explain why the same algebra keeps showing up: min distributes over max, $\wedge$ over $\vee$, and `A × (B ⊕ C)` is isomorphic to `(A × B) ⊕ (A × C)` (Chapter 11); all three are one statement, "products distribute over coproducts", in three categories. Chapter 19 adds the missing column, the exponential (function types, implication), and shows that Chapter 5's Heyting algebras are exactly the thin categories that have all of these.

16.12 In Mathlib

Everything in this chapter has a counterpart in Mathlib's category theory library. The differences are generality (morphisms in any universe), automation, and one notational choice that you must get used to.

`CategoryTheory.Category`

Mathlib's class is `CategoryTheory.Category C`, for a type of objects `C`. Its morphisms come from a more primitive class, `Quiver` (a type of arrows between any two objects, without composition), and live in a universe `v` that you can specify as `Category.{v} C`. Its notation:

- `X ⟶ Y` is the type of morphisms, as in the book;
- `𝟙 X` (typed `\b1`) is the identity on `X`;
- `f ≫ g` (typed `\gg`) is composition **in diagrammatic order**: first `f`, then `g`. It is the book's `g ⊚ f`.

The diagrammatic order matches how arrows are drawn and how pipelines are written (`f ≫ g` reads like `x.f().g()`). The laws `id_comp`, `comp_id` and `assoc` have *default proofs*: an instance that omits them makes Mathlib try a tactic, `cat_disch`, that proves simple ones automatically. They are also `simp` lemmas, so `simp` normalizes identities and brackets:

```
/-- A statement about every Mathlib category. `f ≫ g` is "first `f`, then `g`":
the diagrammatic order, the reverse of `g ∘ f`. -/
example {C : Type*} [Category C] {X Y Z : C} (f : X ⟶ Y) (g : Y ⟶ Z) :
    𝟙 X ≫ (f ≫ g) ≫ 𝟙 Z = f ≫ g := by
  rw [Category.id_comp, Category.comp_id]

/-- The same goal, closed by `simp`: the unit and associativity laws are
`simp` lemmas in Mathlib. -/
example {C : Type*} [Category C] {W X Y Z : C}
    (f : W ⟶ X) (g : X ⟶ Y) (h : Y ⟶ Z) :
    (𝟙 W ≫ f) ≫ (g ≫ h) = f ≫ g ≫ h := by
  simp
```

These examples, and the ones below, are in `code/lean/Book/Ch16/InMathlib.lean`, under `open CategoryTheory` (`𝟙` and `≫` are scoped to that namespace, like the book's notations are to `Ch16`).

The two definitions describe the same notion, and every category of the book's class can be translated into Mathlib's. The translation, in `code/lean/Book/Ch16/Bridge.lean`, shows exactly what the change of order does:

```
/-- The book's category class, translated into Mathlib's. Mathlib's `f ≫ g`
("first `f`, then `g`") is the book's `g ⊚ f`. -/
abbrev toMathlib (C : Type u) [Category C] : CategoryTheory.Category.{0} C where
  Hom a b := Category.Hom a b
  id a := Category.id a
  comp f g := Category.comp g f
  -- Mathlib's `𝟙 a ≫ f = f` is the book's `f ⊚ id a = f`: our `comp_id`.
  id_comp f := Category.comp_id f
  -- Mathlib's `f ≫ 𝟙 b = f` is the book's `id b ⊚ f = f`: our `id_comp`.
  comp_id f := Category.id_comp f
  -- Mathlib's `(f ≫ g) ≫ h = f ≫ (g ≫ h)` is the book's
  -- `h ⊚ (g ⊚ f) = (h ⊚ g) ⊚ f`: our `assoc`, unchanged.
  assoc f g h := Category.assoc h g f
```

```
/-- Mathlib's composition in the translated category is the book's composition
with the arguments swapped, by definition. -/
example (C : Type u) [Category C] {a b c : C}
    (f : Category.Hom a b) (g : Category.Hom b c) :
    (toMathlib C).comp f g = Category.comp g f :=
  rfl
```

Because the order of composition is reversed, Mathlib's left unit law is the book's right unit law and vice versa, while associativity survives unchanged. `.{0}` fixes Mathlib's morphism universe to `Type`, which

is where the book's `Hom` lives, and `abbrev` marks the definition as reducible, as Lean recommends for definitions whose type is a class.

> **WARNING**
>
> Mathlib's `→` is *global* notation, while the book's is scoped to `Ch16`. Inside `namespace Ch16` (or after `open Ch16`) in a file that imports Mathlib's category theory, `X → Y` is therefore ambiguous and Lean reports an error. That is why `InMathlib.lean` does not import the book's class, and why `Bridge.lean` never writes the arrow. In your own files, keep the two worlds apart, or write the book's arrow as `Ch16.Category.Hom X Y`.

`Type u` as a category

Mathlib makes `Type u` a category for every universe `u`, and here it made a choice that differs from ours: **a morphism `X → Y` in `Type u` is not literally a function**, but a one-field structure (`TypeCat.Hom`) that wraps one. The wrapper keeps Lean from confusing morphisms with functions during automation, which matters in a library of this size. You build a morphism from a function with `↾f`, and you apply a morphism to an element as if it were a function, through a coercion:

```
/-- In Mathlib, a morphism of `Type` is not literally a function: it is a
one-field structure wrapping one. `↾f` turns a function into a morphism. -/
def lenM : String → ℕ := ↾String.length
def doubleM : ℕ → ℕ := ↾(fun n => 2 * n)

/-- A morphism can still be applied to an argument, through a coercion. -/
example : (lenM ≫ doubleM) "hello" = 10 := rfl

/-- info: 10 -/
#guard_msgs in
#eval (lenM ≫ doubleM) "hello"

/-- Applying a composite applies the two functions in diagrammatic order. -/
example {X Y Z : Type} (f : X → Y) (g : Y → Z) (x : X) : (f ≫ g) x = g (f x) := by
  simp
```

The last example is the simp lemma that makes this convenient: `(f ≫ g) x = g (f x)`, the diagrammatic order turned into ordinary application.

Preorders, monos and epis

Every preorder is a category in Mathlib too, with `homOfLE` turning a proof of `x ≤ y` into a morphism and `leOfHom` going back. Thinness is the statement that the type of morphisms is a `Subsingleton` (it has at most one element):

```
/-- Every preorder is a category in Mathlib too; `homOfLE` and `leOfHom`
convert between proofs of `≤` and morphisms. -/
def threeToFive : (3 : ℕ) → 5 := homOfLE (by decide)
def fiveToEight : (5 : ℕ) → 8 := homOfLE (by decide)

example : (3 : ℕ) ≤ 8 := leOfHom (threeToFive ≫ fiveToEight)

/-- Parallel morphisms of a preorder are equal (`Subsingleton.elim`). -/
example (f g : (3 : ℕ) → 8) : f = g := Subsingleton.elim f g
```

Mono and epi are classes (`Mono f`, `Epi f`) and the theorems we proved by hand are in the library:

```
/-- Mathlib's versions of the two theorems we proved by hand. -/
example {X Y : Type} (f : X → Y) : Mono f ↔ Function.Injective f :=
  mono_iff_injective f

example {X Y : Type} (f : X → Y) : Epi f ↔ Function.Surjective f :=
  epi_iff_surjective f
```

Products through `Limits`

Mathlib does not define products on their own. Terminal objects, products, equalizers, pullbacks and many other universal constructions are all special cases of one notion, a **limit**, whose definition needs functors and natural transformations (Chapters 17 and 18). Products are limits of a diagram with two objects and no arrows: a discrete category with two objects, which is why we said those trivial-looking categories are building blocks. You can use the library without knowing the general definition:

```
/-- In any category with binary products, `prod.lift f g : X → A × B` is the
morphism of the universal property, and these are its two equations. -/
example {C : Type*} [Category C] [HasBinaryProducts C] {X A B : C}
    (f : X → A) (g : X → B) :
    prod.lift f g ≫ prod.fst = f ∧ prod.lift f g ≫ prod.snd = g :=
  ⟨prod.lift_fst f g, prod.lift_snd f g⟩

/-- Uniqueness: a morphism into a product is determined by its projections. -/
example {C : Type*} [Category C] [HasBinaryProducts C] {X A B : C}
    (f : X → A) (g : X → B) (h : X → A × B)
    (h₁ : h ≫ prod.fst = f) (h₂ : h ≫ prod.snd = g) : h = prod.lift f g := by
  apply prod.hom_ext   -- it suffices to compare the two projections
  · simp [h₁]          -- h ≫ fst = f = prod.lift f g ≫ fst
  · simp [h₂]          -- h ≫ snd = g = prod.lift f g ≫ snd

/-- In `Type`, the pair type is a limit (a product) in Mathlib's sense... -/
example (X Y : Type) : IsLimit (Types.binaryProductCone X Y) :=
  Types.binaryProductLimit X Y

/-- ...so the abstract product `X × Y` is isomorphic to `X × Y`... -/
noncomputable example (X Y : Type) : X × Y ≅ X × Y :=
  Types.binaryProductIso X Y

/-- ...and `PUnit` is terminal. -/
example : IsTerminal PUnit := Types.isTerminalPUnit
```

- `HasBinaryProducts C` says that every two objects of `C` have a chosen product, written `A × B` (a special symbol, typed `\X`, not the `×` of pair types). `prod.lift`, `prod.fst` and `prod.snd` are our `lift`, `fst` and `snd`, and `prod.lift_fst` and `prod.lift_snd` are the two triangles.
- Uniqueness takes the form `prod.hom_ext`: two morphisms into a product are equal if their projections are. Our `uniq` follows, as the second example shows: `apply prod.hom_ext` leaves the two projection equations, and `simp` closes each with the hypothesis and `prod.lift_fst` (or `prod.lift_snd`).
- In `Type`, `Types.binaryProductLimit` is our `pairIsProduct`, so the library's `X × Y` is isomorphic to `X × Y`. That isomorphism is `noncomputable`: the library's product is chosen by a non-constructive principle, so Lean has no program to run.
- `Types.isTerminalPUnit` says that `PUnit` (a `Unit` that exists in every universe) is terminal.

16.13 Common confusions

"**The objects are sets and the morphisms are functions.**" Only in categories like the category of types. In a preorder the morphisms are proofs, in a monoid they are elements, in a free category they are paths.

"**$g \circ f$ means first g.**" It means first f: $(g \circ f)(x) = g(f(x))$. The book's `g ⊙ f` and `comp g f` follow this convention. Mathlib's `f ≫ g` uses the opposite, diagrammatic order. When you move between the two, the unit laws swap names.

"**A product is the pair type.**" A product is *anything* with the universal property: `min`, `∧`, pairs. Within one category all products of the same two objects are isomorphic, in a unique way that respects the projections, so it does not matter which one you pick.

"**Rust's `(A, B)` is a categorical product.**" Only if you can duplicate the input of the pairing: the diagonal $A \to A \times A$ needs `Clone`. For types that cannot be copied, `(A, B)` is a weaker structure that Chapter 19 calls a *monoidal* product. The coproduct, `enum`, has no such problem.

"**Mono means injective, and mono plus epi means isomorphism.**" True in the category of types, false in general: in `ℕ` with `≤`, every arrow is mono and epi, and only the identities are isomorphisms.

16.14 Summary

- A category has objects, morphisms, identities and an associative composition with the identities as units; composition is only defined when arrows meet.
- The book's `Ch16.Category` class states the laws as fields, so every instance carries proofs; its `Hom` lives in `Type`, a simplification of Mathlib's universe-polymorphic version.
- Types and functions, preorders (thin categories), monoids (one object), discrete categories (only identities) and free categories (paths, such as the traces of a protocol) are all categories. Rust can represent them as runtime data and test the laws; Lean checks composability by types and proves the laws.
- A commutative diagram is an equation between composites: a property test with a proof.
- Inverses are unique. In the category of types, monos are the injective and epis the surjective functions, but mono and epi together do not make an isomorphism in general.
- Terminal and initial objects, products and coproducts are defined by universal properties, like interface specifications, and are unique up to unique isomorphism. In Lean's types they are `Unit`, `Empty`, pairs and sums; in `ℕ`, `0`, `min` and `max`; in `Prop`, `True`, `False`, `∧` and `∨`.
- The opposite category reverses all arrows, and duality turns every definition and theorem into its mirror image for free.
- Rust's pairing of functions needs `Clone` (the diagonal is contraction), and side effects break the uniqueness of maps into `()`.
- Mathlib writes `X → Y`, `𝟙 X` and `f ≫ g` (diagrammatic order), wraps the morphisms of `Type u` in a structure, and defines products as limits.

16.15 Exercises

1. Prove in Lean that every isomorphism is a monomorphism, `theorem iso_mono (i : a ≅ b) : Mono i.hom`, and then that it is an epimorphism. *Hint:* given `i.hom ≫ g = i.hom ≫ h`, compose both sides with `i.inv` on the left: a `calc` chain from `g` to `Category.id a ≫ g` to `(i.inv ≫ i.hom) ≫ g`, regrouped with `Category.assoc`.
2. Prove that coproducts are unique up to isomorphism, once directly (copying `IsProduct.iso`) and once by duality, through a definition `IsCoproduct.op : IsCoproduct inl inr → IsProduct ...` in the opposite category. *Hint:* the fields of the product in `Op Obj` are those of the coproduct, unchanged (`lift f g := h.desc f g`, and so on); named arguments such as `(a := (⟨a⟩ : Op Obj))` tell Lean which objects you mean.
3. Define in Lean the category `FinObj` whose objects are natural numbers (wrapped in a structure) and whose morphisms from `m` to `n` are the functions `Fin m → Fin n` (where `Fin n` is the type of numbers below `n`). Then show that `⟨0⟩` is initial and `⟨1⟩` is terminal. *Hint:* the instance is a copy of `typeCategory`, with every law `rfl`. For the universal properties, `Fin 0` has no elements (`nomatch` or `Fin.elim0`) and every element of `Fin 1` is `0` (`Fin.ext` and `omega`, or `Subsingleton.elim`).
4. In Rust, implement the `Category` trait for **matrices**: objects are dimensions (`usize`), a morphism $n \to m$ is an $m \times n$ matrix of `i64`, composition is matrix multiplication, identities are identity matrices. Check the laws with `check_laws` on a few small matrices. *Hint:* store each matrix with its dimensions. For $f : n \to m$ and $g : m \to k$, $g \circ f$ is the product GF, a $k \times n$ matrix; return `None` when the inner dimensions differ.
5. In the preorder of natural numbers ordered by divisibility (now including 0), what are the product and the coproduct of a and b? Which number is initial, and which is terminal? Compare with the table in "Putting it together". *Hint:* translate "a morphism $x \to a$" into "x divides a" and read the universal property aloud. Every number divides 0.
6. The Rust function `pair` requires `X: Clone`. Write a version `pair_ref` whose functions take `&X`, so that no bound is needed, and explain why it does not contradict the claim that the diagonal needs a copy. *Hint:* what is duplicated now? A shared reference `&X` is `Copy`, whatever `X` is (Chapters 23 and 26 explain why).

CHAPTER 17

Functors and Higher-Kinded Types

Rust's standard library is full of methods called `map`. `Option::map` applies a function to the value inside a `Some`. `Result::map` applies it to the value inside an `Ok`. `Iterator::map` applies it to every element; arrays have `map`, and so does `Poll`. Each of these is written separately, for one type, and yet they all behave the same way: mapping the identity function changes nothing, and mapping `f` and then `g` is the same as mapping "`f`, then `g`" once. If you have ever rewritten `.map(f).map(g)` as `.map(|x| g(f(x)))` without a second thought, you relied on that second rule.

That common shape has a name: a **functor**. Functors are the second fundamental notion of category theory, after categories themselves. A functor is a map from one category to another that preserves what makes a category a category: identities and composition. Every `map` method above is the "arrow part" of a functor from the category of types to itself.

And here is the twist. Rust has no `Functor` trait, and cannot have one in the form other languages do, because a trait would have to be generic over `Option` itself, not over `Option<i32>`. `Option` is not a type; it is a *type constructor*, a function from types to types, and Rust's generics cannot range over those. This missing feature is called **higher-kinded types**. This chapter explains kinds, defines functors with their laws and proves them in Lean, shows how far Rust can get with generic associated types, and then generalizes in two directions: functors that reverse arrows, and functors between arbitrary categories.

In this chapter you will learn:

- what kinds are (`Type`, `Type → Type`, `Type → Type → Type`), what a type constructor is, and what partial application means for types;
- the definition of a functor between categories, and the functor laws, for `Option`, `Vec`, `Result<_, E>`, `(E, _)` and `fn(E) -> _`;
- a `Functor` class with the laws as fields, with instances proved for `Option`, `List` and a binary tree, and core Lean's `Functor` and `LawfulFunctor`;
- why Rust has no `Functor` trait, two encodings with generic associated types, and exactly where each one falls short;
- contravariant functors (predicates, comparators), bifunctors (pairs, `Result`) and profunctors (functions);
- functors between arbitrary categories of the book's `Category` class: monotone maps, the interpretation of a free category, hom functors;
- how variance (Chapter 27) is functoriality, and how Mathlib writes functors (`C ⇒ D`).

17.1 Kinds: the types of types

Values have types: `3` has type `i32`, `"hi"` has type `&str`. Do types have types? In a sense they do, and these "types of types" are called **kinds**. A type such as `i32` or `Vec<String>`, which classifies values, has the kind `Type` (some texts write $*$). `Option` on its own classifies no values: there is no value of type `Option`. It needs an argument first. So its kind is `Type → Type`: give it a type, get a type. `Result` needs two arguments and has kind `Type → Type → Type`.

Kinds are checked by a rule of the same shape as function application in the λ-calculus (Chapter 8):

$$\frac{F : \kappa_1 \to \kappa_2 \qquad A : \kappa_1}{F\,A : \kappa_2}\ \text{K-App}$$

With $F = \texttt{Option}$, $\kappa_1 = \kappa_2 = \texttt{Type}$ and $A = \texttt{i32}$, the rule concludes that `Option<i32>` is a type. Things of kind `Type → Type` (or longer) are called **type constructors**. Rust has many, and some take arguments that are not types:

Rust constructor	Kind
`Option`, `Vec`, `Box`, `Rc`	`Type → Type`
`Result`, `HashMap` (ignoring the hasher parameter)	`Type → Type → Type`
`&'a T`	`Lifetime → Type → Type`
`[T; N]`	`Type → usize → Type`

A **partial application** fixes some arguments and leaves the others open. `Result<_, io::Error>` has kind `Type → Type`: give it a success type, get a type. Rust has no syntax for this `_`, but a type alias with a parameter does the job, and the standard library uses one: `std::io::Result<T>` is defined as `Result<T, std::io::Error>`. What Rust does not have is a way to *pass* a type constructor around. A generic parameter always stands for a type of kind `Type`, so you cannot write a struct generic over "some container":

```
/// A column of a table, stored in a container of our choice (`Vec`,
/// `VecDeque`, ...): we would like to pass the container's type constructor.
struct Column<F> {
    values: F<String>,
}
```

```
error[E0109]: type arguments are not allowed on type parameter `F`
 --> compile_fail/ch17_no_hkt.rs:9:15
  |
9 |     values: F<String>,
  |             - ^^^^^^ type argument not allowed
  |             |
  |             not allowed on type parameter `F`
  |
note: type parameter `F` defined here
 --> compile_fail/ch17_no_hkt.rs:8:15
  |
8 | struct Column<F> {
  |               ^
```

The error code E0109 says that `F` is a type parameter, so it cannot take arguments. Rust has plenty of type constructors (every generic type is one, and `Column` itself has kind `Type → Type`), but a generic *parameter* always stands for a type (or a lifetime, or a constant), never for a constructor: you can

name a constructor only by applying it. The usual workaround is to parameterize by the *applied* type (`struct Column<C> { values: C }` with `C = Vec<String>`), which works until you need the same container at two different element types. That is exactly the situation of `map`.

Kinds in Lean

In Lean, type constructors are not special at all. `Type → Type` is an ordinary function type, and `Option` is an ordinary function, from types to types. The file `code/lean/Book/Ch17/Functor.lean` starts with:

```
/-- info: Option.{u} (α : Type u) : Type u -/
#guard_msgs in
#check Option

/-- info: Option ℕ : Type -/
#guard_msgs in
#check Option ℕ

/-- A type constructor of kind `Type → Type → Type`, partially applied. -/
example : Type → Type := Except String

/-- Type-level functions can be written with `fun`, like any other function. -/
def Twice : Type → Type := fun α => α × α

/-- info: (1, 2) -/
#guard_msgs in
#eval ((1, 2) : Twice ℕ)
```

`#check Option` prints `Option.{u} (α : Type u) : Type u`: `Option` takes a type `α` in any universe `u` (Chapter 12) and returns a type in the same universe. Applied to `ℕ` it gives `Option ℕ : Type`. `Except String` is a partial application of `Except : Type → Type → Type`, and it *is* a value of type `Type → Type`, accepted where one is expected. `Twice` is a type-level function written with `fun`, a *type-level* λ, which Rust does not have at all. The function type has no constructor name that could be partially applied, so to fix its input type we give it a name with a definition:

```
/-- Functions out of a fixed type `ε`: the kind is `Type → Type` once `ε` is
fixed. A named definition makes this partial application possible. -/
def Reader (ε α : Type) : Type := ε → α
```

`Reader ε` is "functions out of `ε`", of kind `Type → Type`. Because types are values in Lean, everything we do with functions (pass them as arguments, store them in structures, quantify over them) we can do with type constructors. That is all "higher-kinded types" means: being allowed to do that.

17.2 Functors

The definition

Let $\mathcal{C}$ and $\mathcal{D}$ be categories. A **functor** $F : \mathcal{C} \to \mathcal{D}$ consists of

- a map on objects: for every object a of $\mathcal{C}$, an object $F\,a$ of $\mathcal{D}$;
- a map on morphisms: for every $f : a \to b$ in $\mathcal{C}$, a morphism $F\,f : F\,a \to F\,b$ in $\mathcal{D}$;

such that identities and composites are preserved:

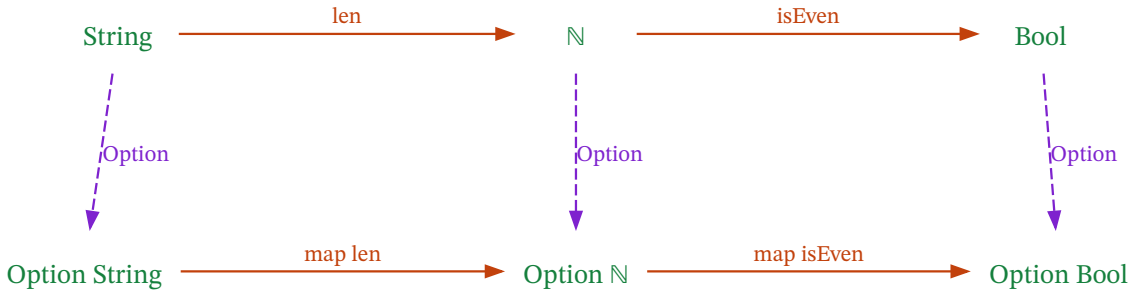
$$F\,\mathrm{id}_a = \mathrm{id}_{F\,a} \qquad\qquad F\,(g \circ f) = F\,g \circ F\,f$$

These are the **functor laws**. Notice the types: an arrow $f : a \to b$ goes to an arrow between the *images* of a and b. So whatever can be composed in $\mathcal{C}$ can still be composed after applying F, and the composite of the images is the image of the composite. A functor carries the whole web of arrows of $\mathcal{C}$ into $\mathcal{D}$ without tearing it.

A functor from a category to itself is an **endofunctor**. The endofunctors of the category of types are the ones programmers meet daily: a type constructor `F` (the map on objects, `α ↦ F α`) together with a function `map : (α → β) → F α → F β` (the map on morphisms). For them, the laws read

$$\mathrm{map}\ \mathrm{id} = \mathrm{id} \qquad\qquad \mathrm{map}\ (g \circ f) = \mathrm{map}\ g \circ \mathrm{map}\ f$$

The picture shows `Option` applied to the arrows `len` and `isEven` of Chapter 16. Every type in the top row has an image in the bottom row, every arrow has an image arrow, and composites go to composites.



The functor Option maps each type to a type and each function to a function between the images, preserving composition.

Endofunctors you already use

Here is what `map` does for five type constructors that every Rust programmer uses:

Constructor	`map f` does	In Rust
`Option<_>`	apply `f` inside `Some`, keep `None`	`Option::map`
`Vec<_>`	apply `f` to every element	`iter().map(f).collect()`
`Result<_, E>`	apply `f` inside `Ok`, keep `Err`	`Result::map`
`(E, _)`	apply `f` to the second component	(none: write it)
`fn(E) -> _`	compose `f` after the function	(none: a closure)

The last two have no `map` in the standard library, so the example program, `code/rust/examples/ch17_functors.rs`, defines them:

```
/// The standard library has no `map` for pairs; this one maps the second
/// component and keeps the first, a label of type `E`.
fn map_snd<E, A, B>((e, a): (E, A), f: impl FnOnce(A) -> B) -> (E, B) {
    (e, f(a))
}

/// Mapping over a function's result is composing after it.
fn map_fn<E, A, B>(r: impl Fn(E) -> A, f: impl Fn(A) -> B) -> impl Fn(E) -> B {
```

```
    move |e| f(r(e))
}
```

The pair functor `(E, _)` keeps a label of type `E` (a sensor name, a log message, a source position) attached to a value. The function functor `fn(E) -> _` describes computations that need an input of type `E` (a configuration, an environment) to produce their result; mapping over it means post-processing the result. Both are functors, as the example checks on samples:

```
let f = |n: i32| n + 1;
let g = |n: i32| n * 10;

// Option: `Option::map`.
for x in [None, Some(0), Some(7)] {
    assert_eq!(x.map(|a| a), x); // map id = id
    assert_eq!(x.map(|a| g(f(a))), x.map(f).map(g)); // map (g ∘ f) = map g ∘ map f
}
// Vec: through an iterator, `map`, then `collect`.
for v in [vec![], vec![1, 2, 3]] {
    let map = |v: &Vec<i32>, h: &dyn Fn(i32) -> i32| -> Vec<i32> {
        v.iter().map(|&a| h(a)).collect()
    };
    assert_eq!(map(&v, &|a| a), v);
    assert_eq!(map(&v, &|a| g(f(a))), map(&map(&v, &f), &g));
}
// Result<_, E>: `Result::map` changes the `Ok` value, leaves `Err` alone.
for r in [Ok(3), Err("bad input")] {
    assert_eq!(r.map(|a| a), r);
    assert_eq!(r.map(|a| g(f(a))), r.map(f).map(g));
}
// (E, _): a value with a label; map the value, keep the label.
let p = ("sensor-1", 4);
assert_eq!(map_snd(p, |a| a), p);
assert_eq!(map_snd(p, |a| g(f(a))), map_snd(map_snd(p, f), g));
// fn(E) -> _: mapping is post-composition; compare the results pointwise.
let reading = |sensor: u32| sensor as i32 * 3;
for e in [0, 5] {
    assert_eq!(map_fn(reading, |a| a)(e), reading(e));
    assert_eq!(
        map_fn(reading, |a| g(f(a)))(e),
        map_fn(map_fn(reading, f), g)(e)
    );
}
println!("laws: Option, Vec, Result, (E, _) and fn(E) -> _ pass on samples");
```

```
laws: Option, Vec, Result, (E, _) and fn(E) -> _ pass on samples
```

Each block checks the identity law (`map` with `|a| a` gives back the input) and the composition law (mapping the composite equals mapping twice) on a few samples. For `fn(E) -> _`, equality of functions cannot be tested, so we compare the two sides on sample inputs.

Laws that fail

Not every function called `map` is a functor. The laws rule out maps that do something besides applying the function. In Lean we can define such maps and *prove* that they break the laws:

```
/-- A `map` for `Option` that forgets everything. It satisfies the composition
law but not the identity law. -/
def forgetMap {α β : Type} (_ : α → β) (_ : Option α) : Option β := none

theorem forgetMap_comp {α β γ : Type} (f : α → β) (g : β → γ) (x : Option α) :
    forgetMap (g ∘ f) x = forgetMap g (forgetMap f x) := rfl
```

```
theorem forgetMap_not_id : ¬ ∀ x : Option ℕ, forgetMap id x = x := by
  intro h
  have h1 := h (some 1)   -- forgetMap id (some 1) = some 1, that is, none = some 1
  simp [forgetMap] at h1

/-- A `map` on pairs that counts how often it is called (like a Rust `map`
that also increments a counter). It breaks both laws. -/
def countingMap {α β : Type} (f : α → β) (p : ℕ × α) : ℕ × β := (p.1 + 1, f p.2)

theorem countingMap_not_id : countingMap id (0, "a") ≠ (0, "a") := by
  simp [countingMap]
```

`forgetMap` throws the contents away and always returns `none`. It satisfies the composition law (both sides are `none`, so `rfl` proves it), but not the identity law: applied to `some 1`, the identity law would claim `none = some 1`. The proof of `forgetMap_not_id` assumes the law for all `x`, specializes it to `some 1`, and `simp [forgetMap] at h1` unfolds `forgetMap`, sees that `none = some 1` is false, and closes the goal. So the identity law is what forbids "losing" data.

`countingMap` is the kind of `map` a Rust programmer might write to collect statistics: it applies the function and increments a counter stored next to the value. It breaks both laws; the theorem shows the identity law failing on one input, by `simp` computing both sides. The laws say that `map` must apply the function and do nothing else. In Rust you would also have to say "and must not print, panic or mutate", which the type of `map` cannot express.

Why do we care? Because the laws are what make `map` safe to refactor. Removing a `.map(|x| x)` is correct because of the identity law. Fusing `.map(f).map(g)` into one pass, or splitting one complicated closure into two `map` calls for readability, is correct because of the composition law. Code reviewers apply these rewrites without thinking, and the laws are the reason it is safe.

17.3 Functors in Lean

A class with laws

Here is a functor class in Lean, with the laws as fields, exactly like the `Category` class of Chapter 16:

```
/-- A functor on types: a type constructor `F` with a `map` that satisfies the
two functor laws. (This class shadows core Lean's `Functor` inside `Ch17`;
core's is `_root_.Functor`.) -/
class Functor (F : Type → Type) where
  /-- Apply a function to every value of type `α` inside an `F α`. -/
  map : {α β : Type} → (α → β) → F α → F β
  /-- Mapping the identity changes nothing. -/
  map_id : ∀ {α : Type} (x : F α), map id x = x
  /-- Mapping a composite is mapping one function, then the other. -/
  map_comp : ∀ {α β γ : Type} (f : α → β) (g : β → γ) (x : F α),
    map (g ∘ f) x = map g (map f x)
```

The parameter `F : Type → Type` is the type constructor: this is the higher-kinded quantification that Rust lacks, written as an ordinary argument. The field `map` has implicit type arguments `{α β : Type}`, so `map` works at every pair of types. The laws are stated *pointwise* ("for every `x : F α`") rather than as equations between functions; the two forms are equivalent by `funext`, and the pointwise form is

easier to use in proofs. The class is named `Functor`, like core Lean's own class; inside `namespace Ch17` our name wins, and core's is still available as `_root_.Functor`.

The instance for `Option`:

```
instance : Functor Option where
  map f
    | none => none
    | some a => some (f a)
  map_id x := by
    cases x <;> rfl          -- check `none` and `some a` separately
  map_comp f g x := by
    cases x <;> rfl
```

`map f` followed by two alternatives is a function defined by pattern matching on its last argument. Both laws are proved by `cases x <;> rfl`: split on whether `x` is `none` or `some a`, and in each case both sides compute to the same value. (`<;>` runs the tactic on its right on every goal produced by the tactic on its left.)

For lists, the proofs need induction:

```
/-- `map` on lists, written out (it is `List.map`). -/
def listMap {α β : Type} (f : α → β) : List α → List β
  | [] => []
  | a :: as => f a :: listMap f as

theorem listMap_id {α : Type} (xs : List α) : listMap id xs = xs := by
  induction xs with
  | nil => rfl
  | cons a as ih =>
    -- Goal: listMap id (a :: as) = a :: as
    simp only [listMap, id, ih]

theorem listMap_comp {α β γ : Type} (f : α → β) (g : β → γ) (xs : List α) :
    listMap (g ∘ f) xs = listMap g (listMap f xs) := by
  induction xs with
  | nil => rfl
  | cons a as ih =>
    -- Goal: (g ∘ f) a :: listMap (g ∘ f) as = g (f a) :: listMap g (listMap f as)
    simp only [listMap, Function.comp, ih]

instance : Functor List where
  map := listMap
  map_id := listMap_id
  map_comp := listMap_comp
```

`listMap` is `List.map`, written out so that every step of the proof is visible. In `listMap_id`, the `nil` case is `rfl`. In the `cons a as` case the goal is `listMap id (a :: as) = a :: as`; `simp only [listMap, id, ih]` unfolds one step of `listMap` (giving `id a :: listMap id as`), unfolds `id` (giving `a :: listMap id as`), and rewrites with the induction hypothesis `ih : listMap id as = as`, which leaves `a :: as = a :: as`, closed automatically. `listMap_comp` has the same structure; `Function.comp` unfolds `(g ∘ f) a` into `g (f a)`.

Trees are the third example, and the first that is not built into the language:

```
/-- Binary trees with values at the nodes. -/
inductive Tree (α : Type) where
  | leaf : Tree α
  | node : Tree α → α → Tree α → Tree α
  deriving Repr
```

```
def Tree.map {α β : Type} (f : α → β) : Tree α → Tree β
  | .leaf => .leaf
  | .node l a r => .node (l.map f) (f a) (r.map f)

theorem Tree.map_id {α : Type} (t : Tree α) : t.map id = t := by
  induction t with
  | leaf => rfl
  | node l a r ihl ihr =>
    -- Goal: (node l a r).map id = node l a r; one induction hypothesis per subtree
    simp only [Tree.map, id, ihl, ihr]

theorem Tree.map_comp {α β γ : Type} (f : α → β) (g : β → γ) (t : Tree α) :
    t.map (g ∘ f) = (t.map f).map g := by
  induction t with
  | leaf => rfl
  | node l a r ihl ihr => simp only [Tree.map, Function.comp, ihl, ihr]

instance : Functor Tree where
  map := Tree.map
  map_id := Tree.map_id
  map_comp := Tree.map_comp
```

`Tree.map` keeps the shape and applies `f` at every node. In the induction on a tree, the `node l a r` case has *two* induction hypotheses, one per subtree (`ihl` and `ihr`), and `simp only` uses both. This is structural induction on an inductive type, the general form of what you did on lists.

```
def sample : Tree ℕ := .node (.node .leaf 1 .leaf) 2 (.node .leaf 3 .leaf)

/--
info: Ch17.Tree.node
  (Ch17.Tree.node (Ch17.Tree.leaf) "1" (Ch17.Tree.leaf))
  "2"
  (Ch17.Tree.node (Ch17.Tree.leaf) "3" (Ch17.Tree.leaf))
-/
#guard_msgs in
#eval Functor.map toString sample
```

In `Functor.map toString sample`, Lean finds the `Functor Tree` instance and turns every number into a string, keeping the shape.

Three more instances, with the laws for free:

```
/-- `Except ε` maps the success value and leaves errors alone, like Rust's
`Result::map`. -/
instance (ε : Type) : Functor (Except ε) where
  map f
    | .error e => .error e
    | .ok a => .ok (f a)
  map_id x := by cases x <;> rfl
  map_comp f g x := by cases x <;> rfl

/-- `Prod ε` (pairs `ε × α`) maps the second component, like a Rust tuple
`(E, A)` with a label of type `E` attached to the value. -/
instance (ε : Type) : Functor (Prod ε) where
  map f p := (p.1, f p.2)
  map_id _ := rfl
  map_comp _ _ _ := rfl

/-- `Reader ε` maps the result: `map f r` is "first `r`, then `f`". -/
instance (ε : Type) : Functor (Reader ε) where
  map f r := fun e => f (r e)
  map_id _ := rfl
  map_comp _ _ _ := rfl
```

`Except ε` is Lean's `Result` with the error type first: `Except ε α` is either `.error e` or `.ok a`. Its `map`, like Rust's `Result::map`, changes the success value and leaves errors alone. `Prod ε` is the partial application of the pair type to its first argument, so `Prod ε α` is `ε × α`, the `(E, _)` functor. Its laws hold by `rfl`: `(p.1, id p.2)` is `p` by η for pairs. `Reader ε` maps by post-composition, and its laws are the unit and associativity laws of function composition, which hold by computation.

Proving things once for all functors

With the laws available as hypotheses, we can prove theorems about *every* lawful functor at once. The simplest is map fusion:

```
/-- Map fusion, for every lawful functor: mapping twice is mapping once with the
composite. A compiler (or an iterator adapter) may rewrite one into the other. -/
theorem map_map {F : Type → Type} [Functor F] {α β γ : Type}
    (f : α → β) (g : β → γ) (x : F α) :
    Functor.map g (Functor.map f x) = Functor.map (fun a => g (f a)) x :=
  (Functor.map_comp f g x).symm
```

`{F : Type → Type} [Functor F]` quantifies over all type constructors with a lawful `map`. The proof is the composition law, read from right to left. Rust can only *test* such a statement, one functor at a time.

Core Lean's `Functor` and `LawfulFunctor`

Lean's core library has its own `Functor` class, which `do` notation and many library functions use. It splits the operation from the laws: `Functor` has `map` (with the infix notation `<$>`) and `mapConst` (map a constant function, with a default implementation), and a separate class `LawfulFunctor` has the laws `id_map`, `comp_map` and `map_const` (the last one says that `mapConst` agrees with its default). Our tree joins in:

```
/-- Core Lean's class has only the operation; `<$>` is its notation. -/
instance : _root_.Functor Tree where
  map := Tree.map

/-- The laws live in a separate class, `LawfulFunctor`. -/
instance : LawfulFunctor Tree where
  map_const := rfl                          -- `mapConst` has its default definition
  id_map t := Tree.map_id t
  comp_map f g t := Tree.map_comp f g t

/-- info: Ch17.Tree.node (Ch17.Tree.leaf) 20 (Ch17.Tree.leaf) -/
#guard_msgs in
#eval (· * 10) <$> Tree.node .leaf 2 .leaf

/-- Lemmas proved once for all lawful functors now apply to `Tree`. -/
example (t : Tree ℕ) : (· + 1) <$> (· * 2) <$> t = (fun n => n * 2 + 1) <$> t := by
  simp only [Functor.map_map]
```

The proofs are our tree lemmas; `map_const` is `rfl` because we kept the default `mapConst`. With the `LawfulFunctor` instance, library lemmas proved once for every lawful functor apply to `Tree`: the last example uses `Functor.map_map`, core's version of map fusion, through `simp only`.

17.4 Functors in Rust: the missing trait

What we would like to write

In Haskell, the functor class is written (in Haskell syntax, not compiled here):

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

The class is indexed by `f`, a type constructor of kind `Type → Type`, and the types `f a` and `f b` apply it to two different arguments. Lean's class has the same shape. A direct Rust translation would need a trait generic over a type constructor, something like this (invalid Rust):

```
trait Functor<F<_>> {
    fn fmap<A, B>(x: F<A>, f: impl FnMut(A) -> B) -> F<B>;
}
```

As we saw, a Rust type parameter cannot be applied to arguments. Higher-kinded types have been discussed in the Rust community for years without being added; the difficulties are not only syntax, but also their interaction with type inference and with the trait system, as the encodings below will show. Instead, Rust gained a smaller feature that covers many of the use cases: **generic associated types**.

Encoding 1: a GAT on the applied type

An associated type (Chapter 14) is a type chosen by each `impl` of a trait, like `Iterator::Item`. A **generic associated type** (GAT, stable since Rust 1.65) is an associated type that takes parameters of its own. That is enough to say "the same container, with a different element type":

```
/// A `Functor` trait with a generic associated type. `Self` is an *applied*
/// type such as `Option<A>`; `Wrapped<B>` is meant to be "the same type
/// constructor, applied to `B`".
trait Functor: Sized {
    type Elem;
    type Wrapped<B>: Functor<Elem = B>;
    fn fmap<B>(self, f: impl FnMut(Self::Elem) -> B) -> Self::Wrapped<B>;
}

impl<A> Functor for Option<A> {
    type Elem = A;
    type Wrapped<B> = Option<B>;
    fn fmap<B>(self, f: impl FnMut(A) -> B) -> Option<B> {
        self.map(f)
    }
}

impl<A> Functor for Vec<A> {
    type Elem = A;
    type Wrapped<B> = Vec<B>;
    fn fmap<B>(self, f: impl FnMut(A) -> B) -> Vec<B> {
        self.into_iter().map(f).collect()
    }
}

/// `Result<A, E>` fixes `E` and maps `A`: the partial application `Result<_, E>`.
impl<A, E> Functor for Result<A, E> {
    type Elem = A;
    type Wrapped<B> = Result<B, E>;
    fn fmap<B>(self, f: impl FnMut(A) -> B) -> Result<B, E> {
        self.map(f)
    }
}

impl<A> Functor for Tree<A> {
    type Elem = A;
    type Wrapped<B> = Tree<B>;
    fn fmap<B>(self, mut f: impl FnMut(A) -> B) -> Tree<B> {
        map_tree(self, &mut f)
    }
```

```
}

/// Generic over the container: one function for every `Functor`.
fn double_all<F: Functor<Elem = i32>>(x: F) -> F::Wrapped<i32> {
    x.fmap(|n| n * 2)
}
```

The trait is implemented by an *applied* type such as `Option<A>`. `Elem` names the element type `A`, and the GAT `Wrapped<B>` names `Option<B>`: the constructor applied to another argument. The bound `Functor<Elem = B>` says that the result is again a functor, with element type `B`. The impl for `Result<A, E>` fixes `E` and maps `A`, which is the partial application `Result<_, E>` expressed through the impl. The tree uses a recursive helper:

```
/// A binary tree with values at the nodes.
#[derive(Clone, PartialEq, Debug)]
enum Tree<A> {
    Leaf,
    Node(Box<Tree<A>>, A, Box<Tree<A>>),
}

fn node<A>(left: Tree<A>, value: A, right: Tree<A>) -> Tree<A> {
    Tree::Node(Box::new(left), value, Box::new(right))
}

/// `map` for trees: apply `f` to every value, keeping the shape.
fn map_tree<A, B>(t: Tree<A>, f: &mut impl FnMut(A) -> B) -> Tree<B> {
    match t {
        Tree::Leaf => Tree::Leaf,
        Tree::Node(l, a, r) => {
            let l = map_tree(*l, f);
            let b = f(a);
            node(l, b, map_tree(*r, f))
        }
    }
}
```

The helper takes `f` by mutable reference so that the same closure can be used in both recursive calls: a generic `FnMut` closure cannot be copied, only lent.

Now `double_all` is written once and works for every implementor:

```
println!(
    "gat: double_all(Some(21))      = {:?}",
    double_all(Some(21))
);
println!(
    "gat: double_all(vec![1, 2, 3]) = {:?}",
    double_all(vec![1, 2, 3])
);
let ok: Result<i32, String> = Ok(5);
println!("gat: double_all(Ok(5))         = {:?}", double_all(ok));
let tree = node(node(Tree::Leaf, 1, Tree::Leaf), 2, Tree::Leaf);
println!(
    "gat: double_all(tree) has root = {:?}",
    root(&double_all(tree))
);
println!(
    "gat: double_all(Box::new(4))   = {:?}",
    double_all(Box::new(4))
);
```

```
gat: double_all(Some(21))      = Some(42)
gat: double_all(vec![1, 2, 3]) = [2, 4, 6]
```

```
gat: double_all(Ok(5))         = Ok(10)
gat: double_all(tree) has root = Some(4)
gat: double_all(Box::new(4))   = Some(8)
```

So far, so good. The last line deserves suspicion, though. It comes from this impl:

```
/// Nothing forces `Wrapped<B>` to be built from the same constructor as
/// `Self`: this impl compiles, and turns a `Box` into an `Option`.
impl<A> Functor for Box<A> {
    type Elem = A;
    type Wrapped<B> = Option<B>;
    fn fmap<B>(self, mut f: impl FnMut(A) -> B) -> Option<B> {
        Some(f(*self))
    }
}
```

The trait *meant* "`Wrapped<B>` is the same constructor applied to `B`", but nothing enforces it. Any type will do, as long as it is a `Functor` with the right `Elem`, so a `Box` can "map" to an `Option`. The trait describes a *relation between applied types*, not a type constructor.

A worse problem shows up when we try to state the composition law generically:

```
fn comp_law<F, B, C>(x: F, f: impl Fn(F::Elem) -> B, g: impl Fn(B) -> C) -> bool
where
    F: Functor + Clone,
    F::Wrapped<C>: PartialEq,
{
    let once = x.clone().fmap(|a| g(f(a)));
    let twice = x.fmap(&f).fmap(&g);
    once == twice
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch17_gat_composition.rs:19:13
   |
12 | fn comp_law<F, B, C>(x: F, f: impl Fn(F::Elem) -> B, g: impl Fn(B) -> C) -> bool
   |             - expected this type parameter
...
19 |     once == twice
   |             ^^^^^ expected type parameter `F`, found associated type
   |
   = note: expected associated type `<F as Functor>::Wrapped<_>`
              found associated type `<<F as Functor>::Wrapped<B> as Functor>::Wrapped<_>`
   = note: an associated type was expected, but a different one was found
help: consider further restricting this bound
   |
14 |     F: Functor<Wrapped<B> = F> + Clone,
   |                ++++++++++++++++
```

The two sides have different types. Mapping once gives `F::Wrapped<C>`. Mapping twice gives `<F::Wrapped<B> as Functor>::Wrapped<C>`: first wrap at `B`, then ask *that* type for its own `Wrapped<C>`. We know these are both `Option<C>` when `F` is `Option<A>`, but the trait never says that wrapping twice is the same as wrapping once, so for a generic `F` the compiler cannot prove it. (The message words this awkwardly, and its suggested fix, `F: Functor<Wrapped<B> = F>`, would claim that mapping into `B` gives back `F` itself, which is wrong whenever `B` is not `A`.) In this encoding, the composition law cannot even be *written* as a generic test.

Encoding 2: a family marker for the constructor

The second encoding gives the type constructor a name of its own. A **marker type** (an empty struct) stands for the constructor, and a GAT applies it:

```
/// The "family" pattern: a marker type stands for a type constructor, and the
/// generic associated type `Of<T>` applies it to `T`.
trait Family {
    type Of<T>;
}

/// A functor is a family with a `map`.
trait FunctorFamily: Family {
    fn fmap<A, B>(x: Self::Of<A>, f: impl FnMut(A) -> B) -> Self::Of<B>;
}

struct OptionFamily;
struct VecFamily;
/// `Result<_, E>`: the marker carries the fixed error type.
struct ResultFamily<E>(PhantomData<E>);
struct TreeFamily;

impl Family for OptionFamily {
    type Of<T> = Option<T>;
}
impl FunctorFamily for OptionFamily {
    fn fmap<A, B>(x: Option<A>, f: impl FnMut(A) -> B) -> Option<B> {
        x.map(f)
    }
}
```

`OptionFamily` is not an option; it stands for `Option` itself, the thing of kind `Type → Type`, and `OptionFamily::Of<T>` is `Option<T>`. A partial application such as `Result<_, E>` becomes a marker with a parameter, `ResultFamily<E>` (`PhantomData<E>` lets a struct mention a type it does not store). `FunctorFamily` is now indexed by the constructor, exactly as Haskell's class is indexed by `f`: Rust's generics range over marker types, and the GAT `Of` turns each marker into the type constructor it represents. The other families (`VecFamily`, `ResultFamily<E>`, `TreeFamily`) follow the same pattern in the example file.

This time, the laws can be checked by one generic function:

```
/// The functor laws, checked once for every family `F` on sample values.
/// Here `F` is a type constructor, so both sides of each law have the same
/// type (`F::Of<i32>` and `F::Of<usize>`) and can be compared.
fn law_report<F: FunctorFamily>(samples: &[F::Of<i32>]) -> (bool, bool)
where
    F::Of<i32>: Clone + PartialEq,
    F::Of<usize>: PartialEq,
{
    let f = |n: i32| format!("{n}"); // i32 -> String
    let g = |s: String| s.len(); // String -> usize
    let identity = samples.iter().all(|x| F::fmap(x.clone(), |a| a) == *x);
    let composition = samples
        .iter()
        .all(|x| F::fmap(x.clone(), |a| g(f(a))) == F::fmap(F::fmap(x.clone(), f), g));
    (identity, composition)
}

fn show(name: &str, (identity, composition): (bool, bool)) {
    let verdict = |ok: bool| if ok { "holds" } else { "FAILS" };
    println!(
        "family: {name:<13} identity {}, composition {}",
        verdict(identity),
        verdict(composition)
```

```
    );
}
```

Both sides of each law now have the same type, because there is one family `F` and every type is `F::Of<_>` of something. The function maps numbers to strings and strings to lengths, so the composition law is checked at three different types. Here is a lawless family, which maps and then reverses:

```
/// A lawless "functor": it maps, then reverses the vector.
struct ReversingVec;

impl Family for ReversingVec {
    type Of<T> = Vec<T>;
}
impl FunctorFamily for ReversingVec {
    fn fmap<A, B>(x: Vec<A>, f: impl FnMut(A) -> B) -> Vec<B> {
        let mut v: Vec<B> = x.into_iter().map(f).collect();
        v.reverse();
        v
    }
}
```

```
show(
    "OptionFamily",
    law_report::<OptionFamily>(&[None, Some(-5), Some(42)]),
);
show(
    "VecFamily",
    law_report::<VecFamily>(&[vec![], vec![1, 22, 333]]),
);
let results = [Ok(10), Err("timeout".to_string())];
show("ResultFamily", law_report::<ResultFamily<String>>(&results));
let tree = node(Tree::Leaf, 7, node(Tree::Leaf, 100, Tree::Leaf));
show("TreeFamily", law_report::<TreeFamily>(&[Tree::Leaf, tree]));
show(
    "ReversingVec",
    law_report::<ReversingVec>(&[vec![1, 22, 333]]),
);
```

```
family: OptionFamily  identity holds, composition holds
family: VecFamily     identity holds, composition holds
family: ResultFamily  identity holds, composition holds
family: TreeFamily    identity holds, composition holds
family: ReversingVec  identity FAILS, composition FAILS
```

`ReversingVec` compiles, type-checks and is caught only by the test, on a sample that happens to expose it (on `vec![7, 7, 7]` it would pass). This is Chapter 14's lesson again: Rust can name an algebraic structure, but not demand its laws.

The family encoding has its own limit, and it is about inference. Here is the generic `double_all` in this style, called on an `Option`:

```
fn double_all<F: FunctorFamily>(x: F::Of<i32>) -> F::Of<i32> {
    F::fmap(x, |n| n * 2)
}

fn caller() -> Option<i32> {
    double_all(Some(21)) // works with `double_all::<OptionFamily>`
}
```

```
error[E0283]: type annotations needed
  --> compile_fail/ch17_family_inference.rs:28:5
   |
28 |     double_all(Some(21)) // works with `double_all::<OptionFamily>`
   |     ^^^^^^^^^^ cannot infer type of the type parameter `F` declared on the function
`double_all`
   |
   = note: the type must implement `FunctorFamily`
help: the trait `FunctorFamily` is implemented for `OptionFamily`
  --> compile_fail/ch17_family_inference.rs:16:1
   |
16 | impl FunctorFamily for OptionFamily {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   = note: cannot satisfy `<_ as Family>::Of<i32> == Option<i32>`
note: required by a bound in `double_all`
  --> compile_fail/ch17_family_inference.rs:23:18
   |
23 | fn double_all<F: FunctorFamily>(x: F::Of<i32>) -> F::Of<i32> {
   |                  ^^^^^^^^^^^^^ required by this bound in `double_all`
help: consider specifying a concrete type for the type parameter `F`
   |
28 |     double_all::</* Type */>(Some(21)) // works with `double_all::<OptionFamily>`
   |               ++++++++++++++
```

Error E0283 means that the compiler cannot infer `F`. Read the note "cannot satisfy `<_ as Family>::Of<i32> == Option<i32>`": the compiler would have to find the family whose `Of<i32>` is `Option<i32>`, and nothing guarantees that there is only one (another crate could define a second marker with `Of<T> = Option<T>`). In the jargon, `Of` is not *injective*, so the argument type does not determine the family. Every call needs a turbofish, `double_all::<OptionFamily>(Some(21))`. Lean has the same question to answer when it infers `F` from `F α`, and solves it by unification: there `Option` is a real term, and `Option α` determines it.

Taking stock

	GAT on applied type	Family marker	Lean class
Quantify over constructors	indirectly	yes (markers)	yes (`F : Type → Type`)
"Same constructor" enforced	no	yes	yes
Composition law statable	no	yes	yes
Inference from arguments	yes	no (turbofish)	yes
Laws enforced	no, tested	no, tested	yes, proved

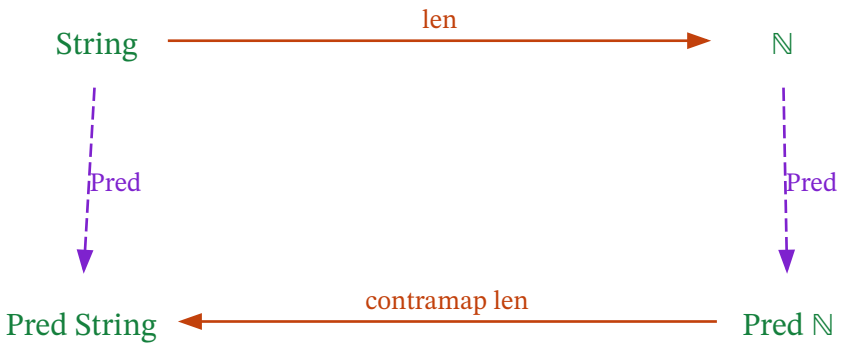
There is also a Rust-specific dimension that no encoding removes: **ownership**. Our `fmap` takes `self` by value and an `FnMut` closure. That is right for `Vec` (the closure runs many times, possibly updating state) but more than `Option` needs (one call at most, so `FnOnce` would do, as in `Option::map`). A version that maps through a reference (`&Vec<A>` to `Vec<B>`) is yet another trait. In a language with ownership, "the" functor interface splits into several, according to how often the function is called and whether the container is consumed: the `Fn` / `FnMut` / `FnOnce` distinctions of Chapter 8, and a first sign of the resource reading of types in Part V. Chapter 20 meets the same problem for monads.

17.5 Other variances

The functors so far are **covariant**: an arrow $a \to b$ becomes an arrow $F\,a \to F\,b$, in the same direction. Some useful constructions turn arrows around, and some take two arguments.

Contravariant functors

A predicate on numbers, `ℕ → Bool`, can be turned into a predicate on strings with a function `String → ℕ`: test the length. Notice the direction. The function goes from strings to numbers, and the predicate transformation goes from predicates on numbers to predicates on strings:



A contravariant functor: Pred sends the arrow len from String to ℕ to an arrow in the opposite direction, from Pred ℕ to Pred String.

A **contravariant functor** sends $f : a \to b$ to $F\,f : F\,b \to F\,a$, and its composition law reverses the order:

$$F\,\mathrm{id}_a = \mathrm{id}_{F\,a} \qquad\qquad F\,(g \circ f) = F\,f \circ F\,g$$

Equivalently, it is an ordinary functor from the opposite category $\mathcal{C}^{\mathrm{op}}$ of Chapter 16, where every arrow is already reversed. We will build it that way below. First, as a class:

```
/-- A contravariant functor: `contramap` turns `α → β` into `F β → F α`, and the
composition law reverses the order. -/
class Contravariant (F : Type → Type) where
  contramap : {α β : Type} → (α → β) → F β → F α
  contramap_id : ∀ {α : Type} (x : F α), contramap id x = x
  contramap_comp : ∀ {α β γ : Type} (f : α → β) (g : β → γ) (x : F γ),
    contramap (g ∘ f) x = contramap f (contramap g x)

/-- A predicate on `α`: a test that every value passes or fails. -/
def Pred (α : Type) : Type := α → Bool

/-- To test a `β` given a test on `γ`, convert first: precomposition. -/
instance : Contravariant Pred where
  contramap f p := fun a => p (f a)
  contramap_id _ := rfl
  contramap_comp _ _ _ := rfl

/-- A comparator, like the closures passed to Rust's `sort_by`. -/
def Comparator (α : Type) : Type := α → α → Ordering

/-- Compare values by comparing a key computed from them (`sort_by_key`). -/
instance : Contravariant Comparator where
  contramap f c := fun a b => c (f a) (f b)
```

```
  contramap_id _ := rfl
  contramap_comp _ _ _ := rfl
```

The operation is usually called `contramap`. For predicates it is precomposition: to test an `α` with a test on `β`, convert the `α` first. For comparators (functions `α → α → Ordering`, what Rust's `sort_by` takes), it compares two values by comparing their images: exactly what `sort_by_key` does. All the laws hold by `rfl`, because precomposition is function composition.

```
def isLong : Pred ℕ := fun n => n > 3

/-- "Is the string long?" is "is its length long?". -/
def isLongString : Pred String := Contravariant.contramap String.length isLong

/-- info: [false, true] -/
#guard_msgs in
#eval ["abc", "abcd"].map isLongString
```

In Rust:

```
/// A predicate: a test on values of type `T`.
struct Predicate<T>(Box<dyn Fn(&T) -> bool>);

impl<T: 'static> Predicate<T> {
    fn test(&self, value: &T) -> bool {
        (self.0)(value)
    }

    /// Precompose with `f`: a test on `T` becomes a test on `S`.
    /// The arrow `S -> T` is turned around into `Predicate<T> -> Predicate<S>`.
    fn contramap<S>(self, f: impl Fn(&S) -> T + 'static) -> Predicate<S> {
        Predicate(Box::new(move |s| self.test(&f(s))))
    }
}

/// A comparator on keys becomes a comparator on values: the same contramap,
/// and what `sort_by_key` does for you.
fn by_key<T, K: Ord>(key: impl Fn(&T) -> K) -> impl Fn(&T, &T) -> Ordering {
    move |a, b| key(a).cmp(&key(b))
}
```

```
let is_large = Predicate(Box::new(|n: &usize| *n > 3));
let is_long: Predicate<String> = is_large.contramap(|s: &String| s.len());
println!(
    "contravariant: is_long(\"abc\") = {}, is_long(\"abcd\") = {}",
    is_long.test(&"abc".to_string()),
    is_long.test(&"abcd".to_string())
);
let mut words = vec!["category", "map", "functor", "id"];
words.sort_by(by_key(|w: &&str| w.len()));
println!("contravariant: sorted by length = {words:?}");
```

```
contravariant: is_long("abc") = false, is_long("abcd") = true
contravariant: sorted by length = ["id", "map", "functor", "category"]
```

`Predicate<T>` stores a boxed closure (a `dyn Fn`, Chapter 15), and `contramap` builds a new one that converts before testing. `by_key` is the comparator version: from a key function it builds a comparator on values, by contramapping `Ord::cmp` on keys. Contravariance is common in Rust, even where it has no name: anything that *consumes* values of a type, such as a predicate, a comparator, a serializer or a callback, is contravariant in that type.

Bifunctors

`Result<T, E>` has two type parameters, and you can map both: `map` changes the `Ok` side, `map_err` the `Err` side. A **bifunctor** is a functor of two arguments, covariant in each, with a `bimap` that maps both sides at once:

```
/-- A bifunctor maps both of its arguments, covariantly. -/
class Bifunctor (F : Type → Type → Type) where
  bimap : {α β γ δ : Type} → (α → γ) → (β → δ) → F α β → F γ δ
  bimap_id : ∀ {α β : Type} (x : F α β), bimap id id x = x
  bimap_comp : ∀ {α β γ δ ε ζ : Type} (f : α → γ) (f' : γ → ε) (g : β → δ) (g' : δ → ζ)
    (x : F α β), bimap (f' ∘ f) (g' ∘ g) x = bimap f' g' (bimap f g x)

instance : Bifunctor Prod where
  bimap f g p := (f p.1, g p.2)
  bimap_id _ := rfl
  bimap_comp _ _ _ _ _ := rfl

/-- `Except ε α` maps the error with the first function and the value with the
second: Rust's `map_err` and `map` together. -/
instance : Bifunctor Except where
  bimap f g
    | .error e => .error (f e)
    | .ok a => .ok (g a)
  bimap_id x := by cases x <;> rfl
  bimap_comp _ _ _ _ x := by cases x <;> rfl
```

The laws say that mapping two identities is the identity and that `bimap` respects composition on both sides at once. For pairs they hold by `rfl`; for `Except` by cases. Mathematically, a bifunctor is a functor out of a *product category* $\mathcal{C} \times \mathcal{D}$, whose objects are pairs of objects and whose morphisms are pairs of morphisms.

In Rust, `map` and `map_err` act on different sides, so they commute:

```
/// A pair maps both components at once.
fn bimap_pair<A, B, C, D>(
    (a, b): (A, B),
    f: impl FnOnce(A) -> C,
    g: impl FnOnce(B) -> D,
) -> (C, D) {
    (f(a), g(b))
}
```

```
let to_message = |code: u16| format!("error {code}");
for r in [Ok::<i32, u16>(20), Err(404)] {
    // `map` and `map_err` act on different sides, so their order does not matter.
    let one = r.map(|n| n + 1).map_err(to_message);
    let other = r.map_err(to_message).map(|n| n + 1);
    assert_eq!(one, other);
    println!("bifunctor: {r:?} -> {one:?}");
}
let p = bimap_pair(("temperature", 21.5), str::len, |t: f64| t.round() as i64);
println!("bifunctor: bimap_pair = {p:?}");
```

```
bifunctor: Ok(20) -> Ok(21)
bifunctor: Err(404) -> Err("error 404")
bifunctor: bimap_pair = (11, 22)
```

Profunctors: functions

The function type `A -> B` has two parameters and is a bifunctor of mixed variance: contravariant in the input (you can precompose) and covariant in the output (you can postcompose). Such a thing is called a **profunctor**, and its operation `dimap` does both:

```
/-- Functions are contravariant in the input and covariant in the output:
`dimap f g h` is "first `f`, then `h`, then `g`". -/
def dimap {α β γ δ : Type} (f : γ → α) (g : β → δ) (h : α → β) : γ → δ :=
  g ∘ h ∘ f

theorem dimap_id {α β : Type} (h : α → β) : dimap id id h = h := rfl

theorem dimap_comp {α β γ δ ε ζ : Type} (f : γ → α) (f' : ε → γ)
    (g : β → δ) (g' : δ → ζ) (h : α → β) :
    dimap (f ∘ f') (g' ∘ g) h = dimap f' g' (dimap f g h) := rfl
```

Both laws are `rfl`. Note the order in `dimap_comp`: the input adapters compose in the *reverse* order (`f ∘ f'`) and the output adapters in the same order (`g' ∘ g`), which is contravariance and covariance side by side. Every closure adapter you write in Rust, wrapping a callback to convert its argument and its result, is a `dimap`.

17.6 Functors between arbitrary categories

So far, every functor went from the category of types to itself. The definition works between any two categories, and the book's `Category` class lets us state it in full generality. The file `code/lean/Book/Ch17/CatFunctor.lean` imports `Book.Ch16.Examples`, so that this chapter can reuse Chapter 16's preorder, opposite and free categories instead of redefining them, and it opens the namespace `Ch16` to get the notations `⟶` and `⊚`:

```
/-- A functor from `C` to `D`: a map on objects and a map on morphisms that
preserves identities and composition. -/
structure CatFunctor (C : Type u) (D : Type v) [Category C] [Category D] where
  /-- The action on objects. -/
  obj : C → D
  /-- The action on morphisms: an arrow `a ⟶ b` goes to `obj a ⟶ obj b`. -/
  map : {a b : C} → (a ⟶ b) → (obj a ⟶ obj b)
  /-- Identities go to identities. -/
  map_id : ∀ (a : C), map (Category.id a) = Category.id (obj a)
  /-- Composites go to composites. -/
  map_comp : ∀ {a b c : C} (g : b ⟶ c) (f : a ⟶ b), map (g ⊚ f) = map g ⊚ map f
```

`CatFunctor C D` is a structure, not a class, because there are usually many functors between two categories, and we want to name and pass them around. `obj` is the map on objects, and `map` is the map on morphisms; the type of `map` says that it sends `a ⟶ b` to `obj a ⟶ obj b`, so the "preserves sources and targets" part of the definition is enforced by Lean's types. `map_id` and `map_comp` are the functor laws.

Composing functors

Functors compose, and there is an identity functor:

```
/-- The identity functor. -/
def id (C : Type u) [Category C] : CatFunctor C C where
  obj a := a
```

```
  map f := f
  map_id _ := rfl
  map_comp _ _ := rfl

/-- The composite functor: first `F`, then `G`. -/
def comp {C : Type u} {D : Type v} {E : Type w} [Category C] [Category D] [Category E]
    (G : CatFunctor D E) (F : CatFunctor C D) : CatFunctor C E where
  obj a := G.obj (F.obj a)
  map f := G.map (F.map f)
  map_id a := by rw [F.map_id, G.map_id]
  map_comp g f := by rw [F.map_comp, G.map_comp]

/-- Composition of functors is associative and has the identity as a unit;
all three equations hold by `rfl`. -/
theorem comp_assoc {C D E B : Type} [Category C] [Category D] [Category E] [Category B]
    (H : CatFunctor E B) (G : CatFunctor D E) (F : CatFunctor C D) :
    comp H (comp G F) = comp (comp H G) F := rfl

theorem id_comp {C D : Type} [Category C] [Category D] (F : CatFunctor C D) :
    comp (id D) F = F := rfl

theorem comp_id {C D : Type} [Category C] [Category D] (F : CatFunctor C D) :
    comp F (id C) = F := rfl
```

The laws of the composite follow from the laws of its parts: `map_id` for `comp G F` rewrites first with `F.map_id` (so `F` sends the identity to an identity) and then with `G.map_id`; `map_comp` is the same with the composition law. The three theorems at the end say that functor composition is associative and has the identity functor as a unit, and they hold by `rfl`: two functors with the same `obj` and `map` are equal, and the proof fields do not matter because of proof irrelevance. This should look familiar: categories and functors form a category themselves, usually called $\mathbf{Cat}$. (One of the exercises asks you to build it as an instance of the book's class.)

Functors on types are functors

Every lawful `Functor` instance from the first half of the chapter is a `CatFunctor` from the category of types to itself:

```
/-- Every lawful functor on types (the class of `Functor.lean`) is a functor
from the category of types to itself. -/
def ofFunctor (F : Type → Type) [Functor F] : CatFunctor Type Type where
  obj α := F α
  map f := Functor.map f
  map_id _ := funext fun x => Functor.map_id x
  map_comp g f := funext fun x => Functor.map_comp f g x

/-- info: some 10 -/
#guard_msgs in
#eval (ofFunctor Option).map (double ∘ len) (some "hello")
```

The laws convert between the pointwise form of the class and equations between morphisms (functions), using `funext`. The example maps the composite `double ∘ len` of Chapter 16 inside an `Option`.

Monotone maps are functors between preorders

In a preorder, a morphism $a \to b$ is a proof of $a \le b$. A functor between two preorders therefore sends elements to elements, and proofs of $a \le b$ to proofs of $F\,a \le F\,b$: it is a **monotone** map. The functor laws are automatic, because parallel morphisms in a preorder are equal.

```
/-- A monotone map between preorders is a functor between their categories.
The laws hold by `rfl`: parallel morphisms of a preorder are equal. -/
def monotoneFunctor {α β : Type} [Preorder α] [Preorder β]
    (f : α → β) (hf : Monotone f) : CatFunctor α β where
  obj := f
  map h := ⟨hf h.down⟩    -- a proof of `a ≤ b` becomes a proof of `f a ≤ f b`
  map_id _ := rfl
  map_comp _ _ := rfl

/-- Conversely, the object map of any functor between preorders is monotone. -/
theorem CatFunctor.monotone {α β : Type} [Preorder α] [Preorder β]
    (F : CatFunctor α β) : Monotone F.obj :=
  fun _ _ h => (F.map ⟨h⟩).down

/-- Doubling is monotone on `ℕ`, hence a functor `ℕ ⇒ ℕ`. -/
def doubling : CatFunctor ℕ ℕ :=
  monotoneFunctor (fun n => 2 * n) (fun _ _ h => Nat.mul_le_mul_left 2 h)

/-- It sends the morphism `threeToFive : 3 → 5` to a morphism `6 → 10`. -/
example : doubling.map threeToFive = (⟨Nat.mul_le_mul_left 2 threeToFive.down⟩ :
    doubling.obj 3 → doubling.obj 5) := rfl

example : (doubling.obj 3, doubling.obj 5) = (6, 10) := rfl
```

`Monotone f` is Mathlib's predicate "`a ≤ b` implies `f a ≤ f b`". The two directions are both here: `monotoneFunctor` turns a monotone function into a functor, and `CatFunctor.monotone` shows that the object map of *any* functor between preorders is monotone, by applying `map` to the morphism `⟨h⟩` and taking the resulting proof out of its `PLift`. So "functor between preorders" and "monotone function" are the same notion. `doubling` is the example, with `Nat.mul_le_mul_left` as the monotonicity proof.

Running a free category: an interpreter is a functor

The free category on a graph (Chapter 16) has a remarkable property: to define a functor out of it, you only need to say where each vertex and each *edge* goes. Paths are then sent to the composites of the images of their edges, and the functor laws hold automatically. This is the universal property that justifies the name "free".

```
/-- Interpret a path in a category `D`, given an object for every vertex and a
morphism for every edge: compose the morphisms of the edges along the path. -/
def interp (o : V → D) (m : {a b : V} → E a b → (o a ⟶ o b)) :
    {a b : V} → Path E a b → (o a ⟶ o b)
  | _, _, .nil a => Category.id (o a)
  | _, _, .cons e p => interp o m p ≫ m e

/-- Interpreting a concatenation composes the interpretations. -/
theorem interp_append (o : V → D) (m : {a b : V} → E a b → (o a ⟶ o b))
    {a b c : V} (p : Path E a b) (q : Path E b c) :
    interp o m (p.append q) = interp o m q ≫ interp o m p := by
  induction p with
  | nil a =>
    -- Goal: interp q = interp q ≫ id
    exact (Category.comp_id _).symm
  | cons e p ih =>
    -- Goal: interp (append p q) ≫ m e = interp q ≫ (interp p ≫ m e)
    simp only [Path.append, interp]
    rw [ih, Category.assoc]

/-- So the interpretation is a functor out of the free category. -/
def freeFunctor (o : V → D) (m : {a b : V} → E a b → (o a ⟶ o b)) :
    CatFunctor (Free E) D where
  obj v := o v.vertex
  map p := interp o m p
```

```
  map_id _ := rfl
  map_comp g f := interp_append o m f g
```

`interp o m` interprets a path in any category `D`, given an object `o v` for each vertex and a morphism `m e` for each edge: the empty path goes to an identity, and an edge followed by a path goes to the composite. The functor law for composition is `interp_append`, proved by induction on the first path:

- In the `nil` case, the goal is `interp q = interp q ⊚ id`, the right unit law read backwards.
- In the `cons e p` case, after unfolding one step of `append` and `interp`, the goal is `interp (append p q) ⊚ m e = interp q ⊚ (interp p ⊚ m e)`. Rewriting with the induction hypothesis turns the left side into `(interp q ⊚ interp p) ⊚ m e`, and `Category.assoc` rebrackets the right side to the same thing.

The identity law, `map_id`, is `rfl`: the empty path is interpreted as an identity by definition.

Now apply it to the connection protocol of Chapter 16, sending each state to the type of data a client holds in that state, and each operation to a function on that data:

```
/-- The data a client holds in each state of the protocol. -/
def sessionData : Conn → Type
  | .closed => Unit
  | .connected => String                -- the server
  | .authenticated => String × String  -- the server and the user

/-- What each operation does to the data. -/
def runStep : {a b : Conn} → Step a b → (sessionData a → sessionData b)
  | _, _, .connect => fun () => "db.example.org"
  | _, _, .login => fun server => (server, "alice")
  | _, _, .logout => fun (server, _) => server
  | _, _, .close => fun _ => ()

/-- Running traces is a functor from the free category to the category of types. -/
def runTrace : CatFunctor (Free Step) Type :=
  freeFunctor sessionData runStep

/-- The data after signing in, starting from the closed state. -/
def afterSignIn : String × String := runTrace.map signIn ()

/-- info: ("db.example.org", "alice") -/
#guard_msgs in
#eval afterSignIn

/-- Functoriality: running a composite trace is running its parts in turn. -/
example :
    runTrace.map (signOut ⊚ signIn) = runTrace.map signOut ⊚ runTrace.map signIn :=
  runTrace.map_comp signOut signIn
```

`runTrace` is a functor from the free category of the protocol to the category of types: it turns every valid trace into a function, an *interpreter* for traces. The functor law for composition says that running a composite trace is running its parts one after the other, and we did not have to prove it for this protocol: it came with `freeFunctor`. This is a pattern worth recognizing in your own code. Whenever you define the meaning of a sequence of commands by the meanings of the individual commands, you are defining a functor out of a free category, and the composition law is the guarantee that "run `a` then `b`" equals "run `a`, then run `b`".

Contravariant functors, again

A contravariant functor is an ordinary functor out of the opposite category, whose arrows are already reversed. Predicates are the example:

```
/-- Predicates form a functor out of the *opposite* of the category of types:
a morphism `a → b` in `Op Type` is a function `b.unop → a.unop`, and it turns
a test on `a.unop` into a test on `b.unop` by precomposition. -/
def predFunctor : CatFunctor (Op Type) Type where
  obj a := a.unop → Bool
  map f := fun p => p ∘ f
  map_id _ := rfl
  map_comp _ _ := rfl
```

A morphism `a → b` in `Op Type` is a function `b.unop → a.unop`, so `p ∘ f` turns a predicate on `a.unop` into a predicate on `b.unop`, in the direction a functor `Op Type → Type` requires. Both laws are `rfl`. The `Contravariant` class of the first half was a convenient packaging of the same thing.

Hom functors

Here is a functor that every category has. Fix an object `a`; then "the arrows out of `a`" form a functor into the category of types:

```
/-- For every object `a`, the morphisms out of `a` form a functor to `Type`:
`b` goes to the type `a → b`, and `g : b → c` acts by post-composition. This
needs `a → b` to be a type, which is why the book's `Hom` lives in `Type`. -/
def homFunctor {C : Type u} [Category C] (a : C) : CatFunctor C Type where
  obj b := a → b
  map g := fun f => g ∘ f
  map_id _ := funext fun f => Category.id_comp f
  map_comp h g := funext fun f => (Category.assoc h g f).symm

/-- In the category of types, the hom functor out of `ε` is `Reader ε`. -/
example {ε α β : Type} (g : α → β) (r : Reader ε α) :
    (homFunctor ε).map g r = Functor.map g r := rfl
```

It sends each object `b` to the *type* `a → b`, and an arrow `g : b → c` to the function "compose with `g`", from `a → b` to `a → c`. The functor laws are the left unit law and associativity of the category. This is where the decision of Chapter 16 to make `Hom` land in `Type` pays off: `a → b` must be an object of the category of types for `homFunctor` to make sense.

In the category of types, `homFunctor ε` is our `Reader ε`: the arrows out of `ε` are the functions `ε → β`, and composing with `g` is mapping. Hom functors are the subject of the Yoneda lemma, which Chapter 18 proves.

17.7 Variance is functoriality

Rust has exactly one kind of subtyping, between lifetimes: if `'a` outlives `'b` (written `'a: 'b`), a `&'a T` can be used where a `&'b T` is expected. Subtyping is reflexive and transitive, so it is a preorder, and a preorder is a category (Chapter 16). Now ask how a type constructor behaves with respect to it:

- `&'a T`, `Box<T>`, `Vec<T>` and `Option<T>` are **covariant**: if `S` is a subtype of `T`, then `Vec<S>` is a subtype of `Vec<T>`. That is a monotone map on the subtyping preorder, which is to say a *functor*.

- `fn(T)` is **contravariant** in `T`: a function that accepts any `&'b str` can be used where a function accepting `&'static str` is expected, but not the other way around. That is a functor from the opposite preorder.
- `&mut T` and `Cell<T>` are **invariant** in `T`: neither direction is allowed, and they are not functors on the subtyping preorder in either direction. Chapter 27 shows the unsound program that covariance of `&mut T` would allow.

So "variance" is the word type theorists use for functoriality on the subtyping preorder, and the variances of Rust's type constructors are the answers to "is this constructor a covariant functor, a contravariant one, or neither?". The function type `fn(A) -> B` is contravariant in `A` and covariant in `B`, the profunctor of the previous section. Chapter 27 develops this in full.

17.8 Putting it together: map fusion, proved once

A pipeline that maps several functions over a container in turn can be optimized into a single `map` of the composite function: one traversal instead of many. Rust's lazy iterators perform a version of this automatically (`v.iter().map(f).map(g)` makes one pass), and compilers for functional languages apply it as a rewrite rule. Is it always correct? For every lawful functor, yes, and we can prove it once, for all functors and all pipelines.

A pipeline is a list of functions, run either one `map` at a time or fused:

```
/-- A pipeline of transformations of numbers, applied in order. -/
def Pipeline := List (ℕ → ℕ)

/-- Run the pipeline "naively": one `map` per stage. -/
def runNaive {F : Type → Type} [Functor F] : Pipeline → F ℕ → F ℕ
  | [], x => x
  | f :: fs, x => runNaive fs (Functor.map f x)

/-- Compose all the stages into a single function. -/
def fuse : Pipeline → (ℕ → ℕ)
  | [] => id
  | f :: fs => fuse fs ∘ f

/-- Fusion is correct for every lawful functor: one `map` of the fused function
gives the same result as one `map` per stage. -/
theorem runNaive_eq_fuse {F : Type → Type} [Functor F] (p : Pipeline) (x : F ℕ) :
    runNaive p x = Functor.map (fuse p) x := by
  induction p generalizing x with
  | nil =>
    -- Goal: x = map id x. This is the identity law, read right to left.
    exact (Functor.map_id x).symm
  | cons f fs ih =>
    -- Goal: runNaive fs (map f x) = map (fuse fs ∘ f) x
    simp only [runNaive, fuse]
    rw [ih, Functor.map_comp]  -- both sides become map (fuse fs) (map f x)
```

`runNaive` maps each stage in order; `fuse` composes all the stages into one function, the first stage innermost. The theorem says that the two agree on every input, in every lawful functor. The proof is by induction on the pipeline, with `generalizing x` because the recursive call works on a different container (`map f x` instead of `x`), so the induction hypothesis must hold for every container:

- For the empty pipeline, `runNaive [] x` is `x`, and `fuse []` is `id`, so the goal is `x = map id x`: the identity law, read from right to left.

- For a pipeline `f :: fs`, unfolding gives `runNaive fs (map f x) = map (fuse fs ∘ f) x`. The induction hypothesis (instantiated at `map f x`) rewrites the left side to `map (fuse fs) (map f x)`, and the composition law rewrites the right side to the same thing.

Both laws were needed, one per case. Run on two different functors:

```
def stages : Pipeline := [(· + 1), (· * 10), (· - 3)]

/-- info: some 17 -/
#guard_msgs in
#eval Functor.map (fuse stages) (some 1)

/-- info: [17, 27, 37] -/
#guard_msgs in
#eval runNaive stages [1, 2, 3]
```

The stages add 1, multiply by 10 and subtract 3, so `1` becomes `17`. The theorem guarantees that the fused and the naive runs agree for `Option`, `List`, `Tree`, `Except ε`, and every other lawful functor, including ones that do not exist yet. The Rust example's `law_report` is the testing counterpart: one generic function, every family, finitely many samples.

17.9 In Mathlib

Mathlib's functors between categories are `CategoryTheory.Functor C D`, written `C ⥤ D` (typed `\functor`). The fields are `obj`, `map`, `map_id` and `map_comp`, like our `CatFunctor`, but in Mathlib's conventions: composition in diagrammatic order (`map_comp : F.map (f ≫ g) = F.map f ≫ F.map g`), and laws with default proofs, so a functor whose laws are easy can omit them:

```
/-- `Option` as a Mathlib functor `Type ⥤ Type`. The fields `map_id` and
`map_comp` are omitted: Mathlib proves them automatically (`cat_disch`). -/
def optionFunctor : Type ⥤ Type where
  obj α := Option α
  map f := ↾(Option.map f)

/-- `F.obj` acts on objects, `F.map` on morphisms. -/
example : optionFunctor.obj ℕ = Option ℕ := rfl

example (f : ℕ → ℕ) : optionFunctor.map (↾f) (some 3) = some (f 3) := rfl

/-- The laws are available as `Functor.map_id` and `Functor.map_comp`, with
composition in diagrammatic order. -/
example {X Y Z : Type} (f : X → Y) (g : Y → Z) :
    optionFunctor.map (f ≫ g) = optionFunctor.map f ≫ optionFunctor.map g :=
  optionFunctor.map_comp f g
```

The examples are in `code/lean/Book/Ch17/InMathlib.lean`. Recall from Chapter 16 that a morphism of `Type` in Mathlib wraps a function. That is why the code writes `map f := ↾(Option.map f)`: the morphism `f` is coerced to a function for `Option.map`, and `↾` wraps the resulting function back into a morphism.

Core Lean's lawful functors convert to Mathlib functors with `ofTypeFunctor`, which needs exactly the `Functor` and `LawfulFunctor` instances of core Lean. Functors compose with `⋙` (typed `\ggg`; again "first the left one") and `𝟭 C` is the identity functor:

```
/-- Any lawful functor of core Lean gives a Mathlib functor. -/
example : Type ⥤ Type := ofTypeFunctor Option

example : (ofTypeFunctor Option).obj ℕ = Option ℕ := rfl

/-- Functors compose with `⋙` (first the left one), and `𝟭` is the identity. -/
example : (optionFunctor ⋙ ofTypeFunctor Option).obj ℕ = Option (Option ℕ) := rfl

example : (𝟭 Type ⋙ optionFunctor) = optionFunctor := rfl
```

The last example says that `𝟭 Type ⋙ optionFunctor` *is* `optionFunctor`, by `rfl`, just like our `CatFunctor.id_comp`. Monotone maps are functors in Mathlib too:

```
/-- A monotone function between preorders is a functor between them. -/
def doubling : ℕ ⥤ ℕ :=
  Monotone.functor ⟨fun n => 2 * n, fun _ _ h => Nat.mul_le_mul_left 2 h⟩

example : doubling.obj 5 = 10 := rfl

/-- Its action on morphisms is the monotonicity proof, wrapped by `homOfLE`. -/
example : (6 : ℕ) ≤ 10 := leOfHom (doubling.map (homOfLE (by decide : (3 : ℕ) ≤ 5)))
```

There are now three things called "functor" in a Lean project that imports Mathlib, and it is worth keeping them apart: core Lean's `Functor` (an operation on type constructors, without laws), `LawfulFunctor` (the laws, as a separate class), and Mathlib's `CategoryTheory.Functor` (a structure for functors between arbitrary categories, laws included). This chapter's `Ch17.Functor` and `Ch17.CatFunctor` are our from-scratch versions of the first two and the third.

17.10 Common confusions

"**A functor is a container.**" Many functors are containers, but `Reader ε` contains nothing: a function `ε → α` has no `α` inside until you give it an `ε`. A functor is a type constructor with a lawful `map`. Contravariant functors, such as predicates, do not even have a `map` in the usual direction.

"**Any type with a `map` method is a functor.**" Only if the laws hold. A `map` that drops elements, reorders them, counts its calls or prints is not a functor `map`, and the refactorings that rely on the laws become unsafe.

"**`Option` is a type.**" `Option<i32>` is a type. `Option` is a type constructor, of kind `Type → Type`. Rust lets you name it only when you apply it.

"**GATs give Rust higher-kinded types.**" They give encodings, each with gaps: the applied-type encoding cannot state the composition law, and the family encoding loses inference. Neither enforces the laws.

"**A contravariant functor is the opposite of a functor.**" It is a functor, from the opposite category. Every theorem about functors applies to it.

"**Mathlib's `Functor` is Lean's `Functor`.**" Core's `Functor` is a class on `Type u → Type v`; Mathlib's `CategoryTheory.Functor` is a structure between any two categories. `ofTypeFunctor` converts the first (with its laws) into the second.

17.11 Summary

- Kinds classify types: `Type` for types of values, `Type → Type` for type constructors such as `Option` and `Vec`, `Type → Type → Type` for `Result`. Partial application fixes some arguments (`Result<_, E>`, Lean's `Except ε`). Rust's type parameters always have kind `Type` (E0109); in Lean, `Type → Type` is an ordinary function type.
- A functor maps objects to objects and morphisms to morphisms, preserving identities and composition. Endofunctors on types are type constructors with a lawful `map`: `Option`, `Vec`, `Result<_, E>`, `(E, _)`, `fn(E) -> _`, trees.
- The laws exclude maps that lose, reorder or add data, and they justify refactorings such as map fusion. Lean proves them (by `cases`, induction or `rfl`); Rust can only test them.
- Rust has no `Functor` trait. A GAT on the applied type cannot state the composition law and does not enforce "same constructor"; a family marker can state the laws but needs turbofish annotations (E0283). Ownership splits the interface further (`FnOnce`, `FnMut`, by value or by reference).
- Contravariant functors reverse arrows (predicates, comparators: precomposition) and are functors from the opposite category. Bifunctors map two arguments (pairs, `Result` with `map` and `map_err`); functions are profunctors.
- Functors between arbitrary categories compose and have identities. Monotone maps are the functors between preorders; a functor out of a free category is an interpreter determined by the edges; hom functors exist in every category.
- Variance is functoriality on the subtyping preorder: covariant, contravariant or invariant (Chapter 27).
- Mathlib writes `C ⥤ D`, `F.obj`, `F.map`, `⋙` and `𝟭`, and `ofTypeFunctor` turns core Lean's lawful functors into categorical ones.

17.12 Exercises

1. Give the kind of each of these Rust type constructors: `Box`, `Cell`, `PhantomData`, `&'a mut T`, `HashMap` with its hasher parameter, and `Result<_, String>`. Which ones take a parameter that is not a type? *Hint:* count the parameters and their sorts; lifetimes and const values count as parameters of other kinds (`Lifetime`, `usize`).
2. In Lean, make `Twice` (pairs `α × α`, with `map` applied to both components) an instance of `Ch17.Functor`, and prove the laws. Then show that the function "swap the two components" commutes with your `map`. *Hint:* define `map f p := (f p.1, f p.2)`; both laws are `rfl`, as is the commutation `swap (map f p) = map f (swap p)`. Chapter 18 calls such a function a natural transformation.
3. In Rust, implement `FunctorFamily` for `Box` (`BoxFamily`, with `Of<T> = Box<T>`) and for pairs of equal types (`PairFamily`, with `Of<T> = (T, T)`), and run `law_report` on both. Then write a lawless `FunctorFamily` for `Option` that passes the composition test but fails the identity test. *Hint:* for the lawless one, imitate `forgetMap` from the Lean file and return `None` always.

4. In Lean, prove that functors preserve isomorphisms: given `F : CatFunctor C D` and `i : a ≅ b` (Chapter 16's `Iso`), build an isomorphism `F.obj a ≅ F.obj b`. *Hint:* take `F.map i.hom` and `F.map i.inv`; for `inv_hom`, rewrite with `← F.map_comp`, then `i.inv_hom`, then `F.map_id`.
5. In Lean, build the category of small categories: objects are pairs of a type and a category structure on it, `(C : Type) × Category C`, and morphisms are `CatFunctor`s. *Hint:* `Hom a b := @CatFunctor a.1 b.1 a.2 b.2`, and similarly with `@` for `id` and `comp` (write `comp {a b c} g f := ...` to name the implicit objects). All three laws are `rfl`, by the theorems of the section "Composing functors".
6. Explain, with a short Rust example, why `fn(T) -> T` is neither covariant nor contravariant in `T` with respect to lifetimes, and why `Cell<T>` must be invariant. *Hint:* `T` appears both as an input (a contravariant position) and as an output (a covariant position). For `Cell<T>`, think of `set` as an input and `get` as an output; Chapter 27 has the details.

CHAPTER 18

Natural Transformations and Free Theorems

Here is a refactoring you have probably done without a second thought. A function builds a vector of formatted strings from a vector of numbers and then takes the first one:

```
let label = numbers.iter().map(format_number).collect::<Vec<_>>().first().cloned();
```

and you rewrite it to take the first number and format only that one:

```
let label = numbers.first().map(format_number);
```

You did not write a test to check that the two versions agree. You did not need to: "take the first element" does not care what the elements are, so it cannot matter whether you format them before or after. The same intuition tells you that reversing a list commutes with mapping over it, that mapping does not change a length, and that a function with the signature `fn<T>(x: T) -> T` can only hand back its argument.

This chapter turns that intuition into mathematics, from two directions. From category theory comes the **natural transformation**: a family of functions, one for each type, that commutes with `map`. From type theory comes **parametricity**: a generic function that cannot inspect its type parameter satisfies equations that can be read off its type, the **free theorems**. The two meet in the slogan *polymorphic functions are natural transformations*. The chapter ends with the most famous result of elementary category theory, the **Yoneda lemma**, which turns out to be continuation-passing style seen through a categorical lens.

In this chapter you will learn:

- what a natural transformation is, how to read a naturality square, and why `first`, `Option::into_iter`, `reverse` and `len` are natural;
- how to check naturality squares in Rust on samples, and how to prove them in Lean for concrete transformations;
- how natural transformations compose, and why functors and natural transformations form a category of their own;
- Reynolds' parametricity and Wadler's free theorems: how a type alone yields an equation, and why the general statement is a meta-theorem that Lean cannot prove (and, with classical logic, can even refute);

- where Rust leaks parametricity: trait bounds, `size_of`, `Any`, panics and side effects, specialization;
- the Yoneda lemma, as continuation-passing style in Rust and as a proved isomorphism in Lean, and Mathlib's `NatTrans` and `yoneda`.

18.1 From functors to maps between functors

Chapter 17 introduced functors. On the category of types, a functor is a type constructor `F` together with a function

$$\mathrm{map} : (\alpha \to \beta) \to F\,\alpha \to F\,\beta$$

that satisfies two laws: mapping the identity does nothing, and mapping a composite is the same as mapping twice.

$$\mathrm{map}\ \mathrm{id} = \mathrm{id} \qquad\qquad \mathrm{map}\,(g \circ f) = \mathrm{map}\,g \circ \mathrm{map}\,f$$

`Option`, `Vec`, `Result<_, E>` and the pair `(E, _)` are functors in this sense; in Rust each comes with its own `map` method, and Chapter 17 explained why Rust has no `Functor` trait to collect them (no higher-kinded types).

Once you have functors, a natural next question is: what is a function *from one functor to another*? A functor is not a type, so the question needs care. Look at this Rust function:

```rust
/// Vec ⇒ Option: the first element, if any.
fn first<T>(v: Vec<T>) -> Option<T> {
    v.into_iter().next()
}

/// Option ⇒ Vec: zero or one element (this is `Option::into_iter`, collected).
fn option_to_vec<T>(o: Option<T>) -> Vec<T> {
    o.into_iter().collect()
}

/// Vec ⇒ Vec: the same elements in the opposite order.
fn reverse<T>(mut v: Vec<T>) -> Vec<T> {
    v.reverse();
    v
}

/// Vec ⇒ Const<usize>: the length. Mapping never changes it.
fn len<T>(v: Vec<T>) -> usize {
    v.len()
}
```

The full program is in `code/rust/examples/ch18_naturality.rs`. `first` is not one function but a family of them: `first::<i32>` takes a `Vec<i32>` to an `Option<i32>`, `first::<String>` takes a `Vec<String>` to an `Option<String>`, and so on for every type. Monomorphization (Chapter 12) makes this literal: the compiler emits one machine-code function per type at which `first` is used. So `first` is a family of functions $F\,\alpha \to G\,\alpha$, one for each type α, where the functor F is `Vec` and the functor G is `Option`. The same is true of `option_to_vec` (from `Option` to `Vec`), of `reverse` (from `Vec` to itself) and, with a little imagination, of `len`: its result type `usize` does not mention `T`, but we can see it as the **constant functor** $\mathrm{Const}_{\texttt{usize}}$, which sends every type to `usize` and every function to the identity.

A family of functions of this shape is the raw material. A natural transformation is such a family that, in addition, behaves *uniformly*.

18.2 Naturality

The definition

Let $\mathcal{C}$ and $\mathcal{D}$ be categories and F, G two functors from $\mathcal{C}$ to $\mathcal{D}$. A **natural transformation** $\eta : F \Rightarrow G$ consists of

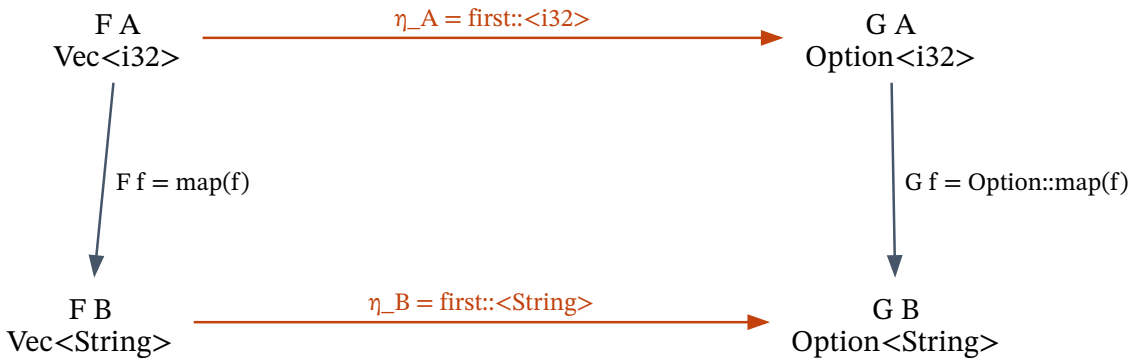
- for every object a of $\mathcal{C}$, a morphism $\eta_a : F\,a \to G\,a$ of $\mathcal{D}$, called the **component** of η at a;

such that

- for every morphism $f : a \to b$ of $\mathcal{C}$, the **naturality equation** holds:

$$G\,f \circ \eta_a = \eta_b \circ F\,f$$

Here $F\,f : F\,a \to F\,b$ and $G\,f : G\,a \to G\,b$ are what the functors do to the morphism f; for functors on types, they are `map f` for `F` and for `G`. Both sides of the equation are morphisms from $F\,a$ to $G\,b$, and the equation says that the two ways of getting there agree. It is usually drawn as a square, the **naturality square**, and "naturality" means "this square commutes" (Chapter 16 introduced commutative diagrams):



The naturality square for first at $f : i32 \to String$. Going right then down (take the first element, then map f over the Option) gives the same result as going down then right (map f over the vector, then take the first element).

Read the square as two programs. The path along the top and down the right side transforms first and maps afterwards: `first(v).map(f)`. The path down the left side and along the bottom maps first and transforms afterwards: `first(v.into_iter().map(f).collect())`. Naturality says that for every `f` and every `v`, the two programs return the same value. In words: *a natural transformation changes the shape of the container without looking at, or depending on, the elements*. Mapping changes the elements without touching the shape; the two operations do not interfere.

Examples on types

For functors on types the naturality equation is an equation between functions, which we can state pointwise. Here are the transformations of the Rust listing, with their equations (x is any input, f any function):

Transformation	From ⇒ To	Naturality equation
`first`	`Vec` ⇒ `Option`	$\text{first}(\text{map}\, f\, v) = \text{map}\, f\, (\text{first}\, v)$
`option_to_vec`	`Option` ⇒ `Vec`	$\text{toVec}(\text{map}\, f\, o) = \text{map}\, f\, (\text{toVec}\, o)$
`reverse`	`Vec` ⇒ `Vec`	$\text{rev}(\text{map}\, f\, v) = \text{map}\, f\, (\text{rev}\, v)$
`len`	`Vec` ⇒ $\text{Const}_{\texttt{usize}}$	$\text{len}(\text{map}\, f\, v) = \text{len}\, v$
`Some`	Id ⇒ `Option`	$\text{Some}(f\, x) = \text{map}\, f\, (\text{Some}\, x)$

The last two rows deserve a comment. For `len`, the target functor is constant, so its `map f` is the identity and the equation says "mapping does not change the length". For `Some`, the source is the **identity functor** Id, which sends every type to itself and every function to itself; the equation says that wrapping and then mapping is the same as applying and then wrapping.

Not every family of functions is natural. Imagine a trait `Pick` with a method `fn pick(v: Vec<Self>) -> Option<Self>`, implemented for `i32` by returning the *last* element and for `String` by returning the *first*. That is a perfectly good family of functions `Vec<α> → Option<α>` (for the two types that implement the trait), but with `f = |n: i32| n.to_string()` and `v = vec![1, 2]` the two paths around the square give `Some("2")` and `Some("1")`. Ad-hoc polymorphism (Chapter 14) lets each type have its own unrelated code, and naturality is exactly the property that such independent choices destroy.

18.3 Checking naturality in Rust

Since Rust has no higher-kinded types, we cannot write a function that takes "a functor `F`" as a parameter. We can still check a naturality square, by passing the pieces of the square as ordinary closures: the two components and the two lifted maps.

```
/// Checks one naturality square on one input `x : F<A>`.
///
/// Without higher-kinded types we cannot write "for every functor F", so the
/// caller passes the two components (`eta_a`, `eta_b`) and the two lifted
/// maps (`map_f` is F(f), `map_g` is G(f)) as ordinary closures.
fn square<FA, FB, GA, GB>(
    x: FA,
    eta_a: impl Fn(FA) -> GA,
    eta_b: impl Fn(FB) -> GB,
    map_f: impl Fn(FA) -> FB,
    map_g: impl Fn(GA) -> GB,
) -> (GB, GB)
where
    FA: Clone,
{
    let right_then_down = eta_b(map_f(x.clone())); // map first, then transform
    let down_then_right = map_g(eta_a(x)); // transform first, then map
    (right_then_down, down_then_right)
}
```

```
/// `map` for vectors, written out so that the squares below read clearly.
fn vmap<A, B>(f: impl Fn(A) -> B) -> impl Fn(Vec<A>) -> Vec<B> {
    move |v| v.into_iter().map(&f).collect()
}

fn report<T: PartialEq + Debug>(name: &str, input: &str, (lhs, rhs): (T, T)) {
    let verdict = if lhs == rhs { "commutes" } else { "FAILS" };
    println!("{name:<14} {input:<11} {lhs:?} vs {rhs:?}: {verdict}");
}
```

`square` takes an input `x : F<A>` and returns the two corners it reaches in $G\,B$, one along each path. `eta_a` and `eta_b` are the components at A and B (for `first`, the instantiations `first::<i32>` and `first::<String>`, which Rust infers from the closure types), `map_f` is $F\,f$ and `map_g` is $G\,f$. The helper `vmap` builds map f for vectors. Then `main` checks a few squares:

```
let tens = |n: i32| n * 10;
let show = |n: i32| format!("<{n}>");
report(
    "first",
    "[3, 1, 2]",
    square(vec![3, 1, 2], first, first, vmap(show), |o| o.map(show)),
);
report(
    "first",
    "[]",
    square(vec![], first, first, vmap(show), |o| o.map(show)),
);
report(
    "option_to_vec",
    "Some(7)",
    square(
        Some(7),
        option_to_vec,
        option_to_vec,
        |o| o.map(tens),
        vmap(tens),
    ),
);
report(
    "reverse",
    "[3, 1, 2]",
    square(vec![3, 1, 2], reverse, reverse, vmap(show), vmap(show)),
);
report(
    "len",
    "[3, 1, 2]",
    square(vec![3, 1, 2], len, len, vmap(show), |n| n),
);
```

```
first          [3, 1, 2]   Some("<3>") vs Some("<3>"): commutes
first          []          None vs None: commutes
option_to_vec  Some(7)     [70] vs [70]: commutes
reverse        [3, 1, 2]   ["<2>", "<1>", "<3>"] vs ["<2>", "<1>", "<3>"]: commutes
len            [3, 1, 2]   3 vs 3: commutes
```

Every square commutes, including the one for the empty vector. Each line is a test of one input. The rest of the chapter shows that for these functions the tests are, in a precise sense, unnecessary: the equation holds for *every* input and *every* `f`, and the reason is visible in the type.

18.4 Naturality in Lean

A functor class, and natural transformations

The Lean code for this section is in `code/lean/Book/Ch18/Natural.lean`. Chapter 17 built a functor class with laws; to keep this chapter independent of Chapter 17's files, we redefine a small one here. We call it `TypeFunctor`, because core Lean already has a class called `Functor` (the one behind `<$>`), and we want no confusion between the two:

```
/-- A functor from the category of Lean types to itself: a type constructor `F`
with a `map` that preserves identities and composition. -/
class TypeFunctor (F : Type → Type) where
  map : {α β : Type} → (α → β) → F α → F β
  map_id : ∀ {α : Type} (x : F α), map id x = x
  map_comp : ∀ {α β γ : Type} (g : β → γ) (f : α → β) (x : F α),
    map (g ∘ f) x = map g (map f x)
```

A `TypeFunctor` is a type constructor `F : Type → Type` with a `map` and proofs of the two functor laws. Here are the instances we need: `Option` and `List` (whose laws are library lemmas or short case analyses), the identity functor `Ident` and the constant functor `Const C`.

```
instance : TypeFunctor Option where
  map := Option.map
  map_id x := by cases x <;> rfl
  map_comp _ _ x := by cases x <;> rfl

instance : TypeFunctor List where
  map := List.map
  map_id x := List.map_id x
  map_comp _ _ _ := List.map_map.symm

/-- The identity functor: `Ident α` is just `α`. -/
def Ident (α : Type) : Type := α

instance : TypeFunctor Ident where
  map f x := f x
  map_id _ := rfl
  map_comp _ _ _ := rfl

/-- The constant functor: `Const C α` is `C`, whatever `α` is, and `map f`
does nothing. -/
def Const (C : Type) (_ : Type) : Type := C

instance (C : Type) : TypeFunctor (Const C) where
  map _ x := x
  map_id _ := rfl
  map_comp _ _ _ := rfl
```

`Ident α` *is* `α` and `Const C α` *is* `C`; giving them names lets instances attach to them. For both, the laws hold by `rfl`: `Ident`'s `map f` is `f` itself, and `Const`'s `map f` returns its argument unchanged.

A natural transformation is now a structure with two fields, the components and the proof of naturality:

```
/-- A natural transformation from `F` to `G`: one function `F α → G α` for
every type `α` (the *components*), such that every naturality square
commutes. -/
structure NatTrans (F G : Type → Type) [TypeFunctor F] [TypeFunctor G] where
  app : (α : Type) → F α → G α
  naturality : ∀ {α β : Type} (f : α → β) (x : F α),
    app β (map f x) = map f (app α x)
```

`app α` is the component η_α (the name `app` is Mathlib's). `naturality` is the equation, stated pointwise: for every `f : α → β` and every `x : F α`, transforming the mapped input equals mapping the transformed input. Instance resolution picks `F`'s `map` on the left and `G`'s on the right. A value of type `NatTrans F G` is the family *together with* a proof that it is natural; Lean will not accept the family alone.

Proving the squares

Here is `List.head?`, Lean's version of `first`, as a natural transformation:

```
/-- `List.head?` is natural: taking the head and then mapping is the same as
mapping and then taking the head. -/
def headNat : NatTrans List Option where
  app _ := List.head?
  naturality f xs := by
    -- Goal: (List.map f xs).head? = Option.map f xs.head?
    cases xs with
    | nil => rfl            -- both sides are `none`
    | cons a rest => rfl    -- both sides are `some (f a)`
```

The component at every type is `List.head?`. The proof of naturality starts from the goal written in the comment: `List.head?` of the mapped list equals `Option.map f` of the head. The tactic `cases xs with` splits on the two ways a list can be built. If `xs` is `[]`, both sides compute to `none`, and `rfl` closes the goal. If `xs` is `a :: rest`, the left side computes to `(f a :: List.map f rest).head?`, which is `some (f a)`, and the right side to `Option.map f (some a)`, which is also `some (f a)`; again `rfl`. The proof mirrors the reason the square commutes: the head of a list depends only on whether the list is empty, which mapping does not change, and on its first element, which mapping changes in exactly the way `Option.map` does.

`Option.toList` (Rust's `Option::into_iter` followed by `collect`) is the same kind of proof:

```
/-- `Option.toList` is natural (Rust's `Option::into_iter`, collected). -/
def toListNat : NatTrans Option List where
  app _ := Option.toList
  naturality f o := by
    cases o <;> rfl
```

The combinator `<;>` runs the tactic on its right in every goal produced by the tactic on its left: `cases o` produces the `none` and `some` cases, and `rfl` closes both.

`List.reverse` needs an induction, because the reverse of `a :: rest` is defined in terms of the reverse of `rest`:

```
/-- `List.reverse` is natural. This time the two paths agree only after an
induction on the list. -/
theorem map_reverse' {α β : Type} (f : α → β) (xs : List α) :
    List.map f xs.reverse = (List.map f xs).reverse := by
  induction xs with
  | nil => rfl
  | cons a rest ih =>
    -- Goal: map f (reverse (a :: rest)) = reverse (map f (a :: rest))
    simp only [List.reverse_cons, List.map_append, List.map_cons, List.map_nil, ih]

def reverseNat : NatTrans List List where
  app _ := List.reverse
  naturality f xs := (map_reverse' f xs).symm
```

The base case is `rfl`: both sides are `[]`. In the inductive case the goal is `map f (reverse (a :: rest)) = reverse (map f (a :: rest))`, and the induction hypothesis `ih` says the same thing for `rest`. The `simp only [...]` call rewrites with exactly the listed lemmas and nothing else: `List.reverse_cons` turns `reverse (a :: rest)` into `reverse rest ++ [a]` (on both sides, since `map f (a :: rest)` becomes `f a :: map f rest` by `List.map_cons`), `List.map_append` distributes `map` over `++`, `List.map_nil` handles the singleton's tail, and `ih` replaces `map f (reverse rest)` by `reverse (map f rest)`. After these rewrites both sides read `reverse (map f rest) ++ [f a]`, and `simp` closes the goal. (Lean's library already has this lemma as `List.map_reverse`; proving it once by hand shows what is inside.) The natural transformation `reverseNat` then just packages the lemma, with `.symm` flipping the equation into the orientation `NatTrans` expects.

The remaining two rows of the table are one-liners:

```
/-- `List.length` is a natural transformation into a constant functor:
mapping does not change the length. -/
def lengthNat : NatTrans List (Const ℕ) where
  app _ := List.length
  naturality f xs := List.length_map (as := xs) f

/-- `some` is a natural transformation from the identity functor to `Option`. -/
def someNat : NatTrans Ident Option where
  app _ x := some x
  naturality _ _ := rfl
```

For `lengthNat`, the goal is `(List.map f xs).length = xs.length` (the constant functor's `map` disappears), which is the library lemma `List.length_map`. For `someNat`, both sides are `some (f x)` by definition. And the components really are the functions we started from:

```
/-- info: some 3 -/
#guard_msgs in
#eval headNat.app ℕ [3, 1, 2]

/-- info: [2, 1, 3] -/
#guard_msgs in
#eval reverseNat.app ℕ [3, 1, 2]
```

Compare this with Rust. The Rust program checked five squares on five inputs. The Lean file proves the same five squares for all inputs, all types and all functions `f`, once each.

18.5 Composing natural transformations

Vertical composition

If $\eta : F \Rightarrow G$ and $\theta : G \Rightarrow H$, you can follow one by the other: the components $\theta_a \circ \eta_a : F\,a \to H\,a$ form a family from F to H. This is called **vertical composition** (picture the three functors stacked vertically, with the transformations as arrows between them), and the composite is again natural. On paper: for $f : a \to b$,

$$H f \circ \theta_a \circ \eta_a = \theta_b \circ G f \circ \eta_a = \theta_b \circ \eta_b \circ F f$$

using the naturality of θ for the first step and of η for the second. In other words, you can paste two commuting squares side by side and the outer rectangle commutes. In Lean:

```
/-- Vertical composition: first `η : F ⇒ G`, then `θ : G ⇒ H`. -/
def NatTrans.vcomp {F G H : Type → Type} [TypeFunctor F] [TypeFunctor G]
    [TypeFunctor H] (θ : NatTrans G H) (η : NatTrans F G) : NatTrans F H where
  app α x := θ.app α (η.app α x)
  naturality f x := by
    -- Goal: θ.app β (η.app β (map f x)) = map f (θ.app α (η.app α x))
    rw [η.naturality, θ.naturality]

/-- The identity natural transformation. -/
def NatTrans.id (F : Type → Type) [TypeFunctor F] : NatTrans F F where
  app _ x := x
  naturality _ _ := rfl
```

The proof is the paper proof, read from the left: `rw [η.naturality]` rewrites `η.app β (map f x)` into `map f (η.app α x)`, so the left side becomes `θ.app β (map f (η.app α x))`; then `rw [θ.naturality]` rewrites that into `map f (θ.app α (η.app α x))`, which is the right side, and `rw` closes the goal by reflexivity. The identity transformation, whose components are identity functions, is natural for the trivial reason that both sides are `map f x`.

Composition gives naturality for free. "The first element, as a list of length at most one" is the composite of `headNat` and `toListNat`, and its naturality needs no new proof:

```
/-- "The first element, as a list of length at most one": natural with no new
proof, because it is a composite of natural transformations. -/
def firstAsList : NatTrans List List := toListNat.vcomp headNat

/-- info: [3] -/
#guard_msgs in
#eval firstAsList.app ℕ [3, 1, 2]

example (f : ℕ → String) (xs : List ℕ) :
    firstAsList.app String (xs.map f) = (firstAsList.app ℕ xs).map f :=
  firstAsList.naturality f xs
```

The `example` uses the composite's `naturality` field, which `vcomp` built from the naturality of the two pieces.

Vertical composition is associative and has the identity as a unit. To prove equations between natural transformations, we first need to know when two of them are equal:

```
/-- Two natural transformations are equal when their components are. The
naturality fields are proofs, and any two proofs of the same proposition are
equal, so they take care of themselves. -/
theorem NatTrans.ext' {F G : Type → Type} [TypeFunctor F] [TypeFunctor G]
    {η θ : NatTrans F G} (h : ∀ α x, η.app α x = θ.app α x) : η = θ := by
  cases η with
  | mk app₁ nat₁ =>
    cases θ with
    | mk app₂ nat₂ =>
      -- Turn the pointwise equality into an equality of functions.
      have : app₁ = app₂ := funext fun α => funext fun x => h α x
      subst this
      rfl
```

`cases` takes both structures apart into their fields. `funext` (function extensionality, Chapter 16) turns the pointwise hypothesis `h` into an equality `app₁ = app₂`, and `subst` replaces `app₂` by `app₁` everywhere. What remains is `⟨app₁, nat₁⟩ = ⟨app₁, nat₂⟩`, where `nat₁` and `nat₂` are proofs of the same proposition; any two such proofs are equal by definition (proof irrelevance, Chapter 10), so `rfl` closes the goal. With this lemma the category laws are immediate:

```
variable {F G H K : Type → Type} [TypeFunctor F] [TypeFunctor G] [TypeFunctor H]
  [TypeFunctor K]

theorem NatTrans.id_vcomp (η : NatTrans F G) : (NatTrans.id G).vcomp η = η :=
  NatTrans.ext' fun _ _ => rfl

theorem NatTrans.vcomp_id (η : NatTrans F G) : η.vcomp (NatTrans.id F) = η :=
  NatTrans.ext' fun _ _ => rfl

theorem NatTrans.vcomp_assoc (ι : NatTrans H K) (θ : NatTrans G H)
    (η : NatTrans F G) : ι.vcomp (θ.vcomp η) = (ι.vcomp θ).vcomp η :=
  NatTrans.ext' fun _ _ => rfl
```

Each law holds componentwise by `rfl`, because composing and identity functions satisfy these laws by computation.

The functor category

Everything so far was about functors on types, but the definitions make sense for any two categories, and Lean can say so with the book's own `Category` class from Chapter 16. The file `code/lean/Book/Ch18/FunctorCategory.lean` imports `Book.Ch16.Category` and redefines Chapter 17's notion of a functor between categories (again, to keep the chapters independent):

```
/-- A functor between two categories: a map on objects and a map on morphisms
that preserves identities and composition. -/
structure CatFunctor (C D : Type) [Category C] [Category D] where
  obj : C → D
  map : {a b : C} → (a → b) → (obj a → obj b)
  map_id : ∀ a : C, map (Category.id a) = Category.id (obj a)
  map_comp : ∀ {a b c : C} (g : b → c) (f : a → b), map (g ∘ f) = map g ∘ map f
```

```
/-- A natural transformation `F ⇒ G`: a morphism `F a → G a` in `D` for every
object `a` of `C`, such that every naturality square commutes. -/
structure CatNatTrans (F G : CatFunctor C D) where
  app : (a : C) → (F.obj a → G.obj a)
  naturality : ∀ {a b : C} (f : a → b), app b ∘ F.map f = G.map f ∘ app a
```

The notation is Chapter 16's: `a → b` is the type of morphisms, and `g ∘ f` is "first `f`, then `g`". The naturality field is the square, now as an equation between morphisms of `D` with no points in sight: `app b ∘ F.map f` (map, then transform) equals `G.map f ∘ app a` (transform, then map). Vertical composition is proved by moving parentheses:

```
namespace CatNatTrans

/-- The identity natural transformation. -/
def id (F : CatFunctor C D) : CatNatTrans F F where
  app a := Category.id (F.obj a)
  naturality f := by
    -- id ∘ F.map f = F.map f ∘ id
    rw [Category.id_comp, Category.comp_id]

/-- Vertical composition: first `η : F ⇒ G`, then `θ : G ⇒ H`. -/
def vcomp {F G H : CatFunctor C D} (θ : CatNatTrans G H) (η : CatNatTrans F G) :
    CatNatTrans F H where
  app a := θ.app a ∘ η.app a
  naturality f := by
    -- Goal: (θ_b ∘ η_b) ∘ F f = H f ∘ (θ_a ∘ η_a)
    rw [← Category.assoc, η.naturality, Category.assoc, θ.naturality,
      ← Category.assoc]
```

Read the rewrite chain as the paper proof. The goal is `(θ_b ∘ η_b) ∘ F f = H f ∘ (θ_a ∘ η_a)`. `← Category.assoc` regroups the left side as `θ_b ∘ (η_b ∘ F f)`; `η.naturality` replaces the inner part by `G f ∘ η_a`; `Category.assoc` regroups to `(θ_b ∘ G f) ∘ η_a`; `θ.naturality` replaces `θ_b ∘ G f` by `H f ∘ θ_a`; and a final `← Category.assoc` produces `H f ∘ (θ_a ∘ η_a)`, the right side. With an extensionality lemma like the one above (`CatNatTrans.ext`, in the file), we can state the punchline: functors from `C` to `D` are the objects of a category whose morphisms are natural transformations.

```
/-- The functor category: objects are functors `C ⇒ D`, morphisms are natural
transformations, composition is vertical composition. The laws are inherited,
component by component, from the laws of `D`. -/
instance functorCategory : Category (CatFunctor C D) where
  Hom F G := CatNatTrans F G
  id F := CatNatTrans.id F
  comp θ η := CatNatTrans.vcomp θ η
  id_comp _ := CatNatTrans.ext fun _ => Category.id_comp _
  comp_id _ := CatNatTrans.ext fun _ => Category.comp_id _
  assoc _ _ _ := CatNatTrans.ext fun _ => Category.assoc _ _ _
```

This is the **functor category**, often written $[\mathcal{C}, \mathcal{D}]$ or $\mathcal{D}^{\mathcal{C}}$. Each of its laws is `CatNatTrans.ext` followed by the corresponding law of `D` at every object. Historically, natural transformations came first: Samuel Eilenberg and Saunders Mac Lane invented category theory in the 1940s largely to say precisely what "natural" means in mathematics, and introduced categories and functors because they were needed to define it.

A small example shows how much the general definition can mean. In a preorder viewed as a category (Chapter 16), there is a morphism $a \to b$ exactly when $a \le b$, and a functor between preorders is a monotone function.

```
/-- The natural numbers ordered by `≤`, as a category: there is one morphism
`a → b` exactly when `a ≤ b` (`PLift` turns the proof into data). The instance
is local to this file. -/
local instance leCategory : Category ℕ where
  Hom a b := PLift (a ≤ b)
  id a := ⟨Nat.le_refl a⟩
  comp g f := ⟨Nat.le_trans f.down g.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

/-- A monotone function is a functor on `(ℕ, ≤)`. The functor laws hold by
`rfl`: any two proofs of the same inequality are equal. -/
def monotone (f : ℕ → ℕ) (mono : ∀ {a b : ℕ}, a ≤ b → f a ≤ f b) :
    CatFunctor ℕ ℕ where
  obj := f
  map h := ⟨mono h.down⟩
  map_id _ := rfl
  map_comp _ _ := rfl
```

A morphism `a → b` is a proof of `a ≤ b`, wrapped with `PLift` to turn a proof into data (the `Category` class wants morphisms in `Type`, and proofs live in `Prop`). The instance is `local`, so that it does not attach itself to `ℕ` in any other file. Since any two proofs of `a ≤ b` are equal, every diagram in this category commutes, and in particular every naturality square. A natural transformation between monotone maps is therefore nothing more than a family of inequalities:

```
/-- Between monotone maps, a natural transformation is exactly a pointwise
inequality: naturality squares commute automatically in a preorder. -/
```

```
theorem natTrans_iff_le (F G : CatFunctor ℕ ℕ) :
    Nonempty (CatNatTrans F G) ↔ ∀ n, F.obj n ≤ G.obj n := by
  constructor
  · -- A natural transformation gives a morphism `F n → G n`, a proof of `≤`.
    rintro ⟨η⟩ n
    exact (η.app n).down
  · -- Conversely, the inequalities are the components, and every naturality
    -- square commutes because both paths are proofs of the same `≤`.
    intro h
    exact ⟨{ app := fun n => ⟨h n⟩, naturality := fun _ => rfl }⟩

def double : CatFunctor ℕ ℕ := monotone (fun n => 2 * n) (by omega)
def succ : CatFunctor ℕ ℕ := monotone (fun n => n + 1) (by omega)

/-- There is a natural transformation from the identity to `double`... -/
example : Nonempty (CatNatTrans (monotone id fun h => h) double) :=
  (natTrans_iff_le _ _).mpr fun n => show n ≤ 2 * n by omega

/-- ...but none from `succ` to `double`: the component at `0` would be a proof
of `1 ≤ 0`. -/
example : ¬ Nonempty (CatNatTrans succ double) := by
  rw [natTrans_iff_le]
  intro h
  have : 0 + 1 ≤ 2 * 0 := h 0
  omega
```

`natTrans_iff_le` says that a natural transformation from `F` to `G` exists if and only if `F n ≤ G n` for every `n`. In one direction, the component at `n` *is* a proof of `F n ≤ G n`. In the other, the inequalities serve as components, and the naturality obligation `fun _ => rfl` holds because both sides of each square are proofs of the same inequality. (`rintro ⟨η⟩ n` introduces the hypothesis and takes the `Nonempty` apart in one step.) There is a natural transformation from the identity to doubling, since $n \le 2n$, but none from successor to doubling: its component at `0` would prove $1 \le 0$, which `omega` refutes. Natural transformations between monotone maps are the pointwise order on them. Chapter 27 will view lifetimes as a preorder of this kind, and variance as functoriality on it.

18.6 Parametricity: theorems for free

One function at a time, or all at once?

We proved naturality one function at a time. But the intuition from the start of the chapter says more: *any* function with the signature `fn<T>(Vec<T>) -> Option<T>` should commute with `map`. It can drop elements, rearrange them, and choose which one to return based on positions or on the length, but not based on values, because it knows nothing about their type.

That claim is a theorem, due to John Reynolds (*Types, Abstraction and Parametric Polymorphism*, 1983), and Philip Wadler made it famous as a recipe for deriving equations from types (*Theorems for free!*, 1989). The property is called **parametricity**, and the equations it produces are **free theorems**: free because you get them from the type alone, without looking at the code.

Relations instead of functions

Reynolds' insight was to interpret every type not as a set of values but as a *relation* between values. Fix two types A and B and a relation $R \subseteq A \times B$ (any relation: think of it as "a pairs with b"). For a type

expression τ that mentions the type variable α, define a relation $[\![\tau]\!]_R$ between $\tau[A/\alpha]$ and $\tau[B/\alpha]$ by recursion on τ:

- $[\![\alpha]\!]_R = R$: at the type variable, use the given relation.
- $[\![\texttt{bool}]\!]_R$ is equality, and the same for every other type that does not mention α: a boolean on the left is related only to the same boolean on the right.
- $[\![\tau \times \sigma]\!]_R$ relates (x_1, x_2) to (y_1, y_2) when x_1 is related to y_1 at τ and x_2 to y_2 at σ.
- $[\![\mathrm{List}\,\tau]\!]_R$ relates two lists when they have the same length and their elements are related position by position.
- $[\![\tau \to \sigma]\!]_R$ relates two functions g and h when they send related arguments to related results: whenever x is related to y at τ, $g\,x$ is related to $h\,y$ at σ.

The last clause is the heart of it. A related pair of functions is one that "respects" the relation: it cannot tell related inputs apart in a way that would make the outputs unrelated.

The abstraction theorem (Reynolds). If t is a closed term of type $\forall\alpha.\,\tau$ in a language like System F (Chapter 12) with no way to inspect types, no effects and no non-termination, then for all types A, B and every relation $R \subseteq A \times B$, the two instances t_A and t_B are related by $[\![\tau]\!]_R$.

In words: every polymorphic term respects every relation. The proof is by induction on the typing derivation of t, one case per typing rule, and it is a *meta-theorem*: a theorem about all the terms of a language, proved from outside the language. We will return to this point, because it is exactly what Lean cannot do internally.

The free theorem for the identity

Let $t : \forall\alpha.\,\alpha \to \alpha$. Pick any type A and any value $a : A$, and choose the relation that relates a to itself and nothing else:

$$R = \{(a, a)\} \subseteq A \times A$$

By the abstraction theorem, t_A is related to t_A at $[\![\alpha \to \alpha]\!]_R$. By the function clause, since $(a, a) \in R$, we get $(t_A\,a, t_A\,a) \in R$. The only pair in R is (a, a), so $t_A\,a = a$. Since A and a were arbitrary, t is the identity: **a total, pure function of type $\forall\alpha.\,\alpha \to \alpha$ can only return its argument**. You knew this informally (what else could `fn id<T>(x: T) -> T` return?); now you have a proof, and its assumptions are spelled out: no type inspection, no effects, termination.

The free theorem for lists

Let $r : \forall\alpha.\,\mathrm{List}\,\alpha \to \mathrm{List}\,\alpha$ and take any function $f : A \to B$. This time choose the **graph** of f as the relation:

$$R = \{(a, f\,a) \mid a \in A\}$$

Two lists are related at $[\![\mathrm{List}\,\alpha]\!]_R$ exactly when the second is the first with f applied to every element, that is, $ys = \mathrm{map}\,f\,xs$. The abstraction theorem says that r_A and r_B send related lists to related lists. Feed them the related pair $(xs, \mathrm{map}\,f\,xs)$: the outputs $r_A\,xs$ and $r_B\,(\mathrm{map}\,f\,xs)$ are related, which means

$$r_B\,(\mathrm{map}\,f\,xs) = \mathrm{map}\,f\,(r_A\,xs)$$

This is precisely the naturality square. The same argument works for any functorial types built from α with products, sums, lists and similar constructors, and gives the slogan of the chapter: **every parametric polymorphic function $\forall\alpha.\, F\,\alpha \to G\,\alpha$ is a natural transformation**. `first`, `reverse` and `len` are natural not because of how they are implemented but because of their types.

Other types give other free theorems. A few, all obtained the same way (choose graphs of functions as the relations and unfold the clauses):

Type	Free theorem
$\forall\alpha.\, \alpha \to \alpha$	the function is the identity
$\forall\alpha\,\beta.\, \alpha \to \beta \to \alpha$	the function returns its first argument
$\forall\alpha.\, \alpha \to$ `usize`	the function is constant
$\forall\alpha.\, \mathrm{List}\,\alpha \to$ `usize`	the result depends only on the length
$\forall\alpha.\, \mathrm{List}\,\alpha \to \mathrm{List}\,\alpha$	it commutes with $\mathrm{map}\,f$ for every f
$\forall\alpha\,\beta.\, \alpha \to \beta$	there is no such total function

The last row is the logical reading from Chapter 10: $\forall\alpha\,\beta.\, \alpha \to \beta$ would prove every proposition from every other, and the free theorem confirms that no total, parametric program has this type. Rust can write `fn coerce<A, B>(a: A) -> B` only with `panic!`, `loop {}` or `unsafe`.

Parametricity for `reverse`, in Lean

Lean cannot prove the abstraction theorem for itself (the next section explains why), but it can prove the relational property for one specific function, and that shows exactly how the free theorem falls out. The file is `code/lean/Book/Ch18/Parametricity.lean`. First, the relational lifting of a relation to lists, as an inductive predicate: two lists are related when both are empty, or when their heads are related and their tails are related.

```
/-- The relational lifting of `R` to lists: two lists are related when they
have the same length and related elements in every position. -/
inductive ListRel {α β : Type} (R : α → β → Prop) : List α → List β → Prop
  | nil : ListRel R [] []
  | cons {a b xs ys} : R a b → ListRel R xs ys → ListRel R (a :: xs) (b :: ys)
```

This is the clause for $[\![\mathrm{List}\,\tau]\!]_R$. Related lists can be concatenated, by induction on the first relatedness proof:

```
/-- Related lists concatenate to related lists. -/
theorem ListRel.append {α β : Type} {R : α → β → Prop} {xs ys xs' ys'}
    (h : ListRel R xs ys) (h' : ListRel R xs' ys') :
    ListRel R (xs ++ xs') (ys ++ ys') := by
  induction h with
  | nil => exact h'
  | cons hab _ ih => exact ListRel.cons hab ih
```

`induction h with` does induction on the *derivation* of `ListRel R xs ys`, the way Chapter 4 did induction on derivations: in the `nil` case `xs` and `ys` are both empty and the goal is `h'` itself; in the `cons` case we rebuild the `cons` with the induction hypothesis for the tails. Now the relational property of `reverse`, for every relation between any two types:

```
/-- **Parametricity of `reverse`.** For every relation `R` between any two
types, `reverse` sends related lists to related lists. -/
theorem reverse_param {α β : Type} (R : α → β → Prop) {xs : List α} {ys : List β}
    (h : ListRel R xs ys) : ListRel R xs.reverse ys.reverse := by
  induction h with
  | nil => exact ListRel.nil
  | cons hab _ ih =>
    -- reverse (a :: xs) = reverse xs ++ [a], and the same on the right.
    simp only [List.reverse_cons]
    exact ih.append (ListRel.cons hab ListRel.nil)
```

Again by induction on the relatedness derivation. In the `cons` case, `simp only [List.reverse_cons]` rewrites `reverse (a :: xs)` into `reverse xs ++ [a]` on both sides, and the related reversed tails (`ih`) are appended to the related singletons `[a]` and `[b]`.

This theorem says nothing about functions yet. To get the free theorem, we choose the graph of a function as the relation, and check that relatedness then means "is the image under `map f`":

```
/-- When `R` is the graph of a function `f`, related lists are exactly a list
and its image under `map f`. -/
theorem listRel_graph {α β : Type} (f : α → β) (xs : List α) (ys : List β) :
    ListRel (fun a b => f a = b) xs ys ↔ xs.map f = ys := by
  constructor
  · intro h
    induction h with
    | nil => rfl
    | cons hab _ ih => simp [hab, ih]
  · intro h
    subst h
    induction xs with
    | nil => exact ListRel.nil
    | cons a rest ih => exact ListRel.cons rfl ih
```

Each direction is an induction (the left-to-right one on the relatedness proof, the right-to-left one on the list, after `subst h` replaces `ys` by `xs.map f`). And now naturality comes out exactly as in the paper argument:

```
/-- The free theorem for `reverse`, obtained from parametricity by choosing the
graph of `f` as the relation: naturality. -/
theorem reverse_natural {α β : Type} (f : α → β) (xs : List α) :
    xs.reverse.map f = (xs.map f).reverse := by
  have related : ListRel (fun a b => f a = b) xs (xs.map f) :=
    (listRel_graph f xs (xs.map f)).mpr rfl
  exact (listRel_graph f _ _).mp (reverse_param _ related)
```

`related` says that `xs` and `xs.map f` are related by the graph of `f`; `reverse_param` turns that into a relation between the reversed lists, and `listRel_graph` reads it back as an equation. We proved the same equation by direct induction in `Natural.lean`; the point of this detour is to see the general recipe, abstraction theorem plus graph relation, work on one instance.

18.7 Why Lean cannot prove the free theorem

A meta-theorem

Why not prove the abstraction theorem in Lean once and for all, for every function of type `(α : Type) → List α → List α`? Here is the statement for the simplest case, the identity:

```
/-- The free theorem for `∀ α, α → α`, stated *inside* Lean as a proposition
about every function of that type. -/
def FreeTheoremId : Prop :=
  ∀ (f : (α : Type) → α → α) (α : Type) (a : α), f α a = a
```

`FreeTheoremId` is a perfectly good Lean proposition: for every function `f` of type `(α : Type) → α → α` (Lean's way of writing $\forall \alpha.\, \alpha \to \alpha$, Chapter 12), `f α a = a`. But Reynolds' theorem is not about "every function of that type". It is about every *term*, every piece of syntax that can be written in a particular language, and its proof goes by induction on how the term was built. Inside Lean, a variable `f : (α : Type) → α → α` does not come with a construction we could do induction on; it ranges over every function of that type that Lean's logic admits. Nothing in the logic says that those functions were all written uniformly, without looking at `α`.

And in fact, Lean's logic admits a function of that type that is not the identity. With classical logic, Lean can ask a question about types that no program could answer:

```
open Classical in
/-- A "polymorphic identity" that inspects its type argument. Classical logic
lets it ask whether `α` is `Bool` (a question no program could decide), and on
`Bool` it negates. -/
noncomputable def notId (α : Type) (a : α) : α :=
  if h : α = Bool then cast h.symm (!(cast h a)) else a

theorem notId_true : notId Bool true = false := by
  simp [notId]

/-- So the internal free theorem is not just unprovable: in Lean (with its
standard axioms) it is false. -/
theorem not_freeTheoremId : ¬ FreeTheoremId := by
  intro h
  have : notId Bool true = true := h notId Bool true
  rw [notId_true] at this
  exact Bool.false_ne_true this
```

`open Classical in` makes every proposition decidable for the next declaration (Chapter 5 explained `Decidable` and why `if` on a `Prop` needs it). So `if h : α = Bool then ... else ...` is allowed: if the type argument happens to be `Bool`, `notId` uses the proof `h` to `cast` the argument to a `Bool`, negates it, and casts back; otherwise it returns the argument. The definition is marked `noncomputable` because there is no algorithm behind the decision "is `α` equal to `Bool`?": Lean accepts it as a mathematical function but will not compile it to code. `notId_true` checks, by `simp` (which unfolds `notId`, decides the `if` using `h : Bool = Bool` and simplifies the casts), that `notId Bool true = false`. Then `not_freeTheoremId` applies the supposed free theorem to `notId` and derives `false = true`, a contradiction.

So inside Lean, with its standard axioms, the internal free theorem is not merely unprovable: it is *false*. The axioms show up where they should:

```
/-- info: 'Ch18.Param.not_freeTheoremId' depends on axioms: [propext, Classical.choice,
Quot.sound] -/
#guard_msgs in
#print axioms not_freeTheoremId

/-- info: 'Ch18.Param.reverse_natural' depends on axioms: [propext] -/
#guard_msgs in
#print axioms reverse_natural
```

The refutation uses `Classical.choice`. The free theorem for `reverse`, proved directly, uses only `propext` (through `simp`), and no choice. What about Lean without classical axioms? Then `notId` cannot be written, but the free theorem cannot be proved either: if it could be proved without axioms, it would remain provable after adding `Classical.choice`, and we have just refuted it there. (This argument assumes Lean with choice is consistent, which is the standard belief and the basis of all of Mathlib.)

> **NOTE**
>
> Parametricity is a property of a *language*, proved about its terms from the outside. Researchers have designed type theories in which parametricity is an internal principle that you can invoke in proofs; Lean is not one of them, and its classical axioms are incompatible with that design. In Lean, free theorems are proved function by function, as we did for `reverse`.

In practice, a free theorem is a reasoning principle about the code you *write*, in a language whose rules keep generic code from looking at types. In Lean, you prove the equation for the function in front of you, usually in a few lines; in Rust, you rely on it as long as the function stays inside the parametric fragment of the language. How big is that fragment?

18.8 Where Rust leaks parametricity

Rust's generics are designed to be parametric by default. Inside `fn f<T>(x: T) -> T`, the body cannot compare `x` with anything, print it or copy it. The compiler enforces this, as this rejected program (`code/rust/compile_fail/ch18_no_peeking.rs`) shows:

```rust
fn keep_if_same<T>(a: T, b: T) -> Option<T> {
    if a == b { Some(a) } else { None }
}
```

```
error[E0369]: binary operation `==` cannot be applied to type `T`
 --> compile_fail/ch18_no_peeking.rs:7:10
  |
7 |     if a == b { Some(a) } else { None }
  |        - ^^ - T
  |        |
  |        T
  |
help: consider restricting type parameter `T` with trait `PartialEq`
  |
6 | fn keep_if_same<T: std::cmp::PartialEq>(a: T, b: T) -> Option<T> {
  |                  ++++++++++++++++++++++
```

The error E0369 says that `==` is not available for an arbitrary `T`, and the help suggests the fix: add a bound `T: PartialEq`. That fix changes the signature, and with it the free theorem. This is the first and most honest way Rust leaks parametricity: **trait bounds**. There are several others.

Trait bounds

A bound gives the function an operation on `T`, and the function may use it to make decisions based on values. The free theorem then holds only for functions `f` that respect the operation:

```rust
/// Needs `Ord`: it compares elements, so it is natural only for maps that
/// preserve the order.
fn largest<T: Ord>(v: Vec<T>) -> Option<T> {
    v.into_iter().max()
}

/// Needs `PartialEq`: it compares neighbors, so it is natural only for maps
/// that never make two different elements equal.
fn dedup<T: PartialEq>(mut v: Vec<T>) -> Vec<T> {
    v.dedup();
    v
}
```

```rust
println!();
let negate = |n: i32| -n;
let parity = |n: i32| n % 2;
let input = vec![3, 1, 2];
report(
    "largest, *10",
    "[3, 1, 2]",
    square(input.clone(), largest, largest, vmap(tens), |o| o.map(tens)),
);
report(
    "largest, neg",
    "[3, 1, 2]",
    square(input.clone(), largest, largest, vmap(negate), |o| {
        o.map(negate)
    }),
);
report(
    "dedup, *10",
    "[3, 1, 2]",
    square(input.clone(), dedup, dedup, vmap(tens), vmap(tens)),
);
report(
    "dedup, %2",
    "[3, 1, 2]",
    square(input, dedup, dedup, vmap(parity), vmap(parity)),
);
```

```
largest, *10   [3, 1, 2]   Some(30) vs Some(30): commutes
largest, neg   [3, 1, 2]   Some(-1) vs Some(-3): FAILS
dedup, *10     [3, 1, 2]   [30, 10, 20] vs [30, 10, 20]: commutes
dedup, %2      [3, 1, 2]   [1, 0] vs [1, 1, 0]: FAILS
```

`largest` commutes with multiplying by 10, which preserves the order, and fails for negation, which reverses it: the largest of the negated numbers is -1, while the negation of the largest is -3. `dedup` commutes with an injective map and fails for `n % 2`, which makes `3` and `1` equal and so creates a duplicate that was not there before. These are still free theorems, just weaker ones: parametricity with a bound `T: Ord` yields naturality for monotone maps, and with `T: PartialEq` for injective ones. Mathlib proves the second as a library lemma, and Lean refutes the general version with `decide`:

```
/-- `dedup` needs `DecidableEq`: it compares elements. It commutes with `map f`
for injective `f` (Mathlib proves this as `List.dedup_map_of_injective`)... -/
example (f : ℕ → ℕ) (hf : Function.Injective f) (xs : List ℕ) :
    (xs.map f).dedup = xs.dedup.map f :=
  List.dedup_map_of_injective hf xs

/-- ...but not for a map that identifies two elements. -/
example : ([1, 2].map fun _ => (0 : ℕ)).dedup ≠ ([1, 2] : List ℕ).dedup.map fun _ => 0 := by
  decide
```

The pleasant thing about bounds is that they are visible: `largest<T: Ord>` announces in its signature that it looks at the elements, and a reader knows that the free theorem has a side condition.

`size_of`, `type_name` and `TypeId`

Some leaks need no visible bound at all:

```
/// Type `fn<T>(&T) -> usize`, no bound at all, yet not constant.
fn weigh<T>(_: &T) -> usize {
    size_of::<T>()
}

/// Type `fn<T: Any>(T) -> T`, but not the identity: it negates booleans.
fn sneaky_id<T: Any>(x: T) -> T {
    let mut boxed: Box<dyn Any> = Box::new(x);
    if let Some(b) = boxed.downcast_mut::<bool>() {
        *b = !*b;
    }
    *boxed.downcast::<T>().expect("the box still holds a T")
}

/// Type `fn<T>(T) -> T`, and never returns: `panic!` has type `!`.
fn stubborn<T>(_: T) -> T {
    panic!("no value of type T to return")
}
```

```
weigh(&1u8) = 1, weigh(&1u64) = 8
weigh(&String::new()) = 24
sneaky_id(5) = 5, sneaky_id(true) = false
stubborn(7) returned normally: false
```

`weigh` has the type $\forall\alpha.\ \&\,\alpha \to \texttt{usize}$, whose free theorem says it is constant, and yet it returns 1 for `u8`, 8 for `u64` and 24 for `String` (on a 64-bit platform). The function `std::mem::size_of::<T>()` reads a property of the type argument at compile time, and the same is true of `align_of` and `std::any::type_name`. Strictly speaking there *is* a bound: every type parameter in Rust has an implicit `T: Sized` bound unless you opt out with `T: ?Sized`, and `size_of` requires it. But nobody reads `Sized` as "this function may behave differently for different types", and the signature looks fully parametric.

`sneaky_id` is the Rust version of Lean's `notId`. Its type is `fn<T: Any>(T) -> T`, the type of the identity with a bound, and it negates booleans. `Any` (Chapter 15) gives every `'static` type a runtime type identifier, `TypeId`, and `downcast_mut::<bool>()` succeeds exactly when the boxed value is a `bool`. Here the bound is visible, and that is the right way to read `T: Any`: it grants the function permission to ask what `T` is, which voids the free theorem.

Panics, non-termination and effects

`stubborn` has the type `fn<T>(T) -> T` with no bound at all, and it is not the identity: it never returns. The free theorems assume total functions, and Rust (like every general-purpose language, Chapter 10) lets every type be "inhabited" by `panic!()` and `loop {}`. The honest free theorem for `fn<T>(T) -> T` in Rust is "the identity, or it diverges". Similarly, a generic function may print, write to a file, read the clock or update a global counter; the free theorem is about the value it returns and says nothing about those effects. The walkthrough at the end of the chapter uses this on purpose.

Specialization and `unsafe`

Two more holes, for completeness. **Specialization** is an unstable feature (nightly only, with soundness problems still being worked out; check its current status) that would let a generic `impl<T> Trait for T` be overridden by a more specific `impl Trait for i32`, so that generic code calling the trait method behaves differently at `i32`. And `unsafe` code can do anything, including reinterpreting the bytes of a `T` with `std::mem::transmute`. Both are outside the parametric fragment.

Leak	Visible in the signature?	Free theorem becomes
trait bound (`Ord`, `PartialEq`, ...)	yes	natural for maps that respect the operation
`T: Any` (`TypeId`, downcasting)	yes	none
`size_of`, `align_of`, `type_name`	no (implicit `Sized`)	none
`panic!`, `loop {}`	no	"... or diverges"
side effects	no	holds for the returned values only
specialization (unstable), `unsafe`	no	none

The table is less alarming than it looks: most generic Rust code never calls `size_of` or `Any`, and for pure, bound-free code the free theorems are as reliable as the type checker.

18.9 The Yoneda lemma, gently

Continuation-passing style

Here is a puzzle. What is the difference between a value of type α and a value of type

$$\forall \beta.\, (\alpha \to \beta) \to \beta\,?$$

The second is a function that, for any result type β, accepts a **continuation** $k : \alpha \to \beta$ ("what to do with the value") and returns a β. If you have a value a, you can build such a function: $\lambda k.\, k\, a$, "pass a to whatever continuation comes". Conversely, if you have such a function, you can get a value out of it by choosing $\beta = \alpha$ and the identity continuation. This is **continuation-passing style** (CPS), a program transformation that some compilers for functional languages use to make control flow explicit. The two types are isomorphic, and the reason is a free theorem.

In Rust, the type $\forall \beta.\, (\alpha \to \beta) \to \beta$ is rank 2 (the $\forall \beta$ sits inside), and Chapter 12 showed how to write such types: a trait with a generic method. The program is `code/rust/examples/ch18_yoneda.rs`.

```
/// `∀B. (A → B) → B`: a value of type `A` in continuation-passing style.
trait Cps<A> {
    fn run<B>(self, k: impl FnOnce(A) -> B) -> B;
}

/// The obvious implementation: hold the value, hand it to the continuation.
struct Value<A>(A);

impl<A> Cps<A> for Value<A> {
    fn run<B>(self, k: impl FnOnce(A) -> B) -> B {
        k(self.0)
    }
}
```

```
/// A less obvious one: compute the value only when a continuation arrives.
struct Deferred<F>(F);

impl<A, F: FnOnce() -> A> Cps<A> for Deferred<F> {
    fn run<B>(self, k: impl FnOnce(A) -> B) -> B {
        k((self.0)())
    }
}

/// Lowering: run with the identity continuation.
fn lower_cps<A>(c: impl Cps<A>) -> A {
    c.run(|a| a)
}
```

```
let show = |n: u32| format!("<{n}>");
println!("Value(42).run(show) = {}", Value(42).run(show));
println!("lower_cps(Value(42)) = {}", lower_cps(Value(42)));
let deferred = Deferred(|| 6 * 7);
println!("lower_cps(Deferred(|| 6 * 7)) = {}", lower_cps(deferred));
```

```
Value(42).run(show) = <42>
lower_cps(Value(42)) = 42
lower_cps(Deferred(|| 6 * 7)) = 42
```

`Value` holds a value and passes it on. `Deferred` holds only a recipe and computes the value when a continuation arrives. Both lower to `42`. In Lean the CPS type is an ordinary dependent function type (living in `Type 1`, because it quantifies over `Type`, Chapter 12), and one direction of the round trip holds by `rfl`:

```
/-- A value of type `α` in continuation-passing style: give me any
continuation `k : α → β` and I will produce a `β`. -/
def Cps (α : Type) : Type 1 := (β : Type) → (α → β) → β

/-- Pass the value to the continuation. -/
def toCps {α : Type} (a : α) : Cps α := fun _ k => k a

/-- Run with the identity continuation to get the value back. -/
def fromCps {α : Type} (c : Cps α) : α := c α id

theorem fromCps_toCps {α : Type} (a : α) : fromCps (toCps a) = a := rfl

/-- info: 42 -/
#guard_msgs in
#eval fromCps (toCps 42)

/-- info: "42!" -/
#guard_msgs in
#eval toCps 42 String (fun n => toString n ++ "!")
```

The other direction, "if I lower `c` and then lift the result, I get `c` back", is the interesting one. It says that `c β k = k (c α id)` for every `β` and `k`: the function `c` must compute its answer by applying `k` to one fixed value. That is the free theorem for the type $\forall \beta.\, (\alpha \to \beta) \to \beta$, and as we just saw, Lean cannot prove it for every `c`. We will need to assume it, in the form of naturality.

The lemma

Now replace the final β by $F\,\beta$ for a functor F. The **Yoneda lemma** (for functors on types) says that for every functor F and every type α,

$$\left(\forall \beta.\, (\alpha \to \beta) \to F\,\beta \right) \cong F\,\alpha$$

where the left side contains only the *natural* families. In categorical language, the left side is the set of natural transformations from the functor $\mathrm{Hom}(\alpha, -)$ (which sends β to the type of functions $\alpha \to \beta$, and maps by post-composition) to F, and the lemma reads

$$\mathrm{Nat}\ \big(\ \mathrm{Hom}(\alpha, -), F\ \big) \cong F\,\alpha$$

The two directions are the CPS ones, generalized:

- **lower**: given a natural family η, apply it to the identity: $\eta_\alpha(\mathrm{id}) : F\,\alpha$.
- **lift**: given $x : F\,\alpha$, define $\eta_\beta(k) = \mathrm{map}\,k\,x$.

Lowering a lifted x gives $\mathrm{map}\,\mathrm{id}\,x = x$, by the identity law of F. Lifting a lowered η gives the family $k \mapsto \mathrm{map}\,k\,(\eta_\alpha(\mathrm{id}))$, and naturality of η at the morphism k, applied to the element id, says exactly that this is $\eta_\beta(k \circ \mathrm{id}) = \eta_\beta(k)$. So a natural family is completely determined by one value, its component at α applied to the identity. CPS is the special case $F = \mathrm{Id}$.

In Lean

The file is `code/lean/Book/Ch18/Yoneda.lean`, which imports `Natural.lean` for the `TypeFunctor` class. The left side of the lemma is a structure that carries the family and its naturality:

```
/-- The left-hand side of the Yoneda lemma: a family of functions
`(α → β) → F β`, one for each type `β`, that is natural in `β`. -/
structure Yoneda (F : Type → Type) [TypeFunctor F] (α : Type) : Type 1 where
  run : (β : Type) → (α → β) → F β
  natural : ∀ {β γ : Type} (k : α → β) (g : β → γ),
    run γ (g ∘ k) = map g (run β k)
```

The `natural` field says: running with the continuation `g ∘ k` is the same as running with `k` and then mapping `g` over the result. (It is the naturality square of a transformation from $\mathrm{Hom}(\alpha, -)$ to F, written pointwise.) The two maps of the lemma:

```
/-- From `F α` to the natural family: map the continuation over `x`. The
family is natural because `map` preserves composition. -/
def Yoneda.lift (x : F α) : Yoneda F α where
  run _ k := map k x
  natural k g := map_comp g k x

/-- From the natural family back to `F α`: use the identity continuation. -/
def Yoneda.lower (y : Yoneda F α) : F α := y.run α id
```

`lift` must prove that its family is natural, and the proof is the composition law of `F`: `map (g ∘ k) x = map g (map k x)`. The round trips:

```
/-- Lowering a lifted value gives it back: this is the identity law of `F`. -/
theorem Yoneda.lower_lift (x : F α) : (Yoneda.lift x).lower = x :=
  map_id x

/-- Lifting a lowered family gives it back: this is naturality. Every
component `run β k` is determined by the single value `run α id`. -/
theorem Yoneda.lift_lower (y : Yoneda F α) : Yoneda.lift y.lower = y := by
  cases y with
  | mk run natural =>
    -- Goal: (fun β k => map k (run α id), _) = (run, natural)
    have h : (fun β (k : α → β) => map k (run α id)) = run := by
      funext β k
      -- `k ∘ id` is `k` by definition, so naturality at `id` and `k` gives
      -- run β k = map k (run α id).
```

```
      exact (natural id k).symm
    simp only [Yoneda.lift, Yoneda.lower, h]
```

`lower_lift` is the identity law `map id x = x`, verbatim. `lift_lower` takes the structure `y` apart into `run` and `natural`. The heart of the proof is the `have`: the family `fun β k => map k (run α id)` is equal to `run`. After `funext` fixes `β` and `k`, the goal is `map k (run α id) = run β k`, and `natural id k` states `run β (k ∘ id) = map k (run α id)`; since `k ∘ id` is `k` by definition, its symmetric form is exactly the goal. The final `simp only` unfolds `lift` and `lower` and rewrites with `h`, leaving `(run, _) = (run, natural)`, which holds by proof irrelevance. Packaged as an `Equiv` (Mathlib's type of isomorphisms between types, from Chapter 11):

```
/-- **The Yoneda lemma** (for lawful functors on types). -/
def yonedaEquiv : Yoneda F α ≃ F α where
  toFun := Yoneda.lower
  invFun := Yoneda.lift
  left_inv := Yoneda.lift_lower
  right_inv := Yoneda.lower_lift
```

A few uses. A list lifted into the Yoneda form is "whatever you map over me"; the CPS isomorphism is the case of the identity functor; and with the functor $\gamma \to (\cdot)$, often called `Reader`, the lemma says that natural families $\forall \beta.\, (\alpha \to \beta) \to (\gamma \to \beta)$ are the same thing as plain functions $\gamma \to \alpha$:

```
/-- A list, seen as "whatever you map over me". -/
def digits : Yoneda List ℕ := Yoneda.lift [3, 1, 2]

/-- info: ["3", "1", "2"] -/
#guard_msgs in
#eval digits.run String toString

/-- info: [3, 1, 2] -/
#guard_msgs in
#eval yonedaEquiv digits

/-- CPS is the case `F = Ident`: natural CPS values are plain values. -/
def cpsEquiv : Yoneda Ident α ≃ α := yonedaEquiv
```

```
/-- The functor `γ → ·` (often called `Reader γ`): `map` is post-composition. -/
def Reader (γ β : Type) : Type := γ → β

instance (γ : Type) : TypeFunctor (Reader γ) where
  map g r := fun c => g (r c)
  map_id _ := rfl
  map_comp _ _ _ := rfl

/-- The Yoneda embedding, for types: natural transformations from `α → ·` to
`γ → ·` are the same as plain functions `γ → α`. -/
def yonedaEmbedding (γ : Type) : Yoneda (Reader γ) α ≃ (γ → α) := yonedaEquiv
```

The last statement is called the **Yoneda embedding**, and it carries a philosophy: a type (more generally, an object of any category) is completely determined, up to isomorphism, by the functions out of it, that is, by how it relates to everything else. If two types α and γ have naturally isomorphic hom-functors, then $\alpha \cong \gamma$.

In Rust

The Rust version uses a trait with a generic method for the rank-2 type $\forall \beta.\, (\alpha \to \beta) \to \texttt{Vec}\langle\beta\rangle$:

```
/// `∀B. (A → B) → Vec<B>`: "whatever you map over me".
trait YonedaVec<A> {
    fn run<B>(self, k: impl FnMut(A) -> B) -> Vec<B>;
}

/// Lifting a vector: map the continuation over it.
struct Lift<A>(Vec<A>);

impl<A> YonedaVec<A> for Lift<A> {
    fn run<B>(self, k: impl FnMut(A) -> B) -> Vec<B> {
        self.0.into_iter().map(k).collect()
    }
}

/// Lowering: run with the identity continuation.
fn lower<A>(y: impl YonedaVec<A>) -> Vec<A> {
    y.run(|a| a)
}

/// An implementation that never stores a `Vec<u32>`: it generates the
/// numbers n, n-1, ..., 1 on demand and feeds them to the continuation.
struct Countdown(u32);

impl YonedaVec<u32> for Countdown {
    fn run<B>(self, k: impl FnMut(u32) -> B) -> Vec<B> {
        (1..=self.0).rev().map(k).collect()
    }
}
```

`Lift` is the lifting map of the lemma; `lower` runs with the identity continuation. `Countdown` is an implementation that never stores a vector at all: it generates its elements when a continuation arrives. Yoneda says it is nevertheless equivalent to the vector `[3, 2, 1]`, and the round trip agrees:

```
// Round trip 1: lower(lift(v)) == v.
let v = vec![3, 1, 2];
assert_eq!(lower(Lift(v.clone())), v);
println!("lower(Lift([3, 1, 2])) = {:?}", lower(Lift(v)));

// Round trip 2: lift(lower(y)) behaves like y, for every continuation.
let direct = Countdown(3).run(show);
let round_trip = Lift(lower(Countdown(3))).run(show);
assert_eq!(direct, round_trip);
println!("Countdown(3).run(show)        = {direct:?}");
println!("Lift(lower(Countdown(3))).run = {round_trip:?}");

// A non-parametric implementation breaks round trip 2.
let direct = Picky(vec![3, 1, 2]).run(show);
let round_trip = Lift(lower(Picky(vec![3, 1, 2]))).run(show);
assert_ne!(direct, round_trip);
println!("Picky([3, 1, 2]).run(show)        = {direct:?}");
println!("Lift(lower(Picky([3, 1, 2]))).run = {round_trip:?}");
```

```
lower(Lift([3, 1, 2])) = [3, 1, 2]
Countdown(3).run(show)        = ["<3>", "<2>", "<1>"]
Lift(lower(Countdown(3))).run = ["<3>", "<2>", "<1>"]
Picky([3, 1, 2]).run(show)        = []
Lift(lower(Picky([3, 1, 2]))).run = ["<3>", "<1>", "<2>"]
```

The last two lines are the interesting ones. `Picky` is an implementation that is not natural:

```
/// Not parametric in `B`: it returns nothing when `B` is larger than 4 bytes.
/// The trait's signature cannot prevent this, because `size_of` needs no bound.
struct Picky(Vec<u32>);

impl YonedaVec<u32> for Picky {
```

```
    fn run<B>(self, k: impl FnMut(u32) -> B) -> Vec<B> {
        if size_of::<B>() > 4 {
            Vec::new()
        } else {
            self.0.into_iter().map(k).collect()
        }
    }
}
```

The trait promises "for every `B`", and the implementation keeps the promise as far as the type checker can tell, but it consults `size_of::<B>()` and returns nothing when `B` is larger than four bytes. Lowering uses `B = u32` (four bytes) and gets the full vector; a continuation producing `String`s (24 bytes) gets nothing. The round trip breaks exactly where Lean's version needed the `natural` field. (Since `run<B>` has no `'static` bound, an implementation cannot call `TypeId::of::<B>()`: only the bound-free leaks are available.)

> **TIP**
>
> The Yoneda form of a functor is also a well-known optimization. Mapping over a lifted value only composes continuations, and the real `map` happens once, when you lower. Rust's iterator adapters are lazy in a similar spirit: `v.iter().map(f).map(g)` stores both functions and traverses the vector once, when the iterator is consumed.

18.10 In Mathlib

Mathlib's category theory library has natural transformations as the structure `CategoryTheory.NatTrans`, with the same two fields as ours. The file `code/lean/Book/Ch18/InMathlib.lean` shows its naturality field:

```
/--
info: @NatTrans.naturality : ∀ {C : Type u_3} [inst : Category.{u_1, u_3} C] {D : Type u_4}
[inst_1 : Category.{u_2, u_4} D]
  {F G : C ⥤ D} (self : NatTrans F G) {X Y : C} (f : X ⟶ Y), F.map f ≫ self.app Y =
self.app X ≫ G.map f
-/
#guard_msgs in
#check @NatTrans.naturality
```

Composition is written `≫` in *diagrammatic* order (`f ≫ g` is "first `f`, then `g`", Chapter 16), so the square reads left to right as "map, then transform equals transform, then map". The universe annotations make it work for categories of any size, and when the proof is routine the field is filled in automatically (by the `cat_disch` tactic). Mathlib also builds a functor `Type ⥤ Type` from any lawful Lean functor (`ofTypeFunctor`), so `List.head?` can be stated as a morphism in the functor category:

```
/-- `List.head?` as a morphism in Mathlib's functor category `Type ⥤ Type`.
`ofTypeFunctor` turns a lawful Lean functor into a Mathlib functor, and `↾f`
turns a function into a morphism of Mathlib's category of types. -/
def headNT : ofTypeFunctor List ⟶ ofTypeFunctor Option where
  app α := ↾(List.head? : List α → Option α)
  naturality := by
    intro α β f
    ext xs
    cases xs <;> rfl
```

```
/-- info: some 3 -/
#guard_msgs in
#eval headNT.app ℕ [3, 1, 2]
```

In this version of Mathlib, morphisms in the category of types are not bare functions but a thin wrapper around them; `↾f` wraps a function as a morphism, and the `ext` tactic reduces an equation between morphisms to an equation between their values at an arbitrary point `xs`. After that, the proof is our `cases xs <;> rfl`. Finally, the Yoneda lemma:

```
/--
info: @yonedaEquiv : {C : Type u_2} →
  [inst : Category.{u_1, u_2} C] → {X : C} → {F : Cᵒᵖ ⥤ Type u_1} → (yoneda.obj X ⟶ F) ≃
F.obj (Opposite.op X)
-/
#guard_msgs in
#check @yonedaEquiv

/--
info: @coyonedaEquiv : {C : Type u_2} →
  [inst : Category.{u_1, u_2} C] → {X : C} → {F : C ⥤ Type u_1} → (coyoneda.obj (Opposite.op
X) ⟶ F) ≃ F.obj X
-/
#guard_msgs in
#check @coyonedaEquiv
```

`yonedaEquiv` is the version for contravariant functors $\mathcal{C}^{op} \to \mathbf{Type}$ (natural transformations out of $\mathrm{Hom}(-, X)$), and `coyonedaEquiv` is the covariant one we proved: natural transformations from $\mathrm{Hom}(X, -)$ to F correspond to elements of $F\,X$. Both work for every category, not just types. The opposite category $\mathcal{C}^{op}$ is the one from Chapter 16, and `Opposite.op X` is `X` seen as an object of it.

18.11 Putting it together: moving work past a transformation

Back to the refactoring that opened the chapter, now in a realistic setting and with a payoff. A log viewer keeps the last few entries of an event log and shows them formatted. The program is `code/rust/examples/ch18_log_tail.rs`:

```
#[derive(Clone, Debug, PartialEq)]
enum Level {
    Info,
    Warn,
    Error,
}

impl Level {
    fn label(&self) -> &'static str {
        match self {
            Level::Info => "INFO",
            Level::Warn => "WARN",
            Level::Error => "ERROR",
        }
    }
}

#[derive(Clone, Debug, PartialEq)]
struct Event {
    seq: u32,
    level: Level,
    message: String,
```

```
}

/// A synthetic log: every seventh event is a warning, every 50th an error.
fn sample_log(len: u32) -> Vec<Event> {
    (1..=len)
        .map(|seq| Event {
            seq,
            level: if seq % 50 == 0 {
                Level::Error
            } else if seq % 7 == 0 {
                Level::Warn
            } else {
                Level::Info
            },
            message: format!("request {seq} handled"),
        })
        .collect()
}
```

The function that keeps the tail of the log is generic:

```
/// The last `n` entries. The signature says it all: for every `T`, a
/// `Vec<T>` in, a `Vec<T>` out, and no bound on `T`. It can drop, keep and
/// reorder entries, but it cannot look at them.
fn latest<T>(mut log: Vec<T>, n: usize) -> Vec<T> {
    let start = log.len().saturating_sub(n);
    log.split_off(start)
}
```

Its type is $\forall \alpha.\, \mathrm{Vec}\,\alpha \times \texttt{usize} \to \mathrm{Vec}\,\alpha$, with no bound on `T`. The parametricity argument for this type, with the graph of `f` as the relation for α and equality for `usize`, gives the free theorem

$$\mathrm{latest}(\mathrm{map}\, f\, v, n) = \mathrm{map}\, f\, (\mathrm{latest}(v, n))$$

for every `f`, `v` and `n`: `latest(_, n)` is a natural transformation from `Vec` to `Vec` for each `n`. So formatting every entry and then keeping three gives the same lines as keeping three entries and formatting them. The second is much cheaper, and the program measures how much, with a counter inside `render`:

```
let log = sample_log(1000);
let calls = Cell::new(0u32);
let render = |e: Event| {
    calls.set(calls.get() + 1);
    format!("#{:04} {:<5} {}", e.seq, e.level.label(), e.message)
};

// Format everything, then keep the last three lines.
calls.set(0);
let eager: Vec<String> = latest(log.clone().into_iter().map(render).collect(), 3);
let eager_calls = calls.get();

// Keep the last three events, then format them.
calls.set(0);
let lazy: Vec<String> = latest(log, 3).into_iter().map(render).collect();
let lazy_calls = calls.get();

assert_eq!(eager, lazy);
for line in &lazy {
    println!("{line}");
}
println!("render called {eager_calls} times (map, then latest)");
println!("render called {lazy_calls} times (latest, then map)");
```

```
#0998 INFO  request 998 handled
#0999 INFO  request 999 handled
#1000 ERROR request 1000 handled
render called 1000 times (map, then latest)
render called 3 times (latest, then map)
```

The `assert_eq!` passes, and the lazy pipeline formats 3 entries instead of 1000. Three observations make this more than a micro-optimization.

First, the justification does not depend on the body of `latest`. If someone reimplements it with a ring buffer, the optimization stays valid as long as the signature stays `fn<T>(Vec<T>, usize) -> Vec<T>` with no bounds (and no `size_of` or `Any` inside). If someone adds `T: Ord` to sort the log by severity, the free theorem weakens and the reordering must be re-examined. The signature is the contract.

Second, `render` has a side effect: it increments a counter. The free theorem is about values and says nothing about effects, which here differ by 997 calls. That is the point of the optimization; but if `render` wrote each line to a file, the two pipelines would no longer be interchangeable. Free theorems license reordering *pure* computations.

Third, Lean can state and prove the naturality of the same function for all inputs:

```
/-- The last `n` elements of a list (Rust's `latest` in `ch18_log_tail.rs`). -/
def latest {α : Type} (n : ℕ) (xs : List α) : List α := xs.drop (xs.length - n)

/-- `latest n` is natural, so `map f` can be moved after it. -/
def latestNat (n : ℕ) : NatTrans List List where
  app _ := latest n
  naturality f xs := by
    -- Unfold `latest` and the list functor's `map`:
    show (xs.map f).drop ((xs.map f).length - n) = (xs.drop (xs.length - n)).map f
    -- `map` does not change the length, and it commutes with `drop`.
    rw [List.length_map, List.map_drop]

/-- info: ["#8", "#9", "#10"] -/
#guard_msgs in
#eval (latestNat 3).app String ((List.range' 1 10).map fun i => s!"#{i}")
```

`latest n xs` drops all but the last `n` elements. The proof first uses `show` to restate the goal with the definitions unfolded (Lean checks that the new statement is the same as the old one up to computation): the mapped list, minus its first `(xs.map f).length - n` elements, equals the map of the dropped list. Then `List.length_map` replaces `(xs.map f).length` by `xs.length`, and `List.map_drop` (`map f (drop i l) = drop i (map f l)`) turns the right side into the left. The Rust program checked the square on one log; the Lean theorem holds for every log, every `n` and every `f`.

18.12 Common confusions

"**Every generic function is a natural transformation.**" Only a function of type $\forall \alpha.\, F\,\alpha \to G\,\alpha$ for functors F and G, and only if it is parametric: no bounds that let it inspect values, no `size_of` or `Any`, and total. `fn<T: Display>(Vec<T>) -> String` has the right shape (its target is a constant functor, like `len`), but `Display` lets its result depend on the values, so it is not natural.

"**A natural transformation is a kind of functor.**" A functor maps types to types and functions to functions. A natural transformation maps *one functor to another*: it is a family of functions, one per

type, from $F\,\alpha$ to $G\,\alpha$. In the functor category, functors are the objects and natural transformations are the morphisms.

"**Naturality is a law I have to test, like the functor laws.**" For a parametric function, naturality is guaranteed by its type, and tests can only confirm it. The functor laws are different: `map` itself is not determined by its type in a way that rules out every bug (a `map` for `Vec` could drop the last element and still have the right type, while `map id` would then not be `id`). Test the laws; trust the free theorems, unless a bound or a leak is involved.

"**The Yoneda lemma is deep magic.**" For functors on types it is two short proofs: lowering a lifted value is the identity law, and lifting a lowered family is naturality. Its depth lies in how often it applies (to every functor, in every category), not in its proof.

18.13 Summary

- A natural transformation $\eta : F \Rightarrow G$ is a family of morphisms $\eta_a : F\,a \to G\,a$ such that every naturality square commutes: $G\,f \circ \eta_a = \eta_b \circ F\,f$. For functors on types: transforming commutes with `map`.
- `first`, `Option::into_iter`, `reverse`, `len` and `Some` are natural. In Rust we checked the squares on samples; in Lean we proved them for all inputs, by case analysis or induction.
- Natural transformations compose vertically, with identities, associatively; functors $\mathcal{C} \to \mathcal{D}$ and natural transformations form the functor category. Between monotone maps on a preorder, a natural transformation is a pointwise inequality.
- Parametricity (Reynolds): every polymorphic term maps related inputs to related outputs, for every relation. Choosing graphs of functions as relations yields Wadler's free theorems: $\forall\alpha.\,\alpha \to \alpha$ is the identity, and every parametric $\forall\alpha.\,F\,\alpha \to G\,\alpha$ is natural.
- Parametricity is a meta-theorem about the terms of a language. Lean cannot prove it internally, and with `Classical.choice` it can define a non-identity `(α : Type) → α → α`; free theorems are proved per function.
- Rust leaks parametricity through trait bounds (visible; the free theorem gets a side condition), `size_of` / `type_name` (invisible), `Any`, panics and effects, and unstable specialization.
- The Yoneda lemma: natural families $\forall\beta.\,(\alpha \to \beta) \to F\,\beta$ correspond to values of $F\,\alpha$. The special case $F = \mathrm{Id}$ is continuation-passing style. Mathlib has it as `yonedaEquiv` and `coyonedaEquiv`.

18.14 Exercises

1. Write `fn last<T>(v: Vec<T>) -> Option<T>` in Rust and check its naturality square with `square` on three inputs. Then answer with the free theorem: which functions of type `fn<T>(Vec<T>) -> Option<T>` exist, and what can determine *which* element is returned? *Hint:* the graph-relation argument shows that the result is either `None` or the element at some position, and that both the choice between them and the position may depend only on the length of the vector.

2. In Lean, define `takeNat (n : ℕ) : NatTrans List List` with components `List.take n`, and prove its naturality. *Hint:* follow `latestNat`. The library lemma you need is `List.map_take`; check its exact statement with `#check @List.map_take` and use `.symm` if the orientation is reversed.
3. On paper, derive the free theorems for $\mathrm{swap} : \forall \alpha\, \beta.\, \alpha \times \beta \to \beta \times \alpha$ and for $\mathrm{filter} : \forall \alpha.\, (\alpha \to \texttt{bool}) \to \mathrm{List}\,\alpha \to \mathrm{List}\,\alpha$. *Hint:* for `swap`, use one graph relation for each type variable. For `filter`, the predicate argument is a function, so its relation is given by the function clause: p on A and q on B are related when $p = q \circ f$. The result is $\mathrm{map}\, f\, (\mathrm{filter}\, (q \circ f)\, xs) = \mathrm{filter}\, q\, (\mathrm{map}\, f\, xs)$.
4. Write a Rust function `fn<T: Any>(v: Vec<T>) -> Vec<T>` that reverses the vector when `T` is `String` and leaves it alone otherwise, and find a function `f` and an input for which its naturality square fails. *Hint:* take `f: i32 -> String`. One path applies the function at `i32`, the other at `String`.
5. In Lean, use `yonedaEquiv` to show that natural families `(β : Type) → (Bool → β) → List β` correspond to lists of booleans. Write the family that corresponds to `[true, false, true]` explicitly, and check with `#eval` that running it with the continuation `fun b => if b then 1 else 0` gives `[1, 0, 1]`. *Hint:* `Yoneda.lift [true, false, true]` is the family; its `run` field takes the type `ℕ` and the continuation.
6. Define the composite functor `abbrev Comp (F G : Type → Type) (α : Type) : Type := F (G α)` with a `TypeFunctor` instance (when `F` and `G` have one), and prove that the family `fun α => List.map (List.head? (α := α))` is a natural transformation from `Comp List List` to `Comp List Option`. This is called *whiskering* `headNat` with `List`. *Hint:* use `abbrev` rather than `def`, so that Lean unfolds `Comp` while rewriting. The instance's `map f` is `map (map f)`; for its laws, first prove `map (F := G) id = id` and `map (F := G) (g ∘ f) = map g ∘ map f` with `funext`. For naturality, `show` the goal in terms of `List.map`, rewrite both sides with `List.map_map`, and finish with `List.map_congr_left` and `headNat.naturality`.

CHAPTER 19

Cartesian Closed Categories

Chapter 10 showed that the simply typed λ-calculus and intuitionistic logic are the same system seen twice: types are propositions, terms are proofs, β-reduction is proof simplification. Chapter 16 began a third reading, in which types are objects and programs are arrows. This chapter finishes it. We ask: what must a category have so that the simply typed λ-calculus can be interpreted in it? The answer is short. It needs a terminal object (for the unit type), products (for pairs) and **exponentials** (for function types), and a category with all three is called **cartesian closed**. The correspondence between the λ-calculus and cartesian closed categories, due to Joachim Lambek, completes the Curry–Howard–Lambek triangle that this book has been drawing since Chapter 1.

Two other ideas come along. **Currying**, the bijection between functions of two arguments and functions that return functions, turns out to be the first example of an **adjunction**, the concept that organizes much of category theory (and produces monads, in Chapter 20). And the "cartesian" in "cartesian closed" is precisely what Rust gives up for values that are not `Copy`. A pair of owned values has projections but no diagonal $A \to A \times A$: you cannot use a `String` twice without cloning it. That one missing arrow is why Part V needs linear logic, whose models are a different kind of category.

In this chapter you will learn:

- terminal objects, products and exponentials as universal properties, built as Lean classes on top of Chapter 16's `Category`, with the instance for Lean's types;
- what a cartesian closed category is, and why Lean's types, and every Heyting algebra, are examples;
- currying as a natural bijection $\mathrm{Hom}(A \times B, C) \cong \mathrm{Hom}(A, C^B)$: the first adjunction;
- Lambek's correspondence: an interpretation of the simply typed λ-calculus in any cartesian closed category, with β and η proved sound in Lean (and β holding by `rfl` in `Type`);
- the Curry–Howard–Lambek triangle;
- why Rust is not cartesian for non-`Copy` types: the diagonal needs `Clone`, and `(A, B)` with moves is a *monoidal* product, the structure behind linear logic.

19.1 Products and the terminal object, again

Chapter 16 defined terminal objects and binary products by their universal properties and proved that Lean's `Unit` and `×` have them. We restate both, now as Lean classes that a category may or may not have, because cartesian closed categories are built on top of them. The code is in

`code/lean/Book/Ch19/CCC.lean`, which imports the book's `Category` class from `Book.Ch16.Category`; the notation is Chapter 16's (`a → b` for morphisms, `g ∘ f` for "first `f`, then `g`").

A **terminal object** 1 is an object with exactly one morphism into it from every object:

```
/-- A terminal object: exactly one morphism into it from every object. -/
class HasTerminal (Obj : Type u) [Category Obj] where
  one : Obj
  toOne : (a : Obj) → (a → one)
  toOne_unique : ∀ {a : Obj} (f : a → one), f = toOne a
```

The class has a field for the object, one for the morphisms into it (`toOne a`, often written $!_a$), and one for uniqueness: every morphism `a → one` *is* `toOne a`.

A **product** of a and b is an object $a \times b$ with two projections $\pi_1 : a \times b \to a$ and $\pi_2 : a \times b \to b$, such that for every object x and every pair of morphisms $f : x \to a$, $g : x \to b$ there is exactly one morphism $\langle f, g \rangle : x \to a \times b$ with

$$\pi_1 \circ \langle f, g \rangle = f \qquad \pi_2 \circ \langle f, g \rangle = g$$

In Lean, `fst` and `snd` are the projections, `pair f g` is $\langle f, g \rangle$, and the product object is written `a × b` (a special cross, so that it does not clash with Lean's `×` on types):

```
/-- Binary products, by their universal property: projections `fst` and
`snd`, and for every pair of morphisms `f : x → a`, `g : x → b` a morphism
`pair f g : x → a × b` that is the only one with `fst ∘ _ = f` and
`snd ∘ _ = g`. -/
class HasProducts (Obj : Type u) [Category Obj] where
  /-- The product object `a × b`. -/
  prod : Obj → Obj → Obj
  fst : {a b : Obj} → (prod a b → a)
  snd : {a b : Obj} → (prod a b → b)
  pair : {x a b : Obj} → (x → a) → (x → b) → (x → prod a b)
  fst_pair : ∀ {x a b : Obj} (f : x → a) (g : x → b), fst ∘ pair f g = f
  snd_pair : ∀ {x a b : Obj} (f : x → a) (g : x → b), snd ∘ pair f g = g
  pair_unique : ∀ {x a b : Obj} (h : x → prod a b), pair (fst ∘ h) (snd ∘ h) = h

@[inherit_doc] scoped infixr:35 " × " => HasProducts.prod
```

`fst_pair` and `snd_pair` are the two equations, and they say that `pair f g` *exists* with the required property. Uniqueness is stated in a compact form that is convenient in proofs: every morphism `h` into the product equals the pair of its own projections. This is equivalent to the usual statement, as the first lemma shows:

```
/-- Uniqueness, in the form used most: a morphism into a product is
determined by its two projections. -/
theorem pair_eq {x a b : Obj} {f : x → a} {g : x → b} {h : x → a × b}
    (h₁ : fst ∘ h = f) (h₂ : snd ∘ h = g) : h = pair f g := by
  rw [← pair_unique h, h₁, h₂]

/-- Precomposing a pair precomposes both components. -/
theorem pair_comp {y x a b : Obj} (f : x → a) (g : x → b) (h : y → x) :
    pair f g ∘ h = pair (f ∘ h) (g ∘ h) :=
  pair_eq (by rw [Category.assoc, fst_pair]) (by rw [Category.assoc, snd_pair])

/-- `pair fst snd` is the identity of the product. -/
theorem pair_fst_snd {a b : Obj} : pair fst snd = Category.id (a × b) := by
  have h := pair_unique (Category.id (a × b))
  rw [Category.comp_id, Category.comp_id] at h
  exact h
```

`pair_eq` says: if `h` has the right projections `f` and `g`, then `h` is `pair f g`. The proof rewrites `h` into `pair (fst ≫ h) (snd ≫ h)` (reading `pair_unique h` backwards, with `←`) and then replaces the projections using the hypotheses. `pair_comp` says that precomposing a pair precomposes both components, $\langle f, g\rangle \circ h = \langle f \circ h, g \circ h\rangle$; it is `pair_eq` applied to the right side, and the two projection equations are proved by reassociating (`Category.assoc`) and using `fst_pair` or `snd_pair`. `pair_fst_snd` says that pairing the two projections gives the identity, which is `pair_unique` at the identity morphism, cleaned up by `Category.comp_id` ($f \circ \mathrm{id} = f$). The `rw ... at h` form rewrites in the hypothesis `h` instead of the goal.

Two more constructions will be used all the time. The **product of morphisms** $f \times g : a \times b \to c \times d$ applies f to the first component and g to the second; it is $\langle f \circ \pi_1, g \circ \pi_2\rangle$:

```
/-- The product of two morphisms, `f × g`, acting on each component. -/
def prodMap {a b c d : Obj} (f : a ⟶ c) (g : b ⟶ d) : a × b ⟶ c × d :=
  pair (f ≫ fst) (g ≫ snd)

theorem prodMap_comp {a b c d e k : Obj} (f : c ⟶ e) (g : d ⟶ k) (h : a ⟶ c)
    (m : b ⟶ d) : prodMap f g ≫ prodMap h m = prodMap (f ≫ h) (g ≫ m) := by
  unfold prodMap
  rw [pair_comp, ← Category.assoc, fst_pair, ← Category.assoc, snd_pair,
    Category.assoc, Category.assoc]

theorem prodMap_comp_id {a b c d : Obj} (g : c ⟶ d) (h : a ⟶ c) :
    prodMap (g ≫ h) (Category.id b) = prodMap g (Category.id b) ≫ prodMap h (Category.id
b) := by
  rw [prodMap_comp, Category.id_comp]

/-- A pair splits into "keep the second component, then apply `h` to the
first". -/
theorem pair_eq_prodMap {x a b : Obj} (h : x ⟶ a) (g : x ⟶ b) :
    pair h g = prodMap h (Category.id b) ≫ pair (Category.id x) g := by
  unfold prodMap
  rw [pair_comp, ← Category.assoc, fst_pair, Category.comp_id, ← Category.assoc,
    snd_pair, Category.id_comp]
```

`prodMap_comp` says that $(f \times g) \circ (h \times m) = (f \circ h) \times (g \circ m)$ (`prodMap_comp_id` is its special case with identities on the second component), and `pair_eq_prodMap` splits a pair $\langle h, g\rangle$ into "make the pair $\langle \mathrm{id}, g\rangle$, then apply h to the first component". Both proofs are chains of the same moves: distribute a composite over a pair with `pair_comp`, reassociate, and simplify a projection of a pair.

The second construction is the **diagonal**. In a category with products, every object can be copied:

```
/-- The diagonal `a ⟶ a × a`: every object of a category with products can be
copied. -/
def diag (a : Obj) : a ⟶ a × a := pair (Category.id a) (Category.id a)

/-- The diagonal is natural: copying and then applying `f` to both copies is
the same as applying `f` and then copying. -/
theorem diag_natural {a b : Obj} (f : a ⟶ b) : prodMap f f ≫ diag a = diag b ≫ f := by
  unfold diag prodMap
  rw [pair_comp, pair_comp, ← Category.assoc, fst_pair, ← Category.assoc, snd_pair,
    Category.comp_id, Category.id_comp]

/-- Copying and then forgetting one copy does nothing. -/
theorem fst_diag (a : Obj) : fst ≫ diag a = Category.id a := fst_pair _ _
```

$\Delta_a = \langle \mathrm{id}_a, \mathrm{id}_a\rangle$, and `diag_natural` says that it is a natural transformation (Chapter 18) from the identity functor to the functor $a \mapsto a \times a$: copying and then applying f to both copies is the same as

applying f and copying. Keep the diagonal in mind. In logic it is contraction (Chapter 6), and it is exactly what Rust will refuse at the end of the chapter.

In Rust, the projections and the pairing look like this (the full program is `code/rust/examples/ch19_ccc.rs`):

```
/// The two projections out of a product. `fst` drops the second component:
/// discarding is allowed in Rust (it runs `Drop`, if there is one).
fn fst<A, B>((a, _): (A, B)) -> A {
    a
}

fn snd<A, B>((_, b): (A, B)) -> B {
    b
}

/// Pairing ⟨f, g⟩ : X → A × B. Feeding one `x` to two functions needs two
/// copies of `x`, so the input type must be `Clone`.
fn pair<X: Clone, A, B>(
    f: impl Fn(X) -> A,
    g: impl Fn(X) -> B,
) -> impl Fn(X) -> (A, B) {
    move |x: X| (f(x.clone()), g(x))
}

/// The unique map into the terminal object `()`.
fn to_unit<A>(_: A) {}
```

```
let price_of = |item: u32| item * 100;
let label_of = |item: u32| format!("item-{item}");
let both = pair(price_of, label_of);
let p = both(7);
// The universal property: projecting the pair gives back f and g.
assert_eq!(fst(p.clone()), price_of(7));
assert_eq!(snd(p.clone()), label_of(7));
println!("pair(price_of, label_of)(7) = {p:?}");
to_unit(p);
```

```
pair(price_of, label_of)(7) = (700, "item-7")
```

Look at the signature of `pair`: it requires `X: Clone`. To feed one input to two functions you need two copies of it. For `u32` that costs nothing, but the bound is there, and we will come back to it.

19.2 Exponentials

Functions as objects

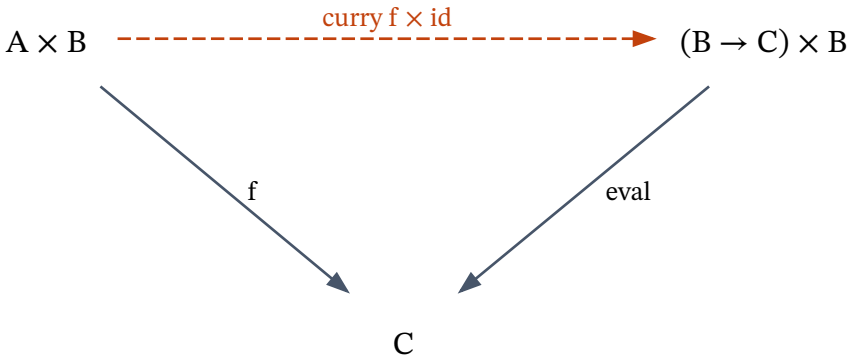
In a category, the morphisms from b to c form a collection $\mathrm{Hom}(b, c)$ that lives *outside* the category: it is a set (or, in Lean, a type), not an object. For Lean's types, however, the functions from `β` to `γ` are themselves the values of a type, `β → γ`. A function type is an object that *internalizes* a hom-set. To say what it takes for an object to play this role, we need to say what can be done with a function value: it can be *applied*, and it can be *created* from an expression with a free variable. The categorical versions are evaluation and currying.

An **exponential** of b and c is an object $b \Rightarrow c$ (also written c^b) together with an **evaluation** morphism

$$\mathrm{eval} : (b \Rightarrow c) \times b \to c$$

such that for every object a and every morphism $f : a \times b \to c$, there is exactly one morphism $\mathrm{curry}\, f : a \to (b \Rightarrow c)$ with

$$\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}_b) = f$$



The universal property of the exponential, drawn for types, where the exponential object $B \Longrightarrow C$ is the function type $B \to C$. Every $f : A \times B \to C$ factors through eval, by exactly one $\mathrm{curry}\, f : A \to (B \Longrightarrow C)$.

Read it with functions in mind. A morphism $f : a \times b \to c$ is a function of two arguments. $\mathrm{curry}\, f$ takes only the first argument and returns a function value waiting for the second. The equation says that evaluating that function value at the second argument gives what f would have given; uniqueness says that $\mathrm{curry}\, f$ is the only morphism with this property. A **cartesian closed category** (CCC) is a category with a terminal object, binary products, and an exponential for every pair of objects.

In Lean

The class mirrors the definition, with the exponential written `b ⟹ c` (the notation Mathlib uses too):

```
/-- A cartesian closed category: a terminal object, binary products, and for
all objects `b`, `c` an exponential `b ⟹ c` (the object of morphisms from `b`
to `c`) with an evaluation morphism, such that every `f : a × b → c` factors
through `eval` by exactly one `curry f : a → (b ⟹ c)`. -/
class CartesianClosed (Obj : Type u) [Category Obj] [HasTerminal Obj]
    [HasProducts Obj] where
  /-- The exponential object `b ⟹ c`. -/
  exp : Obj → Obj → Obj
  eval : {b c : Obj} → (exp b c × b → c)
  curry : {a b c : Obj} → (a × b → c) → (a → exp b c)
  eval_curry : ∀ {a b c : Obj} (f : a × b → c),
    eval ∘ prodMap (curry f) (Category.id b) = f
  curry_unique : ∀ {a b c : Obj} (g : a → exp b c),
    curry (eval ∘ prodMap g (Category.id b)) = g

@[inherit_doc] scoped infixr:30 " ⟹ " => CartesianClosed.exp
```

The class takes the terminal object and the products as instance arguments (a CCC is a category with those *and* exponentials) and adds four fields. `eval_curry` is the factorization equation. Uniqueness is again stated in compact form: `curry_unique` says that every `g : a → (b ⟹ c)` is the curried form of `eval ∘ prodMap g id`, the two-argument morphism it determines. This is the categorical form of **η**: a function is equal to the function that applies it. In the same way, `eval_curry` is the categorical form of **β**: applying a function built from f computes f.

The inverse of `curry` is `uncurry`, and the two laws make them a bijection of hom-sets, packaged as an `Equiv` (Chapter 11):

```
/-- `uncurry g`: evaluate after applying `g` to the first component. -/
def uncurry {a b c : Obj} (g : a ⟶ (b ⟹ c)) : a ⨯ b ⟶ c :=
  eval ∘ prodMap g (Category.id b)

theorem uncurry_curry {a b c : Obj} (f : a ⨯ b ⟶ c) : uncurry (curry f) = f :=
  eval_curry f

theorem curry_uncurry {a b c : Obj} (g : a ⟶ (b ⟹ c)) : curry (uncurry g) = g :=
  curry_unique g

/-- Currying is a bijection between the two hom-sets. -/
def curryEquiv (a b c : Obj) : (a ⨯ b ⟶ c) ≃ (a ⟶ (b ⟹ c)) where
  toFun := curry
  invFun := uncurry
  left_inv := uncurry_curry
  right_inv := curry_uncurry
```

Lean's types form a CCC

The instance for `Type` uses `Unit`, `×` and `→`:

```
instance : HasTerminal Type where
  one := Unit
  toOne _ := fun _ => ()
  toOne_unique _ := rfl

instance : HasProducts Type where
  prod a b := a × b
  fst := fun p => p.1
  snd := fun p => p.2
  pair f g := fun x => (f x, g x)
  fst_pair _ _ := rfl
  snd_pair _ _ := rfl
  pair_unique _ := rfl

instance : CartesianClosed Type where
  exp b c := b → c
  eval := fun p => p.1 p.2
  curry f := fun a b => f (a, b)
  eval_curry _ := rfl
  curry_unique _ := rfl
```

Every law is `rfl`, and it is worth seeing why. `fst_pair` holds because `(fun x => (f x, g x))` followed by `.1` computes to `f`. `pair_unique` needs `(fun x => ((h x).1, (h x).2)) = h`, which holds because Lean's definitional equality includes **η for structures** (a pair is equal to the pair of its components) and **η for functions** (`fun x => h x` is `h`). `toOne_unique` holds because every value of `Unit` is definitionally `()`. And `curry_unique` is `(fun a b => g a b) = g`, η for functions twice. In Lean's type theory, the CCC equations are part of computation:

```
/-- In `Type`, `curry` is ordinary currying. -/
example : (curry (fun p : ℕ × ℕ => p.1 + p.2) : ℕ → (ℕ ⟹ ℕ)) 2 3 = 5 := rfl

/-- And the diagonal is `fun x => (x, x)`. -/
example : (diag ℕ : ℕ → ℕ × ℕ) 7 = (7, 7) := rfl
```

In Rust

Rust has a wrinkle: there is no single type of "functions from `B` to `C`". Every closure has its own anonymous type (Chapter 8), and `impl Fn(B) -> C` is a different type at every use. The closest thing to an exponential object is the existential `Box<dyn Fn(B) -> C>` (Chapter 15), which hides the closure's type behind a vtable:

```
/// The exponential object C^B: a function from B to C, as a value.
type Exp<B, C> = Box<dyn Fn(B) -> C>;

/// eval : C^B × B → C
fn eval<B, C>((g, b): (Exp<B, C>, B)) -> C {
    g(b)
}

/// curry : Hom(A × B, C) → Hom(A, C^B). The returned function may be called
/// many times, and every closure it returns needs its own `a` (hence
/// `A: Clone`) and access to `f` (shared through an `Rc`).
fn curry<A, B, C>(f: impl Fn((A, B)) -> C + 'static) -> impl Fn(A) -> Exp<B, C>
where
    A: Clone + 'static,
    B: 'static,
    C: 'static,
{
    let f = Rc::new(f);
    move |a: A| {
        let f = Rc::clone(&f);
        Box::new(move |b: B| f((a.clone(), b)))
    }
}

/// uncurry : Hom(A, C^B) → Hom(A × B, C), that is, eval ∘ (g × id).
fn uncurry<A, B, C>(g: impl Fn(A) -> Exp<B, C>) -> impl Fn((A, B)) -> C {
    move |(a, b)| eval((g(a), b))
}
```

`eval` is application. `curry` returns a closure that, given an `a`, builds and boxes a closure waiting for `b`. Notice the costs. The boxed closure is an `Fn`, so it may be called many times, and each call must hand `f` an owned pair `(a, b)`: it can only do so by cloning the `a` it captured, hence `A: Clone`. And all the boxed closures need the same `f`, which they share through an `Rc` (a reference-counted pointer). The `'static` bounds say that the boxed closures may not capture short-lived references (Chapter 26). The laws hold on samples:

```
// f : (unit price, quantity) -> total
let total = |(price, qty): (u32, u32)| price * qty;
let samples = [(250, 4), (99, 0), (1200, 3)];

// β: uncurry(curry(f)) = f
let round_trip = uncurry(curry(total));
for s in samples {
    assert_eq!(round_trip(s), total(s));
}
// η: curry(uncurry(g)) = g, for a g given in curried form
let discount =
    |pct: u32| -> Exp<u32, u32> { Box::new(move |c| c * (100 - pct) / 100) };
let again = curry(uncurry(discount));
for (pct, cents) in [(10, 2000), (0, 1999), (50, 3)] {
    assert_eq!(again(pct)(cents), discount(pct)(cents));
}
let price_250 = curry(total)(250);
println!("curry(total)(250)(4) = {}", price_250(4));
println!(
    "eval((curry(total)(99), 3)) = {}",
```

```
    eval((curry(total)(99), 3))
);
println!("beta and eta hold on all samples");
```

```
curry(total)(250)(4) = 1000
eval((curry(total)(99), 3)) = 297
beta and eta hold on all samples
```

`uncurry(curry(total))` agrees with `total` (β), and `curry(uncurry(discount))` agrees with `discount` (η), on every sample.

19.3 Currying is an adjunction

A natural bijection

The bijection `curryEquiv` relates two hom-sets:

$$\mathrm{Hom}(a \times b,\, c) \cong \mathrm{Hom}(a,\, b \Rightarrow c)$$

Chapter 11 proved the corresponding isomorphism of *types*, `(A × B → C) ≅ (A → B → C)`, in Lean. Here it holds in any CCC, and it has one more property that makes it special: it is **natural in a**. If you precompose a two-argument morphism with $h : x \to a$ on its first argument and then curry, you get the same as currying first and then precomposing with h:

$$\mathrm{curry}\big(\, f \circ (h \times \mathrm{id}_b)\big) = \mathrm{curry}\, f \circ h$$

In programming terms: currying commutes with "preprocessing the first argument". In Lean:

```
/-- The bijection is natural in `a`: currying commutes with precomposition.
Together with `curryEquiv`, this is the statement that `(· × b)` is left
adjoint to `(b ⟹ ·)`. -/
theorem curry_natural {x a b c : Obj} (f : a × b ⟶ c) (h : x ⟶ a) :
    curry (f ∘ prodMap h (Category.id b)) = curry f ∘ h := by
  -- Write the right-hand side as `curry (uncurry (curry f ∘ h))`...
  conv_rhs => rw [← curry_unique (curry f ∘ h)]
  -- ...and compare the arguments of `curry`.
  rw [prodMap_comp_id, Category.assoc, eval_curry]

/-- The categorical β-law: evaluating a curried morphism at an argument. -/
theorem eval_pair_curry {x b c : Obj} (f : x × b ⟶ c) (g : x ⟶ b) :
    eval ∘ pair (curry f) g = f ∘ pair (Category.id x) g := by
  rw [pair_eq_prodMap, Category.assoc, eval_curry]
```

The proof of `curry_natural` uses a technique worth learning: to show that two morphisms into an exponential are equal, write both as `curry` of something and compare the somethings. `conv_rhs => rw [...]` rewrites only the right-hand side of the goal (`conv` is Lean's mode for rewriting inside a chosen part of an expression): `curry_unique` read backwards turns `curry f ∘ h` into `curry (eval ∘ prodMap (curry f ∘ h) id)`. Now both sides are `curry` of something, and the rest is the paper proof: `prodMap_comp_id` splits `prodMap (curry f ∘ h) id` into `prodMap (curry f) id ∘ prodMap h id`, `Category.assoc` regroups, and `eval_curry` turns `eval ∘ prodMap (curry f) id` into `f`. Both sides are then `curry (f ∘ prodMap h id)`.

The second theorem, `eval_pair_curry`, is the form of β that the λ-calculus will need: evaluating curry f paired with an argument g is f applied to the pair of "the current context" and g, $\mathrm{eval} \circ \langle \mathrm{curry}\, f, g\rangle = f \circ \langle \mathrm{id}, g\rangle$. The proof splits the pair with `pair_eq_prodMap` and uses `eval_curry`.

Adjunctions

A bijection between hom-sets that is natural in both variables has a name. Let $L : \mathcal{C} \to \mathcal{D}$ and $R : \mathcal{D} \to \mathcal{C}$ be two functors. An **adjunction** between them is a family of bijections

$$\mathrm{Hom}_{\mathcal{D}}(L\,a,\, c) \cong \mathrm{Hom}_{\mathcal{C}}(a,\, R\,c)$$

one for each object a of $\mathcal{C}$ and c of $\mathcal{D}$, natural in a and in c. We say that L is **left adjoint** to R, written $L \dashv R$. In a CCC, take both categories to be the CCC itself, $L = (-) \times b$ ("pair with a b") and $R = b \Rightarrow (-)$ ("functions from b"). The bijection is `curry`, naturality in a is `curry_natural`, and naturality in c (currying commutes with post-processing the result) is Exercise 2. So in a CCC, for every object b,

$$(-) \times b \dashv b \Rightarrow (-)$$

and this single sentence can serve as the definition: *a CCC is a category with finite products in which every functor $(-) \times b$ has a right adjoint*.

Adjunctions are everywhere once you look for them, and they are a recurring theme of the rest of the book. Chapter 5 already had one in disguise: the residuation law of a Heyting algebra, $a \wedge b \le c \Leftrightarrow a \le (b \Rightarrow c)$, which we will meet again in a moment. Chapter 20 will show that every adjunction produces a monad, and the adjunction of this section produces a familiar one: $b \Rightarrow ((-) \times b)$, functions from a state b to a result paired with a new state, is the state monad. And in Part V, linear logic keeps the adjunction while giving up the products, as we will see at the end of this chapter.

> **NOTE**
>
> "Natural in a" deserves a word, since Chapter 18 defined naturality for families of morphisms. Here the family is the bijection itself: one function $\mathrm{curry}_a : \mathrm{Hom}(a \times b, c) \to \mathrm{Hom}(a, b \Rightarrow c)$ for each a, and naturality says that these functions commute with the maps between hom-sets induced by any $h : x \to a$. `curry_natural` is precisely the naturality square of this family.

19.4 Heyting algebras are thin CCCs

Recall from Chapter 5 that a **Heyting algebra** is a bounded lattice (with $\top$, $\bot$, meets $\sqcap$ and joins $\sqcup$) with an implication $\Rightarrow$ (written `⇨` in Mathlib) satisfying residuation:

$$a \sqcap b \le c \quad \Leftrightarrow \quad a \le (b \Rightarrow c)$$

Heyting algebras are the algebraic semantics of intuitionistic logic, just as Boolean algebras are the semantics of classical logic. Now compare residuation with the currying bijection $\mathrm{Hom}(a \times b, c) \cong \mathrm{Hom}(a, b \Rightarrow c)$. They have the same shape. The connection is precise.

A preorder is a category in which there is at most one morphism between any two objects (Chapter 16), a **thin** category: a morphism $a \to b$ is just the fact $a \le b$. In a thin category:

- a **terminal object** is a top element: everything is $\le \top$;

- a **product** of a and b is a greatest lower bound $a \sqcap b$: the universal property says that $x \le a \sqcap b$ exactly when $x \le a$ and $x \le b$, which is the definition of a meet;
- an **exponential** $b \Rightarrow c$ is an element with $(b \Rightarrow c) \sqcap b \le c$ (evaluation is modus ponens) such that $a \sqcap b \le c$ implies $a \le b \Rightarrow c$ (currying is residuation).

All the equations between morphisms hold automatically, since any two parallel morphisms are equal. In Lean, a thin category is built from any preorder, with morphisms that are proofs of `≤` turned into data by `PLift`:

```
/-- A copy of a preordered type, so that the category structure below does not
attach itself to the type `α` everywhere. -/
structure Thin (α : Type) where
  val : α

/-- A preorder as a category: one morphism `a ⟶ b` exactly when `a ≤ b`.
Every diagram commutes, because any two morphisms with the same ends are
equal (they are proofs of the same `≤`). -/
instance thinCategory (α : Type) [Preorder α] : Category (Thin α) where
  Hom a b := PLift (a.val ≤ b.val)
  id a := ⟨le_refl a.val⟩
  comp g f := ⟨le_trans f.down g.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

`Thin α` is a one-field structure wrapping `α`, for the same reason as the `Op` wrapper of Chapter 16: so that the category structure attaches to the wrapper and not to `α` itself, which may be an ordinary type with other uses. The laws are `rfl` because two proofs of the same `≤` are equal by definition. Then every Heyting algebra (Mathlib's `HeytingAlgebra`, which Chapter 5 introduced) is a cartesian closed thin category:

```
variable (α : Type) [HeytingAlgebra α]

/-- Top is terminal: everything is `≤ ⊤`. -/
instance : HasTerminal (Thin α) where
  one := ⟨⊤⟩
  toOne _ := ⟨le_top⟩
  toOne_unique _ := rfl

/-- Meet is the product: `x ≤ a ⊓ b` exactly when `x ≤ a` and `x ≤ b`. -/
instance : HasProducts (Thin α) where
  prod a b := ⟨a.val ⊓ b.val⟩
  fst := ⟨inf_le_left⟩
  snd := ⟨inf_le_right⟩
  pair f g := ⟨le_inf f.down g.down⟩
  fst_pair _ _ := rfl
  snd_pair _ _ := rfl
  pair_unique _ := rfl

/-- Heyting implication is the exponential: `eval` is modus ponens, `curry` is
one direction of residuation. -/
instance : CartesianClosed (Thin α) where
  exp b c := ⟨b.val ⇨ c.val⟩
  eval := ⟨himp_inf_le⟩
  curry f := ⟨le_himp_iff.mpr f.down⟩
  eval_curry _ := rfl
  curry_unique _ := rfl
```

The fields are filled with Mathlib's lattice lemmas, whose names describe their statements: `le_top` ($a \le \top$), `inf_le_left` ($a \sqcap b \le a$), `inf_le_right`, `le_inf` (from $x \le a$ and $x \le b$ conclude $x \le a \sqcap$

b), `himp_inf_le` ($(a \Rightarrow b) \sqcap a \le b$) and `le_himp_iff` (residuation, as an `↔`; `.mpr` uses its right-to-left direction). Every equation is `rfl`. Conversely, the CCC structure gives residuation back, as `curry` and `uncurry` read in a category whose morphisms are proofs:

```
/-- Residuation, `a ⊓ b ≤ c ↔ a ≤ b ⇨ c`, read off from the cartesian closed
structure: it is `curry` and `uncurry` in a category where morphisms are
proofs of `≤`. -/
theorem residuation {α : Type} [HeytingAlgebra α] (a b c : α) :
    a ⊓ b ≤ c ↔ a ≤ b ⇨ c := by
  constructor
  · intro h
    -- `h` is a morphism `a × b ⟶ c` of the thin category; curry it.
    let f : (Thin.mk a × Thin.mk b : Thin α) ⟶ Thin.mk c := ⟨h⟩
    exact (curry f).down
  · intro h
    -- `h` is a morphism `a ⟶ (b ⟹ c)`; uncurry it.
    let g : (Thin.mk a : Thin α) ⟶ (Thin.mk b ⟹ Thin.mk c) := ⟨h⟩
    exact (uncurry g).down

/-- In `Prop`, the exponential is implication and `eval` is modus ponens. -/
example (p q : Prop) : (p → q) ∧ p → q :=
  (eval : (Thin.mk p ⟹ Thin.mk q) × Thin.mk p ⟶ Thin.mk q).down
```

In the proof, `let f : ... := ⟨h⟩` packages the hypothesis `h : a ⊓ b ≤ c` as a morphism of the thin category, `curry f` is a morphism `a ⟶ (b ⟹ c)`, and `.down` unwraps it into the proof `a ≤ b ⇨ c`. The last example takes α to be `Prop`, which is a Heyting algebra with $\le$ as implication, $\sqcap$ as `∧` and $\Rightarrow$ as `→`: the evaluation morphism of the thin category of propositions is modus ponens.

How much of a Heyting algebra does a thin CCC capture? Exactly the part with $\top$, $\sqcap$ and $\Rightarrow$. The joins $\sqcup$ and the bottom $\bot$ are the *coproducts* and the *initial object* of the thin category (Chapter 16's duals of products and terminal objects). A category that is cartesian closed and also has finite coproducts is called **bicartesian closed**, and a Heyting algebra is exactly a thin bicartesian closed category in which isomorphic objects are equal.

There is a lesson in the comparison. In a Heyting algebra, there is at most one morphism $a \to b$: we only know *whether* b follows from a. In the CCC of types, there are many morphisms $a \to b$, all the different programs. A thin CCC is a CCC that has forgotten which proof was used; a general CCC remembers proofs as morphisms. This is Curry–Howard one level up, and it is what makes CCCs the right semantics for *proofs*, not just for provability.

19.5 Lambek: the λ-calculus is the language of CCCs

The dictionary

Look again at the typing rules of the simply typed λ-calculus (Chapter 8) with products and unit, and at the structure of a CCC. Every rule has a categorical counterpart. A typing judgment $\Gamma \vdash t : \tau$ becomes a morphism from the interpretation of the context to the interpretation of the type:

$$[\![\Gamma \vdash t : \tau]\!] : [\![\Gamma]\!] \to [\![\tau]\!]$$

where a context $x_1 : \tau_1, \ldots, x_n : \tau_n$ is interpreted as the product $((1 \times [\![\tau_1]\!]) \times \ldots) \times [\![\tau_n]\!]$, an "environment" holding one value per variable. Then:

λ-calculus	**CCC**
type $\tau \to \sigma$, $\tau \times \sigma$, unit	exponential, product, terminal object
variable x_i	projection out of $[\![\Gamma]\!]$
$\lambda x.\, t$	curry $[\![t]\!]$
application $t\, u$	$\mathrm{eval} \circ \langle [\![t]\!], [\![u]\!] \rangle$
pair, `fst`, `snd`, `()`	$\langle -, - \rangle$, π_1, π_2, $!$
substitution $t[u/x]$	composition $[\![t]\!] \circ \langle \mathrm{id}, [\![u]\!] \rangle$
β: $(\lambda x.\, t)\, u = t[u/x]$	$\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}) = f$
η: $\lambda x.\, t\, x = t$	$\mathrm{curry}(\mathrm{eval} \circ (g \times \mathrm{id})) = g$

The correspondence goes both ways. Interpreting terms as morphisms is the direction we will formalize: it works in *every* CCC, and β and η are sound, meaning that terms equal by β and η have equal interpretations. In the other direction, the types and the terms of the λ-calculus, with terms identified up to β and η, form a CCC themselves (the *syntactic category*), and it is the **free** CCC on the base types: a CCC that satisfies no equations except those the CCC axioms force. As a consequence, two terms are βη-equal exactly when their interpretations agree in every CCC. This is Lambek's correspondence, presented in full in Lambek and Scott's *Introduction to Higher Order Categorical Logic* (1986). In slogan form: **the simply typed λ-calculus is the internal language of cartesian closed categories**.

The syntax, in Lean

The file `code/lean/Book/Ch19/Semantics.lean` imports `CCC.lean`. It redefines Chapter 8's intrinsically typed terms, with unit, products and functions over numbered base types `o₀, o₁, ...`:

```
inductive Ty where
  | base : ℕ → Ty           -- base types `o₀, o₁, ...`
  | unit : Ty
  | prod : Ty → Ty → Ty
  | arr : Ty → Ty → Ty

abbrev Ctx := List Ty

/-- A variable is a typed position in the context (a de Bruijn index). -/
inductive Var : Ctx → Ty → Type where
  | here {Γ : Ctx} {τ : Ty} : Var (τ :: Γ) τ
  | there {Γ : Ctx} {σ τ : Ty} : Var Γ τ → Var (σ :: Γ) τ

/-- Terms in context: `Term Γ τ` is the type of derivations of `Γ ⊢ t : τ`. -/
inductive Term : Ctx → Ty → Type where
  | var {Γ τ} : Var Γ τ → Term Γ τ
  | lam {Γ σ τ} : Term (σ :: Γ) τ → Term Γ (.arr σ τ)
  | app {Γ σ τ} : Term Γ (.arr σ τ) → Term Γ σ → Term Γ τ
  | unit {Γ} : Term Γ .unit
  | pair {Γ σ τ} : Term Γ σ → Term Γ τ → Term Γ (.prod σ τ)
  | fst {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ σ
  | snd {Γ σ τ} : Term Γ (.prod σ τ) → Term Γ τ
```

As in Chapter 8, `Term Γ τ` contains only well-typed terms: a value of this type *is* a typing derivation of $\Gamma \vdash t : \tau$. Variables are de Bruijn indices (Chapter 7) that carry their type: a value of `Var Γ τ` is a position in Γ together with the evidence that the type at that position is τ. Since β involves substitution, we need it too, built in the standard way for intrinsic terms: renamings map variables to variables, substitutions map variables to terms, and both must be "lifted" when they go under a binder.

```
/-- Renamings and substitutions: what to put in place of each variable. -/
abbrev Ren (Γ Δ : Ctx) := ∀ {τ}, Var Γ τ → Var Δ τ
abbrev Sub (Γ Δ : Ctx) := ∀ {τ}, Var Γ τ → Term Δ τ

/-- Push a renaming under a binder. -/
def Ren.lift {Γ Δ σ} (r : Ren Γ Δ) : Ren (σ :: Γ) (σ :: Δ)
  | _, .here => .here
  | _, .there v => .there (r v)

def Term.rename {Γ Δ} (r : Ren Γ Δ) : {τ : Ty} → Term Γ τ → Term Δ τ
  | _, .var v => .var (r v)
  | _, .lam t => .lam (t.rename r.lift)
  | _, .app f a => .app (f.rename r) (a.rename r)
  | _, .unit => .unit
  | _, .pair a b => .pair (a.rename r) (b.rename r)
  | _, .fst p => .fst (p.rename r)
  | _, .snd p => .snd (p.rename r)

/-- Push a substitution under a binder (the bound variable stays itself; the
other terms are weakened by one). -/
def Sub.lift {Γ Δ σ} (s : Sub Γ Δ) : Sub (σ :: Γ) (σ :: Δ)
  | _, .here => .var .here
  | _, .there v => (s v).rename .there

def Term.subst {Γ Δ} (s : Sub Γ Δ) : {τ : Ty} → Term Γ τ → Term Δ τ
  | _, .var v => s v
  | _, .lam t => .lam (t.subst s.lift)
  | _, .app f a => .app (f.subst s) (a.subst s)
  | _, .unit => .unit
  | _, .pair a b => .pair (a.subst s) (b.subst s)
  | _, .fst p => .fst (p.subst s)
  | _, .snd p => .snd (p.subst s)

/-- The substitution `[a/x]` for the innermost variable `x`. -/
def Sub.single {Γ σ} (a : Term Γ σ) : Sub (σ :: Γ) Γ
  | _, .here => a
  | _, .there v => .var v
```

`Sub.single a` is the substitution $[a/x]$ for the innermost variable x: it sends `here` to `a` and every other variable to itself, one position lower. So β-reduction is the step from `.app (.lam t) a` to `t.subst (Sub.single a)`.

The interpretation

Now fix a CCC `Obj` and an object `B i` for every base type, and interpret everything, following the dictionary:

```
/-- Types are objects. -/
@[reducible] def Ty.interp : Ty → Obj
  | .base i => B i
  | .unit => one
  | .prod σ τ => σ.interp × τ.interp
  | .arr σ τ => σ.interp ⟹ τ.interp

/-- Contexts are products of objects, the newest variable on the right. -/
@[reducible] def Ctx.interp : Ctx → Obj
  | [] => one
  | τ :: Γ => Ctx.interp Γ × τ.interp B

/-- A variable is a projection out of the context. -/
def Var.interp : {Γ : Ctx} → {τ : Ty} → Var Γ τ → (Ctx.interp B Γ ⟶ τ.interp B)
  | _, _, .here => snd
  | _, _, .there v => v.interp ≫ fst

/-- A term in context `Γ` of type `τ` is a morphism `⟦Γ⟧ ⟶ ⟦τ⟧`. -/
def Term.interp : {Γ : Ctx} → {τ : Ty} → Term Γ τ → (Ctx.interp B Γ ⟶ τ.interp B)
```

```
  | _, _, .var v => v.interp B
  | _, _, .lam t => curry t.interp
  | _, _, .app f a => eval ∘ HasProducts.pair f.interp a.interp
  | _, _, .unit => toOne _
  | _, _, .pair a b => HasProducts.pair a.interp b.interp
  | _, _, .fst p => HasProducts.fst ∘ p.interp
  | _, _, .snd p => HasProducts.snd ∘ p.interp
```

Read `Term.interp` clause by clause against the table. A variable is a projection: `here` is `snd`, the last component of the environment, and `there v` first drops the last component with `fst` and then projects with `v`. A λ-abstraction curries the interpretation of its body, whose context has one more variable: that is the whole meaning of "binding a variable". An application pairs the function with its argument and evaluates. The unit is the unique morphism to the terminal object, and pairs and projections are themselves. Notice that the interpretation uses `HasProducts.pair`, `HasProducts.fst` and `HasProducts.snd` with their full names: inside `Term.interp`, Lean opens the namespace `Term`, where `pair`, `fst` and `snd` are the *syntactic* constructors. The difference between the two is the difference between syntax and semantics.

`Ty.interp` and `Ctx.interp` are marked `@[reducible]`, which tells Lean to unfold them eagerly. Without it, Lean's rewriting tactics would not see that the interpretation of the context `σ :: Γ` *is* the product `⟦Γ⟧ × ⟦σ⟧`, and many `rw` steps below would fail to find their patterns.

β and η are sound in every CCC

Substitution must be interpreted too. A renaming or substitution from Γ to Δ tells how to build each variable of Γ from the variables of Δ, so its meaning is a morphism from $⟦Δ⟧$ to $⟦Γ⟧$, built as a tuple:

```
/-- A renaming `Γ → Δ` is a morphism `⟦Δ⟧ → ⟦Γ⟧` (note the direction: it
builds the old environment from the new one). -/
def Ren.interp {Δ : Ctx} : (Γ : Ctx) → Ren Γ Δ → (Ctx.interp B Δ → Ctx.interp B Γ)
  | [], _ => toOne _
  | _ :: Γ, r => pair (Ren.interp Γ fun v => r (.there v)) ((r .here).interp B)

/-- The same for substitutions. -/
def Sub.interp {Δ : Ctx} : (Γ : Ctx) → Sub Γ Δ → (Ctx.interp B Δ → Ctx.interp B Γ)
  | [], _ => toOne _
  | _ :: Γ, s => pair (Sub.interp Γ fun v => s (.there v)) ((s .here).interp B)
```

The central result is the **substitution lemma**: substituting in the syntax is composing in the semantics.

$$⟦t[s]⟧ = ⟦t⟧ \circ ⟦s⟧$$

Its proof needs a handful of lemmas about renamings, all in the file: looking up a variable in a renaming or substitution is projecting from its interpretation (`Ren.interp_var`, `Sub.interp_var`); weakening by one variable composes with `fst` (`Ren.interp_there`, `Sub.interp_weaken`); the identity renaming means the identity morphism (`Ren.interp_id`, `Sub.interp_vars`); and lifting under a binder means $⟦s⟧ \times \mathrm{id}$ (`Ren.interp_lift`, `Sub.interp_lift`). Each is a short induction on the context or on the variable, using `pair_comp`, `pair_fst_snd` and associativity. With these, the substitution lemma is an induction on the term, and the λ case is where the adjunction earns its keep:

```
/-- **The substitution lemma**: substituting is composing. -/
theorem Term.interp_subst {Γ Δ τ} (s : Sub Γ Δ) (t : Term Γ τ) :
    (t.subst s).interp B = t.interp B ∘ Sub.interp B Γ s := by
```

```
  induction t generalizing Δ with
  | var v =>
    -- A variable: look it up in the substitution.
    exact Sub.interp_var B s v
  | lam t ih =>
    -- Under a binder the substitution is lifted, which means `⟦s⟧ × id`,
    -- and `curry` absorbs it by naturality.
    rw [Term.subst, Term.interp, ih, Sub.interp_lift, curry_natural, Term.interp]
  | app f a ihf iha =>
    -- `pair (⟦f⟧ ⊚ ⟦s⟧) (⟦a⟧ ⊚ ⟦s⟧) = pair ⟦f⟧ ⟦a⟧ ⊚ ⟦s⟧`, then associativity.
    rw [Term.subst, Term.interp, ihf, iha, Term.interp, ← pair_comp, Category.assoc]
  | unit =>
    -- Two morphisms into the terminal object are equal.
    exact (toOne_unique _).trans (toOne_unique _).symm
  | pair a b iha ihb => rw [Term.subst, Term.interp, iha, ihb, Term.interp, pair_comp]
  | fst p ih => rw [Term.subst, Term.interp, ih, Term.interp, Category.assoc]
  | snd p ih => rw [Term.subst, Term.interp, ih, Term.interp, Category.assoc]
```

`induction t generalizing Δ` makes the induction hypothesis hold for every target context, which the λ case needs because a lifted substitution goes from `σ :: Γ` to `σ :: Δ`. In that case, the chain of rewrites is: unfold `subst` and `interp` for `lam` on the left (`Term.subst`, `Term.interp`), use the induction hypothesis on the body (`ih`), replace the meaning of the lifted substitution by `prodMap ⟦s⟧ id` (`Sub.interp_lift`), and apply `curry_natural`, which turns `curry (⟦t⟧ ⊚ prodMap ⟦s⟧ id)` into `curry ⟦t⟧ ⊚ ⟦s⟧`; the final `Term.interp` folds the right side. The application case distributes over the pair (`← pair_comp`) and reassociates. The other cases are similar or immediate.

And now the two theorems that justify the whole correspondence:

```
/-- **β is sound in every CCC**: `(λx. t) a` and `t[a/x]` have the same
meaning. -/
theorem beta {Γ σ τ} (t : Term (σ :: Γ) τ) (a : Term Γ σ) :
    (Term.app (.lam t) a).interp B = (t.subst (Sub.single a)).interp B := by
  -- Right side: ⟦t⟧ ⊚ ⟦[a/x]⟧ (substitution lemma).
  -- Left side:  eval ⊚ pair (curry ⟦t⟧) ⟦a⟧ = ⟦t⟧ ⊚ pair id ⟦a⟧ (categorical β).
  rw [Term.interp_subst, Term.interp, Term.interp, eval_pair_curry]
  -- Both sides are now `⟦t⟧ ⊚ _`; compare what follows `⟦t⟧`.
  congr 1
  -- The meaning of `[a/x]` is `pair ⟦vars⟧ ⟦a⟧`, and `⟦vars⟧ = id`.
  show _ = pair (Sub.interp B Γ fun v => Term.var v) (a.interp B)
  rw [Sub.interp_vars]

/-- **η is sound in every CCC**: `λx. f x` (with `f` weakened past `x`) and
`f` have the same meaning. -/
theorem eta {Γ σ τ} (f : Term Γ (.arr σ τ)) :
    (Term.lam (.app (f.rename .there) (.var .here))).interp B = f.interp B := by
  -- Unfold, and use that weakening `f` means precomposing with `fst`.
  rw [Term.interp, Term.interp, Term.interp_rename, Ren.interp_there B Γ (fun v => v),
    Ren.interp_id, Category.id_comp]
  -- Goal: curry (eval ⊚ pair (⟦f⟧ ⊚ fst) snd) = ⟦f⟧
  show curry (eval ⊚ pair (f.interp B ⊚ fst) snd) = _
  -- `pair (⟦f⟧ ⊚ fst) (id ⊚ snd)` is `prodMap ⟦f⟧ id`: this is `curry_unique`.
  rw [← Category.id_comp (snd (a := Ctx.interp B Γ) (b := σ.interp B))]
  exact curry_unique (f.interp B)
```

For **β**, the right side is rewritten by the substitution lemma into `⟦t⟧ ⊚ ⟦Sub.single a⟧`, and the left side, `eval ⊚ pair (curry ⟦t⟧) ⟦a⟧`, by the categorical β-law `eval_pair_curry` into `⟦t⟧ ⊚ pair id ⟦a⟧`. `congr 1` reduces the goal to the parts after `⟦t⟧`, and what remains is that the meaning of $[a/x]$ is $\langle \mathrm{id}, [\![a]\!] \rangle$: `show` exposes it as `pair ⟦vars⟧ ⟦a⟧`, and `Sub.interp_vars` says that the substitution sending every variable to itself means the identity.

For **η**, the term `λx. f x` must weaken `f` past the new variable `x` (`f.rename .there`), and the weakening means precomposition with `fst` (`Ren.interp_there` with the identity renaming, then `Ren.interp_id` and `Category.id_comp`). The goal becomes `curry (eval ∘ pair (⟦f⟧ ∘ fst) snd) = ⟦f⟧`. Since `pair (⟦f⟧ ∘ fst) (id ∘ snd)` is by definition `prodMap ⟦f⟧ id`, inserting an `id ∘` in front of `snd` (the `rw [← Category.id_comp ...]`) turns the goal into exactly `curry_unique ⟦f⟧`.

So β and η, the two equations of the λ-calculus, are consequences of the two halves of the universal property of exponentials: existence gives β, uniqueness gives η. This is the soundness half of Lambek's correspondence, proved in Lean for every CCC at once.

19.6 Putting it together: one term, three meanings

The interpretation works in any CCC, so we can apply it to the two CCCs of this chapter. Here are three closed terms: swapping a pair, the diagonal (which uses its variable twice) and currying.

```
/-- `λp. (snd p, fst p) : o₀ × o₁ → o₁ × o₀` -/
def swap : Term [] (.arr (.prod (.base 0) (.base 1)) (.prod (.base 1) (.base 0))) :=
  .lam (.pair (.snd (.var .here)) (.fst (.var .here)))

/-- `λx. (x, x) : o₀ → o₀ × o₀`: the diagonal, which uses `x` twice. -/
def dup : Term [] (.arr (.base 0) (.prod (.base 0) (.base 0))) :=
  .lam (.pair (.var .here) (.var .here))

/-- `λf. λa. λb. f (a, b)`: currying, as a closed term. -/
def curryTerm (a b c : Ty) : Term [] (.arr (.arr (.prod a b) c) (.arr a (.arr b c))) :=
  .lam (.lam (.lam (.app (.var (.there (.there .here)))
    (.pair (.var (.there .here)) (.var .here)))))
```

In `Type`, with `ℕ` and `String` as base types, the terms mean what a programmer expects: `swap` swaps, `dup` copies, `curryTerm` curries. And β holds by `rfl`: the meaning of a redex *is* the meaning of its body in an environment extended with the argument's value, with no lemma needed, because Lean's own evaluation performs the step.

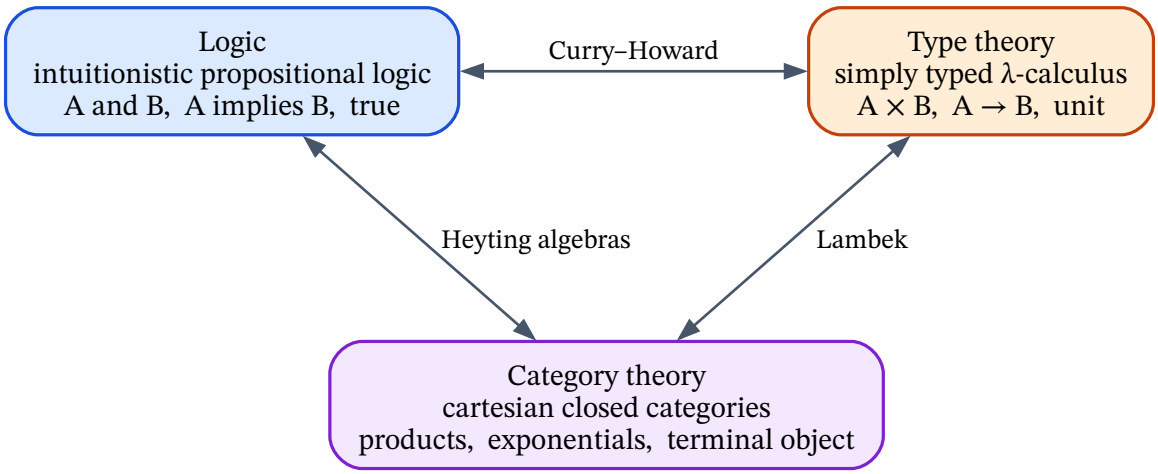
```
/-- In `Type`, with `o₀ = ℕ` and `o₁ = String`, `swap` means swapping. -/
def typeBase : ℕ → Type
  | 0 => ℕ
  | _ => String

example : (swap.interp typeBase) () ((7 : ℕ), "seven") = ("seven", (7 : ℕ)) := rfl

/-- `dup` means copying... -/
example : (dup.interp typeBase) () (3 : ℕ) = ((3 : ℕ), (3 : ℕ)) := rfl

/-- ...and `curryTerm` means currying. -/
example : (curryTerm (.base 0) (.base 0) (.base 0)).interp typeBase ()
    (fun (p : ℕ × ℕ) => p.1 + p.2) (2 : ℕ) (3 : ℕ) = (5 : ℕ) := rfl

/-- In `Type`, the β-law holds by computation alone: the meaning of a redex
*is* the meaning of the body in the extended environment. -/
example {Γ σ τ} (t : Term (σ :: Γ) τ) (a : Term Γ σ) (ρ : Ctx.interp typeBase Γ) :
    (Term.app (.lam t) a).interp typeBase ρ = t.interp typeBase (ρ, a.interp typeBase ρ) :=
  rfl
```

(The type ascriptions `(7 : ℕ)` help Lean see that the numeral lives in the interpretation of the base type `o₀`, which is `ℕ`. The `()` is the empty environment, the single value of the terminal object.)

In the Heyting algebra `Prop`, with propositions as base types, the same terms mean proofs. A closed term of type τ is interpreted as a morphism from the empty context, $\top$, to $[\![\tau]\!]$, that is, a proof of `True → ⟦τ⟧`, and applying it to `trivial` (the proof of `True`) gives a proof of the proposition $[\![\tau]\!]$:

```
/-- Three propositions as base types, in the thin category of `Prop`. -/
def propBase (p q r : Prop) : ℕ → Thin Prop
  | 0 => ⟨p⟩
  | 1 => ⟨q⟩
  | _ => ⟨r⟩

/-- The same term `swap`, interpreted in the Heyting algebra `Prop`: a proof
that `p ∧ q` implies `q ∧ p`. -/
example (p q : Prop) : p ∧ q → q ∧ p :=
  (swap.interp (propBase p q q)).down trivial

/-- `dup` becomes a proof of `p → p ∧ p`... -/
example (p : Prop) : p → p ∧ p :=
  (dup.interp (propBase p p p)).down trivial

/-- ...and `curryTerm` a proof of `((p ∧ q) → r) → p → q → r`. -/
example (p q r : Prop) : ((p ∧ q) → r) → p → q → r :=
  ((curryTerm (.base 0) (.base 1) (.base 2)).interp (propBase p q r)).down trivial
```

`swap` proves $p \wedge q \to q \wedge p$, `dup` proves $p \to p \wedge p$, and `curryTerm` proves $((p \wedge q) \to r) \to p \to q \to r$. Program in one model, proof in another, one definition behind both: this is the triangle of the chapter's title, made executable.



The Curry–Howard–Lambek triangle: intuitionistic logic, the simply typed λ-calculus and cartesian closed categories are three presentations of one structure.

In Rust, the story is different, and the difference is the subject of the next section. `swap` is fine (it is `braid` below). `curryTerm` is `curry`, which needed `A: Clone` because the closure it returns may use `a` many times. And `dup`, $\lambda x.\,(x, x)$, which in both Lean models is the most innocent term imaginable, does not compile for a type that is not `Copy`.

Logic	λ-calculus	CCC	Rust	Lean
$A \wedge B$	$A \times B$	product	`(A, B)`	`A × B`
$A \to B$	$A \to B$	exponential	`Box<dyn Fn(A) -> B>`	`A → B`
$\top$	unit	terminal object	`()`	`Unit`
proof	term	morphism	function	function
normalization	β-reduction	$\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}) = f$	calling a closure	`rfl`
contraction	using x twice	diagonal Δ	needs `Clone`	free

19.7 In Mathlib

Mathlib's version of a cartesian closed category is assembled from two pieces: a **monoidal** structure that happens to be cartesian (the class `CartesianMonoidalCategory`), and a **closed** structure for it (`MonoidalClosed`, which provides a right adjoint to tensoring with each object). Earlier versions of Mathlib had a separate `CartesianClosed` class; in the version this book uses it has been folded into `MonoidalClosed`, and the name `CartesianClosed` survives as a namespace with notation such as `A ⟹ B`. The file `code/lean/Book/Ch19/InMathlib.lean` checks that `Type` has both structures:

```
/-- info: inferInstance : CartesianMonoidalCategory Type -/
#guard_msgs in
#check (inferInstance : CartesianMonoidalCategory Type)

/-- info: inferInstance : MonoidalClosed Type -/
#guard_msgs in
#check (inferInstance : MonoidalClosed Type)

/-- In `Type`, the monoidal product `⊗` is the cartesian product `×`. -/
example (X Y : Type) : (X ⊗ Y) = (X × Y) := rfl
```

```
/--
info: @MonoidalClosed.curry : {C : Type u_2} →
  [inst : Category.{u_1, u_2} C] →
    [inst_1 : MonoidalCategory C] → {A X Y : C} → [inst_2 : Closed A] → (A ⊗ Y → X) → (Y → A
⟹ X)
-/
#guard_msgs in
#check @MonoidalClosed.curry

/--
info: @MonoidalClosed.uncurry_curry : ∀ {C : Type u_2} [inst : Category.{u_1, u_2} C]
[inst_1 : MonoidalCategory C]
  {A X Y : C} [inst_2 : Closed A] (f : A ⊗ X → Y), MonoidalClosed.uncurry
(MonoidalClosed.curry f) = f
-/
#guard_msgs in
#check @MonoidalClosed.uncurry_curry
```

The monoidal product `⊗` of `Type` is `×`, by `rfl`. Mathlib's `curry` takes `A ⊗ Y → X` to `Y → A ⟹ X`, with the exponent on the *left* of the product (ours was on the right; the two conventions are isomorphic by swapping), and `uncurry_curry` is our `uncurry_curry`. The design is worth noticing, because it anticipates the next section: in Mathlib a CCC is literally *a monoidal closed category whose monoidal product is cartesian*. Remove the word "cartesian" and you get the structure that models linear logic.

19.8 Rust is not cartesian

The diagonal needs `Clone`

In every category with products, every object has a diagonal $\Delta_a = \langle \mathrm{id}, \mathrm{id} \rangle : a \to a \times a$, and the λ-term $\lambda x.\,(x, x)$ denotes it. Here it is in Rust, for an arbitrary type (`code/rust/compile_fail/ch19_diagonal.rs`):

```rust
fn diagonal<A>(a: A) -> (A, A) {
    (a, a)
}
```

```
error[E0382]: use of moved value: `a`
 --> compile_fail/ch19_diagonal.rs:7:9
  |
6 | fn diagonal<A>(a: A) -> (A, A) {
  |             - move occurs because `a` has type `A`, which does not implement the
`Copy` trait
7 |     (a, a)
  |      -  ^ value used here after move
  |      |
  |      value moved here
  |
help: if `A` implemented `Clone`, you could clone the value
 --> compile_fail/ch19_diagonal.rs:6:13
  |
6 | fn diagonal<A>(a: A) -> (A, A) {
  |             ^ consider constraining this type parameter with `Clone`
7 |     (a, a)
  |      - you could clone this value
help: consider restricting type parameter `A` with trait `Copy`
  |
6 | fn diagonal<A: Copy>(a: A) -> (A, A) {
  |              ++++++
```

Error E0382 is the error of Chapter 6 and Chapter 10: the first component moves `a`, and the second uses it after the move. Read the help messages: "if `A` implemented `Clone`, you could clone the value", and "consider restricting type parameter `A` with trait `Copy`". The compiler is telling us, in its own words, that the diagonal exists only for types that come with a way to duplicate their values. Logically, the diagonal is **contraction**: it is the categorical form of using an assumption twice, and Rust's ownership discipline removes contraction for owned values (Chapter 6).

With a `Clone` bound the diagonal exists, but it is an operation that runs code, which may be expensive or have effects:

```rust
/// A payload whose duplication is visible: cloning it says so.
#[derive(Debug, PartialEq)]
struct Payload {
    bytes: Vec<u8>,
}

impl Clone for Payload {
    fn clone(&self) -> Self {
        println!("  (cloning a payload of {} bytes)", self.bytes.len());
        Payload {
            bytes: self.bytes.clone(),
        }
    }
}

/// The diagonal A → A × A exists only for duplicable types.
```

```
fn diagonal<A: Clone>(a: A) -> (A, A) {
    (a.clone(), a)
}
```

```
let payload = Payload {
    bytes: vec![0; 1024],
};
println!("diagonal(payload):");
let (left, right) = diagonal(payload);
assert_eq!(left, right);

// Pairing calls the diagonal on its input.
let summary = pair(|p: Payload| p.bytes.len(), |p: Payload| p.bytes[0]);
println!("pair(len, first_byte)(payload):");
println!("  = {:?}", summary(left));
```

```
diagonal(payload):
  (cloning a payload of 1024 bytes)
pair(len, first_byte)(payload):
  (cloning a payload of 1024 bytes)
  = (1024, 0)
```

`diagonal(payload)` clones a kilobyte. And `pair(len, first_byte)` clones it too, silently: the universal property of the product promises a pairing $\langle f, g \rangle$ for *every* f and g out of the same object, and in Rust that promise can be kept only by duplicating the input. For a type that cannot be duplicated at all (a file handle, a lock guard, a connection), there is no $\langle \mathrm{id}, \mathrm{id} \rangle$, and so `(A, B)` is not a categorical product.

What survives: a monoidal product

The pair type has not lost everything. These functions compile for all types, with no bounds:

```
/// The structure that survives without `Clone`: the symmetric monoidal maps.
fn braid<A, B>((a, b): (A, B)) -> (B, A) {
    (b, a)
}

fn associate<A, B, C>(((a, b), c): ((A, B), C)) -> (A, (B, C)) {
    (a, (b, c))
}

fn unit_right<A>((a, ()): (A, ())) -> A {
    a
}

/// Currying without `Clone`: the result can be called once (A ⊸ (B ⊸ C)).
fn curry_once<A, B, C>(
    f: impl FnOnce((A, B)) -> C + 'static,
) -> impl FnOnce(A) -> Box<dyn FnOnce(B) -> C>
where
    A: 'static,
    B: 'static,
    C: 'static,
{
    // `a` and `f` are moved into the inner closure: no copies are made.
    move |a: A| -> Box<dyn FnOnce(B) -> C> { Box::new(move |b: B| f((a, b))) }
}
```

```
println!("braid((1, \"a\")) = {:?}", braid((1, "a")));
println!("associate(((1, 2), 3)) = {:?}", associate(((1, 2), 3)));
println!("unit_right((5, ())) = {}", unit_right((5, ())));

let greet =
    curry_once(|(name, greeting): (String, String)| format!("{greeting}, {name}"));
```

```
let to_alice = greet(String::from("Alice"));
println!("curry_once: {}", to_alice(String::from("Hello")));
```

```
braid((1, "a")) = ("a", 1)
associate(((1, 2), 3)) = (1, (2, 3))
unit_right((5, ())) = 5
curry_once: Hello, Alice
```

`braid` swaps, `associate` regroups, `unit_right` removes a `()`, all without copying anything. These are the structure maps of a **symmetric monoidal category**: a category with a product-like operation $\otimes$ (the "tensor") and a unit object I, with isomorphisms $a \otimes b \cong b \otimes a$, $(a \otimes b) \otimes c \cong a \otimes (b \otimes c)$ and $a \otimes I \cong a$ satisfying coherence laws, but with no diagonals and no projections required. A cartesian product is the special case where the tensor has projections and diagonals for every object.

Rust's owned values sit in between. They have projections: `fst((a, b))` compiles, because Rust may drop `b` (weakening, with `Drop` as its side effect, Chapter 24). Categorically, this is because the unit `()` is *terminal*: `to_unit` is the unique way to discard a value, and `fst` is "discard the second component, then remove the unit". A symmetric monoidal category whose unit is terminal is the categorical model of an **affine** logic: weakening yes, contraction no. And Rust's pairs are *closed* in the monoidal sense: `curry_once` turns `FnOnce((A, B)) -> C` into `FnOnce(A) -> Box<dyn FnOnce(B) -> C>` with no `Clone` bound, a bijection between $\mathrm{Hom}(a \otimes b, c)$ and $\mathrm{Hom}(a, b \multimap c)$, where $b \multimap c$ ("b linearly implies c") is a function that may be called once. The adjunction survives; only the cartesian structure is gone.

So, as a model (in the sense of $\approx$ that this book uses throughout):

owned Rust values, moves and `FnOnce` $\approx$ a symmetric monoidal closed category with terminal unit

and the types that *do* have diagonals are the `Copy` types (free, implicit duplication) and the `Clone` types (explicit duplication with a cost). In categorical terms, these are objects equipped with a comonoid structure: a copy map $a \to a \otimes a$ and a discard map $a \to I$ that fit together. Chapter 23 makes this precise as the exponential modality $!A$ of linear logic.

Symmetric monoidal closed categories are the models of (multiplicative, intuitionistic) linear logic, in the same way that CCCs are the models of intuitionistic logic: tensor for $\otimes$, the internal hom for $\multimap$, unit for $\mathbf{1}$. That is the bridge into Part V. Chapter 21 starts from the logic side, by removing structural rules from the sequent calculus, and arrives at the same place we just reached from the categorical side, by removing the diagonal.

19.9 Common confusions

"**A cartesian closed category is a category whose morphisms are functions.**" Heyting algebras are CCCs whose morphisms are inequalities, and there are CCCs of sets with structure, of presheaves, of domains for programming-language semantics. What a CCC requires is the universal properties, not that morphisms be functions.

"**The exponential $b \Rightarrow c$ is the hom-set $\mathrm{Hom}(b, c)$.**" The hom-set lives outside the category; the exponential is an object inside it. In `Type` they happen to look the same (both are `β → γ`), but in a Heyting algebra the hom-set $\mathrm{Hom}(b, c)$ is a proof of $b \leq c$ (or nothing), while the exponential $b \Rightarrow c$ is an element of the algebra. What connects them is that morphisms $1 \to (b \Rightarrow c)$ correspond to morphisms $b \to c$ (Exercise 3).

"**An adjunction is an isomorphism between the two functors.**" The functors $(-) \times b$ and $b \Rightarrow (-)$ are not inverse to each other at all. What is invertible is the correspondence between *morphisms* $L\,a \to c$ and $a \to R\,c$, naturally in a and c.

"**Rust's `(A, B)` is the categorical product of `A` and `B`.**" Only for types whose values can be duplicated. In general it is a monoidal product: it has the structure maps and, thanks to `drop`, the projections, but not the pairing of two functions out of one non-`Clone` value.

"**η is an optimization rule.**" Compilers do use η, but its role in the theory is uniqueness: η is the statement that `curry` is the *only* morphism with the β-property. Without η, the exponential would have a factorization but not a universal property.

"**Lambek's correspondence means every CCC is a programming language.**" Every CCC is a *model* of the simply typed λ-calculus: programs can be interpreted in it. The λ-calculus itself (modulo βη) is one particular CCC, the free one.

19.10 Summary

- A terminal object, binary products and exponentials are defined by universal properties; a category with all three is cartesian closed. Lean's types, with `Unit`, `×` and `→`, form a CCC whose laws hold by `rfl`.
- The exponential's universal property has two halves: existence of $\mathrm{curry}\, f$ with $\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}) = f$ (β) and its uniqueness (η).
- Currying is a bijection $\mathrm{Hom}(a \times b, c) \cong \mathrm{Hom}(a, b \Rightarrow c)$, natural in a and c: the adjunction $(-) \times b \dashv b \Rightarrow (-)$.
- Heyting algebras are thin CCCs (bicartesian, with joins and $\bot$): meet is the product, $\top$ is terminal, implication is the exponential, modus ponens is `eval` and residuation is currying.
- Lambek: the simply typed λ-calculus is the internal language of CCCs. We interpreted it in every CCC in Lean and proved the substitution lemma and the soundness of β and η; in `Type`, β holds by `rfl`; in `Prop`, terms become proofs.
- Rust is not cartesian for non-`Copy` types: the diagonal $\lambda x.\,(x, x)$ is rejected (E0382) and pairing needs `Clone`. Owned values form ($\approx$) a symmetric monoidal closed category with terminal unit, the model of affine logic, and `Copy` / `Clone` types carry the missing diagonals.

19.11 Exercises

1. In Lean, prove `prodMap (Category.id a) (Category.id b) = Category.id (a × b)` in any category with products (add it to a scratch copy of `CCC.lean`, inside the `Products` section). *Hint:* `unfold prodMap`, then rewrite with `Category.id_comp` twice and finish with `pair_fst_snd`.
2. Prove that currying is natural in c. Define `expMap (k : c ⟶ d) : (b ⟹ c) ⟶ (b ⟹ d) := curry (k ⊚ eval)` and prove `curry (k ⊚ f) = expMap k ⊚ curry f`. *Hint:* after `unfold expMap`, rewrite the right side with `← curry_natural`, then reassociate with `← Category.assoc` and use `eval_curry`.
3. On paper, show that in any CCC the morphisms $1 \to (b \Rightarrow c)$ are in bijection with the morphisms $b \to c$. What does this say for `Type`? For a Heyting algebra? *Hint:* use currying with $a = 1$, and the isomorphism $1 \times b \cong b$ (the projection π_2 and $\langle !, \mathrm{id} \rangle$). In a Heyting algebra the statement becomes $\top \le (b \Rightarrow c) \Leftrightarrow b \le c$.
4. In Rust, write `uncurry_once`, the inverse of `curry_once`, and check both round trips on a type that is neither `Copy` nor `Clone` (for example a `struct Ticket(String)` without derives). *Hint:* `uncurry_once(g)` is `move |(a, b)| g(a)(b)`. For the check, compare the results (derive `PartialEq` on the result type, not on `Ticket`).
5. Interpret the term `dup` (λx. (x, x)) in the thin CCC of a Heyting algebra `α`, and read off the inequality it proves. Then explain why the corresponding inequality in a model of affine logic, $a \le a \otimes a$, should not hold in general. *Hint:* in a Heyting algebra it is $a \le a \sqcap a$, which is true because $\sqcap$ is idempotent. In a resource reading, $a \otimes a$ is "two copies of a", which one copy does not provide.
6. Using only the CCC structure of `Thin α` (no lattice lemmas), derive `a ⊓ (a ⇨ b) ≤ b` in any Heyting algebra. *Hint:* build the morphism `pair snd fst` from `a × (a ⟹ b)` to `(a ⟹ b) × a` (this is `braid`), compose it with `eval`, and take `.down`.

CHAPTER 20

Monads and Effects

Count the question marks in the last Rust function you wrote. Each one says the same thing: "evaluate this; if it produced a value, continue with that value; otherwise stop here and hand the failure to my caller". You call `and_then` on an `Option` for the same reason. You call `flat_map` on an iterator when every element produces several results and you want all of them in one sequence. You write `.await` when a step may have to wait for the network before the rest of the function can run. Four spellings, in four corners of the language, of one pattern: *run a step that does something more than return a value, then feed its value to the rest of the computation.*

The pattern has a name and a theory. In Chapter 16 you saw that types and functions form a category. But a parser that may fail, a lookup that may find several answers, a step that updates a counter or waits for a socket: each of these has an **effect**, and functions with effects do not compose the way the category of types expects. At the end of the 1980s Eugenio Moggi proposed a uniform account of such "notions of computation" (his paper *Notions of computation and monads* appeared in 1991): an effectful function from A to B is a pure function from A to $M\,B$, where the type constructor M describes the effect, and a **monad** is exactly the structure that makes such functions compose again. Philip Wadler then showed functional programmers how to structure programs with the idea (*Monads for functional programming* is his best-known tutorial), and in the 1990s Haskell adopted monads to bring input and output into a pure language. Lean inherited the design.

This chapter, the last of Part IV, defines monads in three equivalent ways and proves that they agree. Then it follows the idea into both languages. Rust has many monads and no `Monad`: we will see exactly which features of Rust stand in the way (the missing higher-kinded types of Chapter 17, and the difference between `FnOnce` and `FnMut`), and what Rust does instead. Lean has a `Monad` class, `do` notation and monad transformers, and it lets us state and prove the laws that Rust can only test.

In this chapter you will learn:

- why functions with effects break "functions as morphisms", and how Kleisli arrows $A \to M\,B$ repair composition;
- the definition of a monad (`M`, `pure`, `bind`) with its three laws, and the equivalent definitions with `join` and with Kleisli composition;
- how `?`, `and_then`, `flat_map` and `.await` implement `bind` in Rust, and why Rust has no `Monad` trait;

- how Lean's `Monad` class and `do` notation work, what a `do` block desugars into, and what `LawfulMonad` and `IO` are;
- how to build a monad class with laws from scratch, prove the laws for `Option`, `List` and `State`, and build the Kleisli category of any monad on the `Category` class of Chapter 16;
- how monad transformers (`StateT`, `ExceptT`) combine effects, how Rust combines them ad hoc, and how adjunctions give rise to monads.

20.1 When functions have effects

Composition breaks

Here are two small functions from this chapter's main Rust example, `code/rust/examples/ch20_monads.rs`. The first parses a quantity typed by a user, the second checks that it is in range:

```
/// Parse a quantity: fails on text that is not a number.
fn parse_qty(s: &str) -> Option<u32> {
    s.trim().parse().ok()
}

/// Accept quantities between 1 and 100.
fn check_qty(n: u32) -> Option<u32> {
    if (1..=100).contains(&n) {
        Some(n)
    } else {
        None
    }
}
```

In the category of types and functions of Chapter 16, each is a morphism: `parse_qty` from `&str` to `Option<u32>`, `check_qty` from `u32` to `Option<u32>`. We would like to *compose* them, but composition requires the target of the first morphism to be the source of the second, and `Option<u32>` is not `u32`: `check_qty(parse_qty(s))` does not compile.

You know how to fix this. You write the glue by hand:

```
/// Kleisli composition for `Option`: run `f`, and if it produced a value,
/// run `g` on it.
fn compose_option<A, B, C>(
    f: impl Fn(A) -> Option<B>,
    g: impl Fn(B) -> Option<C>,
) -> impl Fn(A) -> Option<C> {
    move |a| f(a).and_then(|b| g(b))
}
```

`compose_option` runs `f`; if it produced a value, it runs `g` on it; if not, the composite fails. Look at what we did: we started reading `parse_qty` as a *partial* function from `&str` to `u32`, a function with an effect (it may fail), and defined a new way of composing such functions that takes care of the effect.

The same move works for every effect. Suppose M is a type constructor that describes an effect. A function with that effect from A to B is represented by a plain function

$$f : A \to M\,B$$

and such a function is called a **Kleisli arrow** from A to B (after the mathematician Heinrich Kleisli). Here are the effects this chapter meets, with the type constructor that describes each:

Effect	$M\,B$	Rust	Lean
may fail	$B + 1$	`Option<B>`	`Option B`
may fail with an error E	$B + E$	`Result<B, E>`	`Except E B`
has many possible results	lists of B	`Vec<B>`, iterators	`List B`
reads an environment R	$R \to B$	a `&R` parameter	`ReaderM R B`
reads and writes a state S	$S \to B \times S$	a `&mut S` parameter	`StateM S B`
writes a log W	$B \times W$	a `&mut Vec<_>` parameter	Mathlib's `Writer W B`
interacts with the world	(opaque)	any function at all	`IO B`
may suspend and resume	(opaque)	`impl Future<Output = B>`	(not in this book)

Two things stand out in the Rust column. Several entries are not type constructors at all: Rust handles state with `&mut`, and it does not record input and output in types, since any Rust function may print or open a file. And where Rust does have a type constructor, each one comes with its own syntax: `?`, `for`, `.await`. Both observations will matter.

Effects as typing rules

Moggi's idea can be stated as two typing rules. Distinguish *values* of type τ from *computations* of type $M\,\tau$, which may have the effect M when they run and then produce a τ. Two rules connect the two worlds:

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathit{pure}\, e : M\,\tau}\ \text{T-Pure} \qquad \frac{\Gamma \vdash e_1 : M\,\tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : M\,\tau_2}{\Gamma \vdash \texttt{let}\ x \Leftarrow e_1; e_2 : M\,\tau_2}\ \text{T-Bind}$$

Rule T-Pure says that a value can always be seen as a computation that has no effect. Rule T-Bind is rule T-Let of Chapter 1, for computations: run e_1, which may have its effect and then produces a τ_1; bind that result to x; continue with e_2, in which x is a plain τ_1. The whole is again a computation in M.

Notice what is missing: no rule takes you from $M\,\tau$ back to τ. You can use the result of a computation only *inside a larger computation of the same kind*. You know this discipline from Rust: once a step may fail, everything that depends on its result may fail too, unless you handle the failure explicitly. The arrow $\Leftarrow$ in T-Bind is spelled `let x = e1?;` in Rust for `Option` and `Result`, `let x = e1.await;` for futures, and `let x ← e1` in a Lean `do` block.

20.2 Monads: `pure`, `bind` and three laws

The definition

Definition. A **monad** consists of

- a type constructor M, which takes a type A to a type $M\,A$;
- for every type A, an operation $\mathit{pure} : A \to M\,A$;

- for all types A and B, an operation $\mathit{bind} : M\,A \to (A \to M\,B) \to M\,B$;

such that, for all values a, computations m and functions f and g of the appropriate types, three **laws** hold:

$$\mathit{bind}(\mathit{pure}\,a, f) = f\,a \qquad \text{(left identity)}$$

$$\mathit{bind}(m, \mathit{pure}) = m \qquad \text{(right identity)}$$

$$\mathit{bind}(\mathit{bind}(m, f), g) = \mathit{bind}(m, \lambda a.\ \mathit{bind}(f\,a, g)) \qquad \text{(associativity)}$$

`pure` and `bind` are rules T-Pure and T-Bind turned into functions. `pure a` is the computation that does nothing and returns `a`. `bind m f` runs `m`, passes its result to `f`, and runs the computation that `f` returns. For `Option`, `pure` is `Some` and `bind` is `and_then`. For `Vec`, `pure a` is `vec![a]` and `bind` is `flat_map` followed by `collect`. Mathematicians call this presentation a *Kleisli triple*; Haskell spells `bind m f` as `m >>= f`, and so does Lean.

The laws are easiest to understand as **refactorings that must not change what a program means**:

- *Left identity*: `let x ⇐ pure a; e` can be replaced by `e` with `a` for `x`. A step without an effect can be inlined.
- *Right identity*: `let x ⇐ m; pure x` can be replaced by `m`. A block that ends by returning the result of its last step unchanged *is* that last step.
- *Associativity*: given three steps, it does not matter whether you extract the first two or the last two into a helper function. "Extract function" is a safe refactoring.

The laws as category laws

The glue we wrote for `Option` works for any monad. **Kleisli composition** is defined, for $f : A \to M\,B$ and $g : B \to M\,C$, by

$$g \circ_M f = \lambda a.\ \mathit{bind}(f\,a, g) : A \to M\,C$$

Rewrite the three laws with it. Left identity says $g \circ_M \mathit{pure} = g$, right identity says $\mathit{pure} \circ_M f = f$, and associativity says

$$h \circ_M (g \circ_M f) = (h \circ_M g) \circ_M f$$

These are exactly the laws of a category from Chapter 16, with pure as the identity. (The names "left" and "right" cross over because $\circ$ is written right to left.) So we have found a new category:

> For every monad M, the **Kleisli category** of M has types as objects, Kleisli arrows $A \to M\,B$ as morphisms from A to B, pure as identities and $\circ_M$ as composition. The monad laws are the category laws.

Later in the chapter Lean checks this in both directions: the monad laws imply the category laws, and conversely.

`map` and `join`

Category theorists usually present a monad differently: as a functor with two natural transformations. Every monad is a functor in the sense of Chapter 17, because `map` can be defined from the other two operations:

$$\mathit{map}\ f\ m = \mathit{bind}(m, \lambda a.\ \mathit{pure}\,(f\ a))$$

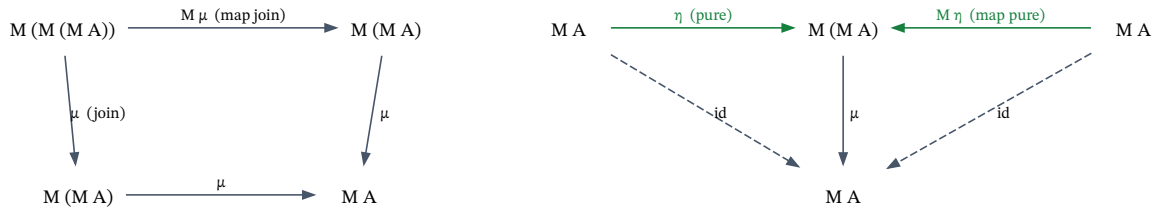
and the functor laws follow from the monad laws (we prove it in Lean below). Next to `pure`, which category theorists call η (eta, the *unit*), there is a flattening operation μ (mu, the *multiplication*), which programmers call `join`:

$$\mu_A : M\,(M\ A) \to M\ A \qquad\qquad \mu_A(mm) = \mathit{bind}(mm, \lambda m.\ m)$$

You have used it: `Option::flatten` turns an `Option<Option<T>>` into an `Option<T>`, and `Iterator::flatten` turns an iterator of iterators into one iterator. Both η and μ are natural transformations (Chapter 18), from the identity functor to M and from $M \circ M$ to M, and the monad laws become three equations between them:

$$\mu_A \circ M\mu_A = \mu_A \circ \mu_{MA} \qquad\qquad \mu_A \circ \eta_{MA} = \mathrm{id}_{MA} = \mu_A \circ M\eta_A$$

The first says that a computation with three layers can be flattened from the inside or from the outside with the same result. The other two say that adding a trivial layer, outside or inside, and flattening it again does nothing.

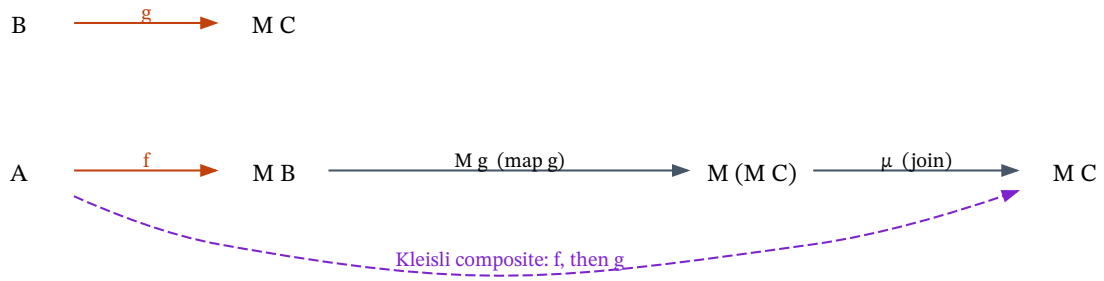


The monad laws in terms of join (μ) and pure (η). Left: flattening three layers from the outside or from the inside gives the same result. Right: adding a trivial layer with pure, outside (η) or inside (M η), and flattening again is the identity.

The two presentations are interchangeable. We just defined `map` and `join` from `bind`; conversely, `bind` is "map, then flatten":

$$\mathit{bind}(m, f) = \mu\,(\mathit{map}\ f\ m)$$

which is why Rust's `flat_map` has that name. In this language the Kleisli composite is $g \circ_M f = \mu_C \circ Mg \circ f$: apply f, map g over the result (which gives two layers of effects), and flatten.



Kleisli composition. Given $f : A \to M\,B$ and $g : B \to M\,C$, the composite runs f, maps g over its result to get $M\,(M\,C)$, and flattens with μ.

> **NOTE**
>
> The μ/η presentation explains a famous line from Saunders Mac Lane's *Categories for the Working Mathematician*: a monad is a monoid in the category of endofunctors. The "product" of two endofunctors is their composite; μ is a multiplication $M \circ M \to M$, η a unit $\mathrm{Id} \to M$, and the three equations above are the associativity and unit laws of a monoid (Chapter 14). You will see a down-to-earth instance of this below: the laws of the `Writer` monad are literally the monoid laws of its log.

20.3 Monads you already use in Rust

`?` is bind

Here is one function written three ways. All three parse a quantity, check it, and compute how many boxes it fills at two units per box:

```rust
/// With `?`: every `?` is a bind, and the rest of the function body is the
/// continuation that receives the value.
fn boxes_question(s: &str) -> Option<u32> {
    let n = parse_qty(s)?;
    let q = check_qty(n)?;
    Some(q.div_ceil(2))
}

/// With `and_then`: the continuations are written as closures.
fn boxes_and_then(s: &str) -> Option<u32> {
    parse_qty(s).and_then(|n| check_qty(n).and_then(|q| Some(q.div_ceil(2))))
}

/// With `match` and `return`: what `?` does on an `Option`, written out.
fn boxes_match(s: &str) -> Option<u32> {
    let n = match parse_qty(s) {
        Some(v) => v,
        None => return None,
    };
    let q = match check_qty(n) {
        Some(v) => v,
        None => return None,
    };
    Some(q.div_ceil(2))
}
```

The program checks that the three agree on every sample and prints the results of `compose_option(parse_qty, check_qty)` (as `read_qty`) and of the three versions (as `boxes`):

```
-- Option: ?, and_then and match agree --
      12: read_qty = Some(12), boxes = Some(6)
  twelve: read_qty = None, boxes = None
     500: read_qty = None, boxes = None
```

The `match` version is, conceptually, what the compiler does with `?` on an `Option`. Read `let n = parse_qty(s)?; REST` as `parse_qty(s).and_then(|n| REST)`, where `REST` is *the rest of the function body*: the continuation that receives the value. If there is no value, the continuation is skipped with an early `return`. That is `bind`, with an implicit continuation.

The real desugaring goes through two standard library traits. Roughly, `e?` becomes a `match` on `Try::branch(e)`, which returns `ControlFlow::Continue(v)` (continue with `v`) or `ControlFlow::Break(r)`, in which case the function returns `FromResidual::from_residual(r)`. Both traits are unstable (feature `try_trait_v2`), so on stable Rust `?` works only for the standard library's types (`Option`, `Result`, `ControlFlow` and `Poll` values wrapping them): `bind` for a fixed list of monads.

For `Result`, `?` does one more thing:

```
#[derive(Debug, PartialEq)]
enum OrderError {
    BadNumber(ParseIntError),
    OutOfRange(u32),
}

impl From<ParseIntError> for OrderError {
    fn from(e: ParseIntError) -> Self {
        OrderError::BadNumber(e)
    }
}

fn check_range(n: u32) -> Result<u32, OrderError> {
    if (1..=100).contains(&n) {
        Ok(n)
    } else {
        Err(OrderError::OutOfRange(n))
    }
}

/// `?` on a `Result`: bind, plus a conversion of the error through `From`.
fn boxes_result(s: &str) -> Result<u32, OrderError> {
    let n: u32 = s.trim().parse()?;
    let q = check_range(n)?;
    Ok(q.div_ceil(2))
}

/// The same function with the early returns and conversions written out.
fn boxes_result_desugared(s: &str) -> Result<u32, OrderError> {
    let n: u32 = match s.trim().parse::<u32>() {
        Ok(v) => v,
        Err(e) => return Err(From::from(e)),
    };
    let q = match check_range(n) {
        Ok(v) => v,
        Err(e) => return Err(From::from(e)),
    };
    Ok(q.div_ceil(2))
}
```

```
-- Result: ? converts errors with From --
      12: Ok(6)
  twelve: Err(BadNumber(ParseIntError { kind: InvalidDigit }))
     500: Err(OutOfRange(500))
```

The first `?` meets a `ParseIntError`, but the function returns `Result<u32, OrderError>`. `bind` for `Result<_, E>` would keep the error type; `?` converts it with `From::from`, hence our `From<ParseIntError>` impl. (For the second `?`, the blanket `impl<T> From<T> for T` makes the conversion the identity.) As a typing rule, a model in the style of Chapter 1, with *ret* the return type of the enclosing function:

$$\frac{\Gamma \vdash e : \texttt{Result}\langle \tau, E\rangle \qquad \mathit{ret} = \texttt{Result}\langle \sigma, F\rangle \qquad F : \texttt{From}\langle E\rangle}{\Gamma \vdash e? : \tau}\ \text{T-Try}$$

The conclusion gives `e?` the *unwrapped* type τ: that is how the rest of the function gets to see a plain value, as in rule T-Bind. And the premise about *ret* shows that the monad is fixed by the function's return type: one function body, one monad. Using `?` on an `Option` inside a function that returns a `Result` is rejected:

```rust
pub fn first_qty(lines: &[&str]) -> Result<u32, String> {
    let first = lines.first()?;
    first.parse::<u32>().map_err(|e| e.to_string())
}
```

```
error[E0277]: the `?` operator can only be used on `Result`s, not `Option`s, in a function
that returns `Result`
 --> compile_fail/ch20_question_mark_option.rs:6:30
  |
5 | pub fn first_qty(lines: &[&str]) -> Result<u32, String> {
  | ------------------------------------------------------- this function returns a `Result`
6 |     let first = lines.first()?;
  |                              ^ use `.ok_or(...)?` to provide an error compatible with
`Result<u32, String>`
```

The suggested fix, `.ok_or(...)?`, first translates the `Option` into a `Result`, a translation between monads that respects `pure` and `bind` (a *monad morphism*). Lean inserts such translations automatically, as we will see when we combine effects.

`and_then`, `flat_map` and iterator adapters

`Option::and_then` and `Result::and_then` are `bind` with an explicit closure. For sequences, `bind` is `Iterator::flat_map`: apply a function that returns many results to every element, and concatenate. Here it is at work on a nondeterministic question, "which warehouses and carriers could ship this item?":

```rust
/// Every warehouse that stocks `item`.
fn warehouses(item: &str) -> Vec<&'static str> {
    match item {
        "bolts" => vec!["Hamburg", "Lyon"],
        "nuts" => vec!["Lyon"],
        _ => vec![],
    }
}

/// Every carrier that collects from a warehouse.
fn carriers(site: &str) -> Vec<&'static str> {
```

```
    match site {
        "Hamburg" => vec!["DHL", "UPS"],
        "Lyon" => vec!["Geodis"],
        _ => vec![],
    }
}

/// `bind` for vectors: apply `f` to every element and concatenate.
fn bind_vec<A, B>(m: Vec<A>, f: impl FnMut(A) -> Vec<B>) -> Vec<B> {
    m.into_iter().flat_map(f).collect()
}

/// Every (warehouse, carrier) pair that can ship `item`.
fn shipping_options(item: &str) -> Vec<(&'static str, &'static str)> {
    bind_vec(warehouses(item), |w| {
        bind_vec(carriers(w), |c| vec![(w, c)])
    })
}
```

```
-- Vec: flat_map is bind --
   bolts: [("Hamburg", "DHL"), ("Hamburg", "UPS"), ("Lyon", "Geodis")]
    nuts: [("Lyon", "Geodis")]
  screws: []
```

Read `shipping_options` as T-Bind twice: choose a warehouse `w`, then a carrier `c` for it, then return the pair. The list monad explores every choice. An empty list means "no possible result", so `Vec` generalizes `Option`, which is a list with at most one element.

Other iterator adapters are monad operations in disguise: `flatten` is `join`, and `iter.collect::<Option<Vec<_>>>()` turns a sequence of computations that may fail into one computation that fails if any of them does, an operation we will meet again as `sequence`.

The example also checks the three laws for `Option` and `Vec` on samples:

```
/// The three monad laws, checked on samples: for `Option` (pure = `Some`,
/// bind = `and_then`) and for `Vec` (pure = `vec![a]`, bind = `flat_map`).
fn check_laws() {
    for s in ["12", "twelve", "500", "0", " 7 "] {
        let m = parse_qty(s);
        // Left identity: pure(a) >>= f  ==  f(a)
        assert_eq!(Some(s).and_then(parse_qty), parse_qty(s));
        // Right identity: m >>= pure  ==  m
        assert_eq!(m.and_then(Some), m);
        // Associativity: (m >>= f) >>= g  ==  m >>= (|x| f(x) >>= g)
        let half = |n: u32| n.checked_sub(1).map(|k| k / 2);
        assert_eq!(
            m.and_then(check_qty).and_then(half),
            m.and_then(|n| check_qty(n).and_then(half))
        );
    }
    for item in ["bolts", "nuts", "screws"] {
        let m = warehouses(item);
        assert_eq!(bind_vec(vec![item], warehouses), warehouses(item));
        assert_eq!(bind_vec(m.clone(), |x| vec![x]), m);
        assert_eq!(
            bind_vec(bind_vec(m.clone(), carriers), services),
            bind_vec(m.clone(), |w| bind_vec(carriers(w), services))
        );
    }
}
```

```
-- the monad laws hold on every sample --
```

As in Chapter 14, a Rust test can confirm the laws on the inputs we thought of and nothing more. The Lean instances below prove them for all inputs.

State, the hard way and the Rust way

`Option`, `Result` and `Vec` look like containers. The state monad does not: a computation in it is a *function* from the current state to a result and a new state. We can build it in Rust with closures:

```
/// A stateful computation: a function from the current state to a result and
/// a new state. It is boxed because every closure has its own type, and
/// `bind` must return one type, whatever closure it was given.
struct State<'a, S, A>(Box<dyn FnOnce(S) -> (A, S) + 'a>);

impl<'a, S: 'a, A: 'a> State<'a, S, A> {
    fn pure(a: A) -> Self {
        State(Box::new(move |s| (a, s)))
    }

    fn bind<B: 'a>(self, f: impl FnOnce(A) -> State<'a, S, B> + 'a) -> State<'a, S, B> {
        State(Box::new(move |s| {
            let (a, s1) = (self.0)(s);
            (f(a).0)(s1)
        }))
    }

    fn run(self, initial: S) -> (A, S) {
        (self.0)(initial)
    }
}

/// Read the state.
fn get<'a, S: Clone + 'a>() -> State<'a, S, S> {
    State(Box::new(|s: S| (s.clone(), s)))
}

/// Replace the state.
fn put<'a, S: 'a>(new: S) -> State<'a, S, ()> {
    State(Box::new(move |_| ((), new)))
}
```

Each closure has its own anonymous type (Chapter 8), and `bind` must return a single type whatever closures it was given, so the closure goes into a `Box<dyn FnOnce(S) -> (A, S)>`, an existential package (Chapter 15); `+ 'a` says that what it captures must live at least as long as `'a`. `bind` builds a closure that runs `self`, feeds the result to `f`, and runs the computation `f` returns from the intermediate state `s1`. Here is a program that gives every order line a fresh ticket number, next to what a Rust programmer would write:

```
/// A fresh ticket number: read the counter, advance it, return the old value.
fn fresh<'a>() -> State<'a, u32, u32> {
    get().bind(|n| put(n + 1).bind(move |()| State::pure(n)))
}

/// Give every order line a ticket number, as `label` does in `State.lean`.
fn label<'a>(items: &'a [&'a str]) -> State<'a, u32, Vec<(u32, &'a str)>> {
    match items.split_first() {
        None => State::pure(Vec::new()),
        Some((first, rest)) => fresh().bind(move |n| {
            label(rest).bind(move |tail| {
                let mut lines = vec![(n, *first)];
                lines.extend(tail);
                State::pure(lines)
            })
        }),
```

```
    }
}

/// What a Rust programmer writes instead: the state is a `&mut` borrow.
fn label_mut<'a>(items: &[&'a str], counter: &mut u32) -> Vec<(u32, &'a str)> {
    let mut lines = Vec::new();
    for item in items {
        lines.push((*counter, *item));
        *counter += 1;
    }
    lines
}
```

```
-- State: a monad of closures, and &mut --
label     : [(100, "bolts"), (101, "nuts"), (102, "washers")], counter = 103
label_mut : same lines, counter = 103
```

Both versions produce the same lines and leave the counter at 103. The monadic one allocates a box for every `bind`, needs lifetime annotations, and reads inside out; the `&mut` one is short, fast and obvious. The state monad is how languages without mutation express state. Rust has *controlled* mutation instead: a `&mut u32` is a unique, temporary permission to change the counter, and the borrow checker (Part VI) guarantees that nobody else observes the counter meanwhile. Threading the state through a function $S \to A \times S$ and lending a unique mutable reference are two answers to the same question. Likewise, the reader monad is an extra `&Config` parameter in Rust, and the writer monad a `&mut Vec<String>` parameter.

20.4 async and await: a monad built into the compiler

A future is a computation that may have to wait. The `Future` trait has one method, `poll`, which returns `Poll::Ready(value)` or `Poll::Pending` ("not yet, poll me again later"). Something has to keep calling `poll`: an **executor**. `code/rust/examples/ch20_async.rs` contains the smallest executor that works with the standard library alone:

```
/// A minimal executor: poll the future until it is ready, and report how many
/// polls it took. A real executor sleeps until a waker fires; ours retries at
/// once, which is enough for futures that never wait for the outside world.
fn block_on<F: Future>(fut: F) -> (F::Output, u32) {
    let mut fut = pin!(fut);
    let mut cx = Context::from_waker(Waker::noop());
    let mut polls = 0;
    loop {
        polls += 1;
        if let Poll::Ready(value) = fut.as_mut().poll(&mut cx) {
            return (value, polls);
        }
    }
}
```

`pin!` fixes the future in place on the stack (a future may hold references into itself, so it must not move once polled), and `Waker::noop()` is a waker that does nothing, enough for an executor that polls again at once. Here is a future that is not ready the first time, standing in for a network round trip:

```
/// A future that is not ready the first time it is polled, standing in for a
/// network round trip.
struct NetworkDelay {
```

```
    waited: bool,
}

impl Future for NetworkDelay {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        if self.waited {
            Poll::Ready(())
        } else {
            self.waited = true;
            cx.waker().wake_by_ref(); // "poll me again"
            Poll::Pending
        }
    }
}

fn network_delay() -> NetworkDelay {
    NetworkDelay { waited: false }
}
```

And here is a price quote that fetches prices from a simulated service:

```
#[derive(Debug, PartialEq)]
enum ShopError {
    UnknownItem(String),
}

/// Ask the (simulated) price service for the unit price of an item.
async fn fetch_price(item: &str) -> Result<u32, ShopError> {
    network_delay().await;
    match item {
        "bolts" => Ok(25),
        "nuts" => Ok(10),
        _ => Err(ShopError::UnknownItem(item.to_string())),
    }
}

/// Two effects in one function: `.await` may suspend, `?` may fail. Each is a
/// bind, and the compiler supports both.
async fn quote(lines: &[(&str, u32)]) -> Result<u32, ShopError> {
    let mut total = 0;
    for &(item, qty) in lines {
        let price = fetch_price(item).await?;
        total += price * qty;
    }
    Ok(total)
}
```

```
quote([("bolts", 2), ("nuts", 3)]) = Ok(80) after 3 polls
quote([("bolts", 2), ("screws", 1), ("nuts", 3)]) = Err(UnknownItem("screws")) after 3 polls
  (stopped at "screws": the line after it was never priced)
and_then(ready(20), |n| ready(n + 1)) = 21 after 1 poll
```

Each `fetch_price` waits once, so the first quote needs three polls: two that stop at a `network_delay`, and one that finishes. The second quote stops at `"screws"`: the `?` returns the error and the nuts are never priced.

In monadic terms, `impl Future<Output = B>` is a computation $M\ B$, `.await` is `bind`, an `async` block is a block of T-Bind steps, `std::future::ready` is `pure`, and the executor is what *runs* the monad, as `run` did for our `State`. We can even write `bind` for futures with `async` itself:

```
/// `bind` for futures, written with `async` itself: run `first`, feed its
/// output to `f`, run the future that `f` returns.
```

```
fn and_then<A, B, Fut1, Fut2>(
    first: Fut1,
    f: impl FnOnce(A) -> Fut2,
) -> impl Future<Output = B>
where
    Fut1: Future<Output = A>,
    Fut2: Future<Output = B>,
{
    async move {
        let a = first.await;
        f(a).await
    }
}
```

The last line of the output shows `and_then(ready(20), |n| ready(n + 1))` completing at the first poll, as left identity predicts. (The program also checks left identity on `fetch_price`.)

The compiler does not implement `async` with closures and `bind`, though. It turns the body of an `async fn` into a state machine, roughly an `enum` with a variant for every `.await` at which the function can be suspended, holding the local variables alive at that point. That is how `quote` can update `total` in a `for` loop across suspension points. And notice `.await?`: `quote` has *two* effects, suspension and failure, stacked on top of each other. Lean needs a library construction for that, monad transformers; Rust builds it into the syntax.

20.5 Why Rust has no `Monad` trait

Rust has all these monads, but no trait that they share, and no way to write a function once for all of them. There are three reasons, and the second is the most interesting.

No higher-kinded types

A `Monad` trait would have to abstract over `Option` itself, a type constructor of kind $Type \to Type$, but Rust's traits abstract over types. Chapter 17 met this obstacle for `Functor`, and the workaround: a "family" trait, in which a marker type stands for the type constructor and a generic associated type applies it. The monad version is in `code/rust/examples/ch20_monad_trait.rs`:

```
/// A family of types with `pure` and `bind`. `Self` is a marker type that
/// stands for a type constructor; `Self::Of<T>` applies it to `T`.
trait Monad {
    type Of<T>;

    fn pure<A>(a: A) -> Self::Of<A>;

    /// `FnMut`, not `FnOnce`: the `Vec` instance calls `f` once per element.
    fn bind<A, B, F>(m: Self::Of<A>, f: F) -> Self::Of<B>
    where
        F: FnMut(A) -> Self::Of<B>;
}

/// The marker for `Option`.
struct OptionM;

/// The marker for `Vec`.
struct VecM;

impl Monad for OptionM {
    type Of<T> = Option<T>;
```

```
    fn pure<A>(a: A) -> Option<A> {
        Some(a)
    }

    fn bind<A, B, F>(m: Option<A>, f: F) -> Option<B>
    where
        F: FnMut(A) -> Option<B>,
    {
        m.and_then(f)
    }
}

impl Monad for VecM {
    type Of<T> = Vec<T>;

    fn pure<A>(a: A) -> Vec<A> {
        vec![a]
    }

    fn bind<A, B, F>(m: Vec<A>, f: F) -> Vec<B>
    where
        F: FnMut(A) -> Vec<B>,
    {
        m.into_iter().flat_map(f).collect()
    }
}
```

With it we can finally write code once for every monad:

```
/// Kleisli composition, once for every monad.
fn kleisli<M: Monad, A, B, C>(
    mut f: impl FnMut(A) -> M::Of<B>,
    g: impl FnMut(B) -> M::Of<C> + Clone,
) -> impl FnMut(A) -> M::Of<C> {
    move |a| M::bind(f(a), g.clone())
}

/// Every combination of a result of `ma` with a result of `mb`.
fn pair<M: Monad, A: Clone, B>(ma: M::Of<A>, mb: M::Of<B>) -> M::Of<(A, B)>
where
    M::Of<B>: Clone,
{
    M::bind(ma, move |a: A| {
        M::bind(mb.clone(), move |b: B| M::pure((a.clone(), b)))
    })
}

/// Turn a list of computations into one computation of a list.
fn sequence<M: Monad, A: Clone>(ms: Vec<M::Of<A>>) -> M::Of<Vec<A>>
where
    M::Of<A>: Clone,
{
    let mut acc = M::pure(Vec::new());
    for m in ms {
        acc = M::bind(acc, move |done: Vec<A>| {
            M::bind(m.clone(), move |x: A| {
                let mut next = done.clone();
                next.push(x);
                M::pure(next)
            })
        });
    }
    acc
}
```

`pair` combines two computations into a computation of pairs, and `sequence` turns a list of computations into a computation of a list. The same code means very different things in different monads:

```
// The same generic functions, used with two different monads. The marker
// type must be named: the compiler cannot work out `M` from `Option<u32>`.
let mut read_qty = kleisli::<OptionM, _, _, _>(parse_qty, check_qty);
let mut variants = kleisli::<VecM, _, _, _>(sizes, colors);
println!("read_qty(\"12\")  = {:?}", read_qty("12"));
println!("read_qty(\"500\") = {:?}", read_qty("500"));
println!("variants(\"tee\") = {:?}", variants("tee"));

println!(
    "pair, Option  = {:?}",
    pair::<OptionM, _, _>(Some(3), Some("bolts"))
);
println!(
    "pair, Vec     = {:?}",
    pair::<VecM, _, _>(vec![1, 2], vec!['a', 'b'])
);

let all = sequence::<OptionM, _>(vec![Some(1), Some(2), Some(3)]);
let one_missing = sequence::<OptionM, _>(vec![Some(1), None, Some(3)]);
let grid = sequence::<VecM, _>(vec![vec!["S", "M"], vec!["red", "blue"]]);
println!("sequence, Option = {all:?} and {one_missing:?}");
println!("sequence, Vec    = {grid:?}");
```

```
read_qty("12")  = Some(12)
read_qty("500") = None
variants("tee") = ["tee/S/red", "tee/S/blue", "tee/M/red", "tee/M/blue"]
pair, Option  = Some((3, "bolts"))
pair, Vec     = [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
sequence, Option = Some([1, 2, 3]) and None
sequence, Vec    = [["S", "red"], ["S", "blue"], ["M", "red"], ["M", "blue"]]
```

For `Option`, `pair` succeeds only if both sides do, and `sequence` is all-or-nothing, like `collect::<Option<Vec<_>>>()`. For `Vec`, `pair` is the cartesian product and `sequence` enumerates every combination of choices. One definition, two effects: that is what an abstraction over monads buys.

It comes at a price, visible in every call: the turbofish `::<OptionM, ...>`. Leave it out, and the compiler gives up:

```
trait Monad {
    type Of<T>;
    fn pure<A>(a: A) -> Self::Of<A>;
}

struct OptionM;

impl Monad for OptionM {
    type Of<T> = Option<T>;
    fn pure<A>(a: A) -> Option<A> {
        Some(a)
    }
}

fn twice<M: Monad>(n: u32) -> M::Of<u32> {
    M::pure(n * 2)
}

pub fn answer() -> Option<u32> {
    twice(21)
}
```

```
error[E0283]: type annotations needed
  --> compile_fail/ch20_monad_family_inference.rs:25:5
   |
25 |     twice(21)
```

```
   |     ^^^^^ cannot infer type of the type parameter `M` declared on the function `twice`
   |
   = note: the type must implement `Monad`
help: the trait `Monad` is implemented for `OptionM`
  --> compile_fail/ch20_monad_family_inference.rs:13:1
   |
13 | impl Monad for OptionM {
   | ^^^^^^^^^^^^^^^^^^^^^^
   = note: cannot satisfy `<_ as Monad>::Of<u32> == Option<u32>`
note: required by a bound in `twice`
  --> compile_fail/ch20_monad_family_inference.rs:20:13
   |
20 | fn twice<M: Monad>(n: u32) -> M::Of<u32> {
   |             ^^^^^ required by this bound in `twice`
help: consider specifying a concrete type for the type parameter `M`
   |
25 |     twice::</* Type */>(21)
   |          ++++++++++++++
```

The note is the heart of the matter: the compiler must solve `<_ as Monad>::Of<u32> == Option<u32>` for the blank, and it will not, because two different markers could both declare `type Of<T> = Option<T>`. A real higher-kinded parameter `M` with `M<u32> = Option<u32>` would determine `M = Option` by unification (Chapter 13); a type family does not.

`FnOnce` or `FnMut`?

Look again at the closure bounds. `Option::and_then` takes an `FnOnce`, while `Iterator::flat_map` takes an `FnMut`. This is no accident: `bind` for `Option` calls its continuation at most once, `bind` for `Vec` once per element. In Chapter 8 the three closure traits answered the question "how many times may this closure be called?", and the answer differs between the two monads.

A single `Monad` trait has to choose one bound. If it chooses `FnOnce`, the `Vec` instance cannot be written:

```
pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
    let mut out = Vec::new();
    for a in m {
        out.extend(f(a));
    }
    out
}
```

```
error[E0382]: use of moved value: `f`
 --> compile_fail/ch20_vec_bind_fnonce.rs:8:20
  |
5 | pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
  |                                                           - move occurs because `f` has
type `F`, which does not implement the `Copy` trait
6 |     let mut out = Vec::new();
7 |     for a in m {
  |     ---------- inside of this loop
8 |         out.extend(f(a));
  |                    ^---
  |                    |
  |                    `f` moved due to this call, in previous iteration of loop
  |
note: `FnOnce` closures can only be called once
 --> compile_fail/ch20_vec_bind_fnonce.rs:5:26
  |
5 | pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
  |                          ^^^^^^^^^^^^^^^^^^^ `F` is made to be an `FnOnce` closure here
...
8 |         out.extend(f(a));
```

```
  |                    ---- this value implements `FnOnce`, which causes it to be moved when
called
```

Calling an `FnOnce` consumes it, so the second iteration of the loop would use a moved value. This is error E0382, the same error as using a moved `String` twice: in Part V's terms, an `FnOnce` closure is a resource that can be used once, and calling it twice would need contraction. If the trait chooses `FnMut`, as ours does, then `Vec` is fine, but every `Option` user pays:

```
fn bind_option<A, B, F>(m: Option<A>, mut f: F) -> Option<B>
where
    F: FnMut(A) -> Option<B>,
{
    match m {
        Some(a) => f(a),
        None => None,
    }
}

pub fn ticket(id: Option<u32>, customer: String) -> Option<(u32, String)> {
    bind_option(id, move |n| Some((n, customer)))
}
```

```
error[E0507]: cannot move out of `customer`, a captured variable in an `FnMut` closure
  --> compile_fail/ch20_option_bind_fnmut.rs:17:39
   |
16 | pub fn ticket(id: Option<u32>, customer: String) -> Option<(u32, String)> {
   |                                --------  ------ move occurs because `customer` has type
`String`, which does not implement the `Copy` trait
   |                                |
   |                                captured outer variable
17 |     bind_option(id, move |n| Some((n, customer)))
   |                     --------          ^^^^^^^^ `customer` is moved here
   |                     |
   |                     captured by this `FnMut` closure
   |
help: `Fn` and `FnMut` closures require captured values to be able to be consumed multiple
times, but `FnOnce` closures may consume them only once
  --> compile_fail/ch20_option_bind_fnmut.rs:8:8
   |
 8 |     F: FnMut(A) -> Option<B>,
   |        ^^^^^^^^^^^^^^^^^^^^^
```

The closure gives away the `String` it captured. That is fine for a closure that runs once, and `id.and_then(move |n| Some((n, customer)))` compiles. But an `FnMut` closure must be ready to run again, and a second run would have no `customer` left to give away, as the help message explains. The same cost appears in our generic functions: `pair` and `sequence` need `Clone` bounds and call `clone()` inside their closures, copies that `Option` never needs.

In Haskell or Lean every value can be used as often as you like, so the question "how many times will `bind` run the continuation?" never comes up, and one `Monad` class covers both cases. In Rust the question is part of the type, because closures own what they capture. The textbook definition of a monad does not ask how often a function $A \to M\ B$ will be called; Rust's ownership discipline makes that a real difference. It is Part V's resource reading of types, showing up far from ownership itself.

Control flow and borrowing

The third reason is practical. `?` and `.await` are not closures: they can sit inside a loop or a `match` arm, `return` from the function, and hold a mutable borrow of a local variable across several steps, as

`quote` does with `total`. A closure passed to `and_then` cannot `return` from the enclosing function or `break` out of its loop, and a closure that captures `&mut total` conflicts with every other use of `total` while it exists.

So Rust took a different road. It has the *instances* (`Option`, `Result`, iterators, futures) and *dedicated syntax* for the most important ones (`?`, `for`, `.await`), each zero-cost and each composing with the others (`.await?` works). Other effects are expressed with `&mut`, with parameters, or not at all (input and output). What Rust gives up is the abstraction: there is no `sequence` for every monad, and no way to give a new effect its own `?` on stable Rust.

20.6 Monads in Lean

The `Monad` class and `do` notation

Lean's core library defines monads with a small hierarchy of type classes (Chapter 14). The two operations come from the classes `Pure` and `Bind`, and `Monad` extends both (through `Applicative`, which this chapter does not need). The code is in `code/lean/Book/Ch20/DoNotation.lean`:

```
/-- info: @pure : {f : Type u_1 → Type u_2} → [self : Pure f] → {α : Type u_1} → α → f α -/
#guard_msgs in
#check @pure

/--
info: @bind : {m : Type u_1 → Type u_2} → [self : Bind m] → {α β : Type u_1} → m α → (α → m
β) → m β
-/
#guard_msgs in
#check @bind
```

Read the second signature with all implicit arguments shown: for any type constructor `m` (`u_1` and `u_2` are universe variables, Chapter 12) that has a `Bind` instance (the square brackets mark an instance argument), and any types `α` and `β`, `bind` takes an `m α` and a function `α → m β` and returns an `m β`: the definition of a monad, minus the laws. `x >>= f` is notation for `bind x f`. The core library has instances for `Option`, `Except ε`, `Id`, the state and reader monads, `IO` and more, but none for `List`; Mathlib adds one in `Mathlib.Data.List.Monad`.

As you saw in Chapter 2, monadic code in Lean is usually written with `do` notation. Here is the boxes computation of the Rust example:

```
def parseQty (s : String) : Option Nat := s.toNat?

def checkQty (n : Nat) : Option Nat := if 1 ≤ n ∧ n ≤ 100 then some n else none

/-- Parse, check, and compute the number of boxes (two units per box). -/
def boxes (s : String) : Option Nat := do
  let n ← parseQty s
  let q ← checkQty n
  return (q + 1) / 2

/-- info: [some 6, none, none] -/
#guard_msgs in
#eval ["12", "twelve", "500"].map boxes
```

`let n ← parseQty s` is rule T-Bind: run `parseQty s`, and if it produces a value, call it `n`. `return` is `pure`. To see what Lean makes of the block, print the definition with notation switched off (`set_option pp.notation false in` turns off all notation for the next command, so that `>>=`, `do` and even `+` and `/` are shown as plain function applications):

```
/--
info: def Ch20.Do.boxes : String → Option Nat :=
fun s => bind (parseQty s) fun n => bind (checkQty n) fun q => pure (HDiv.hDiv (HAdd.hAdd q
1) 2)
-/
#guard_msgs in
set_option pp.notation false in
#print boxes

/-- The `do` block and the chain of binds are the same term. -/
example (s : String) :
    boxes s = (parseQty s >>= fun n => checkQty n >>= fun q => pure ((q + 1) / 2)) :=
  rfl
```

There is no magic. Each `let x ← e` becomes `bind e (fun x => ...)`, with the rest of the block as the body of the function: the continuation, exactly as for Rust's `?`. The `example` confirms that the `do` block and the hand-written chain of `>>=` are the same term (`rfl` proves it). Nested actions follow the same scheme: `(← e)` inside an expression means "run `e` first, and use its result here":

```
/-- `(← e)` runs `e` first and uses its result in place. -/
def totalQty (a b : String) : Option Nat := do
  return (← parseQty a) + (← parseQty b)

/--
info: def Ch20.Do.totalQty : String → String → Option Nat :=
fun a b =>
  bind (parseQty a) fun __do_lift => bind (parseQty b) fun __do_lift_1 => pure (HAdd.hAdd
__do_lift __do_lift_1)
-/
#guard_msgs in
set_option pp.notation false in
#print totalQty
```

The elaborator named the two results `__do_lift` and `__do_lift_1` and moved both binds in front of the addition, left to right. The other constructs of `do` desugar in the same spirit: a line `e` whose result is unused becomes `bind e (fun _ => ...)`, `let x := e` is an ordinary `let`, and the `let mut` variables and `for` loops of Chapter 2 become state passed from one iteration to the next.

`LawfulMonad`

Lean's `Monad` class does not mention the laws, so an instance may break them. A separate `Prop`-valued class, `LawfulMonad m`, collects them as proof obligations, like the `LawfulMonoid` of Chapter 14. Its fields include `pure_bind` (`pure x >>= f = f x`) and `bind_assoc`, together with laws relating `bind` to `map` and `<*>` from `Functor` and `Applicative`. Right identity is not a field but a theorem, `bind_pure`, derived from the others. For a lawful monad the laws become rewriting rules for `do` blocks, proved here for an arbitrary monad `m`:

```
section Laws

variable {m : Type → Type} [Monad m] [LawfulMonad m] {α β γ : Type}

/-- Left identity: a `pure` step can be inlined. -/
```

```
example (a : α) (f : α → m β) : (do let x ← pure a; f x) = f a := by
  simp only [pure_bind]

/-- Right identity: binding into `pure` changes nothing. -/
example (x : m α) : (x >>= fun a => pure a) = x := by
  simp only [bind_pure]

/-- The `do` elaborator applies right identity by itself: this block *is* `x`. -/
example (x : m α) : (do let a ← x; pure a) = x := rfl

/-- Associativity: extracting a sub-block into a helper (or inlining it) does
not change the program. -/
example (x : m α) (f : α → m β) (g : β → m γ) :
    (do let b ← (do let a ← x; f a); g b) = (do let a ← x; let b ← f a; g b) := by
  simp only [bind_assoc]

end Laws
```

`simp only [l]` rewrites the goal with the lemma `l` and closes it if both sides become equal. The third `example` is a curiosity worth knowing: the `do` elaborator *itself* applies right identity, turning `let a ← x; pure a` into plain `x`, so `rfl` proves the equation. The language relies on a law that the `Monad` class does not require. The core library's monads are lawful:

```
example : LawfulMonad Option := inferInstance
example : LawfulMonad (Except String) := inferInstance
example : LawfulMonad (StateM Nat) := inferInstance
```

IO

`IO α` is the type of computations that may interact with the outside world and then produce an `α`. Defining one does nothing; running it is a separate step, performed by the Lean runtime when the action is a program's `main`, or by `#eval` in the editor:

```
/-- An `IO` action: a description of a computation that interacts with the
world. Defining it prints nothing. -/
def greet (name : String) : IO Unit := do
  IO.println s!"Hello, {name}."
  IO.println "Your order has shipped."

/--
info: Hello, Ada.
Your order has shipped.
-/
#guard_msgs in
#eval greet "Ada"
```

Because actions are values, you can store them and decide later in which order to run them. Here a list of three actions is built (printing nothing), then run in reverse; each action also increments a mutable reference created with `IO.mkRef`:

```
/-- Actions are values: we can put them in a list, reorder them, and run them
later. A mutable reference (`IO.mkRef`) is an effect of `IO` too. -/
def countdown : IO Nat := do
  let counter ← IO.mkRef 0
  let steps : List (IO Unit) :=
    [1, 2, 3].map fun i => do
      IO.println s!"step {i}"
      counter.modify (· + 1)
  for step in steps.reverse do
    step
  counter.get
```

```
/--
info: step 3
step 2
step 1
---
info: 3
-/
#guard_msgs in
#eval countdown
```

The messages come out in the order the actions were *run*, not the order in which they were *built*, and the final `3` is the value of the whole action. A Rust function call performs its effects; a Lean `IO` value only describes them.

What is an `IO` action, then? Lean's own definitions answer:

```
/-- info: @[reducible] def IO : Type → Type :=
EIO IO.Error -/
#guard_msgs in
#print IO

/-- info: def EIO : Type → Type → Type :=
fun ε α => EST ε IO.RealWorld α -/
#guard_msgs in
#print EIO

/-- info: def EST : Type → Type → Type → Type :=
fun ε σ α => Void σ → EST.Out ε σ α -/
#guard_msgs in
#print EST
```

`IO` is `EIO IO.Error`, `EIO ε` is `EST ε IO.RealWorld`, and an `EST ε σ α` is a function that takes a token of type `Void σ` and returns an `EST.Out ε σ α`: a result or an error, each paired with a new token. So `IO` is a state monad with exceptions, whose "state" is a token standing for the whole world (`IO.RealWorld`). The token carries no data; passing it from step to step forces the steps to happen in order. (This `Void σ` is an opaque token type, unrelated to the empty `Void` of Chapter 10.) The most impure-looking monad of all is, in its type, the state monad you are about to build, plus errors.

20.7 A monad class from scratch

Lean's `Monad` and `LawfulMonad` split the operations from the laws and carry extra structure. To see the essentials, we build our own class, with the laws as fields, in `code/lean/Book/Ch20/Monad.lean`. Inside `namespace Ch20` the name `Monad` refers to this class, which shadows Lean's (still available as `_root_.Monad`); that file does not use `do` notation, so we write `Monad.bind` and `Monad.pure` explicitly.

The class and three instances

```
/-- A monad: a type constructor `M` with two operations and three laws. -/
class Monad (M : Type → Type) where
  /-- Wrap a plain value in a computation that has no effect. -/
  pure : {α : Type} → α → M α
  /-- Run `m`, then feed its result to `f`. -/
  bind : {α β : Type} → M α → (α → M β) → M β
  /-- Left identity: binding a pure value is just function application. -/
  pure_bind : ∀ {α β : Type} (a : α) (f : α → M β), bind (pure a) f = f a
  /-- Right identity: binding into `pure` changes nothing. -/
```

```
  bind_pure : ∀ {α : Type} (m : M α), bind m pure = m
  /-- Associativity: only the order of the steps matters, not their nesting. -/
  bind_assoc : ∀ {α β γ : Type} (m : M α) (f : α → M β) (g : β → M γ),
    bind (bind m f) g = bind m (fun a => bind (f a) g)
```

This is the mathematical definition, word for word, in the style of the `Category` class of Chapter 16: two operations, then three fields whose types are the laws. An instance must supply the operations *and* the proofs. `Option` first:

```
/-- `Option` is the monad of computations that may fail without a reason. -/
instance optionMonad : Monad Option where
  pure a := some a
  bind m f :=
    match m with
    | none => none
    | some a => f a
  pure_bind _ _ := rfl
  bind_pure m := by
    cases m <;> rfl
  bind_assoc m _ _ := by
    cases m <;> rfl
```

Left identity holds by `rfl`: `bind (pure a) f` is a `match` on `some a`, which computes to `f a`. The other two laws need a case split on `m`, because a `match` on an unknown `m` cannot compute. `cases m` produces two goals, one for `none` and one for `some a`, and the combinator `<;>` runs the tactic after it, `rfl`, on every goal that `cases` produced. In each case both sides compute to the same value: for `bind_assoc`, both sides are `none` in the first case and `bind (f a) g` in the second. `Except ε` is identical except for the constructor names:

```
/-- `Except ε` is the monad of computations that may fail with an error of
type `ε`: Lean's `Result`. -/
instance exceptMonad (ε : Type) : Monad (Except ε) where
  pure a := .ok a
  bind m f :=
    match m with
    | .error e => .error e
    | .ok a => f a
  pure_bind _ _ := rfl
  bind_pure m := by
    cases m <;> rfl
  bind_assoc m _ _ := by
    cases m <;> rfl
```

Lists need induction. We write `bind` for lists ourselves (it is `flatMap`, the `flat_map` of Rust) and prove the lemma that associativity needs: `bind` distributes over concatenation.

```
/-- `bind` for lists: apply `f` to every element and concatenate the results.
(This is `List.flatMap`, Rust's `flat_map`; we write it out to prove things
about it.) -/
def listBind {α β : Type} : List α → (α → List β) → List β
  | [], _ => []
  | a :: as, f => f a ++ listBind as f

/-- `listBind` distributes over `++`. -/
theorem listBind_append {α β : Type} (xs ys : List α) (f : α → List β) :
    listBind (xs ++ ys) f = listBind xs f ++ listBind ys f := by
  induction xs with
  | nil => rfl
  | cons x xs ih =>
    -- Goal: listBind (x :: xs ++ ys) f = listBind (x :: xs) f ++ listBind ys f
    simp only [List.cons_append, listBind, ih, List.append_assoc]
```

The proof is by induction on `xs`. The empty case holds by computation. In the `cons` case, `simp only` rewrites with four facts: `List.cons_append` moves the head out of the concatenation, `(x :: xs) ++ ys = x :: (xs ++ ys)`; the equations of `listBind` unfold one step on both sides; the induction hypothesis `ih` replaces `listBind (xs ++ ys) f`; and `List.append_assoc` regroups `f x ++ (listBind xs f ++ listBind ys f)` so that both sides become identical. With the lemma, the instance follows:

```
/-- `List` is the monad of nondeterministic computations: a computation returns
every possible result. -/
instance listMonad : Monad List where
  pure a := [a]
  bind := listBind
  pure_bind a f := by
    -- listBind [a] f = f a ++ [] = f a
    simp [listBind]
  bind_pure m := by
    induction m with
    | nil => rfl
    | cons a as ih => simp [listBind, ih]
  bind_assoc m f g := by
    induction m with
    | nil => rfl
    | cons a as ih => simp [listBind, listBind_append, ih]
```

`pure_bind` needs only `simp [listBind]`: `listBind [a] f` unfolds to `f a ++ []`, and `simp` knows that appending `[]` changes nothing. The other two laws go by induction on `m`. Here are the instances at work on the examples of the Rust program:

```
/-- A quantity parsed from text: fails on anything that is not a number. -/
def parseQty (s : String) : Option Nat := s.toNat?

/-- A quantity is accepted only between 1 and 100. -/
def checkQty (n : Nat) : Option Nat := if 1 ≤ n ∧ n ≤ 100 then some n else none

/-- info: (some 12, none, none) -/
#guard_msgs in
#eval (Monad.bind (parseQty "12") checkQty,
       Monad.bind (parseQty "twelve") checkQty,
       Monad.bind (parseQty "500") checkQty)

/-- Every warehouse that has `item`, nondeterministically. -/
def warehouses : String → List String
  | "bolts" => ["Hamburg", "Lyon"]
  | "nuts" => ["Lyon"]
  | _ => []

/-- Every carrier that collects from a warehouse. -/
def carriers : String → List String
  | "Hamburg" => ["DHL", "UPS"]
  | "Lyon" => ["Geodis"]
  | _ => []

/-- info: ["DHL", "UPS", "Geodis"] -/
#guard_msgs in
#eval Monad.bind (warehouses "bolts") carriers
```

Derived operations and their laws

`map`, `join` and Kleisli composition can now be defined once, for every instance of the class:

```
/-- Every monad is a functor: `map f m` runs `m` and applies `f` to the result. -/
def map (f : α → β) (m : M α) : M β :=
```

```
  Monad.bind m (fun a => Monad.pure (f a))

/-- Flatten a computation that returns a computation. -/
def join (mm : M (M α)) : M α :=
  Monad.bind mm (fun m => m)

/-- Kleisli composition: first `f`, then `g`, effects included. (Haskell and
Lean's own library write it `f >=> g`.) -/
def kcomp (f : α → M β) (g : β → M γ) : α → M γ :=
  fun a => Monad.bind (f a) g
```

The claims of the first part of the chapter become theorems. The functor laws of Chapter 17 hold for `map`:

```
/-- The functor laws of Chapter 17 follow from the monad laws. -/
theorem map_id (m : M α) : map (fun a => a) m = m :=
  Monad.bind_pure m

theorem map_comp (f : α → β) (g : β → γ) (m : M α) :
    map (fun a => g (f a)) m = map g (map f m) := by
  -- Unfold `map`; the right side is a bind of a bind.
  simp only [map]
  -- Reassociate, then the inner `bind (pure (f a)) _` is a left identity.
  rw [Monad.bind_assoc]
  simp only [Monad.pure_bind]
```

`map_id` is right identity in disguise: `map (fun a => a) m` *is* `bind m (fun a => pure a)`, which Lean accepts as `bind m pure`, and the proof term is the law itself. For `map_comp`, `simp only [map]` unfolds `map` everywhere, `rw [Monad.bind_assoc]` reassociates the double `bind` on the right-hand side, and the inner `bind (pure (f a)) _` that appears is removed by left identity. The equivalence of the `bind` and `join` presentations, and the three laws of `join` from the diagram above, are proved the same way:

```
/-- `bind` is `map` followed by `join`: the categorical definition of a monad
uses `pure`, `map` and `join` instead of `pure` and `bind`. -/
theorem bind_eq_join_map (m : M α) (f : α → M β) :
    Monad.bind m f = join (map f m) := by
  -- Right side: bind (bind m (fun a => pure (f a))) (fun m => m)
  simp only [join, map]
  -- Reassociate: bind m (fun a => bind (pure (f a)) (fun m => m))
  rw [Monad.bind_assoc]
  -- Left identity turns the inner bind into f a.
  simp only [Monad.pure_bind]

/-- Unit laws for `join`: wrapping with `pure` outside or inside, then
flattening, gives back what we started with. -/
theorem join_pure (m : M α) : join (Monad.pure m) = m :=
  Monad.pure_bind m (fun m => m)

theorem join_map_pure (m : M α) : join (map Monad.pure m) = m := by
  rw [← bind_eq_join_map, Monad.bind_pure]

/-- Associativity of `join`: a three-layer computation can be flattened from
the outside or from the inside. -/
theorem join_join (mmm : M (M (M α))) : join (join mmm) = join (map join mmm) := by
  simp only [join, map]
  -- Reassociate the outer bind on each side.
  rw [Monad.bind_assoc, Monad.bind_assoc]
  -- On the right, `bind (pure (join mm)) id` is `join mm` by left identity.
  simp only [Monad.pure_bind]
```

`join_map_pure` shows a useful trick: rewriting *backwards* with `← bind_eq_join_map` turns `join (map pure m)` into `bind m pure`, and then right identity finishes. Finally, the three laws restated for Kleisli composition:

```
/-- The three monad laws, restated for Kleisli composition: `pure` is a left
and a right identity, and `kcomp` is associative. -/
theorem pure_kcomp (f : α → M β) : kcomp Monad.pure f = f := by
  funext a
  exact Monad.pure_bind a f

theorem kcomp_pure (f : α → M β) : kcomp f Monad.pure = f := by
  funext a
  exact Monad.bind_pure (f a)

theorem kcomp_assoc (f : α → M β) (g : β → M γ) (h : γ → M δ) :
    kcomp (kcomp f g) h = kcomp f (kcomp g h) := by
  funext a
  exact Monad.bind_assoc (f a) g h
```

Each statement is an equation between functions, so the proof starts with `funext a` (two functions are equal if they agree on every argument, as in Chapter 16) and finishes with the corresponding monad law applied at `a`. The library's lawful monads are instances of our class too:

```
/-- Every monad of Lean's library that comes with a `LawfulMonad` instance
gives an instance of our class: the laws are already proved there. -/
abbrev Monad.ofLawful (M : Type → Type) [_root_.Monad M] [LawfulMonad M] : Monad M where
  pure := Pure.pure
  bind := Bind.bind
  pure_bind := LawfulMonad.pure_bind
  bind_pure := _root_.bind_pure
  bind_assoc := LawfulMonad.bind_assoc
```

(`abbrev` declares a definition that Lean unfolds eagerly, as instance-like definitions should be.) It is not an `instance`, so that it does not compete with the instances above; use it as `Monad.ofLawful (StateM Nat)`.

State, Reader and Writer

The function-shaped monads are in `code/lean/Book/Ch20/State.lean`. The state monad first:

```
/-- A computation with a state of type `σ`: given the current state, it returns
a result and the new state. -/
def State (σ α : Type) : Type := σ → α × σ

/-- Run a stateful computation from an initial state. -/
def State.run {σ α : Type} (m : State σ α) (s : σ) : α × σ := m s
```

```
instance stateMonad (σ : Type) : Monad (State σ) where
  pure a := fun s => (a, s)
  bind m f := fun s =>
    let (a, s') := m s
    f a s'
  -- All three laws hold by computation: both sides of each equation unfold to
  -- the same function.
  pure_bind _ _ := rfl
  bind_pure _ := rfl
  bind_assoc _ _ _ := rfl
```

All three laws are proved by `rfl`. That may look like cheating, so take associativity: both sides are functions of a state `s` that run `m` from `s`, then `f` from the resulting state, then `g` from the state after that; Lean unfolds the definitions and finds the same function. For right identity Lean also uses the *eta rule for pairs*: a pair `p` is definitionally equal to `(p.1, p.2)`. (The Rust `State` could not even be tested with `assert_eq!`: closures cannot be compared.)

Two primitive operations read and replace the state, and they combine into `fresh`, which hands out numbers from a counter. With it, the labeling program of the Rust example:

```
/-- Read the state. -/
def get {σ : Type} : State σ σ := fun s => (s, s)

/-- Replace the state. -/
def set {σ : Type} (s : σ) : State σ Unit := fun _ => ((), s)

/-- Return a fresh number from a counter and advance the counter. -/
def fresh : State Nat Nat :=
  Monad.bind get (fun n => Monad.bind (set (n + 1)) (fun _ => Monad.pure n))
```

```
/-- Give every order line a fresh ticket number, in order. -/
def label : List String → State Nat (List (Nat × String))
  | [] => Monad.pure []
  | x :: xs =>
    Monad.bind fresh (fun n =>
      Monad.bind (label xs) (fun rest =>
        Monad.pure ((n, x) :: rest)))

/-- info: ([(100, "bolts"), (101, "nuts"), (102, "washers")], 103) -/
#guard_msgs in
#eval (label ["bolts", "nuts", "washers"]).run 100
```

The Rust program checked the counter for one input; in Lean we prove what it is for every input:

```
/-- Labeling `xs` advances the counter by exactly `xs.length`, whatever the
starting value. -/
theorem label_counter (xs : List String) (n : Nat) :
    ((label xs).run n).2 = n + xs.length := by
  induction xs generalizing n with
  | nil => rfl
  | cons x xs ih =>
    -- Running `label (x :: xs)` from `n` runs `label xs` from `n + 1`.
    show ((label xs).run (n + 1)).2 = n + (x :: xs).length
    rw [ih (n + 1), List.length_cons]
    omega
```

The induction is on the list, and `generalizing n` makes the induction hypothesis hold for *every* starting value of the counter (we need it for `n + 1`, not for `n`). In the `cons` case, `show` replaces the goal by an equivalent one that Lean can check by computation: running `label (x :: xs)` from `n` runs `fresh` (which moves the counter to `n + 1`) and then `label xs` from `n + 1`, and the final `pure` leaves the counter alone. Then the induction hypothesis at `n + 1` and `List.length_cons` turn the goal into `n + 1 + xs.length = n + (xs.length + 1)`, which `omega` closes.

The reader and writer monads complete the family:

```
/-- A computation that reads an environment of type `ρ`. -/
def Reader (ρ α : Type) : Type := ρ → α

instance readerMonad (ρ : Type) : Monad (Reader ρ) where
  pure a := fun _ => a
  bind m f := fun r => f (m r) r
  pure_bind _ _ := rfl
  bind_pure _ := rfl
  bind_assoc _ _ _ := rfl

/-- Ask for the environment. -/
def ask {ρ : Type} : Reader ρ ρ := fun r => r

/-- Pricing parameters, read by every step but changed by none. -/
```

```
structure Pricing where
  unitPrice : Nat
  shipping : Nat

def subtotal (qty : Nat) : Reader Pricing Nat :=
  Monad.bind ask (fun p => Monad.pure (qty * p.unitPrice))

def total (qty : Nat) : Reader Pricing Nat :=
  Monad.bind (subtotal qty) (fun s =>
    Monad.bind ask (fun p => Monad.pure (s + p.shipping)))

/-- info: 85 -/
#guard_msgs in
#eval total 3 { unitPrice := 25, shipping := 10 }
```

```
/-- A computation that also produces a log. -/
def Writer (α : Type) : Type := α × List String

instance writerMonad : Monad Writer where
  pure a := (a, [])
  bind m f :=
    let (b, w) := f m.1
    (b, m.2 ++ w)
  -- Each law of the monad is a law of the log: `[]` is a unit for `++`,
  -- and `++` is associative.
  pure_bind a f := by
    show ((f a).1, [] ++ (f a).2) = f a
    rw [List.nil_append]
    -- Goal: ((f a).1, (f a).2) = f a, true by the eta rule for pairs.
    rfl
  bind_pure m := by
    show (m.1, m.2 ++ []) = m
    rw [List.append_nil]
    rfl
  bind_assoc m f g := by
    show ((g (f m.1).1).1, (m.2 ++ (f m.1).2) ++ (g (f m.1).1).2)
       = ((g (f m.1).1).1, m.2 ++ ((f m.1).2 ++ (g (f m.1).1).2))
    rw [List.append_assoc]

/-- Add a line to the log. -/
def tell (line : String) : Writer Unit := ((), [line])

def reserveLogged (item : String) (qty : Nat) : Writer Nat :=
  Monad.bind (tell s!"reserve {qty} x {item}") (fun _ => Monad.pure qty)

/-- info: (5, ["reserve 2 x bolts", "reserve 3 x nuts"]) -/
#guard_msgs in
#eval Monad.bind (reserveLogged "bolts" 2) (fun a =>
        Monad.bind (reserveLogged "nuts" 3) (fun b => Monad.pure (a + b)))
```

The reader laws hold by `rfl`. The writer laws are more instructive. In each proof, `show` spells out what the two sides compute to, and what remains is a fact about the log: `[] ++ w = w`, `w ++ [] = w`, and the associativity of `++` (the final `rfl` uses the eta rule for pairs). The three monad laws are the three monoid laws of the log. Any monoid would do as a log type (Mathlib's `WriterT` accepts one), and this is the "monoid in the category of endofunctors" slogan in miniature.

20.8 The Kleisli category

Now we build the category of effectful functions on the book's own `Category` class, in `code/lean/Book/Ch20/Kleisli.lean`, which imports `Book.Ch16.Category`. As with the opposite category

of Chapter 16, the objects are wrapped in a structure, so that this category does not clash with `typeCategory`:

```
/-- An object of the Kleisli category of `M`: a Lean type. The wrapper keeps
this category apart from `typeCategory`, whose objects are also types. -/
structure Kleisli (M : Type → Type) where
  /-- The underlying type. -/
  base : Type
```

```
/-- The Kleisli category of a monad: a morphism from `a` to `b` is an effectful
function `a.base → M b.base`, the identity is `pure` and composition is
Kleisli composition. -/
instance kleisliCategory (M : Type → Type) [Monad M] : Category (Kleisli M) where
  Hom a b := a.base → M b.base
  id _ := Monad.pure
  comp g f := kcomp f g
  -- `id ⊚ f = f`: binding into `pure` changes nothing (right identity).
  id_comp f := kcomp_pure f
  -- `f ⊚ id = f`: binding a pure value is application (left identity).
  comp_id f := pure_kcomp f
  -- `h ⊚ (g ⊚ f) = (h ⊚ g) ⊚ f`: associativity of bind.
  assoc h g f := kcomp_assoc f g h
```

A morphism from `a` to `b` is a Kleisli arrow `a.base → M b.base`, the identity is `pure`, and `comp g f` (first `f`, then `g`, in the order of Chapter 16) is `kcomp f g`. Each category law is discharged by one of the Kleisli-form theorems, and each of those came from one monad law. Two of these categories are worth naming:

```
/-- The Kleisli category of `Option`: types and partial functions. -/
abbrev Partial := Kleisli Option

def text : Partial := ⟨String⟩
def qty : Partial := ⟨Nat⟩

/-- Two partial functions, as morphisms. -/
def parseM : text → qty := parseQty
def checkM : qty → qty := checkQty

/-- Their composite, written with the category's `⊚`: first parse, then check. -/
def readQtyM : text → qty := checkM ⊚ parseM

/-- info: [some 12, none, none] -/
#guard_msgs in
#eval ["12", "twelve", "500"].map readQtyM

/-- The Kleisli category of `List`: types and nondeterministic functions,
which are relations. -/
def item : Kleisli List := ⟨String⟩
def site : Kleisli List := ⟨String⟩

def stockedAt : item → site := warehouses
def collectedBy : site → site := carriers

/-- info: ["DHL", "UPS", "Geodis"] -/
#guard_msgs in
#eval (collectedBy ⊚ stockedAt) "bolts"
```

The Kleisli category of `Option` is the category of types and *partial functions*, and `checkM ⊚ parseM` is the composite that Rust's `compose_option` built by hand. The Kleisli category of `List` is the category of types and *nondeterministic functions*, which are relations: `stockedAt` relates an item to every warehouse that has it. (`⟨String⟩` is the anonymous constructor notation, building an object whose `base` is `String`. The type `text → qty` unfolds to `String → Option Nat`, so `parseQty` itself is the morphism.)

The instance shows that the monad laws *imply* the category laws. They are in fact equivalent, and to state the converse we need the operations without the laws:

```
/-- The operations of a monad without the laws. -/
structure MonadOps (M : Type → Type) where
  pure : {α : Type} → α → M α
  bind : {α β : Type} → M α → (α → M β) → M β

/-- Kleisli composition, defined from the raw operations. -/
def MonadOps.kcomp {M : Type → Type} (ops : MonadOps M) {α β γ : Type}
    (f : α → M β) (g : β → M γ) : α → M γ :=
  fun a => ops.bind (f a) g

/-- The monad laws, stated with `bind`. -/
def MonadOps.BindLaws {M : Type → Type} (ops : MonadOps M) : Prop :=
  (∀ {α β : Type} (a : α) (f : α → M β), ops.bind (ops.pure a) f = f a) ∧
  (∀ {α : Type} (m : M α), ops.bind m ops.pure = m) ∧
  (∀ {α β γ : Type} (m : M α) (f : α → M β) (g : β → M γ),
    ops.bind (ops.bind m f) g = ops.bind m (fun a => ops.bind (f a) g))

/-- The category laws of the Kleisli category, stated with `kcomp`. -/
def MonadOps.KleisliLaws {M : Type → Type} (ops : MonadOps M) : Prop :=
  (∀ {α β : Type} (f : α → M β), ops.kcomp ops.pure f = f) ∧
  (∀ {α β : Type} (f : α → M β), ops.kcomp f ops.pure = f) ∧
  (∀ {α β γ δ : Type} (f : α → M β) (g : β → M γ) (h : γ → M δ),
    ops.kcomp (ops.kcomp f g) h = ops.kcomp f (ops.kcomp g h))
```

```
/-- The monad laws hold if and only if Kleisli composition satisfies the laws of
a category. -/
theorem bindLaws_iff_kleisliLaws {M : Type → Type} (ops : MonadOps M) :
    ops.BindLaws ↔ ops.KleisliLaws := by
  constructor
  · -- From bind laws to category laws: apply each law pointwise.
    intro ⟨pure_bind, bind_pure, bind_assoc⟩
    refine ⟨?_, ?_, ?_⟩
    · intro α β f
      funext a
      exact pure_bind a f
    · intro α β f
      funext a
      exact bind_pure (f a)
    · intro α β γ δ f g h
      funext a
      exact bind_assoc (f a) g h
  · -- From category laws to bind laws: a computation `m : M α` is the
    -- Kleisli arrow `fun () => m` applied to `()`.
    intro ⟨pure_kcomp, kcomp_pure, kcomp_assoc⟩
    refine ⟨?_, ?_, ?_⟩
    · intro α β a f
      exact congrFun (pure_kcomp f) a
    · intro α m
      exact congrFun (kcomp_pure (fun (_ : Unit) => m)) ()
    · intro α β γ m f g
      exact congrFun (kcomp_assoc (fun (_ : Unit) => m) f g) ()
```

`constructor` splits the `↔` into two implications. In the first, the destructuring `intro ⟨pure_bind, bind_pure, bind_assoc⟩` names the three bind laws, `refine ⟨?_, ?_, ?_⟩` leaves one goal for each Kleisli law, and each goal is the corresponding bind law applied pointwise after `funext`. The converse contains the one real idea. A Kleisli law talks about functions, but a bind law talks about a computation `m : M α`. The trick is to see `m` as a Kleisli arrow out of the unit type, `fun () => m`, applied to `()`. Then `ops.bind m f` is `ops.kcomp (fun () => m) f ()` by definition, and every bind law is a Kleisli law applied at `()`. `congrFun h x` turns an equation between functions `h : g₁ = g₂` into

the equation `g₁ x = g₂ x` between their values. (The left identity case does not even need the trick: `congrFun` at `a` suffices.)

20.9 Putting it together: an order workflow

Real programs have more than one effect. An order-processing step may fail (an item is unknown or out of stock) *and* it updates a stock (state). Lean combines effects with **monad transformers**: a transformer takes a monad `m` and adds one effect to it. The code is in `code/lean/Book/Ch20/Transformers.lean`, and the definitions are short enough to state as equations that Lean checks by `rfl`:

```
example (σ α : Type) (m : Type → Type) : StateT σ m α = (σ → m (α × σ)) := rfl
example (ε α : Type) (m : Type → Type) : ExceptT ε m α = m (Except ε α) := rfl
example (ρ α : Type) (m : Type → Type) : ReaderT ρ m α = (ρ → m α) := rfl
example (σ α : Type) : StateM σ α = StateT σ Id α := rfl
```

`StateT σ m` is the state monad with `m`'s effect on each step: a function from a state to an `m`-computation of a result and a new state. `ExceptT ε m` is an `m`-computation of a result that may be an error. `ReaderT ρ m` adds an environment. `StateM σ`, the plain state monad, is `StateT σ` over `Id`, the monad with no effect at all.

One workflow, written once

The domain is a small warehouse:

```
/-- What can go wrong with an order. -/
inductive OrderError where
  | emptyOrder
  | unknownItem (item : String)
  | outOfStock (item : String) (wanted available : Nat)
  deriving Repr

/-- The stock of a warehouse: item names and the quantities on hand. -/
abbrev Stock := List (String × Nat)

/-- Set the quantity of `item` to `n`. -/
def Stock.update (stock : Stock) (item : String) (n : Nat) : Stock :=
  stock.map fun (i, q) => if i == item then (i, n) else (i, q)
```

The workflow does not commit to a particular stack of transformers. It is written against two classes: `MonadStateOf Stock m` ("`m` has a state of type `Stock`", providing `get` and `set`) and `MonadExceptOf OrderError m` ("`m` can fail with an `OrderError`", providing `throw` and `try ... catch`). They play the role of trait bounds, in the dictionary-passing sense of Chapter 14:

```
/-- Take `qty` units of `item` out of the stock. The function works in every
monad `m` that has a state of type `Stock` and errors of type `OrderError`. -/
def reserve {m : Type → Type} [Monad m] [MonadStateOf Stock m]
    [MonadExceptOf OrderError m] (item : String) (qty : Nat) : m Unit := do
  let stock ← get
  match stock.lookup item with
  | none => throw (.unknownItem item)
  | some available =>
    if qty ≤ available then
      set (stock.update item (available - qty))
    else
      throw (.outOfStock item qty available)
```

```
/-- Reserve every line of an order; return the number of units. -/
def placeOrder {m : Type → Type} [Monad m] [MonadStateOf Stock m]
    [MonadExceptOf OrderError m] (lines : List (String × Nat)) : m Nat := do
  if lines.isEmpty then
    throw .emptyOrder
  for (item, qty) in lines do
    reserve item qty
  return (lines.map (·.2)).sum
```

(`List.lookup` returns the quantity paired with an item, if there is one, and `(·.2)` is a function that takes the second component of a pair: the `·` marks the argument.) Now there are two natural ways to stack the effects:

```
/-- State outside, errors inside: a failed run returns no state at all. -/
abbrev AllOrNothing := StateT Stock (Except OrderError)

/-- Errors outside, state inside: a failed run still returns the state. -/
abbrev KeepGoing := ExceptT OrderError (StateM Stock)

example (α : Type) : AllOrNothing α = (Stock → Except OrderError (α × Stock)) := rfl
example (α : Type) : KeepGoing α = (Stock → Except OrderError α × Stock) := rfl
```

The `rfl` equations show what the two stacks *are*. In `AllOrNothing`, a run returns either an error, or a result with the new stock. In `KeepGoing`, a run always returns a stock, with either an error or a result. The types already tell you what happens when an order fails halfway, and the runs confirm it:

```
def warehouse : Stock := [("bolts", 10), ("nuts", 4)]

def good : List (String × Nat) := [("bolts", 3), ("nuts", 2)]
def bad : List (String × Nat) := [("bolts", 3), ("nuts", 9)]

/-- info: Except.ok (5, [("bolts", 7), ("nuts", 2)]) -/
#guard_msgs in
#eval (placeOrder good : AllOrNothing Nat).run warehouse

/-- info: Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4) -/
#guard_msgs in
#eval (placeOrder bad : AllOrNothing Nat).run warehouse

/-- info: (Except.ok 5, [("bolts", 7), ("nuts", 2)]) -/
#guard_msgs in
#eval (placeOrder good : KeepGoing Nat).run.run warehouse

/--
info: (Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4), [("bolts", 7), ("nuts",
4)])
-/
#guard_msgs in
#eval (placeOrder bad : KeepGoing Nat).run.run warehouse
```

The good order gives the same result in both stacks. The bad order reserves three bolts, then fails on the nuts. In `AllOrNothing` the failure discards the state, and the reservation of the bolts with it. In `KeepGoing` the caller gets the error *and* a stock in which the bolts are reserved, though the order failed. Transformers do not commute: the order of the stack is a design decision about what a failure means.

If you want `KeepGoing`'s shape but all-or-nothing behavior, restore the stock yourself:

```
/-- Run `action`; if it fails, put the stock back the way it was and fail with
the same error. -/
def atomically {m : Type → Type} {α : Type} [Monad m] [MonadStateOf Stock m]
```

```
    [MonadExceptOf OrderError m] (action : m α) : m α := do
  let saved ← get
  try
    action
  catch e =>
    set saved
    throw e

/--
info: (Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4), [("bolts", 10), ("nuts",
4)])
-/
#guard_msgs in
#eval (atomically (placeOrder bad) : KeepGoing Nat).run.run warehouse
```

`try ... catch e => ...` runs the handler if the action throws; the handler puts the saved stock back and rethrows. The bad order now leaves ten bolts in the warehouse.

Finally, code written for a smaller monad can be used in a bigger one. `checkLines` lives in plain `Except OrderError`, and `placeChecked` uses it inside `AllOrNothing` without any conversion:

```
/-- A check that needs only errors, written in the plain `Except` monad. -/
def checkLines (lines : List (String × Nat)) : Except OrderError Unit :=
  if lines.isEmpty then throw .emptyOrder else pure ()

/-- Inside the stack, the `Except` computation is lifted automatically. -/
def placeChecked (lines : List (String × Nat)) : AllOrNothing Nat := do
  checkLines lines
  for (item, qty) in lines do
    reserve item qty
  return (lines.map (·.2)).sum

/-- info: Except.error (Ch20.Shop.OrderError.emptyOrder) -/
#guard_msgs in
#eval (placeChecked []).run warehouse
```

Lean inserts the translation automatically, through instances of a class `MonadLift` that says how to run an `m`-computation inside an `n`-computation. This is the automatic version of Rust's `.ok_or(...)?`.

The same workflow in Rust

In Rust, the stock is a `&mut` borrow and failure is a `Result` with `?` (`code/rust/examples/ch20_order_workflow.rs`):

```
#[derive(Debug, PartialEq)]
enum OrderError {
    EmptyOrder,
    UnknownItem(String),
    OutOfStock {
        item: String,
        wanted: u32,
        available: u32,
    },
}

/// Item names and quantities on hand. A `BTreeMap` prints in a fixed order.
type Stock = BTreeMap<String, u32>;
```

```
/// Take `qty` units of `item` out of the stock.
fn reserve(stock: &mut Stock, item: &str, qty: u32) -> Result<(), OrderError> {
    let available = stock
        .get_mut(item)
        .ok_or_else(|| OrderError::UnknownItem(item.to_string()))?;
```

```
    if qty > *available {
        return Err(OrderError::OutOfStock {
            item: item.to_string(),
            wanted: qty,
            available: *available,
        });
    }
    *available -= qty;
    Ok(())
}

/// Reserve every line of an order; return the number of units.
fn place_order(stock: &mut Stock, lines: &[(&str, u32)]) -> Result<u32, OrderError> {
    if lines.is_empty() {
        return Err(OrderError::EmptyOrder);
    }
    for &(item, qty) in lines {
        reserve(stock, item, qty)?;
    }
    Ok(lines.iter().map(|&(_, qty)| qty).sum())
}
```

```
/// All or nothing: work on a copy of the stock and commit only on success.
fn place_order_atomically(
    stock: &mut Stock,
    lines: &[(&str, u32)],
) -> Result<u32, OrderError> {
    let mut draft = stock.clone();
    let units = place_order(&mut draft, lines)?;
    *stock = draft;
    Ok(units)
}
```

```
let mut stock = warehouse();
let result = place_order(&mut stock, &good);
println!("plain,  good order: {result:?}, stock {stock:?}");

let mut stock = warehouse();
let result = place_order(&mut stock, &bad);
println!("plain,  bad order : {result:?}, stock {stock:?}");

let mut stock = warehouse();
let result = place_order_atomically(&mut stock, &bad);
println!("atomic, bad order : {result:?}, stock {stock:?}");
```

```
plain,  good order: Ok(5), stock {"bolts": 7, "nuts": 2}
plain,  bad order : Err(OutOfStock { item: "nuts", wanted: 9, available: 4 }), stock
{"bolts": 7, "nuts": 4}
atomic, bad order : Err(OutOfStock { item: "nuts", wanted: 9, available: 4 }), stock
{"bolts": 10, "nuts": 4}
```

The results match Lean's line for line. Plain `place_order` behaves like `KeepGoing`: mutations made through `&mut` before the `?` returned are not undone. `place_order_atomically` behaves like `AllOrNothing`: it works on a copy and commits only after every step succeeded. Rust reaches both semantics with two ordinary features, mutation through a borrow and early return, and you choose by how you write the function. The price is that the type `fn(&mut Stock, ...) -> Result<u32, OrderError>` does not say which semantics you got, while `AllOrNothing` and `KeepGoing` do. That is the trade-off of the whole chapter: Rust's built-in, effect-specific features are cheaper and more flexible; Lean's monads are more explicit and more abstract, and they come with laws you can prove.

20.10 In Mathlib: monads and adjunctions

Mathlib defines monads in the categorical style, in `Mathlib.CategoryTheory.Monad.Basic`: a `CategoryTheory.Monad C` on a category `C` is an endofunctor `T : C ⥤ C` (Chapter 17) with natural transformations `η : 𝟭 C ⟶ T` and `μ : T ⋙ T ⟶ T` (Chapter 18; `𝟭 C` is the identity functor and `⋙` composes functors), satisfying the three equations of our diagram. The file `code/lean/Book/Ch20/InMathlib.lean` states them in a readable form:

```
variable {C : Type*} [Category C] (T : CategoryTheory.Monad C) (X : C)

/-- `T` is an endofunctor: it acts on objects and on morphisms. -/
example : C := T.obj X

/-- The unit `η : 𝟭 C ⟶ T` and the multiplication `μ : T ⋙ T ⟶ T` are natural
transformations (Chapter 18); here are their components at `X`. -/
example : X → T.obj X := T.η.app X
example : T.obj (T.obj X) → T.obj X := T.μ.app X

/-- The three laws, in diagrammatic order (`f ≫ g` is "first `f`, then `g`"). -/
example : T.map (T.μ.app X) ≫ T.μ.app X = T.μ.app (T.obj X) ≫ T.μ.app X :=
  T.assoc X
example : T.η.app (T.obj X) ≫ T.μ.app X = 𝟙 (T.obj X) := T.left_unit X
example : T.map (T.η.app X) ≫ T.μ.app X = 𝟙 (T.obj X) := T.right_unit X
```

Each law is a field of the structure (`T.assoc`, `T.left_unit`, `T.right_unit`), written in Mathlib's diagrammatic order, where `f ≫ g` means "first `f`, then `g`" (Chapter 16). A lawful Lean monad gives such a monad on the category `Type`:

```
/-- A lawful Lean monad, seen as a monad on the category `Type`. -/
def optionMonad : CategoryTheory.Monad Type := ofTypeMonad Option

example (α : Type) : optionMonad.obj α = Option α := rfl
```

Adjunctions give monads

Chapter 19 introduced adjunctions through currying: functions $A \times S \to B$ correspond exactly to functions $A \to (S \to B)$, naturally in A and B. In general, an adjunction $L \dashv R$ between functors $L : \mathcal{C} \to \mathcal{D}$ and $R : \mathcal{D} \to \mathcal{C}$ is a natural correspondence between morphisms $L\,A \to B$ in $\mathcal{D}$ and morphisms $A \to R\,B$ in $\mathcal{C}$. Every adjunction yields a monad on $\mathcal{C}$: the functor is the composite $T = R \circ L$, the unit $\eta : A \to R\,(L\,A)$ is what the correspondence assigns to the identity of $L\,A$, and μ is built from the opposite map $L\,(R\,B) \to B$ (the *counit*). Mathlib's `Adjunction.toMonad` performs this construction. Apply it to currying, and out comes a monad you know:

```
/-- The currying adjunction of Chapter 19 on `Type`:
`(S ⊗ A → B) ≃ (A → (S → B))`. -/
example (S : Type) : tensorLeft S ⊣ coyoneda.obj (Opposite.op S) :=
  Types.tensorProductAdjunction S

/-- Every adjunction gives a monad: this one gives the state monad. -/
noncomputable def stateMonad (S : Type) : CategoryTheory.Monad Type :=
  (Types.tensorProductAdjunction S).toMonad

/-- On objects: a computation is a morphism from states to (state, result)
pairs. -/
example (S α : Type) : (stateMonad S).obj α = (S → S ⊗ α) := rfl

/-- The unit pairs the value with the unchanged state: it is `pure`. -/
```

```
example (S α : Type) :
    (stateMonad S).η.app α = ↾(fun (a : α) => ↾(fun (s : S) => (s, a))) := rfl

/-- The multiplication runs the outer computation, then the inner one from the
new state: it is `join`. -/
example (S α : Type) :
    (stateMonad S).μ.app α =
      ↾(fun (k : S → S ⊗ (S → S ⊗ α)) => ↾(fun (s : S) => (k s).2 (k s).1)) := rfl
```

On `Type`, `tensorLeft S` is the functor $A \mapsto S \times A$ (written `S ⊗ A`, the state on the left), and `coyoneda.obj (Opposite.op S)` is the functor $B \mapsto (S \to B)$, written `S → B`. (Mathlib wraps the functions of its category `Type` in a structure; `↾f` turns a plain function into such a morphism.) The monad of the currying adjunction sends A to $S \to S \times A$: the state monad. The `rfl` proofs confirm that its unit is our `pure` and its multiplication is `join` for state: run the outer computation, then the inner computation it returned, from the new state. Its laws come for free from the adjunction.

The construction can be reversed: every monad arises from an adjunction, in fact from two canonical ones:

```
/-- Conversely, every monad comes from an adjunction: the free-algebra functor
is left adjoint to the forgetful functor of its Eilenberg–Moore category ... -/
example : T.free ⊣ T.forget := T.adj
/-- ... and the monad of that adjunction is `T` again, up to isomorphism. -/
example : T.adj.toMonad ≅ T := Adjunction.adjToMonadIso T
/-- The Kleisli category gives another such adjunction. -/
example : Kleisli.Adjunction.toKleisli T ⊣ Kleisli.Adjunction.fromKleisli T :=
  Kleisli.Adjunction.adj T
```

The first uses the *Eilenberg–Moore category* of `T`, whose objects are `T`-algebras (objects `X` with a map `T X → X` compatible with `η` and `μ`), and `adjToMonadIso` says that its monad is `T` again. The second uses the Kleisli category, and you can understand it right now: the left adjoint sends a type A to A as an object of the Kleisli category, the right adjoint sends A to $M\,A$, and the correspondence between Kleisli morphisms $A \to B$ and plain functions $A \to M\,B$ is the identity, because that is how Kleisli morphisms were defined. Monads and adjunctions are two sides of one idea.

20.11 Common confusions

"**A monad is a container.**" `Option` and `Vec` look like containers, and the metaphor helps for them. It fails for `State`, `Reader`, `IO` and futures: a state computation contains no value at all until you run it from some state. A monad is a way of *sequencing computations* that have a certain kind of effect; a container is one special case.

"**Monads make Lean impure.**" The opposite. A `State σ α` is a function, an `IO α` is a value that describes interactions, and every Lean function is still a mathematical function; the runtime runs `IO` actions outside the logic. In Rust, effects happen when the code runs, and the type `fn() -> u32` does not tell you whether the function prints, reads a file or panics.

"**`?` is just `and_then`.**" It is `bind` for the rest of the function body, plus an error conversion through `From`, plus an early `return`. That is why `?` inside a closure returns from the *closure*, not from the

function around it, and why one function body can use `?` for only one monad, the one named by its return type (error E0277).

"**The compiler checks the monad laws.**" Neither Rust nor Lean's `Monad` class does. Lean checks them only where you ask for it: a `LawfulMonad` instance, or a class like ours, requires proofs. Lean's `do` elaborator nonetheless assumes right identity, so an unlawful instance can behave differently from what its `do` blocks seem to say.

"**Monad transformers can be stacked in any order.**" They cannot: the order-processing example shows that `StateT σ (Except ε)` and `ExceptT ε (StateM σ)` give different answers to "what happens to the state when a step fails?".

20.12 Summary

- An effectful function from A to B is a Kleisli arrow $A \to M\,B$, where the type constructor M describes the effect: failure, errors, nondeterminism, state, environment, logging, input and output, suspension.
- A monad is a type constructor with `pure : A → M A` and `bind : M A → (A → M B) → M B` satisfying left identity, right identity and associativity. The laws say that inlining and extracting steps are safe refactorings.
- Equivalently, a monad is a functor with natural transformations η (`pure`) and μ (`join`) satisfying unit and associativity laws, with `bind` $=$ `map` then `join`. Equivalently again, Kleisli arrows form a category with `pure` as identity: the monad laws *are* the category laws, as proved in Lean in both directions.
- Rust uses monads constantly: `?` is `bind` for `Option` and `Result` (plus a `From` conversion), `and_then` and `flat_map` are `bind`, and `.await` is `bind` for futures, compiled into a state machine.
- Rust has no `Monad` trait: there are no higher-kinded types (a GAT family trait works but needs turbofishes), and `bind` must choose between `FnOnce` (which `Vec` cannot implement) and `FnMut` (which costs `Option` users and forces `Clone` bounds). Rust handles effects with dedicated syntax, `&mut` and parameters instead.
- Lean has `Monad`, `do` notation that desugars into nested `bind`s, and `LawfulMonad` for the laws. `IO` is a state-and-exception monad over a token that stands for the world.
- Our own class carries the laws as fields; the instances for `Option`, `Except`, `List`, `State`, `Reader` and `Writer` prove them, and the Kleisli category of any instance is an instance of Chapter 16's `Category`.
- Monad transformers (`StateT`, `ExceptT`, `ReaderT`) combine effects, and the order of the stack matters. Rust gets the same two behaviors from `&mut` and `?`, but its types do not say which one a function has.
- Every adjunction gives a monad (currying gives the state monad), and every monad comes from an adjunction (Eilenberg–Moore, Kleisli).

20.13 Exercises

1. **A counting monad (Lean).** Define `Counted α := α × Nat`, a writer monad whose log is the number of steps performed, as an instance of the class in `Monad.lean` (`pure a := (a, 0)`, and `bind` adds the counts). Prove the three laws, then define `tick : Counted Unit` and check with `#eval` that a three-step computation reports a count of 3. *Hint:* copy `writerMonad` and replace `[]` and `++` with `0` and `+`; the lemmas you need are `Nat.zero_add`, `Nat.add_zero` and `Nat.add_assoc`.
2. **Naturality of `pure` and `join` (Lean).** For an arbitrary instance of our class, prove `map f (Monad.pure a) = Monad.pure (f a)` and `map f (join mm) = join (map (map f) mm)`. These say that η and μ are natural transformations (Chapter 18). *Hint:* the first is one `simp only [map, Monad.pure_bind]`. For the second, unfold `map` and `join`, reassociate with `rw [Monad.bind_assoc]` (as often as needed), and use `Monad.pure_bind` to remove the `pure` steps.
3. **Writer in Rust.** Write `struct Writer<A> { value: A, log: Vec<String> }` with `pure`, `bind` and a `tell` operation, and check the three laws with `assert_eq!` on a few samples. Can your `bind` take an `FnOnce`? Why is that compatible with the discussion of `FnOnce` and `FnMut` in this chapter? *Hint:* `bind` calls the continuation exactly once, as for `Option`; derive `PartialEq` and `Debug` to compare results.
4. **Desugar by hand (Lean, pen and paper).** Write `boxes_result` from `ch20_monads.rs` as a Lean function in `Except String` with a `do` block. Write down, on paper, the term it desugars into, then check your answer with `set_option pp.notation false in #print`. *Hint:* `String.toNat?` returns an `Option`; turn it into an `Except` with a `match`, or with `.elim`. Every `let x ← e` becomes `bind e (fun x => ...)`.
5. **A third layer (Lean).** Add a limit on the number of units per order line to the workflow of `Transformers.lean`, read from an environment with `ReaderT`. Run the good and bad orders in `ReaderT Nat AllOrNothing` with limits 5 and 10, and explain the outputs. *Hint:* inside the new `reserve`, `let limit ← read` gives the environment; `get`, `set` and `throw` are lifted through `ReaderT` automatically; run with `((placeOrder good).run 10).run warehouse`.
6. **The price of `FnMut` (Rust).** Make `compile_fail/ch20_option_bind_fnmut.rs` compile without changing the signature of `bind_option`. Then explain, in terms of Chapter 8's closure traits and Chapter 10's remark that duplicating a value needs `Clone`, why your fix is necessary for an `FnMut` closure but not for an `FnOnce` one. *Hint:* the closure must still own a `String` after it runs; return `customer.clone()`.

PART V

Ownership as Logic

Part V returns to the structural rules of Part I and removes them, one at a time. Without contraction, an assumption can be used at most once; without weakening, it must be used. The resulting substructural logics, linear and affine, are logics of resources, and the affine λ-calculus built from them is the formal core of Rust's ownership: moves, `Copy`, `Clone` and `Drop` each find their place in it, with a type checker in Rust and the metatheory in Lean. The part ends with typestate, where the affine discipline turns protocols into types.

CHAPTER 21

Substructural Logics

Chapter 6 ended with a design decision. Ordinary logic treats an assumption as a fact, which may be used any number of times or not at all; the rules that permit this are the structural rules, exchange, weakening and contraction. If an assumption stands for a *resource* instead (a coin, a file handle, a heap buffer), some of those rules are wrong, and the way to build a logic of resources is to delete them. We now have the tools to carry that decision out.

This chapter opens Part V. It deletes the structural rules one by one and maps out the family of logics that results. It finds that, without contraction, the familiar connectives come apart: there are two conjunctions and two disjunctions, which ordinary logic had merged into one each. It reads the new connectives as Rust types, builds a proof search that decides which sequents survive in linear and affine logic, and proves in Lean that the most important one does not: from one A you cannot get two.

In this chapter you will learn:

- how deleting structural rules produces a family of logics: intuitionistic, affine, relevant, linear and ordered;
- why, without contraction, conjunction splits into a multiplicative $A \otimes B$ and an additive $A \,\&\, B$, and what each one means for resources;
- the sequent calculus of intuitionistic linear logic: context splitting versus context sharing, linear implication $\multimap$, the units, and the exponentials for orientation;
- to read Rust types as linear formulas: a tuple as $\otimes$, an offer with consuming methods as $A \,\&\, B$, an `enum` as $\oplus$, a `FnOnce` closure as $\multimap$;
- how a proof search for linear and affine logic works, and what it says about $A \vdash A \otimes A$, $A \otimes B \vdash A$ and $A \,\&\, B \vdash A$;
- a complete Lean proof that $A \vdash A \otimes A$ has no derivation, by an invariant that counts atoms.

21.1 The structural rules, again

Here are the three rules as Chapter 6 stated them:

$$\frac{\Gamma_1, A, B, \Gamma_2 \vdash C}{\Gamma_1, B, A, \Gamma_2 \vdash C}\ \textsf{Exch} \qquad \frac{\Gamma \vdash C}{\Gamma, A \vdash C}\ \textsf{Weak} \qquad \frac{\Gamma, A, A \vdash C}{\Gamma, A \vdash C}\ \textsf{Contr}$$

Exchange says the order of assumptions does not matter. Weakening, read from the conclusion up, discards an assumption; contraction duplicates one. Chapter 6 drew four lessons from them.

1. **Natural deduction has all three, admissibly.** None of them is a rule of Chapter 4's system, yet all three hold: Chapter 6 proved in Lean that derivability is monotone in the context.
2. **They hide in two places.** The hypothesis rule accepts any context that contains the formula, and rules with several premises hand the same context to all of them (contraction by sharing).
3. **They control usage.** Weakening introduces an assumption used zero times; contraction adds up the uses of two assumptions, and is the only rule that can push a count above one.
4. **Rust keeps some and refuses one.** Rust keeps exchange and weakening (an unused value is dropped) and refuses contraction for values that are not `Copy` (error E0382).

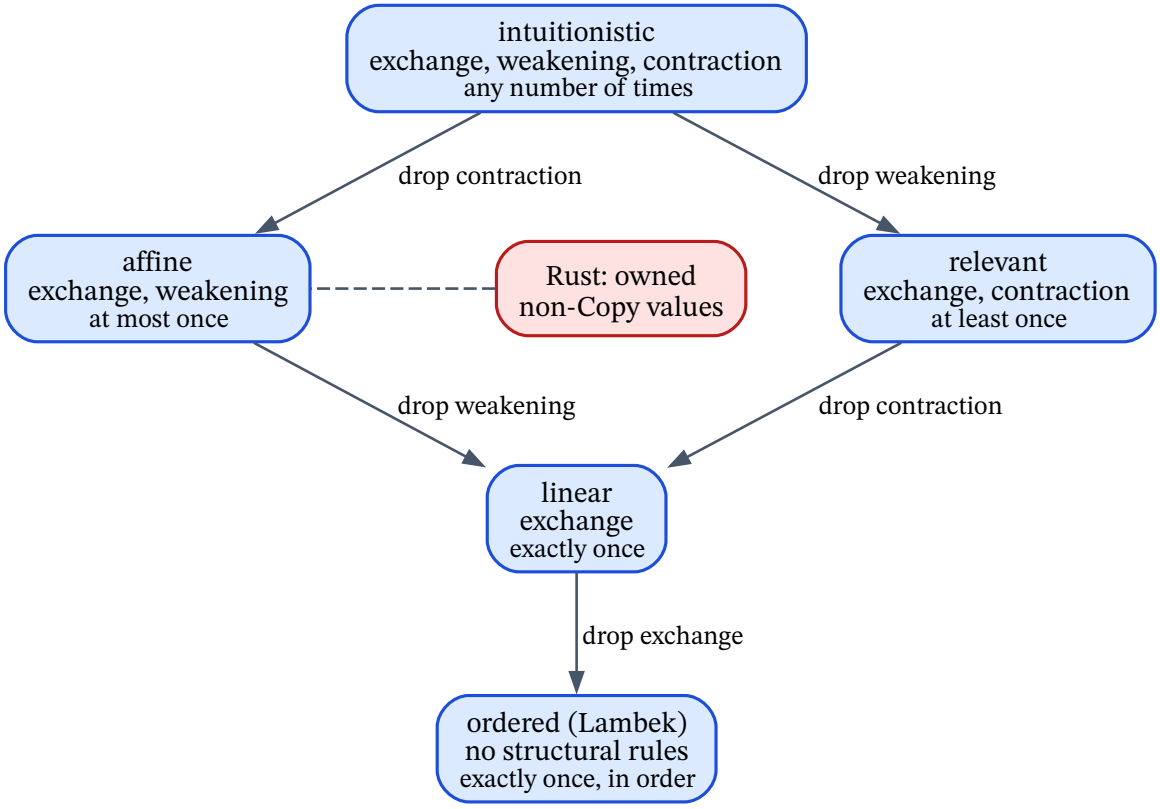
The second lesson is the warning label for this chapter. Deleting the rule Contr from a list does not delete contraction if a connective rule smuggles it back through context sharing. To build a logic without contraction, we must also redesign the connectives.

21.2 The family of logics

Deleting subsets of $\{\mathsf{Exch}, \mathsf{Weak}, \mathsf{Contr}\}$ gives a family of logics. Five of them have names worth knowing:

Logic	Exchange	Weakening	Contraction	Each assumption is used
intuitionistic	yes	yes	yes	any number of times
affine	yes	yes	no	at most once
relevant	yes	no	yes	at least once
linear	yes	no	no	exactly once
ordered	no	no	no	exactly once, in order

The last column is the bottom-up reading of the rules. Without contraction an assumption cannot be duplicated, so it is used at most once; without weakening it cannot be discarded, so it is used at least once; without both, exactly once. Without exchange, the assumptions must even be used in the order in which they appear.



The family of substructural logics. Each arrow deletes one structural rule; Rust's owned values that are not Copy follow the affine discipline.

A few remarks on the members of the family.

Ordered logic is the oldest. Joachim Lambek designed a calculus without any structural rules in 1958, for the grammar of natural languages: the words of a sentence are resources that must be used exactly once *and in order*, and a sentence is grammatical when the types of its words combine into the type of a sentence. Rust does not drop exchange: the order in which you declare independent bindings never makes a program ill typed. We keep exchange for the rest of the book.

Relevant logic comes from philosophy: its authors objected that $A \to (B \to A)$ is a theorem although B is irrelevant to A, and wanted every premise to be used. In programming terms it is a "must use" discipline, of which Rust has a weak echo in `#[must_use]` and the unused-variable lint, both warnings rather than type errors (Chapter 24).

Linear logic was introduced by Jean-Yves Girard in 1987. Every assumption is used exactly once, and the logic is a precise language for resources that must be consumed: a payment that must be settled, a channel that must be closed.

Affine logic keeps weakening and drops contraction: resources may be thrown away but not copied. That is Rust's discipline for owned values that are not `Copy`, and the affine λ-calculus of Chapter 22 makes it a type system.

Each structural rule has a classic *test formula*, a formula provable in intuitionistic logic whose every proof uses that rule. In the notation of the λ-calculus they are combinators, named K, W and C:

Test formula	Combinator	Needs
$A \to (B \to A)$	K, $\lambda x.\, \lambda y.\, x$	weakening (y unused)
$(A \to A \to B) \to (A \to B)$	W, $\lambda f.\, \lambda x.\, f\, x\, x$	contraction (x used twice)
$(A \to B \to C) \to (B \to A \to C)$	C, $\lambda f.\, \lambda y.\, \lambda x.\, f\, x\, y$	exchange (arguments swapped)

Affine logic proves the linear version of K but not of W; relevant logic proves W but not K; linear logic proves neither; and all of them except ordered logic prove C. The prover we build below confirms the claims about affine and linear logic.

21.3 Conjunction splits in two

Here is the rule $\wedge$I of natural deduction again, together with the two projections:

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\ \wedge \mathsf{I} \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\ \wedge \mathsf{E}_1 \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\ \wedge \mathsf{E}_2$$

Rule $\wedge$I shares Γ between its premises. As Chapter 6 showed, this is contraction in disguise: the context $\Gamma = A$ proves $A \wedge A$ by using A in both premises. In a logic without contraction we must decide what to do with the context of $\wedge$I, and there are two honest options.

- **Split it.** The conclusion's assumptions are divided between the premises, each going to exactly one of them. The result is the **multiplicative** conjunction, written $A \otimes B$ and read "A tensor B" or "A times B".
- **Share it, and pay for the sharing at the other end.** Both premises receive the whole context, but then a proof of the conjunction can only be *used* for one of its components, never both. The result is the **additive** conjunction, written $A \,\&\, B$ and read "A with B".

The sequent calculus

From now on we work in the **sequent calculus**, which Chapter 4 previewed. Instead of introduction and elimination rules, it has *right rules*, which say how to prove a formula (the formula is on the right of $\vdash$), and *left rules*, which say how to use an assumption (the formula is on the left). The sequent calculus is better suited to the study of structural rules, because every rule shows exactly what happens to the context, and to proof search, because it is easy to run backward.

In linear logic the identity axiom has an *exact* context, with nothing else in it:

$$\frac{}{A \vdash A}\ \mathsf{Ax}$$

Compare it with rule Hyp of natural deduction, $A \in \Gamma$ implies $\Gamma \vdash A$, which ignores the rest of Γ: that tolerance was weakening. The exact axiom is half of what makes the logic linear. The other half is the shape of the connective rules.

There is also the **cut** rule, which lets you prove a lemma A and then use it:

$$\frac{\Gamma \vdash A \qquad \Delta, A \vdash C}{\Gamma, \Delta \vdash C}\ \mathsf{Cut}$$

Gentzen proved for ordinary logic, and Girard for linear logic, that cut can be eliminated: every sequent provable with cut is provable without it. Cut-free proofs have the *subformula property* (every formula in the proof is a subformula of the conclusion), which makes proof search finite. The systems in this chapter, in Rust and in Lean, are cut-free.

Tensor

The rules for $\otimes$:

$$\frac{\Gamma \vdash A \qquad \Delta \vdash B}{\Gamma, \Delta \vdash A \otimes B}\otimes\mathsf{R} \qquad\qquad \frac{\Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C}\otimes\mathsf{L}$$

To *prove* $A \otimes B$ from the resources Γ, Δ, divide them: prove A from Γ and B from Δ. To *use* an assumption $A \otimes B$, replace it by its two components, both available at once. Read as resources, $A \otimes B$ means "an A and a B, side by side": producing it costs the resources of both, and consuming it gives you both.

Now try to prove $A \vdash A \otimes A$. The only rule that applies is $\otimes$R (the assumption A is atomic, so no left rule applies), and $\otimes$R must divide the context A between two premises. One premise gets A and the other gets nothing, and $\vdash A$ is not provable: there is no rule that produces an A from nothing. So there is no proof. From one A you cannot make two.

With

The rules for $A \,\&\, B$:

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \,\&\, B}\,\&\,\mathsf{R} \qquad \frac{\Gamma, A \vdash C}{\Gamma, A \,\&\, B \vdash C}\,\&\,\mathsf{L}_1 \qquad \frac{\Gamma, B \vdash C}{\Gamma, A \,\&\, B \vdash C}\,\&\,\mathsf{L}_2$$

To prove $A \,\&\, B$, show that the *same* resources Γ could produce an A, and could also produce a B. To use $A \,\&\, B$, pick one of the two and carry on with it; the other is gone. Read as resources, $A \,\&\, B$ is an *offer*: "an A or a B, your choice", where the choice belongs to whoever consumes the formula.

The sharing in rule &R is now harmless. Both premises see Γ, but the left rules ensure that only one of them is ever cashed in, so the resources are spent once.

Girard's own explanations of linear logic used purchases, and a vending machine makes the point well. Write $coin$ for a coin. A machine that, for one coin, lets you have a coffee or a tea is the formula

$$coin \multimap (coffee \,\&\, tea)$$

(where $\multimap$, explained below, means "consume the left side to produce the right side"). You choose which drink you get. You do not get $coffee \otimes tea$, both drinks at once: that would take two coins.

Two conjunctions, and how ordinary logic merges them

The two conjunctions are genuinely different. In linear logic:

- $A \,\&\, B \vdash A$ is provable (rule $\&\mathsf{L}_1$, then Ax), but $A \otimes B \vdash A$ is not: the B would have to be thrown away.
- $A \vdash A \,\&\, A$ is provable (rule &R, with Ax in both premises), but $A \vdash A \otimes A$ is not, as we just saw.
- Neither $A \otimes B \vdash A \,\&\, B$ nor $A \,\&\, B \vdash A \otimes B$ is provable.

The structural rules are exactly what collapses them. With weakening, $A \otimes B$ implies $A \,\&\, B$:

$$\dfrac{\dfrac{\dfrac{}{A \vdash A}\,\mathsf{Ax}}{A, B \vdash A}\,\mathsf{Weak} \qquad \dfrac{\dfrac{}{B \vdash B}\,\mathsf{Ax}}{A, B \vdash B}\,\mathsf{Weak}}{A, B \vdash A \,\&\, B}\,\&\,\mathsf{R}}{A \otimes B \vdash A \,\&\, B}\,\otimes\mathsf{L}$$

(the second weakening adds A in front, which exchange allows). With contraction, $A \,\&\, B$ implies $A \otimes B$:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{A \vdash A}\,\mathsf{Ax}}{A \,\&\, B \vdash A}\,\&\,\mathsf{L}_1 \qquad \dfrac{\dfrac{}{B \vdash B}\,\mathsf{Ax}}{A \,\&\, B \vdash B}\,\&\,\mathsf{L}_2}{A \,\&\, B, A \,\&\, B \vdash A \otimes B}\,\otimes\mathsf{R}}{A \,\&\, B \vdash A \otimes B}\,\mathsf{Contr}$$

In a logic with both rules, the two conjunctions prove the same things, and it needs only one, which it calls $\wedge$. That is why you never met two conjunctions in Chapter 4. In affine logic, which has weakening but not contraction, only the first derivation exists: $A \otimes B$ is *stronger* than $A \,\&\, B$. Having both is worth more than being offered a choice, when you are allowed to throw away the one you do not want.

21.4 Disjunction, implication and the units

Plus

The additive disjunction $A \oplus B$ ("A plus B") is the dual of $A \,\&\, B$: the choice belongs to the *producer*.

$$\dfrac{\Gamma \vdash A}{\Gamma \vdash A \oplus B}\,\oplus\mathsf{R}_1 \qquad \dfrac{\Gamma \vdash B}{\Gamma \vdash A \oplus B}\,\oplus\mathsf{R}_2 \qquad \dfrac{\Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma, A \oplus B \vdash C}\,\oplus\mathsf{L}$$

To prove $A \oplus B$, prove one of them, and you decide which. To use an assumption $A \oplus B$, you must be ready for both cases, because you do not know which one you will get. A machine $coin \multimap (coffee \oplus tea)$ takes your coin and gives you a drink of *its* choosing.

Rule $\oplus$L shares the context Γ between its two premises, like &R, and for the same reason this is not contraction: the two premises are alternatives, and only one of them will happen. Rules that share their context between alternatives are called **additive**; rules that split their context between parts that coexist are called **multiplicative**. The names, like the symbols $\otimes$ and $\oplus$, come from an analogy with linear algebra, where tensor products multiply dimensions and direct sums add them.

There is also a multiplicative disjunction, written $A \parr B$ ("par"). It needs sequents with several formulas on the right of $\vdash$, which is the setting of classical linear logic; in the intuitionistic setting of this book, with one formula on the right, it has no rules, and it has no convincing reading as a Rust type. We mention it for orientation only.

Linear implication

Linear implication $A \multimap B$ ("A lollipop B") consumes an A to produce a B:

$$\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B}\,\multimap\mathsf{R} \qquad \dfrac{\Gamma \vdash A \qquad \Delta, B \vdash C}{\Gamma, \Delta, A \multimap B \vdash C}\,\multimap\mathsf{L}$$

Rule $\multimap$R is $\to$I: to build a consumer of A from the resources Γ, assume the A and produce the B. Rule $\multimap$L says how to use one: split the remaining resources, pay for the argument A with Γ, receive the B, and continue with B and Δ. It is a multiplicative rule, with splitting, and it is what makes $\multimap$ consume its argument: modus ponens becomes $A \multimap B, A \vdash B$, and the A is gone afterward.

The two conjunction-like connectives relate to implication differently. Currying works for $\otimes$: $(A \otimes B) \multimap C$ and $A \multimap (B \multimap C)$ prove each other, because each of A and B is used exactly once on both sides. It does not work for $A \,\&\, B$.

The units

Each binary connective has a unit, a formula that changes nothing when combined with another:

$$\frac{}{\vdash 1}\ 1\mathsf{R} \qquad \frac{\Gamma \vdash C}{\Gamma, 1 \vdash C}\ 1\mathsf{L} \qquad \frac{}{\Gamma \vdash \top}\ \top\ \mathsf{R} \qquad \frac{}{\Gamma, 0 \vdash C}\ 0\mathsf{L}$$

- 1 is the unit of $\otimes$ ($A \otimes 1$ is equivalent to A). It stands for "no resources at all": it can be produced from an empty context, and an assumption 1 can be dropped.
- $\top$ is the unit of with ($A \,\&\, \top$ is equivalent to A). It can be produced from *any* context, consuming everything in it, and it has no left rule: it offers nothing.
- 0 is the unit of $\oplus$. It has no right rule, so it cannot be produced, and an assumption 0 proves anything: it describes a situation that cannot happen. It is the linear version of $\bot$, and in Rust it is an empty `enum`.

Ordinary logic has one truth constant $\top$; linear logic splits it into 1 and $\top$, just as it split $A \wedge B$ into $A \otimes B$ and $A \,\&\, B$. In affine logic the split closes again: weakening lets you produce 1 from anything, so 1 and $\top$ become equivalent.

Here is the whole vocabulary, with the Rust readings that the next sections develop:

Formula	Name	Kind	Unit	Rust $\approx$
$A \otimes B$	tensor	multiplicative conjunction	1	`(A, B)`, a struct
$A \,\&\, B$	with	additive conjunction	$\top$	an offer with consuming methods
$A \oplus B$	plus	additive disjunction	0	`enum`
$A \parr B$	par	multiplicative disjunction	$\bot$	(classical only)
$A \multimap B$	lollipop	linear implication		`FnOnce(A) -> B`
$!A$	of course	exponential		`A: Copy` (Chapter 23)

Full linear logic, for orientation

Girard's linear logic is classical: it has a linear negation $A^\bot$ with $A^{\bot\bot} = A$, De Morgan laws that exchange the multiplicatives ($(A \otimes B)^\bot = A^\bot \parr B^\bot$) and the additives ($(A \,\&\, B)^\bot = A^\bot \oplus B^\bot$), and the definition $A \multimap B = A^\bot \parr B$. This book uses only the intuitionistic fragment, and you can safely ignore the classical one.

What you cannot ignore are the **exponentials**. The modality $!A$ ("of course A") marks a formula that may be used any number of times. Its rules give back the structural rules, but only for formulas marked with !:

$$\frac{\Gamma, A \vdash C}{\Gamma, !A \vdash C}\ \text{Der} \qquad \frac{\Gamma \vdash C}{\Gamma, !A \vdash C}\ !\text{W} \qquad \frac{\Gamma, !A, !A \vdash C}{\Gamma, !A \vdash C}\ !\text{C} \qquad \frac{!\Gamma \vdash A}{!\Gamma \vdash !A}\ \text{Prom}$$

Dereliction (Der) uses a $!A$ once, as an A. Weakening and contraction are back, for $!A$ only. Promotion (Prom) says that A may be marked as reusable if it was proved from reusable assumptions only ($!\Gamma$ means that every formula of the context starts with !). With !, ordinary logic fits inside linear logic: the intuitionistic implication $A \to B$ becomes $!A \multimap B$, a function that may use its argument as often as it likes. The dual modality $?A$ plays the same role on the right of $\vdash$ in the classical system. Chapter 23 reads Rust's `Copy` as !: a class of types for which contraction and weakening are back.

The exponentials come at a price: provability in propositional linear logic with exponentials is known to be undecidable, whereas the fragment without them (the *multiplicative-additive* fragment, MALL) is decidable, by exactly the kind of finite proof search we build below.

21.5 Splitting versus sharing

Write $\Gamma_1 \uplus \Gamma_2$ for a context *split* into two parts, every assumption going to exactly one part. (Contexts in linear logic are multisets: order does not matter, the number of copies does.) The multiplicative rules split, and the additive rules share:

Rule	Context	Why it is safe
$\otimes$R, $\multimap$L	split, $\Gamma_1 \uplus \Gamma_2$	the two premises coexist, so each resource goes to one
&R, $\oplus$L	shared, Γ in both	the two premises are alternatives, only one will happen
Ax	exactly A	nothing may be left over

The affine version of the calculus differs only at the leaves. Its axiom and its rule for 1 tolerate extra assumptions, which are discarded:

$$\frac{}{\Gamma, A \vdash A}\ \text{Ax}_{aff} \qquad \frac{}{\Gamma \vdash 1}\ 1\text{R}_{aff}$$

With these leaves, weakening is admissible, by an induction like Chapter 6's: an extra assumption is passed up the derivation until a leaf absorbs it.

This table is also a description of Rust. A call `f(a, b)` or a tuple `(a, b)` *splits* the owned variables in scope between its parts: if both parts need the same non-`Copy` variable, no split exists, and the compiler reports E0382. The arms of a `match` *share* them: each arm may move the same variable, because only one arm runs. Here is a function from this chapter's Rust example that relies on sharing:

```rust
/// `Coffee ⊕ Tea`: the machine decides, and the customer must handle both.
#[derive(Debug)]
enum Drink {
    Coffee(Coffee),
    Tea(Tea),
}
```

```
/// ⊕R: the machine picks a drink (here, by the coin's serial number).
fn surprise(coin: Coin) -> Drink {
    if coin.serial % 2 == 0 {
        Drink::Coffee(brew_coffee(coin))
    } else {
        Drink::Tea(brew_tea(coin))
    }
}

/// A receipt: another resource, consumed by whoever files it.
struct Receipt {
    number: u32,
}

/// Filing a receipt consumes it.
fn file(receipt: Receipt) -> u32 {
    receipt.number
}

/// ⊕L: both arms consume the same `receipt`, because only one arm runs.
fn serve(drink: Drink, receipt: Receipt) -> String {
    match drink {
        Drink::Coffee(c) => {
            format!("coffee (coin {}), filed {}", c.paid_with, file(receipt))
        }
        Drink::Tea(t) => format!("tea (coin {}), filed {}", t.paid_with, file(receipt)),
    }
}
```

Both arms of the `match` in `serve` consume `receipt` (by passing it to `file`, which takes it by value), and the compiler accepts this, exactly as rule ⊕L shares Γ between its premises. The tuple `(consume(s), consume(s))` would be rejected, exactly as ⊗R refuses to give s to both premises.

21.6 Linear logic in Rust

The full program is in `code/rust/examples/ch21_resources.rs`. As always, reading Rust types as formulas is a model, and Rust's discipline is affine, not linear: any value may be dropped. The comparisons below hold with that caveat. The resources are coins, which are neither `Copy` nor `Clone`, and drinks:

```
/// A coin: the resource. Neither `Copy` nor `Clone`: it can be spent once.
#[derive(Debug)]
struct Coin {
    serial: u32,
}

#[derive(Debug)]
struct Coffee {
    paid_with: u32,
}

#[derive(Debug)]
struct Tea {
    paid_with: u32,
}

/// `Coin ⊸ Coffee`: spending a coin produces a coffee.
fn brew_coffee(coin: Coin) -> Coffee {
    Coffee {
        paid_with: coin.serial,
    }
}

/// `Coin ⊸ Tea`.
```

```
fn brew_tea(coin: Coin) -> Tea {
    Tea {
        paid_with: coin.serial,
    }
}
```

The functions `brew_coffee` and `brew_tea` have the types $coin \multimap coffee$ and $coin \multimap tea$: each takes a coin by value and consumes it.

Tensor: tuples and structs

A tuple `(A, B)` holds an `A` and a `B` at the same time, each owned independently. It is $A \otimes B$:

```
/// ⊗R: two coins, split between the two drinks, give a coffee *and* a tea.
fn coffee_and_tea(c1: Coin, c2: Coin) -> (Coffee, Tea) {
    (brew_coffee(c1), brew_tea(c2))
}
```

```
let (coffee, tea) = coffee_and_tea(Coin { serial: 1 }, Coin { serial: 2 });
println!("⊗: {coffee:?} and {tea:?}");
```

Building the tuple is rule ⊗R: the two coins are split between the two components. Taking it apart with the pattern `let (coffee, tea) = ...` is rule ⊗L: one step, and both components become available as separate variables. This is why `fn swap<A, B>((a, b): (A, B)) -> (B, A)` needs no `Clone`, while the natural-deduction proof of $A \wedge B \to B \wedge A$ in Chapter 6 used its assumption twice: Rust's pair is $\otimes$, eliminated by pattern matching, not $\wedge$ with two projections. (Rust also lets you move the fields out one at a time, `pair.0` and later `pair.1`, tracking each field separately. Chapter 22 explains this refinement.) Structs are tensors with named components.

With: an offer

The additive conjunction is an offer: the consumer chooses one alternative, and both alternatives are paid for by the same resources. In Rust, that is a value holding the resources, with one *consuming* method per alternative:

```
/// `Coffee & Tea`: the machine after one coin was inserted. Both choices are
/// paid for by the *same* coin (&R shares its context); the customer picks
/// one by calling one of the consuming methods (&L1 or &L2).
struct Offer {
    coin: Coin,
}

impl Offer {
    fn insert(coin: Coin) -> Offer {
        Offer { coin }
    }

    fn coffee(self) -> Coffee {
        brew_coffee(self.coin)
    }

    fn tea(self) -> Tea {
        brew_tea(self.coin)
    }
}
```

`Offer::insert` is rule &R: from one coin it builds something that could become a coffee and could also become a tea, and both methods use the same coin. Calling a method is rule $\&L_1$ or $\&L_2$: `coffee(self)`

takes the offer by value, so after the call the offer no longer exists and the other method cannot be called. The compiler enforces this:

```
pub fn both(offer: Offer) -> (Coffee, Tea) {
    (offer.coffee(), offer.tea())
}
```

```
error[E0382]: use of moved value: `offer`
  --> compile_fail/ch21_offer_twice.rs:31:22
   |
30 | pub fn both(offer: Offer) -> (Coffee, Tea) {
   |             ----- move occurs because `offer` has type `Offer`, which does not
implement the `Copy` trait
31 |     (offer.coffee(), offer.tea())
   |            --------  ^^^^^ value used here after move
   |            |
   |            `offer` moved due to this method call
   |
note: `Offer::coffee` takes ownership of the receiver `self`, which moves `offer`
  --> compile_fail/ch21_offer_twice.rs:18:19
   |
18 |     pub fn coffee(self) -> Coffee {
   |                   ^^^^
```

The note says it exactly: the method takes ownership of the receiver `self`, which moves `offer`. Taking both alternatives would require contraction on `offer`.

A generic version of $A \,\&\, B$ stores the shared resources and two ways of consuming them:

```
/// `A & B` in general: shared resources `S` and two ways to consume them.
struct With<S, F, G> {
    resources: S,
    left: F,
    right: G,
}

impl<S, A, B, F: FnOnce(S) -> A, G: FnOnce(S) -> B> With<S, F, G> {
    fn left(self) -> A {
        (self.left)(self.resources)
    }

    fn right(self) -> B {
        (self.right)(self.resources)
    }
}
```

```
let first = Offer::insert(Coin { serial: 3 });
let choice = first.tea(); // `first` is consumed: no coffee from it now
println!("&: chose {choice:?}");
let second = Offer::insert(Coin { serial: 4 });
println!("&: chose {:?}", second.coffee());
let make = |serial| With {
    resources: Coin { serial },
    left: brew_coffee,
    right: brew_tea,
};
println!("&: chose {:?} and {:?}", make(5).left(), make(6).right());
```

A second encoding is a closure that takes the choice as an argument, `impl FnOnce(Choice) -> Drink`. It loses information: the result type cannot depend on the *value* of the choice, so the closure returns the sum type `Drink`, and the customer must still `match` on both cases. A dependent type can say

more: in Lean, `(b : Bool) → if b then A else B` gives an `A` when asked for `true`. Chapter 30 is about such types.

Plus: enums

An `enum` with two variants holds one of two things, and the producer decides which. It is $A \oplus B$. In the `plus` listing above, `surprise` is rule $\oplus$R: the machine picks a drink (here by the coin's serial number), and its return type `Drink` hides which. `serve` is rule $\oplus$L: the consumer must handle both variants, and both arms share the same `receipt`. The output shows the two cases:

```
⊕: served tea (coin 7), filed 107
⊕: served coffee (coin 8), filed 108
```

Exhaustiveness checking, which makes you handle every variant, is the requirement that $\oplus$L has a premise for each disjunct.

Lollipop: `FnOnce`

A closure that consumes what it captured can be called once. Its type, `impl FnOnce(A) -> B`, is $A \multimap B$:

```
/// A paper cup: a resource that a coffee machine needs once.
struct Cup;

/// `Coin ⊸ Coffee` as a value: a closure that has captured the cup. It can be
/// called once, because calling it moves the cup out.
fn machine_with_cup(cup: Cup) -> impl FnOnce(Coin) -> Coffee {
    move |coin| {
        let Cup = cup; // the cup is used up by this call
        brew_coffee(coin)
    }
}
```

```
let machine = machine_with_cup(Cup);
let coffee = machine(Coin { serial: 9 });
println!("⊸: {coffee:?}");
```

Creating the closure is rule $\multimap$R: from the resources Γ (the captured cup) and an assumed argument A (the coin), produce a B (the coffee). Calling it is rule $\multimap$L: the argument is paid for with one part of the context, and the result joins the rest. Calling it a second time is a use of a moved value, as Chapter 8 showed. Closures that implement `Fn` can be called any number of times, which makes them resemble $!(A \multimap B)$; the resemblance is only suggestive, since `Fn` and `FnMut` are about *how* the closure accesses its captures (shared or unique borrows), the subject of Part VI.

The units

```
/// `0`, the empty type: from a value of it, anything follows (0L).
enum Void {}

fn absurd<T>(v: Void) -> T {
    match v {}
}

/// `1` is `()`. In an affine language, any value can be turned into `()` by
/// discarding it, so `T` and `1` collapse into one type.
fn discard<T>(_value: T) {}
```

The unit type `()` is 1. An empty `enum` is 0: no value of it can be built, and `match v {}` with no arms turns one into anything, which is rule 0L. There is no separate $\top$ in Rust, and there does not need to be: Rust is affine, any value can be discarded, and in affine logic $\top$ and 1 are equivalent. The function `discard` is rule $\top$R and 1R at once: it consumes anything and returns `()`.

Here is the complete output of the example:

```
⊗: Coffee { paid_with: 1 } and Tea { paid_with: 2 }
&: chose Tea { paid_with: 3 }
&: chose Coffee { paid_with: 4 }
&: chose Coffee { paid_with: 5 } and Tea { paid_with: 6 }
⊕: served tea (coin 7), filed 107
⊕: served coffee (coin 8), filed 108
⊸: Coffee { paid_with: 9 }
1: a coin discarded, () returned
```

21.7 Proof search: the `linlogic` crate

Which sequents are provable in linear logic, and which in affine logic? For the multiplicative-additive fragment the question is decidable, and the crate `linlogic` (in `code/rust/crates/linlogic`) decides it by searching for a cut-free derivation, backward from the goal. It prints the derivations it finds, checks them with an independent checker, and explains its failures.

Formulas and sequents

The formulas are an enum with one variant per connective and unit:

```rust
/// A formula of intuitionistic multiplicative-additive linear logic.
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum Formula {
    /// An atomic proposition: a kind of resource, such as `coin` or `A`.
    Atom(String),
    /// `1`, the unit of ⊗: no resources at all.
    One,
    /// `⊤`, the unit of &: accepts any resources and offers nothing.
    Top,
    /// `0`, the unit of ⊕: an impossible situation.
    Zero,
    /// `A ⊗ B` (tensor): both A and B, at the same time.
    Tensor(Box<Formula>, Box<Formula>),
    /// `A & B` (with): A or B, and the consumer chooses which.
    With(Box<Formula>, Box<Formula>),
    /// `A ⊕ B` (plus): A or B, and the producer chooses which.
    Plus(Box<Formula>, Box<Formula>),
    /// `A ⊸ B` (lollipop): consume an A to produce a B.
    Lolli(Box<Formula>, Box<Formula>),
}
```

A parser reads an ASCII syntax (`A * B` for $A \otimes B$, `A & B`, `A + B` for $A \oplus B$, `A -o B` for $A \multimap B$, and `1`, `0`, `top`), and the printer uses the Unicode symbols. A sequent is a context and a goal:

```rust
/// A sequent `Γ ⊢ C` of intuitionistic linear logic: a multiset of resources
/// `Γ` (the context) and one goal `C`. The order of the context is irrelevant
/// (exchange is built in); its multiplicities are not.
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Sequent {
    pub context: Vec<Formula>,
    pub goal: Formula,
}
```

The context is a `Vec`, but the prover treats it as a multiset by sorting it before it looks a sequent up or stores it, so exchange is built in, as in the table of Chapter 6. Weakening and contraction are not. The logic is a parameter:

```
/// Which structural rules the logic has. Both have exchange (contexts are
/// multisets) and neither has contraction. Affine logic also has weakening.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Logic {
    Linear,
    Affine,
}
```

The rules, backward

The heart of the crate is a function that takes a sequent and returns every way of applying a rule *backward*: every rule whose conclusion matches the sequent, with the premises that remain to be proved. Here are the leaves, where the two logics differ:

```
// Leaves. In linear logic the identity axiom needs the context to be
// exactly the goal; affine logic may first discard everything else.
if context.len() == 1 && context[0] == *goal {
    steps.push(step(Rule::Axiom, vec![]));
} else if logic == Logic::Affine && context.contains(goal) {
    let axiom = Sequent::new(vec![goal.clone()], goal.clone());
    steps.push(step(Rule::Weaken, vec![axiom]));
}
match goal {
    Formula::One if context.is_empty() => steps.push(step(Rule::OneR, vec![])),
    Formula::One if logic == Logic::Affine => {
        let unit = Sequent::new(vec![], Formula::One);
        steps.push(step(Rule::Weaken, vec![unit]));
    }
    Formula::Top => steps.push(step(Rule::TopR, vec![])),
    _ => {}
}
if context.contains(&Formula::Zero) {
    steps.push(step(Rule::ZeroL, vec![]));
}
```

In linear logic the identity axiom applies only when the context is exactly the goal. In affine logic, if the goal is somewhere in the context, a `Weaken` step discards everything else and leaves the exact axiom to prove. The same happens for the unit 1. Because the affine prover discards resources only at the leaves, every weakening in a derivation is visible, as a line marked `W`, right where the resource is thrown away.

The rules that split their context must try every split. A context with n assumptions has 2^n ways of dividing it into a left part and a right part (fewer when some assumptions are equal):

```
/// Every way to divide a multiset of resources between two premises. A
/// context of n formulas has 2^n splits (fewer when formulas repeat).
pub fn splits(context: &[Formula]) -> Vec<(Vec<Formula>, Vec<Formula>)> {
    let mut seen = BTreeSet::new();
    let mut result = Vec::new();
    for mask in 0..(1_u32 << context.len()) {
        let (mut left, mut right) = (Vec::new(), Vec::new());
        for (i, formula) in context.iter().enumerate() {
            if mask & (1 << i) != 0 {
                left.push(formula.clone());
            } else {
                right.push(formula.clone());
```

```
            }
        }
        left.sort();
        right.sort();
        if seen.insert(left.clone()) {
            result.push((left, right));
        }
    }
    result
}
```

Rule ⊗R, backward, produces one candidate step per split:

```
Formula::Tensor(a, b) => {
    for (left, right) in splits(context) {
        let p1 = Sequent::new(left, (**a).clone());
        let p2 = Sequent::new(right, (**b).clone());
        steps.push(step(Rule::TensorR, vec![p1, p2]));
    }
}
```

and rule &L produces two candidate steps, one per side:

```
Formula::With(a, b) => {
    let p1 = Sequent::new(with_added(&rest, &[a]), goal.clone());
    let p2 = Sequent::new(with_added(&rest, &[b]), goal.clone());
    steps.push(step(Rule::WithL1, vec![p1]));
    steps.push(step(Rule::WithL2, vec![p2]));
}
```

The function `steps` lists the candidates in a fixed order: first the rules that close a branch (Ax, 1R, ⊤R, 0L), then the rules that never lose provability (⊸R, &R, ⊗L, 1L, ⊕L: if the conclusion is provable, so are their premises, so they never lead the search into a dead end), and last the rules that make a choice (which split, which side of $A \oplus B$ or $A \,\&\, B$). The order affects only which proof is found first and how fast; correctness does not depend on it.

The search

```
/// A proof search for one logic. It remembers every sequent it has
/// examined, with the proof it found or `None`, so no sequent is searched
/// twice.
pub struct Prover {
    logic: Logic,
    memo: BTreeMap<Sequent, Option<Proof>>,
}

impl Prover {
    pub fn new(logic: Logic) -> Prover {
        Prover {
            logic,
            memo: BTreeMap::new(),
        }
    }

    /// Searches for a derivation of `seq`. The search is complete: if it
    /// returns `None`, the sequent has no cut-free derivation at all.
    pub fn prove(&mut self, seq: &Sequent) -> Option<Proof> {
        let mut proof = self.search(&seq.normalized())?;
        // Show the root with the context in the order the user wrote it.
        proof.conclusion = seq.clone();
        Some(proof)
    }
```

```
    fn search(&mut self, seq: &Sequent) -> Option<Proof> {
        if let Some(known) = self.memo.get(seq) {
            return known.clone();
        }
        let mut found = None;
        'steps: for step in steps(self.logic, seq) {
            let mut premises = Vec::new();
            for premise in &step.premises {
                match self.search(&premise.normalized()) {
                    Some(proof) => premises.push(proof),
                    None => continue 'steps,
                }
            }
            found = Some(Proof {
                rule: step.rule,
                conclusion: seq.clone(),
                premises,
            });
            break;
        }
        self.memo.insert(seq.clone(), found.clone());
        found
    }
}
```

`search` tries the candidate steps in order, and the first step whose premises are all provable gives the proof; the labeled `continue 'steps` abandons a candidate as soon as one premise fails. Every result is stored in a `BTreeMap` keyed by the sorted sequent (deterministic, unlike a `HashMap`), so no sequent is searched twice.

Three properties make this a decision procedure.

- **Termination.** Every premise of every rule is strictly smaller than its conclusion, counting the total size of the formulas in the sequent. A unit test (`every_premise_is_smaller`) checks it on a sequent that exercises every rule. So the recursion cannot go on forever.
- **Completeness.** The search tries every rule application, so if a cut-free derivation exists, it finds one. By cut elimination, every provable sequent has a cut-free derivation. In the affine logic, discarding only at the leaves loses nothing, because weakening is admissible in a calculus with those leaves.
- **Soundness.** Every derivation the prover returns is checked by `Proof::check`, which knows nothing about the search: it looks at each node and verifies, by comparing multisets, that the node is a correct instance of the rule it claims.

```
/// Checks every step of the derivation against the rules of `logic`.
/// This does not trust the prover: it only compares sequents.
pub fn check(&self, logic: Logic) -> Result<(), String> {
    for premise in &self.premises {
        premise.check(logic)?;
    }
    let premises: Vec<&Sequent> =
        self.premises.iter().map(|p| &p.conclusion).collect();
    if check_step(logic, self.rule, &self.conclusion, &premises) {
        Ok(())
    } else {
        Err(format!(
            "{} is not a correct use of {}",
            self.conclusion, self.rule
        ))
```

```
    }
}
```

Do not trust the prover, check its output: it is the same principle that makes Lean trustworthy, where tactics may be as clever as they like because the kernel checks the proof terms they produce.

Running it

The example `crates/linlogic/examples/ch21_prover.rs` runs the prover on a list of sequents, in both logics:

```
const TABLE: [(&str, &str); 12] = [
    ("A |- A * A", "contraction"),
    ("A * B |- A", "weakening"),
    ("A & B |- A", "a choice"),
    ("A |- A & A", "sharing"),
    ("A * B |- B * A", "exchange"),
    ("A * B |- A & B", "& from ⊗ needs weakening"),
    ("A & B |- A * B", "⊗ from & needs contraction"),
    ("A |- B -o A", "the K combinator"),
    ("A -o A -o B |- A -o B", "the W combinator"),
    ("A -o B -o C |- A * B -o C", "currying"),
    ("top |- 1", "1 from ⊤ needs weakening"),
    ("A * (B + C) |- (A * B) + (A * C)", "distributivity"),
];
```

```
cargo run -p linlogic --example ch21_prover
```

```
sequent                          linear  affine  shows
A ⊢ A ⊗ A                        no      no      contraction
A ⊗ B ⊢ A                        no      yes     weakening
A & B ⊢ A                        yes     yes     a choice
A ⊢ A & A                        yes     yes     sharing
A ⊗ B ⊢ B ⊗ A                    yes     yes     exchange
A ⊗ B ⊢ A & B                    no      yes     & from ⊗ needs weakening
A & B ⊢ A ⊗ B                    no      no      ⊗ from & needs contraction
A ⊢ B ⊸ A                        no      yes     the K combinator
A ⊸ A ⊸ B ⊢ A ⊸ B                no      no      the W combinator
A ⊸ B ⊸ C ⊢ (A ⊗ B) ⊸ C          yes     yes     currying
⊤ ⊢ 1                            no      yes     1 from ⊤ needs weakening
A ⊗ (B ⊕ C) ⊢ (A ⊗ B) ⊕ (A ⊗ C)  yes     yes     distributivity
```

Every row is a fact of this chapter, now checked by a machine. Neither logic has contraction, so $A \vdash A \otimes A$ fails in both; $A \otimes B \vdash A$ needs weakening, so only affine logic proves it; choosing and sharing ($A \,\&\, B \vdash A$, $A \vdash A \,\&\, A$) are not structural rules, so both logics prove them. The two conjunctions are related only through the structural rules, K is affine but not linear, W is neither, and $\top \vdash 1$ needs weakening, which is why the two units merge in affine logic.

Here are the derivations for three of the provable sequents, as the prover prints them. Each line shows the rule applied, then the sequent it proves; the lines indented below it are the derivations of its premises.

```
linear proof of A ⊗ B ⊢ B ⊗ A:
⊗L   A ⊗ B ⊢ B ⊗ A
⊗R     A, B ⊢ B ⊗ A
Ax       B ⊢ B
Ax       A ⊢ A
```

```
affine proof of A ⊗ B ⊢ A:
⊗L   A ⊗ B ⊢ A
W      A, B ⊢ A
Ax       A ⊢ A
```

The first derivation takes the pair apart (⊗L) and rebuilds it in the other order (⊗R). The second takes the pair apart and then discards B at the leaf: the `W` line is the weakening, at the exact place where a resource is thrown away, which in Rust is where the destructor of the unused component runs.

When a sequent is not provable, the prover can list every rule that applies and the verdict on each premise:

```
no linear proof of A ⊢ A ⊗ A; the rules that apply:
  ⊗R   ⊢ A (fails);  A ⊢ A (provable)
  ⊗R   A ⊢ A (provable);  ⊢ A (fails)
  (3 sequents examined)
```

This is the argument we made by hand for $A \vdash A \otimes A$, produced mechanically: there are two ways to split the single A, and each of them leaves one side with nothing.

21.8 A linear sequent calculus in Lean

The prover decides each sequent we give it. It cannot tell us that $A \vdash A \otimes A$ fails for *every* formula A, or explain why. For that we turn to Lean. The file is `code/lean/Book/Ch21/Linear.lean`.

Formulas and rules

```
/-- Formulas of intuitionistic linear logic (multiplicative-additive). -/
inductive Formula where
  /-- An atomic resource. -/
  | atom (name : String)
  /-- `1`, the unit of `⊗`: nothing at all. -/
  | one
  /-- `⊤`, the unit of `&`: accepts anything, offers nothing. -/
  | top
  /-- `0`, the unit of `⊕`: an impossible situation. -/
  | zero
  /-- `A ⊗ B`: both, at the same time. -/
  | tensor (A B : Formula)
  /-- `A & B`: either, and the consumer chooses. -/
  | «with» (A B : Formula)
  /-- `A ⊕ B`: either, and the producer chooses. -/
  | plus (A B : Formula)
  /-- `A ⊸ B`: consume an `A`, produce a `B`. -/
  | lolli (A B : Formula)
  deriving DecidableEq, Repr

@[inherit_doc] scoped infixr:40 " ⊗ " => Formula.tensor
@[inherit_doc] scoped infixr:35 " & " => Formula.with
@[inherit_doc] scoped infixr:30 " ⊕ " => Formula.plus
@[inherit_doc] scoped infixr:25 " ⊸ " => Formula.lolli
```

The constructor for $A \,\&\, B$ is named `«with»`: `with` is a keyword of Lean, and the French quotes make it usable as a name. The `scoped infixr` lines define notations active inside the namespace `Ch21`, with $\otimes$ binding tightest and $\multimap$ loosest. Core Lean already uses $\oplus$ for its sum type `Sum`; Lean tells the two apart in ordinary terms, by their types, but not in patterns, so in pattern matching we write `plus A B` instead.

The rules are an inductive relation, exactly like `Proves` in Chapter 6, with the sequent calculus in place of natural deduction:

```
/-- `LProves Γ C`: the sequent `Γ ⊢ C` is derivable in linear logic. Left
rules act on the first formula of the context; `exch` moves formulas. -/
inductive LProves : List Formula → Formula → Prop where
  | ax (A : Formula) : LProves [A] A
  | exch {Γ Δ C} : Γ.Perm Δ → LProves Γ C → LProves Δ C
  | oneR : LProves [] one
  | oneL {Γ C} : LProves Γ C → LProves (one :: Γ) C
  | topR {Γ} : LProves Γ top
  | zeroL {Γ C} : LProves (zero :: Γ) C
  | tensorR {Γ Δ A B} : LProves Γ A → LProves Δ B → LProves (Γ ++ Δ) (A ⊗ B)
  | tensorL {Γ A B C} : LProves (A :: B :: Γ) C → LProves ((A ⊗ B) :: Γ) C
  | withR {Γ A B} : LProves Γ A → LProves Γ B → LProves Γ (A & B)
  | withL₁ {Γ A B C} : LProves (A :: Γ) C → LProves ((A & B) :: Γ) C
  | withL₂ {Γ A B C} : LProves (B :: Γ) C → LProves ((A & B) :: Γ) C
  | plusR₁ {Γ A B} : LProves Γ A → LProves Γ (A ⊕ B)
  | plusR₂ {Γ A B} : LProves Γ B → LProves Γ (A ⊕ B)
  | plusL {Γ A B C} :
      LProves (A :: Γ) C → LProves (B :: Γ) C → LProves ((A ⊕ B) :: Γ) C
  | lolliR {Γ A B} : LProves (A :: Γ) B → LProves Γ (A ⊸ B)
  | lolliL {Γ Δ A B C} :
      LProves Γ A → LProves (B :: Δ) C → LProves ((A ⊸ B) :: (Γ ++ Δ)) C

@[inherit_doc] scoped notation:20 Γ " ⊢ " C:21 => LProves Γ C
```

Lean's contexts are lists, so exchange is an explicit rule, `exch`, which reorders a context by any permutation. `Γ.Perm Δ` ("`Δ` is a permutation of `Γ`") is defined inductively in Lean's core library, with constructors for the empty list, for adding the same element in front, for swapping the first two elements, and for transitivity. The left rules act on the first formula of the context, and `exch` brings the formula you want to the front. Rule ⊗R produces the context `Γ ++ Δ`: the split, read top-down. There is no weakening, no contraction and no cut, and the axiom has the exact context `[A]`. The last line gives the relation the notation `Γ ⊢ C`.

A few derivations:

```
/-- A choice: from `A & B` take `A`. -/
example : [A & B] ⊢ A := .withL₁ (.ax A)

/-- Sharing: both sides of `A & A` are built from the same `A`. -/
example : [A] ⊢ A & A := .withR (.ax A) (.ax A)

/-- Exchange: `⊗R` produces the context `[B] ++ [A]`, and `exch` reorders it. -/
example : [A ⊗ B] ⊢ B ⊗ A :=
  .tensorL (.exch (List.Perm.swap A B []) (.tensorR (.ax B) (.ax A)))

/-- Modus ponens: `⊸L` splits `[A]` off for the argument. -/
example : [A ⊸ B, A] ⊢ B := .lolliL (Γ := [A]) (Δ := []) (.ax A) (.ax B)

/-- Currying: `A ⊸ B ⊸ C ⊢ A ⊗ B ⊸ C`. -/
example : [A ⊸ B ⊸ C] ⊢ A ⊗ B ⊸ C := by
  apply LProves.lolliR
  -- [A ⊗ B, A ⊸ B ⊸ C] ⊢ C
  apply LProves.tensorL
  -- [A, B, A ⊸ B ⊸ C] ⊢ C: bring the implication to the front
  apply LProves.exch (List.perm_middle (l₁ := [A, B]) (l₂ := [])).symm
  -- [A ⊸ B ⊸ C, A, B] ⊢ C: the argument A, then B ⊸ C with B
  exact .lolliL (Γ := [A]) (Δ := [B]) (.ax A)
    (.lolliL (Γ := [B]) (Δ := []) (.ax B) (.ax C))
```

The first two are the choice and the sharing of $A \,\&\, B$. In the third, `.tensorR (.ax B) (.ax A)` proves `[B] ++ [A] ⊢ B ⊗ A`, which Lean computes to `[B, A] ⊢ B ⊗ A`. `List.Perm.swap A B []` is the permutation from `[B, A]` to `[A, B]`, so `exch` turns the derivation into one of `[A, B] ⊢ B ⊗ A`, which `tensorL` turns into `[A ⊗ B] ⊢ B ⊗ A`. In the modus ponens example, the named arguments `(Γ := [A]) (Δ := [])` tell Lean how to split the context, which it cannot guess. The currying proof is a tactic proof whose comments show the goal at each step; `List.perm_middle` says that moving an element from the middle of a list to the front is a permutation.

How to prove that something is not provable

Now the main question. To show that a sequent *is* derivable, we build a derivation. To show that it is *not*, we would have to rule out every possible derivation. The standard technique is the one Chapter 4 used for soundness: find a property that every derivable sequent has, prove it by induction on derivations, and check that the sequent we care about lacks it. Such a property is called an **invariant**.

For linear logic the natural invariant is about counting: if every resource is used exactly once, every atom that goes in must come out. In $A \vdash A \otimes A$, the left side has one A and the right side two, so the sequent should not be derivable.

Two complications must be handled. First, $A \multimap B$ produces a B but *uses up* an A, so it counts $+1$ for B and -1 for A: a negative count is a debt. Second, $A \,\&\, B$ might end up providing an A or a B, depending on a choice made later, so a formula has a *set* of possible counts. And $\top$ and 0 absorb any count at all: $\top$ consumes whatever is left, and from 0 anything follows.

So we define, for each formula, the **bags** of atoms it can provide:

```
/-- A bag of resources: how many copies of each atom. Counts may be negative,
which records a debt: `A ⊸ B` provides one `B` and owes one `A`. -/
abbrev Bag : Type := String → ℤ

/-- The bag that holds exactly one copy of the atom `a`. -/
def single (a : String) : Bag := fun x => if x = a then 1 else 0
```

```
/-- `Provides F n`: one way of resolving the choices inside `F` yields the
bag `n`. The additives offer several ways; `⊤` and `0` fit any bag. -/
def Provides : Formula → Bag → Prop
  | atom a, n => n = single a
  | one, n => n = 0
  | top, _ => True
  | zero, _ => True
  | A ⊗ B, n => ∃ a b, Provides A a ∧ Provides B b ∧ n = a + b
  | A & B, n => Provides A n ∨ Provides B n
  | plus A B, n => Provides A n ∨ Provides B n
  | A ⊸ B, n => ∃ a b, Provides A a ∧ Provides B b ∧ n = b - a

/-- A whole context provides the sum of what its formulas provide. -/
def ProvidesAll : List Formula → Bag → Prop
  | [], n => n = 0
  | A :: Γ, n => ∃ a g, Provides A a ∧ ProvidesAll Γ g ∧ n = a + g

/-- A sequent is balanced when the context and the goal can provide the same
bag of atoms. -/
def Balanced (Γ : List Formula) (C : Formula) : Prop :=
  ∃ n, ProvidesAll Γ n ∧ Provides C n
```

A `Bag` is a function from atom names to integers: how many copies of each atom, possibly negative. `single a` is the bag with one copy of `a`. Bags can be added and subtracted pointwise, because Mathlib makes functions into an additive group pointwise (`(f + g) x = f x + g x`).

`Provides F n` says that, for some way of resolving the choices inside `F`, the formula provides exactly the bag `n`. Read the clauses one at a time. An atom provides one copy of itself; 1 provides nothing; $\top$ and 0 fit any bag. $A \otimes B$ provides the sum of a bag of A and a bag of B: both are there. $A \,\&\, B$ and $A \oplus B$ provide whatever one of their components provides. $A \multimap B$ provides a bag of B minus a bag of A. A context provides the sum of what its formulas provide, and a sequent is **balanced** if its context and its goal can provide the same bag.

The invariant

The theorem is that every derivable sequent is balanced. Its proof uses three small lemmas: every formula and every context provides *some* bag (needed for the rules $\top$R, 0L and Ax), a concatenation of contexts provides the sum of the bags (needed for the splitting rules), and a permutation of a context provides the same bags (needed for `exch`):

```
/-- Every formula provides some bag. -/
theorem provides_exists : ∀ F : Formula, ∃ n, Provides F n
  | atom a => ⟨single a, rfl⟩
  | one => ⟨0, rfl⟩
  | top => ⟨0, trivial⟩
  | zero => ⟨0, trivial⟩
  | A ⊗ B =>
    let ⟨a, ha⟩ := provides_exists A
    let ⟨b, hb⟩ := provides_exists B
    ⟨a + b, a, b, ha, hb, rfl⟩
  | A & _ =>
    let ⟨a, ha⟩ := provides_exists A
    ⟨a, .inl ha⟩
  | plus A _ =>
    let ⟨a, ha⟩ := provides_exists A
    ⟨a, .inl ha⟩
  | A ⊸ B =>
    let ⟨a, ha⟩ := provides_exists A
    let ⟨b, hb⟩ := provides_exists B
    ⟨b - a, a, b, ha, hb, rfl⟩

/-- Every context provides some bag. -/
theorem providesAll_exists : ∀ Γ : List Formula, ∃ n, ProvidesAll Γ n
  | [] => ⟨0, rfl⟩
  | A :: Γ =>
    let ⟨a, ha⟩ := provides_exists A
    let ⟨g, hg⟩ := providesAll_exists Γ
    ⟨a + g, a, g, ha, hg, rfl⟩

/-- Concatenated contexts provide the sum of the bags. -/
theorem providesAll_append {Γ Δ : List Formula} {g d : Bag}
    (hΓ : ProvidesAll Γ g) (hΔ : ProvidesAll Δ d) : ProvidesAll (Γ ++ Δ) (g + d) := by
  induction Γ generalizing g with
  | nil =>
    -- hΓ : g = 0
    simp only [ProvidesAll] at hΓ
    simpa [hΓ] using hΔ
  | cons A Γ ih =>
    obtain ⟨a, g', ha, hg', rfl⟩ := hΓ
    exact ⟨a, g' + d, ha, ih hg', by abel⟩

/-- Reordering a context does not change what it provides. -/
theorem providesAll_perm {Γ Δ : List Formula} (p : Γ.Perm Δ) :
    ∀ {n : Bag}, ProvidesAll Γ n → ProvidesAll Δ n := by
```

```
  induction p with
  | nil => exact id
  | cons A _ ih =>
    rintro n ⟨a, g, ha, hg, rfl⟩
    exact ⟨a, g, ha, ih hg, rfl⟩
  | swap A B Γ =>
    rintro n ⟨b, r, hb, ⟨a, g, ha, hg, rfl⟩, rfl⟩
    exact ⟨a, b + g, ha, ⟨b, g, hb, hg, rfl⟩, by abel⟩
  | trans _ _ ih₁ ih₂ => exact fun h => ih₂ (ih₁ h)
```

The two existence lemmas are proved by recursion on the formula or the list, written as pattern matching, like `Deriv.toProves` in Chapter 6. In `providesAll_append`, the `nil` case has `hΓ : g = 0`, and `simpa [hΓ] using hΔ` rewrites `g + d` into `d` and closes the goal with `hΔ`. Two tools appear here for the first time. `abel` is a Mathlib tactic that proves equations in commutative additive groups, such as `a + (b + g) = b + (a + g)`, by normalizing both sides; our bags form such a group, so `abel` does all the bookkeeping of sums and differences. And the patterns of `obtain` and `rintro` can end in `rfl`, as in `⟨a, g', ha, hg', rfl⟩`: that takes apart an existential statement and, for the final equation `n = a + g'`, substitutes it everywhere, so that `n` disappears. The proof of `providesAll_perm` is an induction on the permutation, with one case for each of its four constructors.

Now the invariant itself:

```
/-- **The invariant.** Every derivable sequent is balanced. -/
theorem balanced_of_proves {Γ : List Formula} {C : Formula}
    (h : Γ ⊢ C) : Balanced Γ C := by
  induction h with
  | ax A =>
    -- [A] ⊢ A: both sides provide whatever A provides.
    obtain ⟨a, ha⟩ := provides_exists A
    exact ⟨a, ⟨a, 0, ha, rfl, by abel⟩, ha⟩
  | exch p _ ih =>
    obtain ⟨n, hΓ, hC⟩ := ih
    exact ⟨n, providesAll_perm p hΓ, hC⟩
  | oneR => exact ⟨0, rfl, rfl⟩
  | oneL _ ih =>
    obtain ⟨n, hΓ, hC⟩ := ih
    exact ⟨n, ⟨0, n, rfl, hΓ, by abel⟩, hC⟩
  | @topR Γ =>
    -- ⊤ accepts whatever the context provides.
    obtain ⟨g, hg⟩ := providesAll_exists Γ
    exact ⟨g, hg, trivial⟩
  | @zeroL Γ C =>
    -- 0 makes up the difference between the goal and the rest.
    obtain ⟨c, hc⟩ := provides_exists C
    obtain ⟨g, hg⟩ := providesAll_exists Γ
    exact ⟨c, ⟨c - g, g, trivial, hg, by abel⟩, hc⟩
  | tensorR _ _ ih₁ ih₂ =>
    -- The two halves of the context pay for the two halves of A ⊗ B.
    obtain ⟨n₁, hΓ, hA⟩ := ih₁
    obtain ⟨n₂, hΔ, hB⟩ := ih₂
    exact ⟨n₁ + n₂, providesAll_append hΓ hΔ, n₁, n₂, hA, hB, rfl⟩
  | tensorL _ ih =>
    obtain ⟨n, ⟨a, r, ha, ⟨b, g, hb, hg, rfl⟩, rfl⟩, hC⟩ := ih
    exact ⟨a + (b + g), ⟨a + b, g, ⟨a, b, ha, hb, rfl⟩, hg, by abel⟩, hC⟩
  | withR _ _ ih₁ _ =>
    -- One premise is enough: A & B can provide what A provides.
    obtain ⟨n, hΓ, hA⟩ := ih₁
    exact ⟨n, hΓ, .inl hA⟩
  | withL₁ _ ih =>
    obtain ⟨n, ⟨a, g, ha, hg, rfl⟩, hC⟩ := ih
    exact ⟨a + g, ⟨a, g, .inl ha, hg, rfl⟩, hC⟩
  | withL₂ _ ih =>
    obtain ⟨n, ⟨b, g, hb, hg, rfl⟩, hC⟩ := ih
```

```
    exact ⟨b + g, ⟨b, g, .inr hb, hg, rfl⟩, hC⟩
| plusR₁ _ ih =>
    obtain ⟨n, hΓ, hA⟩ := ih
    exact ⟨n, hΓ, .inl hA⟩
| plusR₂ _ ih =>
    obtain ⟨n, hΓ, hB⟩ := ih
    exact ⟨n, hΓ, .inr hB⟩
| plusL _ _ ih₁ _ =>
    obtain ⟨n, ⟨a, g, ha, hg, rfl⟩, hC⟩ := ih₁
    exact ⟨a + g, ⟨a, g, .inl ha, hg, rfl⟩, hC⟩
| lolliR _ ih =>
    -- From A, Γ ⊢ B: Γ provides g and B provides a + g, so A ⊸ B provides g.
    obtain ⟨n, ⟨a, g, ha, hg, rfl⟩, hB⟩ := ih
    exact ⟨g, hg, a, a + g, ha, hB, by abel⟩
| lolliL _ _ ih₁ ih₂ =>
    -- Γ pays for the argument A; the result B joins Δ.
    obtain ⟨m, hΓ, hA⟩ := ih₁
    obtain ⟨n, ⟨b, d, hb, hd, rfl⟩, hC⟩ := ih₂
    exact ⟨b + d, ⟨b - m, m + d, ⟨m, b, hA, hb, rfl⟩,
      providesAll_append hΓ hd, by abel⟩, hC⟩
```

Each case takes the balance of the premises (the induction hypotheses `ih`) and builds the balance of the conclusion. Let us go through the interesting ones.

- **ax.** Pick any bag `a` that `A` provides. The context `[A]` provides `a + 0`, and `A` provides `a`; `abel` proves `a = a + 0`.
- **exch.** The bag is the same, and `providesAll_perm` moves it to the permuted context.
- **tensorR.** The left premise gives a bag `n₁` provided by both `Γ` and `A`, the right premise a bag `n₂` provided by both `Δ` and `B`. Then `Γ ++ Δ` provides `n₁ + n₂` (by `providesAll_append`), and so does `A ⊗ B`, by definition. This is the case that makes contraction impossible: the bag of the conclusion's context is the *sum* of the premises' bags, so each resource is counted once on each side.
- **tensorL.** The premise's context `A :: B :: Γ` provides `a + (b + g)`. In the conclusion, `A ⊗ B` provides `a + b`, and the context provides `(a + b) + g`, which `abel` identifies with `a + (b + g)`.
- **withR.** Only the first premise is needed: if `Γ` and `A` provide the same bag, then so do `Γ` and `A & B`, by choosing `A`. The invariant does not care that `B` is also available, because `&` provides one of its components, not both.
- **lolliR.** The premise says that `A :: Γ` and `B` both provide `a + g`, where `a` is a bag of `A` and `g` a bag of `Γ`. Then `A ⊸ B` provides `(a + g) - a`, which is `g`, what `Γ` provides.
- **lolliL.** The left premise gives a bag `m` provided by both `Γ` and `A`; the right premise says that `B :: Δ` provides `b + d`, which `C` also provides. In the conclusion, `A ⊸ B` provides `b - m`, the context `Γ ++ Δ` provides `m + d`, and the sum `(b - m) + (m + d)` is `b + d` again: the debt of `A` is paid by `Γ`.
- **topR and zeroL.** These are the absorbing units. For ⊤R, take any bag of the context; `top` provides it. For 0L, take a bag `c` of the goal and a bag `g` of the rest of the context, and let `zero` provide the difference `c - g`.

The remaining cases (`oneR`, `oneL`, `withL₁`, `withL₂`, `plusR₁`, `plusR₂`, `plusL`) follow the same pattern: the bag stays the same, and the additive formula provides it by the side that the rule chose (`.inl` or `.inr`). Like `withR`, rule `plusL` needs only one of its premises.

The case for weakening is missing, because there is no weakening rule, and so is the case for contraction. If either rule were added, the theorem would be false: weakening would let the left side have more than the right, and contraction would let it have less. The invariant captures exactly the absence of the two rules.

From one A you cannot get two

The main theorem is now a short calculation:

```
/-- **No contraction.** `A ⊢ A ⊗ A` has no derivation in linear logic. -/
theorem not_proves_dup (a : String) : ¬ ([atom a] ⊢ atom a ⊗ atom a) := by
  intro h
  obtain ⟨n, ⟨x, g, hx, hg, hn⟩, ⟨y, z, hy, hz, hyz⟩⟩ := balanced_of_proves h
  -- hx : x = single a,  hg : g = 0,  hy : y = single a,  hz : z = single a
  subst hx hg hy hz
  -- hn : n = single a + 0,  hyz : n = single a + single a
  -- Count the copies of `a` on both sides: 1 + 0 = 1 + 1 is false.
  have count := congrFun (hn.symm.trans hyz) a
  simp [single] at count
```

Step by step:

- `intro h` assumes a derivation `h` of `[atom a] ⊢ atom a ⊗ atom a`; the goal becomes `False`.
- `balanced_of_proves h` says that the sequent is balanced, and `obtain` takes the statement apart. The context `[atom a]` provides `n = x + g`, where `x` is provided by `atom a` and `g` by the empty context. The goal `atom a ⊗ atom a` provides `n = y + z`, where both `y` and `z` are provided by `atom a`.
- By the definition of `Provides`, the hypotheses `hx`, `hg`, `hy`, `hz` are the equations `x = single a`, `g = 0`, `y = single a` and `z = single a`, and `subst` replaces the variables by their values everywhere.
- `hn.symm.trans hyz` chains the equations `single a + 0 = n` and `n = single a + single a` into `single a + 0 = single a + single a`, an equation between functions. `congrFun _ a` evaluates both sides at the atom `a`.
- `simp [single]` computes: the left side is `1 + 0`, the right side `1 + 1`, and `1 = 2` is false, which closes the goal.

The same invariant shows that linear logic has no weakening, using the atom that would have to be thrown away:

```
/-- **No weakening.** `A ⊗ B ⊢ A` has no derivation when `A` and `B` are
different atoms: the `B` on the left has nowhere to go. -/
theorem not_proves_fst {a b : String} (hab : a ≠ b) :
    ¬ ([atom a ⊗ atom b] ⊢ atom a) := by
  intro h
  obtain ⟨n, ⟨t, g, ⟨x, y, hx, hy, ht⟩, hg, hn⟩, hC⟩ := balanced_of_proves h
  subst hx hy ht hg hC
  -- hn : single a = single a + single b + 0. Count the copies of `b`.
  have count := congrFun hn b
  simp [single, hab.symm] at count
```

Here the context provides one `a` and one `b`, the goal only one `a`, and counting the copies of `b` gives `1 = 0`. The hypothesis `hab.symm : b ≠ a` lets `simp` decide that `single a b = 0`.

The invariant is necessary for provability, not sufficient. The sequent $(A \otimes B)\,\&\,(A \otimes C) \vdash A \otimes (B\,\&\,C)$ is balanced (both sides can provide $A + B$, and both can provide $A + C$), yet the prover

above reports that it has no derivation in linear logic. The invariant answers the questions it was designed for, contraction and weakening; for the rest, there is the prover.

Forgetting resources

One more theorem connects this chapter to Chapter 6. If you forget the resources, reading both $A \otimes B$ and $A \,\&\, B$ as $A \wedge B$, $\oplus$ as $\vee$, $\multimap$ as $\to$, 1 and $\top$ as $\top$ and 0 as $\bot$, every linear derivation becomes a derivation in natural deduction. The file `code/lean/Book/Ch21/Forget.lean` imports Chapter 6's `Structural.lean` (both chapters are part of the same development) and proves it:

```
/-- Forget resources: translate a linear formula into Chapter 6's formulas. -/
def Formula.forget : Formula → Ch06.Formula
  | atom a => .atom a
  | one => .top
  | top => .top
  | zero => .bot
  | A ⊗ B => .conj A.forget B.forget
  | .with A B => .conj A.forget B.forget
  | plus A B => .disj A.forget B.forget
  | A ⊸ B => .imp A.forget B.forget
```

```
/-- Every linear derivation, with resources forgotten, is an intuitionistic
derivation. -/
theorem LProves.forget {Γ : List Formula} {C : Formula} (h : Γ ⊢ C) :
    Ch06.Proves (Γ.map Formula.forget) C.forget := by
  induction h with
  | ax A => exact .hyp List.mem_cons_self
  | exch p _ ih =>
    -- Exchange: the translated contexts have the same elements.
    exact ih.mono (p.map _).subset
  | oneR => exact .topI
  | oneL _ ih => exact Ch06.weakening ih
  | topR => exact .topI
  | zeroL => exact .botE (.hyp List.mem_cons_self)
  | tensorR _ _ ih₁ ih₂ =>
    -- Splitting becomes sharing: each half is weakened to the whole context.
    rw [List.map_append]
    exact .conjI (ih₁.mono (List.subset_append_left _ _))
      (ih₂.mono (List.subset_append_right _ _))
  | tensorL _ ih => exact conj_left ih
  | withR _ _ ih₁ ih₂ => exact .conjI ih₁ ih₂
  | withL₁ _ ih => exact conj_left₁ ih
  | withL₂ _ ih => exact conj_left₂ ih
  | plusR₁ _ ih => exact .disjI₁ ih
  | plusR₂ _ ih => exact .disjI₂ ih
  | plusL _ _ ih₁ ih₂ =>
    exact .disjE (.hyp List.mem_cons_self)
      (ih₁.mono (List.cons_subset_cons _ (List.subset_cons_self _ _)))
      (ih₂.mono (List.cons_subset_cons _ (List.subset_cons_self _ _)))
  | lolliR _ ih => exact .impI ih
  | lolliL _ _ ih₁ ih₂ =>
    -- Apply the implication (first in the context) to a proof of its
    -- argument, then use the result as the hypothesis of the second premise.
    rw [List.map_cons, List.map_append]
    refine .impE (Ch06.weakening (.impI (ih₂.mono ?_)))
      (.impE (.hyp List.mem_cons_self) ?_)
    · exact List.cons_subset_cons _ (List.subset_append_right _ _)
    · exact ih₁.mono (List.subset_cons_of_subset _ (List.subset_append_left _ _))
```

The proof is an induction on the linear derivation, and each case uses a tool from Chapter 6. Exchange becomes `Proves.mono`, since a permutation has the same elements. Rule ⊗R splits its context while natural deduction shares it, so each half is *weakened* to the whole context, again with `Proves.mono`. Rule 1L becomes `weakening`. The left rules for $A \otimes B$ and $A \,\&\, B$ are handled by three small lemmas

in the same file (`conj_left`, `conj_left₁`, `conj_left₂`), which use an assumption $A \wedge B$ through its projections and also rely on `weakening`. Linear logic proves fewer things than intuitionistic logic, never different things, and the admissible structural rules of Chapter 6 are the glue that turns a linear proof into an ordinary one. The converse fails:

```
/-- The converse fails: intuitionistic logic duplicates what linear logic
cannot. -/
example (a : String) :
    Ch06.Proves [(atom a).forget] ((atom a ⊗ atom a).forget) ∧
      ¬ ([atom a] ⊢ atom a ⊗ atom a) :=
  ⟨.conjI (.hyp List.mem_cons_self) (.hyp List.mem_cons_self), not_proves_dup a⟩
```

21.9 Putting it together: the vending machine

Let us follow Girard's vending machine through every tool of the chapter. The machine is the formula $coin \multimap (coffee \,\&\, tea)$: for a coin, it offers a coffee or a tea, at your choice.

The prover settles the questions, with variations: one coin buys a coffee but not a coffee and a tea; two coins and two machines buy both; and a machine that chooses the drink itself, with $\oplus$, cannot promise you a coffee, only a coffee or a tea.

```
lin  aff  sequent
yes  yes  coin, coin ⊸ (coffee & tea) ⊢ coffee
no   no   coin, coin ⊸ (coffee & tea) ⊢ coffee ⊗ tea
yes  yes  coin, coin, coin ⊸ (coffee & tea), coin ⊸ (coffee & tea) ⊢ coffee ⊗ tea
no   no   coin, coin ⊸ (coffee ⊕ tea) ⊢ coffee
yes  yes  coin, coin ⊸ (coffee ⊕ tea) ⊢ coffee ⊕ tea
```

Here is the derivation for one coin and one coffee. Rule ⊸L splits the context: the coin pays for the machine's argument, and the offer joins the (empty) rest; rule &L₁ chooses the coffee:

```
linear proof of coin, coin ⊸ (coffee & tea) ⊢ coffee:
⊸L   coin, coin ⊸ (coffee & tea) ⊢ coffee
Ax     coin ⊢ coin
&L1    coffee & tea ⊢ coffee
Ax       coffee ⊢ coffee
```

and here is why one coin does not buy two drinks. Every possible first step is listed, and every one of them has a failing premise:

```
no linear proof of coin, coin ⊸ (coffee & tea) ⊢ coffee ⊗ tea; the rules that apply:
  ⊗R   ⊢ coffee (fails);  coin, coin ⊸ (coffee & tea) ⊢ tea (provable)
  ⊗R   coin ⊢ coffee (fails);  coin ⊸ (coffee & tea) ⊢ tea (fails)
  ⊗R   coin ⊸ (coffee & tea) ⊢ coffee (fails);  coin ⊢ tea (fails)
  ⊗R   coin, coin ⊸ (coffee & tea) ⊢ coffee (provable);  ⊢ tea (fails)
  ⊸L   ⊢ coin (fails);  coin, coffee & tea ⊢ coffee ⊗ tea (fails)
  ⊸L   coin ⊢ coin (provable);  coffee & tea ⊢ coffee ⊗ tea (fails)
  (26 sequents examined)
```

Look at the four ⊗R lines: they are the four ways to divide the coin and the machine between the coffee and the tea. The two ⊸L lines are the two ways to use the machine first, which either leave no coin to pay with or produce an offer that cannot become both drinks.

Lean proves the same two facts for good, independently of any search:

```
/-- The vending machine: a coin buys a coffee... -/
example : [atom "coin", atom "coin" ⊸ (atom "coffee" & atom "tea")] ⊢ atom "coffee" :=
  .exch (List.Perm.swap _ _ [])
    (.lolliL (Γ := [atom "coin"]) (Δ := []) (.ax _) (.withL₁ (.ax _)))

/-- ...or a tea, but not both. -/
theorem one_coin_one_drink :
    ¬ ([atom "coin", atom "coin" ⊸ (atom "coffee" & atom "tea")] ⊢
        atom "coffee" ⊗ atom "tea") := by
  intro h
  obtain ⟨n, ⟨c, r, hc, ⟨m, g, ⟨k, d, hk, hd, hm⟩, hg, hr⟩, hn⟩,
    ⟨x, y, hx, hy, hxy⟩⟩ := balanced_of_proves h
  subst hc hk hm hg hr hx hy
  -- The machine's offer was resolved one way or the other.
  rcases hd with rfl | rfl
  · -- Chose coffee: there is no tea on the left. Count teas.
    have count := congrFun (hn.symm.trans hxy) "tea"
    simp [single] at count
  · -- Chose tea: count coffees.
    have count := congrFun (hn.symm.trans hxy) "coffee"
    simp [single] at count
```

The negative theorem uses the invariant with all atoms counted at once. The offer inside the machine was resolved one way or the other (`rcases hd`), and in each case one of the two drinks is missing from the left side: if the machine provided a coffee, counting teas gives `0 = 1`; if it provided a tea, counting coffees does. Notice that this proof needs bags of *all* the atoms at once, not one count per atom: counted separately, coffees and teas could each balance under different choices.

Rust has the same structure in its types. The machine with a coin inserted is the `Offer` value; choosing is calling a consuming method; and asking for both drinks is the rejected program `ch21_offer_twice.rs`, with error E0382. Buying both drinks with two coins is `coffee_and_tea`, which splits two coins between two brews and returns a tuple: a $\otimes$, honestly paid for.

21.10 Common confusions

"**Linear logic is about linear functions, or linear time.**" The name comes from the mathematics Girard used to give the logic a meaning, where proofs become *linear* maps. In practice, "linear" means that every assumption is used exactly once; it has nothing to do with running time or with straight-line code.

"***A* & *B* is just a pair.**" A value of $A \,\&\, B$ contains the *means* to produce either an A or a B, from one set of resources, and only one of them can ever be produced. A pair, $A \otimes B$, contains both. In Rust the difference is between an offer whose methods take `self` and a tuple.

"**Sharing the context in &R and ⊕L is contraction.**" Contraction lets one resource serve two uses that both happen. In the additive rules, the two premises are alternatives, and only one of them is ever carried out, so every resource is still spent once. Rust's `match` arms, each moving the same variable, are the everyday example.

"**Rust is linear.**" Rust is affine: any value may be dropped, and dropping is weakening (with an effect: Chapter 24). A linear discipline would force every value to be consumed explicitly. The difference shows up in the prover's table as the column of sequents that are affine but not linear.

"**If a sequent is balanced, it is provable.**" Balance is a necessary condition, which is enough to prove that $A \vdash A \otimes A$ and $A \otimes B \vdash A$ fail. It is not sufficient; the example in the text is $(A \otimes B) \,\&\, (A \otimes C) \vdash A \otimes (B \,\&\, C)$.

21.11 Summary

- Deleting structural rules gives a family of logics: intuitionistic, affine (at most once), relevant (at least once), linear (exactly once) and ordered (in order).
- Connective rules must not smuggle the rules back: multiplicative rules ($\otimes$R, $\multimap$L) split the context, additive rules (&R, $\oplus$L) share it between alternatives, and the linear axiom has an exact context.
- Conjunction splits into $A \otimes B$ (both) and $A \,\&\, B$ (one, chosen by the consumer); $A \oplus B$ is one, chosen by the producer. Weakening and contraction together merge the conjunctions back into $\wedge$.
- The units are 1, $\top$ and 0; the exponential $!A$ brings the structural rules back for marked formulas (Chapter 23 reads it as `Copy`).
- In Rust, `(A, B)` is $\otimes$, an offer with consuming methods is $A \,\&\, B$, an `enum` is $\oplus$, `FnOnce(A) -> B` is $A \multimap B$, `()` is 1 and an empty `enum` is 0.
- The `linlogic` crate decides linear and affine provability by a terminating, complete, memoizing backward search, and checks every proof it finds.
- In Lean, every derivable linear sequent is balanced, so $A \vdash A \otimes A$ and $A \otimes B \vdash A$ have no derivation; forgetting resources turns every linear derivation into an intuitionistic one.

21.12 Exercises

1. Derive, on paper, the sequent $A \otimes (B \oplus C) \vdash (A \otimes B) \oplus (A \otimes C)$ in the linear sequent calculus, and then the converse. Check your answers with the prover. *Hint:* start with $\otimes$L, then use $\oplus$L on $B \oplus C$; in each branch choose the matching side with $\oplus\mathrm{R}_1$ or $\oplus\mathrm{R}_2$ before splitting with $\otimes$R.
2. Construct the derivation of Exercise 1 in Lean, as a value of type `[A ⊗ (B ⊕ C)] ⊢ (A ⊗ B) ⊕ (A ⊗ C)`. *Hint:* after `.tensorL`, the context is `[A, B ⊕ C]`, and `plusL` needs the disjunction at the front: use `.exch (List.Perm.swap _ _ [])` first. In each branch, the context `[B, A]` must be reordered again before `tensorR` with `(.ax A) (.ax B)`.
3. Prove in Lean that the combinator K is not linear: for different atoms `a` and `b`, `¬ ([atom a] ⊢ atom b ⊸ atom a)`. *Hint:* follow `not_proves_fst`. The goal `atom b ⊸ atom a` provides `single a − single b`; count the copies of `b`.
4. Run the prover (write a small `main`, or add rows to the example) on $(A \otimes B) \,\&\, (A \otimes C) \vdash A \otimes (B \,\&\, C)$ and on its converse. Which one is provable? For the unprovable one, compute by hand two different bags that both sides can provide, confirming that the invariant cannot refute it. *Hint:* `prover.explain(&seq)` shows the first steps of the failed search. The bags are $A + B$ and $A + C$.
5. In Rust, model a network connection that, once opened, must end in exactly one of two ways: `close(self) -> Report` or `abort(self, reason: String) -> Report`. Which connective is it? Write

the type, show that calling both methods is rejected, and then encode the same choice as a closure that takes an enum `Ending` as its argument. What does the closure version lose? *Hint:* the two methods share the connection, and the caller chooses. In the closure version, think about what the return type can say about the choice.

6. Find an invariant for *affine* logic restricted to formulas without $\multimap$: show that every derivable sequent has, for some resolution of its choices, a left bag that is greater than or equal to the right bag at every atom. Use it to show that $A \vdash A \otimes A$ is not derivable in affine logic either. Why does the invariant break as soon as $\multimap$ is allowed? *Hint:* weakening adds a formula to the left, which can only increase a bag if all bags are non-negative. With $\multimap$, the formula $A \multimap 1$ provides the bag $-A$: weaken $A \vdash A$ by it and see what happens to the left bag.

CHAPTER 22

The Affine Lambda Calculus

Chapter 21 took the structural rules away from logic one at a time and looked at what was left. Without contraction, an assumption stops being a fact that can be quoted as often as you like and becomes a *resource* that is spent when it is used. The logic that keeps weakening but drops contraction is **affine logic**: every assumption is used at most once.

This chapter does for affine logic what Chapter 8 did for intuitionistic logic. It builds the programming language whose type system *is* the logic: the **affine λ-calculus**, λ^{aff}. Its typing rules fit on one page, and they are the formal content of the first half of the claim that "Rust's ownership is based on affine types": moves, the error "use of moved value" (E0382), partial moves, closures that can be called only once. There are no references and no lifetimes here; those are Part VI. This chapter is pure move semantics.

We will write every rule down, build derivations by hand and in Lean, and prove the theorem that makes the calculus affine: in a well-typed term, every variable is used at most once. Then we turn the rules into an algorithm, prove it sound and complete in Lean, implement it in Rust as the `affine` crate, and put the crate's error messages side by side with rustc's.

In this chapter you will learn:

- the syntax and typing rules of λ^{aff}, and how *splitting* the context replaces *sharing* it in rules with two subterms;
- how to build derivations, and why swapping a pair has one while duplicating a value has none;
- why a pair of type $\tau \otimes \sigma$ is taken apart with `let` and not with projections, and how Rust's partial moves refine this one place at a time;
- the usage theorem and its complete proof in Lean;
- the metatheory around it: weakening is admissible, contraction is not, and substitution and preservation hold;
- how to check the rules with input and output contexts ("leftovers"), with a Lean proof that this algorithm is sound and complete;
- how the `affine` crate reports errors modeled on E0382, and where it agrees and disagrees with rustc.

22.1 From affine logic to a programming language

Here is what we need from earlier chapters. Chapter 6 introduced the three structural rules: exchange (the order of assumptions does not matter), weakening (an assumption may go unused) and contraction (an assumption may be used twice). Chapter 21 arranged the logics that drop some of them in a lattice: **linear** logic drops weakening and contraction (every assumption used exactly once), **affine** logic drops only contraction (at most once).

Chapter 21 also found that, once contraction is gone, the familiar connectives split in two. Conjunction becomes $A \otimes B$ ("tensor", I have both, and they were paid for with *disjoint* resources) and $A \,\&\, B$ ("with", I can give you either one, your choice, both built from the *same* resources). Disjunction becomes $A \oplus B$, and implication becomes $A \multimap B$ ("lollipop", a function that uses its argument at most once). Girard's coin is the standard illustration: with one coin you can have a coffee *or* a tea ($\mathit{Coffee} \,\&\, \mathit{Tea}$), but not a coffee *and* a tea ($\mathit{Coffee} \otimes \mathit{Tea}$).

The difference between the two conjunctions lives in one operation on contexts, **context splitting**. The introduction rule of $\otimes$ divides the assumptions between its two premises, $\Gamma = \Gamma_1 \uplus \Gamma_2$, so that each assumption goes to one premise only. The introduction rule of $\&$ gives the *whole* context to both premises, which is safe because only one of the two will ever be used.

Chapter 10 showed how to read a natural-deduction system as a programming language: propositions become types, proofs become terms, assumptions become variables, and each rule becomes a typing rule that says how to build a term. Applied to the intuitionistic logic of Chapter 4, the recipe gives the simply typed λ-calculus of Chapter 8. Applied to affine logic, it gives λ^{aff}.

> **NOTE**
> λ^{aff} is a model of a fragment of Rust: values that are moved, never borrowed. When we say "this rule is the move checker", we mean that the rule rejects the same programs that rustc rejects with E0382, for the same reason, on the programs that both of them understand. How rustc actually implements the check (a dataflow analysis on a control-flow graph) is the subject of Chapter 29.

22.2 Syntax

Types

$$\tau, \sigma, \rho ::= b \mid \mathbf{1} \mid \tau \multimap \sigma \mid \tau \otimes \sigma \mid \tau \,\&\, \sigma \mid \tau \oplus \sigma$$

Type	Read as	Rust ≈	Logic
b	base type	`String`, `File`, `Conn`	atom
$\mathbf{1}$	unit	`()`	unit of $\otimes$
$\tau \multimap \sigma$	affine function	`FnOnce(τ) -> σ`	linear implication
$\tau \otimes \sigma$	pair	`(τ, σ)`	multiplicative "and"
$\tau \,\&\, \sigma$	offer of either	one value, two consuming methods	additive "and"
$\tau \oplus \sigma$	sum	an `enum` with two variants	additive "or"

A base type b is an opaque resource: the calculus has no operations on it and no constants of that type. Values of base type enter a program through its context, as the parameters of a Rust function do, and the calculus only describes how they are passed around.

Terms

$$e ::= x \mid \lambda x : \tau.\, e \mid e_1\, e_2 \mid () \mid (e_1, e_2) \mid \texttt{let}\ (x, y) = e_1\ \texttt{in}\ e_2 \mid \texttt{let}\ x = e_1\ \texttt{in}\ e_2$$
$$\mid \langle e_1, e_2 \rangle \mid \mathit{fst}\, e \mid \mathit{snd}\, e \mid \mathit{inl}\, e \mid \mathit{inr}\, e \mid \texttt{case}\ e\ \{x.\, e_1 \mid y.\, e_2\}$$

Most of these are familiar from Chapter 8; the differences are in what the typing rules will allow.

- $\lambda x : \tau.\, e$ is a function that may use its argument at most once. It also *takes* every variable of the context that its body uses, the way a Rust closure takes ownership of what its body consumes. A function is itself a resource: a value of type $\tau \multimap \sigma$ can be called once, like an `FnOnce`.
- (e_1, e_2) builds a pair of type $\tau \otimes \sigma$, and $\texttt{let}\ (x, y) = e_1\ \texttt{in}\ e_2$ takes one apart, binding *both* components at once. There are no projections *fst* and *snd* for $\otimes$; a section below explains why.
- $\langle e_1, e_2 \rangle$ is an offer of type $\tau \,\&\, \sigma$: two computations, of which the consumer will run exactly one, chosen with *fst* or *snd*. The components are not evaluated when the offer is built.
- *inl* e and *inr* e inject into a sum, and $\texttt{case}\ e\ \{x.\, e_1 \mid y.\, e_2\}$ inspects one: if e is *inl* v it runs e_1 with x bound to v, otherwise e_2 with y bound to v. It is Rust's `match` on a two-variant `enum`.
- $\texttt{let}\ x = e_1\ \texttt{in}\ e_2$ is Rust's `let x = e1; e2`. It could be defined as $(\lambda x : \tau.\, e_2)\, e_1$, but it is more convenient as a construct of its own.

Two things are missing on purpose. There is no construct for throwing a value away: discarding is built into the typing rules, as we will see, and Chapter 24 adds an explicit $\texttt{drop}\ x; e$ when it needs to talk about the moment of discarding. And there is no recursion, so every program terminates; general recursion would not change anything in this chapter's theorems (Exercise 6 asks you to check one of them).

In Lean

The Lean development of this chapter lives in four files in `code/lean/Book/Ch22/`: `Syntax.lean`, `Typing.lean`, `Usage.lean` and `Checker.lean`, each importing the previous one. The types are an ordinary inductive type:

```
/-- The types of `λ^aff`. -/
inductive Ty where
  /-- An opaque base type, named like a Rust type: `String`, `File`, ... -/
  | base : String → Ty
  /-- The unit type `1`, Rust's `()`. -/
  | unit : Ty
  /-- Linear (affine) functions `τ ⊸ σ`, Rust's `FnOnce(τ) -> σ`. -/
  | lolli : Ty → Ty → Ty
  /-- The multiplicative pair `τ ⊗ σ`, Rust's `(τ, σ)`. -/
  | tensor : Ty → Ty → Ty
  /-- The additive pair `τ & σ` ("with"): a choice of one of the two. -/
  | amp : Ty → Ty → Ty
  /-- The sum `τ ⊕ σ`, a Rust `enum` with two variants. -/
  | plus : Ty → Ty → Ty
  deriving DecidableEq
```

The additive pair is called `amp` (for the ampersand) because `with` is a keyword in Lean. A few notations make the types readable:

```
@[inherit_doc] scoped infixr:25 " ⊸ " => Ty.lolli
@[inherit_doc] scoped infixl:35 " ⊗ " => Ty.tensor
@[inherit_doc] scoped infixl:35 " & " => Ty.amp
@[inherit_doc] scoped infixl:30 " ⊕ " => Ty.plus
```

The numbers are precedences, and `infixr` makes $\multimap$ associate to the right as $\to$ does. The keyword `scoped` makes each notation active only inside the namespace `Ch22` (or where that namespace is opened), so that it does not leak into other chapters, where $\otimes$ means something else. The symbol `⊕` is also Lean's notation for the sum of two *Lean* types; Lean chooses between the two meanings by the types of the arguments, and here they are always `Ty`s.

For the terms, we use de Bruijn indices, as Chapter 7 introduced them and Chapter 9 used them for its proof of type safety: a variable is the number of binders between it and its binder. Every binding construct says how many variables it binds:

```
/-- The terms of `λ^aff`. Variables are de Bruijn indices: `var 0` is the
nearest enclosing binder. The comments give the named notation of the text. -/
inductive Term where
  | var : Nat → Term                    -- x
  | lam : Ty → Term → Term              -- λx:τ. e          (binds x)
  | app : Term → Term → Term            -- e₁ e₂
  | unit : Term                         -- ()
  | pair : Term → Term → Term           -- (e₁, e₂)          ⊗ introduction
  | letPair : Term → Term → Term        -- let (x, y) = e₁ in e₂  (binds x, y)
  | wpair : Term → Term → Term          -- ⟨e₁, e₂⟩          & introduction
  | fst : Term → Term                   -- fst e
  | snd : Term → Term                   -- snd e
  | inl : Ty → Term → Term              -- inl e  (the Ty is the right summand)
  | inr : Ty → Term → Term              -- inr e  (the Ty is the left summand)
  | case : Term → Term → Term → Term    -- case e of inl x ⇒ e₁ | inr y ⇒ e₂
  | letIn : Term → Term → Term          -- let x = e₁ in e₂  (binds x)
  deriving DecidableEq, Repr
```

Inside the body of a `letPair`, the second component y is `var 0` and the first component x is `var 1`: the innermost binder gets index 0, and we treat y as bound after x. The injections carry the *other* summand as an annotation, so that every term has at most one type, which keeps the checker later in the chapter simple. Here are three of the chapter's examples, written with this convention:

```
/-- Two base types for the examples. -/
def Str : Ty := .base "String"
def File : Ty := .base "File"

/-- `λp. let (x, y) = p in (y, x)`. Inside the `let`, `y` is `var 0`, `x` is
`var 1` and `p` would be `var 2`. -/
def swap (τ σ : Ty) : Term :=
  .lam (τ ⊗ σ) (.letPair (.var 0) (.pair (.var 0) (.var 1)))

/-- `λx. λy. x`: the second argument is discarded. -/
def discard (τ σ : Ty) : Term :=
  .lam τ (.lam σ (.var 1))

/-- `λx. (x, x)`: the argument is used twice. -/
def dup (τ : Ty) : Term :=
  .lam τ (.pair (.var 0) (.var 0))
```

The function at the heart of the chapter counts how many times a term uses a variable:

```
/-- `uses i e` counts the occurrences of the free variable with index `i` in
`e`. Under a binder the index of the same variable goes up by one (by two under
`letPair`, which binds two variables). The two alternatives of an additive pair
and the two branches of a `case` are counted with `max`, not `+`: only one of
them will ever run. -/
def uses (i : Nat) : Term → Nat
  | .var j => if i = j then 1 else 0
  | .lam _ e => uses (i + 1) e
  | .app e₁ e₂ => uses i e₁ + uses i e₂
  | .unit => 0
  | .pair e₁ e₂ => uses i e₁ + uses i e₂
  | .letPair e₁ e₂ => uses i e₁ + uses (i + 2) e₂
  | .wpair e₁ e₂ => max (uses i e₁) (uses i e₂)
  | .fst e => uses i e
  | .snd e => uses i e
  | .inl _ e => uses i e
  | .inr _ e => uses i e
  | .case e e₁ e₂ => uses i e + max (uses (i + 1) e₁) (uses (i + 1) e₂)
  | .letIn e₁ e₂ => uses i e₁ + uses (i + 1) e₂
```

Under one binder the same free variable has a larger index, so the count goes from `i` to `i + 1` (to `i + 2` under `letPair`). Where two subterms both run, as in an application or a pair, the counts add up. Where only one of two alternatives will ever run, in an offer and in the branches of a `case`, the count is the *maximum* of the two. That is the counting convention that the usage theorem needs, and we will come back to it.

```
-- The body of `swap` uses `p` once: the `let` consumes it.
/-- info: 1 -/
#guard_msgs in
#eval uses 0 (.letPair (.var 0) (.pair (.var 0) (.var 1)))

-- Under one binder, the free variable 0 is written `var 1`.
/-- info: 1 -/
#guard_msgs in
#eval uses 0 (.lam Str (.var 1))

-- The body of `dup` uses its variable twice.
/-- info: 2 -/
#guard_msgs in
#eval uses 0 (.pair (.var 0) (.var 0))

-- An additive pair counts the larger of its two components.
/-- info: 1 -/
#guard_msgs in
#eval uses 0 (.wpair (.var 0) (.var 0))
```

Finally, `Syntax.lean` gives `Ty` a printer, so that Lean shows types the way this chapter writes them:

```
/-- info: (String ⊗ File) ⊸ (File ⊗ String) -/
#guard_msgs in
#eval (Str ⊗ File) ⊸ (File ⊗ Str)
```

22.3 Contexts and splitting

A context is a list of distinct variables with their types, $\Gamma = x_1 : \tau_1, \ldots, x_n : \tau_n$. We keep the exchange rule, as every substructural logic of interest to Rust does, so the order of a context does not matter.

The splitting judgment $\Gamma = \Gamma_1 \uplus \Gamma_2$ says that Γ is divided between Γ_1 and Γ_2. It is defined by three rules that walk through the context and send each variable left or right:

$$\frac{}{\cdot = \cdot \uplus \cdot}\ \text{S-Empty} \qquad \frac{\Gamma = \Gamma_1 \uplus \Gamma_2}{\Gamma, x : \tau = (\Gamma_1, x : \tau) \uplus \Gamma_2}\ \text{S-Left} \qquad \frac{\Gamma = \Gamma_1 \uplus \Gamma_2}{\Gamma, x : \tau = \Gamma_1 \uplus (\Gamma_2, x : \tau)}\ \text{S-Right}$$

The dot $\cdot$ is the empty context. Every variable goes to exactly one side, so a context with n variables has 2^n splits. For $x : A, y : B$ there are four: both left, x left and y right, x right and y left, both right. Chapter 23 will add a fourth rule, S-Both, that sends a variable to both sides when its type is `Copy`; Chapter 28 uses all four.

Contexts with holes

In Lean a context cannot simply be a list of types, because de Bruijn indices are positions: if a split removed a variable from the list, every variable after it would be renumbered, and a term would mean something else in each half of the split. So a Lean context keeps every position and marks the variables that are not available with a **hole**:

```
/-- A context: one entry per de Bruijn index. `some τ` is a variable of type
`τ` that the term may use; `none` is a hole, a variable that exists but is not
available to this part of the term (it was given to another part). -/
abbrev Ctx := List (Option Ty)
```

```
/-- `Split Γ Γ₁ Γ₂`: the context `Γ` is divided between `Γ₁` and `Γ₂`. Every
available variable of `Γ` goes to exactly one side and leaves a hole in the
other. Positions never move, so de Bruijn indices mean the same thing in all
three contexts. -/
inductive Split : Ctx → Ctx → Ctx → Prop where
  | nil : Split [] [] []
  | left {Γ Γ₁ Γ₂ : Ctx} {τ : Ty} :
      Split Γ Γ₁ Γ₂ → Split (some τ :: Γ) (some τ :: Γ₁) (none :: Γ₂)
  | right {Γ Γ₁ Γ₂ : Ctx} {τ : Ty} :
      Split Γ Γ₁ Γ₂ → Split (some τ :: Γ) (none :: Γ₁) (some τ :: Γ₂)
  | hole {Γ Γ₁ Γ₂ : Ctx} :
      Split Γ Γ₁ Γ₂ → Split (none :: Γ) (none :: Γ₁) (none :: Γ₂)
```

Rules `left` and `right` are S-Left and S-Right: an available variable goes to one side and leaves a hole on the other. Rule `hole` says that a position that is already a hole stays a hole on both sides. With holes, the three contexts of a split always have the same length, and `var 3` means the same variable in all of them. For example, `Split [some A, some B] [some A, none] [none, some B]` gives variable 0 to the left and variable 1 to the right. The same idea will return in the checker, where "a hole" will mean "already moved".

22.4 The typing rules

The judgment $\Gamma \vdash e : \tau$ reads as before: in context Γ, the term e has type τ. What is new is that the context is a supply of resources, and the rules decide who gets them. Here are all the rules. A bound variable is always assumed to be fresh (not already in the context), as in Chapter 7.

$$\frac{}{\Gamma, x : \tau \vdash x : \tau}\ \text{A-Var} \qquad \frac{}{\Gamma \vdash () : \mathbf{1}}\ \text{A-Unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \sigma}{\Gamma \vdash \lambda x : \tau.\, e : \tau \multimap \sigma}\ \text{A-Abs} \qquad \frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \Gamma_1 \vdash e_1 : \tau \multimap \sigma \qquad \Gamma_2 \vdash e_2 : \tau}{\Gamma \vdash e_1\, e_2 : \sigma}\ \text{A-App}$$

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \Gamma_1 \vdash e_1 : \tau \qquad \Gamma_2 \vdash e_2 : \sigma}{\Gamma \vdash (e_1, e_2) : \tau \otimes \sigma}\ \text{A-Pair}$$

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \Gamma_1 \vdash e_1 : \tau \otimes \sigma \qquad \Gamma_2, x : \tau, y : \sigma \vdash e_2 : \rho}{\Gamma \vdash \texttt{let}\,(x, y) = e_1\,\texttt{in}\,e_2 : \rho}\ \text{A-LetPair}$$

$$\frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash \langle e_1, e_2 \rangle : \tau \,\&\, \sigma}\ \text{A-With} \qquad \frac{\Gamma \vdash e : \tau \,\&\, \sigma}{\Gamma \vdash \mathit{fst}\, e : \tau}\ \text{A-Fst} \qquad \frac{\Gamma \vdash e : \tau \,\&\, \sigma}{\Gamma \vdash \mathit{snd}\, e : \sigma}\ \text{A-Snd}$$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathit{inl}\, e : \tau \oplus \sigma}\ \text{A-Inl} \qquad \frac{\Gamma \vdash e : \sigma}{\Gamma \vdash \mathit{inr}\, e : \tau \oplus \sigma}\ \text{A-Inr}$$

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \Gamma_1 \vdash e : \tau \oplus \sigma \qquad \Gamma_2, x : \tau \vdash e_1 : \rho \qquad \Gamma_2, y : \sigma \vdash e_2 : \rho}{\Gamma \vdash \texttt{case}\, e\, \{x.\, e_1 \mid y.\, e_2\} : \rho}\ \text{A-Case}$$

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \Gamma_1 \vdash e_1 : \tau \qquad \Gamma_2, x : \tau \vdash e_2 : \sigma}{\Gamma \vdash \texttt{let}\, x = e_1\,\texttt{in}\,e_2 : \sigma}\ \text{A-Let}$$

Read aloud, with the Rust construct each rule models:

Rule	In words	Rust ≈
A-Var	a variable has its type; others may idle	using a variable by value moves it
A-Abs	the body may use the argument and anything else	a closure takes what its body uses
A-App	function and argument use disjoint variables	a call partitions what it moves
A-Pair	the components use disjoint variables	`(a, b)` moves `a` and `b`
A-LetPair	take a pair apart, getting both parts	`let (x, y) = p;`
A-With	both alternatives may use everything	a value with two `self` methods
A-Case	scrutinee and branches split; branches share	`match` arms
A-Let	the value and the rest split	`let x = e1; e2`

Three rules to stare at

A-Var: where weakening lives. The axiom allows an arbitrary Γ next to the variable, and A-Unit allows an arbitrary context too. Those extra variables are not used. In a *linear* calculus the axiom would be

$$\frac{}{x : \tau \vdash x : \tau}\ \text{L-Var}$$

with nothing else in the context, and A-Unit would demand an empty context. That one difference is the whole difference between an affine language and a language in which every value must be consumed. Rust is on the affine side: a function may ignore a parameter, and the value is then simply dropped at the end of the function (Chapter 24 is about what "dropped" costs).

A-App and A-Pair: splitting is the move checker. To type (e_1, e_2) you must divide the variables in scope between the two components, and no variable can go to both. When you write `f(a, b)` in Rust, the compiler must be able to partition the owned values in scope between the argument positions; if both arguments mention the same non-`Copy` variable, no partition exists, and you get E0382. Every rule whose premises *both run* splits the context: A-App, A-Pair, A-LetPair, A-Let, and A-Case between the scrutinee and the branches.

A-With and A-Case: sharing because only one alternative runs. In A-With both premises have the same Γ, and in A-Case the two branches have the same Γ_2. This is *not* contraction. An offer $\langle e_1, e_2 \rangle$

runs only the alternative the consumer chooses, and a `case` runs only one branch, so each resource is still consumed at most once *when the program runs*. Rust does the same: the borrow checker checks each arm of a `match` against the same starting state, so a value moved in one arm is not thereby moved in the other.

In Lean

Each rule becomes a constructor of an inductive predicate, with the premises as arguments and the conclusion as the result type:

```
/-- The typing judgment `Γ ⊢ e : τ` of `λ^aff`. Every rule with two
subterms that both run splits the context; the additive rules (`wpair`, and the
two branches of `case`) share it. -/
inductive HasType : Ctx → Term → Ty → Prop where
  | var {Γ : Ctx} {x : Nat} {τ : Ty} :
      Γ[x]? = some (some τ) →
      HasType Γ (.var x) τ
  | lam {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasType (some τ :: Γ) e σ →
      HasType Γ (.lam τ e) (τ ⊸ σ)
  | app {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      Split Γ Γ₁ Γ₂ →
      HasType Γ₁ e₁ (τ ⊸ σ) → HasType Γ₂ e₂ τ →
      HasType Γ (.app e₁ e₂) σ
  | unit {Γ : Ctx} :
      HasType Γ .unit .unit
  | pair {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      Split Γ Γ₁ Γ₂ →
      HasType Γ₁ e₁ τ → HasType Γ₂ e₂ σ →
      HasType Γ (.pair e₁ e₂) (τ ⊗ σ)
  | letPair {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ ρ : Ty} :
      Split Γ Γ₁ Γ₂ →
      HasType Γ₁ e₁ (τ ⊗ σ) → HasType (some σ :: some τ :: Γ₂) e₂ ρ →
      HasType Γ (.letPair e₁ e₂) ρ
  | wpair {Γ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      HasType Γ e₁ τ → HasType Γ e₂ σ →
      HasType Γ (.wpair e₁ e₂) (τ & σ)
  | fst {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasType Γ e (τ & σ) → HasType Γ (.fst e) τ
  | snd {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasType Γ e (τ & σ) → HasType Γ (.snd e) σ
  | inl {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasType Γ e τ → HasType Γ (.inl σ e) (τ ⊕ σ)
  | inr {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasType Γ e σ → HasType Γ (.inr τ e) (τ ⊕ σ)
  | case {Γ Γ₁ Γ₂ : Ctx} {e e₁ e₂ : Term} {τ σ ρ : Ty} :
      Split Γ Γ₁ Γ₂ →
      HasType Γ₁ e (τ ⊕ σ) →
      HasType (some τ :: Γ₂) e₁ ρ → HasType (some σ :: Γ₂) e₂ ρ →
      HasType Γ (.case e e₁ e₂) ρ
  | letIn {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      Split Γ Γ₁ Γ₂ →
      HasType Γ₁ e₁ τ → HasType (some τ :: Γ₂) e₂ σ →
      HasType Γ (.letIn e₁ e₂) σ
```

Rule `var` says that variable `x` has type `τ` when position `x` of the context holds `some τ`. The notation `Γ[x]?` is Lean's indexing that returns an `Option` (`none` when `x` is out of range), like Rust's `slice.get(x)`; we write `some (some τ)` because the context entries are themselves options. Nothing is said about the other positions: that is A-Var's weakening. The binding rules extend the context at the front, with `some τ :: Γ`, because the innermost variable has index 0. The implicit arguments in braces are the metavariables of the rule, which Lean will infer whenever we use a constructor.

22.5 Worked derivations

Swap is typable

The function that swaps a pair uses each of its variables exactly once:

$$\vdash \lambda p.\,\texttt{let}\ (x, y) = p\ \texttt{in}\ (y, x) : (\tau \otimes \sigma) \multimap (\sigma \otimes \tau)$$

Read the derivation from the bottom. A-Abs puts p in the context. A-LetPair must split $p : \tau \otimes \sigma$ between the scrutinee and the body; the scrutinee is p itself, so p goes left and the body gets the empty context, extended with x and y. Finally A-Pair splits $x : \tau, y : \sigma$ so that y goes to the first component and x to the second. Writing the pair's sub-derivation separately, and leaving the splits out of the trees:

$$\mathcal{D} = \dfrac{\dfrac{}{y : \sigma \vdash y : \sigma}\ \text{A-Var} \qquad \dfrac{}{x : \tau \vdash x : \tau}\ \text{A-Var}}{x : \tau, y : \sigma \vdash (y, x) : \sigma \otimes \tau}\ \text{A-Pair}$$

$$\dfrac{\dfrac{\dfrac{}{p : \tau \otimes \sigma \vdash p : \tau \otimes \sigma}\ \text{A-Var} \qquad \mathcal{D}}{p : \tau \otimes \sigma \vdash \texttt{let}\ (x, y) = p\ \texttt{in}\ (y, x) : \sigma \otimes \tau}\ \text{A-LetPair}}{\vdash \lambda p.\,\texttt{let}\ (x, y) = p\ \texttt{in}\ (y, x) : (\tau \otimes \sigma) \multimap (\sigma \otimes \tau)}\ \text{A-Abs}$$

with the splits $p : \tau \otimes \sigma = (p : \tau \otimes \sigma) \uplus \cdot$ and $x : \tau, y : \sigma = (y : \sigma) \uplus (x : \tau)$. In Lean the same derivation is a term, one constructor per rule, and the splits appear as arguments:

```
/-- Swap is typable: `⊢ λp. let (x, y) = p in (y, x) : (τ ⊗ σ) ⊸ (σ ⊗ τ)`. -/
theorem swap_typed (τ σ : Ty) : HasType [] (swap τ σ) ((τ ⊗ σ) ⊸ (σ ⊗ τ)) :=
  .lam                                  -- context: p
    (.letPair (.left .nil)              -- p goes to the `let`, nothing else
      (.var rfl)                        -- p : τ ⊗ σ
      (.pair (.left (.right (.hole .nil)))  -- y left, x right, p is a hole
        (.var rfl)                      -- y : σ
        (.var rfl)))                    -- x : τ
```

The split for the `letPair` is `.left .nil`: the context has one entry, p, and it goes left. The body's context is then `[some σ, some τ, none]`: y, x, and the hole where p was. The pair's split `.left (.right (.hole .nil))` reads position by position: y left, x right, the hole stays a hole. Each leaf is `.var rfl`: the premise `Γ[x]? = some (some τ)` is about a concrete list, so Lean computes the lookup and `rfl` confirms it.

The `affine` crate, which we meet properly later in the chapter, prints the derivation it finds for the same function written in Rust syntax. Each line is a judgment with the context that the subterm actually used:

```
ok: fn swap(p: (String, File)) -> (File, String)
[A-LetPair] p: (String, File) ⊢ let (s, f) = p; (f, s) : (File, String)
  [A-Var] p: (String, File) ⊢ p : (String, File)
  [A-Pair] s: String, f: File ⊢ (f, s) : (File, String)
    [A-Var] f: File ⊢ f : File
    [A-Var] s: String ⊢ s : String
```

And Rust, of course, accepts the function. This chapter's Rust example, `code/rust/examples/ch22_affine_in_rust.rs`, writes the terms of the calculus as Rust functions:

```
/// `λp. let (x, y) = p in (y, x)`: each variable is used exactly once.
fn swap<A, B>(p: (A, B)) -> (B, A) {
    let (x, y) = p;
```

```
    (y, x)
}

/// The same function with Rust's projections. Rust tracks moves per place,
/// and `p.0` and `p.1` are different places.
fn swap_fields<A, B>(p: (A, B)) -> (B, A) {
    (p.1, p.0)
}
```

Discarding is typable

The function $\lambda x.\, \lambda y.\, x$ ignores its second argument:

$$\dfrac{\dfrac{\dfrac{}{x : \tau, y : \sigma \vdash x : \tau}\,\text{A-Var}}{x : \tau \vdash \lambda y.\, x : \sigma \multimap \tau}\,\text{A-Abs}}{\vdash \lambda x.\, \lambda y.\, x : \tau \multimap \sigma \multimap \tau}\,\text{A-Abs}$$

The only interesting step is the top one: A-Var accepts the unused y in the context. In Lean the derivation is three constructors:

```
/-- Discarding is typable: `⊢ λx. λy. x : τ ⊸ σ ⊸ τ`. Rule `var` accepts the
unused `y` in the context: that is weakening. -/
theorem discard_typed (τ σ : Ty) : HasType [] (discard τ σ) (τ ⊸ σ ⊸ τ) :=
  .lam (.lam (.var rfl))
```

In Rust, discarding is legal too, but it is not invisible. Here is the Rust version, called with a `File` whose destructor writes to a log:

```
/// `λx. λy. x`: `y` is discarded. In the calculus that is weakening and
/// nothing happens; in Rust the discarded value is dropped.
fn first<A, B>(x: A, _y: B) -> A {
    x
}
```

```
log: first: start
log: close c.txt
log: first: returned "kept"
```

The file is closed *inside* `first`, before it returns: the discarded value was dropped at the end of the function. The typing rule is the calculus's weakening; the closing is what Chapter 24 calls weakening with a side effect.

Duplication is not

Now try $\lambda x.\, (x, x)$ at type $\tau \multimap \tau \otimes \tau$. A-Abs leaves us with $x : \tau \vdash (x, x) : \tau \otimes \tau$, and the only rule for a pair is A-Pair, which needs a split $x : \tau = \Gamma_1 \uplus \Gamma_2$ with $\Gamma_1 \vdash x : \tau$ and $\Gamma_2 \vdash x : \tau$. The only rule that concludes a variable is A-Var, which needs x in the context. So x would have to be in both Γ_1 and Γ_2, and no split does that. There is no derivation.

This argument proves more than it seems. The rule of contraction,

$$\dfrac{\Gamma, x : \tau, y : \tau \vdash e : \rho}{\Gamma, z : \tau \vdash e[z/x, z/y] : \rho}\,\text{Contr}$$

is **not admissible** in λ^{aff} (in the sense of Chapter 6: we cannot add it without changing what is derivable). Its premise is derivable with $e = (x, y)$, and its conclusion would be $z : \tau \vdash (z, z)$, which we just showed is not. In Lean, the argument becomes a proof by inversion:

```
/-- `(z, z)` has no type in a context where `z` is the variable 0, whatever the
rest of the context and whatever the type: rule `pair` must split the context,
and only one side receives `z`. -/
theorem no_pair_twice (Γ : Ctx) (τ ρ : Ty) :
    ¬ HasType (some τ :: Γ) (.pair (.var 0) (.var 0)) ρ := by
  intro h
  -- The only rule that concludes a pair is `pair`: it gives a split and two
  -- derivations, one for each component.
  cases h with
  | pair hs h₁ h₂ =>
    -- The only rule that concludes a variable is `var`: each side must have
    -- `some _` at position 0.
    cases h₁ with
    | var g₁ =>
      cases h₂ with
      | var g₂ =>
        -- The split puts position 0 on one side only; the other side has a
        -- hole there, which contradicts `g₁` or `g₂`.
        cases hs <;> simp at g₁ g₂
```

`cases h` asks which constructor could have produced a derivation of a pair; the only candidate is `pair`, and Lean names its premises: the split `hs` and the two derivations `h₁` and `h₂`. `cases h₁` and `cases h₂` do the same for the variables: only `var` fits, which gives `g₁ : Γ₁[0]? = some (some _)` and the same for `Γ₂`. Finally `cases hs` tries the constructors of `Split` for a context whose first entry is `some τ`: `left` puts a hole at position 0 of `Γ₂`, `right` puts one in `Γ₁`, and `hole` and `nil` do not fit at all. The combinator `<;>` runs `simp at g₁ g₂` on every remaining case, and in each one `simp` evaluates the lookup of the hole to `none`, turns the hypothesis into `none = some _`, recognizes it as false and closes the goal.

Rust rejects the same function, and its message describes the failed split:

```
fn dup(s: String) -> (String, String) {
    (s, s)
}
```

```
error[E0382]: use of moved value: `s`
 --> compile_fail/ch22_dup.rs:6:9
  |
5 | fn dup(s: String) -> (String, String) {
  |        - move occurs because `s` has type `String`, which does not implement the `Copy`
trait
6 |     (s, s)
  |      -  ^ value used here after move
  |      |
  |      value moved here
  |
help: consider cloning the value if the performance cost is acceptable
  |
6 |     (s.clone(), s)
  |       ++++++++
```

"Value moved here" is the first component taking s; "value used here after move" is the second component finding nothing left. The help at the end is a glimpse of Chapter 23: `clone` is an *explicit* duplication, a term rather than a structural rule.

22.6 Sharing: offers and branches

The additive rules are where Girard's coin becomes a program. Suppose the environment provides two functions, $\mathit{coffee} : \mathit{Coin} \multimap \mathit{Coffee}$ and $\mathit{tea} : \mathit{Coin} \multimap \mathit{Tea}$. (In the calculus a function in the context would be a resource too, usable once; we treat these two as constants, available in every context, the way a Rust program can call a declared function as often as it likes. Chapter 23 explains why Rust may do that.) Then

$$c : \mathit{Coin} \vdash \langle \mathit{coffee}\, c, \mathit{tea}\, c \rangle : \mathit{Coffee} \,\&\, \mathit{Tea}$$

is derivable by A-With, both alternatives using c, while

$$c : \mathit{Coin} \vdash (\mathit{coffee}\, c, \mathit{tea}\, c) : \mathit{Coffee} \otimes \mathit{Tea}$$

is not, for the same reason as (x, x). The crate's derivation of the offer shows the coin in both premises:

```
ok: fn offer(c: Coin) -> With<Coffee, Tea>
[A-With] c: Coin ⊢ with(buy_coffee(c), buy_tea(c)) : With<Coffee, Tea>
  [Call] c: Coin ⊢ buy_coffee(c) : Coffee
    [A-Var] c: Coin ⊢ c : Coin
  [Call] c: Coin ⊢ buy_tea(c) : Tea
    [A-Var] c: Coin ⊢ c : Coin
```

Rust has no built-in $\&$, but the idea is common in Rust APIs: a value with several methods that take `self`. You may call any one of them, and calling it uses the value up. The chapter's Rust example has a voucher, and a generic version that stores the shared resources together with the two alternatives:

```rust
struct Coin;
#[derive(Debug)]
struct Coffee;
#[derive(Debug)]
struct Tea;

fn buy_coffee(_c: Coin) -> Coffee {
    Coffee
}

fn buy_tea(_c: Coin) -> Tea {
    Tea
}

/// `Coffee & Tea`: one coin, two ways to spend it. Both methods take `self`,
/// so whichever you call consumes the voucher, and the other is gone.
struct Voucher {
    coin: Coin,
}

impl Voucher {
    fn coffee(self) -> Coffee {
        buy_coffee(self.coin)
    }
    fn tea(self) -> Tea {
        buy_tea(self.coin)
    }
}

/// The general form: the shared resources `R` and the two alternatives.
struct With<R, A, B> {
    resources: R,
    fst: fn(R) -> A,
    snd: fn(R) -> B,
}
```

```
impl<R, A, B> With<R, A, B> {
    fn fst(self) -> A {
        (self.fst)(self.resources)
    }
    fn snd(self) -> B {
        (self.snd)(self.resources)
    }
}
```

```
// Each voucher (each offer) is spent once, on the alternative we choose.
let coffee = Voucher { coin: Coin }.coffee();
let tea = Voucher { coin: Coin }.tea();
println!("vouchers: {coffee:?}, {tea:?}");
let offer = || With {
    resources: Coin,
    fst: buy_coffee,
    snd: buy_tea,
};
let (coffee, tea) = (offer().fst(), offer().snd());
println!("offers:   {coffee:?}, {tea:?}");
```

Each voucher, each offer, is spent once. Spending one voucher twice is exactly the $\otimes$ derivation that does not exist:

```
impl Voucher {
    pub fn coffee(self) -> Coffee {
        drop(self.coin);
        Coffee
    }
    pub fn tea(self) -> Tea {
        drop(self.coin);
        Tea
    }
}

pub fn both(v: Voucher) -> (Coffee, Tea) {
    (v.coffee(), v.tea())
}
```

```
error[E0382]: use of moved value: `v`
  --> compile_fail/ch22_voucher_both.rs:26:18
   |
25 | pub fn both(v: Voucher) -> (Coffee, Tea) {
   |             - move occurs because `v` has type `Voucher`, which does not implement the
`Copy` trait
26 |     (v.coffee(), v.tea())
   |        --------  ^ value used here after move
   |        |
   |        `v` moved due to this method call
   |
note: `Voucher::coffee` takes ownership of the receiver `self`, which moves `v`
  --> compile_fail/ch22_voucher_both.rs:15:19
   |
15 |     pub fn coffee(self) -> Coffee {
   |                   ^^^^
```

The note names the mechanism: `coffee` takes its receiver `self` by value, so calling it is a move of `v`, like passing `v` to a function.

The two branches of a `case` share their context in the same way. Here is a settlement that ships an order or refunds it, using the connection `c` in both branches, in the crate's syntax and with the crate's derivation:

```
ok: fn settle(m: Either<Order, Refund>, c: Conn) -> Conn
[A-Case] m: Either<Order, Refund>, c: Conn ⊢ match m { Left(o) => …, Right(r) => … } : Conn
  [A-Var] m: Either<Order, Refund> ⊢ m : Either<Order, Refund>
  [Call] c: Conn, o: Order ⊢ ship(c, o) : Conn
    [A-Var] c: Conn ⊢ c : Conn
    [A-Var] o: Order ⊢ o : Order
  [Call] c: Conn, r: Refund ⊢ refund(c, r) : Conn
    [A-Var] c: Conn ⊢ c : Conn
    [A-Var] r: Refund ⊢ r : Refund
```

Each branch premise has `c` in its context. In Rust:

```rust
enum Either<A, B> {
    Left(A),
    Right(B),
}

/// Both arms use `c`: only one of them runs.
fn settle(m: Either<String, String>, c: Conn) -> Conn {
    match m {
        Either::Left(order) => send(c, format!("ship {order}")),
        Either::Right(refund) => send(c, format!("refund {refund}")),
    }
}
```

This sharing is also why `uses` takes the maximum over branches. A variable that appears once in each branch is used once on every run, so its count is $\max(1, 1) = 1$, not 2. The other side of the coin is that a variable moved in *one* branch must be considered moved after the `case`, because the type checker does not know which branch will run. The crate reports this case with the move in the left arm:

```
error[E0382]: use of moved value: `log`
 --> ch22_route.aff:8:9
  |
3 | fn route(m: Either<String, String>, c: Conn, log: String) -> (Conn, String) {
  |                                              --- move occurs because `log` has type
`String`, which does not implement the `Copy` trait
4 |     let c = match m {
5 |         Left(order) => send(c, log),
  |                                --- value moved here
...
8 |     (c, log)
  |         ^^^ value used here after move
```

22.7 Why ⊗ has no projections

In the simply typed λ-calculus a pair has two projections, and $\lambda p.\,(\mathit{snd}\,p, \mathit{fst}\,p)$ swaps it. In λ^{aff} that term is not typable: it uses p twice. Projections are the wrong eliminator for $\otimes$. A projection keeps one component and silently abandons the other, the *orphaned* component; and two projections need two uses of the pair.

In an affine calculus a projection can still be *defined*, because abandoning a value is weakening, which we have:

```
/-- A first projection for `⊗`, defined with the `let`: `let (x, y) = e in x`.
The second component `y` is discarded. -/
def tfst (e : Term) : Term := .letPair e (.var 1)

/-- The second projection, `let (x, y) = e in y`. -/
def tsnd (e : Term) : Term := .letPair e (.var 0)
```

```
theorem tfst_typed {Γ : Ctx} {e : Term} {τ σ : Ty} (h : HasType Γ e (τ ⊗ σ)) :
    HasType Γ (tfst e) τ :=
  .letPair (Split.all_left Γ) h (.var rfl)

/-- But the two projections cannot both be applied to the same pair. -/
theorem no_both_projections (Γ : Ctx) (τ σ ρ : Ty) :
    ¬ HasType (some (τ ⊗ σ) :: Γ) (.pair (tfst (.var 0)) (tsnd (.var 0))) ρ := by
  intro h
  have := usage h 0
  simp [tfst, tsnd, uses, avail] at this
```

`tfst e` is `let` $(x, y) = e$ `in` x, with y discarded, and `tfst_typed` derives it for any well-typed pair (`Split.all_left` sends the whole context to the scrutinee). But `no_both_projections` shows that the two projections cannot both be applied to the same variable: the proof is one application of the usage theorem of the next section, which finds variable 0 used twice. So the `let` that binds both components at once is not a convenience. It is the only way to get both halves of a pair while using the pair once.

Rust refines this. It tracks moves not per variable but per **place**: `p`, `p.0` and `p.1` are different places, and moving out of `p.0` leaves `p.1` where it was. That is why `swap_fields` above, written `(p.1, p.0)`, compiles. What Rust does not allow is using `p` as a whole after one of its parts has been moved:

```
fn split(p: (String, File)) -> (String, (String, File)) {
    let s = p.0;
    (s, p)
}
```

```
error[E0382]: use of partially moved value: `p`
  --> compile_fail/ch22_partial_move.rs:10:9
   |
 9 |     let s = p.0;
   |             --- value partially moved here
10 |     (s, p)
   |         ^ value used here after partial move
   |
   = note: partial move occurs because `p.0` has type `String`, which does not implement the
`Copy` trait
```

After `let s = p.0;` the variable `p` is *partially moved*: a pair with a hole in it. Its remaining part is still owned, and if nothing moves it, it is dropped at the end of the scope (Chapter 24 describes how the compiler drops only the fields that are still there). In the language of this chapter, rustc applies the splitting discipline to places instead of variables, and the calculus's `let` elimination is the special case in which every field of the pair is moved at the same time.

22.8 The usage theorem

We can now state and prove the theorem that the whole chapter has been preparing.

> **Theorem (usage).** If $\Gamma \vdash e : \tau$, then every variable of Γ occurs free in e at most once, counting occurrences in subterms that both run with $+$ and occurrences in the two alternatives of an offer or a `case` with $\max$. A variable that is not in Γ does not occur free in e at all.

The proof on paper is a short induction on the derivation. A-Var: the variable of the axiom occurs once, every other variable zero times. A-Abs and the one-premise rules: the induction hypothesis, for the same variable. The splitting rules: a variable of Γ goes to exactly one premise, where by induction it occurs at most once, and it is absent from the other premise's context, where by induction it does not occur; one plus zero is at most one. The sharing rules: by induction the variable occurs at most once in each alternative, and the maximum of two numbers at most one is at most one.

The Lean proof follows the paper proof, but it needs one idea to make "goes to exactly one premise" precise. We count how many times each position of a context *may* be used:

```
/-- How many times the term may use the variable `i` of `Γ`: once if it is
available, never if it is a hole or does not exist. -/
def avail (Γ : Ctx) (i : Nat) : Nat :=
  match Γ[i]? with
  | some (some _) => 1
  | _ => 0
```

The key lemma says that a split hands out exactly what the context had:

```
/-- Splitting divides resources and creates none: what `Γ` offers for the
variable `i` is the sum of what the two parts offer. -/
theorem Split.avail_add {Γ Γ₁ Γ₂ : Ctx} (hs : Split Γ Γ₁ Γ₂) (i : Nat) :
    avail Γ i = avail Γ₁ i + avail Γ₂ i := by
  induction hs generalizing i with
  | nil => simp [avail]
  | left _ ih | right _ ih | hole _ ih =>
    cases i with
    | zero => simp [avail]            -- position 0: 1 = 1 + 0, 1 = 0 + 1, 0 = 0 + 0
    | succ i => simpa using ih i      -- other positions: the induction hypothesis
```

It is proved by induction on the split, for every position `i`. At position 0, each of the three constructors gives a different equation, $1 = 1 + 0$, $1 = 0 + 1$ or $0 = 0 + 0$, and `simp` checks each by computing `avail`. At a position `i + 1`, all three constructors behave the same: the head entries are irrelevant, and the equation is the induction hypothesis at `i`. (`simpa using ih i` simplifies both the goal and `ih i` with the same lemmas, among them `avail_cons_succ`, which drops a head entry, and then closes the goal with the simplified hypothesis.) The three cases share one proof thanks to the `| left _ ih | right _ ih | hole _ ih =>` pattern, which handles several cases with the same script.

Now the theorem itself. It is stated for every variable at once, with `avail` playing the role of "is in Γ":

```
/-- **The usage theorem.** In a term typed in `Γ`, each variable is used at
most as often as `Γ` makes it available. -/
theorem usage {Γ : Ctx} {e : Term} {τ : Ty} (h : HasType Γ e τ) :
    ∀ i, uses i e ≤ avail Γ i := by
  induction h with
  | @var Γ x τ hx =>
    intro i
    by_cases hi : i = x
    · subst hi; simp [uses, avail, hx]   -- the variable itself: 1 ≤ 1
    · simp [uses, hi]                    -- any other variable: 0 ≤ ...
  | unit => intro i; simp [uses]
  | lam _ ih =>
    -- Under the binder, our variable has index i + 1.
    intro i; simpa [uses] using ih (i + 1)
  | app hs _ _ ih₁ ih₂ | pair hs _ _ ih₁ ih₂ =>
    -- The two subterms are typed in the two parts of a split.
    intro i
    have h₁ := ih₁ i
```

```
    have h₂ := ih₂ i
    have hs' := hs.avail_add i
    simp only [uses]; omega
  | letPair hs _ _ ih₁ ih₂ =>
    intro i
    have h₁ := ih₁ i
    have h₂ := ih₂ (i + 2)
    have hs' := hs.avail_add i
    simp only [uses]; simp at h₂; omega
  | letIn hs _ _ ih₁ ih₂ =>
    intro i
    have h₁ := ih₁ i
    have h₂ := ih₂ (i + 1)
    have hs' := hs.avail_add i
    simp only [uses]; simp at h₂; omega
  | case hs _ _ _ ih ih₁ ih₂ =>
    -- The scrutinee and the branches split; the two branches share.
    intro i
    have h := ih i
    have h₁ := ih₁ (i + 1)
    have h₂ := ih₂ (i + 1)
    have hs' := hs.avail_add i
    simp only [uses]; simp at h₁ h₂; omega
  | wpair _ _ ih₁ ih₂ =>
    -- The two components share the context: count the larger of the two.
    intro i
    have h₁ := ih₁ i
    have h₂ := ih₂ i
    simp only [uses]; omega
  | fst _ ih | snd _ ih | inl _ ih | inr _ ih =>
    intro i; simpa [uses] using ih i
```

Each case is one line of the paper proof.

- **`var`**. The pattern `@var Γ x τ hx` names the implicit arguments of the constructor, so that we can talk about `x`. `by_cases hi : i = x` splits the proof in two. If `i = x`, then `subst` replaces `i` with `x` everywhere, and `simp` computes `uses x (.var x) = 1` and, with `hx`, `avail Γ x = 1`. If not, `uses i (.var x)` is 0, and $0 \le$ anything.
- **`unit`**. `uses i .unit = 0`.
- **`lam`**. Under the binder our variable has index `i + 1`, so we apply the induction hypothesis there; `simpa [uses]` turns `uses i (.lam τ e)` into `uses (i + 1) e` and `avail (some τ :: Γ) (i + 1)` into `avail Γ i`.
- **`app`** and **`pair`**. The heart of the proof. We collect three facts with `have`: the two induction hypotheses, one for each part of the split, and the splitting lemma $avail\ \Gamma\ i = avail\ \Gamma_1\ i + avail\ \Gamma_2\ i$. After `simp only [uses]` the goal is `uses i e₁ + uses i e₂ ≤ avail Γ i`, and `omega`, the decision procedure for linear arithmetic over the naturals, puts the three facts together.
- **`letPair`**, **`letIn`** and **`case`**. The same, except that the body's induction hypothesis is at index `i + 2` or `i + 1`, and `simp at h₂` first rewrites the extended context back to `Γ₂`.
- **`wpair`**. No split: both components have the whole context, and the goal is `max (uses i e₁) (uses i e₂) ≤ avail Γ i`. `omega` understands `max`, so the two induction hypotheses are enough. Had `uses` added the two counts instead, this case, and the corresponding part of `case`, would be false.
- **`fst`**, **`snd`**, **`inl`**, **`inr`**. The induction hypothesis, unchanged.

Two corollaries put the theorem in its familiar form:

```
/-- Every variable is used at most once. -/
theorem at_most_once {Γ : Ctx} {e : Term} {τ : Ty} (h : HasType Γ e τ) (i : Nat) :
    uses i e ≤ 1 :=
  Nat.le_trans (usage h i) (avail_le_one Γ i)

/-- A hole is never used. -/
theorem hole_unused {Γ : Ctx} {e : Term} {τ : Ty} (h : HasType Γ e τ) {i : Nat}
    (hi : Γ[i]? = some none) : uses i e = 0 := by
  have := usage h i
  simp [avail, hi] at this
  exact this
```

The first is "every variable is used at most once", since `avail` is never more than 1. The second says that a hole, a variable that this part of the program did not receive, is not used at all. And the impossibility proofs of the previous section become one-liners. `dup` has no type in *any* context, and contraction fails:

```
/-- `λx. (x, x)` has no type, in any context. -/
theorem dup_untypable (Γ : Ctx) (τ ρ : Ty) : ¬ HasType Γ (dup τ) ρ := by
  intro h
  cases h with
  | lam h =>
    -- The body uses variable 0 twice, but the context offers it once.
    have := usage h 0
    simp [uses, avail] at this
```

```
/-- Contraction is not admissible: `x : String, y : String ⊢ (x, y)` is
derivable, but identifying `x` and `y` gives `z : String ⊢ (z, z)`, which is not.
(In the de Bruijn notation, `y` is variable 0 and `x` is variable 1.) -/
theorem contraction_fails :
    HasType [some Str, some Str] (.pair (.var 1) (.var 0)) (Str ⊗ Str) ∧
    ¬ HasType [some Str] (.pair (.var 0) (.var 0)) (Str ⊗ Str) := by
  constructor
  · exact .pair (.right (.left .nil)) (.var rfl) (.var rfl)
  · intro h
    have := usage h 0
    simp [uses, avail] at this
```

In `contraction_fails`, the first part is the premise of the contraction rule (a derivation, with the split `.right (.left .nil)`: variable 0 goes right, variable 1 left), and the second part shows that its conclusion has none.

The theorem is a statement about the program *text*. Its meaning for the program's *runs* is that a resource bound to a variable reaches at most one place that will be executed: the job of the next section is to say why that survives evaluation. It is also a statement about *variables*. Rust counts uses of places, which is finer, as we saw; and Rust's `Copy` types are exempt, which is Chapter 23.

22.9 More metatheory

Weakening is admissible

A-Var and A-Unit accept unused variables, but that alone does not show that *every* derivation survives the addition of a variable to its context: the splitting rules must decide where the new variable goes. The statement and proof are about contexts in which some holes have been filled:

```
/-- `Wk Γ Γ'`: the context `Γ'` is `Γ` with some holes filled in by
variables. -/
inductive Wk : Ctx → Ctx → Prop where
```

```
  | nil : Wk [] []
  | keep {Γ Γ' : Ctx} {o : Option Ty} : Wk Γ Γ' → Wk (o :: Γ) (o :: Γ')
  | fill {Γ Γ' : Ctx} {τ : Ty} : Wk Γ Γ' → Wk (none :: Γ) (some τ :: Γ')
```

```
/-- If `Γ` splits into `Γ₁` and `Γ₂`, and `Γ'` has more variables than `Γ`, then
`Γ'` splits into `Γ₂` and a weakening of `Γ₁`: the new variables go left. -/
theorem Split.wk {Γ Γ₁ Γ₂ Γ' : Ctx} (hs : Split Γ Γ₁ Γ₂) (hw : Wk Γ Γ') :
    ∃ Γ₁', Split Γ' Γ₁' Γ₂ ∧ Wk Γ₁ Γ₁' := by
  induction hs generalizing Γ' with
  | nil => cases hw; exact ⟨[], .nil, .nil⟩
  | left _ ih =>
    cases hw with
    | keep hw =>
      obtain ⟨Γ₁', hs', hw'⟩ := ih hw
      exact ⟨_, .left hs', .keep hw'⟩
  | right _ ih =>
    cases hw with
    | keep hw =>
      obtain ⟨Γ₁', hs', hw'⟩ := ih hw
      exact ⟨_, .right hs', .keep hw'⟩
  | hole _ ih =>
    cases hw with
    | keep hw =>
      obtain ⟨Γ₁', hs', hw'⟩ := ih hw
      exact ⟨_, .hole hs', .keep hw'⟩
    | fill hw =>
      obtain ⟨Γ₁', hs', hw'⟩ := ih hw
      exact ⟨_, .left hs', .fill hw'⟩
```

`Split.wk` makes the choice: the new variables go left. The proof walks along the split; at a position where the old context had a hole and the new one has a variable (`fill`), it uses `left` instead of `hole` . The `obtain` tactic takes apart the existential of the induction hypothesis (as `let (a, b) = ...` does in Rust), giving names to the witness and to the two facts about it. Weakening itself is then an induction on the derivation:

```
/-- **Weakening is admissible**: a term that is well typed in `Γ` is well
typed in any context with more variables. -/
theorem weaken {Γ : Ctx} {e : Term} {τ : Ty} (h : HasType Γ e τ) :
    ∀ {Γ'}, Wk Γ Γ' → HasType Γ' e τ := by
  induction h with
  | var hx => intro _ hw; exact .var (hw.get hx)
  | lam _ ih => intro _ hw; exact .lam (ih (.keep hw))
  | unit => intro _ _; exact .unit
  | app hs _ h₂ ih₁ _ =>
    -- Split the bigger context so that the new variables go to e₁.
    intro _ hw
    obtain ⟨_, hs', hw'⟩ := hs.wk hw
    exact .app hs' (ih₁ hw') h₂
  | pair hs _ h₂ ih₁ _ =>
    intro _ hw
    obtain ⟨_, hs', hw'⟩ := hs.wk hw
    exact .pair hs' (ih₁ hw') h₂
  | letPair hs _ h₂ ih₁ _ =>
    intro _ hw
    obtain ⟨_, hs', hw'⟩ := hs.wk hw
    exact .letPair hs' (ih₁ hw') h₂
  | case hs _ h₁ h₂ ih _ _ =>
    intro _ hw
    obtain ⟨_, hs', hw'⟩ := hs.wk hw
    exact .case hs' (ih hw') h₁ h₂
  | letIn hs _ h₂ ih₁ _ =>
    intro _ hw
    obtain ⟨_, hs', hw'⟩ := hs.wk hw
    exact .letIn hs' (ih₁ hw') h₂
  | wpair _ _ ih₁ ih₂ => intro _ hw; exact .wpair (ih₁ hw) (ih₂ hw)
```

```
  | fst _ ih => intro _ hw; exact .fst (ih hw)
  | snd _ ih => intro _ hw; exact .snd (ih hw)
  | inl _ ih => intro _ hw; exact .inl (ih hw)
  | inr _ ih => intro _ hw; exact .inr (ih hw)
```

In each splitting rule, the bigger context is split with `hs.wk` and the induction hypothesis is applied to the left premise. Putting the variables on the right would work as well and give a different derivation: weakening is admissible, but the derivation it produces is not unique. Weakening is not a curiosity here; the checker's soundness proof below uses it in several places.

Substitution and preservation

To say how programs *run*, we give λ^{aff} a call-by-value semantics. The values are

$$v ::= \lambda x : \tau.\, e \mid () \mid (v_1, v_2) \mid \langle e_1, e_2 \rangle \mid \mathit{inl}\, v \mid \mathit{inr}\, v$$

(an offer is a value with *unevaluated* components: nothing runs until the consumer chooses), and the computation rules are

$$(\lambda x : \tau.\, e)\, v \to e[v/x] \qquad \texttt{let}\ (x, y) = (v_1, v_2)\ \texttt{in}\ e \to e[v_1/x][v_2/y] \qquad \texttt{let}\ x = v\ \texttt{in}\ e \to e[v/x]$$

$$\mathit{fst}\, \langle e_1, e_2 \rangle \to e_1 \qquad \mathit{snd}\, \langle e_1, e_2 \rangle \to e_2 \qquad \texttt{case}\ \mathit{inl}\, v\ \{x.\, e_1 \mid y.\, e_2\} \to e_1[v/x] \qquad \texttt{case}\ \mathit{inr}\, v\ \{x.\, e_1 \mid y.\, e_2\} \to e_2[v/y]$$

together with rules that evaluate subterms from left to right, as in Chapter 9. Notice the two rules for offers: the alternative that is not chosen is thrown away without running. That is weakening at run time.

Lemma (substitution). If $\Gamma, x : \sigma \vdash e : \tau$ and $\Delta \vdash v : \sigma$, where Γ and Δ have no variable in common, then $\Gamma \uplus \Delta \vdash e[v/x] : \tau$.

The proof is by induction on the derivation of e, and its interesting cases are exactly the two kinds of rules. In a **splitting** rule, x went to exactly one premise. Substitute there, using the induction hypothesis, and leave the other premise alone: by the usage theorem, x does not occur in it, so substitution does not change it. The resources Δ of v join the side that received x. In a **sharing** rule ($\langle e_1, e_2 \rangle$ or the branches of a `case`), x may occur in both alternatives, and we substitute in both. The value v is then copied *syntactically*, but only one copy will ever run, and the same derivation $\Delta \vdash v : \sigma$ serves in each alternative. This is the precise sense in which sharing is not contraction.

Theorem (preservation). If $\Gamma \vdash e : \tau$ and $e \to e'$, then $\Gamma \vdash e' : \tau$.

Each computation rule replaces a destructor applied to a constructor by a substitution, and the substitution lemma retypes the result, in the part of the context that the redex used; weakening gives back the rest. The rules for offers need only weakening. Progress (a closed well-typed term is a value or steps) holds as in Chapter 9. Neither proof is mechanized in this chapter: they have exactly the shape of Chapter 9's mechanized proofs, with the extra bookkeeping of holes in the substitution lemma, and the usage theorem is the part that is new.

Together, preservation and the usage theorem say what affinity means at run time: at every step of a run, the program is well typed, so every resource variable occurs in at most one place that can still execute.

22.10 Checking with leftovers

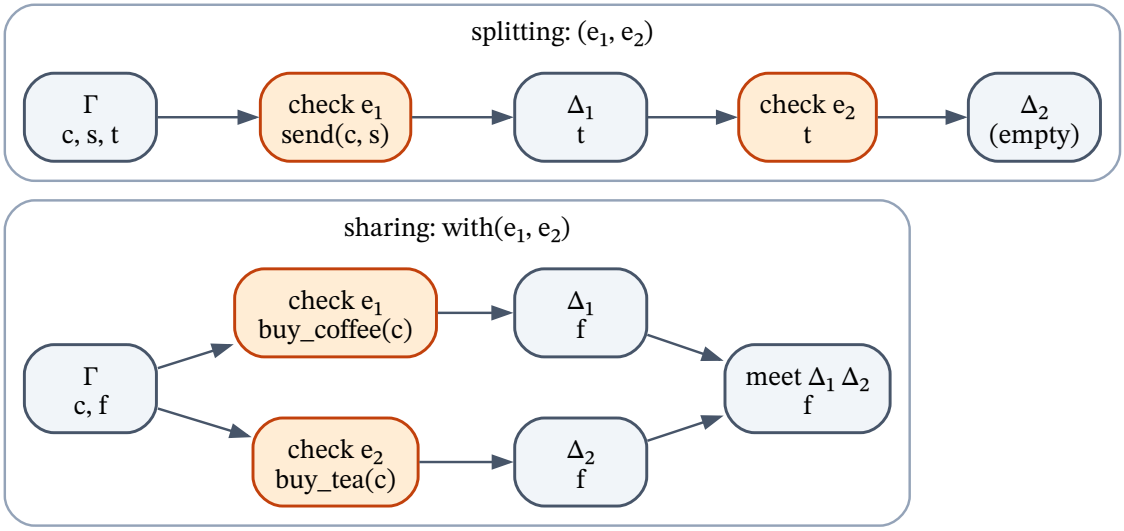
The declarative rules say *what* a derivation looks like, but not how to find one. Each splitting rule asks us to choose one of 2^n splits of the context, and a type checker that tried them all would be hopeless. The way out is to stop guessing. Give the *whole* context to the first subterm, and let it report what it did not use: its **leftovers**. Give the leftovers to the second subterm, and so on. A variable that the first subterm consumed is simply not there when the second one looks for it.

This is an algorithmic judgment with an input and an output context, $\Gamma \vdash e : \tau \dashv \Delta$, read "with Γ available, e has type τ and leaves Δ". Writing $\Gamma - x$ for Γ with x marked as used, here are four of its rules:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau \dashv \Gamma - x} \qquad \frac{\Gamma \vdash e_1 : \tau \dashv \Delta_1 \qquad \Delta_1 \vdash e_2 : \sigma \dashv \Delta_2}{\Gamma \vdash (e_1, e_2) : \tau \otimes \sigma \dashv \Delta_2}$$

$$\frac{\Gamma \vdash e_1 : \tau \dashv \Delta_1 \qquad \Gamma \vdash e_2 : \sigma \dashv \Delta_2}{\Gamma \vdash \langle e_1, e_2 \rangle : \tau \,\&\, \sigma \dashv \Delta_1 \sqcap \Delta_2} \qquad \frac{\Gamma, x : \tau \vdash e : \sigma \dashv \Delta}{\Gamma \vdash \lambda x : \tau.\, e : \tau \multimap \sigma \dashv \Delta - x}$$

The variable rule consumes. The pair *threads* the context: the output of e_1 is the input of e_2. The offer *forks* it: both alternatives start from Γ, and what remains afterwards is the **meet** $\Delta_1 \sqcap \Delta_2$, the variables that neither alternative used. The abstraction removes its parameter from the leftovers, whether the body used it or not. The other rules follow the same two patterns, threading for the splitting rules and forking for the sharing ones.



Checking with leftovers. Above, a pair threads the context: the first component consumes c and s, and the second receives only t. Below, an offer forks it: both alternatives start from the same context, and only what neither consumed is left.

The checker in Lean

With contexts with holes, "marked as used" is "turned into a hole":

```
/-- Consume the variable `x`: return its type and the context with a hole at
`x`. Fails if `x` is a hole (already consumed) or does not exist. -/
def take : Ctx → Nat → Option (Ty × Ctx)
  | some τ :: Γ, 0 => some (τ, none :: Γ)
  | none :: _, 0 => none
  | o :: Γ, x + 1 =>
    match take Γ x with
    | some (τ, Δ) => some (τ, o :: Δ)
    | none => none
  | [], _ => none
```

```
/-- What is left after two alternatives: a variable survives only if neither
alternative consumed it. -/
def meet : Ctx → Ctx → Ctx
  | some τ :: Γ, some _ :: Δ => some τ :: meet Γ Δ
  | _ :: Γ, _ :: Δ => none :: meet Γ Δ
  | _, _ => []
```

```
/-- The checker. `infer Γ e = some (τ, Δ)` means: `e` has type `τ`, and `Δ` is
what remains of `Γ` after `e` has consumed the variables it uses. -/
def infer : Ctx → Term → Option (Ty × Ctx)
  | Γ, .var x => take Γ x
  | Γ, .unit => some (.unit, Γ)
  | Γ, .lam τ e =>
    match infer (some τ :: Γ) e with
    | some (σ, _ :: Δ) => some (τ ⊸ σ, Δ)        -- the parameter leaves scope
    | _ => none
  | Γ, .app e₁ e₂ =>
    match infer Γ e₁ with
    | some (.lolli τ σ, Δ₁) =>
      match infer Δ₁ e₂ with                     -- e₂ gets the leftovers of e₁
      | some (τ', Δ₂) => if τ = τ' then some (σ, Δ₂) else none
      | none => none
    | _ => none
  | Γ, .pair e₁ e₂ =>
    match infer Γ e₁ with
    | some (τ, Δ₁) =>
      match infer Δ₁ e₂ with
      | some (σ, Δ₂) => some (τ ⊗ σ, Δ₂)
      | none => none
    | none => none
  | Γ, .letPair e₁ e₂ =>
    match infer Γ e₁ with
    | some (.tensor τ σ, Δ₁) =>
      match infer (some σ :: some τ :: Δ₁) e₂ with
      | some (ρ, _ :: _ :: Δ₂) => some (ρ, Δ₂)
      | _ => none
    | _ => none
  | Γ, .wpair e₁ e₂ =>
    match infer Γ e₁, infer Γ e₂ with            -- both start from Γ
    | some (τ, Δ₁), some (σ, Δ₂) => some (τ & σ, meet Δ₁ Δ₂)
    | _, _ => none
  | Γ, .fst e =>
    match infer Γ e with
    | some (.amp τ _, Δ) => some (τ, Δ)
    | _ => none
  | Γ, .snd e =>
    match infer Γ e with
    | some (.amp _ σ, Δ) => some (σ, Δ)
    | _ => none
  | Γ, .inl σ e =>
    match infer Γ e with
    | some (τ, Δ) => some (τ ⊕ σ, Δ)
    | none => none
```

```
  | Γ, .inr τ e =>
    match infer Γ e with
    | some (σ, Δ) => some (τ ⊕ σ, Δ)
    | none => none
  | Γ, .case e e₁ e₂ =>
    match infer Γ e with
    | some (.plus τ σ, Δ) =>
      match infer (some τ :: Δ) e₁, infer (some σ :: Δ) e₂ with
      | some (ρ₁, _ :: Δ₁), some (ρ₂, _ :: Δ₂) =>
        if ρ₁ = ρ₂ then some (ρ₁, meet Δ₁ Δ₂) else none
      | _, _ => none
    | _ => none
  | Γ, .letIn e₁ e₂ =>
    match infer Γ e₁ with
    | some (τ, Δ₁) =>
      match infer (some τ :: Δ₁) e₂ with
      | some (σ, _ :: Δ₂) => some (σ, Δ₂)
      | _ => none
    | none => none
```

`infer` is a direct transcription of the algorithmic rules, in the `Option` monad written out by hand: every `match` on a recursive call either continues with the leftovers or gives up with `none`. A rule that needs a particular kind of type, such as `app` needing a function, matches on the type's constructor. The pattern `_ :: Δ` in `lam` drops the parameter's entry from the leftovers, consumed or not. And we can run it:

```
-- swap is accepted, in the empty context:
/-- info: some ((String ⊗ File) ⊸ (File ⊗ String), []) -/
#guard_msgs in
#eval infer [] (swap Str File)

-- dup is rejected:
/-- info: none -/
#guard_msgs in
#eval infer [] (dup Str)

-- Leftovers: with s : String (variable 1) and f : File (variable 0), the term
-- `s` consumes s and leaves f.
/-- info: some (String, [some File, none]) -/
#guard_msgs in
#eval infer [some File, some Str] (.var 1)

-- An additive pair whose components use different variables consumes both.
/-- info: some (File & String, [none, none]) -/
#guard_msgs in
#eval infer [some File, some Str] (.wpair (.var 0) (.var 1))
```

Soundness

An algorithm is only as good as its relation to the rules. The first half is **soundness**: whatever the checker accepts has a derivation, and the part of the context that the checker consumed is exactly the part the derivation uses.

```
/-- **Soundness of the checker.** If `infer Γ e = some (τ, Δ)`, then `Γ` splits
into a used part `U` and the leftovers `Δ`, and `U ⊢ e : τ`. -/
theorem infer_sound (e : Term) :
    ∀ {Γ Δ : Ctx} {τ : Ty}, infer Γ e = some (τ, Δ) →
      ∃ U, Split Γ U Δ ∧ HasType U e τ := by
  induction e with
  | var x =>
    intro Γ Δ τ h
    obtain ⟨U, hs, hx⟩ := take_sound h
    exact ⟨U, hs, .var hx⟩
```

```
  | unit =>
    intro Γ Δ τ h
    simp only [infer, Option.some.injEq, Prod.mk.injEq] at h
    obtain ⟨rfl, rfl⟩ := h
    exact ⟨Γ.holes, Split.all_right Γ, .unit⟩
  | lam τ e ih =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i σ o Δ' he
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      -- The body was checked with the parameter in front of Γ.
      obtain ⟨V, hs, hv⟩ := ih he
      obtain ⟨V', hs', hw⟩ := hs.uncons
      exact ⟨V', hs', .lam (weaken hv hw)⟩
    · simp at h
  | app e₁ e₂ ih₁ ih₂ =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ σ Δ₁ h₁
      split at h
      · rename_i τ' Δ₂ h₂
        split at h
        · rename_i hτ
          subst hτ
          simp only [Option.some.injEq, Prod.mk.injEq] at h
          obtain ⟨rfl, rfl⟩ := h
          obtain ⟨U₁, hs₁, d₁⟩ := ih₁ h₁
          obtain ⟨U₂, hs₂, d₂⟩ := ih₂ h₂
          obtain ⟨U, hs, hU⟩ := hs₁.assoc hs₂
          exact ⟨U, hs, .app hU d₁ d₂⟩
        · simp at h
      · simp at h
    · simp at h
  | pair e₁ e₂ ih₁ ih₂ =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ Δ₁ h₁
      split at h
      · rename_i σ Δ₂ h₂
        simp only [Option.some.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        obtain ⟨U₁, hs₁, d₁⟩ := ih₁ h₁
        obtain ⟨U₂, hs₂, d₂⟩ := ih₂ h₂
        obtain ⟨U, hs, hU⟩ := hs₁.assoc hs₂
        exact ⟨U, hs, .pair hU d₁ d₂⟩
      · simp at h
    · simp at h
  | letPair e₁ e₂ ih₁ ih₂ =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ σ Δ₁ h₁
      split at h
      · rename_i ρ' o o' Δ₂ h₂
        simp only [Option.some.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        obtain ⟨U₁, hs₁, d₁⟩ := ih₁ h₁
        obtain ⟨V, hs₂, d₂⟩ := ih₂ h₂
        obtain ⟨V₁, hs₂', hw₁⟩ := hs₂.uncons
        obtain ⟨V₂, hs₂'', hw₂⟩ := hs₂'.uncons
        obtain ⟨U, hs, hU⟩ := hs₁.assoc hs₂''
        exact ⟨U, hs, .letPair hU d₁ (weaken d₂ (hw₁.trans (.keep hw₂)))⟩
      · simp at h
    · simp at h
  | wpair e₁ e₂ ih₁ ih₂ =>
    intro Γ Δ ρ h
```

```
    simp only [infer] at h
    split at h
    · rename_i τ Δ₁ σ Δ₂ h₁ h₂
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      obtain ⟨U₁, hs₁, d₁⟩ := ih₁ h₁
      obtain ⟨U₂, hs₂, d₂⟩ := ih₂ h₂
      obtain ⟨U, hs, w₁, w₂⟩ := hs₁.meet hs₂
      exact ⟨U, hs, .wpair (weaken d₁ w₁) (weaken d₂ w₂)⟩
    · simp at h
  | fst e ih =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ σ Δ' he
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      obtain ⟨U, hs, d⟩ := ih he
      exact ⟨U, hs, .fst d⟩
    · simp at h
  | snd e ih =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ σ Δ' he
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      obtain ⟨U, hs, d⟩ := ih he
      exact ⟨U, hs, .snd d⟩
    · simp at h
  | inl σ e ih =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ Δ' he
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      obtain ⟨U, hs, d⟩ := ih he
      exact ⟨U, hs, .inl d⟩
    · simp at h
  | inr τ e ih =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i σ Δ' he
      simp only [Option.some.injEq, Prod.mk.injEq] at h
      obtain ⟨rfl, rfl⟩ := h
      obtain ⟨U, hs, d⟩ := ih he
      exact ⟨U, hs, .inr d⟩
    · simp at h
  | case e e₁ e₂ ih ih₁ ih₂ =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ σ Δ₀ he
      split at h
      · rename_i ρ₁ o₁ Δ₁ ρ₂ o₂ Δ₂ h₁ h₂
        split at h
        · rename_i hρ
          subst hρ
          simp only [Option.some.injEq, Prod.mk.injEq] at h
          obtain ⟨rfl, rfl⟩ := h
          obtain ⟨U, hs, d⟩ := ih he
          obtain ⟨V₁, hs₁, d₁⟩ := ih₁ h₁
          obtain ⟨V₂, hs₂, d₂⟩ := ih₂ h₂
          obtain ⟨V₁', hs₁', hw₁⟩ := hs₁.uncons
          obtain ⟨V₂', hs₂', hw₂⟩ := hs₂.uncons
          obtain ⟨W, hW, w₁, w₂⟩ := hs₁'.meet hs₂'
          obtain ⟨X, hX, hX'⟩ := hs.assoc hW
          exact ⟨X, hX, .case hX' d
```

```
            (weaken d₁ (hw₁.trans (.keep w₁)))
            (weaken d₂ (hw₂.trans (.keep w₂)))⟩
        · simp at h
      · simp at h
    · simp at h
  | letIn e₁ e₂ ih₁ ih₂ =>
    intro Γ Δ ρ h
    simp only [infer] at h
    split at h
    · rename_i τ Δ₁ h₁
      split at h
      · rename_i σ o Δ₂ h₂
        simp only [Option.some.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        obtain ⟨U₁, hs₁, d₁⟩ := ih₁ h₁
        obtain ⟨V, hs₂, d₂⟩ := ih₂ h₂
        obtain ⟨V', hs₂', hw⟩ := hs₂.uncons
        obtain ⟨U, hs, hU⟩ := hs₁.assoc hs₂'
        exact ⟨U, hs, .letIn hU d₁ (weaken d₂ hw)⟩
      · simp at h
    · simp at h
```

The proof is by induction on the term, and it needs three facts about splits. The first is that two splits in a row are one split:

```
/-- Two successive splits are one split: if `Γ` gives `U₁` to `e₁` and keeps
`Δ₁`, and `Δ₁` gives `U₂` to `e₂` and keeps `Δ₂`, then `Γ` gives `U = U₁ ⊎ U₂`
to the pair and keeps `Δ₂`. -/
theorem Split.assoc {Γ U₁ Δ₁ U₂ Δ₂ : Ctx} (h₁ : Split Γ U₁ Δ₁) (h₂ : Split Δ₁ U₂ Δ₂) :
    ∃ U, Split Γ U Δ₂ ∧ Split U U₁ U₂ := by
  induction h₁ generalizing U₂ Δ₂ with
  | nil => cases h₂; exact ⟨[], .nil, .nil⟩
  | left _ ih =>
    cases h₂ with
    | hole h₂ =>
      obtain ⟨U, hU, hU'⟩ := ih h₂
      exact ⟨_, .left hU, .left hU'⟩
  | right _ ih =>
    cases h₂ with
    | left h₂ =>
      obtain ⟨U, hU, hU'⟩ := ih h₂
      exact ⟨_, .left hU, .right hU'⟩
    | right h₂ =>
      obtain ⟨U, hU, hU'⟩ := ih h₂
      exact ⟨_, .right hU, .hole hU'⟩
  | hole _ ih =>
    cases h₂ with
    | hole h₂ =>
      obtain ⟨U, hU, hU'⟩ := ih h₂
      exact ⟨_, .hole hU, .hole hU'⟩
```

This is what the threading rules need. Take the `app` case. Unfolding `infer` with `simp only [infer] at h` turns the hypothesis into nested `match` expressions, and each `split at h` considers the cases of one `match`: either the recursive call succeeded, with values that `rename_i` gives names to, or it failed, and then `h` says `none = some _` and `simp at h` closes the case. In the successful case, the induction hypotheses give a split of Γ into U_1 (used by e_1) and Δ_1, and a split of Δ_1 into U_2 (used by e_2) and Δ_2. `Split.assoc` combines them into a split of Γ into U and Δ_2 together with a split of U into U_1 and U_2, which is exactly the premise of rule `app`.

The second fact is for the forking rules. Two alternatives that start from the same context use two parts of it; together they use the union, which splits off the meet of the leftovers, and each alternative's part can be weakened to the union:

```
/-- Two alternatives that both start from `Γ`: the union of what they consume
is split off from `Γ`, leaving the meet of their leftovers. -/
theorem Split.meet {Γ U₁ Δ₁ U₂ Δ₂ : Ctx} (h₁ : Split Γ U₁ Δ₁) (h₂ : Split Γ U₂ Δ₂) :
    ∃ U, Split Γ U (meet Δ₁ Δ₂) ∧ Wk U₁ U ∧ Wk U₂ U := by
  induction h₁ generalizing U₂ Δ₂ with
  | nil => cases h₂; exact ⟨[], .nil, .nil, .nil⟩
  | left _ ih =>
    cases h₂ with
    | left h₂ =>
      obtain ⟨U, hU, w₁, w₂⟩ := ih h₂
      exact ⟨_, .left hU, .keep w₁, .keep w₂⟩
    | right h₂ =>
      obtain ⟨U, hU, w₁, w₂⟩ := ih h₂
      exact ⟨_, .left hU, .keep w₁, .fill w₂⟩
  | right _ ih =>
    cases h₂ with
    | left h₂ =>
      obtain ⟨U, hU, w₁, w₂⟩ := ih h₂
      exact ⟨_, .left hU, .fill w₁, .keep w₂⟩
    | right h₂ =>
      obtain ⟨U, hU, w₁, w₂⟩ := ih h₂
      exact ⟨_, .right hU, .keep w₁, .keep w₂⟩
  | hole _ ih =>
    cases h₂ with
    | hole h₂ =>
      obtain ⟨U, hU, w₁, w₂⟩ := ih h₂
      exact ⟨_, .hole hU, .keep w₁, .keep w₂⟩
```

In the `wpair` case, the two derivations are weakened to that common context, and rule `wpair` applies. This is where weakening becomes essential: the declarative rule wants both alternatives typed in *the same* context, while the checker found two different ones. The third fact, `Split.uncons`, handles binders: when the body of a `lam` or `let` was checked with the bound variable in front, the split of the extended context gives a split of the original one, and the bound variable, used or not, can be put back by weakening. The `case` rule uses all three facts.

Two corollaries close the circle with the previous sections:

```
/-- In particular, whatever the checker accepts is well typed in the whole
context (the leftovers are discarded by weakening). -/
theorem infer_typed {Γ Δ : Ctx} {e : Term} {τ : Ty} (h : infer Γ e = some (τ, Δ)) :
    HasType Γ e τ := by
  obtain ⟨U, hs, d⟩ := infer_sound e h
  exact weaken d hs.wk_left

/-- And so the usage theorem applies to every term the checker accepts. -/
theorem infer_at_most_once {Γ Δ : Ctx} {e : Term} {τ : Ty}
    (h : infer Γ e = some (τ, Δ)) (i : Nat) : uses i e ≤ 1 :=
  at_most_once (infer_typed h) i
```

Completeness

The second half is **completeness**: the checker never rejects a typable term. For this we need to know exactly what the checker leaves behind. It removes precisely the variables that the term uses:

```
/-- `consumed Γ f`: the context `Γ` with a hole at every position `i` such that
`f i > 0`. With `f i = uses i e` it is what remains of `Γ` after `e`. -/
def consumed : Ctx → (Nat → Nat) → Ctx
  | [], _ => []
  | o :: Γ, f => (if f 0 = 0 then o else none) :: consumed Γ (fun i => f (i + 1))
```

The delicate point is threading. When `app` checks e_2 with the leftovers of e_1, everything e_2 needs must still be there. It is, and the reason is the usage theorem: e_1 uses only variables that its own part of the split offered, so the variables of e_2's part are untouched:

```
/-- If `U` splits into `U₁` and `U₂` and `Γ` has at least the variables of
`U`, then removing from `Γ` only variables of `U₁` leaves every variable of
`U₂` in place. -/
theorem Split.wk_consumed {U U₁ U₂ : Ctx} (hs : Split U U₁ U₂) :
    ∀ {Γ : Ctx} {f : Nat → Nat}, Wk U Γ → (∀ i, f i ≤ avail U₁ i) →
      Wk U₂ (consumed Γ f) := by
  induction hs with
  | nil => intro Γ f hw _; cases hw; exact .nil
  | left _ ih =>
    intro Γ f hw hf
    cases hw with
    | keep hw =>
      have := ih (f := fun i => f (i + 1)) hw (fun i => by simpa using hf (i + 1))
      simp only [consumed]
      split
      · exact .fill this
      · exact .keep this
  | right _ ih =>
    intro Γ f hw hf
    cases hw with
    | keep hw =>
      have h0 : f 0 = 0 := by simpa [avail] using hf 0
      have := ih (f := fun i => f (i + 1)) hw (fun i => by simpa using hf (i + 1))
      simp only [consumed, h0, ↓reduceIte]
      exact .keep this
  | hole _ ih =>
    intro Γ f hw hf
    cases hw with
    | keep hw =>
      have := ih (f := fun i => f (i + 1)) hw (fun i => by simpa using hf (i + 1))
      simp only [consumed]
      split <;> exact .keep this
    | fill hw =>
      have := ih (f := fun i => f (i + 1)) hw (fun i => by simpa using hf (i + 1))
      simp only [consumed]
      split
      · exact .fill this
      · exact .keep this
```

The hypothesis `∀ i, f i ≤ avail U₁ i` is the usage theorem for e_1. With that lemma, completeness is an induction on the derivation in which each case unfolds `infer`, rewrites the recursive calls with the induction hypotheses, and checks that the leftovers have the promised form:

```
/-- **Completeness of the checker.** If `U ⊢ e : τ` and the checker's context
`Γ` has at least the variables of `U`, the checker succeeds with the type `τ`,
and its leftovers are `Γ` minus exactly the variables that `e` uses. -/
theorem infer_complete {U : Ctx} {e : Term} {τ : Ty} (h : HasType U e τ) :
    ∀ {Γ : Ctx}, Wk U Γ → infer Γ e = some (τ, consumed Γ (fun i => uses i e)) := by
  induction h with
  | @var U x τ hx =>
    intro Γ hw
    simp only [infer, uses]
    exact take_of_get Γ x (hw.get hx)
  | unit =>
    intro Γ _
    simp [infer, uses, consumed_zero]
  | lam _ ih =>
    intro Γ hw
    have := ih (Wk.keep (o := some _) hw)
    simp only [infer, this, consumed]
    rfl
  | app hs d₁ _ ih₁ ih₂ =>
```

```
    intro Γ hw
    have h₁ := ih₁ (hs.wk_left.trans hw)
    have h₂ := ih₂ (hs.wk_consumed hw (usage d₁))
    simp only [infer, h₁, h₂, consumed_consumed, ↓reduceIte]
    rfl
  | pair hs d₁ _ ih₁ ih₂ =>
    intro Γ hw
    have h₁ := ih₁ (hs.wk_left.trans hw)
    have h₂ := ih₂ (hs.wk_consumed hw (usage d₁))
    simp only [infer, h₁, h₂, consumed_consumed]
    rfl
  | letPair hs d₁ _ ih₁ ih₂ =>
    intro Γ hw
    have h₁ := ih₁ (hs.wk_left.trans hw)
    have h₂ := ih₂ (Wk.keep (Wk.keep (hs.wk_consumed hw (usage d₁))))
    simp only [infer, h₁, h₂, consumed, consumed_consumed]
    rfl
  | letIn hs d₁ _ ih₁ ih₂ =>
    intro Γ hw
    have h₁ := ih₁ (hs.wk_left.trans hw)
    have h₂ := ih₂ (Wk.keep (hs.wk_consumed hw (usage d₁)))
    simp only [infer, h₁, h₂, consumed, consumed_consumed]
    rfl
  | wpair _ _ ih₁ ih₂ =>
    intro Γ hw
    simp only [infer, ih₁ hw, ih₂ hw, meet_consumed]
    rfl
  | fst _ ih =>
    intro Γ hw
    simp only [infer, ih hw]
    rfl
  | snd _ ih =>
    intro Γ hw
    simp only [infer, ih hw]
    rfl
  | inl _ ih =>
    intro Γ hw
    simp only [infer, ih hw]
    rfl
  | inr _ ih =>
    intro Γ hw
    simp only [infer, ih hw]
    rfl
  | case hs d _ _ ih ih₁ ih₂ =>
    intro Γ hw
    have h := ih (hs.wk_left.trans hw)
    have hw₂ := hs.wk_consumed hw (usage d)
    have h₁ := ih₁ (Wk.keep hw₂)
    have h₂ := ih₂ (Wk.keep hw₂)
    simp only [infer, h, h₁, h₂, consumed, ↓reduceIte, meet_consumed, consumed_consumed]
    -- Left: max (u + v₁) (u + v₂); right: u + max v₁ v₂. Equal, by arithmetic.
    congr 2
    apply congrArg
    funext i
    simp only [uses]
    omega
```

Most cases end with `rfl` once `simp only` has rewritten the calls, because the leftovers agree by definition. The `case` case needs one extra step: the leftovers come out as $\max(u + v_1, u + v_2)$ and the specification says $u + \max(v_1, v_2)$, which are equal but not by definition. `congr 2` and `apply congrArg` reduce the goal to the equality of the two counting functions, `funext` to their equality at every `i`, and `omega` proves it. Soundness and completeness together say that the checker *decides* the typing relation:

```
/-- Together: the checker decides typability in a context. -/
theorem infer_iff {Γ : Ctx} {e : Term} {τ : Ty} :
    HasType Γ e τ ↔ ∃ Δ, infer Γ e = some (τ, Δ) := by
  constructor
  · intro h
    exact ⟨_, infer_complete h (Wk.refl Γ)⟩
  · intro ⟨Δ, h⟩
    exact infer_typed h
```

22.11 The `affine` crate

The crate `code/rust/crates/affine` implements the same algorithm for a language that looks like Rust, so that its programs and errors can be compared with real ones. A program is a list of functions. Functions without a body are provided by the environment, and calling a declared function uses its arguments but not the function itself (it is a constant, like a Rust `fn` item):

λ^{aff}	`affine` crate
$\tau \otimes \sigma$, $\tau \multimap \sigma$	`(T, U)`, `FnOnce(T) -> U`
$\tau \,\&\, \sigma$, $\tau \oplus \sigma$	`With<T, U>`, `Either<T, U>`
`let` $(x, y) = e_1$ `in` e_2	`let (x, y) = e1; e2`
$\langle e_1, e_2 \rangle$, *fst* e, *snd* e	`with(e1, e2)`, `e.fst`, `e.snd`
`case` $e\ \{x.\, e_1 \mid y.\, e_2\}$	`match e { Left(x) => e1, Right(y) => e2 }`

A λ-abstraction $\lambda x : \tau.\, e$ is written as a closure, `|x: T| e`. The crate also has what λ^{aff} deliberately lacks and Rust has: projections `p.0` and `r.field` on places, with per-place move tracking, structs, and (for Chapter 23) `Copy`, `.clone()` and `Rc`. It consists of a parser, the checker (`check.rs`), the diagnostics (`diagnostic.rs`), and derivation trees (`derivation.rs`).

Contexts that remember moves

The crate's context is a stack of bindings, each recording where it was declared and every move out of it:

```
/// One variable in the context, with everything the checker knows about it.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Binding {
    pub name: String,
    pub ty: Ty,
    /// Where the variable was introduced (a parameter or a `let`).
    pub decl: Span,
    /// How many closures enclose the place where the variable was introduced.
    pub depth: usize,
    /// The moves out of this variable or out of parts of it. No moves: the
    /// variable is available. A move with an empty path: it is gone.
    pub moves: Vec<Move>,
    /// How many times the variable was used (moved, copied or cloned).
    pub uses: usize,
}

/// A move out of a place `x.path`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Move {
    pub path: Vec<Proj>,
    pub span: Span,
    pub kind: MoveKind,
```

```
}

/// How a move happened, for the labels of error messages.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum MoveKind {
    /// An ordinary use by value: "value moved here".
    Plain,
    /// A use inside a closure body: the closure captured the variable.
    IntoClosure { use_span: Span },
    /// Calling a closure consumes it: "`f` moved due to this call".
    Call,
}

/// The typing context: a stack of bindings, innermost last. Shadowing is
/// allowed, as in Rust: a name refers to its innermost binding.
#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct Ctx {
    pub bindings: Vec<Binding>,
}
```

A binding with a move whose path is empty is the crate's version of Lean's hole. Unlike a hole, it remembers *where* the value went, which is what the error message needs. The checker's main function has the shape of the algorithmic judgment $\Gamma \vdash e : \tau \dashv \Delta$, and its Rust signature is itself affine: it takes the context by value and gives back the leftovers.

```
/// Check `e` in the context `ctx`, which it consumes; return the type, the
/// leftover context and the derivation. `expected` is the type that the
/// surrounding code expects, if it is known.
pub fn infer(
    &mut self,
    ctx: Ctx,
    e: &Expr,
    expected: Option<&Ty>,
) -> Result<Checked, Diagnostic> {
    let before = ctx.uses_snapshot();
    let (ty, ctx, rule, premises) = self.infer_kind(ctx, e, expected)?;
    if let Some(want) = expected {
        if *want != ty {
            return Err(mismatch(e.span, want, &ty));
        }
    }
    let deriv = Derivation {
        rule,
        ctx: ctx.used_since(&before),
        expr: show_expr_brief(e),
        ty: ty.clone(),
        premises,
    };
    Ok(Checked { ty, ctx, deriv })
}
```

The threading rule is two calls in a row, the second one receiving the first one's output:

```
ExprKind::Tuple(a, b) => {
    let (ea, eb) = match expected {
        Some(Ty::Tuple(x, y)) => (Some(&**x), Some(&**y)),
        _ => (None, None),
    };
    // The first component gets the whole context; the second gets
    // what the first one left over.
    let first = self.infer(ctx, a, ea)?;
    let second = self.infer(first.ctx, b, eb)?;
    let ty = Ty::tuple(first.ty, second.ty);
    Ok((ty, second.ctx, "A-Pair", vec![first.deriv, second.deriv]))
}
```

and the forking rule clones the context, the only place where the checker does so, and merges the leftovers:

```
ExprKind::With(a, b) => {
    let (ea, eb) = match expected {
        Some(Ty::With(x, y)) => (Some(&**x), Some(&**y)),
        _ => (None, None),
    };
    // Both alternatives start from the same context: the only
    // place where the checker copies a context is where the
    // rules share one.
    let left = self.infer(ctx.clone(), a, ea)?;
    let right = self.infer(ctx, b, eb)?;
    let ty = Ty::With(Box::new(left.ty), Box::new(right.ty));
    let out = merge(left.ctx, right.ctx);
    Ok((ty, out, "A-With", vec![left.deriv, right.deriv]))
}
```

```
/// The leftovers of two alternatives that started from the same context (the
/// two components of `with(a, b)`, the two arms of a `match`): a variable is
/// moved afterwards if *either* alternative moved it.
fn merge(mut a: Ctx, b: Ctx) -> Ctx {
    for (x, y) in a.bindings.iter_mut().zip(b.bindings) {
        for m in y.moves {
            if !x.moves.contains(&m) {
                x.moves.push(m);
            }
        }
        x.uses = x.uses.max(y.uses);
    }
    a
}
```

Every error E0382 comes from one function, the use of a place:

```
/// Use the place `root.path` by value: move it, or copy it if its type is
/// `Copy`. This is where E0382 comes from.
fn use_place(
    &mut self,
    mut ctx: Ctx,
    root: &Ident,
    path: &[Proj],
    use_span: Span,
    (move_span, kind): (Span, MoveKind),
) -> Result<(Ty, Ctx), Diagnostic> {
    let i = ctx
        .lookup(&root.name)
        .expect("the caller looked the variable up");
    let b = &ctx.bindings[i];
    let mut ty = b.ty.clone();
    for p in path {
        ty = self
            .place_step(&ty, p)
            .expect("the caller checked the path");
    }
    // Moved before: this place, or a place that contains it.
    let moved: Vec<&Move> = b
        .moves
        .iter()
        .filter(|m| path.starts_with(&m.path))
        .collect();
    if !moved.is_empty() {
        return Err(self.use_after_move(b, &moved, path, use_span));
    }
    // A part of this place was moved before.
    let partly: Vec<&Move> = b
        .moves
```

```
            .iter()
            .filter(|m| m.path.starts_with(path))
            .collect();
        if !partly.is_empty() {
            return Err(self.use_after_partial_move(b, &partly, path, use_span));
        }
        let b = &mut ctx.bindings[i];
        b.uses += 1;
        if !self.is_copy(&ty) {
            // A variable from outside the innermost closures is captured.
            let (span, kind) = if b.depth < self.closures.len() {
                (self.closures[b.depth], MoveKind::IntoClosure { use_span })
            } else {
                (move_span, kind)
            };
            b.moves.push(Move {
                path: path.to_vec(),
                span,
                kind,
            });
        }
        Ok((ty, ctx))
    }
```

A use is an error if the same place, or a place that contains it, has been moved (`path.starts_with(&m.path)`), or if a part of it has been moved. Otherwise the use records a move, unless the type is `Copy`, which in this chapter's discipline nothing is. A use inside a closure body of a variable declared outside the closure is recorded as a move *into the closure*, at the closure's head, which is what A-Abs says: the closure takes what its body uses.

Each call of `infer` also builds a node of a derivation, whose context is the set of variables that the subterm used: the part of the input context that is not in the output. That is the context U of the soundness theorem, and it is how the crate prints derivations of the declarative system from a run of the algorithmic one:

```
/// A node of a typing derivation: `ctx ⊢ expr : ty`, justified by `rule`
/// from the premises. The context is the part of the checker's input context
/// that this subterm actually used (the soundness theorem of Chapter 22 says
/// that this part is enough to derive the judgment).
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Derivation {
    pub rule: &'static str,
    pub ctx: Vec<(String, Ty)>,
    pub expr: String,
    pub ty: Ty,
    pub premises: Vec<Derivation>,
}
```

The example `crates/affine/examples/ch22_affine_checker.rs` checks the programs of `crates/affine/examples/programs/ch22_accepted.aff`:

```
// Functions provided by the environment. A call consumes its arguments.
fn send(c: Conn, msg: String) -> Conn;
fn ship(c: Conn, o: Order) -> Conn;
fn refund(c: Conn, r: Refund) -> Conn;
fn buy_coffee(c: Coin) -> Coffee;
fn buy_tea(c: Coin) -> Tea;

// ⊗: take the pair apart, and build it again the other way round.
fn swap(p: (String, File)) -> (File, String) {
    let (s, f) = p;
    (f, s)
```

```
}

// Weakening: `f` is never used.
fn first(s: String, f: File) -> String {
    s
}

// &: both alternatives may spend the coin, because only one will be chosen.
fn offer(c: Coin) -> With<Coffee, Tea> {
    with(buy_coffee(c), buy_tea(c))
}

// ⊕: both arms may use the connection, because only one arm runs.
fn settle(m: Either<Order, Refund>, c: Conn) -> Conn {
    match m {
        Left(o) => ship(c, o),
        Right(r) => refund(c, r),
    }
}

// ⊸: the closure captures `c` and `msg`; it can be called once.
fn later(c: Conn, msg: String) -> FnOnce(()) -> Conn {
    |u: ()| send(c, msg)
}
```

and prints the derivations; we have seen most of them already. Two lines are new. After `first`, the crate notes that `f` was discarded without use, the weakening of A-Var. And `later` shows A-Abs: the closure's context holds the two captured variables, and the premise adds the parameter `u`, unused:

```
ok: fn first(s: String, f: File) -> String
[A-Var] s: String ⊢ s : String
    (discarded without use: f)
```

```
ok: fn later(c: Conn, msg: String) -> FnOnce(()) -> Conn
[A-Abs] c: Conn, msg: String ⊢ |u: ()| send(c, msg) : FnOnce(()) -> Conn
  [Call] c: Conn, msg: String, u: () ⊢ send(c, msg) : Conn
    [A-Var] c: Conn ⊢ c : Conn
    [A-Var] msg: String ⊢ msg : String
```

Side by side with rustc

The example `ch22_move_errors.rs` runs the checker on six programs that it rejects. The first is `dup`:

```
error[E0382]: use of moved value: `s`
 --> ch22_dup.aff:2:9
  |
1 | fn dup(s: String) -> (String, String) {
  |        - move occurs because `s` has type `String`, which does not implement the `Copy`
trait
2 |     (s, s)
  |      -  ^ value used here after move
  |      |
  |      value moved here
```

Compare it with rustc's output for the same function earlier in the chapter. The labels are the same, in the same places: the declaration that explains why the value moves, the move, and the use after the move. rustc adds a suggestion to clone. The coin is the crate's version of the voucher:

```
error[E0382]: use of moved value: `c`
 --> ch22_coin.aff:5:29
  |
4 | fn both(c: Coin) -> (Coffee, Tea) {
```

```
  |         - move occurs because `c` has type `Coin`, which does not implement the `Copy`
trait
5 |     (buy_coffee(c), buy_tea(c))
  |                  -           ^ value used here after move
  |                  |
  |                  value moved here
```

The partial move, with the crate's per-place tracking, matches rustc's message word for word, note included:

```
error[E0382]: use of partially moved value: `p`
 --> ch22_split.aff:3:9
  |
2 |     let s = p.0;
  |             --- value partially moved here
3 |     (s, p)
  |         ^ value used here after partial move
  |
  = note: partial move occurs because `p.0` has type `String`, which does not implement the
`Copy` trait
```

A closure that uses a variable takes it:

```
error[E0382]: use of moved value: `msg`
 --> ch22_capture.aff:5:9
  |
3 | fn capture(c: Conn, msg: String) -> (FnOnce(()) -> Conn, String) {
  |                     --- move occurs because `msg` has type `String`, which does not
implement the `Copy` trait
4 |     let f = |u: ()| send(c, msg);
  |             -------         --- variable moved due to use in closure
  |             |
  |             value moved into closure here
5 |     (f, msg)
  |         ^^^ value used here after move
```

and rustc says the same about the closure `|| send(c, msg)`:

```
fn capture(c: Conn, msg: String) -> (impl FnOnce() -> Conn, String) {
    let f = || send(c, msg);
    (f, msg)
}
```

```
error[E0382]: use of moved value: `msg`
  --> compile_fail/ch22_closure_capture.rs:14:9
   |
12 | fn capture(c: Conn, msg: String) -> (impl FnOnce() -> Conn, String) {
   |                     --- move occurs because `msg` has type `String`, which does not
implement the `Copy` trait
13 |     let f = || send(c, msg);
   |             --          --- variable moved due to use in closure
   |             |
   |             value moved into closure here
14 |     (f, msg)
   |         ^^^ value used here after move
   |
note: consider changing this parameter type in function `send` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch22_closure_capture.rs:6:23
   |
 6 | fn send(c: Conn, msg: String) -> Conn {
   |    ----               ^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
help: consider cloning the value if the performance cost is acceptable
```

```
   |
13 |     let f = || send(c, msg.clone());
   |                          ++++++++
```

The first eleven lines are the same diagnosis; rustc then explains two ways out (borrow instead of owning, or clone). Finally, a function of type $\tau \multimap \sigma$ is a resource, so calling it twice is a second use of it, as rustc says for an `FnOnce` (Chapter 8 showed rustc's version of this error):

```
error[E0382]: use of moved value: `f`
 --> ch22_twice.aff:2:12
  |
1 | fn twice(f: FnOnce(String) -> Conn, a: String, b: String) -> (Conn, Conn) {
  |          - move occurs because `f` has type `FnOnce(String) -> Conn`, which does not
implement the `Copy` trait
2 |     (f(a), f(b))
  |      ----  ^ value used here after move
  |      |
  |      `f` moved due to this call
```

The crate and rustc agree on these programs, but they get there differently. The crate follows the syntax tree, rule by rule; rustc lowers the function to a control-flow graph and runs a dataflow analysis over it (Chapter 29), which is how it handles loops, early returns and conditional moves that the calculus does not have. rustc also infers more types than the crate (where the crate needs `let x: Either<A, B> = Left(a);`, rustc infers the type from later uses), and its closures capture by reference whenever the body only reads a variable, which is borrowing, Part VI. On the affine fragment, though, the two agree on which programs to reject and on why.

22.12 Putting it together: an upload

Here is a small program that does something a real service does: read a file, send its contents over a connection, close the file, and keep the connection for later. It is followed by two buggy versions:

```
fn read_all(file: File) -> (String, File);
fn send(conn: Conn, msg: String) -> Conn;
fn close_file(file: File);

// Read a file, send its contents, close the file, keep the connection.
fn upload(conn: Conn, file: File) -> Conn {
    let (data, opened) = read_all(file);
    let sent = send(conn, data);
    close_file(opened);
    sent
}

// A bug: the data is sent twice.
fn upload_twice(conn: Conn, file: File) -> Conn {
    let (data, opened) = read_all(file);
    let sent = send(send(conn, data), data);
    close_file(opened);
    sent
}

// Another bug, which the affine discipline accepts: the file is never closed.
fn upload_leaky(conn: Conn, file: File) -> Conn {
    let (data, opened) = read_all(file);
    send(conn, data)
}
```

The crate accepts `upload`. Read its derivation from the top, watching the contexts shrink:

```
ok: fn upload(conn: Conn, file: File) -> Conn
[A-LetPair] conn: Conn, file: File ⊢ let (data, opened) = read_all(file); … : Conn
  [Call] file: File ⊢ read_all(file) : (String, File)
    [A-Var] file: File ⊢ file : File
  [A-Let] conn: Conn, data: String, opened: File ⊢ let sent = send(conn, data); … : Conn
    [Call] conn: Conn, data: String ⊢ send(conn, data) : Conn
      [A-Var] conn: Conn ⊢ conn : Conn
      [A-Var] data: String ⊢ data : String
    [A-Let] opened: File, sent: Conn ⊢ close_file(opened); sent : Conn
      [Call] opened: File ⊢ close_file(opened) : ()
        [A-Var] opened: File ⊢ opened : File
      [A-Var] sent: Conn ⊢ sent : Conn
```

The whole body uses both parameters. A-LetPair splits them: `file` goes to `read_all(file)`, and `conn` to the rest, which also receives the two new variables `data` and `opened`. The next `let` splits again: `conn` and `data` go to `send`, and `opened` to the rest, which also receives `sent`. The statement `close_file(opened);` is a `let` whose variable is never used, and the last line uses `sent`. Every variable appears in exactly one leaf. That is the usage theorem for one program, and the derivation is the certificate.

The bug in `upload_twice` is a contraction, and the crate finds no split:

```
error[E0382]: use of moved value: `data`
  --> ch22_upload.aff:16:39
   |
15 |     let (data, opened) = read_all(file);
   |          ---- move occurs because `data` has type `String`, which does not implement
the `Copy` trait
16 |     let sent = send(send(conn, data), data);
   |                             ----   ^^^^ value used here after move
   |                             |
   |                             value moved here
```

The bug in `upload_leaky` is different: the file is never closed. The crate *accepts* it, and only notes that `opened` was discarded without use:

```
ok: fn upload_leaky(conn: Conn, file: File) -> Conn
[A-LetPair] conn: Conn, file: File ⊢ let (data, opened) = read_all(file); … : Conn
  [Call] file: File ⊢ read_all(file) : (String, File)
    [A-Var] file: File ⊢ file : File
  [Call] conn: Conn, data: String, opened: File ⊢ send(conn, data) : Conn
    [A-Var] conn: Conn ⊢ conn : Conn
    [A-Var] data: String ⊢ data : String
    (discarded without use: opened)
```

This is affine typing working as designed. Weakening is allowed, so a value may be abandoned, and the type system cannot force us to call `close_file`. In Rust the `File` would be closed anyway, by its destructor, when `opened` goes out of scope. Whether a type system *can* force a value to be consumed, and why Rust chose not to, is the subject of Chapter 24.

The same program in Lean needs the environment in the context. The functions are curried, since the calculus has one-argument functions:

```
def Conn : Ty := .base "Conn"

/-- The context of the upload: the two resources and the three functions of the
```

```
environment. Index 0 is the innermost entry. -/
def uploadCtx : Ctx :=
  [ some File                        -- 0  file
  , some Conn                        -- 1  conn
  , some (File ⊸ .unit)              -- 2  close_file
  , some (Conn ⊸ Str ⊸ Conn)         -- 3  send (curried)
  , some (File ⊸ Str ⊗ File) ]       -- 4  read_all

/-- `let (data, opened) = read_all file in let sent = send conn data in`
`let _ = close_file opened in sent`. Under the `letPair`, `opened` is 0 and
`data` is 1; under the first `letIn`, `sent` is 0; under the second, `sent` is 1. -/
def upload : Term :=
  .letPair (.app (.var 4) (.var 0))
    (.letIn (.app (.app (.var 5) (.var 3)) (.var 1))
      (.letIn (.app (.var 5) (.var 1))
        (.var 1)))

/-- The same without `close_file opened`. -/
def uploadLeaky : Term :=
  .letPair (.app (.var 4) (.var 0))
    (.app (.app (.var 5) (.var 3)) (.var 1))
```

```
-- Everything is consumed:
/-- info: some (Conn, [none, none, none, none, none]) -/
#guard_msgs in
#eval infer uploadCtx upload

-- The leaky version is accepted too; `close_file` is left over, unused.
/-- info: some (Conn, [none, none, some (File ⊸ 1), none, none]) -/
#guard_msgs in
#eval infer uploadCtx uploadLeaky

/-- The checker's answer is a proof: `upload` is well typed. -/
theorem upload_typed : HasType uploadCtx upload Conn :=
  infer_iff.mpr ⟨_, rfl⟩
```

The leftovers say the same as the crate: everything is consumed by `upload`, and `uploadLeaky` leaves `close_file` unused. The last line turns the checker's answer into a proof by completeness and soundness: `rfl` evaluates `infer uploadCtx upload` and checks that it is a `some`, and `infer_iff` converts that into a derivation. There is a difference with the crate, and it is instructive. In Lean the three functions are ordinary variables of the context, so they are resources too: a version of `upload_twice` would be rejected already for calling `send` twice, before anyone looked at `data`. The crate treats declared functions as constants that can be called again and again, as Rust does. Why Rust may treat functions that way, while a closure that captures a `String` can be called only once, is a question about which values may be duplicated, and it is the subject of Chapter 23.

22.13 Common confusions

"**Affine means every variable must be used.**" It means the opposite bound: *at most* once. A variable may be used zero times (weakening). Requiring at least one use is relevant logic, and exactly one use is linear logic; Rust has neither (Chapter 24).

"**The `case` rule uses the context twice, so it is contraction.**" Contraction duplicates a resource for two consumers that *both* run. The branches of a `case` are alternatives, only one of which runs, so each resource is still consumed at most once on every run. The same holds for the two components of an offer. The usage theorem counts them with $\max$ for exactly this reason.

"**E0382 points at the second use, so the second use is the bug.**" The derivation fails at a split, and both uses take part in it: rustc labels both. Fixing the program may mean changing the first use (borrowing instead of moving, or cloning) as often as the second.

"**Partial moves contradict the rule that `⊗` has no projections.**" They refine it. The calculus counts uses of variables; rustc counts uses of places. A pair whose two fields are moved separately is used once per field, which is the same discipline at a finer grain.

"**A closure in the calculus borrows what it uses.**" It takes it. In λ^{aff} every use is a move, so a closure that mentions a variable owns it. Rust closures that only read a variable capture a *reference* to it instead, which is a borrow, and borrowing is not part of the affine calculus (Part VI).

"**The algorithmic checker is a different type system.**" It is the same one, presented differently. Soundness and completeness say that it accepts exactly the typable terms, and the leftovers are exactly the unused part of the context.

22.14 Summary

- λ^{aff} has unit, affine functions $\tau \multimap \sigma$, pairs $\tau \otimes \sigma$, offers $\tau \,\&\, \sigma$ and sums $\tau \oplus \sigma$. Its typing rules split the context when both subterms run and share it when only one of two alternatives runs.
- Weakening is built into A-Var and A-Unit, and it is admissible for all derivations. Contraction is not: $\lambda x.\,(x, x)$ has no type, and rustc's E0382 is the report of the failed split.
- A pair is taken apart with `let`, which binds both components with one use of the pair. Projections can be defined but not both used; Rust refines the discipline to places, which explains partial moves.
- The usage theorem: in a well-typed term every variable is used at most once, counting alternatives with $\max$. Its Lean proof rests on one lemma: a split hands out exactly what the context had.
- Substitution and preservation hold; in a sharing rule a value is copied into both alternatives, but only one copy can run.
- The algorithmic checker threads input and output contexts ("leftovers"), forking and meeting them at offers and branches. Lean proves it sound and complete, so it decides typability.
- The `affine` crate implements the algorithm for a Rust-like syntax, tracks moves per place, prints derivations, and reproduces the core of rustc's E0382 messages.

22.15 Exercises

1. Show that $\tau \multimap \sigma \multimap \tau$ is inhabited in λ^{aff} but would not be in a linear calculus with rule L-Var. Then show that the term $\lambda f.\,\lambda a.\,f\,a\,a$ has no type of the form $(\tau \multimap \tau \multimap \sigma) \multimap \tau \multimap \sigma$ in λ^{aff}. Which structural rule does each term need? *Hint:* the first is the K combinator and needs weakening; the second is W. For W, follow the splits until a has to go to two places.
2. Find closed terms of types $(\tau \otimes \tau) \multimap \tau$ and $(\tau \,\&\, \tau) \multimap \tau$. Which of the two would still be inhabited in the linear calculus? *Hint:* one of them throws a component away, the other chooses an alternative and throws nothing away.

3. In Lean, define the *linear* variant of the typing relation: copy `HasType` to a new `LHasType` in which `var` requires every other position to be a hole and `unit` requires the whole context to be holes. Prove that `discard Str File` has no linear type. *Hint:* define `allHoles : Ctx → Prop` (for example `∀ o ∈ Γ, o = none`). After two `cases` on the `lam` constructors, the `var` premise must hold for the context `[some File, some Str]`, and `simp` refutes it.
4. In Lean, prove that the checker never creates resources: if `infer Γ e = some (τ, Δ)` then `Wk Δ Γ`. *Hint:* `infer_sound` gives a split of `Γ` into `U` and `Δ`. Prove a lemma `Split.wk_right : Split Γ Γ₁ Γ₂ → Wk Γ₂ Γ`, by induction on the split, in the style of `Split.wk_left`.
5. Add an `if` expression to the `affine` crate, with a condition of type `bool` and two branches that share the context like the arms of a `match`. Write tests showing that a variable may be moved in both branches, and that a variable moved in one branch is moved after the `if`. *Hint:* parse `if e { e1 } else { e2 }` in `primary`, and copy the structure of `infer_match`, including the call to `merge`.
6. Suppose we add general recursion, a term $fix\ f.\ e$ with the rule "if $\Gamma, f : \tau \multimap \sigma \vdash e : \tau \multimap \sigma$ then $\Gamma \vdash fix\ f.\ e : \tau \multimap \sigma$". Does the usage theorem survive? What do we lose? *Hint:* look at the proof's case for A-Abs; it has the same shape. What is lost is termination, and with it the reading of types as theorems (Chapter 10): $fix\ f.\ f$ inhabits every function type.

CHAPTER 23

Copy, Clone and the Exponential

The affine λ-calculus of Chapter 22 is strict to the point of being useless. It cannot use an integer twice. It cannot call a function twice: at the end of Chapter 22, the Lean version of the upload program could not even send two messages, because `send` was a variable of the context and every variable is a resource. A real language needs some values to be freely duplicable, and the interesting question is which ones, and on what terms.

Linear logic answers with a **modality**, written $!A$ and read "of course A": a marker on the formulas that may be duplicated and discarded at will. Rust answers with two traits and a library type. `Copy` marks the types whose values are duplicated implicitly and for free; `Clone` marks the types that can be duplicated explicitly, by calling a method that may do real work; and `Rc` and `Arc` share a value among several owners and keep count. These are not three unrelated features. They are three ways of bringing back the structural rule that Chapter 22 removed, contraction, each with a different price.

This chapter presents the exponential modality and its rules, shows that `Copy` behaves like it and that Rust's rules for `Copy` are its rules in disguise, contrasts `Copy` with `Clone` and `Rc`, and then makes the connection formal: λ^{aff} extended with a `Copy` predicate, a splitting rule that may duplicate `Copy` variables, and a generalized usage theorem proved in Lean.

In this chapter you will learn:

- the exponential modality $!A$ and its four rules (weakening, contraction, dereliction, promotion), and what each one says about resources;
- why `Copy` is contraction for free on a class of types, and why the class is what it is: all fields `Copy` (E0204), no destructor (E0184), `&T` but not `&mut T`;
- why `Clone` is a different thing: an explicit duplication *term*, with a cost and possibly an effect;
- how `Rc` and `Arc` share instead of duplicating, and why counting forbids them from being `Copy`;
- the formal statement: λ^{aff} with a `Copy` predicate and a splitting rule S-Both, and the generalized usage theorem, proved in Lean with multiplicities in $\mathbb{N} \cup \{\infty\}$;
- why functions cannot be `Copy` in general, as a Lean proof that preservation would fail;
- how the `affine` crate checks `Copy`, `Clone`, `Rc` and `#[derive(Copy)]`.

23.1 Too affine to be useful

Here is the smallest program that λ^{aff} rejects and every programmer would accept, checked by the `affine` crate in the pure discipline of Chapter 22 (the example is `code/rust/crates/affine/examples/ch23_copy.rs`):

```
== ch23_twice.aff, pure λ^aff (Chapter 22)
error[E0382]: use of moved value: `n`
 --> ch23_twice.aff:2:9
  |
1 | fn twice(n: i32) -> (i32, i32) {
  |          - move occurs because `n` has type `i32`, which does not implement the `Copy`
trait
2 |     (n, n)
  |      -  ^ value used here after move
  |      |
  |      value moved here
```

The message is correct for the calculus: no type is `Copy` there, and the proof of Chapter 22 that $\lambda x.\,(x, x)$ has no type did not depend on the type of x at all. Rust, of course, accepts `twice`. To accept it, a type system has two options, and Rust uses both:

1. let certain types bring contraction back *implicitly*, so that using a variable does not consume it;
2. provide *explicit* duplication functions, $\tau \multimap \tau \otimes \tau$, which the programmer calls when needed.

The first option is the exponential modality of linear logic, and it is `Copy`. The second is `Clone`.

23.2 The exponential modality

Girard's linear logic keeps an assumption from being duplicated or discarded, and then adds a way to say that a particular assumption *may* be. The formula $!A$ means "as many As as you need, including none". On types, $!\tau$ is the type of values that may be used any number of times. Four rules say what that means. We write them as typing rules, with $!\Gamma$ for a context in which every type has the form $!\tau$:

$$\frac{\Gamma \vdash e : \rho}{\Gamma, x : !\tau \vdash e : \rho}\ !\text{-Weak} \qquad \frac{\Gamma, x : !\tau, y : !\tau \vdash e : \rho}{\Gamma, z : !\tau \vdash e[z/x, z/y] : \rho}\ !\text{-Contr}$$

$$\frac{\Gamma \vdash e : !\tau}{\Gamma \vdash \mathit{derelict}\, e : \tau}\ \text{Dereliction} \qquad \frac{!\Gamma \vdash e : \tau}{!\Gamma \vdash !e : !\tau}\ \text{Promotion}$$

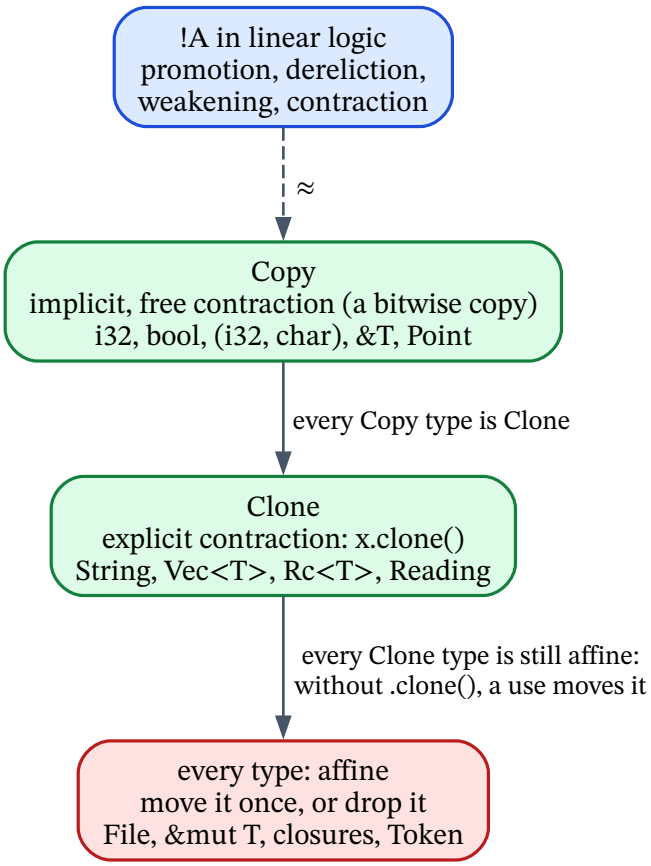
- **!-Weak** says that a $!\tau$ may go unused. In linear logic this is the only form of weakening; in affine logic every variable may go unused already, so the rule adds nothing new.
- **!-Contr** is contraction, restricted to !-types: two variables of type $!\tau$ may be identified, so one $!\tau$ serves two uses.
- **Dereliction** uses a $!\tau$ as an ordinary τ, once. You give up the right to duplicate it and get the value.
- **Promotion** is the subtle one. It builds a duplicable value $!e$, but only from a context in which *everything* is duplicable. The reason is a resource argument. If e used a non-duplicable resource s, then every copy of $!e$ would need its own s, and duplicating $!e$ would duplicate s. So a duplicable value may only be built out of duplicable things.

Together they give the derivations you would expect. From $x : !A$, contraction and two derelictions give a pair $A \otimes A$; weakening gives $\mathbf{1}$. From $x : A$ alone, nothing gives $!A$: promotion needs the whole

context to be of the form !Γ. Girard also showed that ordinary intuitionistic logic can be translated into linear logic by reading an intuitionistic implication $A \to B$ as $!A \multimap B$: an ordinary function is a linear function whose argument may be used any number of times.

Here is how the four rules correspond to Rust. The rest of the chapter explains every row:

Linear logic	Rust
$!\tau$	a type `T` with `T: Copy`
!-Contr	a `Copy` value is copied implicitly at every use
!-Weak	discarding a `Copy` value is free: it has no destructor
Dereliction	a `Copy` value can be used like any other value
Promotion	`#[derive(Copy)]` requires every field to be `Copy`
a term $\tau \multimap \tau \otimes \tau$	`Clone::clone`, called explicitly



Three ways to duplicate in Rust. Copy types are duplicated implicitly and for free; Clone types explicitly, with a call that may cost something; every other type is affine. The exponential of linear logic corresponds to Copy.

In the language of Chapter 19, a !-type comes with a copy map $A \to A \otimes A$ and a discard map $A \to \mathbf{1}$ that fit together: the comonoid structure that Chapter 19 found on the types that have a diagonal. We stay with type theory here.

23.3 `Copy`: contraction for free

In Rust, whether `let b = a;` moves `a` or copies it is decided by the type of `a`, not by anything written in the program. This chapter's Rust example is `code/rust/examples/ch23_copy_clone.rs`:

```rust
let a: i32 = 5;
let b = a; // a copy: `a` is still usable
let s = String::from("report");
let t = s; // a move: `s` is not usable any more
println!("a = {a}, b = {b}, t = {t}");
```

```
a = 5, b = 5, t = report
```

The two `let`s have the same syntax and different rules. For `i32`, the compiler applies contraction silently: `a` stays usable. For `String`, it applies the affine rule: `s` is moved, and a later use would be E0382.

There is one difference between `Copy` and ! that shapes everything else. !τ is a *type constructor*: from any type τ you can form !τ and then build its values by promotion. `Copy` is a *property* of a type: a type is `Copy` or it is not, and there is no `!String`. A `Copy` type behaves as if it were already of the form !τ. Contraction and weakening happen at every use without being written, and dereliction is invisible, because a value of a `Copy` type can always be used as an ordinary value. What remains of promotion is the rule that decides which types may be `Copy`.

Which types are `Copy`

Type	`Copy`?	Why
`i32`, `bool`, `char`, `f64`, `()`	yes	plain bits, nothing owned
`&T`, for every `T`	yes	a shared read permission
`&mut T`	no	a unique permission
`(A, B)`, `[T; N]`, `Option<T>`	if the components are	promotion
a struct or enum	if it derives `Copy` and all fields are `Copy`	promotion
`fn` items and `fn` pointers	yes	they capture nothing
a closure	if everything it captures is `Copy` or a shared reference	promotion
`String`, `Vec<T>`, `Box<T>`	no	they own a heap allocation
any type with a `Drop` impl	no	see below

A copy is a bitwise copy of the value, the same operation as a move. What differs is only whether the source stays usable afterwards:

```rust
/// Contraction for free: a `Copy` value is duplicated by using it twice.
fn dup_copy<T: Copy>(x: T) -> (T, T) {
    (x, x)
}

/// Contraction on request: a `Clone` value is duplicated by calling `clone`.
fn dup_clone<T: Clone>(x: T) -> (T, T) {
```

```
    (x.clone(), x)
}
```

```
let p = Point { x: 1, y: 2 };
let (p1, p2) = dup_copy(p);
println!("{p1:?} {p2:?}, and p is still {p:?}");
// A copy is a new value in a new place, bit for bit the same.
let q = p;
println!(
    "same bits: {}, same place: {}",
    p == q,
    std::ptr::eq(&p, &q)
);
// A shared reference is Copy: the copies point to the same String.
let r = &t;
let (r1, r2) = dup_copy(r);
println!("two references, one String: {}", std::ptr::eq(r1, r2));
```

```
Point { x: 1, y: 2 } Point { x: 1, y: 2 }, and p is still Point { x: 1, y: 2 }
same bits: true, same place: false
two references, one String: true
```

`dup_copy` is $\lambda x.\,(x, x)$, the term that Chapter 22 proved untypable, and its bound `T: Copy` is what makes it legal: Rust's generics have an explicit contraction permission. The copy `q` has the same bits as `p` and lives in a different place (`std::ptr::eq` compares addresses). Copying a shared reference duplicates the *reference*: the two copies point to the same `String`. That is why `&T` can be `Copy` for every `T`: a shared reference is a permission to read, and handing out more read permissions does not duplicate the data. Chapter 26 takes this further, and shows that it is precisely why affinity alone cannot explain references.

Promotion: every field must be `Copy`

A struct can derive `Copy` only if all its fields are `Copy`:

```
/// All fields are `Copy`, and there is no destructor: the derive is accepted.
#[derive(Clone, Copy, Debug, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}
```

```
#[derive(Clone, Copy)]
struct Reading {
    sensor: String,
    value: f64,
}
```

```
error[E0204]: the trait `Copy` cannot be implemented for this type
 --> compile_fail/ch23_copy_string_field.rs:6:8
  |
5 | #[derive(Clone, Copy)]
  |                 ---- in this derive macro expansion
6 | struct Reading {
  |        ^^^^^^^
7 |     sensor: String,
  |     -------------- this field does not implement `Copy`
```

This is promotion. To make `Reading` duplicable, every resource it is built from must be duplicable, and a `String` is not: copying a `Reading` bit by bit would give two owners of one heap buffer, and two frees. The rule is not specific to `derive`; a hand-written `impl Copy for Reading {}` is rejected with the same error.

No destructors

A type with a destructor cannot be `Copy`, even if all its fields are:

```
#[derive(Clone, Copy)]
struct Token {
    id: u64,
}

impl Drop for Token {
    fn drop(&mut self) {
        println!("releasing token {}", self.id);
    }
}
```

```
error[E0184]: the trait `Copy` cannot be implemented for this type; the type has a
destructor
  --> compile_fail/ch23_copy_with_drop.rs:7:8
   |
 6 | #[derive(Clone, Copy)]
   |                 ---- in this derive macro expansion
 7 | struct Token {
   |        ^^^^^ `Copy` not allowed on types with destructors
   |
note: destructor declared here
  --> compile_fail/ch23_copy_with_drop.rs:12:5
   |
12 |     fn drop(&mut self) {
   |     ^^^^^^^^^^^^^^^^^^
```

`Copy` makes contraction implicit, and a destructor makes weakening an action (Chapter 24). Put the two together and every silent copy would bring one more silent destructor call: a token copied three times would be released four times. In the rules above, that is !-Weak: discarding a $!\tau$ must be free, and a value whose discarding runs code is not a $!\tau$. This is the reason Chapter 24 gives for E0184.

`Copy` needs `Clone`

In the standard library the trait is declared as `pub trait Copy: Clone`, so deriving `Copy` without `Clone` is an error too (E0277, "the trait bound `Point: Clone` is not satisfied"). Any type that may be duplicated implicitly may also be duplicated explicitly, and the documentation of `Clone` asks that, for a `Copy` type, `clone` be equivalent to a copy (`unsafe` code is not allowed to rely on that). We will prove the type-theoretic version of "`Copy` implies `Clone`" in Lean below.

`&mut T` is not `Copy`

A mutable reference is a *unique* permission to read and write. Duplicating it would give two writers, which is exactly what the borrow checker exists to prevent, so `&mut T` is affine:

```
fn writers(w: &mut String) -> (&mut String, &mut String) {
    let a = w;
    let b = w;
```

```
    (a, b)
}
```

```
error[E0382]: use of moved value: `w`
 --> compile_fail/ch23_mut_ref_not_copy.rs:7:13
  |
5 | fn writers(w: &mut String) -> (&mut String, &mut String) {
  |            - move occurs because `w` has type `&mut String`, which does not implement
the `Copy` trait
6 |     let a = w;
  |             - value moved here
7 |     let b = w;
  |             ^ value used here after move
```

There is a caveat that every Rust programmer has run into. A `&mut` reference *can* be passed to two calls in a row:

```
fn record(log: &mut Vec<&'static str>, event: &'static str) {
    log.push(event);
}

/// `&mut` is not `Copy`, yet `log` is passed twice: each call receives a
/// fresh reborrow `&mut *log`, not a copy of `log` (Chapter 28).
fn record_twice(log: &mut Vec<&'static str>) {
    record(log, "first");
    record(log, "second");
}
```

```
log: ["first", "second"]
```

That is not a copy. Where a `&mut` reference is expected, the compiler passes a fresh *reborrow* `&mut *log`, a new unique reference that lives only as long as the call, and `log` becomes usable again afterwards. Reborrowing belongs to the calculus of borrows, and Chapter 28 derives this very program. (It is also why the tuple `(a, w)` in place of `(a, b)` above gets a slightly different message, "borrow of moved value": the compiler tries to reborrow `w` for the tuple.)

Functions

`fn` items and function pointers are `Copy`: a function that captures nothing can be duplicated freely, even as a value (`let g = send; g(a); g(b)` compiles). That is what justifies the `affine` crate in treating declared functions as constants that can be called again and again. A closure is a function *with captured values*, and it is `Copy` exactly when what it captures may be duplicated: promotion again. A closure that has captured a `String` by value is not `Copy`, and if calling it consumes the `String`, it is only `FnOnce`, the $\multimap$ of Chapter 22. In λ^{aff} the type $\tau \multimap \sigma$ says nothing about what a function captured, so the safe choice is never to make a function type `Copy`. We will see in Lean what goes wrong otherwise.

23.4 `Clone`: duplication as a term

`Clone` has one method, `fn clone(&self) -> Self`. It is an ordinary function that the programmer calls, and it may do anything: allocate a new buffer and copy bytes into it, increment a counter, log a message. The difference between `Copy` and `Clone` is the difference between a structural rule and a term:

	`Copy`	`Clone`
status	a structural rule (S-Both below)	a term, $\tau \multimap \tau \otimes \tau$
written in the program	never	always: `x.clone()`
cost	a bitwise copy	anything
effects	none	possible

In the calculus, a `Clone` type comes with a constant $dup_\tau : \tau \multimap \tau \otimes \tau$, and a program that needs two copies of x writes `let` $(x_1, x_2) = dup_\tau\ x$ `in` ..., using x once. (Rust's `clone` takes `&self`, a borrow, and leaves the original where it was, so the second copy is the original itself; borrowing is Part VI, and $\tau \multimap \tau \otimes \tau$ is its affine shadow.) The generic function `dup_clone` above is exactly this term.

The call is visible in the source, and so is its cost:

```
/// A type whose duplication has a visible effect: every clone is counted.
#[derive(Debug)]
struct Specimen {
    id: u32,
    clones: Rc<Cell<u32>>,
}

impl Clone for Specimen {
    fn clone(&self) -> Specimen {
        self.clones.set(self.clones.get() + 1);
        Specimen {
            id: self.id,
            clones: Rc::clone(&self.clones),
        }
    }
}
```

```
let (u, v) = dup_clone(String::from("calibration"));
println!(
    "cloned strings equal: {}, same buffer: {}",
    u == v,
    u.as_ptr() == v.as_ptr()
);
let counter = Rc::new(Cell::new(0));
let specimen = Specimen {
    id: 7,
    clones: Rc::clone(&counter),
};
let (s1, s2) = dup_clone(specimen);
let (s3, _s4) = dup_clone(s1);
println!(
    "specimens {} and {}: {} clones so far",
    s2.id,
    s3.id,
    counter.get()
);
```

```
cloned strings equal: true, same buffer: false
specimens 7 and 7: 2 clones so far
```

Cloning a `String` allocates a new buffer (the two strings are equal, their buffers are not the same), and cloning a `Specimen` runs code that updates a counter: two calls to `dup_clone`, two clones. Neither could happen silently, which is the point. A type for which duplication has a cost or an effect should make every duplication visible, and Rust achieves that by making such a type `Clone` but not `Copy`.

Not every type is `Clone`. A file handle is not: duplicating a handle to an open file would need a decision about what two owners of one file descriptor mean (the standard library's `File` offers a separate `try_clone` for that). For such types the calculus has no dup_τ at all, and they stay strictly affine.

23.5 `Rc` and `Arc`: counted sharing

`Rc<T>` takes a third way. It does not duplicate the `T` at all. It puts the `T` in a heap allocation together with a counter, and an `Rc<T>` is a pointer to that allocation. `Rc::clone` copies the pointer and increments the counter; each drop of an `Rc` decrements it; when the counter reaches zero, the `T` is dropped and the allocation freed:

```
let config = Rc::new(String::from("lab-7"));
println!("count after new: {}", Rc::strong_count(&config));
let reader = Rc::clone(&config);
let writer = config.clone();
println!("count after 2 clones: {}", Rc::strong_count(&config));
println!("one allocation: {}", Rc::ptr_eq(&reader, &writer));
drop(reader);
drop(writer);
println!("count after 2 drops: {}", Rc::strong_count(&config));
```

```
count after new: 1
count after 2 clones: 3
one allocation: true
count after 2 drops: 1
```

So `Rc` turns contraction into bookkeeping. Every duplication of the pointer is recorded (the count goes up), and so is every discard (the count goes down). The shared `T` itself is never duplicated. An `Rc<T>` dereferences to a shared reference `&T`, so the owners can all read the value; to mutate it you need either proof that you are the only owner (`Rc::get_mut` checks the count) or the interior mutability of Chapter 26. In the vocabulary of this chapter, `Rc<T>` is *not* $!T$: it makes both contraction and weakening *explicit and counted*, where ! makes them silent.

That is also why `Rc<T>` cannot be `Copy`, whatever `T` is. A bitwise copy of the pointer would not increment the counter, and the two copies would later decrement it twice. And `Rc` has a destructor, the decrement, which E0184 forbids anyway. rustc knows the situation well enough to give it a special message:

```
fn share(cfg: Rc<String>) -> (Rc<String>, Rc<String>) {
    (cfg, cfg)
}
```

```
error[E0382]: the type `Rc` does not implement `Copy`
 --> compile_fail/ch23_rc_not_copy.rs:9:11
  |
8 | fn share(cfg: Rc<String>) -> (Rc<String>, Rc<String>) {
  |          --- this move could be avoided by cloning the original `Rc`, which is
inexpensive
9 |     (cfg, cfg)
  |      ---  ^^^ value used here after move
  |      |
  |      value moved here
  |
  = note: consider using `Rc::clone`
```

```
help: clone the value to increment its reference count
  |
9 |     (cfg.clone(), cfg)
  |         ++++++++
```

`Arc<T>` is the same with an atomic counter, so that clones can be sent to other threads:

```
let readings = Arc::new(vec![10, 20, 30]);
let handles: Vec<_> = (0..3)
    .map(|i| {
        let mine = Arc::clone(&readings);
        thread::spawn(move || mine[i] * 2)
    })
    .collect();
let doubled: Vec<i32> = handles.into_iter().map(|h| h.join().unwrap()).collect();
println!(
    "{doubled:?}, count back to {}",
    Arc::strong_count(&readings)
);
```

```
[20, 40, 60], count back to 1
```

Each thread receives its own clone of the pointer and drops it when it finishes, so after the joins the count is back to 1. Counting has a known blind spot: two `Rc`s that point to each other keep each other's counts above zero forever. Such a cycle is never freed, and Chapter 24 explains why that is a leak and not a safety problem.

23.6 The formal statement

We now extend λ^{aff} to make all of this precise. The extension has two parts, and the typing rules of Chapter 22 do not change at all.

A `Copy` predicate on types. Fix a set $\mathcal{C}$ of base types that are `Copy` (for us, the scalar types `i32`, `u64`, `bool`, `char`). Then

$$\frac{b \in \mathcal{C}}{\mathit{Copy}(b)} \qquad \frac{}{\mathit{Copy}(\mathbf{1})} \qquad \frac{\mathit{Copy}(\tau) \qquad \mathit{Copy}(\sigma)}{\mathit{Copy}(\tau \otimes \sigma)} \qquad \frac{\mathit{Copy}(\tau) \qquad \mathit{Copy}(\sigma)}{\mathit{Copy}(\tau \oplus \sigma)}$$

There is no rule for $\tau \multimap \sigma$ and none for $\tau \,\&\, \sigma$. A function value may hold captured resources, and an offer is a suspended computation over resources; in both cases the type does not mention them, so the type cannot promise that duplicating the value is harmless.

A fourth splitting rule. A `Copy` variable may go to *both* sides of a split:

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \mathit{Copy}(\tau)}{\Gamma, x : \tau = (\Gamma_1, x : \tau) \uplus (\Gamma_2, x : \tau)}\ \text{S-Both}$$

That is the whole extension. Compare it with the rules of !:

- S-Both is !-Contr, folded into the splitting relation instead of being a rule of its own. Wherever the rules of Chapter 22 split a context, a `Copy` variable may now be given to both premises.
- !-Weak is already there, as for every variable of an affine calculus.
- Dereliction is invisible: a `Copy` variable is still used by A-Var, like any other.

- Promotion survives as the definition of *Copy*: a compound type is `Copy` only if it is built from `Copy` types. That is `#[derive(Copy)]`'s rule.

Chapter 28 uses exactly these four splitting rules, S-Empty, S-Left, S-Right and S-Both, for its calculus of borrows, where shared references are `Copy` and mutable references are not.

In Lean

The Lean file of this chapter, `code/lean/Book/Ch23/Exponential.lean`, imports Chapter 22's files and reuses their types, terms and `uses` function. The `Copy` predicate is a function to `Bool`, decided by the structure of the type, as rustc decides it:

```
/-- The base types that are `Copy`: a few of Rust's scalar types. -/
def copyBase (name : String) : Bool :=
  name == "i32" || name == "u64" || name == "bool" || name == "char"

/-- The `Copy` predicate, decided by the structure of the type, the way rustc
decides it. A pair or a sum is `Copy` when its components are (this is the rule
of *promotion*). Functions and additive pairs are never `Copy`: their values
may hold resources that their types do not mention. -/
def isCopy : Ty → Bool
  | .base n => copyBase n
  | .unit => true
  | .tensor a b => isCopy a && isCopy b
  | .plus a b => isCopy a && isCopy b
  | .lolli _ _ => false
  | .amp _ _ => false
```

```
def I32 : Ty := .base "i32"

example : isCopy I32 = true := rfl
example : isCopy (I32 ⊗ (I32 ⊕ .unit)) = true := rfl
example : isCopy Str = false := rfl
example : isCopy (I32 ⊗ Str) = false := rfl
example : isCopy (I32 ⊸ I32) = false := rfl

/-- Promotion: a pair is `Copy` exactly when both components are. -/
theorem isCopy_tensor (τ σ : Ty) :
    isCopy (τ ⊗ σ) = true ↔ isCopy τ = true ∧ isCopy σ = true := by
  simp [isCopy]
```

Each `example` is proved by `rfl`: Lean evaluates `isCopy` and finds `true` or `false`. The theorem `isCopy_tensor` is promotion for pairs, and `simp [isCopy]` proves it by unfolding the definition and simplifying the Boolean conjunction.

The splitting relation gets the fourth constructor. We make the predicate a *parameter* `cp` of the relation, so that at the end of the chapter we can compare the right choice of `Copy` types with a wrong one; the book's choice is `isCopy`:

```
/-- Splitting with duplication: the four rules of Chapter 22, plus `both`,
which gives a variable to both sides when its type satisfies the predicate
`cp`. Rule `both` is contraction, restricted to the types that `cp` accepts.
The predicate is a parameter so that we can compare choices; the book's is
`isCopy`. -/
inductive SplitC (cp : Ty → Bool) : Ctx → Ctx → Ctx → Prop where
  | nil : SplitC cp [] [] []
  | left {Γ Γ₁ Γ₂ : Ctx} {τ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → SplitC cp (some τ :: Γ) (some τ :: Γ₁) (none :: Γ₂)
  | right {Γ Γ₁ Γ₂ : Ctx} {τ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → SplitC cp (some τ :: Γ) (none :: Γ₁) (some τ :: Γ₂)
  | hole {Γ Γ₁ Γ₂ : Ctx} :
```

```
      SplitC cp Γ Γ₁ Γ₂ → SplitC cp (none :: Γ) (none :: Γ₁) (none :: Γ₂)
  | both {Γ Γ₁ Γ₂ : Ctx} {τ : Ty} :
      cp τ = true →
      SplitC cp Γ Γ₁ Γ₂ → SplitC cp (some τ :: Γ) (some τ :: Γ₁) (some τ :: Γ₂)
```

The typing rules are those of Chapter 22, with `SplitC cp` in place of `Split`:

```
/-- The typing rules of `λ^aff` with duplicable types: the rules of Chapter 22,
word for word, with `SplitC cp` in place of `Split`. -/
inductive HasTypeC (cp : Ty → Bool) : Ctx → Term → Ty → Prop where
  | var {Γ : Ctx} {x : Nat} {τ : Ty} :
      Γ[x]? = some (some τ) → HasTypeC cp Γ (.var x) τ
  | lam {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasTypeC cp (some τ :: Γ) e σ → HasTypeC cp Γ (.lam τ e) (τ ⊸ σ)
  | app {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → HasTypeC cp Γ₁ e₁ (τ ⊸ σ) → HasTypeC cp Γ₂ e₂ τ →
      HasTypeC cp Γ (.app e₁ e₂) σ
  | unit {Γ : Ctx} : HasTypeC cp Γ .unit .unit
  | pair {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → HasTypeC cp Γ₁ e₁ τ → HasTypeC cp Γ₂ e₂ σ →
      HasTypeC cp Γ (.pair e₁ e₂) (τ ⊗ σ)
  | letPair {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ ρ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → HasTypeC cp Γ₁ e₁ (τ ⊗ σ) →
      HasTypeC cp (some σ :: some τ :: Γ₂) e₂ ρ → HasTypeC cp Γ (.letPair e₁ e₂) ρ
  | wpair {Γ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      HasTypeC cp Γ e₁ τ → HasTypeC cp Γ e₂ σ → HasTypeC cp Γ (.wpair e₁ e₂) (τ & σ)
  | fst {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasTypeC cp Γ e (τ & σ) → HasTypeC cp Γ (.fst e) τ
  | snd {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasTypeC cp Γ e (τ & σ) → HasTypeC cp Γ (.snd e) σ
  | inl {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasTypeC cp Γ e τ → HasTypeC cp Γ (.inl σ e) (τ ⊕ σ)
  | inr {Γ : Ctx} {e : Term} {τ σ : Ty} :
      HasTypeC cp Γ e σ → HasTypeC cp Γ (.inr τ e) (τ ⊕ σ)
  | case {Γ Γ₁ Γ₂ : Ctx} {e e₁ e₂ : Term} {τ σ ρ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → HasTypeC cp Γ₁ e (τ ⊕ σ) →
      HasTypeC cp (some τ :: Γ₂) e₁ ρ → HasTypeC cp (some σ :: Γ₂) e₂ ρ →
      HasTypeC cp Γ (.case e e₁ e₂) ρ
  | letIn {Γ Γ₁ Γ₂ : Ctx} {e₁ e₂ : Term} {τ σ : Ty} :
      SplitC cp Γ Γ₁ Γ₂ → HasTypeC cp Γ₁ e₁ τ → HasTypeC cp (some τ :: Γ₂) e₂ σ →
      HasTypeC cp Γ (.letIn e₁ e₂) σ
```

Since the new splitting relation contains the old one, every derivation of Chapter 22 is still a derivation, whatever `cp` is; the proof is one line per rule:

```
/-- Every derivation of Chapter 22 is a derivation here, whatever the
predicate: adding duplication only adds programs. -/
theorem HasType.toC {cp : Ty → Bool} {Γ : Ctx} {e : Term} {τ : Ty}
    (h : HasType Γ e τ) : HasTypeC cp Γ e τ := by
  induction h with
  | var hx => exact .var hx
  | lam _ ih => exact .lam ih
  | app hs _ _ ih₁ ih₂ => exact .app (Split.toC hs) ih₁ ih₂
  | unit => exact .unit
  | pair hs _ _ ih₁ ih₂ => exact .pair (Split.toC hs) ih₁ ih₂
  | letPair hs _ _ ih₁ ih₂ => exact .letPair (Split.toC hs) ih₁ ih₂
  | wpair _ _ ih₁ ih₂ => exact .wpair ih₁ ih₂
  | fst _ ih => exact .fst ih
  | snd _ ih => exact .snd ih
  | inl _ ih => exact .inl ih
  | inr _ ih => exact .inr ih
  | case hs _ _ _ ih ih₁ ih₂ => exact .case (Split.toC hs) ih ih₁ ih₂
  | letIn hs _ _ ih₁ ih₂ => exact .letIn (Split.toC hs) ih₁ ih₂
```

And programs that Chapter 22 rejected are now accepted, when the types allow it:

```
/-- For a `Copy` type, `λx. (x, x)` is well typed: rule `both` gives `x` to the
two components. So every `Copy` type has a `clone`. -/
theorem copy_dup {τ : Ty} (hc : isCopy τ = true) :
    HasTypeC isCopy [] (dup τ) (τ ⊸ τ ⊗ τ) :=
  .lam (.pair (.both hc .nil) (.var rfl) (.var rfl))

/-- `n : i32 ⊢ (n, (n, n)) : i32 ⊗ (i32 ⊗ i32)`: a `Copy` variable can be used
any number of times. -/
example : HasTypeC isCopy [some I32] (.pair (.var 0) (.pair (.var 0) (.var 0)))
    (I32 ⊗ (I32 ⊗ I32)) :=
  .pair (.both rfl .nil) (.var rfl) (.pair (.both rfl .nil) (.var rfl) (.var rfl))
```

`copy_dup` is the type-theoretic version of "`Copy` implies `Clone`": for every `Copy` type, $\lambda x.\,(x, x)$ is a closed term of type $\tau \multimap \tau \otimes \tau$, the type of dup_τ. A `Copy` type needs no library function for duplication; the language provides one. The derivation is Chapter 22's failed derivation of `dup`, with the split that did not exist now supplied by `.both hc .nil`.

The generalized usage theorem

Chapter 22's usage theorem said that every variable is used at most once. With S-Both that is false, and `twice` is the counterexample. The right statement gives each variable a **multiplicity**: one for an ordinary variable, unbounded for a `Copy` variable, zero for a variable the term did not receive.

> **Theorem (generalized usage).** If $\Gamma \vdash e : \tau$ in λ^{aff} with `Copy`, then every variable of Γ whose type is not `Copy` occurs in e at most once (with the counting convention of Chapter 22), and a variable that is not in Γ does not occur at all.

To state the multiplicities in Lean we need a number system with an "unbounded" element. Mathlib provides it: `ℕ∞` (`ENat`), the natural numbers extended with a top element `⊤`, with an addition in which $n + \top = \top$. We import only `Mathlib.Data.ENat.Monoid`, which provides `ℕ∞`'s arithmetic and ordering, and one lemma file about casts:

```
/-- The multiplicity of variable `i` in `Γ`, in `ℕ∞ = ℕ ∪ {⊤}`: unbounded for
a duplicable variable, one for any other variable, zero for a hole. -/
def availC (cp : Ty → Bool) (Γ : Ctx) (i : Nat) : ℕ∞ :=
  match Γ[i]? with
  | some (some τ) => if cp τ then ⊤ else 1
  | _ => 0
```

The key lemma changes from an equation to an inequality. A split now hands out *at most* what the context had, because the `both` rule gives `⊤` to both sides of a position that had `⊤`, and $\top + \top = \top$:

```
/-- Splitting never creates resources. It may duplicate them, but only those
whose multiplicity is already `⊤`, and `⊤ + ⊤ = ⊤`. -/
theorem SplitC.avail_le {cp : Ty → Bool} {Γ Γ₁ Γ₂ : Ctx} (hs : SplitC cp Γ Γ₁ Γ₂)
    (i : Nat) : availC cp Γ₁ i + availC cp Γ₂ i ≤ availC cp Γ i := by
  induction hs generalizing i with
  | nil => simp [availC]
  | left _ ih | right _ ih | hole _ ih =>
    cases i with
    | zero => simp [availC]
    | succ i => simpa using ih i
  | both hc _ ih =>
    cases i with
    | zero => simp [availC, hc]          -- ⊤ + ⊤ ≤ ⊤
    | succ i => simpa using ih i
```

The proof is the same as Chapter 22's, with one more case. In the `both` case at position 0, `simp [availC, hc]` uses the hypothesis `hc : cp τ = true` to rewrite each multiplicity to `⊤`, and Mathlib's simplification lemmas for `ℕ∞` know that $\top + \top \le \top$. The theorem itself also follows Chapter 22's proof step by step:

```
/-- **The generalized usage theorem.** Every variable is used at most as often as
its multiplicity allows, for every choice of the duplicable types. -/
theorem usageC {cp : Ty → Bool} {Γ : Ctx} {e : Term} {τ : Ty} (h : HasTypeC cp Γ e τ) :
    ∀ i, (uses i e : ℕ∞) ≤ availC cp Γ i := by
  induction h with
  | @var Γ x τ hx =>
    intro i
    by_cases hi : i = x
    · -- The variable itself: used once, and its multiplicity is 1 or ⊤.
      subst hi
      simp only [uses, ↓reduceIte, Nat.cast_one, availC, hx]
      split <;> simp
    · simp [uses, hi]
  | unit => intro i; simp [uses]
  | lam _ ih => intro i; simpa [uses] using ih (i + 1)
  | app hs _ _ ih₁ ih₂ | pair hs _ _ ih₁ ih₂ =>
    intro i
    simp only [uses, Nat.cast_add]
    exact (add_le_add (ih₁ i) (ih₂ i)).trans (hs.avail_le i)
  | letPair hs _ _ ih₁ ih₂ =>
    intro i
    have h₂ := ih₂ (i + 2)
    simp only [availC_cons_succ] at h₂
    simp only [uses, Nat.cast_add]
    exact (add_le_add (ih₁ i) h₂).trans (hs.avail_le i)
  | letIn hs _ _ ih₁ ih₂ =>
    intro i
    have h₂ := ih₂ (i + 1)
    simp only [availC_cons_succ] at h₂
    simp only [uses, Nat.cast_add]
    exact (add_le_add (ih₁ i) h₂).trans (hs.avail_le i)
  | case hs _ _ _ ih ih₁ ih₂ =>
    intro i
    have h₁ := ih₁ (i + 1)
    have h₂ := ih₂ (i + 1)
    simp only [availC_cons_succ] at h₁ h₂
    simp only [uses, Nat.cast_add]
    exact (add_le_add (ih i) (cast_max_le h₁ h₂)).trans (hs.avail_le i)
  | wpair _ _ ih₁ ih₂ =>
    intro i
    simp only [uses]
    exact cast_max_le (ih₁ i) (ih₂ i)
  | fst _ ih | snd _ ih | inl _ ih | inr _ ih =>
    intro i; simpa [uses] using ih i
```

The counts `uses i e` are natural numbers, and the multiplicities live in `ℕ∞`, so the statement casts the count with `(uses i e : ℕ∞)`. The proof then has to move between the two. In the `app` case, `Nat.cast_add` rewrites the cast of a sum as a sum of casts; `add_le_add` adds the two induction hypotheses, giving $u_1 + u_2 \le a_1 + a_2$; and `.trans` chains that with the splitting lemma $a_1 + a_2 \le a$. Chapter 22's `omega` cannot be used here, because it works with natural numbers and integers only, and `⊤` is neither. For the `max` of the sharing rules there is a small helper, `cast_max_le`, proved just before the theorem with Mathlib's fact that the cast from `ℕ` to `ℕ∞` is monotone. In the `var` case, `split <;> simp` considers the two possible multiplicities of the variable, `⊤` and `1`, and checks that `1` is below each.

The familiar form is a corollary, and so is the fact that `dup` is still untypable at a non-`Copy` type:

```
/-- **Non-duplicable variables are used at most once.** -/
theorem non_copy_at_most_once {cp : Ty → Bool} {Γ : Ctx} {e : Term} {τ σ : Ty}
    (h : HasTypeC cp Γ e τ) {i : Nat} (hi : Γ[i]? = some (some σ)) (hσ : cp σ = false) :
    uses i e ≤ 1 := by
  have := usageC h i
  simp [availC, hi, hσ] at this
  exact_mod_cast this

/-- So `λx. (x, x)` is still untypable at every non-`Copy` type. -/
theorem dup_untypable_c {τ ρ : Ty} (hσ : isCopy τ = false) (Γ : Ctx) :
    ¬ HasTypeC isCopy Γ (dup τ) ρ := by
  intro h
  cases h with
  | lam h =>
    have := non_copy_at_most_once h (i := 0) rfl hσ
    simp [uses] at this
```

For a non-`Copy` variable, `simp [availC, hi, hσ]` turns the theorem's right-hand side into `1`, leaving `(uses i e : ℕ∞) ≤ 1`. The tactic `exact_mod_cast` removes the casts from that hypothesis and from the goal, and finds that they are the same statement about natural numbers.

The theorem holds for *every* predicate `cp`, not just for `isCopy`. That is worth pausing on, because it means that the usage theorem alone cannot tell a good choice of `Copy` types from a bad one. The next section shows what can.

Why functions are not `Copy`

Suppose we made every function type `Copy`, whatever the function captures. Here is a program that uses that freedom, with $s : \mathit{String}$ in the context:

$$\texttt{let}\ f = \lambda u : \mathbf{1}.\, s\ \texttt{in}\ (f\,(), f\,())$$

The closure f captures s; the body calls f twice. With function types `Copy`, the `let` body may give f to both components of the pair, and s occurs once in the program text, so everything type-checks. Now run one step: the `let` substitutes the closure for f, and the result is

$$((\lambda u : \mathbf{1}.\, s)\,(), (\lambda u : \mathbf{1}.\, s)\,())$$

in which s occurs twice and must be given to both components. String is not `Copy`, so this term has no type. A well-typed program stepped to an ill-typed one: **preservation fails**. In Lean:

```
/-- A bad choice: every function type is duplicable, whatever it captures. -/
def fnCopy : Ty → Bool
  | .lolli _ _ => true
  | t => isCopy t

/-- With `s : String` (variable 0): `let f = λu:1. s in (f (), f ())`. The
closure captures `s`; the body calls the closure twice. -/
def callTwice : Term :=
  .letIn (.lam .unit (.var 1))
    (.pair (.app (.var 0) .unit) (.app (.var 0) .unit))

/-- What `callTwice` becomes after one step, `let f = v in e → e[v/f]`: the
closure, and with it `s`, now appears twice. -/
def callTwice' : Term :=
  .pair (.app (.lam .unit (.var 1)) .unit) (.app (.lam .unit (.var 1)) .unit)

/-- If closures were `Copy` whatever they capture, preservation would fail:
`callTwice` is typable, but the term it steps to is not. -/
theorem promotion_matters :
    HasTypeC fnCopy [some Str] callTwice (Str ⊗ Str) ∧
```

```
    ¬ HasTypeC fnCopy [some Str] callTwice' (Str ⊗ Str) := by
  constructor
  · -- `s` goes to the closure; the closure is shared by the two calls.
    exact .letIn (.left .nil) (.lam (.var rfl))
      (.pair (.both rfl (.hole .nil))
        (.app (.left (.hole .nil)) (.var rfl) .unit)
        (.app (.left (.hole .nil)) (.var rfl) .unit))
  · -- After the step, `s` occurs twice, and it is not duplicable.
    intro h
    have := non_copy_at_most_once h (i := 0) rfl rfl
    simp [callTwice', uses] at this

/-- With the real `isCopy`, `callTwice` is rejected in the first place: the
closure `f` is used twice, and a function type is never `Copy`. -/
theorem callTwice_rejected : ¬ HasTypeC isCopy [some Str] callTwice (Str ⊗ Str) := by
  intro h
  cases h with
  | letIn _ hf hbody =>
    -- The let-bound value is a λ, so `f` has a function type.
    cases hf with
    | lam _ =>
      have := non_copy_at_most_once hbody (i := 0) rfl rfl
      simp [uses] at this
```

The first half of `promotion_matters` is the derivation of the program, by hand. The split `.left .nil` gives `s` to the closure, the split `.both rfl (.hole .nil)` gives the closure `f` to both calls (allowed, since `fnCopy` says yes to function types), and each call's split `.left (.hole .nil)` gives `f` to the function position. The second half is the generalized usage theorem applied to the reduct: variable 0, which is `s`, occurs twice and has multiplicity 1. The last theorem shows that the real predicate rejects the program from the start: after inverting the `let`, and the λ to learn that `f` has a function type, the usage theorem finds the non-`Copy` variable `f` used twice.

This is promotion, seen from the other side. A value may be made duplicable only if everything it holds is duplicable. Rust's rule for closures, `Copy` only when every capture is, is that rule; in λ^{aff}, where a function type does not record its captures, the only safe rule is "never".

23.7 `Copy` in the `affine` crate

The crate implements the extension with a `Discipline` switch: `Affine` is Chapter 22's calculus, and `Copy` turns on the predicates of `copy.rs`, which follow Rust's rules rather than the minimal calculus (they include references, structs and `Rc`):

```
/// The `Copy` predicate, decided by the structure of the type, as in rustc.
pub fn is_copy(program: &Program, ty: &Ty) -> bool {
    match ty {
        Ty::Unit => true,
        Ty::Named(name) => {
            COPY_SCALARS.contains(&name.as_str())
                || program.structure(name).is_some_and(|s| derives(s, "Copy"))
        }
        // Promotion: a pair or a sum is Copy when its components are.
        Ty::Tuple(a, b) | Ty::Either(a, b) => {
            is_copy(program, a) && is_copy(program, b)
        }
        // A shared reference is a read permission: duplicating it is harmless.
        // A mutable reference is a unique permission: it is not Copy.
        Ty::Ref { mutable, .. } => !mutable,
        // Closures may own captured resources that their type does not show,
        // an offer `With<A, B>` is a suspended computation over resources, and
```

```
        // an `Rc` must count its copies: none of them is Copy.
        Ty::Fn(..) | Ty::With(..) | Ty::Rc(..) => false,
    }
}
```

```
/// The `Clone` predicate: every `Copy` type, plus types with an explicit,
/// possibly expensive duplication (`String`, `Rc<T>`, structs that derive it).
pub fn is_clone(program: &Program, ty: &Ty) -> bool {
    match ty {
        Ty::Unit => true,
        Ty::Named(name) => {
            COPY_SCALARS.contains(&name.as_str())
                || name == "String"
                || program.structure(name).is_some_and(|s| derives(s, "Clone"))
        }
        Ty::Tuple(a, b) | Ty::Either(a, b) => {
            is_clone(program, a) && is_clone(program, b)
        }
        Ty::Ref { mutable, .. } => !mutable,
        Ty::Rc(_) => true,
        Ty::Fn(..) | Ty::With(..) => false,
    }
}
```

The checker uses `is_copy` in one place, the use of a place that we saw in Chapter 22: a use of a `Copy` place records no move, which is rule S-Both in algorithmic form (the variable stays in the leftovers, for the next subterm to use). A call `x.clone()` requires `is_clone` and reads `x` without moving it. With the `Copy` discipline, `twice` is accepted, and its derivation shows the same variable in both premises of A-Pair:

```
== ch23_twice.aff, with Copy (Chapter 23)
ok: fn twice(n: i32) -> (i32, i32)
[A-Pair] n: i32 ⊢ (n, n) : (i32, i32)
  [A-Var] n: i32 ⊢ n : i32
  [A-Var] n: i32 ⊢ n : i32
```

The crate also accepts the programs of `ch23_copy.aff`, each of which the pure affine discipline rejects (the example checks that too):

```
#[derive(Clone, Copy)]
struct Point { x: i32, y: i32 }

struct Reading { sensor: String, value: i32 }

// A Copy struct can be used twice, and its fields read freely.
fn mirror(p: Point) -> (Point, Point) {
    (p, Point { x: p.y, y: p.x })
}

// Copy fields are copied out; the String field stays where it is.
fn values(r: Reading) -> ((i32, i32), Reading) {
    ((r.value, r.value), r)
}

// A shared reference is Copy.
fn readers(r: &String) -> (&String, &String) {
    (r, r)
}

// Clone is explicit, and leaves the original in place.
fn backup(s: String) -> (String, String) {
    (s.clone(), s)
}
```

```
// Cloning an Rc copies a pointer and counts the copies.
fn share(cfg: Rc<Config>) -> (Rc<Config>, Rc<Config>) {
    (cfg.clone(), cfg)
}
```

```
== ch23_copy.aff, with Copy
ok: fn mirror(p: Point) -> (Point, Point)
ok: fn values(r: Reading) -> ((i32, i32), Reading)
ok: fn readers(r: &String) -> (&String, &String)
ok: fn backup(s: String) -> (String, String)
ok: fn share(cfg: Rc<Config>) -> (Rc<Config>, Rc<Config>)
```

`values` combines `Copy` with Chapter 22's places: copying the `Copy` field `r.value` twice moves nothing, so `r` is still whole afterwards. Rust accepts the same function. Finally, the checks that rustc makes on `#[derive(Copy)]` are implemented as a direct transcription of the three rules:

```
/// The checks rustc makes when a struct derives `Copy`: `Clone` must be
/// derived too (E0277), every field must be `Copy` (E0204), and the type must
/// not have a destructor (E0184).
pub fn check_derives(program: &Program, s: &StructDef) -> Vec<Diagnostic> {
    let mut errors = Vec::new();
    let Some(copy) = s.derives.iter().find(|d| d.name == "Copy") else {
        return errors;
    };
    let name = &s.name.name;
    if !derives(s, "Clone") {
        errors.push(
            Diagnostic::new(
                "E0277",
                &format!("the trait bound `{name}: Clone` is not satisfied"),
            )
            .with(Label::secondary(
                copy.span,
                "in this derive macro expansion",
            ))
            .with(Label::primary(
                s.name.span,
                &format!("the trait `Clone` is not implemented for `{name}`"),
            ))
            .note("note: `Copy` requires `Clone`: `pub trait Copy: Clone`"),
        );
    }
    let bad_fields: Vec<_> = s
        .fields
        .iter()
        .filter(|(_, ty, _)| !is_copy(program, ty))
        .collect();
    if !bad_fields.is_empty() {
        let mut d = Diagnostic::new(
            "E0204",
            "the trait `Copy` cannot be implemented for this type",
        )
        .with(Label::secondary(
            copy.span,
            "in this derive macro expansion",
        ))
        .with(Label::primary(s.name.span, ""));
        for (_, _, span) in bad_fields {
            d = d.with(Label::secondary(
                *span,
                "this field does not implement `Copy`",
            ));
        }
        errors.push(d);
    }
```

```
    if let Some(span) = program.items.iter().find_map(|item| match item {
        crate::syntax::Item::DropImpl { ty, span } if ty.name == *name => Some(*span),
        _ => None,
    }) {
        errors.push(
            Diagnostic::new(
                "E0184",
                "the trait `Copy` cannot be implemented for this type; the type has a
destructor",
            )
            .with(Label::secondary(copy.span, "in this derive macro expansion"))
            .with(Label::primary(s.name.span, "`Copy` not allowed on types with
destructors"))
            .child("destructor declared here", Label::primary(span, "")),
        );
    }
    errors
}
```

Here are the crate's errors for the program `ch23_errors.aff`, next to rustc's from earlier in the chapter. Promotion:

```
error[E0204]: the trait `Copy` cannot be implemented for this type
 --> ch23_errors.aff:2:8
  |
1 | #[derive(Clone, Copy)]
  |                 ---- in this derive macro expansion
2 | struct Reading { sensor: String, value: f64 }
  |        ^^^^^^^   -------------- this field does not implement `Copy`
```

The destructor:

```
error[E0184]: the trait `Copy` cannot be implemented for this type; the type has a
destructor
 --> ch23_errors.aff:5:8
  |
4 | #[derive(Clone, Copy)]
  |                 ---- in this derive macro expansion
5 | struct Token { id: u64 }
  |        ^^^^^ `Copy` not allowed on types with destructors
  |
note: destructor declared here
 --> ch23_errors.aff:7:1
  |
7 | impl Drop for Token {}
  | ^^^^^^^^^^^^^^^^^^^^^^
```

Both match rustc's diagnostics label for label. (rustc points the note at the `fn drop` inside the `impl`; the crate's `impl Drop for Token {}` has no body, so it points at the whole `impl`.) A mutable reference used twice is an ordinary E0382:

```
error[E0382]: use of moved value: `w`
  --> ch23_errors.aff:11:13
   |
 9 | fn writers(w: &mut String) -> (&mut String, &mut String) {
   |            - move occurs because `w` has type `&mut String`, which does not implement
the `Copy` trait
10 |     let a = w;
   |             - value moved here
11 |     let b = w;
   |             ^ value used here after move
```

A file handle cannot be cloned:

```
error[E0599]: no method named `clone` found for `File` in the current scope
  --> ch23_errors.aff:16:8
   |
16 |     (f.clone(), f)
   |        ^^^^^ method not found in `File`
   |
   = note: `clone` is the method of the trait `Clone`, which this type does not implement
```

And an `Rc` used twice gets the generic E0382, with a hint that `Clone` is available. rustc's special wording for `Rc` ("the type `Rc` does not implement `Copy`", "clone the value to increment its reference count") says the same thing more helpfully:

```
error[E0382]: use of moved value: `cfg`
  --> ch23_errors.aff:20:11
   |
19 | fn share(cfg: Rc<Config>) -> (Rc<Config>, Rc<Config>) {
   |          --- move occurs because `cfg` has type `Rc<Config>`, which does not implement
the `Copy` trait
20 |     (cfg, cfg)
   |      ---  ^^^ value used here after move
   |      |
   |      value moved here
   |
   = help: consider cloning the value if the performance cost is acceptable
```

23.8 Putting it together: a measurement session

Here is a program from a laboratory, in which all three kinds of duplication meet. A device is measured twice on the same channel, and both samples are labeled with the operator's name:

```
/// A measurement: `Copy`, so it can be used as often as we like.
#[derive(Clone, Copy)]
struct Sample {
    channel: i32,
    value: i32,
}

/// The instrument: one handle, neither `Copy` nor `Clone`.
struct Device {
    reads: i32,
}

fn measure(dev: Device, ch: i32) -> (Sample, Device) {
    let value = 100 + 10 * ch + dev.reads;
    let sample = Sample { channel: ch, value };
    (
        sample,
        Device {
            reads: dev.reads + 1,
        },
    )
}

/// A labeled measurement. The operator's name is a `String`: `Clone`.
struct Record {
    op: String,
    sample: Sample,
}

fn label(op: String, sample: Sample) -> Record {
    Record { op, sample }
}
```

```
/// Measure one channel twice; label both samples with the operator's name.
fn session(dev: Device, ch: i32, op: String) -> ((Record, Record), Device) {
    let (a, dev2) = measure(dev, ch);
    let (b, dev3) = measure(dev2, ch);
    ((label(op.clone(), a), label(op, b)), dev3)
}
```

```
let ((r1, r2), dev) = session(Device { reads: 0 }, 3, String::from("ada"));
for r in [&r1, &r2] {
    println!(
        "{}: channel {} = {}",
        r.op, r.sample.channel, r.sample.value
    );
}
println!("device used {} times", dev.reads);
```

```
ada: channel 3 = 130
ada: channel 3 = 131
device used 2 times
```

Each variable gets the treatment its type calls for.

- `ch: i32` is `Copy`. It is used twice, once per measurement, and nothing is written to make that possible: contraction for free.
- `a` and `b` are `Sample`s, a struct that derives `Copy` because both its fields are `i32` (promotion). They are used once each here, but could be used more.
- `dev: Device` is affine, and the program *threads* it: each `measure` consumes the device and returns it, and the returned one is the only one we may use next. Chapter 25 builds protocols on this pattern.
- `op: String` is `Clone` but not `Copy`, so the program duplicates it explicitly, once, with `op.clone()`, paying for one allocation.

The same program in the crate's syntax, with two buggy variants, is `ch23_session.aff`:

```
#[derive(Clone, Copy)]
struct Sample { channel: i32, value: i32 }

fn measure(dev: Dev, ch: i32) -> (Sample, Dev);
fn label(op: String, s: Sample) -> Rec;

// Measure one channel twice; label both samples with the operator's name.
fn session(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev) {
    let (a, dev2) = measure(dev, ch);
    let (b, dev3) = measure(dev2, ch);
    ((label(op.clone(), a), label(op, b)), dev3)
}

// Forgot the clone.
fn session_unlabeled(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev) {
    let (a, dev2) = measure(dev, ch);
    let (b, dev3) = measure(dev2, ch);
    ((label(op, a), label(op, b)), dev3)
}

// Measured with the old device.
fn session_stale(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev) {
    let (a, dev2) = measure(dev, ch);
    let (b, dev3) = measure(dev, ch);
    ((label(op.clone(), a), label(op, b)), dev3)
}
```

The crate accepts `session`. Its derivation, printed with the contexts as variable names only so that it fits the page, shows each of the four treatments:

```
ok: fn session(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev)
[A-LetPair] dev, ch, op ⊢ let (a, dev2) = measure(dev, ch); … : ((Rec, Rec), Dev)
  [Call] dev, ch ⊢ measure(dev, ch) : (Sample, Dev)
    [A-Var] dev ⊢ dev : Dev
    [A-Var] ch ⊢ ch : i32
  [A-LetPair] ch, op, a, dev2 ⊢ let (b, dev3) = measure(dev2, ch); … : ((Rec, Rec), Dev)
    [Call] ch, dev2 ⊢ measure(dev2, ch) : (Sample, Dev)
      [A-Var] dev2 ⊢ dev2 : Dev
      [A-Var] ch ⊢ ch : i32
    [A-Pair] op, a, b, dev3 ⊢ (…, dev3) : ((Rec, Rec), Dev)
      [A-Pair] op, a, b ⊢ (label(op.clone(), a), label(op, b)) : (Rec, Rec)
        [Call] op, a ⊢ label(op.clone(), a) : Rec
          [Clone] op ⊢ op.clone() : String
          [A-Var] a ⊢ a : Sample
        [Call] op, b ⊢ label(op, b) : Rec
          [A-Var] op ⊢ op : String
          [A-Var] b ⊢ b : Sample
      [A-Var] dev3 ⊢ dev3 : Dev
```

`ch` appears in the contexts of *both* premises of each `measure` split: the first `measure` receives it, and so does the rest of the program. That is rule S-Both. `dev`, `dev2` and `dev3` each appear in exactly one leaf. The interesting line is the one for the pair of labels: `op` appears in both of its premises, although `String` is not `Copy`. The first premise uses `op` only through `op.clone()`, which in Rust takes `&self`: it reads `op` through a shared borrow and leaves it in place. The crate models that as a read that does not move, and the calculus, which has no borrows, would write it with the constant of the previous sections, `let` $(op_1, op_2) =$ ***dup*** op `in` …, using `op` exactly once.

Forgetting the clone is a contraction of a non-`Copy` variable, and the crate suggests the clone:

```
error[E0382]: use of moved value: `op`
  --> ch23_session.aff:18:27
   |
15 | fn session_unlabeled(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev) {
   |                                         -- move occurs because `op` has type `String`,
which does not implement the `Copy` trait
...
18 |     ((label(op, a), label(op, b)), dev3)
   |              --            ^^ value used here after move
   |              |
   |              value moved here
   |
   = help: consider cloning the value if the performance cost is acceptable
```

Measuring with the old device is the error that typestate (Chapter 25) relies on: the first `measure` consumed `dev`, and only `dev2` may be used now:

```
error[E0382]: use of moved value: `dev`
  --> ch23_session.aff:24:29
   |
22 | fn session_stale(dev: Dev, ch: i32, op: String) -> ((Rec, Rec), Dev) {
   |                  --- move occurs because `dev` has type `Dev`, which does not implement
the `Copy` trait
23 |     let (a, dev2) = measure(dev, ch);
   |                             --- value moved here
24 |     let (b, dev3) = measure(dev, ch);
   |                             ^^^ value used here after move
```

In the formal system, the program's context is `dev : Dev, ch : i32, op : String`, with multiplicities 1, ∞ and 1. The generalized usage theorem says that `dev` and `op` are used at most once in any derivation, and that `ch` may be used freely: exactly the three treatments above, with the clone as the explicit term that turns one `op` into two.

23.9 Common confusions

"**`Copy` is a cheap `Clone`.**" They are different kinds of thing. `Copy` is a property that makes duplication implicit, at every use, as a bitwise copy; it has no method. `Clone` is a method that must be called. Every `Copy` type is `Clone`, but a cheap `clone` does not make a type `Copy`: `Rc::clone` is cheap and `Rc` is not `Copy`, because its duplication must be counted.

"**Moving is a `memcpy` and copying is a `memcpy`, so they are the same.**" At the machine level they often are. The difference is in the types: after a move the source is no longer usable, and after a copy it is. That is the difference between a split and S-Both, and it is decided by the type.

"**`&mut T` is used twice in my code, so it must be `Copy`.**" The second use receives a reborrow, a new and shorter-lived unique reference, created implicitly at each call (Chapter 28). `let b = w;` after `let a = w;` is still E0382.

"**`Rc<T>` makes `T` duplicable.**" It makes the *pointer* duplicable, explicitly and with counting. The `T` is shared, not duplicated, and is reachable only through shared references.

"**`!A` is a type, so `Copy` should be a type constructor too.**" Linear logic has ! as a constructor; Rust, and the calculus of this chapter, use a predicate on types. The two presentations agree on what may be duplicated; the predicate is simpler to use (there is no need to write promotions and derelictions) at the price of fixing, once per type, whether it is duplicable.

"**A type with only `Copy` fields can always be `Copy`.**" Only if it has no destructor. A destructor makes discarding an action, and an action cannot be duplicated silently.

23.10 Summary

- The exponential $!\tau$ marks the types for which contraction and weakening are allowed. Its rules are !-Weak, !-Contr, Dereliction and Promotion; promotion says that a duplicable value may only be built from duplicable resources.
- `Copy` behaves as if a type were already of the form $!\tau$: contraction and weakening are silent, dereliction is invisible, and promotion is the rule for which types may be `Copy` (all fields `Copy`: E0204). Destructors are excluded (E0184), shared references are `Copy`, and mutable references are not.
- `Clone` is a different mechanism: an explicit duplication term $\tau \multimap \tau \otimes \tau$, visible in the source, with a cost and possibly an effect. Every `Copy` type is `Clone`, and Lean proves the type-theoretic version: for a `Copy` type, $\lambda x.\,(x, x)$ is such a term.
- `Rc` and `Arc` share a value instead of duplicating it, and count the duplications and discards of the pointer; the counting is why they cannot be `Copy`.

- Formally, λ^{aff} with `Copy` adds a predicate on types and one splitting rule, S-Both; the typing rules are unchanged. The generalized usage theorem, proved in Lean with multiplicities in `ℕ∞`, says that non-`Copy` variables are still used at most once.
- The usage theorem holds for any choice of duplicable types, but preservation does not: if function types were `Copy` regardless of their captures, a well-typed program could step to an ill-typed one. That is why promotion matters, and why Rust's closures are `Copy` only when their captures are.
- The `affine` crate implements the extension: `Copy` places are not moved, `.clone()` reads without moving, and `#[derive(Copy)]` is checked as rustc checks it.

23.11 Exercises

1. Using the four rules of !, derive $x : !A \vdash (\mathit{derelict}\, x, \mathit{derelict}\, x) : A \otimes A$ and $x : !A \vdash () : \mathbf{1}$. Then explain why $x : A \vdash !x : !A$ has no derivation. *Hint:* for the pair, apply !-Contr to a derivation in which two variables of type $!A$ go to the two components. For the last one, look at the context that Promotion requires.
2. Which of these types are `Copy` in Rust? `(i32, &String)`, `[&mut i32; 2]`, `Option<fn(i32) -> i32>`, the type of `move || n + 1` where `n: i32`, and the type of `move || s.len()` where `s: String`. Check your answers with a generic function `fn is_copy<T: Copy>(_: &T) {}`. *Hint:* apply the promotion rules component by component; for closures, look at what is captured, and how.
3. In Lean, prove that for every `Copy` type the discard map exists as well as the copy map: `HasTypeC isCopy [] (.lam τ .unit) (τ ⊸ .unit)`. Then prove that it exists for *every* type. Which structural rule makes the second statement true, and how would it fail in a linear calculus? *Hint:* `.lam .unit` needs no split at all; rule `unit` accepts any context.
4. In Lean, prove that `isCopy` is closed under the sum: `isCopy (τ ⊕ σ) = true ↔ isCopy τ = true ∧ isCopy σ = true`. Then add a type former for Rust's arrays to a copy of `Ty` and extend `isCopy` accordingly. *Hint:* the proof is the same as `isCopy_tensor`.
5. Give the `affine` crate a type `Arc<T>`, which behaves like `Rc<T>` for `Copy` and `Clone`, and a test showing that `(a.clone(), a)` is accepted and `(a, a)` rejected for `a: Arc<Config>`. *Hint:* add a variant to `Ty`, parse it like `Rc`, and add it to the same arms of `is_copy` and `is_clone` as `Rc`.
6. Write a Rust type whose `clone` panics on the second call, and use it to explain why the compiler may not insert calls to `clone` on its own, the way it inserts copies for `Copy` types. *Hint:* store a `Cell<bool>` (or an `Rc<Cell<u32>>`) in the value and check it in `clone`. An implicit duplication would run that code at places the programmer cannot see.

CHAPTER 24

Drop: Weakening with a Side Effect

In the affine λ-calculus of Chapter 22, throwing a value away costs nothing. The weakening rule lets a variable sit in the context unused, and when the program finishes, the variable simply vanishes: the calculus has no files, no locks and no memory to give back.

Rust is affine too, but in Rust, throwing a value away *runs code*. When a `MutexGuard` goes out of scope, a lock is released; when a `File` does, a file descriptor is closed; when a `Vec<String>` does, every string is freed, then the vector's buffer. The typing rule is the same as in the calculus (you may discard), but the operational rule is not: discarding is an *event*, at a precise point of the program, and programs can observe it.

This chapter is about that event: when Rust runs destructors and in which order, what the compiler generates for them, and what it forbids so that no resource is released twice. Then comes the other side of the coin. Rust guarantees that a destructor runs *at most once*, not that it runs *at least once*: leaking a value is safe. That decision, taken a few weeks before Rust 1.0, is the precise reason why Rust is affine and not linear, and it still shapes how safe APIs are designed. In Lean, which has no destructors, we look at reference counting and in-place updates, and then prove, for a small resource language with a log of releases, "at most once" and, in the absence of `forget`, "exactly once".

In this chapter you will learn:

- why discarding a value in Rust is "weakening with a side effect", and what RAII guards (locks, transactions, buffered writers) do with it;
- the rules of drop order, explained line by line on a running program: locals, fields, temporaries, moves, drop flags, assignment and `mem::drop`;
- what drop glue is, what `ManuallyDrop` does, and why you can neither call `Drop::drop` yourself (E0040) nor move a field out of a type with a destructor (E0509);
- why leaking is safe in Rust: `mem::forget`, `Box::leak`, `Rc` cycles and `Weak` counts, and the history of `thread::scoped` that settled the question;
- how to design APIs that stay sound when a destructor does not run (`Vec::drain`, `thread::scope`), and what linear types would add, with `#[must_use]` as a lint-level approximation;
- how Lean manages memory without destructors, and how reference counts let a pure language update values in place;

- how to model drops in Lean and prove that no resource is released twice, that without `forget` every resource is released exactly once, and that `forget` breaks the second property.

24.1 Discarding is not free

Recall the shape of weakening. In the logic of Chapter 6 it says that an unused assumption does no harm: if $\Gamma \vdash B$ then $\Gamma, A \vdash B$. In the affine λ-calculus of Chapter 22 it says that a variable may stay unused. To talk about the moment of discarding, it helps to make it explicit with a construct $\texttt{drop}\ x; e$ that throws x away and continues with e. Its typing rule is weakening in disguise:

$$\frac{\Gamma \vdash e : \rho}{\Gamma, x : \tau \vdash \texttt{drop}\ x; e : \rho}\ \text{T-Drop}$$

and in the pure calculus its computation rule does nothing at all:

$$\texttt{drop}\ x; e \to e$$

Rust keeps the typing rule: any value may be discarded, whatever its type (a choice with consequences, as we will see). What Rust changes is the computation rule. A Rust program runs in a world with resources: memory, file descriptors, locks, connections. Writing σ for the state of that world, the rule becomes

$$\langle \sigma, \texttt{drop}\ x; e \rangle \to \langle \mathit{release}_\tau(\sigma, x), e \rangle$$

where $\mathit{release}_\tau$ is the *destructor* of the type τ: the code that gives back whatever a value of type τ holds. Weakening is still allowed, but it has become an effect. That is the title of this chapter.

RAII

The programming pattern built on this rule has a name inherited from C++: **RAII**, "resource acquisition is initialization". The idea is simpler than the name. Tie each resource to a value: acquiring the resource creates the value, and the value's destructor releases the resource. The compiler then releases it on every path out of the scope: the normal end, an early `return`, a `?`, a `break`, even a panic.

The standard library is full of such values, usually called *guards*. Here is the most common one, a `MutexGuard`, from this chapter's example `code/rust/examples/ch24_raii.rs`:

```rust
fn lock_guard() {
    let balance = Mutex::new(100_u32);
    {
        let mut guard = balance.lock().unwrap(); // acquire
        *guard -= 30;
        println!("  inside: locked by us? {}", balance.try_lock().is_err());
    } // the MutexGuard is dropped here: release
    println!("  after the block: free? {}", balance.try_lock().is_ok());
    println!("  balance = {}", balance.lock().unwrap());
}
```

```
lock guard:
  inside: locked by us? true
  after the block: free? true
  balance = 70
```

Inside the block the mutex is locked, so `try_lock` fails (it returns an error instead of waiting, which is why this single-threaded program does not deadlock). At the closing brace the guard is discarded, its destructor unlocks the mutex, and `try_lock` succeeds. There is no `unlock` method to forget to call: the only way to unlock is to stop holding the guard.

You can write your own guards by implementing the `Drop` trait, whose single method `fn drop(&mut self)` is the destructor. A good example is a database transaction that rolls back unless it is explicitly committed:

```
/// A transaction on an in-memory table. Rows are staged in `pending` and
/// reach the table only on `commit`; a transaction that is dropped without
/// being committed rolls back.
#[must_use = "a transaction rolls back unless it is committed"]
struct Transaction<'db> {
    table: &'db mut Vec<String>,
    pending: Vec<String>,
    committed: bool,
}

impl<'db> Transaction<'db> {
    fn begin(table: &'db mut Vec<String>) -> Self {
        Transaction {
            table,
            pending: Vec::new(),
            committed: false,
        }
    }

    fn insert(&mut self, row: &str) {
        self.pending.push(row.to_string());
    }

    /// Consumes the transaction: after `commit` it cannot be used again.
    fn commit(mut self) {
        self.table.append(&mut self.pending);
        self.committed = true;
    } // `self` is dropped here, and `drop` sees `committed == true`
}

impl Drop for Transaction<'_> {
    fn drop(&mut self) {
        if !self.committed {
            println!(
                "  rollback: {} pending row(s) discarded",
                self.pending.len()
            );
        }
    }
}

fn transactions() {
    let mut orders = Vec::new();

    let mut tx = Transaction::begin(&mut orders);
    tx.insert("order 1: 3 x sensor");
    tx.insert("order 2: 1 x cable");
    tx.commit();

    let mut tx = Transaction::begin(&mut orders);
    tx.insert("order 3: 9 x probe");
    // An early return, a `?` or a panic here would skip the commit:
    // the destructor rolls back either way.
    drop(tx);

    println!("  table = {orders:?}");
}
```

```
transactions:
  rollback: 1 pending row(s) discarded
  table = ["order 1: 3 x sensor", "order 2: 1 x cable"]
```

The design has two exits. The *explicit* one is `commit`, which takes `self` by value: after `tx.commit()`, `tx` has been moved and cannot be used again (Chapter 22's affine typing at work). The *implicit* one is the destructor, which rolls back every transaction discarded without a commit. The first transaction is committed, so its destructor stays silent; the second is dropped and rolls back. As the comment in the code says, the rollback also happens if anything between `begin` and `commit` returns early or panics.

Notice two things the destructor *cannot* do. It cannot take arguments, so it cannot ask the caller whether to commit. And it cannot return a value, so it cannot report failure. Both limits are real, and the standard library's `BufWriter` shows the second one:

```
fn buffered_writer() {
    let mut file_contents: Vec<u8> = Vec::new();
    {
        let mut writer = BufWriter::new(&mut file_contents);
        writer.write_all(b"t,celsius\n0,21.5\n").unwrap();
        println!(
            "  buffered, not yet written: {} bytes",
            writer.buffer().len()
        );
    } // BufWriter's destructor flushes the buffer (and ignores any error)
    println!("  after the drop, written: {} bytes", file_contents.len());
}
```

```
buffered writer:
  buffered, not yet written: 17 bytes
  after the drop, written: 17 bytes
```

A `BufWriter` collects small writes in a buffer. While it is alive, the 17 bytes sit in the buffer; when it is dropped, its destructor flushes them. If flushing failed (a full disk, a closed socket), the destructor could not tell anyone, so it ignores the error. That is why `BufWriter` also has an explicit `flush` method that returns a `Result`, and why careful code calls it: the destructor is a safety net, not the main road.

The slogan

Put the typing rule and the computation rule side by side and you get a slogan that is worth remembering, with one caveat that the rest of the chapter explains:

> Rust is **affine in its typing and linear in its dynamics**. The type checker lets you discard any value, but the compiler inserts the discarding for you, so every value that is not moved away is released when its owner goes out of scope.

The caveat is in the words "every value". As we will see, safe Rust has ways to make a value disappear *without* its destructor running, and the language explicitly allows them.

24.2 The rules of drop order

Because destructors are observable, the *order* in which they run is part of the meaning of a program. In the pure calculus, the context was a multiset (or a list up to the exchange rule) and the order in which unused variables vanished did not matter. In Rust it matters, and the language fixes it. The example `code/rust/examples/ch24_drop_order.rs` makes the rules visible with a type whose destructor prints its name:

```
/// A value that announces its own destruction.
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}
```

Each function below exercises one rule; `main` calls them in order, printing a header before each. We will go through the output line by line.

Locals: reverse declaration order

```
fn locals() {
    let _a = Noisy("a");
    let _b = Noisy("b");
    let _c = Noisy("c");
    println!("  end of scope");
} // _c, _b, _a are dropped here, in this order
```

```
locals:
  end of scope
  drop c
  drop b
  drop a
```

The three variables are dropped at the closing brace, after the last statement of the function, and in the *reverse* of the order in which they were declared: `c`, `b`, `a`. The reason is that later variables may depend on earlier ones. If `b` were a guard borrowed from `a` (as `guard` was borrowed from `balance` above), dropping `a` first would leave `b` pointing at a dead value. Dropping in reverse order is a stack discipline: the last thing set up is the first thing torn down. (The leading underscore in `_a` only silences the "unused variable" warning. It does not change when the value is dropped. A bare `_` does, as we will see in a moment.)

Fields: declaration order, after the outer destructor

```
/// A struct with its own destructor *and* fields that have destructors.
struct Pair {
    first: Noisy,
    second: Noisy,
}

impl Drop for Pair {
    fn drop(&mut self) {
        // Runs first, while the fields are still intact.
        println!("  drop Pair({}, {})", self.first.0, self.second.0);
    }
```

```
}

fn fields() {
    let _pair = Pair {
        first: Noisy("first"),
        second: Noisy("second"),
    };
    let _tuple = (Noisy("t.0"), Noisy("t.1"));
    println!("  end of scope");
}
```

```
fields:
  end of scope
  drop t.0
  drop t.1
  drop Pair(first, second)
  drop first
  drop second
```

Two locals again, so `_tuple` goes first and `_pair` second (reverse declaration order). Inside each of them, the order is different:

- The tuple's elements are dropped *in order*: `t.0`, then `t.1`. The same holds for the fields of a struct, the elements of an array and the elements of a `Vec`.
- For `Pair`, which has its own destructor, the outer destructor runs *first*, while the fields are still intact (it prints both names), and only then are the fields dropped, in declaration order: `first`, then `second`.

So a compound value is destroyed from the outside in. This is what makes it safe for `Drop::drop` to use its fields: they are still alive when it runs.

Temporaries, and `let _`

A *temporary* is a value that the program creates without giving it a name, such as the `Noisy("temporary")` passed by reference below. Temporaries are dropped at the end of the *statement* that created them:

```
fn name_length(n: &Noisy) -> usize {
    n.0.len()
}

fn temporaries() {
    // The temporary lives until the end of the statement that created it.
    let len = name_length(&Noisy("temporary"));
    println!("  after the call, len = {len}");
    // `_` is not a variable: nothing owns the value, so it is dropped at once.
    let _ = Noisy("underscore");
    println!("  after `let _ =`");
    // `_kept` is a variable: the value lives until the end of the scope.
    let _kept = Noisy("kept");
    println!("  end of scope");
}
```

```
temporaries:
  drop temporary
  after the call, len = 9
  drop underscore
  after `let _ =`
```

```
  end of scope
  drop kept
```

The first line of output is `drop temporary`: the temporary lived just long enough for `name_length` to read it, and died at the semicolon. The next lines show a classic trap. In `let _ = Noisy("underscore")`, the pattern `_` is not a variable: it binds nothing, so nothing owns the value after the statement, and it is dropped at the semicolon. `_kept`, by contrast, is a variable like any other. If you write `let _ = mutex.lock().unwrap();` to hold a lock for the rest of a function, you have in fact released it immediately.

> **NOTE**
>
> The precise rules for temporaries have more cases than we can cover here (for example, `let r = &Noisy("x");` *extends* the temporary's life to the end of the block, because otherwise `r` would dangle), and the 2024 edition, which this book uses, changed two of them: temporaries in the tail expression of a block, and in the scrutinee of an `if let` (the expression after the `=`), are now dropped earlier than in previous editions. The Rust Reference's chapter on destructors is the authority.

Moves cancel drops, and drop flags

A value is dropped by its *owner*. When a value is moved, ownership moves with it, and so does the responsibility for dropping it:

```rust
/// Takes ownership: its parameter is dropped when `consume` returns.
fn consume(n: Noisy) {
    println!("  consume({})", n.0);
}

fn moves(send: bool) {
    let x = Noisy("x");
    let _y = Noisy("y");
    if send {
        consume(x); // x is moved: this scope must no longer drop it
    }
    println!("  end of scope (send = {send})");
} // _y is dropped; x is dropped only if it was not moved (a drop flag)
```

```
moves (send = true):
  consume(x)
  drop x
  end of scope (send = true)
  drop y
moves (send = false):
  end of scope (send = false)
  drop y
  drop x
```

With `send = true`, `x` is moved into `consume`, whose parameter `n` now owns the value and drops it when `consume` returns: `drop x` appears *before* the end of `moves`, and at the end only `_y` is dropped. Dropping `x` there too would release the same value twice, a classic double free. With `send = false`, nothing is moved, and both values are dropped at the end, in reverse order.

It is the *same* function: whether `x` must be dropped at the closing brace depends on a value known only at run time. The compiler handles this with a **drop flag**, a hidden boolean in the stack frame that records whether `x` still owns its value. The move clears it, and the drop at the end of the scope is guarded by it. When the compiler can tell statically that a variable is always moved, or never, it does without the flag. Chapter 29 shows where these drops live in the compiler's intermediate representation.

Assignment and `mem::drop`

```
fn early() {
    let mut slot = Noisy("old");
    println!("  slot holds {}", slot.0);
    slot = Noisy("new"); // the old value is dropped by the assignment
    println!("  slot holds {}", slot.0);
    let guard = Noisy("guard");
    drop(guard); // std::mem::drop: release now, not at the end of the scope
    println!("  end of scope");
}
```

```
early:
  slot holds old
  drop old
  slot holds new
  drop guard
  end of scope
  drop new
```

Assigning a new value to a variable that owns an old one drops the old value *at the assignment*: `drop old` appears between the two `slot holds` lines. (Assigning to a variable whose value has been moved out drops nothing; the drop flag again.) And `drop(guard)` releases `guard` immediately, so `drop guard` appears before `end of scope`, and only `new` is left for the closing brace.

`drop` here is `std::mem::drop`, which is in the prelude. It is worth looking at its definition, because it is the whole story of this chapter in one line:

```
pub fn drop<T>(_x: T) {}
```

The body is empty. `drop` takes ownership of its argument and does nothing with it, so the argument is dropped when the function returns. `mem::drop` is not a special operation: it is weakening, packaged as a function. Since it takes its argument by value, a later use is a use after move (`code/rust/compile_fail/ch24_use_after_drop.rs`):

```
pub fn close_then_use(conn: Connection) -> usize {
    drop(conn);
    conn.host.len()
}
```

```
error[E0382]: borrow of moved value: `conn`
  --> compile_fail/ch24_use_after_drop.rs:12:5
   |
10 | pub fn close_then_use(conn: Connection) -> usize {
   |                       ---- move occurs because `conn` has type `Connection`, which does
not implement the `Copy` trait
11 |     drop(conn);
   |          ---- value moved here
```

```
12 |     conn.host.len()
   |     ^^^^^^^^^ value borrowed here after move
   |
note: if `Connection` implemented `Clone`, you could clone the value
  --> compile_fail/ch24_use_after_drop.rs:5:1
   |
 5 | pub struct Connection {
   | ^^^^^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
11 |     drop(conn);
   |          ---- you could clone this value
```

Read the diagnostic with Chapter 22 in mind: `conn` is not `Copy`, it was consumed by the call `drop(conn)` ("value moved here", as for any other call), and the context no longer contains it.

Order breaks exchange, operationally

In Chapter 6 we saw that Rust keeps the exchange rule: the order of the assumptions in a context does not matter for typing. The `locals` example shows that it matters for the *behavior*: swap the declarations of `_a` and `_b` and the output changes. So exchange holds at the level of the typing judgment, which is where we stated it, but not at the level of observable behavior. This is a small but instructive example of why the claim "Rust is based on logic X" always needs to say *at which level*.

24.3 Drop glue, and two things you cannot do

Most types do not implement `Drop` themselves. A `(u64, String)` has no `drop` method, yet dropping it frees the string. The compiler generates, for every type, a function called the type's **drop glue**: first call `Drop::drop` if the type implements it, then run the drop glue of each field, in declaration order. That is exactly the "outside in" order we observed for `Pair`. A type whose drop glue does nothing at all is one that needs no dropping, and `std::mem::needs_drop::<T>()` tells you which is which:

```
fn drop_glue() {
    println!("  u64                 {}", mem::needs_drop::<u64>());
    println!("  &String             {}", mem::needs_drop::<&String>());
    println!("  String              {}", mem::needs_drop::<String>());
    println!(
        "  (u64, String)       {}",
        mem::needs_drop::<(u64, String)>()
    );
    println!(
        "  Option<Box<u8>>     {}",
        mem::needs_drop::<Option<Box<u8>>>()
    );
    println!("  Noisy               {}", mem::needs_drop::<Noisy>());
    println!(
        "  ManuallyDrop<String> {}",
        mem::needs_drop::<ManuallyDrop<String>>()
    );
}
```

```
needs_drop (does the type have drop glue?):
  u64                 false
  &String             false
  String              true
  (u64, String)       true
  Option<Box<u8>>     true
  Noisy               true
  ManuallyDrop<String> false
```

A `u64` owns nothing. A `&String` is a borrow: it owns nothing either, and dropping the reference leaves the string alone (borrowing is the subject of Part VI). A `String` owns a heap buffer. A tuple needs dropping if any of its fields does, and so does an `Option<Box<u8>>`, whose `Some` case owns a box. `Noisy` has a `Drop` impl. The last line we explain below. (The documentation allows `needs_drop` to answer `true` conservatively for a type that does not really need it; it never answers `false` for one that does.)

Chapter 23 explained why a type cannot be both `Copy` and `Drop` (E0184): a bitwise copy of a value with a destructor would run that destructor twice. The same concern, "never release a resource twice", explains two more rules.

You cannot call `Drop::drop`

```
pub fn release_early(lock: Lock) {
    lock.drop();
}
```

```
error[E0040]: explicit use of destructor method
  --> compile_fail/ch24_explicit_destructor.rs:15:10
   |
15 |     lock.drop();
   |          ^^^^ explicit destructor calls not allowed
   |
help: consider using `drop` function
   |
15 -     lock.drop();
15 +     drop(lock);
   |
```

The method `Drop::drop` takes `&mut self`, not `self`: after the call, `lock` would still be a valid variable that owns its value, and the compiler would drop it again at the end of the scope, running the destructor a second time. So the compiler forbids calling the method directly (E0040), and suggests the function `drop(lock)`, which moves the value and so cancels the automatic drop. The destructor is something that *happens to* a value, at most once; it is not something you call.

You cannot move a field out of a `Drop` type

```
pub fn steal_rows(tx: Transaction) -> Vec<String> {
    tx.pending
}
```

```
error[E0509]: cannot move out of type `Transaction`, which implements the `Drop` trait
  --> compile_fail/ch24_move_out_of_drop.rs:15:5
   |
15 |     tx.pending
   |     ^^^^^^^^^^
   |     |
   |     cannot move out of here
   |     move occurs because `tx.pending` has type `Vec<String>`, which does not implement
the `Copy` trait
   |
help: consider cloning the value if the performance cost is acceptable
   |
15 |     tx.pending.clone()
   |               ++++++++
```

For an ordinary struct, moving one field out is a *partial move* (Chapter 22): the struct becomes partially uninitialized, and at the end of the scope the compiler drops only the fields that are still there. For a struct with a destructor this is impossible, because `Drop::drop` receives `&mut self` and may read every field: it needs the whole value. So the compiler forbids it (E0509). The usual workarounds leave a valid value behind: `std::mem::take` swaps the field with its default (an empty `Vec`), `Option::take` swaps an `Option` field with `None`, and `std::mem::replace` swaps in a value you choose. The `Transaction` above sidesteps the problem in `commit`: it moves the rows with `self.table.append(&mut self.pending)`, which empties `pending` through a mutable reference instead of moving it out.

Destructors and panics

When a thread panics, Rust (by default) *unwinds* the stack: it leaves every active function, and on the way out it drops every live local, exactly as a `return` would.

```
fn unwinding() {
    // Replace the default panic message (printed on stderr) with nothing.
    std::panic::set_hook(Box::new(|_| {}));
    let result = std::panic::catch_unwind(|| {
        let _probe = Noisy("probe guard");
        panic!("sensor disconnected");
    });
    let _ = std::panic::take_hook(); // restore the default hook
    println!("  caught a panic: {}", result.is_err());
}
```

```
unwinding:
  drop probe guard
  caught a panic: true
```

`catch_unwind` runs the closure and catches the panic; on its way out of the closure, the panic dropped `_probe`. This is what makes guards reliable: a `MutexGuard` dropped during a panic releases its lock (and marks the mutex "poisoned", so that the next user knows the data may be half-updated). Two exceptions: a program built with `panic = "abort"` does not unwind, so no destructors run; and a destructor that panics *during* unwinding aborts the process, which is why destructors should not panic.

`ManuallyDrop`

Sometimes you want to switch the automatic drop off. `std::mem::ManuallyDrop<T>` is a wrapper whose drop glue does nothing (hence `false` in the table above):

```
fn manually_drop() {
    let wrapped = ManuallyDrop::new(Noisy("wrapped"));
    println!("  holding {}", wrapped.0);
    let unwrapped = ManuallyDrop::into_inner(ManuallyDrop::new(Noisy("unwrapped")));
    println!("  holding {}", unwrapped.0);
    println!("  end of scope");
} // `unwrapped` is dropped; `wrapped` is not (its destructor never runs)
```

```
ManuallyDrop:
  holding wrapped
  holding unwrapped
  end of scope
  drop unwrapped
```

Only `unwrapped` is dropped: `ManuallyDrop::into_inner` gave the value back, with its ordinary destructor, while `wrapped` was never unwrapped. `ManuallyDrop` is mostly used in `unsafe` code that needs exact control over when a field is destroyed (dropping it in place, `ManuallyDrop::drop`, is `unsafe`, because the compiler can no longer check that it happens at most once; Chapter 32 discusses such obligations). But creating a `ManuallyDrop` is safe: a first hint that in Rust, "the destructor will run" is not a promise.

24.4 Affine, not linear

In the lattice of Chapter 21, a *linear* logic has neither weakening nor contraction (every assumption is used exactly once), and an *affine* logic keeps weakening (at most once). For Rust this means:

Guarantee	Linear system	Rust
a value is consumed at most once	yes	yes (non-`Copy` types)
a value is consumed at least once	yes	no
its destructor runs	by construction	usually, not guaranteed
a protocol is run to completion	can be enforced	only checked at run time

This section is about the second and third lines: Rust lets you discard any value, even *without* running its destructor.

`mem::forget` and `Box::leak`

The function `std::mem::forget<T>(t: T)` takes ownership of a value and makes sure its destructor never runs. It is a safe function. So is `Box::leak`, which turns a `Box<T>` into a reference `&'static mut T` to a heap allocation that will never be freed. From `code/rust/examples/ch24_leaks_and_scopes.rs`:

```
fn forgetting() {
    let kept = Noisy("kept");
    let forgotten = Noisy("forgotten");
    mem::forget(forgotten); // safe: the destructor will never run
    let leaked: &'static mut Noisy = Box::leak(Box::new(Noisy("leaked")));
    leaked.0 = "leaked (renamed)"; // a `'static` reference to a never-freed box
    println!("  end of scope, still holding {}", kept.0);
}
```

```
mem::forget and Box::leak:
  end of scope, still holding kept
  drop kept
```

Three values were created and only one destructor ran. The leaked box lives on, and the reference to it is valid for the rest of the program (that is what `'static` means), which is occasionally useful for data created once at startup. Neither needed `unsafe`. Making `forget` safe may look strange; the surprising fact is that making it unsafe would not help, because safe Rust can leak values without it.

`Rc` cycles and `Weak` counts

An `Rc<T>` (Chapter 23) is a pointer with a reference count. `Rc::clone` copies the pointer and increments the count; dropping an `Rc` decrements it; when the count reaches zero, the value inside is

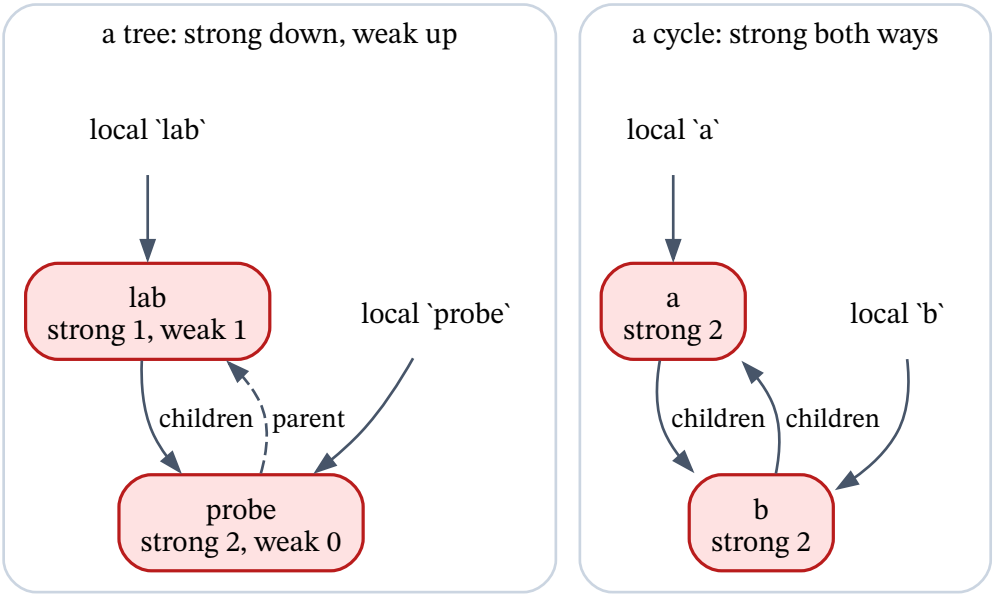
dropped. Combined with interior mutability (`RefCell` , Chapter 26), `Rc` lets you build arbitrary graphs. A tree in which children are owned by their parent, and point back to it, is the textbook example:

```
/// A node of a tree: children are owned (strong `Rc`), the parent pointer is
/// a `Weak`, which does not keep the parent alive.
struct Node {
    name: &'static str,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

impl Node {
    fn new(name: &'static str) -> Rc<Node> {
        Rc::new(Node {
            name,
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }
}

impl Drop for Node {
    fn drop(&mut self) {
        println!("  drop node {}", self.name);
    }
}

fn adopt(parent: &Rc<Node>, child: &Rc<Node>) {
    parent.children.borrow_mut().push(Rc::clone(child));
    *child.parent.borrow_mut() = Rc::downgrade(parent);
}

fn counts(label: &str, node: &Rc<Node>) {
    println!(
        "  {label}: strong = {}, weak = {}",
        Rc::strong_count(node),
        Rc::weak_count(node)
    );
}

fn tree() {
    let lab = Node::new("lab");
    let probe = Node::new("probe");
    adopt(&lab, &probe);
    counts("lab  ", &lab);
    counts("probe", &probe);
    let parent = probe.parent.borrow().upgrade().map(|p| p.name);
    println!("  probe's parent: {parent:?}");
    drop(lab); // the last strong reference to "lab": the whole tree goes
    let parent = probe.parent.borrow().upgrade().map(|p| p.name);
    println!("  probe's parent after drop(lab): {parent:?}");
    counts("probe", &probe);
}
```

```
a tree with Weak parent pointers:
  lab  : strong = 1, weak = 1
  probe: strong = 2, weak = 0
  probe's parent: Some("lab")
  drop node lab
  probe's parent after drop(lab): None
  probe: strong = 1, weak = 0
  drop node probe
```

`Rc<Node>` is a **strong** reference: it owns a share of the node and keeps it alive. `Weak<Node>` is a **weak** reference: it remembers where the node is but does not keep it alive, and must be *upgraded* to an `Rc` (which fails if the node is gone) before use. Read the counts with the left half of the figure below:

`lab` is held strongly by its local and weakly by the child's `parent` field; `probe` is held strongly by its local and by `lab`'s `children`. Then `drop(lab)` drops the only strong reference to `lab`: its destructor runs, the drop glue of its `children` vector releases one strong reference to `probe`, and the child's weak pointer no longer upgrades (`None`).



Two graphs of reference-counted nodes. Left: a tree with strong pointers from parent to child and a weak pointer back; the counts reach zero. Right: two nodes pointing strongly at each other; when the locals go away, each count stays at one and neither node is ever freed.

Now make *both* directions strong:

```
fn cycle() {
    let a = Node::new("a");
    let b = Node::new("b");
    a.children.borrow_mut().push(Rc::clone(&b)); // a -> b (strong)
    b.children.borrow_mut().push(Rc::clone(&a)); // b -> a (strong): a cycle
    counts("a", &a);
    counts("b", &b);
    println!("  end of scope");
} // a and b lose one strong reference each, but neither count reaches 0
```

```
a cycle of strong pointers:
  a: strong = 2, weak = 0
  b: strong = 2, weak = 0
  end of scope
```

Each node has a strong count of 2: its local and the other node's `children`. At the end of `cycle` the locals are dropped, each count goes down to 1, and there it stays: the program's complete output, which ends with `end of main`, never contains `drop node a` or `drop node b`. The two nodes keep each other alive, unreachable, until the process exits. Reference counting, unlike a tracing garbage collector, cannot see that a group of objects is unreachable from outside. The standard fix is the one used in the tree: make the back edges `Weak`.

The consequence for the type theory is direct. Any safe code can build such a cycle, so any safe code can put a value of *any* type `T` into one and lose it forever, without its destructor ever running. In other words, `forget` can be written in safe Rust (with an `Rc` cycle containing the value), and marking `mem::forget` as `unsafe` would only have been a polite fiction.

The Leakpocalypse

This is the moment the language committed to its position. In the spring of 2015, shortly before Rust 1.0, the standard library had a function `std::thread::scoped`. It spawned a thread allowed to *borrow* data from the parent's stack frame, and returned a guard, `JoinGuard`, whose destructor waited for (joined) the thread. The safety argument: the guard cannot outlive the borrowed data (the borrow checker ensures that), and dropping the guard joins the thread; so the thread finishes before the borrowed data goes away.

The argument has a hidden premise: *the guard's destructor runs*. Someone noticed that it need not. Put the guard into an `Rc` cycle and it is never dropped; the parent function returns, its stack frame is reused, and the child thread keeps reading and writing memory that now belongs to someone else: a use-after-free in a program with no `unsafe` in it.

The episode, nicknamed the *Leakpocalypse* in the community, was resolved in a few weeks, and the resolution has three parts that still hold:

1. `thread::scoped` was taken out of the stable standard library before Rust 1.0 (and later removed altogether).
2. `mem::forget`, which until then had been an `unsafe` function, was made safe, recording officially that leaking a value is not a memory-safety violation.
3. A rule for everyone who writes `unsafe` code: **soundness must not depend on a destructor running.** A destructor may be used for cleanup and for the comfortable path; it must never be the only thing that stops undefined behavior.

Leaks are still bugs, but bugs of the same class as infinite loops and deadlocks, which Rust's type system does not prevent either (Chapter 9 discussed what "going wrong" means), not of the class of use-after-free, which it does.

Designing for leaks: `Vec::drain` and `thread::scope`

How do you build an API whose safety needs some cleanup to happen, under this rule? There are two main techniques, and the standard library shows both.

The first is to make the half-finished state *safe*, if not useful, before handing out the guard. `Vec::drain(range)` returns an iterator that removes a range of elements; its destructor shifts the tail down to close the gap. While the iterator is alive, the vector has a hole in the middle, and if the iterator were forgotten with the vector left as it was, the vector would later drop the elements in the hole a second time. So `drain` shrinks the vector's length *first*:

```
fn forgotten_drain() {
    let mut readings = vec![10, 11, 12, 13, 14, 15];
    let drain = readings.drain(1..3); // would remove 11 and 12
```

```
    mem::forget(drain); // its destructor, which repairs the vector, never runs
    println!("  after forgetting the drain: {readings:?}");
}
```

```
a forgotten Vec::drain:
  after forgetting the drain: [10]
```

After forgetting the drain, the vector reports only the element before the range; the other five are leaked. The documentation warns that a forgotten `Drain` may lose elements (exactly which is an implementation detail), but the result is *safe*: nothing is dropped twice and nothing dangling is read. The worst case was turned from undefined behavior into lost data.

The second technique is to not hand out a guard at all, and keep control instead. The modern replacement for `thread::scoped` is `std::thread::scope`, stable since Rust 1.63:

```
fn scoped_threads() {
    let samples = vec![3, 1, 4, 1, 5, 9, 2, 6];
    let (left, right) = samples.split_at(samples.len() / 2);
    // Both threads borrow `samples` from this stack frame. `thread::scope`
    // joins them before it returns, so the borrow cannot outlive the data.
    let (sum_left, sum_right) = thread::scope(|s| {
        let h1 = s.spawn(|| left.iter().sum::<i32>());
        let h2 = s.spawn(|| right.iter().sum::<i32>());
        (h1.join().unwrap(), h2.join().unwrap())
    });
    println!(
        "  sums: {sum_left} + {sum_right} = {}",
        sum_left + sum_right
    );
    println!("  samples still owned here: {samples:?}");
}
```

```
thread::scope:
  sums: 9 + 22 = 31
  samples still owned here: [3, 1, 4, 1, 5, 9, 2, 6]
```

`thread::scope` passes its closure a scope handle `s`. Threads spawned with `s.spawn` may borrow anything that outlives the scope, here the two halves of `samples`. The crucial difference from the old design is *who* waits for the threads: not a destructor, but `thread::scope` itself, which joins every thread after the closure returns, and only then returns. A caller cannot forget a guard it never receives. The guarantee comes from control flow instead of a destructor, and this closure-passing style is the standard way to get "exactly once" behavior in an affine language; Chapter 25 uses it again.

24.5 What linearity would buy

Suppose Rust had genuinely linear types, whose values *must* be consumed by one of a designated set of operations (no weakening, in the terms of Chapter 21). What could we express that we cannot today?

- **Mandatory decisions.** A `Transaction` that must be either committed or rolled back explicitly, so that forgetting to decide is a compile-time error rather than a silent rollback.
- **Fallible cleanup.** A file that must be closed by a `close(self) -> Result<(), Error>` call, so that the error from the final flush cannot be ignored, as `BufWriter`'s destructor ignores it today.

- **Protocol completion.** A network session or a lab instrument (Chapter 25) whose protocol must be run to its final state, not abandoned half-way.

In each case the consuming operation, being an ordinary method that the programmer must call, can take arguments and return results, which a destructor cannot.

`#[must_use]`: a lint, not a rule

Rust has one feature that points in this direction. The attribute `#[must_use]` on a type (or a function) makes the compiler *warn* when a value of that type is produced and immediately thrown away. Our `Transaction` carries one:

```
#[must_use = "a transaction rolls back unless it is committed"]
struct Transaction<'db> { /* ... */ }
```

If you write the statement `Transaction::begin(&mut orders);` on its own, the compiler warns "unused `Transaction` that must be used", repeats the note from the attribute, and suggests "use `let _ = ...` to ignore the resulting value". That suggestion shows how weak the guarantee is. `#[must_use]` checks only that the value of an *expression statement* is not discarded on the spot; `let _ =`, an unused variable or a call to `drop` all satisfy it. It is a lint: not part of type checking, possible to turn off, and nothing `unsafe` code may rely on. It approximates "must be used" the way a unit test approximates a proof.

Why Rust is not linear, and the state of the discussion

Adding linear types to Rust after the fact is hard. Every generic function may drop a value of its type parameter (`fn store<T>(x: T) {}` compiles for every `T`), and the standard library is full of such functions; unwinding drops every live value; `mem::forget` and `Rc` cycles are safe. Linear types would need a new kind of bound meaning "may not be dropped implicitly" (or "may not be leaked"), and every generic API would have to be revisited to decide whether it supports such types.

Proposals along these lines have been discussed in the Rust community for years, under names such as "linear types", "must-move types" and a `Leak` trait. None is part of the language this book uses: the standard library of Rust 1.99 has no such marker trait. If the topic interests you, check the current state of the discussion. Other languages have taken the step: GHC Haskell, for example, has an extension that adds linear function types.

24.6 Lean: memory without destructors

Lean has no `Drop`: a Lean programmer never writes code that runs when a value dies. Memory is still managed precisely and deterministically, by **reference counting**: every heap object carries a count of the references to it and is freed when the count reaches zero, at that moment. This is Rust's `Rc`, applied by the runtime to every object. Values built by pure code cannot form cycles (an immutable value cannot point to something created after it), so for pure data reference counting frees everything; mutable references such as `IO.Ref` can be made to form cycles, and those are not freed, as in Rust.

Other resources are handled by the runtime too. The documentation of Lean's file handles, `IO.FS.Handle`, says that there is no explicit operation to close a file: when the last reference to a handle

is dropped, the file is closed automatically. So Lean has RAII for the resources its runtime provides, implemented in C, but a Lean program cannot attach its own code to the death of a value.

Functional, but in place

Chapter 2 previewed the most interesting consequence of reference counting. In pure Lean, `xs.set! 0 99` does not modify `xs`; it returns a new array that differs from `xs` in position 0. Done naively, that copies the whole array on every update. But the runtime knows the reference count of `xs`. If it is 1, nobody else can see `xs`, and nobody will notice if the update is done *in place*. Only a shared array has to be copied. This is often summed up as "functional but in place".

The function `dbgTraceIfShared msg a` lets us watch the runtime decide. It returns `a` unchanged, and as a side effect, if `a` is shared, prints the message `msg` prefixed by `shared RC`. The Lean file `code/lean/Book/Ch24/InPlace.lean` uses it on a function that overwrites the first element of an array:

```
/-- Overwrite the first element. `Array.set!` updates the array in place when
it is the only reference to it, and copies it otherwise; `dbgTraceIfShared`
reports the copying case. -/
def bump (xs : Array Nat) : Array Nat :=
  (dbgTraceIfShared "bump" xs).set! 0 99

/-- A fresh array whose contents depend on the argument `n`, so that Lean
cannot precompute it once and share it. -/
def fresh (n : Nat) : Array Nat := Array.replicate n 0
```

The reason for `fresh` is subtle and comes back below. First, an array with a single reference:

```
/-- The array from `fresh n` has exactly one reference, which `bump` consumes. -/
def bumpFresh (n : Nat) : Array Nat := bump (fresh n)

/-- info: #[99, 0, 0] -/
#guard_msgs in
#eval bumpFresh 3
```

The only output is the result: no `shared RC` message, so the array built by `fresh 3` was updated in place. Now keep a second reference to the array:

```
/-- Here `xs` is still needed after the call: it is shared, so `bump` copies. -/
def bumpAndKeep (n : Nat) : Array Nat × Array Nat :=
  let xs := fresh n
  (bump xs, xs)

/--
info: shared RC bump
---
info: (#[99, 0, 0], #[0, 0, 0])
-/
#guard_msgs in
#eval bumpAndKeep 3
```

Here `#guard_msgs` checks two messages, separated by `---`. The first, `shared RC bump`, is printed by the runtime while `bump` runs: `xs` is still needed for the second component of the pair, so its count is 2, and `set!` copies. The second, the result, shows why the copy was necessary: the pair holds the updated array *and* the original one, unchanged, as an immutable language demands.

The same reasoning applies to loops. A `for` loop in `Id.run do` that updates a `let mut` array is, underneath, a chain of functions each of which consumes the array and returns the next version. If no one else holds a reference, every update is in place:

```
/-- A loop that writes every slot. The array is never shared, so all `n`
updates happen in place: no message is printed. -/
def stamp (n : Nat) : Array Nat := Id.run do
  let mut xs := fresh n
  for i in [0:n] do
    xs := (dbgTraceIfShared "stamp" xs).set! i (i + 1)
  return xs

/-- info: #[1, 2, 3, 4, 5] -/
#guard_msgs in
#eval stamp 5

/-- The same loop, but every intermediate version is kept in a history. Each
update now finds the array shared (the history holds a reference), and copies. -/
def stampWithHistory (n : Nat) : List (Array Nat) := Id.run do
  let mut xs := fresh n
  let mut history := []
  for i in [0:n] do
    history := xs :: history
    xs := (dbgTraceIfShared "history" xs).set! i (i + 1)
  return (xs :: history).reverse

/--
info: shared RC history
shared RC history
shared RC history
---
info: [#[0, 0, 0], #[1, 0, 0], #[1, 2, 0], #[1, 2, 3]]
-/
#guard_msgs in
#eval stampWithHistory 3
```

`stamp 5` prints only its result: five updates, no copies. `stampWithHistory 3` keeps every intermediate version in a list. Now each update finds the array shared (the list holds a reference to it), and the runtime reports three copies, one per iteration. The result shows that the copies were needed: the history really contains four different arrays.

Finally, the reason for `fresh`:

```
/-- A closed term: computed once, stored, and shared by every use. -/
def calibration : Array Nat := #[5, 6, 7]

/--
info: shared RC bump
---
info: #[99, 6, 7]
-/
#guard_msgs in
#eval bump calibration
```

`calibration` is a *closed term*: a value that does not depend on any argument. Lean computes such values once and stores them, like a Rust `static`, so every use of `calibration` refers to the same stored array, which is therefore shared, and `bump` must copy it. That is why the earlier examples build their arrays inside a function, from an argument.

Compare the two languages. In Rust, the uniqueness of a `Vec` you own, or of a `&mut` borrow, is established *statically*, so in-place mutation is simply what the code says. In Lean, uniqueness is checked

dynamically, and the outcome affects only performance, never the result. Rust has the dynamic version too: `Rc::make_mut` returns a mutable reference to the value inside an `Rc`, cloning it first if the count is above one. That is Lean's strategy as a library function.

What the logic sees

The Lean counterpart of "the typing rule does not see the destructor" is this: to Lean's logic, `dbgTraceIfShared` is the identity function. Its definition in the core library is `a`; the message comes from a native implementation that the compiler substitutes at run time. So proofs cannot observe sharing:

```
/-- To the logic, `dbgTraceIfShared` is the identity function: the message is
a side channel that proofs cannot observe. -/
theorem dbgTraceIfShared_eq {α : Type} (msg : String) (a : α) :
    dbgTraceIfShared msg a = a := rfl

/-- Whether `bump` copies or not, its result is the same value. -/
theorem bump_size (xs : Array Nat) : (bump xs).size = xs.size := by
  simp [bump, dbgTraceIfShared_eq]

theorem bump_first (xs : Array Nat) (h : 0 < xs.size) :
    (bump xs)[0]? = some 99 := by
  simp [bump, dbgTraceIfShared_eq, h]

theorem bump_rest (xs : Array Nat) (i : Nat) (hi : 0 < i) :
    (bump xs)[i]? = xs[i]? := by
  simp [bump, dbgTraceIfShared_eq, Nat.ne_of_lt hi]

/-- The two results of `bumpAndKeep` are different values, as the semantics
of an immutable language demands: the copy is what makes this true. -/
theorem bumpAndKeep_spec (n : Nat) (h : 0 < n) :
    (bumpAndKeep n).1[0]? = some 99 ∧ (bumpAndKeep n).2[0]? = some 0 := by
  simp [bumpAndKeep, bump, fresh, dbgTraceIfShared_eq, h]
```

`dbgTraceIfShared_eq` is proved by `rfl`: both sides are equal by definition. With it, `simp` sees through `bump`, and the other theorems describe its result exactly: the size is unchanged, position 0 holds 99, every other position is unchanged (`Nat.ne_of_lt hi` turns `0 < i` into `0 ≠ i`, which `simp` needs), and the two components of `bumpAndKeep` differ. None of these statements mentions reference counts, and none could: the runtime's message is a side channel, as a Rust destructor is invisible to rule T-Drop.

24.7 A drop log in Lean

We now turn the slogan "affine in typing, linear in dynamics" into theorems. The plan is to define a tiny language in which variables own resources, an evaluator that records every acquisition and release in a log (the formal counterpart of the lines our `Noisy` values print), and then prove three properties of the log and one of a static checker. Everything is in `code/lean/Book/Ch24/DropLog.lean`.

The language

```
/-- Variables are names; resources are numbered in order of acquisition. -/
abbrev Var := String
abbrev Res := Nat

/-- The statements of the language. -/
inductive Stmt where
  | new (x : Var)          -- let x = Resource::new();
```

```
  | move (x y : Var)       -- let y = x;
  | drop (x : Var)         -- drop(x);
  | forget (x : Var)       -- mem::forget(x);
  | read (x : Var)         -- x.read();  (uses x without consuming it)
  deriving Repr, DecidableEq

/-- What the log records. -/
inductive Event where
  | acquire (r : Res)
  | release (r : Res)
  deriving DecidableEq
```

Each statement is a miniature Rust statement, as the comments show: `drop` and `forget` consume a variable, with and without running the destructor, and `read` uses one without consuming it. (Resources are numbered `0, 1, 2, ...`; a `Repr` instance in the file prints them as `r0, r1, ...`.)

The state of a running program is the list of declared variables, the next fresh resource number, and the log:

```
/-- A slot is a declared variable with its *drop flag*: `some r` if the
variable owns resource `r`, `none` if its value has been moved out. -/
abbrev Slot := Var × Option Res

structure State where
  slots : List Slot      -- every declared variable, newest first
  next  : Res            -- the next fresh resource
  log   : List Event     -- what has happened, oldest first

def State.init : State := ⟨[], 0, []⟩
```

The `Option Res` in a slot is exactly Rust's drop flag: `some r` means "owns `r`, drop it at the end of the scope"; `none` means "moved out". A variable stays in the list after a move, as it stays in scope in Rust. New variables go at the front, so the list is in reverse declaration order, and a variable that shadows an older one with the same name is found first, as in Rust.

The heart of the evaluator is `take`, which moves a value out of a variable:

```
/-- Take the value out of the newest slot named `x`, clearing its drop flag.
This is the run-time version of "use of moved value". -/
def take (x : Var) : List Slot → Except String (Res × List Slot)
  | [] => .error s!"cannot find value `{x}` in this scope"
  | (y, o) :: rest =>
    if y = x then
      match o with
      | some r => .ok (r, (y, none) :: rest)
      | none => .error s!"use of moved value: `{x}`"
    else
      match take x rest with
      | .ok (r, rest') => .ok (r, (y, o) :: rest')
      | .error e => .error e
```

It walks the list to the newest slot named `x`. If that slot owns a resource, it returns the resource and the list with the slot's flag cleared; if the flag is already clear, it fails with Rust's message for E0382. `Except String α` is Lean's counterpart of `Result<α, String>` (Chapter 2). Every statement except `new` uses `take`:

```
/-- One statement. Only `new` and `drop` write to the log. -/
def step (s : State) : Stmt → Except String State
  | .new x =>
```

```
    .ok ⟨(x, some s.next) :: s.slots, s.next + 1, s.log ++ [.acquire s.next]⟩
  | .move x y =>
    match take x s.slots with
    | .ok (r, slots) => .ok ⟨(y, some r) :: slots, s.next, s.log⟩
    | .error e => .error e
  | .drop x =>
    match take x s.slots with
    | .ok (r, slots) => .ok ⟨slots, s.next, s.log ++ [.release r]⟩
    | .error e => .error e
  | .forget x =>
    match take x s.slots with
    | .ok (_, slots) => .ok ⟨slots, s.next, s.log⟩
    | .error e => .error e
  | .read x =>
    match take x s.slots with
    | .ok _ => .ok s
    | .error e => .error e
```

The anonymous constructor `⟨slots, next, log⟩` builds a `State`. `new` pushes a slot and logs an acquisition. `move` gives the value to a new slot `y` and logs nothing, because a move runs no code. `drop` logs a release. `forget` takes the value and logs *nothing*: no one owns the resource any more, and no one released it. `read` only checks that `x` still owns its value. Finally, the end of the scope:

```
/-- The end of the scope: release every resource that is still owned, newest
slot first (reverse declaration order). Moved-out slots are skipped: that is
what the drop flag is for. -/
def dropAll : List Slot → List Event → List Event
  | [], log => log
  | (_, some r) :: rest, log => dropAll rest (log ++ [.release r])
  | (_, none) :: rest, log => dropAll rest log

def exec (s : State) : List Stmt → Except String State
  | [] => .ok s
  | st :: prog =>
    match step s st with
    | .ok s' => exec s' prog
    | .error e => .error e

/-- Run a program in a fresh scope and return the complete log. -/
def run (prog : List Stmt) : Except String (List Event) :=
  match exec State.init prog with
  | .ok s => .ok (dropAll s.slots s.log)
  | .error e => .error e
```

`dropAll` releases, newest first, every resource whose flag is set. `exec` runs the statements, stopping at the first error, and `run` runs a program in a fresh scope and then drops what is left:

```
/-- The `moves` function of `ch24_drop_order.rs`, with `send = true`:
`x` is moved into `consume`, which drops it; `y` is dropped at the end. -/
def movesProgram : List Stmt :=
  [.new "x", .new "y", .move "x" "n", .drop "n"]

/-- info: Except.ok [acquire r0, acquire r1, release r0, release r1] -/
#guard_msgs in
#eval run movesProgram

/-- Without the move, both are dropped at the end, newest first. -/
def localsProgram : List Stmt := [.new "a", .new "b", .new "c"]

/--
info: Except.ok [acquire r0, acquire r1, acquire r2, release r2, release r1, release r0]
-/
#guard_msgs in
#eval run localsProgram
```

```
/-- info: Except.error "use of moved value: `conn`" -/
#guard_msgs in
#eval run [.new "conn", .drop "conn", .read "conn"]

/-- info: Except.ok [acquire r0] -/
#guard_msgs in
#eval run [.new "conn", .forget "conn"]
```

Compare with the Rust output. `movesProgram` is `moves` with `send = true`: `x` (`r0`) is moved into `n` and dropped there, before the end of the scope drops `y` (`r1`), the order the Rust program printed. `localsProgram` is `locals`: three releases in reverse order. The third program fails with the message of E0382. The fourth forgets its resource: an acquisition with no release, a leak.

Counting

The properties we want are about how many times each resource appears in the log. We count three things:

```
/-- How many slots own resource `r` right now. -/
def held : List Slot → Res → Nat
  | [], _ => 0
  | (_, o) :: rest, r => (if o = some r then 1 else 0) + held rest r

/-- How many times the log says `r` was acquired, and released. -/
def acquired (log : List Event) (r : Res) : Nat := log.count (.acquire r)
def released (log : List Event) (r : Res) : Nat := log.count (.release r)
```

`held slots r` counts the variables that currently own `r`; `acquired` and `released` use the library's `List.count`. (The file also proves small `@[simp]` lemmas such as `released (log ++ [.release r']) r = released log r + (if r' = r then 1 else 0)`, which `simp` then uses automatically.) The one lemma with real content says that `take` removes exactly one owner of the taken resource and changes no other count:

```
/-- Taking a value out of the slots removes exactly one owner of `r` (if the
taken resource is `r`) and leaves every other count unchanged. -/
theorem take_held {x : Var} {slots slots' : List Slot} {r : Res}
    (h : take x slots = .ok (r, slots')) (q : Res) :
    held slots q = held slots' q + (if r = q then 1 else 0) := by
  induction slots generalizing slots' with
  | nil =>
    -- `take x []` is an error, so `h` is impossible.
    simp [take] at h
  | cons slot rest ih =>
    obtain ⟨y, o⟩ := slot
    by_cases hyx : y = x
    · -- The newest slot is `x`: it must own a resource, which is taken.
      cases o with
      | none => simp [take, hyx] at h
      | some r₀ =>
        simp only [take, hyx, ↓reduceIte, Except.ok.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        simp only [held_cons, Option.some.injEq, reduceCtorEq, ↓reduceIte]
        omega
    · -- Another variable: the value is taken from the rest of the list.
      simp only [take, hyx, ↓reduceIte] at h
      split at h
      · rename_i r₁ rest' htake
        simp only [Except.ok.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        have := ih htake
        simp only [held_cons]
```

```
        omega
      · simp at h
```

For every resource `q`, the old number of owners equals the new number plus one if `q` is the taken resource. The proof is by induction on the list of slots, following the definition of `take`:

- `induction slots generalizing slots'` makes the induction hypothesis `ih` hold for *every* `slots'`, which we need because the recursive call produces a different list.
- In the `nil` case, `take x []` is an error, so `h : take x [] = .ok ...` is false: `simp [take] at h` computes the left side and closes the goal.
- In the `cons` case, `obtain ⟨y, o⟩ := slot` splits the first slot into its name and flag, and `by_cases hyx : y = x` asks whether it is the slot we want.
- If it is, `cases o` splits on the flag. With `none`, `take` fails and `h` is again impossible. With `some r₀`, `simp only [...] at h` computes `take` and turns `h` into `r₀ = r` and `(y, none) :: rest = slots'`, which `obtain ⟨rfl, rfl⟩ := h` substitutes everywhere. `simp only` then rewrites `some r₀ = some q` to `r₀ = q` and the cleared slot's contribution to zero, and `omega` finishes. (`omega` decides linear arithmetic over `Nat` and `Int`, and treats any other term, such as an `if`, as an unknown number.)
- If it is not, `take` recurses. `split at h` considers the two results of the recursive call; in the successful one, `rename_i` names the pieces, `ih htake` gives the equation for `rest`, and `omega` adds the first slot's unchanged contribution to both sides.

At most once

To prove that no resource is ever released twice, we find an *invariant*: a property of states that holds at the start and that every statement preserves.

```
/-- The invariant of every reachable state. -/
structure Inv (s : State) : Prop where
  /-- Owners plus releases never exceed acquisitions. -/
  conserve : ∀ r, held s.slots r + released s.log r ≤ acquired s.log r
  /-- Each resource is acquired at most once. -/
  once : ∀ r, acquired s.log r ≤ 1
  /-- Resources from `next` on have not been acquired yet. -/
  fresh : ∀ r, s.next ≤ r → acquired s.log r = 0
```

The first field is a conservation law: owners plus releases never exceed acquisitions. The second says that each resource is acquired at most once, the third that resources from `next` on have not been acquired yet (which makes `new` safe). A `structure` in `Prop` bundles several proofs: `Inv s` holds when all three fields do.

Here is the preservation proof, one case per statement:

```
theorem step_inv {s s' : State} {st : Stmt} (hs : Inv s)
    (h : step s st = .ok s') : Inv s' := by
  obtain ⟨hcons, honce, hfresh⟩ := hs
  cases st with
  | new x =>
    -- A fresh resource `s.next` gets one owner and one acquisition.
    simp only [step, Except.ok.injEq] at h
    subst h
    -- Before the step, `s.next` had no owner and no release.
    have h0 := hfresh s.next (Nat.le_refl _)
```

```
    have hc0 := hcons s.next
    refine ⟨fun r => ?_, fun r => ?_, fun r hr => ?_⟩
    · have := hcons r
      by_cases hr : s.next = r
      · subst hr; simp; omega
      · simp [hr]; omega
    · have := honce r
      by_cases hr : s.next = r
      · subst hr; simp; omega
      · simp [hr]; omega
    · -- `r` is at least `s.next + 1`: it is neither `s.next` nor acquired.
      have hlt : s.next < r := hr
      simp [Nat.ne_of_lt hlt, hfresh r (Nat.le_of_lt hlt)]
  | move x y =>
    -- The resource changes owner: every count is unchanged.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, honce, hfresh⟩
      have h1 := take_held htake q
      have h2 := hcons q
      simp only [held_cons, Option.some.injEq]
      omega
    · simp at h
  | drop x =>
    -- One owner fewer, one release more.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, fun q => ?_, fun q hq => ?_⟩
      · have h1 := take_held htake q
        have h2 := hcons q
        simp only [released_release, acquired_release]
        omega
      · simpa using honce q
      · simpa using hfresh q hq
    · simp at h
  | forget x =>
    -- One owner fewer, and nothing in the log.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, honce, hfresh⟩
      have h1 := take_held htake q
      have h2 := hcons q
      dsimp only
      omega
    · simp at h
  | read x =>
    simp only [step] at h
    split at h
    · simp only [Except.ok.injEq] at h
      subst h
      exact ⟨hcons, honce, hfresh⟩
    · simp at h
```

Each case starts the same way: `simp only [step] at h` unfolds the evaluator in `h : step s st = .ok s'`, `split at h` separates the success and failure of `take`, and `subst h` replaces `s'` by the state the step built. Failure cases end with `simp at h`, since `.error e = .ok s'` is impossible. Then:

- **`new`**: the fresh resource `s.next` gains one owner and one acquisition. `by_cases hr : s.next = r` separates it from the others. It had not been acquired (`h0`), hence (by `hc0`) had no owners or releases, and its new counts are 1, 0 and 1; the other counts do not change. `simp` applies the counting lemmas and `omega` does the arithmetic. For the third field, `hr` is `s.next < r` once the structure is unfolded, which is all `have hlt` says.
- **`move`**: by `take_held`, the old slots had one more owner of `r` than the remaining ones, and the new slot `(y, some r)` adds it back: nothing changes.
- **`drop`**: one owner fewer (`take_held`), one release more (`released_release`); the sum is unchanged.
- **`forget`**: one owner fewer and no release. The sum *decreases*, and `≤` survives a decrease. This is the only place where `forget` matters, and it already shows that an equality would not survive.
- **`read`**: nothing changes.

Induction over the statements gives the same for whole programs (`exec_inv` in the file), and the theorem follows:

```
/-- **At most once.** For every program, whatever it does, no resource
appears twice as released in the final log. -/
theorem run_at_most_once {prog : List Stmt} {log : List Event}
    (h : run prog = .ok log) (r : Res) : log.count (.release r) ≤ 1 := by
  simp only [run] at h
  split at h
  · rename_i s hexec
    simp only [Except.ok.injEq] at h
    subst h
    have hs := exec_inv inv_init hexec
    have h1 := released_dropAll s.slots s.log r
    have h2 := hs.conserve r
    have h3 := hs.once r
    simp only [released] at h1 h2
    omega
  · simp at h
```

After unfolding `run`, the final log is `dropAll s.slots s.log`, where `s` is the final state. Lemma `released_dropAll` (proved in the file by induction on the slots) says that the end of the scope adds one release per owner. So the final releases are the earlier releases plus the owners, which by the invariant is at most the acquisitions, which is at most 1; `omega` chains the facts. The theorem holds for *every* program, with `forget` or without: a resource can only be released by taking it out of a slot, and taking it clears the slot.

Exactly once, and how `forget` breaks it

For "exactly once" we need the equality version of the conservation law, and we need to exclude `forget`:

```
/-- A state is *balanced* when owners plus releases equal acquisitions:
nothing has been lost. -/
def Balanced (s : State) : Prop :=
  ∀ r, held s.slots r + released s.log r = acquired s.log r

def NoForget : Stmt → Prop
  | .forget _ => False
  | _ => True
```

`NoForget` is a predicate on statements defined by pattern matching: it is the proposition `False` for `forget` and `True` for everything else. The proof that `Balanced` is preserved by every statement other than `forget` (`step_balanced` in the file) is the same case analysis as before with `=` in place of `≤`; in the `forget` case the hypothesis `hf : NoForget (.forget x)` is a proof of `False`, and `absurd` concludes anything from it. Then:

```
/-- **Exactly once.** Without `forget`, every resource that was acquired is
released exactly once by the end of the scope. -/
theorem run_exactly_once {prog : List Stmt} {log : List Event}
    (hf : ∀ st ∈ prog, NoForget st) (h : run prog = .ok log)
    (r : Res) (hr : Event.acquire r ∈ log) : log.count (.release r) = 1 := by
  simp only [run] at h
  split at h
  · rename_i s hexec
    simp only [Except.ok.injEq] at h
    subst h
    have hs := exec_inv inv_init hexec
    have hb := exec_balanced (fun r => by
      simp [State.init, held, released, acquired]) hf hexec
    -- The final log has the same acquisitions, and one release per owner.
    have h1 := released_dropAll s.slots s.log r
    have h2 := acquired_dropAll s.slots s.log r
    have h3 := hb r
    have h4 := hs.once r
    have h5 : 0 < acquired (dropAll s.slots s.log) r := List.count_pos_iff.mpr hr
    simp only [released, acquired] at h1 h2 h3 h4 h5
    omega
  · simp at h
```

At the end of a program without `forget`, owners plus releases equal acquisitions (`hb`); the end of the scope turns every owner into a release (`h1`) and acquires nothing (`h2`), so in the final log releases equal acquisitions. Acquisitions are at most 1 (`h4`) and at least 1 (`h5`: `List.count_pos_iff` turns "`acquire r` is in the log" into "its count is positive"). So releases are exactly 1.

Finally, the hypothesis `hf` cannot be dropped:

```
/-- **`forget` breaks "exactly once".** The statement of `run_exactly_once`
without its hypothesis is false: one program is a counterexample. -/
theorem forget_breaks_exactly_once :
    ¬ ∀ (prog : List Stmt) (log : List Event), run prog = .ok log →
        ∀ r, Event.acquire r ∈ log → log.count (.release r) = 1 := by
  intro claim
  -- the program `let conn = ...; mem::forget(conn);`
  have h := claim [.new "conn", .forget "conn"] [.acquire 0] rfl 0 (by simp)
  -- its log has no `release`: the count is 0, not 1
  simp at h
```

This proves a negation: we assume the general claim and derive a contradiction. We apply it to the program that creates a resource and forgets it, whose log is `[.acquire 0]`; the argument `rfl` proves `run [...] = .ok [.acquire 0]` by running the program. The claim then says that `release 0` occurs exactly once in `[.acquire 0]`; `simp at h` computes the count, finds `0 = 1`, and closes the goal.

Affine in typing: a move checker

The evaluator detects use-after-move at run time, by failing. Rust programs carry no ownership information at run time: the type checker guarantees the failure cannot happen. The last part of the file makes that precise with a static checker that sees only names and flags, never resources:

```
/-- The checker's view of a slot: the name and whether it is initialized. -/
abbrev Flag := Var × Bool

def takeFlag (x : Var) : List Flag → Option (List Flag)
  | [] => none
  | (y, b) :: rest =>
    if y = x then (if b then some ((y, false) :: rest) else none)
    else (takeFlag x rest).map ((y, b) :: ·)

def checkStmt (flags : List Flag) : Stmt → Option (List Flag)
  | .new x => some ((x, true) :: flags)
  | .move x y => (takeFlag x flags).map ((y, true) :: ·)
  | .drop x => takeFlag x flags
  | .forget x => takeFlag x flags
  | .read x => (takeFlag x flags).map (fun _ => flags)

/-- Accept a program if no statement uses a moved (or undeclared) variable. -/
def check (flags : List Flag) : List Stmt → Bool
  | [] => true
  | st :: prog =>
    match checkStmt flags st with
    | some flags' => check flags' prog
    | none => false

/-- info: false -/
#guard_msgs in
#eval check [] [.new "conn", .drop "conn", .read "conn"]
```

`takeFlag` is `take` with a `Bool` instead of an `Option Res`. This is the affine discipline of Chapter 22 in miniature: a variable may be consumed at most once, and not used afterwards. Its soundness theorem:

```
/-- **Well-checked programs do not go wrong**: if the move checker accepts a
program, running it never reports a use of a moved value. -/
theorem check_sound {prog : List Stmt} (h : check [] prog = true) :
    ∃ log, run prog = .ok log := by
  obtain ⟨s, hs⟩ := exec_of_check (s := State.init) h
  exact ⟨dropAll s.slots s.log, by simp [run, hs]⟩
```

The proof (in the file) relates the two views with a function `erase` that keeps only the flags, and shows by induction that whenever the checker can take a flag, the evaluator can take the resource, and the erased states stay equal. Together with the previous theorems, the picture is complete:

- **typing** (`check`) is affine: it allows weakening (a variable may be left alone) and forbids contraction (no use after a move);
- **dynamics** (`run`) is linear without `forget`: every acquired resource is released exactly once, because the end of the scope inserts the releases the program did not perform;
- `forget` is the one operation that is well typed and breaks linearity of the dynamics, while keeping "at most once".

That is the slogan of this chapter, now as theorems.

24.8 Putting it together: a transaction, from Rust to the log

Let us follow the `Transaction` of `ch24_raii.rs` through the whole chapter. `Transaction::begin` acquires a resource (in a real database, a transaction on the server); in the log model, that is `new tx`. The type is not `Copy` and has a destructor, so it will be released exactly once, unless it is forgotten.

There are two ways to consume it. `tx.commit()` takes `self`: in the model, `move tx c`, and the end of `commit`'s scope drops `c`. Inside `commit`, the rows reach the table through `append`, because moving `pending` out of a `Drop` type is forbidden (E0509); then `committed` is set, and the destructor, which still runs at the end of `commit`, stays silent. The other way is to drop it, on any path out of the scope, and the destructor rolls back. Either way the transaction is released exactly once, as `run_exactly_once` says for programs without `forget`.

What about `mem::forget(tx)`? Nothing is committed, the destructor does not run: the resource leaks, as in `forget_breaks_exactly_once`. The design is still *sound*, because the table is modified only inside `commit`: a forgotten transaction loses its pending rows, which is what a rollback would have done. That is the rule of the Leakpocalypse applied. Had we written the rows immediately and relied on the destructor to *remove* them on rollback, a forgotten transaction would leave half-finished data behind.

A linear `Transaction` would go further: the compiler would reject a function that neither commits nor rolls back, and `rollback` could be an explicit method returning a `Result`. Today, `#[must_use]` warns about the most common slip, and the destructor catches the rest at run time.

24.9 Common confusions

"**A value is dropped when its variable goes out of scope.**" Only if the variable still owns it. A value moved into a function is dropped when that function's parameter goes out of scope; a value moved into a collection is dropped with the collection; a value whose variable was moved from is not dropped at that variable's scope end at all. Ownership, not scope, decides.

"**`let _ = x;` drops `x`.**" It does not even move it. The pattern `_` binds nothing, so when the right-hand side is a variable, `let _ = x;` does nothing at all, and `x` lives on. When the right-hand side is a temporary, such as `let _ = make_guard();`, the temporary is dropped at the end of the statement, which is usually not what the author wanted. Use `drop(x)` to drop, and a named variable such as `_guard` to keep.

"**Leaking memory is unsafe.**" It is safe, and sometimes intended (`Box::leak`). It is usually a bug, but like an infinite loop, not like a use-after-free; unsafe code must stay sound even when values are leaked.

"**`Rc` cleans up cycles eventually.**" It never does. Reference counting frees a value when its count reaches zero, and a cycle keeps every count in it above zero. Use `Weak` for back edges.

24.10 Summary

- Rust keeps the typing rule of weakening but gives discarding an effect, the destructor: "affine in typing, linear in dynamics".
- RAII ties resources to values, released on every path out of a scope, including panics. Destructors cannot take arguments or report errors, so good APIs also offer an explicit consuming method.
- Drop order is fixed: locals in reverse declaration order; fields and elements in declaration order, after the outer `Drop::drop`; temporaries at the end of their statement; `let _ =` binds nothing.

Moves transfer the duty to drop, and drop flags handle conditional moves. Assignment drops the old value; `mem::drop` is weakening as a function.

- Drop glue runs `Drop::drop` and then the fields' glue. To keep "at most once", the compiler forbids calling `Drop::drop` (E0040), moving fields out of a `Drop` type (E0509) and `Copy` with `Drop` (E0184). `ManuallyDrop` turns the glue off.
- Rust is affine, not linear: `mem::forget` and `Box::leak` are safe, and `Rc` cycles leak even without them. Since 2015, soundness must not depend on a destructor running; `Vec::drain` and `thread::scope` show how to design for it.
- Linear types would allow mandatory consumption, fallible cleanup and protocol completion. `#[must_use]` is only a lint; Rust 1.99 has no way to declare a type that must be consumed.
- Lean has no destructors; reference counting frees memory deterministically and lets pure code update unshared values in place, invisibly to the logic.
- For a small resource language with a drop log we proved in Lean: no resource is released twice; without `forget`, every resource is released exactly once; `forget` breaks the second property; an affine move checker rules out use-after-move.

24.11 Exercises

1. Predict the output of the following function, then check your answer by adding it to a copy of `ch24_drop_order.rs`:

```
fn puzzle() {
    let a = Noisy("a");
    let v = vec![Noisy("v0"), Noisy("v1")];
    let a = Noisy("a2");
    let _ = Noisy("tmp");
    let b = (Noisy("b0"), a);
    println!("  end of scope");
}
```

   Hint: shadowing does not drop the shadowed value; the first `a` is still alive (and still owned by its now unnameable variable). The second `a` is moved into the tuple. Locals go in reverse order, elements in order.

2. In `ch24_leaks_and_scopes.rs`, change the cycle so that `b` points back to `a` with a `Weak` instead of an `Rc`. Predict the counts printed by `counts` and the destructor messages, then run it. *Hint:* `a` then has one strong and one weak reference; when the locals go, `a`'s strong count reaches zero first, and its `children` vector releases `b`.

3. A `Pool` hands out a buffer with `fn borrow_buffer(&self) -> Guard<'_>`, and the guard's destructor marks the buffer free again. Unsafe code elsewhere reuses the buffer as soon as it is marked free. Is the API sound if a guard is forgotten? Would it be if the guard's destructor were what *stopped* other code from writing into the buffer? Redesign the second version in the style of `thread::scope`. *Hint:* a forgotten guard that keeps the buffer marked busy is only a leak. A forgotten guard that never revokes someone's access is the `thread::scoped` bug. A closure-based `with_buffer(&self, f: impl FnOnce(&mut [u8]))` does the cleanup itself after `f` returns.

4. In `DropLog.lean`, add a statement `swap (x y : Var)` that exchanges the resources owned by two variables (failing if either is moved out). Extend `step`, `checkStmt` and the proofs. Does `step_inv` need a new idea? *Hint:* implement it with two `take`s and two new slots; then each count of owners is unchanged, as for `move`, and the `move` case of the proofs is a good template.
5. Prove in `DropLog.lean` that the checker rejects `[.new "a", .drop "a", .drop "a"]` and that running it fails, both by `rfl`. Then suppose `move` forgot to clear the drop flag of `x`. Find a program whose log releases a resource twice, and say which field of `Inv` the `move` case of `step_inv` could no longer preserve. *Hint:* two statements suffice, and the end of the scope does the rest. Count the owners of `r0` after the move.
6. In `InPlace.lean`, write `reverseInPlace (xs : Array Nat) : Array Nat` that swaps elements `i` and `n - 1 - i` for `i < n / 2`, with `dbgTraceIfShared` before each swap. Check with `#guard_msgs` that reversing `Array.range n` (built inside a function of `n`) prints no `shared RC` message, then make a small change that makes it print one. *Hint:* `Array.swapIfInBounds xs i j` swaps two positions. To force a copy, keep the original array alive, for example by returning it in a pair as `bumpAndKeep` does.

CHAPTER 25

Typestate: Protocols in Types

Most interfaces come with rules about order. Connect before you log in; log in before you query; calibrate an instrument before you start a measurement, and stop the measurement before you power it down; set the URL of a request before you send it. Such rules form a **protocol**, and usually they live in the documentation. The code enforces them at run time, if at all: a `state` field, a check at the top of every method, an error such as `NotConnected` that someone has to handle and some test has to provoke.

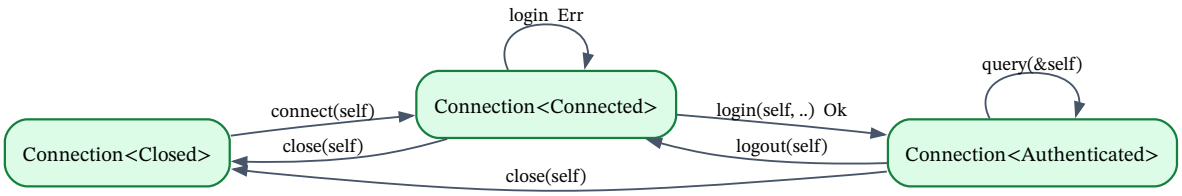
Typestate moves the protocol into the types. Each state of the protocol becomes a type; each operation becomes a function from the type of the state where it is allowed to the type of the state it leads to. Calling an operation in the wrong state is then not a run-time error but a type error, found by the compiler before the program runs. In Rust, the pattern works because of the property that Part V has been studying: values are affine, so a transition can *consume* the old state, and the old state cannot be used again. In this chapter we build typestate APIs in Rust for a database connection, a request builder and a lab instrument; we look briefly at session types, typestate for communication channels; we meet the limit that Chapter 24 predicted (an affine language cannot force a protocol to be completed); and in Lean we use dependent types to go further, with families of types indexed by states, a vault whose type records its balance, and proofs about the traces a protocol accepts.

In this chapter you will learn:

- how to turn a protocol (a state machine) into types, and why consuming methods and affinity are what make the old state unusable;
- the Rust toolkit for typestate: states as type parameters, `PhantomData`, sealed traits for closed sets of states, states that carry data, fallible transitions;
- how to build a builder whose required fields are checked at compile time;
- what session types are, what duality means, and how to encode them in Rust;
- where affinity falls short (nothing forces a protocol to finish), and what `#[must_use]` and a complaining destructor can and cannot do about it;
- how Lean expresses typestate with indexed families, including states that depend on run-time values;
- how to describe a protocol as an inductive relation in Lean and prove that well-typed programs follow it, finish in the final state and stop every measurement they start.

25.1 Protocols as state machines

A protocol is a *labelled transition system*: a set of states, a set of actions, and a transition relation saying which action is allowed in which state and where it leads. Here is the protocol of the database connection we will implement:



The connection protocol. Each box is a state, which will be a type; each arrow is an operation. Operations that take self by value consume the old state and return the new one; query takes &self and stays where it is.

Writing $\delta(s, a) = s'$ for "in state s, action a is allowed and leads to s'", the diagram says $\delta(\mathit{Closed}, \texttt{connect}) = \mathit{Connected}$, $\delta(\mathit{Authenticated}, \texttt{query}) = \mathit{Authenticated}$, and so on; δ is undefined everywhere else.

The run-time approach stores the current state in a field and checks it:

```
fn query(&self, sql: &str) -> Result<Rows, ConnError> {
    if self.state != State::Authenticated {
        return Err(ConnError::NotAuthenticated);
    }
    // ...
}
```

The typestate approach gives the connection a type parameter S for its state, and makes each action a method that exists only for the right S. As a typing rule (a *model* of what the Rust code below does, in the sense of Chapter 1):

$$\frac{\Gamma \vdash c : \texttt{Connection}\langle s \rangle \qquad \delta(s, a) = s'}{\Gamma \vdash c.a() : \texttt{Connection}\langle s' \rangle}\ \textsc{T-Transition}$$

The second premise is not a judgment the compiler derives; it is a fact built into the program by writing an `impl` block for each defined $\delta(s, a)$. If $\delta(s, a)$ is undefined, no rule applies, and the compiler reports that the method does not exist for that type.

The idea is old. Robert Strom and Shaula Yemini proposed typestate in the 1980s as a compiler analysis, and early, pre-1.0 versions of Rust had a typestate system built into the language, later removed. What Rust programmers call the typestate pattern today needs no special support: it is built from generics, traits and ownership.

Why affinity matters

The rule T-Transition has a hidden assumption. After `let c2 = c.connect();`, what happens to `c`? If `c` can still be used, the program has two views of the same connection, one saying `Closed` and one saying `Connected`, and at most one of them is telling the truth. The type of `c` would be a lie.

In a language with contraction, nothing stops you from using `c` again. In Rust, `connect` takes `self` by value, so the call *moves* `c`, and every later use is rejected (E0382). That is the affine discipline of Chapter 22 doing exactly the job we need: the old state is consumed by the transition, and only the new state remains. Here is the error for using a connection after closing it, from `code/rust/compile_fail/ch25_use_after_close.rs`:

```
pub fn finish(mut conn: Connection<Connected>) -> Connection<Closed> {
    let closed = conn.close();
    conn.send("QUIT");
    closed
}
```

```
error[E0382]: borrow of moved value: `conn`
  --> compile_fail/ch25_use_after_close.rs:31:5
   |
29 | pub fn finish(mut conn: Connection<Connected>) -> Connection<Closed> {
   |               -------- move occurs because `conn` has type `Connection<Connected>`,
which does not implement the `Copy` trait
30 |     let closed = conn.close();
   |                       ------- `conn` moved due to this method call
31 |     conn.send("QUIT");
   |     ^^^^ value borrowed here after move
   |
note: `Connection::<Connected>::close` takes ownership of the receiver `self`, which moves
`conn`
  --> compile_fail/ch25_use_after_close.rs:20:18
   |
20 |     pub fn close(self) -> Connection<Closed> {
   |                  ^^^^
```

The note at the end names the culprit precisely: `close` takes ownership of the receiver `self`. In typestate terms, `close` consumed the `Connection<Connected>`, and the only connection left is the `Connection<Closed>` it returned.

25.2 Typestate in Rust: a connection

The full program is `code/rust/examples/ch25_typestate.rs`. It contains two modules, `connection` and `instrument`, and a `main` that uses them.

The states

```
mod sealed {
    /// Only this module can implement `Sealed`, so only the three states
    /// below can ever implement `State`.
    pub trait Sealed {}
}

/// The states of the protocol: a closed set of types.
pub trait State: sealed::Sealed {
    const NAME: &'static str;
}

/// States in which the connection is open.
pub trait Open: State {}

// Empty enums: types with no values, used only as labels.
pub enum Closed {}
pub enum Connected {}
pub enum Authenticated {}
```

```
impl sealed::Sealed for Closed {}
impl sealed::Sealed for Connected {}
impl sealed::Sealed for Authenticated {}
impl State for Closed {
    const NAME: &'static str = "Closed";
}
impl State for Connected {
    const NAME: &'static str = "Connected";
}
impl State for Authenticated {
    const NAME: &'static str = "Authenticated";
}
impl Open for Connected {}
impl Open for Authenticated {}
```

Each state is a type with *no values*: an empty enum, like the `Void` of Chapters 5 and 10. We never create a `Closed`; we only use the type as a label, which is why an empty enum is the right choice (a unit struct would work too, but it would suggest that values of it exist). The trait `State` gathers the three labels and gives each a name for printing.

`State` is **sealed**. Its supertrait `sealed::Sealed` lives in a private module, so code outside `connection` can *use* `State` but cannot *implement* it: no one else can add a fourth state, and the set of states is closed, like the variants of an enum. Without the seal, a user could implement `State` for their own type and create a `Connection<TheirState>` that the protocol knows nothing about. With it, the attempt fails (`code/rust/compile_fail/ch25_sealed.rs`):

```
pub struct Hijacked;

impl connection::State for Hijacked {}
```

```
error[E0277]: the trait bound `Hijacked: Sealed` is not satisfied
  --> compile_fail/ch25_sealed.rs:19:28
   |
19 | impl connection::State for Hijacked {}
   |                            ^^^^^^^^ unsatisfied trait bound
   |
help: the trait `Sealed` is not implemented for `Hijacked`
  --> compile_fail/ch25_sealed.rs:17:1
   |
17 | pub struct Hijacked;
   | ^^^^^^^^^^^^^^^^^^^
help: the trait `Sealed` is implemented for `Closed`
  --> compile_fail/ch25_sealed.rs:12:5
   |
12 |     impl sealed::Sealed for Closed {}
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: required by a bound in `State`
  --> compile_fail/ch25_sealed.rs:9:22
   |
 9 |     pub trait State: sealed::Sealed {}
   |                      ^^^^^^^^^^^^^^ required by this bound in `State`
   = note: `State` is a "sealed trait", because to implement it you also need to implement
`connection::sealed::Sealed`, which is not accessible; this is usually done to force you to
use one of the provided types that already implement it
   = help: the following type implements the trait:
             connection::Closed
```

The compiler's last note even names the pattern. A second trait, `Open`, describes a *subset* of the states, the two in which the connection is open; we will use it to define `close` once for both.

The connection type

```
/// A connection in state `S`. The state exists only in the type:
/// `PhantomData<S>` has size zero.
#[must_use = "transitions return the connection in its new state"]
pub struct Connection<S: State> {
    host: String,
    _state: PhantomData<S>,
}

impl<S: State> Connection<S> {
    /// Available in every state.
    pub fn describe(&self) -> String {
        format!("{} [{}]", self.host, S::NAME)
    }

    /// Change the state: same data, new type.
    fn into_state<T: State>(self) -> Connection<T> {
        Connection {
            host: self.host,
            _state: PhantomData,
        }
    }
}
```

The state appears only in the type. The connection's data, a host name, is the same in every state, and the field `_state: PhantomData<S>` exists only to mention `S`. Rust insists that every type parameter of a struct be used by some field, because it needs to know how the struct relates to the parameter (Chapter 27 explains the *variance* it infers from the fields):

```
pub struct Connection<S> {
    host: String,
}
```

```
error[E0392]: type parameter `S` is never used
 --> compile_fail/ch25_unused_state.rs:6:23
  |
6 | pub struct Connection<S> {
  |                       ^ unused type parameter
  |
  = help: consider removing `S`, referring to it in a field, or using a marker such as
`PhantomData`
  = help: if you intended `S` to be a const parameter, use `const S: /* Type */` instead
```

`PhantomData<S>` is the standard answer: a zero-sized field that tells the compiler "treat this struct as if it contained an `S`". It costs nothing at run time, as the last line of the program's output will confirm.

The `impl<S: State>` block defines methods that exist in *every* state: `describe`, which prints the host and the state's name, and the private helper `into_state`, which rebuilds the same data with a different state type. Every transition will be a call to `into_state`. The attribute `#[must_use]` comes back later in the chapter.

The transitions

```
impl Connection<Closed> {
    pub fn new(host: &str) -> Self {
        Connection {
            host: host.to_string(),
            _state: PhantomData,
        }
    }
```

```
    pub fn connect(self) -> Connection<Connected> {
        self.into_state()
    }
}

impl Connection<Connected> {
    /// A fallible transition: on failure, the caller gets the connection
    /// back, still in state `Connected`.
    pub fn login(
        self,
        user: &str,
        password: &str,
    ) -> Result<Connection<Authenticated>, Connection<Connected>> {
        if user == "lab" && password == "secret" {
            Ok(self.into_state())
        } else {
            Err(self)
        }
    }
}

impl Connection<Authenticated> {
    /// Does not change the state: takes `&self`, not `self`.
    pub fn query(&self, sql: &str) -> String {
        format!("result of `{sql}` from {}", self.host)
    }

    pub fn logout(self) -> Connection<Connected> {
        self.into_state()
    }
}

impl<S: Open> Connection<S> {
    /// Close from any open state.
    pub fn close(self) -> Connection<Closed> {
        self.into_state()
    }
}
```

Each `impl` block is one row of the transition table. `impl Connection<Closed>` defines `new` and `connect`, which exist only for closed connections; `impl Connection<Authenticated>` defines `query` and `logout`; and `impl<S: Open> Connection<S>` defines `close` once for every open state. Three details are worth a closer look.

- **Transitions take `self`, observations take `&self`.** `connect`, `login`, `logout` and `close` consume the connection and return it at a new type. `query` borrows it and leaves the state unchanged, which is why it is a loop in the diagram.
- **Fallible transitions return both outcomes.** A login can fail. `login` returns `Result<Connection<Authenticated>, Connection<Connected>>`: on success the authenticated connection, on failure the *original* connection, still connected, so the caller can try again. Since `login` consumed `self`, returning it in the error case is the only way to give it back. When a transition depends on run-time information, the type cannot know the outcome, and an enum of possible states (`Result` here) is how the uncertainty shows up in the types.
- **Same data, different types.** Every state has the same representation, so `into_state` just moves the host name into a struct of a different type. The compiler sees `Connection<Closed>` and `Connection<Connected>` as two unrelated types; at run time they are the same bytes.

Using it

```
fn session() {
    let conn = Connection::new("db.lab.internal");
    println!("  {}", conn.describe());
    let conn = conn.connect();
    println!("  {}", conn.describe());
    let conn = match conn.login("lab", "wrong") {
        Ok(_) => unreachable!("the password is wrong"),
        Err(conn) => {
            println!("  login failed, still {}", conn.describe());
            conn
        }
    };
    let Ok(conn) = conn.login("lab", "secret") else {
        unreachable!("the password is right");
    };
    println!("  {}", conn.describe());
    println!("  {}", conn.query("SELECT * FROM samples"));
    let conn = conn.logout().close();
    println!("  {}", conn.describe());
}
```

```
connection:
  db.lab.internal [Closed]
  db.lab.internal [Connected]
  login failed, still db.lab.internal [Connected]
  db.lab.internal [Authenticated]
  result of `SELECT * FROM samples` from db.lab.internal
  db.lab.internal [Closed]
```

The code reads like the protocol diagram. Each `let conn = ...` shadows the previous `conn` with a value of a new type: the old one was consumed by the transition, so there is nothing to confuse it with. The failed login hands back the connection, still `Connected`, and the second attempt succeeds. `let ... else` destructures the `Ok` case, and otherwise runs the `else` block, which must diverge (here with `unreachable!`).

Now break the protocol. Query before logging in (`code/rust/compile_fail/ch25_wrong_state.rs`):

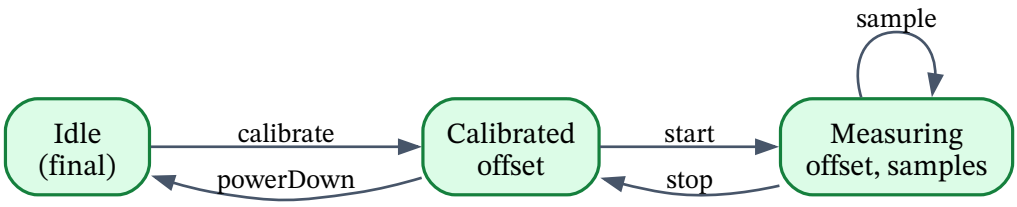
```
pub fn report(conn: Connection<Connected>) -> String {
    conn.query("SELECT * FROM samples")
}
```

```
error[E0599]: no method named `query` found for struct `Connection<Connected>` in the
current scope
  --> compile_fail/ch25_wrong_state.rs:32:10
   |
10 | pub struct Connection<S> {
   | ------------------------ method `query` not found for this struct
...
32 |     conn.query("SELECT * FROM samples")
   |          ^^^^^ method not found in `Connection<Connected>`
   |
   = note: the method was found for `Connection<Authenticated>`
```

This is E0599, "no method found", and it is exactly rule T-Transition failing: $\delta(\mathit{Connected}, \texttt{query})$ is undefined, so there is no `query` for `Connection<Connected>`. The note, "the method was found for `Connection<Authenticated>`", is the compiler pointing at the state where the operation *is* allowed. A protocol violation has become a type error with a good error message.

25.3 States that carry data: a lab instrument

In the connection, every state had the same data. Often the states differ in what is *known*. A lab instrument starts `Idle`. Calibrating it against a reference gives an offset, which every later reading must be corrected by. While `Measuring`, it accumulates samples. A run-time design would put `offset: Option<f64>` and `samples: Vec<f64>` in one struct and `unwrap` the offset wherever it is needed, hoping it was set. With typestate, each state holds exactly the data that exists in it:



The lab instrument protocol. Calibrated knows its offset; Measuring knows its offset and its samples. Idle is the final state: a session should end there.

```
/// The states carry data: what is known in that state, and nothing more.
pub struct Idle;
pub struct Calibrated {
    offset: f64,
}
pub struct Measuring {
    offset: f64,
    samples: Vec<f64>,
    armed: bool,
}

pub struct Instrument<S> {
    name: String,
    state: S,
}
```

Here the type parameter `S` *is* the type of a real field, `state: S`, so no `PhantomData` is needed. A `Calibrated` contains an offset, so an `Instrument<Calibrated>` without an offset cannot be constructed; the `unwrap` and the bug it might hide are gone. This is the slogan "make illegal states unrepresentable" applied to protocols. The transitions move data from state to state:

```
impl Instrument<Idle> {
    pub fn new(name: &str) -> Self {
        Instrument {
            name: name.to_string(),
            state: Idle,
        }
    }

    /// Compare a reading of a known reference with its true value.
    pub fn calibrate(self, reference: f64, reading: f64) -> Instrument<Calibrated> {
        let offset = reference - reading;
        Instrument {
            name: self.name,
            state: Calibrated { offset },
        }
    }
}
```

```
impl Instrument<Calibrated> {
    pub fn start(self) -> Instrument<Measuring> {
        let offset = self.state.offset;
        let state = Measuring {
            offset,
            samples: Vec::new(),
            armed: true,
        };
        Instrument {
            name: self.name,
            state,
        }
    }

    pub fn power_down(self) -> Instrument<Idle> {
        Instrument {
            name: self.name,
            state: Idle,
        }
    }
}

impl Instrument<Measuring> {
    pub fn record(&mut self, raw: f64) {
        self.state.samples.push(raw + self.state.offset);
    }

    /// Finish the measurement: back to `Calibrated`, with the samples.
    pub fn stop(mut self) -> (Instrument<Calibrated>, Vec<f64>) {
        // `Measuring` implements `Drop`, so its fields cannot be moved out
        // (E0509): take the samples, and disarm the complaint.
        let samples = std::mem::take(&mut self.state.samples);
        self.state.armed = false;
        let state = Calibrated {
            offset: self.state.offset,
        };
        (
            Instrument {
                name: self.name,
                state,
            },
            samples,
        )
    }
}

impl<S> Instrument<S> {
    pub fn name(&self) -> &str {
        &self.name
    }
}
```

`calibrate` computes the offset and stores it in the new state. `start` copies the offset into a fresh `Measuring`. `record` takes `&mut self`: recording a sample changes the data but not the state. `stop` returns the instrument in state `Calibrated` together with the samples. Its body is more complicated than you might expect, because of a destructor we will meet in the section on what affinity cannot do. Here is the program using it:

```
fn measure() {
    let probe = Instrument::new("thermometer").calibrate(100.0, 99.5);
    let mut run = probe.start();
    for raw in [20.0, 20.5, 21.0] {
        run.record(raw);
    }
    let (probe, samples) = run.stop();
    println!("  {}: samples {samples:?}", probe.name());
```

```
    let probe = probe.power_down().calibrate(100.0, 100.0);
    let mut run = probe.start();
    run.record(22.0);
    run.record(22.5);
    println!("  dropping {} in the middle of a measurement", run.name());
} // `run` is dropped here, still in state Measuring
```

```
instrument:
  thermometer: samples [20.5, 21.0, 21.5]
  dropping thermometer in the middle of a measurement
  warning: measurement abandoned, 2 sample(s) lost
size of Connection<Authenticated>: 24 bytes (a String)
```

The first measurement is complete: the probe read 99.5 for a reference of 100, so the offset is 0.5, and the three raw readings come back corrected. The second measurement is *abandoned*: the function ends while `run` is still `Instrument<Measuring>`. Rust accepts this, because a value may always be dropped, and the only trace of the mistake is the warning printed by a destructor. That line is the subject of a later section. The last line of output shows the cost of the typestate machinery in the connection: `Connection<Authenticated>` is 24 bytes, the size of its `String`, and the state type adds nothing.

25.4 Builders with required fields

The builder pattern constructs a complicated value step by step: `Request::builder().method(...).url(...).header(...).build()`. Some fields are required, and the usual implementation checks them in `build` (`code/rust/examples/ch25_builder.rs`):

```
/// The usual builder: required fields are `Option`s, checked by `build`.
#[derive(Default)]
pub struct DynBuilder {
    method: Option<Method>,
    url: Option<String>,
    timeout_ms: Option<u64>,
}

#[derive(Debug)]
pub enum BuildError {
    MissingMethod,
    MissingUrl,
}

impl DynBuilder {
    pub fn method(mut self, method: Method) -> Self {
        self.method = Some(method);
        self
    }

    pub fn url(mut self, url: &str) -> Self {
        self.url = Some(url.to_string());
        self
    }

    pub fn build(self) -> Result<Request, BuildError> {
        Ok(Request {
            method: self.method.ok_or(BuildError::MissingMethod)?,
            url: self.url.ok_or(BuildError::MissingUrl)?,
            headers: Vec::new(),
            timeout_ms: self.timeout_ms.unwrap_or(30_000),
        })
```

```
    }
}
```

```
let missing = DynBuilder::default().method(Method::Get).build();
println!("dynamic, URL forgotten: {missing:?}");
let ok = DynBuilder::default()
    .url("/samples")
    .method(Method::Get)
    .build();
println!("dynamic, complete:      {}", ok.expect("both fields set"));
```

```
dynamic, URL forgotten: Err(MissingUrl)
dynamic, complete:      Get /samples (timeout 30000 ms)
```

Forgetting the URL is an error at run time, in a `Result` that every caller must handle even when, as in most calls, every field was set. A builder is a protocol too ("set the method and the URL, in any order, then build"), and typestate can check it. The state of the builder is "which required fields are set", one type parameter per field:

```
/// Marker types: "not set yet". The set versions carry the value itself.
pub struct NoMethod;
pub struct NoUrl;
pub struct Url(String);

/// `M` is `NoMethod` or `Method`; `U` is `NoUrl` or `Url`. No `PhantomData`
/// is needed: the type parameters are the types of real fields.
pub struct RequestBuilder<M, U> {
    method: M,
    url: U,
    headers: Vec<(String, String)>,
    timeout_ms: u64,
}

impl RequestBuilder<NoMethod, NoUrl> {
    pub fn new() -> Self {
        RequestBuilder {
            method: NoMethod,
            url: NoUrl,
            headers: Vec::new(),
            timeout_ms: 30_000,
        }
    }
}

/// Setting the method: only possible while it is not set, whatever `U` is.
impl<U> RequestBuilder<NoMethod, U> {
    pub fn method(self, method: Method) -> RequestBuilder<Method, U> {
        RequestBuilder {
            method,
            url: self.url,
            headers: self.headers,
            timeout_ms: self.timeout_ms,
        }
    }
}

/// Setting the URL: only possible while it is not set, whatever `M` is.
impl<M> RequestBuilder<M, NoUrl> {
    pub fn url(self, url: &str) -> RequestBuilder<M, Url> {
        RequestBuilder {
            method: self.method,
            url: Url(url.to_string()),
            headers: self.headers,
            timeout_ms: self.timeout_ms,
```

```
        }
    }
}

/// Optional settings: available in every state, and they keep the state.
impl<M, U> RequestBuilder<M, U> {
    pub fn header(mut self, name: &str, value: &str) -> Self {
        self.headers.push((name.to_string(), value.to_string()));
        self
    }

    pub fn timeout_ms(mut self, ms: u64) -> Self {
        self.timeout_ms = ms;
        self
    }
}

/// `build` exists only when both required fields are set.
impl RequestBuilder<Method, Url> {
    pub fn build(self) -> Request {
        Request {
            method: self.method,
            url: self.url.0,
            headers: self.headers,
            timeout_ms: self.timeout_ms,
        }
    }
}
```

The type parameter `M` is `NoMethod` until a method is set, and then it is `Method`, the type of the method itself; likewise `U` is `NoUrl` or `Url`. The markers for "not set" are unit structs, and the "set" types hold the value, so again every parameter is the type of a real field and no `PhantomData` is needed. Read the `impl` blocks as transitions:

- `impl<U> RequestBuilder<NoMethod, U>` defines `method` for every builder whose method is not set yet, *whatever* the state of the URL. That generic `U` is what makes the order free: the method can be set before or after the URL.
- `impl<M> RequestBuilder<M, NoUrl>` does the same for `url`.
- `impl<M, U> RequestBuilder<M, U>` defines the optional settings for every state; they return `Self`, the same type, so they do not change the state.
- `impl RequestBuilder<Method, Url>` defines `build`, only when both are set. It returns a `Request`, not a `Result`: there is nothing left to go wrong.

```
let a = RequestBuilder::new()
    .method(Method::Post)
    .header("content-type", "application/json")
    .url("/samples")
    .timeout_ms(500)
    .build();
let b = RequestBuilder::new()
    .url("/samples/42")
    .method(Method::Get)
    .build();
println!("typestate, method first: {a}");
println!("typestate, URL first:    {b}");
```

```
typestate, method first: Post /samples (timeout 500 ms, content-type: application/json)
typestate, URL first:    Get /samples/42 (timeout 30000 ms)
size of RequestBuilder<NoMethod, NoUrl>: 32 bytes
size of RequestBuilder<Method, Url>:     64 bytes
```

Both orders work, optional settings can appear anywhere in the chain, and the two sizes show that the type of the builder changes as it fills up. A missing field is now a compile-time error (`code/rust/compile_fail/ch25_builder_incomplete.rs`):

```
pub fn request() -> (String, String) {
    RequestBuilder::new().method("GET").build()
}
```

```
error[E0599]: no method named `build` found for struct `RequestBuilder<Method, NoUrl>` in
the current scope
  --> compile_fail/ch25_builder_incomplete.rs:41:41
   |
10 | pub struct RequestBuilder<M, U> {
   | ------------------------------- method `build` not found for this struct
...
41 |     RequestBuilder::new().method("GET").build()
   |                                         ^^^^^ method not found in
`RequestBuilder<Method, NoUrl>`
   |
   = note: the method was found for `RequestBuilder<Method, Url>`
```

The type in the message, `RequestBuilder<Method, NoUrl>`, tells you which field is missing. Setting the method twice is rejected the same way, since `method` exists only while `M` is `NoMethod`.

The technique has costs, and they grow with the number of required fields. Each field adds a type parameter and doubles the number of possible builder types; each transition method has to move every field into the new struct (the field lists in `method` and `url`); and the error messages mention types that the user never wrote. For two or three required fields it is usually worth it; for ten, crates that generate typestate builders with a derive macro exist, and a run-time check may also be the more readable choice.

25.5 Session types, briefly

The protocols so far govern one object. A **session type** governs a *conversation*: the type of a communication channel describes the whole sequence of messages that will travel on it. Session types were introduced by Kohei Honda and his colleagues in the 1990s. A small syntax:

$$S ::= !\tau.\, S \mid ?\,\tau.\, S \mid \mathit{end}$$

Here $!\tau.\, S$ means "send a value of type τ, then continue as S", $?\,\tau.\, S$ means "receive a τ, then continue as S", and *end* means that the conversation is over. A client that sends a sensor number and receives a reading has type $!\texttt{u32}.\, ?\,\texttt{f64}.\, \mathit{end}$.

The server at the other end of the channel must do the opposite: receive where the client sends, send where the client receives. This mirror image is the **dual** of a session type:

$$\overline{!\tau.\, S} = ?\,\tau.\, \overline{S} \qquad \overline{?\,\tau.\, S} = !\tau.\, \overline{S} \qquad \overline{\mathit{end}} = \mathit{end}$$

Duality is the central idea: if the two ends of a channel have dual types, every send is matched by a receive of the same type, and neither side waits for a message the other will never send. Duality is also an *involution*: the dual of the dual is the original, $\overline{\overline{S}} = S$. (If you know linear logic, this is the duality of linear negation: session types can be read as propositions of linear logic, the logic of Chapter 21.)

In Rust

The example `code/rust/examples/ch25_session.rs` encodes session types as types, and duality as a trait with an associated type:

```rust
/// "Send a `T`, then continue as `S`."
pub struct Send<T, S>(PhantomData<(T, S)>);
/// "Receive a `T`, then continue as `S`."
pub struct Recv<T, S>(PhantomData<(T, S)>);
/// "The conversation is over."
pub struct End;

/// Every session type has a dual: the same protocol seen from the other side.
pub trait Session {
    type Dual: Session;
}

impl Session for End {
    type Dual = End;
}
impl<T, S: Session> Session for Send<T, S> {
    type Dual = Recv<T, S::Dual>;
}
impl<T, S: Session> Session for Recv<T, S> {
    type Dual = Send<T, S::Dual>;
}
```

The three `impl` blocks are the three equations of duality, computed by the trait solver at compile time. A protocol is a type, and its dual is computed, not written:

```rust
/// The client names a sensor, receives its reading, sends back a correction
/// and learns whether the corrected value is in range.
type Client = Send<u32, Recv<f64, Send<f64, Recv<bool, End>>>>;
/// The server's protocol is computed, not written by hand.
type Server = <Client as Session>::Dual;

/// Compiles only if the dual of `A` is `B`.
fn assert_dual<A: Session<Dual = B>, B>() {}
```

`assert_dual::<A, B>()` compiles only if `B` is the dual of `A`, so the two calls in `main` make the compiler check $\overline{Client} = Server$ and $\overline{Server} = Client$. A channel endpoint is a typestate object whose state is the rest of the protocol:

```rust
/// What travels on the wire.
#[derive(Debug)]
enum Msg {
    Nat(u32),
    Real(f64),
    Flag(bool),
}

/// The payload types the wire can carry.
trait Payload: Sized {
    fn wrap(self) -> Msg;
    fn unwrap(msg: Msg) -> Option<Self>;
}

impl Payload for u32 {
    fn wrap(self) -> Msg {
        Msg::Nat(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Nat(n) = msg { Some(n) } else { None }
    }
}
```

```
impl Payload for f64 {
    fn wrap(self) -> Msg {
        Msg::Real(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Real(x) = msg {
            Some(x)
        } else {
            None
        }
    }
}

impl Payload for bool {
    fn wrap(self) -> Msg {
        Msg::Flag(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Flag(b) = msg {
            Some(b)
        } else {
            None
        }
    }
}

type Queue = Rc<RefCell<VecDeque<Msg>>>;

/// An endpoint that must follow protocol `S`. Every operation consumes the
/// endpoint and returns it at the next step of the protocol.
struct Chan<S> {
    incoming: Queue,
    outgoing: Queue,
    _protocol: PhantomData<S>,
}

impl<S: Session> Chan<S> {
    /// Two connected endpoints, with dual protocols.
    fn pair() -> (Chan<S>, Chan<S::Dual>) {
        let a_to_b: Queue = Rc::default();
        let b_to_a: Queue = Rc::default();
        let a = Chan {
            incoming: Rc::clone(&b_to_a),
            outgoing: Rc::clone(&a_to_b),
            _protocol: PhantomData,
        };
        let b = Chan {
            incoming: a_to_b,
            outgoing: b_to_a,
            _protocol: PhantomData,
        };
        (a, b)
    }
}

impl<S> Chan<S> {
    fn advance<S2>(self) -> Chan<S2> {
        Chan {
            incoming: self.incoming,
            outgoing: self.outgoing,
            _protocol: PhantomData,
        }
    }
}

impl<T: Payload, S> Chan<Send<T, S>> {
    fn send(self, value: T) -> Chan<S> {
        self.outgoing.borrow_mut().push_back(value.wrap());
        self.advance()
```

```
    }
}

impl<T: Payload, S> Chan<Recv<T, S>> {
    fn recv(self) -> (T, Chan<S>) {
        let msg = self.incoming.borrow_mut().pop_front();
        let value = msg
            .and_then(T::unwrap)
            .expect("the peer follows the dual protocol");
        (value, self.advance())
    }
}

impl Chan<End> {
    fn close(self) {}
}
```

The parts to read are the four `impl` blocks at the end. `send` exists only on a `Chan<Send<T, S>>`, takes a `T` and returns a `Chan<S>`: one step of the protocol consumed. `recv` exists only on a `Chan<Recv<T, S>>` and returns the received value with a `Chan<S>`. `close` exists only on a `Chan<End>`. And `pair` creates two connected endpoints whose types are dual *by construction*: `Chan<S>` and `Chan<S::Dual>`. (The queues and the `Payload` trait are plumbing for an in-memory wire; a real library would use threads or an async runtime.)

```
assert_dual::<Client, Server>();
assert_dual::<Server, Client>();
let short = |name: &str| name.replace("ch25_session::", "");
println!("Client = {}", short(std::any::type_name::<Client>()));
println!("Server = {}", short(std::any::type_name::<Server>()));

// The two sides take turns (in a real program, two threads or tasks).
let (client, server) = Chan::<Client>::pair();
let client = client.send(7_u32);
let (sensor, server) = server.recv();
let server = server.send(21.5_f64);
let (reading, client) = client.recv();
let client = client.send(reading - 0.5);
let (corrected, server) = server.recv();
let server = server.send((15.0..=25.0).contains(&corrected));
let (in_range, client) = client.recv();
client.close();
server.close();
println!("sensor {sensor}: reading {reading}, corrected {corrected}");
println!("in range: {in_range}");
```

```
Client = Send<u32, Recv<f64, Send<f64, Recv<bool, End>>>>
Server = Recv<u32, Send<f64, Recv<f64, Send<bool, End>>>>
sensor 7: reading 21.5, corrected 21
in range: true
```

The two type names in the output are each other's mirror image. In the conversation, notice what is *not* written: no type annotations on `recv`. When the server calls `server.recv()`, the type of its endpoint is `Chan<Recv<u32, ...>>`, so the compiler knows the value is a `u32`. Calling `client.recv()` first, or sending a `bool` where a `u32` is expected, is a type error.

The encoding has the limits you would now expect. It guarantees "no wrong message": each endpoint can only perform the next step of its protocol. It does not guarantee "no missing message": an endpoint can be dropped (or forgotten) in the middle of the conversation, and the peer then waits for a message that never comes (in our in-memory version, `recv` would panic). Real protocols also need choice

(offer several continuations) and recursion (loops), which make the encodings considerably heavier. The Lean section below shows what a language that can check *totality* adds.

25.6 Where affinity falls short

Typestate turns "operation in the wrong state" into a type error. It does not turn "protocol left unfinished" into one. The second measurement of the lab instrument was abandoned while `Measuring`, and the program compiled; a `Connection` can be dropped while authenticated; a session endpoint can be dropped before `End`. The reason is the one Chapter 24 gave: Rust keeps weakening, so any value may be discarded at any point, and `mem::forget` and `Rc` cycles may even discard it without running its destructor. A protocol that *must* reach a final state needs linear types, which Rust does not have.

What we can do is approximate. There are three common mitigations, each with its own limits.

`#[must_use]`

The `Connection` struct carries `#[must_use = "transitions return the connection in its new state"]`. Since every transition returns a new `Connection`, the most likely way to lose one is to call a transition and ignore its result: `conn.connect();` on its own line. The compiler then warns "unused `Connection` that must be used", followed by the note from the attribute. As we saw in Chapter 24, this is a lint about one expression statement: it does not notice a connection stored in a variable and never used again, and `let _ = ...` silences it.

A destructor that complains

The second mitigation is a run-time check at the point of abandonment: a `Drop` implementation that notices when a value is dropped in a non-final state, and reports it. That is what produced the warning line in the instrument's output:

```
/// A destructor that complains: the protocol says a measurement must be
/// stopped, and the type system cannot force it.
impl Drop for Measuring {
    fn drop(&mut self) {
        if self.armed {
            let n = self.samples.len();
            println!("  warning: measurement abandoned, {n} sample(s) lost");
        }
    }
}
```

There are three things to understand about this small piece of code.

The destructor is on the state, not on the instrument. The natural place for it would be `impl Drop for Instrument<Measuring>`, a destructor for one state only. Rust forbids that (`code/rust/compile_fail/ch25_drop_one_state.rs`):

```
impl Drop for Instrument<Measuring> {
    fn drop(&mut self) {
        println!("{} dropped while measuring", self.name);
    }
}
```

```
error[E0366]: `Drop` impls cannot be specialized
  --> compile_fail/ch25_drop_one_state.rs:14:1
   |
14 | impl Drop for Instrument<Measuring> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: `Measuring` is not a generic parameter
note: use the same sequence of generic lifetime, type and const parameters as the struct
definition
  --> compile_fail/ch25_drop_one_state.rs:8:1
   |
 8 | pub struct Instrument<S> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^
```

A `Drop` impl must be exactly as generic as the type (E0366): either every `Instrument<S>` has the destructor or none does. This is a deliberate restriction: the destructor of a generic type is one piece of generic code, the same for all its instances. Since our states carry data, we can put the destructor on the state type `Measuring` instead, which has no type parameters, and `Instrument<Measuring>` inherits it through its field. (With zero-sized marker states, the workaround is an `impl<S: State> Drop` that looks at a constant of the state, such as `S::NAME`.)

The destructor gets in the way of the transitions. Look again at `stop`. Since `Measuring` implements `Drop`, its fields cannot be moved out (E0509, Chapter 24), so `stop` cannot simply take `self.state.samples`. It uses `mem::take` to swap the samples out for an empty vector, copies the offset (an `f64` is `Copy`), and then sets `armed` to `false` before returning. The old `Measuring` value is dropped at the end of `stop`, as every consumed state is, and the `armed` flag is how the destructor tells "consumed by `stop`" from "abandoned". Every typestate API with a complaining destructor needs some version of this disarming step.

It complains, it does not prevent. The check happens at run time, when the damage is done. It cannot run at all if the value is forgotten. And it should not panic: a destructor that panics while the thread is already unwinding from another panic aborts the process (Chapter 24). Logging, counting or, in debug builds, an assertion are the usual choices.

Keeping control

The strongest mitigation is the one `thread::scope` used in Chapter 24: do not hand the caller a value that must be finished; take a closure, and finish the protocol yourself after the closure returns. For the instrument:

```
impl Instrument<Calibrated> {
    fn measure(
        self,
        f: impl FnOnce(&mut Recorder),
    ) -> (Instrument<Calibrated>, Vec<f64>) {
        let mut run = self.start();
        f(&mut Recorder(&mut run)); // the caller records samples
        run.stop() // and we always stop
    }
}
```

The caller never holds an `Instrument<Measuring>`, so it cannot abandon one (only a panic inside `f` can interrupt the protocol, and then unwinding runs the destructors). Exercise 4 asks you to write it. The price is flexibility: the measurement must fit inside one closure call.

25.7 Typestate in Lean: indexed families

Lean has no ownership and no affinity, but it has something Rust lacks: types that depend on values. That changes what typestate looks like. The Lean files of this chapter are in `code/lean/Book/Ch25/`; we start with `Typestate.lean`.

```
/-- The states of the protocol, as an ordinary inductive type. -/
inductive ConnState where
  | closed | connected | authenticated
  deriving Repr, DecidableEq

/-- A connection in state `s`. The state appears only in the type: no field
mentions it, and Lean, unlike Rust, does not mind. -/
structure Conn (s : ConnState) where
  host : String
  sent : List String    -- what has gone over the wire, for the demo
  deriving Repr
```

The states are an ordinary inductive type, `ConnState`, with three values. That is already a difference from Rust, where the states had to be three separate types: in Lean, a type can be parameterized by a *value* of `ConnState`, so one type of states suffices. `Conn s` is a structure with a parameter `s`, and Lean does not mind that no field mentions it, so there is no `PhantomData`. Here is the type of `Conn` itself:

```
/-- info: Conn : ConnState → Type -/
#guard_msgs in
#check @Conn
```

`Conn` is a *function from states to types*: a family of types, one for each state. Such a function is called an **indexed family** of types. (Strictly speaking, `s` is a *parameter* of the structure; Chapter 30 explains the difference between parameters and indices, which does not matter here.) The operations are functions between members of the family:

```
def Conn.new (host : String) : Conn .closed := ⟨host, []⟩

def Conn.connect (c : Conn .closed) : Conn .connected :=
  ⟨c.host, c.sent ++ ["HELLO"]⟩

def Conn.login (c : Conn .connected) (user : String) : Conn .authenticated :=
  ⟨c.host, c.sent ++ [s!"LOGIN {user}"]⟩

def Conn.query (c : Conn .authenticated) (sql : String) : Conn .authenticated :=
  ⟨c.host, c.sent ++ [s!"QUERY {sql}"]⟩

/-- Available in every state: `s` is an implicit argument. -/
def Conn.close {s : ConnState} (c : Conn s) : Conn .closed :=
  ⟨c.host, c.sent ++ ["BYE"]⟩
```

The types are the transition table: `connect` maps `Conn .closed` to `Conn .connected`, and so on. `close` takes an implicit `{s : ConnState}`, so it works in every state; this is Lean's version of Rust's `impl<S: State> Connection<S>` (and we could restrict it to open states with an extra hypothesis, much as `Open` did). A complete session runs as expected:

```
/-- info: { host := "db", sent := ["HELLO", "LOGIN lab", "QUERY SELECT 1", "BYE"] } -/
#guard_msgs in
#eval ((Conn.new "db").connect.login "lab" |>.query "SELECT 1").close
```

(`e |>.query "SELECT 1"` is Lean's notation for `(e).query "SELECT 1"`: it calls `query` on the result of everything to its left, without extra parentheses.) And querying before logging in is a type error:

```
/--
error: Application type mismatch: The argument
  (Conn.new "db").connect
has type
  Conn ConnState.connected
but is expected to have type
  Conn ConnState.authenticated
in the application
  (Conn.new "db").connect.query
-/
#guard_msgs in
def tooEarly := (Conn.new "db").connect.query "SELECT 1"
```

The message has the same content as Rust's E0599, phrased as a mismatch between the type of the argument and the type `query` expects.

But Lean is not affine, and this is perfectly legal:

```
/-- Lean is not affine: the old state `c` can be used twice. For an
immutable value this is harmless; it would not be for a live socket. -/
def connectTwice (c : Conn .closed) : Conn .connected × Conn .connected :=
  (c.connect, c.connect)
```

The old state `c` is used twice. In Lean this is harmless: a `Conn .closed` is an immutable value, a *description* of a closed connection, not the connection itself. Both results are correct descriptions of what you would get by connecting it. Rust needs affinity for typestate because its values *are* the resources (a socket, a file, an instrument on a serial port), and two live views of one resource can disagree. Lean programs that talk to the outside world do it inside `IO`, where the order of effects is fixed by the monad (Chapter 20), and Lean values that stand for external resources, such as file handles, are not typestated by the standard library.

States that depend on run-time values: a vault

So far the states were a fixed, finite set. In Lean the index of a family can be any value, including one computed at run time. Here is a vault whose type records its balance:

```
/-- A vault whose type records its balance. The balance is not a field: it
is the index, and `Vault.balance` reads it back from the type. -/
structure Vault (balance : Nat) where
  owner : String

def Vault.open (owner : String) : Vault 0 := ⟨owner⟩

def Vault.deposit {b : Nat} (v : Vault b) (amount : Nat) : Vault (b + amount) :=
  ⟨v.owner⟩

/-- Withdrawing requires a proof that the balance suffices. The body does not
use the proof (hence the underscore); the type checker does. -/
def Vault.withdraw {b : Nat} (v : Vault b) (amount : Nat)
    (_enough : amount ≤ b) : Vault (b - amount) :=
  ⟨v.owner⟩

def Vault.balance {b : Nat} (_ : Vault b) : Nat := b
```

`Vault 100` is the type of vaults holding 100 units. The balance is not a field: it is the index, and `Vault.balance` reads it back from the type (the implicit argument `{b : Nat}` is a real argument at run

time, filled in by the type checker). `deposit` returns a vault whose type says the balance went up. `withdraw` takes an extra argument, a *proof* that the amount does not exceed the balance. That is a Lean feature with no counterpart in Rust: a function can demand evidence of a fact about its arguments (Chapter 1 met it with array indexing). The body ignores the proof (hence the name `_enough`); its only job is to make the call type-check.

```
/-- The type checker computes the balance: `0 + 100 - 30` is `70`. -/
def vault70 : Vault 70 :=
  ((Vault.open "lab").deposit 100).withdraw 30 (by decide)

/-- info: 70 -/
#guard_msgs in
#eval vault70.balance
```

The type checker does the bookkeeping: depositing 100 into a `Vault 0` gives a `Vault (0 + 100)`, and withdrawing 30 gives a `Vault (0 + 100 - 30)`, which Lean accepts as a `Vault 70` because the two indices compute to the same number. The proof obligation `30 ≤ 0 + 100` is discharged by `decide`. Overdrawing fails:

```
/--
error: Tactic `decide` proved that the proposition
  30 ≤ 0 + 10
is false
-/
#guard_msgs in
def overdrawn := ((Vault.open "lab").deposit 10).withdraw 30 (by decide)
```

This is a rejection of the same kind as Rust's E0599, a protocol violation found before the program runs, except that the "state" is a number and the rule is arithmetic.

When the amount is only known at run time, no one can write the proof in advance. The program must test the condition, and Lean's *dependent* `if` turns the test into a proof:

```
/-- An amount known only at run time: the dependent `if` tests the condition
and, in the `then` branch, hands us a proof `h` of it. -/
def tryWithdraw {b : Nat} (v : Vault b) (amount : Nat) :
    Option (Vault (b - amount)) :=
  if h : amount ≤ b then some (v.withdraw amount h) else none

/-- info: none -/
#guard_msgs in
#eval (tryWithdraw ((Vault.open "lab").deposit 10) 30).map Vault.balance

/-- info: some 20 -/
#guard_msgs in
#eval (tryWithdraw ((Vault.open "lab").deposit 50) 30).map Vault.balance
```

In `if h : amount ≤ b then ... else ...`, the name `h` is bound to a proof of `amount ≤ b` inside the `then` branch (and to a proof of its negation in the `else` branch). The run-time check and the static guarantee meet: the check produces exactly the evidence that `withdraw` demands. The result type `Option (Vault (b - amount))` is still precise about the balance in the success case.

If even the balance is only known at run time, for example after applying a list of transactions read from a file, we can no longer write it in the type. We package it with the vault instead:

```
/-- A vault with *some* balance: a dependent pair of the balance and a vault
of that balance. This is how a balance computed at run time is packaged. -/
abbrev AnyVault := (b : Nat) × Vault b

/-- Apply a list of transactions (positive: deposit, negative: withdrawal).
Withdrawals that would overdraw are skipped. -/
def applyAll : AnyVault → List Int → AnyVault
  | v, [] => v
  | ⟨b, v⟩, d :: ds =>
    if d ≥ 0 then
      applyAll ⟨b + d.toNat, v.deposit d.toNat⟩ ds
    else if h : d.natAbs ≤ b then
      applyAll ⟨b - d.natAbs, v.withdraw d.natAbs h⟩ ds
    else
      applyAll ⟨b, v⟩ ds

/-- info: 90 -/
#guard_msgs in
#eval (applyAll ⟨0, Vault.open "lab"⟩ [100, -30, -500, 20]).1
```

`(b : Nat) × Vault b` is a **dependent pair**: a number `b` together with a vault whose type mentions *that* `b` (Chapter 10 introduced these Σ-types). It is the Lean counterpart of an existential type (Chapter 15): "a vault with some balance". `applyAll` takes such a pair apart with the pattern `⟨b, v⟩`, and builds a new one after each transaction; withdrawals are guarded by a dependent `if`, so even inside this loop no withdrawal happens without a proof. Here, 100 is deposited, 30 withdrawn, the withdrawal of 500 is skipped, and 20 deposited: 90.

We can still prove things about the result:

```
/-- Whatever the transactions, the result is a well-formed vault: its type
index *is* its balance, and it is a natural number, so it never went
negative. What we can still prove is how deposits add up. -/
theorem applyAll_deposits (b : Nat) (v : Vault b) (ds : List Nat) :
    (applyAll ⟨b, v⟩ (ds.map Int.ofNat)).1 = b + ds.sum := by
  induction ds generalizing b with
  | nil => simp [applyAll]
  | cons d ds ih =>
    -- a deposit: the `d ≥ 0` branch is taken
    simp only [List.map_cons, applyAll, Int.ofNat_eq_natCast, Int.natCast_nonneg,
      ↓reduceIte, Int.toNat_natCast, List.sum_cons]
    rw [ih]
    omega
```

The statement says that applying a list of deposits (natural numbers, converted to integers by `Int.ofNat`) adds their sum to the balance. The proof is by induction on the list, generalizing the starting balance `b`. In the step case, `simp only` unfolds one step of `applyAll`, and the listed lemmas let it decide the `if`: a converted natural number is nonnegative (`Int.natCast_nonneg`), so the deposit branch is taken (`↓reduceIte` simplifies an `if` whose condition is now `True`), and converting back gives the original number (`Int.toNat_natCast`). Rewriting with the induction hypothesis leaves `b + d + sum ds = b + (d + sum ds)`, which `omega` proves.

Rust's closest tool is const generics (Chapter 30): a `Vault<const B: u64>` can record a balance known at compile time. But stable Rust cannot compute with const parameters in types (`Vault<{ B - A }>` requires an unstable feature), and a value known only at run time can never appear in a type. A Rust vault would check the balance at run time and return a `Result`, perhaps with a newtype whose constructor is the only place the check happens.

25.8 Protocols as inductive relations

The typestate API encodes the protocol *implicitly*, in the types of the operations. To reason about protocols, we also want them as explicit objects. The file `code/lean/Book/Ch25/Protocol.lean` does this for the lab instrument:

```
inductive Mode where
  | idle | calibrated | measuring
  deriving DecidableEq

inductive Action where
  | calibrate | start | sample | stop | powerDown
  deriving DecidableEq
```

```
/-- `Step m a m'`: in mode `m`, action `a` is allowed and leads to mode `m'`. -/
inductive Step : Mode → Action → Mode → Prop where
  | calibrate : Step .idle .calibrate .calibrated
  | start     : Step .calibrated .start .measuring
  | sample    : Step .measuring .sample .measuring
  | stop      : Step .measuring .stop .calibrated
  | powerDown : Step .calibrated .powerDown .idle

/-- `Run m tr m'`: the trace `tr` is allowed from `m` and ends in `m'`. -/
inductive Run : Mode → List Action → Mode → Prop where
  | nil  {m : Mode} : Run m [] m
  | cons {m₁ m₂ m₃ : Mode} {a : Action} {as : List Action} :
      Step m₁ a m₂ → Run m₂ as m₃ → Run m₁ (a :: as) m₃
```

`Step` is the transition relation δ, written as an inductive predicate in the style of the typing rules of Chapter 3: each constructor is an axiom, one per arrow of the diagram. `Run m tr m'` says that the trace `tr`, a list of actions, is allowed from mode `m` and ends in `m'`. It has two rules: the empty trace stays where it is; and a step followed by a run is a run.

A relation is good for proofs; for computation we want a function. `next` is the transition function, which returns `none` for a forbidden action, and `accepts` runs it along a trace:

```
/-- The transition function: the protocol as a run-time check. -/
def next : Mode → Action → Option Mode
  | .idle, .calibrate => some .calibrated
  | .calibrated, .start => some .measuring
  | .measuring, .sample => some .measuring
  | .measuring, .stop => some .calibrated
  | .calibrated, .powerDown => some .idle
  | _, _ => none

def accepts : Mode → List Action → Option Mode
  | m, [] => some m
  | m, a :: as => (next m a).bind fun m' => accepts m' as

/-- info: some idle -/
#guard_msgs in
#eval accepts .idle [.calibrate, .start, .sample, .sample, .stop, .powerDown]

/-- info: none -/
#guard_msgs in
#eval accepts .idle [.calibrate, .sample]
```

(`Option.bind` continues with the next action if the previous one was allowed, and propagates `none` otherwise, like `?` in Rust.) This is the run-time version of the protocol, what a `state` field and a check in each method would compute. The two versions agree, and proving it is our first theorem:

```
theorem next_iff {m m' : Mode} {a : Action} : next m a = some m' ↔ Step m a m' := by
  constructor
  · intro h
    -- 15 combinations of mode and action; `simp [next]` discards the ones
    -- where `next` is `none`, and each remaining one is a constructor.
    cases m <;> cases a <;> simp [next] at h <;> subst h <;> constructor
  · intro h
    cases h <;> rfl

theorem accepts_iff {m m' : Mode} {tr : List Action} :
    accepts m tr = some m' ↔ Run m tr m' := by
  induction tr generalizing m with
  | nil =>
    constructor
    · intro h
      simp only [accepts, Option.some.injEq] at h
      subst h
      exact .nil
    · intro h
      cases h
      rfl
  | cons a as ih =>
    constructor
    · intro h
      simp only [accepts, Option.bind_eq_some_iff] at h
      obtain ⟨m₂, h₁, h₂⟩ := h
      exact .cons (next_iff.mp h₁) (ih.mp h₂)
    · intro h
      cases h with
      | cons hs hr => simp [accepts, next_iff.mpr hs, ih.mpr hr]

/-- A concrete run, proved by computation through `accepts_iff`. -/
example : Run .idle [.calibrate, .start, .sample, .stop, .powerDown] .idle :=
  accepts_iff.mp rfl
```

`next_iff` says that the function and the relation agree on single steps. Left to right, we split on the mode and the action with `cases m <;> cases a`, which produces 15 goals (the combinator `<;>` applies the next tactic to every goal the previous one produced). In each, `simp [next] at h` computes `next`: where it is `none`, the hypothesis `none = some m'` is false and the goal closes; where it is `some`, `subst h` replaces `m'` by the computed mode, and `constructor` finds the `Step` constructor that proves it. Right to left, `cases h` considers each of the five constructors, and in each `next` computes to the right value, so `rfl` closes it.

`accepts_iff` lifts this to traces by induction on the trace. In the `nil` case, `accepts m []` is `some m`, which matches `Run.nil`. In the `cons` case, left to right: `Option.bind_eq_some_iff` says that a `bind` succeeded exactly when the first step gave some `m₂` (`h₁`) and the rest succeeded from there (`h₂`); then `next_iff` turns `h₁` into a `Step` and the induction hypothesis turns `h₂` into a `Run`. Right to left, we take the `Run` apart and rebuild the computation with `simp`. The final `example` shows the payoff: to prove a concrete run, we just let `accepts` compute (`rfl`), and `accepts_iff.mp` converts the computation into a proof.

Properties of traces

Now we can prove properties of *all* runs. The first one says that measurements are properly bracketed: along any run, the number of `start`s and `stop`s agree, up to a measurement that may be in progress at either end.

```
/-- 1 in the mode where a measurement is running, 0 elsewhere. -/
def running : Mode → Nat
  | .measuring => 1
  | _ => 0

/-- Along any run, starts and stops balance, up to the measurement that may
be running at the beginning or at the end. -/
theorem run_balance {m m' : Mode} {tr : List Action} (h : Run m tr m') :
    tr.count .start + running m = tr.count .stop + running m' := by
  induction h with
  | nil => simp
  | cons hs hr ih =>
    -- one case per kind of step; `simp_all` unfolds the counts, `omega`
    -- does the arithmetic
    cases hs <;> simp_all [running] <;> omega

/-- A complete session (from idle back to idle) stops every measurement it
starts. -/
theorem complete_balanced {tr : List Action} (h : Run .idle tr .idle) :
    tr.count .start = tr.count .stop := by
  simpa [running] using run_balance h
```

The function `running` is 1 in the measuring mode and 0 elsewhere. The statement of `run_balance` is the *generalized* form of what we want: it holds between any two modes, which is what makes the induction go through. The proof is by induction on the derivation of `Run` (Chapter 4 introduced induction on derivations): in the `nil` case both sides are the same; in the `cons` case, `cases hs` considers each kind of step, and for each one `simp_all` computes the counts of the extended trace from the induction hypothesis, and `omega` checks the arithmetic. For example, a `start` step from `calibrated` to `measuring` adds 1 to the `start` count on the left and moves `running` from 0 to 1 on the right. The corollary `complete_balanced` specializes it to runs from `idle` to `idle`, where `running` is 0 at both ends: a complete session stops every measurement it starts.

The second property: a nonempty run that ends in `idle` ends with `powerDown`, the only action that leads to `idle`.

```
/-- A nonempty run that ends in `idle` ends with `powerDown`: it is the only
action that leads to `idle`. -/
theorem ends_with_powerDown {m m' : Mode} {tr : List Action} (h : Run m tr m')
    (hm : m' = .idle) (hne : tr ≠ []) : tr.getLast? = some .powerDown := by
  induction h with
  | nil => contradiction
  | cons hs hr ih =>
    rename_i m₁ m₂ m₃ a as
    cases as with
    | nil =>
      -- the step `hs` is the last one, and it leads to `idle`
      cases hr
      subst hm
      cases hs
      rfl
    | cons b bs =>
      -- the last action is the last action of the rest of the run
      have := ih hm (by simp)
      simpa [List.getLast?_cons_cons] using this
```

The statement takes the final mode as a variable `m'` with a separate hypothesis `hm : m' = .idle`. That is a standard trick: `induction` on `h` needs the indices of `Run m tr m'` to be variables, and if we wrote `.idle` directly in the type of `h`, Lean would refuse (try it: the error says the index is not a variable). In the `cons` case, `rename_i` names the pieces of the step, and we look at the rest of the trace `as`.

If it is empty, the step was the last one: `cases hr` learns from `Run m₂ [] m₃` that `m₂ = m₃`, `subst hm` makes that mode `idle`, and `cases hs` finds that the only step into `idle` is `powerDown`. If the rest is not empty, the last action of the whole trace is the last action of the rest (`List.getLast?_cons_cons`), and the induction hypothesis gives it.

Typed programs follow the protocol

Finally, we connect the protocol back to typestate. In Rust, a sequence of calls like `probe.calibrate(..).start()` is accepted when each call is allowed in the state the previous one produced. We can describe such sequences as data, a family of types indexed by *two* modes, the start and the end:

```
/-- Programs that drive the instrument from mode `i` to mode `j`. Each
constructor is one operation of the typestate API: `calibrate` turns a
program that starts calibrated into one that starts idle. -/
inductive Prog : Mode → Mode → Type where
  | done {m : Mode} : Prog m m
  | calibrate {m : Mode} : Prog .calibrated m → Prog .idle m
  | start {m : Mode} : Prog .measuring m → Prog .calibrated m
  | sample {m : Mode} : Prog .measuring m → Prog .measuring m
  | stop {m : Mode} : Prog .calibrated m → Prog .measuring m
  | powerDown {m : Mode} : Prog .idle m → Prog .calibrated m

/-- The actions a program performs, in order. -/
def Prog.trace {i j : Mode} : Prog i j → List Action
  | .done => []
  | .calibrate p => .calibrate :: p.trace
  | .start p => .start :: p.trace
  | .sample p => .sample :: p.trace
  | .stop p => .stop :: p.trace
  | .powerDown p => .powerDown :: p.trace

def shift : Prog .idle .idle :=
  .calibrate <| .start <| .sample <| .sample <| .stop <| .powerDown .done

/-- info: [calibrate, start, sample, sample, stop, powerDown] -/
#guard_msgs in
#eval shift.trace
```

`Prog i j` is the type of programs that take the instrument from mode `i` to mode `j`. Each constructor adds one operation at the front: `calibrate` takes a program that starts in `calibrated` and returns one that starts in `idle`. This is the typestate API turned into a data type, with the same discipline: you cannot build a `Prog` that samples before calibrating, because the types do not fit. `trace` lists the actions of a program, and `shift` is one complete session.

The theorem says that the types guarantee protocol compliance:

```
/-- **Typed programs follow the protocol**: the trace of a `Prog i j` is a run
from `i` to `j`. -/
theorem Prog.trace_run {i j : Mode} (p : Prog i j) : Run i p.trace j := by
  induction p with
  | done => exact .nil
  | calibrate p ih => exact .cons .calibrate ih
  | start p ih => exact .cons .start ih
  | sample p ih => exact .cons .sample ih
  | stop p ih => exact .cons .stop ih
  | powerDown p ih => exact .cons .powerDown ih

/-- **Every complete session ends in the final state**, and so the run-time
check accepts its trace with result `idle`, its starts and stops balance, and
```

```
(if it does anything) it ends by powering down. -/
theorem session_complete (p : Prog .idle .idle) :
    accepts .idle p.trace = some .idle ∧
    p.trace.count .start = p.trace.count .stop ∧
    (p.trace ≠ [] → p.trace.getLast? = some .powerDown) :=
  ⟨accepts_iff.mpr p.trace_run,
   complete_balanced p.trace_run,
   ends_with_powerDown p.trace_run rfl⟩
```

`Prog.trace_run` is proved by induction on the program: each constructor corresponds to one `Step` constructor, and `Run.cons` adds it to the run of the rest. Then `session_complete` collects the consequences for a complete session, a `Prog .idle .idle`: the run-time check `accepts` agrees that the trace is allowed and ends in the final state `idle`, every measurement started is stopped, and a session that does anything ends by powering down. The proof is a triple of applications of the theorems above to `p.trace_run`; the anonymous constructor `⟨_, _, _⟩` builds the conjunction.

Compare this with Rust. The function signature `fn shift(i: Instrument<Idle>) -> Instrument<Idle>` *looks* like it demands a complete session, but it does not: the body may drop the measuring instrument half-way and return a brand-new idle one made with `Instrument::new`. A `Prog` has no constructor for "abandon" or "start over", so the type `Prog .idle .idle` really does describe only complete sessions.

Session types in Lean

The same idea gives session types their full strength. In `code/lean/Book/Ch25/Session.lean`, sessions are an inductive type and duality is a function:

```
/-- The types of the messages. -/
inductive Ty where
  | nat | bool
  deriving DecidableEq

/-- The Lean type of a message of type `t`. -/
abbrev Ty.denote : Ty → Type
  | .nat => Nat
  | .bool => Bool

inductive Session where
  | send (t : Ty) (k : Session)   -- send a `t`, continue as `k`
  | recv (t : Ty) (k : Session)   -- receive a `t`, continue as `k`
  | done
  deriving DecidableEq

def Session.dual : Session → Session
  | .send t k => .recv t k.dual
  | .recv t k => .send t k.dual
  | .done => .done
```

Message types are a small inductive `Ty` with a function `Ty.denote` into Lean types, the same technique as the intrinsically typed evaluator of Chapter 8. Duality is an involution, by induction on the session:

```
/-- Duality is an involution: the other side of the other side is you. -/
theorem Session.dual_dual (s : Session) : s.dual.dual = s := by
  induction s with
  | send t k ih => simp [dual, ih]
  | recv t k ih => simp [dual, ih]
  | done => rfl
```

In each `send` and `recv` case, `simp [dual, ih]` unfolds `dual` twice and uses the induction hypothesis for the continuation; `done` is its own dual by definition. Now a process is indexed by the session it follows, and two processes with dual sessions can be run against each other:

```
/-- A process that follows session `s`. Sending needs a value of the right
type; receiving needs a continuation for every value that may arrive. -/
inductive Proc : Session → Type where
  | send {t : Ty} {k : Session} : t.denote → Proc k → Proc (.send t k)
  | recv {t : Ty} {k : Session} : (t.denote → Proc k) → Proc (.recv t k)
  | done : Proc .done

def Ty.show : (t : Ty) → t.denote → String
  | .nat, n => toString n
  | .bool, b => toString b

/-- Run two processes with dual sessions against each other, recording the
messages. There are only three cases: the types rule out every mismatch
(both sending, both receiving, one finished early), and the recursion is
structural, so the conversation always completes. -/
def talk : {s : Session} → Proc s → Proc s.dual → List String
  | .send t _, .send v p, .recv f => s!"→ {t.show v}" :: talk p (f v)
  | .recv t _, .recv f, .send v q => s!"← {t.show v}" :: talk (f v) q
  | .done, .done, .done => []
```

The function `talk` is the most interesting definition of the chapter, because of what it *lacks*. It matches the two processes simultaneously, and there are only three cases. If the first process is sending, its session is `.send t k`, so the second process has type `Proc (.send t k).dual`, which Lean computes to `Proc (.recv t k.dual)`: the second process *must* be receiving, a value of the same type `t`. Lean checks that every combination not listed (both sending, both receiving, one finished while the other is not) is impossible because the types cannot match, so no case is missing. And the recursion is structural, so `talk` terminates. A total function from two dual processes to a transcript is a proof that dual processes always communicate to completion, with no mismatched message and no deadlock.

```
/-- The client's protocol: send a sensor number, receive whether it is
in service. -/
def Query : Session := .send .nat (.recv .bool .done)

def client : Proc Query :=
  .send (7 : Nat) (.recv fun _inService => .done)

/-- The server follows the dual protocol: receive, then send. -/
def server : Proc Query.dual :=
  .recv fun (sensor : Nat) => .send (decide (sensor < 10)) .done

/-- info: ["→ 7", "← true"] -/
#guard_msgs in
#eval talk client server
```

The server is written against `Query.dual`, and its `recv` receives a `Nat` because that is what the dual session says. This is the guarantee the Rust encoding could not give: Lean's totality ensures that the protocol finishes, which is the "at least once" half that affinity does not provide.

25.9 Putting it together: the lab instrument, end to end

Let us line up everything this chapter did with one protocol, the lab instrument.

Concept	Rust	Lean
state	a type: `Idle`, `Calibrated`, `Measuring`	a value of `Mode`
data known in a state	fields of the state type	(not modeled)
allowed transition	a method in an `impl Instrument<S>` block	a constructor of `Step`
wrong-state call	E0599, no method found	no constructor applies
old state reused	E0382, it was moved	allowed: values are immutable
a program	a chain of method calls	a value of `Prog i j`
run-time check	a destructor that complains	`accepts`
complete session	not enforceable	`Prog .idle .idle`, proved complete

Read the rows from top to bottom and the story of the chapter appears. The protocol's diagram becomes types in both languages. In Rust, the types reject operations in the wrong state (E0599), and affinity rejects the reuse of a state that was consumed (E0382), which matters because a Rust value is the resource itself. Rust cannot make a measurement finish: a value may always be dropped. What it offers instead is a run-time complaint, which needs care to write (E0366 and E0509 shape the code), and the closure-based design that keeps the protocol under the library's control. In Lean, the same diagram becomes an inductive relation; typed programs are a data type whose values provably follow the relation; complete sessions provably end in the final state with every measurement stopped; and the runtime checker provably agrees with the relation.

The comparison also says something about the question of this book. Typestate is one of the most practical consequences of Rust's ownership discipline, and it works because ownership is affine: "at most once" is exactly what makes a consumed state unusable. The guarantee it misses, "at least once", is exactly the difference between affine and linear. Chapter 33 collects this observation with the others.

25.10 Common confusions

"**Typestate needs `PhantomData`.**" Only when the state carries no data. The builder and the instrument use their type parameters as the types of real fields, and need no marker. `PhantomData` is for the case where the state is purely a label, as in the connection.

"**With typestate, protocol errors are impossible.**" Calling an operation in the wrong state is impossible. Abandoning the protocol half-way is not, and when a transition depends on run-time data (a login that may fail), the outcome comes back as a `Result` or an enum of states that must be handled at run time.

"**A sealed trait is a security feature.**" It is an API design tool. It prevents other crates from implementing the trait, so that the set of states stays closed and the library can rely on knowing all of them. It has nothing to do with access control at run time.

"**`#[must_use]` on a typestate type makes the protocol linear.**" It warns when a transition's result is discarded on the spot, which catches a common slip. It does nothing about a value that is bound, kept, and dropped later in the wrong state.

"**Lean's typestate is weaker because it is not affine.**" It is different. Lean values are immutable, so reusing an old state is not a lie, and Lean's types can mention run-time values (the vault) and its totality can guarantee completion (the session types), which Rust cannot express.

25.11 Summary

- Typestate turns a protocol's states into types and its transitions into functions between them; calling an operation in the wrong state becomes a type error (E0599 in Rust), checked before the program runs.
- In Rust, transitions take `self` by value. Affinity makes the old state unusable (E0382); without it, two views of one resource could disagree.
- The toolkit: empty enums or unit structs as state labels, `PhantomData` when the state is only a label (E0392 otherwise), sealed traits to close the set of states (E0277 for outsiders), trait bounds for groups of states, states that carry the data known in them, `Result` for fallible transitions.
- Builders with required fields use one type parameter per field; generic `impl` blocks allow any order, and `build` exists only when everything is set.
- Session types describe conversations; duality (! and ? swapped) is an involution, and dual endpoints match message for message. A Rust encoding guarantees no wrong message, but not that the conversation finishes.
- Affinity cannot force completion. `#[must_use]` warns about ignored results; a complaining destructor reports abandonment at run time, but must live on a type without parameters or be generic (E0366) and must be disarmed by the normal transitions (E0509); closure-based APIs keep the protocol under control.
- Lean expresses typestate with indexed families (`Conn : ConnState → Type`), lets states depend on run-time values (a vault indexed by its balance, with a proof obligation for withdrawals and dependent `if` and pairs for run-time amounts), and describes protocols as inductive relations.
- We proved that a run-time checker agrees with the protocol relation, that runs from `idle` to `idle` stop every measurement and end with `powerDown`, that typed programs follow the protocol, and that dual Lean processes always talk to completion.

25.12 Exercises

1. Design a `ServerBuilder` whose `build()` method exists only after both `name` and `port` have been set, in either order, with an optional `max_connections` available in every state. How many type parameters does it need, and how many distinct builder types can occur? *Hint:* follow `RequestBuilder`: one parameter per required field, with a "missing" marker and a "present" type for each; four types in all.
2. Add a `Maintenance` mode to the instrument, reachable from `Idle` by `service` and returning to `Idle` by `release`. Update the Rust module, the Lean `Mode`, `Action`, `Step`, `next` and `Prog`, and check which proofs still go through unchanged. *Hint:* `next_iff` and `accepts_iff` need no new

ideas, but the automation in `next_iff` now has more cases; `ends_with_powerDown` is now *false*. Why? State and prove the corrected version.

3. Prove in `Protocol.lean` that every complete run calibrates as often as it powers down: `Run .idle tr .idle → tr.count .calibrate = tr.count .powerDown`. *Hint:* generalize as in `run_balance`, with a function that is 0 in `idle` and 1 in the two other modes.
4. Write the closure-based `measure` method sketched in "Keeping control" for `ch25_typestate.rs`, with a `Recorder` type that only allows recording samples. Then explain why a caller can no longer abandon a measurement, and the one way in which the measurement can still be interrupted. *Hint:* `Recorder<'a>(&'a mut Instrument<Measuring>)` with a single `record` method; the interruption is a panic inside the closure.
5. Extend `Session.lean` with *choice*: a constructor `offer (k₁ k₂ : Session)` (the other side chooses a branch) and `choose (k₁ k₂ : Session)` (this side chooses), dual to each other. Extend `dual`, `dual_dual`, `Proc` and `talk`. *Hint:* a `Proc (.choose k₁ k₂)` is a choice of one branch with a process for it (a `Bool` and a dependent `if`, or two constructors); a `Proc (.offer k₁ k₂)` needs a process for each branch.
6. In `Typestate.lean`, prove that `tryWithdraw v amount` returns `none` exactly when `b < amount`. *Hint:* unfold `tryWithdraw` and use `split` (or `by_cases h : amount ≤ b`) to follow the dependent `if`; `simp` and `omega` handle the rest.

PART VI

Borrowing and Regions

Part VI is about the other half of Rust's discipline: references. Affinity alone cannot explain borrowing (a shared reference can be copied freely), so we turn to regions, permissions and subtyping. We build a small calculus with ownership, lifetimes, shared and mutable borrows and reborrowing, check it in Rust and formalize it in Lean, and then look at how the real compiler does it: not by building derivations, but by dataflow analysis over a control-flow graph.

CHAPTER 26

References, Aliasing and Regions

Part V gave Rust's ownership an exact account. A value of a non-`Copy` type is an affine resource: it may be used at most once, so a move is the absence of contraction, `Copy` is contraction restored for a class of types, and `Drop` is weakening with a side effect. The affine λ-calculus of Chapter 22 made this precise, and its usage theorem proved it. That account is correct, and it explains a great deal of Rust.

It does not explain a single `&`. Yet references are what you write most, and almost every borrow-checker error you have fought was about a reference, not a move. This chapter asks what goes wrong when a language hands out pointers, why the affine account cannot be stretched to cover them, and what does cover them: **regions**, which is what lifetimes are, and **permissions**, which is what `&` and `&mut` are.

This is where the answer to the book's question starts to take shape. Chapter 1 asked whether Rust's ownership and lifetimes are "based on affine types". By the end of this chapter you will see why the honest answer has two halves, and why the second half is a different theory altogether.

In this chapter you will learn:

- the two ways a reference can go wrong (outliving its referent, and aliasing combined with mutation), and which compiler errors rule out each;
- why affine typing cannot explain borrowing: `&T` is `Copy`, and a duplicable pointer type is unsound without extra apparatus;
- what region calculi are (Tofte and Talpin, Cyclone), and how lifetimes, outlives constraints, the escape check, elision and `'static` fit them;
- the permission reading of references: a shared reference is a duplicable read permission, a mutable reference a unique write permission, a borrow a temporary transfer of permission;
- how interior mutability moves the aliasing check from compile time to run time;
- how to define a heap language in Lean whose semantics gets stuck on a use after free, and how to prove that a simple region discipline rules out every stuck state.

26.1 What goes wrong with references

A reference is a pointer with rules. In C a pointer has no rules at all: it can be copied anywhere, and nothing stops you from following it after the memory behind it has been freed. Rust's references are

the same machine pointers, and their rules are the subject of this chapter. First, let us collect the bugs the rules prevent. Each program below is a file in `code/rust/compile_fail/`.

Outliving the referent

The first family of bugs is *temporal*: a reference is used after the thing it points to has gone away.

```
pub fn dangling() {
    let r;
    {
        let reading = 42;
        r = &reading;
    }
    println!("{r}");
}
```

```
error[E0597]: `reading` does not live long enough
  --> compile_fail/ch26_dangling.rs:10:13
   |
 9 |         let reading = 42;
   |             ------- binding `reading` declared here
10 |         r = &reading;
   |             ^^^^^^^^ borrowed value does not live long enough
11 |     }
   |     - `reading` dropped here while still borrowed
12 |     println!("{r}");
   |                - borrow later used here
```

Read the three labels in order. The borrow `&reading` creates a reference to a local variable; at the closing brace the variable is dropped, its stack slot is dead; and the reference is used afterward. In C the equivalent program compiles and prints whatever happens to be in that stack slot. The principle the compiler enforces is simple to state: **no reference outlives its referent**. Error E0597 ("does not live long enough") is its most common form.

The same principle at a function boundary:

```
pub fn shout(text: &str) -> &str {
    let loud = text.to_uppercase();
    &loud
}
```

```
error[E0515]: cannot return reference to local variable `loud`
 --> compile_fail/ch26_return_local_ref.rs:8:5
  |
8 |     &loud
  |     ^^^^^ returns a reference to data owned by the current function
```

`loud` is a `String` owned by `shout`; when `shout` returns, `loud` is dropped and its heap buffer freed. A reference into it would dangle the moment the caller received it. Error E0515 is the compiler noticing that the returned reference *escapes* the region in which its referent exists, a phrase that will become precise later in this chapter.

The third temporal bug is the one we will formalize in Lean. A box is allocated, a second pointer to its contents is made, the box is freed through the first pointer, and the contents are read through the second:

```
pub fn use_after_free() -> i32 {
    let x = Box::new(42);
    let y = &*x;
    drop(x);
    *y
}
```

```
error[E0505]: cannot move out of `x` because it is borrowed
  --> compile_fail/ch26_free_while_borrowed.rs:9:10
   |
 7 |     let x = Box::new(42);
   |         - binding `x` declared here
 8 |     let y = &*x;
   |             --- borrow of `*x` occurs here
 9 |     drop(x);
   |          ^ move out of `x` occurs here
10 |     *y
   |     -- borrow later used here
   |
help: consider cloning the value if the performance cost is acceptable
   |
 8 -     let y = &*x;
 8 +     let y = &x.clone();
   |
```

`drop(x)` frees the heap cell, and `*y` would read it afterward: a classic use after free *through an alias*. Rust does not let the owner `x` give the box away (here to `drop`, which frees it) while a borrow of it is still going to be used. The help message suggests cloning, which would give `y` its own copy and break the aliasing.

Aliasing and mutation

The second family is *spatial*: two paths to the same memory, at least one of which writes.

```
pub fn first_then_push(orders: &mut Vec<u32>) {
    let first = &orders[0];
    orders.push(99);
    println!("first order: {first}");
}
```

```
error[E0502]: cannot borrow `*orders` as mutable because it is also borrowed as immutable
 --> compile_fail/ch26_shared_then_mut.rs:8:5
  |
7 |     let first = &orders[0];
  |                  ------ immutable borrow occurs here
8 |     orders.push(99);
  |     ^^^^^^^^^^^^^^^ mutable borrow occurs here
9 |     println!("first order: {first}");
  |                             ----- immutable borrow later used here
```

Why is this dangerous? A `Vec` keeps its elements in a heap buffer. `push` may find the buffer full, allocate a bigger one, move the elements, and free the old buffer. `first` points into the old buffer. So `first` would dangle, but nothing in the program frees anything explicitly: the free is hidden inside `push`. The compiler cannot know whether this particular `push` reallocates, and it does not try. It enforces a rule that covers every case: **while a shared reference to a place is in use, the place is not mutated**. Error E0502 is that rule.

The most famous instance has a name, *iterator invalidation*:

```
pub fn repeat_large(orders: &mut Vec<u32>) {
    for amount in orders.iter() {
        if *amount > 100 {
            orders.push(*amount);
        }
    }
}
```

```
error[E0502]: cannot borrow `*orders` as mutable because it is also borrowed as immutable
  --> compile_fail/ch26_iterator_invalidation.rs:9:13
   |
 7 |       for amount in orders.iter() {
   |       -             -------------
   |       |             |
   |       |             immutable borrow occurs here
   |  _____|             immutable borrow later used here
   | |
 8 | |         if *amount > 100 {
 9 | |             orders.push(*amount);
   | |             ^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
10 | |         }
11 | |     }
   | |_____- this for loop borrows `*orders` immutably, preventing mutation within its body
   |
   = help: consider using an index-based loop instead, or collecting modifications into a
separate collection
```

The iterator holds a pointer into the buffer (and a pointer to its end) for the whole loop. A `push` in the body may reallocate the buffer, and the next iteration would read freed memory. In C++ this is undefined behavior; Java's collections detect it at run time, on a best-effort basis, by throwing `ConcurrentModificationException`. Rust rejects it before the program runs, and the diagnostic even points out that the `for` loop is the borrow.

The dual rule forbids two writers:

```
pub fn transfer(accounts: &mut Vec<i64>, amount: i64) {
    let from = &mut accounts[0];
    let to = &mut accounts[1];
    *from -= amount;
    *to += amount;
}
```

```
error[E0499]: cannot borrow `*accounts` as mutable more than once at a time
 --> compile_fail/ch26_two_mut.rs:8:19
  |
7 |     let from = &mut accounts[0];
  |                -------- first mutable borrow occurs here
8 |     let to = &mut accounts[1];
  |                   ^^^^^^^^ second mutable borrow occurs here
9 |     *from -= amount;
  |     --------------- first borrow later used here
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

Index 0 and index 1 are different elements, so this particular program would be harmless. But `accounts[0]` is a call to `IndexMut::index_mut(&mut accounts, 0)`, which borrows the *whole* vector, and the checker reasons about the vector as a single place. The rule it enforces is: **a mutable reference to a place excludes every other reference to that place**. Error E0499 is that rule, and the `split_at_mut` that the help message suggests is the standard way to tell the compiler that two mutable references cover disjoint parts.

The same rule, applied across threads, rules out *data races*. A data race is two unsynchronized accesses to the same memory from different threads, at least one of them a write, and it is undefined behavior in C, C++ and unsafe Rust alike:

```
use std::thread;

pub fn race() {
    let mut counter = 0;
    let first = &mut counter;
    let second = &mut counter;
    thread::scope(|s| {
        s.spawn(move || *first += 1);
        s.spawn(move || *second += 1);
    });
}
```

```
error[E0499]: cannot borrow `counter` as mutable more than once at a time
  --> compile_fail/ch26_data_race.rs:11:18
   |
10 |     let first = &mut counter;
   |                 ------------ first mutable borrow occurs here
11 |     let second = &mut counter;
   |                  ^^^^^^^^^^^^ second mutable borrow occurs here
12 |     thread::scope(|s| {
13 |         s.spawn(move || *first += 1);
   |                         ------ first borrow later captured here by closure
```

The two threads would each increment `counter` through their own `&mut`, with no synchronization. The compiler does not need a separate theory of threads to reject it: two simultaneous `&mut` to one place is already an error. (The traits `Send` and `Sync` decide which types may cross threads at all; together with the aliasing rule they give Rust's "fearless concurrency".)

Two principles

Every error above enforces one of two principles:

Program	Error	Principle
`ch26_dangling`	E0597	no reference outlives its referent
`ch26_return_local_ref`	E0515	the same, at a function boundary
`ch26_free_while_borrowed`	E0505	the owner cannot free while lent
`ch26_shared_then_mut`	E0502	no mutation while shared
`ch26_iterator_invalidation`	E0502	the same, for a loop-long borrow
`ch26_two_mut`	E0499	at most one writer
`ch26_data_race`	E0499	at most one writer, across threads

The first is about *time*: how long a reference may be used. The second is the slogan **aliasing XOR mutation**: at any moment, a place may have many readers or one writer, never both. Repaired versions of these programs are in `code/rust/examples/ch26_aliasing.rs`; two of them show how small the fix usually is:

```
/// The fix for `ch26_shared_then_mut`: copy the value out, so that the shared
/// borrow ends (at its last use) before the vector is mutated.
fn first_then_push(orders: &mut Vec<u32>) {
```

```
    let first = orders[0]; // a `u32` copied out of the vector, not a reference
    orders.push(99);
    println!("first order: {first}, now {} orders", orders.len());
}

/// The fix for `ch26_iterator_invalidation`: read everything first, then
/// mutate. The shared borrow of the loop ends before `extend` begins.
fn repeat_large(orders: &mut Vec<u32>) {
    let large: Vec<u32> = orders.iter().copied().filter(|&a| a > 100).collect();
    orders.extend(large);
    println!("orders after repeating the large ones: {orders:?}");
}
```

```
/// The fix for `ch26_two_mut`: `split_at_mut` turns one unique permission on
/// the whole slice into two unique permissions on disjoint halves.
fn transfer(accounts: &mut [i64], amount: i64) {
    let (left, right) = accounts.split_at_mut(1);
    let from = &mut left[0];
    let to = &mut right[0];
    *from -= amount;
    *to += amount;
}

/// The fix for `ch26_data_race`: two threads, two disjoint unique permissions.
fn count_in_parallel() -> [u32; 2] {
    let mut counters = [0u32; 2];
    let (first, second) = counters.split_at_mut(1);
    thread::scope(|s| {
        s.spawn(|| first[0] += 10);
        s.spawn(|| second[0] += 20);
    });
    counters
}
```

```
first order: 120, now 4 orders
orders after repeating the large ones: [120, 80, 300, 99, 120, 300]
accounts after transfer: [70, 80]
counters from two threads: [10, 20]
```

In `first_then_push` the value is copied out of the vector, so no reference is alive when `push` runs. In `transfer` and `count_in_parallel`, `split_at_mut` splits one mutable reference to the whole slice into two mutable references to two halves that cannot overlap.

26.2 Why affinity is not enough

It is natural to hope that references can be explained like moves, perhaps with a cleverer affine system. This section shows why that hope fails, first in Rust and then formally.

`&T` is `Copy`

Recall from Chapter 23 that `Copy` is free contraction: a variable of a `Copy` type may be used any number of times. And `&T` is `Copy`, for *every* `T`:

```
/// Contraction, available only for `Copy` types: the value is used twice.
fn duplicate<T: Copy>(x: T) -> (T, T) {
    (x, x)
}

fn references_are_copy() {
    let report = String::from("sensor report");
```

```
    // `String` is not `Copy`, but `&String` is: a shared reference can be
    // duplicated freely, and both copies point to the same string.
    let (r1, r2) = duplicate(&report);
    println!(
        "r1 = {r1:?}, r2 = {r2:?}, same address: {}",
        std::ptr::eq(r1, r2)
    );
    // `duplicate(&mut report)` would not compile: `&mut String` is not `Copy`.
}
```

```
r1 = "sensor report", r2 = "sensor report", same address: true
```

`duplicate` uses its argument twice, which is contraction, and its `T: Copy` bound makes that legal. Called with `&report` it returns two shared references to the same `String`. So a shared reference is a *fully structural* value: it can be copied (contraction), ignored (weakening) and used in any order (exchange). An affine type system says nothing at all about such a value. Whatever rejects the programs of the previous section, it is not a restriction on how many times a `&T` is used.

A mutable reference, in contrast, is not `Copy`. It behaves exactly like an owned value under the affine rules:

```
pub fn two_writers(log: &mut String) {
    let first = log;
    let second = log;
    *first = String::from("a");
    *second = String::from("b");
}
```

```
error[E0382]: use of moved value: `log`
 --> compile_fail/ch26_mut_ref_moves.rs:8:18
  |
6 | pub fn two_writers(log: &mut String) {
  |                    --- move occurs because `log` has type `&mut String`, which does not
implement the `Copy` trait
7 |     let first = log;
  |                 --- value moved here
8 |     let second = log;
  |                  ^^^ value used here after move
```

This is error E0382 from Chapter 6, word for word: `&mut String` is moved by `let first = log`, and `log` cannot be used again. (In everyday code you rarely see this error, because the compiler silently *reborrows* a `&mut` when you pass it to a function: `f(log)` means `f(&mut *log)`. An assignment without a type annotation does not reborrow, which is why the program above moves.)

So `&mut T` is affine. Does that explain E0499? Look again at `ch26_two_mut`: `from` and `to` are two *different* variables, each used exactly once. An affine checker that counts uses of variables would accept the program. The conflict is not between two uses of one variable but between two values that point to the same place, and an affine type system cannot see that, because it counts uses of variables, not what they point to.

The dangling-reference term

The same observation can be made formally. Take the affine λ-calculus of Chapter 22 and add a heap: a type $Box\,\tau$ of owned heap cells, with

$$new : \tau \multimap Box\,\tau \qquad free : Box\,\tau \multimap \tau$$

Nothing new is needed so far. $\mathit{Box}\,\tau$ is an affine type, ownership of the cell travels with the variable that holds it, and the usage theorem of Chapter 22 guarantees that each cell is freed at most once. This is Rust's `Box<T>`.

Now add references: a type $\mathit{Ref}\,\tau$ of pointers that may read a cell but do not own it. Reading should not consume anything, so references should be duplicable (like `&T`), and taking a reference should not consume the box. The natural rules are

$$\frac{(b : \mathit{Box}\,\tau) \in \Gamma}{\Gamma \vdash \mathit{borrow}\; b : \mathit{Ref}\,\tau}\ \text{T-Borrow} \qquad\qquad \frac{\Gamma \vdash e : \mathit{Ref}\,\tau}{\Gamma \vdash \mathit{read}\; e : \tau}\ \text{T-Read}$$

with $\mathit{Ref}\,\tau$ declared `Copy` and τ a `Copy` type in T-Read. The premise of T-Borrow is a membership test, not a context split: b stays available after it is borrowed, exactly as `x` stays available after `&x` in Rust. Every rule is affine-respectable. And yet the term

$$\texttt{let}\; b = \mathit{new}\; 42\; \texttt{in let}\; r = \mathit{borrow}\; b\; \texttt{in let}\; _ = \mathit{free}\; b\; \texttt{in}\; \mathit{read}\; r$$

is well typed: b is consumed once (by free), r is used once, and nothing is duplicated that is not `Copy`. It reads freed memory.

There are exactly two ways out, and Rust uses both:

1. **Lend and return.** Make *borrow consume* the box, and give it back at the end of a delimited scope. Ownership is then never shared, only lent: during the scope the reference exists and the owner does not; after it, the owner exists again and the reference does not. Chapter 28 builds a calculus around this idea.
2. **Regions.** Index every reference type by a *region*, a name for the stretch of the program during which the reference may be used, and require that no reference is used outside its region.

Neither is about how many times a variable is used. The first needs a scope and a side condition that the reference does not leak out of it; the second needs a new sort of name in the types. Both are about *time*.

> **IMPORTANT**
>
> Affinity is a statement about **multiplicity**: how many times a variable may be used. Lifetimes are a statement about **extent**: during which part of the program a reference is valid. They are orthogonal mechanisms, which Rust happens to check in the same pass. No amount of affine typing produces lifetimes.

26.3 A heap language that goes wrong

Before we look at regions, we make the problem precise. The Lean file `code/lean/Book/Ch26/Heap.lean` defines a tiny imperative language with a heap, gives it a semantics that gets stuck on a use after free, and proves that a program goes wrong. It is deliberately small: no loops, no functions, only straight-line code, so that the proofs stay short and every idea is visible.

Syntax and states

```
/-- What a variable holds: a number, or the address of a heap cell. -/
inductive Val where
  | num (n : ℕ)
  | addr (l : ℕ)
  deriving DecidableEq, Repr

/-- The commands of the language. A program is a list of commands, run from
left to right. -/
inductive Cmd where
  | alloc (x : Var) (n : ℕ)  -- x := new n     a fresh cell holding n
  | copy (y x : Var)         -- y := x         y now holds what x holds
  | read (y x : Var)         -- y := *x        read the cell x points to
  | write (x : Var) (n : ℕ)  -- *x := n        write the cell x points to
  | free (x : Var)           -- free x         deallocate the cell
  deriving Repr

/-- A program is a list of commands. -/
abbrev Prog := List Cmd
```

A variable holds either a number or the address of a heap cell. The five commands correspond to familiar Rust operations: `alloc x n` is `let x = Box::new(n)`, `copy y x` makes `y` a second pointer to whatever `x` points to (like copying a `&T`), `read` and `write` follow a pointer, and `free x` deallocates the cell, as `drop(x)` does for a box. Variables are strings and addresses are natural numbers.

```
/-- The machine state: the variables, the heap and the next fresh address.
A cell that was never allocated, or has been freed, holds `none`. -/
structure State where
  env : Var → Option Val
  heap : ℕ → Option ℕ
  next : ℕ

/-- The empty state: no variables, no cells. -/
def State.init : State where
  env := fun _ => none
  heap := fun _ => none
  next := 0
```

The state has three parts. `env` gives each variable its value (or `none` if it has none yet), `heap` gives each address its contents (or `none` if the cell was never allocated or has been freed), and `next` is the next address that `alloc` will hand out. Both `env` and `heap` are functions, which is the simplest way to model a finite map in a proof; the price is that a `State` cannot be printed, which is why later we print only some variables.

The `where` syntax after `def State.init : State` builds a structure value field by field. It is the Lean counterpart of a Rust struct literal.

The step relation

The semantics is a small-step relation, as in Chapter 9. A *configuration* is a pair of the commands still to run and the current state. Written as rules, the cases for reading and freeing are

$$\frac{s.env(x) = \mathit{addr}\ \ell \qquad s.heap(\ell) = n}{\langle \texttt{read}\ y\ x :: k, s\rangle \to \langle k, s[env(y) := \mathit{num}\ n]\rangle}\ \text{E-Read}$$

$$\frac{s.env(x) = \mathit{addr}\ \ell \qquad s.heap(\ell) = m}{\langle \texttt{free}\ x :: k, s\rangle \to \langle k, s[heap(\ell) := \mathit{none}]\rangle}\ \text{E-Free}$$

Each premise is a *requirement*: to read through `x`, `x` must hold an address, and the cell at that address must be allocated. If a requirement fails, the rule does not apply. In Lean:

```
/-- One step of execution. A configuration is the program still to run
together with the current state. -/
inductive Step : Prog × State → Prog × State → Prop where
  | alloc {x : Var} {n : ℕ} {k : Prog} {s : State} :
      Step (.alloc x n :: k, s)
        (k, { env := update s.env x (some (.addr s.next))
              heap := update s.heap s.next (some n)
              next := s.next + 1 })
  | copy {y x : Var} {v : Val} {k : Prog} {s : State}
      (hx : s.env x = some v) :
      Step (.copy y x :: k, s) (k, { s with env := update s.env y (some v) })
  | read {y x : Var} {l n : ℕ} {k : Prog} {s : State}
      (hx : s.env x = some (.addr l)) (hl : s.heap l = some n) :
      Step (.read y x :: k, s) (k, { s with env := update s.env y (some (.num n)) })
  | write {x : Var} {n l m : ℕ} {k : Prog} {s : State}
      (hx : s.env x = some (.addr l)) (hl : s.heap l = some m) :
      Step (.write x n :: k, s) (k, { s with heap := update s.heap l (some n) })
  | free {x : Var} {l m : ℕ} {k : Prog} {s : State}
      (hx : s.env x = some (.addr l)) (hl : s.heap l = some m) :
      Step (.free x :: k, s) (k, { s with heap := update s.heap l none })
```

Two pieces of notation are new. `update f a v` is Mathlib's `Function.update`: the function that agrees with `f` everywhere except at `a`, where it returns `v` (the file says `open Function (update)` so that we can write it without the prefix). And `{ s with env := e }` is *structure update*: a copy of `s` with the field `env` replaced, like Rust's `State { env: e, ..s }`.

Notice what the rules do *not* say. There is no rule for reading a freed cell, or for freeing it twice. When a program reaches such a command, no rule applies: the program is **stuck**, which is our formal version of undefined behavior.

```
/-- Zero or more steps: Mathlib's reflexive-transitive closure of `Step`. -/
abbrev Steps := Relation.ReflTransGen Step

/-- A configuration is stuck when there is still a command to run but no
rule of `Step` applies to it. -/
def Stuck (c : Prog × State) : Prop := c.1 ≠ [] ∧ ¬ ∃ c', Step c c'

/-- A program goes wrong if, started in the empty state, it can reach a stuck
configuration. -/
def GoesWrong (p : Prog) : Prop := ∃ c, Steps (p, State.init) c ∧ Stuck c
```

`Relation.ReflTransGen` is Mathlib's reflexive-transitive closure: `Steps c c'` holds when `c'` can be reached from `c` in zero or more steps. (Chapter 9 defined its own `Steps`; here we use the library's.) A configuration is `Stuck` if it still has commands to run but no step is possible, and a program `GoesWrong` if it can reach a stuck configuration from the empty state.

Three programs

```
/-- Two pointers to one cell, both used to read it, then the cell is freed.
A perfectly safe program. -/
def share : Prog :=
  [.alloc "x" 42, .copy "y" "x", .read "a" "x", .read "b" "y", .free "x"]

/-- The same aliasing, but the cell is freed *before* the alias is read. -/
def uaf : Prog :=
  [.alloc "x" 42, .copy "y" "x", .free "x", .read "z" "y"]
```

```
/-- Freeing the same cell twice, once through each pointer. -/
def doubleFree : Prog :=
  [.alloc "x" 1, .copy "y" "x", .free "x", .free "y"]
```

`share` makes two pointers to one cell, reads through both and frees the cell: fine. `uaf` is the Rust program `ch26_free_while_borrowed` in our language: the cell is freed through `x` and then read through `y`. `doubleFree` frees the same cell through each of its two pointers.

To run programs, the file also defines an interpreter, `exec` for one command and `run` for a program, that returns either the final state or a `Fault` explaining which requirement failed: an unassigned variable, a number used as an address, or a `dangling` access to a cell that is not allocated. (`Except` is Lean's `Result`, as Chapter 2 explained.) Here is what happens:

```
/-- info: Except.ok [("a", some (Ch26.Val.num 42)), ("b", some (Ch26.Val.num 42))] -/
#guard_msgs in
#eval observe ["a", "b"] (run share State.init)

/-- info: Except.error (Ch26.Fault.dangling "y" 0) -/
#guard_msgs in
#eval observe ["z"] (run uaf State.init)

/-- info: Except.error (Ch26.Fault.dangling "y" 0) -/
#guard_msgs in
#eval observe [] (run doubleFree State.init)
```

`observe` picks out the values of the listed variables from the final state. The safe program ends with `a` and `b` both holding 42. The other two fail with the same fault: `y` points to cell 0, which is no longer allocated.

Proving that a program goes wrong

An interpreter is a program, and a program can have bugs. To make sure ours says the same thing as the step relation, the file proves two theorems. The first says that a fault really is a stuck configuration:

```
/-- A command on which `exec` reports a fault is stuck: no rule applies. -/
theorem exec_error {s : State} {c : Cmd} {e : Fault} (k : Prog)
    (h : exec s c = .error e) : ¬ ∃ c', Step (c :: k, s) c' := by
  rintro ⟨c', hstep⟩
  -- Which rule could have fired? Each one contradicts the fault.
  cases hstep with
  | alloc => simp [exec] at h
  | copy hx => simp [exec, hx] at h
  | read hx hl =>
    simp only [exec] at h
    split at h
    · rename_i e' hd
      exact deref_error hd hx hl
    · cases h
  | write hx hl =>
    simp only [exec] at h
    split at h
    · rename_i e' hd
      exact deref_error hd hx hl
    · cases h
  | free hx hl =>
    simp only [exec] at h
    split at h
    · rename_i e' hd
      exact deref_error hd hx hl
    · cases h
```

If `exec` reports a fault on command `c`, then `(c :: k, s)` cannot step. The proof assumes a step, `rintro ⟨c', hstep⟩`, and derives a contradiction. `cases hstep` is *inversion*, as in Chapter 9: one goal per rule of `Step` that could have produced `hstep`, each with that rule's premises as hypotheses. For `alloc`, `exec` never fails, so `simp [exec] at h` turns `h` into a contradiction. For `copy`, the premise `hx` says that `x` has a value, and `simp [exec, hx] at h` shows that `exec` would have succeeded. The other three commands follow a pointer with a helper `deref`, and `split at h` splits the `match` on its result into its two branches. In the error branch, `hd` says that `deref` failed, while the rule's premises `hx` and `hl` say that `x` points to an allocated cell; the lemma `deref_error` (in the same file) shows that these cannot both hold. In the success branch, `h` equates `.ok` with `.error`, and `cases h` closes the goal because the constructors differ. (`rename_i` names hypotheses that Lean introduced without a name.)

The second pair of theorems lifts this to whole programs:

```
/-- A successful run is a sequence of steps that ends with nothing left to do. -/
theorem run_ok {p : Prog} {s s' : State} (h : run p s = .ok s') :
    Steps (p, s) ([], s') := by
  induction p generalizing s with
  | nil =>
    simp only [run, Except.ok.injEq] at h
    subst h
    exact Relation.ReflTransGen.refl
  | cons c k ih =>
    simp only [run] at h
    split at h
    · cases h
    · rename_i s₁ hc
      exact Relation.ReflTransGen.head (exec_ok k hc) (ih h)

/-- A run that reports a fault has reached a stuck configuration. -/
theorem run_error {p : Prog} {s : State} {e : Fault} (h : run p s = .error e) :
    ∃ c, Steps (p, s) c ∧ Stuck c := by
  induction p generalizing s with
  | nil => simp [run] at h
  | cons c k ih =>
    simp only [run] at h
    split at h
    · -- The first command itself is stuck.
      rename_i e' hc
      exact ⟨(c :: k, s), Relation.ReflTransGen.refl,
        List.cons_ne_nil c k, exec_error k hc⟩
    · -- The first command runs; the fault comes later.
      rename_i s₁ hc
      obtain ⟨c', hsteps, hstuck⟩ := ih h
      exact ⟨c', Relation.ReflTransGen.head (exec_ok k hc) hsteps, hstuck⟩
```

Both are proved by induction on the program, `generalizing s` because the state changes from one command to the next. In the `cons` case of `run_error`, `split at h` asks whether the *first* command faulted. If it did, the stuck configuration is the current one, reached in zero steps (`ReflTransGen.refl`), and `exec_error` shows it is stuck. If it did not, `exec_ok` (a companion lemma in the file) gives one step, the induction hypothesis gives the rest, and `ReflTransGen.head` glues the first step onto the remaining ones.

With these, proving that `uaf` goes wrong takes one line:

```
/-- The use after free goes wrong. -/
theorem uaf_goes_wrong : GoesWrong uaf := run_error (e := .dangling "y" 0) rfl

/-- So does the double free. -/
```

```
theorem doubleFree_goes_wrong : GoesWrong doubleFree :=
  run_error (e := .dangling "y" 0) rfl

/-- The safe program runs to completion. -/
theorem share_finishes : ∃ s, Steps (share, State.init) ([], s) :=
  ⟨_, run_ok rfl⟩
```

The argument `rfl` is a proof that `run uaf State.init` equals `.error (.dangling "y" 0)`: Lean evaluates the interpreter during type checking and finds that both sides are the same. This is a pattern worth remembering: *a verified interpreter turns a computation into a proof*.

Two disciplines that fail

Now we can test the claims of the previous section on a real language. The file `code/lean/Book/Ch26/Regions.lean` starts with two static checks that do not use regions.

The first tracks *variables*: after `free x`, the variable `x` is dead, and using it is an error. Pointers may be copied freely, as `&T` may.

```
/-- Track *variables*: `free x` marks `x` as dead, and a dead variable may not
be used again. Pointers can be copied freely, like Rust's `&T`. -/
def naiveCheck (dead : List Var) : Prog → Bool
  | [] => true
  | .alloc x _ :: k => naiveCheck (dead.erase x) k
  | .copy y x :: k => !dead.contains x && naiveCheck (dead.erase y) k
  | .read y x :: k => !dead.contains x && naiveCheck (dead.erase y) k
  | .write x _ :: k => !dead.contains x && naiveCheck dead k
  | .free x :: k => !dead.contains x && naiveCheck (x :: dead) k

/-- The naive check accepts the use after free... -/
theorem naive_accepts_uaf : naiveCheck [] uaf = true := rfl

/-- ...so it does not guarantee safety. -/
theorem naive_unsound : ¬ ∀ p, naiveCheck [] p = true → ¬ GoesWrong p :=
  fun h => h uaf naive_accepts_uaf uaf_goes_wrong
```

`naive_accepts_uaf` is proved by `rfl`: Lean runs the check and gets `true`. The check looks reasonable, and it does catch `free x; read y x`. But `uaf` reads through `y`, and `y` was never freed *by name*, so the check is satisfied. Combined with `uaf_goes_wrong`, this gives `naive_unsound`: the check accepts a program that goes wrong. Tracking names cannot work when one cell has many names.

The second check treats pointers as affine resources, as `Box<T>` is: copying a pointer moves it, so the source variable becomes dead.

```
/-- Treat pointers as affine resources, like Rust's `Box<T>`: `copy y x`
*moves* the pointer from `x` to `y`, so `x` is dead afterwards. -/
def affineCheck (dead : List Var) : Prog → Bool
  | [] => true
  | .alloc x _ :: k => affineCheck (dead.erase x) k
  | .copy y x :: k => !dead.contains x && affineCheck ((x :: dead).erase y) k
  | .read y x :: k => !dead.contains x && affineCheck (dead.erase y) k
  | .write x _ :: k => !dead.contains x && affineCheck dead k
  | .free x :: k => !dead.contains x && affineCheck (x :: dead) k

/-- The affine check rejects the use after free... -/
theorem affine_rejects_uaf : affineCheck [] uaf = false := rfl

/-- ...but it also rejects `share`, which runs safely (`share_finishes`). -/
theorem affine_rejects_share : affineCheck [] share = false := rfl
```

This one rejects `uaf`: after `copy "y" "x"` the variable `x` is dead, so `free "x"` is an error. But it also rejects `share`, which runs perfectly well, because `share` reads through `x` after copying it. Here is the dilemma of the previous section, now as two theorems about a real language: **duplicable pointers are unsafe without more information; affine pointers are safe but cannot be shared.** References in Rust are shared and safe, so they need more information. That information is the region.

26.4 Regions

Tofte and Talpin, and Cyclone

Region-based memory management was developed in the 1990s by Mads Tofte and Jean-Pierre Talpin, for Standard ML. Their idea was to divide memory into *regions* that are allocated and freed as a whole, in a stack discipline: a construct `letregion` ρ `in` e creates a fresh region ρ, evaluates e, and then frees ρ with everything in it. Values are allocated "at" a region, and every type that can point into memory carries the name of the region it points into. A type-and-effect system checks that no value of a region is used after the region is freed, and an inference algorithm places the `letregion`s automatically, so that ML programs could run without a garbage collector. The approach was implemented in the ML Kit, a compiler for Standard ML.

In the early 2000s, the Cyclone language (by Trevor Jim, Greg Morrisett, Dan Grossman and others) brought regions to a safe dialect of C. Cyclone pointers carry region annotations; a function can be polymorphic over the regions of its arguments; regions are ordered by an *outlives* relation, so that a pointer into a longer-lived region can be used where a shorter-lived one is expected; and there are unique pointers for memory that is not shared. Every one of these is a feature you know from Rust. Cyclone is Rust's most direct ancestor on this side of the language.

The important observation for our question is negative: **none of these is a substructural system**. A region type system is an ordinary type system, with weakening, contraction and exchange, extended with a second sort of variable (region names) and constraints between them. Rust combines a region system with an affine one. Either alone would be unremarkable; the combination is what makes Rust distinctive.

A region discipline for the heap language

Here is a region discipline for our heap language. Types are numbers or pointers into a named region:

$$\tau ::= \mathit{num} \mid \mathit{ref}\ \rho$$

A typing context Γ assigns types to variables, and a set Δ records which regions are *live*. In our straight-line language every `alloc` creates a fresh region holding exactly one cell; `free` ends the region. The typing judgment for a command has an input and an output, because commands change both Γ and Δ: $\Delta; \Gamma \vdash c \dashv \Delta'; \Gamma'$ reads "in context $\Delta; \Gamma$ the command c is well typed and leaves the context $\Delta'; \Gamma'$". The rules:

$$\frac{\rho \text{ fresh}}{\Delta; \Gamma \vdash \texttt{alloc}\ x\ n \dashv \Delta \cup \{\rho\}; \Gamma[x : \mathit{ref}\ \rho]}\ \text{R-Alloc}$$

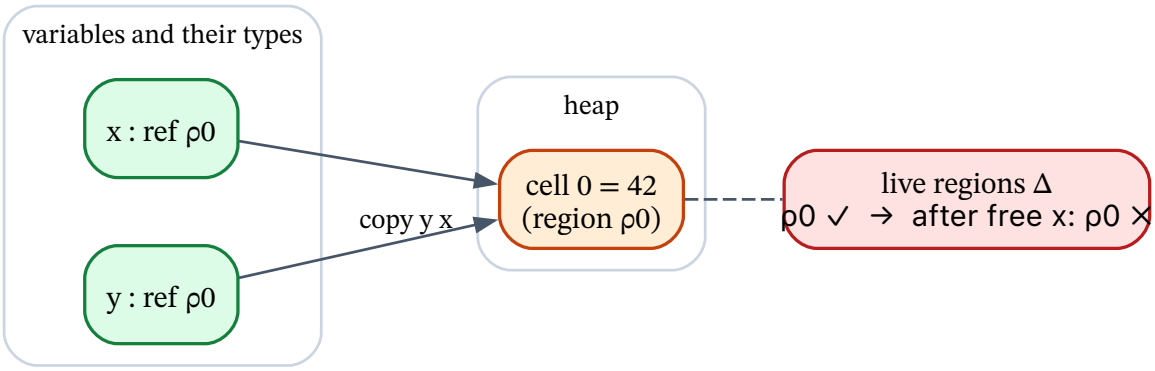
$$\frac{\Gamma(x) = \mathit{ref}\ \rho \qquad \rho \in \Delta}{\Delta; \Gamma \vdash \texttt{copy}\ y\ x \dashv \Delta; \Gamma[y : \mathit{ref}\ \rho]}\ \text{R-Copy} \qquad \frac{\Gamma(x) = \mathit{ref}\ \rho \qquad \rho \in \Delta}{\Delta; \Gamma \vdash \texttt{read}\ y\ x \dashv \Delta; \Gamma[y : \mathit{num}]}\ \text{R-Read}$$

$$\frac{\Gamma(x) = \mathit{ref}\ \rho \qquad \rho \in \Delta}{\Delta; \Gamma \vdash \texttt{free}\ x \dashv \Delta \setminus \{\rho\}; \Gamma}\ \text{R-Free}$$

plus a rule for `write` like R-Read, and one for copying a number. Two things make this work, and both are worth saying slowly.

First, **a copy shares the region**. Rule R-Copy gives `y` the type *ref* ρ, the *same* ρ as `x`. The type system now knows that `x` and `y` are aliases: not by counting them, but because their types mention the same region name. Copies are unrestricted, so pointers are fully structural, exactly like `&T`.

Second, **`free` ends the region for every alias at once**. Rule R-Free removes ρ from Δ but leaves Γ alone: `y` still has type *ref* ρ. What has changed is that ρ is no longer live, and every rule that uses a pointer requires its region to be live. So `read z y` after `free x` is rejected, without the type system ever tracking that `y` was copied from `x`.



The alias y shares the region name ρ0 with x, so ending ρ0 with free x makes both pointers unusable.

The premise $\rho \in \Delta$ in every rule is the **escape check** of this discipline: a pointer may be used only while its region is live. In Tofte and Talpin's lexical version the same check takes the form "the result of a `letregion` ρ block must not mention ρ"; we will meet Rust's version shortly.

The rules in Lean

The Lean version follows the mathematics closely:

```
/-- The type of a variable: a number, or a pointer into region `ρ`. -/
inductive Ty where
  | num
  | ref (ρ : ℕ)
  deriving DecidableEq, Repr

/-- The static context: the type of each variable, the set of live regions,
and the next unused region name. -/
structure Ctx where
  ty : Var → Option Ty
  live : ℕ → Bool
  fresh : ℕ
```

```
/-- The context of the empty program: no variables, no live regions. -/
def Ctx.init : Ctx where
  ty := fun _ => none
  live := fun _ => false
  fresh := 0
```

The live set Δ is the Boolean function `live`, and `fresh` is a counter that supplies new region names, like `next` supplies new addresses.

```
/-- `CmdOK Γ c Γ'`: in the context `Γ` the command `c` is well typed and
leaves the context `Γ'`. Every use of a pointer requires its region to be
live; `free` ends the region. -/
inductive CmdOK : Ctx → Cmd → Ctx → Prop where
  | alloc {Γ : Ctx} {x : Var} {n : ℕ} :
      CmdOK Γ (.alloc x n)
        { ty := update Γ.ty x (some (.ref Γ.fresh))
          live := update Γ.live Γ.fresh true
          fresh := Γ.fresh + 1 }
  | copyNum {Γ : Ctx} {y x : Var} (hx : Γ.ty x = some .num) :
      CmdOK Γ (.copy y x) { Γ with ty := update Γ.ty y (some .num) }
  | copyRef {Γ : Ctx} {y x : Var} {ρ : ℕ}
      (hx : Γ.ty x = some (.ref ρ)) (hρ : Γ.live ρ = true) :
      CmdOK Γ (.copy y x) { Γ with ty := update Γ.ty y (some (.ref ρ)) }
  | read {Γ : Ctx} {y x : Var} {ρ : ℕ}
      (hx : Γ.ty x = some (.ref ρ)) (hρ : Γ.live ρ = true) :
      CmdOK Γ (.read y x) { Γ with ty := update Γ.ty y (some .num) }
  | write {Γ : Ctx} {x : Var} {n : ℕ} {ρ : ℕ}
      (hx : Γ.ty x = some (.ref ρ)) (hρ : Γ.live ρ = true) :
      CmdOK Γ (.write x n) Γ
  | free {Γ : Ctx} {x : Var} {ρ : ℕ}
      (hx : Γ.ty x = some (.ref ρ)) (hρ : Γ.live ρ = true) :
      CmdOK Γ (.free x) { Γ with live := update Γ.live ρ false }

/-- A program is well typed when each command is, in the context that the
previous command leaves. -/
inductive ProgOK : Ctx → Prog → Prop where
  | nil {Γ : Ctx} : ProgOK Γ []
  | cons {Γ Γ' : Ctx} {c : Cmd} {k : Prog} :
      CmdOK Γ c Γ' → ProgOK Γ' k → ProgOK Γ (c :: k)
```

Each constructor of `CmdOK` is one rule, its arguments the premises. `ProgOK` threads the context through a list of commands: the context that one command leaves is the context in which the next is checked.

The rules are *syntax-directed*: for each command at most one rule applies (two for `copy`, distinguished by the type of `x`), and its premises can be checked by looking them up. So a checker is easy to write, and it can report which premise failed:

```
/-- Why a command is rejected. -/
inductive RegionError where
  | unknown (x : Var)               -- `x` has no type yet
  | notAPointer (x : Var)           -- `x` holds a number
  | deadRegion (x : Var) (ρ : ℕ)  -- `x` points into region `ρ`, which has ended
  deriving Repr

/-- The region that the pointer in `x` points into, if that region is live. -/
def liveRegion (Γ : Ctx) (x : Var) : Except RegionError ℕ :=
  match Γ.ty x with
  | none => .error (.unknown x)
  | some .num => .error (.notAPointer x)
  | some (.ref ρ) => if Γ.live ρ then .ok ρ else .error (.deadRegion x ρ)

/-- Check one command, computing the context it leaves. -/
def checkCmd (Γ : Ctx) : Cmd → Except RegionError Ctx
  | .alloc x _ => .ok { ty := update Γ.ty x (some (.ref Γ.fresh))
                        live := update Γ.live Γ.fresh true
```

```
                        fresh := Γ.fresh + 1 }
  | .copy y x =>
    match Γ.ty x with
    | some .num => .ok { Γ with ty := update Γ.ty y (some .num) }
    | _ =>
      match liveRegion Γ x with
      | .error e => .error e
      | .ok ρ => .ok { Γ with ty := update Γ.ty y (some (.ref ρ)) }
  | .read y x =>
    match liveRegion Γ x with
    | .error e => .error e
    | .ok _ => .ok { Γ with ty := update Γ.ty y (some .num) }
  | .write x _ =>
    match liveRegion Γ x with
    | .error e => .error e
    | .ok _ => .ok Γ
  | .free x =>
    match liveRegion Γ x with
    | .error e => .error e
    | .ok ρ => .ok { Γ with live := update Γ.live ρ false }

/-- Check a whole program, stopping at the first error. -/
def check (Γ : Ctx) : Prog → Except RegionError Unit
  | [] => .ok ()
  | c :: k =>
    match checkCmd Γ c with
    | .error e => .error e
    | .ok Γ' => check Γ' k
```

```
/-- info: Except.ok () -/
#guard_msgs in
#eval check Ctx.init share

/-- info: Except.error (Ch26.RegionError.deadRegion "y" 0) -/
#guard_msgs in
#eval check Ctx.init uaf

/-- info: Except.error (Ch26.RegionError.deadRegion "y" 0) -/
#guard_msgs in
#eval check Ctx.init doubleFree
```

The checker accepts `share` and rejects both `uaf` and `doubleFree`, in both cases at the use of `y`, which points into region 0, already ended. Compare with the naive check, which accepted both, and the affine check, which rejected `share`.

Is the checker faithful to the rules? The file proves that it is, in both directions:

```
/-- The checker accepts a program exactly when the typing rules do. -/
theorem check_ok {Γ : Ctx} {p : Prog} : check Γ p = .ok () ↔ ProgOK Γ p := by
  induction p generalizing Γ with
  | nil => exact ⟨fun _ => ProgOK.nil, fun _ => rfl⟩
  | cons c k ih =>
    constructor
    · intro h
      simp only [check] at h
      split at h
      · cases h
      · rename_i Γ' hc
        exact ProgOK.cons (checkCmd_ok.mp hc) (ih.mp h)
    · intro h
      cases h with
      | cons hc hk =>
        simp only [check, checkCmd_ok.mpr hc]
        exact ih.mpr hk

/-- So the typing judgment is decidable, and `decide` can use the checker. -/
```

```
instance (Γ : Ctx) (p : Prog) : Decidable (ProgOK Γ p) :=
  match h : check Γ p with
  | .ok () => isTrue (check_ok.mp h)
  | .error _ => isFalse fun hp => by rw [check_ok.mpr hp] at h; cases h

theorem share_typed : ProgOK Ctx.init share := by decide

theorem uaf_rejected : ¬ ProgOK Ctx.init uaf := by decide

theorem doubleFree_rejected : ¬ ProgOK Ctx.init doubleFree := by decide
```

`check_ok` is proved by induction on the program, using a lemma `checkCmd_ok` that relates `checkCmd` to `CmdOK` one command at a time (both are in the file; their proofs are the same kind of case analysis as `exec_error`). With the equivalence in hand, the `instance` makes `ProgOK Γ p` *decidable*: to decide it, run the checker. The `match h : check Γ p with` form names the equation between `check Γ p` and the branch taken, so that each branch can use it: `isTrue` packages a proof, `isFalse` a refutation. After that, `decide` proves that `share` is well typed and that `uaf` and `doubleFree` are not.

Safety, proved

A type system is only as good as its safety theorem. Ours says that a program accepted by the region discipline never goes wrong, and its proof has the shape you know from Chapter 9: progress and preservation. The new ingredient is the invariant that connects a static context to a run-time state:

```
/-- How a static context describes a run-time state. `owner` says which region
each heap cell belongs to. It is *ghost* information: it exists in the proof,
not in the program or the machine. -/
structure Agree (Γ : Ctx) (s : State) (owner : ℕ → ℕ) : Prop where
  /-- Variables typed `num` hold numbers. -/
  nums : ∀ x, Γ.ty x = some .num → ∃ n, s.env x = some (.num n)
  /-- Variables typed `ref ρ` hold the address of a cell of region `ρ`. -/
  refs : ∀ x ρ, Γ.ty x = some (.ref ρ) →
    ∃ l, s.env x = some (.addr l) ∧ l < s.next ∧ owner l = ρ
  /-- Every cell allocated so far belongs to a region that has been named. -/
  named : ∀ l, l < s.next → owner l < Γ.fresh
  /-- The cells of a live region are allocated. -/
  alive : ∀ l, l < s.next → Γ.live (owner l) = true → s.heap l ≠ none
```

A `structure` whose type is `Prop` is a conjunction with named components: a proof of `Agree Γ s owner` is a bundle of four proofs, accessed as `ha.nums`, `ha.refs` and so on. The four fields say: variables typed `num` hold numbers; variables typed `ref ρ` hold the address `l` of a cell that has been allocated at some point (`l < s.next`) and belongs to region `ρ`; every cell allocated so far belongs to a region that has already been named (`owner l < Γ.fresh`); and the cells of every live region are still allocated.

The parameter `owner` deserves a comment. It says which region each heap cell belongs to, and it appears nowhere in the program or in the machine: it is **ghost** information, which exists only to state and prove the invariant. Keep that in mind, because Rust's lifetimes are ghost information in exactly the same sense. The compiler uses them to check the program and then erases them; no lifetime exists at run time.

Progress says that a well-typed command can always take a step:

```
/-- Progress: a well-typed command can run in any state that agrees with
its context. -/
theorem progress {Γ Γ' : Ctx} {c : Cmd} {s : State} {owner : ℕ → ℕ}
```

```
    (hc : CmdOK Γ c Γ') (ha : Agree Γ s owner) (k : Prog) :
    ∃ s', Step (c :: k, s) (k, s') := by
  cases hc with
  | alloc => exact ⟨_, Step.alloc⟩
  | copyNum hx =>
    obtain ⟨n, hn⟩ := ha.nums _ hx
    exact ⟨_, Step.copy hn⟩
  | copyRef hx _ =>
    obtain ⟨l, hl, -, -⟩ := ha.refs _ _ hx
    exact ⟨_, Step.copy hl⟩
  | read hx hρ =>
    -- `x` points to a cell `l` of region `ρ`; `ρ` is live, so `l` is allocated.
    obtain ⟨l, hl, hlt, rfl⟩ := ha.refs _ _ hx
    obtain ⟨n, hn⟩ := Option.ne_none_iff_exists'.mp (ha.alive l hlt hρ)
    exact ⟨_, Step.read hl hn⟩
  | write hx hρ =>
    obtain ⟨l, hl, hlt, rfl⟩ := ha.refs _ _ hx
    obtain ⟨m, hm⟩ := Option.ne_none_iff_exists'.mp (ha.alive l hlt hρ)
    exact ⟨_, Step.write hl hm⟩
  | free hx hρ =>
    obtain ⟨l, hl, hlt, rfl⟩ := ha.refs _ _ hx
    obtain ⟨m, hm⟩ := Option.ne_none_iff_exists'.mp (ha.alive l hlt hρ)
    exact ⟨_, Step.free hl hm⟩
```

The proof is by cases on the typing rule. For `alloc` a step is always possible. For a copy of a number, `ha.nums` provides the number, and `Step.copy` the step. The interesting cases are the three that follow a pointer. Take `read`: the typing rule gives `hx`, that `x` has type `ref ρ`, and `hρ`, that `ρ` is live. From `ha.refs` we obtain the address `l` that `x` holds, the fact `hlt` that it was allocated at some point, and (by the pattern `rfl`) that its owner is `ρ`. Then `ha.alive l hlt hρ` says that the cell is still allocated, in the form `s.heap l ≠ none`; the core lemma `Option.ne_none_iff_exists'` turns that into an actual value `n` with `s.heap l = some n`, and `Step.read` applies. Here the premise $\rho \in \Delta$ of the typing rule does its work.

Preservation says that after a step, a (possibly different) ghost `owner` makes the new state agree with the new context. It is proved by cases, and each case is a lemma of the form "this change to the context and this change to the state preserve `Agree`". The heart of the matter is `free`:

```
/-- The heart of the matter: after `free x`, every alias of `x` is dead in
the context, and every pointer that is still live points to an allocated cell. -/
theorem agree_free {Γ : Ctx} {s : State} {owner : ℕ → ℕ} {x : Var}
    {ρ : ℕ} {l : ℕ} (ha : Agree Γ s owner) (hx : Γ.ty x = some (.ref ρ))
    (hl : s.env x = some (.addr l)) :
    Agree { Γ with live := update Γ.live ρ false }
      { s with heap := update s.heap l none } owner where
  nums := ha.nums
  refs := ha.refs
  named := ha.named
  alive l' hl' hlive := by
    -- The cell `l` that was freed belongs to `ρ`, the region that just ended.
    obtain ⟨l₀, hl₀, -, hown⟩ := ha.refs x ρ hx
    rw [hl, Option.some.injEq, Val.addr.injEq] at hl₀
    subst hl₀
    -- `l'` belongs to a region that is still live, so it is not `ρ`...
    have hne : owner l' ≠ ρ := by
      intro h
      simp [h] at hlive
    -- ...hence `l'` is not the freed cell, and its contents are unchanged.
    have hl'l : l' ≠ l := by
      rintro rfl
      exact hne hown
    simp only [update_of_ne hl'l]
```

```
    simp only [update_of_ne hne] at hlive
    exact ha.alive l' hl' hlive
```

The new context has region `ρ` dead; the new state has cell `l` freed. Three of the four fields are untouched, so they are inherited from `ha`. The fourth, `alive`, must be proved: take any cell `l'` whose region is still live (that is `hlive`, stated for the *updated* `live` function). The proof proceeds in four steps.

1. `ha.refs x ρ hx` says that `x` holds some address `l₀` owned by `ρ`. Since `x` holds `l` (hypothesis `hl`), rewriting with `hl` and the injectivity of `some` and `addr` turns `hl₀` into `l = l₀`, and `subst` replaces `l₀` by `l` everywhere. Now `hown : owner l = ρ`.
2. The owner of `l'` is not `ρ`: if it were, `hlive` would say that the updated `live` returns `true` at `ρ`, but it returns `false` there; `simp [h] at hlive` derives that contradiction.
3. Hence `l'` is not `l`: if it were, its owner would be `ρ` (by `hown`), contradicting step 2. (`rintro rfl` introduces the equation `l' = l` and substitutes it at once.)
4. Since `l' ≠ l`, freeing `l` did not change the cell `l'` (`update_of_ne`), and since `owner l' ≠ ρ`, killing `ρ` did not change whether `owner l'` is live; the old invariant `ha.alive` finishes the proof.

Read step 3 again: it is where aliasing is handled. A cell that is still reachable through a live pointer cannot be the freed cell, *because the freed cell's region is dead and the pointer's region is live*. The proof never asks how many pointers to `l` exist.

The rest is assembly:

```
/-- A configuration is well typed when some context describes both the
program still to run and the current state. -/
def Typed (c : Prog × State) : Prop :=
  ∃ Γ owner, ProgOK Γ c.1 ∧ Agree Γ c.2 owner

/-- A well-typed configuration is not stuck. -/
theorem Typed.not_stuck {c : Prog × State} (h : Typed c) : ¬ Stuck c := by
  obtain ⟨p, s⟩ := c
  obtain ⟨Γ, owner, hp, ha⟩ := h
  rintro ⟨hne, hno⟩
  cases hp with
  | nil => exact hne rfl
  | cons hc _ =>
    obtain ⟨s', hs⟩ := progress hc ha _
    exact hno ⟨_, hs⟩

/-- A step from a well-typed configuration leads to a well-typed one. -/
theorem Typed.step {c c' : Prog × State} (h : Typed c) (hs : Step c c') :
    Typed c' := by
  obtain ⟨p, s⟩ := c
  obtain ⟨p', s'⟩ := c'
  obtain ⟨Γ, owner, hp, ha⟩ := h
  cases hp with
  | nil => cases hs
  | cons hc hk =>
    obtain ⟨rfl, owner', ha'⟩ := preservation hc ha hs
    exact ⟨_, owner', hk, ha'⟩

/-- Any number of steps preserve typing. -/
theorem Typed.steps {c c' : Prog × State} (h : Typed c) (hs : Steps c c') :
    Typed c' := by
  induction hs with
  | refl => exact h
  | tail _ hstep ih => exact ih.step hstep
```

```
/-- **Region safety.** A program accepted by the region discipline never goes
wrong: no use after free, no double free, no dangling write. -/
theorem regions_safe {p : Prog} (h : ProgOK Ctx.init p) : ¬ GoesWrong p := by
  rintro ⟨c, hs, hstuck⟩
  have ht : Typed (p, State.init) := ⟨Ctx.init, fun _ => 0, h, agree_init⟩
  exact (ht.steps hs).not_stuck hstuck
```

A configuration is `Typed` if some context and some ghost describe it. `Typed.not_stuck` combines the two ways a configuration could fail to be stuck: if no commands remain, it is finished (`hne rfl` contradicts "the program is not empty"); otherwise `progress` provides a step. `Typed.step` is preservation for whole configurations, and `Typed.steps` extends it to any number of steps by induction on `Steps` : the `tail` case of `ReflTransGen` adds one step at the end of a path. Finally, `regions_safe` starts from the empty state, which agrees with the empty context (`agree_init`), and concludes that no reachable configuration is stuck. **Every program the region discipline accepts runs without a use after free, a double free or a dangling write.**

The discipline is a toy: straight-line programs, one cell per region, no functions. Real region systems need functions that are polymorphic over regions, regions that contain many objects, and some way to relate regions to each other. Rust has all three, and they are called lifetimes.

26.5 Lifetimes are region variables

Lifetime parameters

A lifetime parameter `'a` is a region variable, and a function with lifetime parameters is polymorphic over regions, just as a generic function is polymorphic over types (Chapter 12):

```
/// One region variable for both inputs and the output: the result is valid
/// wherever *both* inputs are valid.
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}
```

The signature says: for every region `'a` , if both arguments are valid throughout `'a` , then so is the result. At a call site, the compiler picks `'a` :

```
let station = String::from("north station");
let result;
{
    let probe = String::from("probe 7");
    result = longer(&station, &probe);
    println!("longer: {result}"); // fine: both inputs are alive here
}
// Using `result` here would be E0597: its region is that of `probe` too.
```

Both `&station` and `&probe` must be valid throughout `'a` , so `'a` can be no larger than the part of the program where *both* `station` and `probe` exist, which ends at the closing brace. The result has type `&'a str` , so it may be used only inside that region, and using `result` after the brace would be E0597. Notice that the function body is checked once, for an arbitrary `'a` , and never looks at a call site; each call site is checked against the signature only. That is what makes lifetime checking modular.

When the signature does not say how the result relates to the inputs, there is nothing to check against:

```
pub fn longer(a: &str, b: &str) -> &str {
    if a.len() >= b.len() { a } else { b }
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch26_missing_lifetime.rs:6:36
  |
6 | pub fn longer(a: &str, b: &str) -> &str {
  |                  ----     ----     ^ expected named lifetime parameter
  |
  = help: this function's return type contains a borrowed value, but the signature does not
say whether it is borrowed from `a` or `b`
help: consider introducing a named lifetime parameter
  |
6 | pub fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
  |              ++++     ++          ++          ++
```

The help text says it exactly: the signature does not say whether the result is borrowed from `a` or from `b`. Without that information a caller could not know which argument must stay alive, so the function cannot be typed.

Lifetimes become most interesting when a type holds references. A tokenizer that hands out slices of its input is a good example:

```
/// A tokenizer over a borrowed input. The region `'src` is the region of the
/// *input*, not of the tokenizer.
struct Tokenizer<'src> {
    input: &'src str,
    pos: usize,
}

impl<'src> Tokenizer<'src> {
    fn new(input: &'src str) -> Self {
        Tokenizer { input, pos: 0 }
    }

    /// `&mut self` is borrowed only for the call, but the token borrows from
    /// the input, so it lives as long as `'src`, not as long as the call.
    fn next_token(&mut self) -> Option<&'src str> {
        let rest = &self.input[self.pos..];
        let start = rest.find(|c: char| !c.is_whitespace())?;
        let rest = &rest[start..];
        let len = rest.find(char::is_whitespace).unwrap_or(rest.len());
        self.pos += start + len;
        Some(&rest[..len])
    }
}

fn tokens(input: &str) -> Vec<&str> {
    let mut tokenizer = Tokenizer::new(input);
    let mut out = Vec::new();
    while let Some(token) = tokenizer.next_token() {
        out.push(token); // tokens outlive the tokenizer: they borrow `input`
    }
    out
}
```

```
tokens: ["GET", "/index.html", "HTTP/1.1"]
```

There are two regions in `next_token`: that of the `&mut self` borrow, which lasts for one call, and `'src`, the region of the input. The return type `Option<&'src str>` puts tokens in the second. That is why `tokens` can keep them in a vector while calling `next_token` again: the tokens do not borrow the

tokenizer at all. With `Option<&str>`, the elision rules below would tie each token to the `&mut self` borrow, and the second call would be rejected.

Outlives

Regions are related by *outlives*: `'a: 'b` (read "`'a` outlives `'b`") means that the region `'a` contains the region `'b`, so a reference valid throughout `'a` is also valid throughout `'b`. Write $\Delta \vdash \alpha : \beta$ for "under the assumptions Δ, α outlives β". It is the smallest relation closed under these rules:

$$\frac{}{\Delta \vdash \alpha : \alpha}\ \text{O-Refl} \qquad \frac{(\alpha : \beta) \in \Delta}{\Delta \vdash \alpha : \beta}\ \text{O-Hyp} \qquad \frac{}{\Delta \vdash \mathit{static} : \alpha}\ \text{O-Static}$$

$$\frac{\Delta \vdash \alpha : \beta \qquad \Delta \vdash \beta : \gamma}{\Delta \vdash \alpha : \gamma}\ \text{O-Trans}$$

Every region outlives itself; assumptions written in the signature (a `where 'a: 'b` clause, or `<'b, 'a: 'b>`) may be used; `'static`, the region of the whole program, outlives every region; and outlives is transitive. A relation that is reflexive and transitive is a *preorder*, and Chapter 16 showed that a preorder is a category. Chapter 27 builds subtyping and variance on exactly this preorder.

In Rust the assumptions appear in signatures:

```
/// An explicit outlives bound: `'long: 'short` lets a `&'long` reference be
/// returned where a `&'short` one is promised.
fn pick<'short, 'long: 'short>(
    fallback: &'long str,
    choice: &'short str,
) -> &'short str {
    if choice.is_empty() { fallback } else { choice }
}

/// `T: 'a` says that every reference inside `T` outlives `'a`, which is what
/// makes a `&'a T` meaningful.
struct Labeled<'a, T: 'a> {
    label: &'static str,
    value: &'a T,
}
```

In `pick` the body returns `fallback`, of type `&'long str`, where the signature promises `&'short str`. That is allowed because `'long: 'short` is an assumption (rule O-Hyp), and a reference valid for the longer region is valid for the shorter one. The bound `T: 'a` on `Labeled` generalizes outlives from regions to types: it says that every reference *inside* `T` outlives `'a`, so that a `&'a T` never points to a value that contains dangling references. (Rust infers this bound for struct fields, so writing it is optional.)

The escape check

In our Lean discipline, the escape check was the premise "the region is live" on every use of a pointer. Rust's lifetimes are not created and destroyed by explicit commands, so its check takes a different form, but the content is the same. Roughly, and as a model of what the compiler does: every reference has a region, which must include every point of the program where the reference may still be used, and which must be contained in the scope of the referent. When no region satisfies both requirements, the reference would escape its referent's scope, and the compiler reports it. E0597 is the failure inside one function (the use comes after the referent's scope ends), E0515 is the failure at a function's boundary

(the result's region would have to extend into the caller, past the local's scope), and E0505 is the case where the owner moves the referent away while the reference's region is still running. Chapter 28 states an escape check as a typing rule, in the style of Tofte and Talpin; Chapter 29 shows how rustc computes regions as sets of points in a control-flow graph.

Elision

Most Rust code writes few lifetimes, because the compiler fills them in by three *elision* rules, applied to function signatures:

1. Each elided lifetime in the parameters becomes a distinct lifetime parameter.
2. If there is exactly one lifetime in the parameters (elided or not), it is assigned to all elided lifetimes in the result.
3. If there are several, but one of them is the lifetime of `&self` or `&mut self`, that one is assigned to all elided lifetimes in the result.

So `fn first_word(s: &str) -> &str` means `fn first_word<'a>(s: &'a str) -> &'a str` by rules 1 and 2, and a method `fn name(&self) -> &str` returns a borrow of `self` by rule 3. In any other case the result's lifetime cannot be elided, and you get E0106, as with `longer` above. Elision is pure abbreviation: it never changes what is checked, only how much you have to write. It is also the reason for the explicit `'src` in the tokenizer: rule 3 would have picked the wrong region.

`'static`

`'static` is the region of the entire execution. A string literal has type `&'static str` because it is stored in the program's binary, and `Box::leak` produces a `&'static mut T` because the box is never freed (Chapter 24 explained why leaking is safe). A *bound* `T: 'static` means something weaker than "lives forever": that `T` contains no references with a shorter region. An owned `Vec<i32>` satisfies it, because it contains no references at all.

The most common place to meet `'static` is `std::thread::spawn`, whose closure must be `'static`, because the new thread may outlive every region of the function that spawned it:

```
use std::thread;

pub fn spawn_with_borrow() {
    let readings = vec![1, 2, 3];
    let handle = thread::spawn(|| readings.len());
    handle.join().unwrap();
}
```

```
error[E0373]: closure may outlive the current function, but it borrows `readings`, which is
owned by the current function
  --> compile_fail/ch26_spawn_borrow.rs:10:32
   |
10 |     let handle = thread::spawn(|| readings.len());
   |                                ^^ --------- `readings` is borrowed here
   |                                |
   |                                may outlive borrowed value `readings`
   |
note: function requires argument type to outlive `'static`
  --> compile_fail/ch26_spawn_borrow.rs:10:18
   |
10 |     let handle = thread::spawn(|| readings.len());
```

```
   |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: to force the closure to take ownership of `readings` (and any other referenced
variables), use the `move` keyword
   |
10 |     let handle = thread::spawn(move || readings.len());
   |                                ++++
```

The note names the requirement, and the help names the fix: `move` the vector into the closure, so that the closure owns it and contains no borrow. (Chapter 24's `thread::scope` is the other fix: it guarantees that the threads are joined before the scope ends, so they may borrow.)

```rust
/// `'static`: string literals live in the program's binary; a leaked box
/// lives until the process ends; an owned value moved into a thread satisfies
/// a `'static` bound because it contains no borrowed references at all.
fn statics() -> usize {
    let literal: &'static str = "compiled into the binary";
    let leaked: &'static [u8] = Box::leak(vec![1, 2, 3].into_boxed_slice());
    let readings = vec![4, 5, 6];
    let handle = thread::spawn(move || readings.len()); // the fix for ch26_spawn_borrow
    literal.len() + leaked.len() + handle.join().unwrap()
}
```

26.6 Permissions, not multiplicities

Regions say *when* a reference may be used. The aliasing rule says *how* it may be used, and here the right vocabulary is **permissions**:

Rust type	Permission	Duplicable?
`T` (the owner)	read, write, free	no
`&'a mut T`	read and write, during `'a`	no (unique)
`&'a T`	read, during `'a`	yes (`Copy`)

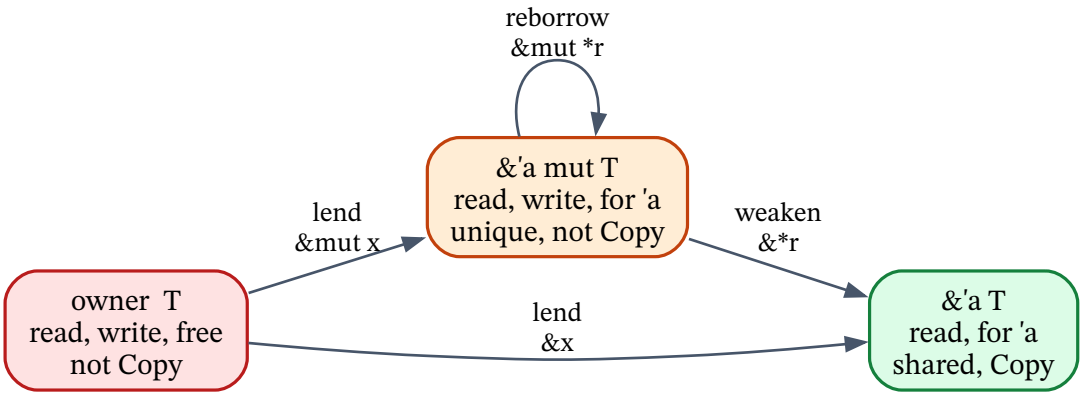
The governing law is the aliasing rule, stated in terms of permissions:

> **NOTE**
>
> **The aliasing rule.** At every point of the program, each place is in one of two situations: any number of shared (read) permissions and no unique one, or exactly one unique (read-write) permission and nothing else. While a place is lent, its owner's own permission is suspended accordingly: it may still read while all loans are shared, and it may do nothing while a unique loan is live.

"Many readers or one writer" is not a statement about how many times a variable is used. It is a *disjointness* condition on claims to memory: two claims to the same place are compatible only if both are read-only. Logicians and verification researchers know this pattern well. John Boyland's *fractional permissions* split a write permission into fractions that each allow reading and that must be reassembled to write; *separation logic*, which Chapter 32 introduces, is built on a connective $P * Q$ that holds when P and Q describe disjoint parts of the heap. Compare what is being counted:

Discipline	Counts
affine types (Part V)	uses of a variable
permissions, separation logic	claims on a place in memory

A shared reference can be used as many times as you like, while the claim it represents stays the same: a read-only claim for the duration of its region. No discipline on the uses of variables can express that.



Permissions flow from the owner to loans: a unique loan can be reborrowed or weakened to a shared one, and shared loans are freely copied.

A borrow is a temporary transfer

The diagram shows how permissions move. `&mut x` *lends* the owner's read-write permission for a region; while that region lasts, the owner has nothing, and when it ends, the permission comes back. `&x` lends a read permission, which the owner keeps too, so the owner can read but not write. A unique loan can be *reborrowed* (`&mut *r`, which the compiler inserts when you pass `r` to a function) for a shorter region, after which it is usable again, or *weakened* to a shared one. And `split_at_mut` splits a unique permission on a slice into two unique permissions on disjoint halves: this is how `transfer` and `count_in_parallel` above were made legal.

This is the "lend and return" idea from the beginning of the chapter, and it is the honest way to make borrowing *look* affine: the owner really is unusable while it is lent, and Chapter 28 builds a calculus on this view. What the affine reading leaves out is *when* the permission comes back. That is a region.

A ledger of permissions

The aliasing rule is simple enough to run. The example `code/rust/examples/ch26_permissions.rs` checks it dynamically, on traces of events: a place comes into existence (`Own`), is borrowed shared or unique (`Borrow`), is read or written through a loan (`Through`) or directly by its owner (`Direct`), and is freed (`Free`). The ledger keeps every loan, and considers a loan *live* from its creation to its last use in the trace. That is the idea of non-lexical lifetimes, which Chapter 29 develops on real control-flow graphs. The rule itself is the `step` method:

```
/// The aliasing rule, one event at a time.
fn step(&mut self, now: usize, event: Event) -> Result<(), String> {
```

```
    match event {
        Event::Own(place) => self.owned.push(place),
        Event::Borrow {
            loan,
            place,
            unique,
        } => {
            // Shared loans coexist; a unique loan coexists with nothing.
            for (other, other_unique) in self.live_loans(place, now) {
                if unique || other_unique {
                    let how = if unique { "uniquely" } else { "shared" };
                    return Err(format!(
                        "cannot borrow `{place}` {how} while `{other}` is live"
                    ));
                }
            }
            self.loans.insert(loan, Loan { place, unique });
        }
        Event::Through(loan, access) => {
            let l = &self.loans[loan];
            if !self.owned.contains(&l.place) {
                return Err(format!("`{loan}` points to freed `{}`", l.place));
            }
            // A shared loan is a read permission only.
            if access == Access::Write && !l.unique {
                return Err(format!("cannot write through shared loan `{loan}`"));
            }
        }
        Event::Direct(place, access) => {
            // While lent, the owner may read (if all loans are shared)
            // but not write.
            for (other, other_unique) in self.live_loans(place, now) {
                if access == Access::Write || other_unique {
                    let verb = if access == Access::Write {
                        "write"
                    } else {
                        "read"
                    };
                    return Err(format!(
                        "cannot {verb} `{place}` while `{other}` is live"
                    ));
                }
            }
        }
        Event::Free(place) => {
            // Freeing needs every permission back.
            if let Some((other, _)) = self.live_loans(place, now).first() {
                return Err(format!(
                    "cannot free `{place}` while `{other}` is live"
                ));
            }
            self.owned.retain(|p| *p != place);
        }
    }
    Ok(())
}
```

Each arm is one clause of the aliasing rule. A new loan is compatible with the live loans of the same place only if both are shared. Writing through a loan needs a unique one. The owner may read while all loans are shared and may not write while any loan is live. Freeing needs every loan to be over. The traces are transcriptions of this chapter's programs:

```
use Access::{Read, Write};
use Event::{Borrow, Direct, Free, Own, Through};

const fn shared(loan: &'static str, place: &'static str) -> Event {
    Borrow {
```

```
        loan,
        place,
        unique: false,
    }
}

const fn unique(loan: &'static str, place: &'static str) -> Event {
    Borrow {
        loan,
        place,
        unique: true,
    }
}

/// The traces, each named after the program it comes from.
#[rustfmt::skip]
const TRACES: &[(&str, &[Event])] = &[
    // Lean's `share`: two readers, then the owner frees. Accepted.
    ("share", &[Own("x"), shared("y", "x"), shared("z", "x"),
        Through("y", Read), Through("z", Read), Free("x")]),
    // ch26_dangling (E0597) and ch26_free_while_borrowed (E0505).
    ("dangling", &[Own("reading"), shared("r", "reading"), Free("reading"),
        Through("r", Read)]),
    // ch26_shared_then_mut (E0502).
    ("shared_then_mut", &[Own("orders"), shared("first", "orders"),
        Direct("orders", Write), Through("first", Read)]),
    // The fix: the shared loan is dead after its last use. Accepted.
    ("copy_out_first", &[Own("orders"), shared("first", "orders"),
        Through("first", Read), Direct("orders", Write)]),
    // ch26_two_mut (E0499).
    ("two_mut", &[Own("accounts"), unique("from", "accounts"),
        unique("to", "accounts"), Through("from", Write), Through("to", Write)]),
    // ch26_iterator_invalidation (E0502).
    ("iterator_invalidation", &[Own("orders"), shared("iter", "orders"),
        Through("iter", Read), Direct("orders", Write), Through("iter", Read)]),
    // Writing through a shared reference (rustc: E0594).
    ("write_through_shared", &[Own("n"), shared("r", "n"), Through("r", Write)]),
    // Lend and return: a unique loan, then the owner uses the place again.
    ("lend_and_return", &[Own("log"), unique("w", "log"), Through("w", Write),
        Through("w", Write), Direct("log", Read)]),
];
```

```
share                 ok
dangling              event 2: cannot free `reading` while `r` is live
shared_then_mut       event 2: cannot write `orders` while `first` is live
copy_out_first        ok
two_mut               event 2: cannot borrow `accounts` uniquely while `from` is live
iterator_invalidation event 3: cannot write `orders` while `iter` is live
write_through_shared  event 2: cannot write through shared loan `r`
lend_and_return       ok
```

Every rejected program is rejected for the reason rustc gives. `share` and `copy_out_first` are accepted because each loan is dead (used for the last time) before the conflicting event, and `lend_and_return` shows the owner getting its permission back. The ledger counts claims on places, never uses of variables: in `share`, the loans `y` and `z` coexist on one place, and each could be used a thousand times without changing the verdict.

26.7 Interior mutability

The aliasing rule is checked at compile time, and that is sometimes too strict. A cache that a shared object updates when it is read, a counter of how often a sensor was queried, a graph whose nodes

point to each other: all want mutation through a shared reference. Rust's answer is *interior mutability*: types that allow it, by moving the check somewhere else.

```
/// `Cell`: mutation through a shared reference, safe because no reference
/// into the cell's contents is ever handed out (values are copied in and out).
struct Sensor {
    name: &'static str,
    reads: Cell<u32>,
}

impl Sensor {
    fn read(&self) -> u32 {
        self.reads.set(self.reads.get() + 1);
        42
    }
}

/// `RefCell`: the aliasing rule checked at run time. The cell counts its
/// shared and unique borrows and refuses a conflicting one.
fn refcell_at_run_time() {
    let log = RefCell::new(vec!["boot"]);
    let reader = log.borrow(); // a shared borrow, counted by the cell
    println!(
        "while reading: try_borrow_mut is {:?}",
        log.try_borrow_mut().map(|_| ())
    );
    drop(reader); // the shared borrow ends
    log.borrow_mut().push("ready"); // now a unique borrow succeeds
    println!("after the reader is gone: {:?}", log.borrow());
}
```

```
thermometer: read twice through two shared references, 2 reads, sum 84
while reading: try_borrow_mut is Err(BorrowMutError)
after the reader is gone: ["boot", "ready"]
```

`Cell<T>` allows mutation through `&Cell<T>` because it never hands out a reference to its contents: `get` copies the value out and `set` replaces it, so there is never a reader holding a pointer into the cell while it changes. (A `Cell` also cannot be shared between threads: it is not `Sync`.) In `main`, the example duplicates `&sensor` with the same `duplicate` function as before and reads through both copies; the count reaches 2.

`RefCell<T>` does hand out references (`Ref` and `RefMut` guards), and it keeps the aliasing rule by *counting* them at run time: a borrow flag records how many shared borrows exist, or that a unique one does. `try_borrow_mut` while a `Ref` is alive returns `Err(BorrowMutError)`, and `borrow_mut` would panic. The rule is the same rule; only the moment of checking has moved, from compile time to run time, and with it the cost: a counter to maintain, and a failure that is now a panic in production instead of an error message at your desk. `Mutex` and `RwLock` make the same trade for threads, with blocking instead of panicking. Chapter 27 will show that interior mutability also has consequences for subtyping: `Cell<T>` must be invariant in `T`.

26.8 Putting it together: one bug, four checkers

Let us follow the use after free through an alias through everything this chapter built. In Rust it is `ch26_free_while_borrowed`:

```
let x = Box::new(42);   // alloc x 42
let y = &*x;            // copy y x
drop(x);                // free x
*y                      // read z y
```

In the Lean heap language it is `uaf`, and `uaf_goes_wrong` proves that its semantics gets stuck at the last command with `dangling "y" 0`: the read follows a pointer to a freed cell.

The naive checker, which tracks freed variables by name, accepts it (`naive_accepts_uaf`), and is therefore unsound (`naive_unsound`). Its mistake is the one that makes C's pointers dangerous: it does not know that `x` and `y` name the same cell.

The affine checker rejects it, because `copy y x` moves the pointer out of `x`. But it also rejects `share`, where both pointers are only read. Affine pointers are `Box<T>`: safe, but not shareable.

The region checker rejects it with `deadRegion "y" 0`: `y` has type $\mathit{ref}\ \rho_0$, the same as `x`, and `free x` ended ρ_0. It accepts `share`. And `regions_safe` proves, once and for all, that nothing it accepts can get stuck.

rustc rejects it with E0505, and **the permission ledger** rejects the same shape of trace ("cannot free `reading` while `r` is live"). Notice *where* each one reports the error. rustc and the ledger complain at the `drop`, because they know that the borrow will be used later, so the loan is live at the free. Our Lean checker complains at the read, because it only learns of the conflict when the dead region is used. Both are sound; they differ in which line they blame.

The naive checker counts names, the affine one counts uses. The region and permission checkers track what neither count can see: which pointers may refer to the same memory, and for how long they may be used.

26.9 Common confusions

"**A lifetime is how long a value lives.**" A value lives in a scope; it is created and later dropped. A lifetime is the region of the program during which a *reference* may be used. The two are related by a constraint (a reference's region must fit inside its referent's scope), but they are different things: a `String` has no lifetime parameter, and a `&'a String` can have a region much shorter than the string's scope.

"**`&T` is immutable, `&mut T` is mutable.**" The more accurate words are *shared* and *unique*. `Cell` and `RefCell` mutate through `&`, and what `&mut` really guarantees is that nobody else can observe the place while you hold it.

"**The borrow checker counts references.**" It tracks loans, but its rule is not a count of uses. A shared reference can be copied and used any number of times; what matters is which loans are live at the same point for the same place.

"**Rust's lifetimes are Tofte–Talpin regions.**" They are descendants, not copies. In Tofte and Talpin's system, regions are run-time areas of memory that are allocated and freed. In Rust, memory is

managed by ownership (a value is freed when its owner drops it) and regions are purely static: they check references and are erased before the program runs.

"**Regions replace ownership.**" Rust uses both. Ownership, which is affine, decides who frees a value and when. Regions and permissions decide how long and in what way the value may be borrowed in the meantime.

26.10 Summary

- References can go wrong in two ways: by outliving their referent (E0597, E0515, E0505) and by aliasing combined with mutation (E0502, E0499, iterator invalidation, data races). The second principle is "aliasing XOR mutation".
- `&T` is `Copy`, so shared references are fully structural; `&mut T` is affine. Neither fact explains why the bad programs are rejected: affine typing counts uses of variables, and the conflicts are between different variables that point to the same place.
- A duplicable reference type added to an affine calculus with a heap is unsound (the dangling-reference term). The two ways out are lend-and-return and regions; both are about time, not multiplicity.
- In Lean, a straight-line heap language gets stuck on a use after free. Tracking freed variables is unsound; affine pointers are safe but cannot be shared; a region discipline, in which copies share a region name and `free` ends the region for all aliases, is proved safe by progress and preservation, with a ghost map from cells to regions.
- Lifetimes are region variables: functions are polymorphic over them, outlives (`'a: 'b`, `T: 'a`, `'static`) is a preorder on them, the escape check keeps references inside their regions, and elision fills in the obvious cases.
- References are permissions: the owner can read, write and free; `&mut` is a unique read-write permission; `&` is a duplicable read permission. A borrow is a temporary transfer of permission, and the aliasing rule is a disjointness condition, the same idea as fractional permissions and separation logic.
- Interior mutability (`Cell`, `RefCell`, `Mutex`) keeps the aliasing rule but checks it at run time, or avoids handing out references altogether.

26.11 Exercises

1. For each of the programs `ch26_dangling`, `ch26_shared_then_mut` and `ch26_data_race`, write the trace of events that `ch26_permissions.rs` would need, and predict the ledger's verdict before running it. Then add a trace for the repaired `count_in_parallel` from `ch26_aliasing.rs`. *Hint:* `split_at_mut` produces two unique loans of *different* places; treat the two halves as two places owned by the same function.
2. In `code/lean/Book/Ch26/Regions.lean`, show that the region discipline is *incomplete*: it rejects a program that never goes wrong. Use `[.alloc "x" 1, .copy "y" "x", .free "x", .copy "z" "y"]`,

prove that it is rejected with `decide`, and prove that it runs to completion. *Hint:* copying a dangling pointer does not follow it, so the semantics happily performs the copy. For the second part, imitate `share_finishes`. Which Rust error corresponds to the rejection?

3. Add a command `move (y x : Var)` to the heap language: `y` receives `x`'s value, and `x` becomes unassigned. Extend `Step`, `exec`, `CmdOK` and the checker, and repair the proofs. *Hint:* in the region discipline, `y` gets `x`'s type and `x`'s type becomes `none`. In `Agree`, the variable `x` no longer constrains anything, so the preservation case is a variant of `agree_copy`.
4. Rewrite `transfer` from `ch26_two_mut.rs` without `split_at_mut`, using two statements `accounts[0] -= amount;` and `accounts[1] += amount;`. Explain in terms of loans why this version compiles while the original does not. *Hint:* each statement creates a unique loan of the whole vector, and the loan is dead at the end of the statement.
5. In `ch26_aliasing.rs`, change the return type of `Tokenizer::next_token` to `Option<&str>`, compile, and read the error. Which elision rule assigned which lifetime to the result, and why does that make `tokens` fail? *Hint:* rule 3. The error appears in `tokens`, not in `next_token`: the body still type-checks, because returning a shorter borrow is always allowed.
6. (Pen and paper.) Write a typing rule for a lexical construct `letregion` $\rho\,\{\,k_1\,\}\,k_2$ for the heap language: allocate in ρ during k_1, free the whole region at the end of k_1, and continue with k_2. What side condition on the context after k_1 plays the role of the escape check? *Hint:* in Tofte and Talpin's version the result and the remaining context must not mention ρ. In our flow-sensitive version it is enough to remove ρ from Δ, as R-Free does; compare which programs each version accepts.

CHAPTER 27

Subtyping and Variance

In Chapter 1, Rust rejected `if ready { 1 } else { "not yet" }`, and we noted what the compiler did *not* do: it did not look for a common supertype of the two branches, the way a language with subtyping might. Rust has no class hierarchy, no `Cat` that is also an `Animal`, no `i32` that is also an `i64`. It would be easy to conclude that Rust has no subtyping at all.

It has some, and you rely on it every day. When you pass a string literal, of type `&'static str`, to a function that expects a `&'a str` for some short `'a`, the compiler silently uses the fact that a reference valid for the whole program is also valid for a shorter region. That is subtyping, on lifetimes. And as soon as there is subtyping on lifetimes, every type that contains a lifetime raises a question: if `&'static str` can be used as `&'a str`, can `Vec<&'static str>` be used as `Vec<&'a str>`? Can `fn(&'static str)` be used as `fn(&'a str)`? Can `&mut &'static str` be used as `&mut &'a str`? The answers are yes, no, and *absolutely not*, and the reasons are the theory of **variance**. Getting variance wrong is one of the few ways to write a use after free in a type system that otherwise looks sound, which is why it deserves a chapter.

In this chapter you will learn:

- what subtyping is, how the subsumption rule brings it into a type system, and why it makes the types of a program a preorder, hence a category;
- why lifetimes are Rust's only source of subtyping, and how it shows up in ordinary code;
- the four variances (covariant, contravariant, invariant, bivariant), the variance of `&'a T`, `&'a mut T`, `fn(T) -> U`, `Box<T>`, `Cell<T>` and `*mut T`, and how to make the compiler prove each one;
- why `&mut T` must be invariant in `T`, through the classic unsound program that invariance rejects;
- how Rust infers the variance of your own types, and how `PhantomData` chooses it when the compiler cannot;
- variance as functoriality: covariant and contravariant functors on the category of lifetimes, and variances as a small monoid;
- how to formalize a tiny language with subtyping in Lean and prove that the contravariant rule for functions is what makes it sound, and why Lean itself has coercions and subtypes but no subtyping.

27.1 Subtyping and subsumption

A type σ is a **subtype** of a type τ, written $\sigma <: \tau$, when every value of type σ can be used wherever a value of type τ is expected. In an object-oriented language, a `Cat` class extending `Animal` makes `Cat` a subtype of `Animal`; in a language with refinement types, the type of positive integers is a subtype of the type of integers. A type system brings subtyping into its judgments with one rule, called **subsumption**:

$$\frac{\Gamma \vdash e : \sigma \qquad \sigma <: \tau}{\Gamma \vdash e : \tau}\ \text{T-Sub}$$

If e has type σ, and σ is a subtype of τ, then e also has type τ. The subtyping relation itself is defined by its own rules. Two of them are always present:

$$\frac{}{\tau <: \tau}\ \text{S-Refl} \qquad\qquad \frac{\sigma <: \tau \qquad \tau <: \upsilon}{\sigma <: \upsilon}\ \text{S-Trans}$$

A relation that is reflexive and transitive is a **preorder**, and Chapter 16 showed that a preorder is a category: the objects are the types, and there is exactly one arrow $\sigma \to \tau$ when $\sigma <: \tau$ (and none otherwise). Identity arrows are S-Refl, composition is S-Trans, and the category laws hold automatically because there is never more than one arrow between two objects. Keep this picture in mind; it will turn variance into something you already know.

Subsumption has a cost. Every other rule you have seen is *syntax-directed*: the shape of the term determines which rule applies. T-Sub applies to every term, at any point, so a derivation can use it anywhere, any number of times. A type checker cannot try all of them. Practical checkers use an *algorithmic* formulation instead, in which subtyping is checked only where it is needed (for instance, when an argument is passed to a function) and in which transitivity is built in rather than being a rule. We will build one in Lean later in this chapter and prove that it decides the same relation.

There are two ways to read $\sigma <: \tau$. The *inclusion* reading says that the values of σ are among the values of τ: every positive integer *is* an integer, with no conversion. The *coercion* reading says that there is a conversion from σ to τ that the compiler inserts silently. The two readings coincide only when the conversion does nothing. Rust's lifetime subtyping is pure inclusion; its many coercions (deref, unsizing) are something else, and the last part of the chapter is about the difference.

27.2 Rust's only subtyping: lifetimes

Rust has no subtyping between nominal types. A struct cannot extend another, an `i32` must be converted to an `i64` explicitly, and a `Box<Valve>` that becomes a `Box<dyn Describe>` goes through a coercion that builds a new fat pointer, not through subtyping. The one source of subtyping is the outlives relation of Chapter 26. If `'long` outlives `'short`, a reference valid throughout `'long` is valid throughout `'short`:

$$\frac{\Delta \vdash \alpha : \beta}{\Delta \vdash \&\,\alpha\,\tau <: \&\,\beta\,\tau}\ \text{S-Ref}$$

Here $\&\,\alpha\,\tau$ is our notation for `&'a T`, and $\Delta \vdash \alpha : \beta$ is the outlives judgment of Chapter 26. Since `'static` outlives every region, `&'static T` is a subtype of every `&'a T`. The program `code/rust/examples/ch27_variance.rs` starts with the most ordinary use:

```rust
/// One region for both arguments: a `&'static str` is accepted where a
/// `&'a str` is expected, because `&'static str <: &'a str`.
fn longer<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

fn subsumption() {
    let fallback: &'static str = "default";
    let configured = String::from("from the config file");
    let chosen = longer(fallback, &configured); // 'a := the region of `configured`
    println!("chosen: {chosen}");
}
```

```
chosen: from the config file
```

`longer` wants two arguments of the same type `&'a str`. The compiler picks `'a` to be the region of `configured`, and uses subsumption to view `fallback`, of type `&'static str`, at the type `&'a str`. Nothing happens at run time: the pointer is the same pointer. Subtyping on lifetimes is invisible in the generated code, which is exactly what the inclusion reading predicts.

(There is a second, rarer kind of subtyping in Rust, between higher-ranked types: a function pointer of type `for<'a> fn(&'a str)`, which accepts a reference with any lifetime, is a subtype of `fn(&'x str)` for each particular `'x`. It is the same idea as instantiating a generic function, Chapter 12's universal quantifier, and the call `grow_argument(count)` below relies on it.)

27.3 Variance

Subtyping between two lifetimes induces subtyping between bigger types that contain them, and how it does so depends on the type constructor. This is **variance**. For a type constructor F with one parameter:

- F is **covariant** if $\sigma <: \tau$ implies $F\,\sigma <: F\,\tau$: the subtyping goes the same way.
- F is **contravariant** if $\sigma <: \tau$ implies $F\,\tau <: F\,\sigma$: the subtyping flips.
- F is **invariant** if $F\,\sigma <: F\,\tau$ holds only when σ and τ are the same type (or, more precisely, are subtypes of each other).
- F is **bivariant** if $F\,\sigma <: F\,\tau$ always holds: the parameter does not matter, which happens when it is not used at all.

A type constructor with several parameters has a variance in each. The rules for the classic constructors are these:

$$\frac{\sigma <: \sigma' \qquad \tau <: \tau'}{\sigma \times \tau <: \sigma' \times \tau'}\ \text{S-Pair} \qquad\qquad \frac{\sigma' <: \sigma \qquad \tau <: \tau'}{\sigma \to \tau <: \sigma' \to \tau'}\ \text{S-Arrow}$$

Pairs are covariant in both components: a pair of a cat and a dog is a pair of two animals. Functions are covariant in the result, but **contravariant in the argument**: notice that the premise about arguments is written $\sigma' <: \sigma$, the "wrong" way round. The reason is clearest with callbacks. Suppose a scheduler

wants a callback of type $Cat \to Unit$, which it will call with cats. A function of type $Animal \to Unit$, which handles any animal, is perfectly acceptable: whatever the scheduler passes, it can handle. So $Animal \to Unit <: Cat \to Unit$, even though $Cat <: Animal$. A function that *accepts more* is the more useful one, hence the subtype. The opposite, a function that handles only cats used where any animal may arrive, would be a disaster.

In Rust, the variances of the standard type constructors are fixed by the language and the standard library:

Type	Variance in `'a`	Variance in `T`
`&'a T`	covariant	covariant
`&'a mut T`	covariant	**invariant**
`Box<T>`, `Vec<T>`, `Option<T>`		covariant
`Cell<T>`, `RefCell<T>`, `UnsafeCell<T>`		**invariant**
`*const T` / `*mut T`		covariant / **invariant**
`fn(T) -> U`		**contravariant** in `T`, covariant in `U`
`PhantomData<T>`		covariant

Making the compiler prove it

You can make the compiler confirm every entry of the table. A function whose body is just `x`, with a signature that converts one type into another, compiles only if the conversion is a subtyping step. Each such function is a *witness*: its existence is a proof, checked by the compiler, that the claimed variance holds.

```
// `'long: 'short` reads "'long outlives 'short", so `&'long T <: &'short T`.

/// `&'a T` is covariant in `'a`...
fn shrink_ref<'long: 'short, 'short>(x: &'long str) -> &'short str {
    x
}

/// ...and in `T`: a reference to a longer-lived reference can be shortened.
fn shrink_ref_ref<'long: 'short, 'short, 'r>(x: &'r &'long str) -> &'r &'short str {
    x
}

/// `&'a mut T` is covariant in `'a` (but invariant in `T`: see
/// `ch27_mut_invariance`).
fn shrink_mut<'long: 'short, 'short, 'v>(x: &'long mut &'v str) -> &'short mut &'v str {
    x
}

/// `Box<T>`, `Vec<T>` and `Option<T>` are covariant in `T`.
fn shrink_box<'long: 'short, 'short>(x: Box<&'long str>) -> Box<&'short str> {
    x
}

fn shrink_vec<'long: 'short, 'short>(x: Vec<&'long str>) -> Vec<&'short str> {
    x
}

fn shrink_option<'long: 'short, 'short>(x: Option<&'long str>) -> Option<&'short str> {
    x
}
```

```
/// `fn() -> T` is covariant in `T`: a function that returns long-lived
/// references can stand in for one that returns short-lived ones.
fn shrink_result<'long: 'short, 'short>(f: fn() -> &'long str) -> fn() -> &'short str {
    f
}

/// `fn(T)` is *contravariant* in `T`: the conversion goes the other way. A
/// callback that accepts any short-lived `&str` can stand in for one that will
/// only ever be given long-lived ones.
fn grow_argument<'long: 'short, 'short>(
    f: fn(&'short str) -> usize,
) -> fn(&'long str) -> usize {
    f
}

/// Two contravariances make a covariance: `fn(fn(T))` is covariant in `T`.
fn shrink_twice<'long: 'short, 'short>(
    f: fn(fn(&'long str) -> usize) -> usize,
) -> fn(fn(&'short str) -> usize) -> usize {
    f
}
```

```
shrink_ref, shrink_ref_ref: static
shrink_mut, then a write through it: replaced
shrink_box, shrink_vec, shrink_option: boxed ["a", "b"] Some("some")
shrink_result: a static string
grow_argument: 18
shrink_twice: 6
```

Read the signatures carefully, because the direction of each one is the content. `shrink_ref` turns `&'long str` into `&'short str`: covariance in the lifetime. `shrink_ref_ref` does the same one level down, inside a reference: covariance in `T`. `shrink_mut` shortens the *outer* lifetime of a mutable reference, but leaves the referent type `&'v str` alone: that is all that is allowed, as we are about to see. `grow_argument` goes the other way, from a callback that accepts `&'short str` to one that accepts `&'long str`: the callback that accepts more is the subtype. And `shrink_twice` shows what happens when a contravariant position is nested in another contravariant position: the two flips cancel, and `fn(fn(T))` is covariant in `T`. Every one of these functions compiles; reversing the direction of any of them would not.

The contravariance of `fn(T)` is also responsible for errors. Here a function that only accepts `'static` strings is passed where a callback for a short-lived string is needed:

```
fn needs_static(msg: &'static str) -> usize {
    msg.len()
}

fn call_with<'a>(callback: fn(&'a str) -> usize, msg: &'a str) -> usize {
    callback(msg)
}

pub fn report() {
    let msg = String::from("built at run time");
    call_with(needs_static, &msg);
}
```

```
error[E0597]: `msg` does not live long enough
  --> compile_fail/ch27_contravariance.rs:16:29
   |
15 |     let msg = String::from("built at run time");
```

```
   |         --- binding `msg` declared here
16 |     call_with(needs_static, &msg);
   |               ------------  ^^^^ borrowed value does not live long enough
   |               |
   |               coercion requires that `msg` is borrowed for `'static`
17 | }
   | - `msg` dropped here while still borrowed
```

`call_with` wants a callback of type `fn(&'a str) -> usize` and a message of type `&'a str`, with one `'a`. The callback `needs_static` has type `fn(&'static str) -> usize`. For it to be a `fn(&'a str) -> usize`, contravariance requires `&'a str <: &'static str`, which holds only if `'a` is `'static`. So the compiler sets `'a` to `'static`, and then `&msg` would have to be a `&'static str`, which it is not: `msg` is a local `String`. The compiler reports the consequence (E0597), and its label mentions the "coercion" of the function item `needs_static` to a function pointer, the place where the subtyping requirement arose. This is the scheduler disaster in Rust form: a callback that can handle only long-lived strings, offered to code that will call it with a short-lived one.

27.4 Why `&mut T` must be invariant

Covariance of `&'a T` in `T` is safe because a shared reference only reads: if you can read a `&'static str` out of it, you can certainly treat what you read as a `&'short str`. A mutable reference also *writes*, and writing reverses the direction. Here is the classic program that shows why, kept in `code/rust/compile_fail/ch27_mut_invariance.rs`:

```rust
fn overwrite<T>(dst: &mut T, src: T) {
    *dst = src;
}

pub fn evil() {
    let mut s: &'static str = "static data";
    {
        let local = String::from("short-lived");
        overwrite(&mut s, &local);
    }
    println!("{s}");
}
```

Suppose, for a moment, that `&mut T` were covariant in `T`. Then `&mut s : &mut &'static str` could be viewed as `&mut &'short str`, where `'short` is the region of `local`. The call `overwrite(&mut s, &local)` would type-check with `T = &'short str`, and it would store a pointer to `local` into `s`. At the closing brace `local` is dropped, and `println!("{s}")` reads freed memory, through a variable whose type still promises a string that lives forever. Every step is locally reasonable; only the covariance assumption is wrong.

Rust's actual rule rejects the program:

```
error[E0597]: `local` does not live long enough
  --> compile_fail/ch27_mut_invariance.rs:15:27
   |
12 |     let mut s: &'static str = "static data";
   |                ------------ type annotation requires that `local` is borrowed for
`'static`
13 |     {
14 |         let local = String::from("short-lived");
```

```
   |             ----- binding `local` declared here
15 |         overwrite(&mut s, &local);
   |                           ^^^^^^ borrowed value does not live long enough
16 |     }
   |     - `local` dropped here while still borrowed
```

Because `&mut T` is invariant in `T`, the type `&mut &'static str` of the first argument fixes `T = &'static str` exactly, with no room to shorten it. Then the second argument `&local` must be a `&'static str`, and the compiler explains that the type annotation on `s` "requires that `local` is borrowed for `'static`", which is impossible.

The same attack can be written inside a single function, with the conflict in the signature itself. Here the compiler names the culprit explicitly:

```
pub fn remember(log: &mut Vec<&'static str>, line: &str) {
    log.push(line);
}
```

```
error[E0521]: borrowed data escapes outside of function
 --> compile_fail/ch27_invariance_escape.rs:8:5
  |
7 | pub fn remember(log: &mut Vec<&'static str>, line: &str) {
  |                 ---                          ----  - let's call the lifetime of this
reference `'1`
  |                 |                            |
  |                 |                            `line` is a reference that is only valid in
the function body
  |                 `log` declared here, outside of the function body
8 |     log.push(line);
  |     ^^^^^^^^^^^^^^
  |     |
  |     `line` escapes the function body here
  |     argument requires that `'1` must outlive `'static`
  |
  = note: requirement occurs because of a mutable reference to `Vec<&str>`
  = note: mutable references are invariant over their type parameter
  = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

If `&mut Vec<&'static str>` could be viewed as `&mut Vec<&'1 str>`, the `push` would type-check, and the caller would later read a dangling `&'static str` out of its log. The two notes at the end say exactly what this section says: "mutable references are invariant over their type parameter".

Interior mutability writes through a *shared* reference, so it has the same problem, and `Cell<T>` is invariant too:

```
use std::cell::Cell;

pub fn set_label(slot: &Cell<&'static str>, label: &str) {
    slot.set(label);
}
```

```
error[E0521]: borrowed data escapes outside of function
 --> compile_fail/ch27_cell_invariance.rs:9:5
  |
8 | pub fn set_label(slot: &Cell<&'static str>, label: &str) {
  |                  ----                       -----  - let's call the lifetime of this
reference `'1`
  |                  |                          |
  |                  |                          `label` is a reference that is only valid in
```

```
the function body
  |                  `slot` declared here, outside of the function body
9 |     slot.set(label);
  |     ^^^^^^^^^^^^^^^
  |     |
  |     `label` escapes the function body here
  |     argument requires that `'1` must outlive `'static`
  |
  = note: requirement occurs because of the type `Cell<&str>`, which makes the generic
argument `&str` invariant
  = note: the struct `Cell<T>` is invariant over the parameter `T`
  = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about
variance
```

The principle behind all three is worth stating as a rule of thumb: **reading is covariant, writing is contravariant, and both is invariant.** A place you can read from is a *source* of τ values, like a function $Unit \to \tau$, covariant in τ. A place you can write to is a *sink* for τ values, like a function $\tau \to Unit$, contravariant in τ. A mutable reference is both at once, and the only way for both views to be subtypes is for the types to be equal. The same reasoning explains the rest of the table: `*const T` only reads and is covariant, `*mut T` writes and is invariant, and the lifetime `'a` of `&'a mut T` stays covariant because shortening a loan only gives the permission back sooner, which is always safe.

Variance is not a style rule. A type with the wrong variance, typically a hand-written abstraction over raw pointers, lets safe code produce exactly the dangling reference of the classic program. That is why the next section matters to anyone who writes `unsafe` code.

27.5 The variance of your own types

You never declare the variance of a struct in Rust; the compiler **infers** it from the fields. Each field's type has a variance in each parameter, and the struct's variance is the strongest constraint among them: if a parameter appears covariantly in one field and contravariantly in another, the struct is invariant in it; if it appears in no field at all, the struct would be bivariant in it. Nesting composes: a parameter inside a `fn` argument inside a `fn` argument is covariant, as `shrink_twice` showed. We will make this calculation precise in Lean.

A parameter that appears in no field is a problem, because a bivariant parameter means nothing at all, and Rust rejects it (Chapter 25 met the same error for a typestate parameter):

```
pub struct RawView<'a, T> {
    ptr: *const T,
    len: usize,
}
```

```
error[E0392]: lifetime parameter `'a` is never used
 --> compile_fail/ch27_unused_param.rs:6:20
  |
6 | pub struct RawView<'a, T> {
  |                    ^^ unused lifetime parameter
  |
  = help: consider removing `'a`, referring to it in a field, or using a marker such as
`PhantomData`
```

The help names the solution: `PhantomData`, a zero-sized marker type in `std::marker`. A field of type `PhantomData<X>` takes no space, and tells the compiler to treat the struct, for variance and for a few other purposes, *as if* it contained an `X`. By choosing `X`, you choose the variance:

Marker	Variance in `T`	Notes
`PhantomData<T>`	covariant	"owns a `T`": inherits `T`'s `Send` / `Sync`
`PhantomData<&'a T>`	covariant (and in `'a`)	a borrow of `T` for `'a`
`PhantomData<&'a mut T>`	invariant (covariant in `'a`)	a unique borrow
`PhantomData<fn() -> T>`	covariant	produces `T`s; always `Send` and `Sync`
`PhantomData<fn(T)>`	contravariant	consumes `T`s; always `Send` and `Sync`
`PhantomData<fn(T) -> T>`	invariant	always `Send` and `Sync`

The last column matters as much as the second. Auto traits such as `Send` and `Sync` are also computed from the fields, so the marker decides them too. A function pointer is always `Send` and `Sync`, whatever its argument type, so the `fn` markers give the variance you want without dragging in the thread-safety of `T`. `PhantomData<T>` also tells the compiler's drop checker that dropping the struct may drop a `T`, which matters for types that really own values through raw pointers.

Here is a common pattern, a typed identifier that cannot be confused across tables:

```
/// A typed identifier: an `Id<Order>` cannot be passed where an
/// `Id<Customer>` is expected. The `T` is only a marker, and the choice of
/// marker matters:
/// - `PhantomData<T>` would say "an `Id<T>` owns a `T`", so `Id<Rc<..>>`
///   would not be `Send` and the drop checker would assume a `T` is dropped;
/// - `PhantomData<fn() -> T>` says "an `Id<T>` produces `T`s": covariant in
///   `T`, always `Send` and `Sync`, owns nothing.
struct Id<T> {
    raw: u64,
    _marker: PhantomData<fn() -> T>,
}

// Derived `Clone` and `Copy` would require `T: Clone`; an ID is always copyable.
impl<T> Clone for Id<T> {
    fn clone(&self) -> Self {
        *self
    }
}
impl<T> Copy for Id<T> {}

impl<T> Id<T> {
    fn new(raw: u64) -> Self {
        Id {
            raw,
            _marker: PhantomData,
        }
    }
}

struct Order;
struct Customer;

fn customer_of(order: Id<Order>) -> Id<Customer> {
    Id::new(order.raw / 100)
}

fn require_send_sync<T: Send + Sync>(_: &T) -> &'static str {
```

```
    "Send + Sync"
}
```

`Id<Order>` and `Id<Customer>` are different types, so passing one where the other is expected is a type error, while at run time both are a plain `u64`. The marker `PhantomData<fn() -> T>` makes `Id<T>` covariant, owning nothing, and thread-safe regardless of `T`, which is what the program checks with an `Rc`, the standard example of a type that is neither `Send` nor `Sync`:

```
order 4217 belongs to customer 42
Id<Rc<String>> is Send + Sync
```

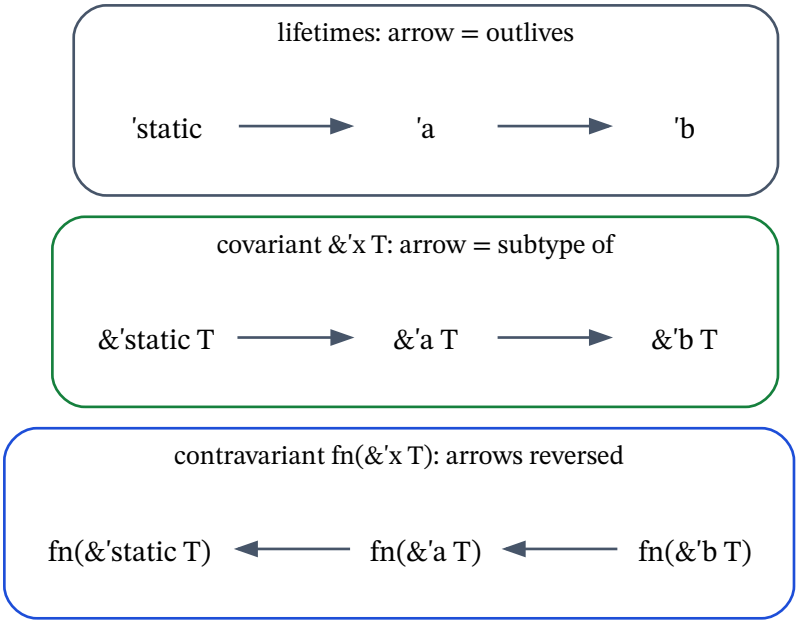
With `PhantomData<T>` instead, `Id<Rc<String>>` would be neither `Send` nor `Sync`, for no good reason.

27.6 Variance as functoriality

Now return to the picture from the beginning of the chapter. Lifetimes ordered by outlives form a preorder, so a category $\mathcal{L}$: one object per lifetime, one arrow $\alpha \to \beta$ when α outlives β. Types ordered by subtyping form another category $\mathcal{T}$. A type with a lifetime parameter, such as `&'x i32`, is a map from lifetimes to types: $\alpha \mapsto \& \, \alpha$ `i32`.

Chapter 17 defined a **functor** between categories: a map on objects and a map on arrows that preserves identities and composition. Between preorders, there is at most one arrow between two objects, so preserving identities and composition is automatic, and a functor is simply a map on objects that sends arrows to arrows: a **monotone** map. Covariance of `&'x i32` in `'x` says exactly that whenever $\alpha \to \beta$ in $\mathcal{L}$, there is an arrow $\& \, \alpha$ `i32` $\to \& \, \beta$ `i32` in $\mathcal{T}$. **A covariant type constructor is a functor $\mathcal{L} \to \mathcal{T}$.**

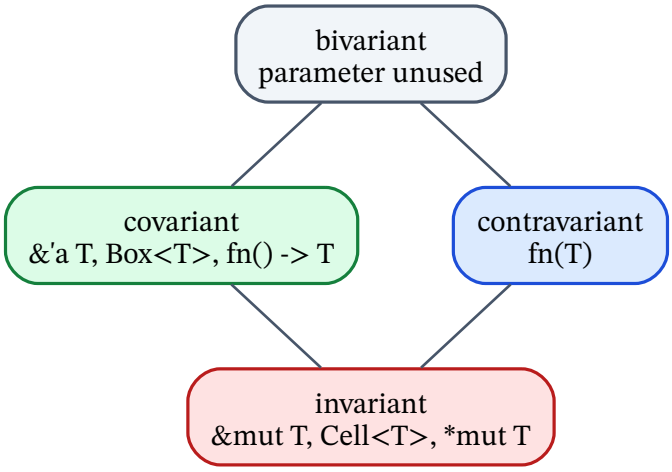
Contravariance reverses arrows. Chapter 16 built the *opposite* category $\mathcal{L}^{op}$, with the same objects and every arrow turned around. A contravariant constructor such as `fn(&'x i32)` sends an arrow $\alpha \to \beta$ to an arrow $F\beta \to F\alpha$, which is an arrow-preserving map from $\mathcal{L}^{op}$: **a contravariant type constructor is a functor $\mathcal{L}^{op} \to \mathcal{T}$.** This is the same "contravariant functor" that Chapter 17 met with predicates and comparators.



Covariance and contravariance as functors from the category of lifetimes: &'x T sends each outlives arrow to a subtyping arrow in the same direction, fn(&'x T) to one in the opposite direction.

The two other variances fit the same pattern. An invariant constructor sends an arrow $\alpha \to \beta$ to an arrow only when $\alpha = \beta$: it is a functor only from the *discrete* category on the lifetimes, which keeps the objects and throws away every arrow except the identities. A bivariant constructor sends every pair of lifetimes to related types: it is a functor from the category with an arrow between any two objects. So the four variances are the four ways a type can be functorial in its parameter.

Composition of functors explains the sign rule. If F and G are both contravariant, $F \circ G$ turns arrows around twice, which is not turning them around at all: a functor $(\mathcal{L}^{op})^{op} = \mathcal{L} \to \mathcal{T}$, covariant. That is `shrink_twice`. The variances, with composition, behave like signs under multiplication, with two extra elements: invariance absorbs covariance and contravariance, and bivariance absorbs everything. When a parameter occurs in several places, the constraints are combined by a *meet* in the lattice below: covariant and contravariant together give invariant, and bivariant (no occurrence) is neutral.



The four variances ordered by how much they allow: bivariant on top, covariant and contravariant in the middle, invariant at the bottom. Combining two occurrences of a parameter takes the meet.

Functors in Lean

The file `code/lean/Book/Ch27/Functoriality.lean` makes this literal, with the book's `Category` class from Chapter 16 (`Book.Ch16.Category`, which it imports). Lifetimes are modeled by the point where they end, so `'a` outlives `'b` when `b ≤ a`, and types by a small language of Rust-like types, `RTy ℕ`, defined in `Variance.lean` below together with its subtyping relation `RSub`:

```
/-- Lifetimes, named by their end points. -/
structure Lifetime where
  endsAt : ℕ

/-- The category of lifetimes: a morphism `a → b` is a proof that `a`
outlives `b`. There is at most one. -/
instance : Category Lifetime where
  Hom a b := PLift (b.endsAt ≤ a.endsAt)
  id _ := ⟨Nat.le_refl _⟩
  comp g f := ⟨Nat.le_trans g.down f.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

/-- Closed Rust-like types. -/
structure RType where
  ty : RTy ℕ

/-- The category of types: a morphism `t → u` is a proof that `t` is a
subtype of `u`. -/
instance : Category RType where
  Hom t u := PLift (RSub t.ty u.ty)
  id t := ⟨RSub.refl t.ty⟩
  comp g f := ⟨RSub.trans f.down g.down⟩
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

A morphism is a *proof* wrapped in `PLift`. The class requires morphisms to live in `Type`, and a proof of `b ≤ a` lives in `Prop`; `PLift` lifts a proposition into a type with one element per proof. Since all proofs of a proposition are equal (proof irrelevance, Chapter 10), there is at most one morphism between two objects, and the category laws hold by `rfl`.

We need a notion of functor for the book's class. Chapter 17 defines one, `CatFunctor`; since a file of this chapter may import only Chapter 16 and its own files, we redefine a minimal version here under the name `Fun`:

```
/-- A functor between two categories of the book's `Category` class. -/
structure Fun (C D : Type) [Category C] [Category D] where
  obj : C → D
  map : {a b : C} → (a ⟶ b) → (obj a ⟶ obj b)
  map_id : ∀ a, map (Category.id a) = Category.id (obj a)
  map_comp : ∀ {a b c : C} (g : b ⟶ c) (f : a ⟶ b), map (g ≫ f) = map g ≫ map f

/-- In these categories any two morphisms with the same ends are equal, so
the functor laws hold automatically: a functor is just a monotone map. -/
def Fun.ofMonotone {C : Type} [Category C] (obj : C → RType)
    (map : {a b : C} → (a ⟶ b) → (obj a ⟶ obj b)) : Fun C RType where
  obj := obj
  map := map
  map_id _ := rfl
  map_comp _ _ := rfl
```

`Fun.ofMonotone` is the observation above, as a definition (Chapter 17's `monotoneFunctor` is the same idea for Mathlib's preorders): to build a functor into a thin category, a map on objects and a map on arrows suffice, and the laws are proved by `rfl`, because two morphisms with the same ends are definitionally equal. Now covariant and contravariant types become functors:

```
/-- A covariant type (in its lifetime parameter `'x`) is a functor from
lifetimes to types. -/
def covariantFunctor (t : RTy Lft) (h : t.var = .co) : Fun Lifetime RType :=
  Fun.ofMonotone (fun a => ⟨t.inst a.endsAt⟩)
    (fun f => ⟨RTy.covariant h f.down⟩)

/-- A contravariant type is a functor from the *opposite* category of
lifetimes, where every arrow is reversed. -/
def contravariantFunctor (t : RTy Lft) (h : t.var = .contra) :
    Fun (Op Lifetime) RType :=
  Fun.ofMonotone (fun a => ⟨t.inst a.unop.endsAt⟩)
    (fun f => ⟨RTy.contravariant h f.down⟩)

/-- `&'x i32` is a functor `Lifetime ⥤ RType`... -/
def refFunctor : Fun Lifetime RType := covariantFunctor refX rfl

/-- ...and `fn(&'x i32)` a functor `Lifetimeᵒᵖ ⥤ RType`. -/
def callbackFunctor : Fun (Op Lifetime) RType :=
  contravariantFunctor (.fn refX .i32) rfl
```

`covariantFunctor` takes a type `t` with a lifetime parameter and a proof that its inferred variance is `co`, and produces a functor from lifetimes. The map on arrows is the theorem `RTy.covariant`, which we prove in the next section from the soundness of variance inference. `contravariantFunctor` does the same from `Op Lifetime`, Chapter 16's opposite category, whose arrows are the reversed arrows of `Lifetime`. The two examples at the end pass `rfl` as the variance proof: Lean computes the variance of `&'x i32` and of `fn(&'x i32)` and checks that they are `co` and `contra`.

Mathlib has the same construction in general: every preorder is a category, and a monotone function is a functor.

```
/-- In Mathlib, every preorder is a category (`a ⟶ b` when `a ≤ b`), and a
monotone function is a functor between such categories. -/
example : CategoryTheory.Functor ℕ ℕ :=
  Monotone.functor (f := fun n : ℕ => 2 * n) (fun _ _ h => Nat.mul_le_mul_left 2 h)
```

`Monotone.functor` takes a proof of monotonicity and returns a `CategoryTheory.Functor` (written `ℕ ⥤ ℕ` in Mathlib's notation).

27.7 Variance in Lean

The file `code/lean/Book/Ch27/Variance.lean` turns the algebra of variances into definitions and theorems, and uses them to prove that variance inference, the calculation the compiler performs for your structs, is sound.

The algebra of variances

```
/-- How a type depends on a parameter. -/
inductive Variance where
  | co      -- covariant: a longer parameter gives a subtype
  | contra  -- contravariant: a shorter parameter gives a subtype
  | inv     -- invariant: only the same parameter
  | bi      -- bivariant: the parameter does not matter
  deriving DecidableEq, Repr
```

```
/-- `v * w` is the variance of a position of variance `w` nested inside a
position of variance `v`: like signs, two contravariances make a covariance. -/
def mul : Variance → Variance → Variance
  | bi, _ => bi
  | _, bi => bi
  | inv, _ => inv
  | _, inv => inv
  | co, w => w
  | contra, co => contra
  | contra, contra => co

instance : Mul Variance := ⟨mul⟩
instance : One Variance := ⟨co⟩
```

`mul` is composition: `v * w` is the variance of a position of variance `w` nested inside a position of variance `v`. Bivariance absorbs everything, then invariance absorbs what is left, covariance is the identity, and two contravariances make a covariance. The instances `Mul` and `One` let us write `*` and `1`.

```
/-- Two occurrences of a parameter, with variances `v` and `w`: the parameter
may vary only in ways that both allow. -/
def meet : Variance → Variance → Variance
  | bi, w => w
  | v, bi => v
  | co, co => co
  | contra, contra => contra
  | _, _ => inv

@[inherit_doc] scoped infixl:65 " ⊓ " => meet
```

`meet` combines two occurrences of the same parameter: bivariance (no occurrence) is neutral, two equal variances stay as they are, and any other combination is invariant.

```
/-- Composition of variances is associative, commutative, and `co` is its
unit: the variances form a commutative monoid. -/
instance : CommMonoid Variance where
  mul_assoc a b c := by cases a <;> cases b <;> cases c <;> rfl
  one_mul a := by cases a <;> rfl
  mul_one a := by cases a <;> rfl
  mul_comm a b := by cases a <;> cases b <;> rfl
```

```
/-- info: Ch27.Variance.co -/
#guard_msgs in
#eval contra * contra

/-- info: Ch27.Variance.contra -/
#guard_msgs in
#eval contra ^ 3

/-- info: Ch27.Variance.inv -/
#guard_msgs in
#eval co ⊓ contra
```

`CommMonoid` is Mathlib's class of commutative monoids. Its laws are proved by brute force: `cases a <;> cases b <;> cases c <;> rfl` splits into the $4 \times 4 \times 4 = 64$ combinations and checks each by computation. Being a monoid instance gives us Mathlib's notation for free, including powers: `contra ^ 3` is three nested contravariant positions, which is contravariant.

What a variance means

Tables of rules are only as trustworthy as their meaning. A variance has a precise meaning: given the relation that holds between two parameters (for lifetimes, "outlives"), it says which relation must hold for the types to be in the subtyping relation.

```
/-- The meaning of a variance: how it transforms a relation `R` on parameters
(for us, "outlives" on lifetimes) into a relation on parameters that is safe
to use for the type. -/
def apply {α : Type} : Variance → (α → α → Prop) → (α → α → Prop)
  | co, R => R
  | contra, R => fun a b => R b a
  | inv, R => fun a b => R a b ∧ R b a
  | bi, _ => fun _ _ => True
```

Covariance uses the relation as it is, contravariance flips it, invariance needs it in both directions, and bivariance needs nothing. With this meaning, the algebra is no longer arbitrary: the two operations are exactly composition and intersection of meanings.

```
/-- Composing variances is composing their meanings. -/
theorem apply_mul {α : Type} (v w : Variance) (R : α → α → Prop) (a b : α) :
    (v * w).apply R a b ↔ v.apply (w.apply R) a b := by
  cases v <;> cases w <;> simp [HMul.hMul, Mul.mul, mul, apply] <;> tauto

/-- Meeting variances is intersecting their meanings. -/
theorem apply_meet {α : Type} (v w : Variance) (R : α → α → Prop) (a b : α) :
    (v ⊓ w).apply R a b ↔ v.apply R a b ∧ w.apply R a b := by
  cases v <;> cases w <;> simp [meet, apply] <;> tauto
```

Both proofs are case analyses over the 16 combinations, with `simp` unfolding the definitions. What remains in some cases is a small propositional fact such as `R a b ∧ R b a ↔ R b a ∧ R a b`, which `tauto` (a Mathlib tactic that proves tautologies of propositional logic) finishes. In `apply_mul`, the `simp` set includes `HMul.hMul` and `Mul.mul` so that the `*` notation unfolds to our `mul`. These two theorems say that the map from variances to relation transformers turns `*` into composition and the meet into conjunction: the variance algebra is a faithful summary of how relations are transformed.

Variance inference, proved sound

Now a small language of Rust-like types, with the lifetime of each reference drawn from a set `L`:

```
/-- Rust-like types whose lifetimes are drawn from `L`. -/
inductive RTy (L : Type) where
  | i32
  | ref (a : L) (t : RTy L)       -- &'a T
  | refMut (a : L) (t : RTy L)    -- &'a mut T
  | fn (arg res : RTy L)          -- fn(T) -> U
  | cell (t : RTy L)              -- Cell<T>
  | box (t : RTy L)               -- Box<T>
  | pair (t u : RTy L)            -- a struct with two fields
  deriving Repr

/-- A lifetime in a type with one parameter: the parameter `'x`, or a fixed
scope, named by the point where it ends. -/
inductive Lft where
  | x
  | scope (n : ℕ)
  deriving Repr
```

`RTy ℕ` is a closed type, whose lifetimes are end points, and `RTy Lft` is a type with one lifetime parameter `'x`. Subtyping on closed types follows the Rust table:

```
/-- Subtyping on closed types, whose lifetimes are end points: `'a` outlives
`'b` when `b ≤ a`. -/
inductive RSub : RTy ℕ → RTy ℕ → Prop where
  | refl (t : RTy ℕ) : RSub t t
  | trans {t u w : RTy ℕ} : RSub t u → RSub u w → RSub t w
  | ref {a b : ℕ} {t u : RTy ℕ} : b ≤ a → RSub t u → RSub (.ref a t) (.ref b u)
  | refMut {a b : ℕ} {t u : RTy ℕ} :
      b ≤ a → RSub t u → RSub u t → RSub (.refMut a t) (.refMut b u)
  | fn {t t' u u' : RTy ℕ} : RSub t' t → RSub u u' → RSub (.fn t u) (.fn t' u')
  | cell {t u : RTy ℕ} : RSub t u → RSub u t → RSub (.cell t) (.cell u)
  | box {t u : RTy ℕ} : RSub t u → RSub (.box t) (.box u)
  | pair {t t' u u' : RTy ℕ} : RSub t t' → RSub u u' → RSub (.pair t u) (.pair t' u')
```

Invariance is written as subtyping in both directions (`RSub t u → RSub u t`), as in the definition at the start of the chapter. The variance that the compiler would infer for a type with a parameter `'x` is computed by recursion, and `inst` replaces `'x` by a concrete lifetime:

```
/-- The variance of a type in its lifetime parameter `'x`. -/
def Lft.var : Lft → Variance
  | .x => co
  | .scope _ => bi

/-- The variance of a type in `'x`: the meet over every occurrence of `'x`,
each weighted by the variances of the positions it sits in. -/
def RTy.var : RTy Lft → Variance
  | .i32 => bi
  | .ref a t => a.var ⊓ (co * t.var)
  | .refMut a t => a.var ⊓ (inv * t.var)
  | .fn t u => (contra * t.var) ⊓ (co * u.var)
  | .cell t => inv * t.var
  | .box t => co * t.var
  | .pair t u => t.var ⊓ u.var

/-- Replace the parameter `'x` by the scope ending at `a`. -/
def Lft.inst (a : ℕ) : Lft → ℕ
  | .x => a
  | .scope n => n

/-- Replace `'x` by the scope ending at `a` everywhere in a type. -/
def RTy.inst (a : ℕ) : RTy Lft → RTy ℕ
  | .i32 => .i32
  | .ref l t => .ref (l.inst a) (t.inst a)
  | .refMut l t => .refMut (l.inst a) (t.inst a)
  | .fn t u => .fn (t.inst a) (u.inst a)
  | .cell t => .cell (t.inst a)
```

```
  | .box t => .box (t.inst a)
  | .pair t u => .pair (t.inst a) (u.inst a)
```

Each case is a line of the Rust table, written with the algebra: a reference `&'a T` is covariant in `'a` (if `'a` is `'x`) and covariant in `T`, so its variance is the meet of the two; a mutable reference has `inv` in front of its referent; a function has `contra` in front of its argument and `co` in front of its result; a two-field struct is the meet of its fields. Lean computes:

```
/-- `&'x i32` -/
def refX : RTy Lft := .ref .x .i32

/-- info: Ch27.Variance.co -/
#guard_msgs in
#eval refX.var

-- `&'s mut &'x i32`: the classic invariant position.
/-- info: Ch27.Variance.inv -/
#guard_msgs in
#eval (RTy.refMut (.scope 0) refX).var

-- `fn(&'x i32)` and `fn(fn(&'x i32))`
/-- info: Ch27.Variance.contra -/
#guard_msgs in
#eval (RTy.fn refX .i32).var

/-- info: Ch27.Variance.co -/
#guard_msgs in
#eval (RTy.fn (.fn refX .i32) .i32).var

-- `struct Holder<'x> { r: &'x i32, hits: Cell<i32> }`
/-- info: Ch27.Variance.co -/
#guard_msgs in
#eval (RTy.pair refX (.cell .i32)).var
```

The last line is a struct `Holder<'x>` with a reference field and a `Cell<i32>` field. The `Cell` makes its contents invariant, but its contents do not mention `'x`, so the field is bivariant in `'x` and the struct is covariant.

The theorem that makes all this trustworthy says that the inferred variance is *sound*: whatever relation between two lifetimes the variance asks for, it suffices for subtyping.

```
/-- "`'a` outlives `'b`" on end points. -/
def Outlives (a b : ℕ) : Prop := b ≤ a

/-- **Variance inference is sound.** Whatever relation between `a` and `b` the
inferred variance asks for, instantiating `'x` with `a` gives a subtype of
instantiating it with `b`. -/
theorem RTy.var_sound (t : RTy Lft) :
    ∀ a b, t.var.apply Outlives a b → RSub (t.inst a) (t.inst b) := by
  induction t with
  | i32 => exact fun _ _ _ => RSub.refl _
  | ref l t ih =>
    intro a b h
    rw [RTy.var, apply_meet, apply_mul] at h
    obtain ⟨hl, ht⟩ := h
    apply RSub.ref _ (ih a b ht)
    -- The lifetime of the reference: `'x` itself, or a fixed scope.
    cases l with
    | x => exact hl
    | scope n => exact Nat.le_refl n
  | refMut l t ih =>
    intro a b h
    rw [RTy.var, apply_meet, apply_mul] at h
```

```
    -- Invariance gives both directions for the referent.
    obtain ⟨hl, ht₁, ht₂⟩ := h
    apply RSub.refMut _ (ih a b ht₁) (ih b a ht₂)
    cases l with
    | x => exact hl
    | scope n => exact Nat.le_refl n
  | fn t u iht ihu =>
    intro a b h
    rw [RTy.var, apply_meet, apply_mul, apply_mul] at h
    obtain ⟨ht, hu⟩ := h
    -- Contravariance swaps `a` and `b` for the argument.
    exact RSub.fn (iht b a ht) (ihu a b hu)
  | cell t ih =>
    intro a b h
    rw [RTy.var, apply_mul] at h
    exact RSub.cell (ih a b h.1) (ih b a h.2)
  | box t ih =>
    intro a b h
    rw [RTy.var, apply_mul] at h
    exact RSub.box (ih a b h)
  | pair t u iht ihu =>
    intro a b h
    rw [RTy.var, apply_meet] at h
    exact RSub.pair (iht a b h.1) (ihu a b h.2)
```

The proof is by induction on the type, with one case per constructor, and each case has the same shape: rewrite the hypothesis `h` with the definition of `RTy.var` and the two meaning theorems, then build the subtyping proof from the pieces. Take the `fn t u` case. After `rw [RTy.var, apply_meet, apply_mul, apply_mul] at h`, the hypothesis says that `contra.apply (t.var.apply Outlives) a b` and `co.apply (u.var.apply Outlives) a b` hold. The first is, by the definition of `apply`, the relation for `t` with `a` and `b` *swapped*. So the induction hypothesis for the argument is used at `b a`, giving `RSub (t.inst b) (t.inst a)`, which is exactly the contravariant premise of `RSub.fn`. The `refMut` and `cell` cases use their induction hypothesis twice, at `a b` and at `b a`, for the two directions that invariance demands. In the `ref` and `refMut` cases, `cases l` distinguishes the parameter `'x` (where the outlives fact comes from `h`) from a fixed scope (where both instantiations give the same lifetime, and `Nat.le_refl` applies).

The corollaries specialize the theorem to the two variances we used to build functors:

```
/-- A covariant type: lengthening `'x` gives a subtype. -/
theorem RTy.covariant {t : RTy Lft} (h : t.var = co) {a b : ℕ} (hab : b ≤ a) :
    RSub (t.inst a) (t.inst b) :=
  t.var_sound a b (by rw [h]; exact hab)

/-- A contravariant type: *shortening* `'x` gives a subtype. -/
theorem RTy.contravariant {t : RTy Lft} (h : t.var = contra) {a b : ℕ}
    (hab : a ≤ b) : RSub (t.inst a) (t.inst b) :=
  t.var_sound a b (by rw [h]; exact hab)
```

27.8 A tiny language with subtyping

`RSub` was given by rules, chosen to match Rust. Why are those the *right* rules? In particular, why must the function rule be contravariant? To answer, a type system needs a meaning, and the subtyping rules must be sound for it. The file `code/lean/Book/Ch27/Subtyping.lean` does this for a tiny language, small enough that the whole argument fits in a few theorems.

Types as refinements

The language has three base types, `pos`, `nat` and `int`, all represented at run time by integers, and function types. To give types a meaning, each type is indexed by its *shape*, which says what its values look like at run time:

```
/-- The shape of a type: what its values look like at run time. -/
inductive Shape where
  | base
  | arrow (s t : Shape)

/-- The run-time values of each shape: integers, and functions. -/
@[reducible] def Shape.Val : Shape → Type
  | .base => ℤ
  | .arrow s t => s.Val → t.Val
```

`Shape.Val` computes a Lean type for each shape: integers for base types, functions for arrows. It is marked `@[reducible]` so that Lean unfolds it freely when checking types, which lets us write `0 < z` for a `z` of type `Shape.base.Val`.

```
/-- Types, indexed by their shape. -/
inductive Ty : Shape → Type where
  | pos : Ty .base
  | nat : Ty .base
  | int : Ty .base
  | arrow {s t : Shape} (σ : Ty s) (τ : Ty t) : Ty (.arrow s t)

/-- Which values belong to a type. A function belongs to `σ ⇒ τ` when it maps
every value of `σ` to a value of `τ`. -/
def Ty.Holds : {s : Shape} → Ty s → s.Val → Prop
  | _, .pos, z => 0 < z
  | _, .nat, z => 0 ≤ z
  | _, .int, _ => True
  | _, .arrow σ τ, f => ∀ x, σ.Holds x → τ.Holds (f x)

/-- `σ ⇒ τ` is the type of functions from `σ` to `τ`. -/
scoped infixr:30 " ⇒ " => Ty.arrow
```

`Ty` is an indexed family (Chapter 25 used them for typestate): a `Ty s` is a type of shape `s`. `Ty.Holds τ v` says that the run-time value `v` belongs to the type `τ`. A positive integer is one that is greater than zero, and a function belongs to $\sigma \Rightarrow \tau$ when it sends every value of σ to a value of τ. This is the inclusion reading of types, made precise: a type is a *predicate* on run-time values, also called a *refinement*.

The subtyping rules:

```
/-- The subtyping relation. -/
inductive Sub : {s : Shape} → Ty s → Ty s → Prop where
  | refl {s : Shape} (τ : Ty s) : Sub τ τ
  | trans {s : Shape} {σ τ υ : Ty s} : Sub σ τ → Sub τ υ → Sub σ υ
  | pos_nat : Sub .pos .nat
  | nat_int : Sub .nat .int
  | arrow {s t : Shape} {σ σ' : Ty s} {τ τ' : Ty t} :
      Sub σ' σ → Sub τ τ' → Sub (σ ⇒ τ) (σ' ⇒ τ')

/-- `σ <: τ`: `σ` is a subtype of `τ`. -/
scoped infix:50 " <: " => Sub
```

Because the relation is indexed by shape, only types of the same shape can be compared, and an arrow is never a subtype of a base type. The arrow rule has the contravariant premise `Sub σ' σ`.

Soundness, and where contravariance is needed

Soundness says that subtyping is inclusion: every value of a subtype is a value of the supertype. That is what makes rule T-Sub safe: a term that produces a value of σ can be used where a value of τ is expected, because that value *is* a value of τ.

```
/-- **Subsumption is sound.** A value of a subtype is a value of the
supertype. -/
theorem Sub.sound {s : Shape} {σ τ : Ty s} (h : σ <: τ) :
    ∀ v, σ.Holds v → τ.Holds v := by
  induction h with
  | refl τ => exact fun v hv => hv
  | trans _ _ ih₁ ih₂ => exact fun v hv => ih₂ v (ih₁ v hv)
  | pos_nat =>
    -- 0 < v implies 0 ≤ v.
    intro v hv
    exact le_of_lt hv
  | nat_int => exact fun _ _ => trivial
  | arrow _ _ ihσ ihτ =>
    -- `f` maps σ to τ; we must show it maps σ' to τ'.
    intro f hf x hx
    -- `x` is a σ'. Since σ' <: σ (contravariance!), `x` is also a σ...
    have hxσ := ihσ x hx
    -- ...so `f x` is a τ, and since τ <: τ', it is a τ'.
    exact ihτ (f x) (hf x hxσ)
```

The proof is by induction on the derivation of `σ <: τ`. Reflexivity is immediate, and transitivity chains the two induction hypotheses. For `pos_nat`, `Ty.Holds` unfolds to `0 < v` and the goal to `0 ≤ v`, and Mathlib's `le_of_lt` converts one into the other; for `nat_int` the goal is `True`.

The arrow case is the one that matters. We have a function `f` with `hf : (σ ⇒ τ).Holds f`, which says that `f` sends values of `σ` to values of `τ`, and we must show that it sends values of `σ'` to values of `τ'`. So take `x` with `hx : σ'.Holds x`. To apply `hf`, we need `x` to be a value of `σ`. The premise of the rule is `σ' <: σ`, so the induction hypothesis `ihσ` converts `σ'.Holds x` into `σ.Holds x`: this is the step that needs the argument premise to point from `σ'` to `σ`. Now `hf x hxσ` says that `f x` is a value of `τ`, and `ihτ` (from `τ <: τ'`) makes it a value of `τ'`.

With a covariant argument premise, `σ <: σ'`, the induction hypothesis would convert `σ.Holds x` into `σ'.Holds x`, the wrong direction, and the proof would be stuck. Being stuck in a proof is not yet a counterexample, so the file proves that the covariant rule really is unsound:

```
/-- The same relation with a *covariant* rule for function arguments. -/
inductive BadSub : {s : Shape} → Ty s → Ty s → Prop where
  | refl {s : Shape} (τ : Ty s) : BadSub τ τ
  | trans {s : Shape} {σ τ υ : Ty s} : BadSub σ τ → BadSub τ υ → BadSub σ υ
  | pos_nat : BadSub .pos .nat
  | nat_int : BadSub .nat .int
  | arrow {s t : Shape} {σ σ' : Ty s} {τ τ' : Ty t} :
      BadSub σ σ' → BadSub τ τ' → BadSub (σ ⇒ τ) (σ' ⇒ τ')

/-- With the covariant rule, subsumption is unsound. -/
theorem badSub_unsound :
    ¬ ∀ {s : Shape} (σ τ : Ty s), BadSub σ τ → ∀ v, σ.Holds v → τ.Holds v := by
  intro h
  -- The bad rule derives `nat ⇒ nat <: int ⇒ nat`.
  have hsub : BadSub (Ty.nat ⇒ .nat) (Ty.int ⇒ .nat) :=
    BadSub.arrow BadSub.nat_int (BadSub.refl _)
  -- The identity function maps naturals to naturals...
  have hid : (Ty.nat ⇒ .nat).Holds (fun z : ℤ => z) := fun _ hz => hz
  -- ...so it would also map every integer to a natural, -1 included.
```

```
  have hneg : (0 : ℤ) ≤ -1 := h _ _ hsub (fun z : ℤ => z) hid (-1) trivial
  linarith
```

The covariant rule derives `nat ⇒ nat <: int ⇒ nat` from `nat <: int`. The identity function on integers belongs to `nat ⇒ nat`, since it sends naturals to naturals. If subsumption were sound for the bad rule, the identity would also belong to `int ⇒ nat`, and applied to `-1` it would give a natural number, that is `0 ≤ -1`. The hypothesis `hneg` states exactly this, and `linarith`, Mathlib's tactic for linear arithmetic, derives the contradiction. In Rust terms: the identity is a callback that is fine for `'static` strings; treating it as a callback for any string is fine too, but treating a callback *for long-lived values only* as a callback for short-lived ones is exactly the `ch27_contravariance` error.

An algorithm, and why transitivity can go

To check subtyping, an implementation cannot search for derivations that use S-Trans, because the middle type τ of a transitivity step can be anything. The algorithmic version builds transitivity in:

```
/-- The base types are a chain: `pos` below `nat` below `int`. -/
def Ty.rank : Ty .base → ℕ
  | .pos => 0
  | .nat => 1
  | .int => 2

/-- Subtyping as a function, with no transitivity rule: compare the ranks of
base types, and treat function types by their variance. -/
def subtype : {s : Shape} → Ty s → Ty s → Bool
  | .base, σ, τ => σ.rank ≤ τ.rank
  | .arrow _ _, .arrow σ τ, .arrow σ' τ' => subtype σ' σ && subtype τ τ'
```

On base types it compares ranks in the chain `pos < nat < int`; on arrows it recurses with the variance of each side. There is no case for transitivity. `subtype` is defined by *dependent* pattern matching: matching on the shape first tells Lean that in the `.arrow` case both types must be arrows, so no other combination needs a case.

The file proves the algorithm sound (`subtype_sound`: if it says `true`, there is a derivation) and complete (`subtype_complete`: if there is a derivation, it says `true`). Completeness is the interesting direction, because the rules have S-Trans and the algorithm does not, so we must show that transitivity is *admissible* for the algorithm, in the sense of Chapter 6: it can be derived rather than assumed.

```
/-- Transitivity is *admissible*: the algorithm does not need a rule for it. -/
theorem subtype_trans : {s : Shape} → {σ τ υ : Ty s} →
    subtype σ τ = true → subtype τ υ = true → subtype σ υ = true
  | .base, _, _, _, h₁, h₂ => by
    simp only [subtype, decide_eq_true_eq] at h₁ h₂ ⊢
    omega
  | .arrow _ _, .arrow _ _, .arrow _ _, .arrow _ _, h₁, h₂ => by
    simp only [subtype, Bool.and_eq_true] at h₁ h₂ ⊢
    -- Arguments: υ's argument <: τ's <: σ's, the order flips.
    exact ⟨subtype_trans h₂.1 h₁.1, subtype_trans h₁.2 h₂.2⟩
```

This theorem is written as a recursive function, by pattern matching on the shape: a proof by structural recursion is a proof by induction (Chapter 10 made the same observation about derivations). On base types, `simp only` turns the Boolean comparisons into inequalities between ranks, and `omega` chains them. On arrows, the argument comparisons go the other way: from `σ ⇒ τ <: σ' ⇒ τ'` and `σ' ⇒ τ' <: σ'' ⇒ τ''` we have `σ' <: σ` and `σ'' <: σ'`, so the recursive call chains them as

`σ'' <: σ' <: σ`, *in the reverse order*: `subtype_trans h₂.1 h₁.1`. Contravariance shows up even in the proof of transitivity.

With both directions proved, the algorithm decides the relation, and `decide` can prove and refute subtyping judgments:

```
/-- The algorithm decides subtyping, so `decide` can prove (and refute)
subtyping judgments. -/
instance {s : Shape} (σ τ : Ty s) : Decidable (σ <: τ) :=
  decidable_of_iff (subtype σ τ = true) ⟨subtype_sound, subtype_complete⟩

example : (Ty.int ⇒ .pos) <: (Ty.pos ⇒ .int) := by decide

example : ¬ (Ty.nat ⇒ .nat) <: (Ty.int ⇒ .nat) := by decide

example : ((Ty.pos ⇒ .int) ⇒ .pos) <: ((Ty.nat ⇒ .int) ⇒ .nat) := by decide
```

The third example is the type-level version of `shrink_twice`: in an argument of an argument, the subtyping goes in the same direction, `pos` on the subtype side and `nat` on the supertype side.

27.9 Coercions are not subtyping

Rust performs many implicit conversions that look like subtyping and are not. They are called **coercions**, and they happen only at certain *coercion sites* (a `let` with a type annotation, a function argument, a struct field, a return value). The main ones are:

- *deref coercion*: `&String` to `&str`, `&Box<T>` to `&T`, `&Vec<T>` to `&[T]`, and in general `&U` to `&T` when `U: Deref<Target = T>`;
- *unsizing*: `&[T; N]` to `&[T]`, `Box<T>` to `Box<dyn Trait>`;
- *weakening a permission*: `&mut T` to `&T`;
- *function items and non-capturing closures to function pointers*;
- the never type `!` to any type (Chapter 1).

A subtyping step reuses a value as it is. A coercion converts it, and the example `code/rust/examples/ch27_coercions.rs` makes the conversions visible. First, a deref coercion calls code, here a `Deref` implementation that counts its calls:

```
/// A smart pointer that counts how often it is dereferenced.
struct Tracked<T> {
    value: T,
    derefs: Cell<u32>,
}

impl<T> Deref for Tracked<T> {
    type Target = T;
    fn deref(&self) -> &T {
        self.derefs.set(self.derefs.get() + 1);
        &self.value
    }
}

fn shout(s: &str) -> String {
    s.to_uppercase()
}

fn deref_coercions() {
```

```
    let name = Tracked {
        value: String::from("pump 3"),
        derefs: Cell::new(0),
    };
    // `&Tracked<String>` -> `&String` -> `&str`: two coercions, and the first
    // one is a call to our `deref`.
    let loud = shout(&name);
    println!("{loud}: Deref::deref ran {} time(s)", name.derefs.get());
}
```

Second, unsizing and function-pointer coercions change the representation:

```
trait Describe {
    fn describe(&self) -> String;
}

struct Valve {
    open: bool,
}

impl Describe for Valve {
    fn describe(&self) -> String {
        format!("valve open: {}", self.open)
    }
}

fn double(x: i32) -> i32 {
    2 * x
}

fn representations() {
    // Unsizing a reference to an array adds the length: a thin pointer
    // becomes a fat one.
    let readings = [1, 2, 3];
    let array_ref: &[i32; 3] = &readings;
    let slice_ref: &[i32] = array_ref;
    println!(
        "&[i32; 3]: {} bytes, &[i32]: {} bytes",
        size_of_val(&array_ref),
        size_of_val(&slice_ref)
    );
    // Unsizing a box to a trait object adds a vtable pointer.
    let boxed: Box<Valve> = Box::new(Valve { open: true });
    let size_before = size_of_val(&boxed);
    let object: Box<dyn Describe> = boxed;
    println!(
        "Box<Valve>: {size_before} bytes, Box<dyn Describe>: {} bytes ({})",
        size_of_val(&object),
        object.describe()
    );
    // A function item has its own zero-sized type; coercing it to a function
    // pointer produces an actual address.
    let item = double;
    let pointer: fn(i32) -> i32 = double;
    println!(
        "fn item: {} bytes, fn pointer: {} bytes, both give {}",
        size_of_val(&item),
        size_of_val(&pointer),
        item(21) + pointer(0)
    );
}
```

Third, lifetime subtyping changes nothing:

```
/// Subtyping changes nothing: the same pointer, viewed at a shorter lifetime.
fn shorten<'long: 'short, 'short>(x: &'long str) -> &'short str {
    x
}
```

```
fn subtyping_is_free() {
    let long: &'static str = "a static string";
    let short: &str = shorten(long);
    println!(
        "&'static str -> &str: {} bytes -> {} bytes, same address: {}",
        size_of_val(&long),
        size_of_val(&short),
        std::ptr::eq(long, short)
    );
}
```

```
PUMP 3: Deref::deref ran 1 time(s)
&[i32; 3]: 8 bytes, &[i32]: 16 bytes
Box<Valve>: 8 bytes, Box<dyn Describe>: 16 bytes (valve open: true)
fn item: 0 bytes, fn pointer: 8 bytes, both give 42
&'static str -> &str: 16 bytes -> 16 bytes, same address: true
```

The `Deref` implementation ran once while `shout(&name)` was being called. A reference to an array is a thin pointer (8 bytes on a 64-bit machine); as a slice it becomes a fat pointer with a length (16 bytes), and the same happens when a `Box<Valve>` becomes a `Box<dyn Describe>`, which carries a vtable pointer (Chapter 15). A function item has a zero-sized type of its own, and only the coercion to a function pointer materializes an address. The lifetime subtyping step, in contrast, leaves the size and the address unchanged.

There is a second, structural difference. Subtyping lifts through type constructors according to their variance: `Vec<&'static str>` is a subtype of `Vec<&'a str>` because `Vec` is covariant, so no work is needed to use one as the other. Coercions do not lift:

```
pub fn names(owned: Vec<&String>) -> Vec<&str> {
    owned
}
```

```
error[E0308]: mismatched types
 --> compile_fail/ch27_no_coercion_in_vec.rs:7:5
  |
6 | pub fn names(owned: Vec<&String>) -> Vec<&str> {
  |                                      --------- expected `Vec<&str>` because of return
type
7 |     owned
  |     ^^^^^ expected `Vec<&str>`, found `Vec<&String>`
  |
  = note: expected struct `Vec<&str>`
             found struct `Vec<&String>`
```

`&String` coerces to `&str`, but `Vec<&String>` does not coerce to `Vec<&str>`. It could not, cheaply: each element would have to be converted, which means building a new vector.

```
/// Subtyping lifts through `Vec` for free (`Vec` is covariant)...
fn shorten_all<'long: 'short, 'short>(v: Vec<&'long str>) -> Vec<&'short str> {
    v
}

/// ...but a coercion does not: `Vec<&String>` is not a `Vec<&str>`
/// (`ch27_no_coercion_in_vec`). Converting means building a new vector.
fn as_strs(owned: &[String]) -> Vec<&str> {
    owned.iter().map(|s| s.as_str()).collect()
}
```

The weakening of `&mut T` to `&T` is worth a separate look, because it connects to Chapter 26: a unique permission can always be given up in favor of a shared one.

```
/// `&mut T` coerces to `&T`: a unique permission can always be weakened to a
/// shared one, for as long as the unique one is not used.
fn total(values: &[u32]) -> u32 {
    values.iter().sum()
}

fn weaken(values: &mut Vec<u32>) -> u32 {
    values.push(4);
    total(values) // `&mut Vec<u32>` -> `&Vec<u32>` -> `&[u32]`
}
```

Lean: coercions and subtypes, no subtyping

Lean has no subtyping at all: a term has a type, and no rule lets it be used at a bigger one. What looks like subtyping is a coercion that the elaborator inserts, and Lean shows it to you, marked `↑`. The examples are in `code/lean/Book/Ch27/Coercions.lean`:

```
/-- A function on integers. -/
def double (z : ℤ) : ℤ := 2 * z

/-- info: double ↑3 : ℤ -/
#guard_msgs in
#check double (3 : ℕ)

/-- info: fun n => double ↑n : ℕ → ℤ -/
#guard_msgs in
#check fun (n : ℕ) => double n

/-- The `↑` is an ordinary function, `Int.ofNat`. -/
example (n : ℕ) : (n : ℤ) = Int.ofNat n := rfl
```

Passing a natural number to a function on integers works, and `#check` reveals how: the argument became `↑3`, a call to the conversion function `Int.ofNat`, as the `example` confirms by `rfl`. The conversion goes only one way:

```
/--
error: Type mismatch
  z
has type
  ℤ
but is expected to have type
  ℕ
-/
#guard_msgs in
def toNat (z : ℤ) : ℕ := z
```

You can add coercions for your own types with an instance of the class `Coe`, which is how the elaborator finds them:

```
/-- An amount of money, in cents. -/
structure Cents where
  amount : ℕ

/-- Cents can be used where an integer is expected. -/
instance : Coe Cents ℤ where
  coe c := c.amount

/-- The sum of a list of signed amounts. -/
def balance (deltas : List ℤ) : ℤ := deltas.sum
```

```
/-- info: fun c => double ↑c.amount : Cents → ℤ -/
#guard_msgs in
#check fun (c : Cents) => double c

/-- info: 150 -/
#guard_msgs in
#eval balance [Cents.mk 250, -100]
```

The `#check` shows the inserted call (the field access `c.amount`, followed by the `ℕ`-to-`ℤ` coercion), and the `#eval` shows that the list literal mixing `Cents` and integers is accepted. This is Rust's deref coercion in Lean form: a function call that the compiler writes for you.

Lean's **subtypes** are something else again, despite the name. The type `{n : ℕ // n % 2 = 0}` is a *refinement*: its values are pairs of a number and a proof that the number is even (Chapter 10 introduced `Subtype`).

```
/-- The even natural numbers, as a subtype: each value is a pair of a number
and a proof that it is even. -/
abbrev Even := {n : ℕ // n % 2 = 0}

def four : Even := ⟨4, rfl⟩

/-- info: 5 -/
#guard_msgs in
#eval four.val + 1

/-- The proof travels with the value, and can be used. -/
theorem four_even : four.val % 2 = 0 := four.property
```

An `Even` is not a natural number; `.val` (or the coercion `↑`, which unfolds to it) extracts one. And like Rust's coercions, this one does not lift through `List`:

```
/--
error: Type mismatch
  xs
has type
  List Even
but is expected to have type
  List ℕ
-/
#guard_msgs in
def evens (xs : List Even) : List ℕ := xs

/-- What works is an explicit map, which builds a new list. -/
def evens' (xs : List Even) : List ℕ := xs.map Subtype.val
```

Our tiny language connects the two ideas. Its types *are* refinements of the run-time values, so each type gives a Lean subtype, and a subtyping proof gives a coercion between them that does nothing to the value:

```
/-- The values of a type, as a Lean subtype of the run-time values. -/
abbrev Ty.Sem {s : Shape} (τ : Ty s) : Type := {v : s.Val // τ.Holds v}

/-- A subtyping proof turns a value of `σ` into a value of `τ`... -/
def Sub.coe {s : Shape} {σ τ : Ty s} (h : σ <: τ) : σ.Sem → τ.Sem :=
  fun v => ⟨v.val, h.sound v.val v.property⟩

/-- ...without changing it: the underlying run-time value is the same. -/
theorem Sub.coe_val {s : Shape} {σ τ : Ty s} (h : σ <: τ) (v : σ.Sem) :
    (h.coe v).val = v.val := rfl
```

`Ty.Sem τ` is the Lean subtype of run-time values that belong to `τ`. `Sub.coe` keeps the value `v.val` and only replaces the proof, using `Sub.sound`; `Sub.coe_val` states, by `rfl`, that the value is untouched. In the *coercion* reading of subtyping, our language's subtyping is a coercion whose run-time part is the identity, which is exactly what Rust's lifetime subtyping is, and what Rust's deref and unsizing coercions are not.

27.10 Putting it together: the variance of a raw-pointer view

Let us design a type with a variance that we choose on purpose: a borrowed view of a slice, implemented with a raw pointer, the kind of type that sits at the bottom of many data-structure crates. The first attempt was `ch27_unused_param.rs`: a pointer and a length, with a lifetime parameter `'a` that nothing mentions, rejected with E0392 because the compiler cannot infer a variance for an unused parameter.

Step 1: say what the type means. A `RawView<'a, T>` should behave exactly like a `&'a [T]`: it borrows the elements for `'a` and only reads them. So it should have the variance of `&'a [T]`: covariant in `'a` and in `T`.

Step 2: choose the marker. "Behaves like a `&'a T`" is precisely what `PhantomData<&'a T>` says:

```
/// A borrowed view of a slice through a raw pointer. The raw pointer carries
/// no lifetime, so `PhantomData<&'a T>` records the borrow: the view is
/// covariant in `'a` and `T`, exactly like `&'a [T]`.
struct RawView<'a, T> {
    ptr: *const T,
    len: usize,
    _borrow: PhantomData<&'a T>,
}

impl<'a, T> RawView<'a, T> {
    fn new(slice: &'a [T]) -> Self {
        RawView {
            ptr: slice.as_ptr(),
            len: slice.len(),
            _borrow: PhantomData,
        }
    }

    fn get(&self, i: usize) -> Option<&'a T> {
        if i < self.len {
            // SAFETY: `ptr` points to `len` initialized elements borrowed for
            // `'a` (see `new`), and `i < len`.
            Some(unsafe { &*self.ptr.add(i) })
        } else {
            None
        }
    }
}

/// The witness: the view is covariant in `'a`.
fn shrink_view<'long: 'short, 'short, T>(v: RawView<'long, T>) -> RawView<'short, T> {
    v
}

/// With `PhantomData<&'a mut T>` the type would be invariant in `T`, and with
/// `PhantomData<fn(T)>` contravariant; `PhantomData<Cell<T>>` also invariant.
#[allow(dead_code)]
struct Invariant<T>(PhantomData<Cell<T>>);
```

The inferred variance is now the meet of the fields' variances. `ptr: *const T` is covariant in `T` and does not mention `'a`; `len` mentions nothing; `_borrow: PhantomData<&'a T>` is covariant in both. The meet is covariant in both, as intended. In the notation of `Variance.lean`, the variance in `'a` is $bi \sqcap bi \sqcap co = co$.

Step 3: make the compiler confirm it. `shrink_view` is the witness: it compiles, so `RawView` is covariant in `'a`. The output shows the view at work after shrinking:

```
view[1] = Some(4.0), view[3] = None
```

Step 4: ask what would go wrong with another choice. Suppose the view were mutable, a `RawViewMut<'a, T>` that can write through its pointer. If it were covariant in `T`, a `RawViewMut<'a, &'static str>` could be viewed as a `RawViewMut<'a, &'short str>`, a short-lived string written into it, and the original owner of the `&'static str` slots would read a dangling reference: the classic program of `ch27_mut_invariance`, in a library instead of in your code. The right marker is `PhantomData<&'a mut T>`, which makes the type invariant in `T` and covariant in `'a`, like `&'a mut [T]`. With `*mut T` as the pointer type, the field `ptr` would already be invariant in `T`, and the marker would only be needed for `'a`.

Step 5: check the auto traits. A raw pointer is neither `Send` nor `Sync`, so `RawView` is neither, whatever the marker. Since the view behaves like `&'a [T]`, which is `Send` and `Sync` exactly when `T` is `Sync`, a library would restore that with `unsafe impl<'a, T: Sync> Send for RawView<'a, T> {}` (and the same for `Sync`), an `unsafe` promise that Chapter 32 teaches you to justify. For the typed identifier the marker alone decided the auto traits, and that is why `Id<T>` used `PhantomData<fn() -> T>` rather than `PhantomData<T>`.

The whole design took one line of code. The reasoning behind that line is the entire theory of this chapter: subtyping on lifetimes, the variance of each field, the meet that combines them, and the classic program that tells you which answer is unsafe.

27.11 Common confusions

"**Rust has no subtyping.**" It has subtyping between lifetimes (and between higher-ranked function types), and through variance this subtyping reaches every type that contains a lifetime. You use it whenever a `&'static str` is passed where a shorter `&str` is expected.

"**`&mut T` is invariant.**" It is invariant in `T` and covariant in its lifetime. Shortening the loan is always safe; changing what it points to is not.

"**Contravariance is an exotic corner case.**" Every callback parameter is a contravariant position. In Rust it shows up mostly with function pointers and closures that take references, and in the `PhantomData<fn(T)>` marker.

"**Variance is declared.**" In C#, Kotlin and Scala you declare it (`in` / `out`, `-T` / `+T`). In Rust it is always inferred from the fields; `PhantomData` is how you influence it.

"**A coercion is a kind of subtyping.**" A coercion converts a value, may run code, may change the representation, and does not lift through type constructors. A subtyping step reuses the value unchanged and lifts according to variance.

"**Lean's subtypes are subtyping.**" A value of `{x // p x}` is a pair of a value and a proof; to use it as an `α` you take its `.val`, which is a coercion. Lean has no subsumption rule.

27.12 Summary

- Subtyping $\sigma <: \tau$ enters a type system through the subsumption rule T-Sub. It is a preorder, hence a thin category, and it makes typing non-syntax-directed, which algorithmic formulations fix.
- Rust's only subtyping is on lifetimes: `'long: 'short` gives `&'long T <: &'short T`. It is pure inclusion: no code runs.
- Variance says how subtyping lifts through a type constructor: covariant, contravariant, invariant or bivariant. Functions are contravariant in their argument; `&'a T` is covariant in both parameters; `&'a mut T`, `Cell<T>` and `*mut T` are invariant in `T`.
- `&mut T` must be invariant: with covariance, the classic `overwrite` program stores a short-lived reference in a `'static` slot. Reading is covariant, writing contravariant, both invariant.
- Rust infers the variance of structs from their fields (a meet over occurrences, a product along nesting). Unused parameters are rejected (E0392); `PhantomData` chooses variance and auto traits.
- Variance is functoriality: covariant types are functors from the category of lifetimes, contravariant ones from its opposite. Variances form a commutative monoid under composition, with a meet for combining occurrences, and in Lean variance inference is proved sound.
- In a tiny language whose types are refinements, subsumption is proved sound, the contravariant arrow rule is exactly what the proof needs, a covariant one is refuted, and transitivity is admissible for the algorithm.
- Coercions (deref, unsizing, `&mut` to `&`, function pointers) are conversions, not subtyping. Lean has coercions (`↑`, `Coe`) and subtypes (refinements), but no subtyping.

27.13 Exercises

1. Determine the variance in `'a` and in `T` of `struct Callback<'a, T> { f: fn(&'a T) -> usize }`. Confirm your answer by writing witness functions in the style of `ch27_variance.rs`, and show that the opposite directions do not compile. *Hint:* both parameters occur only inside the argument of a function pointer.
2. Assume, contrary to fact, that `Cell<T>` were covariant in `T`. Write a safe Rust program that would then read a dangling reference, modeled on `ch27_mut_invariance.rs`. Then compile it and read the actual error. *Hint:* create a `Cell<&'static str>`, view it as a `&Cell<&'short str>`, and `set` a reference to a local `String`.

3. Why is it safe for `&'a mut T` to be covariant in `'a`? Give an informal argument in terms of permissions (Chapter 26), and a witness function. *Hint:* `shrink_mut` in `ch27_variance.rs` is the witness. What can the holder of a shorter loan do that the holder of the longer loan could not?
4. In `code/lean/Book/Ch27/Variance.lean`, prove that `meet` is commutative, associative and idempotent, and that `bi` is its unit, so that the variances form a semilattice. Then prove `Variance.apply_meet` again, from these laws, for `v ⊓ v`. *Hint:* `cases v <;> cases w <;> rfl` for the laws.
5. Add a constructor `ptrMut (t : RTy L)`, standing for `*mut T`, to `RTy`, with an `RSub` rule and a `var` case, and repair `RTy.var_sound`. Which existing case does your proof resemble? *Hint:* `*mut T` is invariant in `T`, like `Cell<T>`.
6. Add pair types to `Subtyping.lean`: a shape `prod s t`, values `s.Val × t.Val`, a type `σ ⊗ τ` with `Holds` componentwise, the covariant rule S-Pair, and the corresponding case of `subtype`. Repair `Sub.sound`, `subtype_sound`, `subtype_trans` and `subtype_complete`. *Hint:* every new case follows the arrow case, but without the flip.

CHAPTER 28

A Core Calculus for Borrowing

Over the last six chapters we have collected the pieces of Rust's ownership discipline one at a time: an affine λ-calculus (Chapter 22), `Copy` (Chapter 23), drops (Chapter 24), regions and the escape check (Chapter 26), subtyping between lifetimes (Chapter 27). Each piece made sense on its own, and each came with a promise that it explained some part of what the borrow checker does.

This chapter keeps those promises in the only way that is fully convincing: by putting every piece into a single formal system and checking that it works. We build a small calculus with owners, moves, drops, `Copy`, shared and mutable borrows, lifetimes, reborrowing and the escape check, write down every typing rule with the Rust program each premise models, and give it an operational semantics in which reading freed memory or writing to a shared location are *stuck* states, the formal counterpart of undefined behavior. Then we work through two derivations that every Rust programmer has met in disguise: why a `&mut` reference can apparently be used twice, and why the classic "push while iterating" bug has no derivation at all.

The calculus is in the spirit of the research calculi that formalize Rust's core (Featherweight Rust and Oxide are two well-known ones), but much smaller: small enough that a Rust crate can check its programs and print their derivations, and that Lean can prove theorems about it in a few hundred lines.

In this chapter you will learn:

- how ownership, `Copy`, borrows and lifetimes fit into one type system, λ^{aff}_{rgn}, with thirteen typing rules;
- what every premise of the three borrow rules does, and which premises are affine book-keeping and which are region reasoning;
- an operational semantics with a store and loan states, and what progress and preservation promise for it;
- why `bump(r); bump(r);` type-checks although `&mut` is not `Copy` (derivation I), and why the aliasing bug is rejected (derivation II);
- how the `borrowck` crate checks programs of the calculus, prints derivation trees, reports errors and runs programs on an abstract machine;
- how to state the rules in Lean, build a derivation as a proof term, prove that *no* derivation exists, and prove that a mutable reference is used at most once.

28.1 What we need from Parts V and VI

Three ideas from earlier chapters carry this one. Here they are in the form we will use them.

Context splitting. In the affine λ-calculus of Chapter 22, a rule with two subterms does not give both of them the whole context. It *splits* the context: $\Gamma = \Gamma_1 \uplus \Gamma_2$ means that every variable of Γ goes to Γ_1 or to Γ_2, so a variable can be used by one subterm or the other, but not by both. That is how the type system forbids contraction, and it is the formal version of a move: after `let t = s;` the variable `s` belongs to the `let`'s right-hand side, so the rest of the block cannot use it (E0382). Weakening is allowed: a variable may go unused, as in Rust, where an unused value is simply dropped.

`Copy` as permission to share a variable. Chapter 23 refined the split: a variable whose type satisfies the predicate Copy may go to *both* halves. Contraction comes back, but only for the types where it is harmless.

Regions and permissions. Chapter 26 explained that affinity alone cannot account for references: a shared reference is duplicable, so counting uses says nothing about whether it outlives the data it points to. Lifetimes are *region variables*: names for a stretch of the program during which a borrow is valid. A reference type mentions its region (`&'a T`), and the **escape check** forbids a value whose type mentions a region from leaving that region. A shared reference is a duplicable read permission, a mutable reference a unique write permission, and a borrow is a temporary transfer of permission from an owner.

Chapter 27's subtyping does not appear in this chapter. Our rules demand exact equality of types where Rust would accept a reference with a longer lifetime than needed, so the calculus rejects some programs that subtyping would rescue; none of the programs we study needs it.

28.2 The design of the calculus

Before the rules, four design decisions. Each one trades fidelity to Rust for a system small enough to hold in your head.

Owners are locations. Borrowing needs something to point at. In our calculus, a value that can be borrowed lives in a *store* (a heap), and its owner is a pointer to that location, of type $\mathit{Box}\,\tau$. Rust lets you borrow any local variable (`&mut x`) as well as the contents of a `Box`; a local variable lives in a stack frame, which is memory too, so modelling every borrowable variable as an owned location loses nothing essential. A new location is created with $\mathit{new}\ e$, Rust's `Box::new(e)`.

Borrows are lexically scoped. A borrow is a construct with a body: $\mathit{borrow}\ y = \&\,^{\alpha} x\ \mathit{in}\ e$ creates a reference y that is valid exactly while e runs. The lifetime α is introduced by the construct, and its scope is the body. This is roughly how Rust worked before non-lexical lifetimes arrived with the 2018 edition, when a borrow held in a variable lasted until the end of that variable's scope; today's compiler computes shorter lifetimes from the control-flow graph, and Chapter 29 is about how.

A borrow lends the owner and gives it back. The borrow construct consumes its owner, exactly as a function call consumes a moved argument, and returns it when the body finishes, paired with the

body's result. So the type of a borrow is a pair $\mathit{Box}\,\tau \otimes \sigma$: the owner, and whatever the body computed. The program then rebinds the owner with a `let`:

$$\mathit{let}\ (x, n) = (\mathit{borrow}\ y = \&^{\alpha} x\ \mathit{in} * y)\ \mathit{in} \ldots$$

Rust hides this rebinding: after `let n = *(&x);` you still call the owner `x`. Making it explicit is what lets plain affine context splitting express "you cannot use the owner while it is borrowed": during the body, the owner belongs to the borrow, not to anybody else.

No functions. The calculus has no λ, no function types and no lifetime polymorphism. A Rust function `fn f<'a>(r: &'a mut i32) { body }` is modelled by checking its body in a context where `'a` is a lifetime in scope and `r` has type $\&^{\alpha}\mathit{mut}\ \mathit{Int}$, which is exactly what a function signature means (Chapter 1 read signatures as judgments). A call is modelled by inlining the callee. We keep the name $\lambda^{\mathit{aff}}_{\mathit{rgn}}$, "affine, with regions", as a family name.

Here is the full list of what we leave out, so that you know what the calculus does *not* claim to explain: functions and lifetime parameters on them, the `'static` lifetime, outlives constraints (in a lexically scoped calculus they are implicit: a lifetime introduced inside another one's scope is shorter), subtyping and variance, structs and enums, partial moves out of places, moving a value out of a box, reading through a `&mut`, `Drop` implementations, interior mutability, and non-lexical lifetimes. Everything else, the things that make borrowing borrowing, is in.

28.3 Syntax

The types are integers, the unit type, pairs, owned locations and the two kinds of reference. We write κ for any lifetime and α, β for lifetimes that a borrow introduces:

$$\tau, \sigma ::= \mathit{Int} \mid \mathit{Unit} \mid \tau_1 \otimes \tau_2 \mid \mathit{Box}\,\tau \mid \&^{\kappa}\tau \mid \&^{\kappa}\mathit{mut}\,\tau$$

The pair type $\tau_1 \otimes \tau_2$ is the multiplicative conjunction of Chapter 21, Rust's tuple `(T, U)`. $\&^{\kappa}\tau$ is Rust's `&'k T` and $\&^{\kappa}\mathit{mut}\,\tau$ is `&'k mut T`. The free lifetimes of a type, $\mathrm{flv}(\tau)$, are the lifetimes it mentions: $\mathrm{flv}(\,\&^{\kappa}\mathit{mut}\,(\mathit{Int} \otimes \&^{\beta}\mathit{Int})) = \{\kappa, \beta\}$.

The terms are the affine core of Chapter 22 without functions, plus allocation, dropping, reading, writing and three ways to borrow:

$$e ::= x \mid n \mid () \mid (e_1, e_2) \mid \mathit{let}\ x = e_1\ \mathit{in}\ e_2 \mid \mathit{let}\ (x, y) = e_1\ \mathit{in}\ e_2 \mid \mathit{new}\ e \mid \mathit{drop}\ e$$

$$\mid *e \mid *e_1 := e_2 \mid \mathit{borrow}\ y = \&^{\alpha} e_1\ \mathit{in}\ e_2 \mid \mathit{borrow}\ y = \&^{\alpha}\mathit{mut}\ e_1\ \mathit{in}\ e_2$$

$$\mid \mathit{borrow}\ y = \&^{\alpha}\mathit{mut} * e_1\ \mathit{in}\ e_2$$

Each form has a Rust counterpart, and the `borrowck` crate reads programs in a concrete syntax that stays close to Rust's:

Calculus	Crate syntax	Rust
$new\ e$	`new e`	`Box::new(e)`
$drop\ e$	`drop e`	`std::mem::drop(e)`
$*e$	`*e`	`*e` (copying a `Copy` value out)
$*e_1 := e_2$	`*e1 := e2`	`*e1 = e2;`
$borrow\ y = \&^{\alpha} e_1\ in\ e_2$	`borrow y = &'a e1 in e2`	`{ let y = &e1; e2 }`
$borrow\ y = \&^{\alpha} mut\ e_1\ in\ e_2$	`borrow y = &'a mut e1 in e2`	`{ let y = &mut e1; e2 }`
$borrow\ y = \&^{\alpha} mut * e_1\ in\ e_2$	`borrow y = &'a mut *e1 in e2`	`{ let y = &mut *e1; e2 }`

The last form is a **reborrow**: it creates a new mutable reference from an existing one, the `&mut *r` that you rarely write because the compiler inserts it for you. In all three borrow forms the lifetime α and the variable y are bound in the body e_2, the way a `let` binds its variable.

The syntax in Rust and in Lean

The crate's types and terms are ordinary Rust enums (`code/rust/crates/borrowck/src/syntax.rs`):

```rust
/// The types of the calculus.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    /// Integers (Rust's `i32`).
    Int,
    /// The unit type `()`.
    Unit,
    /// The pair type `τ₁ ⊗ τ₂` (Rust's tuple `(T, U)`).
    Pair(Box<Ty>, Box<Ty>),
    /// An owned location holding a `τ` (Rust's `Box<T>`).
    Box(Box<Ty>),
    /// A shared reference `&'κ τ`.
    Ref(Lifetime, Box<Ty>),
    /// A mutable reference `&'κ mut τ`.
    RefMut(Lifetime, Box<Ty>),
}
```

```rust
/// The three ways to create a reference.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum BorrowKind {
    /// `borrow y = &'a e₁ in e₂`
    Shared,
    /// `borrow y = &'a mut e₁ in e₂`
    Mut,
    /// `borrow y = &'a mut *e₁ in e₂`
    Reborrow,
}

/// The terms of the calculus.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(String),
    Lit(i64),
    Unit,
    /// `(e₁, e₂)`
    Pair(Box<Term>, Box<Term>),
    /// `let x = e₁ in e₂`
    Let(String, Box<Term>, Box<Term>),
    /// `let (x, y) = e₁ in e₂`
    LetPair(String, String, Box<Term>, Box<Term>),
    /// `new e`: allocate a location (`Box::new`).
    New(Box<Term>),
    /// `drop e`: consume a value, freeing what it owns (`std::mem::drop`).
    Drop(Box<Term>),
```

```
    /// `*e`: read through a shared reference.
    Deref(Box<Term>),
    /// `*e₁ := e₂`: write through a mutable reference.
    Assign(Box<Term>, Box<Term>),
    /// The three borrow forms. They bind `lifetime` and `var` in `body`.
    Borrow {
        kind: BorrowKind,
        lifetime: Lifetime,
        var: String,
        owner: Box<Term>,
        body: Box<Term>,
    },
    /// Run-time only: a pointer to store location `ℓ`.
    Ptr(Ptr, usize),
    /// Run-time only: `restore w after e`, the frame of a borrow in progress.
    Restore(Box<Term>, Box<Term>),
}

/// The three kinds of pointer values that exist at run time.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Ptr {
    /// `own ℓ`, the value of an owner of type `Box τ`.
    Own,
    /// `shr ℓ`, the value of a shared reference.
    Shr,
    /// `uniq ℓ`, the value of a mutable reference.
    Uniq,
}
```

The last two variants of `Term` never appear in a program you write. They are the run-time forms of the operational semantics, which we meet later: pointer values into the store, and the frame of a borrow in progress. Keeping them in the same enum mirrors the mathematics, where run-time forms extend the grammar of terms.

The Lean version (`code/lean/Book/Ch28/Syntax.lean`) is the same two inductive types, with variables and lifetimes named by strings so that the derivations we build later read like the ones on paper:

```
/-- The types of the calculus. -/
inductive Ty where
  /-- Machine integers (Rust's `i32`). -/
  | int
  /-- The unit type `()`. -/
  | unit
  /-- The pair type `τ₁ ⊗ τ₂` (Rust's tuple `(T, U)`). -/
  | pair (τ₁ τ₂ : Ty)
  /-- An owned location in the store holding a `τ` (Rust's `Box<T>`). -/
  | box (τ : Ty)
  /-- A shared reference `&'κ τ`. -/
  | ref (κ : Lft) (τ : Ty)
  /-- A mutable reference `&'κ mut τ`. -/
  | refMut (κ : Lft) (τ : Ty)
  deriving DecidableEq, Repr
```

```
/-- The terms of the calculus. The three `borrow` forms bind the lifetime `α`
and the reference variable `y` in the body `e₂`. -/
inductive Term where
  | var (x : Var)
  | lit (n : Nat)
  | unit
  /-- `(e₁, e₂)` -/
  | pair (e₁ e₂ : Term)
  /-- `let x = e₁ in e₂` -/
  | letIn (x : Var) (e₁ e₂ : Term)
  /-- `let (x, y) = e₁ in e₂` -/
  | letPair (x y : Var) (e₁ e₂ : Term)
```

```
  /-- `new e`: allocate a location holding the value of `e` (`Box::new`). -/
  | new (e : Term)
  /-- `drop e`: consume a value and free what it owns (`std::mem::drop`). -/
  | drop (e : Term)
  /-- `*e`: read through a shared reference. -/
  | deref (e : Term)
  /-- `*e₁ := e₂`: write through a mutable reference. -/
  | assign (e₁ e₂ : Term)
  /-- `borrow y = &'α e₁ in e₂`: a shared borrow of the owner `e₁`. -/
  | borrow (α : Lft) (y : Var) (e₁ e₂ : Term)
  /-- `borrow y = &'α mut e₁ in e₂`: a mutable borrow of the owner `e₁`. -/
  | borrowMut (α : Lft) (y : Var) (e₁ e₂ : Term)
  /-- `borrow y = &'α mut *e₁ in e₂`: a reborrow of the mutable reference `e₁`. -/
  | reborrow (α : Lft) (y : Var) (e₁ e₂ : Term)
  deriving Repr
```

(The constructor names `letIn` and `refMut` avoid Lean's keywords `let` and `mut`.) The free lifetimes of a type are a recursive function:

```
/-- `flv(τ)`: the lifetimes mentioned in a type. The escape check asks that a
fresh lifetime is not in this list. -/
def Ty.lfts : Ty → List Lft
  | .int | .unit => []
  | .pair τ₁ τ₂ => τ₁.lfts ++ τ₂.lfts
  | .box τ => τ.lfts
  | .ref κ τ | .refMut κ τ => κ :: τ.lfts
```

28.4 `Copy` and context splitting

The predicate $Copy(\tau)$ says that values of type τ may be duplicated by simply using them twice. It is the least predicate closed under four rules:

$$\frac{}{Copy(Int)}\ \text{C-Int} \qquad \frac{}{Copy(Unit)}\ \text{C-Unit} \qquad \frac{}{Copy(\,\&^{\kappa}\,\tau)}\ \text{C-Ref} \qquad \frac{Copy(\tau_1) \qquad Copy(\tau_2)}{Copy(\tau_1 \otimes \tau_2)}\ \text{C-Pair}$$

"Least" means that nothing else is `Copy`. In particular there is no rule for $Box\,\tau$ (an owner of a location: duplicating it would give the location two owners, and two frees) and none for $\&^{\kappa} mut\,\tau$ (a unique write permission: duplicating it would give two writers). Rule C-Ref says that *every* shared reference is `Copy`, whatever it points to: `&String` is `Copy` although `String` is not, because copying the reference copies the read permission, not the string. Rule C-Pair is Rust's requirement that a type can derive `Copy` only if all its fields are `Copy`.

In Lean the predicate is an inductive family with one constructor per rule, and "no rule concludes `Copy (.box τ)`" becomes a one-line proof:

```
/-- `Copy τ`: values of type `τ` may be duplicated. The least predicate closed
under these four rules; in particular `box` and `refMut` are never `Copy`. -/
inductive Copy : Ty → Prop where
  | int : Copy .int
  | unit : Copy .unit
  | ref {κ τ} : Copy (.ref κ τ)
  | pair {τ₁ τ₂} : Copy τ₁ → Copy τ₂ → Copy (.pair τ₁ τ₂)
```

```
/-- An owner is not `Copy`: no rule concludes `Copy (.box τ)`, so a proof of it
cannot exist, and `nomatch` checks exactly that. -/
theorem not_copy_box (τ : Ty) : ¬ Copy (.box τ) := fun h => nomatch h
```

```
/-- A mutable reference is not `Copy` either. -/
theorem not_copy_refMut (κ : Lft) (τ : Ty) : ¬ Copy (.refMut κ τ) :=
  fun h => nomatch h
```

`nomatch h` asks Lean to consider every constructor that could have produced `h : Copy (.box τ)`. None of the four has a conclusion of that shape, so there is nothing to consider and the proof is complete. You will see the same idea, "no rule fits", carry the impossibility proof of derivation II.

The split $\Gamma = \Gamma_1 \uplus \Gamma_2$ is defined by four rules that walk through the context one entry at a time and send each entry left, right, or, if its type is `Copy`, to both sides:

$$\frac{}{\cdot = \cdot \uplus \cdot}\ \text{S-Empty} \qquad \frac{\Gamma = \Gamma_1 \uplus \Gamma_2}{\Gamma, x:\tau = (\Gamma_1, x:\tau) \uplus \Gamma_2}\ \text{S-Left}$$

$$\frac{\Gamma = \Gamma_1 \uplus \Gamma_2}{\Gamma, x:\tau = \Gamma_1 \uplus (\Gamma_2, x:\tau)}\ \text{S-Right} \qquad \frac{\Gamma = \Gamma_1 \uplus \Gamma_2 \qquad \mathit{Copy}(\tau)}{\Gamma, x:\tau = (\Gamma_1, x:\tau) \uplus (\Gamma_2, x:\tau)}\ \text{S-Both}$$

The dot $\cdot$ is the empty context. Rule S-Both is the whole of Chapter 23 in one line: the exponential ! of linear logic, folded into the splitting operation instead of being a type constructor. Rust makes the same choice: `Copy` is a property of a type, not a wrapper around it. The Lean definition transcribes the four rules, with contexts as lists whose head is the most recent entry:

```
/-- A typing context: a list of variables with their types, most recent
first. -/
abbrev Ctx := List (Var × Ty)

/-- The type of `x` in `Γ`, if `x` is in `Γ`. -/
def lookup : Ctx → Var → Option Ty
  | [], _ => none
  | (y, τ) :: Γ, x => if x = y then some τ else lookup Γ x

/-- The variables of a context. -/
def dom (Γ : Ctx) : List Var := Γ.map Prod.fst
```

```
/-- `Split Γ Γ₁ Γ₂` is the text's `Γ = Γ₁ ⊎ Γ₂`: every entry of `Γ` goes to
the left part, to the right part, or, if its type is `Copy`, to both. -/
inductive Split : Ctx → Ctx → Ctx → Prop where
  | nil : Split [] [] []
  | left {b Γ Γ₁ Γ₂} : Split Γ Γ₁ Γ₂ → Split (b :: Γ) (b :: Γ₁) Γ₂
  | right {b Γ Γ₁ Γ₂} : Split Γ Γ₁ Γ₂ → Split (b :: Γ) Γ₁ (b :: Γ₂)
  | both {b Γ Γ₁ Γ₂} : Copy b.2 → Split Γ Γ₁ Γ₂ → Split (b :: Γ) (b :: Γ₁) (b :: Γ₂)
```

A type checker should not try all the ways to split a context: there are up to three choices per variable, so 3^n splits of a context with n variables. The crate instead decides by looking at the two subterms. A variable goes to the subterm that mentions it; a variable that both subterms mention must be `Copy`, and goes to both; otherwise the split fails, and the failure *is* the type error:

```
/// Split `Γ` between a first premise `e1` and a second premise `e2` (in which
/// the variables `bound` are about to be bound). A variable goes where it is
/// used; a variable used by both premises must be `Copy`. Unused variables go
/// to the second premise, unless their name is about to be rebound there.
fn split(
    ctx: &Ctx,
    e1: &Term,
    e2: &Term,
    bound: &[&str],
    site: Site,
    whole: &Term,
```

```
) -> Result<(Ctx, Ctx), TypeError> {
    let (mut left, mut right) = (Ctx::new(), Ctx::new());
    for (x, ty) in ctx {
        let in_first = e1.occurs(x);
        let rebound = bound.contains(&x.as_str());
        let in_second = !rebound && e2.occurs(x);
        match (in_first, in_second) {
            (true, true) if ty.is_copy() => {
                left.push((x.clone(), ty.clone()));
                right.push((x.clone(), ty.clone()));
            }
            (true, true) => {
                return Err(TypeError::UsedTwice {
                    var: x.clone(),
                    ty: ty.clone(),
                    site,
                    term: whole.clone(),
                });
            }
            (true, false) => left.push((x.clone(), ty.clone())),
            (false, true) => right.push((x.clone(), ty.clone())),
            (false, false) if rebound => left.push((x.clone(), ty.clone())),
            (false, false) => right.push((x.clone(), ty.clone())),
        }
    }
    Ok((left, right))
}
```

The parameter `bound` handles rebinding. In $\mathit{let}\ (x, n) = e_1\ \mathit{in}\ e_2$ an occurrence of x inside e_2 refers to the new x, so it must not pull the old x into e_2's part of the context. A variable that neither side uses can go to either side: rule T-Var below accepts extra variables in the context, and that is weakening.

28.5 The typing rules

The judgment of the calculus is

$$\Delta; \Gamma \vdash e : \tau$$

read "with the lifetimes Δ in scope and the variables Γ, the term e has type τ". Δ is a list of lifetime names, Rust's `<'a, 'b>`; Γ is an affine context. Two conventions keep the rules short. First, $\Gamma, x : \tau$ is only written when x is not already in Γ; you can always rename a bound variable to make this true. (Rebinding an owner after a borrow, as in the `let` above, is allowed: by then the old x has been consumed and is no longer in the context.) Second, $\Gamma_1 \uplus \Gamma_2$ in the conclusion of a rule means "any context that splits into Γ_1 and Γ_2". There are thirteen rules. As in Chapter 1, they are a *model*: Rust has no official formal semantics, and these rules describe a fragment of what the compiler enforces, in a simplified form.

The affine core

$$\frac{(x : \tau) \in \Gamma}{\Delta; \Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{}{\Delta; \Gamma \vdash n : \mathit{Int}}\ \text{T-Int} \qquad \frac{}{\Delta; \Gamma \vdash () : \mathit{Unit}}\ \text{T-Unit}$$

$$\frac{\Delta; \Gamma_1 \vdash e_1 : \tau_1 \qquad \Delta; \Gamma_2 \vdash e_2 : \tau_2}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash (e_1, e_2) : \tau_1 \otimes \tau_2}\ \text{T-Pair}$$

$$\frac{\Delta; \Gamma_1 \vdash e_1 : \tau_1 \qquad \Delta; \Gamma_2, x : \tau_1 \vdash e_2 : \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \mathit{let}\ x = e_1\ \mathit{in}\ e_2 : \sigma}\ \text{T-Let}$$

$$\frac{\Delta; \Gamma_1 \vdash e_1 : \tau_1 \otimes \tau_2 \qquad \Delta; \Gamma_2, x : \tau_1, y : \tau_2 \vdash e_2 : \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \mathit{let}\ (x, y) = e_1\ \mathit{in}\ e_2 : \sigma}\ \text{T-LetPair}$$

$$\frac{\Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \mathit{new}\ e : \mathit{Box}\ \tau}\ \text{T-New} \qquad \frac{\Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \mathit{drop}\ e : \mathit{Unit}}\ \text{T-Drop}$$

These are Chapter 22's rules, with the moves and drops that Part V explained.

- **T-Var**, **T-Int** and **T-Unit** are the leaves. They accept any other variables in Γ, which is how weakening enters the system: a value that is never used is fine. (In our calculus an unused owner is *leaked*, never freed, which is safe; Chapter 24 explained why Rust can afford leaks, and a real compiler would insert the drop for you.)
- **T-Pair**, **T-Let** and **T-LetPair** split the context between their two premises. A variable of a non-`Copy` type is therefore used by at most one of them. This is the rule behind E0382: in `let c = b in (b, c)`, both the right-hand side and the body need `b`, and `Box Int` is not `Copy`.
- **T-LetPair** is the only way to take a pair apart. It consumes the pair once and binds both components, which is how the result of a borrow is unpacked.
- **T-New** allocates, Rust's `Box::new`. **T-Drop** consumes a value of any type, Rust's `std::mem::drop`; at run time it frees the locations the value owns.

Using references

$$\frac{\Delta; \Gamma \vdash e : \&^{\kappa}\ \tau \qquad \mathit{Copy}(\tau)}{\Delta; \Gamma \vdash *e : \tau}\ \text{T-Deref} \qquad \frac{\Delta; \Gamma_1 \vdash e_1 : \&^{\kappa}\ \mathit{mut}\ \tau \qquad \Delta; \Gamma_2 \vdash e_2 : \tau}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash *e_1 := e_2 : \mathit{Unit}}\ \text{T-Assign}$$

Rule **T-Deref** reads through a shared reference. Its second premise matters: reading `*e` produces a copy of the value behind the reference while the original stays where it is, so the value must be duplicable. Without $\mathit{Copy}(\tau)$ you could read a *Box Int* out of a shared reference and end up with two owners of one location. Rust reports exactly this situation as E0507, "cannot move out of `*s` which is behind a shared reference".

Rule **T-Assign** writes through a mutable reference, and only through a mutable one: there is no rule that assigns through $\&^{\kappa}\ \tau$, which is why `*s = 1` with `s: &i32` is E0594. The two premises split the context, so the reference and the new value cannot share a non-`Copy` variable. At run time the old value is dropped, which is what Rust's `*r = v` does: an assignment to an initialized place runs the old value's destructor first. (A purely affine calculus would rather use a *swap* that returns the old value, Rust's `std::mem::replace`; dropping is closer to what you write every day.)

Borrowing

Now the three rules that go beyond affine logic:

$$\frac{\Delta; \Gamma_1 \vdash e_1 : \mathit{Box}\ \tau \qquad \Delta, \alpha; \Gamma_2, y : \&^{\alpha}\ \tau \vdash e_2 : \sigma \qquad \alpha \notin \Delta \qquad \alpha \notin \mathrm{flv}(\sigma)}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \mathit{borrow}\ y = \&^{\alpha} e_1\ \mathit{in}\ e_2 : \mathit{Box}\ \tau \otimes \sigma}\ \text{T-Borrow}$$

$$\frac{\Delta; \Gamma_1 \vdash e_1 : \mathit{Box}\ \tau \qquad \Delta, \alpha; \Gamma_2, y : \&^{\alpha}\ \mathit{mut}\ \tau \vdash e_2 : \sigma \qquad \alpha \notin \Delta \qquad \alpha \notin \mathrm{flv}(\sigma)}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \mathit{borrow}\ y = \&^{\alpha}\ \mathit{mut}\ e_1\ \mathit{in}\ e_2 : \mathit{Box}\ \tau \otimes \sigma}\ \text{T-BorrowMut}$$

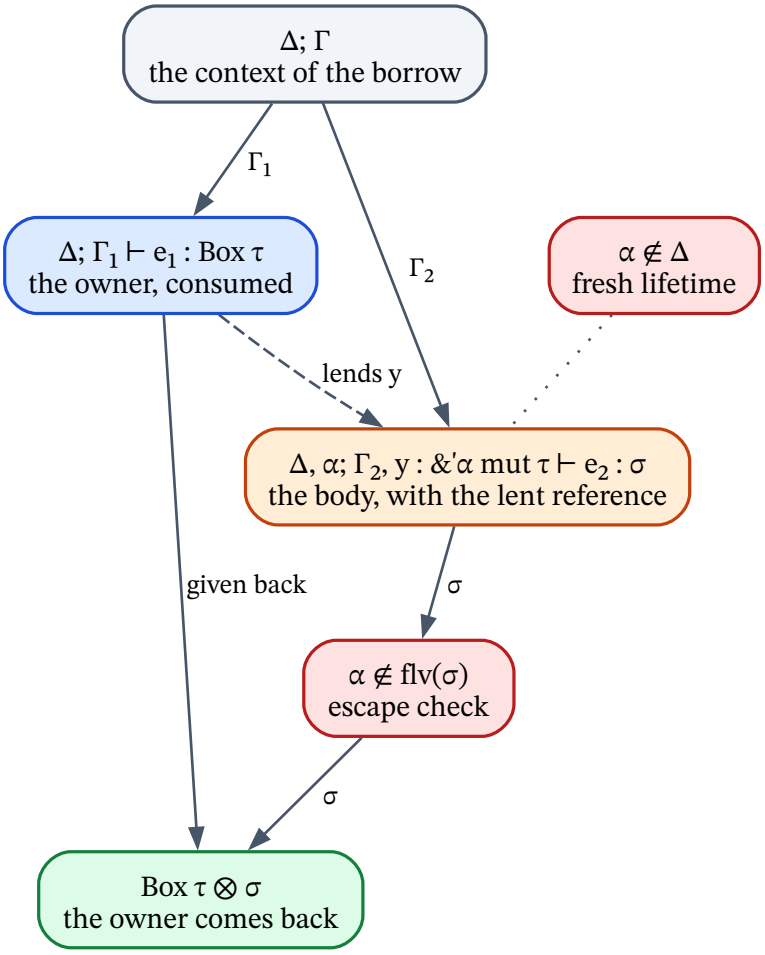
$$\frac{\Delta; \Gamma_1 \vdash e_1 : \&^{\kappa}\ \mathit{mut}\ \tau \qquad \Delta, \alpha; \Gamma_2, y : \&^{\alpha}\ \mathit{mut}\ \tau \vdash e_2 : \sigma \qquad \alpha \notin \Delta \qquad \alpha \notin \mathrm{flv}(\sigma)}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \mathit{borrow}\ y = \&^{\alpha}\ \mathit{mut} * e_1\ \mathit{in}\ e_2 : \&^{\kappa}\ \mathit{mut}\ \tau \otimes \sigma}\ \text{T-Reborrow}$$

The three rules have the same shape, so let us read T-BorrowMut premise by premise, together with the Rust it models:

```
let mut owner: Box<T> = ...;
let result = {
    let y = &mut *owner;   // 'α starts here
    body                   // uses y; must not use `owner`
};                         // 'α ends here; `owner` is usable again
```

1. $\boldsymbol{\Delta; \Gamma_1 \vdash e_1 : \mathit{Box}\,\tau}$**: the owner is consumed.** The owner gets its own part Γ_1 of the context. Because of the split, a non-`Copy` variable used to produce the owner (usually the owner variable itself) is not available to the body. This premise is the formal reason why you cannot touch `owner` while it is mutably borrowed, and it is pure affine bookkeeping.
2. $\boldsymbol{\Delta, \alpha; \Gamma_2, y : \&^{\alpha} \mathit{mut}\,\tau \vdash e_2 : \sigma}$**: the body gets the reference.** The body is checked with the rest of the context, one more variable y of reference type, and one more lifetime α in scope. Since $\&^{\alpha} \mathit{mut}\,\tau$ is not `Copy`, the body can use y at most once; to write through it twice, the body must reborrow it, as derivation I will show.
3. $\boldsymbol{\alpha \notin \Delta}$**: the lifetime is fresh.** The borrow introduces a new region, distinct from every lifetime already in scope, and binds it for the duration of the body. In Rust you never name this lifetime; the compiler invents one for every borrow expression.
4. $\boldsymbol{\alpha \notin \mathrm{flv}(\sigma)}$**: the escape check.** The type of the body's result must not mention α. A value of type σ is what survives the borrow, and if σ mentioned α it could contain a reference created by this borrow, still pointing at the location after the owner is back in charge of it and free to drop it. This is the premise that rejects a dangling reference.
5. **The conclusion** $\boldsymbol{\mathit{Box}\,\tau \otimes \sigma}$**: the owner comes back.** Ownership was lent, not given away.

T-Borrow differs only in the kind of reference the body receives, a shared one, which the body may copy freely by rule S-Both. T-Reborrow differs in what is lent: not an owner but a mutable reference $\&^{\kappa} \mathit{mut}\,\tau$, and what comes back is that same reference, *with its original lifetime* κ. The new reference lives for the fresh α, which is introduced inside the scope of κ and therefore shorter: the outlives relation of Chapter 26 holds by construction, which is why our calculus can do without explicit outlives constraints. The figure below shows the shape that the three rules share.



The anatomy of rule T-BorrowMut. The context is split; the owner's part is consumed and lends a reference to the body; the body runs under a fresh lifetime; the escape check filters the body's result; the owner comes back beside it. Blue marks affine bookkeeping, red the region apparatus.

Which premises are affine and which are not? The question matters for the question of this book, so here is the tally:

Ingredient	Mechanism	Affine?
$\Gamma_1 \uplus \Gamma_2$	context splitting	yes
owner consumed by the first premise	affine use	yes
owner returned in the conclusion	the lend-and-return shape	neutral
$\&^{\alpha} mut\ \tau$ not `Copy`	no contraction	yes
$\&^{\alpha}\ \tau$ is `Copy`	contraction restored	no: the opposite
α fresh and scoped to the body	region binding	no
$\alpha \notin \mathrm{flv}(\sigma)$	escape check	no

Half of the ingredients are affine typing; the other half are a region system. Keep this table in mind for Chapter 33.

The rules in Lean and in Rust

In Lean the judgment is an inductive relation, one constructor per rule (`code/lean/Book/Ch28/Typing.lean`):

```
/-- The typing judgment `Δ; Γ ⊢ e : τ`: `Δ` lists the lifetimes in scope,
`Γ` the variables. The premises `x ∉ dom Γ₂` spell out the text's convention
that a context is only extended with a variable it does not already contain. -/
inductive HasType : List Lft → Ctx → Term → Ty → Prop where
  | var {Δ Γ x τ} :
      lookup Γ x = some τ →
      HasType Δ Γ (.var x) τ
  | lit {Δ Γ n} : HasType Δ Γ (.lit n) .int
  | unit {Δ Γ} : HasType Δ Γ .unit .unit
  | pair {Δ Γ Γ₁ Γ₂ e₁ e₂ τ₁ τ₂} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ τ₁ →
      HasType Δ Γ₂ e₂ τ₂ →
      HasType Δ Γ (.pair e₁ e₂) (.pair τ₁ τ₂)
  | letIn {Δ Γ Γ₁ Γ₂ x e₁ e₂ τ₁ σ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ τ₁ →
      x ∉ dom Γ₂ →
      HasType Δ ((x, τ₁) :: Γ₂) e₂ σ →
      HasType Δ Γ (.letIn x e₁ e₂) σ
  | letPair {Δ Γ Γ₁ Γ₂ x y e₁ e₂ τ₁ τ₂ σ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ (.pair τ₁ τ₂) →
      x ∉ dom Γ₂ → y ∉ dom Γ₂ → x ≠ y →
      HasType Δ ((x, τ₁) :: (y, τ₂) :: Γ₂) e₂ σ →
      HasType Δ Γ (.letPair x y e₁ e₂) σ
  | new {Δ Γ e τ} :
      HasType Δ Γ e τ →
      HasType Δ Γ (.new e) (.box τ)
  | drop {Δ Γ e τ} :
      HasType Δ Γ e τ →
      HasType Δ Γ (.drop e) .unit
  | deref {Δ Γ e κ τ} :
      HasType Δ Γ e (.ref κ τ) →
      Copy τ →
      HasType Δ Γ (.deref e) τ
  | assign {Δ Γ Γ₁ Γ₂ e₁ e₂ κ τ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ (.refMut κ τ) →
      HasType Δ Γ₂ e₂ τ →
      HasType Δ Γ (.assign e₁ e₂) .unit
  | borrow {Δ Γ Γ₁ Γ₂ α y e₁ e₂ τ σ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ (.box τ) →
      y ∉ dom Γ₂ →
      HasType (α :: Δ) ((y, .ref α τ) :: Γ₂) e₂ σ →
      α ∉ Δ →
      α ∉ σ.lfts →
      HasType Δ Γ (.borrow α y e₁ e₂) (.pair (.box τ) σ)
  | borrowMut {Δ Γ Γ₁ Γ₂ α y e₁ e₂ τ σ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ (.box τ) →
      y ∉ dom Γ₂ →
      HasType (α :: Δ) ((y, .refMut α τ) :: Γ₂) e₂ σ →
      α ∉ Δ →
      α ∉ σ.lfts →
      HasType Δ Γ (.borrowMut α y e₁ e₂) (.pair (.box τ) σ)
  | reborrow {Δ Γ Γ₁ Γ₂ α y e₁ e₂ κ τ σ} :
      Split Γ Γ₁ Γ₂ →
      HasType Δ Γ₁ e₁ (.refMut κ τ) →
      y ∉ dom Γ₂ →
      HasType (α :: Δ) ((y, .refMut α τ) :: Γ₂) e₂ σ →
      α ∉ Δ →
      α ∉ σ.lfts →
      HasType Δ Γ (.reborrow α y e₁ e₂) (.pair (.refMut κ τ) σ)
```

Compare a constructor with its rule. The split, written $\Gamma_1 \uplus \Gamma_2$ in the conclusion on paper, is a premise `Split Γ Γ₁ Γ₂`. The convention that a context is only extended with a new variable becomes explicit premises such as `y ∉ dom Γ₂` (and `x ≠ y` for a pair pattern): a proof assistant does not accept conventions, so everything that the paper leaves implicit must be written down. `α :: Δ` is Δ, α, and `α ∉ σ.lfts` is the escape check.

In the crate, the three borrow rules share one function. It follows the premises in order: freshness, split, the owner, the body with the new lifetime and variable, and the escape check last, because only then is σ known:

```
/// Rules T-Borrow, T-BorrowMut and T-Reborrow.
fn check_borrow(
    delta: &[Lifetime],
    ctx: &Ctx,
    e: &Term,
    kind: BorrowKind,
    alpha: &Lifetime,
    y: &str,
    owner: &Term,
    body: &Term,
) -> Result<Derivation, TypeError> {
    let rule = borrow_rule(kind);
    // Freshness: α ∉ Δ.
    if delta.contains(alpha) {
        return Err(TypeError::LifetimeInScope {
            lifetime: alpha.clone(),
            rule,
            term: e.clone(),
        });
    }
    // Γ = Γ₁ ⊎ Γ₂: the owner (or the reborrowed reference) gets Γ₁.
    let (c1, mut c2) = split(ctx, owner, body, &[y], Site::Borrow(kind), e)?;
    let d1 = check(delta, &c1, owner)?;
    // What the body receives (y's type), and what comes back afterwards.
    let (y_ty, returned) = match (kind, d1.conclusion.ty.clone()) {
        (BorrowKind::Shared, Ty::Box(t)) => {
            (Ty::Ref(alpha.clone(), t.clone()), Ty::Box(t))
        }
        (BorrowKind::Mut, Ty::Box(t)) => {
            (Ty::RefMut(alpha.clone(), t.clone()), Ty::Box(t))
        }
        (BorrowKind::Reborrow, Ty::RefMut(k, t)) => {
            (Ty::RefMut(alpha.clone(), t.clone()), Ty::RefMut(k, t))
        }
        (kind, found) => {
            let expected = match kind {
                BorrowKind::Reborrow => "a mutable reference &'κ mut τ",
                _ => "an owner Box τ",
            };
            return Err(TypeError::Mismatch {
                rule,
                expected: expected.into(),
                found,
                term: e.clone(),
            });
        }
    };
    // The body is checked with α in scope and y bound to the new reference.
    let mut delta2 = delta.to_vec();
    delta2.push(alpha.clone());
    let mut notes = vec![
        split_note(&c1, &c2),
        format!("fresh: '{alpha} ∉ Δ = {{{}}}", quoted(delta)),
    ];
    c2.push((y.to_string(), y_ty));
```

```
    let d2 = check(&delta2, &c2, body)?;
    // The escape check: α ∉ flv(σ).
    let sigma = d2.conclusion.ty.clone();
    if sigma.mentions(alpha) {
        return Err(TypeError::Escape {
            lifetime: alpha.clone(),
            ty: sigma,
            rule,
            term: e.clone(),
        });
    }
    notes.push(format!("escape check: '{alpha} ∉ flv({sigma})"));
    let ty = Ty::pair(returned, sigma);
    Ok(node(rule, delta, ctx, e, ty, notes, vec![d1, d2]))
}
```

Every rule of the calculus is *syntax-directed*: for each form of term exactly one rule can conclude it, so the checker never searches. When it succeeds, it returns a `Derivation`, a tree of rule applications with their conclusions and side conditions:

```
/// A judgment `Δ; Γ ⊢ e : τ`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Judgment {
    pub lifetimes: Vec<Lifetime>,
    pub ctx: Ctx,
    pub term: Term,
    pub ty: Ty,
}

/// A derivation tree: the rule used at the root, its conclusion, the side
/// conditions it checked, and the derivations of its premises.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Derivation {
    pub rule: &'static str,
    pub conclusion: Judgment,
    pub side_conditions: Vec<String>,
    pub premises: Vec<Derivation>,
}
```

28.6 Operational semantics

A type system is only as good as the promise it keeps, so we need to say what programs *do*. The semantics runs terms against a store, and its central device is a **loan state** for every location.

Values are integers, unit, pairs of values and three kinds of pointer:

$$v, w ::= n \mid () \mid (v_1, v_2) \mid \mathit{own}\,\ell \mid \mathit{shr}\,\ell \mid \mathit{uniq}\,\ell$$

$\mathit{own}\,\ell$ is the value of an owner, $\mathit{shr}\,\ell$ of a shared reference and $\mathit{uniq}\,\ell$ of a mutable one, each pointing to a location ℓ. At run time there are no lifetimes: references are plain pointers, exactly as in compiled Rust.

The store S maps each allocated location to its contents and its loan state. We write $S(\ell) = v^m$ when ℓ holds v in state m, where m is *none* (not lent), *shared* (lent through a shared borrow) or *mut* (lent through a mutable borrow). The loan states play no part in the computation itself; they are the bookkeeping that lets the semantics *detect* a violation of the aliasing rules.

A borrow in progress is a run-time term $\mathit{restore}\,w\,\mathit{after}\,e$: run e, then hand back w (the owner, or the reborrowed reference) next to the result. A configuration $\langle S; e\rangle$ steps to $\langle S'; e'\rangle$ by these rules:

$$\frac{\ell \notin \mathrm{dom}(S)}{\langle S;\ \mathit{new}\ v\rangle \to \langle S[\ell \mapsto v^{\mathit{none}}];\ \mathit{own}\,\ell\rangle}\ \text{E-New} \qquad \frac{S' = \mathrm{free}(S, v)}{\langle S;\ \mathit{drop}\ v\rangle \to \langle S';\ ()\rangle}\ \text{E-Drop}$$

$$\frac{}{\langle S;\ \mathit{let}\ x = v\ \mathit{in}\ e\rangle \to \langle S;\ e[v/x]\rangle}\ \text{E-Let} \qquad \frac{}{\langle S;\ \mathit{let}\ (x, y) = (v_1, v_2)\ \mathit{in}\ e\rangle \to \langle S;\ e[v_1/x, v_2/y]\rangle}\ \text{E-LetPair}$$

$$\frac{S(\ell) = v^{\mathit{shared}}}{\langle S;\ *\,\mathit{shr}\,\ell\rangle \to \langle S;\ v\rangle}\ \text{E-Deref} \qquad \frac{S(\ell) = w^{\mathit{mut}} \qquad S' = \mathrm{free}(S[\ell \mapsto v^{\mathit{mut}}], w)}{\langle S;\ *\,\mathit{uniq}\,\ell := v\rangle \to \langle S';\ ()\rangle}\ \text{E-Assign}$$

$$\frac{S(\ell) = v^{\mathit{none}}}{\langle S;\ \mathit{borrow}\ y = \&^{\alpha}\ \mathit{own}\,\ell\ \mathit{in}\ e\rangle \to \langle S[\ell \mapsto v^{\mathit{shared}}];\ \mathit{restore}\ \mathit{own}\,\ell\ \mathit{after}\ e[\mathit{shr}\,\ell/y]\rangle}\ \text{E-Borrow}$$

$$\frac{S(\ell) = v^{\mathit{none}}}{\langle S;\ \mathit{borrow}\ y = \&^{\alpha}\mathit{mut}\ \mathit{own}\,\ell\ \mathit{in}\ e\rangle \to \langle S[\ell \mapsto v^{\mathit{mut}}];\ \mathit{restore}\ \mathit{own}\,\ell\ \mathit{after}\ e[\mathit{uniq}\,\ell/y]\rangle}\ \text{E-BorrowMut}$$

$$\frac{S(\ell) = v^{\mathit{mut}}}{\langle S;\ \mathit{borrow}\ y = \&^{\alpha}\mathit{mut} * \mathit{uniq}\,\ell\ \mathit{in}\ e\rangle \to \langle S;\ \mathit{restore}\ \mathit{uniq}\,\ell\ \mathit{after}\ e[\mathit{uniq}\,\ell/y]\rangle}\ \text{E-Reborrow}$$

$$\frac{S(\ell) = w^{m}}{\langle S;\ \mathit{restore}\ \mathit{own}\,\ell\ \mathit{after}\ v\rangle \to \langle S[\ell \mapsto w^{\mathit{none}}];\ (\mathit{own}\,\ell, v)\rangle}\ \text{E-EndBorrow}$$

$$\frac{}{\langle S;\ \mathit{restore}\ \mathit{uniq}\,\ell\ \mathit{after}\ v\rangle \to \langle S;\ (\mathit{uniq}\,\ell, v)\rangle}\ \text{E-EndReborrow}$$

Here $e[v/x]$ is substitution (Chapter 7), and $\mathrm{free}(S, v)$ removes from the store every location that v owns: for each $\mathit{own}\,\ell$ inside v, the location ℓ, and recursively whatever its contents own. It is defined only if each of those locations is still allocated and in state *none*. One more rule, the congruence rule, lets a step happen inside a larger term, always in the leftmost subterm that is not yet a value (a pair is evaluated left to right, a `let` evaluates its right-hand side before its body, a borrow evaluates its owner before lending it, and a frame runs its body).

The premises above the lines are **guards**. Reading requires the location to be lent shared; writing requires it to be lent mutably; borrowing an owner requires it not to be lent; dropping requires everything dropped to be allocated and not lent. A configuration that is not a value and where no rule applies is **stuck**, and every way of getting stuck corresponds to a memory-safety bug: reading or writing freed memory, freeing twice, freeing something that is lent, writing to a location while it is shared. Being stuck is the calculus's undefined behavior. A *panic* would be something else entirely: a defined, orderly way to stop.

Notice what E-Reborrow does to the store: nothing. At run time `&mut *r` is just `r`; everything a reborrow means happens in the type system.

The semantics in Rust

The crate's `eval` module implements these rules as a small-step interpreter. The store is a map from locations to cells with a loan state:

```
/// The loan state of a location.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Loan {
    /// Not borrowed: the owner may be borrowed, dropped or moved.
    None,
    /// Lent through a shared borrow: readable through `shr ℓ`.
    Shared,
    /// Lent through a mutable borrow: writable through `uniq ℓ`.
```

```
    Mut,
}

/// One store location: its value and its loan state.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Cell {
    pub value: Term,
    pub loan: Loan,
}

/// The store: a finite map from locations to cells.
#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct Store {
    cells: BTreeMap<usize, Cell>,
    next: usize,
}
```

and the three borrow rules share one function that checks the guard, updates the loan state, substitutes a pointer for y and wraps the body in its frame:

```
/// The three borrow rules. A borrow lends the location (guard: it is not
/// lent already), substitutes a reference for `y` in the body, and wraps
/// the body in a frame that gives the owner back at the end.
fn borrow(
    &mut self,
    kind: BorrowKind,
    y: &str,
    owner: Term,
    body: &Term,
    whole: &Term,
) -> Result<(Term, &'static str), Stuck> {
    let (l, needs, lend_as, reference, rule) = match (kind, &owner) {
        (BorrowKind::Shared, Term::Ptr(Ptr::Own, l)) => {
            (*l, Loan::None, Loan::Shared, Ptr::Shr, "E-Borrow")
        }
        (BorrowKind::Mut, Term::Ptr(Ptr::Own, l)) => {
            (*l, Loan::None, Loan::Mut, Ptr::Uniq, "E-BorrowMut")
        }
        (BorrowKind::Reborrow, Term::Ptr(Ptr::Uniq, l)) => {
            (*l, Loan::Mut, Loan::Mut, Ptr::Uniq, "E-Reborrow")
        }
        _ => return Err(ill_formed(whole, "cannot borrow this value")),
    };
    let cell = self.store.cell(l, whole)?;
    if cell.loan != needs {
        let reason = format!("ℓ{l} is {}, so it cannot be {}", cell.loan, lend_as);
        return Err(Stuck {
            redex: whole.summary(),
            reason,
        });
    }
    cell.loan = lend_as;
    let body = subst(body, y, &Term::Ptr(reference, l));
    Ok((Term::Restore(Box::new(owner), Box::new(body)), rule))
}
```

Here is a complete well-typed program, closed so that it can run: it allocates a location, borrows it mutably, writes through two reborrows, gives the owner back, reads the result through a shared borrow and frees the location (`crates/borrowck/examples/ch28_run.rs`):

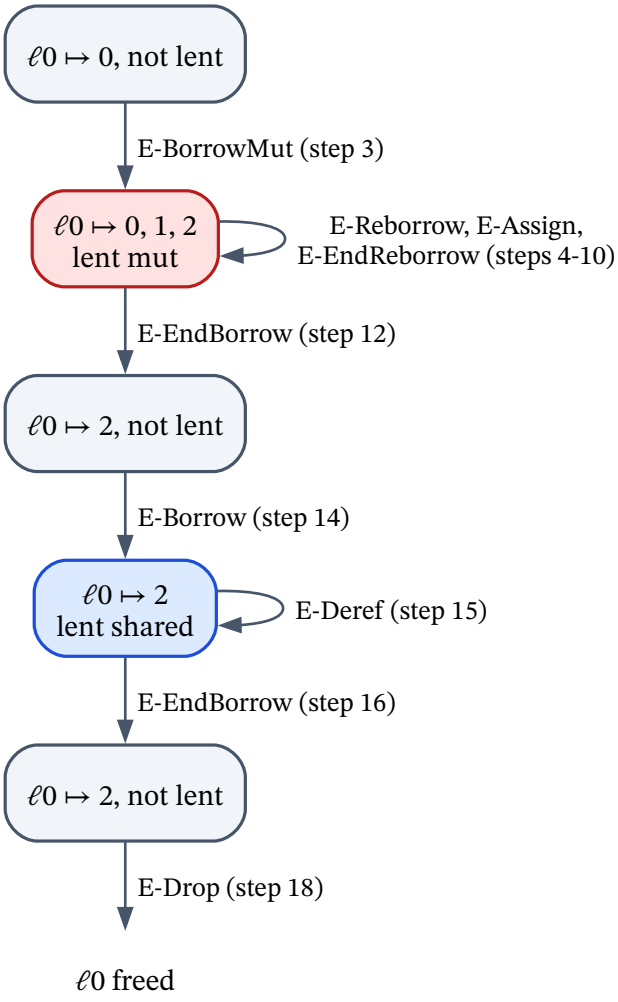
```
const PROGRAM: &str = "
fn main() {
    let b = new 0 in
    let (b, u) = (borrow r = &'a mut b in
                    let (r1, u1) = (borrow y1 = &'b1 mut *r in *y1 := 1) in
                    let (r2, u2) = (borrow y2 = &'b2 mut *r1 in *y2 := 2) in
```

```
                    ()) in
    let (b, n) = (borrow s = &'c b in *s) in
    let u = drop b in
    n
}";
```

The example checks it and runs it, printing the rule of every step and the store after it:

```
type: Int
 1  E-New          ℓ0 ↦ 0
 2  E-Let          ℓ0 ↦ 0
 3  E-BorrowMut    ℓ0 ↦ 0 [mut]
 4  E-Reborrow     ℓ0 ↦ 0 [mut]
 5  E-Assign       ℓ0 ↦ 1 [mut]
 6  E-EndReborrow  ℓ0 ↦ 1 [mut]
 7  E-LetPair      ℓ0 ↦ 1 [mut]
 8  E-Reborrow     ℓ0 ↦ 1 [mut]
 9  E-Assign       ℓ0 ↦ 2 [mut]
10  E-EndReborrow  ℓ0 ↦ 2 [mut]
11  E-LetPair      ℓ0 ↦ 2 [mut]
12  E-EndBorrow    ℓ0 ↦ 2
13  E-LetPair      ℓ0 ↦ 2
14  E-Borrow       ℓ0 ↦ 2 [shared]
15  E-Deref        ℓ0 ↦ 2 [shared]
16  E-EndBorrow    ℓ0 ↦ 2
17  E-LetPair      ℓ0 ↦ 2
18  E-Drop         ∅
19  E-Let          ∅
value: 2
```

Follow the loan state of ℓ_0 down the right column. It becomes *mut* at step 3, stays *mut* through both reborrows (steps 4 to 10: reborrows do not touch the store), goes back to not lent when the mutable borrow ends at step 12, is lent shared for the read at steps 14 and 15, and the location disappears at step 18. The figure below draws the same history.



The loan state of location ℓ0 while the program of ch28_run runs, with the reduction rule that causes each change.

28.7 Metatheory

What does the type system guarantee about the semantics? The statements follow the pattern of Chapter 9, adapted to a store. To type a configuration we need a **store typing** Σ that gives each location the type of its contents, typing rules for the run-time values ($own\,\ell$: $Box\,\Sigma(\ell)$ and so on), and a notion of a **well-formed configuration**: the stored values have the types Σ says, and the loan state of every location agrees with the term (a location is lent exactly when a `restore` frame for it is running, shared or mutable according to the kind of borrow, and its references occur only inside that frame).

Theorem (progress). If $\langle S; e\rangle$ is well formed and $\cdot; \cdot \underset{\Sigma}{\vdash} e : \tau$, then e is a value or $\langle S; e\rangle \to \langle S'; e'\rangle$ for some S' and e'.

Theorem (preservation). If $\langle S; e\rangle$ is well formed, $\cdot; \cdot \underset{\Sigma}{\vdash} e : \tau$ and $\langle S; e\rangle \to \langle S'; e'\rangle$, then $\langle S'; e'\rangle$ is well formed and $\cdot; \cdot \underset{\Sigma'}{\vdash} e' : \tau$ for a store typing Σ' that agrees with Σ on the locations that are still allocated.

Together they give safety: a well-typed closed program never gets stuck, so it never reads freed memory, never frees twice, never frees what is lent and never writes to a shared location. We will not prove the two theorems here; a full proof is long (the run-time typing must record which lifetime lends which location, so that the escape check can be connected to the values that leave a frame), and RustBelt, which Chapter 32 describes, proves a much stronger result for a much larger language. What we can do is see *why* each guard holds in a well-typed program, because each one rests on a single premise of the rules:

- **Borrowing or dropping an owner finds it not lent.** While ℓ is lent, its owner $own\,\ell$ sits inside a `restore` frame, where no rule can reach it, and there is no other copy of it because $Box\,\tau$ is not `Copy`. *Affinity.*
- **Reading through $shr\,\ell$ finds ℓ lent shared.** A shared reference is created inside a frame and cannot leave it, because its type mentions the borrow's lifetime and the escape check forbids exactly that. *Regions.*
- **Writing through $uniq\,\ell$ finds ℓ lent mutably.** The same argument, for mutable references.
- **No location is freed twice.** Only an owner frees, and owners cannot be duplicated. *Affinity again.*

Here is the division of labor in one table:

Guarantee	Comes from
no double free	$Box\,\tau$ is not `Copy` (affinity)
a mutable reference is unique	$\&^{\kappa} mut\,\tau$ is not `Copy` (affinity)
the owner is untouchable while lent	the borrow consumes it until its body ends (affinity plus scoping)
no dangling reference	the escape check (regions)
lifetimes do not get confused	freshness (regions)

The semantics is deliberately coarse, and it is worth knowing where. A reborrow does not change the store, so if an ill-typed program let a reborrowed reference escape, two $uniq\,\ell$ pointers would coexist and the machine would not notice. Semantic models of Rust such as Stacked Borrows and Tree Borrows, which Chapter 32 describes, tag every pointer to catch exactly such cases.

The guards are not decoration. The `ch28_go_wrong` example takes two programs that the checker rejects and runs them anyway:

```
/// A reference escapes its borrow and is used after the owner is dropped.
const DANGLING: &str = "
fn dangling() {
    let b = new 5 in
    let (b, r) = (borrow y = &'a b in y) in
    let u = drop b in
    *r
}";
```

```
/// Derivation II, closed: the owner is created by the program itself.
const ALIASING: &str = "
fn aliasing() {
    let v = new 1 in
    borrow f = &'a v in
      let (v2, u) = (borrow m = &'b mut v in *m := 4) in
      *f
}";
```

```
== dangling
error: borrowed value does not live long enough
-- run without checking:
 1  E-New          ℓ0 ↦ 5
 2  E-Let          ℓ0 ↦ 5
 3  E-Borrow       ℓ0 ↦ 5 [shared]
 4  E-EndBorrow    ℓ0 ↦ 5
 5  E-LetPair      ℓ0 ↦ 5
 6  E-Drop         ∅
 7  E-Let          ∅
stuck at `*shr ℓ0`:
  ℓ0 has been freed (use after free)

== aliasing
error: `v` is used while it is borrowed
-- run without checking:
 1  E-New          ℓ0 ↦ 1
 2  E-Let          ℓ0 ↦ 1
 3  E-Borrow       ℓ0 ↦ 1 [shared]
stuck at `borrow m = &'b mut own ℓ0 in …`:
  ℓ0 is lent shared, so it cannot be lent mut
```

The first program lets a shared reference escape its borrow, drops the owner and reads through the reference: a use after free, which the machine catches as a failed guard. The checker had rejected it with the escape check. The second is derivation II, which we are about to study: the machine gets stuck when the body tries to lend ℓ_0 mutably while it is lent shared.

28.8 Worked derivation I: a `&mut` reborrow

Here is a puzzle that every Rust programmer meets eventually. A `&mut i32` is not `Copy`; by everything Part V said, it is an affine value that can be used at most once. Yet this compiles (`code/rust/examples/ch28_reborrows.rs`):

```
fn set(r: &mut i32, value: i32) {
    *r = value;
}

/// Derivation I in Rust: `r` is used twice. Each call receives a fresh
/// reborrow `&mut *r`, which the compiler inserts.
fn twice(r: &mut i32) {
    set(r, 1);
    set(r, 2);
}
```

The resolution is that `set(r, 1)` does not pass `r`. Because `set` expects a `&mut i32`, the compiler inserts an **implicit reborrow** and passes `&mut *r`, a new reference that lives only for the call. During the call `r` is lent; after it, `r` is usable again. Written out by hand, with one block per reborrow:

```
/// The same function with the reborrows written out, one block for each
/// `borrow y = &'b mut *r in ...` of the calculus.
```

```
fn twice_explicit(r: &mut i32) {
    {
        let y1 = &mut *r;
        *y1 = 1;
    }
    {
        let y2 = &mut *r;
        *y2 = 2;
    }
}
```

Inlining `set` turns this into a term of the calculus. In the crate's syntax, as a program whose parameters form the initial context (`crates/borrowck/examples/ch28_borrowck.rs`):

```
/// Derivation I. In Rust: `fn twice(r: &mut i32) { set(r, 1); set(r, 2); }`,
/// where each call passes an implicit reborrow `&mut *r`.
const TWICE: &str = "
fn twice<'a>(r: &'a mut Int) {
    let (r1, u1) = (borrow y1 = &'b1 mut *r in *y1 := 1) in
    let (r2, u2) = (borrow y2 = &'b2 mut *r1 in *y2 := 2) in
    ()
}";
```

We want to derive $\alpha; r : M_\alpha \vdash \mathit{twice} : \mathit{Unit}$, where M_κ abbreviates $\&^\kappa \mathit{mut}\ \mathit{Int}$ and we write α for `'a`, β_1 and β_2 for `'b1` and `'b2`. Call the two reborrows E_1 and E_2.

Step 1: the body of the first reborrow. In the lifetime context $\Delta_1 = \alpha, \beta_1$, the body writes `1` through y_1. Rule T-Assign splits the context $y_1 : M_{\beta_1}$, giving y_1 to the reference and nothing to the literal:

$$\mathcal{D}_1 = \dfrac{\dfrac{}{\Delta_1; y_1 : M_{\beta_1} \vdash y_1 : M_{\beta_1}}\ \text{T-Var} \qquad \dfrac{}{\Delta_1; \cdot \vdash 1 : \mathit{Int}}\ \text{T-Int}}{\Delta_1; y_1 : M_{\beta_1} \vdash * y_1 := 1 : \mathit{Unit}}\ \text{T-Assign}$$

Step 2: the reborrow. Rule T-Reborrow, with $\kappa := \alpha$, $\tau := \mathit{Int}$ and $\sigma := \mathit{Unit}$, splits $r : M_\alpha$ by giving r to the reborrowed expression and nothing to the body:

$$\dfrac{\dfrac{}{\alpha; r : M_\alpha \vdash r : M_\alpha}\ \text{T-Var} \qquad \mathcal{D}_1 \qquad \beta_1 \notin \{\alpha\} \qquad \beta_1 \notin \mathrm{flv}(\mathit{Unit})}{\alpha; r : M_\alpha \vdash E_1 : M_\alpha \otimes \mathit{Unit}}\ \text{T-Reborrow}$$

All four premises hold: the first by T-Var, the second is $\mathcal{D}_1$, the lifetime β_1 is fresh, and the body's type Unit mentions no lifetime at all, so the escape check passes.

Now look at the conclusion. The type is $M_\alpha \otimes \mathit{Unit}$: the reborrow **consumed** r, exactly as affinity demands, and **produced a new reference at the same lifetime** α, as the first component of the pair. So `r` really is used only once. What makes `set(r, 1); set(r, 2);` legal is not that `&mut` can be duplicated, but that each reborrow hands back a fresh binding of the same type. Rust hides the rebinding; the calculus makes it a `let`.

Step 3: unpack and repeat. Rule T-LetPair unpacks the pair into $r_1 : M_\alpha$ and $u_1 : \mathit{Unit}$. The second premise's context, $r_1 : M_\alpha, u_1 : \mathit{Unit}$, contains a reference of exactly the same type as the original r, so Steps 1 and 2 apply again with r_1, β_2 and y_2, giving $\alpha; r_1 : M_\alpha \vdash E_2 : M_\alpha \otimes \mathit{Unit}$. A second T-LetPair unpacks that pair, and the final $()$ is typed by T-Unit, whose context u_1, r_2, u_2 is simply ignored (weakening). The derivation is complete.

Here is the same derivation as the crate prints it. Each line is a rule application; the lines below a rule list its side conditions (the split and, for borrows, freshness and the escape check), followed by its premises:

```
== twice (Δ = {'a})
T-LetPair  r: &'a mut Int ⊢ let (r1, u1) = … in … : Unit
│ split: Γ₁ = r | Γ₂ = ·
├─ T-Reborrow  r: &'a mut Int ⊢ borrow y1 = &'b1 mut *r in … : &'a mut Int ⊗ Unit
│  │ split: Γ₁ = r | Γ₂ = ·
│  │ fresh: 'b1 ∉ Δ = {'a}
│  │ escape check: 'b1 ∉ flv(Unit)
│  ├─ T-Var  r: &'a mut Int ⊢ r : &'a mut Int
│  └─ T-Assign  y1: &'b1 mut Int ⊢ *y1 := 1 : Unit
│     │ split: Γ₁ = y1 | Γ₂ = ·
│     ├─ T-Var  y1: &'b1 mut Int ⊢ y1 : &'b1 mut Int
│     └─ T-Int  · ⊢ 1 : Int
└─ T-LetPair  r1: &'a mut Int, u1: Unit ⊢ let (r2, u2) = … in () : Unit
   │ split: Γ₁ = r1 | Γ₂ = u1
   ├─ T-Reborrow  r1: &'a mut Int ⊢ borrow y2 = &'b2 mut *r1 in … : &'a mut Int ⊗ Unit
   │  │ split: Γ₁ = r1 | Γ₂ = ·
   │  │ fresh: 'b2 ∉ Δ = {'a}
   │  │ escape check: 'b2 ∉ flv(Unit)
   │  ├─ T-Var  r1: &'a mut Int ⊢ r1 : &'a mut Int
   │  └─ T-Assign  y2: &'b2 mut Int ⊢ *y2 := 2 : Unit
   │     │ split: Γ₁ = y2 | Γ₂ = ·
   │     ├─ T-Var  y2: &'b2 mut Int ⊢ y2 : &'b2 mut Int
   │     └─ T-Int  · ⊢ 2 : Int
   └─ T-Unit  u1: Unit, r2: &'a mut Int, u2: Unit ⊢ () : Unit
```

What each side condition bought

Every premise of T-Reborrow rules out a real bug. Take them away one at a time:

Ingredient	What would go wrong without it
the split $\Gamma_1 \uplus \Gamma_2$	the body could use r as well as y_1: two live `&mut` to one location
$\neg \mathit{Copy}(M_{\beta_1})$	the body could duplicate y_1
β_1 fresh	two regions could share a name; here that only makes the escape check stricter, but the substitution lemmas of a full proof need distinct names
$\beta_1 \notin \mathrm{flv}(\sigma)$	the body could return y_1, which would then alias the returned r_1
the conclusion returns M_α	one reborrow would consume r for good, and the second write would be rejected

The last row is not hypothetical: Rust behaves exactly like that when it does not insert the reborrow. The compiler inserts implicit reborrows only where the expected type is known to be a mutable reference. When the parameter is generic, the argument is simply moved:

```rust
fn consume<T>(_value: T) {}

pub fn twice_generic(r: &mut i32) {
    consume(r);
    consume(r);
}
```

```
error[E0382]: use of moved value: `r`
  --> compile_fail/ch28_no_reborrow.rs:11:13
   |
 9 | pub fn twice_generic(r: &mut i32) {
   |                      - move occurs because `r` has type `&mut i32`, which does not
```

```
implement the `Copy` trait
10 |     consume(r);
   |             - value moved here
11 |     consume(r);
   |             ^ value used here after move
   |
help: consider creating a fresh reborrow of `r` here
   |
10 |     consume(&mut *r);
   |             ++++++
```

Read the error with the calculus in mind: "move occurs because `r` has type `&mut i32`, which does not implement the `Copy` trait". Without a reborrow, `&mut` is affine, and the compiler's suggestion is literally rule T-Reborrow: "consider creating a fresh reborrow of `r` here". With the explicit reborrows, the generic version compiles (`twice_generic` in the example). This is good empirical evidence that the account in this chapter is the right one.

The borrow rules as Rust functions

You can write the three borrow rules as Rust functions, and their signatures say everything the rules say:

```
/// Rule T-Borrow as a Rust function. The owner goes in and comes back out,
/// next to the result of the body. `for<'a>` makes the lifetime fresh, and
/// `S` is chosen outside its scope, so `S` cannot mention `'a`: that is the
/// escape check.
fn lend<T, S>(owner: Box<T>, body: impl for<'a> FnOnce(&'a T) -> S) -> (Box<T>, S) {
    let result = body(&*owner);
    (owner, result)
}

/// Rule T-BorrowMut.
fn lend_mut<T, S>(
    mut owner: Box<T>,
    body: impl for<'a> FnOnce(&'a mut T) -> S,
) -> (Box<T>, S) {
    let result = body(&mut *owner);
    (owner, result)
}

/// Rule T-Reborrow. The reference comes back with its original lifetime `'k`.
fn reborrow<'k, T, S>(
    r: &'k mut T,
    body: impl for<'a> FnOnce(&'a mut T) -> S,
) -> (&'k mut T, S) {
    let result = body(&mut *r);
    (r, result)
}
```

Each function takes the owner (or the reference) by value: it is consumed. The body is a closure that receives the new reference. The higher-ranked bound `for<'a>` (Chapter 12) says that the closure must work for *every* lifetime `'a`, which makes `'a` fresh: the closure cannot assume anything about it. The result type `S` is a type parameter of the outer function, chosen before `'a` is even in scope, so `S` cannot mention `'a`: that is the escape check, enforced by Rust's own type checker. (Try `lend(b, |y| y)`: rustc rejects the closure, because returning `y` would require its lifetime to outlive the closure call.) And the owner comes back in the result pair. With these functions, derivation I can be transcribed into Rust term by term, and it compiles:

```
/// Derivation I, transcribed from the calculus term by term.
fn twice_calculus(r: &mut i32) {
    let (r1, ()) = reborrow(r, |y1| *y1 = 1);
    let (_r2, ()) = reborrow(r1, |y2| *y2 = 2);
}
```

```
twice:            n = 2
twice_explicit:   n = 2
twice_generic:    n = 1
twice_calculus:   n = 2
read_then_push:   first = 1, v = [1, 2, 3, 4]
```

Derivation I in Lean

In Lean a derivation is a proof term, built from the constructors of `HasType` (`code/lean/Book/Ch28/Derivations.lean`). First the term itself and the abbreviation M_κ:

```
/-- `M κ` abbreviates `&'κ mut Int`, as in the text. -/
abbrev M (κ : Lft) : Ty := .refMut κ .int
```

```
/-- The body of `fn twice<'a>(r: &'a mut i32)`: two writes through `r`, each
through a reborrow that hands `r` back under a new name.

    let (r1, u1) = borrow y1 = &'b1 mut *r in *y1 := 1 in
    let (r2, u2) = borrow y2 = &'b2 mut *r1 in *y2 := 2 in
    () -/
def twice : Term :=
  .letPair "r1" "u1" (.reborrow "b1" "y1" (.var "r") (.assign (.var "y1") (.lit 1)))
    (.letPair "r2" "u2" (.reborrow "b2" "y2" (.var "r1") (.assign (.var "y2") (.lit 2)))
      .unit)
```

Then $\mathcal{D}_1$, which is one line: rule T-Assign (`.assign`) applied to the split that sends `y1` left (`.left .nil`), a T-Var leaf whose premise `lookup [("y1", M "b1")] "y1" = some (M "b1")` holds by computation (`rfl`), and a T-Int leaf (`.lit`):

```
/-- `𝒟₁`, the body of the first reborrow:
`'b1, 'a; y1 : &'b1 mut Int ⊢ *y1 := 1 : Unit`. -/
theorem d₁ :
    HasType ["b1", "a"] [("y1", M "b1")] (.assign (.var "y1") (.lit 1)) .unit :=
  .assign (.left .nil) (.var rfl) .lit
```

The first reborrow is rule T-Reborrow applied to its six premises, in the order of the constructor. The side conditions are decidable facts about concrete strings and lists, so `decide` proves them by evaluation, as it did in Chapter 1:

```
/-- The first reborrow, `E₁`, by rule T-Reborrow:
`'a; r : &'a mut Int ⊢ E₁ : &'a mut Int ⊗ Unit`. -/
theorem e₁ :
    HasType ["a"] [("r", M "a")]
      (.reborrow "b1" "y1" (.var "r") (.assign (.var "y1") (.lit 1)))
      (.pair (M "a") .unit) :=
  .reborrow
    (.left .nil)   -- split: `r` goes to the reborrowed expression
    (.var rfl)     -- 'a; r : &'a mut Int ⊢ r : &'a mut Int
    (by decide)    -- `y1` is a new variable
    d₁             -- the body, with 'b1 in scope
    (by decide)    -- freshness: 'b1 ∉ ['a]
    (by decide)    -- escape check: 'b1 ∉ flv(Unit)
```

The second reborrow is the same derivation for `r1` and `'b2`, and the whole derivation puts the pieces together with two T-LetPair steps. The second split, `.left (.right .nil)`, sends `r1` to the reborrow and `u1` to the body:

```
/-- The second reborrow, `E₂`: the same derivation, for `r1` and `'b2`. -/
theorem e₂ :
    HasType ["a"] [("r1", M "a")]
      (.reborrow "b2" "y2" (.var "r1") (.assign (.var "y2") (.lit 2)))
      (.pair (M "a") .unit) :=
  .reborrow (.left .nil) (.var rfl) (by decide)
    (.assign (.left .nil) (.var rfl) .lit) (by decide) (by decide)
```

```
/-- Derivation I: `'a; r : &'a mut Int ⊢ twice : Unit`. -/
theorem twice_typed : HasType ["a"] [("r", M "a")] twice .unit :=
  .letPair (.left .nil) e₁ (by decide) (by decide) (by decide)
    (.letPair (.left (.right .nil)) e₂ (by decide) (by decide) (by decide)
      .unit)
```

Lean checks every step: if a split sent a variable to the wrong side, or a lifetime were not fresh, the term would not type-check and the theorem would not exist.

28.9 Worked derivation II: the aliasing bug

The classic bug: take a reference into a vector, push onto the vector (which may reallocate its buffer), then use the reference.

```
pub fn aliasing_bug() {
    let mut v: Vec<i32> = vec![1, 2, 3];
    let first: &i32 = &v[0];
    v.push(4);
    println!("{first}");
}
```

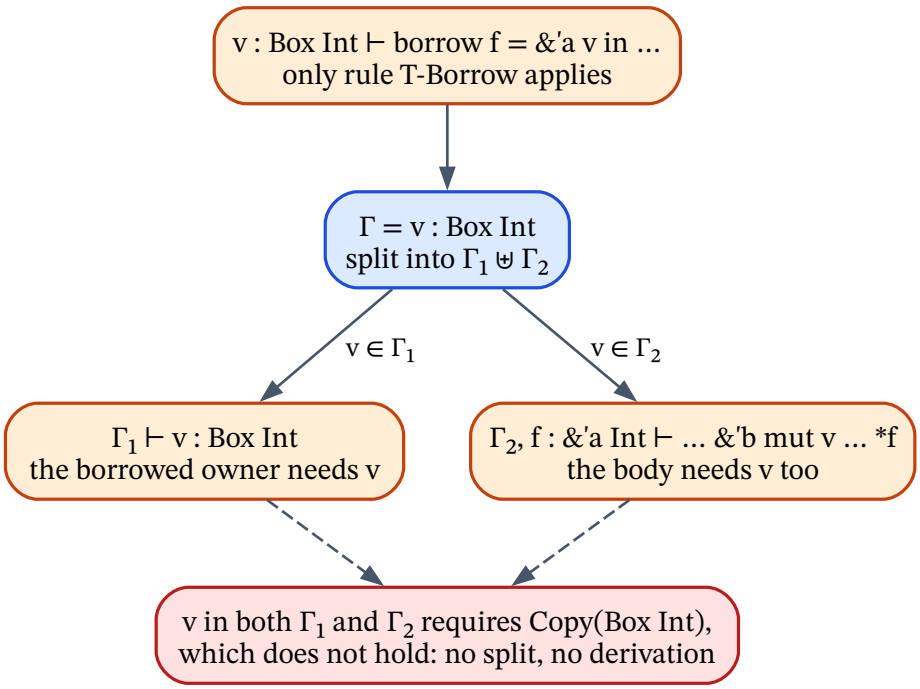
```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> compile_fail/ch28_aliasing_bug.rs:9:5
   |
 8 |     let first: &i32 = &v[0];
   |                       - immutable borrow occurs here
 9 |     v.push(4);
   |     ^^^^^^^^^ mutable borrow occurs here
10 |     println!("{first}");
   |                ----- immutable borrow later used here
```

In the calculus, the vector is an owner `v : Box Int`, the reference `first` is a shared borrow `f` of `v` (of the whole vector rather than of its first element, a simplification), and `v.push(4)` is a write through a mutable borrow of `v`. The use of `first` after the push puts the push inside the shared borrow's body:

```
/// Derivation II. In Rust: `let first = &v[0]; v.push(4); println!("{first}");`
const ALIASING: &str = "
fn aliasing(v: Box Int) {
    borrow f = &'a v in
      let (v2, u) = (borrow m = &'b mut v in *m := 4) in
      *f
}";
```

Why no derivation exists. The term is a shared borrow, and since the rules are syntax-directed, only T-Borrow can conclude it. T-Borrow requires the context $v : \mathit{Box}\ \mathit{Int}$ to be split into Γ_1, for the owner,

and Γ_2, for the body. The owner is the variable v, and the only rule for a variable is T-Var, so v must be in Γ_1. The body contains $\mathit{borrow}\ m = \&^{\beta}\mathit{mut}\ v\ \mathit{in}\ \ldots$, whose owner is v again, so by the same reasoning v must be in the body's context, which is built from Γ_2. The variable would have to be on both sides, and rule S-Both allows that only if $\mathit{Copy}(\mathit{Box}\ \mathit{Int})$, which no rule concludes. There is no split, so there is no derivation. The figure below draws the argument.



Derivation II fails at the very first rule: the context v : Box Int would have to go to both premises of T-Borrow, which requires Copy(Box Int).

The crate reports the failed split, naming the variable, the rule and the two premises that both need it:

```
== aliasing (Δ = {})
error: `v` is used while it is borrowed
  in: borrow f = &'a v in …
  `v` is needed by the borrowed owner and by the body of the borrow,
  but `v: Box Int` is not Copy, so the split Γ = Γ₁ ⊎ Γ₂
  of T-Borrow can give it to only one of them
```

rustc's message speaks of *borrows*: "cannot borrow `v` as mutable because it is also borrowed as immutable". The calculus's reason is a failed *context split*. They agree because rule T-Borrow consumes the owner for the whole duration of the borrow. You can watch the same agreement inside rustc by transcribing the term with the `lend` functions from derivation I:

```
pub fn aliasing_lend(v: Box<Vec<i32>>) -> i32 {
    let (_v, first) = lend(v, |f| {
        let (_v2, ()) = lend_mut(v, |m| m.push(4));
        f[0]
    });
    first
}
```

```
error[E0382]: use of moved value: `v`
  --> compile_fail/ch28_aliasing_lend.rs:20:31
   |
19 | pub fn aliasing_lend(v: Box<Vec<i32>>) -> i32 {
   |                      - move occurs because `v` has type `Box<Vec<i32>>`, which does not
implement the `Copy` trait
20 |     let (_v, first) = lend(v, |f| {
   |                            -  ^^^ value used here after move
   |                            |
   |                            value moved here
21 |         let (_v2, ()) = lend_mut(v, |m| m.push(4));
   |                                  - use occurs due to use in closure
   |
```

Now rustc reports E0382, use of a moved value: the owner was moved into `lend`, and the closure (the body) needs it again. This is the affine phrasing of the same bug, and it is exactly the calculus's phrasing.

What non-lexical lifetimes changed. Delete the `println!` from the Rust program and rustc accepts it, because `first` is no longer used after the push: its borrow is dead. Our calculus would still reject the corresponding term, because a lexical borrow lasts for its whole body. To make the program typable you narrow the borrow by hand, reading before pushing, and both the calculus and Rust accept the result:

```
/// The aliasing program repaired: the shared borrow ends (and gives `v` back)
/// before the mutable borrow begins.
fn read_then_push(v: Box<Vec<i32>>) -> (Box<Vec<i32>>, i32) {
    let (v, first) = lend(v, |f| f[0]);
    let (v, ()) = lend_mut(v, |m| m.push(4));
    (v, first)
}
```

Closing this gap is why the real borrow checker is a dataflow analysis rather than a type system (Chapter 29).

Derivation II in Lean: proving that no derivation exists

Lean lets us turn "there is no derivation" into a theorem. The statement says: for every lifetime context `Δ` and every type `τ`, there is no proof of `HasType Δ [("v", .box .int)] aliasing τ`.

```
/-- The classic aliasing bug, with `v : Box Int` standing for the vector:

    borrow f = &'a v in
      let (v2, u) = borrow m = &'b mut v in *m := 4 in
      *f -/
def aliasing : Term :=
  .borrow "a" "f" (.var "v")
    (.letPair "v2" "u" (.borrowMut "b" "m" (.var "v") (.assign (.var "m") (.lit 4)))
      (.deref (.var "f")))
```

```
/-- No derivation exists, whatever the lifetimes in scope and the result
type. The proof inverts the rules: each `cases` asks "which rule could have
produced this judgment, and with which premises?". -/
theorem aliasing_untypable (Δ : List Lft) (τ : Ty) :
    ¬ HasType Δ [("v", .box .int)] aliasing τ := by
  intro h
  -- Only T-Borrow concludes a shared borrow.
  cases h with
  | borrow hs hv _ hbody _ _ =>
    -- The context `v : Box Int` must be split between `v` and the body.
```

```
    cases hs with
    | both hc _ =>
      -- Give `v` to both sides? Only if `Box Int` were `Copy`.
      exact not_copy_box _ hc
    | right hs =>
      -- Give `v` to the body: the borrowed expression `v` has no context.
      cases hs
      have hmem := hv.var_mem
      simp [dom] at hmem
    | left hs =>
      -- Give `v` to the borrowed expression: the body only has `f`.
      cases hs
      cases hbody with
      | letPair hs₂ hbm _ _ _ _ =>
        cases hbm with
        | borrowMut hs₃ hv₃ _ _ _ _ =>
          -- The inner `&mut v` needs `v`, in a part of a part of `[f]`.
          have hmem :=
            dom_subset hs₂.sublist.1 (dom_subset hs₃.sublist.1 hv₃.var_mem)
          simp [dom] at hmem
```

The proof is the paper argument, one inversion at a time. `intro h` assumes a derivation `h` and aims for a contradiction. `cases h` asks Lean which constructors of `HasType` could have produced `h`. Lean compares the conclusion of every constructor with `HasType Δ [("v", .box .int)] aliasing τ`, discards those whose term does not match (only `.borrow` produces a shared borrow), and gives us the premises of the one that is left: the split `hs`, the derivation `hv` of the owner, and the derivation `hbody` of the body.

`cases hs` does the same for the split of the one-entry context. S-Empty cannot match a non-empty list, so three cases remain, one for each place `v` can go:

- **Both sides** (`both`): the premise `hc` claims that `Box Int` is `Copy`, and `not_copy_box` refutes it.
- **Right** (`right`): the owner's context is empty. (The inner `cases hs` splits the rest of the context, which is empty, so it has a single case.) `hv.var_mem` is a small lemma from `Typing.lean`, the inversion of T-Var: a variable that has a type is in the context. Here it says `"v" ∈ dom []`, and `simp` reduces that to `False`.
- **Left** (`left`): after the same inner `cases hs`, the body's context is `[("f", &'a Int)]`. Two more inversions open the body: it is a `let`-pair (`.letPair`), whose right-hand side is a mutable borrow (`.borrowMut`), whose owner is the variable `v`. Its context `Γ₅` is a part of a part of `[f]` (each split produces sublists of its context, a lemma called `Split.sublist`), so `v` would be in `dom [f]`, and `simp` sees that `"v" ∈ ["f"]` is false.

Every branch ends in a contradiction, so `h` cannot exist. There is also a shorter route, through the metatheorem of the next section: `v` occurs twice in the term, and the theorem says that a non-`Copy` variable occurs at most once in any well-typed term:

```
/-- info: 2 -/
#guard_msgs in
#eval aliasing.occ "v"

/-- The same fact, as a corollary of the usage theorem: `v` occurs twice. -/
theorem aliasing_untypable' (Δ : List Lft) (τ : Ty) :
    ¬ HasType Δ [("v", .box .int)] aliasing τ := fun h =>
  have once : aliasing.occ "v" ≤ 1 :=
```

```
    h.used_at_most_once (by decide) rfl (not_copy_box .int)
  absurd once (by decide)
```

The escape check, in Lean

The usage theorem explains derivation II, but not every rejection is affine. The dangling program of the `ch28_go_wrong` example reduces to its core, $\mathit{borrow}\ y = \&\,^{\alpha} b\ \mathit{in}\ y$, which uses b only once. It is rejected by the escape check alone, and the Lean proof finds exactly that premise:

```
/-- A borrow whose body returns the reference itself:
`borrow y = &'a b in y`. -/
def escape : Term := .borrow "a" "y" (.var "b") (.var "y")

/-- Rejected by the escape check, and by nothing else. -/
theorem escape_untypable (Δ : List Lft) (τ : Ty) :
    ¬ HasType Δ [("b", .box .int)] escape τ := by
  intro h
  cases h with
  | borrow _ _ _ hbody _ hesc =>
    -- The body `y` has the type of `y` in its context: `&'a τ'`.
    cases hbody with
    | var hl =>
      simp [lookup] at hl
      subst hl
      -- So the escape check demands `'a ∉ flv(&'a τ')`, which is false.
      simp [Ty.lfts] at hesc
```

After inverting T-Borrow, the body `y` must be typed by T-Var, so its type is the type of `y` in the body's context, `&'a τ'` for the owner's content type `τ'`. `simp [lookup] at hl` computes the lookup and turns `hl` into the equation `Ty.ref "a" τ' = σ`, and `subst` replaces σ everywhere. The escape check premise `hesc` now reads `"a" ∉ (Ty.ref "a" τ').lfts`, and `simp` unfolds `lfts` and finds `"a"` at the head of the list. The usage theorem has nothing to say here: `b` is used once.

```
/-- The usage theorem cannot explain this rejection: `b` is used once. -/
example : escape.occ "b" = 1 := by decide
```

In real Rust, the same bug (keep a reference to the box, drop the box, use the reference) is E0505:

```
error[E0505]: cannot move out of `b` because it is borrowed
  --> compile_fail/ch28_drop_while_borrowed.rs:9:10
   |
 7 |     let b = Box::new(5);
   |         - binding `b` declared here
 8 |     let r: &i32 = &b;
   |                   -- borrow of `b` occurs here
 9 |     drop(b);
   |          ^ move out of `b` occurs here
10 |     *r
   |     -- borrow later used here
   |
help: consider cloning the value if the performance cost is acceptable
   |
 8 |     let r: &i32 = &b.clone();
   |                     ++++++++
```

28.10 The metatheorem: a mutable reference is used at most once

We close the formal part with a real theorem about all derivations at once, the calculus's version of Chapter 22's usage theorem. If a variable has a type that is not `Copy` (an owner or a mutable reference) then it occurs at most once in any well-typed term. "Occurs" means *free* occurrences, counted by a recursive function on terms that skips the scope of any binder for the same name:

```
/-- The number of free occurrences of `x` in a term. A binder for `x` hides
the occurrences in its scope. -/
def Term.occ (x : Var) : Term → Nat
  | .var y => if x = y then 1 else 0
  | .lit _ | .unit => 0
  | .pair e₁ e₂ | .assign e₁ e₂ => e₁.occ x + e₂.occ x
  | .new e | .drop e | .deref e => e.occ x
  | .letIn y e₁ e₂ | .borrow _ y e₁ e₂ | .borrowMut _ y e₁ e₂
  | .reborrow _ y e₁ e₂ => e₁.occ x + (if x = y then 0 else e₂.occ x)
  | .letPair y z e₁ e₂ => e₁.occ x + (if x = y ∨ x = z then 0 else e₂.occ x)
```

Two lemmas carry the proof. The first is about splits, and it is the formal core of the whole chapter: a non-`Copy` variable goes to exactly one side.

```
/-- The heart of the matter. When `Γ` splits into `Γ₁` and `Γ₂`, a variable
whose type is not `Copy` ends up in exactly one of the two parts. -/
theorem Split.non_copy {Γ Γ₁ Γ₂ : Ctx} (hs : Split Γ Γ₁ Γ₂) (hnd : (dom Γ).Nodup)
    {x : Var} {σ : Ty} (hx : lookup Γ x = some σ) (hσ : ¬ Copy σ) :
    (lookup Γ₁ x = some σ ∧ x ∉ dom Γ₂) ∨ (lookup Γ₂ x = some σ ∧ x ∉ dom Γ₁) := by
  induction hs with
  | nil => simp [lookup] at hx
  | @left b Γ Γ₁ Γ₂ hs ih =>
    obtain ⟨y, τ⟩ := b
    rw [dom_cons, List.nodup_cons] at hnd
    by_cases hxy : x = y
    · -- `x` is the entry that went left; it is not repeated further down.
      subst hxy
      simp [lookup] at hx
      subst hx
      exact Or.inl ⟨by simp [lookup], fun hm => hnd.1 (dom_subset hs.sublist.2 hm)⟩
    · rw [lookup_cons_ne hxy] at hx
      rcases ih hnd.2 hx with ⟨h₁, h₂⟩ | ⟨h₁, h₂⟩
      · exact Or.inl ⟨by rw [lookup_cons_ne hxy]; exact h₁, h₂⟩
      · exact Or.inr ⟨h₁, by simp [hxy, h₂]⟩
  | @right b Γ Γ₁ Γ₂ hs ih =>
    obtain ⟨y, τ⟩ := b
    rw [dom_cons, List.nodup_cons] at hnd
    by_cases hxy : x = y
    · subst hxy
      simp [lookup] at hx
      subst hx
      exact Or.inr ⟨by simp [lookup], fun hm => hnd.1 (dom_subset hs.sublist.1 hm)⟩
    · rw [lookup_cons_ne hxy] at hx
      rcases ih hnd.2 hx with ⟨h₁, h₂⟩ | ⟨h₁, h₂⟩
      · exact Or.inl ⟨h₁, by simp [hxy, h₂]⟩
      · exact Or.inr ⟨by rw [lookup_cons_ne hxy]; exact h₁, h₂⟩
  | @both b Γ Γ₁ Γ₂ hc hs ih =>
    obtain ⟨y, τ⟩ := b
    rw [dom_cons, List.nodup_cons] at hnd
    by_cases hxy : x = y
    · -- The entry shared by both parts is `Copy`, but `x` is not.
      subst hxy
      simp [lookup] at hx
      subst hx
      exact absurd hc hσ
    · rw [lookup_cons_ne hxy] at hx
      rcases ih hnd.2 hx with ⟨h₁, h₂⟩ | ⟨h₁, h₂⟩
```

```
      · exact Or.inl ⟨by rw [lookup_cons_ne hxy]; exact h₁, by simp [hxy, h₂]⟩
      · exact Or.inr ⟨by rw [lookup_cons_ne hxy]; exact h₁, by simp [hxy, h₂]⟩
```

The proof is by induction on the split. In each step case it asks whether the entry just handled is `x` itself (`by_cases hxy : x = y`). If it is, then in the cases `left` and `right` it went to one side, and the other side cannot contain `x` because the context has no repeated variables (`hnd`, the `Nodup` hypothesis) and each side is a sublist of the context. In the case `both` the entry must be `Copy`, contradicting `hσ`. If the entry is some other variable, the induction hypothesis applies to the rest of the context, and `lookup_cons_ne` (looking up `x` past a different variable) moves the conclusion back up.

The second lemma says that a variable absent from the context does not occur in a well-typed term (`HasType.occ_eq_zero` in `Typing.lean`). It is a straightforward induction on the derivation, and we use it without showing it.

Here is the theorem. The first cases are the easy ones:

```
/-- **Usage theorem.** In a well-typed term, a variable whose type is not `Copy`
occurs free at most once. (The context must not repeat a variable; contexts
built by the rules never do.) -/
theorem HasType.used_at_most_once {Δ : List Lft} {Γ : Ctx} {e : Term} {τ : Ty}
    (h : HasType Δ Γ e τ) (hnd : (dom Γ).Nodup)
    {x : Var} {σ : Ty} (hx : lookup Γ x = some σ) (hσ : ¬ Copy σ) :
    e.occ x ≤ 1 := by
  induction h generalizing x with
  | var _ =>
    -- A variable occurs once in itself and zero times in any other variable.
    simp only [Term.occ]
    split <;> omega
  | lit | unit => simp [Term.occ]
  | new _ ih | drop _ ih | deref _ _ ih =>
    -- One premise, same context: the induction hypothesis is the answer.
    exact ih hnd hx
```

The proof is by induction on the derivation `h`, `generalizing x` so that the induction hypothesis can be used for whatever variable a premise needs.

- **T-Var**: the term is a variable `y`, and `occ` is `1` if `x = y` and `0` otherwise. `simp only [Term.occ]` unfolds the definition, `split` considers the two branches of the `if`, and `omega` (the arithmetic decision procedure you met in Chapter 2) checks `1 ≤ 1` and `0 ≤ 1`.
- **T-Int and T-Unit**: there are no variables at all.
- **T-New, T-Drop, T-Deref**: one premise with the same context; the induction hypothesis is exactly the goal.

The two-premise rules without binders, T-Pair and T-Assign, are where the split lemma does its work:

```
| pair hs h₁ h₂ ih₁ ih₂ | assign hs h₁ h₂ ih₁ ih₂ =>
  simp only [Term.occ]
  -- `x` went to exactly one side of the split.
  rcases hs.non_copy hnd hx hσ with ⟨hin, hout⟩ | ⟨hin, hout⟩
  · have := ih₁ (nodup_of_sublist hs.sublist.1 hnd) hin
    have := h₂.occ_eq_zero hout
    omega
  · have := h₁.occ_eq_zero hout
    have := ih₂ (nodup_of_sublist hs.sublist.2 hnd) hin
    omega
```

`rcases` takes apart the disjunction from `Split.non_copy`. If `x` went left, the induction hypothesis for the left premise bounds its occurrences in `e₁` by 1 (the left part has no repeated variables either, being a sublist), and `occ_eq_zero` says it does not occur in `e₂` at all. `omega` adds the two facts. If `x` went right, the roles are swapped.

The rules with a binder (T-Let and the three borrow rules, which all bind a variable in their second premise) need one more observation:

```
| letIn hs h₁ hy h₂ ih₁ ih₂ | borrow hs h₁ hy h₂ _ _ ih₁ ih₂
| borrowMut hs h₁ hy h₂ _ _ ih₁ ih₂ | reborrow hs h₁ hy h₂ _ _ ih₁ ih₂ =>
  simp only [Term.occ]
  rcases hs.non_copy hnd hx hσ with ⟨hin, hout⟩ | ⟨hin, hout⟩
  · -- `x` went to the first premise; the body cannot mention it.
    have := ih₁ (nodup_of_sublist hs.sublist.1 hnd) hin
    split
    · omega
    · rename_i hne
      have := h₂.occ_eq_zero (x := x) (by simp [hne, hout])
      omega
  · -- `x` went to the body, so it differs from the variable bound there.
    have := h₁.occ_eq_zero hout
    have hne := ne_of_mem_dom (mem_dom_of_lookup hin) hy
    have hnd₂ := nodup_of_sublist hs.sublist.2 hnd
    have := ih₂ (by rw [dom_cons, List.nodup_cons]; exact ⟨hy, hnd₂⟩)
      (by rw [lookup_cons_ne hne]; exact hin)
    simp only [hne, ite_false]
    omega
```

If `x` went to the first premise, the body either rebinds `x` (then `occ` counts nothing there, by definition) or does not mention it at all (`occ_eq_zero`, since `x` is neither in `Γ₂` nor the bound variable). If `x` went to the body, then `x` is different from the bound variable `y`, because `y ∉ dom Γ₂` while `x` is in `Γ₂` (`ne_of_mem_dom`). So the body's context `(y, _) :: Γ₂` still has no repeated variables, looking up `x` in it finds the same type, and the induction hypothesis applies. T-LetPair, not shown, is the same argument with two bound variables. The theorem holds for all derivations, whatever the term.

The case this chapter cares about is a corollary:

```
/-- The case the chapter cares about: a mutable reference is used at most
once. -/
theorem HasType.ref_mut_used_at_most_once {Δ : List Lft} {Γ : Ctx} {e : Term}
    {τ : Ty} (h : HasType Δ Γ e τ) (hnd : (dom Γ).Nodup)
    {r : Var} {κ : Lft} {σ : Ty} (hr : lookup Γ r = some (.refMut κ σ)) :
    e.occ r ≤ 1 :=
  h.used_at_most_once hnd hr (not_copy_refMut κ σ)
```

and the contrast with shared references is two small examples: a shared reference goes to both sides of a pair by S-Both, while a mutable one makes the pair untypable.

```
/-- A shared reference may go to both sides of a split: `(s, s)` is fine. -/
theorem shared_twice :
    HasType ["a"] [("s", .ref "a" .int)] (.pair (.var "s") (.var "s"))
      (.pair (.ref "a" .int) (.ref "a" .int)) :=
  .pair (.both .ref .nil) (.var rfl) (.var rfl)

/-- A mutable reference may not: `(r, r)` has no type at all. -/
theorem mut_twice_untypable (τ : Ty) :
    ¬ HasType ["a"] [("r", M "a")] (.pair (.var "r") (.var "r")) τ := fun h =>
  absurd (h.ref_mut_used_at_most_once (r := "r") (by decide) rfl) (by decide)
```

Note what the theorem does *not* say. It says nothing about lifetimes: the escape check, freshness and the scoping of borrows play no role in its proof. The usage theorem is the affine half of the calculus, proved; the region half needs the semantics, which is why safety needs both.

28.11 Putting it together: a checker for the calculus

The `borrowck` crate is a small pipeline with one module per stage: `parse` reads a program in the concrete syntax into the types of `syntax`, `check` builds a derivation or explains which premise fails, and `eval` runs the program on the abstract machine. The main example prints both worked derivations:

```
fn main() {
    for src in [TWICE, ALIASING] {
        let program = parse_program(src).expect("the sample programs parse");
        let delta: Vec<String> =
            program.lifetimes.iter().map(|k| format!("'{k}")).collect();
        println!("== {} (Δ = {{{}}})", program.name, delta.join(", "));
        match check_program(&program) {
            Ok(derivation) => println!("{derivation}"),
            Err(error) => println!("{error}"),
        }
        println!();
    }
}
```

To see the calculus as a whole, here is a suite of programs, each paired with the rustc error code that the corresponding real Rust program produces (every one of those was checked against the compiler) (`crates/borrowck/examples/ch28_suite.rs`):

```
/// Each program is preceded by rustc's verdict on the corresponding Rust
/// program: `ok`, or the error code it reports.
const SUITE: &str = "
// rustc: ok
fn twice<'a>(r: &'a mut Int) {
    let (r1, u1) = (borrow y1 = &'b1 mut *r in *y1 := 1) in
    let (r2, u2) = (borrow y2 = &'b2 mut *r1 in *y2 := 2) in ()
}
// rustc: E0502
fn aliasing(v: Box Int) {
    borrow f = &'a v in let (v2, u) = (borrow m = &'b mut v in *m := 4) in *f
}
// rustc: ok
fn read_then_push(v: Box Int) {
    let (v, n) = (borrow f = &'a v in *f) in
    let (v, u) = (borrow m = &'b mut v in *m := 4) in n
}
// rustc: E0499
fn two_mut(v: Box Int) {
    borrow m1 = &'a mut v in borrow m2 = &'b mut v in *m1 := 1
}
// rustc: E0505
fn dangling(b: Box Int) {
    let (b, r) = (borrow y = &'a b in y) in let u = drop b in *r
}
// rustc: E0505
fn drop_while_borrowed(b: Box Int) { borrow y = &'a b in let u = drop b in *y }
// rustc: E0382
fn use_after_move(b: Box Int) { let c = b in (b, c) }
// rustc: ok
fn share<'a>(s: &'a Int) { (s, s) }
// rustc: E0507
fn move_out<'a>(s: &'a Box Int) { *s }
// rustc: E0594
```

```
fn write_shared<'a>(s: &'a Int) { *s := 1 }
";
```

```
program              rustc   the calculus
twice                ok      ok: Unit
aliasing             E0502   error: `v` is used while it is borrowed
read_then_push       ok      ok: Int
two_mut              E0499   error: `v` is used while it is borrowed
dangling             E0505   error: borrowed value does not live long enough
drop_while_borrowed  E0505   error: `b` is used while it is borrowed
use_after_move       E0382   error: use of moved value: `b`
share                ok      ok: &'a Int ⊗ &'a Int
move_out             E0507   error: cannot move out of a shared reference
write_shared         E0594   error: cannot assign through a shared reference
```

Read the table row by row and match each verdict to the rule that produced it:

- `twice` and `read_then_push` are the two programs we derived, and `share` copies a shared reference by S-Both.
- `aliasing` (E0502), `two_mut` (E0499), `drop_while_borrowed` (E0505) and `use_after_move` (E0382) are four different rustc errors, but in the calculus they are one error: a non-`Copy` variable needed by both premises of a split. The compiler distinguishes them by what the second use does (borrow, borrow mutably, move, use after a move); the calculus sees a single structural rule missing.
- `dangling` is the escape check, the only region error in the suite.
- `move_out` (E0507) is the $\mathit{Copy}(\tau)$ premise of T-Deref, and `write_shared` (E0594) is the absence of any rule that writes through a shared reference.

The Lean relation `HasType` is the specification: it states what a derivation is, without saying how to find one. The Rust checker is an algorithm that finds derivations by deciding each split from the subterms. We did not prove the algorithm sound with respect to the specification (Chapter 3 shows the technique on a simpler language); the crate's tests and the Lean proofs check the same derivations from both ends.

28.12 Common confusions

"**A reborrow copies the `&mut`.**" It does not, at least not in the type system. The reborrow consumes the reference and hands it back afterwards, with the same lifetime, under a new name. At run time it is a pointer copy (rule E-Reborrow does not touch the store), but statically the original is unusable while the reborrow lives: that is the split in T-Reborrow.

"**Lifetimes are affine resources.**" Nothing in the calculus counts uses of a lifetime. A lifetime is a name for a region; the rules that mention it (freshness, the escape check) are about *scope*, not about *multiplicity*. The shared reference, the paradigm example of a type with a lifetime, is the most freely duplicable value of the calculus.

"**This is how rustc checks borrows.**" No. The calculus is a *specification*, stricter than today's compiler (it rejects programs that NLL accepts). rustc runs a dataflow analysis on MIR, which Chapter 29 builds in miniature.

"**Getting stuck is a panic.**" A panic is defined behavior: the program stops in an orderly way, running destructors. Getting stuck means the semantics has no answer, which corresponds to undefined behavior. The type system rules out the latter and says nothing about the former.

"**The escape check is about returning references from functions.**" That is one instance (E0515). The check applies to every borrow, and it also rejects a reference stored in a variable that outlives the borrow (E0597) and, as the `dangling` program shows, a reference used after the owner is dropped (E0505).

28.13 Summary

- λ_{rgn}^{aff} combines the affine core of Part V (context splitting, `Copy`, drops) with a region system (lexically scoped borrows, fresh lifetimes, the escape check) in thirteen syntax-directed typing rules.
- A borrow lends its owner and gives it back: its type is $\mathit{Box}\,\tau \otimes \sigma$. Because the owner is consumed by the borrow, "no use of the owner while it is borrowed" is just context splitting.
- `Copy` is a predicate that lets a variable go to both sides of a split. Shared references are `Copy`; owners and mutable references are not.
- The operational semantics gives every location a loan state; reading, writing, borrowing and freeing have guards, and a failed guard is a stuck state, the calculus's undefined behavior. Progress and preservation say that well-typed programs never get stuck. A reborrow changes nothing at run time.
- Derivation I: `bump(r); bump(r);` works because each implicit reborrow consumes `r` and returns a reference of the same type. Without the reborrow (a generic parameter), `&mut` is affine and rustc reports E0382.
- Derivation II: the aliasing bug fails at the first rule, because the owner would have to go to both premises of T-Borrow and `Box` is not `Copy`. rustc calls it E0502; the calculus calls it a failed split.
- In Lean, the rules are an inductive relation, derivations are proof terms, impossibility is proved by inversion (`cases`, `nomatch`), and the usage theorem proves that a non-`Copy` variable, in particular a mutable reference, is used at most once. The escape check is the part of the system that the usage theorem cannot explain.

28.14 Exercises

1. Write the term $\mathit{borrow}\ y = \&^{\alpha}\mathit{mut}\ b\ \mathit{in}\ y$ with $b : \mathit{Box}\ \mathit{Int}$ in the crate's syntax and check it. Which premise of T-BorrowMut fails? Write a Rust function with the same bug and compare rustc's error code with the ones in this chapter. *Hint:* the body's type is $\&^{\alpha}\mathit{mut}\ \mathit{Int}$; in Rust, try returning `&mut *b` from a function that owns the box `b`.
2. Write the complete derivation tree of `read_then_push` from the suite, with every split. Then construct it in Lean as a proof of `HasType [] [("v", .box .int)] readThenPush .int` for a suitable definition of `readThenPush`. *Hint:* in the first `let`-pair the variable `v` goes left; in the second, `v`

goes left and `n` goes right, all the way down to the final `n`. Build the derivation from the leaves up, as `twice_typed` does.

3. Add a rule T-DerefMut that reads a `Copy` value through a mutable reference, to the crate's checker and to Lean's `HasType`. Does the usage theorem still hold? Repair its proof. *Hint:* the new constructor has one premise and the same context, so the new case of `used_at_most_once` is the same as the case for `deref`. Note that the reference is consumed by the read, so reading and then writing needs a reborrow.
4. Prove in Lean that `borrow y = &'a mut b in (y, y)` has no type in the context `b : Box Int`. *Hint:* invert T-BorrowMut with `cases`, then apply `HasType.ref_mut_used_at_most_once` to the derivation of the body, whose context starts with `("y", .refMut "a" _)`. You need to show that this context has no repeated variables: `y` is not in the rest of it (a premise of the rule), and the rest is a sublist of `[b]`.
5. Find a program that the checker rejects but that the interpreter runs to a value without getting stuck. Does that mean the checker is too strict? What would go wrong if you extended the program? *Hint:* try `use_after_move` from the suite, made closed with `new`. Look at the value it produces: how many owners does location ℓ_0 have, and what would two `drop`s do?
6. Design a *shared reborrow*, `borrow y = &'a *e1 in e2`, which takes a mutable reference $e_1 : \&^{\kappa}\, \mathit{mut}\, \tau$ and gives the body a shared reference $y : \&^{\alpha}\, \tau$ (Rust's `&*r`). Write the typing rule and the reduction rule. What loan state does the location need while the shared reborrow runs, and what must happen when it ends? *Hint:* during the body the location must be readable but not writable, so its state must change from *mut* to *shared* and back. The frame must remember which state to restore.

CHAPTER 29

How rustc Really Checks Borrows

Chapter 28 ended with a type system for borrowing: a dozen rules, derivations that Lean checks, and a proof that the classic aliasing bug has no derivation at all. It also ended with a confession. Its calculus rejects this program, and the Rust compiler accepts it:

```
/// The borrow `first` is last used before the push: NLL accepts this, a
/// lexical borrow checker (Rust before 2018) rejected it.
fn first_then_push() -> usize {
    let mut readings = vec![10, 20, 30];
    let first = &readings[0];
    println!("first = {first}");
    readings.push(40);
    readings.len()
}
```

The reference `first` is used for the last time on the `println!` line, so by the time `readings.push(40)` runs, nobody can observe the old buffer through `first` any more. A borrow that is lexically scoped, as in Chapter 28's calculus and in Rust before 2018, lasts until the closing brace and makes the `push` an error. Today's compiler sees that the borrow is *dead* after its last use.

To see that, the compiler must follow the program as a graph: which statements can run after which, which values may still be used later. This chapter opens the hood. We follow a function from source text to MIR, the control-flow graph the borrow checker works on; we build the dataflow analyses that run on it in a Rust crate that reproduces rustc's error messages; and we prove in Lean the facts about fixpoints that make those analyses trustworthy. On the way we meet two-phase borrows, a safe program that the stable borrow checker still rejects, and Polonius, the reformulation designed to accept it.

In this chapter you will learn:

- why rustc's borrow checker is a static analysis rather than a type system, and what each kind of checker is good at;
- what MIR is: basic blocks, places, `StorageLive` and `StorageDead`, drops, and how to look at the real thing;
- how dataflow analysis works (lattices, transfer functions, the worklist algorithm) and how it finds moved values (E0382) and live variables;
- how non-lexical lifetimes compute regions as sets of program points, and how loans and their conflicts produce E0499, E0502 and E0506;

- what two-phase borrows are, why `v.push(v.len())` compiles and `v.push(v.pop().unwrap())` does not;
- why NLL rejects "problem case #3", and how Polonius' formulation in terms of origins and loans accepts it;
- how to prove in Lean that a dataflow solver returns a fixpoint, the least one, and always terminates on a finite lattice.

29.1 A type system or an analysis?

Chapter 28's calculus is a type system in the sense of Chapter 3. There is a judgment, there are inference rules, and a program is accepted when a derivation exists. The rules are *syntax-directed*: each one looks at one form of term, and the derivation has the shape of the program's syntax tree. A borrow is a construct with a body, $\mathit{borrow}\ y = \&\,{}^{\alpha}x\ \mathit{in}\ e$, and its lifetime is that body. That design has real virtues. The derivation is a certificate that anybody can check, the rules compose (if the parts are well typed, so is the whole), and the metatheory is a matter of induction on derivations, which is what made the Lean proofs of Chapter 28 possible.

rustc's borrow checker works differently. It does not look for a derivation. It turns the function body into a control-flow graph, computes *facts* at every point of the graph ("`job` may have been moved", "`first` will be used again", "the loan of `readings` is still in force"), and reports an error wherever some point violates a rule. That is a **static analysis**, and in particular a *dataflow* analysis: facts flow along the edges of the graph until nothing changes.

	Type system (Chapter 28)	Analysis (rustc)
Works on	the syntax tree	the control-flow graph
Produces	a derivation	facts at every program point
A borrow lasts	its lexical scope	while a reference made from it may be used
Rejects when	no derivation exists	some path reaches a conflict

Both are *conservative*: they reject some programs that would never misbehave, because deciding exactly which programs misbehave is impossible in general (it would decide the halting problem). The analysis is conservative in a specific, well-defined way: it considers every path through the graph as possible, without evaluating conditions. rustc says so itself, in a note it attaches to one of its errors:

```
fn calibrate(warm: bool) -> i32 {
    let offset: i32;
    if warm {
        offset = 2;
    }
    offset
}
```

```
error[E0381]: used binding `offset` is possibly-uninitialized
  --> compile_fail/ch29_maybe_uninit.rs:10:5
```

```
   |
 6 |     let offset: i32;
   |         ------ binding declared here but left uninitialized
 7 |     if warm {
   |        ---- if this `if` condition is `false`, `offset` is not initialized
 8 |         offset = 2;
 9 |     }
   |      - an `else` arm might be missing here, initializing `offset`
10 |     offset
   |     ^^^^^^ `offset` used here but it is possibly-uninitialized
   |
   = note: when checking initialization, the compiler describes possible control-flow paths
without evaluating whether branch conditions can actually have the values shown
```

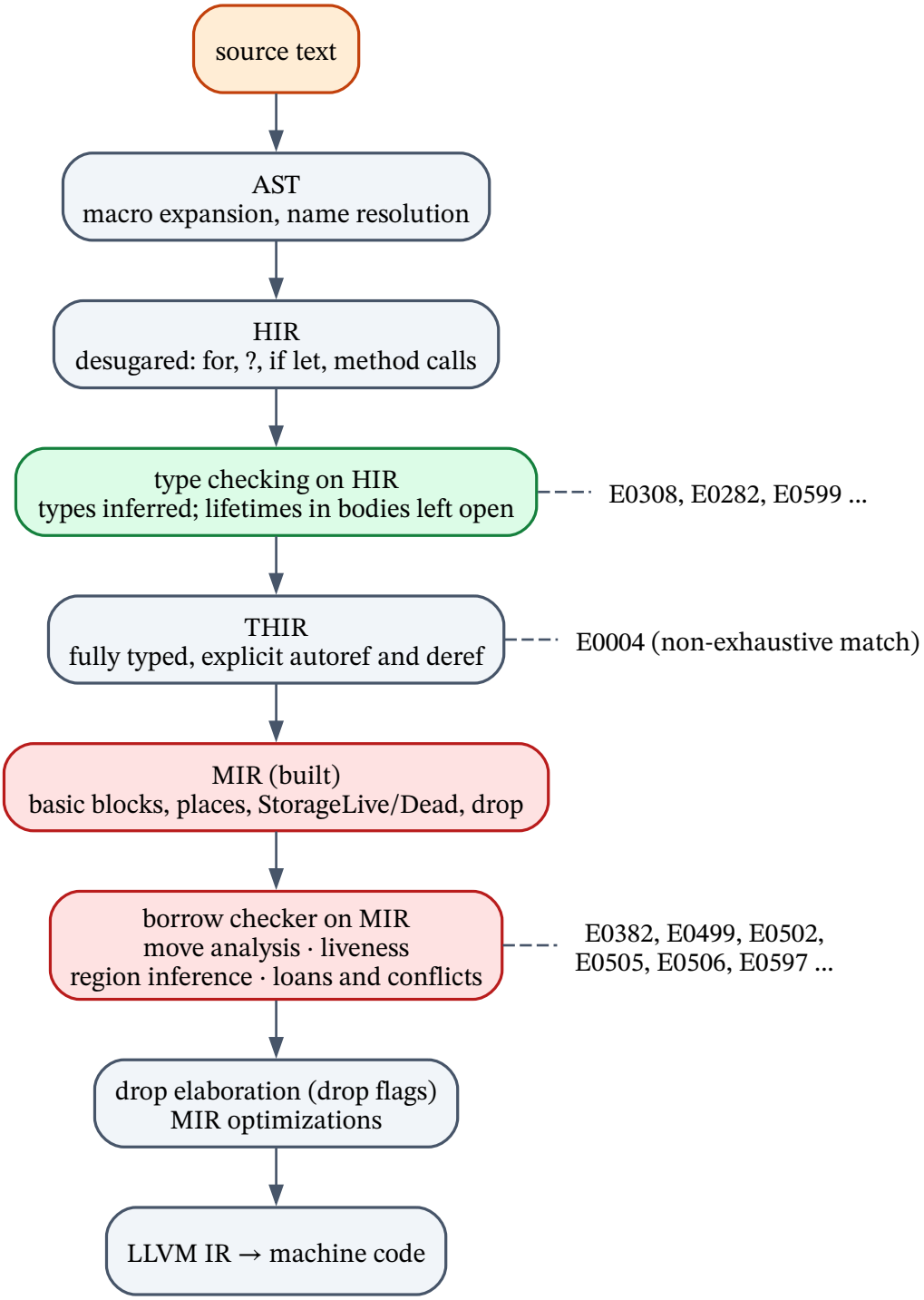
Read the note at the end. The compiler is not claiming that `warm` can be false at run time; it is telling you that its reasoning is about *paths*.

What the analysis buys is *flow sensitivity*: the facts can differ from one point to the next and from one branch to the other, which is exactly what `first_then_push` needs. What it gives up is the certificate: there is no derivation to print, and an error message has to reconstruct a story ("borrow later used here") from a table of facts. Chapter 33 will build on this trade: the theories of Parts V and VI describe *what* the borrow checker enforces; *how* it enforces it is the subject of this chapter.

> **NOTE**
>
> Everything this chapter says about rustc describes the pinned compiler, Rust 1.99, and its documented design. The internals of rustc are not a stable interface: names, passes and dump formats change between releases. The ideas (MIR, dataflow, regions as sets of points, loans) have been stable for years.

29.2 From source text to MIR

A Rust function goes through several representations before machine code comes out, and different errors come from different stages:



The stages of rustc that matter for borrow checking, with the errors each stage reports. The borrow checker runs on MIR, after type checking.

The **AST** is the syntax tree of Chapter 3, after macro expansion and name resolution. The **HIR** (high-level intermediate representation) is the AST with some sugar removed: `for` loops become `loop` plus `match`, `?` and `if let` become `match`. **Type checking** runs on the HIR: it infers types (Chapter 13), resolves method calls, and inserts the implicit operations such as auto-referencing of method receivers. E0308 and E0282 come from here.

Type checking does *not* decide the lifetimes inside a function body. It needs lifetimes to check signatures and trait bounds, but the lifetime of each borrow expression inside the body is left open: the type checker records types with the regions *erased*. The **THIR** (typed HIR) is the fully typed, fully explicit version of the body, every autoref and deref spelled out; pattern exhaustiveness (E0004) is checked on it, and it is the input to MIR building.

MIR (mid-level intermediate representation) is a control-flow graph, and the borrow checker runs on it. It gives every region in the body a fresh inference variable (rustc calls this step *renumbering*), type-checks the MIR once more to collect constraints between those variables, computes which variables are live and which places are initialized, solves the region constraints, and checks every access against the loans in force. After borrow checking, *drop elaboration* decides how to run the destructors, and the MIR is optimized and translated to LLVM IR.

What MIR looks like

Here is a function that moves a `String` on one path only, from `code/rust/examples/ch29_nll_accepts.rs`:

```
/// `job` is moved on one path only, so at the end of the function it is
/// "maybe initialized": the compiler drops it under a run-time drop flag.
fn maybe_archive(urgent: bool) {
    let job = String::from("backup");
    if urgent {
        archive(job);
    }
}
```

And here is its MIR, exactly as the borrow checker sees it. A nightly compiler can dump MIR at any stage with the unstable flag `-Zdump-mir`; this listing was produced by `rustc 1.101.0-nightly (2026-10-08)` with

```
rustc +nightly --edition 2024 --emit=metadata \
  -Zdump-mir=maybe_archive -Zdump-mir-dir=mir_dump examples/ch29_nll_accepts.rs
```

taking the file `mir_dump/ch29_nll_accepts.maybe_archive.-------.nll.0.mir` and removing the source-position comments at the end of each line:

```
fn maybe_archive(_1: bool) -> () {
    debug urgent => _1;
    let mut _0: ();
    let _2: std::string::String;
    let mut _3: bool;
    let _4: usize;
    let mut _5: std::string::String;
    scope 1 {
        debug job => _2;
    }

    bb0: {
        StorageLive(_2);
        _2 = <String as From<&str>>::from(const "backup") -> [return: bb1, unwind: bb9];
    }

    bb1: {
        FakeRead(ForLet(None), _2);
        StorageLive(_3);
        _3 = copy _1;
```

```
        switchInt(move _3) -> [0: bb4, otherwise: bb2];
    }

    bb2: {
        StorageLive(_4);
        StorageLive(_5);
        _5 = move _2;
        _4 = archive(move _5) -> [return: bb3, unwind: bb7];
    }

    bb3: {
        StorageDead(_5);
        StorageDead(_4);
        _0 = const ();
        goto -> bb5;
    }

    bb4: {
        _0 = const ();
        goto -> bb5;
    }

    bb5: {
        StorageDead(_3);
        drop(_2) -> [return: bb6, unwind: bb9];
    }

    bb6: {
        StorageDead(_2);
        return;
    }

    bb7 (cleanup): {
        drop(_5) -> [return: bb8, unwind terminate(cleanup)];
    }

    bb8 (cleanup): {
        drop(_2) -> [return: bb9, unwind terminate(cleanup)];
    }

    bb9 (cleanup): {
        resume;
    }
}
```

Let us read it.

- **Locals.** `_0` is the *return place*, where the function's result is written. `_1` to `_n` are the arguments, here `_1` for `urgent`. The rest are user variables and compiler temporaries: `_2` is `job` (the `debug` lines map names to locals), `_3` holds the condition, `_4` the discarded result of `archive`, `_5` the argument passed to it.
- **Basic blocks.** `bb0` to `bb9` are straight-line sequences of *statements* ending in one *terminator*, the only place where control can branch. `switchInt(move _3) -> [0: bb4, otherwise: bb2]` is the `if`; a call is a terminator too, because the callee may unwind.
- **Places.** Assignments, moves and borrows act on *places*: a local followed by *projections*, such as `(*_1)` (follow a reference) or `(_3 as Some).0` (a field of an enum variant). This is where Rust tracks moves per place rather than per variable, as Chapter 22 anticipated.
- **Operands.** `copy _1` reads a `Copy` value, `move _2` moves a value out (after it, `_2` is uninitialized), `const "backup"` is a constant.

- **Storage.** `StorageLive(_2)` and `StorageDead(_2)` mark where the local's stack slot comes into existence and where it is freed: the lexical scope of the variable, from `let` to the closing brace.
- **Drops.** `drop(_2) -> bb6` is a terminator that runs the destructor of `_2` *if* the value is still there. At this stage the compiler does not yet know whether it is, and that is the question drop elaboration answers later.
- **Unwinding.** The `unwind` targets and the `cleanup` blocks are the paths taken if a call panics. `FakeRead` statements are markers for the borrow checker that do nothing at run time.

You do not need a nightly compiler to see MIR. The stable flag `--emit=mir` writes the MIR *after* borrow checking and optimization. Running `rustc --edition 2024 --emit=mir -o accepts.mir examples/ch29_nll_accepts.rs` with the pinned 1.99 and keeping the same function gives:

```
fn maybe_archive(_1: bool) -> () {
    debug urgent => _1;
    let mut _0: ();
    let _2: std::string::String;
    let _3: usize;
    let mut _4: std::string::String;
    let mut _5: bool;
    scope 1 {
        debug job => _2;
    }

    bb0: {
        _5 = const false;
        _5 = const true;
        _2 = <String as From<&str>>::from(const "backup") -> [return: bb1, unwind continue];
    }

    bb1: {
        switchInt(copy _1) -> [0: bb3, otherwise: bb2];
    }

    bb2: {
        _5 = const false;
        _4 = move _2;
        _3 = archive(move _4) -> [return: bb3, unwind continue];
    }

    bb3: {
        switchInt(copy _5) -> [0: bb4, otherwise: bb5];
    }

    bb4: {
        _5 = const false;
        return;
    }

    bb5: {
        drop(_2) -> [return: bb4, unwind continue];
    }
}
```

Two things changed. The storage markers and cleanup blocks are gone, and a new local `_5: bool` has appeared: a **drop flag**, the hidden boolean of Chapter 24. It is set to `true` in `bb0`, set back to `false` in `bb2` just before `job` is moved into `archive`, and tested in `bb3`, which runs `drop(_2)` (in `bb5`) only if the flag is still `true`. By the end of the section on move analysis you will be able to compute that this flag is needed.

The miniature MIR

The crate `mini-nll` (`code/rust/crates/mini-nll`) has a miniature MIR with the same shapes. Its locals, blocks, program points and region variables are newtypes around indices:

```
/// A local: `_0` is the return place, `_1 ..= _n` are the arguments, and the
/// rest are user variables and compiler temporaries.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Local(pub usize);

/// A basic block, printed `bb3`.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct BlockId(pub usize);

/// A program point: statement `index` of `block`. The terminator of a block
/// with `n` statements is at index `n`. Printed `bb3[1]`, as rustc does.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Location {
    pub block: BlockId,
    pub index: usize,
}

/// A region (lifetime) variable, printed `'?4` like rustc's inference
/// variables, or by its name (`'r`) if it is a named lifetime parameter.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct RegionVid(pub usize);
```

A place is a local and a list of projections:

```
/// One step of a path into memory.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum ProjectionElem {
    /// `*p`: follow a reference.
    Deref,
    /// `p.0`, `p.1`: a field of a struct, a tuple or an enum variant.
    Field(usize),
    /// `(p as Some)`: view an enum value as one of its variants.
    Downcast(&'static str),
}

/// A place: a local followed by projections, such as `_1`, `(*_1)` or
/// `(_3 as Some).0`. Moves, borrows and assignments all act on places.
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct Place {
    pub local: Local,
    pub projection: Vec<ProjectionElem>,
}
```

Types are reduced to what the borrow checker needs: which regions they mention, whether they are `Copy`, whether they have a destructor. A `String` or a `Vec<i32>` is `Owned`; a reference carries its region; `Option<&'a mut String>` is an `Adt` with an argument:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Mutability {
    Not,
    Mut,
}

/// Just enough of a type to know the regions it mentions, whether it is
/// `Copy`, and whether it has a destructor.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    /// `i32`, `usize`, `bool`, `()`: `Copy`, no destructor.
    Scalar(&'static str),
    /// `String`, `Vec<i32>`, `HashMap<u32, String>`: owns memory, is not
    /// `Copy`, has a destructor, and mentions no regions.
```

```
    Owned(&'static str),
    /// `&'a T` and `&'a mut T`.
    Ref(RegionVid, Mutability, Box<Ty>),
    /// A generic enum or struct applied to arguments: `Option<&'a mut String>`.
    Adt(&'static str, Vec<Ty>),
}
```

Statements and terminators are those of the real dump, minus what this chapter does not use. A borrow `&'k p` carries its own region variable `'k`, and a call carries one piece of its signature, `result_borrows_from`, saying that the result borrows from an argument, as in `fn index<'a>(&'a Vec<T>, usize) -> &'a T`:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum BorrowKind {
    /// `&p`
    Shared,
    /// `&mut p`
    Mut,
    /// A `&mut p` inserted by the compiler (method receivers, implicit
    /// reborrows of arguments, overloaded compound assignment). It is only
    /// *reserved* when created and *activated* at its first use.
    TwoPhaseMut,
}

/// An operand: how a value is read.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Operand {
    /// Read a `Copy` value; the place stays initialized.
    Copy(Place),
    /// Move the value out; the place becomes uninitialized.
    Move(Place),
    /// A constant such as `const 40_i32`.
    Const(&'static str),
}

/// The right-hand side of an assignment.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Rvalue {
    Use(Operand),
    /// `&'k p` or `&'k mut p`. Every borrow expression creates a *loan*, and
    /// `'k` is the loan's region.
    Ref(RegionVid, BorrowKind, Place),
    BinaryOp(&'static str, Operand, Operand),
    /// `discriminant(p)`: which variant an enum value is (used by `match`).
    Discriminant(Place),
}
```

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum StatementKind {
    Assign(Place, Rvalue),
    /// The local's storage is allocated: it is now in scope, uninitialized.
    StorageLive(Local),
    /// The local's storage is freed: it goes out of scope.
    StorageDead(Local),
    /// `let x = ...;` marks the binding as read (rustc's `FakeRead(ForLet)`).
    FakeRead(Local),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TerminatorKind {
    Goto(BlockId),
    /// Jump on the value of an integer: `[0: bb4, otherwise: bb2]`.
    SwitchInt {
        discr: Operand,
        targets: Vec<(&'static str, BlockId)>,
    },
```

```
    /// `dest = func(args) -> target`. If `result_borrows_from` is
    /// `Some(i)`, the signature ties the regions of the result to the regions
    /// of argument `i`, as in `fn index<'a>(&'a Vec<T>, usize) -> &'a T`.
    Call {
        func: &'static str,
        args: Vec<Operand>,
        dest: Place,
        target: BlockId,
        result_borrows_from: Option<usize>,
    },
    /// Run the destructor of `place` (if it is still initialized).
    Drop {
        place: Place,
        target: BlockId,
    },
    Return,
}
```

The bodies in `src/programs.rs` are translated from real functions by hand, following the MIR that rustc builds for them. A small builder keeps the translation readable, and attaches spans to statements *by content*, so that error messages can point into the real source file:

```
/// ```text
/// fn maybe_archive(urgent: bool) {
///     let job = String::from("backup");
///     if urgent {
///         archive(job);
///     }
/// }
/// ```
pub fn maybe_archive() -> Body {
    let mut b = Builder::new("maybe_archive", ACCEPTS_FILE, ACCEPTS);
    b.ret(scalar("()"));
    let urgent = b.arg(
        "urgent",
        scalar("bool"),
        b.span("fn maybe_archive", "urgent"),
    );
    let job = b.var("job", owned("String"), b.span("let job", "job"));
    let cond = b.temp(scalar("bool"));
    let archived = b.temp(scalar("usize"));
    let arg = b.temp(owned("String"));
    let [bb0, bb1, bb2, bb3, bb4, bb5, bb6] = [(); 7].map(|_| b.block());

    b.storage_live(bb0, job);
    let from = b.span("let job", "String::from(\"backup\")");
    let callee = b.span("let job", "String::from");
    b.call(bb0, job, "String::from", vec![cst("\"backup\"")], bb1)
        .spans(from, callee);

    b.fake_read(bb1, job);
    b.storage_live(bb1, cond);
    b.assign(bb1, cond, Rvalue::Use(copy(urgent)), None);
    b.switch(bb1, mv(cond), vec![("0", bb4), ("otherwise", bb2)]);

    b.storage_live(bb2, archived);
    b.storage_live(bb2, arg);
    let moved = b.span("archive(job)", "job");
    b.assign(bb2, arg, Rvalue::Use(mv(job)), Some(moved));
    let call = b.span("archive(job)", "archive(job)");
    let callee = b.span("archive(job)", "archive");
    b.call(bb2, archived, "archive", vec![mv(arg)], bb3)
        .spans(call, callee);

    b.storage_dead(bb3, arg);
    b.storage_dead(bb3, archived);
    b.assign(bb3, Local(0), Rvalue::Use(cst("()")), None);
    b.goto(bb3, bb5);
```

```
    b.assign(bb4, Local(0), Rvalue::Use(cst("()")), None);
    b.goto(bb4, bb5);

    b.storage_dead(bb5, cond);
    b.drop(bb5, job, bb6);

    let end = b.closing_brace();
    b.storage_dead_at(bb6, job, end);
    b.ret_term(bb6);
    b.finish()
}
```

Printed in rustc's format by `crates/mini-nll/examples/ch29_mir.rs`, the result is the real dump without the unwinding machinery:

```
fn maybe_archive(_1: bool) -> () {
    debug urgent => _1;
    debug job => _2;
    let _0: ();
    let _2: String;
    let _3: bool;
    let _4: usize;
    let _5: String;

    bb0: {
        StorageLive(_2);
        _2 = String::from(const "backup") -> bb1;
    }

    bb1: {
        FakeRead(ForLet, _2);
        StorageLive(_3);
        _3 = copy _1;
        switchInt(move _3) -> [0: bb4, otherwise: bb2];
    }

    bb2: {
        StorageLive(_4);
        StorageLive(_5);
        _5 = move _2;
        _4 = archive(move _5) -> bb3;
    }

    bb3: {
        StorageDead(_5);
        StorageDead(_4);
        _0 = const ();
        goto -> bb5;
    }

    bb4: {
        _0 = const ();
        goto -> bb5;
    }

    bb5: {
        StorageDead(_3);
        drop(_2) -> bb6;
    }

    bb6: {
        StorageDead(_2);
        return;
    }
}
edges: bb0->bb1 bb1->bb4 bb1->bb2 bb2->bb3 bb3->bb5 bb4->bb5 bb5->bb6
```

The crate has nine such programs: the four functions of `examples/ch29_nll_accepts.rs` that rustc accepts and five of this chapter's compile-fail programs. Every analysis below runs on all of them.

29.3 Dataflow analysis

A dataflow analysis attaches a *fact* to every program point. For the move check, the fact at a point is "the set of places that may be uninitialized here"; for liveness, "the set of locals whose value may still be used". Three ingredients define an analysis.

A lattice of facts. The facts must be ordered by "knows less, knows more", with a least element $\bot$ ("nothing yet") and a *join* $x \sqcup y$, the least fact that is above both (Chapter 5 met joins in Heyting algebras). For sets of places the order is $\subseteq$, the join is $\cup$ and $\bot$ is $\varnothing$. Joining is what happens where control-flow paths merge: a place that *may* be uninitialized on one incoming path may be uninitialized after the merge.

A transfer function f_n for every node n of the graph, saying how executing n changes the facts. Most transfer functions in this chapter have the *gen/kill* form $f_n(x) = gen_n \cup (x \setminus kill_n)$: the statement adds some facts and removes others. A move out of `_2` *gens* "`_2` may be uninitialized"; an assignment to `_2` *kills* it.

A direction. In a *forward* analysis facts flow along the edges (initialization: what happened before determines what holds now). In a *backward* analysis they flow against them (liveness: what happens later determines what is live now).

The facts we want satisfy, at every node n of a forward analysis, the equation

$$in(n) = entry(n) \sqcup \bigsqcup_{m \in \mathrm{pred}(n)} f_m(in(m))$$

where $entry(n)$ holds the boundary facts (at the start of the function: "every local except the arguments is uninitialized") and the big join runs over the predecessors of n. A backward analysis is the same with successors in place of predecessors. In a graph with loops the equations are recursive, so we cannot just compute them in order. We solve them by *iteration*: start with $\bot$ everywhere, apply the equations, and repeat until nothing changes. If the transfer functions are **monotone** (more facts in, more facts out) and the lattice has no infinite ascending chains, the iteration stops, and what it stops at is the *least* solution, the most precise facts that satisfy all the equations. We will prove all of this in Lean shortly.

A framework in Rust

rustc has a generic dataflow framework (`rustc_mir_dataflow`) in which each analysis supplies its lattice and transfer functions. `mini-nll` has a miniature one, in `src/dataflow.rs` :

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Direction {
    /// Facts flow along the edges of the CFG (initialization, loans).
    Forward,
    /// Facts flow against the edges (liveness: "used later").
    Backward,
}
```

```
/// A dataflow analysis. `Domain` must be a join-semilattice with `bottom` as
/// its least element, and the two `apply_*` functions must be monotone; then
/// the worklist algorithm terminates (if the lattice has finite height) and
/// computes the least solution of the dataflow equations.
pub trait Analysis {
    type Domain: Clone + PartialEq + Debug;
    const DIRECTION: Direction;

    /// The least element: "no facts yet".
    fn bottom(&self, body: &Body) -> Self::Domain;

    /// The facts at the boundary: on entry to `bb0` for a forward analysis,
    /// at every `return` for a backward one.
    fn boundary(&self, body: &Body) -> Self::Domain;

    /// Join `other` into `state`; return `true` if `state` grew.
    fn join(&self, state: &mut Self::Domain, other: &Self::Domain) -> bool;

    /// The effect of a statement (applied "backwards" for a backward analysis).
    fn apply_statement(
        &self,
        state: &mut Self::Domain,
        stmt: &Statement,
        loc: Location,
    );

    /// The effect of a terminator.
    fn apply_terminator(
        &self,
        state: &mut Self::Domain,
        term: &Terminator,
        loc: Location,
    );
}
```

The solver keeps one state per basic block (the facts on entry, in the direction of the analysis) and a *worklist* of blocks to process. Processing a block pushes its state through the block's statements and joins the result into the next blocks; any block whose state grew goes back on the worklist:

```
/// Iterate to the fixpoint with a worklist of basic blocks.
pub fn solve<A: Analysis>(body: &Body, analysis: A) -> Results<A> {
    let n = body.blocks.len();
    let preds = body.predecessors();
    let mut states = vec![analysis.bottom(body); n];
    let mut worklist: VecDeque<usize> = (0..n).collect();
    match A::DIRECTION {
        Direction::Forward => states[0] = analysis.boundary(body),
        Direction::Backward => {
            for bb in body.block_ids() {
                if body.successors(bb).is_empty() {
                    states[bb.0] = analysis.boundary(body);
                }
            }
            worklist = (0..n).rev().collect();
        }
    }
    let mut queued = vec![true; n];
    let mut block_visits = 0;

    while let Some(bb) = worklist.pop_front() {
        queued[bb] = false;
        block_visits += 1;
        // Push the state through the block...
        let mut state = states[bb].clone();
        apply_block(body, &analysis, bb, &mut state, |_, _| {});
        // ...and join it into the neighbors that come next in our direction.
        let next: Vec<usize> = match A::DIRECTION {
```

```
            Direction::Forward => body.successors(BlockId(bb)),
            Direction::Backward => preds[bb].clone(),
        }
        .into_iter()
        .map(|b| b.0)
        .collect();
        for succ in next {
            if analysis.join(&mut states[succ], &state) && !queued[succ] {
                queued[succ] = true;
                worklist.push_back(succ);
            }
        }
    }
    Results {
        analysis,
        block_states: states,
        block_visits,
    }
}
```

The `join` method returns `true` exactly when the state grew, so the worklist only revisits blocks that can learn something new. With monotone transfer functions and a finite lattice, a state can only grow a finite number of times, so the loop ends.

29.4 Move analysis

The first analysis rustc runs answers the question behind E0382: at this point, may this place have been moved out of (or never initialized)? rustc tracks *move paths*, places without dereferences (`_2`, `(_3 as Some).0`), since what lies behind a reference is not owned by the function. The analysis is `MaybeUninit`, in `src/moves.rs`:

```
/// Forward "may" analysis: a move path is in the set at a point if on *some*
/// path from the start of the function to that point it was never
/// initialized, or was moved out of, or its storage died.
pub struct MaybeUninit;

impl Analysis for MaybeUninit {
    type Domain = PlaceSet;
    const DIRECTION: Direction = Direction::Forward;

    fn bottom(&self, _body: &Body) -> PlaceSet {
        PlaceSet::new()
    }

    fn boundary(&self, body: &Body) -> PlaceSet {
        // On entry, everything except the arguments is uninitialized.
        (0..body.locals.len())
            .filter(|&i| i == 0 || i > body.arg_count)
            .map(|i| Place::from(Local(i)))
            .collect()
    }

    fn join(&self, state: &mut PlaceSet, other: &PlaceSet) -> bool {
        let before = state.len();
        state.extend(other.iter().cloned());
        state.len() != before
    }

    fn apply_statement(&self, state: &mut PlaceSet, stmt: &Statement, _: Location) {
        match &stmt.kind {
            StatementKind::Assign(dest, rv) => {
                for p in moved_places(&rv.operands()) {
                    mark_uninit(state, &p); // gen: moved out
```

```
                }
                mark_init(state, dest); // kill: (re)initialized
            }
            StatementKind::StorageLive(l) | StatementKind::StorageDead(l) => {
                mark_uninit(state, &Place::from(*l));
            }
            StatementKind::FakeRead(_) => {}
        }
    }

    fn apply_terminator(&self, state: &mut PlaceSet, term: &Terminator, _: Location) {
        match &term.kind {
            TerminatorKind::Call { args, dest, .. } => {
                let args: Vec<&Operand> = args.iter().collect();
                for p in moved_places(&args) {
                    mark_uninit(state, &p);
                }
                mark_init(state, dest);
            }
            TerminatorKind::Drop { place, .. } => mark_uninit(state, place),
            _ => {}
        }
    }
}
```

Read it as gen/kill: a `move` gens its place, an assignment kills its destination, `StorageLive` and `StorageDead` gen the local (a fresh or freed slot holds no value), and a `drop` leaves the place uninitialized. The example `crates/mini-nll/examples/ch29_dataflow.rs` runs it on `maybe_archive`, with two more analyses that we will need, and prints the state on entry to each block:

```
/// Solve `analysis` on `body` and print the state at the entry of every
/// block (for a backward analysis: at the exit, where it starts).
fn show<A, T>(body: &Body, title: &str, analysis: A)
where
    A: Analysis<Domain = BTreeSet<T>>,
    T: Display,
{
    let results = solve(body, analysis);
    println!("{title} ({} block visits)", results.block_visits);
    for (bb, state) in body.block_ids().zip(&results.block_states) {
        println!("  {bb}: {}", set(state));
    }
}
```

```
maybe-uninitialized, on entry (7 block visits)
  bb0: {_0, _2, _3, _4, _5}
  bb1: {_0, _3, _4, _5}
  bb2: {_0, _4, _5}
  bb3: {_0, _2, _5}
  bb4: {_0, _4, _5}
  bb5: {_2, _4, _5}
  bb6: {_2, _3, _4, _5}
maybe-initialized, on entry (7 block visits)
  bb0: {_1}
  bb1: {_1, _2}
  bb2: {_1, _2, _3}
  bb3: {_1, _3, _4}
  bb4: {_1, _2, _3}
  bb5: {_0, _1, _2, _3}
  bb6: {_0, _1}
```

Follow `_2` (`job`) through the first table. Uninitialized on entry to `bb0`, initialized by the call to `String::from`, it is absent in `bb1` and `bb2`; moved into `_5` in `bb2`, it is back on entry to `bb3`. Block

`bb5` has two predecessors, `bb3` (where `_2` may be uninitialized) and `bb4` (where it is not), and the join keeps it. With no loop in the graph, each block was processed exactly once.

The E0382 check

With the facts in hand, the check is a loop over every use of every place:

```
/// Report every use of a place that may be uninitialized: E0382 if some
/// path moved out of it, E0381 if it was never initialized.
pub fn check_moves(body: &Body) -> Vec<Diagnostic> {
    let uninit = solve(body, MaybeUninit).states_before(body);
    let mut diags = Vec::new();
    for loc in body.locations() {
        for (place, kind) in uses_at(body, loc) {
            if let Some(missing) = overlaps(&uninit[&loc], &place) {
                diags.push(report_uninit_use(body, loc, &place, kind, &missing));
            }
        }
    }
    diags
}
```

When a use finds its place in the maybe-uninitialized set, the error message needs the moves responsible, so `report_uninit_use` walks *backwards* from the use, along every path, until it meets a move of the place (or an assignment that initializes it, which ends that path). Here is the analysis on the variant of `maybe_archive` that uses `job` after the `if`:

```
fn conditional_move(urgent: bool) -> usize {
    let job = String::from("backup");
    if urgent {
        archive(job);
    }
    job.len()
}
```

`mini-nll` reports, in the last lines of the same output:

```
error[E0382]: borrow of moved value: `job`
  --> compile_fail/ch29_conditional_move.rs:14:5
   |
10 |     let job = String::from("backup");
   |         --- move occurs because `job` has type `String`, which does not implement the
`Copy` trait
11 |     if urgent {
12 |         archive(job);
   |                 --- value moved here
13 |     }
14 |     job.len()
   |     ^^^ value borrowed here after move
```

and the real compiler reports:

```
error[E0382]: borrow of moved value: `job`
  --> compile_fail/ch29_conditional_move.rs:14:5
   |
10 |     let job = String::from("backup");
   |         --- move occurs because `job` has type `String`, which does not implement the
`Copy` trait
11 |     if urgent {
12 |         archive(job);
   |                 --- value moved here
13 |     }
14 |     job.len()
```

```
   |     ^^^ value borrowed here after move
   |
note: consider changing this parameter type in function `archive` to borrow instead if
owning the value isn't necessary
  --> compile_fail/ch29_conditional_move.rs:4:17
   |
 4 | fn archive(job: String) -> usize {
   |    -------      ^^^^^^ this parameter takes ownership of the value
   |    |
   |    in this function
help: consider cloning the value if the performance cost is acceptable
   |
12 |         archive(job.clone());
   |                     ++++++++
```

Up to rustc's extra note and suggestion, the two are identical, down to the columns of the underlines. That is no accident of formatting: the facts are the same facts. "Borrow of moved value" because the use is `job.len()`, which borrows `job` (in MIR, `_7 = &_2`); "value moved here" because the backward walk found the move `_6 = move _2` in `bb2`.

The same walk explains the loop version of the error from Chapter 6, "value moved here, in previous iteration of loop": when the backward search from a use in a loop body comes back around the back edge to the use itself, the move it finds is the use's own move, one iteration earlier. (The crate's tests build exactly this case.) And a place that was never initialized on some path gives E0381 instead, as in `calibrate` above.

Drop elaboration and drop flags

The dual analysis, `MaybeInit`, computes the places that *may be initialized*. Its table is the second one in the output above. Drop elaboration combines the two at every `drop`:

```rust
/// What drop elaboration does with a `drop(place)` terminator.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum DropStyle {
    /// The place is definitely uninitialized here: the drop is deleted.
    Dead,
    /// The place is definitely initialized: drop it unconditionally.
    Static,
    /// Initialized on some paths only: drop it if a run-time flag says so.
    Conditional,
    /// Partly moved out: drop the fields that are still there.
    Open,
}

/// Classify every `drop` terminator, as rustc's drop elaboration does after
/// borrow checking.
pub fn drop_styles(body: &Body) -> Vec<(Location, Place, DropStyle)> {
    let init = solve(body, MaybeInit).states_before(body);
    let uninit = solve(body, MaybeUninit).states_before(body);
    let mut out = Vec::new();
    for bb in body.block_ids() {
        if let TerminatorKind::Drop { place, .. } = &body.blocks[bb.0].terminator.kind {
            let loc = body.terminator_location(bb);
            let maybe_init = overlaps(&init[&loc], place).is_some();
            let style = match overlaps(&uninit[&loc], place) {
                _ if !maybe_init => DropStyle::Dead,
                None => DropStyle::Static,
                Some(q) if q.is_prefix_of(place) => DropStyle::Conditional,
                Some(_) => DropStyle::Open,
            };
            out.push((loc, place.clone(), style));
        }
```

```
    }
    out
}
```

```
drop elaboration:
  maybe_archive: bb5[1] drop(_2) is Conditional
  record_count: bb3[4] drop(_1) is Dead
```

At the `drop(_2)` in `bb5`, `_2` is in both sets: maybe initialized (through `bb4`) and maybe uninitialized (through `bb3`). Only a run-time flag can tell, which is the `_5` you saw in the optimized MIR. In `record_count`, which we meet below, the value is moved into the return place on every path before the `drop`, so it is definitely uninitialized there and the drop is deleted: the optimized MIR of `record_count` has no `drop` on its normal path at all.

29.5 Liveness

The second analysis is the one non-lexical lifetimes are built on. A local is **live** at a point if the value it holds there may be read later, along some path. It is a backward analysis: a *use* gens the local (reading it later makes it live now), a *definition* kills it (whatever it held before is overwritten, so that value is dead):

```
/// Backward "may" analysis: the locals whose current value may be used later.
pub struct Liveness;

impl Liveness {
    /// An assignment to `dest`: a *def* of the local if the whole local is
    /// overwritten, a *use* if we write through a reference stored in it.
    fn def(state: &mut LocalSet, dest: &Place) {
        if dest.projection.is_empty() {
            state.remove(&dest.local);
        } else if dest.has_deref() {
            state.insert(dest.local);
        }
    }
}

impl Analysis for Liveness {
    type Domain = LocalSet;
    const DIRECTION: Direction = Direction::Backward;

    fn bottom(&self, _body: &Body) -> LocalSet {
        LocalSet::new()
    }

    fn boundary(&self, _body: &Body) -> LocalSet {
        LocalSet::new() // nothing is used after `return`
    }

    fn join(&self, state: &mut LocalSet, other: &LocalSet) -> bool {
        union(state, other)
    }

    // Backward: first remove what the statement defines, then add what it
    // uses (the uses happen before the definition).
    fn apply_statement(&self, state: &mut LocalSet, stmt: &Statement, _: Location) {
        match &stmt.kind {
            StatementKind::Assign(dest, rv) => {
                Self::def(state, dest);
                for p in rv.places() {
                    state.insert(p.local);
```

```
                }
            }
            StatementKind::StorageLive(l) | StatementKind::StorageDead(l) => {
                state.remove(l);
            }
            StatementKind::FakeRead(l) => {
                state.insert(*l);
            }
        }
    }

    fn apply_terminator(&self, state: &mut LocalSet, term: &Terminator, _: Location) {
        match &term.kind {
            TerminatorKind::Call { args, dest, .. } => {
                Self::def(state, dest);
                for p in args.iter().filter_map(Operand::place) {
                    state.insert(p.local);
                }
            }
            TerminatorKind::SwitchInt { discr, .. } => {
                if let Some(p) = discr.place() {
                    state.insert(p.local);
                }
            }
            // A drop uses the value: its destructor may read it.
            TerminatorKind::Drop { place, .. } => {
                state.insert(place.local);
            }
            TerminatorKind::Return => {
                state.insert(Local(0));
            }
            TerminatorKind::Goto(_) => {}
        }
    }
}
```

A `drop` counts as a use: the destructor may read the value. rustc refines this distinction (a variable that is live only because of its drop is *drop-live*, and only the regions its destructor may actually touch are kept alive), which matters for types with a `Drop` impl that hold references. Our programs do not need the refinement.

On `maybe_archive` (the third table of the output, the state on *exit* from each block, where a backward analysis starts):

```
live, on exit (7 block visits)
  bb0: {_1, _2}
  bb1: {_2}
  bb2: {_2}
  bb3: {_0, _2}
  bb4: {_0, _2}
  bb5: {_0}
  bb6: {}
```

`_2` is live on exit from `bb0` through `bb4` because the `drop(_2)` in `bb5` reads it, and `_0` is live from the moment it is assigned until the `return`. Notice that liveness says nothing about initialization: on exit from `bb3`, `job` has been moved, yet it is "live" because the `drop` will look at it. Each analysis answers one question, and the borrow checker combines them.

29.6 Dataflow, proved

The Rust solver *is* a fixpoint computation, and the claims we made about it (it stops, the result satisfies the equations, the result is the most precise one) are theorems. The file `code/lean/Book/Ch29/Dataflow.lean` proves them. Chapter 28 proved things about derivations; here we prove things about an algorithm, so this is where the contrast between "type system" and "analysis" becomes concrete in Lean too.

A dataflow problem has nodes `ι` and facts in a type `α` that is a join-semilattice with a least element: Mathlib's classes `SemilatticeSup` (a partial order with `⊔`) and `OrderBot` (with `⊥`). The `inputs` of a node are its predecessors for a forward analysis and its successors for a backward one, so one definition serves both directions:

```
/-- A dataflow problem with nodes `ι` and facts `α`. For a forward analysis
the `inputs` of a node are its predecessors in the control-flow graph; for a
backward analysis they are its successors. -/
structure Problem (ι α : Type) [SemilatticeSup α] [OrderBot α] where
  /-- The nodes whose facts flow into each node. -/
  inputs : ι → List ι
  /-- Facts that hold at a node whatever flows in (the boundary condition). -/
  entry : ι → α
  /-- The transfer function of each node. -/
  transfer : ι → α → α
```

Joining a list of facts is a fold, with two lemmas that say it is the least upper bound of the list:

```
/-- The join of a list of facts: `x₁ ⊔ x₂ ⊔ ... ⊔ ⊥`. -/
def joinAll : List α → α
  | [] => ⊥
  | x :: xs => x ⊔ joinAll xs

/-- Every element of the list is below the join... -/
theorem le_joinAll {x : α} : ∀ {xs : List α}, x ∈ xs → x ≤ joinAll xs
  | _ :: _, h => by
    rcases List.mem_cons.mp h with rfl | h
    · exact le_sup_left
    · exact le_trans (le_joinAll h) le_sup_right

/-- ...and the join is below every upper bound of the list. -/
theorem joinAll_le {y : α} : ∀ {xs : List α}, (∀ x ∈ xs, x ≤ y) → joinAll xs ≤ y
  | [], _ => bot_le
  | x :: _, h =>
    sup_le (h x List.mem_cons_self)
      (joinAll_le fun z hz => h z (List.mem_cons_of_mem x hz))
```

The first lemma is proved by recursion on the list, with `rcases` splitting the membership hypothesis into "it is the head" (and then `le_sup_left` applies) or "it is in the tail" (and then the recursive call gives `x ≤ joinAll xs`, which `le_sup_right` lifts past the head). The second is a term: an upper bound of the head and of the tail is an upper bound of their join (`sup_le`).

`step` computes the right-hand side of the dataflow equations at every node at once, and `IsSolution` says that a state satisfies every constraint: the boundary facts are included, and so is everything that flows along an edge:

```
/-- One round of the dataflow equations, at every node at once. -/
def step (P : Problem ι α) (s : ι → α) : ι → α :=
  fun v => P.entry v ⊔ joinAll ((P.inputs v).map fun u => P.transfer u (s u))
```

```
/-- A solution satisfies every constraint: the boundary facts, and every edge
`u → v`, whose transferred facts must be included in the facts at `v`. -/
def IsSolution (P : Problem ι α) (s : ι → α) : Prop :=
  ∀ v, P.entry v ≤ s v ∧ ∀ u ∈ P.inputs v, P.transfer u (s u) ≤ s v
```

The solver is the iteration, with fuel, as in Chapter 7's normalizer: Lean insists that every function terminates, and "repeat until nothing changes" does not obviously do so.

```
/-- Iterate `step` until nothing changes, for at most `fuel` rounds. -/
def solve [DecidableEq (ι → α)] (P : Problem ι α) : ℕ → (ι → α) → Option (ι → α)
  | 0, _ => none
  | fuel + 1, s => if P.step s = s then some s else P.solve fuel (P.step s)
```

The test `P.step s = s` compares two functions `ι → α`; Lean can decide it when `ι` is finite and equality on `α` is decidable, which is the instance the `[DecidableEq (ι → α)]` argument asks for.

The first theorem is the basic guarantee: a converged result is a fixpoint.

```
/-- If the solver returns `r`, then `r` is a fixpoint of `step`. -/
theorem solve_fixpoint [DecidableEq (ι → α)] (P : Problem ι α) :
    ∀ (fuel : ℕ) (s r : ι → α), P.solve fuel s = some r → P.step r = r := by
  intro fuel
  induction fuel with
  | zero =>
    -- With no fuel the solver returns `none`, so there is nothing to prove.
    intro s r h
    simp [solve] at h
  | succ fuel ih =>
    intro s r h
    -- Unfold one round: either `step s = s` and `r = s`, or we recurse.
    unfold solve at h
    split at h
    · -- Converged: `h : some s = some r` and the test said `step s = s`.
      rename_i hfix
      cases h
      exact hfix
    · -- Not yet: the recursive call returned `r`; the induction hypothesis
      -- (for one unit of fuel less, starting from `step s`) finishes.
      exact ih (P.step s) r h
```

The proof is an induction on the fuel. With zero fuel the solver returned `none`, so the hypothesis `none = some r` is absurd and `simp` closes the goal. With `fuel + 1`, `unfold solve` exposes the `if`, and `split` gives one goal per branch. In the first, `cases h` identifies `r` with `s`, and the test's hypothesis `step s = s` is the goal. In the second, the solver recursed on `step s`, and the induction hypothesis (stated for every starting state, which is why `intro s r` comes after `induction`) finishes.

Next, fixpoints and solutions. A state is a solution exactly when one more round of `step` does not take it any higher:

```
/-- Being a solution is the same as being a *post-fixpoint*: `step s ≤ s`. -/
theorem isSolution_iff (P : Problem ι α) (s : ι → α) :
    P.IsSolution s ↔ P.step s ≤ s := by
  constructor
  · intro hs v
    obtain ⟨hentry, hedges⟩ := hs v
    apply sup_le hentry
    apply joinAll_le
    intro x hx
    obtain ⟨u, hu, rfl⟩ := List.mem_map.mp hx
    exact hedges u hu
  · intro hs v
```

```
    constructor
    · exact le_trans le_sup_left (hs v)
    · intro u hu
      have hmem : P.transfer u (s u) ∈ (P.inputs v).map fun w => P.transfer w (s w) :=
        List.mem_map.mpr ⟨u, hu, rfl⟩
      exact le_trans (le_joinAll hmem) (le_trans le_sup_right (hs v))
```

Left to right: `step s v` is a join, so it is below `s v` if both halves are (`sup_le`): the boundary half by the first part of the hypothesis, the list half by `joinAll_le`, each element of the list being, by `List.mem_map`, the transfer of some input `u`. Right to left: `entry v` is below `step s v` (`le_sup_left`), which is below `s v`; and each input's contribution is in the list, hence below its join, hence below `step s v` and `s v`. A fixpoint is in particular a solution:

```
/-- A fixpoint of `step` satisfies all the dataflow constraints. -/
theorem fixpoint_isSolution (P : Problem ι α) {r : ι → α} (h : P.step r = r) :
    P.IsSolution r :=
  (P.isSolution_iff r).mpr (le_of_eq h)
```

So the solver's answer satisfies every dataflow constraint. Is it the *best* answer? Any state with at least the right facts at every node is a solution: "every place may be uninitialized everywhere" satisfies all the equations of the move analysis, and it would reject every program. We want the least solution, and for that the transfer functions must be monotone (Chapter 17 introduced Mathlib's `Monotone`). Then `step` is monotone too:

```
/-- With monotone transfer functions, `step` is monotone. -/
theorem step_mono (P : Problem ι α) (hmono : ∀ u, Monotone (P.transfer u)) :
    Monotone P.step := by
  intro s t hst v
  apply sup_le_sup (le_refl _)
  exact joinAll_map_mono (fun u => hmono u (hst u)) _
```

and the result of the solver is below every solution:

```
/-- The solver computes the *least* solution: starting below any solution
`t`, every round stays below `t`, so the result does too. -/
theorem solve_least [DecidableEq (ι → α)] (P : Problem ι α)
    (hmono : ∀ u, Monotone (P.transfer u)) {t : ι → α} (ht : P.IsSolution t) :
    ∀ (fuel : ℕ) (s r : ι → α), s ≤ t → P.solve fuel s = some r → r ≤ t := by
  have hpost : P.step t ≤ t := (P.isSolution_iff t).mp ht
  intro fuel
  induction fuel with
  | zero => intro s r _ h; simp [solve] at h
  | succ fuel ih =>
    intro s r hs h
    unfold solve at h
    split at h
    · cases h; exact hs
    · -- `step s ≤ step t ≤ t`: one more round keeps us below `t`.
      exact ih (P.step s) r (le_trans (P.step_mono hmono hs) hpost) h

/-- In particular, starting from `⊥`. -/
theorem solve_bot_least [DecidableEq (ι → α)] (P : Problem ι α)
    (hmono : ∀ u, Monotone (P.transfer u)) {t r : ι → α} (ht : P.IsSolution t)
    {fuel : ℕ} (h : P.solve fuel ⊥ = some r) : r ≤ t :=
  P.solve_least hmono ht fuel ⊥ r bot_le h
```

The proof is the classic argument for least fixpoints, again by induction on the fuel. If the current state `s` is below a solution `t`, then `step s` is below `step t` (monotonicity), which is below `t` (`t` is a solution, so a post-fixpoint). Starting from `⊥`, which is below everything, every iterate stays below

`t`, and so does the final answer. In the language of the borrow checker: the solver flags a place as maybe-uninitialized only if *every* consistent assignment of facts would flag it too.

A useful corollary compares two problems that differ only in their boundary facts. We will need it for regions:

```
/-- More boundary facts give a larger least solution. (Chapter 29 uses this to
compare lexical and non-lexical regions.) -/
theorem solve_mono_entry [DecidableEq (ι → α)] (P Q : Problem ι α)
    (hinputs : P.inputs = Q.inputs) (htransfer : P.transfer = Q.transfer)
    (hentry : P.entry ≤ Q.entry) (hmono : ∀ u, Monotone (P.transfer u))
    {fp fq : ℕ} {rp rq : ι → α}
    (hp : P.solve fp ⊥ = some rp) (hq : Q.solve fq ⊥ = some rq) : rp ≤ rq := by
  -- `rq` solves `Q`, hence also `P`, whose constraints are weaker.
  have hq_sol : Q.IsSolution rq := Q.fixpoint_isSolution (Q.solve_fixpoint fq ⊥ rq hq)
  have hp_sol : P.IsSolution rq := by
    intro v
    obtain ⟨he, hedges⟩ := hq_sol v
    refine ⟨le_trans (hentry v) he, ?_⟩
    rw [hinputs, htransfer]
    exact hedges
  exact P.solve_bot_least hmono hp_sol hp
```

Finally, termination. With fuel the solver may give up, so we prove that on a finite lattice a certain amount of fuel is always enough:

```
/-- If the `k`-th iterate is already a fixpoint and the fuel exceeds `k`, the
solver returns something. -/
theorem solve_isSome [DecidableEq (ι → α)] (P : Problem ι α) :
    ∀ (fuel k : ℕ) (s : ι → α), P.step^[k + 1] s = P.step^[k] s → k < fuel →
      (P.solve fuel s).isSome := by
  intro fuel
  induction fuel with
  | zero => intro k s _ hk; omega
  | succ fuel ih =>
    intro k s hfix hk
    unfold solve
    split
    · rfl
    · rename_i hne
      cases k with
      | zero => exact absurd hfix hne
      | succ k =>
        -- Iterating `k + 1` times from `s` is iterating `k` times from `step s`.
        rw [Function.iterate_succ_apply, Function.iterate_succ_apply] at hfix
        exact ih k (P.step s) hfix (by omega)

/-- On a finite lattice, monotone transfer functions and `|ι → α| + 1` rounds
of fuel are enough. (The bound is crude: the real one is the number of nodes
times the height of the lattice, plus one.) -/
theorem solve_terminates [Fintype ι] [DecidableEq ι] [Fintype α] [DecidableEq α]
    (P : Problem ι α) (hmono : ∀ u, Monotone (P.transfer u)) :
    (P.solve (Fintype.card (ι → α) + 1) ⊥).isSome := by
  set N := Fintype.card (ι → α)
  let it : ℕ → ι → α := fun k => P.step^[k] ⊥
  -- The iterates form an increasing chain ⊥ ≤ step ⊥ ≤ step (step ⊥) ≤ ...
  have hchain : Monotone it := by
    apply monotone_nat_of_le_succ
    intro k
    induction k with
    | zero => exact bot_le
    | succ k ihk =>
      simp only [it, Function.iterate_succ_apply'] at ihk ⊢
      exact P.step_mono hmono ihk
  -- Pigeonhole: N + 1 iterates in a set of size N, so two of them are equal.
  obtain ⟨i, j, hij, heq⟩ := Fintype.exists_ne_map_eq_of_card_lt
    (fun k : Fin (N + 1) => it k) (by simp [N])
```

```
  -- Between two equal iterates the chain is flat, so the smaller index is a fixpoint.
  have flat : ∀ a b : ℕ, a < b → it a = it b → it (a + 1) = it a := by
    intro a b hab hab'
    apply le_antisymm
    · rw [hab']; exact hchain (by omega)
    · exact hchain (by omega)
  have key : ∃ k, k < N + 1 ∧ it (k + 1) = it k := by
    rcases lt_or_gt_of_ne (Fin.val_ne_of_ne hij) with h | h
    · exact ⟨i, i.isLt, flat i j h heq⟩
    · exact ⟨j, j.isLt, flat j i h heq.symm⟩
  obtain ⟨k, hk, hfix⟩ := key
  exact P.solve_isSome (N + 1) k ⊥ hfix hk
```

`solve_isSome` says that if the `k`-th iterate is already a fixpoint, fuel `k + 1` suffices; its induction peels one iteration off with `Function.iterate_succ_apply`. `solve_terminates` finds such a `k`. The iterates `⊥, step ⊥, step (step ⊥), ...` form an ascending chain (`monotone_nat_of_le_succ` reduces this to one step at a time, proved by induction with `step_mono`). The first `N + 1` iterates live in a set of `N` states, so by the pigeonhole principle (Mathlib's `Fintype.exists_ne_map_eq_of_card_lt`) two coincide, and between two equal elements of an ascending chain the chain is flat (`le_antisymm`): the earlier one is a fixpoint. The bound is wildly pessimistic (the real one is the number of nodes times the height of the lattice), but it needs no computation.

The same analyses, in Lean

`code/lean/Book/Ch29/MoveAnalysis.lean` runs these definitions on the control flow graph of `maybe_archive`: the same seven blocks, the same six locals. Block-level transfer functions are gen/kill functions again (composing gen/kill functions gives a gen/kill function), and every gen/kill function is monotone:

```
/-- A gen/kill transfer function: add the facts `gen`, remove the facts
`kill`, keep the rest. -/
def genKill {β : Type} [DecidableEq β] (gen kill : Finset β) (x : Finset β) : Finset β :=
  gen ∪ (x \ kill)

/-- Every gen/kill function is monotone. -/
theorem genKill_mono {β : Type} [DecidableEq β] (gen kill : Finset β) :
    Monotone (genKill gen kill) := by
  intro x y hxy
  -- `≤` on finite sets is `⊆`; union and difference respect it.
  exact Finset.union_subset_union (le_refl gen) (Finset.sdiff_subset_sdiff hxy (le_refl
kill))
```

```
/-- Maybe-uninitialized locals (forward). The gen/kill sets summarize each
block: in `bb2`, `_5 = move _2` and `archive(move _5)` move out of `_2` and
`_5` (gen) and the call initializes `_4` (kill). -/
def maybeUninit : Problem Block (Finset Loc) where
  inputs := preds
  entry := fun b => if b = 0 then {0, 2, 3, 4, 5} else ∅
  transfer := fun b =>
    match b.val with
    | 0 => genKill ∅ {2}          -- `_2 = String::from(..)`
    | 1 => genKill ∅ {3}          -- `_3 = copy _1`
    | 2 => genKill {2, 5} {4}     -- moves of `_2`, `_5`; `_4 = archive(..)`
    | 3 => genKill {4, 5} {0}     -- `StorageDead(_5)`, `StorageDead(_4)`, `_0 = ()`
    | 4 => genKill ∅ {0}          -- `_0 = ()`
    | 5 => genKill {2, 3} ∅       -- `StorageDead(_3)`, `drop(_2)`
    | _ => genKill {2} ∅          -- `StorageDead(_2)`
```

`#eval` runs the solver, and the results are exactly the tables printed by the Rust example (maybe-uninitialized, maybe-initialized, and live on exit):

```
/-- info: some [[0, 2, 3, 4, 5], [0, 3, 4, 5], [0, 4, 5], [0, 2, 5], [0, 4, 5], [2, 4, 5],
[2, 3, 4, 5]] -/
#guard_msgs in
#eval (maybeUninit.solve 10 ⊥).map table

/-- info: some [[1], [1, 2], [1, 2, 3], [1, 3, 4], [1, 2, 3], [0, 1, 2, 3], [0, 1]] -/
#guard_msgs in
#eval (maybeInit.solve 10 ⊥).map table

/-- info: some [[1, 2], [2], [2], [0, 2], [0, 2], [0], []] -/
#guard_msgs in
#eval (liveness.solve 10 ⊥).map table
```

The drop-flag decision becomes a boolean that `decide` evaluates, and the general theorems apply to the concrete analysis: its answer is the least solution, and the solver terminates (this time without running it):

```
/-- Drop elaboration's question at `drop(_2)` in `bb5`: is `_2` both maybe
initialized and maybe uninitialized there? -/
def needsDropFlag : Bool :=
  match maybeInit.solve 10 ⊥, maybeUninit.solve 10 ⊥ with
  | some init, some uninit => decide (2 ∈ init 5 ∧ 2 ∈ uninit 5)
  | _, _ => false

theorem drop_needs_flag : needsDropFlag = true := by decide
```

```
/-- All three analyses have monotone transfer functions... -/
theorem maybeUninit_mono : ∀ b, Monotone (maybeUninit.transfer b) := by
  intro b
  simp only [maybeUninit]
  split <;> exact genKill_mono _ _

/-- ...so the solver's answer is below every other solution: it is the most
precise one. Any `t` that satisfies the dataflow constraints says "maybe
uninitialized" at least wherever `r` does. -/
theorem maybeUninit_least {r t : Block → Finset Loc}
    (h : maybeUninit.solve 10 ⊥ = some r) (ht : maybeUninit.IsSolution t) : r ≤ t :=
  maybeUninit.solve_bot_least maybeUninit_mono ht h

/-- And the solver always terminates, without our having to run it. -/
theorem maybeUninit_terminates :
    (maybeUninit.solve (Fintype.card (Block → Finset Loc) + 1) ⊥).isSome :=
  maybeUninit.solve_terminates maybeUninit_mono
```

`simp only [maybeUninit]` unfolds the definition so that the `match` becomes visible, and `split` produces one goal per case, each closed by `genKill_mono` (the `<;>` combinator runs the second tactic on every goal the first produced).

29.7 Non-lexical lifetimes

Now the borrows. In Chapter 26 a lifetime was a *region*, and Chapter 27 ordered regions by "outlives". Non-lexical lifetimes (NLL) give the word a precise meaning inside a function body: **a region is a set of program points**. A reference of type `&'a T` may be used only at points that belong to `'a`, and a loan of a place, made by a borrow expression `&'k p`, is in force at the points of `'k`.

The regions are the unknowns of a constraint problem. Each region variable starts empty and must satisfy two kinds of constraints:

$$\frac{x \text{ is live at } P \qquad \rho \text{ occurs in the type of } x}{P \in \rho}\ \text{R-Live} \qquad\qquad \frac{\rho_1 : \rho_2 \qquad P \in \rho_2}{P \in \rho_1}\ \text{R-Outlives}$$

Rule R-Live is the heart of NLL: a reference keeps its region alive where it may still be used, not where it is in scope. Rule R-Outlives reads Chapter 27's outlives relation as inclusion of point sets. The outlives constraints come from the statements of the body, by the subtyping rules of Chapter 27:

- an assignment `x = y` where `y: &'a T` and `x: &'b T` needs `&'a T` to be a subtype of `&'b T`, which (references being covariant in their region) is `'a: 'b`;
- a borrow `x = &'k p` gives the rvalue the type `&'k T`, so it generates `'k: 'b` for the region `'b` of `x`; and the point of the borrow belongs to `'k`;
- a **reborrow** `x = &'k *r`, where `r: &'r T`, also generates `'r: 'k`: the original reference must stay valid as long as the reborrow (the T-Reborrow rule of Chapter 28, as a constraint);
- a call relates arguments to results as the callee's signature says: `index` returns a reference with its argument's region.

In `mini-nll` a constraint remembers where it was generated:

```
/// `sup: sub`, generated at `location`: the region `sup` must outlive (as a
/// set of points: contain) the region `sub`.
#[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord)]
pub struct OutlivesConstraint {
    pub sup: RegionVid,
    pub sub: RegionVid,
    pub location: Location,
}
```

The liveness constraints give each region its starting points, using the liveness analysis (or, for the lexical checker we will compare against, the `StorageLive` / `StorageDead` scopes):

```
/// The liveness constraints: the starting value of every region.
pub fn initial_values(body: &Body, mode: RegionMode) -> RegionValues {
    let n = body.regions.len();
    let mut points = vec![BTreeSet::new(); n];
    let mut ends = vec![BTreeSet::new(); n];
    // A region in the type of a live (or in-scope) local contains the point.
    for (loc, locals) in alive_locals(body, mode) {
        for local in locals {
            for r in body.decl(local).ty.regions() {
                points[r.0].insert(loc);
            }
        }
    }
    // A borrow's region contains the point where the borrow happens.
    for loc in body.locations() {
        if let Some(Statement {
            kind: StatementKind::Assign(_, Rvalue::Ref(k, _, _)),
            ..
        }) = body.blocks[loc.block.0].statements.get(loc.index)
        {
            points[k.0].insert(loc);
        }
    }
    // A named region outlives the whole body, and itself.
    for r in body.universal_regions() {
        points[r.0] = body.locations().into_iter().collect();
```

```
        ends[r.0].insert(r);
    }
    RegionValues { points, ends }
}
```

A *named* lifetime parameter like the `'r` of `fn get_default<'r>` is a **universal** region: the caller chooses it, and all the function knows is that it outlives the whole call. So it contains every point of the body, plus a marker `end('r)` for "the part of the caller after the call". Solving is again a fixpoint, this time over the constraint graph:

```
/// Grow the regions until every outlives constraint holds. Returns the
/// solution and the number of passes over the constraints.
pub fn propagate(
    mut values: RegionValues,
    constraints: &[OutlivesConstraint],
) -> (RegionValues, usize) {
    let mut passes = 0;
    loop {
        passes += 1;
        let mut changed = false;
        for c in constraints {
            // 'sup: 'sub  means  value('sup) ⊇ value('sub)
            let sub_points = values.points[c.sub.0].clone();
            let sub_ends = values.ends[c.sub.0].clone();
            let before = (values.points[c.sup.0].len(), values.ends[c.sup.0].len());
            values.points[c.sup.0].extend(sub_points);
            values.ends[c.sup.0].extend(sub_ends);
            changed |=
                before != (values.points[c.sup.0].len(), values.ends[c.sup.0].len());
        }
        if !changed {
            return (values, passes);
        }
    }
}
```

rustc solves the same problem more cleverly (it computes the strongly connected components of the constraint graph and propagates once over them), but the answer is the same least solution. Note one thing the solver *ignores*: the `location` field. NLL treats `'a: 'b` as plain set inclusion, whatever the point where it was generated. Remember this; it is the root of problem case #3.

rustc's regions, and ours

Here is a function that needs everything at once:

```
/// `counts.push(counts.len())`: the receiver is borrowed mutably *before* the
/// argument is evaluated. Two-phase borrows make this legal.
fn record_count() -> Vec<usize> {
    let mut counts = Vec::new();
    counts.push(counts.len());
    counts
}
```

A nightly compiler's dump of its borrow-checking stage begins with the region values that rustc computed. Produced like the dump of `maybe_archive` above, with `-Zdump-mir=record_count`, with the long reasons for each constraint shortened to `Boring(..)` and `CallArgument(..)`:

```
| Free Region Mapping
| '?0 | Global | ['?0, '?1]
| '?1 | Local | ['?1]
```

```
|
| Inferred Region Values
| '?0 | U0 | {bb0[0..=1], bb1[0..=7], bb2[0..=1], bb3[0..=4], bb4[0..=1], bb5[0], bb6[0], '?
0, '?1}
| '?1 | U0 | {bb0[0..=1], bb1[0..=7], bb2[0..=1], bb3[0..=4], bb4[0..=1], bb5[0], bb6[0], '?
1}
| '?2 | U0 | {bb1[3..=7], bb2[0..=1]}
| '?3 | U0 | {bb1[6..=7]}
| '?4 | U0 | {bb1[4..=7], bb2[0..=1]}
| '?5 | U0 | {bb1[7]}
| '?6 | U0 | {bb1[7]}
| '?7 | U0 | {bb2[1]}
|
| Inference Constraints
| '?0 live at {bb0[0..=1], bb1[0..=7], bb2[0..=1], bb3[0..=4], bb4[0..=1], bb5[0], bb6[0]}
| '?1 live at {bb0[0..=1], bb1[0..=7], bb2[0..=1], bb3[0..=4], bb4[0..=1], bb5[0], bb6[0]}
| '?2 live at {bb1[3]}
| '?3 live at {bb1[6]}
| '?4 live at {bb1[4..=7], bb2[0..=1]}
| '?5 live at {bb1[7]}
| '?6 live at {bb1[7]}
| '?7 live at {bb2[1]}
| '?2: '?4 due to Boring(..) at bb1[3]
| '?3: '?5 due to Boring(..) at bb1[6]
| '?4: '?7 due to CallArgument(..) at bb2[1]
| '?5: '?6 due to CallArgument(..) at bb1[7]
|
| Borrows
| bw0: issued at bb1[3] in '?2
| bw1: issued at bb1[6] in '?3
```

Below the header comes the MIR; here are its first three blocks, which `mini-nll` reproduces statement for statement (without the `unwind` edges):

```
bb0: {
    StorageLive(_1);
    _1 = Vec::<usize>::new() -> [return: bb1, unwind: bb6];
}

bb1: {
    FakeRead(ForLet(None), _1);
    StorageLive(_2);
    StorageLive(_3);
    _3 = &mut _1;
    StorageLive(_4);
    StorageLive(_5);
    _5 = &_1;
    _4 = Vec::<usize>::len(move _5) -> [return: bb2, unwind: bb5];
}

bb2: {
    StorageDead(_5);
    _2 = Vec::<usize>::push(move _3, move _4) -> [return: bb3, unwind: bb5];
}
```

The points are numbered as before: `bb1[3]` is `_3 = &mut _1`, `bb1[6]` is `_5 = &_1`, `bb1[7]` the call to `len`, `bb2[1]` the call to `push`.

`'?0` and `'?1` are free regions that rustc always creates (`'?0` is `'static`). `'?2` is the loan of `&mut _1` at `bb1[3]` (the "Borrows" section says so), and `'?4` the region in the type of `_3`, which receives it: hence `'?2: '?4`. `'?4` is live from `bb1[4]` to `bb2[1]`, the call to `push` that uses `_3`. `'?3` and `'?5` are the same story for `&_1` and `_5`, and `'?6`, `'?7` are the regions of the callees' parameters. Now our own region inference, from `crates/mini-nll/examples/ch29_regions.rs`:

```
record_count: outlives constraints
  '?2: '?0  at bb1[3]
  '?3: '?1  at bb1[6]
record_count: region values (2 passes)
  '?0 = {bb1[4..=7], bb2[0..=1]}   type of _3
  '?1 = {bb1[7]}                   type of _5
  '?2 = {bb1[3..=7], bb2[0..=1]}   loan L0 = &'?2 mut _1 at bb1[3]
  '?3 = {bb1[6..=7]}               loan L1 = &'?3 _1 at bb1[6]
```

The numbering differs, but the sets are the same: the loan of `&mut _1` is `{bb1[3..=7], bb2[0..=1]}` in both, and the shared loan `{bb1[6..=7]}`. Two passes were enough: one to propagate, one to see that nothing changes. Note how short the shared loan's region is: born at `bb1[6]`, dead after the call to `len`. That will matter for two-phase borrows.

Lexical versus non-lexical

Back to `first_then_push`. Its loan of `readings` (the `&_1` passed to `Index::index`) has, under the two rules:

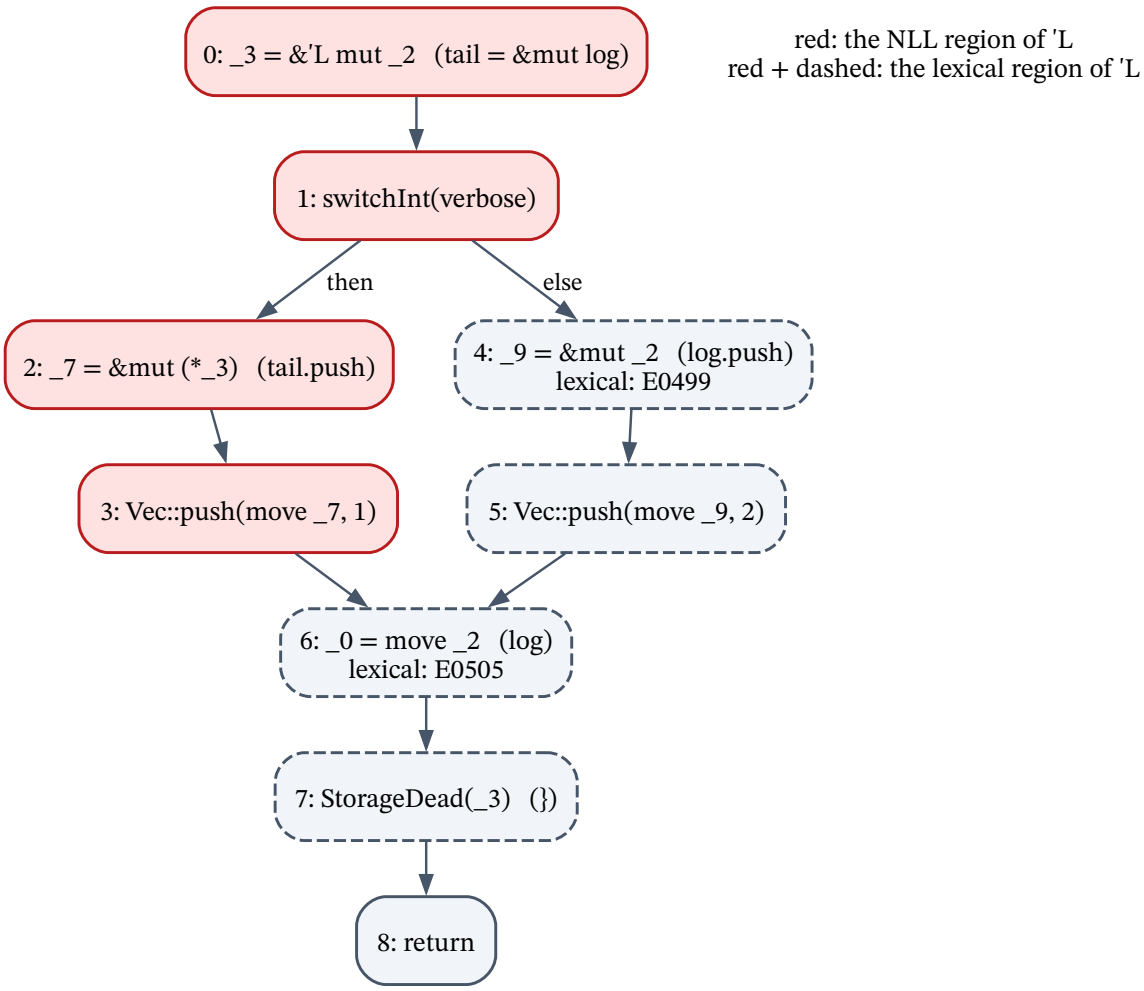
```
first_then_push: loan L0 = &'?2 _1 at bb1[3]
  NLL     '?2 = {bb1[3..=4], bb2[0..=5]}
  lexical '?2 = {bb1[2..=4], bb2[0..=5], bb3[0..=5], bb4[0..=4], bb5[0..=1]}
```

Under NLL the region ends inside `bb2`, where `println!` uses `first` for the last time; the push activates its borrow at `bb3[5]`, outside it. Under the lexical rule the region runs to `bb5[1]`, the `StorageDead` of `first` at the closing brace, and the lexical checker reports, in the style of pre-2018 compilers:

```
error[E0502]: cannot borrow `readings` as mutable because it is also borrowed as immutable
  --> examples/ch29_nll_accepts.rs:24:5
   |
22 |     let first = &readings[0];
   |                 -------- immutable borrow occurs here
23 |     println!("first = {first}");
24 |     readings.push(40);
   |     ^^^^^^^^ mutable borrow occurs here
25 |     readings.len()
26 | }
   | - immutable borrow ends here
```

Regions as sets of points can do something intervals of source text cannot: leave out a branch. In `branch_borrow`, `tail` borrows `log` and is used on one branch only:

```
/// `tail` is used on one branch only; on the other branch it is dead, so its
/// loan is not in scope there (NLL "problem case #2").
fn branch_borrow(verbose: bool) -> Vec<u32> {
    let mut log = Vec::new();
    let tail = &mut log;
    if verbose {
        tail.push(1);
    } else {
        log.push(2);
    }
    log
}
```



The program points of branch_borrow. The NLL region of the loan made by `&mut log` covers the red points only; the lexical region also covers the dashed ones, where `log` is borrowed again and moved.

The NLL region of the loan contains the `then` branch but not the `else` branch, because on the `else` branch `tail` is dead. The NLL RFC (RFC 2094) called this situation "problem case #2: conditional control flow", and NLL solves it.

`code/lean/Book/Ch29/Regions.lean` makes the point in Lean, on the program points of the figure:

```
/-- The program points of `branch_borrow`:

    0: _3 = &'L mut _2          let tail = &mut log;
    1: switchInt(_5)            if verbose
    2: _7 = &'R mut (*_3)       tail.push(1)       (then)
    3: Vec::push(move _7, 1)
    4: _9 = &'E mut _2          log.push(2)        (else)
    5: Vec::push(move _9, 2)
    6: _0 = move _2             log
    7: StorageDead(_3)          }
    8: return
-/
abbrev Point := Fin 9

def next : Point → List Point := fun p =>
  match p.val with
  | 0 => [1] | 1 => [2, 4] | 2 => [3] | 3 => [6] | 4 => [5] | 5 => [6]
```

```
  | 6 => [7] | 7 => [8] | _ => []

/-- The references: `tail` (`_3`), and the receivers `_7` and `_9`. -/
abbrev Ref := Fin 3
```

Liveness of the three references is one more backward `Problem` over these points (`liveIn` in the file). Region inference is one more instance too: the nodes are the region variables, the inputs of `'a` are the regions `'b` with `'a: 'b`, every transfer function is the identity, and the boundary facts are the liveness constraints. Rule R-Outlives *is* the dataflow equation.

```
/-- The region variables: `'t`, `'7`, `'9` (in the types of `tail`, `_7` and
`_9`) and the loan regions `'L`, `'R`, `'E`. -/
abbrev Region := Fin 6

/-- The region in the type of each reference. -/
def typeRegion : Ref → Region := fun x => ⟨x.val, by omega⟩

/-- The outlives constraints `'a: 'b`, listed as the regions `'b` whose points
must flow into each `'a`:
`'L: 't` (point 0), `'R: '7` (point 2), `'t: 'R` (point 2, the reborrow of
`*tail`), `'E: '9` (point 4). -/
def outlives : Region → List Region := fun r =>
  match r.val with
  | 3 => [0] | 4 => [1] | 0 => [4] | 5 => [2] | _ => []

/-- Each loan region contains the point of its borrow. -/
def borrowPoint : Region → Finset Point := fun r =>
  match r.val with
  | 3 => {0} | 4 => {2} | 5 => {4} | _ => ∅

/-- Region inference as a dataflow problem: identity transfer functions,
the constraint graph as edges, and the starting points as boundary facts. -/
def inference (start : Region → Finset Point) : Problem Region (Finset Point) where
  inputs := outlives
  entry := start
  transfer := fun _ x => x

/-- NLL: a region contains the points where a reference whose type mentions
it is live. -/
def nllStart (r : Region) : Finset Point :=
  borrowPoint r ∪ Finset.univ.filter fun p => ∃ x ∈ liveIn p, typeRegion x = r

/-- Lexical: a region contains the points where such a reference is in scope
(from its `StorageLive` to its `StorageDead`). -/
def scope (x : Ref) : Finset Point :=
  match x.val with
  | 0 => {0, 1, 2, 3, 4, 5, 6, 7}    -- tail: to the end of the function
  | 1 => {2, 3}                      -- _7: the statement `tail.push(1)`
  | _ => {4, 5}                      -- _9: the statement `log.push(2)`

def lexicalStart (r : Region) : Finset Point :=
  borrowPoint r ∪ Finset.univ.filter fun p => ∃ x, p ∈ scope x ∧ typeRegion x = r

def regionsOf (start : Region → Finset Point) : Region → Finset Point :=
  match (inference start).solve 20 ⊥ with
  | some r => r
  | none => fun _ => Finset.univ

def regionTable (start : Region → Finset Point) : List (List ℕ) :=
  (List.finRange 6).map fun r => ((regionsOf start r).sort (· ≤ ·)).map Fin.val

/-- info: [[1, 2, 3], [3], [5], [0, 1, 2, 3], [2, 3], [4, 5]] -/
#guard_msgs in
#eval regionTable nllStart

/-- info: [[0, 1, 2, 3, 4, 5, 6, 7], [2, 3], [4, 5], [0, 1, 2, 3, 4, 5, 6, 7], [2, 3], [4,
5]] -/
```

```
#guard_msgs in
#eval regionTable lexicalStart
```

The regions are numbered as in `mini-nll`: `'?0` for `tail`'s type, `'?3` for the loan `'L`. Under NLL the loan's region is `{0, 1, 2, 3}`, under the lexical rule `{0, ..., 7}`, and `decide` checks the consequences:

```
/-- The loan `'L` of `log` (for `tail`) is not in scope at point 4, where the
`else` branch borrows `log` again, nor at point 6, where `log` is moved: NLL
accepts the program. -/
theorem nll_accepts : 4 ∉ regionsOf nllStart 3 ∧ 6 ∉ regionsOf nllStart 3 := by decide

/-- With lexical regions the loan reaches both points: E0499 at 4, E0505 at 6. -/
theorem lexical_rejects : 4 ∈ regionsOf lexicalStart 3 ∧ 6 ∈ regionsOf lexicalStart 3 := by
  decide

/-- The NLL region of the loan is not an interval of the source: it contains
the `then` branch (points 2 and 3) but not the `else` branch. -/
theorem nll_region : regionsOf nllStart 3 = {0, 1, 2, 3} := by decide
```

The last theorem of the file is general rather than computed. If every live reference is in scope (which is how liveness and scopes relate in any program, since a variable can only be used while it exists), every NLL region is contained in the corresponding lexical one. The proof is `solve_mono_entry`: the two problems have the same edges and the same transfer functions, and differ only in their boundary facts.

```
/-- In general: if every live reference is in scope, every NLL region is
contained in the corresponding lexical region. The proof is `solve_mono_entry`:
the two problems differ only in their boundary facts. -/
theorem nll_le_lexical (hstart : nllStart ≤ lexicalStart) :
    regionsOf nllStart ≤ regionsOf lexicalStart := by
  unfold regionsOf
  split
  · rename_i rn hn
    split
    · rename_i rl hl
      exact Problem.solve_mono_entry (inference nllStart) (inference lexicalStart)
        rfl rfl hstart (fun _ => monotone_id) hn hl
    · intro r; exact Finset.subset_univ _
  · -- The NLL solver did not converge: impossible here, as `decide` shows.
    rename_i hn
    exact absurd hn (by decide)

/-- For `branch_borrow` the hypothesis holds: a live reference is in scope. -/
theorem live_in_scope : nllStart ≤ lexicalStart := by decide

theorem nll_regions_smaller : regionsOf nllStart ≤ regionsOf lexicalStart :=
  nll_le_lexical live_in_scope
```

Since an error needs a loan in force at a conflicting point, smaller regions mean fewer errors: in this model, NLL never rejects a program that the lexical checker accepts. (The crate's tests check the inclusion on all nine programs.)

29.8 Loans and conflicts

The last stage puts regions to work. Every borrow expression in the body creates a **loan**:

```
/// A loan: the result of one borrow expression `&'k p` / `&'k mut p`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Loan {
```

```
    pub kind: BorrowKind,
    /// The borrowed place.
    pub place: Place,
    /// The loan's region `'k`.
    pub region: RegionVid,
    /// Where the borrow happens (for a two-phase borrow: its reservation).
    pub location: Location,
    /// The local that receives the reference.
    pub temp: Local,
    /// For a two-phase borrow: the first use of `temp`, where it activates.
    pub activation: Option<Location>,
    pub span: Option<Span>,
}
```

A loan is *in scope* at the points reachable from its creation without leaving its region, and without passing a point that kills it (overwriting or freeing the local the borrowed place is based on). That is a forward analysis too, the one rustc calls `Borrows`:

```
impl BorrowsInScope<'_> {
    fn apply(&self, state: &mut BTreeSet<LoanId>, loc: Location) {
        // Leaving the region: the loan can no longer be used from here on.
        state.retain(|&id| self.values.contains(self.loans[id].region, loc));
        for id in loans_killed_at(self.body, self.loans, loc) {
            state.remove(&id);
        }
        for (id, loan) in self.loans.iter().enumerate() {
            if loan.location == loc {
                state.insert(id);
            }
        }
    }
}
```

Then every statement is decomposed into **accesses**, each a place, a depth and a kind:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Depth {
    /// Touches the place itself, not what it points to (assignment, `StorageDead`).
    Shallow,
    /// Touches the place and everything reachable from it.
    Deep,
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum AccessKind {
    /// `copy p`, `discriminant(p)`.
    ReadCopy,
    /// `&p`.
    ReadBorrow,
    /// `p = ...`.
    Mutate,
    /// `move p`.
    Move,
    /// `&mut p`.
    MutBorrow,
    /// A two-phase `&mut p` at its reservation: acts like a shared borrow.
    Reservation,
    /// A two-phase `&mut p` at its activation: now acts like `&mut p`.
    Activation(LoanId),
    /// `StorageDead(p)` or `drop(p)`.
    StorageDeadOrDrop,
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Access {
    pub place: Place,
    pub depth: Depth,
```

```
    pub kind: AccessKind,
    pub span: Option<Span>,
}
```

and each access is checked against the loans in scope at its point. Two questions decide whether there is an error. Does the access touch the borrowed memory? Two places conflict when one is a prefix of the other, with two refinements: different fields are disjoint, and a *shallow* access such as overwriting a reference does not reach what the reference points to:

```
/// Can an access to `access` (at `depth`) touch memory of the borrowed place?
pub fn places_conflict(borrowed: &Place, access: &Place, depth: Depth) -> bool {
    if borrowed.local != access.local {
        return false;
    }
    for (b, a) in borrowed.projection.iter().zip(&access.projection) {
        if b != a {
            return false; // different fields or variants: disjoint memory
        }
    }
    if borrowed.projection.len() <= access.projection.len() {
        return true; // the access is inside the borrowed place
    }
    // The access is to a prefix of the borrowed place, e.g. `x` vs `(*x).0`.
    // A shallow access does not go through a dereference.
    let rest = &borrowed.projection[access.projection.len()..];
    depth == Depth::Deep || !rest.contains(&ProjectionElem::Deref)
}
```

And are the access and the loan compatible? This is "aliasing XOR mutation" from Chapter 26, as a table:

Access	Shared loan	Mutable loan	Two-phase loan, reserved
read (`copy p` , `&p`)	ok	E0503, E0502	ok
reservation of a two-phase `&mut p`	ok	E0499	E0499
`&mut p` , or an activation	E0502	E0499	E0499
assignment `p = v`	E0506	E0506	E0506
move out of `p`	E0505	E0505	E0505
`StorageDead(p)` , `drop(p)`	E0597	E0597	E0597

The two-phase column, reservations and activations are explained in the next section. In code:

```
/// The error code if `access` conflicts with a loan of kind `loan`, given
/// whether the loan (if two-phase) has been activated.
pub fn conflict(
    access: AccessKind,
    loan: BorrowKind,
    loan_active: bool,
) -> Option<&'static str> {
    use AccessKind::*;
    match (access, loan) {
        // Reading is compatible with shared loans, and so is a reservation.
        (ReadCopy | ReadBorrow | Reservation, BorrowKind::Shared) => None,
        // Reading is compatible with a two-phase loan that is only reserved.
        (ReadCopy | ReadBorrow, _) if !loan_active => None,
        (ReadCopy, _) => Some("E0503"),
        (ReadBorrow, _) => Some("E0502"),
        (MutBorrow | Activation(_), BorrowKind::Shared) => Some("E0502"),
```

```
        (MutBorrow | Reservation | Activation(_), _) => Some("E0499"),
        (Mutate, _) => Some("E0506"),
        (Move, _) => Some("E0505"),
        (StorageDeadOrDrop, _) => Some("E0597"),
    }
}
```

When an access conflicts with a loan, the error message explains *why* the loan is still in force. `explain` walks forward from the error, inside the loan's region, to the first use of a reference that may carry the loan ("borrow later used here"); for the lexical checker it looks for the end of the scope instead; and when the loan flows into a named region, it points at the return. Here is `still_used`, the version of `first_then_push` with the two middle lines swapped:

```
fn still_used() {
    let mut readings = vec![10, 20, 30];
    let first = &readings[0];
    readings.push(40);
    println!("first = {first}");
}
```

`crates/mini-nll/examples/ch29_nll.rs` runs four checkers on all nine programs and prints a table of error codes, then this program's error under NLL:

```
let mut header = format!("{:<22}", "program");
for checker in Checker::ALL {
    header += &format!(" {:<12}", checker.name());
}
println!("{}", header.trim_end());
for body in programs::all() {
    let mut row = format!("{:<22}", body.name);
    for checker in Checker::ALL {
        let codes: Vec<&str> =
            check(&body, checker).iter().map(|d| d.code).collect();
        let cell = if codes.is_empty() {
            "ok".to_string()
        } else {
            codes.join(",")
        };
        row += &format!(" {cell:<12}");
    }
    println!("{}", row.trim_end());
}
```

```
program                lexical      NLL, no 2PB  NLL          Polonius
maybe_archive          ok           ok           ok           ok
conditional_move       E0382        E0382        E0382        E0382
first_then_push        E0502        ok           ok           ok
still_used             E0502        E0502        E0502        E0502
branch_borrow          E0499,E0505  ok           ok           ok
assign_while_borrowed  E0506        E0506        E0506        E0506
record_count           E0502        E0502        ok           ok
rotate_top             E0499        E0499        E0499        E0499
get_default            E0499,E0499  E0499,E0499  E0499,E0499  ok

error[E0502]: cannot borrow `readings` as mutable because it is also borrowed as immutable
 --> compile_fail/ch29_still_used.rs:8:5
  |
7 |     let first = &readings[0];
  |                 -------- immutable borrow occurs here
8 |     readings.push(40);
  |     ^^^^^^^^^^^^^^^^^ mutable borrow occurs here
9 |     println!("first = {first}");
  |                       ----- immutable borrow later used here
```

And here is rustc:

```
error[E0502]: cannot borrow `readings` as mutable because it is also borrowed as immutable
 --> compile_fail/ch29_still_used.rs:8:5
  |
7 |     let first = &readings[0];
  |                 -------- immutable borrow occurs here
8 |     readings.push(40);
  |     ^^^^^^^^^^^^^^^^^ mutable borrow occurs here
9 |     println!("first = {first}");
  |                       ----- immutable borrow later used here
```

The two messages are identical, and the crate's tests check that for this program and three others: our rendering must be a prefix of rustc's `.stderr` snapshot. The same machinery finds E0506 in `assign_while_borrowed`, where the access is an assignment (a shallow write) to a place with a loan in scope:

```
fn assign_while_borrowed() -> i32 {
    let mut total = 10;
    let r = &total;
    total = 20;
    *r + total
}
```

```
error[E0506]: cannot assign to `total` because it is borrowed
 --> compile_fail/ch29_assign_while_borrowed.rs:8:5
  |
7 |     let r = &total;
  |             ------ `total` is borrowed here
8 |     total = 20;
  |     ^^^^^^^^^^ `total` is assigned to here but it was already borrowed
9 |     *r + total
  |     -- borrow later used here
```

Read the table column by column. The *lexical* checker rejects `first_then_push` and `branch_borrow`, which today's compiler accepts. *NLL without two-phase borrows* fixes those, but rejects `record_count`. *NLL*, which is what rustc 1.99 implements, agrees with the real compiler on every one of the nine programs: it accepts the four from `ch29_nll_accepts.rs`, and reports exactly rustc's error codes for the five compile-fail programs. The last column, *Polonius*, differs from NLL in one place, `get_default`. The next two sections are about the last two columns.

29.9 Two-phase borrows

`counts.push(counts.len())` looks innocent, but desugar it. A method call evaluates its receiver first, so it is

```
Vec::push(&mut counts, Vec::len(&counts))
```

with the `&mut counts` taken *before* `&counts`, while the mutable loan is in scope. Under the rules so far that is E0502, and that is the verdict of the "NLL, no 2PB" column. Rust accepts it because of **two-phase borrows**: a mutable borrow that the compiler inserts is first only *reserved*, and becomes *active* at its first use, which here is the call to `push`. While reserved it behaves like a shared borrow: other reads, including new shared borrows, are allowed; writes are not. At activation it is checked again,

now as a mutable borrow against every other loan in scope. In `record_count` the shared loan of `counts` has region `{bb1[6..=7]}`, so by the time the reservation activates at `bb2[1]` it is gone:

```
L0 = &'?2 mut _1 at bb1[3], two-phase, activated at bb2[1]
L1 = &'?3 _1 at bb1[6]
record_count, NLL, no 2PB: 1 error(s)
error[E0502]: cannot borrow `counts` as immutable because it is also borrowed as mutable
  --> examples/ch29_nll_accepts.rs:45:17
   |
45 |     counts.push(counts.len());
   |     ------ ---- ^^^^^^ immutable borrow occurs here
   |     |      |
   |     |      mutable borrow later used by call
   |     mutable borrow occurs here
record_count, NLL: 0 error(s)
```

(That is `crates/mini-nll/examples/ch29_two_phase.rs`. The real dump does not show which borrows are two-phase; `mini-nll` prints a `// two-phase` comment.)

Two-phase borrows apply to borrows the compiler inserts: the auto-referenced receiver of a method call, the implicit reborrow of a `&mut` argument, and the mutable borrow behind an overloaded compound assignment such as `+=` on a `String`. All three compile, and `main` (in the same example file) checks the result:

```
/// More two-phase borrows: an implicit reborrow of a `&mut` argument, and the
/// mutable borrow behind an overloaded `+=`.
fn put(dst: &mut Vec<usize>, n: usize) {
    dst.push(n);
}

fn more_two_phase(v: &mut Vec<usize>) -> String {
    put(v, v.len()); // `v` is reborrowed as `&mut *v` (two-phase)
    v.push(*v.last().unwrap()); // shared reads before the activation are fine
    let mut banner = String::from("ab");
    banner += &banner.to_uppercase(); // `AddAssign::add_assign(&mut banner, ..)`
    banner
}
```

```
more_two_phase: v = [7, 1, 1], banner = abAB
```

An explicit `&mut` is an ordinary mutable borrow, active at once. Writing the desugaring of `record_count` by hand is rejected:

```
fn record_count_explicit() -> Vec<usize> {
    let mut counts = Vec::new();
    Vec::push(&mut counts, counts.len());
    counts
}
```

```
error[E0502]: cannot borrow `counts` as immutable because it is also borrowed as mutable
 --> compile_fail/ch29_explicit_mut_borrow.rs:8:28
  |
8 |     Vec::push(&mut counts, counts.len());
  |     --------- ------------  ^^^^^^ immutable borrow occurs here
  |     |         |
  |     |         mutable borrow occurs here
  |     mutable borrow later used by call
```

And a reservation tolerates *reads* before the activation, not writes. If the argument needs a mutable borrow of the receiver, the reservation conflicts with it:

```
fn rotate_top(stack: &mut Vec<i32>) {
    stack.push(stack.pop().unwrap());
}
```

```
error[E0499]: cannot borrow `*stack` as mutable more than once at a time
 --> compile_fail/ch29_two_phase_limit.rs:6:16
  |
6 |     stack.push(stack.pop().unwrap());
  |     ----- ---- ^^^^^ second mutable borrow occurs here
  |     |     |
  |     |     first borrow later used by call
  |     first mutable borrow occurs here
  |
help: try adding a local storing this argument...
 --> compile_fail/ch29_two_phase_limit.rs:6:16
  |
6 |     stack.push(stack.pop().unwrap());
  |                ^^^^^^^^^^^
help: ...and then using that local as the argument to this call
 --> compile_fail/ch29_two_phase_limit.rs:6:5
  |
6 |     stack.push(stack.pop().unwrap());
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

`mini-nll` (the last lines of `ch29_two_phase`) reproduces the error and its three labels, including rustc's layout of several labels on one line:

```
rotate_top, NLL: 1 error(s)
error[E0499]: cannot borrow `*stack` as mutable more than once at a time
 --> compile_fail/ch29_two_phase_limit.rs:6:16
  |
6 |     stack.push(stack.pop().unwrap());
  |     ----- ---- ^^^^^ second mutable borrow occurs here
  |     |     |
  |     |     first borrow later used by call
  |     first mutable borrow occurs here
```

"First borrow later used by call" points at `push`: the first loan is still reserved, but its activation (the call) comes after the second borrow, so its region contains the second borrow's point. rustc suggests the fix in the part of the message we do not reproduce: evaluate `stack.pop().unwrap()` into a local first.

29.10 Problem case #3

The NLL RFC listed several "problem cases" that the lexical checker got wrong. NLL solved most of them. Number 3, "conditional control flow across functions", it did not:

```
fn get_default<'r>(cache: &'r mut HashMap<u32, String>, key: u32) -> &'r mut String {
    match cache.get_mut(&key) {
        Some(value) => value,
        None => {
            cache.insert(key, String::new());
            cache.get_mut(&key).unwrap()
        }
    }
}
```

The function looks a key up in a cache and returns a mutable reference to the value, inserting a default first if the key is missing. It is perfectly safe: on the `None` path the first borrow of `cache` returned nothing, so nobody can use it. The pinned compiler rejects it twice:

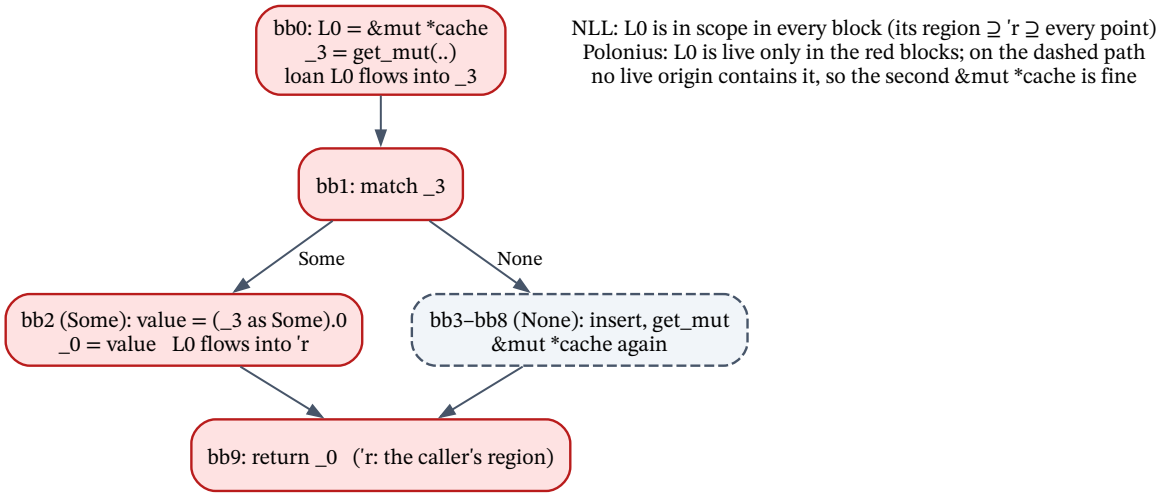
```
error[E0499]: cannot borrow `*cache` as mutable more than once at a time
  --> compile_fail/ch29_problem_case_3.rs:12:13
   |
 8 |   fn get_default<'r>(cache: &'r mut HashMap<u32, String>, key: u32) -> &'r mut String {
   |                  -- lifetime `'r` defined here
 9 |       match cache.get_mut(&key) {
   |       -     ----- first mutable borrow occurs here
   |  _____|
   | |
10 | |         Some(value) => value,
11 | |         None => {
12 | |             cache.insert(key, String::new());
   | |             ^^^^^ second mutable borrow occurs here
...  |
15 | |     }
   | |_____- returning this value requires that `*cache` is borrowed for `'r`

error[E0499]: cannot borrow `*cache` as mutable more than once at a time
  --> compile_fail/ch29_problem_case_3.rs:13:13
   |
 8 |   fn get_default<'r>(cache: &'r mut HashMap<u32, String>, key: u32) -> &'r mut String {
   |                  -- lifetime `'r` defined here
 9 |       match cache.get_mut(&key) {
   |       -     ----- first mutable borrow occurs here
   |  _____|
   | |
10 | |         Some(value) => value,
11 | |         None => {
12 | |             cache.insert(key, String::new());
13 | |             cache.get_mut(&key).unwrap()
   | |             ^^^^^ second mutable borrow occurs here
14 | |         }
15 | |     }
   | |_____- returning this value requires that `*cache` is borrowed for `'r`
```

Follow the constraints. The first `get_mut` reborrows `*cache` (loan L0) and returns `Option<&'?1 mut String>`, tied to L0's region by the signature. On the `Some` path the reference flows into `value` and then into the return place, whose type is `&'r mut String`; that generates `'?4: 'r` and, down the chain, `L0: 'r`. But `'r` is a universal region: it contains every point of the body. NLL's constraints are location-insensitive, so L0's region must contain every point too, *including those on the `None` path*, where the second and third mutable borrows of `*cache` happen. `crates/mini-nll/examples/ch29_polonius.rs` shows it:

```
L0 = &'?9 mut (*_1) at bb0[2]
NLL: region of L0 = {every point, end('r)}
NLL: 2 errors; the first one:
error[E0499]: cannot borrow `*cache` as mutable more than once at a time
  --> compile_fail/ch29_problem_case_3.rs:12:13
   |
 8 | fn get_default<'r>(cache: &'r mut HashMap<u32, String>, key: u32) -> &'r mut String {
   |                -- lifetime `'r` defined here
 9 |     match cache.get_mut(&key) {
   |           ----- first mutable borrow occurs here
10 |         Some(value) => value,
   |                        ----- returning this value requires that `*cache` is borrowed
for `'r`
11 |         None => {
12 |             cache.insert(key, String::new());
   |             ^^^^^ second mutable borrow occurs here
```

(rustc draws the "returning this value" label under the whole `match`; `mini-nll` puts it on the returned `value`. The rest of the message is the same, and so is the second error, which our table reports as the second E0499.)

The problem is not the constraint `L0: 'r`. On the `Some` path it is right: the caller receives L0's reference and may use it for all of `'r`. The problem is that NLL has one region per loan for the whole function, so a fact that holds on one path leaks to the other.



The control-flow graph of get_default, simplified. Under NLL the loan L0 is in scope in every block; under Polonius it is live only in the red blocks, because on the None path no live origin contains it.

Until the borrow checker learns better, the workarounds trade a little efficiency for a borrow the checker can follow: look the key up twice, or let the standard library's entry API do the case analysis behind a single borrow:

```
/// Problem case #3 rewritten so that NLL accepts it: look the key up twice.
fn get_default_twice<'r>(
    cache: &'r mut HashMap<u32, String>,
    key: u32,
) -> &'r mut String {
    if !cache.contains_key(&key) {
        cache.insert(key, String::new());
    }
    cache.get_mut(&key).unwrap()
}

/// The idiomatic version: the entry API does the case analysis inside the
/// library, behind a single mutable borrow.
fn get_default_entry<'r>(
    cache: &'r mut HashMap<u32, String>,
    key: u32,
) -> &'r mut String {
    cache.entry(key).or_default()
}
```

```
cache = [(1, "one!"), (2, "two")]
```

29.11 Polonius

Polonius is a reformulation of the borrow checker that Niko Matsakis proposed in 2018 as a set of Datalog rules (Datalog is a logic-programming language of facts and rules over relations, whose meaning is a least fixpoint, like our dataflow solver's). Its key move is to turn regions around. In NLL

a region is a set of *points*, and you ask whether a point is in a loan's region. In Polonius a region, called an **origin**, is a set of *loans*: the loans that a reference with that origin in its type might have come from. The outlives constraint flips meaning accordingly. `'a: 'b` in NLL says "`'a` contains every point of `'b`"; in Polonius it says "every loan in `'a` is also in `'b`", written `subset('a, 'b)`. Both say that a reference may flow from `'a` to `'b`.

The other move is to make every fact *located*: an origin contains a loan *at a point*, and a subset relation holds *at a point*. Here are the core rules, a simplified version of the "naive" variant of Polonius:

```
origin_contains_loan_on_entry(O, L, P) :- loan_issued_at(O, L, P).
origin_contains_loan_on_entry(O2, L, P) :-
    origin_contains_loan_on_entry(O1, L, P), subset(O1, O2, P).
origin_contains_loan_on_entry(O, L, Q) :-
    origin_contains_loan_on_entry(O, L, P), !loan_killed_at(L, P),
    cfg_edge(P, Q), origin_live_on_entry(O, Q).

subset(O1, O2, P) :- subset_base(O1, O2, P).
subset(O1, O3, P) :- subset(O1, O2, P), subset(O2, O3, P).
subset(O1, O2, Q) :-
    subset(O1, O2, P), cfg_edge(P, Q),
    origin_live_on_entry(O1, Q), origin_live_on_entry(O2, Q).

loan_live_at(L, P) :-
    origin_contains_loan_on_entry(O, L, P), origin_live_on_entry(O, P).
errors(L, P) :- loan_invalidated_at(L, P), loan_live_at(L, P).
```

Each rule is an inference rule written sideways: the conclusion before `:-`, the premises after it. The *input* relations come from the MIR: `cfg_edge` is the graph, `loan_issued_at(O, L, P)` says that the borrow at `P` creates loan `L` with origin `O`, `subset_base(O1, O2, P)` is an outlives constraint generated at `P`, `loan_killed_at` is our kill rule, `origin_live_on_entry` comes from liveness (a named origin like `'r` is live everywhere), and `loan_invalidated_at` is the conflict table. Rule by rule: a loan is in its origin where it is issued; loans flow along subset relations; facts cross a CFG edge only into points where the origin is still live; subset relations are transitive and survive an edge only while both origins are live. A loan is live where a live origin contains it, and an error is an invalidated live loan.

The difference from NLL is in the edge rules. When an origin dies, its loans are no longer carried forward, *on that path*. In `mini-nll` the rules run as one more forward analysis of our framework, whose state at a point is the pair of relations:

```
#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct PoloniusState {
    /// `origin_contains_loan_on_entry(o, l)`: origin `o` may hold a reference
    /// created by loan `l`.
    pub contains: BTreeSet<(RegionVid, LoanId)>,
    /// `subset(o1, o2)`: every loan in `o1` is also in `o2`.
    pub subset: BTreeSet<(RegionVid, RegionVid)>,
}
```

```
impl PoloniusLoans<'_> {
    fn apply(&self, state: &mut PoloniusState, loc: Location) {
        let live = &self.live_origins[&loc];
        // Facts about dead origins are not carried into this point.
        state.contains.retain(|(o, _)| live.contains(o));
        state
            .subset
            .retain(|(a, b)| live.contains(a) && live.contains(b));
        // loan_issued_at and subset_base for this point.
```

```
        for (id, loan) in self.loans.iter().enumerate() {
            if loan.location == loc {
                state.contains.insert((loan.region, id));
            }
        }
        state
            .subset
            .extend(self.subset_base.get(&loc).into_iter().flatten().copied());
        // Close under transitivity and under "loans flow along subsets".
        loop {
            let mut new_subset = Vec::new();
            for &(a, b) in &state.subset {
                for &(c, d) in &state.subset {
                    if b == c && !state.subset.contains(&(a, d)) {
                        new_subset.push((a, d));
                    }
                }
            }
            let mut new_contains = Vec::new();
            for &(o, l) in &state.contains {
                for &(a, b) in &state.subset {
                    if a == o && !state.contains.contains(&(b, l)) {
                        new_contains.push((b, l));
                    }
                }
            }
            if new_subset.is_empty() && new_contains.is_empty() {
                break;
            }
            state.subset.extend(new_subset);
            state.contains.extend(new_contains);
        }
        // Killed loans do not flow to the successors.
        let killed = loans_killed_at(self.body, self.loans, loc);
        state.contains.retain(|(_, l)| !killed.contains(l));
    }
}
```

Filtering by liveness *on entry* to a point is the edge rule: since the join of two states is a union, filtering after the join is the same as filtering on each incoming edge. On `get_default`, the second half of the `ch29_polonius` output lists, on entry to each block, the live origins that contain L0:

```
Polonius: live origins containing L0, on entry to each block
  bb0: -     StorageLive(_3);
  bb1: '?1   StorageDead(_5);
  bb2: '?1   StorageLive(_7);
  bb3: -     StorageLive(_8);
  bb4: -     _8 = HashMap::insert(move _9, copy _2, move _10) -> bb5;
  bb5: -     StorageDead(_10);
  bb6: -     StorageDead(_8);
  bb7: -     StorageDead(_13);
  bb8: -     StorageDead(_11);
  bb9: 'r    StorageDead(_3);
Polonius: inside bb2 (the `Some` arm)
  bb2[1]: '?1   _7 = move (_3 as Some).0;
  bb2[2]: '?4   _0 = move _7;
  bb2[3]: 'r    StorageDead(_7);
  bb2[4]: 'r    goto -> bb9;
Polonius: 0 errors
```

`'?1` is the origin inside the type of `_3`, the `Option` returned by the first `get_mut`, and `'?4` the origin of `value`. L0 reaches `'?1` at the call. On the `Some` path, `_3` is still live (its payload is about to be moved out), and the loan flows from `'?1` to `'?4` and then into `'r` at `_0 = move _7`. On the `None` path, `_3` is dead from the first point of `bb3`, so its origin's facts are dropped, and `'r` never received L0 on that

path. No live origin contains L0 in `bb3` to `bb8`, the borrows of `*cache` there invalidate nothing live, and there are no errors.

The status of Polonius

Polonius' history is a good example of why this book asks you to check the status of ongoing work yourself. The original implementation was a separate library, also called `polonius`, driven by a small Datalog engine, which a nightly compiler could use instead of NLL with the unstable flag `-Zpolonius`. It established that the formulation accepts problem case #3, but it was never fast enough to replace NLL, and the Rust project's later work has aimed at bringing the same location-sensitive reasoning into rustc's own analyses.

Here is what the compilers installed while this chapter was written do with `compile_fail/ch29_problem_case_3.rs`:

- the pinned stable compiler, **rustc 1.99.0** (2026-09-28), rejects it with the two E0499 errors shown above;
- **rustc 1.101.0-nightly** (2026-10-08) accepts it without any flag, and rejects it again with the two E0499 errors when given `-Zpolonius=off`; its `-Zpolonius` flag accepts the values `legacy`, `off` and `next`.

So on that nightly the default borrow checker already accepts problem case #3. Whether this has reached a stable release by the time you read this, and under which name, is exactly the kind of thing to check rather than to remember: run `rustc --edition 2024 --crate-type lib compile_fail/ch29_problem_case_3.rs` with your own compiler, and look at the Rust project's announcements.

29.12 Putting it together: `branch_borrow`, end to end

Let us take one program through the whole borrow checker, with the output of `crates/mini-nll/examples/ch29_walkthrough.rs`, and see each stage do its job. The program is `branch_borrow`, whose points you saw in the figure above.

MIR and loans. Building MIR makes every borrow explicit. There are three: `tail = &mut log`, the two-phase reborrow `&mut *tail` for `tail.push(1)`, and the two-phase borrow of `log` for `log.push(2)`.

```
L0 = &'?3 mut _2 at bb1[2]
L1 = &'?4 mut (*_3) at bb2[2]
L2 = &'?5 mut _2 at bb4[2]
```

Move analysis. `log` is initialized in `bb0` and only moved at the very end, into the return place; every use finds it initialized, so the move check is silent. (The `drop(_2)` after the move is dead, and drop elaboration will delete it.)

Liveness. `tail` is assigned at `bb1[2]` and used for the last time at `bb2[2]`, in the `then` branch. On the `else` branch it is never used, so it is not live there:

```
tail (_3) live at {bb1[3..=7], bb2[0..=2]}
```

Constraints. The assignment `tail = &'?3 mut log` gives `'?3: '?0`; the reborrow for `tail.push(1)` gives `'?4: '?1` for its temporary and, because it reborrows `*tail`, `'?0: '?4`; the `else` borrow gives `'?5: '?2`:

```
constraint '?3: '?0 at bb1[2]
constraint '?4: '?1 at bb2[2]
constraint '?0: '?4 at bb2[2]
constraint '?5: '?2 at bb4[2]
```

Regions and loans in scope. The point to check is `bb4[2]`, where the `else` branch borrows `log`. The loans in scope there decide everything:

```
log.push(2) borrows `log` at bb4[2]
NLL:
  region of L0 = {bb1[2..=7], bb2[0..=3]}
  loans in scope at bb4[2]: {}
  0 error(s)
lexical:
  region of L0 = {bb1[2..=7], bb2[0..=3], bb3[0..=3], bb4[0..=3], bb5[0..=3], bb6[0..=3]}
  loans in scope at bb4[2]: {L0}
  2 error(s)
```

Under NLL, L0's region is `tail`'s liveness plus the reborrow, entirely inside `bb1` and `bb2`; at `bb4[2]` no loan is in scope, and the program is accepted. Under the lexical rule, L0's region is `tail`'s scope, which runs to its `StorageDead` in `bb6`, so L0 is in scope at `bb4[2]` and again at the final move of `log`:

```
error[E0499]: cannot borrow `log` as mutable more than once at a time
  --> examples/ch29_nll_accepts.rs:59:9
   |
55 |     let tail = &mut log;
   |                -------- first mutable borrow occurs here
...
59 |         log.push(2);
   |         ^^^ second mutable borrow occurs here
...
62 | }
   | - first borrow ends here
error[E0505]: cannot move out of `log` because it is borrowed
  --> examples/ch29_nll_accepts.rs:61:5
   |
54 |     let mut log = Vec::new();
   |             --- binding `log` declared here
55 |     let tail = &mut log;
   |                -------- borrow of `log` occurs here
...
61 |     log
   |     ^^^ move out of `log` occurs here
62 | }
   | - borrow ends here
```

And in Lean. `Regions.lean` computed the same two regions on the simplified points of the figure (`{0, 1, 2, 3}` against `{0, ..., 7}`), proved with `decide` that the second `push` and the final move fall inside the lexical region and outside the NLL one, and proved once and for all that NLL regions are contained in lexical ones.

Every stage was a fixpoint over a graph; none built a derivation. The rules enforced are those of Chapters 26 and 28 (moves are affine, `&mut` is unique, a loan forbids conflicting accesses while it lasts), but *how long a loan lasts* is computed from the graph, point by point.

29.13 Common confusions

"**A lifetime is a scope.**" In a function body, a lifetime is a set of program points, computed from where references are used, and it need not be contiguous in the source (`branch_borrow`'s loan skips the `else` branch). A lifetime *parameter* such as `'a` in a signature is different again: a universal region chosen by the caller, which from inside the body contains every point.

"**With NLL, a borrow ends at the last use of the variable.**" Almost. A loan lasts while *any* reference derived from it may be used: a copy of the reference, a reborrow, a value returned to the caller, a struct holding it. And a drop is a use: a value whose type has a destructor and holds a reference keeps the borrow alive until the end of its scope, because the destructor runs there.

"**The borrow checker runs on my source code.**" It runs on MIR, after desugaring: that is why a message can say "first borrow later used by call" at a method name, the call being a terminator and the autoref a separate borrow.

"**Two-phase borrows make `&mut` a little bit shared.**" Only between the reservation and the activation, only for borrows the compiler inserts, and only against reads. `v.push(v.pop().unwrap())` is still an error, and so is `Vec::push(&mut v, v.len())`.

"**Rejected by the borrow checker means unsafe.**" It means the analysis could not prove the program safe. Problem case #3 is safe; NLL's facts are just not located precisely enough. Every analysis of this kind is conservative somewhere.

"**Polonius is a new language feature.**" It changes no syntax and no typing rule. It is a more precise analysis, designed to accept more of the programs that are already safe.

29.14 Summary

- rustc's borrow checker is a static analysis on MIR, not a search for derivations. It is flow-sensitive and conservative: it considers every path, without evaluating conditions.
- MIR is a control-flow graph of basic blocks whose statements act on places; `StorageLive`/`StorageDead` mark scopes, `drop` runs destructors, and drop elaboration adds drop flags where needed. `--emit=mir` shows MIR on stable, `-Zdump-mir` every stage on nightly.
- A dataflow analysis is a lattice of facts, monotone transfer functions and a direction; the worklist algorithm finds the least solution. In Lean we proved that a converged result is a fixpoint, that it satisfies every constraint, that it is the least solution, and that enough fuel always exists.
- Maybe-uninitialized places give E0382 and E0381; maybe-initialized places together with them decide drop flags; liveness is backward and feeds regions.
- NLL regions are sets of points: liveness constraints plus outlives constraints, solved by propagation. Loans are in scope where their region allows, and an access that conflicts with a loan in scope is E0499, E0502, E0503, E0505, E0506 or E0597. NLL regions are contained in lexical ones.
- Two-phase borrows reserve compiler-inserted mutable borrows and activate them at first use, which accepts `v.push(v.len())`.

- NLL's constraints are location-insensitive, which rejects problem case #3. Polonius makes origins sets of loans and locates every fact on the graph, and accepts it. rustc 1.99 rejects it; the nightly of October 2026 accepts it.

29.15 Exercises

1. Hand-run the move analysis on this loop, block by block, and explain the error rustc gives ("value moved here, in previous iteration of loop"): `fn send_all(msg: String) { loop { consume(msg); } }`. How many times does the worklist process the loop block before the fixpoint? *Hint:* the MIR has `bb0: goto -> bb1` and `bb1: _2 = consume(move _1) -> bb1`. On the first visit `_1` is initialized on entry to `bb1`; the back edge joins in the state after the move.
2. Add a program to `mini-nll`: translate `fn shrink(v: &mut Vec<i32>) { let first = &v[0]; v.clear(); println!("{first}"); }` to the miniature MIR, check that the NLL checker reports E0502, and compare its message with rustc's. *Hint:* copy `still_used`; the borrowed place is now `*v`, so use `deref(v)` for the borrow, and give `v` a type with a region of its own.
3. In Lean, define a *must* analysis ("definitely initialized") on the `maybe_archive` graph. Its merge operation is intersection and it starts from "everything", so it does not fit `Problem` as it stands: write a small solver of your own that starts every block at `Finset.univ` and intersects at merge points, then prove with `decide` that on every block its answer is the complement of `maybeUninit`'s. *Hint:* a place is definitely initialized exactly when it is not maybe uninitialized; `Finset.univ \ s` is the complement of `s`.
4. In Lean, prove that `step` is *inflationary* on the iterates from `⊥`: for every `k`, `P.step^[k] ⊥ ≤ P.step^[k + 1] ⊥`, assuming monotone transfer functions. (The proof of `solve_terminates` contains it; extract it as a lemma of its own.) *Hint:* induction on `k`; the base case is `bot_le`, the step uses `Function.iterate_succ_apply'` and `step_mono`.
5. Rewrite `get_default` with a `match` on `cache.contains_key(&key)` in a way that NLL accepts, and explain in terms of loans and regions why your version is accepted while problem case #3 is not. *Hint:* `contains_key` takes `&self` and returns a `bool`, which carries no loan; after it returns, no reference derived from `cache` is live.
6. Using the Polonius rules of this chapter, compute by hand which origins contain the loan of `readings` at each point of `still_used`, and confirm that `errors` contains the activation of the push. Then do the same for `first_then_push` and show that `errors` is empty. *Hint:* the loan flows from the borrow's origin into `_3`'s, then through `index` into `first`'s; the only question at the push is whether `first` is live there.

PART VII

Beyond

Part VII goes beyond what Rust's type system can express. Dependent types let types mention values, and in Lean they let a type state a complete specification. We use them to prove programs correct, then look at the semantic view of type soundness that makes `unsafe` code tractable, and finally return to the question asked in Chapter 1 with everything we need to answer it.

CHAPTER 30

Dependent Types

In Chapter 1 you met a Lean function that Rust cannot write. It returned the first element of an array, and it took two arguments: the array `xs`, and a proof `h : 0 < xs.size` that the array is not empty. The type of the second argument mentions the *value* of the first. We called this a dependent type, noted that it is the feature that sets Lean apart from Rust, and promised a chapter about it. This is that chapter.

Dependent types have been in the background ever since: Chapter 8's intrinsically typed terms, Chapter 10's `Sigma` and `Subtype`, Chapter 25's vault indexed by its balance. Now we look at them directly: what it means for a type to depend on a term, which type formers this requires, what changes about equality, and how much of it a Rust programmer can have today.

The payoff is large. When types may mention values, a type can state a precise property, such as "a vector of exactly three elements", "an index smaller than the length" or "a proof that these two numbers are equal", and the type checker checks it. The price is that type checking sometimes has to compute, and that some equations which are obviously true need a proof before the type checker accepts them. Chapter 31 then uses dependent types for their most ambitious purpose: writing a program's full specification as its type.

In this chapter you will learn:

- what a type family is, and the two type formers that dependent types add: Π-types (dependent functions) and Σ-types (dependent pairs), with their rules;
- the difference between the parameters and the indices of an inductive family, and how a length-indexed vector makes `head` total and `zip` length-preserving;
- how Lean's `Vector`, `Fin` and the notations `xs[i]`, `xs[i]'h`, `xs[i]?` and `xs[i]!` turn bounds checks into proof obligations;
- equality as a type, rewriting with `rw` and `▸`, the difference between definitional and propositional equality, and why it shapes the way you write indices;
- the λ-cube, and where Rust, Haskell and Lean sit in it;
- how far Rust gets: const generics, Peano numerals at the type level, Σ-types as slices and trait objects, newtypes as refinement types, and what is still unstable;
- how to build dimension-checked matrices in Lean, prove their laws, and connect them to Mathlib's `Matrix`.

30.1 Types that mention terms

Every type constructor you use daily in Rust takes types as arguments: `Vec<T>`, `Option<T>`, `Result<T, E>`, `HashMap<K, V>`. There is one familiar exception. In `[u8; 4]` the `4` is a number, and `[u8; 4]` and `[u8; 5]` are different types: a function that expects one rejects the other. A *value* appears inside a type.

Type theory says that such a type **depends** on a term. Here are three types that we would like to have, each depending on a value:

- `Vec α n`, the lists of elements of type `α` of length exactly `n`;
- `Fin n`, the natural numbers smaller than `n`;
- `a = b`, the proofs that `a` and `b` are equal.

Type families

To talk about types like these we need functions that return types. In Lean this needs no special machinery: `Type` is itself a type (it lives in `Type 1`, as Chapter 12 explained), so a function can return a type just as it returns a number. A function whose result is a type is called a **type family**. The Lean code for this section is in `code/lean/Book/Ch30/Vectors.lean`:

```
/-- A type that depends on a value: a function from `Bool` to `Type`. -/
def Answer : Bool → Type
  | true => ℕ
  | false => String

/-- A dependent function: the *type* of the result depends on the *value* of
the argument. Its type is the Π-type `(b : Bool) → Answer b`. -/
def answer : (b : Bool) → Answer b
  | true => (42 : ℕ)
  | false => "no number today"
```

`Answer` is a family indexed by `Bool`: `Answer true` is `ℕ` and `Answer false` is `String`. The function `answer` is the interesting part. Its result type, `Answer b`, depends on the argument `b`. To check the first clause, Lean must check that `42` has type `Answer true`; it computes `Answer true` by running the definition of `Answer`, gets `ℕ`, and the check succeeds. Notice what just happened: to type-check a program, the type checker *ran* a program. That is the first and most important consequence of letting types mention terms.

```
/-- info: 42 -/
#guard_msgs in
#eval (answer true : ℕ)

/-- info: "no number today" -/
#guard_msgs in
#eval (answer false : String)
```

(The ascriptions `(answer true : ℕ)` tell `#eval` how to print the result; Lean accepts them because `Answer true` computes to `ℕ`.) Rust has no counterpart to `answer`: the return type of a Rust function is fixed by its signature, and generic code can choose a type based on another *type* (through an associated type) but never based on a value.

Π-types: dependent functions

The type of `answer` is written `(b : Bool) → Answer b` in Lean, and $\Pi b : \mathit{Bool}.\ \mathit{Answer}\ b$ in type theory. It is a **Π-type**, or dependent function type: the type of functions whose result type depends on the argument. Three rules describe it. The first says when a Π-type is a well-formed type at all:

$$\frac{\Gamma \vdash A : \mathit{Type} \qquad \Gamma, x : A \vdash B : \mathit{Type}}{\Gamma \vdash \Pi x : A.\ B : \mathit{Type}}\ \text{Pi-Form}$$

The second premise is where dependency enters: B is checked in a context that contains x, so B may mention x. In other words, B is a type family over A. The introduction and elimination rules are the familiar rules for λ and application from Chapter 8, with one change:

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x.\ b : \Pi x : A.\ B}\ \text{Pi-Intro} \qquad \frac{\Gamma \vdash f : \Pi x : A.\ B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\ \text{Pi-Elim}$$

The change is in the conclusion of Pi-Elim. The type of $f\ a$ is $B[a/x]$: the family B with the actual argument a substituted for x. Applying `answer` to `true` gives a value of type `Answer true`; applying it to `false` gives one of type `Answer false`. The substitution in the *type* is the whole point.

When B does not mention x, the substitution does nothing, and the rules become exactly those of the ordinary function type $A \to B$ from Chapter 8: $A \to B$ is the special case of $\Pi x : A.\ B$ in which the result type does not depend on the argument. In Lean, `A → B`, `(x : A) → B` and `∀ x : A, B` are all spellings of Π-types. Chapter 10 used the last one: under Curry–Howard, Π is the universal quantifier. Chapter 12's polymorphic `{α : Type} → α → α` is a Π-type whose argument is a type.

Why the letter Π, for *product*? Think of a dependent function over a finite type as a table with one entry per argument. A function of type `(b : Bool) → Answer b` is completely described by two values, `f true : ℕ` and `f false : String`; so such functions correspond one to one with pairs in `ℕ × String`. In general, $\Pi x : A.\ B$ is the product of the types $B[a/x]$ over all a in A, the way $\prod$ is a product of numbers.

Σ-types: dependent pairs

The second type former is the **Σ-type**, or dependent pair type. An element of $\Sigma x : A.\ B$ is a pair (a, b) in which the *type* of b depends on the *value* of a:

$$\frac{\Gamma \vdash A : \mathit{Type} \qquad \Gamma, x : A \vdash B : \mathit{Type}}{\Gamma \vdash \Sigma x : A.\ B : \mathit{Type}}\ \text{Sigma-Form} \qquad \frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma x : A.\ B}\ \text{Sigma-Intro}$$

$$\frac{\Gamma \vdash p : \Sigma x : A.\ B}{\Gamma \vdash \pi_1\ p : A}\ \text{Sigma-Fst} \qquad \frac{\Gamma \vdash p : \Sigma x : A.\ B}{\Gamma \vdash \pi_2\ p : B[\pi_1\ p/x]}\ \text{Sigma-Snd}$$

Rule Sigma-Intro says that to build a pair, the second component must have the type that the family assigns to the *first component*. Rule Sigma-Snd says that the type of the second projection mentions the first projection. When B does not mention x, the rules collapse to those of the ordinary product $A \times B$. Lean writes the Σ-type `(x : A) × B` or `Σ x, B`, builds pairs with the anonymous constructor `⟨a, b⟩`, and projects with `.1` and `.2`.

Why Σ, for *sum*? Take the family `Answer` again. A pair `⟨b, v⟩` with `v : Answer b` is either `⟨true, n⟩` with `n : ℕ` or `⟨false, s⟩` with `s : String`. That is a tagged union of `ℕ` and `String`:

```
/-- A dependent pair: a tag `b`, and a value whose type depends on the tag.
This is exactly what a Rust `enum` with two variants stores. -/
def mixed : List ((b : Bool) × Answer b) :=
  [⟨true, (7 : ℕ)⟩, ⟨false, "seven"⟩, ⟨true, (8 : ℕ)⟩]

/-- To use a component we look at the tag first, as a `match` on an enum does. -/
def describe : (b : Bool) × Answer b → String
  | ⟨true, n⟩ => "number " ++ Nat.repr n
  | ⟨false, s⟩ => String.append "text " s

/-- info: ["number 7", "text seven", "number 8"] -/
#guard_msgs in
#eval mixed.map describe
```

This is precisely what a Rust `enum` is. The discriminant is the first component, the payload is the second, and the type of the payload depends on the discriminant. A `match` on an enum is Σ-elimination: inspect the tag, and only then do you know the type of what it carries, as `describe` does here. So Rust does have Σ-types, built into the language, as long as the first component ranges over a finite set of variants that you list in advance. What it lacks is a Σ over an arbitrary type, such as "some natural number `n`, and a vector of length `n`". We will meet Rust's partial substitutes for that later in the chapter.

Under Curry–Howard, Σ is the existential quantifier: a proof of $\exists x.\, P(x)$ is a witness together with a proof that the witness has the property. Chapter 10 explained the subtlety that Lean keeps two versions apart: `Sigma`, which lives in `Type` and whose first component you can project out and compute with, and `Exists`, which lives in `Prop` and whose witness cannot be extracted into data. A third relative, the subtype `{ x : A // P x }`, is a Σ whose second component is a proof. It will be everywhere in this chapter and the next.

	Π-type	**Σ-type**
Lean	`(x : A) → B x`	`(x : A) × B x`
non-dependent case	`A → B`	`A × B`
over `Bool`	a pair	a tagged union
logic	$\forall x.\, P(x)$	$\exists x.\, P(x)$
closest in Rust	`fn f<const N: usize>`	`enum`, slices, `dyn Trait`

30.2 Indexed families

Type families defined by a function, like `Answer`, are handy, but the families that matter most are defined *inductively*, constructor by constructor. The standard example is the vector of a given length.

Parameters and indices

```
/-- Lists that know their length. `α` is a *parameter* (the same in every
constructor); the natural number after the colon is an *index* (each
constructor chooses it). -/
inductive Vec (α : Type) : ℕ → Type where
  | nil : Vec α 0
  | cons {n : ℕ} : α → Vec α n → Vec α (n + 1)
```

`Vec α` is a function from `ℕ` to `Type`, one type for each length. The declaration distinguishes two kinds of arguments:

- `α`, written *before* the colon, is a **parameter**: the same in the result type of every constructor. Rust's generic parameters are parameters in this sense: every variant of `Option<T>` produces an `Option<T>` for the same `T`.
- The `ℕ` *after* the colon is an **index**: each constructor chooses its value. `nil` produces index `0`, and `cons` turns index `n` into `n + 1`. Rust has no indices (Haskell has them through an extension called GADTs).

You have seen indexed families before: `Nat.le` in Chapter 10 and `Term Γ τ` in Chapter 8. The braces in `cons {n : ℕ}` make `n` an implicit argument (Chapter 2), inferred from the type of the tail.

Functions whose types depend on their arguments

```
/-- `n` copies of `x`: a Π-type, because the length of the result is the
argument `n`. -/
def replicate : (n : ℕ) → α → Vec α n
  | 0, _ => nil
  | n + 1, x => cons x (replicate n x)

def toList {n : ℕ} : Vec α n → List α
  | nil => []
  | cons x xs => x :: toList xs

/-- From a list to a vector whose length is the length of *that* list. -/
def ofList : (xs : List α) → Vec α xs.length
  | [] => nil
  | x :: xs => cons x (ofList xs)
```

`replicate` has a Π-type: the length of the result is the argument `n`. Each clause is checked against the type with the pattern substituted for `n`: the first must produce a `Vec α 0`, the second a `Vec α (n + 1)`. The type of `ofList` mentions the argument `xs` itself.

```
/-- info: Ch30.Vec.replicate {α : Type} (n : ℕ) : α → Vec α n -/
#guard_msgs in
#check replicate

/-- info: Ch30.Vec.ofList {α : Type} (xs : List α) : Vec α xs.length -/
#guard_msgs in
#check ofList

/-- info: [7, 7, 7] -/
#guard_msgs in
#eval (replicate 3 7).toList
```

A total `head` and a length-preserving `zip`

Here is the function that gives indexed families their reputation:

```
/-- The first element. The argument has length `n + 1`, so the `nil` case
cannot occur, and Lean does not ask for it. -/
def head {n : ℕ} : Vec α (n + 1) → α
  | cons x _ => x

def tail {n : ℕ} : Vec α (n + 1) → Vec α n
  | cons _ xs => xs
```

There is no case for `nil`, and Lean does not complain. The argument has type `Vec α (n + 1)`. If it were `nil`, its type would be `Vec α 0`, so the index `0` would have to equal `n + 1`. Those are built from different constructors of `ℕ` (`Nat.zero` and `Nat.succ`), so no `n` can make them equal, and Lean removes the case. This is **dependent pattern matching**: matching on a value also refines the indices in its type, and cases whose indices cannot fit are discarded. The function is total, and the compiled code contains no check and no panic.

```
def three : Vec ℕ 3 := ofList [10, 20, 30]
def two : Vec Char 2 := ofList ['a', 'b']
```

```
/-- info: 10 -/
#guard_msgs in
#eval three.head

/-- info: [20, 30] -/
#guard_msgs in
#eval three.tail.toList
```

The definitions of `three` and `two` deserve a second look. `ofList [10, 20, 30]` has type `Vec ℕ [10, 20, 30].length`, and we declared the type `Vec ℕ 3`. Lean accepts this because `[10, 20, 30].length` computes to `3`: the type checker evaluates again.

Applying `head` to an empty vector is a type error:

```
/--
error: Application type mismatch: The argument
  nil
has type
  Vec ℕ 0
but is expected to have type
  Vec ℕ (?m.2 + 1)
in the application
  nil.head
-/
#guard_msgs in
example : ℕ := head (nil : Vec ℕ 0)
```

The argument `nil` has type `Vec ℕ 0`, and `head` expected a `Vec ℕ (?m.2 + 1)`, where `?m.2` is a metavariable (Chapter 13) standing for the implicit `n`. Unification found no `n` with `n + 1 = 0`. (Lean prints `head nil` as `nil.head`.) In Rust, `v[0]` on an empty vector panics and `v.first()` returns an `Option`; here the question never arises at run time, because a program that could ask it does not type-check.

`zip` is the second classic:

```
/-- Both arguments have the same length `n`. Mismatched cases such as
`nil, cons ..` are impossible, so they are not written. -/
def zip {n : ℕ} : Vec α n → Vec β n → Vec (α × β) n
  | nil, nil => nil
  | cons x xs, cons y ys => cons (x, y) (zip xs ys)

def map {n : ℕ} (f : α → β) : Vec α n → Vec β n
  | nil => nil
  | cons x xs => cons (f x) (map f xs)
```

Both arguments have the same index `n`. If the first is `nil`, then `n` is `0`, so the second must be `nil` too; if the first is a `cons`, the second must be a `cons`. The mixed cases are impossible, and again Lean

removes them. Rust's `Iterator::zip` silently stops at the shorter input; this `zip` cannot be called with inputs of different lengths:

```
/-- info: [(10, 'x'), (20, 'y'), (30, 'z')] -/
#guard_msgs in
#eval (three.zip (ofList ['x', 'y', 'z'])).toList
```

```
/--
error: Application type mismatch: The argument
  two
has type
  Vec Char 2
but is expected to have type
  Vec ?m.5 3
in the application
  three.zip two
-/
#guard_msgs in
example := three.zip two
```

`Fin`: numbers that know their bound

Indexing needs a type of valid positions. Lean's core library defines it as a structure with two fields, a number and a proof:

```
structure Fin (n : ℕ) where
  val  : ℕ
  isLt : val < n
```

A value of type `Fin n` is a natural number together with evidence that it is smaller than `n`. `Fin 3` has exactly three elements, and `Fin 0` has none, because no proof of `i < 0` exists. Indexing a vector with a `Fin` is total:

```
/-- Total indexing: an index of type `Fin n` is a number together with a proof
that it is less than `n`. There is no `nil` case: `Fin 0` is empty. -/
def get {n : ℕ} : Vec α n → Fin n → α
  | cons x _, ⟨0, _⟩ => x
  | cons _ xs, ⟨i + 1, h⟩ => xs.get ⟨i, by omega⟩
```

In the second clause the vector is `cons _ xs` with `xs : Vec α m`, and the index is `⟨i + 1, h⟩` with `h : i + 1 < m + 1`. The recursive call needs a `Fin m`, so we build `⟨i, _⟩` and prove `i < m` with `omega` (Chapter 2), which decides linear arithmetic and finds the proof from `h`. There is no clause for `nil`, because `Fin 0` is empty.

```
/-- info: 30 -/
#guard_msgs in
#eval three.get 2

/-- info: 30 -/
#guard_msgs in
#eval three.get ⟨2, by decide⟩
```

In `three.get 2` the numeral `2` is read as an element of `Fin 3`. The second line builds the index explicitly, with `decide` proving `2 < 3`. An index that is out of range cannot be built:

```
/--
error: Tactic `decide` proved that the proposition
  5 < 3
```

```
is false
-/
#guard_msgs in
example : ℕ := three.get ⟨5, by decide⟩
```

Numerals, however, are a trap:

```
-- A numeral of type `Fin 3` is reduced modulo 3: `5` means `2`.
/-- info: (2, 30) -/
#guard_msgs in
#eval ((5 : Fin 3), three.get 5)
```

> **WARNING**
>
> A numeral of type `Fin n` is reduced modulo `n`, so `(5 : Fin 3)` is `2`, and `three.get 5` happily returns the element at position 2. The type guarantees that the index is in range, not that it is the index you meant. Build indices that come from outside with `⟨i, proof⟩`, which fails loudly when the proof does not exist.

Concatenation and the order of indices

```
/-- Concatenation. The length of the result is `m + n`, not `n + m`: the
section on equality in this chapter explains why the order matters. -/
def append {n m : ℕ} : Vec α n → Vec α m → Vec α (m + n)
  | nil, ys => ys
  | cons x xs, ys => cons x (append xs ys)
```

The result of appending a vector of length `n` and one of length `m` has length `m + n`. You would probably have written `n + m`, and the reason we did not is one of the most instructive facts about dependent types; the section on equality below explains it. Lean does not simplify the index in the type it displays:

```
/-- info: three.append three : Vec ℕ (3 + 3) -/
#guard_msgs in
#check three.append three
```

`Vec ℕ (3 + 3)` is nevertheless the same type as `Vec ℕ 6`, because `3 + 3` computes to `6`: you can declare a constant of type `Vec ℕ 6` and give it `three.append three` as its value.

An indexed type tells the truth about its index, and we can prove it:

```
/-- The index tells the truth: the list underneath has length `n`. -/
theorem length_toList {n : ℕ} (v : Vec α n) : v.toList.length = n := by
  induction v with
  | nil => rfl
  | cons x xs ih => simp [toList, ih]

theorem toList_ofList (xs : List α) : (ofList xs).toList = xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih => simp [ofList, toList, ih]

theorem toList_append {n m : ℕ} (xs : Vec α n) (ys : Vec α m) :
    (xs.append ys).toList = xs.toList ++ ys.toList := by
  induction xs with
  | nil => rfl
  | cons x xs ih => simp [append, toList, ih]
```

`length_toList` is proved by induction on the vector. In the `nil` case both sides compute to `0`, and `rfl` closes the goal. In the `cons x xs` case the goal is `(cons x xs).toList.length = n + 1` and `ih : xs.toList.length = n`; `simp [toList, ih]` unfolds `toList` one step, uses the library fact that `(a :: l).length = l.length + 1`, rewrites with `ih`, and closes `n + 1 = n + 1`. Induction on an indexed family replaces the index in each case (by `0`, or by `n + 1`) automatically. The other two proofs follow the same pattern.

Σ again: when the length depends on the contents

Some operations produce a vector whose length cannot be computed from the lengths of the inputs. Keeping only the elements that satisfy a predicate is the standard example: the length of the result depends on the *elements*. Its type is a Σ-type, "some length `m`, and a vector of that length":

```
/-- Keeping the elements that satisfy `p`: the length of the result depends on
the *contents* of the vector, not just on its length, so the result is a
dependent pair: some length `m`, and a vector of that length. -/
def filter (p : α → Bool) {n : ℕ} : Vec α n → (m : ℕ) × Vec α m
  | nil => ⟨0, nil⟩
  | cons x xs =>
    let ⟨m, ys⟩ := filter p xs
    if p x then ⟨m + 1, cons x ys⟩ else ⟨m, ys⟩
```

```
/-- info: 2 -/
#guard_msgs in
#eval (three.filter (· > 15)).1

/-- info: [20, 30] -/
#guard_msgs in
#eval (three.filter (· > 15)).2.toList
```

A Σ-type hides the index, but a theorem can recover what we know about it:

```
/-- What the Σ-type forgot, a theorem can recover: filtering never makes a
vector longer. -/
theorem filter_length_le (p : α → Bool) {n : ℕ} (v : Vec α n) :
    (v.filter p).1 ≤ n := by
  induction v with
  | nil => simp [filter]
  | cons x xs ih =>
    simp only [filter]
    split <;> simp <;> omega
```

In the `cons` case, `simp only [filter]` unfolds one step, leaving the first projection of an `if p x then … else …`. `split` makes one goal per branch, and `<;>` runs the next tactic on every goal: `simp` computes the projection, and `omega` finishes with `ih : (filter p xs).1 ≤ n`.

Hiding the length altogether gives back ordinary lists:

```
/-- A vector of *some* length: the Σ-type `Σ n, Vec α n`. Forgetting the length
gives back an ordinary list. -/
abbrev SomeVec (α : Type) := (n : ℕ) × Vec α n

def SomeVec.ofList {α : Type} (xs : List α) : SomeVec α := ⟨xs.length, Vec.ofList xs⟩
```

`SomeVec α` and `List α` carry the same information. A list *is* a Σ-type: a length, and that many elements. We will see the same fact in Rust, where a slice reference stores its length next to the pointer.

30.3 Lean's vectors and indexing with proofs

Our `Vec` is a linked list with a length in its type. Lean's core library has a vector type of its own, built differently:

```
/--
info: structure Vector.{u} (α : Type u) (n : ℕ) : Type u
number of parameters: 2
fields:
  Vector.toArray : Array α
  Vector.size_toArray : self.toArray.size = n
constructor:
  Vector.mk.{u} {α : Type u} {n : ℕ} (toArray : Array α) (size_toArray : toArray.size = n) :
Vector α n
-/
#guard_msgs in
#print Vector
```

A `Vector α n` is an `Array α` (Lean's contiguous array, the counterpart of Rust's `Vec`) bundled with a proof that its size is `n`. Proofs are erased when Lean compiles a program (Chapter 10), so a `Vector` costs exactly what an `Array` costs. The operations have the types you would expect:

```
/-- info: #v[1, 2, 3] ++ #v[4, 5] : Vector ℕ (3 + 2) -/
#guard_msgs in
#check #v[1, 2, 3] ++ #v[4, 5]

/-- info: #[1, 2, 3, 4, 5] -/
#guard_msgs in
#eval (#v[1, 2, 3] ++ #v[4, 5]).toArray

/-- info: @Vector.push : {α : Type u_1} → {n : ℕ} → Vector α n → α → Vector α (n + 1) -/
#guard_msgs in
#check @Vector.push
```

Indexing is where the design pays off. Lean's indexing notation `v[i]` asks for a proof that `i` is in range and tries to find it on its own; when it cannot, you have alternatives:

```
/-- An index of type `Fin n` is always valid. -/
def atFin {n : ℕ} (v : Vector ℕ n) (i : Fin n) : ℕ := v[i]

/-- A proof in the context is found automatically. -/
def atProof {n : ℕ} (v : Vector ℕ n) (i : ℕ) (h : i < n) : ℕ := v[i]

/-- A proof can be given explicitly with `'`: here it needs a small argument. -/
def atNext {n : ℕ} (v : Vector ℕ n) (i : ℕ) (h : i + 1 < n) : ℕ := v[i]'(by omega)

/-- A run-time test produces the proof: in the `then` branch, `h : i < n`. -/
def atOr {n : ℕ} (v : Vector ℕ n) (i : ℕ) (default : ℕ) : ℕ :=
  if h : i < n then v[i] else default

/-- Rust's two options are also available: `v[i]?` and `v[i]!`. -/
def atOpt {n : ℕ} (v : Vector ℕ n) (i : ℕ) : Option ℕ := v[i]?
```

Indexing runs a small tactic, `get_elem_tactic`, which looks for the fact in the context and tries some simple arithmetic; that is how `atProof` works. In `atNext` the fact `i < n` only *follows* from the hypothesis, so `v[i]'proof` supplies it explicitly. In `atOr` the proof comes from a run-time test: the *dependent if* `if h : i < n then … else …` names the condition, and in the `then` branch `h : i < n` is a hypothesis. This is the bridge between a dynamic check and a static guarantee.

```
/-- info: (30, 20, 99, none) -/
#guard_msgs in
#eval (atFin #v[10, 20, 30] 2, atNext #v[10, 20, 30] 1 (by decide),
  atOr #v[10, 20, 30] 7 99, atOpt #v[10, 20, 30] 7)
```

Without any of these, Lean refuses, with the message you first saw in Chapter 1:

```
/--
error: failed to prove index is valid, possible solutions:
  - Use `have`-expressions to prove the index is valid
  - Use `a[i]!` notation instead, runtime check is performed, and 'Panic' error message is
produced if index is not valid
  - Use `a[i]?` notation instead, result is an `Option` type
  - Use `a[i]'h` notation instead, where `h` is a proof that index is valid
n : ℕ
v : Vector ℕ n
i : ℕ
⊢ i < n
-/
#guard_msgs in
def atUnchecked {n : ℕ} (v : Vector ℕ n) (i : ℕ) : ℕ := v[i]
```

The message ends with the goal it could not prove, `⊢ i < n`. Here are the options side by side with Rust's:

Lean	Rust	when `i` is out of range
`v[i]`, proof found	(none)	cannot happen
`v[i]'h`	(none)	cannot happen
`v[i]?`	`v.get(i)`	`none`
`v[i]!`	`v[i]`	panic
`if h : i < n then v[i] else d`	`if i < v.len() { v[i] } else { d }`	`d`

The last row looks the same in both languages, but it is not. In Lean, the test produces a proof that the indexing consumes. In Rust, the test produces a `bool` that the type system forgets, and `v[i]` checks the bound again (the optimizer often removes the second check, but as an optimization, not a guarantee).

Subtypes are the general form of "a value with a proof attached". A percentage is a natural number with a proof that it is at most 100:

```
/-- A subtype: natural numbers together with a proof that they are at most
100. A dependent pair whose second component is a proof. -/
abbrev Percent := { n : ℕ // n ≤ 100 }

/-- Halving stays in range, and Lean demands a proof of it: `p.property` is the
proof stored inside `p` (`p.val ≤ 100`), and `omega` derives the new bound. -/
def half (p : Percent) : Percent :=
  ⟨p.val / 2, by have h := p.property; omega⟩

/-- `Fin n` is the same idea, packaged as a structure. -/
example : Fin 3 := ⟨2, by decide⟩
```

```
/-- info: 21 -/
#guard_msgs in
#eval (half ⟨42, by decide⟩).val
```

To return a `Percent`, `half` must prove that the new value is at most 100, using the evidence carried by its input. We will compare this with Rust's way of guaranteeing the same invariant later in the chapter.

30.4 Equality as a type

The third of our dependent types was `a = b`. In a dependently typed language equality is no longer just a boolean test: it decides which programs type-check.

Equality from scratch

Equality is an inductive family with a single constructor. Here it is from scratch, in `code/lean/Book/Ch30/Equality.lean`:

```
/-- Equality from scratch: an inductive family with a single constructor.
There is a proof of `Same a b` only when `a` and `b` are the same. -/
inductive Same {α : Type} : α → α → Prop where
  | refl (a : α) : Same a a
```

`Same a b` is a proposition indexed by two values. The only constructor, `refl a`, builds a proof of `Same a a`, where both indices are the same. So a proof of `Same a b` can exist only when `a` and `b` are the same value. That single constructor is enough to derive everything we expect from equality:

```
theorem symm {a b : α} (h : Same a b) : Same b a := by
  cases h          -- `h` can only be `refl a`, so Lean replaces `b` by `a`
  exact refl a     -- goal: Same a a

theorem trans {a b c : α} (hab : Same a b) (hbc : Same b c) : Same a c := by
  cases hab        -- now `b` is `a`
  cases hbc        -- now `c` is `a`
  exact refl a

theorem congrArg (f : α → β) {a b : α} (h : Same a b) : Same (f a) (f b) := by
  cases h
  exact refl (f a)

/-- Transport: whatever holds of `a` holds of anything the same as `a`. -/
theorem subst (P : α → Prop) {a b : α} (h : Same a b) (pa : P a) : P b := by
  cases h
  exact pa

/-- Our `Same` and Lean's `Eq` are interchangeable. -/
theorem iff_eq {a b : α} : Same a b ↔ a = b := by
  constructor
  · intro h; cases h; rfl
  · intro h; cases h; exact refl a
```

Every proof starts with `cases h`. Since `h` can only have been built by `refl`, whose indices are equal, Lean unifies `b` with `a` and replaces `b` by `a` in the goal: dependent pattern matching once more, as in `head`. The theorem `subst` is the essence of equality: any property of `a` is a property of anything equal to `a`. This principle, **transport**, is the elimination rule of the equality type. Lean's own equality is defined in exactly the same way:

```
/--
info: inductive Eq.{u_1} : {α : Sort u_1} → α → α → Prop
number of parameters: 2
constructors:
```

```
Eq.refl : ∀ {α : Sort u_1} (a : α), a = a
-/
#guard_msgs in
#print Eq
```

(`Sort u_1` makes `Eq` universe-polymorphic, Chapter 12. `#print` reports two parameters, `α` and the left side `a`: since `a` is uniform in the constructor, Lean promotes it from an index to a parameter.)

Definitional and propositional equality

Definitional equality is what the type checker sees by itself. Two terms are definitionally equal if they compute to the same result: by unfolding definitions, by β-reduction (substituting arguments into function bodies), and by running pattern matching on constructors. The tactic `rfl` (and the term `rfl`) proves `a = b` exactly when `a` and `b` are definitionally equal:

```
example : 2 + 2 = 4 := rfl                      -- computation on numerals
example (n : ℕ) : n + 0 = n := rfl              -- one step of `Nat.add`
example (n : ℕ) : (fun k => k + 1) n = n + 1 := rfl   -- a β-reduction
example : Vec.three.head = 10 := rfl            -- unfolding our own definitions
```

Propositional equality is the type `a = b` itself: anything you can *prove*. Every definitional equality is a propositional one (by `rfl`), but not the other way around. Here is the standard example:

```
/--
error: Type mismatch
  rfl
has type
  ?m.9 = ?m.9
but is expected to have type
  0 + n = n
-/
#guard_msgs in
example (n : ℕ) : 0 + n = n := rfl
```

`n + 0 = n` was accepted by `rfl`, but `0 + n = n` is not. The statement is true, of course; it is just not true *by computation*. To see why, look at how addition on natural numbers is defined in Lean's core library (simplified from the file `Init/Prelude.lean` of the toolchain):

```
protected def Nat.add : Nat → Nat → Nat
  | n, .zero   => n
  | n, .succ m => .succ (Nat.add n m)
```

Addition is defined by recursion on its *second* argument. `n + 0` matches the first equation, whatever `n` is, and computes to `n`: definitional. `0 + n`, for a variable `n`, matches neither equation, because the second argument is not known to be `zero` or `succ m`; the computation is stuck, and `rfl` has nothing to compare. To prove `0 + n = n` we need induction:

```
theorem zero_add' (n : ℕ) : 0 + n = n := by
  induction n with
  | zero => rfl                 -- 0 + 0 = 0 is a computation
  | succ k ih =>
    -- ih   : 0 + k = k
    -- goal : 0 + (k + 1) = k + 1
    show (0 + k) + 1 = k + 1    -- the same goal, after one step of `Nat.add`
    rw [ih]                     -- replace `0 + k` by `k`; `rw` then closes `k + 1 = k + 1`
```

In the `succ` case the goal `0 + (k + 1) = k + 1` *can* compute one step, because the second argument is now a successor; `show` restates it in the computed form `(0 + k) + 1 = k + 1`, which Lean accepts because the two are definitionally equal. Then `rw [ih]` **rewrites**: it replaces the left side of `ih : 0 + k = k` by its right side, and closes the resulting `k + 1 = k + 1` by trying `rfl`. (`rw [← h]` rewrites from right to left.) The lesson: definitional equality is fixed by how your functions recurse; propositional equality is everything you can prove.

Why the order of indices matters

With dependent types the distinction decides which programs type-check, because the type checker compares *types* by definitional equality. Here is `append` with the index `n + m`, the way you would first write it:

```
/--
error: Type mismatch
  ys
has type
  Vec α m
but is expected to have type
  Vec α (0 + m)
---
error: Type mismatch
  Vec.cons x (appendNM xs ys)
has type
  Vec α (n✝ + m + 1)
but is expected to have type
  Vec α (n✝ + 1 + m)
-/
#guard_msgs in
def appendNM {α : Type} {n m : ℕ} : Vec α n → Vec α m → Vec α (n + m)
  | .nil, ys => ys
  | .cons x xs, ys => .cons x (appendNM xs ys)
```

Both clauses fail. In the first, `ys : Vec α m` must have type `Vec α (0 + m)`, and `0 + m` is not definitionally `m`. In the second, the clause produces length `n + m + 1` but must produce `n + 1 + m`, which does not compute for a variable `m`. (The dagger in `n✝` marks a name you cannot refer to, here the implicit length of the tail.) The `append` of `Vectors.lean` used `m + n`, and with that order both clauses check by computation: `m + 0` is `m`, and `m + (n + 1)` is `(m + n) + 1`. The index follows the recursion. That is the first fix, and the one to prefer.

The second fix is to keep the type you want and *transport* the value along a proof of the equation. If `h : a = b` and `v : Vec α a`, then `h ▸ v` is the same vector, seen at type `Vec α b`:

```
/-- Concatenation with the length `n + m`: we reuse `append`, whose result has
length `m + n`, and transport it along the equation `m + n = n + m`. -/
def append' {n m : ℕ} (xs : Vec α n) (ys : Vec α m) : Vec α (n + m) :=
  Nat.add_comm m n ▸ xs.append ys
```

`Nat.add_comm m n : m + n = n + m` is a library theorem, proved by induction. The triangle `▸` is our `subst`, used on data; Lean works out what to rewrite from the expected type. Transport changes the type and nothing else:

```
/-- Transport changes the type, not the elements. -/
theorem toList_cast {n m : ℕ} (h : n = m) (v : Vec α n) :
    (h ▸ v : Vec α m).toList = v.toList := by
```

```
  subst h      -- replace `m` by `n` everywhere; now `h ▸ v` is just `v`
  rfl

theorem toList_append' {n m : ℕ} (xs : Vec α n) (ys : Vec α m) :
    (xs.append' ys).toList = xs.toList ++ ys.toList := by
  rw [append', toList_cast, toList_append]
```

In `toList_cast`, `subst h` replaces `m` by `n` everywhere; then `h ▸ v` is a transport along `n = n`, which computes to `v`, and `rfl` finishes. In `toList_append'`, `rw` unfolds `append'`, removes the transport with `toList_cast`, and applies `toList_append`. Transport works, but every theorem about `append'` must see through the `▸`. Experienced users of dependent types choose indices so that the equations they need hold by computation.

Decidable equality

The last ingredient connects equality with running programs. Chapter 5 introduced `Decidable`: a proposition is decidable when there is an algorithm that answers it, and the answer carries evidence either way:

```
/-- info: true -/
#guard_msgs in
#eval decide (2 + 2 = 4)

inductive Color where
  | red | green | blue
  deriving DecidableEq, Repr

/-- info: false -/
#guard_msgs in
#eval decide (Color.red = Color.blue)

/--
info: inductive Decidable : Prop → Type
number of parameters: 1
constructors:
Decidable.isFalse : {p : Prop} → ¬p → Decidable p
Decidable.isTrue : {p : Prop} → p → Decidable p
-/
#guard_msgs in
#print Decidable
```

A `Decidable p` is `isTrue` with a proof of `p` or `isFalse` with a proof of `¬p`; `decide` runs the algorithm, and `deriving DecidableEq` generates one, as `#[derive(PartialEq)]` does in Rust. The difference is the evidence: Rust's `==` returns a `bool`, and nothing remembers what it was a test of. A dependent `if` turns the evidence into something the types can use:

```
/-- From a vector of unknown length to a vector of length `n`. The run-time
test `h : k = n` produces a proof, and the proof changes the type. -/
def SomeVec.toVec? {α : Type} (n : ℕ) : SomeVec α → Option (Vec α n)
  | ⟨k, v⟩ => if h : k = n then some (h ▸ v) else none
```

```
/-- info: some [1, 2, 3] -/
#guard_msgs in
#eval ((SomeVec.ofList [1, 2, 3]).toVec? 3).map Vec.toList

/-- info: none -/
#guard_msgs in
#eval ((SomeVec.ofList [1, 2, 3]).toVec? 4).map Vec.toList
```

`toVec?` takes a vector whose length `k` is known only at run time (a Σ-type) and tests `k = n`. In the `then` branch the test has produced a proof `h : k = n`, and `h ▸ v` transports the vector to type `Vec α n`. This is the general pattern for crossing from data that arrives at run time (Σ) to code that demands static guarantees (Π): decide, then transport.

Not every proposition is decidable. Equality of functions `ℕ → ℕ` is not: no algorithm can compare two functions on infinitely many inputs. Quantifiers over infinite types are not decidable either, though bounded ones are:

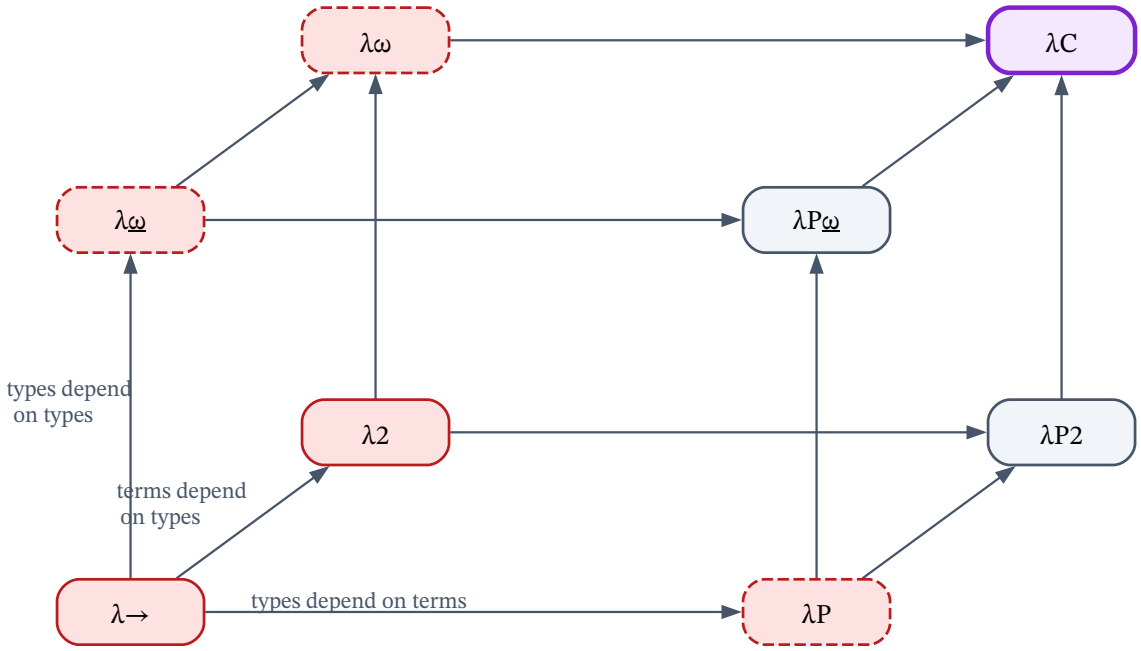
```
/--
error: failed to synthesize
  Decidable (∀ (n : ℕ), n + 0 = n)

Hint: Additional diagnostic information may be available using the `set_option diagnostics
true` command.
-/
#guard_msgs in
example : ∀ n : ℕ, n + 0 = n := by decide

/-- A bounded quantifier is decidable: Lean checks the hundred cases. -/
example : ∀ n < 100, 0 + n = n := by decide
```

30.5 The λ-cube

In the early 1990s Henk Barendregt organized a family of typed λ-calculi as the corners of a cube, which gives us a map for placing Rust and Lean. Start from the simply typed λ-calculus of Chapter 8, where terms depend on terms. There are three independent ways to add a dependency:

- **terms depending on types**: polymorphism, a term that takes a type as an argument, as in `fn id<T>(x: T) -> T` (System F, Chapter 12);
- **types depending on types**: type operators, as in `Vec<T>` viewed as a function from types to types (Chapter 17);
- **types depending on terms**: dependent types, the subject of this chapter.

Each subset of the three gives a system, and the eight systems form the cube:



solid red: in Rust (λ2 as rank-1 generics)
dashed red: only fragments in Rust (generic types, GATs, const generics)
purple: the Calculus of Constructions; Lean's type theory extends it

The λ-cube. Each arrow adds one kind of dependency; λ→ is the simply typed λ-calculus, λ2 is System F, λω is System Fω, λP adds dependent types, and λC, the Calculus of Constructions, has all three. Red marks what Rust has (dashed: only fragments).

The corner λ→ is Chapter 8. Moving back gives λ2, System F (Chapter 12). Moving up gives λω (read "weak lambda omega"), with type operators but no polymorphism, and λω, System Fω, with both: the system in which you can quantify over type constructors, which is what a `Functor` class needs (Chapter 17). Moving right adds dependent types: λP is the system known as LF, used as a logical framework for encoding logics. The far corner, λC, has all three dependencies: it is the **Calculus of Constructions** of Thierry Coquand and Gérard Huet (1988), in which terms and types live in one language and any of them may depend on any other.

Here is where Rust, Lean and Haskell sit, with the caveats that a picture this simple requires:

Language	Polymorphism	Type operators	Dependent types
Rust	rank-1 generics; rank-2 only via traits and `for<'a>`	generic types, but no abstraction over them; GATs as a workaround	const generics only
Haskell (GHC)	higher-rank types (extension)	higher-kinded types, `Functor`	approximated: GADTs, type families, type-level data
Lean	full (Π over types)	full (functions into `Type`)	full: λC and more

GHC's core language is an extension of System Fω, and its dependent-type features simulate Π-types by keeping a type-level copy of the values involved. Lean is beyond the cube altogether: its type theory adds to the Calculus of Constructions inductive types (every `inductive` in this book), a hierarchy of universes and an impredicative `Prop` (Chapter 12), and quotient types. This family of systems is usually called the Calculus of Inductive Constructions; Rocq (formerly Coq) is based on it too.

One more distinction matters in practice. Without dependent types, type checking never runs programs: Rust's types can be checked and erased without evaluating anything but constant expressions. From λP on, deciding whether two types are equal means computing with terms, as with `Answer true` and `Vec ℕ (3 + 3)`. That is why a dependently typed language insists on termination (Chapter 2, and Chapter 31 in detail): if the type checker runs your programs, a non-terminating program would make type checking loop forever.

30.6 Rust's approximations

Rust has no Π-types over arbitrary values, but it has partial substitutes, and they are how working Rust code gets some of this chapter's guarantees.

Const generics

A **const generic** parameter, `const N: usize` in angle brackets, is a value-level parameter of a type or function. Arrays `[T; N]` are the built-in example. The code is in `code/rust/examples/ch30_const_generics.rs`:

```rust
/// The dot product of two arrays of the same length `N`. The length is a
/// const parameter: a value that is part of the type, inferred at each call.
fn dot<const N: usize>(a: [i64; N], b: [i64; N]) -> i64 {
    let mut sum = 0;
    for i in 0..N {
        sum += a[i] * b[i];
    }
    sum
}
```

Calling `dot` with two arrays of length 3 instantiates `N = 3`:

```rust
println!("dot([1, 2, 3], [4, 5, 6]) = {}", dot([1, 2, 3], [4, 5, 6]));
```

```
dot([1, 2, 3], [4, 5, 6]) = 32
```

Arrays of lengths 3 and 4 would be a type error. This is a restricted Π-type: in effect, `dot` : $\Pi N :$ `usize`. $[\texttt{i64}; N] \to [\texttt{i64}; N] \to$ `i64`. A matrix type with dimensions in its type follows the same idea:

```rust
/// An `R × C` matrix of integers. `R` and `C` are values of type `usize`, and
/// they are part of the type: `Matrix<2, 3>` and `Matrix<3, 2>` are different
/// types.
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct Matrix<const R: usize, const C: usize> {
    rows: [[i64; C]; R],
}

impl<const R: usize, const C: usize> Matrix<R, C> {
    fn new(rows: [[i64; C]; R]) -> Self {
```

```
        Matrix { rows }
    }

    /// Rows become columns: an `R × C` matrix becomes a `C × R` matrix.
    fn transpose(&self) -> Matrix<C, R> {
        let mut out = [[0; R]; C];
        for i in 0..R {
            for j in 0..C {
                out[j][i] = self.rows[i][j];
            }
        }
        Matrix { rows: out }
    }
}
```

```
/// `(R × C) * (C × K) = (R × K)`. The left operand's column count and the
/// right operand's row count are the *same* parameter `C`, so a product with
/// mismatched inner dimensions has no implementation to use.
impl<const R: usize, const C: usize, const K: usize> Mul<Matrix<C, K>>
    for Matrix<R, C>
{
    type Output = Matrix<R, K>;

    fn mul(self, rhs: Matrix<C, K>) -> Matrix<R, K> {
        let mut out = [[0; K]; R];
        for i in 0..R {
            for k in 0..K {
                for j in 0..C {
                    out[i][k] += self.rows[i][j] * rhs.rows[j][k];
                }
            }
        }
        Matrix { rows: out }
    }
}
```

The `Mul` implementation exists for a left operand `Matrix<R, C>` and a right operand `Matrix<C, K>` with the *same* `C`. The identity only exists for square shapes, because its `impl` block is for `Matrix<N, N>`:

```
/// The identity exists only for square matrices: this `impl` block is for
/// `Matrix<N, N>`, so `Matrix::<2, 3>::identity()` does not exist.
impl<const N: usize> Matrix<N, N> {
    fn identity() -> Self {
        let mut rows = [[0; N]; N];
        for (i, row) in rows.iter_mut().enumerate() {
            row[i] = 1;
        }
        Matrix { rows }
    }
}
```

```
let a = Matrix::new([[1, 2, 3], [4, 5, 6]]); // Matrix<2, 3>
let b = Matrix::new([[7, 8], [9, 10], [11, 12]]); // Matrix<3, 2>
let ab: Matrix<2, 2> = a * b;
let ba: Matrix<3, 3> = b * a;
println!("a * b       = {:?}", ab.rows);
println!("b * a       = {:?}", ba.rows);
println!("transpose a = {:?}", a.transpose().rows);
assert_eq!(Matrix::<2, 2>::identity() * a, a);
assert_eq!(a * Matrix::<3, 3>::identity(), a);
assert_eq!((a * b).transpose(), b.transpose() * a.transpose());
println!("identity laws and (a * b)^T = b^T * a^T hold");
```

```
a * b       = [[58, 64], [139, 154]]
b * a       = [[39, 54, 69], [49, 68, 87], [59, 82, 105]]
```

```
transpose a = [[1, 4], [2, 5], [3, 6]]
identity laws and (a * b)^T = b^T * a^T hold
```

The annotations `Matrix<2, 2>` and `Matrix<3, 3>` are checked, not trusted. Multiplying a 2×3 matrix by another 2×3 matrix fails with one of the few Rust type errors that talk about numbers (`code/rust/compile_fail/ch30_matrix_mismatch.rs`):

```rust
pub fn product(a: Matrix<2, 3>, b: Matrix<2, 3>) {
    let _c = a * b;
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch30_matrix_mismatch.rs:20:18
   |
20 |     let _c = a * b;
   |                  ^ expected `3`, found `2`
   |
   = note: expected struct `Matrix<3, _>`
              found struct `Matrix<2, 3>`
```

The only candidate implementation requires the right operand to be `Matrix<3, _>`, and `3` is not `2`: E0308, a mismatch between two *values* inside two types.

What const generics cannot do

On stable Rust 1.99 a const parameter may only have an integer type, `bool` or `char`, and inside a type it may only appear on its own, never inside an expression. So the result type of vertical stacking, with `R1 + R2` rows, cannot be written (`code/rust/compile_fail/ch30_const_arithmetic.rs`):

```rust
/// Stack `top` (R1 rows) on `bottom` (R2 rows): the result has R1 + R2 rows.
pub fn vstack<const R1: usize, const R2: usize, const C: usize>(
    top: Matrix<R1, C>,
    bottom: Matrix<R2, C>,
) -> Matrix<{ R1 + R2 }, C> {
    todo!()
}
```

```rust
/// A tensor shape: `N` dimensions, and an array of exactly `N` extents.
pub struct Shape<const N: usize, const DIMS: [usize; N]>;
```

```
error: generic parameters may not be used in const operations
  --> compile_fail/ch30_const_arithmetic.rs:14:15
   |
14 | ) -> Matrix<{ R1 + R2 }, C> {
   |               ^^ cannot perform const operation using `R1`
   |
   = help: const parameters may only be used as standalone arguments here, i.e. `R1`

error: generic parameters may not be used in const operations
  --> compile_fail/ch30_const_arithmetic.rs:14:20
   |
14 | ) -> Matrix<{ R1 + R2 }, C> {
   |                    ^^ cannot perform const operation using `R2`
   |
   = help: const parameters may only be used as standalone arguments here, i.e. `R2`

error[E0770]: the type of const parameters must not depend on other generic parameters
  --> compile_fail/ch30_const_arithmetic.rs:21:54
   |
21 | pub struct Shape<const N: usize, const DIMS: [usize; N]>;
```

```
   |                                                     ^ the type must not depend on the
parameter `N`

error: `[usize; N]` is forbidden as the type of a const generic parameter
  --> compile_fail/ch30_const_arithmetic.rs:21:46
   |
21 | pub struct Shape<const N: usize, const DIMS: [usize; N]>;
   |                                              ^^^^^^^^^^
   |
   = note: the only supported types are integers, `bool`, and `char`
```

The first two errors (which have no error code) reject `R1 + R2` inside a type. The third, E0770, rejects a const parameter whose *type* depends on another parameter: `DIMS` would be an array of `N` extents, a genuinely dependent kind. The fourth rejects arrays as the type of a const parameter at all. Lifting these restrictions is the goal of unstable features, notably `generic_const_exprs` and `adt_const_params`; they are available only on nightly compilers and still being designed, so check their current status before relying on them.

On stable Rust, the usual workaround is to let the caller name the result size and check the arithmetic separately:

```
/// Stable Rust cannot write the result type `Matrix<{ R1 + R2 }, C>`. The
/// workaround: let the caller name the result size `R`, and check `R == R1 + R2`
/// in a `const` block when the function is instantiated.
fn vstack<const R1: usize, const R2: usize, const R: usize, const C: usize>(
    top: Matrix<R1, C>,
    bottom: Matrix<R2, C>,
) -> Matrix<R, C> {
    const { assert!(R == R1 + R2, "vstack: R must be R1 + R2") };
    let mut rows = [[0; C]; R];
    rows[..R1].copy_from_slice(&top.rows);
    rows[R1..].copy_from_slice(&bottom.rows);
    Matrix { rows }
}
```

```
let stacked: Matrix<4, 3> = vstack(a, a);
println!("vstack(a, a) = {:?}", stacked.rows);
```

```
vstack(a, a) = [[1, 2, 3], [4, 5, 6], [1, 2, 3], [4, 5, 6]]
```

An inline `const { … }` block is evaluated at compile time for each set of concrete parameters: had the caller asked for a `Matrix<5, 3>`, the build would stop with error E0080. There is a subtlety, which we checked with rustc 1.99: the check happens only when the function is *instantiated*, during code generation, so `cargo check` does not report it and `cargo build` does. The same technique gives a `first` that rejects empty arrays:

```
/// The first element of a non-empty array. Stable Rust has no way to write
/// `where N > 0` in the signature; the inline `const` block is evaluated when
/// the function is instantiated with a concrete `N`, and a failed assertion
/// stops the compilation.
fn first<T, const N: usize>(items: &[T; N]) -> &T {
    const { assert!(N > 0, "first() needs a non-empty array") };
    &items[0]
}
```

Compare Lean's `head`, and the `vstack` at the end of this chapter: Lean checks the arithmetic in the type, for all sizes at once, when the definition is written. Rust checks each instantiation, after type checking; the generic function itself is never verified.

From run-time data to static shapes

Data from a file or a network has a length known only at run time. Rust bridges the gap the way Lean's `toVec?` did, with a checked conversion:

```
/// From run-time data to a static shape. The number of values on the line is
/// known only at run time; `try_from` checks it once, and from then on the
/// length is part of the type.
fn parse_row<const N: usize>(line: &str) -> Result<[i64; N], String> {
    let mut values = Vec::new();
    for word in line.split_whitespace() {
        let value: i64 = word
            .parse()
            .map_err(|_| format!("not a number: {word:?}"))?;
        values.push(value);
    }
    let found = values.len();
    <[i64; N]>::try_from(values)
        .map_err(|_| format!("expected {N} numbers, found {found}"))
}
```

```
let row: Result<[i64; 3], String> = parse_row("1 2 3");
println!("parse_row::<3>(\"1 2 3\") = {row:?}");
let short: Result<[i64; 3], String> = parse_row("1 2");
println!("parse_row::<3>(\"1 2\")   = {short:?}");
if let Ok(r) = row {
    // From here on the length is static: `dot` accepts only another [i64; 3].
    println!("dot(row, row) = {}", dot(r, r));
}
```

```
parse_row::<3>("1 2 3") = Ok([1, 2, 3])
parse_row::<3>("1 2")   = Err("expected 3 numbers, found 2")
dot(row, row) = 14
```

The difference from Lean is that `N` must be fixed at compile time: the caller decides that it wants three numbers, and the data either has three or is rejected. In Lean, as the walkthrough at the end of the chapter shows, the length itself can come from the data.

Peano numerals at the type level

Before const generics, Rust programmers encoded numbers as *types*, and the technique still helps where const generics cannot do arithmetic. Zero is a type `Z`, and the successor of `N` is `S<N>` (`code/rust/examples/ch30_type_level_nat.rs`):

```
/// Zero, as a type.
struct Z;

/// The successor of `N`, as a type. `PhantomData<N>` lets the struct mention
/// `N` without storing a value of it: `S<N>` has size zero.
struct S<N>(PhantomData<N>);

/// Reflection: from a number that lives in the types back to an ordinary
/// `usize` value. The compiler computes `VALUE` by recursion on the type.
trait Nat {
    const VALUE: usize;
}
```

```
impl Nat for Z {
    const VALUE: usize = 0;
}

impl<N: Nat> Nat for S<N> {
    const VALUE: usize = N::VALUE + 1;
}

type One = S<Z>;
type Two = S<One>;
type Three = S<Two>;
type Five = S<S<Three>>;
```

`PhantomData<N>` is a zero-sized marker from the standard library that lets a struct mention a type parameter without storing a value of it, so `S<S<Z>>` is an empty struct whose *type* encodes the number 2. The trait `Nat` reflects a type-level number back to a value, computed by recursion on the type.

Addition is a trait with an associated type, defined by two `impl` blocks that mirror the two equations of addition by recursion on the first argument:

```
/// Addition as a function on types, by recursion on the first argument:
///   0 + m = m
///   (n + 1) + m = (n + m) + 1
/// `<N as Plus<M>>::Sum` is the type-level number `N + M`.
trait Plus<M> {
    type Sum;
}

impl<M> Plus<M> for Z {
    type Sum = M;
}

impl<N: Plus<M>, M> Plus<M> for S<N> {
    type Sum = S<N::Sum>;
}
```

To find `<S<S<Z>> as Plus<M>>::Sum`, the trait solver uses the second `impl`, which asks for `<S<Z> as Plus<M>>::Sum`, and so on down to `Z`. This is computation by search, as in a logic programming language such as Prolog: the `impl` blocks are rules, and the solver chains them.

```
type TwoPlusThree = <Two as Plus<Three>>::Sum;
println!("Two + Three = {}", short_name::<TwoPlusThree>());
println!("            = {} (as a value)", TwoPlusThree::VALUE);
```

```
Two + Three = S<S<S<S<S<Z>>>>>
            = 5 (as a value)
```

With numbers in types, we can build Rust's version of `Vec α n`:

```
/// A vector whose length `N` is part of its type. The elements live in an
/// ordinary `Vec`; `N` exists only at compile time.
struct Vect<T, N> {
    items: Vec<T>,
    len: PhantomData<N>,
}

impl<T> Vect<T, Z> {
    /// The empty vector has length zero.
    fn nil() -> Self {
        Vect {
            items: Vec::new(),
```

```
            len: PhantomData,
        }
    }
}

impl<T, N> Vect<T, N> {
    /// Adding an element makes the vector one longer: `N` becomes `S<N>`.
    fn push(mut self, x: T) -> Vect<T, S<N>> {
        self.items.push(x);
        Vect {
            items: self.items,
            len: PhantomData,
        }
    }

    /// Both vectors have the same length `N`, so no element is left over.
    fn zip<U>(self, other: Vect<U, N>) -> Vect<(T, U), N> {
        let items = self.items.into_iter().zip(other.items).collect();
        Vect {
            items,
            len: PhantomData,
        }
    }

    /// The result has length `N + M`, computed by the trait solver.
    fn append<M>(mut self, other: Vect<T, M>) -> Vect<T, <N as Plus<M>>::Sum>
    where
        N: Plus<M>,
    {
        self.items.extend(other.items);
        Vect {
            items: self.items,
            len: PhantomData,
        }
    }

    fn as_slice(&self) -> &[T] {
        &self.items
    }
}

impl<T, N: Nat> Vect<T, N> {
    /// The length is read from the type, not from the `Vec`.
    fn len(&self) -> usize {
        N::VALUE
    }
}

/// `head` and `tail` exist only for vectors of length `S<N>`, that is, for
/// non-empty vectors. On `Vect<T, Z>` the methods are simply not there.
impl<T, N> Vect<T, S<N>> {
    fn head(&self) -> &T {
        &self.items[0]
    }

    fn tail(mut self) -> Vect<T, N> {
        self.items.remove(0);
        Vect {
            items: self.items,
            len: PhantomData,
        }
    }
}
```

`Vect<T, N>` stores an ordinary `Vec<T>`; `N` exists only at compile time. Each signature transcribes a dependent type from the Lean section: `push` goes from `N` to `S<N>`, `zip` demands the same `N` twice, `append` returns `<N as Plus<M>>::Sum`, and `head` and `tail` live in an `impl` block for `Vect<T, S<N>>` only.

```
let names: Vect<&str, Three> = Vect::nil().push("ada").push("alan").push("grace");
println!("names       = {:?}, len {}", names.as_slice(), names.len());
println!("head        = {}", names.head());
let years = Vect::nil().push(1815).push(1912).push(1906);
let pairs = names.zip(years);
println!("zipped      = {:?}", pairs.as_slice());
let rest: Vect<(&str, i32), Two> = pairs.tail();
println!("tail        = {:?}", rest.as_slice());
```

```
let xs: Vect<u8, Three> = Vect::nil().push(1).push(2).push(3);
let ys: Vect<u8, Two> = Vect::nil().push(4).push(5);
// This annotation is checked: it compiles only because 3 + 2 = 5.
let zs: Vect<u8, Five> = xs.append(ys);
println!("appended    = {:?}, len {}", zs.as_slice(), zs.len());
```

```
names       = ["ada", "alan", "grace"], len 3
head        = ada
zipped      = [("ada", 1815), ("alan", 1912), ("grace", 1906)]
tail        = [("alan", 1912), ("grace", 1906)]
appended    = [1, 2, 3, 4, 5], len 5
```

The two classic mistakes are type errors. The head of an empty vector is a method that does not exist (`ch30_head_of_empty.rs`):

```
pub fn first_of_nothing() -> i32 {
    let empty: Vect<i32, Z> = Vect(Vec::new(), PhantomData);
    *empty.head()
}
```

```
error[E0599]: no method named `head` found for struct `Vect<i32, Z>` in the current scope
  --> compile_fail/ch30_head_of_empty.rs:18:12
   |
 7 | pub struct Vect<T, N>(Vec<T>, PhantomData<N>);
   | ---------------------- method `head` not found for this struct
...
18 |     *empty.head()
   |            ^^^^ method not found in `Vect<i32, Z>`
   |
   = note: the method was found for `Vect<T, S<N>>`
```

Zipping vectors of lengths 2 and 1 is a mismatch between `S<S<Z>>` and `S<Z>` (`ch30_zip_lengths.rs`):

```
pub fn pair_up(names: Vect<&str, S<S<Z>>>, ages: Vect<u32, S<Z>>) {
    let _pairs = names.zip(ages);
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch30_zip_lengths.rs:17:28
   |
17 |     let _pairs = names.zip(ages);
   |                        --- ^^^^ expected `Vect<_, S<S<Z>>>`, found `Vect<u32, S<Z>>`
   |                        |
   |                        arguments to this method are incorrect
   |
   = note: expected struct `Vect<_, S<S<Z>>>`
              found struct `Vect<u32, S<Z>>`
note: method defined here
  --> compile_fail/ch30_zip_lengths.rs:10:12
   |
10 |     pub fn zip<U>(self, other: Vect<U, N>) -> Vect<(T, U), N> {
   |            ^^^          ------------------
```

The limit: the trait solver does not do induction

Our `Plus` recurses on its *first* argument, the mirror image of Lean's `Nat.add`. So in Rust `Z + m = m` holds by computation (the first `impl` says so), while `n + Z = n` does not. Try to append an empty vector to a vector of unknown length and keep the type (`ch30_plus_zero.rs`):

```
pub fn append_nil<T, N: Plus<Z>>(v: Vect<T, N>) -> Vect<T, N> {
    let nil: Vect<T, Z> = Vect(Vec::new(), PhantomData);
    append(v, nil)
}
```

```
error[E0308]: mismatched types
  --> compile_fail/ch30_plus_zero.rs:30:5
   |
28 | pub fn append_nil<T, N: Plus<Z>>(v: Vect<T, N>) -> Vect<T, N> {
   |                         -                           ---------- expected `Vect<T, N>`
because of return type
   |                         |
   |                         expected this type parameter
29 |     let nil: Vect<T, Z> = Vect(Vec::new(), PhantomData);
30 |     append(v, nil)
   |     ^^^^^^^^^^^^^^ expected `Vect<T, N>`, found `Vect<T, <N as Plus<Z>>::Sum>`
   |
   = note: expected struct `Vect<_, N>`
              found struct `Vect<_, <N as Plus<Z>>::Sum>`
help: consider further restricting this bound
   |
28 | pub fn append_nil<T, N: Plus<Z, Sum = N>>(v: Vect<T, N>) -> Vect<T, N> {
   |                               +++++++++
```

This is the Rust twin of the failed `rfl` in the equality section. For each concrete `N` the solver would compute `<N as Plus<Z>>::Sum` and find `N`; for a *generic* `N` it cannot, because proving `n + 0 = n` for all `n` takes induction, and the trait solver only unfolds `impl` blocks. Rust has no proof terms either, so there is no `▸`. The compiler's suggestion is revealing: add the bound `N: Plus<Z, Sum = N>`, that is, *assume* the equation, and let every caller discharge it with a concrete `N`. Trait bounds are Rust's only proof obligations, and the solver is its only prover.

Σ-types in Rust: hiding the index

Vectors of different lengths have different types, so to store them together we must forget the length: we need $\Sigma n.\,\texttt{Vect}\langle T, n\rangle$. A trait object provides it:

```
/// `Σ n. Vect<T, n>`: a vector of *some* length. The trait object hides the
/// type-level length; `len` still reports it, computed from the hidden type.
trait SomeVect<T> {
    fn len(&self) -> usize;
    fn items(&self) -> &[T];
}

impl<T, N: Nat> SomeVect<T> for Vect<T, N> {
    fn len(&self) -> usize {
        N::VALUE
    }

    fn items(&self) -> &[T] {
        &self.items
    }
}
```

```
// Vectors of different lengths have different types, so they cannot share
// a `Vec`, unless the length is hidden behind a trait object.
let bag: Vec<Box<dyn SomeVect<u8>>> = vec![
    Box::new(Vect::nil().push(7)),
    Box::new(zs),
    Box::new(Vect::<u8, Z>::nil()),
];
for v in &bag {
    println!("some vector of length {}: {:?}", v.len(), v.items());
}
```

```
some vector of length 1: [7]
some vector of length 5: [1, 2, 3, 4, 5]
some vector of length 0: []
```

The trait object is an existential package (Chapter 15): it hides `Vect<u8, N>`, and with it `N`, but its vtable still computes `len` from the hidden type. Rust also has a built-in Σ-type over lengths that you use every day: the slice. Coercing `&[i64; 3]` to `&[i64]` moves the length from the type into the value, next to the pointer:

```
// A slice is a dependent pair: a length, and that many elements. Coercing
// `&[i64; 3]` to `&[i64]` moves the length from the type into the value,
// so the reference doubles in size.
let array: [i64; 3] = [1, 2, 3];
let slice: &[i64] = &array;
println!(
    "size_of::<&[i64; 3]>() = {}, size_of::<&[i64]>() = {}, slice.len() = {}",
    size_of::<&[i64; 3]>(),
    size_of::<&[i64]>(),
    slice.len()
);
```

```
size_of::<&[i64; 3]>() = 8, size_of::<&[i64]>() = 16, slice.len() = 3
```

The slice reference is twice as large because it *is* a pair: a length n and a pointer to n elements. `Vec<T>` is the owning version of the same Σ-type, and converting back to an array with `try_from` is the check-and-transport pattern of `toVec?`.

"Parse, don't validate": newtypes as refinement types

A **refinement type** is a type together with a predicate, such as "integers between 0 and 100"; Lean's subtypes are refinement types with proofs. Rust gets part of the benefit from a newtype with a private field (`code/rust/examples/ch30_refinements.rs`):

```
mod percent {
    /// An integer between 0 and 100. The field is private: outside this
    /// module, the only way to obtain a `Percent` is `Percent::new`.
    #[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord)]
    pub struct Percent(u8);

    impl Percent {
        pub fn new(value: u8) -> Option<Percent> {
            if value <= 100 {
                Some(Percent(value))
            } else {
                None
            }
        }
```

```
        pub fn get(self) -> u8 {
            self.0
        }

        /// Halving keeps the value in range. Nothing checks that claim: inside
        /// the module, the invariant is our responsibility.
        pub fn half(self) -> Percent {
            Percent(self.0 / 2)
        }
    }
}

use percent::Percent;
```

Outside the module `percent` the field is invisible, so every `Percent` comes from `Percent::new`, which checks the bound. Forging one is a privacy error (`ch30_private_constructor.rs`):

```
pub fn forge() -> percent::Percent {
    percent::Percent(250)
}
```

```
error[E0603]: tuple struct constructor `Percent` is private
  --> compile_fail/ch30_private_constructor.rs:19:14
   |
 4 |     pub struct Percent(u8);
   |                        -- a constructor is private if any of the fields is private
...
19 |     percent::Percent(250)
   |              ^^^^^^^ private tuple struct constructor
   |
note: the tuple struct constructor `Percent` is defined here
  --> compile_fail/ch30_private_constructor.rs:4:5
   |
 4 |     pub struct Percent(u8);
   |     ^^^^^^^^^^^^^^^^^^^^^^^
help: consider making the field publicly accessible
   |
 4 |     pub struct Percent(pub u8);
   |                        +++
```

(The compiler's suggestion, making the field public, would dissolve the invariant.) Shapes can encode invariants too: a vector that stores its first element separately cannot be empty, so its `first` and `max` need neither `Option` nor a panic:

```
/// A vector with at least one element. The first element is stored on its own,
/// so an empty `NonEmpty` cannot even be written down.
#[derive(Debug)]
struct NonEmpty<T> {
    first: T,
    rest: Vec<T>,
}

impl<T> NonEmpty<T> {
    fn from_vec(mut items: Vec<T>) -> Option<NonEmpty<T>> {
        if items.is_empty() {
            None
        } else {
            let first = items.remove(0);
            Some(NonEmpty { first, rest: items })
        }
    }

    /// Total: no `Option`, no panic.
    fn first(&self) -> &T {
        &self.first
```

```
    }

    fn len(&self) -> usize {
        1 + self.rest.len()
    }

    /// Total as well: a non-empty collection always has a largest element.
    fn max(&self) -> &T
    where
        T: Ord,
    {
        self.rest
            .iter()
            .fold(&self.first, |best, x| if x > best { x } else { best })
    }
}
```

The slogan "parse, don't validate", popularized by Alexis King, describes how to use such types. *Validating* means checking data and getting a `bool`, which is soon forgotten. *Parsing* means checking it once, at the boundary of the program, and returning a value of a more precise type that records the result:

```
/// Validation: the check returns a `bool`, and the knowledge is lost as soon
/// as the `bool` is. Every later use of the data must check again, or trust.
fn validate_scores(raw: &[u8]) -> bool {
    !raw.is_empty() && raw.iter().all(|&s| s <= 100)
}

/// Parsing: the check returns a value whose *type* records what was checked.
fn parse_scores(raw: &[u8]) -> Option<NonEmpty<Percent>> {
    let mut scores = Vec::new();
    for &s in raw {
        scores.push(Percent::new(s)?);
    }
    NonEmpty::from_vec(scores)
}

/// No checks needed: the argument's type guarantees a non-empty list of
/// percentages.
fn best(scores: &NonEmpty<Percent>) -> Percent {
    *scores.max()
}
```

```
[87, 92, 45]: valid = true, 3 scores, first 87, best 92, half of best 46
[87, 192]: valid = false, rejected by the parser
[]: valid = false, rejected by the parser
```

`best` needs no checks, because its argument's type rules out both empty input and out-of-range scores: the same move as `toVec?`, which turned a test into a more precise type.

The difference is who checks the invariant *inside* the abstraction. `Percent::half` builds `Percent(self.0 / 2)` directly, and nothing verifies that the result is in range; if someone changed it to `self.0 * 2`, the program would still compile. In Lean, `half` had to prove `p.val / 2 ≤ 100`, and the doubling version would be rejected. Rust's newtype is a refinement type whose proofs are trusted, behind an encapsulation boundary; Lean's subtype is one whose proofs are checked.

30.7 Matrices with dimensions in their types

Back to Lean, for the matrices that Rust could multiply but not stack (`code/lean/Book/Ch30/Matrix.lean`). An `m × n` matrix of integers gives an entry for every row index in `Fin m` and column index in `Fin n`:

```
/-- An `m × n` matrix of integers: an entry for every row index `i : Fin m` and
column index `j : Fin n`. -/
@[ext]
structure Mat (m n : ℕ) where
  entry : Fin m → Fin n → ℤ
```

The structure gives the type a name of its own, which keeps error messages readable. `@[ext]` generates an *extensionality* lemma (matrices with equal entries are equal), which lets the `ext` tactic reduce an equation between matrices to one between entries.

```
/-- `(m × n) * (n × p) = (m × p)`: the inner dimensions are the same `n`. -/
def mul (A : Mat m n) (B : Mat n p) : Mat m p :=
  ⟨fun i k => ∑ j, A.entry i j * B.entry j k⟩

instance : HMul (Mat m n) (Mat n p) (Mat m p) := ⟨mul⟩

def transpose (A : Mat m n) : Mat n m := ⟨fun i j => A.entry j i⟩

/-- The identity matrix exists only for square shapes. -/
def one (n : ℕ) : Mat n n := ⟨fun i j => if i = j then 1 else 0⟩

/-- Stack `A` on top of `B`: the result has `m + n` rows. -/
def vstack (A : Mat m q) (B : Mat n q) : Mat (m + n) q :=
  ⟨Fin.append A.entry B.entry⟩
```

As in Rust, the inner dimensions of `mul` are one variable `n`. `∑ j, …` is Mathlib's notation for a sum over all elements of a finite type, here all `j : Fin n`. `vstack`, the operation Rust could not type, has `m + n` rows; its body uses `Fin.append`, which combines functions on `Fin m` and `Fin n` into one on `Fin (m + n)`. To compute and prove with these operations, we state how each entry is computed, as **simp lemmas**:

```
/-- How to compute an entry of each operation. The `simp` attribute lets the
`simp` tactic use these equations, left to right, on its own. -/
@[simp] theorem mul_entry (M : Mat m n) (N : Mat n p) (i : Fin m) (k : Fin p) :
    (M * N).entry i k = ∑ j, M.entry i j * N.entry j k := rfl

@[simp] theorem transpose_entry (M : Mat m n) (i : Fin n) (j : Fin m) :
    M.transpose.entry i j = M.entry j i := rfl

@[simp] theorem one_entry (i j : Fin n) :
    (one n).entry i j = if i = j then 1 else 0 := rfl
```

Each lemma holds by definition (`rfl`); its value is the `@[simp]` attribute, which adds it to the equations that `simp` uses on its own, left to right. Chapter 31 says more about simp lemmas.

```
def A : Mat 2 3 := ⟨![![1, 2, 3], ![4, 5, 6]]⟩
def B : Mat 3 2 := ⟨![![7, 8], ![9, 10], ![11, 12]]⟩
```

```
/-- info: [[58, 64], [139, 154]] -/
#guard_msgs in
#eval (A * B).toLists
```

```
/-- info: [[39, 54, 69], [49, 68, 87], [59, 82, 105]] -/
#guard_msgs in
#eval (B * A).toLists

/-- info: [[1, 2, 3], [4, 5, 6], [1, 4], [2, 5], [3, 6]] -/
#guard_msgs in
#eval A.toLists ++ A.transpose.toLists

/-- info: [[1, 2, 3], [4, 5, 6], [1, 2, 3], [4, 5, 6]] -/
#guard_msgs in
#eval (A.vstack A).toLists
```

Mathlib's notation `![…]` gives a function on `Fin n` by its values. The results agree with the Rust program, and a mismatch is rejected as in Rust:

```
/--
error: Application type mismatch: The last
  A
argument has type
  Mat 2 3
but is expected to have type
  Mat 3 ?m.5
in the application
  A.mul A
-/
#guard_msgs in
example := A.mul A
```

Rust's example *tested* the identity and transpose laws on one pair of matrices. In Lean we prove them for all matrices of all sizes:

```
theorem transpose_transpose (M : Mat m n) : M.transpose.transpose = M := rfl

theorem transpose_mul (M : Mat m n) (N : Mat n p) :
    (M * N).transpose = N.transpose * M.transpose := by
  ext k i
  -- goal: (M * N).transpose.entry k i = (N.transpose * M.transpose).entry k i
  simp only [transpose_entry, mul_entry]
  -- goal: ∑ j, M.entry i j * N.entry j k = ∑ j, N.entry j k * M.entry i j
  simp only [Int.mul_comm]

theorem mul_one (M : Mat m n) : M * one n = M := by
  ext i j
  -- goal after the simp lemmas: ∑ k, M.entry i k * (if k = j then 1 else 0) = M.entry i j
  simp

theorem mul_assoc (L : Mat m n) (M : Mat n p) (N : Mat p q) :
    (L * M) * N = L * (M * N) := by
  ext i l
  simp only [mul_entry]
  -- goal: ∑ k, (∑ j, L i j * M j k) * N k l = ∑ j, L i j * ∑ k, M j k * N k l
  simp only [Finset.sum_mul, Finset.mul_sum]   -- push the products inside the sums
  rw [Finset.sum_comm]                          -- swap the order of summation
  simp only [Int.mul_assoc]
```

- `transpose_transpose` holds by definition: transposing twice swaps the indices back, and `rfl` sees it.
- `transpose_mul` starts with `ext k i`, which reduces the equation between matrices to one between their entries at row `k` and column `i`. The entry lemmas compute both sides, and rewriting with the commutativity of integer multiplication, `Int.mul_comm`, makes the two sums identical.

- `mul_one` is a single `simp`: after the entry lemmas, the goal is a sum of `M.entry i k * (if k = j then 1 else 0)` over all `k`, and Mathlib's simp set knows that such a sum picks out the term `k = j`.
- `mul_assoc` is the real work. `Finset.sum_mul` and `Finset.mul_sum` distribute the products into the sums, so both sides become double sums; `Finset.sum_comm` swaps the order of summation on the left; `Int.mul_assoc` re-brackets the products. Each step is a Mathlib lemma, applied by name.

Stacking distributes over multiplication, and the proof shows how to reason about an index in `Fin (m + n)`:

```
/-- Multiplying a stack multiplies each block. -/
theorem vstack_mul (A₁ : Mat m n) (A₂ : Mat p n) (B : Mat n q) :
    (vstack A₁ A₂) * B = vstack (A₁ * B) (A₂ * B) := by
  ext i k
  -- a row index of the stack is a row of `A₁` or a row of `A₂`
  cases i using Fin.addCases with
  | left i => simp [vstack]
  | right i => simp [vstack]
```

`cases i using Fin.addCases` splits a row index of the stacked matrix into two cases: a row `i` of the top block (the index `Fin.castAdd`, numerically `i`) and a row `i` of the bottom block (the index `Fin.natAdd`, numerically `m + i`). In each case Mathlib's simp lemmas about `Fin.append` select the right block, and the entry lemmas finish.

In Mathlib

Mathlib defines `Matrix m n α := m → n → α`. The difference from our `Mat` is that the indices `m` and `n` are arbitrary *types* (finite ones, for multiplication), not numbers: `Matrix (Fin 2) (Fin 3) ℤ` is our `Mat 2 3`. The notation `!![…]` separates rows with semicolons:

```
/-- info: !![58, 64; 139, 154] -/
#guard_msgs in
#eval !![1, 2, 3; 4, 5, 6] * !![7, 8; 9, 10; 11, 12]
```

Our multiplication is Mathlib's, entry by entry, and that lets us borrow Mathlib's theorems:

```
/-- Our matrices are Mathlib's, with the dimensions turned into index types. -/
def toMatrix (M : Mat m n) : Matrix (Fin m) (Fin n) ℤ := M.entry

theorem toMatrix_mul (M : Mat m n) (N : Mat n p) :
    toMatrix (M * N) = toMatrix M * toMatrix N := by
  ext i k
  rw [Matrix.mul_apply]   -- Mathlib's definition of an entry of a product
  rfl                     -- ... is ours, entry by entry
```

```
theorem toMatrix_injective {M N : Mat m n} (h : toMatrix M = toMatrix N) : M = N :=
  Mat.ext h

/-- Associativity again, this time borrowed from Mathlib's `Matrix.mul_assoc`. -/
theorem mul_assoc' (L : Mat m n) (M : Mat n p) (N : Mat p q) :
    (L * M) * N = L * (M * N) := by
  apply toMatrix_injective
  rw [toMatrix_mul, toMatrix_mul, toMatrix_mul, toMatrix_mul]
  exact Matrix.mul_assoc (toMatrix L) (toMatrix M) (toMatrix N)
```

`apply toMatrix_injective` reduces the goal to an equation between Mathlib matrices, `rw` pushes `toMatrix` inside the products, and Mathlib's `Matrix.mul_assoc` closes the goal: a pattern you will use often with libraries.

Indexing by types instead of numbers has a beautiful consequence for stacking. Mathlib's version of `vstack` is `Matrix.fromRows`, and its result is indexed by the *sum type* of the two row types:

```
/--
info: Matrix.fromRows.{u_1, u_3, u_4, u_5} {R : Type u_1} {m₁ : Type u_3} {m₂ : Type u_4}
{n : Type u_5} (A₁ : Matrix m₁ n R)
  (A₂ : Matrix m₂ n R) : Matrix (m₁ ⊕ m₂) n R
-/
#guard_msgs in
#check Matrix.fromRows
```

A row of the stack is either a row of the top block (`Sum.inl i`) or a row of the bottom block (`Sum.inr i`), and no arithmetic appears in the type at all: the `m + n` of our `vstack` has become `m₁ ⊕ m₂`, and the case split that `Fin.addCases` performed is now an ordinary `match` on a sum. Mathlib's lemma `Matrix.fromRows_mul` is the analogue of our `vstack_mul`. This is the algebra of types from Chapter 11 at work: $|m_1 \oplus m_2| = |m_1| + |m_2|$, so the sum type is the type-level version of addition, and it needs no equations about `+` to use.

30.8 Putting it together: shapes from data

Let us combine everything into one small program: read two matrices whose sizes are known only at run time, and multiply them if their shapes allow it. In Rust, `Matrix<R, C>` cannot help, because `R` and `C` must be fixed at compile time; a Rust program would use a dynamic matrix type with a run-time check in every multiplication. Lean keeps the static guarantee even though the shapes come from data. The first step reads a list of rows into a matrix of a given shape, if it has that shape:

```
/-- The rows, for printing. -/
def toLists (A : Mat m n) : List (List ℤ) :=
  List.ofFn fun i => List.ofFn fun j => A.entry i j

/-- From run-time data: succeeds only if the shape is exactly `m × n`. -/
def ofLists? (m n : ℕ) (rows : List (List ℤ)) : Option (Mat m n) :=
  if h : rows.length = m ∧ ∀ r ∈ rows, r.length = n then
    some ⟨fun i j =>
      let row := rows[i.val]'(by omega)
      row[j.val]'(by
        have hr : row.length = n := h.2 row (List.getElem_mem _)
        omega)⟩
  else
    none
```

`ofLists?` tests a decidable proposition: there are `m` rows, and every row has length `n`. In the `then` branch its proof `h` is used twice: `omega` finds the first half in the context to prove that row index `i` is in range, and `h.2` applied to the selected row proves that column index `j` is. Without the test, the indexing would not type-check.

```
/-- info: some [[1, 2, 3], [4, 5, 6]] -/
#guard_msgs in
#eval (ofLists? 2 3 [[1, 2, 3], [4, 5, 6]]).map toLists
```

```
/-- info: none -/
#guard_msgs in
#eval (ofLists? 2 3 [[1, 2, 3], [4, 5]]).map toLists
```

The second step hides the shape, because before reading the data we do not know it: a matrix of *some* shape is a Σ-type with two indices. Multiplying two such matrices tests the inner dimensions, transports, and multiplies:

```
/-- A matrix of some shape, known only at run time: a Σ-type with two
indices. -/
abbrev SomeMat := (m : ℕ) × (n : ℕ) × Mat m n

/-- Read the shape from the data, then check that the data has that shape. -/
def SomeMat.ofLists (rows : List (List ℤ)) : Option SomeMat :=
  let m := rows.length
  let n := (rows.headD []).length
  (Mat.ofLists? m n rows).map fun M => ⟨m, n, M⟩

/-- Multiply two matrices of unknown shapes. The test `h : n' = n` is decided
at run time; its proof turns `B : Mat n' p` into a `Mat n p`, and from there on
the dimension-checked `*` takes over. -/
def SomeMat.mul : SomeMat → SomeMat → Option SomeMat
  | ⟨m, n, A⟩, ⟨n', p, B⟩ =>
    if h : n' = n then some ⟨m, p, A * (h ▸ B)⟩ else none

def SomeMat.toLists (M : SomeMat) : List (List ℤ) := M.2.2.toLists

/-- Parse both inputs and multiply them, or explain what went wrong. -/
def multiplyText (xs ys : List (List ℤ)) : String :=
  match SomeMat.ofLists xs, SomeMat.ofLists ys with
  | some A, some B =>
    match A.mul B with
    | some C => s!"{A.1}x{A.2.1} * {B.1}x{B.2.1} = {C.toLists}"
    | none => s!"cannot multiply {A.1}x{A.2.1} by {B.1}x{B.2.1}"
  | _, _ => "ragged input"
```

```
/-- info: "2x3 * 3x2 = [[58, 64], [139, 154]]" -/
#guard_msgs in
#eval multiplyText [[1, 2, 3], [4, 5, 6]] [[7, 8], [9, 10], [11, 12]]

/-- info: "cannot multiply 2x3 by 2x3" -/
#guard_msgs in
#eval multiplyText [[1, 2, 3], [4, 5, 6]] [[1, 2, 3], [4, 5, 6]]

/-- info: "ragged input" -/
#guard_msgs in
#eval multiplyText [[1, 2, 3], [4, 5]] [[1], [2], [3]]
```

Follow the types through `SomeMat.mul`. The arguments are dependent pairs `⟨m, n, A⟩` and `⟨n', p, B⟩` with `A : Mat m n` and `B : Mat n' p`, so `A * B` does not type-check: `n` and `n'` are different variables. The dependent `if` decides `n' = n` at run time, and in the `then` branch its proof `h` transports `B` to type `Mat n p`; from there on, the multiplication is the dimension-checked `*` whose laws we proved for all sizes. Every check happens once, where the data enters, and its result is recorded in a type. This is the pattern of the whole chapter: Σ for data whose shape is discovered at run time, decidable tests that produce proofs, transport into the precise types, and Π-typed functions that rely on them.

30.9 Common confusions

"**Dependent types means types are computed at run time.**" No. Type checking still happens before the program runs, and types are still erased afterwards. The type checker reasons about values *symbolically*: it knows that `replicate n x` has length `n` for every `n`, without knowing `n`. Run-time values enter types only through variables, as in `toVec?`.

"**If `rfl` fails, the equation is false.**" `rfl` only checks definitional equality: whether the two sides compute to the same thing. `0 + n = n` is true and needs a proof by induction. When `rfl` fails on a statement you believe, look at how the functions involved recurse.

"**`(5 : Fin 3)` is a type error.**" It is not: numerals of type `Fin n` wrap around modulo `n`. The type promises that an index is in range, not that it is the index you meant. Use `(5, proof)` when an out-of-range number should be an error.

"**Const generics are dependent types.**" They are a first step. The values in Rust's types must be known at compile time, may only be integers, `bool` or `char`, cannot be combined with arithmetic on stable Rust, and come with no proofs. Lean's indices can be any terms, including variables bound at run time, with the whole apparatus of propositional equality.

"**Σ and ∃ are the same thing.**" They correspond under Curry–Howard, but Lean keeps them apart (Chapter 10): from a `Σ n, Vec α n` you can extract the length and compute with it, from a proof of `∃ n, P n` you cannot.

30.10 Summary

- A type **depends** on a term when it mentions it. A function that returns types is a **type family**; Lean has them natively because `Type` is itself a type.
- **Π-types** $\Pi x : A.\, B$ are dependent functions; their elimination rule substitutes the argument into the result type. $A \to B$ is the special case where B does not mention x. Under Curry–Howard Π is ∀.
- **Σ-types** $\Sigma x : A.\, B$ are dependent pairs; $A \times B$ is the special case. A Rust enum is a Σ-type over its finite set of variants; slices and `Vec` are Σ-types over lengths; trait objects hide indices existentially.
- In an inductive family, **parameters** are uniform across constructors and **indices** are chosen by each constructor. Dependent pattern matching refines indices and discards impossible cases, which makes `head` total and `zip` length-preserving.
- `Fin n` and Lean's `Vector α n` turn bounds checks into proof obligations; `v[i]`, `v[i]'h`, `v[i]?`, `v[i]!` and the dependent `if h : …` give the choice between static proof, `Option`, panic and run-time test with evidence.
- Equality is an inductive family with one constructor; its elimination rule is **transport** (`▸`, `subst`, `rw`). **Definitional** equality is decided by computation and depends on how functions recurse (`n + 0 = n` by `rfl`, `0 + n = n` by induction); **propositional** equality is anything you can prove. Indices that follow the recursion avoid transports.

- In the **λ-cube**, Rust has λ2 and fragments of λω and λP; Lean's type theory goes beyond the corner λC with inductive types and universes.
- Rust's approximations: const generics (no arithmetic, no proofs on stable), type-level Peano numerals (computation by trait resolution, no induction), Σ-types as slices and trait objects, and newtypes with private fields as refinement types whose invariants are trusted rather than proved.

30.11 Exercises

1. **Reverse, twice.** In a scratch copy of `Vectors.lean`, define `snoc : Vec α n → α → Vec α (n + 1)`, which adds an element at the end, and use it to define `reverse : Vec α n → Vec α n`. Then try to define the fast version `revAcc : Vec α n → Vec α m → Vec α (n + m)` with an accumulator, and explain the error. *Hint:* `snoc` and `reverse` type-check without any proof, because their indices follow the recursion. For `revAcc`, the recursive call `revAcc xs (cons x acc)` has length `n + (m + 1)`, but the clause needs `(n + 1) + m` (Lean's message may point at `xs` instead, because it matches the expected type first). Transport along `Nat.succ_add_eq_add_succ n m`, which states `n.succ + m = n + m.succ`, and in the `nil` case along `Nat.zero_add m`.
2. **Type-level multiplication.** In `ch30_type_level_nat.rs`, define a trait `Times<M>` with an associated type `Product`, using the equations $0 \cdot m = 0$ and $(n+1) \cdot m = m + n \cdot m$. Check with `short_name` that the compiler computes $2 \cdot 3 = 6$. *Hint:* the second `impl` needs two bounds, one for the recursive product and one to add `M` to it: `impl<N: Times<M>, M> Times<M> for S<N> where M: Plus<<N as Times<M>>::Product>`.
3. **The transpose of the identity.** Prove in Lean that `(Mat.one n).transpose = Mat.one n`, and that `(M * N).transpose.transpose = M * N` (one of the two needs no work at all). *Hint:* for the first, `ext i j` and then `simp [eq_comm]`; the entry of the transposed identity at `(i, j)` is `if j = i then 1 else 0`.
4. **Π and Σ over `Bool`, by hand.** Write, in Lean, functions `toPair : ((b : Bool) → Answer b) → ℕ × String` and `fromPair : ℕ × String → ((b : Bool) → Answer b)`, and prove that `toPair (fromPair p) = p`. Then do the same for `(b : Bool) × Answer b` and `ℕ ⊕ String`. On paper, explain what these two isomorphisms say about the names "product" and "sum". *Hint:* `fromPair p` is defined by pattern matching on the Boolean: `fun | true => p.1 | false => p.2`. The first round trip holds by `rfl` after `cases p`.
5. **A `NonEmpty` with a proof.** Define `NonEmptyList α := { xs : List α // xs ≠ [] }` and a total function `head : NonEmptyList α → α`. Compare with `NonEmpty<T>` in `ch30_refinements.rs`: which invariants does each version check, and where? Does the Lean version need to hide anything? *Hint:* match on the pair: `| ⟨[], h⟩ => absurd rfl h | ⟨x :: _, _⟩ => x`. In the first case `h : [] ≠ []`, and `absurd rfl h` derives anything from the contradiction.
6. **Where does the arithmetic go?** In Rust, write a function `hstack<const R: usize, const C1: usize, const C2: usize, const C: usize>` that places two matrices side by side, with the `const` assertion technique of `vstack`. In Lean, define `hstack` on

`Mat` with result type `Mat m (n₁ + n₂)`. Which of the two can be wrong for some sizes without the compiler noticing until the function is used? *Hint:* in Lean, use `Fin.append` on each row: `(fun i => Fin.append (A.entry i) (B.entry i))`.

CHAPTER 31

Proving Programs Correct

Chapter 1 ended its first look at Lean with a comparison. A Rust loop checked Gauss's formula for 10,001 numbers; a Lean theorem proved it for all of them. The chapter promised that we would come back to this division of labor and use it deliberately. This is the chapter where we do.

Chapter 30 gave us types that can say precise things: a vector of length `n`, an index smaller than the length, a value together with a proof about it. Here we push the idea to its natural end. We write ordinary programs (a sort and a binary search), state what they are supposed to do as types, prove that they do it, and run them. Then we write the same programs in Rust, the way a Rust programmer would, and test the Rust code against the verified Lean model. Along the way we need the tools that make such proofs practical: induction with the right hypothesis, termination arguments for functions that are not structurally recursive, and a working knowledge of the tactics that do the routine steps for us.

The chapter ends with a survey of the tools that verify Rust code directly, and with an honest account of what a proof gives you and what it does not.

In this chapter you will learn:

- how to write a specification as a type, and why a good specification usually needs more than one property;
- induction on lists and trees, and when the induction hypothesis must be generalized;
- structural and well-founded recursion: `termination_by`, `decreasing_by`, `partial`, fuel, and proofs by functional induction;
- a verified insertion sort and a verified binary search, stated with Mathlib's `List.SortedLE` and `List.Perm`;
- tactics in more depth: simp lemmas and `simp?`, `omega`, `linarith`, `ring`, `calc` and `grind`;
- the workflow "model in Lean, implement in Rust", with differential and property-based tests;
- which tools verify Rust code itself, and what a proof does and does not guarantee.

31.1 Specifications as types

Here are three ways to say, in Lean, that a function sorts a list of integers.

The first is the type `List ℤ → List ℤ`. It says almost nothing. The identity function has this type, and so do `List.reverse` and `fun _ => []`. A type like this is all that Rust's `fn sort(v: Vec<i64>) -> Vec<i64>` can say.

The second is a function of that type together with separate theorems about it: "for every `xs`, `sort xs` is sorted" and "for every `xs`, `sort xs` is a permutation of `xs`". This is the *extrinsic* style that Chapter 8 contrasted with intrinsic typing: the program is written first, and its properties are proved afterwards, outside its type.

The third puts the properties *in* the type, with a subtype (Chapter 30):

```
def sort (xs : List ℤ) : { ys : List ℤ // ys.SortedLE ∧ ys ~ xs }
```

A value of this type is a list together with a proof that it is sorted and a proof that it is a permutation of the input. A function of this type cannot return a wrong answer, because a wrong answer would come without its proofs. This is the *intrinsic* style, and it is the form in which "a specification is a type" becomes literal. In practice the two styles combine: we write the program and prove the theorems separately, then package them into the subtype at the end, as you will see.

Notice that the specification has two parts, and that neither is enough alone. "Sorted" alone is satisfied by `fun _ => []`. "A permutation of the input" alone is satisfied by the identity. Together they pin down the result completely: we will *prove* that there is only one sorted permutation of a list. Writing a specification is mostly the art of noticing which programs it still allows.

31.2 Induction, revisited

Proofs about recursive programs are proofs by induction, and you have written many of them since Chapter 1. Two situations need more care than the ones we have met so far. The code is in `code/lean/Book/Ch31/Induction.lean`.

Generalizing the induction hypothesis

Reversing a list by repeatedly appending is quadratic. The standard fix is an accumulator, which also makes the function tail-recursive:

```
/-- Reverse with an accumulator: tail-recursive, and linear time. -/
def revAcc : List α → List α → List α
  | [], acc => acc
  | x :: xs, acc => revAcc xs (x :: acc)

def fastReverse (xs : List α) : List α := revAcc xs []
```

We want to prove `fastReverse xs = xs.reverse`, that is, `revAcc xs [] = xs.reverse`. The obvious induction gets stuck:

```
/--
error: unsolved goals
case cons
α : Type
x : α
xs : List α
ih : revAcc xs [] = xs.reverse
```

```
⊢ revAcc xs [x] = xs.reverse ++ [x]
-/
#guard_msgs in
theorem fastReverse_eq_attempt (xs : List α) : revAcc xs [] = xs.reverse := by
  induction xs with
  | nil => rfl
  | cons x xs ih => simp only [revAcc, List.reverse_cons]
```

Look at the goal in the `cons` case. After one step, `revAcc (x :: xs) []` became `revAcc xs [x]`: the recursive call has a *different* accumulator. But the induction hypothesis `ih` talks only about `revAcc xs []`, with the empty accumulator, so it does not apply. The hypothesis is too weak, because the statement we are proving is too specific.

The cure is to prove a *more general* statement, one that holds for every accumulator, and to tell `induction` to keep the accumulator universally quantified in the hypothesis:

```
/-- The general statement: for *every* accumulator. -/
theorem revAcc_eq (xs acc : List α) : revAcc xs acc = xs.reverse ++ acc := by
  induction xs generalizing acc with
  | nil =>
    -- goal: revAcc [] acc = [].reverse ++ acc
    rfl
  | cons x xs ih =>
    -- ih   : ∀ (acc : List α), revAcc xs acc = xs.reverse ++ acc
    -- goal : revAcc (x :: xs) acc = (x :: xs).reverse ++ acc
    rw [revAcc, ih (x :: acc)]
    -- goal : xs.reverse ++ x :: acc = (x :: xs).reverse ++ acc
    simp

theorem fastReverse_eq (xs : List α) : fastReverse xs = xs.reverse := by
  rw [fastReverse, revAcc_eq, List.append_nil]
```

`induction xs generalizing acc` makes the hypothesis `ih : ∀ (acc : List α), revAcc xs acc = xs.reverse ++ acc`, a statement about *every* accumulator. In the `cons` case we unfold `revAcc` one step and use `ih` with the accumulator `x :: acc`, the one the recursive call actually uses. What remains, `xs.reverse ++ x :: acc = (x :: xs).reverse ++ acc`, is list algebra that `simp` knows (`(x :: xs).reverse` is `xs.reverse ++ [x]`, and append is associative). Then `fastReverse_eq` is the special case `acc = []`.

The rule of thumb: when a recursive call changes an argument other than the one you are recursing on, the theorem must quantify over that argument, and the induction must generalize it. You will meet this again in the binary search, where both ends of the search interval change.

Induction on trees

Induction works on any inductive type, not just lists. For a binary tree, the induction principle has one case per constructor, and the `node` case gives two hypotheses, one for each child:

```
/-- Binary trees with a value at every node. -/
inductive Tree (α : Type) where
  | leaf : Tree α
  | node : Tree α → α → Tree α → Tree α

namespace Tree

def size : Tree α → ℕ
  | leaf => 0
```

```
  | node l _ r => l.size + 1 + r.size

/-- The elements from left to right. -/
def toList : Tree α → List α
  | leaf => []
  | node l x r => l.toList ++ [x] ++ r.toList

/-- Swap the children of every node. -/
def mirror : Tree α → Tree α
  | leaf => leaf
  | node l x r => node r.mirror x l.mirror
```

```
/-- The induction principle for trees gives two hypotheses, one per child. -/
theorem mirror_mirror (t : Tree α) : t.mirror.mirror = t := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    -- ihl : l.mirror.mirror = l,  ihr : r.mirror.mirror = r
    rw [mirror, mirror, ihl, ihr]

theorem length_toList (t : Tree α) : t.toList.length = t.size := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    simp only [toList, size, List.length_append, List.length_singleton, ihl, ihr]

@[simp] theorem toList_mirror (t : Tree α) : t.mirror.toList = t.toList.reverse := by
  induction t with
  | leaf => rfl
  | node l x r ihl ihr =>
    simp [mirror, toList, ihl, ihr]
```

In `mirror_mirror`, the `node` case has `ihl : l.mirror.mirror = l` and `ihr : r.mirror.mirror = r`. Unfolding `mirror` twice turns the goal `(node l x r).mirror.mirror = node l x r` into `node l.mirror.mirror x r.mirror.mirror = node l x r`, and the two hypotheses finish it. In `toList_mirror`, the in-order traversal of a mirrored tree is the reverse of the traversal of the tree; after unfolding, `simp` uses the hypotheses and the library fact that reversing `a ++ b` gives `b.reverse ++ a.reverse`.

Simp lemmas

`toList_mirror` carries the attribute `@[simp]`. That adds it to the set of equations that `simp` uses automatically, oriented from left to right, so any later goal mentioning `t.mirror.toList` gets rewritten to `t.toList.reverse` without being asked. Chapter 30 did the same with the entry lemmas of its matrices. The variant `simp?` runs `simp` and then reports which lemmas it used:

```
/-- Because `toList_mirror` is tagged `@[simp]`, `simp` uses it on its own. -/
example (t : Tree α) : t.mirror.mirror.toList = t.toList := by
  simp

/--
info: Try this:
  [apply] simp only [toList_mirror, List.reverse_reverse]
-/
#guard_msgs in
example (t : Tree α) : t.mirror.mirror.toList = t.toList := by
  simp?
```

The suggestion, `simp only [toList_mirror, List.reverse_reverse]`, is a *closed* version of the same step: it uses exactly those two lemmas and nothing else. A plain `simp` can change behavior when the library

adds new simp lemmas; `simp only` cannot, so it is the better choice in the middle of a long proof. Three guidelines for tagging your own lemmas `@[simp]`: the left side should be more complicated than the right side (so rewriting makes progress); the lemma should not loop (never tag commutativity, `a + b = b + a`, which `simp` could apply forever); and it should be a fact you want applied *every* time, such as how to compute an operation on a constructor. Related forms: `simp at h` simplifies a hypothesis, `simp [h]` adds `h` to the set for one call, and `simp_all` simplifies everything in sight.

31.3 Termination

Every Lean function must terminate. Chapter 2 introduced the rule and Chapter 30 showed one reason for it: type checking runs programs, so a non-terminating program could make the type checker loop. The other reason is logic. If Lean accepted `def loop : False := loop`, then `loop` would be a proof of `False`, and every theorem would be provable. The code for this section is in `code/lean/Book/Ch31/Termination.lean`.

Structural and well-founded recursion

The easy case is **structural recursion**: every recursive call is on a piece of the argument, as in Chapter 1's `sumTo`.

```
/-- Structural recursion: the recursive call is on `xs`, a piece of the
argument `x :: xs`. Lean accepts this without further ado. -/
def total : List ℕ → ℕ
  | [] => 0
  | x :: xs => x + total xs
```

Many natural functions are not structural. Halving a number calls itself on `n / 2`, which is smaller than `n` but not a piece of it:

```
/-- The number of times `n` can be halved before it drops below 2. The call is
on `n / 2`, which is not a piece of `n`, so the recursion is not structural. -/
def log2 (n : ℕ) : ℕ := if n < 2 then 0 else 1 + log2 (n / 2)
```

Lean accepts this anyway, by **well-founded recursion**. It looks for a *measure*, a function from the arguments to a type with a well-founded order (usually `ℕ` with `<`, where every decreasing sequence is finite), that decreases at every recursive call. Here the measure is `n` itself: in the branch that recurses, `n ≥ 2`, so `n / 2 < n`. Lean found the measure and proved the decrease on its own; `termination_by?` asks it to show what it found:

```
/--
info: Try this:
  [apply] termination_by n
-/
#guard_msgs in
def log2' (n : ℕ) : ℕ := if n < 2 then 0 else 1 + log2' (n / 2)
termination_by?
```

The proof that the measure decreases is produced by a tactic called `decreasing_tactic`, which tries simple arithmetic (including `omega`) with the facts known at the call site, such as the negated condition `¬n < 2`.

When Lean needs help

Euclid's algorithm calls itself on `a % b`:

```
/--
error: fail to show termination for
  Ch31.gcdBad
with errors
failed to infer structural recursion:
Cannot use parameter a:
  failed to eliminate recursive application
    gcdBad b (a % b)
Cannot use parameter b:
  failed to eliminate recursive application
    gcdBad b (a % b)


Could not find a decreasing measure.
The basic measures relate at each recursive call as follows:
(<, ≤, =: relation proved, ? all proofs failed, _: no proof attempted)
            a b
1) 61:49-65 ? ?
Please use `termination_by` to specify a decreasing measure.
-/
#guard_msgs in
def gcdBad (a b : ℕ) : ℕ := if b = 0 then a else gcdBad b (a % b)
```

Read the error from the top. Lean first tried structural recursion on each parameter and failed: `b` is not a piece of `a`, and `a % b` is not a piece of `b`. Then it tried the parameters as measures: the table has one row per recursive call (one call, at line 61) and one column per measure (`a`, `b`), and a `?` means that it could not prove the measure decreases. For `b` the fact is true (`a % b < b` whenever `b ≠ 0`), but proving it needs a lemma about `%` that the automatic tactic does not try. So we say which measure to use, and prove the decrease ourselves:

```
/-- Euclid's algorithm. The measure is `b`; the proof that it decreases is
`a % b < b`, which holds because `b ≠ 0` in the branch that recurses. -/
def euclid (a b : ℕ) : ℕ := if b = 0 then a else euclid b (a % b)
termination_by b
decreasing_by
  -- goal: a % b < b, with `h✝ : ¬b = 0` in the context
  apply Nat.mod_lt     -- `Nat.mod_lt : y > 0 → x % y < y`; remaining goal: b > 0
  omega                -- follows from `¬b = 0`
```

`termination_by b` names the measure. `decreasing_by` replaces the automatic proof with ours. The goal is `a % b < b`, and the context contains the negated condition of the `if` (Lean adds it automatically, as an anonymous hypothesis). `apply Nat.mod_lt` uses the library lemma `x % y < y` for `y > 0`, leaving the goal `b > 0`, and `omega` proves it from `¬b = 0`.

```
/-- info: (6, 1, 21) -/
#guard_msgs in
#eval (euclid 48 18, euclid 17 5, euclid 0 21)
```

Some functions need a measure with more than one component. In Ackermann's function, the first argument sometimes stays the same while the second grows enormously. What decreases is the *pair*, in lexicographic order: either the first component decreases, or it stays the same and the second decreases.

```
/-- Ackermann's function: neither argument decreases on every call, but the
pair does, in lexicographic order. -/
```

```
def ack : ℕ → ℕ → ℕ
  | 0, n => n + 1
  | m + 1, 0 => ack m 1
  | m + 1, n + 1 => ack m (ack (m + 1) n)
termination_by m n => (m, n)
```

```
/-- info: [3, 5, 5, 61] -/
#guard_msgs in
#eval [ack 1 1, ack 2 1, ack 3 0, ack 3 3]
```

What `partial` costs

Sometimes no measure exists, or nobody knows one. Whether the Collatz process (halve even numbers, map odd `n` to `3n + 1`) reaches 1 from every start is a famous open problem, so no termination proof can be given. `partial` tells Lean to skip the check:

```
/-- Nobody knows whether this terminates for every `n` (the Collatz
conjecture), so no measure can be given. `partial` skips the check. -/
partial def collatz (n : ℕ) : ℕ :=
  if n ≤ 1 then 0
  else if n % 2 = 0 then 1 + collatz (n / 2)
  else 1 + collatz (3 * n + 1)

/-- info: 111 -/
#guard_msgs in
#eval collatz 27
```

The function runs. But for the logic, a `partial` function is an opaque constant: Lean knows that it exists and has the right type (that is why a `partial` function's result type must be inhabited, Chapter 10), but not what it computes. Nothing can be proved about it, not even a fact that `#eval` confirms:

```
/--
error: Type mismatch
  rfl
has type
  ?m.7 = ?m.7
but is expected to have type
  collatz 6 = 8
-/
#guard_msgs in
example : collatz 6 = 8 := rfl
```

The way out is **fuel**: an extra argument that bounds the number of steps and decreases structurally. The function becomes total, returning `none` when the fuel runs out, and Lean can compute with it inside proofs:

```
/-- The same search with fuel: at most `fuel` steps, then give up with
`none`. Structural recursion on `fuel`, so the function is total, and Lean can
compute with it inside proofs. -/
def collatzFuel : ℕ → ℕ → Option ℕ
  | 0, _ => none
  | fuel + 1, n =>
    if n ≤ 1 then some 0
    else (collatzFuel fuel (if n % 2 = 0 then n / 2 else 3 * n + 1)).map (· + 1)

example : collatzFuel 10 6 = some 8 := by decide
example : collatzFuel 5 6 = none := by decide
```

(Chapter 7 used fuel the same way for the normalizer of the λ-calculus.) The price is that a theorem about `collatzFuel` must say something about the fuel, typically "if the result is `some k`, then ...".

Proving things about well-founded definitions

A function defined by well-founded recursion comes with its own induction principle, which follows the function's recursive structure:

```
/--
info: Ch31.log2.induct (motive : ℕ → Prop) (case1 : ∀ (x : ℕ), x < 2 → motive x)
  (case2 : ∀ (x : ℕ), ¬x < 2 → motive (x / 2) → motive x) (n : ℕ) : motive n
-/
#guard_msgs in
#check log2.induct
```

To prove `motive n` for every `n`, it is enough to prove it in the base case (`x < 2`) and, in the recursive case, to prove it for `x` assuming it for `x / 2`. This is **functional induction**, and the tactic `fun_induction` uses it. It also unfolds the function in the goal, case by case:

```
/-- `log2` really is a logarithm: `2 ^ log2 n ≤ n` for every positive `n`. -/
theorem pow_log2_le (n : ℕ) (h : 1 ≤ n) : 2 ^ log2 n ≤ n := by
  fun_induction log2 n with
  | case1 n hlt =>
    -- `log2 n` was unfolded to `0`; goal: 2 ^ 0 ≤ n
    simp only [pow_zero]
    exact h
  | case2 n hge ih =>
    -- ih   : 1 ≤ n / 2 → 2 ^ log2 (n / 2) ≤ n / 2
    -- goal : 2 ^ (1 + log2 (n / 2)) ≤ n
    calc 2 ^ (1 + log2 (n / 2))
        = 2 * 2 ^ log2 (n / 2) := by ring
      _ ≤ 2 * (n / 2) := by
          have h2 := ih (by omega)   -- n ≥ 2, so n / 2 ≥ 1
          omega
      _ ≤ n := by omega
```

In the first case `log2 n` has become `0`, so the goal is `2 ^ 0 ≤ n`; `simp only [pow_zero]` turns `2 ^ 0` into `1`, and the hypothesis `h : 1 ≤ n` closes it. The second case is a `calc` block, a proof written as a chain of relations, each with its own justification:

- `2 ^ (1 + log2 (n / 2)) = 2 * 2 ^ log2 (n / 2)` by `ring`, which proves equations in commutative rings and knows how exponents add;
- `_ ≤ 2 * (n / 2)` by the induction hypothesis, which needs `1 ≤ n / 2` (true, because `n ≥ 2`) and which `omega` then scales by 2;
- `_ ≤ n` by `omega`, which knows how `/` by a literal behaves.

The underscore `_` at the start of each line stands for the right side of the previous line. `calc` lets a proof read the way you would write it on paper, and Lean chains the steps with transitivity.

31.4 A verified insertion sort

We now have the tools for the main example. The code is in `code/lean/Book/Ch31/Sorting.lean`.

```
/-- Insert `x` into a list, before the first element that is at least `x`.
If the list is sorted, so is the result. -/
def insertSorted (x : ℤ) : List ℤ → List ℤ
  | [] => [x]
  | y :: ys => if x ≤ y then x :: y :: ys else y :: insertSorted x ys

/-- Sort the tail, then insert the head. -/
```

```
def isort : List ℤ → List ℤ
  | [] => []
  | x :: xs => insertSorted x (isort xs)
```

```
/-- info: [-3, 1, 2, 5, 5, 9] -/
#guard_msgs in
#eval isort [5, 2, 9, -3, 5, 1]
```

The specification, in Mathlib's vocabulary

For "is a permutation of" we use `List.Perm` from Lean's core library, written `l₁ ~ l₂` (the notation lives in the `List` namespace, so the file opens it with `open scoped List`). It is an inductive relation with four constructors:

```
/--
info: inductive List.Perm.{u} : {α : Type u} → List α → List α → Prop
number of parameters: 1
constructors:
List.Perm.nil : ∀ {α : Type u}, [] ~ []
List.Perm.cons : ∀ {α : Type u} (x : α) {l₁ l₂ : List α}, l₁ ~ l₂ → x :: l₁ ~ x :: l₂
List.Perm.swap : ∀ {α : Type u} (x y : α) (l : List α), y :: x :: l ~ x :: y :: l
List.Perm.trans : ∀ {α : Type u} {l₁ l₂ l₃ : List α}, l₁ ~ l₂ → l₂ ~ l₃ → l₁ ~ l₃
-/
#guard_msgs in
#print List.Perm
```

Two empty lists are permutations of each other; adding the same head preserves the relation; swapping the first two elements is a permutation; and permutations compose. Every permutation of a list can be built from these four rules.

For "is sorted" we use Mathlib's `List.SortedLE`. It is defined as "`l.get` is a monotone function", and Mathlib proves it equivalent to the inductive form `List.Pairwise (· ≤ ·)`: every element is at most every later element. Our proofs use the `Pairwise` form, which is easier to do induction on, and convert at the end. Sortedness of a concrete list is decidable:

```
example (l : List ℤ) : l.SortedLE ↔ l.Pairwise (· ≤ ·) := List.sortedLE_iff_pairwise

example : [1, 2, 2, 7].SortedLE := by decide
example : ¬ [1, 3, 2].SortedLE := by decide
```

> **NOTE**
>
> Older versions of Mathlib, and much of the material you will find online, use `List.Sorted r l`, with an explicit relation argument. In the Mathlib version pinned by this book, sortedness is expressed with `List.SortedLE`, `List.SortedLT` (and their duals) or directly with `List.Pairwise`. If a name from an older text does not exist, search `.lake/packages/mathlib/Mathlib/Data/List/Sort.lean` for its replacement.

Insertion produces a permutation

```
/-- Inserting `x` gives a permutation of `x :: l`. -/
theorem perm_insertSorted (x : ℤ) (l : List ℤ) : insertSorted x l ~ x :: l := by
  induction l with
  | nil => exact List.Perm.refl [x]
```

```
  | cons y ys ih =>
    -- ih : insertSorted x ys ~ x :: ys
    rw [insertSorted]
    split
    · -- case x ≤ y: the result is literally x :: y :: ys
      exact List.Perm.refl _
    · -- case ¬ x ≤ y: goal y :: insertSorted x ys ~ x :: y :: ys
      calc y :: insertSorted x ys
          ~ y :: x :: ys := List.Perm.cons y ih
        _ ~ x :: y :: ys := List.Perm.swap x y ys

theorem mem_insertSorted {x z : ℤ} {l : List ℤ} :
    z ∈ insertSorted x l ↔ z = x ∨ z ∈ l := by
  rw [(perm_insertSorted x l).mem_iff, List.mem_cons]
```

The proof is by induction on `l`. In the `nil` case, `insertSorted x []` is `[x]`, and every list is a permutation of itself (`List.Perm.refl`). In the `cons y ys` case, `rw [insertSorted]` unfolds one step, which exposes the `if`, and `split` makes one goal for each branch. When `x ≤ y`, the result is literally `x :: y :: ys`, so reflexivity again. When `x > y`, the result is `y :: insertSorted x ys`, and a `calc` with `~` instead of `≤` chains two permutations: the induction hypothesis under the head `y` (`List.Perm.cons`), then a swap of the first two elements (`List.Perm.swap`). `calc` works with any relation that has a transitivity instance, and Lean's library provides one for `~`.

`mem_insertSorted` follows at once: permutations have the same members (`Perm.mem_iff`), and membership in `x :: l` means being `x` or being in `l`.

Insertion preserves sortedness

This is the longest proof of the chapter, and the most typical: a case analysis that mirrors the program, with a small argument in each case.

```
/-- Inserting into a sorted list gives a sorted list. -/
theorem sorted_insertSorted (x : ℤ) (l : List ℤ) (h : l.Pairwise (· ≤ ·)) :
    (insertSorted x l).Pairwise (· ≤ ·) := by
  induction l with
  | nil => simp [insertSorted]
  | cons y ys ih =>
    -- h : (y :: ys).Pairwise (· ≤ ·), which unfolds to two facts:
    rw [List.pairwise_cons] at h
    obtain ⟨hy, hys⟩ := h       -- hy : ∀ b ∈ ys, y ≤ b    hys : ys.Pairwise (· ≤ ·)
    rw [insertSorted]
    split
    · -- case x ≤ y: goal (x :: y :: ys).Pairwise (· ≤ ·)
      rename_i hxy
      rw [List.pairwise_cons, List.pairwise_cons]
      refine ⟨?_, hy, hys⟩
      -- goal: ∀ b ∈ y :: ys, x ≤ b
      intro b hb
      rw [List.mem_cons] at hb
      rcases hb with rfl | hb
      · exact hxy                       -- b = y
      · calc x ≤ y := hxy               -- b ∈ ys
            _ ≤ b := hy b hb
    · -- case ¬ x ≤ y: goal (y :: insertSorted x ys).Pairwise (· ≤ ·)
      rename_i hxy
      rw [List.pairwise_cons]
      refine ⟨?_, ih hys⟩
      -- goal: ∀ b ∈ insertSorted x ys, y ≤ b
      intro b hb
      rw [mem_insertSorted] at hb
      rcases hb with rfl | hb
```

```
      · omega                                -- b = x, and ¬ x ≤ y
      · exact hy b hb
```

Step by step:

- `rw [List.pairwise_cons] at h` rewrites the hypothesis with the library equivalence `(a :: l).Pairwise R ↔ (∀ b ∈ l, R a b) ∧ l.Pairwise R`, and `obtain ⟨hy, hys⟩ := h` splits the conjunction into its two halves.
- `rw [insertSorted]` and `split` expose the two branches of the program, as before. `rename_i hxy` gives a name to the branch condition, which `split` introduced as an anonymous hypothesis.
- In the branch `x ≤ y` the goal is that `x :: y :: ys` is sorted. Rewriting with `pairwise_cons` twice turns it into three facts: `x` is at most every element of `y :: ys`, `y` is at most every element of `ys`, and `ys` is sorted. `refine ⟨?_, hy, hys⟩` supplies the last two from the hypotheses and leaves the first as a new goal (`?_` marks the hole). For an element `b` of `y :: ys`, either `b = y` (then `hxy` is the proof; `rcases hb with rfl | hb` substitutes `y` for `b`), or `b ∈ ys`, and a two-step `calc` chains `x ≤ y` and `y ≤ b`.
- In the branch `¬ x ≤ y` the result is `y :: insertSorted x ys`. The tail is sorted by the induction hypothesis `ih hys`. For the head, every element `b` of `insertSorted x ys` is either `x`, and then `y ≤ x` follows from `¬ x ≤ y` by `omega`, or an element of `ys`, and then `hy` applies.

Putting the two lemmas together gives the correctness of the sort:

```
theorem isort_perm (xs : List ℤ) : isort xs ~ xs := by
  induction xs with
  | nil => exact List.Perm.nil
  | cons x xs ih =>
    -- goal: insertSorted x (isort xs) ~ x :: xs
    calc insertSorted x (isort xs)
        ~ x :: isort xs := perm_insertSorted x (isort xs)
      _ ~ x :: xs := List.Perm.cons x ih

theorem isort_pairwise (xs : List ℤ) : (isort xs).Pairwise (· ≤ ·) := by
  induction xs with
  | nil => exact List.Pairwise.nil
  | cons x xs ih => exact sorted_insertSorted x (isort xs) ih

theorem isort_sorted (xs : List ℤ) : (isort xs).SortedLE :=
  (isort_pairwise xs).sortedLE
```

and the specification as a type:

```
/-- The specification as a type: a list, together with proofs that it is
sorted and that it is a permutation of the input. -/
def sort (xs : List ℤ) : { ys : List ℤ // ys.SortedLE ∧ ys ~ xs } :=
  ⟨isort xs, isort_sorted xs, isort_perm xs⟩
```

```
/-- info: [-3, 1, 2, 5, 5, 9] -/
#guard_msgs in
#eval (sort [5, 2, 9, -3, 5, 1]).val
```

`.val` extracts the list from the subtype; the two proofs travel with it during type checking and vanish when the program is compiled.

Letting `grind` do it

Chapter 1 used `grind` once, to finish Gauss's formula. It is Lean's most powerful general-purpose automation: it collects the facts in the context and the lemmas you give it, follows equalities (congruence closure), splits on `if`s and on disjunctions, instantiates the lemmas where they match, and does linear arithmetic. Given the definition and the two library facts we used by hand, it proves the sortedness lemma in one line:

```
/-- The same theorem as `sorted_insertSorted`, with the case analysis, the
membership reasoning and the arithmetic left to `grind`. -/
theorem sorted_insertSorted' (x : ℤ) (l : List ℤ) (h : l.Pairwise (· ≤ ·)) :
    (insertSorted x l).Pairwise (· ≤ ·) := by
  induction l with
  | nil => simp [insertSorted]
  | cons y ys ih => grind [insertSorted, List.pairwise_cons, mem_insertSorted]
```

When should you write the long version? When you want to understand the proof, when you want it to survive changes to the automation, and when `grind` fails and you need to see where. When should you use `grind`? When the argument is routine and you would rather read the theorem than the proof. (Like the `grind` proof in Chapter 1, it relies on classical reasoning; Chapter 5 explained what that means, and `#print axioms` shows it.)

The specification determines the output

We claimed that "sorted" and "a permutation" together pin down the result. Here is the proof, and two consequences:

```
/-- The specification determines the output: two sorted permutations of the
same list are equal. -/
theorem sorted_perm_unique {ys zs : List ℤ} (hy : ys.SortedLE) (hz : zs.SortedLE)
    (h : ys ~ zs) : ys = zs :=
  List.Perm.eq_of_sortedLE hy hz h

/-- So our sort agrees with Mathlib's insertion sort on every input... -/
theorem isort_eq_insertionSort (xs : List ℤ) :
    isort xs = xs.insertionSort (· ≤ ·) := by
  apply sorted_perm_unique (isort_sorted xs) List.sortedLE_insertionSort
  -- goal: isort xs ~ List.insertionSort (· ≤ ·) xs
  exact (isort_perm xs).trans (List.perm_insertionSort _ xs).symm

/-- ... and with the merge sort of Lean's core library. -/
theorem isort_eq_mergeSort (xs : List ℤ) : isort xs = xs.mergeSort := by
  apply sorted_perm_unique (isort_sorted xs) List.sortedLE_mergeSort
  exact (isort_perm xs).trans (List.mergeSort_perm xs _).symm
```

`sorted_perm_unique` is Mathlib's `List.Perm.eq_of_sortedLE`: two sorted lists that are permutations of each other are equal. So *any* function that meets our specification computes exactly the same function as `isort`. Mathlib's `List.insertionSort` meets it (Mathlib proves sortedness and permutation for it), and so does core Lean's `List.mergeSort`; therefore, for every input, `isort xs` equals both. We have proved two different library sorting algorithms equal to ours without looking at how they work, only at what they promise.

This fact will matter for testing. If the specification determines the output, then *comparing outputs* with a verified model is the same as checking the specification. When it does not, as we will see with binary search, comparing outputs can reject correct implementations.

How many comparisons?

The specification says nothing about cost. Cost is a separate property, and we can prove it separately. We count the comparisons that `insertSorted` and `isort` make, with functions that follow their recursion:

```
/-- The number of comparisons `insertSorted x l` makes. -/
def insertCost (x : ℤ) : List ℤ → ℕ
  | [] => 0
  | y :: ys => if x ≤ y then 1 else 1 + insertCost x ys

/-- The number of comparisons `isort` makes. -/
def isortCost : List ℤ → ℕ
  | [] => 0
  | x :: xs => insertCost x (isort xs) + isortCost xs
```

```
theorem insertCost_le (x : ℤ) (l : List ℤ) : insertCost x l ≤ l.length := by
  induction l with
  | nil => simp [insertCost]
  | cons y ys ih =>
    simp only [insertCost, List.length_cons]
    split <;> omega

/-- At most `n (n - 1) / 2` comparisons for a list of length `n`, stated
without subtraction or division. -/
theorem isortCost_le (xs : List ℤ) :
    2 * isortCost xs + xs.length ≤ xs.length * xs.length := by
  induction xs with
  | nil => simp [isortCost]
  | cons x xs ih =>
    -- one insertion into a list of length `xs.length`
    have hc : insertCost x (isort xs) ≤ xs.length := by
      have h := insertCost_le x (isort xs)
      rwa [(isort_perm xs).length_eq] at h
    rw [isortCost, List.length_cons]
    calc 2 * (insertCost x (isort xs) + isortCost xs) + (xs.length + 1)
        = (2 * isortCost xs + xs.length) + 2 * insertCost x (isort xs) + 1 := by ring
      _ ≤ xs.length * xs.length + 2 * xs.length + 1 := by linarith
      _ = (xs.length + 1) * (xs.length + 1) := by ring
```

`insertCost_le` is a routine induction: after unfolding, `split` creates one goal per branch of the `if`, and `<;> omega` closes both with the induction hypothesis.

`isortCost_le` says that the number of comparisons C for a list of length n satisfies $2C + n \le n^2$, that is, $C \le n(n-1)/2$, written without the subtraction and division that are awkward on `ℕ`. In the `cons` case the insertion costs at most `xs.length` comparisons (`hc`; note the use of `(isort_perm xs).length_eq`, since we insert into the *sorted* tail, whose length equals that of `xs`). The `calc` block then has three steps, each with the right tool:

- an equation that only rearranges terms, by `ring`;
- an inequality that follows *linearly* from `ih` and `hc`, by `linarith`;
- the polynomial identity $n^2 + 2n + 1 = (n+1)^2$, by `ring`.

The middle step is the reason for `linarith`. The goal and the hypotheses mention the product `xs.length * xs.length`, which is not linear arithmetic. `linarith` treats such a product as an opaque quantity and finds a linear combination of the hypotheses that implies the goal: here, `ih` plus twice `hc`. `omega` is the right tool for linear arithmetic with `/`, `%` and literals over `ℕ` and `ℤ`; `linarith`

for linear reasoning over ordered fields (and `ℕ` and `ℤ`), possibly with nonlinear atoms; `ring` for equations that hold as polynomial identities.

```
/-- info: (15, 5, 3) -/
#guard_msgs in
#eval (isortCost [6, 5, 4, 3, 2, 1], isortCost [1, 2, 3, 4, 5, 6], isortCost [1, 1, 1, 1])
```

The bound is attained: the reversed list of six elements needs $15 = 6 \cdot 5/2$ comparisons, and a sorted list needs only $n - 1$.

31.5 A verified binary search

Binary search is a famously easy algorithm to get subtly wrong, and a good test of the techniques so far: its recursion is not structural, and its correctness argument is an invariant about an interval. The code is in `code/lean/Book/Ch31/BinarySearch.lean`.

```
/-- Search for `x` in the index interval `[lo, hi)` of `a`. -/
def searchFrom (a : Array ℤ) (x : ℤ) (lo hi : ℕ) (hhi : hi ≤ a.size) : Option ℕ :=
  if h : lo < hi then
    let mid := lo + (hi - lo) / 2
    -- `a[mid]` below needs this fact; Lean finds it in the context.
    have hmid : mid < a.size := by omega
    if x < a[mid] then searchFrom a x lo mid (by omega)
    else if a[mid] < x then searchFrom a x (mid + 1) hi hhi
    else some mid
  else none
termination_by hi - lo

def search (a : Array ℤ) (x : ℤ) : Option ℕ :=
  searchFrom a x 0 a.size (Nat.le_refl _)
```

`searchFrom` looks for `x` in the index interval `[lo, hi)`. Its extra argument `hhi : hi ≤ a.size` is a proof, carried along so that the indexing `a[mid]` type-checks: from `lo < hi` and `hi ≤ a.size`, `omega` proves `mid < a.size`, and the `have` puts that fact in the context, where the indexing finds it (Chapter 30). The recursive calls must supply the proof for their own intervals, `mid ≤ a.size` and the unchanged `hhi`. The measure is the width of the interval, `hi - lo`, which strictly decreases on both recursive calls; Lean proves this on its own, from `lo < hi` and the definition of `mid`.

```
/-- info: [some 0, some 3, none, none, none] -/
#guard_msgs in
#eval [2, 11, 4, 1, 20].map (search #[2, 3, 7, 11, 13])
```

The midpoint is computed as `lo + (hi - lo) / 2` rather than `(lo + hi) / 2`. On Lean's `ℕ` the two are equal, since there is no overflow; we use the first form because the Rust implementation must, and it is better if the model and the implementation are the same algorithm. We return to this point below.

Soundness: a returned index is right

The first half of the specification does not need the array to be sorted:

```
/-- Soundness: a returned index is in bounds and points at `x`. -/
theorem searchFrom_some (a : Array ℤ) (x : ℤ) (lo hi : ℕ) (hhi : hi ≤ a.size) (i : ℕ) :
    searchFrom a x lo hi hhi = some i → ∃ h : i < a.size, a[i] = x := by
  fun_induction searchFrom a x lo hi hhi with
  | case1 lo hi hhi h mid hmid hlt ih =>
```

```
    -- went left: the goal is literally the induction hypothesis
    exact ih
  | case2 lo hi hhi h mid hmid hnlt hgt ih =>
    -- went right: the same
    exact ih
  | case3 lo hi hhi h mid hmid hnlt hngt =>
    -- found: the result is `some mid`
    intro hsome
    injection hsome with hmi        -- hmi : mid = i
    subst hmi
    exact ⟨hmid, by omega⟩          -- ¬ x < a[mid] and ¬ a[mid] < x give a[mid] = x
  | case4 lo hi hhi h =>
    -- empty interval: the result is `none`, which is not `some i`
    intro hnone
    cases hnone
```

The theorem is stated as an implication in the goal, `searchFrom … = some i → …`, rather than with the hypothesis before the colon, so that functional induction transforms it together with the function call. `fun_induction` produces one case for each way `searchFrom` can return, with the branch conditions as hypotheses and, in each case, the call replaced by what that branch returns:

- `case1`, the branch that recursed to the left: the goal is now about the recursive call `searchFrom a x lo mid _`, which is exactly the induction hypothesis `ih`.
- `case2`, the branch that recursed to the right: the same.
- `case3`, the branch that returned `some mid`. The hypothesis `hsome : some mid = some i` is decomposed by `injection` into `mid = i`, and `subst` replaces `i` by `mid`. What is left is to show `mid < a.size` (that is `hmid`) and `a[mid] = x`, which `omega` derives from the two failed comparisons `¬ x < a[mid]` and `¬ a[mid] < x`. (`omega` treats `a[mid]` as an unknown integer.)
- `case4`, the empty interval: the function returned `none`, and `none = some i` is impossible; `cases` on it finds no constructor that fits and closes the goal.

Completeness: `none` means absent

The second half needs sortedness. We state it for arrays directly:

```
/-- Sorted, for arrays: an element is never larger than a later one. -/
def SortedArr (a : Array ℤ) : Prop :=
  ∀ (i j : ℕ) (hi : i < a.size) (hj : j < a.size), i ≤ j → a[i] ≤ a[j]
```

```
/-- Completeness: on a sorted array, `none` means that no index in `[lo, hi)`
holds `x`. -/
theorem searchFrom_none (a : Array ℤ) (hs : SortedArr a) (x : ℤ) (lo hi : ℕ)
    (hhi : hi ≤ a.size) :
    searchFrom a x lo hi hhi = none →
      ∀ (k : ℕ) (hk : k < a.size), lo ≤ k → k < hi → a[k] ≠ x := by
  fun_induction searchFrom a x lo hi hhi with
  | case1 lo hi hhi h mid hmid hlt ih =>
    -- x < a[mid]: we searched [lo, mid); everything from mid on is too big
    intro hnone k hk hlo hkhi
    by_cases hkm : k < mid
    · exact ih hnone k hk hlo hkm
    · have hle : a[mid] ≤ a[k] := hs mid k hmid hk (by omega)
      omega                         -- x < a[mid] ≤ a[k], so a[k] ≠ x
  | case2 lo hi hhi h mid hmid hnlt hgt ih =>
    -- a[mid] < x: we searched [mid + 1, hi); everything up to mid is too small
    intro hnone k hk hlo hkhi
    by_cases hkm : mid < k
    · exact ih hnone k hk (by omega) hkhi
```

```
    · have hle : a[k] ≤ a[mid] := hs k mid hk hmid (by omega)
      omega                          -- a[k] ≤ a[mid] < x
  | case3 lo hi hhi h mid hmid hnlt hngt =>
    -- found: `some mid` is not `none`
    intro hnone
    cases hnone
  | case4 lo hi hhi h =>
    -- empty interval: there is no k with lo ≤ k < hi
    intro _ k hk hlo hkhi
    omega
```

The statement carries the invariant of the algorithm: if the search of `[lo, hi)` returns `none`, then no index `k` in `[lo, hi)` holds `x`. This is the generalization that the `revAcc` example taught us to look for: both ends of the interval change during the recursion, so the theorem is about an arbitrary interval, and the top-level search is the special case `[0, a.size)`. In `case1` the search went left, because `x < a[mid]`. For an index `k` below `mid`, the induction hypothesis applies. For `k ≥ mid`, sortedness gives `a[mid] ≤ a[k]`, so `x < a[k]`, and `omega` concludes `a[k] ≠ x`. `case2` is the mirror image. `case3` is impossible (the function returned `some mid`), and in `case4` the interval is empty, so `lo ≤ k < hi` is contradictory and `omega` says so.

The whole specification

```
theorem search_some {a : Array ℤ} {x : ℤ} {i : ℕ} (h : search a x = some i) :
    ∃ h : i < a.size, a[i] = x :=
  searchFrom_some a x 0 a.size _ i h

theorem search_none {a : Array ℤ} {x : ℤ} (hs : SortedArr a) (h : search a x = none) :
    x ∉ a := by
  intro hx
  obtain ⟨k, hk, hkx⟩ := Array.mem_iff_getElem.mp hx
  exact searchFrom_none a hs x 0 a.size _ h k hk (Nat.zero_le k) hk hkx

/-- The whole specification in one line, for sorted arrays. -/
theorem search_isSome_iff {a : Array ℤ} {x : ℤ} (hs : SortedArr a) :
    (search a x).isSome ↔ x ∈ a := by
  constructor
  · intro h
    obtain ⟨i, hi⟩ := Option.isSome_iff_exists.mp h
    obtain ⟨hlt, heq⟩ := search_some hi
    exact Array.mem_iff_getElem.mpr ⟨i, hlt, heq⟩
  · intro hx
    cases hr : search a x with
    | none => exact absurd hx (search_none hs hr)
    | some i => rfl
```

`search_none` converts the statement about indices into one about membership with the library lemma `Array.mem_iff_getElem`. `search_isSome_iff` is the specification in one line: on a sorted array, the search succeeds exactly when the key is present. As with the sort, we can package it as a type:

```
/-- The specification as a type, once more. -/
def searchChecked (a : Array ℤ) (hs : SortedArr a) (x : ℤ) :
    { r : Option ℕ //
      (∀ i, r = some i → ∃ h : i < a.size, a[i] = x) ∧ (r = none → x ∉ a) } :=
  ⟨search a x, fun _ h => search_some h, fun h => search_none hs h⟩
```

Notice what this specification does *not* say: which index is returned when the key occurs more than once. Any index holding `x` satisfies it. That freedom is deliberate, since it lets implementations choose, but it will matter when we test.

Connecting the pieces

Our `SortedArr` is a property of arrays; Mathlib's `SortedLE` is a property of lists. A short bridge connects them, and with it we can search the output of the verified sort:

```
/-- Mathlib's sortedness of the underlying list gives ours. -/
theorem sortedArr_of_sortedLE {a : Array ℤ} (h : a.toList.SortedLE) : SortedArr a := by
  intro i j hi hj hij
  have hl := List.sortedLE_iff_getElem_le_getElem_of_le.mp h
    (i := i) (j := j) (hi := by simpa using hi) (hj := by simpa using hj) hij
  simpa using hl

/-- Searching the output of the verified sort finds exactly the input's elements. -/
theorem search_isort (xs : List ℤ) (x : ℤ) :
    (search (isort xs).toArray x).isSome ↔ x ∈ xs := by
  have hs : SortedArr (isort xs).toArray :=
    sortedArr_of_sortedLE (by simpa using isort_sorted xs)
  rw [search_isSome_iff hs, List.mem_toArray]
  exact (isort_perm xs).mem_iff
```

The bridge uses Mathlib's characterization of `SortedLE` by indices, `List.sortedLE_iff_getElem_le_getElem_of_le`, and `simpa` to translate between `a.toList[i]` and `a[i]` (`simpa using e` simplifies both the goal and the expression `e` and then closes the goal with it). `search_isort` combines three results: the sort's output is sorted, the search is complete and sound on sorted arrays, and a permutation has the same members. Finally, here is what the whole development rests on:

```
/-- info: 'Ch31.search_isort' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms search_isort
```

The three standard axioms of Lean's logic (Chapter 1), and nothing else: no `sorry`, no unchecked assumption.

31.6 Running verified code

The functions we verified are ordinary programs. There are two ways to run them, and the difference matters:

```
/-- The kernel can run the sort inside a proof: slow, but trusted. -/
example : isort [3, 1, 2] = [1, 2, 3] := by decide

/-- The numbers below 1009 that `n * 7919 % 1009` hits for `n < 1000`, in a
scrambled order. `#eval` runs the sort through Lean's compiler: fast, but
outside the kernel. -/
def scrambled : List ℤ := (List.range 1000).map fun n => (((n * 7919) % 1009 : ℕ) : ℤ)

/-- info: (1000, [0, 1, 2, 3, 4]) -/
#guard_msgs in
#eval ((sort scrambled).val.length, (sort scrambled).val.take 5)
```

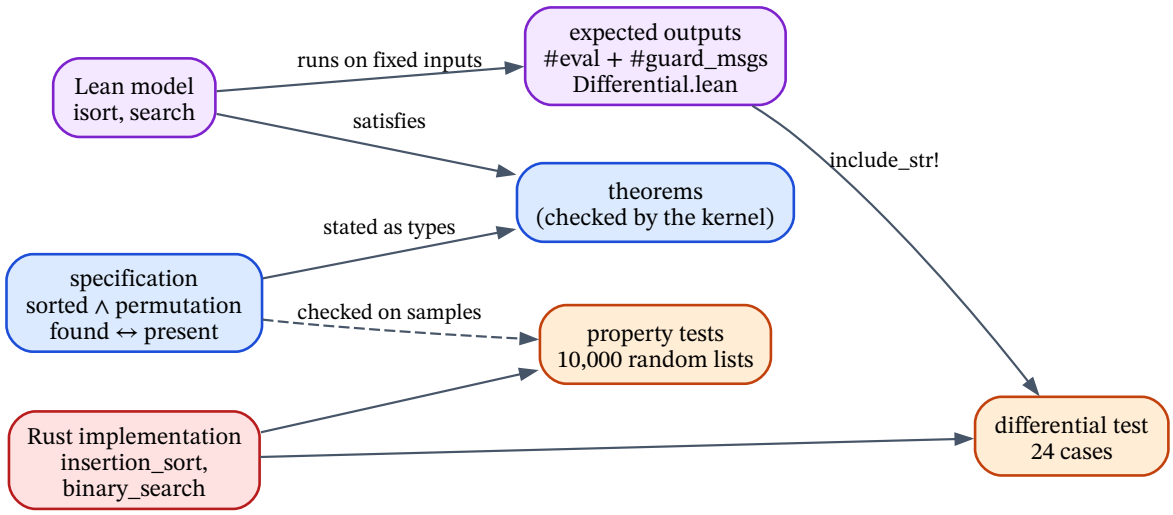
`decide` runs the sort *inside the kernel*, the small trusted checker at the heart of Lean, by reducing the terms step by step. That is slow, but the result is a proof that the kernel itself has checked. `#eval` compiles the program with Lean's code generator and runs it, outside the kernel: fast enough to sort a thousand numbers with a quadratic algorithm in a blink, but the result is trusted only as much as

the compiler. (There is a tactic, `native_decide`, that uses compiled code to prove propositions; it adds the compiler to the trusted base, and this book does not use it.)

The proofs inside the subtype returned by `sort` cost nothing at run time: Lean erases proofs when it compiles, so `(sort xs).val` runs exactly the code of `isort`. Verified Lean code can also be compiled into a native executable with Lake. For this chapter we need something different: not to ship the Lean code, but to use it as a reference for code written in Rust.

31.7 Putting it together: model in Lean, implement in Rust

Here is the workflow, as a diagram:



Model in Lean, implement in Rust. The specification is proved about the Lean model; the model's outputs on fixed inputs are pinned by `#guard_msgs` and read by the Rust tests; property tests check the same specification on random inputs.

Step 1: the model produces expected outputs

`code/lean/Book/Ch31/Differential.lean` runs the verified models on a fixed set of inputs. Some inputs are edge cases chosen by hand; the others come from a small pseudo-random generator, so that they are varied but reproducible:

```
/-- A linear congruential generator: reproducible pseudo-random numbers. -/
def lcgNext (s : ℕ) : ℕ := (s * 1103515245 + 12345) % 2147483648

/-- `n` pseudo-random integers between -9 and 9, starting from seed `s`. -/
def randomList : ℕ → ℕ → List ℤ
  | _, 0 => []
  | s, n + 1 =>
    let s' := lcgNext s
    let x : ℤ := ((s' / 65536 % 19 : ℕ) : ℤ) - 9
    x :: randomList s' n

/-- Edge cases first, then pseudo-random lists with many duplicates. -/
def sortInputs : List (List ℤ) :=
  [[], [5], [2, 1], [4, 3, 2, 1], [5, -2, 5, 0, -2]] ++
    (List.range 4).map fun k => randomList (k + 1) (k + 6)
```

```
/-- Keys to look for in a sorted list: its smallest element (often repeated),
zero (sometimes present) and ten (never present). -/
def searchKeys (ys : List ℤ) : List ℤ := [ys.headD 0, 0, 10]
```

Each test case becomes one line of text:

```
def showResult : Option ℕ → String
  | some i => s!"some {i}"
  | none => "none"

def sortLine (xs : List ℤ) : String :=
  s!"sort {xs} = {(sort xs).val}"

def searchLine (xs : List ℤ) (x : ℤ) : String :=
  s!"search {xs} {x} = {showResult (search xs.toArray x)}"

/-- Every sort case, then every key in the sorted random lists. -/
def table : List String :=
  sortInputs.map sortLine ++
    ((sortInputs.drop 4).map isort).flatMap fun ys => (searchKeys ys).map (searchLine ys)
```

And the table is printed and pinned:

```
/--
info: sort [] = []
sort [5] = [5]
sort [2, 1] = [1, 2]
sort [4, 3, 2, 1] = [1, 2, 3, 4]
sort [5, -2, 5, 0, -2] = [-2, -2, 0, 5, 5]
sort [-5, -8, -4, 7, -4, -6] = [-8, -6, -5, -4, -4, 7]
sort [6, 8, 8, 4, 7, -8, 4] = [-8, 4, 4, 6, 7, 8, 8]
sort [-8, -8, 1, 0, -2, -2, 3, -8] = [-8, -8, -8, -2, -2, 0, 1, 3]
sort [3, 8, -6, -3, 8, 3, -5, 1, -8] = [-8, -6, -5, -3, 1, 3, 3, 8, 8]
search [-2, -2, 0, 5, 5] -2 = some 1
search [-2, -2, 0, 5, 5] 0 = some 2
search [-2, -2, 0, 5, 5] 10 = none
search [-8, -6, -5, -4, -4, 7] -8 = some 0
search [-8, -6, -5, -4, -4, 7] 0 = none
search [-8, -6, -5, -4, -4, 7] 10 = none
search [-8, 4, 4, 6, 7, 8, 8] -8 = some 0
search [-8, 4, 4, 6, 7, 8, 8] 0 = none
search [-8, 4, 4, 6, 7, 8, 8] 10 = none
search [-8, -8, -8, -2, -2, 0, 1, 3] -8 = some 2
search [-8, -8, -8, -2, -2, 0, 1, 3] 0 = some 5
search [-8, -8, -8, -2, -2, 0, 1, 3] 10 = none
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] -8 = some 0
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] 0 = none
search [-8, -6, -5, -3, 1, 3, 3, 8, 8] 10 = none
-/
#guard_msgs in
#eval for line in table do IO.println line
```

This is where `#guard_msgs` earns its keep. The doc comment is the table, and Lean checks, every time the file is built, that the model still prints exactly this table. If someone changed `isort` or `search` in a way that changed an output, the Lean build would fail here, even if the change still satisfied the theorems. The table was produced by running the file once without the doc comment, reading the output, and pasting it in, the same procedure the book uses for every Lean output it shows.

Step 2: the Rust implementation

The Rust code (`code/rust/examples/ch31_sort.rs`) is written the way a Rust programmer would write it, not as a transliteration of the Lean model. The sort works in place on a slice:

```rust
/// Insertion sort, in place: the usual imperative formulation. Each new
/// element is swapped leftwards until the prefix `v[..=i]` is sorted again.
fn insertion_sort(v: &mut [i64]) {
    for i in 1..v.len() {
        let mut j = i;
        while j > 0 && v[j - 1] > v[j] {
            v.swap(j - 1, j);
            j -= 1;
        }
    }
}
```

This is a different algorithm from `isort`: it inserts each element into the sorted prefix on its left, by swapping, while `isort` inserts the head into the sorted tail on its right, building a new list. Because the specification determines the output, the two must agree anyway. The search is a loop with the same midpoint as the model:

```rust
/// Binary search on a sorted slice: the same algorithm as the Lean model
/// `searchFrom`, with a loop instead of tail recursion. Invariant: if `x` is
/// in the slice, it is at an index in `lo..hi`.
fn binary_search(a: &[i64], x: i64) -> Option<usize> {
    let (mut lo, mut hi) = (0, a.len());
    while lo < hi {
        // Not `(lo + hi) / 2`: that sum can overflow; this cannot, since lo < hi.
        let mid = lo + (hi - lo) / 2;
        if x < a[mid] {
            hi = mid;
        } else if a[mid] < x {
            lo = mid + 1;
        } else {
            return Some(mid);
        }
    }
    None
}
```

The midpoint matters. In Rust, `lo + hi` can overflow when the slice is large enough, while `lo + (hi - lo) / 2` cannot, because `lo < hi`. A bug of exactly this kind was found in 2006 in the binary search of Java's standard library: the algorithm was right, but the sum of the two indices overflowed on very large arrays. Our Lean proof works with `ℕ`, which has no overflow, so it *cannot* see this problem: it proves the algorithm correct, not the Rust arithmetic. Using the overflow-free form in both languages keeps the model and the implementation the same algorithm.

Step 3: the differential test

The Rust program reads the Lean file at compile time with `include_str!`, which embeds a file's contents as a `&str`, and extracts the table from the doc comment:

```rust
/// The Lean file whose `#guard_msgs` block pins the model's outputs. Lean
/// checks that the block matches the model; this program checks that the
/// Rust implementation matches the block.
const LEAN_MODEL: &str = include_str!("../../lean/Book/Ch31/Differential.lean");

/// The lines of the expected-output table: from `info: ` to the end of the
/// doc comment.
fn expected_lines(source: &str) -> Vec<&str> {
    let start = source
        .find("info: sort")
        .expect("no table in the Lean file");
    let table = &source[start + "info: ".len()..];
```

```
    let end = table.find("-/").expect("unterminated table");
    table[..end]
        .lines()
        .map(str::trim)
        .filter(|l| !l.is_empty())
        .collect()
}

/// Parses a Lean list of integers such as `[-2, 0, 5]`.
fn parse_list(text: &str) -> Vec<i64> {
    let inner = text.trim().trim_start_matches('[').trim_end_matches(']');
    if inner.trim().is_empty() {
        return Vec::new();
    }
    inner
        .split(',')
        .map(|n| n.trim().parse().expect("an integer"))
        .collect()
}

/// Parses `some 3` or `none`.
fn parse_option(text: &str) -> Option<usize> {
    match text.trim() {
        "none" => None,
        other => Some(other.trim_start_matches("some ").parse().expect("an index")),
    }
}
```

Then it runs its own implementation on every input and compares:

```
/// Runs every line of the table through the Rust implementation.
/// Returns the number of sort and search cases checked.
fn differential_test() -> (usize, usize) {
    let (mut sorts, mut searches) = (0, 0);
    for line in expected_lines(LEAN_MODEL) {
        let (call, expected) = line.split_once(" = ").expect("`call = result`");
        if let Some(input) = call.strip_prefix("sort ") {
            let mut v = parse_list(input);
            insertion_sort(&mut v);
            assert_eq!(
                v,
                parse_list(expected),
                "differs from the Lean model: {line}"
            );
            sorts += 1;
        } else if let Some(args) = call.strip_prefix("search ") {
            let (list, key) = args.rsplit_once(' ').expect("`search list key`");
            let a = parse_list(list);
            let x: i64 = key.parse().expect("an integer key");
            let got = binary_search(&a, x);
            assert_eq!(
                got,
                parse_option(expected),
                "differs from the Lean model: {line}"
            );
            searches += 1;
        } else {
            panic!("unexpected line in the table: {line}");
        }
    }
    (sorts, searches)
}
```

There is a single source of truth. Lean checks that the table matches the model; Rust checks that the implementation matches the table. If the model changes, the Lean build fails until the table is updated, and then the Rust test runs against the new table without any copying. (Cargo notices files read by `include_str!` and recompiles the example when the Lean file changes.)

Step 4: property tests against the specification

A table of 24 cases is small. To cover more ground, the Rust program also checks the *specification* on thousands of random inputs, with a seeded generator written in a dozen lines (the workspace has no external dependencies):

```
/// SplitMix64: a tiny, seeded pseudo-random generator, so that the property
/// tests are reproducible.
struct SplitMix64(u64);

impl SplitMix64 {
    fn next(&mut self) -> u64 {
        self.0 = self.0.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.0;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }

    /// A number in `lo..=hi`.
    fn range(&mut self, lo: i64, hi: i64) -> i64 {
        lo + (self.next() % (hi - lo + 1) as u64) as i64
    }
}
```

```
/// Checks the two specifications on random inputs: the sort output is sorted
/// and a permutation of the input; the search result points at the key, or
/// the key is absent.
fn property_test(cases: usize) -> usize {
    let mut rng = SplitMix64(2026);
    let mut searches = 0;
    for _ in 0..cases {
        let len = rng.range(0, 20) as usize;
        let input: Vec<i64> = (0..len).map(|_| rng.range(-30, 30)).collect();

        let mut sorted = input.clone();
        insertion_sort(&mut sorted);
        assert!(
            sorted.windows(2).all(|w| w[0] <= w[1]),
            "not sorted: {sorted:?}"
        );
        // A permutation: the same multiset of elements. Sorting both sides
        // with the standard library's sort is one way to compare multisets.
        let mut reference = input.clone();
        reference.sort();
        assert_eq!(sorted, reference, "not a permutation of {input:?}");

        for x in -32..=32 {
            match binary_search(&sorted, x) {
                Some(i) => assert_eq!(sorted[i], x),
                None => assert!(!sorted.contains(&x)),
            }
            searches += 1;
        }
    }
    searches
}
```

The two checks complement each other. The differential test compares exact outputs with a model that is proved correct, on few inputs. The property test checks the two halves of the specification, sorted and permutation, found and absent, on many inputs, but with a hand-written checker that is itself unproved. (To check "is a permutation of the input" it compares with the standard library's sort; a different reference would do, as long as it is trustworthy.)

The result

```
let (sorts, searches) = differential_test();
println!("Lean model: {} expected results read", sorts + searches);
println!("  sort:   {sorts} cases agree with the Lean model");
println!("  search: {searches} cases agree with the Lean model");

let cases = 10_000;
let checked = property_test(cases);
println!("property tests: {cases} random lists sorted correctly");
println!("property tests: {checked} searches satisfy the specification");

// With duplicates, the specification of search does not fix the index.
let sevens = [7, 7, 7, 7];
let ours = binary_search(&sevens, 7);
let std_result = sevens.binary_search(&7);
println!("search {sevens:?} 7: ours {ours:?}, std {std_result:?}");
```

```
Lean model: 24 expected results read
  sort:   9 cases agree with the Lean model
  search: 15 cases agree with the Lean model
property tests: 10000 random lists sorted correctly
property tests: 650000 searches satisfy the specification
search [7, 7, 7, 7] 7: ours Some(2), std Ok(3)
```

The last line shows why the difference between "the specification determines the output" and "it does not" matters. On `[7, 7, 7, 7]`, our search returns index 2, and the standard library's `binary_search` returns index 3. Both are correct: the specification only requires *an* index holding 7, and the documentation of `slice::binary_search` says explicitly that any match may be returned when there are several. A differential test that compared our search with the standard library's would report a failure that is not a bug. Our table can compare exact indices only because the Lean model and the Rust code are the same algorithm. For sorting, no such care is needed: we proved that every correct sort returns the same list.

31.8 The landscape: verifying Rust itself

The workflow of this chapter keeps a gap open: the Lean model is proved, the Rust code is only tested against it. Several research and industrial tools aim to close that gap by verifying Rust code directly. They differ in where the specification lives, how much the user proves by hand, and what they guarantee. The descriptions below are brief and general; all of these tools are under active development, so check each project's documentation for its current capabilities and status.

Tool	Approach	Proofs written in
Aeneas	translates Rust into a pure functional model	Lean (and other provers)
Verus	specifications and proofs inside Rust syntax	Rust-like syntax, checked by an SMT solver
Creusot	deductive verification with contracts	annotations, discharged via the Why3 platform
Prusti	deductive verification with contracts	annotations, built on the Viper infrastructure
Kani	bounded model checking	test-like harnesses, no proofs

Aeneas is the closest to this chapter. It translates (a subset of) safe Rust into a purely functional program in Lean (other backends exist too), so that you prove theorems about the translation with

the techniques of this chapter. Its key idea is that, in safe Rust, the borrow checker's discipline makes it possible to model mutable borrows functionally: a function that takes `&mut T` becomes a function that returns the updated value. The model is generated rather than written by hand, which removes the gap between our `isort` and our `insertion_sort`; what remains to trust is the translation.

Verus lets you write specifications (`requires`, `ensures`, loop invariants) and proofs in a Rust-like syntax next to the code, with "ghost" code that exists only for verification, and discharges the proof obligations with an SMT solver, an automatic prover for logical formulas with arithmetic and data structures.

Creusot and **Prusti** are deductive verifiers in the same spirit: contracts written as attributes on Rust functions (preconditions, postconditions, invariants), translated to an intermediate verification platform (Why3 for Creusot, Viper for Prusti) whose automatic provers check them. Verifiers of this kind can also establish properties such as the absence of panics or of arithmetic overflow, which our Lean model could not see.

Kani is a bounded model checker. You write a harness that looks like a test but uses symbolic values (`kani::any()`) instead of concrete ones, and Kani explores *all* values, up to bounds such as the number of loop iterations, looking for panics, overflows, failed assertions and some kinds of undefined behavior in `unsafe` code. It proves nothing beyond its bounds, but within them it is exhaustive, and it needs no proofs from the user. Finally, **Flux** adds refinement types (the idea of Chapter 30's `Percent`, with checked predicates) to Rust.

What they share is the subject of this book: each tool exploits Rust's type system, and the ownership discipline in particular, to make verification tractable. Aliasing is the classic enemy of program proofs, and safe Rust rules out unrestricted aliased mutation (Part VI) before the verifier starts.

31.9 What a proof gives you, and what it does not

A Lean proof about a program is the strongest evidence of correctness this book offers. It is still worth being precise about what it covers.

The specification can be wrong or incomplete. A proof shows that the program meets the specification, not that the specification says what you meant. "Sorted" alone would have accepted `fun _ => []`. Our search specification does not fix which index is returned, which is a choice, not a bug, but it is the kind of thing you have to notice. And no part of our specification mentions cost: `isortCost_le` is a separate theorem, and without it a correct but exponential sort would satisfy everything else.

The model is not the implementation. Our proofs are about `isort` and `search`, Lean functions on lists and arrays of unbounded integers. The Rust code works in place, on `i64`, with `usize` indices. The differential and property tests connect the two on many inputs, not all of them. Tools like Aeneas narrow this gap by generating the model from the code.

Something is always trusted. The theorems rest on Lean's kernel and on three axioms; the `#eval` outputs rest on Lean's compiler; the Rust tests rest on rustc, the standard library, and our small parser

for the table; everything rests on the hardware. A proof does not remove trust; it moves trust from a large program you wrote to a small, carefully reviewed checker.

Rust's guarantees and Lean's are different in kind. Every Lean function terminates, and Lean's theorems hold for all inputs. A well-typed Rust function can loop or panic (Chapter 10). Proving a Rust program correct therefore includes properties, termination and the absence of panics, that a Lean model gets for free.

None of this makes proofs less valuable. It tells you where to look when something goes wrong, and it is the reason this chapter combined proofs with tests rather than choosing between them.

31.10 Common confusions

"**A verified program cannot fail.**" It satisfies its specification, under the assumptions of its trusted base. If the specification is incomplete, or the deployed code differs from the verified model, it can still fail.

"**The differential test proves the Rust code correct.**" It checks 24 inputs against a verified model. That is strong evidence about those inputs and some evidence about others; the property tests add many more inputs. Neither is a proof.

"`partial def` **is fine, because my function terminates in practice.**" It runs fine. But you can prove nothing about it, not even facts that `#eval` shows. If you will want theorems, give a measure or use fuel.

"**The induction hypothesis is whatever I need.**" It is the statement you are proving, for the smaller case. If the recursive call changes an accumulator or an interval, the statement must quantify over it (`generalizing`), or the hypothesis will not apply.

"`simp` **either works or it doesn't.**" `simp` rewrites with a large, changing set of lemmas, so a proof that works today can break when the library grows. Use `simp?` to see what it did and replace it with `simp only [...]` in proofs you want to keep stable.

"`omega` **and** `linarith` **are interchangeable.**" They overlap on linear arithmetic over `ℕ` and `ℤ`. `omega` also handles `/` and `%` by literals and reasons about truncated subtraction on `ℕ`; `linarith` works over ordered fields too and treats nonlinear subterms as opaque atoms. For polynomial *equations*, use `ring`.

31.11 Summary

- A specification is a type: a subtype `{ ys // ys.SortedLE ∧ ys ~ xs }` holds a result together with proofs of its properties. A good specification rules out every wrong answer; "sorted" and "permutation" are each insufficient, and together they determine the output uniquely.
- Induction must sometimes be generalized: when a recursive call changes an argument, prove the statement for all values of it, with `induction … generalizing`. Trees give one induction hypothesis per child.

- Lean requires termination. Structural recursion is automatic; well-founded recursion needs a decreasing measure (`termination_by`) and sometimes a hand-written proof (`decreasing_by`); lexicographic measures handle nested recursion; `partial` gives up reasoning, fuel restores totality. `fun_induction` proves properties of well-founded definitions.
- Insertion sort is verified with `List.Perm` and Mathlib's `List.SortedLE`; it equals Mathlib's `insertionSort` and core Lean's `mergeSort` on every input, because sorted permutations are unique. Its cost is at most $n(n-1)/2$ comparisons.
- Binary search is verified on sorted arrays: a returned index holds the key, and `none` means the key is absent. The specification does not fix which index is returned among duplicates.
- The tactics: simp lemmas and `simp?` / `simp only`, `omega` for linear arithmetic with literals, `linarith` for linear reasoning with opaque atoms, `ring` for polynomial identities, `calc` for readable chains, `grind` for routine case analysis.
- Model in Lean, implement in Rust: the model's outputs, pinned by `#guard_msgs`, drive a differential test of the Rust code; property tests check the specification on random inputs.
- Aeneas, Verus, Creusot, Prusti and Kani verify Rust code directly, with different trade-offs. A proof is only as good as its specification and its trusted base.

31.12 Exercises

1. **Sorting twice changes nothing.** Prove `isort (isort xs) = isort xs`. *Hint:* do not use induction. `isort (isort xs)` is sorted and a permutation of `isort xs`; `isort xs` is sorted and a permutation of itself. Use `sorted_perm_unique`; the whole proof fits on one line.
2. **A merge, and its termination measure.** Define `merge : List ℤ → List ℤ → List ℤ`, which merges two sorted lists, with clauses for `[], ys`, `xs, []` and `x :: xs, y :: ys`. Lean accepts it without help. Use `termination_by?` to find out which measure it used, and explain why neither argument alone would do. Then prove `(merge xs ys).length = xs.length + ys.length`. *Hint:* Lean reports a lexicographic measure on the pair of arguments. For the length theorem, `fun_induction merge xs ys <;> simp_all <;> omega`.
3. **When the specification is not enough to compare outputs.** In `ch31_sort.rs`, change the midpoint to the *lower* middle, `lo + (hi - lo - 1) / 2`. Is the search still correct? Run the example: which checks fail, and why? What would you change in the Lean model, and in the table, to make the test pass again? *Hint:* the property tests still pass, because the search still meets its specification. The differential test fails on a duplicated key: for `-2` in `[-2, -2, 0, 5, 5]` the lower middle finds index 0, and the table says 1.
4. **The first occurrence.** Write a variant `lowerBound` of `searchFrom` that returns the *smallest* index `i` with `x ≤ a[i]` (or `a.size` if there is none), and prove, for sorted arrays, that every index below the result holds an element smaller than `x`. *Hint:* keep the invariant "everything below `lo` is `< x`, everything from `hi` on is `≥ x`", and state the theorem for an arbitrary interval, as in `searchFrom_none`. The recursion has the same measure, with two branches instead of three (`a[mid] < x` goes right, everything else goes left), and returns `lo` when the interval is empty.

5. **A quadratic lower bound.** Prove that sorting a strictly decreasing list of length `n` costs exactly `n * (n - 1) / 2` comparisons, at least for `n ≤ 10` by `decide` on concrete lists, and then, if you are ambitious, for all `n`. *Hint:* for a strictly decreasing list, each insertion goes all the way to the end of the sorted tail, so `insertCost x (isort xs) = xs.length`. For concrete lists, `example : isortCost [5, 4, 3, 2, 1] = 10 := by decide`.
6. **Specifications on paper.** For each specification, find a function of type `List ℤ → List ℤ` that satisfies it but is not a sort: (a) "the output is sorted"; (b) "the output is a permutation of the input"; (c) "the output is sorted and has the same length as the input"; (d) "the output is sorted and every element of the output is an element of the input". *Hint:* for (c) and (d), think of lists with repeated elements, such as `[1, 2]` and `[1, 1]`.

CHAPTER 32

Soundness, unsafe and Semantic Types

In Chapter 9 we proved that a small language is type safe: a well-typed program never gets stuck. It is tempting to believe that safe Rust is protected by a theorem of the same kind, only bigger. It is not, and the reason is in every Rust program you have written. `Vec::push`, `split_at_mut`, `Cell::set` and `Mutex::lock` are implemented with `unsafe` code: raw pointers and operations that the type checker does not check. A proof by induction on typing derivations says nothing about code that has no typing derivation. Yet we say, rightly, that safe Rust has no undefined behavior.

This chapter explains what that sentence means and how it was proved. The key move is to stop asking "does this code have a derivation?" and to ask instead "does this code *behave* like a value of its type?". That is **semantic typing**, and its main tool is the **logical relation**. We build one in Lean for a small language with an unchecked primitive, the analogue of an `unsafe` operation, and verify a library that uses it safely. Then we read `unsafe` Rust as a proof obligation, and meet the logic in which Rust's soundness was actually proved: separation logic, and in particular Iris. At the end, affinity reappears in an unexpected place.

In this chapter you will learn:

- why syntactic soundness cannot account for safe abstractions built with `unsafe`, such as `split_at_mut`, `Vec` and `Cell`;
- what semantic typing is: types as predicates on values, a logical relation and its fundamental theorem, proved in Lean for a language with an unchecked primitive;
- how to read `unsafe` as a proof obligation, with `SAFETY:` comments, safety invariants and validity invariants;
- the basics of separation logic (heaps, points-to, the separating conjunction, the frame rule), with its laws proved in Lean;
- what RustBelt proved, with λRust, Iris and the lifetime logic, and what Stacked Borrows, Tree Borrows and Miri are for;
- what Rust's types do not guarantee, and why the logic used to prove Rust sound is *affine*.

32.1 Syntactic soundness and its limits

What Chapter 9 proved

Chapter 9 had a typing judgment $\Gamma \vdash e : \tau$, defined by rules, and a small-step semantics $e \to e'$. Type safety came in two lemmas: **progress** (a closed well-typed term is a value or can step) and **preservation** (stepping preserves the type), both proved by induction on the typing derivation, one case per rule. This is called **syntactic** type soundness, because "well typed" means "has a derivation built from a fixed list of rules". Andrew Wright and Matthias Felleisen made it the standard method in the 1990s, and Chapter 28 stated the same two lemmas for our borrow calculus.

The closed world of the rules

Syntactic soundness is a theorem about the terms that have derivations, and about nothing else. A term without a derivation gets no guarantee, even if it is harmless. With rule T-If of Chapter 1, the term

$$\texttt{if true then } 1 \texttt{ else false}$$

has no type, since its branches have different types. Yet it runs to 1, and anyone who uses it as a number is safe. The type system is *conservative*: it reasons about the shape of the text, and rejects some safe programs.

For a toy language this does not matter. For Rust it matters a great deal, because the safe programs that the rules reject include the standard library. Splitting a mutable slice into two mutable halves is obviously safe, as long as the halves do not overlap:

```
pub fn naive_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = slice.len();
    (&mut slice[..mid], &mut slice[mid..len])
}
```

```
error[E0499]: cannot borrow `*slice` as mutable more than once at a time
 --> compile_fail/ch32_naive_split.rs:8:30
  |
6 | pub fn naive_split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
  |                                  - let's call the lifetime of this reference `'1`
7 |     let len = slice.len();
8 |     (&mut slice[..mid], &mut slice[mid..len])
  |     ------------------------^^^^^-----------
  |     |     |                 |
  |     |     |                 second mutable borrow occurs here
  |     |     first mutable borrow occurs here
  |     returning this value requires that `*slice` is borrowed for `'1`
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

The borrow checker sees two mutable borrows of `*slice`, both returned, so both must last for the lifetime `'1`. That breaks aliasing XOR mutation (Chapter 26); in the calculus of Chapter 28, a mutable reference is not `Copy`, so the context cannot be split to give it to both reborrows. The checker has no way to know that `slice[..mid]` and `slice[mid..len]` are *disjoint*. Its hint suggests the standard library's `split_at_mut`, which is implemented with `unsafe`.

Two ways out

To prove that programs using `split_at_mut` are safe, we could make it a *primitive*, with its own typing and reduction rules, and redo progress and preservation. That works for one function, but the standard library has hundreds of them (`Vec`, `Box`, `Rc`, `Cell`, `RefCell`, `Mutex`, threads, channels) and crates.io has many more. Every library would need a new language and a new proof, and the proofs would not compose: in the "leakpocalypse" of Chapter 24, `Rc` and the old `thread::scoped` each looked sound on its own, and were unsound together.

The other way is to change what "well typed" means: not "has a derivation" but "behaves like a value of its type, in every context". That is a property of behavior, so it makes sense for any code, including code with raw pointers. The typing rules become *lemmas* about it, and a library written with `unsafe` can be proved to have it by hand, once.

32.2 Semantic types: types as predicates

Truth and provability, once more

You have seen this move before. Chapter 4 had provability, defined by rules, and truth, defined by truth tables, and proved that everything provable is true. The pattern:

- a **syntactic** relation, written $\vdash$, defined by rules and checked by a program;
- a **semantic** relation, written $\vDash$, defined by what things *mean*;
- a **soundness theorem**: $\vdash$ implies $\vDash$.

For types, $\Gamma \vDash e : \tau$ says that whenever e runs with good values for its variables, it produces a good value of type τ. The soundness theorem is called the **fundamental theorem** of the logical relation, and type safety follows from it. $\vdash$ is decidable, and a compiler checks it; $\vDash$ is not, but it is *open*: any code can be proved to satisfy it, by any argument.

A language with an unchecked primitive

We need a language with something that plays the role of `unsafe`. The file `code/lean/Book/Ch32/Language.lean` defines one: naturals, booleans, addition, comparison, `if`, functions, and one more operation.

```
/-- Types: natural numbers, booleans and functions. -/
inductive Ty where
  | nat
  | bool
  | arr (dom cod : Ty)
  deriving DecidableEq, Repr

/-- Terms. Variables are de Bruijn indices: `var 0` is the nearest enclosing
`lam`, `var 1` the one around it, and so on. -/
inductive Expr where
  | var (i : ℕ)
  | nat (n : ℕ)
  | bool (b : Bool)
  | add (e₁ e₂ : Expr)
  | le (e₁ e₂ : Expr)       -- `e₁ ≤ e₂`, a boolean
  | ite (c t f : Expr)      -- `if c then t else f`
  | lam (body : Expr)       -- a function whose parameter is `var 0` in `body`
  | app (f a : Expr)
```

```
  | subRaw (e₁ e₂ : Expr)   -- unchecked `e₁ - e₂`: stuck when `e₁ < e₂`
  deriving Repr
```

Variables are de Bruijn indices, as in Chapters 7 and 9. The odd one out is `subRaw e₁ e₂`, **unchecked subtraction**, which simply gets stuck when $e_1 < e_2$. It models an operation with a precondition, like Rust's `u32::unchecked_sub`, whose documentation says that overflow is undefined behavior. A formal semantics models undefined behavior the standard way: no rule applies, and the program is stuck.

The semantics is **big-step** with **environments**, rather than the small-step substitution semantics of Chapter 9: $\rho \vdash e \Downarrow v$ says that e, run with the variable values ρ, produces v. A function value is a **closure**, a body with the environment it was created in, as in the `stlc` crate of Chapter 8.

```
/-- Values: numbers, booleans and closures. A closure is a function body
together with the environment it was created in. -/
inductive Val where
  | nat (n : ℕ)
  | bool (b : Bool)
  | clo (env : List Val) (body : Expr)
  deriving Repr
```

```
/-- Big-step semantics. `Eval ρ e v` says: in the environment `ρ` (the values
of the variables, nearest first), the term `e` evaluates to the value `v`. -/
inductive Eval : List Val → Expr → Val → Prop where
  | var {ρ i v} : ρ[i]? = some v → Eval ρ (.var i) v
  | nat {ρ n} : Eval ρ (.nat n) (.nat n)
  | bool {ρ b} : Eval ρ (.bool b) (.bool b)
  | add {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) →
      Eval ρ (.add e₁ e₂) (.nat (n₁ + n₂))
  | le {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) →
      Eval ρ (.le e₁ e₂) (.bool (decide (n₁ ≤ n₂)))
  | iteTrue {ρ c t f v} :
      Eval ρ c (.bool true) → Eval ρ t v → Eval ρ (.ite c t f) v
  | iteFalse {ρ c t f v} :
      Eval ρ c (.bool false) → Eval ρ f v → Eval ρ (.ite c t f) v
  | lam {ρ body} : Eval ρ (.lam body) (.clo ρ body)
  | app {ρ f a ρ' body w v} :
      Eval ρ f (.clo ρ' body) → Eval ρ a w → Eval (w :: ρ') body v →
      Eval ρ (.app f a) v
  | subRaw {ρ e₁ e₂ n₁ n₂} :
      Eval ρ e₁ (.nat n₁) → Eval ρ e₂ (.nat n₂) → n₂ ≤ n₁ →
      Eval ρ (.subRaw e₁ e₂) (.nat (n₁ - n₂))
```

Rule `app` evaluates the function to a closure `clo ρ' body`, the argument to `w`, and runs the body in `w :: ρ'`. The interesting rule is the last one:

$$\frac{\rho \vdash e_1 \Downarrow n_1 \qquad \rho \vdash e_2 \Downarrow n_2 \qquad n_2 \le n_1}{\rho \vdash \texttt{subRaw}\; e_1\; e_2 \Downarrow n_1 - n_2}\ \text{E-SubRaw}$$

The third premise is the precondition. When it fails, no rule applies and no derivation exists: that is "stuck" in a big-step semantics. The typing rules are the familiar ones:

```
/-- The syntactic typing rules. There is deliberately no rule for `subRaw`:
the type system cannot check its precondition `e₂ ≤ e₁`. -/
inductive HasType : List Ty → Expr → Ty → Prop where
  | var {Γ i τ} : Γ[i]? = some τ → HasType Γ (.var i) τ
  | nat {Γ n} : HasType Γ (.nat n) .nat
  | bool {Γ b} : HasType Γ (.bool b) .bool
  | add {Γ e₁ e₂} :
```

```
      HasType Γ e₁ .nat → HasType Γ e₂ .nat → HasType Γ (.add e₁ e₂) .nat
  | le {Γ e₁ e₂} :
      HasType Γ e₁ .nat → HasType Γ e₂ .nat → HasType Γ (.le e₁ e₂) .bool
  | ite {Γ c t f τ} :
      HasType Γ c .bool → HasType Γ t τ → HasType Γ f τ →
      HasType Γ (.ite c t f) τ
  | lam {Γ body σ τ} :
      HasType (σ :: Γ) body τ → HasType Γ (.lam body) (.arr σ τ)
  | app {Γ f a σ τ} :
      HasType Γ f (.arr σ τ) → HasType Γ a σ → HasType Γ (.app f a) τ
```

There is **no rule for `subRaw`**: the type system cannot know the values of e_1 and e_2 before the program runs, so it rejects every use of the primitive. That is precisely the status of `unsafe` operations in Rust. (The λ carries no annotation; since `HasType` is a relation, rule `lam` may choose any σ.) Here are two "library functions" built with the primitive:

```
/-- Saturating subtraction, `fun x y => if y ≤ x then x - y else 0`, written
with the unchecked primitive. Inside the body, `var 1` is `x` and `var 0`
is `y`. The `if` establishes the precondition of `subRaw` before calling it. -/
def satSub : Expr :=
  .lam (.lam (.ite (.le (.var 0) (.var 1)) (.subRaw (.var 1) (.var 0)) (.nat 0)))

/-- The careless version: it calls the primitive without checking. -/
def badSub : Expr :=
  .lam (.lam (.subRaw (.var 1) (.var 0)))

/-- Apply a two-argument function to two numbers. -/
def call₂ (f : Expr) (x y : ℕ) : Expr := .app (.app f (.nat x)) (.nat y)
```

`satSub` is saturating subtraction, and calls the primitive only after checking its precondition; `badSub` does not check. Neither has a type. The file also defines an interpreter `run`, with fuel like the normalizer of Chapter 7, which returns a value, `stuck` or `outOfFuel`, and proves it agrees with `Eval` (`run_sound`):

```
/-- info: Ch32.Outcome.value (Ch32.Val.nat 2) -/
#guard_msgs in
#eval run 10 [] (call₂ satSub 5 3)

/-- info: Ch32.Outcome.value (Ch32.Val.nat 0) -/
#guard_msgs in
#eval run 10 [] (call₂ satSub 3 5)

/-- info: Ch32.Outcome.value (Ch32.Val.nat 2) -/
#guard_msgs in
#eval run 10 [] (call₂ badSub 5 3)

/-- info: Ch32.Outcome.stuck -/
#guard_msgs in
#eval run 10 [] (call₂ badSub 3 5)
```

`badSub` gets stuck on `3` and `5`. We want a notion of typing that accepts `satSub`, rejects `badSub`, and accepts everything `HasType` accepts.

Base types: a value of the right shape

A good value of type `nat` is any number; of type `bool`, a boolean. We write $\mathcal{V}[\![\tau]\!]$ for the good values of type τ (double brackets traditionally mean "the meaning of"), and $\mathcal{E}[\![\tau]\!]$ for the closed terms that run to one:

$$\mathcal{V}[\![\mathtt{nat}]\!] = \{\, \mathit{nat}\ n \mid n \in \mathbb{N} \,\} \qquad \mathcal{E}[\![\tau]\!] = \{\, e \mid \exists v.\, e \Downarrow v \ \text{ and } \ v \in \mathcal{V}[\![\tau]\!] \,\}$$

These two definitions already separate the two notions of typing on the term from the beginning of the chapter:

```
/-- `if true then 1 else false`. -/
def mixed : Expr := .ite (.bool true) (.nat 1) (.bool false)

/-- No derivation exists: rule `ite` would need both branches at type `nat`. -/
theorem mixed_not_typed : ¬ HasType [] mixed .nat := by
  intro h
  cases h with
  | ite _ _ hf => cases hf

/-- But the term does run to a number. -/
theorem mixed_sem : SemTyped [] mixed .nat := by
  intro ρ _
  exact ⟨.nat 1, .iteTrue .bool .nat, 1, rfl⟩
```

The first proof shows that *no* derivation exists. `cases h` asks which rule could have produced `h`; only rule `ite` fits, and it yields a derivation `hf` of `HasType [] (.bool false) .nat`. A second `cases hf` finds no rule that concludes such a judgment, so there is nothing left to prove. This is proof by inversion, as in Chapter 28. The second proof exhibits the evaluation `.iteTrue .bool .nat` to `nat 1`, and `⟨1, rfl⟩` shows that `nat 1` is a number. (`SemTyped`, defined below, says for a closed term exactly that it is in $\mathcal{E}[\![\tau]\!]$.)

Function types: the logical step

What is a good value of type `nat → nat`? "A closure" is not enough: `badSub` applied to `3` is a closure, and gets stuck when applied to `5`. A good function turns good arguments into good results:

$$\mathcal{V}[\![\sigma \to \tau]\!] = \{\, \mathit{clo}(\rho, e) \mid \forall w \in \mathcal{V}[\![\sigma]\!].\ \exists r.\ (w :: \rho) \vdash e \Downarrow r \ \text{ and } \ r \in \mathcal{V}[\![\tau]\!] \,\}$$

The meaning of the function type is defined from the meanings of its parts, and has the shape of an implication: "if the input is good, the output is good". That is why such a family of predicates is called a **logical relation** (a *logical predicate*, strictly, since it is unary; binary ones relate two programs and are used to prove equivalences). It is the BHK reading of $A \to B$ from Chapter 5: a method that turns proofs of A into proofs of B. The technique goes back to proofs of normalization for typed λ-calculi in the 1960s and 1970s, starting with work by William Tait. In Lean, the definition is a recursion on the type:

```
/-- `V τ v`: the value `v` is a good value of type `τ`. -/
def V : Ty → Val → Prop
  | .nat, v => ∃ n, v = .nat n
  | .bool, v => ∃ b, v = .bool b
  | .arr σ τ, v =>
    ∃ ρ body, v = .clo ρ body ∧
      ∀ w, V σ w → ∃ r, Eval (w :: ρ) body r ∧ V τ r
```

`V σ w` appears to the *left* of an arrow in the case for `.arr σ τ`. As an inductive predicate, this would be a negative occurrence, which Lean rejects (Chapter 11, positivity). As a recursion on the type it is fine: `V (.arr σ τ)` refers to `V σ` and `V τ`, about smaller types. This is why logical relations are defined by recursion on types (or, with step-indexing, on types and steps).

Open terms

A term with free variables runs in an environment, and semantic typing quantifies over every environment that gives each variable a good value of its type (the set $\mathcal{G}[\![\Gamma]\!]$):

$$\Gamma \vDash e : \tau \quad \text{iff} \quad \forall \rho \in \mathcal{G}[\![\Gamma]\!].\ \rho \vdash e \in \mathcal{E}[\![\tau]\!]$$

```
/-- `E τ ρ e`: in the environment `ρ`, the term `e` runs to a good value of
type `τ`. -/
def E (τ : Ty) (ρ : List Val) (e : Expr) : Prop :=
  ∃ v, Eval ρ e v ∧ V τ v
```

```
/-- `G Γ ρ`: the environment `ρ` gives every variable of `Γ` a good value of
its type. -/
def G : List Ty → List Val → Prop
  | [], [] => True
  | τ :: Γ, v :: ρ => V τ v ∧ G Γ ρ
  | _, _ => False

/-- Semantic typing, written `Γ ⊨ e : τ` in the text: in every environment that
fits `Γ`, the term `e` runs to a good value of type `τ`. -/
def SemTyped (Γ : List Ty) (e : Expr) (τ : Ty) : Prop :=
  ∀ ρ, G Γ ρ → E τ ρ e
```

A small lemma in the file, `G_lookup`, says that if `G Γ ρ` holds and `Γ` gives variable `i` the type `τ`, then `ρ` has at position `i` a value in `V τ`.

Compatibility lemmas

The fundamental theorem is proved by induction on derivations, and each case is a lemma of its own: a typing rule with $\vdash$ replaced by $\vDash$. These are the **compatibility lemmas**:

```
theorem compat_var {Γ i τ} (h : Γ[i]? = some τ) : SemTyped Γ (.var i) τ := by
  intro ρ hG
  obtain ⟨v, hv, hV⟩ := G_lookup hG h
  exact ⟨v, .var hv, hV⟩
```

```
theorem compat_nat {Γ n} : SemTyped Γ (.nat n) .nat :=
  fun _ _ => ⟨.nat n, .nat, n, rfl⟩

theorem compat_bool {Γ b} : SemTyped Γ (.bool b) .bool :=
  fun _ _ => ⟨.bool b, .bool, b, rfl⟩
```

For a variable, `G_lookup` gives the value and its goodness, and `Eval.var` turns the lookup into an evaluation. A constant evaluates to itself. Addition:

```
theorem compat_add {Γ e₁ e₂} (h₁ : SemTyped Γ e₁ .nat)
    (h₂ : SemTyped Γ e₂ .nat) : SemTyped Γ (.add e₁ e₂) .nat := by
  intro ρ hG
  obtain ⟨_, ev₁, n₁, rfl⟩ := h₁ ρ hG
  obtain ⟨_, ev₂, n₂, rfl⟩ := h₂ ρ hG
  exact ⟨.nat (n₁ + n₂), .add ev₁ ev₂, n₁ + n₂, rfl⟩
```

Applied to `ρ` and `hG`, each hypothesis gives a value, its evaluation, and the fact that it is good at `nat`, which unfolds to "it is `nat n` for some `n`". The `obtain` pattern `⟨_, ev₁, n₁, rfl⟩` takes all of this apart, the `rfl` substituting `nat n₁` for the value. `Eval.add` then evaluates the sum, which is a number. For `ite`, the boolean computed by the condition decides which branch's hypothesis to use:

```
theorem compat_ite {Γ c t f τ} (hc : SemTyped Γ c .bool)
    (ht : SemTyped Γ t τ) (hf : SemTyped Γ f τ) :
    SemTyped Γ (.ite c t f) τ := by
  intro ρ hG
  obtain ⟨_, evc, b, rfl⟩ := hc ρ hG
  cases b with
  | true =>
    obtain ⟨v, ev, hv⟩ := ht ρ hG
    exact ⟨v, .iteTrue evc ev, hv⟩
  | false =>
    obtain ⟨v, ev, hv⟩ := hf ρ hG
    exact ⟨v, .iteFalse evc ev, hv⟩
```

The lemmas for functions are where the definition of `V` at function type does its work:

```
theorem compat_lam {Γ body σ τ} (h : SemTyped (σ :: Γ) body τ) :
    SemTyped Γ (.lam body) (.arr σ τ) := by
  intro ρ hG
  refine ⟨.clo ρ body, .lam, ρ, body, rfl, ?_⟩
  intro w hw
  exact h (w :: ρ) ⟨hw, hG⟩
```

`lam body` evaluates to `clo ρ body`, and we must show that for every good argument `w` the body runs to a good result in `w :: ρ`. Hypothesis `h` gives exactly that, because `w :: ρ` fits `σ :: Γ`: the anonymous constructor `⟨hw, hG⟩` proves `G (σ :: Γ) (w :: ρ)`, which unfolds to `V σ w ∧ G Γ ρ`.

```
theorem compat_app {Γ f a σ τ} (hf : SemTyped Γ f (.arr σ τ))
    (ha : SemTyped Γ a σ) : SemTyped Γ (.app f a) τ := by
  intro ρ hG
  obtain ⟨_, evf, ρ', body, rfl, hbody⟩ := hf ρ hG
  obtain ⟨w, eva, hw⟩ := ha ρ hG
  obtain ⟨r, evr, hr⟩ := hbody w hw
  exact ⟨r, .app evf eva evr, hr⟩
```

The function evaluates to a good closure, which carries the promise `hbody` ("give me a good argument and I run to a good result"); the argument evaluates to a good `w`; we call the promise, and `Eval.app` assembles the three evaluations. Compare Chapter 9, where preservation needed a substitution lemma: here the definition of `V` at function type *is* the statement we need.

The fundamental theorem

```
/-- Every syntactically well-typed term is semantically well typed. -/
theorem fundamental {Γ e τ} (h : HasType Γ e τ) : SemTyped Γ e τ := by
  induction h with
  | var h => exact compat_var h
  | nat => exact compat_nat
  | bool => exact compat_bool
  | add _ _ ih₁ ih₂ => exact compat_add ih₁ ih₂
  | le _ _ ih₁ ih₂ => exact compat_le ih₁ ih₂
  | ite _ _ _ ihc iht ihf => exact compat_ite ihc iht ihf
  | lam _ ih => exact compat_lam ih
  | app _ _ ihf iha => exact compat_app ihf iha
```

`induction h` produces one case per typing rule, with the induction hypotheses (`ih₁`, `ihf`, ...) as semantic typings of the premises, and each case is closed by its compatibility lemma. Type safety is a corollary, since the empty environment fits the empty context (`G [] []` is `True`):

```
/-- Type safety as a corollary: a closed well-typed term runs to a value of
its type. It never gets stuck (and, in this language, it always finishes). -/
```

```
theorem safety {e τ} (h : HasType [] e τ) : ∃ v, Eval [] e v ∧ V τ v :=
  fundamental h [] trivial
```

This says a little more than Chapter 9: evaluation *terminates*, since a big-step derivation is a finite tree. That holds because this language, like the simply typed λ-calculus, has no recursion. For a language with loops (Rust), one uses a small-step semantics and a *step-indexed* logical relation, which promises only "never stuck, however many steps you run". Iris, below, has step-indexing built in.

32.3 Verifying a library that uses the primitive

The semantic system accepts code that uses the unchecked primitive correctly. The syntactic one does not:

```
/-- `satSub` has no syntactic type at all: its body uses `subRaw`. -/
theorem satSub_not_typed (τ : Ty) : ¬ HasType [] satSub τ := by
  intro h
  cases h with
  | lam h =>
    cases h with
    | lam h =>
      cases h with
      | ite _ ht _ => cases ht
```

Three inversions (two `lam`s and an `ite`) leave a derivation `ht` for `subRaw ...`, for which no rule exists, so the last `cases ht` has no cases. For the semantic proof we need one fact: in an environment that starts with numbers `y` and `x`, the test `var 0 ≤ var 1` evaluates to the boolean `decide (y ≤ x)` (`decide` turns a decidable proposition into a `Bool`, as in Chapter 5). The two lookups hold by `rfl`, since `(nat y :: nat x :: ρ)[0]?` computes to `some (nat y)`.

```
/-- The comparison `var 0 ≤ var 1` in an environment that starts with two
numbers `y` and `x`. -/
theorem eval_le_vars {y x : ℕ} {ρ : List Val} :
    Eval (.nat y :: .nat x :: ρ) (.le (.var 0) (.var 1)) (.bool (decide (y ≤ x))) :=
  .le (.var rfl) (.var rfl)
```

```
/-- The proof obligation of the library, discharged once and for all:
`satSub` is a good function from two numbers to a number. -/
theorem satSub_sem : SemTyped [] satSub (.arr .nat (.arr .nat .nat)) := by
  intro ρ _
  -- `satSub` evaluates to a closure; we must show it maps numbers to good
  -- functions from numbers to numbers.
  refine ⟨_, .lam, ρ, _, rfl, ?_⟩
  rintro _ ⟨x, rfl⟩
  refine ⟨_, .lam, _, _, rfl, ?_⟩
  rintro _ ⟨y, rfl⟩
  -- The body `if y ≤ x then subRaw x y else 0` runs in `[y, x] ++ ρ`.
  by_cases hyx : y ≤ x
  · -- The `then` branch: `hyx` is exactly the precondition of `subRaw`.
    have hc := eval_le_vars (ρ := ρ) (y := y) (x := x)
    rw [decide_eq_true hyx] at hc
    exact ⟨.nat (x - y), .iteTrue hc (.subRaw (.var rfl) (.var rfl) hyx), _, rfl⟩
  · -- The `else` branch never calls the primitive.
    have hc := eval_le_vars (ρ := ρ) (y := y) (x := x)
    rw [decide_eq_false hyx] at hc
    exact ⟨.nat 0, .iteFalse hc .nat, 0, rfl⟩
```

Step by step:

- `intro ρ _` takes any environment that fits the empty context.
- The first `refine` says that `satSub` evaluates (rule `Eval.lam`) to a closure, and leaves, as `?_`, the condition of `V (.arr .nat (.arr .nat .nat))`.
- `rintro _ ⟨x, rfl⟩` introduces the first argument with the proof that it is a number; the `rfl` pattern replaces it by `nat x`. The inner `lam` is handled the same way, with second argument `nat y`. What remains: the body `if y ≤ x then subRaw x y else 0` runs, in `nat y :: nat x :: ρ`, to a number.
- `by_cases hyx : y ≤ x` splits on the condition. In the first case, `have` takes the evaluation `hc` of the test from `eval_le_vars`, `rw` rewrites `decide (y ≤ x)` in it to `true` (lemma `decide_eq_true`), and we assemble `.iteTrue hc (.subRaw (.var rfl) (.var rfl) hyx)`. The last argument of `.subRaw` is the proof of the precondition $n_2 \le n_1$ demanded by rule E-SubRaw, and we have it because we are in the branch where the program checked it. In the second case the test is `false` (`decide_eq_false`), the `else` branch is the constant `0`, and the primitive is never called.

This proof is the formal version of a `SAFETY:` comment. The sentence you would write in Rust next to the call, "`y <= x` because the `if` just checked it", is the argument `hyx`. The careless library fails the same test, and Lean can prove it:

```
/-- The careless library is not semantically well typed: called with `3` and
`5`, its body is stuck. -/
theorem badSub_not_sem : ¬ SemTyped [] badSub (.arr .nat (.arr .nat .nat)) := by
  intro h
  -- Run `badSub` in the empty environment: it must give a good closure.
  obtain ⟨_, ev, _, _, rfl, hf⟩ := h [] trivial
  cases ev
  -- Apply it to `3`: the result must be a good closure again.
  obtain ⟨_, ev₃, _, _, rfl, hg⟩ := hf (.nat 3) ⟨3, rfl⟩
  cases ev₃
  -- Apply that to `5`: the body `subRaw x y` must run to a value.
  obtain ⟨_, ev₅, _⟩ := hg (.nat 5) ⟨5, rfl⟩
  -- The only rule for `subRaw` needs `5 ≤ 3`.
  cases ev₅ with
  | subRaw ev₁ ev₂ hle =>
    cases ev₁ with
    | var h₁ =>
      cases ev₂ with
      | var h₂ =>
        simp only [List.getElem?_cons_succ, List.getElem?_cons_zero,
          Option.some.injEq, Val.nat.injEq] at h₁ h₂
        omega
```

Assuming `badSub` is semantically well typed, we run it: `cases ev` finds that the evaluation used rule `Eval.lam`, which fixes the closure. Applying it to `nat 3` gives another good closure, and applying that to `nat 5` must give an evaluation `ev₅` of `subRaw (var 1) (var 0)` in `[nat 5, nat 3]`. Inverting `ev₅` leaves only rule `subRaw`, with side condition `hle : n₂ ≤ n₁`; inverting the two lookups shows $n_1 = 3$ and $n_2 = 5$ (`simp only` computes the lookups), and `omega` refutes $5 \le 3$.

Linking a verified library with type-checked clients

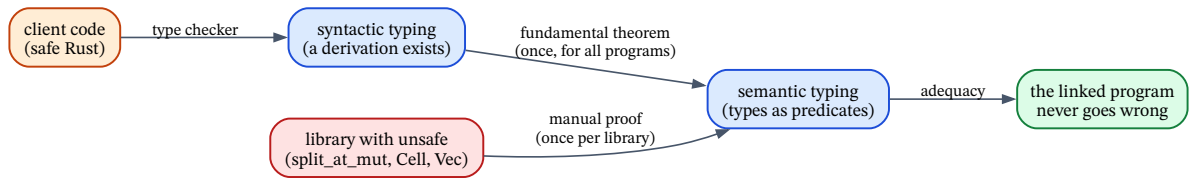
A client of the library should not need its own proof. It should be enough to type-check it against the library's *signature*, as rustc checks your calls to `split_at_mut` without looking inside. We model a client as a term with one free variable of the library's type, and linking as binding that variable:

```
/-- If a library is semantically well typed at `σ`, and a client type-checks
with one variable of type `σ` (the library's signature, nothing more), then
the linked program `(fun lib => client) library` runs to a good value. -/
theorem link {lib c : Expr} {σ τ : Ty}
    (hlib : SemTyped [] lib σ) (hc : HasType [σ] c τ) :
    ∃ v, Eval [] (.app (.lam c) lib) v ∧ V τ v := by
  obtain ⟨w, evw, hw⟩ := hlib [] trivial
  obtain ⟨v, evv, hv⟩ := fundamental hc [w] ⟨hw, trivial⟩
  exact ⟨v, .app .lam evw evv, hv⟩
```

The library's semantic typing gives a good value `w`; the client's *syntactic* typing, through the fundamental theorem, runs the client in `[w]`. A concrete client:

```
/-- A client that only knows `sub : nat → nat → nat` (it is `var 0`):
`sub (sub 10 3) 4`. -/
def client : Expr :=
  .app (.app (.var 0) (.app (.app (.var 0) (.nat 10)) (.nat 3))) (.nat 4)

theorem client_typed : HasType [.arr .nat (.arr .nat .nat)] client .nat :=
  .app (.app (.var rfl) (.app (.app (.var rfl) .nat) .nat)) .nat

/-- The linked program is safe, with no further proof about `satSub`. -/
theorem client_safe : ∃ v, Eval [] (.app (.lam client) satSub) v ∧ V .nat v :=
  link satSub_sem client_typed

/-- info: Ch32.Outcome.value (Ch32.Val.nat 3) -/
#guard_msgs in
#eval run 20 [] (.app (.lam client) satSub)
```

The client's typing is a proof term built from the constructors of `HasType`. `client_safe` costs one line: the work about `satSub` was done once, for every client. The diagram shows this structure, which is also that of the proof of Rust's soundness.



The structure of a semantic soundness proof. Clients are type-checked syntactically; the fundamental theorem, proved once, turns their derivations into semantic typings; libraries that use unsafe operations are proved semantically well typed by hand, once each; and adequacy turns semantic typing into safety of the linked program.

32.4 `unsafe` as a proof obligation

Now the Rust side. The code is in `code/rust/examples/ch32_unsafe_abstractions.rs`.

What `unsafe` unlocks

`unsafe` does not turn off the type checker or the borrow checker. It unlocks five operations, each with preconditions the compiler cannot verify: dereferencing a raw pointer, calling an `unsafe` function, accessing a mutable `static`, implementing an `unsafe` trait, and reading a `union` field. Outside `unsafe`, the safe fragment has no rule for them, as `HasType` has none for `subRaw`:

```
pub fn sub_unchecked(x: u32, y: u32) -> u32 {
    x.unchecked_sub(y)
}
```

```
error[E0133]: call to unsafe function `core::num::<impl u32>::unchecked_sub` is unsafe and
requires unsafe function or block
 --> compile_fail/ch32_unsafe_call.rs:8:5
  |
8 |     x.unchecked_sub(y)
  |     ^^^^^^^^^^^^^^^^^^ call to unsafe function
  |
  = note: consult the function's documentation for information on how to avoid undefined
behavior
```

`unchecked_sub` is the real counterpart of our primitive, and the note says where its precondition lives: in the documentation, in English. The keyword has three roles:

- an **`unsafe` block** says: *I* discharge the obligations of the operations inside (a proof like `satSub_sem` happens here);
- an **`unsafe fn`** says: *the caller* must establish a precondition, written in a `# Safety` section of its documentation;
- an **`unsafe impl`** of an `unsafe trait` promises a property that other unsafe code relies on, as `unsafe impl Sync for T` promises that sharing `&T` across threads is safe.

The convention that makes this workable is the **`SAFETY:` comment** before every `unsafe` block, stating the preconditions involved and why they hold. It is a proof, in English, of the premises of a rule the compiler does not know. Here are the Rust twins of our Lean libraries:

```
/// The Rust twin of `satSub` in `Book/Ch32/Language.lean`.
fn sat_sub(x: u32, y: u32) -> u32 {
    if y <= x {
        // SAFETY: `unchecked_sub` requires that `x - y` does not overflow,
        // that is, `y <= x`, which the `if` has just established.
        unsafe { x.unchecked_sub(y) }
    } else {
        0
    }
}

/// The twin of `badSub`. It compiles and it passes every test below, but it
/// is UNSOUND: it is a safe function, so every caller may pass any `x` and
/// `y`, and `bad_sub(3, 5)` is undefined behavior. No `SAFETY:` comment can
/// be written for this block, because nothing establishes `y <= x`.
fn bad_sub(x: u32, y: u32) -> u32 {
    unsafe { x.unchecked_sub(y) }
}
```

Both compile, and the example's tests pass for both, because they call `bad_sub` only on inputs where it is safe:

```
sat_sub(5, 3) = 2, sat_sub(3, 5) = 0
bad_sub(5, 3) = 2 (never call bad_sub(3, 5))
```

A function without `unsafe` in its signature promises every caller that it is safe *for all inputs*, the same "for all" as in the definition of $\mathcal{V}[\![\sigma \to \tau]\!]$. No finite set of tests establishes it, and `bad_sub` is unsound: it is not semantically well typed at `fn(u32, u32) -> u32`. Tools only see symptoms: Clippy's `undocumented_unsafe_blocks` lint flags exactly one block in this example, the one in `bad_sub`, which

has no `SAFETY:` comment; and in a debug build, `bad_sub(3, 5)` aborts with "unsafe precondition(s) violated: u32::unchecked_sub cannot overflow", a check the standard library calls optional. In a release build it is gone.

An `unsafe fn` passes the obligation up:

```
/// Sum the first `n` readings, without bounds checks.
///
/// # Safety
///
/// The caller must guarantee `n <= readings.len()`.
unsafe fn sum_prefix_unchecked(readings: &[u64], n: usize) -> u64 {
    let mut total = 0;
    for i in 0..n {
        // SAFETY: `i < n <= readings.len()`, by the loop bound and by the
        // precondition of this function, which our caller promised.
        total += unsafe { *readings.get_unchecked(i) };
    }
    total
}

fn sum_first_three(readings: &[u64]) -> Option<u64> {
    if readings.len() < 3 {
        return None;
    }
    // SAFETY: `3 <= readings.len()`, checked just above.
    Some(unsafe { sum_prefix_unchecked(readings, 3) })
}
```

```
sum_first_three([5, 6, 7, 8]) = Some(18)
sum_first_three([5, 6]) = None
```

Since the 2024 edition, an unsafe operation in the body of an `unsafe fn` still needs its own `unsafe` block: otherwise the lint `unsafe_op_in_unsafe_fn` warns that "an unsafe function restricts its caller, but its body is safe by default". So each operation inside gets its own block and justification. The precondition `n <= readings.len()` is what Lean would make an *argument*: recall `first` in Chapter 1, which took a proof `h : 0 < xs.size`. In Lean the caller supplies a proof that the kernel checks; in Rust, an `unsafe` block and a comment that a reviewer checks.

Safety invariants and validity invariants

The **validity invariant** of a type must hold at *all* times, even inside unsafe code, because the compiler relies on it: a `bool` is 0 or 1, a reference is non-null and aligned, a `NonZeroU32` is not zero. Violating it, even briefly, is undefined behavior at once. The compiler uses these facts to lay out data, as in Chapter 11's niche optimization.

The **safety invariant** must hold whenever *safe* code can observe the value, and may be broken temporarily inside a library: a `String` holds UTF-8; a `Vec<T>` has `len <= capacity` with the first `len` elements initialized; the halves returned by `split_at_mut` do not overlap. Breaking it becomes undefined behavior when a safe method that trusts it does something only correct under it. In this chapter's terms, the safety invariant is the semantic type $\mathcal{V}[\![\tau]\!]$; the validity invariant is the much weaker property the compiler needs.

```
/// Safety invariant: a `String` must hold UTF-8. Breaking it is not
/// immediately undefined behavior, but safe methods may misbehave later.
```

```rust
fn shout(s: &str) -> String {
    let bytes: Vec<u8> = s.bytes().map(|b| b.to_ascii_uppercase()).collect();
    // SAFETY: `from_utf8_unchecked` requires valid UTF-8. `s` was valid
    // UTF-8, and `to_ascii_uppercase` only changes bytes in `b'a'..=b'z'`,
    // which never occur inside a multi-byte character (those bytes are all
    // `>= 0x80`), so the result is valid UTF-8 too.
    unsafe { String::from_utf8_unchecked(bytes) }
}

/// Validity invariant: a `NonZeroU32` is never 0, and the compiler relies on
/// it for the layout of `Option<NonZeroU32>`: 0 encodes `None`.
fn port(n: u32) -> Option<NonZeroU32> {
    if n == 0 {
        None
    } else {
        // SAFETY: `new_unchecked` requires `n != 0`, which the `if` checked.
        Some(unsafe { NonZeroU32::new_unchecked(n) })
    }
}
```

```
shout("safe abstraction") = "SAFE ABSTRACTION"
port(8080) = Some(8080), port(0) = None
size_of: u32 = 4, Option<u32> = 8, Option<NonZeroU32> = 4
```

`Option<u32>` needs a separate tag (8 bytes), while `Option<NonZeroU32>` takes 4, because 0 encodes `None`. That is why `NonZeroU32::new_unchecked(0)` is undefined behavior at once, while a `String` holding invalid UTF-8 is "only" a broken promise.

`split_at_mut`, our way

Here is the function the borrow checker rejected, written as the standard library writes it:

```rust
/// Split one unique borrow of a slice into unique borrows of its two halves.
///
/// The signature promises two `&mut` slices that do not overlap. The borrow
/// checker cannot verify that (to it, both halves are borrowed from `slice`),
/// so the body works with a raw pointer, and the `unsafe` block carries the
/// argument that the compiler cannot check.
pub fn my_split_at_mut<'a, T>(
    slice: &'a mut [T],
    mid: usize,
) -> (&'a mut [T], &'a mut [T]) {
    let len = slice.len();
    // This check is what makes the unsafe block sound. Without it, a caller
    // could pass `mid > len` and receive a slice that runs past the end.
    assert!(mid <= len, "mid > len");
    let ptr: *mut T = slice.as_mut_ptr();
    // SAFETY: `from_raw_parts_mut(p, n)` requires that `p` is non-null,
    // aligned, and valid for reads and writes of `n` initialized values of `T`
    // inside one allocation, and that nothing else accesses that memory while
    // the result is alive. `ptr.add(mid)` requires that the result stays in
    // the same allocation (or one past its end).
    // 1. `ptr` comes from `slice`, so it is non-null, aligned and points to
    //    `len` initialized values.
    // 2. `mid <= len` (checked above), so `ptr.add(mid)` is in bounds, and
    //    both ranges `[0, mid)` and `[mid, len)` lie inside `slice`.
    // 3. The two ranges are disjoint, so the two results never alias each
    //    other. This is the fact that the borrow checker cannot see.
    // 4. Both results live for `'a`, the lifetime of the unique borrow
    //    `slice`, so no one else can use the memory while they are alive.
    unsafe {
        (
            std::slice::from_raw_parts_mut(ptr, mid),
            std::slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
```

```
    }
}
```

The `SAFETY:` comment lists the preconditions of `slice::from_raw_parts_mut` and `pointer::add`, then discharges them. Point 2 relies on the `assert!`: the function establishes its own precondition, so a bad `mid` causes a panic, which is safe, instead of undefined behavior. Point 3 is the fact the borrow checker could not see. Point 4 explains why one lifetime `'a` appears three times: while either half is alive, `slice` stays mutably borrowed. (The lifetime is written out for the discussion; elision gives the same signature.)

```
let mut samples = [3, 1, 4, 1, 5, 9, 2, 6];
let (left, right) = my_split_at_mut(&mut samples, 3);
left[0] = 30; // two unique borrows, used side by side
right[0] = 10;
left.swap(1, 2);
right.reverse();
println!("left  = {left:?}");
println!("right = {right:?}");
println!("all   = {samples:?}");
```

```
left  = [30, 4, 1]
right = [6, 2, 9, 5, 10]
all   = [30, 4, 1, 6, 2, 9, 5, 10]
```

In the terms of the diagram, `my_split_at_mut` is a library proved (informally) semantically well typed at its signature, and `main` is a client checked by rustc against that signature.

A `Cell` of our own

`Cell<T>` permits mutation through a shared reference, an apparent violation of aliasing XOR mutation that is sound because of an invariant its API protects:

```
/// A minimal `Cell`: mutation through a shared reference, on one thread.
///
/// The invariant that every method relies on: no reference into `value` ever
/// escapes a method, and no method runs code it does not control (such as a
/// destructor or a `clone`) while it is accessing `value`. So whenever a
/// method touches the contents, no other reference to them is alive.
pub struct MyCell<T> {
    value: UnsafeCell<T>,
}

impl<T> MyCell<T> {
    pub const fn new(value: T) -> Self {
        MyCell {
            value: UnsafeCell::new(value),
        }
    }

    /// Put `value` in the cell and return the old contents.
    pub fn replace(&self, value: T) -> T {
        // SAFETY: we create a `&mut T` to the contents for the duration of
        // `mem::replace`, which only moves bytes and runs no other code.
        // No other reference to the contents is alive: no method lets one
        // escape, and `MyCell` is not `Sync` (because `UnsafeCell` is not),
        // so no other thread can be inside a method of this cell right now.
        unsafe { std::mem::replace(&mut *self.value.get(), value) }
    }

    /// Store `value`. The old contents are dropped *after* the access ends:
    /// a destructor can run arbitrary code, including code that uses this
```

```
    /// very cell.
    pub fn set(&self, value: T) {
        let old = self.replace(value);
        drop(old);
    }

    pub fn into_inner(self) -> T {
        self.value.into_inner()
    }
}

impl<T: Copy> MyCell<T> {
    /// Copy the contents out. The bound `T: Copy` is part of the safety
    /// argument: copying is a plain bitwise read, while a `clone` could run
    /// code that calls `set` on this cell in the middle of our read.
    pub fn get(&self) -> T {
        // SAFETY: we read the contents through a raw pointer and copy them;
        // as in `replace`, no other reference to the contents is alive, and
        // no user code runs during the read.
        unsafe { *self.value.get() }
    }
}
```

`UnsafeCell<T>` is the one primitive through which Rust permits mutation behind a shared reference; `UnsafeCell::get` returns a raw pointer to the contents. The invariant in the doc comment (no reference escapes, no foreign code runs during an access) explains three details:

- `get` requires `T: Copy`. A copy runs no user code. If `get` called `clone` instead, the user's `clone` could call `set` on the same cell while we hold a reference into it. The trait bound is part of the safety argument.
- `set` drops the old value *after* the access, since a destructor is user code.
- `MyCell` is not `Sync`, without our saying so: `UnsafeCell` is not `Sync`, and auto traits propagate to the struct. So "no other thread is inside a method" is enforced by the type system:

```
pub fn bump_in_two_threads(counter: &MyCell<u32>) {
    std::thread::scope(|s| {
        s.spawn(|| counter.bump());
        counter.bump(); // if this compiled, it would be a data race
    });
}
```

```
error[E0277]: `UnsafeCell<u32>` cannot be shared between threads safely
   --> compile_fail/ch32_cell_not_sync.rs:22:17
    |
 22 |         s.spawn(|| counter.bump());
    |           -----  ^^^^^^^^^^^^^^^^^ `UnsafeCell<u32>` cannot be shared between threads
safely
    |           |
    |           required by a bound introduced by this call
    |
    = help: within `MyCell<u32>`, the trait `Sync` is not implemented for `UnsafeCell<u32>`
note: required because it appears within the type `MyCell<u32>`
   --> compile_fail/ch32_cell_not_sync.rs:8:12
    |
  8 | pub struct MyCell<T> {
    |            ^^^^^^
    = note: required for `&MyCell<u32>` to implement `Send`
note: required because it's used within this closure
   --> compile_fail/ch32_cell_not_sync.rs:22:17
    |
 22 |         s.spawn(|| counter.bump());
    |                 ^^
```

```
note: required by a bound in `Scope::<'scope, 'env>::spawn`
   --> /rustc/b940084d7eb6a299eb4bfeb8e34901bc051e7ac4/library/std/src/thread/
scoped.rs:203:28
    |
201 |     pub fn spawn<F, T>(&'scope self, f: F) -> ScopedJoinHandle<'scope, T>
    |            ----- required by a bound in this associated function
202 |     where
203 |         F: FnOnce() -> T + Send + 'scope,
    |                            ^^^^ required by this bound in `Scope::<'scope,
'env>::spawn`
```

Within one thread, a sensor that counts its own reads can be shared freely:

```rust
/// A lab sensor that counts how often it has been read. `read` takes `&self`,
/// so many parts of a program can share the sensor, and still update the
/// counter.
struct Sensor {
    name: &'static str,
    reads: MyCell<u32>,
}

impl Sensor {
    fn read(&self) -> f64 {
        self.reads.set(self.reads.get() + 1);
        20.0 + f64::from(self.reads.get()) / 10.0
    }
}
```

```rust
let sensor = Sensor {
    name: "thermometer",
    reads: MyCell::new(0),
};
let first: &Sensor = &sensor; // two shared references to one sensor
let second: &Sensor = &sensor;
let r1 = first.read();
let r2 = second.read();
let r3 = first.read();
println!(
    "{}: {r1}, {r2}, {r3} after {} reads",
    sensor.name,
    sensor.reads.get()
);
```

```
thermometer: 20.1, 20.2, 20.3 after 3 reads
```

`Vec` and its invariant

A `Vec<T>` is a pointer, a capacity and a length, with the safety invariant above. Safe methods preserve it; unsafe ones let you take over:

```rust
/// Build `[0, 1, 4, 9, ...]` by writing into a vector's spare capacity, then
/// telling the vector how many elements are now initialized.
fn squares(n: usize) -> Vec<u64> {
    let mut v: Vec<u64> = Vec::with_capacity(n);
    let spare: &mut [MaybeUninit<u64>] = v.spare_capacity_mut();
    for (i, slot) in spare.iter_mut().take(n).enumerate() {
        slot.write((i as u64) * (i as u64));
    }
    // SAFETY: `set_len(n)` requires `n <= capacity` and that the elements
    // `0..n` are initialized. The capacity is at least `n` (`with_capacity`),
    // so `spare` has at least `n` slots, and the loop wrote all of the first
    // `n`. This re-establishes `Vec`'s invariant: the first `len` elements
    // are initialized and `len <= capacity`.
    unsafe { v.set_len(n) };
```

```
    v
}
```

```
squares(6) = [0, 1, 4, 9, 16, 25]
```

`spare_capacity_mut` hands out the uninitialized part of the buffer as `MaybeUninit<u64>` slots (a type whose validity invariant allows "uninitialized"); `set_len` declares them elements, and the `SAFETY:` comment proves that the invariant holds again.

32.5 Separation logic

The arguments of the last section all talk about *ownership* of memory, *disjointness*, and the memory an operation *does not touch*. Ordinary logic is clumsy with all three. A Hoare triple (precondition, command, postcondition) for a write,

$$\{\, x \mapsto 1 \land y \mapsto 1 \,\} \; * x = 3 \; \{\, x \mapsto 3 \land y \mapsto 1 \,\}$$

where $x \mapsto 1$ means "the cell at x holds 1", is false if x and y are the same address: the precondition holds, but afterwards the cell holds 3, not 1. So the precondition needs $x \neq y$, and with n pointers you need about $n^2/2$ such facts. **Separation logic**, introduced around 2000 by John Reynolds, Peter O'Hearn and their collaborators (Reynolds's 2002 paper is the standard reference), changes the meaning of assertions: an assertion describes a *piece* of the heap that you own, and the **separating conjunction** $P * Q$ says that the heap splits into two *disjoint* pieces satisfying P and Q. Then $x \mapsto 1 * y \mapsto 2$ implies $x \neq y$ by itself.

We build a *shallow* separation logic (assertions are Lean predicates, not a syntax of formulas) in `code/lean/Book/Ch32/SeparationLogic.lean`.

Heaps and assertions

```
/-- Memory locations. -/
abbrev Loc := ℕ
/-- The values stored in memory. -/
abbrev Val := ℤ

/-- A heap is a partial map from locations to values: `h l = none` means that
`l` is not allocated in `h`. -/
abbrev Heap := Loc → Option Val

/-- The empty heap. -/
def Heap.empty : Heap := fun _ => none

/-- The heap with exactly one allocated cell, `l`, holding `v`. -/
def Heap.single (l : Loc) (v : Val) : Heap :=
  fun l' => if l' = l then some v else none

/-- Two heaps are disjoint when no location is allocated in both. -/
def Heap.Disjoint (h₁ h₂ : Heap) : Prop := ∀ l, h₁ l = none ∨ h₂ l = none

/-- The union of two heaps (meant for disjoint heaps; if both define `l`, the
left one wins). -/
def Heap.union (h₁ h₂ : Heap) : Heap := fun l =>
  match h₁ l with
  | some v => some v
  | none => h₂ l
```

```
@[inherit_doc] scoped infixl:65 " ∪ₕ " => Heap.union
```

A heap is a partial map from locations to values (`abbrev` is like a Rust `type` alias). The union is meant for disjoint heaps; the `scoped` notation `∪ₕ` does not leak into other chapters. Real heaps are *finite*; the laws below do not need it, but finiteness is easy to state and is what allocation needs:

```
/-- A heap is finite when every location from some bound on is free. -/
def Heap.Finite (h : Heap) : Prop := ∃ n, ∀ l, n ≤ l → h l = none

/-- A finite heap always has a free location: this is what allocation needs. -/
theorem Heap.exists_free {h : Heap} (hf : h.Finite) : ∃ l, h l = none := by
  obtain ⟨n, hn⟩ := hf
  exact ⟨n, hn n (Nat.le_refl n)⟩
```

The bound `n` itself is a free location. (The file also proves that unions of finite heaps are finite. Mathlib's `Finmap` builds finiteness into the type.) Union is associative:

```
theorem union_assoc (h₁ h₂ h₃ : Heap) : (h₁ ∪ₕ h₂) ∪ₕ h₃ = h₁ ∪ₕ (h₂ ∪ₕ h₃) := by
  funext l
  simp only [union]
  cases h₁ l <;> rfl
```

`funext l` (function extensionality) reduces an equation between heaps to one location; `simp only [union]` unfolds the definition; `cases h₁ l` considers both possibilities for `h₁ l`, and in each both sides compute to the same thing (`<;>` applies `rfl` to every resulting goal). Commutativity needs disjointness, and disjointness of a union splits into disjointness of its parts; these are proved the same way:

```
/-- info: Ch32.Sep.Heap.union_comm {h₁ h₂ : Heap} (hd : h₁.Disjoint h₂) : h₁ ∪ₕ h₂ = h₂ ∪ₕ
h₁ -/
#guard_msgs in
#check union_comm

/--
info: Ch32.Sep.Heap.disjoint_union_left {h₁ h₂ h₃ : Heap} : (h₁ ∪ₕ h₂).Disjoint h₃ ↔
h₁.Disjoint h₃ ∧ h₂.Disjoint h₃
-/
#guard_msgs in
#check disjoint_union_left
```

Assertions are predicates on heaps:

```
/-- An assertion is a property of heaps: the heaps it describes. -/
abbrev Assertion := Heap → Prop

/-- `emp`: the heap is empty. -/
def emp : Assertion := fun h => h = Heap.empty

/-- `l ↦ v`: the heap is exactly one cell, `l`, holding `v`. -/
def pointsTo (l : Loc) (v : Val) : Assertion := fun h => h = Heap.single l v

/-- `P ∗ Q`: the heap splits into two disjoint parts, one satisfying `P` and
the other satisfying `Q`. -/
def sep (P Q : Assertion) : Assertion := fun h =>
  ∃ h₁ h₂, Heap.Disjoint h₁ h₂ ∧ h = h₁ ∪ₕ h₂ ∧ P h₁ ∧ Q h₂

/-- `P ⊢ Q`: every heap described by `P` is described by `Q`. -/
def Entails (P Q : Assertion) : Prop := ∀ h, P h → Q h
```

```
@[inherit_doc] scoped infix:60 " ↦ " => pointsTo
@[inherit_doc] scoped infixr:35 " ∗ " => sep
@[inherit_doc] scoped infix:25 " ⊢ " => Entails
```

`l ↦ v` holds only of the heap with exactly that one cell. $P * Q$ holds of a union of two disjoint heaps satisfying P and Q. Entailment $P \vdash Q$ is inclusion. (The turnstile here is entailment between assertions, as in the separation logic literature, not a typing judgment.)

The laws of the separating conjunction

Assertions are functions to `Prop`, so we state laws as equations: `funext h` reduces one to a single heap, and `propext` (propositional extensionality, the axiom you saw in Chapter 1's `#print axioms`) turns an equation between propositions into an "if and only if".

```
theorem sep_comm (P Q : Assertion) : (P ∗ Q) = (Q ∗ P) := by
  funext h
  apply propext
  constructor
  all_goals
    rintro ⟨h₁, h₂, hd, rfl, hP, hQ⟩
    exact ⟨h₂, h₁, hd.symm, union_comm hd, hQ, hP⟩
```

`all_goals` runs the same script on both directions. `rintro` takes apart a proof of $P * Q$ (two heaps, disjointness, the equation, which `rfl` substitutes away, and the two proofs), and `exact` rebuilds it with the heaps swapped, using `union_comm`, which needs exactly the disjointness we have.

```
theorem sep_assoc (P Q R : Assertion) : ((P ∗ Q) ∗ R) = (P ∗ (Q ∗ R)) := by
  funext h
  apply propext
  constructor
  · rintro ⟨h₁₂, h₃, hd, rfl, ⟨h₁, h₂, hd₁₂, rfl, hP, hQ⟩, hR⟩
    obtain ⟨hd₁₃, hd₂₃⟩ := disjoint_union_left.mp hd
    exact ⟨h₁, h₂ ∪ₕ h₃, disjoint_union_right.mpr ⟨hd₁₂, hd₁₃⟩,
      union_assoc h₁ h₂ h₃, hP, h₂, h₃, hd₂₃, rfl, hQ, hR⟩
  · rintro ⟨h₁, h₂₃, hd, rfl, hP, ⟨h₂, h₃, hd₂₃, rfl, hQ, hR⟩⟩
    obtain ⟨hd₁₂, hd₁₃⟩ := disjoint_union_right.mp hd
    exact ⟨h₁ ∪ₕ h₂, h₃, disjoint_union_left.mpr ⟨hd₁₃, hd₂₃⟩,
      (union_assoc h₁ h₂ h₃).symm, ⟨h₁, h₂, hd₁₂, rfl, hP, hQ⟩, hR⟩
```

Associativity re-splits $(h_1 \cup h_2) \cup h_3$ as $h_1 \cup (h_2 \cup h_3)$. The work is the disjointness bookkeeping: `disjoint_union_left` extracts from `hd` the facts "h_1 and h_3 are disjoint" and "h_2 and h_3 are disjoint", and `disjoint_union_right` combines what the new split needs. The other direction is the mirror image.

```
theorem emp_sep (P : Assertion) : (emp ∗ P) = P := by
  funext h
  apply propext
  constructor
  · rintro ⟨_, h₂, _, rfl, rfl, hP⟩
    exact hP
  · intro hP
    exact ⟨Heap.empty, h, fun _ => Or.inl rfl, rfl, rfl, hP⟩
```

From left to right, the `rfl` patterns force the first part to be the empty heap, so the whole heap is the second part; from right to left, we split `h` as the empty heap and `h` itself, which are disjoint because the empty heap has no cells. So assertions with $*$ and `emp` form a commutative monoid, like contexts with $\uplus$ in Chapter 21, which is no coincidence. One more law lets entailments act inside a separating conjunction:

```
/-- Monotonicity: entailments can be applied inside either side of `∗`.
Reading it with `Q = Q'`: what `P` entails, `P ∗ Q` entails with the frame `Q`
untouched. -/
theorem sep_mono {P P' Q Q' : Assertion} (hP : P ⊢ P') (hQ : Q ⊢ Q') :
    (P ∗ Q) ⊢ (P' ∗ Q') := by
  rintro h ⟨h₁, h₂, hd, rfl, hp, hq⟩
  exact ⟨h₁, h₂, hd, rfl, hP h₁ hp, hQ h₂ hq⟩
```

The same split works for both sides, with `hP` and `hQ` applied to the two parts. With $Q' = Q$ it is the logical core of the **frame** idea: what you conclude about your part of the heap still holds next to an untouched part Q, the *frame*.

Ownership is exclusive

$A \wedge A$ is the same as A, but $\ell \mapsto v * \ell \mapsto w$ is unsatisfiable:

```
/-- Nobody can own the same cell twice: `l ↦ v ∗ l ↦ w` describes no heap. -/
theorem pointsTo_sep_same (l : Loc) (v w : Val) :
    ((l ↦ v) ∗ (l ↦ w)) ⊢ fun _ => False := by
  rintro _ ⟨h₁, h₂, hd, _, rfl, rfl⟩
  rcases hd l with h | h <;> simp [Heap.single] at h
```

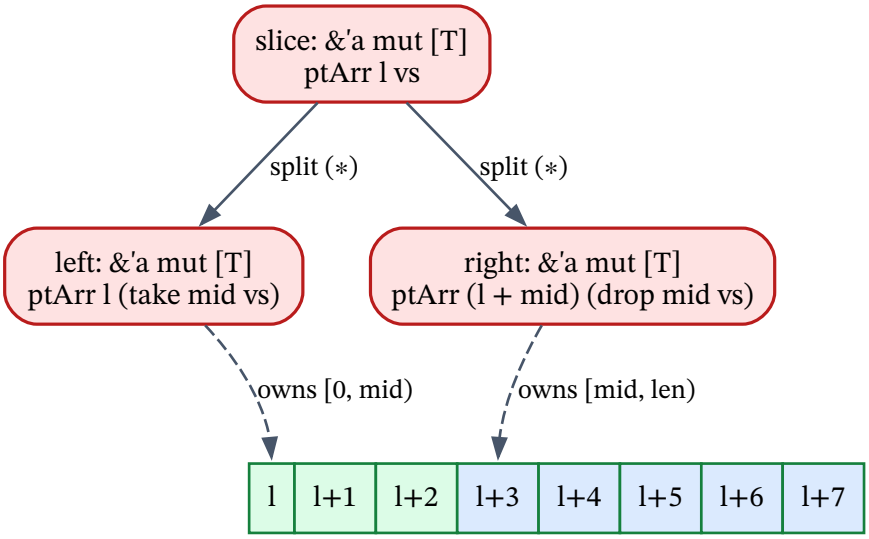
Both parts have the cell `l`, but disjointness says one of them does not. So contraction fails for points-to: $\ell \mapsto v \vdash \ell \mapsto v * \ell \mapsto v$ is false. This is E0499 one level down: you cannot own the same cell twice.

Arrays, and the lemma behind `split_at_mut`

```
/-- `ptArr l [v₀, v₁, …]`: the cells `l, l+1, …` hold `v₀, v₁, …`. -/
def ptArr : Loc → List Val → Assertion
  | _, [] => emp
  | l, v :: vs => (l ↦ v) ∗ ptArr (l + 1) vs

/-- An array is the separating conjunction of its two parts. -/
theorem ptArr_append (xs ys : List Val) (l : Loc) :
    ptArr l (xs ++ ys) = (ptArr l xs ∗ ptArr (l + xs.length) ys) := by
  induction xs generalizing l with
  | nil => simp [ptArr, emp_sep]
  | cons x xs ih =>
    rw [List.cons_append, ptArr, ptArr, ih, sep_assoc, List.length_cons,
      Nat.add_assoc, Nat.add_comm 1 xs.length]

/-- `split_at_mut`, in the logic: owning `vs` at `l` is the same as owning its
first `mid` elements at `l` and the rest at `l + mid`. -/
theorem ptArr_split (vs : List Val) (l mid : ℕ) (h : mid ≤ vs.length) :
    ptArr l vs = (ptArr l (vs.take mid) ∗ ptArr (l + mid) (vs.drop mid)) := by
  have := ptArr_append (vs.take mid) (vs.drop mid) l
  rwa [List.take_append_drop, List.length_take_of_le h] at this
```

`ptArr_append` is proved by induction on `xs`, generalizing the start address (the hypothesis is needed at `l + 1`). In the `nil` case, `ptArr l []` is `emp` and `simp` finishes with `emp_sep`. In the `cons` case, the `rw` chain unfolds `ptArr`, applies the induction hypothesis, reassociates with `sep_assoc`, and fixes the address arithmetic `l + 1 + xs.length = l + (xs.length + 1)`. Then `ptArr_split` instantiates it with `vs.take mid` and `vs.drop mid`, using `List.take_append_drop` and `List.length_take_of_le`.

This is point 3 of the `SAFETY:` comment of `my_split_at_mut` as a theorem: owning a slice *is* owning its two halves separately. Splitting a `&mut [T]` is not contraction (nothing is duplicated); it is the division of one resource into two disjoint ones.



What split_at_mut does, in separation logic. The unique borrow of the whole slice owns cells l to l+7; splitting along the separating conjunction gives two unique borrows that own disjoint ranges of cells.

Hoare triples and the frame rule

`code/lean/Book/Ch32/Frame.lean` adds a tiny command language:

```
/-- Commands. -/
inductive Cmd where
  | skip
  | write (l : Loc) (v : Val)   -- `*l = v`
  | free (l : Loc)              -- deallocate `l`
  | swap (l₁ l₂ : Loc)          -- exchange the contents of `l₁` and `l₂`
  | seq (c₁ c₂ : Cmd)           -- `c₁; c₂`

/-- Change one cell of a heap (`none` deallocates it). -/
def Heap.update (h : Heap) (l : Loc) (o : Option Val) : Heap :=
  fun l' => if l' = l then o else h l'

/-- Run a command. `none` is a fault: the command touched a cell it does not
have. -/
def exec : Cmd → Heap → Option Heap
  | .skip, h => some h
  | .write l v, h => if (h l).isSome then some (h.update l (some v)) else none
  | .free l, h => if (h l).isSome then some (h.update l none) else none
  | .swap l₁ l₂, h =>
    match h l₁, h l₂ with
    | some a, some b => some ((h.update l₁ (some b)).update l₂ (some a))
    | _, _ => none
  | .seq c₁ c₂, h =>
    match exec c₁ h with
    | some h' => exec c₂ h'
    | none => none
```

A command faults (`none`) when it touches a cell that is not in its heap: a command may use only the cells it owns. A **Hoare triple** $\{P\}\ c\ \{Q\}$ says that from any heap satisfying P, c runs without fault to a heap satisfying Q:

```
/-- The Hoare triple `{P} c {Q}`: from every heap satisfying `P`, the command
runs without fault and ends in a heap satisfying `Q`. -/
def Triple (P : Assertion) (c : Cmd) (Q : Assertion) : Prop :=
  ∀ h, P h → ∃ h', exec c h = some h' ∧ Q h'
```

Specifications are **small**: they mention only the cells the command uses.

```
/-- Writing needs the cell, and only the cell. -/
theorem write_spec (l : Loc) (a v : Val) : Triple (l ↦ a) (.write l v) (l ↦ v) := by
  rintro _ rfl
  refine ⟨(Heap.single l a).update l (some v), by simp [exec, Heap.single], ?_⟩
  funext l'
  by_cases e : l' = l <;> simp [Heap.update, Heap.single, e]
```

The precondition makes the heap `Heap.single l a` (the `rfl` pattern); `refine` names the final heap and `simp` shows the write reaches it; then we compare the two heaps at every location, distinguishing `l' = l` from `l' ≠ l`. The specs of `free` and `swap` are proved alike:

```
/-- info: Ch32.Sep.free_spec (l : Loc) (a : Val) : Triple (l ↦ a) (Cmd.free l) emp -/
#guard_msgs in
#check free_spec

/--
info: Ch32.Sep.swap_spec (l₁ l₂ : Loc) (a b : Val) : Triple (l₁ ↦ a ∗ l₂ ↦ b) (Cmd.swap l₁
l₂) (l₁ ↦ b ∗ l₂ ↦ a)
-/
#guard_msgs in
#check swap_spec
```

The precondition of `swap` owns two cells *separately*. Rust's `std::mem::swap(x: &mut T, y: &mut T)` has this specification, and its signature guarantees the precondition: two live `&mut` are always disjoint. To use a small specification in a bigger heap we need the **frame rule**:

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}\ \text{Frame}$$

(With program variables there is a side condition that c does not modify the variables of R; our commands have none.) The rule is a theorem about the commands, resting on **locality**: a command that runs without fault in `h` runs the same way in `h ∪ₕ hf`, for any disjoint `hf`, and leaves `hf` alone. For a write, the key fact is that updating a cell the frame does not have commutes with adding the frame:

```
/-- Updating a cell that the frame does not have commutes with adding the
frame. -/
theorem update_union {h hf : Heap} {l : Loc} (o : Option Val) (hl : hf l = none) :
    (h ∪ₕ hf).update l o = h.update l o ∪ₕ hf := by
  funext l'
  by_cases e : l' = l
  · subst e
    cases o <;> simp [Heap.update, Heap.union, hl]
  · simp [Heap.update, Heap.union, e]
```

```
/-- Locality of `write`: if it runs without fault in `h`, it runs the same way
in `h` plus a disjoint frame `hf`, and leaves the frame alone. -/
theorem write_local {h h' hf : Heap} {l : Loc} {v : Val} (hd : Disjoint h hf)
    (he : exec (.write l v) h = some h') :
    Disjoint h' hf ∧ exec (.write l v) (h ∪ₕ hf) = some (h' ∪ₕ hf) := by
  simp only [exec] at he
  split at he
  · -- The write did not fault, so `l` is allocated in `h`, holding some `a`.
    rename_i hs
    obtain ⟨a, ha⟩ := Option.isSome_iff_exists.mp hs
    -- Then the frame does not have `l`.
    have hl := free_in_frame hd ha
    simp only [Option.some.injEq] at he
```

```
    subst he
    refine ⟨disjoint_update _ hd hl, ?_⟩
    simp [exec, Heap.union, ha, update_union _ hl]
  · -- The write faulted: `none = some h'` is impossible.
    simp at he
```

`simp only [exec] at he` unfolds the successful write into an `if` on whether `l` is allocated, and `split at he` considers both branches. In the first, `rename_i hs` names the hypothesis that `h l` is some value, and `Option.isSome_iff_exists` produces the value `a`. Since `h` has the cell, the disjoint frame does not (`free_in_frame`, a two-line lemma). Two facts remain: the updated heap is still disjoint from the frame (`disjoint_update`), and the write on the big heap gives the update plus the frame (`simp`, using `update_union`). In the second branch the write faulted, and `none = some h'` is absurd. `free` and `swap` are analogous, and locality of every command follows by recursion:

```
/-- Locality of every command, by recursion on the command. -/
theorem exec_frame : ∀ (c : Cmd) {h h' hf : Heap}, Disjoint h hf →
    exec c h = some h' → Disjoint h' hf ∧ exec c (h ∪ₕ hf) = some (h' ∪ₕ hf)
  | .skip, _, _, _, hd, he => by
    simp only [exec, Option.some.injEq] at he
    subst he
    exact ⟨hd, rfl⟩
  | .write _ _, _, _, _, hd, he => write_local hd he
  | .free _, _, _, _, hd, he => free_local hd he
  | .swap _ _, _, _, _, hd, he => swap_local hd he
  | .seq c₁ c₂, _, _, _, hd, he => by
    simp only [exec] at he
    split at he
    · -- `c₁` ran to some `hmid` (`he₁`), then `c₂` ran from `hmid` (`he`).
      rename_i hmid he₁
      obtain ⟨hd₁, he₁'⟩ := exec_frame c₁ hd he₁
      obtain ⟨hd₂, he₂'⟩ := exec_frame c₂ hd₁ he
      exact ⟨hd₂, by simp [exec, he₁', he₂']⟩
    · simp at he
```

For a sequence, `split at he` exposes the intermediate heap; locality is applied to `c₁` with the original frame, then to `c₂`, and the results combine. The frame rule follows:

```
/-- The frame rule: a triple about a small heap holds in any larger heap, with
the extra part `R` carried through unchanged. -/
theorem frame {P Q R : Assertion} {c : Cmd} (h : Triple P c Q) :
    Triple (P ∗ R) c (Q ∗ R) := by
  rintro _ ⟨h₁, h₂, hd, rfl, hp, hr⟩
  obtain ⟨h₁', he, hq⟩ := h h₁ hp
  obtain ⟨hd', he'⟩ := exec_frame c hd he
  exact ⟨h₁' ∪ₕ h₂, he', h₁', h₂, hd', rfl, hq, hr⟩
```

Split the heap as `h₁ ∪ₕ h₂`; the triple runs the command on `h₁` to some `h₁'` satisfying Q; locality says that on the whole heap it runs to `h₁' ∪ₕ h₂`, with the parts still disjoint: a split witnessing $Q * R$.

The frame rule makes separation logic *local*: each function is proved against the memory it uses, and the rule extends the result to any context. Rust's borrow checker gives safe code the same property statically: a function that receives `&mut a` cannot touch `b`.

32.6 RustBelt

We can now describe how the soundness of Rust was established. The RustBelt project (Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers and Derek Dreyer, "RustBelt: Securing the Foundations of

the Rust Programming Language", POPL 2018) followed the structure of this chapter's diagram, at full scale, with every proof checked in the Coq proof assistant (now called Rocq).

λRust. Rust has no formal semantics, so RustBelt defines a core language, λRust, close to MIR (Chapter 29): programs manipulate memory through explicit pointers, functions are in continuation-passing style, and types include owned pointers, shared and mutable references with lifetimes, products, sums and recursive types. Like every core calculus in this book, it is a model of Rust, not Rust. Its semantics makes use-after-free, out-of-bounds accesses and data races stuck.

Iris. Types are interpreted in Iris, a framework for *higher-order concurrent separation logic* developed by Jung, Krebbers, Birkedal, Dreyer and others. Iris adds to the separation logic above what a real language needs: *ghost state* (user-defined logical resources, such as the lifetime tokens below), *invariants* (facts about shared resources that every thread maintains), and *step-indexing* (a "later" modality $\triangleright$ that makes recursive definitions well founded).

Types as predicates, twice. In RustBelt each type has two interpretations. The *ownership predicate* says what it means for a thread to own a value of the type (for `Box<T>`, roughly: a pointer to memory holding an owned `T`, plus the right to free it). The *sharing predicate* says what it means for a location to be shared at the type for a lifetime κ: it interprets `&'κ T`. The sharing predicate must be *persistent*, Iris's word for freely duplicable, because shared references are `Copy`. For most types it means read-only access; for `Cell<T>` it is different, and that is where `Cell`'s soundness argument, and the reason it is not `Sync`, live.

The lifetime logic. References are interpreted with a logic of lifetimes built inside Iris. A lifetime κ is a *token*; a thread holds a fraction of it while κ is alive, and ending κ consumes the whole token and produces a *dead* token $[\dagger\ \kappa]$. A *full borrow* $\&^{\kappa}_{\text{full}} P$ is the right to access the resource P while κ is alive. Simplified (the real rules carry Iris's modalities), two rules say:

- **Borrowing.** Owning P can be exchanged for a full borrow of P at κ together with an *inheritance* that gives P back once $[\dagger\ \kappa]$ is available. This is what `let r = &mut x;` does: `r` gets the borrow, and `x` the inheritance.
- **Accessing.** A full borrow plus a fraction of κ's token can be exchanged for P, with the obligation to give P back to recover the token. This is dereferencing a mutable reference.

Lifetimes here are *resources* in a logic, and the rules of Chapter 28 (reborrowing, outlives, the escape check) become lemmas about them.

What was proved. The fundamental theorem for λRust's type system, adequacy (semantically well-typed programs do not get stuck), and semantic well-typedness of standard library abstractions that use unsafe code, among them `Cell`, `RefCell`, `Rc`, `Arc`, `Mutex`, `RwLock` and `thread::spawn`, each against its public signature. Along the way the authors found a real bug: `MutexGuard<T>` was declared `Sync` under a condition that was too weak. The result is our `link` theorem at the scale of a real language: a well-typed program that uses only verified libraries has no undefined behavior.

32.7 Aliasing models: Stacked Borrows, Tree Borrows and Miri

RustBelt assumes a semantics in which bad things get stuck. But which uses of raw pointers *are* bad? Consider:

```
let mut x = 0;
let p = &mut x as *mut i32;   // a raw pointer to x
let r1 = unsafe { &mut *p };  // a mutable reference made from p
let r2 = unsafe { &mut *p };  // a second one, made from p as well
*r1 += 1;
*r2 += 1;
```

No memory is freed or overrun, but `r1` and `r2` are two live mutable references to one place, which a `&mut` promises cannot happen, and the compiler optimizes on that promise (for instance, keeping `*r1` in a register across writes through other pointers). For such optimizations to be correct, some uses of raw pointers must be undefined behavior. An **aliasing model** is an operational semantics that says which.

Stacked Borrows (Ralf Jung, Hoang-Hai Dang, Jeehoon Kang and Derek Dreyer, POPL 2020) attaches to every location a stack of permissions, one per reference or raw pointer derived from it. Creating a reference pushes an item. Using a pointer requires its item to be in the stack, and writing through it pops every item above it (creating a mutable reference counts as such a use of the pointer it is made from). Using a pointer whose item was popped is undefined behavior. Above, creating `r2` from `p` pops `r1`'s item, so writing through `r1` is undefined behavior. **Tree Borrows**, a successor proposed by Neven Villani and collaborators, records a tree of which pointer was derived from which, and accepts patterns that real unsafe code uses and Stacked Borrows rejects. Neither is, at the time of writing, the official definition of undefined behavior in Rust; check the current status in the Rust project's Unsafe Code Guidelines.

Miri makes these models usable. It interprets MIR step by step and checks every step: in-bounds accesses, no use of freed or uninitialized memory, valid values for every type, the aliasing rules (Stacked Borrows, or Tree Borrows on request) and data races. It reports the first undefined behavior it finds. Miri is a component of the nightly toolchain:

```
rustup +nightly component add miri
cargo +nightly miri run --example ch32_unsafe_abstractions
MIRIFLAGS="-Zmiri-tree-borrows" cargo +nightly miri run --example ch32_unsafe_abstractions
cargo +nightly miri test -p <crate>
```

The book's scripts do not run Miri (it needs nightly and is slow), but it is a good habit for code with `unsafe`. It checks the executions you run, like a very strict test, so it finds nothing wrong with `bad_sub`. Miri finds bugs; only an argument for all inputs rules them out.

32.8 What types do not guarantee

The strongest claim of safe Rust is: no undefined behavior (memory safety, no data races), *provided* that the unsafe code it relies on is sound and the compiler is correct. Outside the guarantee:

- **panics**: out-of-bounds indexing, `unwrap` on `None`, integer overflow in debug builds (release builds wrap); a panic is a defined way to stop;
- **non-termination**: `loop {}` has every type (Chapter 10);
- **leaks**: `mem::forget` and `Rc` cycles are safe (Chapter 24);
- **deadlocks and race conditions**: data races are excluded, two threads waiting for each other's lock are not;
- **protocol completion and logic errors** (Chapter 25), and wrong answers;
- **unsound `unsafe` code and compiler bugs**: one unsound library voids the guarantee for every program that uses it, and rustc has known soundness bugs, tracked under the label `I-unsound` in the Rust issue tracker.

Each has an extension in the literature (totality checking, linear types, session types, refinement and dependent types), each with a cost. Lean, as Chapter 31 showed, can prove functional correctness, at the price of writing the proofs.

32.9 Where affinity reappears

A logic is *affine* (Chapter 21) when it has weakening but not contraction. We saw that contraction fails for points-to. Weakening, in separation logic, is $P * Q \vdash P$: you may forget a part of what you own. In our model it is false:

```
/-- In this exact model, the separating conjunction cannot forget a part: the
heap with two cells satisfies `0 ↦ 1 ∗ 1 ↦ 2` but not `0 ↦ 1`. -/
theorem weakening_fails :
    ¬ (((0 : Loc) ↦ 1) ∗ ((1 : Loc) ↦ 2) ⊢ ((0 : Loc) ↦ 1)) := by
  intro hent
  have hd : Disjoint (Heap.single 0 1) (Heap.single 1 2) := by
    intro l
    by_cases hl : l = 0 <;> simp [Heap.single, hl]
  have h := hent _ (_, _, hd, rfl, rfl, rfl)
  -- `h` claims the two-cell heap equals the one-cell heap; look at cell 1.
  have := congrFun h 1
  simp [Heap.union, Heap.single] at this
```

The proof builds the two-cell heap, shows that it satisfies the left side (the single-cell heaps are disjoint, checked with `by_cases` on `l = 0`), applies the entailment, and gets an equation between the two-cell and the one-cell heap; `congrFun h 1` evaluates both at location 1, where they differ. The cause is the definition of points-to: `l ↦ v` holds of *exactly* one cell, so an assertion accounts for the whole heap. Such a logic is often called *linear* (or *classical*): a postcondition that omits a cell promises that it was freed.

The other natural model lets an assertion describe *at least* some cells:

```
/-- `h ≤ₕ h'`: `h'` extends `h` (it has every cell of `h`, with the same value,
and possibly more). -/
def Heap.Extends (h h' : Heap) : Prop := ∀ l v, h l = some v → h' l = some v

@[inherit_doc] scoped infix:50 " ≤ₕ " => Heap.Extends

/-- An assertion is up-closed (affine) when adding cells to a heap that
satisfies it gives a heap that still satisfies it. -/
def UpClosed (P : Assertion) : Prop := ∀ h h', h ≤ₕ h' → P h → P h'
```

```
theorem Heap.extends_union_left (h₁ h₂ : Heap) : h₁ ≤ₕ (h₁ ∪ₕ h₂) := by
  intro l v hv
  simp [Heap.union, hv]
```

```
/-- Weakening, `P ∗ Q ⊢ P`, holds for every up-closed `P`. -/
theorem sep_weaken {P Q : Assertion} (hP : UpClosed P) : (P ∗ Q) ⊢ P := by
  rintro _ ⟨h₁, h₂, _, rfl, hp, _⟩
  exact hP h₁ (h₁ ∪ₕ h₂) (Heap.extends_union_left h₁ h₂) hp
```

`extends_union_left` (proved by unfolding the union at a cell of `h₁`) says that `h₁` is extended by `h₁ ∪ₕ h₂`, and the up-closure of `P` carries the proof of `P` along. The points-to of this model has both properties of an affine resource:

```
/-- The affine points-to: the heap contains (at least) the cell `l` with `v`. -/
def pointsToA (l : Loc) (v : Val) : Assertion := fun h => h l = some v

theorem pointsToA_upClosed (l : Loc) (v : Val) : UpClosed (pointsToA l v) :=
  fun _ _ hext hp => hext l v hp

/-- Forgetting is allowed in the affine model. -/
example (l l' : Loc) (v v' : Val) :
    (pointsToA l v ∗ pointsToA l' v') ⊢ pointsToA l v :=
  sep_weaken (pointsToA_upClosed l v)

/-- Exclusive ownership survives: the same cell still cannot be owned twice. -/
theorem pointsToA_sep_same (l : Loc) (v w : Val) :
    (pointsToA l v ∗ pointsToA l w) ⊢ fun _ => False := by
  rintro _ ⟨h₁, h₂, hd, _, hv, hw⟩
  rcases hd l with h | h <;> simp_all [pointsToA]
```

It can be forgotten: it is up-closed, because an extension has every cell of the heap it extends, which is the definition of `≤ₕ`. And it cannot be duplicated: $*$ still demands disjoint parts, and the proof is the same case analysis as for `pointsTo_sep_same`. Finally, the up-closed assertions are closed under $*$, so they form a logic of their own; the proof gives the left part to P unchanged and everything else to Q:

```
/-- info: Ch32.Sep.upClosed_sep {P Q : Assertion} (hQ : UpClosed Q) : UpClosed (P ∗ Q) -/
#guard_msgs in
#check upClosed_sep
```

Iris is an affine separation logic: $P * Q \vdash P$ holds for every assertion. Its propositions are resources that may always be forgotten but in general not duplicated; the ones that may be are called *persistent* (the analogue of $!A$ in Chapter 23, and of `Copy`). This fits Rust: a logic in which resources could not be forgotten would be the wrong tool for a language in which `mem::forget` is safe and a leaked `Box` is not undefined behavior. Leaking is weakening.

So "affine" describes something in the foundations of Rust with full precision: the *logic* in which Rust's soundness was proved. But look at what its affine resources are: memory cells, lifetime tokens, full borrows, the ghost resources from which the *meanings* of Rust's types are built, not the values of the affine λ-calculus of Chapter 22. Owning a `Box<T>` and holding a mutable borrow are affine resources of the logic; a shared reference is a persistent (duplicable) predicate; lifetimes are fractional tokens. The affine structure sits one level below the type system, in the model, while the surface discipline for borrows and lifetimes is the region-and-permission system of Part VI. Blurring the two levels is

an easy route to the folklore claim of Chapter 1, and Chapter 33 keeps them apart when it gives the verdict.

32.10 Putting it together: from `split_at_mut` to the frame rule

Let us follow one small program through the levels of this chapter:

```
let mut cells = [10, 20, 30];
let (front, back) = my_split_at_mut(&mut cells, 2);
front.swap(0, 1); // touches cells 0 and 1 only
back[0] = 0; // touches cell 2 only
println!("cells = {cells:?}");
```

```
cells = [20, 10, 0]
```

The type checker. rustc checks `main` against the *signature* of `my_split_at_mut`: two `&mut [i32]` with one lifetime, held in two variables, so `front` and `back` may be used side by side. This is the client's syntactic typing.

The library's obligation. The `SAFETY:` comment of `my_split_at_mut` is an informal proof that it is *semantically* well typed: for every slice and `mid`, it panics or returns two unique borrows of disjoint memory.

The logic. The ownership of `cells` at address ℓ is $\mathit{ptArr}\ \ell\ [10, 20, 30]$, which `ptArr_split` with $mid = 2$ turns into $\mathit{ptArr}\ \ell\ [10, 20] * \mathit{ptArr}\ (\ell + 2)\ [30]$, one resource for `front` and one for `back`. Each statement works on its own resource, and the frame rule says the other is untouched. In Lean, with the cells at locations 0, 1 and 2:

```
/-- Three cells; swap the first two, then clear the third. Each step is proved
with its small specification, and the frame rule supplies the cells that the
step does not touch. -/
theorem three_cells :
    Triple ((0 ↦ 10) ∗ (1 ↦ 20) ∗ (2 ↦ 30))
      (.seq (.swap 0 1) (.write 2 0))
      ((0 ↦ 20) ∗ (1 ↦ 10) ∗ (2 ↦ 0)) := by
  apply seq_rule (Q := (0 ↦ 20) ∗ (1 ↦ 10) ∗ (2 ↦ 30))
  · -- The swap owns cells 0 and 1; cell 2 is the frame.
    have h := frame (R := 2 ↦ 30) (swap_spec 0 1 10 20)
    rw [sep_assoc, sep_assoc] at h
    exact h
  · -- The write owns cell 2; cells 0 and 1 are the frame.
    have h := frame (R := (0 ↦ 20) ∗ (1 ↦ 10)) (write_spec 2 30 0)
    rw [sep_comm (2 ↦ 30), sep_comm (2 ↦ 0), sep_assoc, sep_assoc] at h
    exact h
```

The sequencing rule (proved in the file like the frame rule) needs an intermediate assertion `Q`, the state after the swap. For the swap, the small axiom `swap_spec 0 1 10 20` mentions only cells 0 and 1; `frame` adds cell 2, nested as $((0 \mapsto 10) * (1 \mapsto 20)) * (2 \mapsto 30)$, and two rewrites with `sep_assoc` produce the right-nested form of the goal. For the write, `write_spec 2 30 0` mentions only cell 2; `frame` puts cells 0 and 1 on the right, so `sep_comm` moves cell 2 to the right before reassociating. Each step was proved against the smallest heap it needs, and the frame rule did the rest: the same reason that lets rustc check `front.swap(0, 1)` without thinking about `back`.

32.11 Common confusions

"**`unsafe` turns off the borrow checker.**" It does not: all the usual checks apply inside an `unsafe` block, which only permits five extra operations.

"**If my tests pass and Miri is happy, my unsafe code is sound.**" Soundness of a safe API is a claim about *all* callers and inputs; tests and Miri examine the executions you run. `bad_sub` passes both.

"**A sound function never panics.**" `my_split_at_mut` panics when `mid > len`, and that is what makes it sound. A panic is defined behavior; reading past the end of a slice is not.

"**$P * Q$ is just $P \wedge Q$.**" For resources they differ completely: $\ell \mapsto v \wedge \ell \mapsto v$ is $\ell \mapsto v$, while $\ell \mapsto v * \ell \mapsto v$ is impossible.

"**Iris is affine, so Rust's type system is affine.**" Affinity is a property of the logic in which the model is built. The surface rules for borrows, with duplicable shared references and lifetimes, are a different system, related to the model by a theorem, not by identity.

32.12 Summary

- Syntactic soundness says nothing about code without a derivation, and safe Rust depends on such code: the standard library is built on `unsafe`.
- Semantic typing interprets types as predicates on values; at function type, "good arguments go to good results", which makes it a *logical* relation.
- The fundamental theorem ($\vdash$ implies $\models$) is proved with one compatibility lemma per rule. In Lean we proved it for a language with an unchecked primitive, verified `satSub`, refuted `badSub`, and linked verified libraries with type-checked clients.
- An `unsafe` block discharges proof obligations, an `unsafe fn` passes them to its caller, and a `SAFETY:` comment is the proof. Safety invariants are the semantic types; validity invariants are what the compiler relies on.
- Separation logic describes owned pieces of the heap: $*$ is a commutative monoid with unit `emp`, points-to cannot be duplicated, arrays split along $*$ (the lemma behind `split_at_mut`), and the frame rule follows from locality.
- RustBelt proved Rust's type system sound in this style, with λRust, Iris and a lifetime logic. Stacked and Tree Borrows define what unsafe code may do with aliases; Miri checks the executions you run.
- Iris is affine ($P * Q \vdash P$): affinity reappears one level below Rust's type system, in the logic used to prove it sound.

32.13 Exercises

1. **A second unchecked primitive.** Extend `Language.lean` with `predRaw e`, an unchecked predecessor that is stuck on 0 (a constructor, an `Eval` rule with premise `0 < n`, a case in `run`). Define `safePred := fun x => if x ≤ 0 then 0 else predRaw x` and prove

`SemTyped [] safePred (.arr .nat .nat)`. Which proofs had to change, and why did `fundamental` not need a new case? *Hint:* follow `satSub_sem`. `HasType` has no rule for the primitive, so `induction h` produces the same cases as before; only `run_sound` grows.

2. **Contraction fails, weakening fails.** Prove `sep_emp : (P ∗ emp) = P` from `sep_comm` and `emp_sep`, and prove `¬ (((0 : Loc) ↦ 1) ⊢ ((0 : Loc) ↦ 1) ∗ ((0 : Loc) ↦ 1))`. Which structural rule of Chapter 21 does this fact correspond to, and which does `weakening_fails`? *Hint:* apply the supposed entailment to `Heap.single 0 1` and feed the result to `pointsTo_sep_same`.
3. **A non-local command.** Add to `Frame.lean` a command `checkEmpty` that succeeds (leaving the heap unchanged) only on the empty heap. Show that `Triple emp .checkEmpty emp` holds but the framed triple with `R := (0 : Loc) ↦ 1` does not. Which step of `exec_frame` fails? *Hint:* the command inspects cells it does not own. In Rust the analogue is a function whose behavior depends on memory it has no reference to.
4. **Safe code on top of `my_split_at_mut`.** Using `my_split_at_mut` and no `unsafe` of your own, write `fn swap_ends<T>(v: &mut [T])` (swap the first and last elements of a slice of length at least 2) and `fn first_and_rest<T>(v: &mut [T]) -> Option<(&mut T, &mut [T])>`. Why do they need no `SAFETY:` comments? *Hint:* split at 1 or at `len - 1`. Your functions are clients checked by the rules, like `client` in `LogicalRelation.lean`.
5. **Why `Cell::get` needs `Copy`.** Suppose `MyCell` had `fn get_clone(&self) -> T where T: Clone`, calling `(*self.value.get()).clone()`. Design a type whose `Clone` makes this unsound and describe the execution that goes wrong (if you run it, use Miri). *Hint:* give the type an `Rc` to a structure containing the cell, and let `clone` call `set` on it while `get_clone` holds a reference to the old contents.
6. **Two kinds of invariant.** Classify as validity or safety invariants: a `bool` is 0 or 1; a `Vec` has `len <= capacity`; a `&T` is non-null; a `str` is UTF-8; a `NonZeroU8` is not 0; a `SortedVec` newtype (Chapter 30's "parse, don't validate") is sorted. *Hint:* does the compiler use the fact, for layout or optimization, or only the library's code? `size_of::<Option<T>>()` is a useful experiment.

CHAPTER 33

The Verdict

Chapter 1 asked a question that sounds simple: *is it true that Rust's ownership and lifetime concepts are based on affine types?* At the time, almost every word in it was undefined. We did not know what a type system is in the precise sense, what a structural rule is, what "affine" means, what a region is, or what it would mean for a programming language to be "based on" a logic. Thirty-two chapters later, each of those words has a definition, most of the definitions have Lean formalizations, and the key claims have been proved, tested, or both.

This chapter assembles the answer. It contains no new theory and almost no new code. Instead it goes back through the book, picks up the results that bear on the question, and puts them side by side. The answer, as Chapter 1 promised, is that the claim is **half true**, and the half that is true is not the half most people mean when they repeat it.

In this chapter you will learn:

- what the question means once each of its words has a precise meaning;
- which parts of Rust's discipline are affine, in the sense of a theorem we proved, and with which amendments;
- why borrowing and lifetimes are a different theory, and how much of the aliasing rule affinity does explain;
- the three senses of "based on", and the answer in each;
- the verdict in tables: Rust features, structural rules and error codes;
- how to say it correctly, and what the two languages of this book taught us about each other.

33.1 The question, made precise

The claim has two parts, ownership and lifetimes, and one predicate, "based on affine types". We now know what each of them means.

Affine. A logic or type system is *affine* when it keeps the structural rule of weakening and drops the rule of contraction (Chapters 6 and 21). Read bottom-up, weakening says an assumption may go unused, and contraction says an assumption may be used more than once. So in an affine system every assumption is used *at most once*. A *linear* system drops weakening too, so every assumption is used *exactly once*. The affine λ-calculus λ^{aff} of Chapter 22 is the type-theoretic version: its typing rules

split the context between subterms that both run, and its central metatheorem, the usage theorem, says that in a well-typed term every variable occurs at most once.

Ownership. By "ownership" people mean the discipline of moves: a value of a non-`Copy` type has one owner, passing it to a function or binding it to another variable moves it, and the old name cannot be used again. Around this core are `Copy` (types whose values are duplicated implicitly), `Clone` (explicit duplication) and `Drop` (what happens when a value is discarded).

Lifetimes. By "lifetimes" people mean the annotations `'a` on references, together with everything the borrow checker enforces about references: that they do not outlive their referents, and that a value is not mutated while it is shared.

Based on. This is the slipperiest word, and Chapter 1 split it into three senses:

1. *Historical*: the designers took the theory of affine types as an input.
2. *Specificational*: the rules the compiler enforces are the rules of an affine type system.
3. *Implementational*: the compiler works by building derivations in an affine type system.

The rest of the chapter evaluates the two parts of the claim in these senses.

33.2 Ownership: affine, with three amendments

The core is a theorem

Here is the program that opened Part V's story, from Chapter 6:

```
pub fn checkout(coupon: Coupon) -> u32 {
    let first = redeem(coupon);
    let second = redeem(coupon);
    first + second
}
```

```
error[E0382]: use of moved value: `coupon`
  --> compile_fail/ch06_use_after_move.rs:16:25
   |
14 | pub fn checkout(coupon: Coupon) -> u32 {
   |                 ------ move occurs because `coupon` has type `Coupon`, which does not
implement the `Copy` trait
15 |     let first = redeem(coupon);
   |                         ------ value moved here
16 |     let second = redeem(coupon);
   |                          ^^^^^^ value used here after move
   |
note: consider changing this parameter type in function `redeem` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch06_use_after_move.rs:9:23
   |
 9 | pub fn redeem(coupon: Coupon) -> u32 {
   |        ------         ^^^^^^ this parameter takes ownership of the value
   |        |
   |        in this function
note: if `Coupon` implemented `Clone`, you could clone the value
  --> compile_fail/ch06_use_after_move.rs:5:1
   |
 5 | pub struct Coupon {
   | ^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
15 |     let first = redeem(coupon);
   |                         ------ you could clone this value
```

In the language of Chapter 22, the body of `checkout` is an application of `redeem` to `coupon` paired with another application of `redeem` to `coupon`. The pairing rule splits the context: each variable of the context goes to the left premise or to the right premise, never to both, unless its type is `Copy`. There is no way to split $coupon : \mathit{Coupon}$ so that both premises receive it, so there is no derivation. Error E0382 is rustc's report of the failed split, and the compiler's notes even name the two ways out that the theory offers: borrow instead of consuming (Part VI), or make the type duplicable with `Clone` (Chapter 23).

This is not a loose analogy. For λ^{aff} we proved in Lean the **usage theorem**: in every well-typed term, every variable is used at most once, counting the two alternatives of an offer or a branch with a maximum rather than a sum. Its proof rests on a single lemma about context splitting, and the algorithmic checker that threads "leftover" contexts was proved sound and complete with respect to the typing rules. The `affine` crate implements the same algorithm for a Rust-like syntax and reproduces the core of rustc's E0382 messages. The move checker on the non-`Copy` fragment of Rust enforces exactly the invariant of this theorem. In the *specificational* sense, this part of the claim is true.

It needs three amendments and one refinement, each of which the book worked out.

Amendment 1: `Copy` restores contraction

Not every type in Rust is affine. For `Copy` types, contraction and weakening are silent: `let b = a;` copies, and `a` remains usable. Chapter 23 identified this with linear logic's exponential modality: a `Copy` type behaves as if it had the form $!\tau$, for which the structural rules hold again. The side conditions of `Copy` are the side conditions of the exponential. *Promotion* says that a duplicable value can only be built from duplicable parts, and rustc enforces it as E0204 (a struct with a `String` field cannot be `Copy`). A value with a destructor cannot be duplicated silently, which is E0184.

Formally, λ^{aff} with `Copy` adds one splitting rule, S-Both, which sends a `Copy` variable to both premises, and leaves every typing rule unchanged. The generalized usage theorem, proved in Lean, says that non-`Copy` variables are still used at most once. Chapter 23 also proved why promotion is not a technicality: if function types were `Copy` regardless of what they capture, a well-typed program could step to an ill-typed one. So the accurate statement is that Rust is affine *for non-`Copy` types*, and structural for `Copy` types.

Amendment 2: `Clone` is a term, not a rule

`Clone` is often mentioned together with `Copy`, but in the theory it is something else entirely. A structural rule is invisible: it changes what the context may do without appearing in the term. `Clone` is visible: `x.clone()` is an ordinary function call, a term of type $\tau \multimap \tau \otimes \tau$ that consumes a borrow of its input and produces a new value, at a cost and possibly with effects. Every affine system can contain such terms for particular types; having them does not make the system less affine. `Rc` and `Arc` go one step further and make sharing itself a counted resource: each clone and each drop of the pointer is bookkept, which is precisely why they cannot be `Copy`.

Amendment 3: weakening has an effect, and nothing forces consumption

Weakening is kept: you may let a value go out of scope unused. But in Rust, discarding is not free. It runs the value's destructor (Chapter 24). The typing rule is affine, but the dynamics are not indifferent to the discard. The book's slogan was *affine in typing, linear in dynamics*: every value that is not moved is dropped exactly once, at a point fixed by the rules of drop order.

Then came the second half of that chapter. "Exactly once" is a property of the ordinary dynamics, not a guarantee of the type system: `mem::forget`, `Box::leak` and reference cycles of `Rc` can all prevent a destructor from running, and all of them are safe. Since 2015, Rust's soundness may not depend on destructors running. For the small resource language of Chapter 24 we proved in Lean that no resource is ever released twice, that without `forget` every resource is released exactly once, and that `forget` breaks the second property but not the first. That is the precise difference between affine and linear. Rust guarantees "at most once"; nothing in Rust 1.99 lets you declare a type whose values *must* be consumed. That is why typestate (Chapter 25) can prevent a protocol from being used wrongly but cannot force it to be finished.

The refinement: places, not variables

λ^{aff} counts uses of *variables*. Rust counts moves of *places*: you may move one field out of a struct and keep using the others, and the compiler tracks this per field (Chapter 22). This is a refinement of the same discipline, not a different one: a pair in λ^{aff} is taken apart by a `let` that binds both components at once, and Rust's partial moves are what you get when the pattern is spread out over several statements and the compiler keeps track of which parts are gone.

So the first half of the claim, stated with care, is true: **Rust's ownership discipline is an affine type system on non-`Copy` types, with `Copy` playing the role of the exponential, `Clone` an explicit duplication term, and weakening attached to a destructor.**

33.3 Borrowing: a different theory

Shared references are not affine at all

Now the second half. Start with the simplest fact from Chapter 26:

```
/// Contraction, available only for `Copy` types: the value is used twice.
fn duplicate<T: Copy>(x: T) -> (T, T) {
    (x, x)
}

fn references_are_copy() {
    let report = String::from("sensor report");
    // `String` is not `Copy`, but `&String` is: a shared reference can be
    // duplicated freely, and both copies point to the same string.
    let (r1, r2) = duplicate(&report);
    println!(
        "r1 = {r1:?}, r2 = {r2:?}, same address: {}",
        std::ptr::eq(r1, r2)
    );
    // `duplicate(&mut report)` would not compile: `&mut String` is not `Copy`.
}
```

```
r1 = "sensor report", r2 = "sensor report", same address: true
```

`&T` is `Copy`, for every `T`. A shared reference can be duplicated, discarded and used in any order: it is a *fully structural* value, the exact opposite of an affine one. Whatever rejects the bad programs of Part VI, it cannot be a restriction on how many times a shared reference is used, because there is none.

Mutable references are different: `&mut T` is not `Copy`, and moving one is E0382, word for word as for an owned value. But look at the error that really characterizes mutable references:

```
pub fn transfer(accounts: &mut Vec<i64>, amount: i64) {
    let from = &mut accounts[0];
    let to = &mut accounts[1];
    *from -= amount;
    *to += amount;
}
```

```
error[E0499]: cannot borrow `*accounts` as mutable more than once at a time
 --> compile_fail/ch26_two_mut.rs:8:19
  |
7 |     let from = &mut accounts[0];
  |                ---------- first mutable borrow occurs here
8 |     let to = &mut accounts[1];
  |                   ^^^^^^^^ second mutable borrow occurs here
9 |     *from -= amount;
  |     ---------------- first borrow later used here
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

`from` and `to` are two *different* variables, and each is used exactly once. An affine checker counts uses of variables, so it would accept this program. The conflict is between two values that point into the same place, and a type system that only counts variables cannot see what they point to.

The dangling-reference term

Chapter 26 made the same point formally. Take λ^{aff}, add a heap with owned boxes, and add a duplicable reference type whose borrowing rule is "affine-respectable" (it consumes nothing). Every rule respects affinity, and yet one can type a term that frees a box and then reads it through a reference. Affine typing plus duplicable references is unsound. There are two ways to repair it, and Rust uses both:

- **lend and return**: borrowing consumes the owner for the duration of a delimited scope and gives it back afterwards;
- **regions**: every reference type carries the name of a region of the program, and no reference may be used outside its region.

Neither is about multiplicity. Both are about *time*: during which part of the program a reference is valid. That was the box in Chapter 26 that this whole chapter turns on: affinity is a statement about how many times; lifetimes are a statement about when.

How much of the aliasing rule is affinity?

Here the book's answer is more precise than the folklore, in both directions. Chapter 28 built λ^{aff}_{rgn}, a calculus with moves, drops, `Copy`, shared and mutable borrows, lifetimes and reborrowing, in which a

borrow *lends* its owner and returns it, so that a borrow expression has type $Box\,\tau \otimes \sigma$. In that calculus the classic aliasing bug

```
pub fn aliasing_bug() {
    let mut v: Vec<i32> = vec![1, 2, 3];
    let first: &i32 = &v[0];
    v.push(4);
    println!("{first}");
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> compile_fail/ch28_aliasing_bug.rs:9:5
   |
 8 |     let first: &i32 = &v[0];
   |                       - immutable borrow occurs here
 9 |     v.push(4);
   |     ^^^^^^^^^ mutable borrow occurs here
10 |     println!("{first}");
   |                ----- immutable borrow later used here
```

fails for an affine reason: the owner `v` would have to go to both premises of the borrow rule, once to the borrow and once to the code that pushes, and a `Box` is not `Copy`. Rustc says E0502; the calculus says "failed split". So affinity *does* contribute to the aliasing rule, more than the slogan "lifetimes are not affine" suggests.

But notice what made that possible. The split only works because the borrow is a *scoped construct* that holds the owner for the duration of its body: that is a region. And the calculus needs a second ingredient that no counting argument can supply, the **escape check**: a reference must not leave the scope of the borrow that created it, which is a condition on which region names may appear in a type ($\alpha \notin \mathrm{flv}(\sigma)$). Chapter 28 proved the usage theorem for the calculus (a mutable reference, like any non-`Copy` variable, is used at most once) and noted explicitly that the escape check is the part of the system the usage theorem cannot explain. So the aliasing discipline is affinity of owners *inside* a region discipline. Without the regions, affinity alone produces the unsound dangling-reference term.

Lifetimes are regions; their order is subtyping

What, then, is a lifetime? Chapter 26 answered: a *region variable*, in the tradition of Tofte and Talpin's region calculus and Cyclone. Functions are polymorphic over lifetimes; `'a: 'b` is an outlives constraint that orders them; `'static` is the largest region; elision fills in the obvious cases; the escape check (E0597, E0515) keeps references inside their regions. Chapter 27 added the rest: the outlives order induces Rust's only form of subtyping (`&'long T <: &'short T`), variance says how that subtyping lifts through type constructors, and variance turned out to be functoriality on the preorder category of lifetimes. The classic unsound program that invariance of `&mut T` prevents has nothing to do with how many times anything is used.

Permissions

Finally, Chapter 26 gave references a reading that unifies the rest: a reference is a *permission*. The owner has full permission (read, write, free); `&mut T` is a unique read-write permission; `&T` is a duplicable read permission; a borrow is a temporary transfer of permission from the owner to the reference, and the aliasing rule is a disjointness condition on permissions. That is the vocabulary of

fractional permissions and of separation logic, and Chapter 32 showed that the semantic proof of Rust's soundness, RustBelt, is written exactly in that logic. Permissions can be split and recombined; they are not counted as uses of a variable.

So the second half of the claim is false as stated. **Lifetimes are region variables, the outlives relation is subtyping, and the borrow rules are a permission discipline. Affinity contributes one ingredient (owners and mutable references cannot be duplicated), but the theory of borrowing is the theory of regions and permissions.**

33.4 How the compiler really does it

There remains the third sense of "based on". Does rustc *work* by building derivations in an affine type system, the way our `affine` and `borrowck` crates do?

It does not. Chapter 29 followed a program through the compiler: type checking with lifetimes as inference variables, lowering to MIR (a control-flow graph of basic blocks acting on places, with explicit storage markers and drops), and then a family of *dataflow analyses* over that graph. Moves are checked by computing, at every program point, which places may be uninitialized (E0382, E0381). Borrows are checked by computing regions as sets of program points from liveness and outlives constraints, then reporting every access that conflicts with a loan in scope (E0499, E0502, E0505, E0506, E0597). Two-phase borrows accept `v.push(v.len())`; the location-insensitive constraints of NLL still reject "problem case #3", which Polonius, a reformulation of the analysis in terms of loans and origins, accepts. The `mini-nll` crate reproduced rustc's error codes on real programs with these analyses, and in Lean we proved that the iterative solver computes the least fixpoint and that NLL regions are contained in lexical ones.

None of this is proof search. The relationship between the type systems of Parts V and VI and the compiler is the relationship between a *specification* and an *implementation*: the calculus says which programs are acceptable (and is in fact stricter than today's compiler, which accepts programs that lexically scoped borrows reject); the analysis decides acceptability efficiently, on a representation the calculus does not even have. In the implementational sense, the claim is false for both halves.

33.5 Three senses, three answers

Sense of "based on"	Ownership (moves, `Copy`, `Drop`)	Lifetimes and borrowing
Historical	plausible, but says little	regions, not affinity
Specificational	**true**: an affine system on non-`Copy` types	**false**: regions, subtyping, permissions
Implementational	false: dataflow on MIR	false: dataflow on MIR

The historical row deserves a word of caution, because it is the one this book can say least about. Rust's design drew on several research traditions, among them region-based memory management (Cyclone and the ML Kit), unique pointers and move semantics, and typestate, which early Rust had as a language feature and later removed. Affine and linear types were part of the same landscape. But

a historical influence is not a description: knowing that the designers read about affine types tells you nothing about which rules the compiler enforces. The specificational row is the one that answers the question a programmer actually cares about, and it is the one we could settle with proofs.

33.6 The verdict in tables

Rust features and their accounts

Rust feature	Correct account	Chapters
move of a non-`Copy` value	affine typing: contraction absent	6, 22
unused value, end of scope	weakening, kept, with a destructor as effect	6, 24
`Copy`	the exponential !: contraction and weakening restored	23
`Clone`	an explicit duplication term $\tau \multimap \tau \otimes \tau$	23
`Rc`, `Arc`	counted sharing of a heap value	23, 24
`mem::forget`, leaks	affine, not linear: no "must consume"	24
partial moves	the affine discipline, tracked per place	22
typestate	affinity makes old states unusable	25
`&'a T`	region-indexed duplicable read permission	26, 28
`&'a mut T`	region-indexed unique write permission (affine)	26, 28
`'a: 'b`, variance	outlives preorder, subtyping, functoriality	26, 27
reborrowing	lend-and-return of a reference within a nested region	28
the escape check	region discipline, not counting	26, 28
borrow checking in practice	dataflow over MIR (NLL; Polonius in development)	29
soundness of `unsafe` libraries	semantic typing in an affine separation logic	32

Which structural rules Rust keeps

	Exchange	Weakening	Contraction
owned non-`Copy` values	yes	yes, with `Drop`	no
`Copy` values	yes	yes	yes
`&'a mut T`	yes	yes	no
`&'a T`	yes	yes	yes

Only the first and third rows are affine. Shared references are in the fully structural row, together with integers.

Error codes and the theory behind them

Error	Meaning	Theory
E0382	use of moved value	no contraction (failed context split)
E0204, E0184	invalid `Copy`	side conditions of ! (promotion, no destructor)
E0499	two `&mut` to one place	uniqueness of write permission
E0502	`&mut` while shared	owner lent to a scoped borrow; permissions
E0505, E0506	move or assign while borrowed	the owner is lent for a region
E0597, E0515	reference outlives referent	escape check: regions
E0040, E0509	calling `drop`, moving out of `Drop`	"dropped at most once"
E0381	use of uninitialized	dataflow: maybe-uninitialized places

Of the borrow-related rows, only E0499 can be explained by counting uses of something, and even then the something is a permission to a place, not a variable.

33.7 Where affinity reappears

There is one more place where "affine" is exactly the right word, and it may be the origin of the folklore. Chapter 32 described how RustBelt proved Rust's type system sound: not by a syntactic argument like Chapter 9's, but by interpreting every type as a predicate in Iris, a higher-order concurrent separation logic, and proving that every well-typed program, and every library that uses `unsafe` correctly, satisfies its predicate. Iris is an **affine** separation logic: from $P * Q$ you may conclude P, discarding Q. We proved the corresponding fact for the separation logic of Chapter 32, and saw that it holds in the up-closed model and fails in the exact one.

So affinity is a correct description of the *logic in which Rust's soundness is proved*, one level below the language. It is not a description of the surface rules for borrowing. When someone says "Rust is based on affine logic", they may be compressing two true statements, "ownership is an affine type system" and "Rust's soundness proof lives in an affine separation logic", into one false one, "lifetimes are affine types".

33.8 How to say it correctly

> **IMPORTANT**
>
> **Say:** "Rust's ownership and move semantics implement an affine discipline: a binding of a non-`Copy` type may be used at most once, which is the absence of contraction. `Copy` restores contraction for a class of types, playing the role of linear logic's exponential, and `Clone` is explicit duplication. Weakening is kept, so values may be discarded (with a destructor running), which makes Rust affine rather than linear: it cannot force a value to be consumed. Borrowing and lifetimes are a different mechanism: a region system with an outlives order, variance and an escape check, and a permission discipline in which shared references are freely duplicable and mutable references are unique. The compiler checks all of it with dataflow analyses on MIR."
>
> **Do not say:** "Lifetimes are affine types." A shared reference is `Copy`, the opposite of affine; lifetimes measure extent, not multiplicity.

If you have only one sentence: *Rust's ownership is affine; its borrowing is regions and permissions.*

33.9 Two languages, one idea, revisited

Chapter 1 promised that Rust and Lean would illuminate each other. It is worth looking back at what each of them taught.

Rust's type system is a federation of theories. Its algebraic data types are the products and sums of Chapter 11. Its generics are System F restricted to rank one and implemented by monomorphization (Chapter 12), with local Hindley–Milner inference (Chapter 13). Its traits are type classes elaborated into dictionaries (Chapter 14), and `impl Trait` and `dyn Trait` are existential types (Chapter 15). Its functions are the morphisms of a category that is *not* cartesian closed for non-`Copy` types, because the diagonal needs `Clone` (Chapter 19). Its `?` is the bind of two monads it cannot name (Chapter 20). Its ownership is an affine type system (Part V), its borrowing a region and permission system (Part VI), its const generics a first step toward dependent types (Chapter 30), and its `unsafe` a door into semantic typing (Chapter 32). Each theory was invented separately, decades apart, and Rust's achievement is to make them work together in a practical language. Each also has a seam where Rust departs from the textbook, and those seams (`panic!` inhabiting every type, inference stopping at signatures, no higher-kinded types, leaks being safe, borrow checking by analysis) were among the most instructive parts of the book.

Lean's type theory is one theory. Everything we built in Lean, from the judgments of Chapter 3 to the separation logic of Chapter 32, was built from the same few ingredients: dependent function types, inductive types, a hierarchy of universes, and the impredicative universe `Prop`. A typing judgment was an inductive predicate; a derivation was a value of it; a metatheorem was a function on derivations; a category was a structure whose fields include proofs; a protocol was an indexed family; "this program is correct" was a type. Curry–Howard was not a slogan there but the way the system works.

Rust showed us where the ideas *live*: in error codes, in API design, in the compromises a language makes to be fast and usable. Lean showed us what the ideas *are*: definitions precise enough to be checked by a machine, and theorems that say exactly what a rule guarantees. Rust implements; Lean proves. Neither would have been enough alone.

33.10 Where to go next

Appendix F lists books and papers for every part of the book. A few directions stand out.

- **Lean and Mathlib.** *Theorem Proving in Lean 4*, *Functional Programming in Lean* and *Mathematics in Lean* continue where Chapters 2 and 31 stop. Every Mathlib section of Part IV is an entry point into a library with hundreds of thousands of theorems.
- **Type theory and semantics.** Pierce's *Types and Programming Languages* and Harper's *Practical Foundations for Programming Languages* cover the ground of Parts I–III in depth; *Software Foundations* and *Programming Language Foundations in Agda* mechanize it, as we did for our toy languages.
- **Linear logic and substructural types.** Girard's original paper, Walker's chapter on substructural type systems, and Pfenning's lecture notes take Part V further.
- **Rust's foundations.** The RustBelt papers, Ralf Jung's thesis, Stacked and Tree Borrows, and the work on Polonius are where the questions of Part VI are being answered today.
- **Verifying Rust.** Aeneas, Verus, Creusot, Prusti and Kani bring the methods of Chapter 31 to real Rust code. Aeneas, in particular, translates Rust into Lean, closing the loop between the two languages of this book.

33.11 Summary

- "Affine" means weakening without contraction: every assumption is used at most once. "Based on" can be historical, specificational or implementational.
- Ownership is affine in the specificational sense, and the usage theorem of Chapter 22 is the precise statement: rustc's E0382 is a failed context split. Three amendments: `Copy` is the exponential and restores contraction; `Clone` is an explicit duplication term; weakening runs a destructor, and nothing forces consumption, so Rust is affine, not linear. Rust refines the discipline to places.
- Borrowing is not affine. `&T` is fully structural; the conflicts the borrow checker reports are between different variables pointing to one place; affine typing with duplicable references is unsound. The repairs, lend-and-return and regions, are about time.
- In the borrow calculus of Chapter 28, the aliasing bug is a failed split of the owner, but only inside a scoped borrow, and the escape check is beyond what affinity explains. Lifetimes are region variables, their order is subtyping, variance is functoriality, and references are permissions.
- rustc checks moves and borrows with dataflow analyses on MIR, not by building derivations: the implementational sense is false for both halves.

- Affinity reappears one level down: Iris, the logic of RustBelt's soundness proof, is an affine separation logic.
- Say: Rust's ownership is affine; its borrowing is regions and permissions.

33.12 Exercises

1. For each of the errors E0382, E0499, E0502, E0505, E0597 and E0204, write a five-line program that triggers it, and say in one sentence which part of the theory rejects it: a failed context split, a side condition of the exponential, a permission conflict, or a region constraint. *Hint:* the error-code table above, and the compile-fail programs of Chapters 6, 23, 26 and 29, which you can adapt.
2. Imagine a Rust with an opt-in linear marker trait: values of a `Linear` type may not be dropped implicitly, only consumed by a function that takes them by value. Which structural rule does `Linear` remove? Rewrite the connection typestate of Chapter 25 with it, and explain which mistake it would now catch. Then explain why `mem::forget` and `Rc` would need special treatment. *Hint:* Chapter 24's discussion of what linearity would buy, and its Lean theorem about `forget` breaking "exactly once".
3. Take the dangling-reference term of Chapter 26 and type it in the calculus of Chapter 28. Which rule fails, and is the failure an affine one (a failed split) or a region one (the escape check)? *Hint:* consider which variables the borrow construct consumes, and what type the body of the borrow would need.
4. In Lean, write down the table "which structural rules Rust keeps" as a function from a small inductive type of "kinds of value" (owned, `Copy`, shared reference, mutable reference) to a structure of three booleans, and prove by `decide` that exactly two kinds lack contraction. *Hint:* `deriving DecidableEq` on the inductive type, and a theorem of the form `(kinds.filter (fun k => !(rules k).contraction)).length = 2`.
5. Write the "How to say it correctly" paragraph for three different audiences: a Rust beginner, a programmer who knows Haskell, and a logician who has never used Rust. What does each of them need you to define, and what can you leave out? *Hint:* the beginner needs moves and borrows, not structural rules; the logician needs `Copy` and `&T`, not lifetimes syntax.

Appendices

The appendices collect reference material: how to install and use the two toolchains, a cheat sheet for reading Lean as a Rust programmer, every notation used in the book with a dictionary between logic, type theory, category theory, Rust and Lean, the tactics used in the proofs, a glossary and annotated further reading.

APPENDIX A

Setting Up Rust and Lean

This appendix explains how to install the two toolchains the book uses, how to set up an editor that shows Lean's goals as you work, how to run the code and the verification tools, and how to rebuild the book itself.

In this appendix you will learn:

- how to install Rust with rustup and Lean with elan, and how the repository pins the exact versions;
- how to download the prebuilt Mathlib library;
- how to set up VS Code for Rust and Lean, and how to type Lean's symbols;
- how to run the examples, check the proofs and verify the whole book;
- how to rebuild the Markdown chapters and the PDFs.

A.1 The repository

Clone the repository, or download and unpack it, anywhere you like. All the commands in this appendix are run from the repository root unless a `cd` says otherwise.

```
book/              the chapters
code/
├── rust/          Cargo workspace (Rust 1.99.0, standard library only)
└── lean/          Lake project (Lean 4.34.1 with Mathlib)
tools/             build and verification scripts
```

A.2 Rust

Install Rust with **rustup**, the official toolchain manager. On macOS and Linux:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

On Windows, download and run `rustup-init.exe` from https://rustup.rs; it will also ask you to install the Visual Studio C++ build tools if they are missing.

You do not need to choose a Rust version. The file `code/rust/rust-toolchain.toml` pins the compiler the book was written with:

```
# The book shows real compiler diagnostics, so the exact compiler is pinned.
# rustup installs it automatically the first time you run cargo here.
```

```
# rust-src is the standard library's source, which rust-analyzer needs.
[toolchain]
channel = "1.99.0"
components = ["rustfmt", "rust-src"]
```

The first time you run `cargo` inside `code/rust`, rustup notices the file and downloads Rust 1.99.0 (and `rustfmt`) automatically. Everywhere else your default toolchain is used. The version is pinned because the book shows real compiler diagnostics, and their wording changes between releases. Newer compilers accept the same programs; only some messages may read differently.

Check the installation:

```
cd code/rust
cargo run --example ch01_first_types
```

The workspace has no external dependencies, so this builds in seconds.

A.3 Lean and Mathlib

Install Lean with **elan**, Lean's toolchain manager (the counterpart of rustup). On macOS and Linux:

```
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -sSf | sh
```

On Windows, and for other options, follow the instructions on https://lean-lang.org. Installing the Lean 4 extension for VS Code (see below) also offers to install elan for you.

Again, you do not need to choose a version. The file `code/lean/lean-toolchain` contains a single line, `leanprover/lean4:v4.34.1`, and elan downloads that version the first time you run `lean` or `lake` in the directory.

Lake is Lean's build tool and package manager, the counterpart of Cargo. The project's configuration is `code/lean/lakefile.toml`:

```
# The book's Lean project. It depends on Mathlib, pinned to the release that
# matches the toolchain in lean-toolchain. Before the first build, download the
# prebuilt library (under 1 GB to download, about 8 GB on disk) with
#   lake exe cache get
# otherwise Lake would compile Mathlib from source, which takes hours.
name = "book"
version = "0.1.0"
defaultTargets = ["Book"]

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4"
rev = "v4.34.1"

[[lean_lib]]
name = "Book"
globs = ["Book.+"]
```

It declares one library, `Book`, whose modules are the files under `code/lean/Book/` (the module `Book.Ch05.Kripke` is the file `Book/Ch05/Kripke.lean`), and one dependency, Mathlib, pinned to the release that matches the toolchain. The exact revisions of Mathlib and of its own dependencies are recorded in `lake-manifest.json`, the counterpart of `Cargo.lock`.

Mathlib is large: compiling it from source takes hours. Its maintainers publish prebuilt files for every release, and a command provided by Mathlib downloads them. Run it once, before the first build:

```
cd code/lean
lake exe cache get
```

The download is under a gigabyte of compressed files, which take about 8 GB once unpacked (with the sources of Mathlib and its dependencies) in `code/lean/.lake/`; the compressed files are also kept in a cache in your home directory so that other projects using the same Mathlib release can share it. Then check every Lean file of the book:

```
lake build
```

A build after the first one only rechecks the files that changed. You can also check a single file, which prints every message Lean produces for it:

```
lake env lean Book/Ch01/FirstSteps.lean
```

`lake env` runs a command with the project's search paths set, so that `import Mathlib...` finds the library.

> **WARNING**
>
> If Lean reports `unknown module prefix 'Mathlib'`, or starts compiling hundreds of Mathlib files, the prebuilt library is missing or does not match. Run `lake exe cache get` again in `code/lean`. Never run `lake update` in the book's project: it would move Mathlib to a newer revision than the one the book was checked with.

A.4 An editor for Lean

You can read the Lean files in any editor, but to *work* with Lean you want an editor that runs Lean as you type. The standard choice is **VS Code** with the **Lean 4** extension (published by `leanprover`). For Rust, add the **rust-analyzer** extension.

Open the folder `code/lean` itself in VS Code (not the repository root), so that the extension finds `lakefile.toml` and the toolchain. Open a file such as `Book/Ch01/FirstSteps.lean`. After a moment the extension starts Lean, and a panel called the **infoview** appears on the right. The infoview is the most important tool for learning Lean:

- With the cursor on a command such as `#eval` or `#check`, it shows the output.
- With the cursor inside a proof, it shows the **goal**: the hypotheses available at that point (above the turnstile `⊢`) and what remains to be proved (below it). Moving the cursor from tactic to tactic shows how each step changes the goal. This is how the comments in the book's proofs were written, and it is the best way to read them.
- With the cursor on an error (underlined in red), it shows the full message.

Hovering over a name shows its type and documentation; Ctrl-click (Cmd-click on macOS) jumps to its definition, including definitions inside Lean's core library and Mathlib. Reading the source of a definition is often the quickest way to understand it.

Typing symbols

Lean code uses Unicode symbols such as `→`, `∀`, `ℕ` and `⟨ ⟩`. In VS Code you type them with a backslash followed by a name, and the extension replaces the sequence as you type. A few you will use constantly:

Symbol	Type	Symbol	Type
`→`	`\to`	`ℕ`	`\N`
`∀`	`\forall`	`ℤ`	`\Z`
`∃`	`\exists`	`⟨ ⟩`	`\<` and `\>`
`∧`	`\and`	`≤`	`\le`
`∨`	`\or`	`≠`	`\ne`
`¬`	`\not`	`↔`	`\iff`
`λ`	`\lambda`	`⊢`	`\vdash`
`α β γ`	`\alpha` …	`∘`	`\circ`

Most LaTeX names work (`\Gamma`, `\tau`, `\otimes`, `\times`). When you meet a symbol you do not know how to type, hover over it in the editor: the tooltip shows the input sequence.

A.5 Running and verifying the code

The Rust examples are run with Cargo from `code/rust`. Single-file examples are in `code/rust/examples/`; examples that use one of the book's crates live in that crate's `examples/` directory and need `-p`:

```
cd code/rust
cargo run --example ch05_kripke
cargo run -p stlc --example ch08_stlc
cargo test -p affine
```

The book shows the output of every example. Those outputs are stored as snapshots in `code/rust/output/`, and the programs that must be *rejected* by the compiler are in `code/rust/compile_fail/`, each with a `.stderr` file holding the diagnostics the book shows. A script checks all of it, together with every Lean file:

```
python3 tools/run_examples.py                # everything
python3 tools/run_examples.py --only ch22    # one chapter
python3 tools/run_examples.py --rust --tests # Rust examples and the crates' tests
python3 tools/run_examples.py --lean         # Lean only
```

For each Rust example it compiles the program (failing on any warning), runs it and compares its output with the snapshot. For each compile-fail program it checks that the compiler rejects it with the expected error codes and compares the diagnostics with the `.stderr` file. For Lean it runs `lake build` and fails if Lean prints any message at all: every output shown in the book is wrapped in `#guard_msgs`,

which checks it against the comment above the command. When you change an example on purpose, `--bless` rewrites the snapshots from the real output:

```
python3 tools/run_examples.py --only ch22 --bless
```

The script needs Python 3.11 or newer and no extra packages.

A.6 Building the book

The chapters are written in Markdown in `book/src/`, with `{{#include}}` directives that pull code, outputs and diagnostics from `code/`. The build script expands them into self-contained Markdown files in `book/chapters/`, renders the diagrams, and produces a PDF for each chapter and one for the whole book:

```
python3 tools/build_book.py check    # validate includes, anchors, math and images
python3 tools/build_book.py all      # diagrams, chapters/*.md, PDFs, README toc
python3 tools/build_book.py pdf --only 08 --no-book   # one chapter's PDF
```

It requires pandoc 3.x, Typst 0.13 or newer and Graphviz (`dot`). On macOS all three are available from Homebrew (`brew install pandoc typst graphviz`); on Linux, from your distribution or the projects' release pages. The conventions for writing chapters, examples and proofs are in `tools/AUTHORING.md`.

A.7 Summary

- rustup and elan install the toolchains; `rust-toolchain.toml` and `lean-toolchain` pin Rust 1.99.0 and Lean 4.34.1, and are honored automatically.
- Run `lake exe cache get` once in `code/lean` to download prebuilt Mathlib; never run `lake update` in the book's project.
- Work on Lean files in VS Code with the Lean 4 extension and keep the infoview open; type symbols with backslash sequences.
- `cargo run --example ...` runs an example, `lake build` checks the proofs, and `tools/run_examples.py` verifies everything the book shows.
- `tools/build_book.py all` rebuilds the chapters and PDFs.

APPENDIX B

Lean 4 for Rust Programmers: Cheat Sheet

This appendix is a quick reference for reading and writing Lean when your instincts are Rust's. Chapter 2 explains everything here in detail; the tables are for looking things up. The longer examples come from `code/lean/Book/AppB/CheatSheet.lean`, which compiles with the book's toolchain.

In this appendix you will learn:

- the Lean spelling of the Rust constructs you use every day;
- where the two languages differ in ways that matter (totality, immutability, implicit arguments, propositions);
- the commands, notations and tactics that appear most often in the book.

B.1 Syntax side by side

Rust	Lean	Notes
`fn f(x: u64) -> u64 { ... }`	`def f (x : ℕ) : ℕ := ...`	no `return` needed
`fn g<T>(x: T) -> T`	`def g {α : Type} (x : α) : α`	`{}` = implicit, inferred
`|x| x + 1`	`fun x => x + 1` or `(· + 1)`	`·` is the argument
`f(a, b)`	`f a b`	application is juxtaposition
`let x = e;`	`let x := e`	
`if c { a } else { b }`	`if c then a else b`	
`match e { P => a, ... }`	`match e with | P => a | ...`	must be exhaustive
`struct S { a: A }`	`structure S where a : A`	
`S { a: 1 }, S { a: 2, ..s }`	`{ a := 1 }`, `{ s with a := 2 }`	also `⟨1⟩`
`enum E { A, B(u8) }`	`inductive E where | a | b (n : UInt8)`	constructors `E.a`, `.a`
`impl S { fn m(&self) }`	`def S.m (s : S) ...`	called as `s.m`
`trait T { fn f(&self); }`	`class T (α : Type) where f : α → ...`	
`impl T for S { ... }`	`instance : T S where f := ...`	
`fn h<X: T>(x: X)`	`def h {X : Type} [T X] (x : X)`	`[]` = instance argument
`#[derive(Debug, PartialEq)]`	`deriving Repr, BEq`	also `DecidableEq`, `Hashable`
`mod m { ... }` / `use m::*;`	`namespace m ... end m` / `open m`	
`Option<T>`: `Some(x)`, `None`	`Option α`: `some x`, `none`	
`Result<T, E>`: `Ok`, `Err`	`Except ε α`: `.ok`, `.error`	note the order of arguments
`e?`	`← e` inside `do`	
`Vec<T>`, `vec![1, 2]`	`Array α`, `#[1, 2]`	`List α`, `[1, 2]` in proofs
`HashMap<K, V>`	`Std.HashMap K V`	`import Std.Data.HashMap`
`String`, `format!("{x}")`	`String`, `s!"{x}"`	
`u64`, `i64`, `usize`	`UInt64`, `Int64`, `USize`	
(no equivalent)	`ℕ`, `ℤ`	unbounded naturals and integers
`bool`, `true`	`Bool`, `true`	also `Prop`: see below
`()`	`()` of type `Unit`	
`!`, `enum Void {}`	`Empty` (data), `False` (proposition)	
`(A, B)`, `p.0`	`A × B`, `p.1` / `p.fst`	
`fn main() { println!(...) }`	`def main : IO Unit := IO.println ...`	
`// comment`, `/* */`, `///`	`-- comment`, `/- -/`, `/-- -/`	

B.2 Functions and definitions

```
-- fn square(x: u64) -> u64 { x * x }
def square (x : ℕ) : ℕ := x * x

-- Generic, with the type parameter implicit (inferred at call sites):
-- fn twice<T>(f: impl Fn(T) -> T, x: T) -> T { f(f(x)) }
```

```
def twice {α : Type} (f : α → α) (x : α) : α := f (f x)

-- A closure: |n| n + 1
#guard twice (fun n => n + 1) 5 = 7

-- Pattern matching in the definition itself (like a `match` on the argument):
def fib : ℕ → ℕ
  | 0 => 0
  | 1 => 1
  | n + 2 => fib n + fib (n + 1)
```

Every definition must be **total**: it must return a value for every input, and recursion must terminate. Lean checks structural recursion automatically (each call on a smaller piece of the input), accepts other recursion when you prove termination with `termination_by` / `decreasing_by`, and accepts possibly non-terminating functions only when marked `partial`, which hides their bodies from proofs. `#guard` checks a `Bool` condition at compile time, like a compile-time `assert!`.

B.3 Structures and inductive types

```
-- struct Point { x: f64, y: f64 }  +  #[derive(Debug, Clone, PartialEq)]
structure Point where
  x : Float
  y : Float
  deriving Repr, BEq

-- impl Point { fn norm2(&self) -> f64 { ... } }   (methods live in the namespace)
def Point.norm2 (p : Point) : Float := p.x * p.x + p.y * p.y

def origin : Point := { x := 0, y := 0 }
def p : Point := ⟨3, 4⟩            -- anonymous constructor
def q : Point := { p with y := 0 } -- struct update syntax: Point { y: 0.0, ..p }

/-- info: 25.000000 -/
#guard_msgs in
#eval p.norm2                      -- dot notation: Point.norm2 p
```

```
-- enum Shape { Circle(f64), Rect { w: f64, h: f64 } }
inductive Shape where
  | circle (r : Float)
  | rect (w h : Float)
  deriving Repr

def Shape.area : Shape → Float
  | .circle r => 3.14159 * r * r
  | .rect w h => w * h

-- A recursive enum needs no Box: enum Tree { Leaf, Node(Box<Tree>, u64, Box<Tree>) }
inductive Tree where
  | leaf
  | node (left : Tree) (value : ℕ) (right : Tree)

def Tree.sum : Tree → ℕ
  | .leaf => 0
  | .node l v r => l.sum + v + r.sum
```

Values are immutable. "Updating" a structure builds a new one; Lean's runtime reuses the old memory in place when nothing else refers to it (Chapter 24). Recursive types need no `Box`: values of inductive types are heap-allocated and reference-counted behind the scenes. `.circle` is short for `Shape.circle` when the expected type is known, like Rust's `Self::Circle` in a `match`.

B.4 Option, Except and `do`

```
-- Option<T> and Result<T, E>
def safeDiv (a b : ℕ) : Option ℕ :=
  if b = 0 then none else some (a / b)

def parsePort (s : String) : Except String ℕ :=
  match s.toNat? with
  | some n => if n < 65536 then .ok n else .error s!"{n} is not a port"
  | none => .error s!"not a number: {s}"

/-- info: Except.ok 8080 -/
#guard_msgs in
#eval parsePort "8080"

-- The `?` operator is `←` inside `do`:
def addPorts (a b : String) : Except String ℕ := do
  let x ← parsePort a
  let y ← parsePort b
  return x + y
```

`do` notation works for every monad (Chapter 20): `Option`, `Except`, `IO`, `StateM` and your own. Inside `do`, `let x ← e` runs `e` and binds its result, returning early on failure, which is what Rust's `?` does for `Option` and `Result`.

B.5 Collections

```
-- Vec<u64> ~ Array ℕ (a contiguous array, updated in place when unshared)
def squares : Array ℕ := (Array.range 5).map square

/-- info: #[0, 1, 4, 9, 16] -/
#guard_msgs in
#eval squares

-- Linked lists are the default sequence in proofs: [1, 2, 3] : List ℕ
/-- info: [2, 4, 6] -/
#guard_msgs in
#eval [1, 2, 3].map (· * 2)

/-- info: 6 -/
#guard_msgs in
#eval [1, 2, 3].foldl (· + ·) 0

-- HashMap<String, u64>
def stock : Std.HashMap String ℕ :=
  (∅ : Std.HashMap String ℕ) |>.insert "bolts" 40 |>.insert "nuts" 75

/-- info: some 75 -/
#guard_msgs in
#eval stock.get? "nuts"
```

`Array` is the efficient, contiguous sequence for programs; `List` is the linked list that proofs prefer, because its inductive structure makes induction easy. `(· * 2)` is a function: each `·` stands for the next argument.

B.6 Type classes

```
-- trait Describe { fn describe(&self) -> String; }
class Describe (α : Type) where
```

```
  describe : α → String

-- impl Describe for Point { ... }
instance : Describe Point where
  describe p := s!"({p.x}, {p.y})"

-- fn show_all<T: Describe>(xs: &[T]) -> Vec<String>
def showAll {α : Type} [Describe α] (xs : List α) : List String :=
  xs.map Describe.describe
```

A class is a structure of operations; an instance is a value of that structure, found automatically by **instance resolution** (Chapter 14). Unlike Rust, Lean has no orphan rule and allows overlapping instances, resolved by priority. Classes can also contain *proofs*, so a Lean `Monoid` instance must prove associativity, which a Rust `impl` can only promise in a comment.

B.7 Imperative code

```
-- Imperative-looking code in `do` blocks: `let mut`, `for`, early `return`.
def firstNegativeIndex (xs : Array ℤ) : Option ℕ := Id.run do
  for h : i in [0:xs.size] do
    if xs[i] < 0 then return some i
  return none

def sumTo (n : ℕ) : ℕ := Id.run do
  let mut total := 0
  for i in [0:n+1] do
    total := total + i
  return total

/-- info: 5050 -/
#guard_msgs in
#eval sumTo 100

-- fn main() { println!("..."); }
def main : IO Unit := do
  let name := "Lean"
  IO.println s!"Hello from {name}"
```

`let mut`, `for`, `while` and early `return` are allowed inside `do` blocks, and are translated into pure functional code. `Id.run do ...` uses the "identity" monad to write imperative-looking code with no effects at all. In `for h : i in [0:xs.size]`, the name `h` is a proof that `i` is in range, which lets `xs[i]` be checked at compile time (Chapter 1 explains indexing with proofs).

B.8 Proofs

```
-- A theorem is a definition whose type is a proposition.
theorem square_eq (n : ℕ) : square n = n * n := rfl

example (a b : ℕ) : (a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2 := by ring

example (x y : ℤ) (h₁ : x < y) (h₂ : 0 < x) : 0 < x + y := by linarith

theorem fib_pos (n : ℕ) (h : 0 < n) : 0 < fib n := by
  induction n using fib.induct with
  | case1 => omega
  | case2 => decide
  | case3 n ih₁ ih₂ =>
```

```
    simp only [fib]
    omega
```

Lean	Meaning
`theorem name (args) : P := proof`	a named proof of the proposition `P`
`example : P := proof`	an anonymous proof (checked, then forgotten)
`by tac₁; tac₂` or one tactic per line	build the proof with tactics
`p ∧ q`, `p ∨ q`, `¬p`, `p → q`, `p ↔ q`	and, or, not, implies, iff
`∀ x, P x`, `∃ x, P x`	for all, there exists
`a = b`, `a ≠ b`, `a ≤ b`	equality, inequality, order
`⟨hp, hq⟩`	build an `∧`, an `∃` or a structure from its parts
`h.1`, `h.2`, `h.left`	take an `∧` apart
`Prop` vs `Bool`	propositions (proved) vs booleans (computed); `decide` converts decidable ones
`sorry`	an unproved hole; makes Lean warn; never used in this book

Appendix D describes every tactic the book uses.

B.9 Commands

Command	Does
`#eval e`	evaluate `e` and print the result
`#check e`	print the type of `e`
`#print name`	print a definition or a proof term
`#print axioms name`	list the axioms a theorem depends on
`#reduce e`	normalize `e` by unfolding definitions (slow; for small terms)
`#guard b`	fail unless the boolean `b` evaluates to `true`
`#guard_msgs in cmd`	check `cmd`'s output against the doc comment above
`set_option pp.all true in cmd`	print terms with every implicit detail

B.10 Things that surprise Rust programmers

- **Application binds tightest.** `f x + 1` is `(f x) + 1`; write `f (x + 1)` when you mean it.
- **Natural-number subtraction truncates.** In `ℕ`, `3 - 5 = 0`. Rust would panic (in debug builds); Lean returns `0` because the function must be total. Use `ℤ` when you need negative numbers.
- **Division by zero is defined.** In `ℕ` and `ℤ`, `x / 0 = 0`, again for totality. Theorems about division therefore usually carry a hypothesis such as `b ≠ 0`.

- **No implicit copying or borrowing.** Every value can be used any number of times; there is no ownership system in the types. (There is one in the runtime, through reference counting: Chapter 24.)
- **Types are values.** `Type` is itself a type, functions can take and return types, and a type can depend on a value (`Vector ℕ n`).
- **Indentation matters** inside `do` blocks, tactic blocks and `match` alternatives, as in Python.
- **Errors appear where they happen.** The editor reports each error at its position and continues; a proof with an error elsewhere still shows its goals.

APPENDIX C

Notation and Dictionary

This appendix collects the notation of the book in one place. The symbols are grouped by subject, in roughly the order in which the book meets them: judgments and contexts, logic, type formers, the λ-calculus and evaluation, substructural logic, categories, regions and permissions, and dependent types. Each entry gives a way to read the symbol aloud and the chapter where it is introduced or explained best. The appendix ends with dictionaries that line up the same ideas in logic, type theory, category theory, Rust and Lean, first for the core of the book and then for the vocabulary of ownership and borrowing.

In this appendix you will learn:

- how to read every judgment form in the book, and which chapter defines it;
- the symbols of logic, type theory, the λ-calculus, linear logic and category theory as the book writes them;
- where two chapters use two notations for the same thing, and where one symbol does several jobs;
- the Lean spellings of the same ideas, including the book's own scoped notations and Mathlib's;
- how a single idea appears as a logical connective, a type former, a categorical construction, a Rust feature and a Lean definition.

C.1 Conventions

The letters in rules and judgments are **metavariables** (Chapter 1): placeholders for any context, term or type. The book follows the usual habits of the subject: Γ and Δ for contexts; τ, σ, ρ for types; A, B, C for propositions, or for types when the logical reading matters; e, t, u, s for terms; v, w for values; x, y, z for variables; α, β, γ for type variables. Subscripts and primes make more of them: Γ_1, e', τ_2.

Inference rules are fractions, premises above the line and the conclusion below, with the rule's name on the right in a sans-serif font. A rule with no premises is an axiom. The name usually starts with a prefix that says what kind of judgment the rule derives:

Prefix	Rules for	Ch.
T-	typing (most chapters)	1
E-	evaluation; in Chapter 14, entailment of constraints	3, 9, 14
S-	synthesis (8); splitting a context (22, 23, 28); subtyping (27)	8, 22, 27
C-	checking mode (8); the `Copy` predicate (28)	8, 28
A-	the affine λ-calculus	22
O-	outlives between lifetimes	26
R-	the region discipline (26); region constraints of NLL (29)	26, 29
K-	kinding	17
Pi-, Sigma-	formation, introduction, elimination of Π and Σ	30
∧I, $\wedge E_1$, →I, ...	introduction and elimination (natural deduction)	4
⊗R, ⊗L, ...	right and left rules (sequent calculus)	4, 21

The rule for λ-abstraction is called T-Lam in Chapters 8 to 10 and T-Abs in Chapters 12 and 13; it is the same rule.

The symbol ≈ means "corresponds to, with caveats that the text states" (Preface). When the book writes a typing rule "for Rust", it is a model of what the compiler does, not an official definition.

Symbols with more than one job

A few symbols, and a few letters, are used in several senses. The context always decides, but the list may save you a moment of confusion.

Symbol	Meanings	Ch.
$\Rightarrow$	Heyting implication; exponential object $b \Rightarrow c$	5, 19
	synthesis mode $\Gamma \vdash t \Rightarrow \tau$	8
	arms of a `case`	8, 11
	a constraint, $\pi \Rightarrow \tau$	14, 15
	a natural transformation $F \Rightarrow G$	18
$\Vdash$	forcing in a Kripke model; entailment of constraints	5, 14
$\dashv$	adjunction $L \dashv R$	19, 20
	the output context of a judgment with leftovers	22, 26
$\mapsto$	"maps to", as in $x \mapsto x^2 + 1$	3, 7
	an entry of a substitution, $[\alpha \mapsto \tau]$	13
	a store update $S[\ell \mapsto v]$; points-to $\ell \mapsto v$	28, 32
:	"has type"; between lifetimes, "outlives" ($\Delta \vdash \alpha : \beta$, `'a: 'b`)	1, 26
!	Rust's never type `!`	1, 3
	the exponential $!A$ of linear logic	21, 23
	the unique morphism $!_a$ to a terminal object	19
	"send" in a session type $!\tau.\, S$	25
?	Lean's unknowns `?m`; "why not" $?\, A$; "receive" $?\tau.\, S$	13, 21, 25
$*$	the kind of types; dereference $* e$; separating conjunction	17, 28, 32
$\sqcap$, $\sqcup$	meet and join of a lattice; meet of leftover contexts or of variances	5, 22, 27
$[\![\cdot]\!]$	the meaning of a syntax: in a Heyting algebra (5), as a relation (18), in a CCC (19), as a set of values (32)	5, 18, 19, 32
$\otimes$, $\multimap$	a monoidal product and its closed structure; linear logic	19, 21

Several letters change meaning between chapters too:

Letter	Meanings	Ch.
Δ	a second context; the diagonal Δ_a; a lifetime context	6, 19, 26
ρ	an environment; a type; a renaming; a region	3, 6, 9, 26
σ	a type; a substitution of de Bruijn terms; the state of the world	6, 9, 24
κ	a kind; a lifetime	17, 28
η	the η-rules; a natural transformation; the unit of a monad	8, 18, 20
π	a constraint; a projection π_1, π_2	14, 19
Σ	a Σ-type; a store typing	10, 28
S	a substitution; a session type; a store	13, 25, 28
$\mathcal{C}$	a category; the set of `Copy` base types	16, 23
$\mathcal{D}$	a derivation; a category	3, 17

Two more differences are only typographic. The opposite category is written $\mathcal{C}^{\mathrm{op}}$ in Chapters 16 and 17 and $\mathcal{C}^{op}$ in Chapters 18 and 27. The projections of a product are fst and snd in Chapter 16 and π_1 and π_2 in Chapters 19 and 30.

C.2 Judgments and contexts

Notation	Read as	Ch.
$\Gamma \vdash e : \tau$	in context Γ, the term e has type τ	1
$x_1 : \tau_1, \ldots, x_n : \tau_n$	a context: what we assume about the free variables	1
$\Gamma, x : \tau$	Γ extended with $x : \tau$	1
$(x : \tau) \in \Gamma$, $\Gamma(x) = \tau$	the context gives x the type τ	1, 3
$\vdash e : \tau$, $\varnothing \vdash e : \tau$	e has type τ in the empty context	3
$\cdot$	the empty context	22
$\dfrac{J_1 \qquad J_2}{J}$ Name	from the premises J_1 and J_2, conclude J	1
$\mathcal{D}$, $\mathcal{D}_1$	a derivation, a tree of rule instances	3
$e_1 := x$	instantiate the metavariable e_1 with x	1
$A\ true$	the judgment "A is true"	4
$\Gamma \vdash A$	sequent: assuming Γ, A is true	4
$[A]^u$	assumption A, discharged by the rule labeled u	4
$w \Vdash A$	world w forces A	5
$\Gamma \vdash t \Rightarrow \tau$	t synthesizes the type τ	8
$\Gamma \vdash t \Leftarrow \tau$	t checks against the type τ	8
$\sigma :: \Gamma$, $\Gamma(i)$	a context of de Bruijn positions, and its i-th type	9
$\lvert\Gamma\rvert \vdash \tau$	the context with variable names erased	10
Γ, α	Γ extended with the type variable α	12
$\tau \sqsubseteq \sigma$	the monotype τ is an instance of the scheme σ	13

The judgments of Parts III to VII add constraints, splitting, lifetimes, stores and semantics:

Notation	Read as	Ch.
$P \mid \Gamma \vdash e : \tau$	typing under the constraints P	14
$P \Vdash \pi$	the constraints P entail π	14
$P \Vdash d : \pi$	d is the dictionary that proves π	14
$F\ A : \kappa_2$	the type expression $F\ A$ has kind κ_2	17
$\Gamma = \Gamma_1 \uplus \Gamma_2$	Γ splits into Γ_1 and Γ_2	21, 22
$\Gamma \vdash e : \tau \dashv \Delta$	e has type τ and leaves Δ unused	22
$\mathit{Copy}(\tau)$	τ is a `Copy` type	23, 28
$\Delta; \Gamma \vdash c \dashv \Delta'; \Gamma'$	the command c turns regions Δ and context Γ into Δ' and Γ'	26
$\Delta \vdash \alpha : \beta$	under Δ, the lifetime α outlives β	26
$\sigma <: \tau$	σ is a subtype of τ	27
$\Delta; \Gamma \vdash e : \tau$	typing with the lifetimes Δ in scope	28
$\cdot; \cdot \underset{\Sigma}{\vdash} e : \tau$	a run-time term, typed with the store typing Σ	28
$\Gamma \vDash e : \tau$	e is semantically well typed	32
$\{P\}\ c\ \{Q\}$	from a heap satisfying P, c runs safely to one satisfying Q	32

The turnstile $\vdash$ is a symbol of the metalanguage. It separates assumptions from a conclusion and is never part of a term, which is the difference between $A \vdash B$ (a judgment) and $\vdash A \to B$ (a judgment about the proposition $A \to B$); Chapter 4 explains why the two are derivable together. In Lean's infoview the same symbol `⊢` separates the hypotheses from the goal (Chapter 2).

C.3 Logic

Notation	Read as	Ch.
$A \wedge B$, $A \vee B$	A and B; A or B	4
$A \to B$	A implies B	4
$\top$, $\bot$	true; false	4
$\neg A$	not A, defined as $A \to \bot$	4
$A \leftrightarrow B$	A if and only if B, defined as $(A \to B) \wedge (B \to A)$	4
$\forall x.\, A(x)$, $\exists x.\, A(x)$	for all x; there exists an x	5, 10
$\wedge$I, $\wedge$E$_1$, $\wedge$E$_2$	introduction and eliminations of $\wedge$	4
$\vee$I$_1$, $\vee$I$_2$, $\vee$E	introductions and elimination of $\vee$	4
$\to$I, $\to$E	introduction (discharging) and elimination (*modus ponens*)	4
$\top$I, $\bot$E	truth; *ex falso quodlibet*	4
Hyp	use an assumption from the context	4
Cut	prove a lemma, then use it	4, 21
RAA	*reductio ad absurdum*, the classical rule	4
$A \mathit{tonk} B$	Prior's connective without harmony	4
$\mathcal{D}_1[\mathcal{D}_2/A]$	the derivation $\mathcal{D}_2$ substituted for the assumption A	4
Exch, Weak, Contr	the structural rules: exchange, weakening, contraction	6
$w \le w'$	w' is a possible future of world w	5
$w \Vdash A$, $w \nVdash A$	w forces A; w does not force A	5
$a \sqcap b$, $a \sqcup b$	meet ("and") and join ("or") of truth values	5
$a \Rightarrow b$	Heyting implication, characterized by residuation	5
$c \le (a \Rightarrow b)$ iff $c \sqcap a \le b$	residuation	5
$[\![\Gamma]\!] \le [\![A]\!]$	the sequent is valid in a Heyting algebra	5

In Lean, the connectives on propositions are `∧`, `∨`, `→`, `¬`, `↔`, `True`, `False`, `∀ x, P x` and `∃ x, P x` (Chapters 4 and 10), and Mathlib writes the Heyting operations as `⊓`, `⊔`, `⇨`, `⊤` and `⊥` (Chapter 5).

C.4 Type formers

Notation	Read as	Ch.
$\mathbf{1}$	the unit type, with one value; Rust `()`	1, 11
$\mathbf{0}$	the empty type, with no values; Rust `!`, `Infallible`	1, 11
$A \times B$	product: a pair of an A and a B	8, 11
$A + B$	sum: an A or a B, tagged	8, 11
$A \to B$, B^A	function type, also called exponential	8, 11
$\lvert A \rvert$	the number of values of A	11
$A \cong B$	A and B are isomorphic	11
$\mu X.\, F(X)$	the recursive type X with $X \cong F(X)$	11
∂F	the derivative of a type: F with a hole	11
$\forall \alpha.\, \tau$	for every type α, τ (System F)	12
$\sigma ::= \tau \mid \forall \alpha.\, \sigma$	a type scheme: quantifiers only at the outside	13
$\pi \Rightarrow \tau$	τ qualified by the constraint π	14
$\exists \alpha.\, \tau$	for some hidden type α, τ	15
$\Pi x : A.\, B$	dependent function type	10, 30
$\Sigma x : A.\, B$	dependent pair type	10, 30
$\mathrm{List}\, A$, $\mathrm{Vec}\, \alpha$	type constructors applied to a type	11, 12
$F : \kappa_1 \to \kappa_2$	F takes a type of kind κ_1 to one of kind κ_2	17
$M\, \tau$	a computation with effect M that produces a τ	20
Const_A	the constant functor at A	18
$!\tau.\, S$, $?\, \tau.\, S$, end	send a τ, receive a τ, stop (session types)	25
$\overline{S}$	the dual of the session type S	25
$\delta(s, a) = s'$	in state s, action a leads to s'	25
$\mathit{Box}\, \tau$, $\mathit{Ref}\, \tau$	an owning pointer; a reference	26
$\&^{\kappa}\, \tau$, $\&^{\kappa}\, \mathit{mut}\, \tau$	shared and mutable references with lifetime κ	28

The connectives of linear logic, which are also type formers of the affine λ-calculus, are in the section on substructural logic below.

C.5 The λ-calculus and evaluation

Notation	Read as	Ch.
$\lambda x.\, t$	the function with parameter x and body t	7
$t\ u$	t applied to u (application associates to the left)	7
$\lambda x : \tau.\, t$	a λ with a type annotation (Church style)	8
$\mathrm{FV}(t)$	the free variables of t	7
$t \underset{\alpha}{\equiv} u$	t and u differ only in bound names	7
$t[x := s]$	t with s substituted for x	7
$t[s/x]$	the same operation, in the other common notation	4, 6
$\lambda.\, t$, 0, 1	a nameless binder and de Bruijn indices	7
$\#i$, $\lambda : \tau.\, t$	de Bruijn variables and binders of Chapter 9	9
shift_c, shift_c^d	add 1 (or d) to the indices at least c	7
$t[j := s]$, $t[v]$	substitution for an index; for index 0	7, 9
$(\lambda x.\, t)\, s$	a redex	7
$t \to t'$	t takes one step to t'	1, 7
$t \overset{*}{\to} t'$	t reaches t' in zero or more steps	7
$\rho \vdash e \Downarrow v$	under environment ρ, e evaluates to v (big-step)	3
$\rho, x \mapsto v$	the environment ρ extended with x bound to v	3
Ω	$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$, which steps to itself	7
Y, Z	fixed-point combinators (lazy; call-by-value)	7
K, I	$\lambda x.\, \lambda y.\, x$ and $\lambda x.\, x$	7
$\overline{n}$	the Church numeral n	7
$\lambda x : \sigma.\, f\, x \equiv f$	the η-rule for functions	8
`fix` f, $\mathit{fix}\, f.\, e$	general recursion, which a terminating calculus lacks	8, 22

For polymorphism, existential types and type inference (Chapters 12 to 15) and for the resource languages of Part V:

Notation	Read as	Ch.
$\Lambda\alpha.\, e$, $e\,[\tau]$	type abstraction; type application	12
$\tau[\sigma/\alpha]$	τ with σ for the type variable α	12
$\tau[\alpha := \sigma]$	the same, in the other notation	14, 15
$\mathit{pack}\ \langle \sigma, e\rangle\ \mathit{as}\ \exists\alpha.\,\tau$	an existential package	15
$\mathit{unpack}\ \langle \alpha, x\rangle = e_1\ \mathit{in}\ e_2$	open a package	15
$[\alpha \mapsto \tau]$, $S\,\tau$	a substitution; applying it to τ	13
$S_2 \circ S_1$	first S_1, then S_2	13
$\tau_1 \doteq \tau_2$	an equation to be solved by unification	13
$\mathrm{ftv}(\tau)$, $\mathrm{Gen}(\Gamma, \tau)$	free type variables; generalization	13
mgu, $W(\Gamma, e) = (S, \tau)$	most general unifier; algorithm W	13
$\langle \sigma, e\rangle \to \langle \sigma', e'\rangle$	a step that also changes the state σ	24
$\mathit{release}_\tau$	the destructor of τ	24

Two notations for substitution. Chapters 4 and 6 write $t[s/x]$ ("t with s for x"), and Chapter 7 introduces $t[x := s]$ for the same operation, noting that both are common. After that the book uses both: $t[x := s]$ in Chapters 7, 8 and 15, and $t[s/x]$ in Chapters 12, 19, 22, 23, 28 and 30. For type variables, Chapters 12 and 18 write $\tau[\sigma/\alpha]$ and Chapters 14 and 15 write $\tau[\alpha := \sigma]$. Chapter 13 writes a substitution as a map $[\alpha \mapsto \tau]$ applied on the left, $S\,\tau$. Chapters 9, 10 and 19 also write $t[s]$ for substituting s for the variable of the innermost λ. In Chapter 26, $s[env(y) := \ldots]$ is not a substitution but an update of the state s.

C.6 Substructural logic

Notation	Read as	Ch.
Exch, Weak, Contr	reorder, discard, duplicate an assumption	6
$A \otimes B$	"A tensor B": both, from split resources	6, 21
$A \,\&\, B$	"A with B": either one, chosen by the user	6, 21
$A \oplus B$	"A plus B": one of them, chosen by the producer	21
$A \parr B$	"A par B": the classical multiplicative disjunction	21
$A \multimap B$	"A lollipop B": consume an A, produce a B	8, 21
1, $\top$, $\bot$, 0	the units of $\otimes$, $\&$, $\parr$ and $\oplus$	21
$A^{\bot}$	the linear negation of A	21
$!A$, $?A$	"of course A", "why not A": the exponentials	21, 23
$!\Gamma$	a context in which every formula has the form $!A$	21, 23
$\Gamma_1 \uplus \Gamma_2$	the disjoint union of two contexts	21, 22
Ax, Cut	axiom $A \vdash A$; cut	21
$\otimes$R, $\otimes$L, &R, &L$_{1}$, $\oplus$R$_{1}$, $\multimap$R, $\multimap$L	right and left rules of the connectives	21
Der, !W, !C, Prom	the rules of ! (Chapter 21's names)	21
Dereliction, !-Weak, !-Contr, Promotion	the same rules (Chapter 23's names)	23

The affine λ-calculus of Chapters 22 and 23 writes the same connectives as types, with these term formers and rules:

Notation	Read as	Ch.
λ^{aff}	the affine λ-calculus	22
(e_1, e_2), $\texttt{let}\ (x, y) = e_1\ \texttt{in}\ e_2$	a $\otimes$ pair and its eliminator	22
$\langle e_1, e_2 \rangle$, $\mathit{fst}\, e$, $\mathit{snd}\, e$	an offer (a & pair) and its projections	22
$\texttt{case}\ e\ \{x.\, e_1 \mid y.\, e_2\}$	case analysis on a $\oplus$	22
$\texttt{drop}\ x; e$	discard x, then continue with e	22, 24
S-Empty, S-Left, S-Right	the rules of context splitting	22
S-Both	a `Copy` variable may go to both sides	23
$\Gamma - x$	Γ without x	22
$\Delta_1 \sqcap \Delta_2$	the variables left unused by both alternatives	22
$\mathit{derelict}\, e$, $!e$	use a $!\tau$ once; build a $!\tau$	23
$\mathit{dup}_{\tau} : \tau \multimap \tau \otimes \tau$	an explicit duplication, like `Clone::clone`	23
$P * Q$	separating conjunction: P and Q on disjoint resources	26, 32

Linear logic writes the two conjunctions as $\otimes$ and &, and the book follows it. In Rust, $A \otimes B$ is `(A, B)` and $A \multimap B$ is `FnOnce(A) -> B`; the full correspondence is in the second dictionary at the end of this

appendix. The book's Lean files declare the same connectives as scoped notation for their own formula and type syntax: `⊗`, `&`, `⊕` and `⊸` (Chapters 21 and 22).

C.7 Categories

Notation	Read as	Ch.
$\mathcal{C}$, $\mathcal{D}$	categories	16, 17
$\mathrm{Hom}(a, b)$	the morphisms from a to b	16
$f : a \to b$	f is a morphism from a to b	16
id_a	the identity morphism on a	16
$g \circ f$	g after f: first f, then g	16
$a \cong b$	a and b are isomorphic	16
$\star$	the single object of a monoid seen as a category	16
$a \times b$; $\langle f, g \rangle$	a product; the pairing of f and g	16, 19
fst, snd; π_1, π_2	the projections of a product	16, 19
inl, inr; $[f, g]$	the injections of a coproduct; copairing	16
$\mathcal{C}^{\mathrm{op}}$	the opposite category: every arrow reversed	16
$F : \mathcal{C} \to \mathcal{D}$	a functor	17
$F\,a$, $F\,f$	the functor on an object; on a morphism	17
$\mathcal{C} \times \mathcal{D}$	the product of two categories	17
$\mathbf{Cat}$	the category of categories and functors	17

Natural transformations, cartesian closed categories, monoidal categories and monads (Chapters 18 to 20, and 27):

Notation	Read as	Ch.
$\eta : F \Rightarrow G$, η_a	a natural transformation; its component at a	18
$\theta_a \circ \eta_a$	vertical composition, componentwise	18
$[\mathcal{C}, \mathcal{D}]$, $\mathcal{D}^{\mathcal{C}}$	the functor category	18
$\mathrm{Hom}(\alpha, -)$	the functor of morphisms out of α	18
$\mathrm{Nat}(F, G)$	the natural transformations from F to G	18
$[\![\tau]\!]_R$	the relation that τ assigns to a relation R	18
$!_a$	the unique morphism from a to the terminal object	19
$f \times g$	f and g side by side, $\langle f \circ \pi_1, g \circ \pi_2 \rangle$	19
$\Delta_a = \langle \mathrm{id}_a, \mathrm{id}_a \rangle$	the diagonal $a \to a \times a$	16, 19
$b \Rightarrow c$, c^b	the exponential object	19
eval, curry f	evaluation; the curried form of f	19
$L \dashv R$	L is left adjoint to R	19
$(-) \times b$	the functor "pair with a b"	19
$[\![\Gamma \vdash t : \tau]\!]$	the morphism that interprets a typed term	19
$a \otimes b$, I	a monoidal product and its unit	19
$b \multimap c$	the exponential of a monoidal closed category	19
pure, *bind*	the operations of a monad	20
$g \circ_M f$	Kleisli composition	20
η, μ	unit and multiplication (`join`) of a monad	20
$\mathcal{L}$, $\mathcal{T}$	the categories of lifetimes and of types	27

The book composes from right to left, $g \circ f$, and so does its own Lean notation `g ∘ f`; Mathlib composes in diagrammatic order, `f ≫ g`. The section on Lean notation below lists both libraries' symbols.

C.8 Regions, lifetimes and permissions

Notation	Read as	Ch.
`'a`, `&'a T`, `&'a mut T`	a lifetime; references valid during `'a`	26
`'a: 'b`	`'a` outlives `'b`	26
`T: 'a`	every reference inside `T` outlives `'a`	26
`'static`, *static*	the region of the whole execution	26
`for<'a>`	for every lifetime `'a` (a higher-ranked bound)	12
`letregion` ρ `in` e	a region that lives while e runs	26
ref ρ	a pointer into region ρ	26
$\rho \in \Delta$	the region ρ is still live (escape check)	26
ℓ	a location in memory	26, 28
O-Refl, O-Hyp, O-Static, O-Trans	the rules of outlives	26
$\& \alpha\ \tau <: \& \beta\ \tau$	a reference may be used for a shorter lifetime	27

The core calculus of borrowing of Chapter 28:

Notation	Read as	Ch.
borrow $y = \&^{\alpha} x$ *in* e	borrow x as y for the lifetime α, during e	28
borrow $y = \&^{\alpha}$ *mut* $* e_1$ *in* e_2	a reborrow	28
$* e$, $* e_1 := e_2$	read through a reference; write through `&mut`	28
$\mathrm{flv}(\tau)$	the lifetime variables free in τ	28
$\alpha \notin \mathrm{flv}(\sigma)$	the escape check of the borrow calculus	28
own ℓ, *shr* ℓ, *uniq* ℓ	run-time owner, shared and unique references	28
$S(\ell) = v^m$	location ℓ holds v, with loan state m	28
none, *shared*, *mut*	not lent; lent shared; lent mutably	28
$\langle S; e \rangle$	a configuration: store and term	28
restore w *after* e	give the lent value back when e finishes	28
$\mathrm{dom}(S)$, $\mathrm{free}(S, v)$	allocated locations; release what v owns	28

Non-lexical lifetimes and Polonius (Chapter 29), and the separation logic and lifetime logic of Chapter 32:

Notation	Read as	Ch.
$P \in \rho$	the program point P belongs to the region ρ (NLL)	29
$\rho_1 : \rho_2$	region ρ_1 outlives ρ_2 (contains its points)	29
`subset('a, 'b)`, `loan_live_at(L, P)`	Polonius relations over origins, loans and points	29
$\ell \mapsto v$	points-to: the cell ℓ holds v	32
$\{P\}\ c\ \{Q\}$, Frame	Hoare triple; the frame rule	32
$[\dagger\ \kappa]$	the token saying that lifetime κ has ended	32
$\&^{\kappa}_{\text{full}} P$	a full borrow of P for the lifetime κ	32
$\triangleright$	"later", a modality of Iris	32

Chapters 26 to 28 write lifetimes as Greek letters, α, β, γ and, in Chapter 28, κ, where Rust writes `'a` and `'b`. The regions of Chapter 26's heap language, after Tofte and Talpin, and the regions of NLL in Chapter 29 are written ρ.

C.9 Dependent types

Notation	Read as	Ch.
$\Pi x : A.\ B$	for every $x : A$, a B (which may mention x)	10, 30
$\Sigma x : A.\ B$	an $x : A$ together with a B for that x	10, 30
$B[a/x]$	the type B at the argument a	30
$\pi_1\ p$, $\pi_2\ p$	the components of a dependent pair	30
Pi-Form, Pi-Intro, Pi-Elim	the rules of Π-types	30
Sigma-Form, Sigma-Intro, Sigma-Fst, Sigma-Snd	the rules of Σ-types	30
Type, *Sort* u, imax $u\ v$	universes, and the level of a Π-type	12, 30
$\lambda\!\!\rightarrow$, $\lambda 2$, $\lambda\omega$, $\lambda\underline{\omega}$, λP, λC	the corners of the λ-cube	30

In Lean, $\Pi x : A.\ B$ is written `(x : A) → B` (Chapter 12) and becomes the ordinary arrow `A → B` when B does not mention x. $\Sigma x : A.\ B$ is `(x : A) × B` or `Σ x, B` for data, `(x : A) ×' B` when the second component may be a proof, and `∃ x, B` for propositions (Chapters 10, 15 and 30). Equality is the type `a = b`, proved by `rfl`, and `h ▸ e` transports `e` along `h : a = b` (Chapter 30). The subtype `{x // p x}` pairs a value with a proof (Chapters 10 and 11). `Fin n` is the type of natural numbers below `n` (Chapter 30), and `xs[i]'h` indexes with a proof `h` that `i` is in range (Chapter 1).

C.10 Lean notation

Most of Lean's notation is explained in Chapter 2 and in Appendix B. This table lists the symbols that the chapters use, including the scoped notations that the book's own Lean files declare.

Lean	Meaning	Ch.
`fun x => e`, `(· + 1)`	anonymous functions; `·` is the argument	1, 2
`⟨a, b⟩`	anonymous constructor: a pair, a proof of `∧`, a structure	1
`{α : Type}`, `[C α]`	implicit argument; instance argument	2
`@f`	`f` with all its arguments explicit	12
`⊢`	in the infoview: the goal, after the hypotheses	2
`·` (in a proof)	focus on one goal	2
`t₁ <;> t₂`	run `t₂` on every goal that `t₁` produces	2
`←`	bind in `do` notation; rewrite right to left in `rw`	2
`↑x`	a coercion inserted by Lean	27
`ℕ`, `ℤ`, `ℕ∞`	naturals, integers, naturals with a top element `⊤`	1, 23
`Prop`, `Type`, `Type u`, `Sort u`	the universes	10, 12
`α ⊕ β`, `.inl`, `.inr`	the sum type	10, 11
`?m.5`	a metavariable: an unknown the elaborator must solve	13
`a ⋄ b`	the book's monoid operation	14
`<$>`, `>>=`	`Functor.map`; `bind`	17, 20
`g ∘ f`	function composition	16
`α ≅ β`	the book's `Iso` (in Chapter 16, between objects)	11, 16
`α ≃ β`	Mathlib's `Equiv`: a bijection with both proofs	11
`a ⟶ b`, `g ⊚ f`, `id a`	the book's morphisms, composition, identity	16
`X ⟶ Y`, `f ≫ g`, `𝟙 X`	Mathlib's morphisms, composition (first `f`), identity	16
`C ⥤ D`, `F ⋙ G`, `𝟭 C`	Mathlib's functors, their composition, the identity functor	17
`a × b`, `b ⟹ c`	the book's product and exponential objects	19
`σ ⇒ τ`	the function type of a toy language	8, 13, 27
`σ <: τ`	subtyping in the toy language of Chapter 27	27
`P ∗ Q`, `ℓ ↦ v`, `P ⊢ Q`	separation logic: conjunction, points-to, entailment	32

The commands `#eval`, `#check`, `#guard_msgs` and `#print axioms` are introduced in Chapter 1 and `#print` in Chapter 4; the tactics are described in Appendix D.

C.11 Dictionary: logic, types, categories, Rust, Lean

The table extends the dictionaries of Chapters 10 and 19. A Rust entry is the closest counterpart, with the caveats the chapters state; "—" means that the book gives no counterpart.

Logic	Type theory	Categories	Rust	Lean
proposition	type	object	type	`p : Prop`, `α : Type`
proof	term	morphism	value, function	proof term
assumption	variable in the context	projection out of $[\![\Gamma]\!]$	parameter	hypothesis `h : p`
$A \wedge B$	$A \times B$	product	`(A, B)`, a struct	`p ∧ q`, `α × β`
$A \vee B$	$A + B$	coproduct	an `enum`, `Result`	`p ∨ q`, `α ⊕ β`
$A \to B$	$A \to B$	exponential B^A	`fn`, closures	`p → q`, `α → β`
$\top$	$\mathbf{1}$	terminal object	`()`	`True`, `Unit`
$\bot$	$\mathbf{0}$	initial object	`!`, `enum Void {}`	`False`, `Empty`
$\neg A$	$A \to \mathbf{0}$	arrow into the initial object	`fn(A) -> Void`	`¬p`
$\forall x.\, P(x)$	Π-type	—	generics, over types only	`∀ x, P x`
$\exists x.\, P(x)$	Σ-type	—	`impl Trait`, over types only	`∃ x, P x`
→I	λ-abstraction	curry	closure	`fun`, `intro`
→E	application	eval	call	application, `apply`
⊥E	`absurd`, empty `match`	arrow out of the initial object	`match x {}`, `!` coerces	`nomatch`, `absurd`
replacing an assumption by its proof	substitution $t[u/x]$	composition	inlining a call	—
detour	redex	—	—	—
normalization	β-reduction	$\mathrm{eval} \circ (\mathrm{curry}\, f \times \mathrm{id}) = f$	running a program	`rfl`
local completeness	η	uniqueness of curry	`.map(f)` for `.map(|x| f(x))`	`rfl` (definitional η)
contraction	using x twice	diagonal Δ	needs `Copy` or `Clone`	free
weakening	unused variable	$!_a : a \to 1$	dropping a value	free
consistency	no closed term of type $\mathbf{0}$	—	fails: `loop {}`, `panic!`	holds: totality
induction	recursor, fold	initial algebra	recursion with `match`	`induction`, `T.rec`
intuitionistic $\wedge$, $\to$, $\top$	simply typed λ-calculus	cartesian closed category	—	proofs without `Classical`
Heyting algebra	—	thin bicartesian closed category	—	`HeytingAlgebra`
excluded middle	—	—	—	`Classical.em`

The Rust column for $A \to B$ depends on the reading. Chapter 10 pairs it with `impl FnOnce(A) -> B`, a function that may be called once, which is enough to read a proof; Chapter 19, which needs functions that can be used freely, pairs the exponential with `Box<dyn Fn(A) -> B>`. In the linear reading of Part V the same `FnOnce` is $A \multimap B$.

A second table covers the abstraction mechanisms of Parts III and IV.

Type theory	Logic or categories	Rust	Lean	Ch.
$\forall \alpha.\, \tau$	$\forall$ over propositions (second order)	generic function	`(α : Type) → τ`	12
type constructor	functor on objects	`Option`, `Vec`	`Option : Type → Type`	17
`map` with its laws	functor on morphisms	`map` methods, no trait	`Functor`, `LawfulFunctor`	17
polymorphic $\forall \alpha.\, F\,\alpha \to G\,\alpha$	natural transformation	`fn f<T>(Vec<T>) -> Option<T>`	`NatTrans`	18
parametricity	naturality, free theorems	leaks: `Any`, `size_of`	not provable inside Lean	18
type class	a predicate on types	trait	`class`	14
instance	an entailment rule, a Horn clause	`impl`	`instance`	14
dictionary	evidence for a constraint	vtable, or monomorphized code	instance argument `[C α]`	14
$\exists \alpha.\, \tau$	$\exists$ over types	`impl Trait`, `dyn Trait`	a Σ-type over `Type`	15
$M\ A$ with *pure*, *bind*	monad, Kleisli category	`Option`, `Result`, `?`	`Monad`, `do`	20
currying	adjunction $(-) \times b \dashv b \Rightarrow (-)$	closures returning closures	`fun a b => ...`	19
subtyping $\sigma <: \tau$	a preorder, a thin category	lifetimes only	none (coercions `↑`)	27
variance	covariant or contravariant functor	`&'a T`, `fn(T)`, `&mut T`	—	17, 27

C.12 Dictionary: resources, ownership and borrowing

The vocabulary of Parts V and VI lines up as follows. The first table is about the structural rules, the second about the connectives of linear logic, and the third about references and lifetimes.

Structural rule	On resources	Categories	Rust	Ch.
exchange	use in any order	symmetry $a \otimes b \cong b \otimes a$	order of `let`s is irrelevant to typing	6, 19
weakening	discard	a map $a \to 1$	drop; runs `Drop`	6, 24
contraction	duplicate	diagonal $a \to a \times a$	`Copy` (implicit), `Clone` (explicit)	6, 23
no contraction	use at most once (affine)	symmetric monoidal, terminal unit	moves; E0382	19, 22
no weakening	use at least once (relevant)	—	`#[must_use]`, a lint	21, 24
neither	use exactly once (linear)	symmetric monoidal closed	not enforced: `mem::forget` is safe	19, 21, 24

Linear logic	Name	λ^{aff}	Rust ≈	Ch.
$A \otimes B$	tensor	pair, `let (x, y)`	`(A, B)`, a struct	21, 22
$A \,\&\, B$	with	offer $\langle e_1, e_2 \rangle$	a value with two consuming methods	21, 22
$A \oplus B$	plus	`inl`, `inr`, `case`	an `enum`	21, 22
$A \parr B$	par	—	— (classical only)	21
$A \multimap B$	lollipop	$\lambda x : \tau . e$	`FnOnce(A) -> B`	21, 22
1	unit of $\otimes$	$()$	`()`	21, 22
$!A$	of course	`Copy` predicate, S-Both	`A: Copy`	21, 23
Promotion	—	`Copy` only from `Copy` parts	`#[derive(Copy)]` needs `Copy` fields (E0204)	23
!-Weak	—	discarding a `Copy` value is free	`Copy` excludes `Drop` (E0184)	23
$\tau \multimap \tau \otimes \tau$	a term, not a rule	dup_τ	`Clone::clone`	23
—	counted sharing	—	`Rc<T>`, `Arc<T>`	23

Theory	Rust	Ch.
multiplicity: how many times a variable is used	moves, `Copy`	23, 26
extent: where a reference is valid	lifetimes	26
region variable	lifetime parameter `'a`	26
outlives constraint	`'a: 'b`, `T: 'a`	26
escape check, $\rho \in \Delta$ or $\alpha \notin \mathrm{flv}(\sigma)$	E0597, E0515	26, 28
duplicable read permission	`&T`, which is `Copy`	26
unique read-write permission	`&mut T`, which is affine	26
temporary transfer of permission	a borrow	26, 28
a new unique reference for a shorter lifetime	a reborrow `&mut *r`	28
aliasing XOR mutation	E0499, E0502, E0506	26, 29
run-time check of the aliasing rule	`Cell`, `RefCell`	26
region as a set of program points	non-lexical lifetimes	29
loan	a borrow expression in MIR	29
origin: a set of loans	Polonius	29
variance as functoriality	`&'a T` covariant, `&mut T` invariant in `T`	27
typestate: states as types	consuming methods, `PhantomData`	25
separating conjunction $P * Q$	two live `&mut` are disjoint	32
proof obligation	an `unsafe` block, a `SAFETY:` comment	32

APPENDIX D

The Tactics Used in This Book

A tactic is an instruction that transforms the current goal: it either closes it, or replaces it with simpler goals. Every tactic proof in this book is built from the few dozen tactics described here. Each one has a small example below, taken from `code/lean/Book/AppD/Tactics.lean`, which compiles. For the reasoning behind a tactic, follow the chapter references; for any tactic, hovering over its name in the editor shows its full documentation.

In this appendix you will learn:

- which tactics close goals, and which decision procedures they run;
- how the introduction and elimination rules of natural deduction appear as tactics;
- how to rewrite and simplify, and how to keep `simp` predictable;
- how to structure a longer proof, and how termination is proved;
- which tactics come with Lean and which with Mathlib.

D.1 Reading a goal

Inside a `by` block, the infoview shows the **goal**: the hypotheses, one per line with their names and types, then the turnstile `⊢`, then the proposition still to be proved. A tactic proof is a conversation with this display: every step changes it, and the proof is finished when no goals remain. Chapter 2 introduces the infoview; Chapter 4 explains why the goal is a sequent $\Gamma \vdash A$ of natural deduction, which is why the hypotheses are above the turnstile and the goal below it.

Several goals can be open at once (after `constructor` on `p ∧ q`, for example). Tactics act on the first one; the bullet `·` focuses on one goal until it is closed, and indentation shows the structure.

D.2 Closing goals

```
-- `rfl`: both sides compute to the same value.
example : 2 + 2 = 4 := by rfl

-- `exact e`: the term `e` has exactly the type of the goal.
example (p : Prop) (hp : p) : p := by exact hp

-- `assumption`: some hypothesis has the type of the goal.
example (p q : Prop) (_hq : q) (hp : p) : p := by assumption

-- `trivial`: closes `True`, and goals that `rfl` or `assumption` close.
example : True := by trivial
```

```
-- `contradiction`: the hypotheses are contradictory (here, `False` itself).
example (p : Prop) (h : False) : p := by contradiction

-- `absurd h h'`: `h : p` and `h' : ¬p` prove anything.
example (p q : Prop) (hp : p) (hnp : ¬p) : q := absurd hp hnp

-- `decide`: evaluate a decidable proposition.
example : 7 ∣ 91 := by decide

-- `omega`: linear arithmetic over ℕ and ℤ.
example (a b : ℕ) (h : a < b) : a + 1 ≤ b := by omega

-- `ring`: identities in commutative (semi)rings.
example (x y : ℤ) : (x - y) * (x + y) = x ^ 2 - y ^ 2 := by ring

-- `linarith`: linear inequalities over ordered fields (and ℕ, ℤ).
example (x y : ℚ) (h₁ : x ≤ 2 * y) (h₂ : y < 3) : x < 6 := by linarith

-- `norm_num`: numeric facts.
example : (2 : ℚ) / 3 + 1 / 3 = 1 := by norm_num

-- `positivity`: goals of the form `0 < e` or `0 ≤ e`.
example (x : ℤ) : 0 ≤ x ^ 2 + 1 := by positivity

-- `tauto`: propositional tautologies (classically).
example (p q : Prop) : p ∧ q → q ∨ p := by tauto
```

Tactic	Closes the goal when…
`rfl`	both sides of `=` (or `↔`) are equal by computation
`exact e`	the term `e` has the goal's type
`assumption`	a hypothesis has the goal's type
`trivial`	the goal is `True`, or `rfl` / `assumption` works
`contradiction`	the hypotheses are evidently contradictory
`decide`	the proposition is decidable and evaluates to `true`
`omega`	the goal is linear arithmetic over `ℕ` or `ℤ`
`ring`	the goal is an identity of commutative (semi)rings
`linarith`	the goal follows linearly from hypotheses that are inequalities
`norm_num`	the goal is a numerical fact
`positivity`	the goal is `0 < e` or `0 ≤ e` for a visibly positive `e`
`tauto`	the goal is a propositional tautology (reasoning classically)
`grind`	a combination of rewriting and decision procedures succeeds

Chapter 1 uses `rfl`, `decide`, `omega`, `ring` and `grind` for the first time, and Chapter 2 introduces the arithmetic tactics that come with Mathlib. `decide`, `omega`, `ring`, `linarith` and `norm_num` are **decision procedures**: algorithms that are guaranteed to find a proof for every goal in their class. Use them freely once you understand what they decide. They also produce real proof terms, which the kernel checks; but note that `decide` on a large search space can be slow, and `native_decide` (which this book avoids) trusts compiled code instead of the kernel. `grind` and `tauto` reason classically, so proofs that use them depend on `Classical.choice`, as `#print axioms` shows (Chapters 1 and 5).

D.3 Introduction rules: working on the goal

```
-- `intro`: the →I and ∀I rules; name the hypothesis or variable.
example (p q : Prop) : p → q → p := by
  intro hp _hq
  exact hp

-- `constructor`: apply the (only) constructor: ∧I, ↔I, building a structure.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp
  · exact hq

-- `left` / `right`: ∨I, choosing the side.
example (p q : Prop) (hq : q) : p ∨ q := by
  right
  exact hq

-- `exists` / `use`: ∃I, giving the witness.
example : ∃ n : ℕ, n * n = 49 := by
  exists 7

example : ∃ n : ℕ, n + 3 = 7 := by
  use 4

-- `apply f`: backward reasoning; the remaining premises of `f` become goals.
example (p q r : Prop) (hpq : p → q) (hqr : q → r) (hp : p) : r := by
  apply hqr
  apply hpq
  exact hp

-- `refine`: like `exact`, with `?_` holes that become new goals.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  refine ⟨?_, hq⟩
  exact hp

-- `exfalso`: replace the goal by `False` (⊥E, read backwards).
example (p : Prop) (h : 1 = 0) : p := by
  exfalso
  omega

-- `funext`: two functions are equal if they agree everywhere.
example : (fun n : ℕ => n + 0) = (fun n => n) := by
  funext n
  rfl
```

Each of these tactics applies an *introduction* rule backwards: it looks at the shape of the goal and reduces it to what that rule requires (Chapter 4).

Goal	Tactic	Rule (Chapter 4)
`p → q`, `∀ x, P x`	`intro h`, `intro x`	→I, ∀I
`p ∧ q`, `p ↔ q`, a structure	`constructor`	∧I, ↔I
`p ∨ q`	`left`, `right`	∨I
`∃ x, P x`	`exists w`, `use w`	∃I
`False`-free goal you want to refute	`exfalso`	⊥E, read backwards
`f = g` for functions	`funext x`	function extensionality
anything, using a lemma `f : A → B → goal`	`apply f`, `refine f ?_ ?_`	→E, read backwards

`apply` and `refine` are the general tools: `apply f` unifies the conclusion of `f` with the goal and leaves its remaining premises as new goals; `refine e` does the same for a term `e` with explicit holes `?_`.

D.4 Elimination rules: working on hypotheses

```
-- `cases`: one goal per constructor of the hypothesis (∨E, ∧E, ∃E, pattern matching).
example (p q : Prop) (h : p ∨ q) : q ∨ p := by
  cases h with
  | inl hp => exact Or.inr hp
  | inr hq => exact Or.inl hq

-- `obtain` / `rcases`: take a hypothesis apart with a pattern, in one step.
example (p q r : Prop) (h : p ∧ (q ∧ r)) : r := by
  obtain ⟨_hp, _hq, hr⟩ := h
  exact hr

example (n : ℕ) (h : ∃ k, n = 2 * k) : n % 2 = 0 := by
  rcases h with ⟨k, rfl⟩
  omega

-- `rintro`: `intro` followed by `rcases`.
example (p q : Prop) : p ∧ q → q := by
  rintro ⟨_, hq⟩
  exact hq

-- `induction`: structural induction; one goal per constructor, with hypotheses.
theorem sum_formula (n : ℕ) : 2 * (List.range (n + 1)).sum = n * (n + 1) := by
  induction n with
  | zero => rfl
  | succ k ih =>
    rw [List.range_succ, List.sum_append, mul_add, ih]
    simp
    ring

-- `by_cases`: split on whether a proposition holds.
example (n : ℕ) : n % 2 = 0 ∨ n % 2 = 1 := by
  by_cases h : n % 2 = 0
  · exact Or.inl h
  · right
    omega

-- `split`: one goal per branch of an `if` or `match` in the goal.
example (n : ℕ) : (if n < 5 then n else 5) ≤ 5 := by
  split
  · omega
  · omega

-- `subst`: replace a variable everywhere using an equation `x = e`.
example (a b : ℕ) (h : a = b + 1) : a - 1 = b := by
  subst h
  omega

-- `nomatch`: a hypothesis with no possible constructor proves anything.
example (h : Nat.succ 3 = 0) : False := nomatch h

-- `injection`: constructors are injective.
example (a b : ℕ) (h : Nat.succ a = Nat.succ b) : a = b := by
  injection h
```

Hypothesis	Tactic	Rule
`h : p ∨ q`	`cases h`, `rcases h with hp | hq`	∨E (case analysis)
`h : p ∧ q`, `h : ∃ x, P x`	`obtain ⟨hp, hq⟩ := h`	∧E, ∃E
a value of an inductive type	`cases x`, `induction x`	case analysis, induction
`h : x = e`	`subst h`, `rw [h]`	equality elimination
`h` with no possible constructor	`nomatch h`, `cases h`	⊥E
`h : C a = C b`	`injection h`	constructors are injective
any proposition `p`	`by_cases h : p`	excluded middle (decidable `p`)
`if` / `match` in the goal	`split`	case analysis on the condition

`rcases`, `obtain` and `rintro` take *patterns*: `⟨a, b⟩` takes a pair or an existential apart, `a | b` splits a disjunction, and `rfl` substitutes an equation on the spot. `induction ... with` names each case and its induction hypotheses; adding `generalizing x` keeps `x` universally quantified in the hypothesis, which many proofs need (Chapter 31). When a case introduces names that Lean generated itself (they print with a dagger, `a✝`), `rename_i` gives them names you can use.

D.5 Rewriting and simplification

```
-- `rw [h]`: replace the left side of the equation `h` by its right side.
example (a b c : ℕ) (h₁ : a = b) (h₂ : b = c) : a = c := by
  rw [h₁, h₂]

-- `rw [← h]`: rewrite right to left.
example (a b : ℕ) (h : a = b) (p : ℕ → Prop) (hb : p b) : p a := by
  rw [h]
  exact hb

-- `simp`: rewrite with the simp lemmas (and the ones given) until nothing changes.
example (xs : List ℕ) : (xs ++ []).length = xs.length := by simp

-- `simp only [...]`: only the given lemmas and definitions (predictable).
def double (n : ℕ) : ℕ := n + n

example (n : ℕ) : double n = 2 * n := by
  simp only [double]
  omega

-- `simp at h`, `simp_all`: simplify hypotheses too.
example (a : ℕ) (h : a + 0 = 3) : a = 3 := by
  simp at h
  exact h

-- `simpa using e`: simplify the goal and `e`, then close the goal with `e`.
example (a : ℕ) (h : 0 + a = 3) : a = 3 := by simpa using h

-- `unfold`: unfold a definition.
example (n : ℕ) : double n = n + n := by
  unfold double
  rfl

-- `calc`: a chain of steps, each with its own proof.
example (a b c : ℕ) (h₁ : a ≤ b) (h₂ : b < c) : a < c + 1 :=
  calc a ≤ b := h₁
    _ < c := h₂
    _ < c + 1 := Nat.lt_succ_self c
```

```
-- `symm`, `congr`: symmetry of equality; equal arguments give equal results.
example (a b : ℕ) (h : a = b) : b = a := by
  symm
  exact h

example (f : ℕ → ℕ) (a b : ℕ) (h : a = b) : f a = f b := by
  congr
```

`rw [h]` rewrites with one equation at a time, left to right (`← h` for right to left), and fails if the left side does not occur. `simp` rewrites with a whole database of lemmas tagged `@[simp]`, plus the ones you list, until nothing changes; it is powerful but can be unpredictable, and a `simp` that closes a goal today may stop working when the library changes. Two habits keep proofs robust:

- prefer `simp only [lemma₁, lemma₂, definition]` in the middle of a proof, so that exactly the listed rewrites happen;
- use plain `simp` (or `simpa`) to *finish* a goal, where it does not matter how it gets there.

`simp at h` simplifies a hypothesis, and `simp_all` simplifies everything with everything. `calc` writes a chain of equalities and inequalities as it would appear on paper, each step with its own justification; it is often the clearest way to present an argument.

D.6 Structuring a proof

```
-- `have`: prove an intermediate fact and add it to the context.
example (a b : ℕ) (h : a < b) : a < b + b := by
  have hb : b ≤ b + b := Nat.le_add_right b b
  omega

-- `show`: state the goal (in a form that is equal by computation).
example (n : ℕ) : double n = n + n := by
  show n + n = n + n
  rfl

-- `case`, `next` and `·`: focus on one goal at a time.
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  case left => exact hp
  case right => exact hq

-- `all_goals tac` runs `tac` on every goal; `t₁ <;> t₂` runs `t₂` on every
-- goal that `t₁` produces; `first | t₁ | t₂` tries `t₁`, then `t₂`.
example (p : Prop) (hp : p) : p ∧ p ∧ p := by
  refine (?_, ?_, ?_)
  all_goals exact hp

example (n : ℕ) : n + 0 = n ∧ 0 + n = n := by
  constructor <;> simp

example (a b : ℕ) (h : a = b) : b = a := by
  first
  | rfl
  | exact h.symm
```

- `have h : P := proof` (or `have h : P := by tactics`) proves an intermediate fact. Most readable proofs are a short sequence of `have`s followed by a finishing tactic.
- `show P` restates the goal in an equivalent form; it documents the proof and can unfold definitions.

- `case name => ...` and `next => ...` focus on a named or the next goal; the bullet `·` focuses on the first goal.
- `all_goals t`, `t₁ <;> t₂`, `first | t₁ | t₂` and `repeat t` combine tactics when several goals need the same treatment.

D.7 Recursion and termination

```
-- `termination_by`: the measure that decreases in every recursive call.
def log2 (n : ℕ) : ℕ :=
  if h : n < 2 then 0 else 1 + log2 (n / 2)
termination_by n
decreasing_by omega

/-- info: 10 -/
#guard_msgs in
#eval log2 1024
```

Every definition must terminate (Chapter 2). Structural recursion is checked automatically; for other recursion, `termination_by` names a measure that decreases, and `decreasing_by` proves that it does, usually with `omega` or `simp`. When Lean cannot find the measure by itself, the error message shows the goal it could not prove (Chapters 7 and 31). A function defined this way can be reasoned about with `fun_induction` or with the induction principle Lean generates for it (`f.induct`), whose cases follow the function's own recursion.

D.8 Core Lean and Mathlib

All the tactics above come with Lean itself except these, which come from Mathlib (and need an `import`, for example `import Mathlib.Tactic.Ring`):

Tactic	Import
`ring`	`Mathlib.Tactic.Ring`
`linarith`	`Mathlib.Tactic.Linarith`
`norm_num`	`Mathlib.Tactic.NormNum`
`positivity`	`Mathlib.Tactic.Positivity`
`tauto`	`Mathlib.Tactic.Tauto`
`use`	`Mathlib.Tactic.Use`

D.9 Searching and debugging

- `exact?` searches the library for a lemma that closes the goal and prints the call it found; `apply?` lists lemmas that could be applied. Replace them with the suggestion once found.
- `trace_state` prints the current goal in the messages, which is useful in a terminal without an infoview.
- `sorry` closes any goal without a proof. It is how you sketch a proof top-down, but Lean warns about every use, and `#print axioms` reports `sorryAx` for any theorem that depends on one. No proof in this book uses it; Chapter 2 shows the warning.

APPENDIX E

Glossary

This glossary defines the main terms of the book: the vocabulary of type theory, logic and category theory, and the Rust and Lean terms that the chapters connect to it. Each definition is short and says what the term means in this book; the chapter at the end of the entry is the one that introduces the term or explains it best, and is the place to go for the full story. Closely related terms share an entry, listed under the first of them. Terms that begin with a Greek letter are listed under the letter's name: α under A, β under B, η under E, λ under L, Π under P and Σ under S. Appendix C lists the symbols.

In this appendix you will learn:

- what the central terms of type theory, logic and category theory mean, as the book uses them;
- the Rust and Lean features that the chapters connect to each term;
- which words the book uses in more than one sense;
- which chapter to read for the details of each term.

E.1 A

Abstract syntax. The tree structure of a program, which records which operation is applied to which subexpressions and forgets the characters the programmer typed (the concrete syntax). Type checkers and evaluators work on it. (Chapter 3)

Ad-hoc polymorphism. One name with different code at different types, such as `+`, `==` or `to_string`. Type classes, which Rust calls traits, make it as principled as the parametric kind. (Chapter 14)

Adjunction. A pair of functors L and R with a bijection $\mathrm{Hom}(L\,a, c) \cong \mathrm{Hom}(a, R\,c)$, natural in a and c, written $L \dashv R$. Currying is the first example, and every adjunction gives rise to a monad. (Chapter 19)

Admissible rule, derivable rule. A derivable rule abbreviates a fixed combination of the primitive rules, like a macro. An admissible rule only promises that its conclusion is derivable whenever its premises are; proving that usually takes an induction on derivations. (Chapter 6)

Affine λ-calculus. The programming language whose type system is affine logic, written λ^{aff}: typing rules with context splitting, in which weakening is allowed and contraction absent. It is the formal account of Rust's moves and of error E0382. (Chapter 22)

Affine logic. The logic that keeps weakening and drops contraction: every assumption is used at most once, and resources may be thrown away but not copied. It is the discipline of Rust's owned values that are not `Copy`. (Chapter 21)

Algebraic data types. Types built from products, sums, the unit type, the empty type and function types, such as Rust's structs and enums. If you count values, products multiply, sums add and function types are powers, hence the name. (Chapter 11)

Aliasing XOR mutation. The principle behind Rust's references: at any moment, a place may have many readers or one writer, never both. (Chapter 26)

α-equivalence. Two terms are α-equivalent, $t \underset{\alpha}{\equiv} u$, when they differ only in the names of bound variables, renamed consistently and without capture. They denote the same program. (Chapter 7)

Associated type. A type that each implementation of a trait must define, such as `Iterator::Item`, and that the implementing type therefore determines. Haskell's functional dependencies and Lean's `outParam` express the same idea. (Chapter 14)

Axiom. An inference rule with no premises, whose conclusion holds outright. In Lean the word also names the few assumptions of the logic itself, such as `propext` and `Classical.choice`; `#print axioms` lists the ones a theorem depends on. (Chapter 1)

E.2 B

β-reduction. The computation rule of the λ-calculus: a redex $(\lambda x.\, t)\, s$ reduces to $t[x := s]$. Under Curry–Howard it is the removal of a detour from a proof. (Chapter 7)

Bidirectional type checking. A type checking method that splits the typing judgment in two: synthesis, $\Gamma \vdash t \Rightarrow \tau$, where the type comes out of the term, and checking, $\Gamma \vdash t \Leftarrow \tau$, where an expected type flows in. Annotations are needed only where neither direction knows the type. (Chapter 8)

Big-step and small-step semantics. Two styles of operational semantics. A big-step semantics relates an expression directly to its final value, as in $\rho \vdash e \Downarrow v$ (Chapter 3); a small-step semantics describes one step of computation at a time, $t \to t'$, which is the style of the type safety proof (Chapter 9).

Borrow. A temporary transfer of permission from an owner. `&x` lends a read permission that the owner keeps too; `&mut x` lends the owner's read-write permission, and the owner is unusable until the borrow's region ends. (Chapter 26)

Borrow checker. The part of rustc that enforces Rust's rules for moves, borrows and lifetimes. It is not a type system in the textbook sense but a set of dataflow analyses over MIR. (Chapter 29)

Brouwer–Heyting–Kolmogorov interpretation. BHK. The constructive reading of the connectives as kinds of proof: a proof of $A \vee B$ is a proof of one side together with the information which, and a proof of $A \to B$ is a method that turns proofs of A into proofs of B. (Chapter 5)

E.3 C

Calculus of Constructions. The corner λC of the λ-cube, due to Coquand and Huet (1988), in which terms and types live in one language and any of them may depend on any other. Lean's type theory extends it with inductive types, universes and quotient types. (Chapter 30)

Cartesian closed category. CCC. A category with a terminal object, binary products and exponentials. The simply typed λ-calculus can be interpreted in any CCC, which completes the Curry–Howard–Lambek triangle. (Chapter 19)

Category. A collection of objects and of morphisms (arrows) $f : a \to b$ between them, with an identity morphism on every object and an associative composition $g \circ f$ ("first f, then g") for which the identities are units. Equations between composites are drawn as commutative diagrams. (Chapter 16)

Church encoding. A representation of data by what you can do with it, using only λ-terms: booleans as functions that choose, numerals as functions that iterate, pairs as functions that hand over two components. (Chapter 7)

Classical logic. The logic of truth tables, obtained from intuitionistic natural deduction by adding excluded middle, $A \vee \neg A$, or an equivalent rule such as *reductio ad absurdum*. It is not wrong, but its proofs of $A \vee B$ need not say which side holds. In Lean, excluded middle is `Classical.em`, which follows from the axiom `Classical.choice`. (Chapter 5)

`Clone`. The Rust trait of types that can be duplicated explicitly, by a call to `clone` that may cost something and may have effects. In the affine λ-calculus it is a term of type $\tau \multimap \tau \otimes \tau$, not a structural rule. Every `Copy` type is `Clone`. (Chapter 23)

Closure. A function value together with the values of its free variables, captured from the environment in which it was created. The captures of a Rust closure are the free variables of its body. (Chapter 8)

Coercion. An implicit conversion that the compiler inserts where the expected type is known, such as an unsizing coercion to `Box<dyn Trait>` or a deref coercion. Unlike Rust's subtyping between lifetimes, these coercions do real work at run time. (Chapter 27)

Coherence. The property of a type-class system that all derivations of a constraint elaborate to programs with the same behavior, so that the meaning of a program does not depend on which instance the compiler found. Rust ensures it by forbidding overlapping impls and by the orphan rule: an impl must live in the crate of the trait or in the crate of the type (E0117). (Chapter 14)

Confluence. The property that however two reductions of a term diverge, they can be brought back together. The Church–Rosser theorem says that the λ-calculus is confluent, so a term has at most one normal form. (Chapter 7)

Consistency. A logic is consistent when $\bot$ has no proof. Lean is consistent as a logic thanks to totality; Rust, read as a logic, is not, because `loop {}` and `panic!()` inhabit every type. (Chapter 10)

Const generic. A value-level parameter of a type or function, such as the `N` of `[T; N]`. It is Rust's first step toward dependent types; stable Rust allows no arithmetic on such parameters. (Chapter 30)

Constraint, entailment. A constraint is a statement about types, such as $\mathit{Monoid}\ \alpha$, which a qualified type $\pi \Rightarrow \tau$ requires; Rust's trait bounds are constraints. Entailment, $P \Vdash \pi$, is the judgment that the constraints P imply π, and each instance declaration is one of its rules. (Chapter 14)

Context. The assumptions of a judgment, usually written Γ: in type theory, the variables in scope with their types; in logic, the propositions assumed. (Chapter 1)

Context splitting. Dividing the assumptions of a rule's conclusion between its premises, $\Gamma = \Gamma_1 \uplus \Gamma_2$, so that each assumption goes to exactly one of them. It is how substructural systems refuse contraction, and in the affine λ-calculus it is the move checker. (Chapter 22)

Continuation-passing style. CPS. A style in which a computation, instead of returning its value, passes it to a continuation that says what to do with it. The type $\forall\beta.\,(\alpha \to \beta) \to \beta$ is isomorphic to α. (Chapter 18)

Contraction. The structural rule that merges two copies of an assumption into one, so that an assumption may be used more than once. For programs it is duplication, which Rust refuses for values that are not `Copy`. (Chapter 6)

Coproduct. The dual of a product: an object with two injections through which every pair of morphisms out of the components factors uniquely. In the category of types it is the sum type. (Chapter 16)

`Copy`. The Rust trait of types whose values are duplicated implicitly, by a bitwise copy, so that using them does not move them. It behaves like the exponential ! of linear logic: a type may be `Copy` only if all its parts are, and never if it has a destructor. (Chapter 23)

Covariant, contravariant. A type constructor F is covariant when $\sigma <: \tau$ implies $F\,\sigma <: F\,\tau$, as `&'a T` is in `'a`, and contravariant when it implies $F\,\tau <: F\,\sigma$, as `fn(T)` is in `T`. Categorically, they are functors that keep or reverse the direction of arrows. (Chapters 17 and 27)

Curry–Howard correspondence. The fact that the typing rules of a typed λ-calculus and the rules of a logic are the same rules: propositions are types, proofs are programs, and simplifying a proof is running a program. (Chapter 10)

Currying. Turning a function of two arguments into a function that returns a function, $(A \times B \to C) \cong (A \to B \to C)$. In category theory it is the first example of an adjunction. (Chapter 7)

Cut. The rule that proves a lemma and then uses it. Cut elimination, Gentzen's *Hauptsatz*, says that every derivation in the sequent calculus can be transformed into one without cuts. (Chapter 4)

E.4 D

Dataflow analysis. A static analysis that computes facts at every point of a control-flow graph by letting them flow along the edges until nothing changes, which is a fixpoint. rustc's move analysis, liveness and loan tracking are dataflow analyses. (Chapter 29)

de Bruijn index. A nameless representation of a bound variable: the number of λs between the occurrence and its binder. With de Bruijn indices, α-equivalent terms are identical. (Chapter 7)

`Decidable`. The Lean class of propositions for which an algorithm returns either a proof or a refutation: a decision procedure. It is what lets `if` test a proposition and what the tactic `decide` evaluates. (Chapter 14)

Deep embedding, shallow embedding. Two ways to study a logic in Lean. A shallow embedding uses Lean's own propositions and connectives, which is convenient for proving things in the logic but

not about it. A deep embedding defines the syntax and the rules as ordinary inductive types, so that formulas and derivations are values that theorems can talk about. (Chapter 4)

Definitional equality, propositional equality. Definitional equality is what the type checker sees by itself, by unfolding definitions, β-reduction and pattern matching on constructors; `rfl` proves exactly these equations. Propositional equality is the type `a = b` of everything that can be proved equal, possibly by induction; every definitional equality is propositional, not the other way around. (Chapter 30)

Denotational semantics. A semantics that maps the syntax of a language to mathematical objects, such as Lean values or the morphisms of a cartesian closed category. (Chapter 8)

Dependent type. A type that mentions a value, such as the type of arrays of length `n` or the type of proofs that `i < n`. Lean has them in full; Rust approximates them with const generics. (Chapter 30)

Derivation. A tree of rule instances whose root is the judgment to be established and whose leaves are axioms. A judgment holds exactly when it has a derivation, and type checking is the search for one. (Chapter 1)

Detour. An introduction rule immediately followed by an elimination of the same connective. Local soundness says that every detour can be removed; under Curry–Howard a detour is a redex. (Chapter 4)

Diagonal. The morphism $\langle \mathrm{id}, \mathrm{id} \rangle : a \to a \times a$ that pairs a value with itself: the categorical name for using a value twice, and so for contraction. Rust provides it freely only for `Copy` types. (Chapter 16)

Dictionary passing. The translation of a program with type classes into one without them, in which every constraint becomes an extra argument, a record of the class's operations. Rust's generics usually remove the dictionaries by monomorphization, and `dyn Trait` keeps them as vtables. (Chapter 14)

Disjunction property, existence property. If $A \vee B$ is provable intuitionistically with no assumptions, then A is provable or B is; if $\exists x.\, A(x)$ is provable, then $A(t)$ is provable for some term t. Constructive proofs carry the information which side holds, and a witness. (Chapter 5)

Double-negation translation. An embedding of classical logic into intuitionistic logic that inserts double negations. For propositional logic it is Glivenko's theorem: A is a classical tautology if and only if $\neg\neg A$ is intuitionistically derivable. (Chapter 5)

`Drop`. The Rust trait for destructors: code that runs when a value is discarded, to release what it holds. It turns weakening into an action with an effect, which is why a type with a `Drop` impl cannot be `Copy`. (Chapter 24)

Dyn compatibility. The property of a trait that allows `dyn Trait`, called object safety in older documentation. Its rules follow from the typing of existential packages and from the vtable representation; violations are E0038. (Chapter 15)

E.5 E

Effect. Anything a computation does besides returning a value: failing, producing several answers, reading or updating state, waiting. Functions with effects do not compose like plain functions; monads make them compose again. (Chapter 20)

Elaboration. Translating what the programmer wrote into a fully explicit term. Lean's elaborator fills in implicit arguments, instances and coercions by unification (Chapter 13), and a type checker for type classes elaborates constraints into dictionary arguments (Chapter 14).

Elimination restriction. Lean's rule that a proof of a proposition can be taken apart only to build another proof. That is why the witness of an `∃` cannot be extracted as data. (Chapter 10)

Empty type. A type with no values, written $\mathbf{0}$: Rust's `enum Void {}` and `Infallible`, Lean's `Empty`. It is "false" under Curry–Howard and an initial object in the category of types, and a `match` on it needs no arms. (Chapter 11)

Escape check. The rule that a value whose type mentions a region may not leave that region, written $\rho \in \Delta$ in the region discipline of Chapter 26 and $\alpha \notin \mathrm{flv}(\sigma)$ in the borrow calculus of Chapter 28. Rust reports violations as E0597 or E0515. (Chapter 26)

η-rules. Equations that say a value is determined by what you can do with it: $\lambda x.\, f\, x$ is f, and the pair of the two projections of p is p. Logically they are local completeness. (Chapter 8)

Exchange. The structural rule that lets two neighboring assumptions swap places. Ordered logic drops it; Rust keeps it. (Chapter 6)

Existential type. A type $\exists \alpha.\, \tau$ that packs a hidden type with a value that uses it; clients may use the package only through τ. Abstract data types, `impl Trait` in return position (an opaque type) and `dyn Trait` are existentials. (Chapter 15)

Exponential. The function type $A \to B$, written B^A because it has $|B|^{|A|}$ values (Chapter 11). In a category, the object $b \Rightarrow c$ that comes with eval and curry and satisfies their universal property (Chapter 19).

Exponential modality. The connective $!A$ of linear logic, "of course A", which marks the formulas that may be duplicated and discarded. Its rules are weakening, contraction, dereliction (use a $!A$ once, as an A) and promotion, and Rust's `Copy` behaves like it. (Chapter 23)

E.6 F

Fixed-point combinator. A term such as Curry's Y that, applied to a function g, produces a fixed point of g. It gives the untyped λ-calculus recursion; Z is the variant that works under call-by-value. (Chapter 7)

`Fn`, `FnMut`, `FnOnce`. Rust's closure traits, ordered by how often a closure may be called: `FnOnce` at most once, an affine function; `FnMut` any number of times, one call at a time; `Fn` any number of times, even from several places at once. (Chapter 8)

Free theorem. An equation that every function of a given polymorphic type satisfies, read off the type by parametricity. For example, every function of type $\forall \alpha.\, \alpha \to \alpha$ is the identity. (Chapter 18)

Free variable, bound variable. In $\lambda x.\, x\, y$, the x is bound: it refers to the enclosing binder. The y is free: no binder in the term binds it, and its meaning must come from outside. (Chapter 7)

Functor. A map between categories, on objects and on morphisms, that preserves identities and composition (the functor laws). For types, it is a type constructor with a lawful `map`; a functor from a category to itself is an endofunctor. (Chapter 17)

E.7 G

Generic associated type. GAT. An associated type that takes parameters of its own, stable since Rust 1.65. It lets a trait say "the same container, with a different element type", a partial workaround for higher-kinded types. (Chapter 17)

Goal, infoview. The goal is what remains to be proved in a Lean proof. The infoview, the panel of a Lean editor that shows what Lean knows at the cursor, displays it after the turnstile `⊢`, below the hypotheses. (Chapters 1 and 2)

E.8 H

Harmony. The requirement that the elimination rules of a connective extract exactly what its introduction rules put in. Its two halves are local soundness (a detour can be removed, the source of β-reduction) and local completeness (a formula can be taken apart and rebuilt, the source of η). Prior's `tonk` shows what goes wrong without it. (Chapter 4)

Heyting algebra. A bounded lattice with an implication characterized by residuation: $c \leq (a \Rightarrow b)$ if and only if $c \sqcap a \leq b$. Heyting algebras are the algebraic semantics of intuitionistic logic, as Boolean algebras are of classical logic. Categorically, a Heyting algebra is a thin bicartesian closed category. (Chapter 5)

Higher-kinded types. Type parameters that range over type constructors such as `Option`, rather than over types. Rust lacks them, which is why it has no `Functor` or `Monad` trait. (Chapter 17)

Higher-ranked trait bound. HRTB. A bound written with `for<'a>`, which requires a trait to hold for every lifetime, for example of a callback that must accept references of any lifetime. It gives Rust a form of rank-2 polymorphism. (Chapter 12)

Hindley–Milner. The type system of ML, in which polymorphism is restricted to `let`-bound variables: generalization turns the type of a `let` definition into a type scheme $\forall \alpha_1 \ldots \alpha_n.\, \tau$, and each use takes an instance. Its types can be inferred without annotations, and Milner's Algorithm W finds principal types. Rust's local inference is a restricted cousin. (Chapter 13)

Hoare triple. A specification $\{P\}\ c\ \{Q\}$: from any state satisfying the precondition P, the command c runs without fault to a state satisfying the postcondition Q. (Chapter 32)

E.9 I

Impredicativity. A quantifier is impredicative when its range includes the type it is part of: in Lean, `∀ p : Prop, p → p` is itself a `Prop`. System F is impredicative; Lean's hierarchy of `Type`s is predicative, and impredicativity survives only in `Prop`. (Chapter 10)

Inductive definition, rule induction. An inductive definition defines a set as the smallest set closed under some rules, such as the grammar of expressions or a typing relation. Rule induction proves that every member has a property by showing that each rule preserves it. (Chapter 3)

Inductive family. An inductive type indexed by values, such as Lean's `≤` on natural numbers or the typing relation of a toy language. A function from values to types, such as a typestate `Conn : State → Type`, is called an indexed family. (Chapter 10)

Inductive predicate. An inductive type whose values are proofs, such as `HasType : Ctx → Expr → Ty → Prop`, in which each constructor is one rule. (Chapter 3)

Inference rule. A rule that says how a judgment follows from others: premises above a line, the conclusion below it, the name on the right. A rule is a schema whose instances replace its metavariables. (Chapter 1)

Instance. The book uses the word in two senses. A type is an instance of a type scheme when it is obtained by choosing types for the quantified variables (Chapter 13). An instance of a type class declares that a type belongs to the class and gives its operations; in Rust it is an `impl`, and instance resolution finds the instances a constraint needs, like Prolog solving Horn clauses (Chapter 14).

Interior mutability. Mutation through a shared reference, made safe by moving the aliasing check from compile time to run time, as `RefCell` does, or by never handing out references to the contents, as `Cell` does. (Chapter 26)

Intrinsic and extrinsic typing. In extrinsic typing, terms exist on their own and a separate typing relation says which are well typed. In intrinsic typing, the type of a term is part of its definition, as in a Lean type `Term Γ τ`, so that ill-typed terms cannot be constructed at all. (Chapter 8)

Introduction and elimination rules. An introduction rule says how to prove a formula with a given connective at the root, such as ∧I or →I; an elimination rule says how to use one, such as ∧E or →E. Under Curry–Howard they are the constructors and the destructors of a type. (Chapter 4)

Intuitionistic logic. The constructive logic of natural deduction without classical rules: to prove $A \vee B$ you must say which, and to prove $\exists x.\, A(x)$ you must produce an x. Under Curry–Howard it is the logic of programs. (Chapter 5)

Invariant. The book uses the word in two senses. A type constructor is invariant in a parameter when subtyping between arguments does not carry over in either direction, as `&mut T` is in `T` (Chapter 27). In a proof, an invariant is a property that every derivation or every reachable state has, used to show that something cannot happen (Chapter 21).

Inversion. Reasoning backwards from a derivable judgment to the premises that must have produced it. It works smoothly in a syntax-directed system, where the form of the expression determines the only rule that can conclude it; in Lean it is `cases` on the derivation. (Chapter 3)

Isomorphism. A morphism with an inverse; between types, a pair of functions that undo each other, written $A \cong B$. Isomorphic types are interchangeable but not equal. (Chapter 11)

E.10 J

Judgment. A statement about syntax that is established by a derivation, or not at all, such as $\Gamma \vdash e : \tau$. It belongs to the metalanguage: it is not a boolean expression of the program. (Chapter 3)

E.11 K

Kernel. The small, carefully engineered part of Lean that checks every proof term. Tactics and the elaborator search for proofs, and nothing is accepted that the kernel has not checked. (Chapter 1)

Kind. The type of a type expression: `Type` (some texts write $*$) for types that classify values, `Type → Type` for type constructors such as `Option`. (Chapter 17)

Kleisli arrow. A function $A \to M\,B$ whose effect is described by the monad M. Kleisli arrows are the morphisms of the Kleisli category of M, with `pure` as identities and Kleisli composition, and the monad laws are its category laws. (Chapter 20)

Kripke model. A semantics for intuitionistic logic: a preorder of worlds, read as states of knowledge, in which known atoms stay known. The forcing relation $w \Vdash A$, "at world w we know A", looks at every future world for implication and negation, and a two-world model refutes excluded middle. (Chapter 5)

E.12 L

λ-calculus. Church's calculus of functions, with only variables, abstractions $\lambda x.\, t$ and applications $t\ u$, which can nevertheless express every computation. The untyped version has terms that run forever; its typed versions are the core of type theory. (Chapter 7)

λ-cube. Barendregt's arrangement of eight typed λ-calculi by three kinds of dependency: terms on types (polymorphism), types on types (type operators) and types on terms (dependent types). (Chapter 30)

Leak. A value that is never dropped, through `mem::forget`, `Box::leak` or a cycle of `Rc` pointers. Leaking is safe in Rust: `mem::forget` was made safe before Rust 1.0, after the episode known as the Leakpocalypse, which is why Rust is affine and not linear, and why the soundness of `unsafe` code must not depend on a destructor running. (Chapter 24)

Leftovers. In algorithmic affine type checking, the part of the context that a subterm did not use, which is passed on to the next subterm. The judgment is $\Gamma \vdash e : \tau \dashv \Delta$. (Chapter 22)

Lifetime. In Rust, a name such as `'a` for a region of the program during which a reference may be used: a region variable. It is not how long a value lives. (Chapter 26)

Linear logic. Girard's logic (1987), in which every assumption is used exactly once. It drops both weakening and contraction, has two conjunctions and two disjunctions, and recovers reuse through the exponentials. (Chapter 21)

Liveness. A local is live at a program point if the value it holds there may be read later, along some path. Non-lexical lifetimes are built on liveness. (Chapter 29)

Loan. The record of a borrow expression in the borrow checker: which place was borrowed, how, and with which region. A loan is in scope at the points it can reach without leaving its region. (Chapter 29)

Logical relation. A family of predicates or relations on values, defined by recursion on types, that says what it means to behave like a value of a type. It is the main tool of semantic typing, and its fundamental theorem, from which type safety follows, says that every syntactically well-typed term is semantically well typed. (Chapter 32)

E.13 M

Mathlib. The library of mathematics maintained by Lean's community, with definitions, hundreds of thousands of theorems and many tactics. The book imports only the parts it uses. (Chapter 2)

Metavariable. The book uses the word in two senses. In a rule, a letter such as Γ, e or τ that stands for any context, term or type (Chapter 1). In Lean's elaborator, an unknown such as `?m.5` that unification must solve (Chapter 13).

MIR. rustc's mid-level intermediate representation: a control-flow graph of basic blocks, straight-line statements that act on places and end in a single terminator. The borrow checker runs on it. (Chapter 29)

Miri. An interpreter for MIR that checks every step for undefined behavior, including the aliasing rules of Stacked Borrows or Tree Borrows. It is a component of the nightly toolchain. (Chapter 32)

Monad. A type constructor M with $\mathit{pure} : A \to M\ A$ and $\mathit{bind} : M\ A \to (A \to M\ B) \to M\ B$ satisfying three laws (left identity, right identity and associativity): the structure that makes effectful functions compose. `Option`, `Result`, lists and state are monads, and monad transformers such as `StateT` combine several effects. (Chapter 20)

Monoid. A type with an associative operation and a neutral element, such as numbers with `+` and `0` or strings with concatenation and `""`. As a category, a monoid is a category with one object. (Chapter 14)

Monoidal category. A category with a product-like operation $\otimes$ and a unit object I, associative and unital up to isomorphism, without the diagonals and projections of a cartesian product. Symmetric monoidal closed categories are the models of linear logic. (Chapter 19)

Monomorphism, epimorphism. A monomorphism can be cancelled on the left, $f \circ g = f \circ h$ implies $g = h$; an epimorphism on the right. In the category of types they are exactly the injective and the surjective functions. (Chapter 16)

Monomorphization. Rust's implementation of generics: for every type at which a generic function is used, the compiler generates a separate copy of it. (Chapter 12)

Move. In Rust, using a value of a non-`Copy` type by value: ownership passes to the new place, and the old one may not be used again (E0382). In the affine λ-calculus, a move is a variable that goes to exactly one premise of a split. (Chapter 22)

Multiplicative and additive connectives. The two ways a rule of linear logic can treat the context: split it between the premises, as $\otimes$ does, or give all of it to each premise, as $\&$ does, so that only one of the premises can be used. (Chapter 21)

Multiplicity. How many times a variable may be used: once for an ordinary variable of an affine calculus, without bound for a `Copy` variable, not at all for a variable that a term did not receive. Affine types are about multiplicity; lifetimes are about extent, where a reference is valid. (Chapter 23)

`#[must_use]`. A Rust attribute that makes the compiler warn when a value is discarded on the spot. It is a lint, not a typing rule, and the closest Rust comes to relevant or linear types. (Chapter 24)

E.14 N

Natural deduction. Gentzen's proof system in which each connective has introduction and elimination rules, and a proof is a tree of judgments under assumptions. (Chapter 4)

Natural transformation. A family of morphisms $\eta_a : F\,a \to G\,a$, one for each object, such that the naturality square $G\,f \circ \eta_a = \eta_b \circ F\,f$ commutes. Polymorphic functions such as `Option::into_iter` and `reverse` are natural transformations. (Chapter 18)

Never type. Rust's type `!` of expressions that produce no value, such as `panic!(...)` and `loop {}` without a `break`. It has no values and coerces to every type, the type-theoretic version of "false". (Chapter 1)

Nominal typing, structural typing. A type is nominal when its identity is its declaration, so two structs with the same fields are different types, and structural when its identity is its structure, as for tuples, arrays and function-pointer types. (Chapter 3)

Non-lexical lifetimes. NLL. Rust's borrow checking since the 2018 edition, in which a region is a set of points of the control-flow graph, computed from liveness, rather than a lexical scope. (Chapter 29)

Normal form. A term that contains no redex, the "answer" of a computation. Not every untyped term has one (Ω does not), and by confluence no term has two. (Chapter 7)

Normalization. Reducing a derivation, or a term, until no detour remains. Prawitz's normalization theorem says that every derivation of natural deduction can be normalized; strong normalization, which the simply typed λ-calculus has and Rust does not, says that every reduction sequence ends. (Chapter 4)

E.15 O

Occurs check. The test that prevents unification from binding a type variable to a type that contains it, as in $\alpha \doteq \alpha \to \beta$. It is why $\lambda x.\, x\, x$ has no type. (Chapter 13)

Opposite category. The category $\mathcal{C}^{\mathrm{op}}$ with the same objects as $\mathcal{C}$ and every arrow reversed. By duality, every construction has a mirror image: terminal and initial objects, products and coproducts, monos and epis. (Chapter 16)

Outlives. The order on lifetimes: `'a: 'b`, read "`'a` outlives `'b`", means that a reference valid for `'a` is also valid for `'b`. The bound `T: 'a` says that every reference inside `T` outlives `'a`. (Chapter 26)

Ownership. Rust's rule that every value has an owner, the variable or place responsible for dropping it. When a value is moved, ownership moves with it, and so does the responsibility for dropping it. (Chapter 24)

E.16 P

Parameter, index. The two kinds of arguments of an inductive family. A parameter is the same in the result type of every constructor, like the `T` of Rust's `Option<T>`; an index may be chosen by each constructor, like the length in a type of vectors `Vec α n`. (Chapter 30)

Parametric polymorphism. One piece of code that works uniformly at every type and cannot tell which type it got, as in System F and in Rust's generics. (Chapter 12)

Parametricity. Reynolds's principle that polymorphic functions preserve relations between the types they are used at, from which the free theorems follow. It is a theorem about the language, not provable inside Lean, and Rust breaks it with features such as `Any` and `size_of`. (Chapter 18)

`partial`. A Lean marker for a definition that may not terminate. The function is compiled and runs, but for the logic it is opaque, and Lean accepts it only at a type that is inhabited, so it cannot prove a false proposition. (Chapter 2)

Permission. The reading of references as rights over a place: the owner may read, write and free; `&mut T` is a unique read-write permission and `&T` a duplicable read permission. (Chapter 26)

`PhantomData`. A zero-sized field that tells the Rust compiler to treat a struct as if it contained a value of some type. Typestate uses it to mention a state parameter, and it also chooses the variance of a type. (Chapter 25)

Π-type. The type $\Pi x : A.\, B$ of dependent functions, whose result type may depend on the argument, written `(x : A) → B` in Lean. The function type $A \to B$ is the case where B does not mention x, and under Curry–Howard Π is ∀. (Chapter 30)

Place, partial move. A place is an expression that denotes a location in memory: a local variable, possibly followed by projections such as a field `p.0` or a dereference. Rust tracks moves and borrows per place, so moving one field out of a struct, a partial move, leaves the other fields usable. (Chapter 22)

Polonius. A reformulation of Rust's borrow checking as Datalog rules over loans and origins (regions seen as sets of loans), which accepts safe programs such as "problem case #3" that NLL rejects. Bringing its reasoning into rustc is ongoing work: check the current status. (Chapter 29)

Preorder. A reflexive and transitive relation $\leq$. Every preorder is a thin category, one with at most one morphism between any two objects, and both subtyping and the outlives relation on lifetimes are preorders. (Chapter 16)

Preservation. The half of type safety which says that a well-typed term that takes a step is still well typed, with the same type. (Chapter 9)

Principal type. A type of a program of which every other type of the program is an instance, such as $(a \to a) \to a \to a$ for `twice`. In Hindley–Milner every typable program has one. (Chapter 13)

Product. An object with two projections through which every pair of morphisms into the components factors uniquely, a definition by universal property. In the category of types it is the pair type $A \times B$. (Chapter 16)

Progress. The half of type safety which says that a well-typed closed term is a value or can take a step: it is never stuck. (Chapter 9)

Promotion. The rule of linear logic that builds a duplicable value $!e$ only from a context in which everything is duplicable. In Rust it is the rule that a type can be `Copy` only if all its fields are (E0204). (Chapter 23)

Proof irrelevance. In Lean, any two proofs of the same proposition are definitionally equal, so a proof carries no information beyond the fact that it exists. (Chapter 10)

Proof term. The term that a proof is under Curry–Howard. Tactics build one, the kernel checks it, and `#print` shows it. (Chapter 4)

`Prop`. Lean's universe of propositions. Its elements are proved rather than computed, any two proofs of the same proposition are equal, and it is impredicative. (Chapter 10)

Proposition. Something that may be proved, such as $A \wedge B$: a piece of syntax that we talk about, as opposed to a judgment, which is an act of knowledge about it. Under Curry–Howard, a type. (Chapter 4)

E.17 R

RAII. "Resource acquisition is initialization": tie each resource to a value whose destructor releases it, so that the compiler releases the resource on every path out of the scope, including early returns and panics. (Chapter 24)

`Rc` and `Arc`. Reference-counted pointers that share a value among several owners instead of duplicating it: cloning one increments a counter, dropping one decrements it. `Arc` uses an atomic counter. Because of the counting, neither can be `Copy`. (Chapter 23)

Reborrow. A new reference made from an existing one for a shorter region, such as `&mut *r`, which the compiler inserts when a `&mut` is passed to a function. The original reference is usable again when the reborrow ends. (Chapter 28)

Recursor. The most general elimination principle of an inductive type, which Lean generates automatically for every `inductive` declaration. Folds and proofs by induction are uses of it. (Chapter 11)

Redex. A reducible expression: an application whose function part is an abstraction, $(\lambda x.\, t)\, s$. Under Curry–Howard, a detour. (Chapter 7)

Reduction strategy. A rule that chooses which redex to contract next. Normal order contracts the leftmost, outermost redex, also under λ, and finds a normal form whenever there is one; call-by-name and call-by-value never reduce under λ, and call-by-value, which is what Rust does, first reduces the argument to a value. (Chapter 7)

Refinement type. A type together with a predicate on its values, such as "integers between 0 and 100". Lean's subtypes are refinement types with proofs; Rust approximates them with newtypes that have private fields. (Chapter 30)

Region. A part of a program during which some memory may be used. Region calculi such as Tofte and Talpin's attach regions to pointer types; Rust's lifetimes are region variables, and under NLL a region is a set of program points. (Chapter 26)

Relevant logic. The substructural logic that keeps contraction and drops weakening: every assumption must be used at least once. (Chapter 21)

RustBelt. The project of Jung, Jourdan, Krebbers and Dreyer (POPL 2018) that proved, with proofs checked in Coq, the soundness of λRust, a core language close to MIR, and of standard library abstractions built with `unsafe`, by semantic typing in Iris, a higher-order concurrent separation logic, with a logic of lifetimes. Iris is affine: $P * Q \vdash P$. (Chapter 32)

E.18 S

Safety invariant, validity invariant. The two invariants of a type that `unsafe` code must respect. The validity invariant holds at all times, because the compiler relies on it (a `bool` is 0 or 1); the safety invariant holds whenever safe code can observe the value (a `String` holds UTF-8). (Chapter 32)

Semantic typing. Defining well-typedness by behavior instead of by derivations: $\Gamma \vDash e : \tau$ when e, run with good values for its variables, produces a good value of type τ. It covers safe abstractions built on `unsafe` code that no type checker accepts. (Chapter 32)

Separation logic. A logic for programs with pointers, due to Reynolds, O'Hearn and their collaborators, in which an assertion describes the piece of the heap that you own. Its separating conjunction $P * Q$ says that the heap splits into disjoint parts satisfying P and Q, and its frame rule makes proofs local. (Chapter 32)

Sequent. A judgment $\Gamma \vdash A$: from the assumptions in the context Γ, the formula A is true. (Chapter 4)

Sequent calculus. Gentzen's second presentation of logic, with right rules that prove a connective and left rules that use an assumption, instead of introductions and eliminations. It suits proof search and the study of the structural rules. (Chapter 4)

Session type. The type of a communication channel, describing the whole sequence of messages on it: $!\tau.\, S$ sends a τ, $?\tau.\, S$ receives one, *end* stops. The two ends of a channel have dual types, with sends and receives swapped. (Chapter 25)

Σ-type. The type $\Sigma x : A.\, B$ of dependent pairs, in which the type of the second component depends on the first. $A \times B$ is the case where B does not mention x, and under Curry–Howard Σ is ∃. (Chapter 30)

Simply typed λ-calculus. STLC. The λ-calculus with types: base types, functions, and in this book unit, booleans, naturals, pairs and sums. It is the smallest interesting type theory, every well-typed term terminates, and under Curry–Howard it is intuitionistic logic. (Chapter 8)

`sorry`. A Lean placeholder that proves anything. Lean accepts it with a warning and records it as the axiom `sorryAx`; no proof in this book depends on it. (Chapter 2)

Soundness, completeness. A system is sound for a semantics when everything it derives is true there, and complete when everything true there is derivable. The book proves a type checker sound for its rules (Chapter 3) and natural deduction sound for truth tables (Chapter 4); intuitionistic natural deduction is complete for Kripke models but not for truth tables (Chapter 5). Type safety is the soundness of a type system for its semantics (Chapter 9).

Stacked Borrows, Tree Borrows. Two proposed aliasing models for Rust, which say which uses of references and raw pointers are undefined behavior by tracking a stack, or a tree, of permissions for each location. Neither is, at the time of writing, the official definition; Miri can check programs against both. (Chapter 32)

Strict positivity. Lean's condition on inductive types: the type being defined must never appear to the left of an arrow in the arguments of its constructors. Without it, non-terminating terms could be built, and with them a proof of `False`. (Chapter 11)

Structural recursion, well-founded recursion. Lean's two ways to see that a recursive function terminates. In structural recursion every call is on a piece of the argument, and Lean accepts it automatically; in well-founded recursion a measure decreases at every call, named with `termination_by` and proved with `decreasing_by` when Lean cannot find the proof itself. (Chapter 2)

Structural rules. The rules that mention no connective and talk only about the context: exchange, weakening and contraction. Which ones a logic keeps decides what an assumption is. (Chapter 6)

Stuck. A term that is not a value and cannot take a step, such as `true + 5`: the formal version of a crash. Type safety says that well-typed programs never get stuck. (Chapter 1)

Substitution. Replacing the free occurrences of a variable by a term, written $t[x := s]$ or $t[s/x]$. It must be capture-avoiding: bound variables are renamed when necessary, so that no free variable of the substituted term is captured. In type inference, a substitution is a map from type variables to types. (Chapter 7)

Substitution lemma. Substituting a term of the right type for a variable preserves typing. The proof of preservation depends on it, and in a cartesian closed category it says that substitution is composition. (Chapter 9)

Substructural logic. A logic obtained by deleting some of the structural rules: affine, relevant and linear logic, and ordered logic, which drops exchange as well, so that assumptions must be used in order. Which rules are deleted decides what an assumption is: a fact, a resource that may be discarded, an obligation, or a resource that must be used exactly once. (Chapter 6)

Subtype. The book uses the word in two senses, and Chapter 27 warns against confusing them. In Lean, a subtype `{x // p x}` is the type of values paired with a proof of a property, a refinement type (Chapter 11). In subtyping, σ is a subtype of τ, written $\sigma <: \tau$, when a σ can be used wherever a τ is expected (Chapter 27).

Subtyping. A preorder $\sigma <: \tau$ on types, used through the rule of subsumption: a term of type σ may be used at type τ. Rust's only subtyping is between lifetimes, as when a `&'static T` is used as a `&'a T`; Lean has none. (Chapter 27)

System F. The polymorphic λ-calculus of Girard and Reynolds, with type abstraction $\Lambda\alpha.\, e$, type application and types $\forall\alpha.\, \tau$. It is impredicative; Rust's generics behave like its rank-1 fragment, with trait bounds added. (Chapter 12)

E.19 T

Tactic. An instruction that transforms the current goal of a Lean proof, closing it or replacing it with simpler goals. A proof written with tactics, in tactic mode, builds a proof term. (Chapter 1)

Terminal object, initial object. An object is terminal when every object has exactly one morphism to it, and initial when it has exactly one morphism to every object. In the category of types they are the unit type and the empty type. (Chapter 16)

Totality. The property of a function that returns a value for every input, and in particular terminates. Lean requires it of every definition that proofs can use, because a single non-terminating definition could prove `False`. (Chapter 2)

Trait object. A value of type `dyn Trait`, reached through a pointer that carries the data pointer together with a pointer to a vtable, the table of the trait's methods for the hidden type. It is an existential package, and the vtable is the run-time form of a dictionary. (Chapter 15)

Transport. Using a proof of `a = b` to turn a statement or a value about `a` into one about `b`; in Lean, `h ▸ e`, `subst` and `rw`. It is the elimination rule of the equality type. (Chapter 30)

Two-phase borrows. The rule that a mutable borrow inserted by the compiler, as in `v.push(v.len())`, is first only reserved, behaving like a shared borrow, and becomes active at its first use. (Chapter 29)

Type class. A named set of operations with their types, together with instances that declare which types belong to it; Wadler and Blott's design for principled ad-hoc polymorphism. Rust's traits and Lean's classes are type classes, and a Lean class can also require proofs of laws. (Chapter 14)

Type constructor. Something like `Option` or `Vec`, of kind `Type → Type`: a function from types to types, not a type itself. (Chapter 17)

Type inference. Reconstructing the types a programmer left out, by treating them as unknowns and solving the equations that the program imposes on them. Rust's inference is local to function bodies, with signatures as boundaries. (Chapter 13)

Type safety. The theorem that well-typed programs do not go wrong: by progress and preservation, they never reach a stuck state. For safe Rust, going wrong means undefined behavior, not panics, leaks or non-termination. (Chapter 1)

Type system. A set of rules that decides, before a program runs, which programs are well typed. The mathematical study of type systems is type theory. (Chapter 1)

Typestate. A protocol encoded in types: each state is a type, and each operation consumes a value of the type of the state where it is allowed and returns a value of the type of the next state. It works in Rust because values are affine. (Chapter 25)

E.20 U

Undefined behavior. The kind of going wrong that safe Rust rules out, such as reading freed memory, data races and using uninitialized values, to which the language assigns no meaning at all. (Chapter 1)

Unification. Solving an equation between types, $\tau_1 \doteq \tau_2$, by finding a substitution, a unifier, that makes the two sides equal. Inference wants the most general unifier, through which every other unifier factors. (Chapter 13)

Unit type. The type with exactly one value, `()` in Rust and `Unit` in Lean, written $\mathbf{1}$. It is "true" under Curry–Howard and a terminal object in the category of types. (Chapter 1)

Universal property. A definition of an object by how morphisms relate to it, such as "an object with two projections through which every pair of morphisms factors uniquely". It determines the object up to unique isomorphism. (Chapter 16)

Universe. A type of types. Lean has `Prop` for propositions and a hierarchy `Type 0`, `Type 1`, ... for data, each a member of the next; internally they are all written `Sort u`. (Chapters 10 and 12)

`unsafe`. A Rust keyword that marks a proof obligation the compiler cannot check: an `unsafe` block says that the code inside discharges the obligations of its operations, and an `unsafe fn` passes its obligations to the caller. (Chapter 32)

Usage theorem. The theorem of the affine λ-calculus that in a well-typed term every variable of the context occurs at most once. Chapter 23 generalizes it: variables whose type is not `Copy` occur at most once. (Chapter 22)

E.21 V

Variance. How a subtyping between the arguments of a type constructor carries over to the constructed types: covariant, contravariant, invariant or bivariant. In Rust it matters for lifetimes, and getting it wrong lets safe code produce a dangling reference. (Chapter 27)

E.22 W

Weakening. The structural rule that lets an assumption go unused. In programs it is discarding a value, which in Rust runs the value's destructor: weakening with a side effect. (Chapter 6)

E.23 Y

Yoneda lemma. The theorem that the natural transformations from $\mathrm{Hom}(\alpha, -)$ to a functor F correspond to the elements of $F\,\alpha$. For types it reads $\forall \beta.\, (\alpha \to \beta) \to F\,\beta \cong F\,\alpha$, which is continuation-passing style. (Chapter 18)

E.24 Z

Zero-sized type. A type whose values need no bytes at all, such as `()`, the unique type of a `fn` item or `PhantomData`. (Chapter 11)

APPENDIX F

Further Reading

This book is a first course. Every chapter opens a door into a large field, and this appendix suggests where to go through it. The list is deliberately short and annotated: a few books and papers per topic, chosen because they are classics, because they are readable, or because they continue directly from where the book stops. Online resources are named rather than linked, since addresses change; a search for the title finds them.

In this appendix you will learn:

- where to continue with Lean and Mathlib;
- the standard textbooks on type theory, logic and category theory;
- the original papers behind the main ideas of the book;
- the research on Rust's type system, ownership and verification.

F.1 Lean and Mathlib

- **Functional Programming in Lean**, by David Thrane Christiansen. The best introduction to Lean 4 as a programming language: structures, type classes, monads, `do` notation, and how Lean compiles. A natural continuation of Chapter 2.
- **Theorem Proving in Lean 4**, by Jeremy Avigad, Leonardo de Moura, Soonho Kong and Sebastian Ullrich. The reference introduction to Lean's logic: dependent types, inductive types, propositions, tactics, type classes, axioms.
- **Mathematics in Lean**, by Jeremy Avigad and Patrick Massot. How to do mathematics with Mathlib: the tactics, the library's conventions, and many worked exercises.
- **The Natural Number Game**, by Kevin Buzzard and others. A browser game that teaches tactic proofs by building the natural numbers from scratch; an enjoyable way to make the tactics of this book second nature.
- **The Mechanics of Proof**, by Heather Macbeth. An introduction to writing proofs, using Lean throughout; good if the proofs in this book were your first.
- The Mathlib documentation (searchable online), and Lean's own source code and Mathlib's, which your editor opens with a click on any name.
- Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language* (CADE, 2021): the design of Lean 4.

- Sebastian Ullrich and Leonardo de Moura, *Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming* (2019), and Alex Reinking, Ningning Xie, Leonardo de Moura and Daan Leijen, *Perceus: Garbage Free Reference Counting with Reuse* (PLDI, 2021): how Lean updates unshared values in place (Chapter 24).

F.2 Type theory and programming languages

- Benjamin C. Pierce, **Types and Programming Languages** (MIT Press, 2002). *The* textbook. Its first half covers, in more depth and with more variations, the ground of Chapters 3 and 7–13 of this book: untyped and typed λ-calculi, type safety, subtyping, recursive types, polymorphism, type reconstruction.
- Benjamin C. Pierce (editor), **Advanced Topics in Types and Programming Languages** (MIT Press, 2005). Its first chapter, David Walker's *Substructural Type Systems*, is the standard introduction to linear and affine type systems (Part V).
- Robert Harper, **Practical Foundations for Programming Languages** (Cambridge University Press, second edition 2016). A rigorous, unified treatment of programming languages through judgments and rules; the style of Chapter 3, taken all the way.
- **Software Foundations**, by Benjamin C. Pierce and many coauthors (online, in several volumes). Logic, programming-language semantics and type systems, formalized in the Rocq (formerly Coq) proof assistant; everything this book does in Lean for its toy languages, done systematically.
- **Programming Language Foundations in Agda**, by Philip Wadler, Wen Kokke and Jeremy G. Siek (online). The same ground in Agda, with intrinsically typed terms as in Chapter 8.
- Henk Barendregt, **The Lambda Calculus: Its Syntax and Semantics** (North-Holland, revised edition 1984). The encyclopedia of the untyped λ-calculus (Chapter 7).
- Per Martin-Löf, **Intuitionistic Type Theory** (Bibliopolis, 1984). The lectures that founded dependent type theory, of which Lean's is a descendant (Chapter 30).

F.3 Logic and the Curry–Howard correspondence

- Jean-Yves Girard, Yves Lafont and Paul Taylor, **Proofs and Types** (Cambridge University Press, 1989). A short, dense classic: natural deduction, normalization, System F, and the beginnings of linear logic.
- Morten Heine Sørensen and Paweł Urzyczyn, **Lectures on the Curry–Howard Isomorphism** (Elsevier, 2006). The correspondence in all its variations, from intuitionistic logic to System F and dependent types.
- Philip Wadler, *Propositions as Types* (Communications of the ACM, 2015). An essay on the history and the meaning of the correspondence, and the best short read on the subject of Part II.
- Dag Prawitz, **Natural Deduction: A Proof-Theoretical Study** (1965), and Gerhard Gentzen's *Investigations into Logical Deduction* (1935), where natural deduction and the sequent calculus were born (Chapter 4).

- A. S. Troelstra and H. Schwichtenberg, **Basic Proof Theory** (Cambridge University Press, second edition 2000). Natural deduction, sequent calculi, normalization and cut elimination, for the reader who wants the full metatheory.
- Arthur Prior, *The Runabout Inference-Ticket* (1960): the paper that introduced `tonk` (Chapter 4).
- Frank Pfenning's lecture notes on constructive logic and on linear logic (from his courses at Carnegie Mellon, online): beautifully clear treatments of harmony, verifications and the judgmental approach that Chapters 4, 5 and 21 follow.
- Jean-Yves Girard, *Linear Logic* (Theoretical Computer Science, 1987): the paper that started it all (Chapter 21). Philip Wadler's *Linear Types Can Change the World!* (1990) is a much gentler introduction to linear types for programmers.

F.4 Category theory

- Bartosz Milewski, **Category Theory for Programmers** (2019; also a series of blog posts and video lectures). The friendliest introduction for programmers, with examples in Haskell and C++; complements Part IV well.
- Benjamin C. Pierce, **Basic Category Theory for Computer Scientists** (MIT Press, 1991). Short and to the point.
- Steve Awodey, **Category Theory** (Oxford University Press, second edition 2010). A textbook written for readers coming from logic and computer science rather than from algebra.
- Emily Riehl, **Category Theory in Context** (Dover, 2016). A mathematician's introduction, with an outstanding treatment of the Yoneda lemma, limits and adjunctions.
- Saunders Mac Lane, **Categories for the Working Mathematician** (Springer, second edition 1998). The classic reference.
- J. Lambek and P. J. Scott, **Introduction to Higher Order Categorical Logic** (Cambridge University Press, 1986). Cartesian closed categories and the typed λ-calculus (Chapter 19).
- Eugenio Moggi, *Notions of Computation and Monads* (Information and Computation, 1991), and Philip Wadler, *Monads for Functional Programming* (1995): monads as a structure for effects (Chapter 20).

F.5 Polymorphism, inference and abstraction

- John C. Reynolds, *Types, Abstraction and Parametric Polymorphism* (1983), and Philip Wadler, *Theorems for Free!* (1989): parametricity and free theorems (Chapter 18).
- Robin Milner, *A Theory of Type Polymorphism in Programming* (1978), and Luis Damas and Robin Milner, *Principal Type-Schemes for Functional Programs* (1982): Hindley–Milner type inference (Chapter 13).
- Philip Wadler and Stephen Blott, *How to Make Ad-hoc Polymorphism Less Ad Hoc* (1989): type classes (Chapter 14).

- John C. Mitchell and Gordon D. Plotkin, *Abstract Types Have Existential Type* (1988): existential types (Chapter 15).

F.6 Rust, ownership and regions

- **The Rust Programming Language** ("the book"), **The Rust Reference** and **The Rustonomicon**, all published by the Rust project. The Reference describes the language precisely; the Rustonomicon explains `unsafe`, variance, drop check and the other dark corners that Chapters 24–27 and 32 touch.
- Mads Tofte and Jean-Pierre Talpin, *Region-Based Memory Management* (Information and Computation, 1997), and Dan Grossman and coauthors, *Region-Based Memory Management in Cyclone* (PLDI, 2002): the region calculi behind lifetimes (Chapter 26).
- Rust RFC 2094, *Non-lexical lifetimes*, and Niko Matsakis's blog *Baby Steps*, where NLL, Polonius and many other parts of the borrow checker were designed in public (Chapter 29).
- Aaron Weiss, Olek Gierczak, Daniel Patterson and Amal Ahmed, *Oxide: The Essence of Rust* (2019): a formal core calculus for Rust's ownership and borrowing, in the spirit of Chapter 28.
- Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers and Derek Dreyer, *RustBelt: Securing the Foundations of the Rust Programming Language* (POPL, 2018), and Ralf Jung's doctoral thesis *Understanding and Evolving the Rust Programming Language* (2020): semantic soundness of Rust and its standard library in the Iris separation logic (Chapter 32).
- Ralf Jung, Hoang-Hai Dang, Jeehoon Kang and Derek Dreyer, *Stacked Borrows: An Aliasing Model for Rust* (POPL, 2020), and its successor, Tree Borrows: the aliasing rules that Miri checks.
- John C. Reynolds, *Separation Logic: A Logic for Shared Mutable Data Structures* (LICS, 2002): the logic that RustBelt builds on.

F.7 Verifying Rust programs

- Son Ho and Jonathan Protzenko, *Aeneas: Rust Verification by Functional Translation* (ICFP, 2022): translating safe Rust into pure functional code that can be verified in Lean, among other provers. The most direct continuation of Chapter 31.
- The documentation of **Verus**, **Creusot**, **Prusti** and **Kani**, tools that verify Rust programs with different techniques (specifications checked by SMT solvers, deductive verification, model checking). Read Chapter 31 for a short orientation, then each project's own guide.